
**STM32H745/755 and STM32H747/757 advanced Arm[®]-based
32-bit MCUs**

Introduction

This reference manual targets application developers. It provides complete information on how to use the STM32H745/55/47/57xx microcontroller memory and peripherals.

The STM32H745/755 and STM32H747/757 lines include microcontrollers with different memory sizes, packages and peripherals.

The devices include ST state-of-the-art patented technology.

For ordering information, mechanical, and electrical device characteristics refer to the corresponding datasheets.

For information on the Arm[®] Cortex[®]-M4 with FPU and Arm[®] Cortex[®]-M7 with FPU cores, refer to the corresponding Arm *Technical Reference Manuals*.

Related documents

- Arm[®] Cortex[®]-M4 Technical Reference Manual, available from www.arm.com.
- Arm[®] Cortex[®]-M7 Technical Reference Manual, available from www.arm.com.
- Cortex[®]-M4 programming manual (PM0214).
- Cortex[®]-M7 programming manual (PM0253).
- STM32H745/755 and STM32H747/757 errata sheet

Contents

1	Documentation conventions	106
1.1	General information	106
1.2	List of abbreviations for registers	106
1.3	Glossary	107
1.4	Availability of peripherals	107
2	Memory and bus architecture	108
2.1	System architecture	108
2.1.1	Bus matrices	110
2.1.2	TCM buses	110
2.1.3	Bus-to-bus bridges	110
2.1.4	ART accelerator	111
2.1.5	Inter-domain buses	111
2.1.6	CPU buses	111
2.1.7	Bus master peripherals	112
2.1.8	Clocks to functional blocks	113
2.2	AXI interconnect matrix (AXIM)	114
2.2.1	AXI introduction	114
2.2.2	AXI interconnect main features	114
2.2.3	AXI interconnect functional description	115
2.2.4	AXI interconnect registers	117
2.2.5	AXI interconnect register map	126
2.3	Memory organization	134
2.3.1	Introduction	134
2.3.2	Memory map and register boundary addresses	135
2.4	Embedded SRAM	142
2.5	Flash memory overview	143
2.6	Boot configuration	143
3	RAM ECC monitoring (RAMECC)	146
3.1	Introduction	146
3.2	RAMECC main features	146
3.3	RAMECC functional description	146

3.3.1	RAMECC block diagram	146
3.3.2	RAMECC internal signals	148
3.3.3	RAMECC monitor mapping	148
3.4	RAMECC registers	149
3.4.1	RAMECC interrupt enable register (RAMECC_IER)	149
3.4.2	RAMECC monitor x configuration register (RAMECC_MxCR)	150
3.4.3	RAMECC monitor x status register (RAMECC_MxSR)	150
3.4.4	RAMECC monitor x failing address register (RAMECC_MxFAR)	151
3.4.5	RAMECC monitor x failing data low register (RAMECC_MxFDRL)	151
3.4.6	RAMECC monitor x failing data high register (RAMECC_MxFDRH)	152
3.4.7	RAMECC monitor x failing ECC error code register RAMECC_MxFECCR)	152
3.4.8	RAMECC register map	153
4	Embedded flash memory (FLASH)	154
4.1	Introduction	154
4.2	FLASH main features	154
4.3	FLASH functional description	155
4.3.1	FLASH block diagram	155
4.3.2	FLASH internal signals	156
4.3.3	FLASH architecture and integration in the system	156
4.3.4	Flash memory architecture and usage	158
4.3.5	FLASH system performance enhancements	162
4.3.6	FLASH data protection schemes	162
4.3.7	Overview of FLASH operations	163
4.3.8	FLASH read operations	164
4.3.9	FLASH program operations	167
4.3.10	FLASH erase operations	171
4.3.11	FLASH parallel operations	174
4.3.12	Flash memory error protections	174
4.3.13	Flash bank and register swapping	176
4.3.14	FLASH reset and clocks	179
4.4	FLASH option bytes	179
4.4.1	About option bytes	179
4.4.2	Option byte loading	180
4.4.3	Option byte modification	180
4.4.4	Option bytes overview	183

1 Documentation conventions

1.1 General information

The STM32H745/755 and STM32H747/757 devices embed an Arm^{®(a)} Cortex[®]-M4 with FPU and an Arm[®] Cortex[®]-M7 with FPU core.



1.2 List of abbreviations for registers

The following abbreviations^(b) are used in register descriptions:

read/write (rw)	Software can read and write to this bit.
read-only (r)	Software can only read this bit.
write-only (w)	Software can only write to this bit. Reading this bit returns the reset value.
read/clear write0 (rc_w0)	Software can read as well as clear this bit by writing 0. Writing 1 has no effect on the bit value.
read/clear write1 (rc_w1)	Software can read as well as clear this bit by writing 1. Writing 0 has no effect on the bit value.
read/clear write (rc_w)	Software can read as well as clear this bit by writing to the register. The value written to this bit is not important.
read/clear by read (rc_r)	Software can read this bit. Reading this bit automatically clears it to 0. Writing this bit has no effect on the bit value.
read/set by read (rs_r)	Software can read this bit. Reading this bit automatically sets it to 1. Writing this bit has no effect on the bit value.
read/set (rs)	Software can read as well as set this bit. Writing 0 has no effect on the bit value.
read/write once (rwo)	Software can only write once to this bit and can also read it at any time. Only a reset can return the bit to its reset value.
toggle (t)	The software can toggle this bit by writing 1. Writing 0 has no effect.
read-only write trigger (rt_w1)	Software can read this bit. Writing 1 triggers an event but has no effect on the bit value.
Reserved (Res.)	Reserved bit, must be kept at reset value.

a. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

b. This is an exhaustive list of all abbreviations applicable to STMicroelectronics microcontrollers, some of them may not be used in the current document.

1.3 Glossary

This section gives a brief definition of acronyms and abbreviations used in this document:

- **Word:** data of 32-bit length.
- **Half-word:** data of 16-bit length.
- **Byte:** data of 8-bit length.
- **Double word:** data of 64-bit length.
- **Flash word:** data of 256-bit length
- **IAP (in-application programming):** IAP is the ability to re-program the flash memory of a microcontroller while the user program is running.
- **ICP (in-circuit programming):** ICP is the ability to program the flash memory of a microcontroller using the JTAG protocol, the SWD protocol or the bootloader while the device is mounted on the user application board.
- **Option bytes:** product configuration bits stored in the flash memory.
- **AHB:** advanced high-performance bus.
- **APB:** advanced peripheral bus.
- **AXI:** Advanced extensible interface protocol.
- **PCROP:** proprietary code readout protection.
- **RDP:** readout protection.

1.4 Availability of peripherals

For availability of peripherals and their number across all sales types, refer to the particular device datasheet.

Table 1. Availability of security features

Security feature	STM32H755xI and STM32H757xI	STM32H745xI/G and STM32H747xI/G
Embedded flash memory (FLASH): – Flash Secure-only area	Available	Not available
Security memory management: – Secure access mode – Root secure services (RSS)		
Cryptographic processor (CRYP)		
Hash processor (HASH)		

2 Memory and bus architecture

2.1 System architecture

An AXI bus matrix, two AHB bus matrices and bus bridges allow interconnecting bus masters with bus slaves, as illustrated in [Table 2](#) and [Figure 1](#).

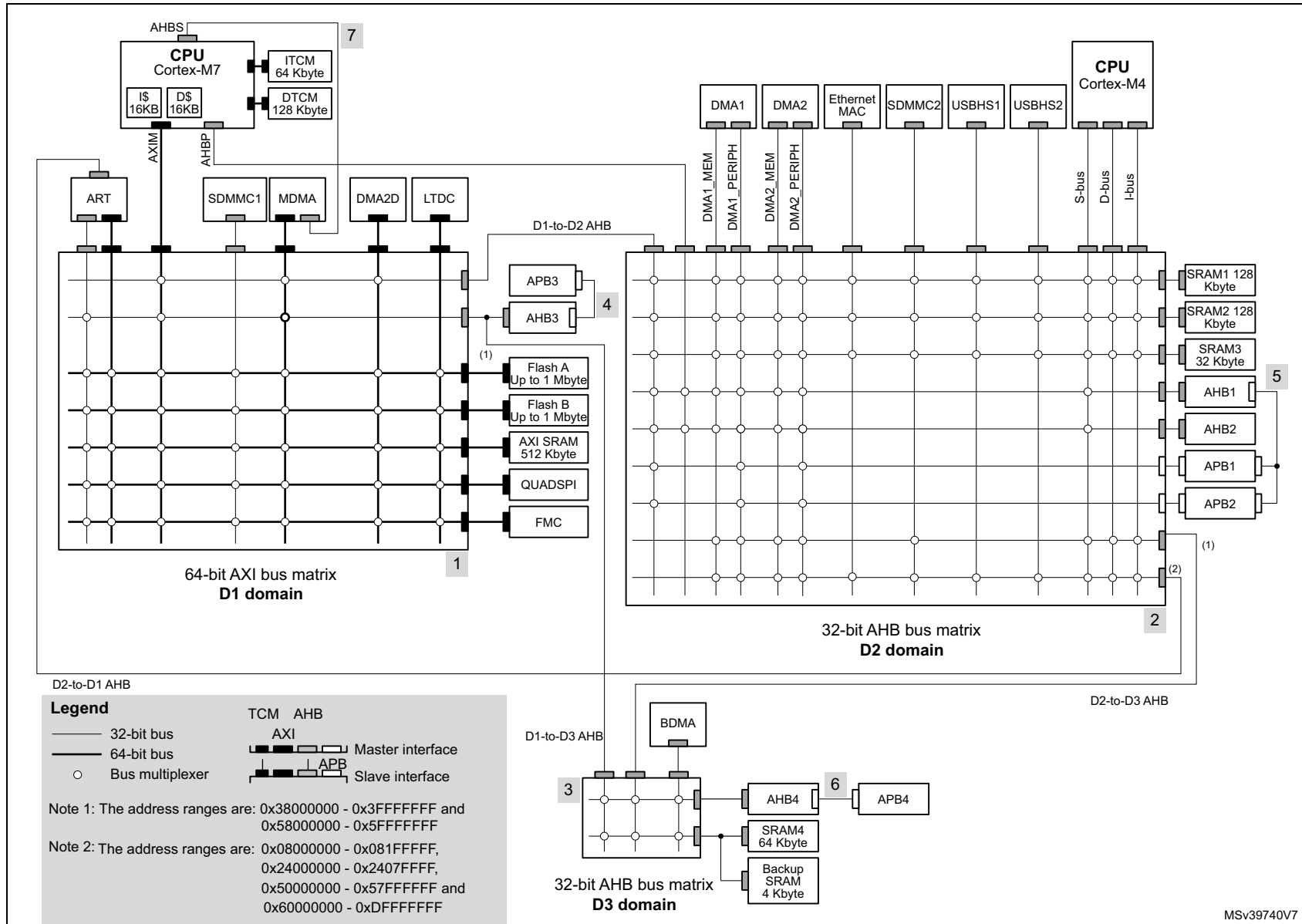
Table 2. Bus-master-to-bus-slave interconnect

	Bus master / type ⁽¹⁾																					
	Cortex-M7 - AXIM	Cortex-M7 - AHBP	Cortex-M7 - ITCM	Cortex-M7 - DTCM	SDMMC1	MDMA	MDMA - AHBS	DMA2D	LTDC	DMA1 - MEM	DMA1 - PERIPH	DMA2 - MEM	DMA2 - PERIPH	Eth. MAC - AHB	SDMMC2 - AHB	USBHS1 - AHB	USBHS2 - AHB	Cortex-M4 - S-bus	Cortex-M4 - D-bus	Cortex-M4 - I-bus	BDMA - AHB	
Bus slave / type ⁽¹⁾	Interconnect path and type ⁽²⁾																					
ITCM	-	-	X	-	-	-	X	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
DTCM	-	-	-	X	-	-	X	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
AHB3 peripherals	X	-	-	-	-	X	-	-	-	X	X	X	X	X	X	X	X	X	X	X	-	
APB3 peripherals	X	-	-	-	-	X	-	-	-	X	X	X	X	X	X	X	X	X	X	X	-	
Flash bank 1	X	-	-	-	X	X	-	X	X	X	X	X	X	X	X	X	X	X	X	X	-	
Flash bank 2	X	-	-	-	X	X	-	X	X	X	X	X	X	X	X	X	X	X	X	X	-	
AXI SRAM	X	-	-	-	X	X	-	X	X	X	X	X	X	X	X	X	X	X	X	X	-	
QUADSPI	X	-	-	-	X	X	-	X	X	X	X	X	X	X	X	X	X	X	X	X	-	
FMC	X	-	-	-	X	X	-	X	X	X	X	X	X	X	X	X	X	X	X	X	-	
SRAM 1	X	-	-	-	-	X	-	X	-	X	X	X	X	X	X	X	X	X	X	X	-	
SRAM 2	X	-	-	-	-	X	-	X	-	X	X	X	X	X	X	X	X	X	X	X	-	
SRAM 3	X	-	-	-	-	X	-	X	-	X	X	X	X	X	X	X	X	X	X	X	-	
AHB1 peripherals	-	X	-	-	-	X	-	X	-	X	X	X	X	-	-	-	-	X	-	-	-	
APB1 peripherals	-	X	-	-	-	X	-	X	-	X	X	X	X	-	-	-	-	X	-	-	-	
AHB2 peripherals	-	X	-	-	-	X	-	X	-	X	X	X	X	-	-	-	-	X	-	-	-	
APB2 peripherals	-	X	-	-	-	X	-	X	-	X	X	X	X	-	-	-	-	X	-	-	-	
AHB4 peripherals	X	-	-	-	-	X	-	-	-	X	X	X	X	-	X	-	-	X	-	-	X	
APB4 peripherals	X	-	-	-	-	X	-	-	-	X	X	X	X	-	X	-	-	X	-	-	X	
SRAM4	X	-	-	-	-	X	-	-	-	X	X	X	X	-	X	-	-	X	-	-	X	
Backup RAM	X	-	-	-	-	X	-	-	-	X	X	X	X	-	X	-	-	X	-	-	X	

1. **Bold** font type denotes 64-bit bus, plain type denotes 32-bit bus.

2. "X" = access possible, "-" = access not possible, shading = access useful/usable.

Figure 1. System architecture for STM32H745/55/47/57xx devices



2.1.1 Bus matrices

AXI bus matrix in D1 domain

The D1 domain multi AXI bus matrix ensures and arbitrates concurrent accesses from multiple masters to multiple slaves. This allows efficient simultaneous operation of high-speed peripherals.

The arbitration uses a round-robin algorithm with QoS capability.

Refer to [Section 2.2: AXI interconnect matrix \(AXIM\)](#) for more information on AXI interconnect.

AHB bus matrices in D2 and D3 domains

The AHB bus matrices in D2 and D3 domains ensure and arbitrate concurrent accesses from multiple masters to multiple slaves. This allows efficient simultaneous operation of high-speed peripherals.

The arbitration uses a round-robin algorithm.

2.1.2 TCM buses

The DTCM and ITCM (data and instruction tightly coupled RAMs) are connected through dedicated TCM buses directly to the Cortex-M7 core. The MDMA controller can access the DTCM and ITCM through AHBS, a specific CPU slave AHB. The ITCM is accessed by Cortex-M7 at CPU clock speed, with zero wait states.

2.1.3 Bus-to-bus bridges

To allow peripherals with different types of buses to communicate together, there is a number of bus-to-bus bridges in the system.

The AHB/APB bridges in D1 and D3 domains allow connecting peripherals on APB3 and APB4 to AHB3 and AHB4, respectively. The AHB/APB bridges in D2 domain allow peripherals on APB1 and APB2 to connect to AHB1. These AHB/APB bridges provide full synchronous interfacing, which allows the APB peripherals to operate with clocks independent of AHB that they connect to.

The AHB/APB bridges also allow APB1 and APB2 peripherals to connect to DMA1 and DMA2 peripheral buses, respectively, without transiting through AHB1.

The AHB/APB bridges convert 8-bit / 16-bit APB data to 32-bit AHB data, by replicating it to the three upper bytes / the upper half-word of the 32-bit word.

The AXI bus matrix incorporates AHB/AXI bus bridge functionality on its slave bus interfaces. The AXI/AHB bus bridges on its master interfaces marked as 32-bit in [Figure 1](#) are outside the matrix.

The Cortex-M7 CPU provides AHB/TCM-bus (ITCM and DTCM buses) translation from its AHBS slave AHB, allowing the MDMA controller to access the ITCM and DTCM.

2.1.4 ART accelerator

Reduced ART (adaptive real-time) memory access accelerator stands between the D2-to-D1 AHB and the AXI bus matrix. It accelerates cacheable AHB instruction fetch accesses, using a dedicated 64-bit AXI bus matrix port to pre-fetch code from the internal and external memories of the D1 domain into a built-in cache. It routes all the other AHB accesses to a dedicated 32-bit AXI bus matrix port connecting the D2-to-D1 AHB with all the internal and external memories and peripherals of the D1 domain excluding GPV, as well as with the D1-to-D3 AHB.

2.1.5 Inter-domain buses

D2-to-D1 AHB

This 32-bit bus connects the D2 domain to the AXI bus matrix in the D1 domain. It allows bus masters in the D2 domain to access resources (bus slaves) in the D1 domain and indirectly, via the D1-to-D3 AHB, in the D3 domain.

D1-to-D2 AHB

This 32-bit bus connects the D1 domain to the D2 domain AHB bus matrix. It allows bus masters in the D1 domain to access resources (bus slaves) in the D2 domain.

D1-to-D3 AHB

This 32-bit bus connects the D1 domain to the D3 domain AHB bus matrix. It allows bus masters in the D1 domain to access resources (bus slaves) in the D3 domain.

D2-to-D3 AHB

This 32-bit bus connects the D2 domain to the D3 domain AHB bus matrix. It allows bus masters in the D2 domain to access resources (bus slaves) in the D3 domain.

2.1.6 CPU buses

Cortex[®]-M7 AXIM bus

The Cortex[®]-M7 CPU uses the 64-bit AXIM bus to access all memories (excluding ITCM, and DTCM) and AHB3, AHB4, APB3 and APB4 peripherals (excluding AHB1, APB1 and APB2 peripherals).

The AXIM bus connects the CPU to the AXI bus matrix in the D1 domain.

Cortex[®]-M7 ITCM bus

The Cortex[®]-M7 CPU uses the 64-bit ITCM bus for fetching instructions from and accessing data in the ITCM.

Cortex[®]-M7 DTCM bus

The Cortex[®]-M7 CPU uses the 2x32-bit DTCM bus for accessing data in the DTCM. The 2x32-bit DTCM bus allows load/load and load/store instruction pairs to be dual-issued on the DTCM memory. It can also fetch instructions.

Cortex[®]-M7 AHBS bus

The Cortex[®]-M7 CPU uses the 32-bit AHBS slave bus to allow the MDMA controller to access the ITCM and the DTCM.

Cortex[®]-M7 AHBP bus

The Cortex[®]-M7 CPU uses the 32-bit AHBP bus for accessing AHB1, AHB2, APB1 and APB2 peripherals via the AHB bus matrix in the D2 domain.

Cortex[®]-M4 I-bus

The Cortex[®]-M4 CPU uses the 32-bit I-bus, via the AHB bus matrix in D2 domain, to fetch instructions from memories containing code and mapped on addresses below 0x2000 0000: the internal SRAM1, SRAM2, SRAM3, and the internal Flash memory.

Cortex[®]-M4 D-bus

The Cortex[®]-M4 CPU uses the 32-bit D-bus, via AHB bus matrix, for literal load and debug access to memories containing code or data and mapped on addresses below 0x2000 0000: the internal SRAM1, SRAM2, SRAM3, and the internal Flash memory.

Cortex[®]-M4 S-bus

The Cortex[®]-M4 CPU uses the 32-bit S-bus, via the AHB bus matrix in D2 domain, to access all memories and all peripherals in the device mapped on addresses above 0x2000 0000. It can also handle instruction fetching, although less efficiently than the I-bus.

2.1.7 Bus master peripherals

SDMMC1

The SDMMC1 uses a 32-bit bus, connected to the AXI bus matrix, through which it can access internal AXI SRAM and Flash memories, and external memories through the Quad-SPI controller and the FMC.

SDMMC2

The SDMMC2 uses a 32-bit bus, connected to the AHB bus matrix in D2 domain. Through the system bus matrices, it can access the internal AXI SRAM, SRAM1, SRAM2, SRAM3 and Flash memories, and external memories through the Quad-SPI controller and the FMC.

MDMA controller

The MDMA controller has two bus masters: an AXI 64-bit bus, connected to the AXI bus matrix and an AHB 32-bit bus connected to the Cortex-M7 AHBS slave bus.

The MDMA is optimized for DMA data transfers between memories since it supports linked list transfers that allow performing a chained list of transfers without the need for CPU intervention. Through the system bus matrices and the Cortex-M7 AHBS slave bus, the MDMA can access all internal and external memories through the Quad-SPI controller and the FMC.

DMA1 and DMA2 controllers

The DMA1 and DMA2 controllers have two 32-bit buses - memory bus and peripheral bus, connected to the AHB bus matrix in D2 domain.

The memory bus allows DMA data transfers between memories. Through the system bus matrices, the memory bus can access all internal memories except ITCM and DTCM, and external memories through the Quad-SPI controller and the FMC.

The peripheral bus allows DMA data transfers between two peripherals, between two memories or between a peripheral and a memory. Through the system bus matrices, the peripheral bus can access all internal memories except ITCM and DTCM, external memories through the Quad-SPI controller and the FMC, and all AHB and APB peripherals. A direct access to APB1 and APB2 is available, without passing through AHB1. Direct path to APB1 and APB2 bridges allows reducing the bandwidth usage on AHB1 bus by improving data treatment efficiency for APB and AHB peripherals.

BDMA controller

The BDMA controller uses a 32-bit bus, connected to the AHB bus matrix in D3 domain, for DMA data transfers between two peripherals, between two memories or between a peripheral and a memory. BDMA transfers are limited to the D3 domain resources. It can access the internal SRAM4, backup RAM, and AHB4 and APB4 peripherals through the AHB bus matrix in the D3 domain.

Chrom-Art Accelerator (DMA2D)

The DMA2D graphics accelerator uses a 64-bit bus, connected to the AXI bus matrix. Through the system bus matrices, internal AXI SRAM, SRAM1, SRAM2, SRAM3 and Flash memories, and external memories through the Quad-SPI controller and the FMC.

LCD-TFT controller (LTDC)

The LCD-TFT display controller, LTDC, uses a 64-bit bus, connected to the AXI bus matrix, through which it can access internal AXI SRAM and Flash memories, and external memories through the Quad-SPI controller and the FMC.

Ethernet MAC

The Ethernet MAC uses a 32-bit bus, connected to the AHB bus matrix in the D2 domain. Through the system bus matrices, it can access all internal memories except ITCM and DTCM, and external memories through the Quad-SPI controller and the FMC.

USBHS1 and USBHS2 peripherals

The USBHS1 and USBHS2 peripherals use 32-bit buses, connected to the AHB bus matrix in the D2 domain. Through the system bus matrices, they can access all internal memories except ITCM and DTCM, and external memories through the Quad-SPI controller and the FMC.

2.1.8 Clocks to functional blocks

Upon reset, clocks to blocks such as peripherals and some memories are disabled (except for the SRAM, DTCM, ITCM and Flash memory). To operate a block with no clock upon reset, the software must first enable its clock through RCC_AHBxENR or RCC_APBxENR register, respectively.

2.2 AXI interconnect matrix (AXIM)

2.2.1 AXI introduction

The AXI (advanced extensible interface) interconnect is based on the Arm® CoreLink™ NIC-400 Network Interconnect. The interconnect has seven initiator ports, or ASIBs (AMBA slave interface blocks), and seven target ports, or AMIBs (AMBA master interface blocks). The ASIBs are connected to the AMIBs via an AXI switch matrix.

Each ASIB is a slave on an AXI bus or AHB (advanced high-performance bus). Similarly, each AMIB is a master on an AXI or AHB bus. Where an ASIB or AMIB is connected to an AHB, it converts between the AHB and the AXI protocol.

The AXI interconnect includes a GPV (global programmer view) which contains registers for configuring certain parameters, such as the QoS (quality of service) level at each ASIB.

Any accesses to unallocated address space are handled by the default slave, which generates the return signals. This ensures that such transactions complete and do not block the issuing master and ASIB.

2.2.2 AXI interconnect main features

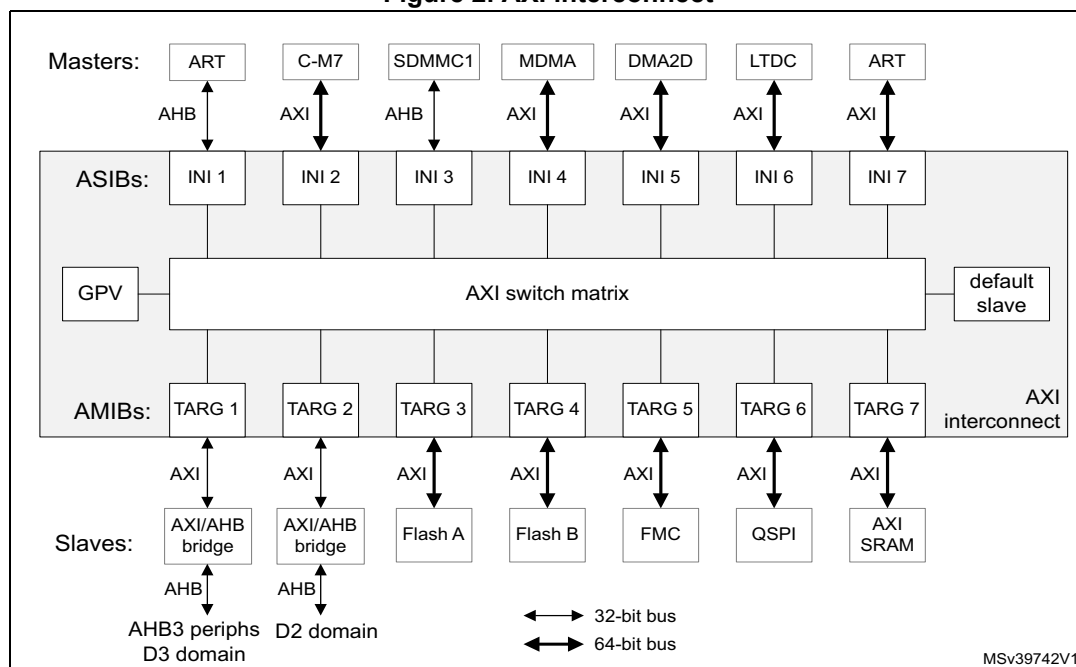
- 64-bit AXI bus switch matrix with seven ASIBs and seven AMIBs, in D1 domain
- AHB/AXI bridge function built into the ASIBs
- concurrent connectivity of multiple ASIBs to multiple AMIBs
- programmable traffic priority management (QoS - quality of service)
- software-configurable via GPV

2.2.3 AXI interconnect functional description

Block diagram

The AXI interconnect is shown in [Figure 2](#).

Figure 2. AXI interconnect



ASIB configuration

[Table 3](#) summarizes the characteristics of the ASIBs.

Table 3. ASIB configuration

ASIB	Connected master	Protocol	Bus width	R/W issuing
INI 1	ART (non-buffered data)	AHB-lite	32	1/4
INI 2	Cortex-M7	AXI4	64	7/32
INI 3	SDMMC1	AHB-lite	32	1/4
INI 4	MDMA	AXI4	64	4/1
INI 5	DMA2D	AXI4	64	2/1
INI 6	LTDC	AXI4	64	1/1
INI 7	ART (instruction fetch)	AXI4	64	1/1

AMIB configuration

[Table 4](#) summarizes the characteristics of the AMIBs.

Table 4. AMIB configuration

AMIB	Connected slave	Protocol	Bus width	R/W/Total acceptance
TARG 1	Peripheral 3 and D3 AHB	AXI4 ⁽¹⁾	32	1/1/1
TARG 2	D2 AHB	AXI4 ⁽¹⁾	32	1/1/1
TARG 3	Flash A	AXI4	64	3/2/5
TARG 4	Flash B	AXI4	64	3/2/5
TARG 5	FMC	AXI4	64	3/3/6
TARG 6	QUADSPI	AXI4	64	2/1/3
TARG 7	AXI SRAM	AXI3	64	2/2/2

1. Conversion to AHB protocol is done via an AXI/AHB bridge sitting between AXI interconnect and the connected slave.

Quality of service (QoS)

The AXI switch matrix uses a priority-based arbitration when two ASIB simultaneously attempt to access the same AMIB. Each ASIB has programmable read channel and write channel priorities, known as QoS, from 0 to 15, such that the higher the value, the higher the priority. The read channel QoS value is programmed in the [AXI interconnect - INI x read QoS register \(AXI_INIx_READ_QOS\)](#), and the write channel in the [AXI interconnect - INI x write QoS register \(AXI_INIx_WRITE_QOS\)](#). The default QoS value for all channels is 0 (lowest priority).

If two coincident transactions arrive at the same AMIB, the higher priority transaction passes before the lower priority. If the two transactions have the same QoS value, then a least-recently-used (LRU) priority scheme is adopted.

The QoS values should be programmed according to the latency requirements for the application. Setting a higher priority for an ASIB ensures a lower latency for transactions initiated by the associated bus master. This can be useful for real-time-constrained tasks, such as graphics processing (LTDC, DMA2D). Assigning a high priority to masters that can make many and frequent accesses to the same slave (such as the Cortex-M7 CPU) can block access to that slave by other lower-priority masters.

Global programmer view (GPV)

The GPV contains configuration registers for the AXI interconnect (see [Section 2.2.4](#)). These registers are only accessible by the Cortex-M7 CPU.

2.2.4 AXI interconnect registers

AXI interconnect - peripheral ID4 register (AXI_PERIPH_ID_4)

Address offset: 0x1FD0

Reset value: 0x0000 0004

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	4KCOUNT[3:0]				JEP106CON[3:0]			
								r				r			

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:4 **4KCOUNT[3:0]**: Register file size
0x0: N/A

Bits 3:0 **JEP106CON[3:0]**: JEP106 continuation code
0x4: Arm®

AXI interconnect - peripheral ID0 register (AXI_PERIPH_ID_0)

Address offset: 0x1FE0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PARTNUM[7:0]							
								r							

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **PARTNUM[7:0]**: Peripheral part number bits 0 to 7
0x00: Part number = 0x400

AXI interconnect - peripheral ID1 register (AXI_PERIPH_ID_1)

Address offset: 0x1FE4

Reset value: 0x0000 00B4

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JEP106ID[3:0]				PARTNUM[11:8]			
								r				r			

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:4 **JEP106ID[3:0]**: JEP106 identity bits 0 to 3

0xB: Arm® JEDEC code

Bits 3:0 **PARTNUM[11:8]**: Peripheral part number bits 8 to 11

0x4: Part number = 0x400

AXI interconnect - peripheral ID2 register (AXI_PERIPH_ID_2)

Address offset: 0x1FE8

Reset value: 0x0000 002B

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	REVISION[3:0]				JEDEC	JEP106ID[6:4]		
								r				r	r		

Bits 7:4 **REVISION[3:0]**: Peripheral revision number

0x2: r0p2

Bit 3 **JEDEC**: JEP106 code flag

0x1: JEDEC allocated code

Bits 2:0 **JEP106ID[6:4]**: JEP106 Identity bits 4 to 6

0x3: Arm® JEDEC code

AXI interconnect - peripheral ID3 register (AXI_PERIPH_ID_3)

Address offset: 0x1FEC

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	REV_AND[3:0]				CUST_MOD_NUM[3:0]			
								r				r			

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:4 **REV_AND[3:0]**: Customer version

0: None

Bits 3:0 **CUST_MOD_NUM[3:0]**: Customer modification

0: None

AXI interconnect - component ID0 register (AXI_COMP_ID_0)

Address offset: 0x1FF0

Reset value: 0x0000 000D

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PREAMBLE[7:0]							
								r							

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **PREAMBLE7:0**: Preamble bits 0 to 7

0xD: Common ID value

AXI interconnect - component ID1 register (AXI_COMP_ID_1)

Address offset: 0x1FF4

Reset value: 0x0000 00F0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CLASS[3:0]				PREAMBLE[11:8]			
								r				r			

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:4 **CLASS[3:0]**: Component class

0xF: Generic IP component class

Bits 3:0 **PREAMBLE[11:8]**: Preamble bits 8 to 11

0x0: Common ID value

AXI interconnect - component ID2 register (AXI_COMP_ID_2)

Address offset: 0x1FF8

Reset value: 0x0000 0005

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PREAMBLE[19:12]							
								r							

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **PREAMBLE[19:12]**: Preamble bits 12 to 19

0x05: Common ID value

AXI interconnect - component ID3 register (AXI_COMP_ID_3)

Address offset: 0x1FFC

Reset value: 0x0000 00B1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PREAMBLE[27:20]							
								r							

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **PREAMBLE[27:20]**: Preamble bits 20 to 27

0xB1: Common ID value

AXI interconnect - TARG x bus matrix issuing functionality register (AXI_TARGx_FN_MOD_ISS_BM)

Address offset: 0x1008 + 0x1000 * x, where x = 1 to 7

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE_ISS_OVERRIDE	READ_ISS_OVERRIDE
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **WRITE_ISS_OVERRIDE**: Switch matrix write issuing override for target

0: Normal issuing capability

1: Set switch matrix write issuing capability to 1

Bit 0 **READ_ISS_OVERRIDE**: Switch matrix read issuing override for target

0: Normal issuing capability

1: Set switch matrix read issuing capability to 1

AXI interconnect - TARG x bus matrix functionality 2 register (AXI_TARGx_FN_MOD2)

Address offset: $0x1024 + 0x1000 * x$, where $x = 1, 2$ and 7

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BYPASS_MERGE
															rw

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **BYPASS_MERGE**: Disable packing of beats to match the output data width. Unaligned transactions are not realigned to the input data word boundary.

0: Normal operation

1: Disable packing

AXI interconnect - TARG x long burst functionality modification register (AXI_TARGx_FN_MOD_LB)

Address offset: $0x102C + 0x1000 * x$, where $x = 1$ and 2

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FN_MOD_LB
															rw

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **FN_MOD_LB**: Controls burst breaking of long bursts

0: Long bursts can not be generated at the output of the ASIB

1: Long bursts can be generated at the output of the ASIB

AXI interconnect - TARG x issuing functionality modification register (AXI_TARGx_FN_MOD)

Address offset: $0x1108 + 0x1000 * x$, where $x = 1, 2$ and 7

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE_ISS_OVERRIDE	READ_ISS_OVERRIDE
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **WRITE_ISS_OVERRIDE**: Override AMIB write issuing capability

0: Normal issuing capability

1: Force issuing capability to 1

Bit 0 **READ_ISS_OVERRIDE**: Override AMIB read issuing capability

0: Normal issuing capability

1: Force issuing capability to 1

AXI interconnect - INI x functionality modification 2 register (AXI_INIx_FN_MOD2)

Address offset: $0x41024 + 0x1000 * x$, where $x = 1$ and 3

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BYPASS_MERGE
															rw

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **BYPASS_MERGE**: Disables alteration of transactions by the up-sizer unless required by the protocol

0: Normal operation

1: Transactions pass through unaltered where allowed

AXI interconnect - INI x AHB functionality modification register (AXI_INIx_FN_MOD_AHB)

Address offset: $0x41028 + 0x1000 * x$, where $x = 1$ and 3

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WR_INC_OVERRIDE	RD_INC_OVERRIDE
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **WR_INC_OVERRIDE**: Converts all AHB-Lite read transactions to a series of single beat AXI transactions.

0: Override disabled

1: Override enabled

Bit 0 **RD_INC_OVERRIDE**: Converts all AHB-Lite write transactions to a series of single beat AXI transactions, and each AHB-Lite write beat is acknowledged with the AXI buffered write response.

0: Override disabled

1: Override enabled

AXI interconnect - INI x read QoS register (AXI_INIx_READ_QOS)

Address offset: $0x41100 + 0x1000 * x$, where $x = 1$ to 76

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	AR_QOS[3:0]			
												rw			

Bits 31:4 Reserved, must be kept at reset value.

Bits 3:0 **AR_QOS[3:0]**: Read channel QoS setting

0x0: Lowest priority

0xF: Highest priority

AXI interconnect - INI x write QoS register (AXI_INIx_WRITE_QOS)Address offset: $0x41104 + 0x1000 * x$, where $x = 1$ to 76

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	AW_QOS[3:0]			
												rw			

Bits 31:4 Reserved, must be kept at reset value.

Bits 3:0 **AW_QOS[3:0]**: Write channel QoS setting

0x0: Lowest priority

0xF: Highest priority

AXI interconnect - INI x issuing functionality modification register (AXI_INIx_FN_MOD)Address offset: $0x41108 + 0x1000 * x$, where $x = 1$ to 76

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE_ISS_OVERRIDE	READ_ISS_OVERRIDE
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **WRITE_ISS_OVERRIDE**: Override ASIB write issuing capability

0: Normal issuing capability

1: Force issuing capability to 1

Bit 0 **READ_ISS_OVERRIDE**: Override ASIB read issuing capability

0: Normal issuing capability

1: Force issuing capability to 1

2.2.5 AXI interconnect register map

Table 5. AXI interconnect register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
0x1FD0	AXI_PERIPH_ID_4	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	4KCOUNT [3:0]				JEP106CON [3:0]							
	Reset value																									0	0	0	0	0	1	0	0				
0x1FD4	AXI_PERIPH_ID_5	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Reserved											
	Reset value																									0	0	0	0	0	0	0	0				
0x1FD8	AXI_PERIPH_ID_6	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Reserved											
	Reset value																									0	0	0	0	0	0	0	0				
0x1FDC	AXI_PERIPH_ID_7	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Reserved											
	Reset value																									0	0	0	0	0	0	0	0				
0x1FE0	AXI_PERIPH_ID_0	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PARTNUM[7:0]											
	Reset value																									0	0	0	0	0	0	0	0				
0x1FE4	AXI_PERIPH_ID_1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JEP106ID [3:0]				PARTNUM [11:8]							
	Reset value																									1	0	1	1	0	1	0	0				
0x1FE8	AXI_PERIPH_ID_2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	REVISION [3:0]				JEDEC JEP106ID [6:4]							
	Reset value																									0	0	1	0	1	0	1	1				
0x1FEC	AXI_PERIPH_ID_3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	REV_AND[3:0]				CUST_MOD_NUM [3:0]							
	Reset value																									0	0	0	0	0	0	0	0				
0x1FF0	AXI_COMP_ID_0																									PREAMBLE[7:0]											
	Reset value																									0	0	0	0	1	1	0	1				
0x1FF4	AXI_COMP_ID_1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CLASS[3:0]				PREAMBLE [11:8]							
	Reset value																									1	1	1	1	0	0	0	0				

Table 5. AXI interconnect register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
0x1FF8	AXI_COMP_ID_2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PREAMBLE[19:12]											
	Reset value																									0	0	0	0	0	0	1	0	1			
0x1FFC	AXI_COMP_ID_3	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PREAMBLE[27:20]											
	Reset value																									1	0	1	1	0	0	0	0	1			
0x2000 - 0x2004	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
0x2008	AXI_TARG1_FN_MOD_ISS_BM	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
	Reset value																															0	WRITE_ISS_OVERRIDE	0	READ_ISS_OVERRIDE		
0x200C - 0x2020	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
0x2024	AXI_TARG1_FN_MOD2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
	Reset value																															0	BYPASS_MERGE				
0x2028	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
0x202C	AXI_TARG1_FN_MOD_LB	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
	Reset value																															0	FN_MOD_LB				
0x2030 - 0x2104	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
0x2108	AXI_TARG1_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
	Reset value																														0	WRITE_ISS_OVERRIDE	0	READ_ISS_OVERRIDE			
0x210C - 0x3004	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			

Table 5. AXI interconnect register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x3008	AXI_TARG2_FN_MOD_ISS_BM	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE ISS OVERRIDE	READ ISS OVERRIDE
	Reset value																															0	0
0x300C - 0x3020	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
0x3024	AXI_TARG2_FN_MOD2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BYPASS_MERGE
	Reset value																																0
0x3028	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
0x302C	AXI_TARG2_FN_MOD_LB	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FN_MOD_LB
	Reset value																																0
0x3030 - 0x3104	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
0x3108	AXI_TARG2_FN_MOD	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE ISS OVERRIDE
	Reset value																																0
0x310C - 0x4004	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
0x4008	AXI_TARG3_FN_MOD_ISS_BM	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE ISS OVERRIDE
	Reset value																																0
0x400C - 0x5004	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Table 5. AXI interconnect register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x5008	AXI_TARG4_ FN_MOD_ ISS_BM	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE ISS OVERRIDE	READ ISS OVERRIDE
	Reset value																															0	0
0x500C - 0x6004	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
0x6008	AXI_TARG5_ FN_MOD_ ISS_BM	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE ISS OVERRIDE	READ ISS OVERRIDE
	Reset value																															0	0
0x600C - 0x7004	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
0x7008	AXI_TARG6_ FN_MOD_ ISS_BM	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE ISS OVERRIDE	READ ISS OVERRIDE
	Reset value																															0	0
0x700C - 0x8004	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
0x8008	AXI_TARG7_ FN_MOD_ ISS_BM	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRITE ISS OVERRIDE	READ ISS OVERRIDE
	Reset value																															0	0
0x800C - 0x8020	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
0x8024	AXI_TARG7_ FN_MOD2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BYPASS_MERGE
	Reset value																															0	

Table 5. AXI interconnect register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x8028 - 0x8104	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
0x8108	AXI_TARG7_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	0	WRITE ISS OVERRIDE
	Reset value																															0	READ ISS OVERRIDE
0x810C-0x42020	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
0x42024	AXI_INI1_FN_MOD2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res		BYPASS_MERGE
	Reset value																															0	
0x42028	AXI_INI1_FN_MOD_AHB	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res		WR_INC_OVERRIDE	RD_INC_OVERRIDE
	Reset value																														0	0	
0x4202C-0x420FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
0x42100	AXI_INI1_READ_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AR_QOS [3:0]			
	Reset value																													0	0	0	0
0x42104	AXI_INI1_WRITE_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AW_QOS [3:0]			
	Reset value																													0	0	0	0
0x42108	AXI_INI1_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	0	WRITE ISS OVERRIDE
	Reset value																														0	READ ISS OVERRIDE	
0x4210C-0x430FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res

Table 5. AXI interconnect register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x43100	AXI_INI2_READ_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AR_QOS [3:0]					
	Reset value																													0	0	0	0		
0x43104	AXI_INI2_WRITE_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AW_QOS [3:0]					
	Reset value																													0	0	0	0		
0x43108	AXI_INI2_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
	Reset value																																0	0	0
0x4310C - 0x44020	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
0x44024	AXI_INI3_FN_MOD2	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
	Reset value																																	0	0
0x44028	AXI_INI3_FN_MOD_AHB	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
	Reset value																																0	0	0
0x4402C-0x440FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
0x44100	AXI_INI3_READ_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AR_QOS [3:0]					
	Reset value																													0	0	0	0	0	
0x44104	AXI_INI3_WRITE_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AW_QOS [3:0]					
	Reset value																													0	0	0	0	0	

Table 5. AXI interconnect register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x44108	AXI_INI3_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WRITE ISS OVERRIDE	READ ISS OVERRIDE
	Reset value																															0	0	
0x4410C-0x450FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
0x45100	AXI_INI4_READ_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AR_QOS [3:0]				
	Reset value																													0	0	0	0	
0x45104	AXI_INI4_WRITE_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AW_QOS [3:0]				
	Reset value																													0	0	0	0	
0x45108	AXI_INI4_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WRITE ISS OVERRIDE	READ ISS OVERRIDE
	Reset value																														0	0		
0x4510C-0x460FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
0x46100	AXI_INI5_READ_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AR_QOS [3:0]				
	Reset value																													0	0	0	0	
0x46104	AXI_INI5_WRITE_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AW_QOS [3:0]				
	Reset value																													0	0	0	0	
0x46108	AXI_INI5_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WRITE ISS OVERRIDE	READ ISS OVERRIDE
	Reset value																														0	0		
0x4610C-0x470FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	
0x47100	AXI_INI6_READ_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AR_QOS [3:0]				
	Reset value																													0	0	0	0	

Table 5. AXI interconnect register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x47104	AXI_INI6_WRITE_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AW_QOS [3:0]			
	Reset value																													0	0	0	0
0x47108	AXI_INI6_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WRITE ISS OVERRIDE		READ ISS OVERRIDE
	Reset value																														0	0	0
0x4710C-0x480FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
0x48100	AXI_INI7_READ_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AR_QOS [3:0]			
	Reset value																													0	0	0	0
0x48104	AXI_INI7_WRITE_QOS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	AW_QOS [3:0]			
	Reset value																													0	0	0	0
0x48108	AXI_INI7_FN_MOD	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WRITE ISS OVERRIDE		READ ISS OVERRIDE
	Reset value																														0	0	0

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

2.3 Memory organization

2.3.1 Introduction

Program memory, data memory, registers and I/O ports are organized within the same linear 4-Gbyte address space.

The bytes are coded in memory in Little Endian format. The lowest numbered byte in a word is considered the word's least significant byte and the highest numbered byte the most significant.

The addressable memory space is divided into eight main blocks, of 512 Mbytes each.

2.3.2 Memory map and register boundary addresses

Table 6. Memory map and default device memory area attributes

Region	Boundary address	Arm® Cortex®-M7	Arm® Cortex®-M4	Type	Attributes	Execute never
External devices	0xD0000000 - 0xD3FFFFFF	FMC SDRAM bank 2 (or Reserved in case of FMC remap)		Device	-	Y
	0xCC000000 - 0xCFFFFFFF	FMC SDRAM bank 1 (or remap of FMC NOR/PSRAM/SRAM 4 bank 1)				
	0xC8000000 - 0xCBFFFFFF	FMC SDRAM bank 1 (or remap of FMC NOR/PSRAM/SRAM 3 bank 1)				
	0xC4000000 - 0xC7FFFFFF	FMC SDRAM bank 1 (or remap of FMC NOR/PSRAM/SRAM 2 bank 1)				
	0xC0000000 - 0xC3FFFFFF	FMC SDRAM bank 1 (or remap of FMC NOR/PSRAM/SRAM 1 bank 1)				
	0xA0000000 - 0xBFFFFFFF	Reserved				
External Memories	0x90000000 - 0x9FFFFFFF	QUADSPI		Normal	Write-through	N
	0x80000000 - 0x8FFFFFFF	FMC NAND Flash memory			Write-Back Write Allocate	
	0x70000000 - 0x7FFFFFFF	Reserved (or remap of FMC SDRAM bank 2)				
	0x6C000000 - 0x6FFFFFFF	FMC NOR/PSRAM/SRAM 4 bank 1 (or remap of FMC SDRAM bank 1)				
	0x68000000 - 0x6BFFFFFF	FMC NOR/PSRAM/SRAM 3 bank 1 (or remap of FMC SDRAM bank 1)				
	0x64000000 - 0x67FFFFFF	FMC NOR/PSRAM/SRAM 2 bank 1 (or remap of FMC SDRAM bank 1)				
	0x60000000 - 0x63FFFFFF	FMC NOR/PSRAM/SRAM 1 bank 1 (or remap of FMC SDRAM bank 1)				
Peripherals	0x40000000 - 0x5FFFFFFF	Peripherals (refer to Table 7: Register boundary addresses)		Device	-	Y

Table 6. Memory map and default device memory area attributes (continued)

Region	Boundary address	Arm® Cortex®-M7	Arm® Cortex®-M4	Type	Attributes	Execute never
RAM	0x38801000 - 0x3FFFFFFF	Reserved		Normal	Write-Back Write Allocate	N
	0x38800000 - 0x38800FFF	Backup SRAM				
	0x38010000 - 0x387FFFFF	Reserved				
	0x38000000 - 0x3800FFFF	SRAM4				
	0x30048000 - 0x37FFFFFF	Reserved				
	0x30040000 - 0x30047FFF	SRAM3				
	0x30020000 - 0x3003FFFF	SRAM2				
	0x30000000 - 0x3001FFFF	SRAM1				
	0x24080000 - 0x2FFFFFFF	Reserved				
	0x24000000 - 0x2407FFFF	AXI SRAM				
	0x20020000 - 0x23FFFFFF	Reserved				
	0x20000000 - 0x2001FFFF	DTCM	Reserved			

Table 6. Memory map and default device memory area attributes (continued)

Region	Boundary address	Arm® Cortex®-M7	Arm® Cortex®-M4	Type	Attributes	Execute never
Code	0x1FF20000 - 0x1FFFFFFF	Reserved		Normal	Write-through	N
	0x1FF00000 - 0x1FF1FFFF	System Memory	Reserved			
	0x10048000 - 0x1FEFFFFF	Reserved				
	0x10040000 - 0x10047FFF	SRAM3 (Alias) ⁽¹⁾				
	0x10020000 - 0x1003FFFF	SRAM2 (Alias) ⁽¹⁾				
	0x10000000 - 0x1001FFFF	SRAM1 (Alias) ⁽¹⁾				
	0x08200000 - 0x0FFFFFFF	Reserved				
	0x08100000 - 0x081FFFFF	Flash memory bank 2 ⁽²⁾				
	0x08000000 - 0x080FFFFF	Flash memory bank 1 ⁽³⁾				
	0x00010000 - 0x07FFFFFF	Reserved				
	0x00000000 - 0x0000FFFF	ITCM	VTOR REMAP ⁽⁴⁾			

1. Alias to maintain Arm® Cortex®-M4 Harvard architecture.
2. Flash memory bank 2 boundary is limited to 0x08100000 - 0x0817FFFF on STM32H745xG/STM32H747xG.
3. Flash memory bank 1 boundary is limited to 0x08000000 - 0x0807FFFF on STM32H745xG/STM32H747xG.
4. Selectable boot memory alias.

All the memory map areas that are not allocated to on-chip memories and peripherals are considered “Reserved”. For the detailed mapping of available memory and register areas, refer to the following table.

The following table gives the boundary addresses of the peripherals available in the devices.

Table 7. Register boundary addresses⁽¹⁾

Boundary address	Peripheral	Bus	Register map
0x58027000 - 0x580273FF	RAMECC3	AHB4 (D3)	Section 3.4: RAMECC registers
0x58026400 - 0x580267FF	HSEM		Section 11.4: HSEM registers
0x58026000 - 0x580263FF	ADC3		Section 26.7: ADC common registers
0x58025800 - 0x58025BFF	DMAMUX2		Section 18.6: DMAMUX registers
0x58025400 - 0x580257FF	BDMA		Section 17.6: BDMA registers
0x58024C00 - 0x58024FFF	CRC		Section 22.4: CRC registers
0x58024800 - 0x58024BFF	PWR		Section 7.8: PWR registers
0x58024400 - 0x580247FF	RCC		Section 9.7: RCC registers
0x58022800 - 0x58022BFF	GPIOK		Section 12.4: GPIO registers
0x58022400 - 0x580227FF	GPIOK		Section 12.4: GPIO registers
0x58022000 - 0x580223FF	GPIOK		Section 12.4: GPIO registers
0x58021C00 - 0x58021FFF	GPIOK		Section 12.4: GPIO registers
0x58021800 - 0x58021BFF	GPIOK		Section 12.4: GPIO registers
0x58021400 - 0x580217FF	GPIOK		Section 12.4: GPIO registers
0x58021000 - 0x580213FF	GPIOK		Section 12.4: GPIO registers
0x58020C00 - 0x58020FFF	GPIOD		Section 12.4: GPIO registers
0x58020800 - 0x58020BFF	GPIOD		Section 12.4: GPIO registers
0x58020400 - 0x580207FF	GPIOD		Section 12.4: GPIO registers
0x58020000 - 0x580203FF	GPIOD		Section 12.4: GPIO registers

Table 7. Register boundary addresses⁽¹⁾ (continued)

Boundary address	Peripheral	Bus	Register map
0x58005800 - 0x580067FF	Reserved	APB4 (D3)	Reserved
0x58005400 - 0x580057FF	SAI4		Section 54.6: SAI registers
0x58004C00 - 0x58004FFF	IWDG2		Section 48.4: IWDG registers
0x58004800 - 0x58004BFF	IWDG1		Section 48.4: IWDG registers
0x58004000 - 0x580043FF	RTC & BKP registers		Section 49.6: RTC registers
0x58003C00 - 0x58003FFF	VREF		Section 28.3: VREFBUF registers
0x58003800 - 0x58003BFF	COMP1 - COMP2		Section 29.7: COMP registers
0x58003000 - 0x580033FF	LPTIM5		Section 45.7: LPTIM registers
0x58002C00 - 0x58002FFF	LPTIM4		Section 45.7: LPTIM registers
0x58002800 - 0x58002BFF	LPTIM3		Section 45.7: LPTIM registers
0x58002400 - 0x580027FF	LPTIM2		Section 45.7: LPTIM registers
0x58001C00 - 0x58001FFF	I2C4		Section 50.7: I2C registers
0x58001400 - 0x580017FF	SPI6		Section 53.11: SPI/I2S registers
0x58000C00 - 0x58000FFF	LPUART1		Section 52.7: LPUART registers
0x58000400 - 0x580007FF	SYSCFG		Section 13.3: SYSCFG registers
0x58000000 - 0x580003FF	EXTI		Section 21.6: EXTI registers
0x52009000 - 0x520093FF	RAMECC1	AHB3 (D1)	Section 3.4: RAMECC registers
0x52008000 - 0x52008FFF	Delay Block SDMMC1		Section 25.4: DLYB registers
0x52007000 - 0x52007FFF	SDMMC1		Section 58.10: SDMMC registers
0x52006000 - 0x52006FFF	Delay Block QUADSPI		Section 25.4: DLYB registers
0x52005000 - 0x52005FFF	QUADSPI control registers		Section 24.5: QUADSPI registers
0x52004000 - 0x52004FFF	FMC control registers		Section 23.7.6: NOR/PSRAM controller registers, Section 23.8.7: NAND flash controller registers, Section 23.9.5: SDRAM controller registers
0x52003000 - 0x52003FFF	JPEG		Section 35.5: JPEG codec registers
0x52002000 - 0x52002FFF	Flash interface registers		Section 4.9: FLASH registers
0x52001000 - 0x52001FFF	Chrom-Art (DMA2D)		Section 19.5: DMA2D registers
0x52000000 - 0x52000FFF	MDMA		Section 15.5: MDMA registers
0x51000000 - 0x510FFFFF	GPV		Section 2.2.4: AXI interconnect registers
0x50003000 - 0x50003FFF	WWDG1	APB3 (D1)	Section 47.5: WWDG registers
0x50001000 - 0x50001FFF	LTDC		Section 33.7: LTDC registers
0x50000000 - 0x50000FFF	DSIHOST		Section 34.15: DSI Host registers, Section 34.16: DSI Wrapper registers

Table 7. Register boundary addresses⁽¹⁾ (continued)

Boundary address	Peripheral	Bus	Register map
0x48023000 - 0x480233FF	RAMECC2	AHB2 (D2)	Section 3.4: RAMECC registers
0x48022800 - 0x48022BFF	Delay Block SDMMC2		Section 25.4: DLYB registers
0x48022400 - 0x480227FF	SDMMC2	AHB2 (D2)	Section 58.10: SDMMC registers
0x48021800 - 0x48021BFF	RNG		Section 36.7: RNG registers
0x48021400 - 0x480217FF	HASH		Section 38.7: HASH registers
0x48021000 - 0x480213FF	CRYPTO		Section 37.7: CRYPT registers
0x48020000 - 0x480203FF	DCMI		Section 32.5: DCMI registers
0x40080000 - 0x400BFFFF	USB2 OTG_FS	AHB1 (D2)	Section 60.14: OTG_HS registers
0x40040000 - 0x4007FFFF	USB1 OTG_HS		Section 60.14: OTG_HS registers
0x40028000 - 0x400293FF	ETHERNET MAC		Section 61.11: Ethernet registers
0x40024400 - 0x400247FF	ART control registers		-
0x40022000 - 0x400223FF	ADC1 - ADC2		Section 26.7: ADC common registers
0x40020800 - 0x40020BFF	DMAMUX1		Section 18.6: DMAMUX registers
0x40020400 - 0x400207FF	DMA2		Section 16.5: DMA registers
0x40020000 - 0x400203FF	DMA1		Section 16.5: DMA registers
0x40017400 - 0x400177FF	HRTIM	APB2 (D2)	Section 39.5: HRTIM registers
0x40017000 - 0x400173FF	DFSDM1		Section 31.7: DFSDM channel y registers (y=0..7), Section 31.8: DFSDM filter x module registers (x=0..3)
0x40016000 - 0x400163FF	SAI3		Section 54.6: SAI registers
0x40015C00 - 0x40015FFF	SAI2		Section 54.6: SAI registers
0x40015800 - 0x40015BFF	SAI1		Section 54.6: SAI registers
0x40015000 - 0x400153FF	SPI5		Section 53.11: SPI/I2S registers
0x40014800 - 0x40014BFF	TIM17		Section 43.6: TIM16/TIM17 registers
0x40014400 - 0x400147FF	TIM16		Section 43.6: TIM16/TIM17 registers
0x40014000 - 0x400143FF	TIM15		Section 43.5: TIM15 registers
0x40013400 - 0x400137FF	SPI4		Section 53.11: SPI/I2S registers
0x40013000 - 0x400133FF	SPI1 / I2S1		Section 53.11: SPI/I2S registers
0x40011400 - 0x400117FF	USART6		Section 51.8: USART registers
0x40011000 - 0x400113FF	USART1		Section 51.8: USART registers
0x40010400 - 0x400107FF	TIM8		Section 40.4: TIM1/TIM8 registers
0x40010000 - 0x400103FF	TIM1		Section 40.4: TIM1/TIM8 registers

Table 7. Register boundary addresses⁽¹⁾ (continued)

Boundary address	Peripheral	Bus	Register map
0x4000AC00 - 0x4000D3FF	CAN Message RAM	APB1 (D2)	Section 59.5: FDCAN registers
0x4000A800 - 0x4000ABFF	CAN CCU		Section 59.5: FDCAN registers
0x4000A400 - 0x4000A7FF	FDCAN2		Section 59.5: FDCAN registers
0x4000A000 - 0x4000A3FF	FDCAN1		Section 59.5: FDCAN registers
0x40009400 - 0x400097FF	MDIOS		Section 57.4: MDIOS registers
0x40009000 - 0x400093FF	OPAMP		Section 30.6: OPAMP registers
0x40008800 - 0x40008BFF	SWPMI		Section 56.6: SWPMI registers
0x40008400 - 0x400087FF	CRS		Section 10.8: CRS registers
0x40007C00 - 0x40007FFF	UART8		Section 51.8: USART registers
0x40007800 - 0x40007BFF	UART7		Section 51.8: USART registers
0x40007400 - 0x400077FF	DAC1		Section 27.7: DAC registers
0x40006C00 - 0x40006FFF	HDMI-CEC		Section 62.7: HDMI-CEC registers
0x40005C00 - 0x40005FFF	I2C3		Section 50.7: I2C registers
0x40005800 - 0x40005BFF	I2C2		Section 50.7: I2C registers
0x40005400 - 0x400057FF	I2C1		Section 50.7: I2C registers
0x40005000 - 0x400053FF	UART5		Section 51.8: USART registers
0x40004C00 - 0x40004FFF	UART4		Section 51.8: USART registers
0x40004800 - 0x40004BFF	USART3		Section 51.8: USART registers
0x40004400 - 0x400047FF	USART2		Section 51.8: USART registers
0x40004000 - 0x400043FF	SPDIFRX		Section 55.5: SPDIFRX interface registers
0x40003C00 - 0x40003FFF	SPI3 / I2S3		Section 53.11: SPI/I2S registers
0x40003800 - 0x40003BFF	SPI2 / I2S2		Section 53.11: SPI/I2S registers
0x40002C00 - 0x40002FFF	WWDG2		Section 47.5: WWDG registers
0x40002400 - 0x400027FF	LPTIM1		Section 45.7: LPTIM registers
0x40002000 - 0x400023FF	TIM14		Section 41.4: TIM2/TIM3/TIM4/TIM5 registers
0x40001C00 - 0x40001FFF	TIM13		Section 41.4: TIM2/TIM3/TIM4/TIM5 registers
0x40001800 - 0x40001BFF	TIM12		Section 41.4: TIM2/TIM3/TIM4/TIM5 registers
0x40001400 - 0x400017FF	TIM7		Section 44.4: TIM6/TIM7 registers
0x40001000 - 0x400013FF	TIM6		Section 44.4: TIM6/TIM7 registers
0x40000C00 - 0x40000FFF	TIM5		Section 41.4: TIM2/TIM3/TIM4/TIM5 registers
0x40000800 - 0x40000BFF	TIM4		Section 41.4: TIM2/TIM3/TIM4/TIM5 registers
0x40000400 - 0x400007FF	TIM3		Section 41.4: TIM2/TIM3/TIM4/TIM5 registers
0x40000000 - 0x400003FF	TIM2		Section 41.4: TIM2/TIM3/TIM4/TIM5 registers

1. Accessing a reserved area results in a bus error. Accessing undefined memory space in a peripheral returns zeros.

2.4 Embedded SRAM

The STM32H745/55/47/57xx devices feature:

- Up to 864 Kbytes of System SRAM
- 128 Kbytes of data TCM RAM
- 64 Kbytes of instruction TCM RAM
- 4 Kbytes of backup SRAM

The embedded system SRAM is divided into up to five blocks over the three power domains:

- D1 domain, AXI SRAM:
 - AXI SRAM is mapped at address 0x2400 0000 and accessible by all system masters except BDMA through D1 domain AXI bus matrix. AXI SRAM can be used for application data which are not allocated in DTCM RAM or reserved for graphic objects (such as frame buffers)
- D2 domain, AHB SRAM:
 - AHB SRAM1 is mapped at address 0x3000 0000 and accessible by all system masters except BDMA through D2 domain AHB matrix. AHB SRAM1 can be used as DMA buffers to store peripheral input/output data in D2 domain, or as code location for Cortex[®]-M4 CPU (application code available when D1 is powered off).
 - AHB SRAM2 is mapped at address 0x3002 0000 and accessible by all system masters except BDMA through D2 domain AHB matrix. AHB SRAM2 can be used as DMA buffers to store peripheral input/output data in D2 domain, or as read-write segment for application running on Cortex[®]-M4 CPU.
 - AHB SRAM3 is mapped at address 0x3004 0000 and accessible by all system masters except BDMA through D2 domain AHB matrix. AHB SRAM3 can be used as buffers to store peripheral input/output data for Ethernet and USB, or as shared memory between the two cores.
- D3 domain, AHB SRAM:
 - AHB SRAM4 is mapped at address 0x3800 0000 and accessible by most of system masters through D3 domain AHB matrix. AHB SRAM4 can be used as BDMA buffers to store peripheral input/output data in D3 domain. It can also be used to retain some application code/data when D1 and D2 domain enter DStandby mode, or as shared memory between the two cores.

The AHB SRAMs of the D2 domain are also aliased to an address range below 0x2000 0000 to maintain the Cortex[®]-M4 Harvard architecture:

- AHB SRAM1 also mapped at address 0x1000 0000 and accessible by all system masters through D2 domain AHB matrix
- AHB SRAM2 also mapped at address 0x1002 0000 and accessible by all system masters through D2 domain AHB matrix
- AHB SRAM3 also mapped at address 0x1004 0000 and accessible by all system masters through D2 domain AHB matrix

The system AHB SRAM can be accessed as bytes, half-words (16-bit units) or words (32-bit units), while the system AXI SRAM can be accessed as bytes, half-words, words or double-words (64-bit units). These memories can be addressed at maximum system clock frequency without wait state.

The AHB masters can read/write-access an SRAM section concurrently with the Ethernet MAC or the USB OTG HS peripheral accessing another SRAM section. For example, the Ethernet MAC accesses the SRAM2 while the CPU accesses the SRAM1, concurrently.

The TCM SRAMs are dedicated to the Cortex[®]-M7:

- DTCM-RAM on TCM interface is mapped at the address 0x2000 0000 and accessible by Cortex[®]-M7, and by MDMA through AHBS slave bus of the Cortex[®]-M7 CPU. The DTCM-RAM can be used as read-write segment to host critical real-time data (such as stack and heap) for application running on Cortex[®]-M7 CPU.
- ITCM-RAM on TCM interface mapped at the address 0x0000 0000 and accessible by Cortex[®]-M7 and by MDMA through AHBS slave bus of the Cortex[®]-M7 CPU. The ITCM-RAM can be used to host code for time-critical routines (such as interrupt handlers) that requires deterministic execution.

The backup RAM is mapped at the address 0x3880 0000 and is accessible by most of the system masters through D3 domain's AHB matrix. With a battery connected to the V_{BAT} pin, the backup SRAM can be used to retain data during low-power mode (Standby and V_{BAT} mode).

Error code correction (ECC)

SRAM data are protected by ECC:

- 7 ECC bits are added per 32-bit word.
- 8 ECC bits are added per 64-bit word for AXI-SRAM and ITCM-RAM.

The ECC mechanism is based on the SECDED algorithm. It supports single-error correction and double-error detection.

When an incomplete word is written to an internal SRAM and a reset occurs, the last incomplete word is not really written. This is due to the ECC behavior. To ensure that an incomplete word is written to SRAM, write an additional dummy incomplete word to the same RAM at a different address before issuing a reset.

2.5 Flash memory overview

The Flash memory interface manages CPU AXI accesses to the Flash memory. It implements the erase and program Flash memory operations and the read and write protection mechanisms.

The Flash memory is organized as follows:

- Two main memory block divided into sectors.
- An information block:
 - System memory from which the device boots in System memory boot mode
 - Option bytes to configure read and write protection, BOR level, watchdog software/hardware and reset when the device is in Standby or Stop mode.

Refer to [Section 4: Embedded flash memory \(FLASH\)](#) for more details.

2.6 Boot configuration

In the STM32H745/55/47/57xx, the two cores can boot alone or in the same time according to the option bytes as shown in [Table 8](#).

Table 8. Boot order

BCM7	BCM4	Boot order
0	0	Cortex [®] -M7 is booting and Cortex [®] -M4 clock is gated
0	1	Cortex [®] -M7 clock is gated and Cortex [®] -M4 is booting
1	0	Cortex [®] -M7 is booting and Cortex [®] -M4 clock is gated
1	1	Both Cortex [®] -M7 and Cortex [®] -M4 are booting

Two different boot areas per core can be selected through the BOOT pin and the boot base address programmed, as shown in the [Table 9](#), in the:

- BCM7_ADD0 and BCM7_ADD1 option bytes for Cortex[®]-M7
- BCM4_ADD0 and BCM4_ADD1 option bytes for Cortex[®]-M4

Table 9. Boot modes

Boot mode selection		Boot area
BOOT	Boot address option bytes	
0	BCM7_ADD0[15:0]	Cortex [®] -M7 boot address defined by user option byte BCM7_ADD0[15:0] ST programmed value: Flash memory at 0x0800 0000
	BCM4_ADD0[15:0]	Cortex [®] -M4 boot address defined by user option byte BCM4_ADD0[15:0] ST programmed value: Flash memory at 0x0810 0000
1	BCM7_ADD1[15:0]	Cortex [®] -M7 boot address defined by user option byte BCM7_ADD1[15:0] ST programmed value: TCM-RAM at 0x000 0000 or bootloader
	BCM4_ADD1[15:0]	Cortex [®] -M4 boot address defined by user option byte BCM4_ADD1[15:0] ST programmed value: SRAM1 at 0x1000 0000

The values on the BOOT pin are latched on the 4th rising edge of SYSCLK after reset release. It is up to the user to set the BOOT pin after reset.

The BOOT pin is also re-sampled when the device exits the Standby mode. Consequently, they must be kept in the required Boot mode configuration when the device is in the Standby mode.

After startup delay, the selection of the boot area is done before releasing the processor reset.

The BOOT_ADD0 and BOOT_ADD1 address option bytes allows to program any boot memory address from 0x0000 0000 to 0x3FFF 0000 which includes:

- All Flash address space
- All RAM address space: ITCM, DTCM RAMs and SRAMs
- The TCM-RAM

The BCM4_ADD0 and BCM4_ADD1 address option bytes allows to program any boot memory address from 0x0000 0000 to 0x3FFF 0000 which includes:

- all Flash memory address space
- all system RAM address space
- SRAM1

The BOOT_ADD0 / BOOT_ADD1 / BCM4_ADD0 / BCM4_ADD1 option bytes can be modified after reset in order to boot from any other boot address after next reset.

If the programmed boot memory address is out of the memory mapped area or a reserved area, the default boot fetch address is programmed as follows:

- Cortex[®]-M7 Boot address 0: FLASH at 0x0800 0000
- Cortex[®]-M7 Boot address 1: ITCM-RAM at 0x0000 0000
- Cortex[®]-M4 Boot address 0: FLASH at 0x0810 0000
- Cortex[®]-M4 Boot address 1: SRAM1 at 0x1000 0000

When the Flash level 2 protection is enabled, only boot from Flash memory is available. If the boot address already programmed in the BOOT_ADD0 / BOOT_ADD1 / BCM4_ADD0 / BCM4_ADD1 option bytes is out of the memory range or belongs to the RAM address range, the default fetch will be forced from Flash memory at address 0x0800 0000 for Cortex[®]-M7 and Flash at address 0x0810 0000 for Cortex[®]-M4.

Embedded bootloader

The embedded bootloader code is located in system memory. It is programmed by ST during production. It is used to reprogram the Flash memory using one of the following serial interfaces:

- USART1 on PA9/PA10 and PB14/PB15 pins, USART2 on PA3/PA2 pins, and USART3 on PB10/PB11 pins.
- I2C1 on PB6/PB9 pins, I2C2 on PF0/PF1 pins, and I2C3 on PA8/PC9 pins.
- USB OTG FS in Device mode (DFU) on PA11/PA12 pins
- SPI1 on PA4/PA5/PA6/PA7 pins, SPI2 on PI0/PI1/PI2/PI3 pins, SPI3 on PC10/PC11/PC12/PA15 pins, and SPI4 on PE11/PE12/PE13/PE14 pins.

For additional information, refer to the application note AN2606.

4 Embedded flash memory (FLASH)

4.1 Introduction

The embedded flash memory (FLASH) manages the accesses of any master to the embedded non-volatile memory, that is 2 Mbytes. It implements the read, program and erase operations, error corrections as well as various integrity and confidentiality protection mechanisms.

The embedded flash memory manages the automatic loading of non-volatile user option bytes at power-on reset, and implements the dynamic update of these options.

4.2 FLASH main features

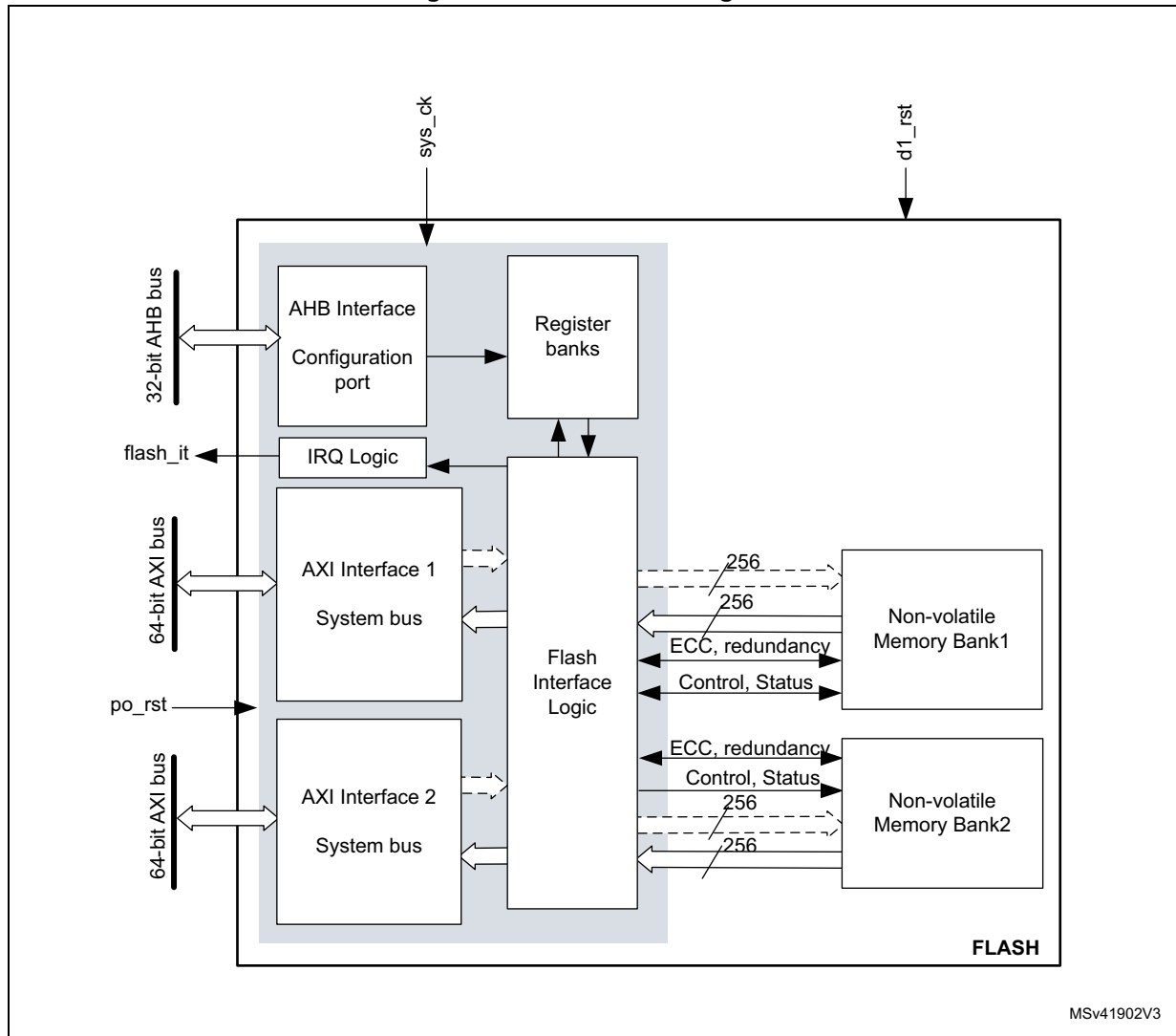
- 2 Mbytes of non-volatile memory divided into two banks of 1 Mbyte each
- flash memory read operations supporting multiple length (64 bits, 32bits, 16bits or one byte)
- Flash memory programming by 256 bits
- 128-Kbyte sector erase, bank erase and dual-bank mass erase
- Dual-bank organization supporting:
 - simultaneous operations: two read/program/erase operations executed in parallel on both banks
 - Bank swapping: the address mapping of the user flash memory of each bank can be swapped, along with the corresponding registers.
- Error Code Correction (ECC): one error detection/correction or two error detections per 256-bit flash word using 10 ECC bits
- Cyclic redundancy check (CRC) hardware module
- User configurable non-volatile option bytes
- Flash memory enhanced protections, activated by option bytes
 - Read protection (RDP), preventing unauthorized flash memory dump to safeguard sensitive application code
 - Write-protection of sectors (WRPS), available per bank (128 Kbyte sectors)
 - Two proprietary code readout protection (PCROP) areas (one per user flash bank). When enabled, this area is execute-only.
 - Two secure-only areas (one per user flash bank). When enabled this area is accessible only if the STM32 microcontroller operates in Secure access mode.
- Read and write command queues to streamline flash operations

4.3 FLASH functional description

4.3.1 FLASH block diagram

Figure 5 shows the embedded flash memory block diagram.

Figure 5. FLASH block diagram



4.3.2 FLASH internal signals

[Table 13](#) describes a list of the useful to know internal signals available at embedded flash memory level. These signals are not available on the microcontroller pads.

Table 13. FLASH internal input/output signals

Internal signal name	Signal type	Description
sys_ck	Input	D1 domain bus clock (embedded flash memory AXI interface clock)
po_rst	Input	Power on reset
d1_rst	Input	D1 domain system reset
flash_it	Output	Embedded flash memory interface interrupt request

4.3.3 FLASH architecture and integration in the system

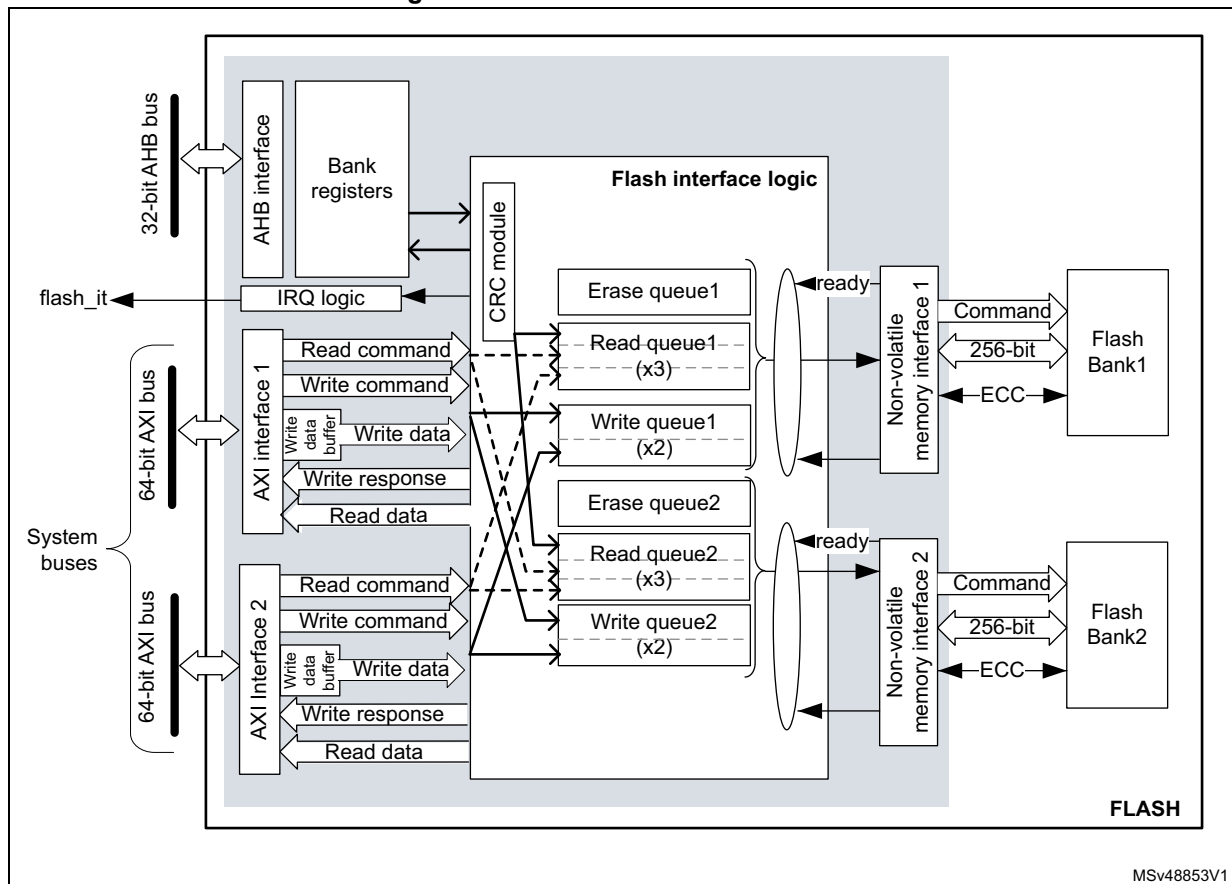
The embedded flash memory is a central resource for the whole microcontroller. It serves as an interface to two non-volatile memory banks, and organizes the memory in a very specific way. The embedded flash memory also proposes a set of security features to protect the assets stored in the non-volatile memory at boot time, at run-time and during firmware and configuration upgrades.

The embedded flash memory offers two 64-bit AXI slave ports for code and data accesses, plus a 32-bit AHB configuration slave port used for register bank accesses.

Note: The application can simultaneously request a read and a write operation through each AXI interface.

The embedded flash memory microarchitecture is shown in [Figure 6](#).

Figure 6. Detailed FLASH architecture



Behind the system interfaces, the embedded flash memory implements various command queues and buffers to perform flash read, write and erase operations with maximum efficiency.

Thanks to the addition of a read and write data buffer, the AXI slave port handles the following access types:

- Multiple length: 64 bits, 32 bits, 16 bits and 8 bits
- Single or burst accesses
- Write wrap burst must not cross 32-byte aligned address boundaries to target exactly one flash word

The AHB configuration slave port supports 8-bit, 16-bit and 32-bit word accesses.

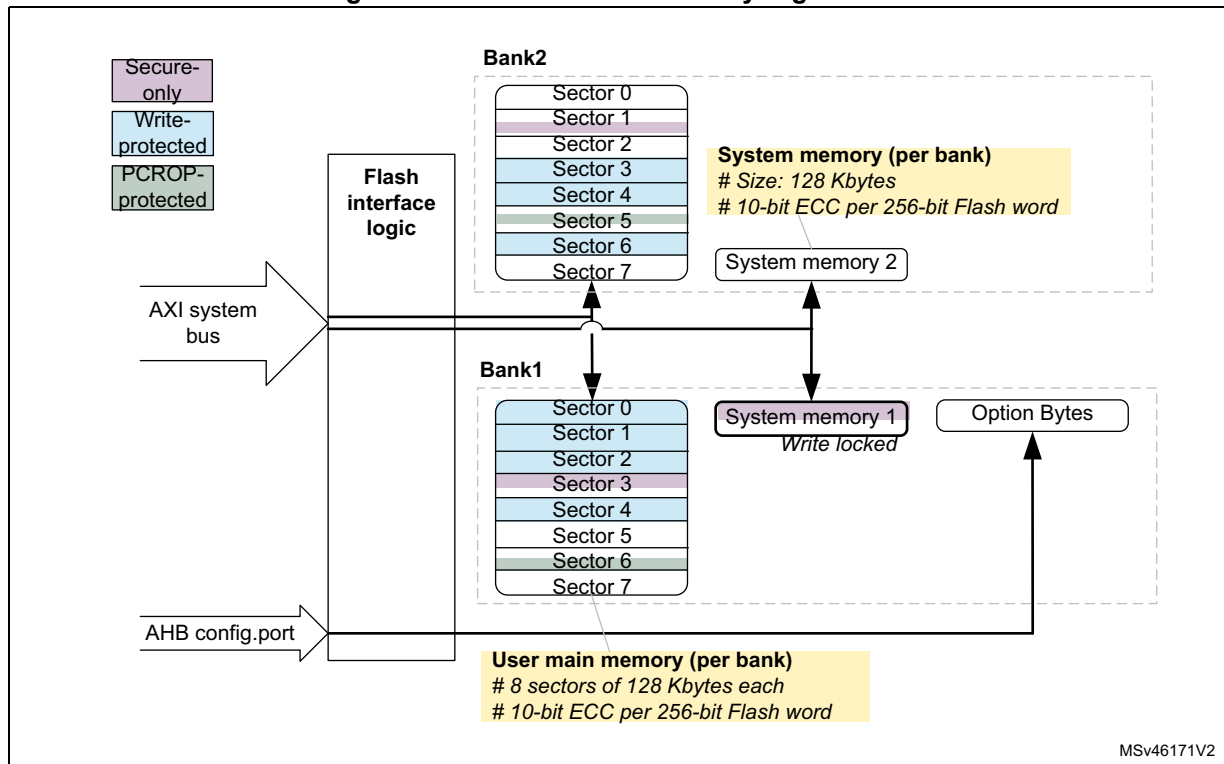
The embedded flash memory is built in such a way that only one read or write operation can be executed at a time on a given bank.

4.3.4 Flash memory architecture and usage

Flash memory architecture

Figure 7 shows the non-volatile memory organization supported by the embedded flash memory.

Figure 7. Embedded flash memory organization



The embedded flash non-volatile memory is composed of:

- A 2-Mbyte main memory block, organized as two banks of 1 Mbyte each. Each bank is in turn divided in eight 128-Kbyte sectors and features flash-word rows of 256 bits + 10 bits of ECC per word
- A system memory block of 256 Kbytes, divided into two 128 Kbyte banks. The system memory is ECC protected.
- A set of non-volatile option bytes loaded at reset by the embedded flash memory and accessible by the application software only through the AHB configuration register interface.

The overall flash memory architecture is summarized in [Table 14](#) and [Table 15](#).

Table 14. Flash memory organization on STM32H745xI/747xI/755xI/757xI devices

Flash memory area		Address range	Size (bytes)	Region name	Access interface	SNB1/2 ⁽¹⁾
User main memory	Bank 1	0x0800 0000-0x0801 FFFF	128 K	Sector 0	AXI ports	0x0
		0x0802 0000-0x0803 FFFF	128 K	Sector 1		0x1
		...	...	...		...
		0x080E 0000-0x080F FFFF	128 K	Sector 7		0x7
	Bank 2	0x0810 0000-0x0811 FFFF	128 K	Sector 0		0x0
		0x0812 0000-0x0813 FFFF	128 K	Sector 1		0x1
		...	...	...		...
		0x081E 0000-0x081F FFFF	128 K	Sector 7		0x7
System memory	Bank 1	0x1FF0 0000-0x1FF1 FFFF	128 K	System flash (read-only)	AXI ports	N/A ⁽²⁾
	Bank 2	0x1FF4 0000-0x1FF5 FFFF	128 K	System flash		N/A ⁽²⁾
Option bytes	Bank 1	N/A	-	User option bytes	Registers only	N/A

1. SNB1/2 contains the target sector number for an erase operation. See [Section 4.3.10](#) for details.

2. Cannot be erased by application software.

Table 15. Flash memory organization on STM32H745xG/STM32H747xG devices

Flash memory area		Address range	Size (bytes)	Region name	Access interface	SNB1/2 ⁽¹⁾
User main memory	Bank 1	0x0800 0000-0x0801 FFFF	128 K	Sector 0	AXI ports	0x0
		0x0802 0000-0x0803 FFFF	128 K	Sector 1		0x1
		...	...	...		...
		0x0806 0000-0x0807 FFFF	128 K	Sector 3		0x3
	Bank 2	0x0810 0000-0x0811 FFFF	128 K	Sector 0		0x0
		0x0812 0000-0x0813 FFFF	128 K	Sector 1		0x1
		...	...	...		...
		0x0816 0000-0x0817 FFFF	128 K	Sector 3		0x3
System memory	Bank 1	0x1FF0 0000-0x1FF1 FFFF	128 K	System flash (read-only)	AXI ports	N/A ⁽²⁾
	Bank 2	0x1FF4 0000-0x1FF5 FFFF	128 K	System flash		N/A ⁽²⁾
Option bytes	Bank 1	N/A	-	User option bytes	Registers only	N/A

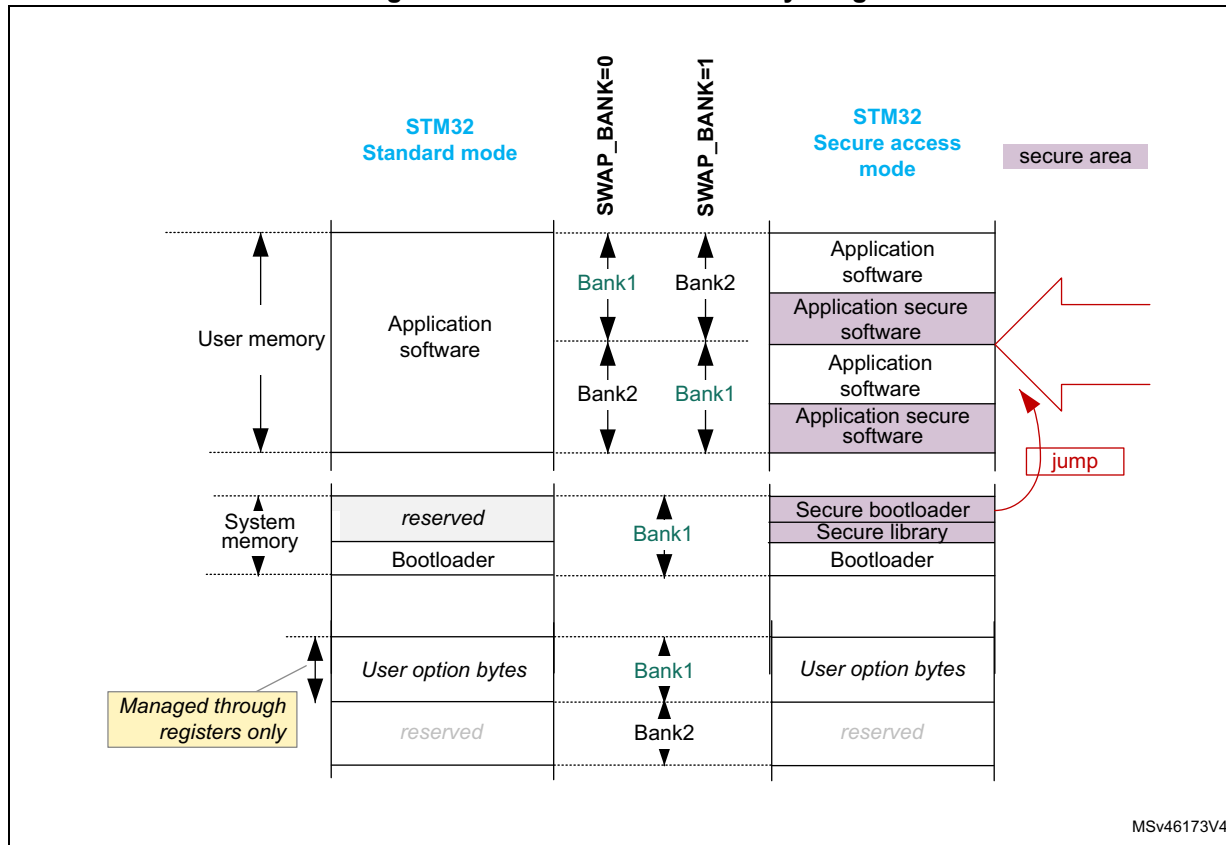
1. SNB1/2 contains the target sector number for an erase operation. See [Section 4.3.10](#) for details.

2. Cannot be erased by application software.

Partition usage

Figure 8 shows how the embedded flash memory is used both by STMicroelectronics and the application software.

Figure 8. Embedded flash memory usage



User and system memories are used differently according to whether the microcontroller is configured by the application software in Standard mode or in Secure access mode. This selection is done through the SECURITY option bit (see [Section 4.4.6](#)):

- In Standard mode, the user memory contains the application code and data, while the system memory is loaded with the STM32 bootloader. When a reset occurs, the executing core jumps to the boot address configured through the BOOT pin and the BOOT_CMx_ADD0/1 option bytes.
- In Secure access mode, dedicated libraries can be used for secure boot. They are located in user flash and system flash memory:
 - ST libraries in system flash memory assist the application software boot with special features such as secure boot and secure firmware install (SFI).
 - Application secure libraries in user flash memory are used for secure firmware update (SFU).

In Secure access mode, the microcontroller always boots into the secure bootloader code (unique entry point). Then, if no secure services are required, this code securely jumps to the requested boot address configured through the BOOT pin and the option bytes, as shown in [Figure 8](#) (see [Section 5: Secure memory management \(SMM\)](#) for details).

Note: For more information on option byte setup for boot, refer to [Section 4.4.7](#).

Additional partition usage is the following:

- The option bytes are used by STMicroelectronics and by the application software as non-volatile product options (e.g. boot address, protection configuration and reset behaviors).

Note: For further information on STM32 bootloader flashing by STMicroelectronics, refer to application note AN2606 “STM32 microcontroller system memory boot mode” available from www.st.com.

Bank swapping

As shown in [Figure 8](#), the embedded flash memory offers a bank swapping feature that can be configured through the SWAP_BANK bit, always available for the application. For more information please refer to [Section 4.3.13](#).

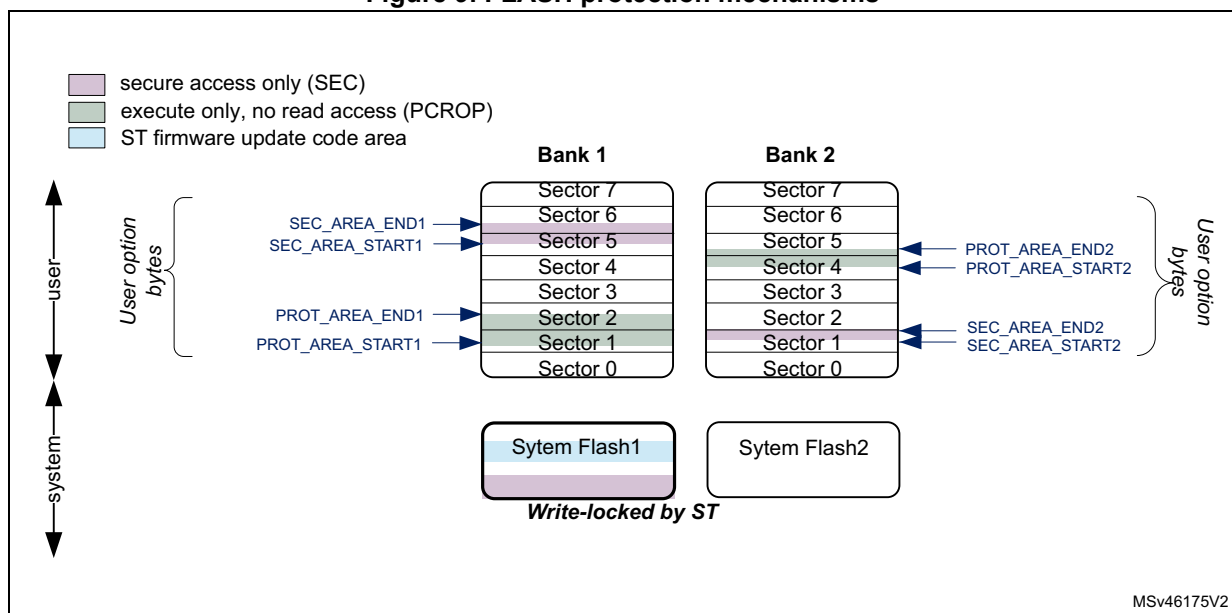
4.3.5 FLASH system performance enhancements

The embedded flash memory uses read and write command queues (one per bank) in order to enhance flash operations.

4.3.6 FLASH data protection schemes

[Figure 9](#) gives an overview of the protection mechanisms supported by the embedded flash memory. A PCROP and a secure-only area can be defined for each bank. The properties of these protected areas are detailed in [Section 4.5](#).

Figure 9. FLASH protection mechanisms



4.3.7 Overview of FLASH operations

Read operations

The embedded flash memory can perform read operations on the whole non-volatile memory using various granularities: 64 bits, 32 bits, 16 bits or one byte. User and system flash memories are read through the AXI interface, while the option bytes are read through the register interface.

The embedded flash memory supports read-while-write operations provided the read and write operations target different banks. Similarly read-while-read operations are supported when two read operations target different banks.

To increase efficiency, the embedded flash memory implements the buffering of consecutive read requests in the same bank.

For more details on read operations, refer to [Section 4.3.8: FLASH read operations](#).

Program/erase operations

The embedded flash memory supports the following program and erase operations:

- Single flash word write (256-bit granularity), with the possibility for the application to force-write a user flash word with less than 256 bits
- Single sector erase
- Bank erase (single or dual)
- Option byte update

Thanks to its dual bank architecture, the embedded flash memory can perform any of the above write or erase operation on one bank while a read or another program/erase operation is executed on the other bank.

Note: *Program and erase operations are subject to the various protection that could be set on the embedded flash memory, such as write protection and global readout protection (see next sections for details).*

To increase efficiency, the embedded flash memory implements the buffering of consecutive write accesses in the same bank.

For more details refer to [Section 4.3.9: FLASH program operations](#) and [Section 4.3.10: FLASH erase operations](#).

Protection mechanisms

The embedded flash memory supports different protection mechanisms:

- Global readout protection (RDP)
- Proprietary code readout protection (PCROP)
- Write protection
- Secure access only protection

For more details refer to [Section 4.5: FLASH protection mechanisms](#).

Option byte loading

Under specific conditions, the embedded flash memory reliably loads the non-volatile option bytes stored in non-volatile memory, thus enforcing boot and security options to the whole system when the embedded flash memory becomes functional again. For more details refer to [Section 4.4: FLASH option bytes](#).

Bank/register swapping

The embedded flash memory allows swapping bank 1 and bank 2 memory mapping. This feature can be used after a firmware upgrade to restart the microcontroller on the new firmware after a system reset. For more details on the feature, refer to [Section 4.3.13: Flash bank and register swapping](#).

4.3.8 FLASH read operations

Read operation overview

The embedded flash memory supports, for each memory bank, the execution of one read command while two are waiting in the read command queue. Multiple read access types are also supported as defined in [Section 4.3.3: FLASH architecture and integration in the system](#).

The read commands to each bank are associated with a 256-bit read data buffer.

Note: The embedded flash memory can perform single error correction and double error detection while read operations are being executed (see [Section 4.3.12: Flash memory error protections](#)).

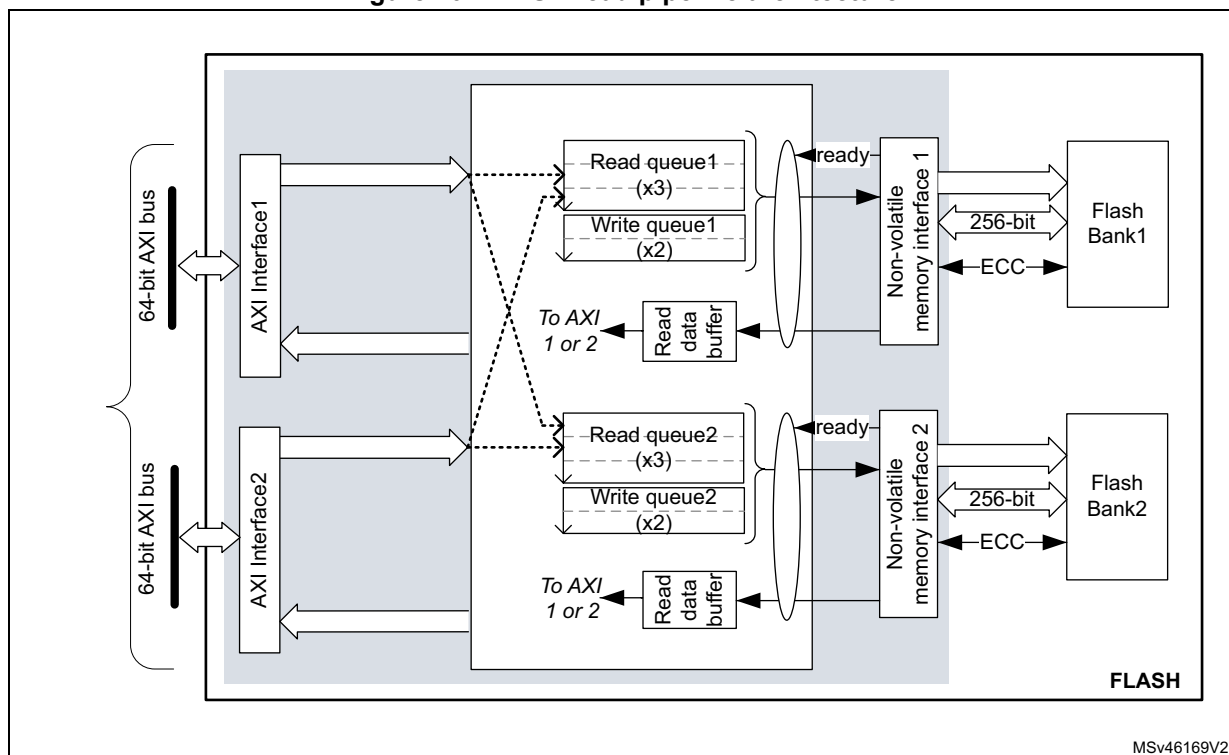
The AXI interface read channel operates as follows:

- When the read command queue is full, any new AXI read request stalls the bus read channel interface and consequently the master that issued that request.
- If several consecutive read accesses request data that belong to the same flash data word (256 bits), the data are read directly from the current data read buffer, without triggering additional flash read operations. This mechanism occurs each time a read access is granted. When a read access is rejected for security reasons (e.g. PCROP protected word), the corresponding read error response is issued by the embedded flash memory and no read operation to flash memory is triggered.

The Read pipeline architecture is summarized in [Figure 10](#).

For more information on bus interfaces, refer to [Section 4.3.3: FLASH architecture and integration in the system](#).

Figure 10. FLASH read pipeline architecture



MSv46169V2

Single read sequence

The recommended simple read sequence is the following:

1. Freely perform read accesses to any AXI-mapped area.
2. The embedded flash memory effectively executes the read operation from the read command queue buffer as soon as the non-volatile memory is ready and the previously requested operations on this specific bank have been served.

Adjusting read timing constraints

The embedded flash memory clock must be enabled and running before reading data from non-volatile memory.

To correctly read data from flash memory, the number of wait states (LATENCY) must be correctly programmed in the flash access control register (FLASH_ACR) according to the embedded flash memory AXI interface clock frequency (sys_ck) and the internal voltage range of the device (V_{core}).

Table 16 shows the correspondence between the number of wait states (LATENCY), the programming delay parameter (WRHIGHFREQ), the embedded flash memory clock frequency and its supply voltage ranges.

Table 16. FLASH recommended number of wait states and programming delay⁽¹⁾

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	AXI Interface clock frequency vs V _{CORE} range			
		VOS3 range 0.95 V - 1.05 V	VOS2 range 1.05 V - 1.15 V	VOS1 range 1.15 V - 1.26 V	VOS0 range 1.26 V - 1.40 V
0 WS (1 FLASH clock cycle)	00	[0;45 MHz]	[0;55 MHz]	[0;70 MHz]	[0;70 MHz]
1 WS (2 FLASH clock cycles)	01	]45 MHz;90 MHz]	]55 MHz;110 MHz]	]70 MHz;140 MHz]	]70 MHz;140 MHz]
2 WS (3 FLASH clock cycles)	01	]90 MHz;135 MHz]	]110 MHz;165 MHz]	]140 MHz;185 MHz]	]140 MHz;185 MHz]
	10	-	-	]185 MHz;210 MHz]	]185 MHz;210 MHz]
3 WS (4 FLASH clock cycles)	10	]135 MHz;180 MHz]	]165 MHz;225 MHz]	]210 MHz;225 MHz]	]210 MHz;225 MHz]
4 WS (5 FLASH clock cycles)	10	]180 MHz;225 MHz]	225 MHz	-	]225 MHz;240 MHz]

1. Refer to the Reset and clock control section RCC section for the maximum product F_{ACLK} frequency.

Adjusting system frequency

After power-on, a default 7 wait-state latency is specified in FLASH_ACR register, in order to accommodate AXI interface clock frequencies with a safety margin (see [Table 16](#)).

When changing the AXI bus frequency, the application software must follow the below sequence in order to tune the number of wait states required to access the non-volatile memory.

To increase the embedded flash memory clock source frequency:

1. If necessary, program the LATENCY and WRHIGHFREQ bits to the right value in the FLASH_ACR register, as described in [Table 16](#).
2. Check that the new number of wait states is taken into account by reading back the FLASH_ACR register.
3. Modify the embedded flash memory clock source and/or the AXI bus clock prescaler in the RCC_CFGR register of the reset and clock controller (RCC).
4. Check that the new embedded flash memory clock source and/or the new AXI bus clock prescaler value are taken in account by reading back the embedded flash memory clock source status and/or the AXI bus prescaler value in the RCC_CFGR register of the reset and clock controller (RCC).

To decrease the embedded flash memory clock source frequency:

1. Modify the embedded flash memory clock source and/or the AXI bus clock prescaler in the RCC_CFGR register of reset and clock controller (RCC).
2. Check that the embedded flash memory new clock source and/or the new AXI bus clock prescaler value are taken into account by reading back the embedded flash

memory clock source status and/or the AXI interface prescaler value in the RCC_CFGR register of reset and clock controller (RCC).

3. If necessary, program the LATENCY and WRHIGHFREQ bits to the right value in FLASH_ACR register, as described in [Table 16](#).
4. Check that the new number of wait states has been taken into account by reading back the FLASH_ACR register.

Error code correction (ECC)

The embedded flash memory embeds an error correction mechanism. Single error correction and double error detection are performed for each read operation. For more details, refer to [Section 4.3.12: Flash memory error protections](#).

Read errors

When the ECC mechanism is not able to correct the read operation, the embedded flash memory reports read errors as described in [Section 4.7.7: Error correction code error \(SNECCERR/DBECCERR\)](#).

Read interrupts

See [Section 4.8: FLASH interrupts](#) for details.

4.3.9 FLASH program operations

Program operation overview

The virgin state of each non-volatile memory bitcell is 1. The embedded flash memory supports programming operations that can change (reset) any memory bitcell to 0. However these operations do not support the return of a bit to its virgin state. In this case an erase operation of the entire sector is required.

A program operation consists in issuing write commands. The embedded flash memory supports, for each memory bank, the execution of one write command while one command is waiting in the write command queue. Since a 10-bit ECC code is associated to each 256-bit data flash word, only write operations by 256 bits are executed in the non-volatile memory.

Note: The application can decide to write as little as 8 bits to a 256 flash word. In this case, a force-write mechanism to the 256 bits + ECC is used (see FW1/2 bit of FLASH_CR1/2 register).

System flash memory bank 1 cannot be written by the application software. System flash memory bank 2 can be written by STMicroelectronics secure firmware only.

It is not recommended to overwrite a flash word that is not virgin. The result may lead to an inconsistent ECC code that will be systematically reported by the embedded flash memory, as described in [Section 4.7.7: Error correction code error \(SNECCERR/DBECCERR\)](#).

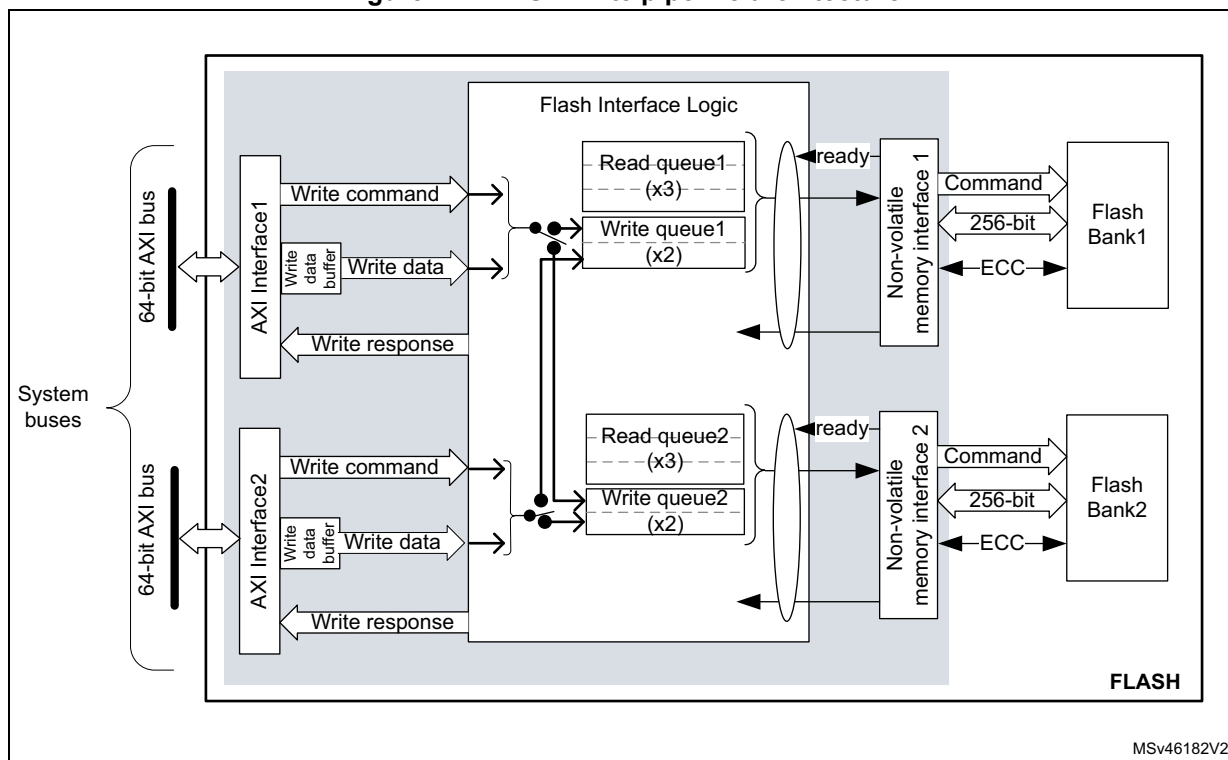
The AXI interface write channel operates as follows:

- A 256-bit write data buffer is associated with each AXI interface. It supports multiple write access types (64 bits, 32 bits, 16 bits and 8 bits).
- When the write queue is full, any new AXI write request stalls the bus write channel interface and consequently the master that issued that request.

The write pipeline architecture is described in [Figure 11](#).

For more information on bus interfaces, refer to [Section 4.3.3: FLASH architecture and integration in the system](#).

Figure 11. FLASH write pipeline architecture



Managing write protections

Before programming a user sector, the application software must check the protection of the targeted flash memory area.

The embedded flash memory checks the protection properties of the write transaction target at the output of the write queue buffer, just before the effective write operation to the non-volatile memory:

- If a write protection violation is detected, the write operation is canceled and write protection error (WRPERR1/2) is raised in FLASH_SR1/2 register.
- If the write operation is valid, the 10-bit ECC code is concatenated to the 256 bits of data and the write to non-volatile memory is effectively executed.

Note: *No write protection check is performed when the embedded flash memory accepts AXI write requests.*

The write protection flag does not need to be cleared before performing a new programming operation.

Monitoring ongoing write operations

The application software can use three status flags located in FLASH_SR1/2 in order to monitor ongoing write operations. Those status are available for each bank.

- **BSY1/2**: this bit indicates that an effective write, erase or option byte change operation is ongoing to the non-volatile memory.
- **QW1/2**: this bit indicates that a write, erase or option byte change operation is pending in the write queue or command queue buffer. It remains high until the write operation is complete. It supersedes the BSY1/2 status bit.
- **WBNE1/2**: this bit indicates that the embedded flash memory is waiting for new data to complete the 256-bit write buffer. In this state the write buffer is not empty. It is reset as soon as the application software fills the write buffer, force-writes the operation using FW1/2 bit in FLASH_CR1/2, or disables all write operations in the corresponding bank.

Enabling write operations

Before programming the user flash memory in bank 1 (respectively bank 2), the application software must make sure that PG1 bit (respectively PG2) is set to 1 in FLASH_CR1 (respectively FLASH_CR2). If it is not the case, an unlock sequence must be used (see [Section 4.5.1: FLASH configuration protection](#)) and the PG1/2 bit must be set.

When the option bytes need to be modified or a mass erase needs to be started, the application software must make sure that FLASH_OPTCR is unlocked. If it is not the case, an unlock sequence must be used (see [Section 4.5.1: FLASH configuration protection](#)).

Note: The application software must not unlock a register that is already unlocked, otherwise this register will remain locked until next system reset.

If needed, the application software can update the programming delay and programming parallelism as described at the end of this section.

Single write sequence

The recommended single write sequence in bank 1/2 is the following:

1. Unlock the FLASH_CR1/2 register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if register is not already unlocked).
2. Enable write operations by setting the PG1/2 bit in the FLASH_CR1/2 register.
3. Check the protection of the targeted memory area.
4. Write one flash-word corresponding to 32-byte data starting at a 32-byte aligned address.
5. Check that QW1 (respectively QW2) has been raised and wait until it is reset to 0.

If step 4 is executed incrementally (e.g. byte per byte), the write buffer can become partially filled. In this case the application software can decide to force-write what is stored in the write buffer by using FW1/2 bit in FLASH_CR1/2 register. In this particular case, the unwritten bits are automatically set to 1. If no bit in the write buffer is cleared to 0, the FW1/2 bit has no effect.

Note: Using a force-write operation prevents the application from updating later the missing bits with a value different from 1, which is likely to lead to a permanent ECC error.

Any write access requested while the PG1/2 bit is cleared to 0 is rejected. In this case, no error is generated on the bus, but the PGSEERR1/2 flag is raised.

Clearing the programming sequence error (PGSEERR) and inconsistency error (INCERR) is mandatory before attempting a write operation (see [Section 4.7: FLASH error management](#) for details).

Adjusting programming timing constraints

Program operation timing constraints depend of the embedded flash memory clock frequency, which directly impacts the performance. If timing constraints are too tight, the non-volatile memory will not operate correctly, if they are too lax, the programming speed will not be optimal.

The user must therefore trim the optimal programming delay through the WRHIGHFREQ parameter in the FLASH_ACR register. Refer to [Table 16](#) in [Section 4.3.8: FLASH read operations](#) for the recommended programming delay depending on the embedded flash memory clock frequency.

FLASH_ACR configuration register is common to both banks.

The application software must check that no program/erase operation is ongoing before modifying WRHIGHFREQ parameter.

Caution: Modifying WRHIGHFREQ while programming/erasing the flash memory might corrupt the flash memory content.

Adjusting programming parallelism

The parallelism is the maximum number of bits that can be written to 0 in one shot during a write operation. The programming parallelism is also used during sector and bank erase.

There is no hardware limitation on programming parallelism. The user can select different parallelisms depending on the application requirements: the lower the parallelism, the lower the peak consumption during a programming operation, but the longer the execution time.

The parallelism is configured through the PSIZE1/2 bits in FLASH_CR1/2 register. Two distinct values can be defined for bank 1 and 2 (refer to [Table 17](#)).

Table 17. FLASH parallelism parameter

PSIZE1/2	Parallelism
00	8 bits (one byte)
01	16 bits
10	32 bits
11	64 bits

Caution: Modifying PSIZE1/2 while programming/erasing the flash memory might corrupt the flash memory content.

Programming errors

When a program operation fails, an error can be reported as described in [Section 4.7: FLASH error management](#).

Programming interrupts

See [Section 4.8: FLASH interrupts](#) for details.

4.3.10 FLASH erase operations

Erase operation overview

The embedded flash memory can perform erase operations on 128-Kbyte user sectors, on one user flash memory bank or on two user flash memory banks (i.e. mass erase).

Note: *System flash memory cannot be erased by the application software.*

The erase operation forces all non-volatile bit cells to high state, which corresponds to the virgin state. It clears existing data and corresponding ECC, allowing a new write operation to be performed. If the application software reads back a word that has been erased, all the bits will be read at 1, without ECC error.

Erase operations are similar to read or program operations except that the commands are queued in a special buffer (a two-command deep erase queue).

Erase commands are issued through the AHB configuration interface. If the embedded flash memory receives simultaneously a write and an erase request for the same bank, both operations are accepted but the write operation is executed first.

Note: *If data cache is enabled after a flash erase operation, it is recommended to invalidate the cache by software to avoid reading old data.*

Erase and security

A user sector can be erased only if it does not contain PCROP, secure-only or write-protected data (see [Section 4.5: FLASH protection mechanisms](#) for details). In other words, if the application software attempts to erase a user sector with at least one flash word that is protected, the sector erase operation is aborted and the WRPERR1/2 flag is raised in the FLASH_SR1/2 register, as described in [Section 4.7.2: Write protection error \(WRPERR\)](#).

The embedded flash memory allows the application software to perform an erase followed by an automatic protection removal (PCROP, secure-only area and write protection), as described hereafter.

Enabling erase operations

Before erasing a sector in bank 1 (respectively bank 2), the application software must make sure that FLASH_CR1 (respectively FLASH_CR2) is unlocked. If it is not the case, an unlock sequence must be used (see [Section 4.5.1: FLASH configuration protection](#)).

Note: *The application software must not unlock a register that is already unlocked, otherwise this register will remain locked until next system reset.*

Similar constraints apply to bank erase requests.

Flash sector erase sequence

To erase a 128-Kbyte user sector, proceed as follows:

1. Check and clear (optional) all the error flags due to previous programming/erase operation. Refer to [Section 4.7: FLASH error management](#) for details.
2. Unlock the FLASH_CR1/2 register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if register is not already unlocked).
3. Set the SER1/2 bit and SNB1/2 bitfield in the corresponding FLASH_CR1/2 register. SER1/2 indicates a sector erase operation, while SNB1/2 contains the target sector number.
4. Set the START1/2 bit in the FLASH_CR1/2 register.
5. Wait for the QW1/2 bit to be cleared in the corresponding FLASH_SR1/2 register.

Note: *If a bank erase is requested simultaneously to the sector erase (BER1/2 bit set), the bank erase operation supersedes the sector erase operation.*

Standard flash bank erase sequence

To erase all bank sectors except for those containing secure-only and protected data, proceed as follows:

1. Check and clear (optional) all the error flags due to previous programming/erase operation. Refer to [Section 4.7: FLASH error management](#) for details.
2. Unlock the FLASH_CR1/2 register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if register is not already unlocked).
3. Set the BER1/2 bit in the FLASH_CR1/2 register corresponding to the targeted bank.
4. Set the START1/2 bit in the FLASH_CR1/2 register to start the bank erase operation. Then wait until the QW1/2 bit is cleared in the corresponding FLASH_SR1/2 register.

Note: *BER1/2 and START1/2 bits can be set together, so above steps 3 and 4 can be merged.*
If a sector erase is requested simultaneously to the bank erase (SER1/2 bit set), the bank erase operation supersedes the sector erase operation.

Flash bank erase with automatic protection-removal sequence

To erase all bank sectors including those containing secure-only and protected data without performing an RDP regression (see [Section 4.5.3](#)), proceed as follows:

1. Check and clear (optional) all the error flags due to previous programming/erase operation. Refer to [Section 4.7: FLASH error management](#) for details.
2. Unlock FLASH_OPTCR register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if register is not already unlocked).
3. If a PCROP-protected area exists set DMEP1/2 bit in FLASH_PRAR_PRG1/2 register. In addition, program the PCROP area end and start addresses so that the difference is negative, i.e. $PROT_AREA_END1/2 < PROT_AREA_START1/2$.
4. If a secure-only area exists set DMES1/2 bit in FLASH_SCAR_PRG1/2 register. In addition, program the secure-only area end and start addresses so that the difference is negative, i.e. $SEC_AREA_END1/2 < SEC_AREA_START1/2$.
5. Set all WRPSn1/2 bits in FLASH_WPSN_PRG1/2R to 1 to disable all sector write protection.
6. Unlock FLASH_CR1/2 register, only if register is not already unlocked.
7. Set the BER1/2 bit in the FLASH_CR1/2 register corresponding to the target bank.
8. Set the START1/2 bit in the FLASH_CR1/2 register to start the bank erase with protection removal operation. Then wait until the QW1/2 bit is cleared in the corresponding FLASH_SR1/2 register. At that point a bank erase operation has erased the whole bank including the sectors containing PCROP-protected and/or secure-only data, and an option byte change has been automatically performed so that all the protections are disabled.

Note: **BER1/2 and START1/2 bits can be set together, so above steps 8 and 9 can be merged.**

Be aware of the following warnings regarding to above sequence:

- It is not possible to perform the above sequence on one bank while modifying the protection parameters of the other bank.
- No other option bytes than the one indicated above must be changed, and no protection change must be performed in the bank that is not targeted by the bank erase with protection removal request.
- When one or both of the events above occurs, a simple bank erase occurs, no option byte change is performed and no option change error is set.

Flash mass erase sequence

To erase all sectors of both banks simultaneously, excepted for those containing secure-only and protected data, the application software can set the MER bit to 1 in FLASH_OPTCR register, as described below:

1. Check and clear (optional) all the error flags due to previous programming/erase operation. Refer to [Section 4.7: FLASH error management](#) for details.
2. Unlock the two FLASH_CR1/2 registers and FLASH_OPTCR register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if the registers are not already unlocked).
3. Set the MER bit to 1 in FLASH_OPTCR register. It automatically sets BER1, BER2, START1 and START2 to 1, thus launching a bank erase operation on both banks. Then wait until both QW1 and QW2 bits are cleared in the corresponding FLASH_SR1/2 register.

Flash mass erase with automatic protection-removal sequence

To erase all sectors of both banks simultaneously, including those containing secure-only and protected data, and without performing an RDP regression, proceed as follows:

1. Check and clear (optional) all the error flags due to previous programming/erase operation.
2. Unlock the two FLASH_CR1/2 registers and FLASH_OPTCR register (only if the registers are not already unlocked).
3. If a PCROP-protected area exists, set DMEP1/2 bit in FLASH_PRAR_PRG1/2 register. In addition, program the PCROP area end and start addresses so that the difference is negative. This operation must be performed for both banks.
4. If a secure-only area exists, set DMES1/2 bit in FLASH_SCAR_PRG1/2 register. In addition, program the secure-only area end and start addresses so that the difference is negative. This operation must be performed for both banks.
5. Set all WRPSn1/2 bits in FLASH_WPSN_PRG1/2R to 1 to disable all sector write protections. This operation must be performed for both banks.
6. Set the MER bit to 1 in FLASH_OPTCR register, then wait until the QW1/2 bit is cleared in the corresponding FLASH_SR1/2 register. At that point, a flash bank erase with automatic protection removal is executed on both banks. The sectors containing PCROP-protected and/or secure-only data become unprotected since an option byte change is automatically performed after the mass erase so that all the protections are disabled.

Caution: No other option bytes than the ones mentioned in the above sequence must be changed, otherwise a simple mass erase is executed, no option byte change is performed and no option change error is raised.

4.3.11 FLASH parallel operations

As the non-volatile memory is divided into two independent banks, the embedded flash memory interface can drive different operations at the same time on each bank. For example a read, write or erase operation can be executed on bank 1 while another read, write or erase operation is executed on bank 2.

In all cases, the sequences described in [Section 4.3.8: FLASH read operations](#), [Section 4.3.9: FLASH program operations](#) and [Section 4.3.10: FLASH erase operations](#) apply.

4.3.12 Flash memory error protections

Error correction codes (ECC)

The embedded flash memory supports an error correction code (ECC) mechanism. It is based on the SECDED algorithm in order to correct single errors and detects double errors.

This mechanism uses 10 ECC bits per 256-bit flash word, and applies to user and system flash memory.

More specifically, during each read operation from a 256-bit flash word, the embedded flash memory also retrieves the 10-bit ECC information, computes the ECC of the flash word, and compares the result with the reference value. If they do not match, the corresponding ECC error is raised as described in [Section 4.7.7: Error correction code error \(SNECCERR/DBECCERR\)](#).

During each program operation, a 10-bit ECC code is associated to each 256-bit data flash word, and the resulting 266-bit flash word information is written in non-volatile memory.

Cyclic redundancy codes (CRC)

The embedded flash memory implements a cyclic redundancy check (CRC) hardware module. This module checks the integrity of a given user flash memory area content (see [Figure 6: Detailed FLASH architecture](#)).

The area processed by the CRC module can be defined either by sectors or by start/end addresses. It can also be defined as the whole bank (user flash memory area only).

Note: *Only one CRC check operation on bank 1 or 2 can be launched at a time. To avoid corruption, do not configure the CRC calculation on the one bank, while calculating the CRC on the other bank.*

When enabled, the CRC hardware module performs multiple reads by chunks of 4, 16, 64 or 256 consecutive flash-words (i.e. chunks of 128, 512, 2048 or 8192 bytes). These consecutive read operations are pushed by the CRC module into the required read command queue together with other AXI read requests, thus avoiding to deny AXI read commands.

CRC computation uses CRC-32 (Ethernet) polynomial 0x4C11DB7:

$$X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$$

The CRC operation is concurrent with option byte change as the same hardware is used for both operations. To avoid the CRC computation from being corrupted, the application shall complete the option byte change (by reading the result of the change) before running a CRC operation, and vice-versa.

The sequence recommended to configure a CRC operation in the bank 1/2 is the following:

1. Unlock FLASH_CR1/2 register, if not already unlocked.
2. Enable the CRC feature by setting the CRC_EN bit in FLASH_CR1/2.
3. Program the desired data size in the CRC_BURST field of FLASH_CRCCR1/2.
4. Define the user flash memory area on which the CRC has to be computed. Two solutions are possible:
 - Define the area start and end addresses by programming FLASH_CRCSADD1/2R and FLASH_CRCEADD1/2R, respectively,
 - or select the targeted sectors by setting the CRC_BY_SECT bit in FLASH_CRCCR1/2 and by programming consecutively the target sector numbers in the CRC_SECT field of the FLASH_CRCCR1/2 register. Set ADD_SECT bit after each CRC_SECT programming.
5. Start the CRC operation by setting the START_CRC bit.
6. Wait until the CRC_BUSY1/2 flag is reset in FLASH_SR1/2 register.
7. Retrieve the CRC result in the FLASH_CRCDATAR register.

The CRC can be computed for a whole bank by setting the ALL_BANK bit in the FLASH_CRCCR1/2 register.

Note: *The application should avoid running a CRC on PCROP- or secure-only user flash memory area since it may alter the expected CRC value. A special error flag defined in [Section 4.7.10: CRC read error \(CRCRDERR\)](#) can be used to detect such a case.*

CRC computation does not raise standard read error flags such as RDSERR1/2, RDPERR1/2 and DBECCERR1/2. Only CRCRDERR1/2 is raised.

4.3.13 Flash bank and register swapping

Flash bank swapping

The embedded flash memory bank 1 and bank 2 can be swapped in the memory map accessible through AXI interface. This feature can be used after a firmware upgrade to restart the device on the new firmware. Bank swapping is controlled by the SWAP_BANK bit of the FLASH_OPTCR register.

[Table 18](#) shows the memory map that can be accessed from the embedded flash memory AXI slave interface, depending on the SWAP_BANK bit configuration.

Table 18. FLASH AXI interface memory map vs swapping option

Flash memory area	Flash memory corresponding bank		Start address	End address	Size (bytes)	Region Name
	SWAP_BANK=0	SWAP_BANK=1				
User main memory	Bank 1	Bank 2	0x0800 0000	0x0801 FFFF	128 K	Sector 0
			0x0802 0000	0x0803 FFFF	128 K	Sector 1
			...	...	...	...
			0x080E 0000	0x080F FFFF	128 K	Sector 7
	Bank 2	Bank 1	0x0810 0000	0x0811 FFFF	128 K	Sector 0
			0x0812 0000	0x0813 FFFF	128 K	Sector 1
			...	...	...	...
			0x081E 0000	0x081F FFFF	128 K	Sector 7
System memory	Bank 1		0x1FF0 0000	0x1FF1 FFFF	128 K	System flash memory (bank 1)
	Bank 2		0x1FF4 0000	0x1FF5 FFFF	128 K	System flash memory (bank 2)

The SWAP_BANK bit in FLASH_OPTCR register is loaded from the SWAP_BANK_OPT option bit **only after system reset or POR**.

To change the SWAP_BANK bit (for example to apply a new firmware update), respect the sequence below:

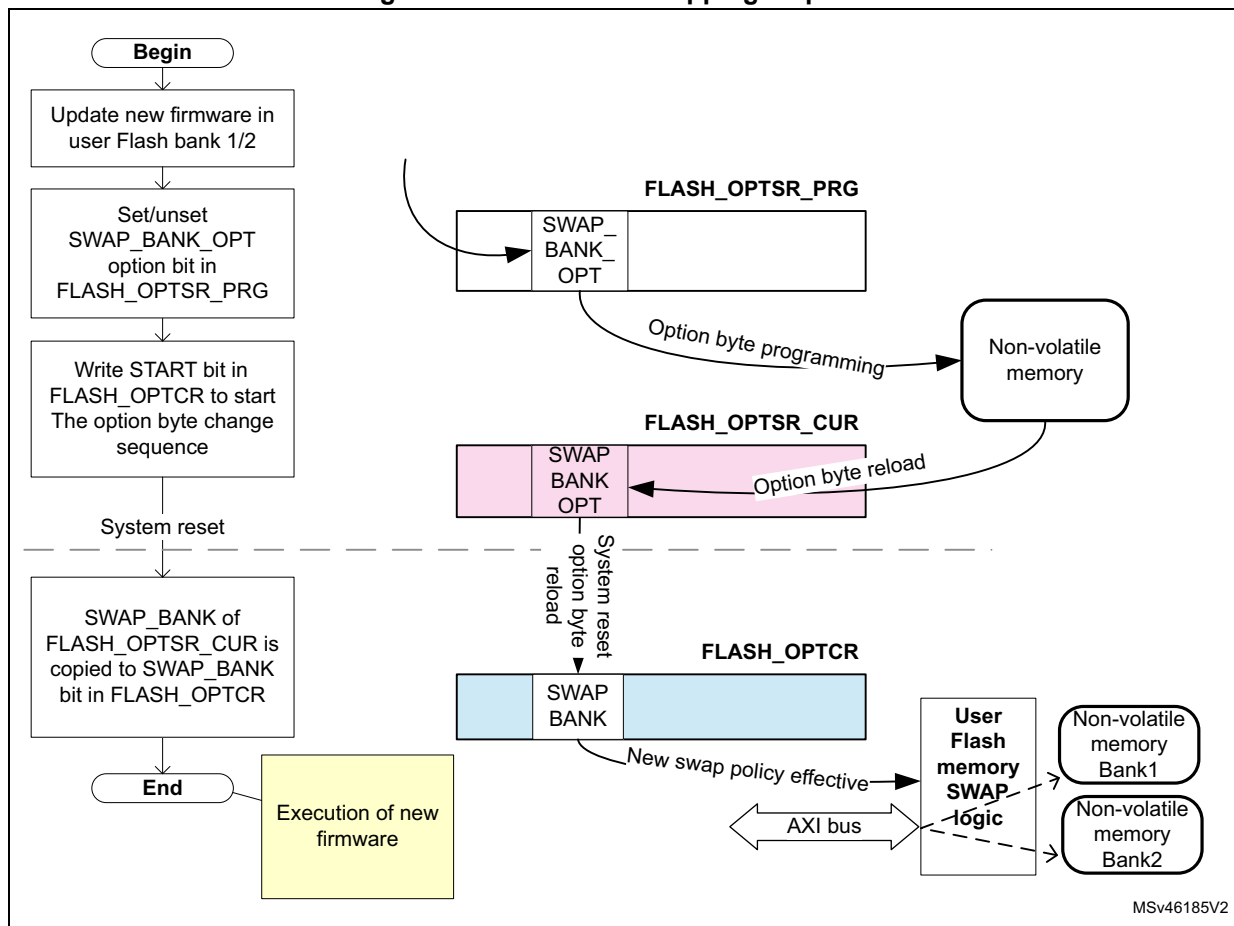
1. Unlock OPTLOCK bit, if not already unlocked.
2. Set the new desired SWAP_BANK_OPT value in the FLASH_OPTSR_PRG register.
3. Start the option byte change sequence by setting the OPTSTART bit in the FLASH_OPTCR register.
4. Once the option byte change has completed, FLASH_OPTSR_CUR contains the expected SWAP_BANK_OPT value, but SWAP_BANK bit in FLASH_OPTCR has not yet been modified and the bank swapping is not yet effective.
5. Force a system reset or a POR. When the reset rises up, the bank swapping is effective (SWAP_BANK value updated in FLASH_OPTCR) and the new firmware shall be executed.

Note: The `SWAP_BANK` bit in `FLASH_OPTCR` is read-only and cannot be modified by the application software.

The `SWAP_BANK_OPT` option bit in `FLASH_OPTSR_PRG` can be modified whatever the RDP level (i.e. even in level 2), thus allowing advanced firmware upgrade in any level of readout protection.

Figure 12 gives an overview of the bank swapping sequence.

Figure 12. Flash bank swapping sequence



Configuration and option byte register swapping

The embedded flash memory bank swapping option controlled by the SWAP_BANK bit also swaps the two sets of configuration and option byte registers, as shown in [Table 19](#). One set of registers is related to bank 1 while the other is related to bank 2. Since some registers are not specific to any particular bank, they are mapped onto two different addresses so that the swapping does not affect the access to these registers.

Table 19. Flash register map vs swapping option

Address offset ⁽¹⁾		Register name		Register targeting bank ⁽¹⁾			
				SWAP_BANK=0		SWAP_BANK=1	
0x000 or 0x100		FLASH_ACR		N/A		N/A	
0x004	0x104	FLASH_KEYR1	FLASH_KEYR2	Bank 1	Bank 2	Bank 2	Bank 1
0x008 or 0x108		FLASH_OPTKEYR		N/A		N/A	
0x00C	0x10C	FLASH_CR1	FLASH_CR2	Bank 1	Bank 2	Bank 2	Bank 1
0x010	0x110	FLASH_SR1	FLASH_SR2	Bank 1	Bank 2	Bank 2	Bank 1
0x014	0x114	FLASH_CCR1	FLASH_CCR2	Bank 1	Bank 2	Bank 2	Bank 1
0x018 or 0x118		FLASH_OPTCR		N/A		N/A	
0x01C or 0x11C		FLASH_OPTSR_CUR		N/A		N/A	
0x020 or 0x120		FLASH_OPTSR_PRG		N/A		N/A	
0x024 or 0x124		FLASH_OPTCCR		N/A		N/A	
0x028	0x128	FLASH_PRAR_CUR1	FLASH_PRAR_CUR2	Bank 1	Bank 2	Bank 2	Bank 1
0x02C	0x12C	FLASH_PRAR_PRG1	FLASH_PRAR_PRG2	Bank 1	Bank 2	Bank 2	Bank 1
0x030	0x130	FLASH_SCAR_CUR1	FLASH_SCAR_CUR2	Bank 1	Bank 2	Bank 2	Bank 1
0x034	0x134	FLASH_SCAR_PRG1	FLASH_SCAR_PRG2	Bank 1	Bank 2	Bank 2	Bank 1
0x038	0x138	FLASH_WPSGN_CUR1	FLASH_WPSGN_CUR2	Bank 1	Bank 2	Bank 2	Bank 1
0x03C	0x13C	FLASH_WPSGN_PRG1	FLASH_WPSGN_PRG2	Bank 1	Bank 2	Bank 2	Bank 1
0x040 or 0x140		FLASH_BOOT7_CUR		N/A		N/A	
0x044 or 0x144		FLASH_BOOT7_PRG		N/A		N/A	
0x048 or 0x148		FLASH_BOOT4_CUR		N/A		N/A	
0x04C or 0x14C		FLASH_BOOT4_PRG		N/A		N/A	
0x050	0x150	FLASH_CRCCR1	FLASH_CRCCR2	Bank 1	Bank 2	Bank 2	Bank 1
0x054	0x154	FLASH_CRCSADD1R	FLASH_CRCSADD2R	Bank 1	Bank 2	Bank 2	Bank 1
0x058	0x158	FLASH_CRCEADD1R	FLASH_CRCEADD2R	Bank 1	Bank 2	Bank 2	Bank 1
0x05C	0x15C	FLASH_CRCDATAR		N/A		N/A	
0x060	0x160	FLASH_ECC_FA1R	FLASH_ECC_FA2R	Bank 1	Bank 2	Bank 2	Bank 1

1. As shown above, some registers are not dedicated to a specific bank and can be accessed at two different addresses.

4.3.14 FLASH reset and clocks

Reset management

The embedded flash memory can be reset by a D1 domain reset (d1_rst), driven by the reset and clock control (RCC). The main effects of this reset are the following:

- All registers, except for option byte registers, are cleared, including read and write latencies. If the bank swapping option is changed, it will be applied.
- Most control registers are automatically protected against write operations. To unprotect them, new unlock sequences must be used as described in [Section 4.5.1: FLASH configuration protection](#).

The embedded flash memory can be reset by a power-on reset (po_rst), driven by the reset and clock control (RCC). When the reset falls, all option byte registers are reset. When the reset rises up, the option bytes are loaded, potentially applying new features. During this loading sequence, the device remains under reset and the embedded flash memory is not accessible.

The Reset signal can have a critical impact on the embedded flash memory:

- The contents of the flash memory are not guaranteed if a device reset occurs during a flash memory write or erase operation.
- If a reset occurs while the option byte modification is ongoing, the old option byte values are kept. When it occurs, a new option byte modification sequence is required to program the new values.

Clock management

The embedded flash memory uses the microcontroller system clock (sys_ck), here the AXI interface clock.

Depending on the device clock and internal supply voltage, specific read and write latency settings usually need to be set in the flash access control register (FLASH_ACR), as explained in [Section 4.3.8: FLASH read operations](#) and [Section 4.3.9: FLASH program operations](#).

4.4 FLASH option bytes

4.4.1 About option bytes

The embedded flash memory includes a set of non-volatile option bytes. They are loaded at power-on reset and can be read and modified only through configuration registers.

These option bytes are configured by the end-user depending on the application requirements. Some option bytes might have been initialized by STMicroelectronics during manufacturing stage.

This section documents:

- When option bytes are loaded
- How application software can modify them
- What is the detailed list of option bytes, together with their default factory values (i.e. before the first option byte change).

4.4.2 Option byte loading

There are multiple ways of loading the option bytes into embedded flash memory:

1. Power-on wakeup

When the device is first powered, the embedded flash memory automatically loads all the option bytes. During the option byte loading sequence, the device remains under reset and the embedded flash memory cannot be accessed.

2. Wakeup from system Standby

When the D1 power domain, which contains the embedded flash memory, is switched from DStandby mode to DRun mode, the embedded flash memory behaves as during a power-on sequence.

3. Dedicated option byte reloading by the application

When the user application successfully modifies the option byte content through the embedded flash memory registers, the non-volatile option bytes are programmed and the embedded flash memory automatically reloads all option bytes to update the option registers.

Note: The option bytes read sequence is enhanced thanks to a specific error correction code. In case of security issue, the option bytes may be loaded with default values (see [Section 4.4.3: Option byte modification](#)).

4.4.3 Option byte modification

Changing user option bytes

A user option byte change operation can be used to modify the configuration and the protection settings saved in the non-volatile option byte area of memory bank 1.

The embedded flash memory features two sets of option byte registers:

- The first register set contains the current values of the option bytes. Their names have the `_CUR` extension. All “`_CUR`” registers are read-only. Their values are automatically loaded from the non-volatile memory after power-on reset, wakeup from system standby or after an option byte change operation.
- The second register set allows the modification of the option bytes. Their names contain the `_PRG` extension. All “`_PRG`” registers can be accessed in read/write mode.

When the `OPTLOCK` bit in `FLASH_OPTCR` register is set, modifying the `_PRG` registers is not possible.

When `OPTSTART` bit is set to 1, the embedded flash memory checks if at least one option byte needs to be programmed by comparing the current values (`_CUR`) with the new ones (`_PRG`). If this is the case and all the other conditions are met (see [Changing security option bytes](#)), the embedded flash memory launches the option byte modification in its non-volatile memory and updates the option byte registers with `_CUR` extension.

If one of the condition described in [Changing security option bytes](#) is not respected, the embedded flash memory sets the `OPTCHANGEERR` flag to 1 in the `FLASH_OPTSR_CUR` register and aborts the option byte change operation. In this case, the `_PRG` registers are not overwritten by current option value. The user application can check what was wrong in their configuration.

Unlocking the option byte modification

After reset, the OPTLOCK bit is set to 1 and the FLASH_OPTCR is locked. As a result, the application software must unlock the option configuration register before attempting to change the option bytes. The FLASH_OPTCR unlock sequence is described in [Section 4.5.1: FLASH configuration protection](#).

Option byte modification sequence

To modify user option bytes, follow the sequence below:

1. Unlock FLASH_OPTCR register as described in [Section 4.5.1: FLASH configuration protection](#), unless the register is already unlocked.
2. Write the desired new option byte values in the corresponding option registers (FLASH_XXX_PRG1/2).
3. Set the option byte start change OPTSTART bit to 1 in the FLASH_OPTCR register.
4. Wait until OPT_BUSY bit is cleared.

Note: If a reset or a power-down occurs while the option byte modification is ongoing, the original option byte value is kept. A new option byte modification sequence is required to program the new value.

Changing security option bytes

On top of OPTLOCK bit, there is a second level of protection for security-sensitive option byte fields. Specific rules must be followed to update them:

- **Readout protection (RDP)**

A detailed description of RDP option bits is given in [Section 4.5.3](#). The following rules must be respected to modify these option bits:

- When RDP is set to level 2, no changes are allowed (except for the SWAP bit). As a result, if the user application attempts to reduce the RDP level, an option byte change error is raised (OPTCHANGEERR bit in FLASH_OPTSR_CUR register), and all the programmed changes are ignored.
- When the RDP is set to level 1, requiring a change to level 2 is always allowed. When requiring a regression to level 0, an option byte change error can occur if some of the recommendations provided in this chapter have not been followed.
- When the RDP is set to level 0, switching to level 1 or level 2 is possible without any restriction.

- **Sector write protection (WRPSn1/2)**

These option bytes manage sector write protection in FLASH_WPSN_CUR1/2R registers. They can be changed without any restriction when the RDP protection level is different from level 2.

- **PCROP area size (PROT_AREA_START1/2 and PROT_AREA_END1/2)**

These option bytes configure the size of the PCROP areas in FLASH_PRAR_CUR1/2 registers. They can be increased without any restriction by the Arm® Cortex®-M7 core. To remove or reduce a PCROP area, an RDP level 1 to 0 regression (see [Section 4.5.3](#)) or a bank erase with protection removal (see [Section 4.3.10](#)) must be

requested at the same time. DMEP must be set to 1 in either FLASH_PRAR_CUR1/2 or FLASH_PRAR_PRG1/2, otherwise an option byte change error is raised.

- **DMEP1/2**

When this option bit is set, the content of the corresponding PCROP area is erased during a RDP level 1 to 0 regression (see [Section 4.5.3](#)) or a bank erase with protection removal (see [Section 4.3.10](#)). It is preserved otherwise.

There are no restrictions in setting DMEP1/2 bit. Resetting DMEP1/2 bit from 1 to 0 can only be done when an RDP level 1 to 0 regression or a bank erase with protection removal is requested at the same time.

- **Secure access mode (SECURITY)**

The SECURITY option bit activates the secure access mode described in [Section 4.5.5](#). This option bit can be freely set by the application software if such mode is activated on the device. If at least one PCROP or secure-only area is defined as not null, the only way to deactivate the security option bit (from 1 to 0) is to perform an RDP level 1 to 0 regression, when DMEP1/2 is set to 1 in either FLASH_PRAR_CUR1/2 or FLASH_PRAR_PRG1/2 registers, and DMES1/2 is set to 1 in either FLASH_SCAR_CUR1/2 or FLASH_SCAR_PRG1/2.

If no valid secure-only area and no valid PCROP area are currently defined, the SECURITY option bit can be freely reset.

Note: It is recommended to have both SEC_AREA_START > SEC_AREA_END and PROT_AREA_START > PROT_AREA_END programmed when deactivating the SECURITY option bit during an RDP level 1 to 0 regression.

- **Secure-only area size (SEC_AREA_START1/2 and SEC_AREA_END1/2)**

These option bytes configure the size of the secure-only areas in FLASH_SCAR_CUR1/2 registers. They can be changed without any restriction by the user secure application or by the ST secure library running on the device. For user non-secure application, the secure-only area size can be removed by performing a bank erase with protection removal (see [Section 4.3.10](#)), or an RDP level 1 to 0 regression when DMES1/2 set to 1 in either FLASH_SCAR_CUR1/2 or FLASH_SCAR_PRG1/2 (otherwise an option byte change error is raised).

- **DMES1/2**

When this option bit is set, the content of the corresponding secure-only area is erased during an RDP level 1 to 0 regression or a bank erase with protection removal, it is preserved otherwise.

DMES1/2 bits can be set without any restriction. Resetting DMES1/2 bit from 1 to 0 can only be performed when an RDP level 1 to 0 regression or a bank erase with protection removal is requested at the same time.

4.4.4 Option bytes overview

Table 20 lists all the user option bytes managed through the embedded flash memory registers, as well as their default values before the first option byte change (default factory value).

Table 20. Option byte organization

Register	Bitfield															
FLASH_OPTSR[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	SWAP_BANK_OP	OPTCHANGEERR	IO_HSLV	Res.	Res.	Res.	NRST_STBY_D2	NRST_STOP_D2	BOOT_CM7	BOOT_CM4	SECURITY	ST_RAM_SIZE		IWDG_FZ_SDBY	IWDG_FZ_STOP	Res.
Default factory value	0	0	0	1	0	1	1	1	1	1	0	0	0	1	1	0
FLASH_OPTSR[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	RDP[7:0]								NRST_STDY_D1	NRST_STOP_D1	IWDG2_SW	IWDG1_SW	BOR_LEV	Res.	Res.	
Default factory value	1	0	1	0	1	0	1	0	1	1	1	1	0	0	0	0
FLASH_BOOT7[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	BOOT_ADD1BOOT_CM7_ADD1[15:0]															
Default factory value	0x1FF0															
FLASH_BOOT7[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	BOOT_ADD0BOOT_CM7_ADD0[15:0]															
Default factory value	0x0800															
FLASH_BOOT4[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	BOOT_CM4_ADD1[15:0]															
Default factory value	0x1000															
FLASH_BOOT4[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	BOOT_CM4_ADD0[15:0]															
Default factory value	0x0810															
FLASH_PRAR_x1[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	DMEP1	Res.	Res.	Res.	PROT_AREA_END1											

Table 20. Option byte organization (continued)

Register	Bitfield															
Default factory value	0	0	0	0	0x000											
FLASH_PRAR_x1[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res.	Res.	Res.	Res.	PROT_AREA_START1											
Default factory value	0	0	0	0	0x0FF											
FLASH_PRAR_x2[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	DMEP2	Res.	Res.	Res.	PROT_AREA_END2											
Default factory value	0	0	0	0	0x000											
FLASH_PRAR_x2[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res.	Res.	Res.	Res.	PROT_AREA_START2											
Default factory value	0	0	0	0	0x0FF											
FLASH_SCAR_x1[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	DMES1	Res.	Res.	Res.	SEC_AREA_END1											
Default factory value	0	0	0	0	0x000											
FLASH_SCAR_x1[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res.	Res.	Res.	Res.	SEC_AREA_START1											
Default factory value	0	0	0	0	0x0FF											
FLASH_SCAR_x2[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	DMES2	Res.	Res.	Res.	SEC_AREA_END2											
Default factory value	0	0	0	0	0x000											
FLASH_SCAR_x2[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res.	Res.	Res.	Res.	SEC_AREA_START2											
Default factory value	0	0	0	0	0x0FF											
FLASH_WPSN_x1[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
Default factory value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 20. Option byte organization (continued)

Register	Bitfield															
FLASH_WPSN_x1[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res	Res	Res	Res	Res	Res	Res	Res	WRPSn1[7:0]							
Default factory value	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
FLASH_WPSN_x2[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
Default factory value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
FLASH_WPSN_x2[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res	Res	Res	Res	Res	Res	Res	Res	WRPSn2[7:0]							
Default factory value	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

4.4.5 Description of user and system option bytes

Below the list of the general-purpose option bytes that can be used by the application:

- Watchdog management
 - IWDG_FZ_STOP: independent watchdogs (IWDG1 and 2) counter active in Stop mode if 1 (stop counting or freeze if 0)
 - IWDG_FZ_SDBY: independent watchdogs (IWDG1 and 2) counter active in Standby mode if 1 (stop counting or freeze if 0)
 - IWDG1_SW: hardware (0) or software (1) IWDG1 watchdog control selection
 - IWDG2_SW: hardware (0) or software (1) IWDG2 watchdog control selection

Note: *If the hardware watchdog “control selection” feature is enabled (set to 0), the corresponding watchdog is automatically enabled at power-on, thus generating a reset unless the watchdog key register is written to or the down-counter is reloaded before the end-of-count is reached.*

Depending on the configuration of IWDG_STOP and IWDG_STBY options, the IWDG can continue counting (1) or not (0) when the device is in Stop or Standby mode, respectively. When the IWDG is kept running during Stop or Standby mode, it can wake up the device from these modes.

- Reset management
 - BOR: Brownout level option, indicating the supply level threshold that activates/releases the reset (see [Section 7.5.2: Brownout reset \(BOR\)](#))
 - NRST_STDBY_D1/2: generates a reset when D1 (respectively D2) domain enters DStandby mode. It is active low.
 - NRST_STOP_D1/2: generates a reset when D1 (respectively D2) domain enters DStop mode. It is active low.

Note: Whenever a Standby (respectively Stop) mode entry sequence is successfully executed, the device is reset instead of entering Standby (respectively Stop) mode if `NRST_STDBY` (respectively `NRST_STOP`) is cleared to 0.

- Bank swapping (see [Section 4.3.13 on page 176](#))
SWAP_BANK_OPT: bank swapping option, set to 1 to swap user sectors and registers after boot.
- Device options
 - IO_HSLV: I/O speed optimization at low-voltage if set to 1.

When STMicroelectronics delivers the device, the values programmed in the general-purpose option bytes are the following:

- Watchdog management
 - IWDG1 and IWDG2 active in Standby and Stop modes (option byte value = 0x1)
 - IWDG1 and IWDG2 not automatically enabled at power-on (option byte value = 0x1)
- Reset management:
 - BOR: brownout level option (reset level) equals brownout reset threshold 0 (option byte value = 0x0)
 - A reset is not generated when D1 or D2 domain enters DStandby or DStop low-power mode (option byte value = 0x1)
- No bank swapping (option byte value = 0x0)
- Device working in the full voltage range with I/O speed optimization at low-voltage disabled (IO_HSLV=0)

Refer to [Section 4.9: FLASH registers](#) for details.

4.4.6 Description of data protection option bytes

Below the list of the option bytes that can be used to enhance data protection:

- RDP[7:0]: Readout protection level (see [Section 4.5.3 on page 191](#) for details).
- WRPSn1/2: write protection option of the corresponding Bank 1 (respectively Bank 2) sector. It is active low. Refer to [Section 4.5.4 on page 195](#) for details.
- PROT_AREAx: Proprietary code readout protection (refer to [Section 4.5.4 on page 195](#) for details)
 - PROT_AREA_START1 (respectively PROT_AREA_END1) contains the first (respectively last) 256-byte block of the PCROP zone in Bank 1
 - PROT_AREA_START2 (respectively PROT_AREA_END2) contains the first (respectively last) 256-byte block of the PCROP zone in Bank 2
 - DMEP1/2: when set to 1, the PCROP area in Bank 1 (respectively Bank 2) is erased during a RDP protection level regression (change from level 1 to 0) or a bank erase with protection removal.
- SEC_AREAx: secure access only zones definition (refer to [Section 4.5.5 on page 196](#) for details).
 - SEC_AREA_START1 (respectively SEC_AREA_END1) contains the first (respectively last) 256-byte block of the secure access only zone in Bank 1
 - SEC_AREA_START2 (respectively SEC_AREA_END2) contains the first (respectively last) 256-byte block of the secure access only zone in Bank 2
 - DMES1/2: when set to 1 the secure access only zone in Bank 1 (respectively Bank 2) is erased during a RDP protection level regression (change from level 1 to 0), or a bank erase with protection removal.
- SECURITY: this non-volatile option can be used by the application to manage secure access mode, as described in [Section 4.5.5](#).
- ST_RAM_SIZE: this non-volatile option defines the amount of DTCM RAM root secure services (RSS) can use during execution when the SECURITY bit is set. The DTCM RAM is always fully available for the application whatever the option byte configuration.

When STMicroelectronics delivers the device, the values programmed in the data protection option bytes are the following:

- RDP level 0 (option byte value = 0xAA)
- Flash bank erase operations do not impact secure-only and PCROP data areas when enabled by the application (DMES1/2=DMEP1/2=0)
- PCROP and secure-only zone protections disabled (start addresses higher than end addresses)
- Write protection enabled (all option byte bits set to 1)
- Secure access mode disabled (SECURITY option byte value = 0)
- RSS can use the DTCM RAM for executing its services (ST_RAM_SIZE=00)

Refer to [Section 4.9: FLASH registers](#) for details.

4.4.7 Description of boot address option bytes

Below the list of option bytes that can be used to configure the appropriate boot address for your application:

- Arm® Cortex®-M7 boot options
 - BOOT_CM7: option bit to enable Arm® Cortex®-M7 boot, when set to 1.
 - BOOT_CM7_ADD0/1: MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is low (respectively high)
- Arm® Cortex®-M4 boot options
 - BOOT_CM4: option bit to enable Arm® Cortex®-M4 boot, when set to 1.
 - BOOT_CM4_ADD0/1: MSB of the Arm® Cortex®-M4 boot address when the BOOT pin is low (respectively high)

When STMicroelectronics delivers the device, the values programmed in the BOOT ADDRESS option bytes are the following:

- Arm® Cortex®-M7 boot: enabled (0x1) with the MSB of the boot address equal to 0x0800 (BOOT pin low for user flash memory) and 0x1FF0 (BOOT pin high for System flash memory)
- Arm® Cortex®-M4 boot: enabled (0x1) with the MSB of the boot address equal to 0x0810 (BOOT pin low) and 0x1000 (BOOT pin high)

Refer to [Section 4.9: FLASH registers](#) for details.

4.5 FLASH protection mechanisms

Since sensitive information can be stored in the flash memory, it is important to protect it against unwanted operations such as reading confidential areas, illegal programming of protected area, or illegal flash memory erasing.

The embedded flash memory implements the following protection mechanisms that can be used by end-user applications to manage the security of embedded non-volatile storage:

- Configuration protection
- Global device Readout protection (RDP)
- Write protection
- Proprietary code readout protection (PCROP)
- Secure access mode areas

This section provides a detailed description of all these security mechanisms.

4.5.1 FLASH configuration protection

The embedded flash memory uses hardware mechanisms to protect the following assets against unwanted or spurious modifications (e.g. software bugs):

- Option bytes change
- Write operations
- Erase commands
- Interrupt masking

More specifically, write operations to embedded flash memory control registers (FLASH_CR1/2 and FLASH_OPTCR) are not allowed after reset.

The following sequence must be used to unlock FLASH_CR1/2 register:

1. Program KEY1 to 0x45670123 in FLASH_KEYR1/2 key register.
2. Program KEY2 to 0xCDEF89AB in FLASH_KEYR1/2 key register.
3. LOCK1/2 bit is now cleared and FLASH_CR1/2 is unlocked.

The following sequence must be used to unlock FLASH_OPTCR register:

1. Program OPTKEY1 to 0x08192A3B in FLASH_OPTKEYR option key register.
2. Program OPTKEY2 to 0x4C5D6E7F in FLASH_OPTKEYR option key register.
3. OPTLOCK bit is now cleared and FLASH_OPTCR register is unlocked.

Any wrong sequence locks up the corresponding register/bit until the next system reset, and generates a bus error.

The FLASH_CR1/2 (respectively FLASH_OPTCR) register can be locked again by software by setting the LOCK1/2 bit in FLASH_CR1/2 register (respectively OPTLOCK bit in FLASH_OPTCR register).

In addition the FLASH_CR1/2 register remains locked and a bus error is generated when the following operations are executed:

- programming a third key value
- writing to a different register belonging to the same bank than FLASH_KEYR1/2 before FLASH_CR1/2 has been completely unlocked (KEY1 programmed but KEY2 not yet programmed)
- writing less than 32 bits to KEY1 or KEY2.

Similarly the FLASH_OPTCR register remains locked and a bus error is generated when the following operations are executed:

- programming a third key value
- writing to a different register before FLASH_OPTCR has been completely unlocked (OPTKEY1 programmed but OPTKEY2 not yet programmed)
- writing less than 32 bits to OPTKEY1 or OPTKEY2.

The embedded flash memory configuration registers protection is summarized in [Table 21](#).

Table 21. Flash interface register protection summary

Register name	Unlocking register	Protected asset
FLASH_ACR	N/A	-
FLASH_KEYR1/2	N/A	-
FLASH_OPTKEYR	N/A	-
FLASH_CR1/2	FLASH_KEYR1/2	Write operations Erase commands Interrupt generation masking sources
FLASH_SR1/2	N/A	-
FLASH_CCR1/2	N/A	-
FLASH_OPTCR	FLASH_OPTKEYR	Option bytes change Mass erase

Table 21. Flash interface register protection summary (continued)

Register name	Unlocking register	Protected asset
FLASH_OPTSR_PRG	FLASH_OPTCR	Option bytes change. See Section 4.4.3: Option byte modification for details.
FLASH_OPTCCR	N/A	-
FLASH_PRAR_PRG1/2	FLASH_OPTCR	Option bytes (PCROP). See Section 4.4.3: Option byte modification for details.
FLASH_SCAR_PRG1/2	FLASH_OPTCR	Option bytes (security). See Section 4.4.3: Option byte modification for details.
FLASH_WPSGN_PRG1/2	FLASH_OPTCR	Option bytes (write protection)
FLASH_BOOT4_PRGR FLASH_BOOT7_PRGR	FLASH_OPTCR	Option bytes (boot)
FLASH_CRCCR1/2	N/A	-
FLASH_CRCSADD1/2R		-
FLASH_CRCEADD1/2R		-
FLASH_CRCDATAR		-
FLASH_ECC_FA1/2R	N/A	-

4.5.2 Write protection

The purpose of embedded flash memory write protection is to protect the embedded flash memory against unwanted modifications of the non-volatile code and/or data.

Any 128-Kbyte flash sector can be independently write-protected or unprotected by clearing/setting the corresponding WRPSn1/2 bit in the FLASH_WPSN_PRG1/2R register.

A write-protected sector can neither be erased nor programmed. As a result, a bank erase cannot be performed if one sector is write-protected, unless a bank erase is executed during an RDP level 1 to 0 regression (see [Section : Flash bank erase with automatic protection-removal sequence](#) for details).

The embedded flash memory write-protection user option bits can be modified without any restriction when the RDP level is set to level 0 or level 1. When it is set to level 2, the write protection bitfield can no more be changed in the option bytes.

Note: PCROP or secure-only areas are write and erase protected.

Write protection errors are documented in [Section 4.7: FLASH error management](#).

4.5.3 Readout protection (RDP)

The embedded flash memory readout protection is global as it does not apply only to the embedded flash memory, but also to the other secured regions. This is done by using dedicated security signals.

In this section *other secured regions* are defined as:

- Backup SRAM
- RTC backup registers

The global readout protection level is set by writing the values given in [Table 22](#) into the readout protection (RDP) option byte (see [Section 4.4.3: Option byte modification](#)).

Table 22. RDP value vs readout protection level

RDP option byte value	Global readout protection level
0xAA	Level 0
0xCC	Level 2
Any other value	Level 1 ⁽¹⁾

1. Default protection level when RDP option byte is erased.

Definitions of RDP global protection level

RDP Level 0 (no protection)

When the global read protection level 0 is set, all read/program/erase operations from/to the user flash memory are allowed (if no others protections are set). This is true whatever the boot configuration (boot from user or system flash memory, boot from RAM), and whether the debugger is connected to the device or not. Accesses to the *other secured regions* are also allowed.

RDP Level 1 (flash memory content protection)

When the global read protection level 1 is set, the below properties apply:

- The flash memory content is protected against debugger and potential malicious code stored in RAM. Hence as soon as any debugger is connected or has been connected, or a boot is configured in embedded RAM (intrusion), the embedded flash memory prevents any accesses to flash memory.
- When no intrusion is detected (no boot in RAM, no boot in System flash memory and no debugger connected), all read/program/erase operations from/to the user flash memory are allowed (if no others protections are set). Accesses to the *other secured regions* are also allowed.
- When an intrusion is detected, no accesses to the user flash memory can be performed. A bus error is generated when a read access is requested to the flash

memory. In addition, no accesses to *other secured regions* (read or write) can be performed.

- When performing an RDP level regression, i.e. programming the RDP protection to level 0, the user flash memory and the *other secured regions* are erased, as described in [RDP protection transitions](#).
- When booting on STM32 bootloader in standard system memory, only the identification services are available (GET_ID_COMMAND, GET_VER_COMMAND and GET_CMD_COMMAND).
- When booting from STMicroelectronics non-secure bootloader, only the identification services are available (GET_ID_COMMAND, GET_VER_COMMAND and GET_CMD_COMMAND).

RDP Level 2 (device protection and intrusion prevention)

When the global read protection level 2 is set, the below rules apply:

- All debugging features are disabled.
- Like level 0, all read/write/erase operations from/to the user flash memory are allowed since the debugger and the boot from RAM and System flash memory are disabled. Accesses to the *other secured regions* are also allowed.
- Booting from RAM is no more allowed.
- The user option bits described in [Section 4.4](#) can no longer be changed except for the SWAP bit.

Caution: Memory read protection level 2 is an irreversible operation. When level 2 is activated, the level of protection cannot be changed back to level 0 or level 1.

Note: *The JTAG port is permanently disabled when level 2 is active (acting as a JTAG fuse). As a consequence, STMicroelectronics is not able to perform analysis on defective parts on which the level 2 protection has been set.*

Apply a power-on reset if the global read protection level 2 is set while the debugger is still connected.

The above RDP global protection is summarized in [Table 23](#).

Table 23. Protection vs RDP Level⁽¹⁾

Boot area	Inputs		Effects				Comment
	RDP	Debugger connected	User flash memory access ⁽²⁾	System flash memory access ⁽³⁾	Other secured regions ⁽⁴⁾	Option Bytes access	
User flash memory	Level 0	Yes ⁽⁵⁾ /No	R/W/E	R	R/W	R/W	-
	Level 1	Yes ⁽³⁾	illegal access ⁽⁶⁾	R	no access	R/W	
	Level 1	No	R/W/E	R	R/W	R/W	
	Level 2	No	R/W/E	R	R/W	R	

Table 23. Protection vs RDP Level⁽¹⁾ (continued)

Boot area	Inputs		Effects				Comment
	RDP	Debugger connected	User flash memory access ⁽²⁾	System flash memory access ⁽³⁾	Other secured regions ⁽⁴⁾	Option Bytes access	
RAM or System flash memory	Level 0	Yes ⁽³⁾ /No	R/W/E	R	R/W	R/W	-
	Level 1	Yes ⁽³⁾ /No	illegal access ⁽⁶⁾	R	no access	R/W	When selected, only ST basic bootloader is executed
	Level 2	No	Not applicable				No boot from RAM or ST system flash memory in RDP level 2

1. R = read, W = write, E = erase.

2. PCROP (see [Section 4.5.4](#)) and secure-only access control (see [Section 4.5.5](#)) applies.

3. Read accesses to secure boot and secure libraries stored in system flash bank 1 possible only from STMicroelectronics code.

4. The “other secured regions” are defined at the beginning of this section.

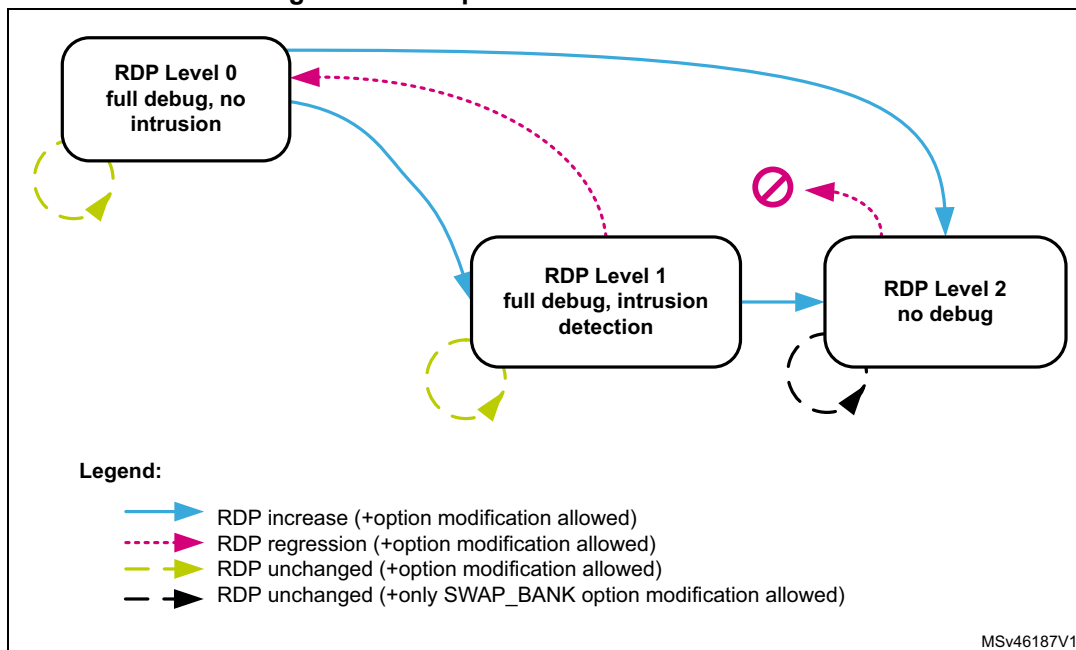
5. JTAG interface disabled while secure libraries are executed.

6. [Read protection error \(RDPEER\)](#) with bus error on read operations, [Write protection error \(WRPEER\)](#) on write/erase operations.

RDP protection transitions

[Figure 13](#) shows how to switch from one RDP level to another. The transition is effective after successfully writing the option bytes including RDP (refer to [Section 4.4.3](#) for details on how to change the option bytes).

Figure 13. RDP protection transition scheme



[Table 24](#) details the RDP transitions and their effects on the product.

Table 24. RDP transition and its effects

RDP transition		RDP option update	Effect on device		
Level before change	Level after change		Debugger disconnect	Option bytes change ⁽¹⁾	Partial/Mass erase
L0	L1	not 0xAA and not 0xCC	No	Allowed	No
	L2	0xCC	Yes	Not allowed	No
L1	L2	0xCC	Yes	Not allowed	No
	L0	0xAA	No	Allowed	Yes
L0	L0	0xAA	No	Allowed	No
L1	L1	not 0xAA and not 0xCC			

1. Except for bank swapping option bit.

When the current RDP level is RDP level 1, requesting a new RDP level 0 can cause a full or partial erase:

- The user flash memory area of the embedded flash memory is fully or partially erased:
 - A partial sector erase occurs if PCROP (respectively secure-only) areas are preserved by the application. It happens when both DMEP1/2 bits (respectively DMES1/2 bits) are cleared to 0 in FLASH_PRAR_CUR1/2 and FLASH_PRAR_PRG1/2 (respectively FLASH_SCAR_CUR1/2 and FLASH_SCAR_PRG1/2). The sectors belonging to the preserved area(s) are not erased.
 - A full bank erase occurs when at least one DMEP1/2 bit is set to 1 in FLASH_PRAR_CUR1/2 or FLASH_PRAR_PRG1/2, and at least one DMES1/2 bit is set to 1 in FLASH_SCAR_CUR1/2 or FLASH_SCAR_PRG1/2.

Note: *Data in write protection area are not preserved during RDP regression.*

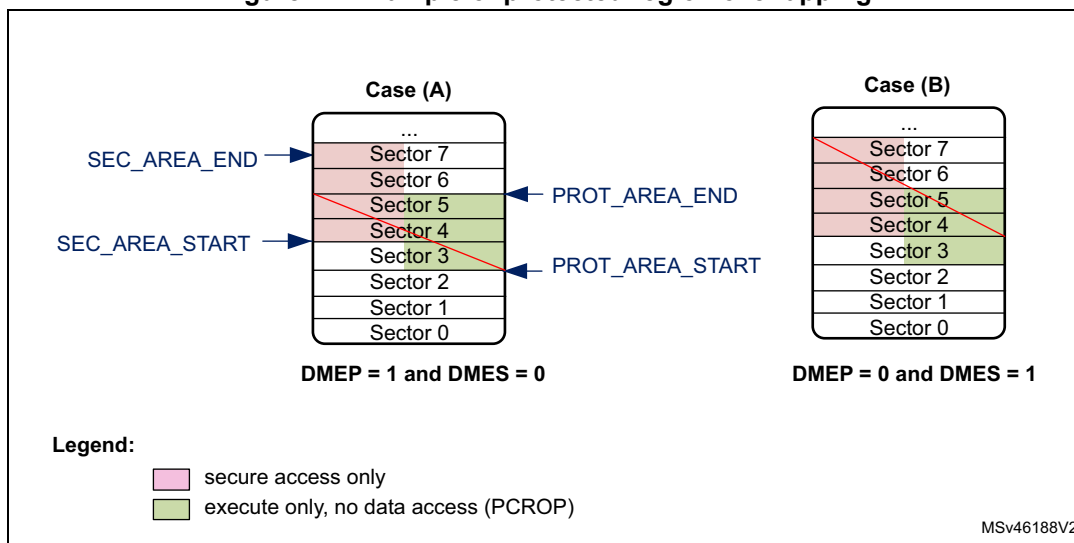
- The other secured regions are also erased (Backup SRAM and RTC backup registers)

During a level regression, if a PCROP area overlaps with a secure-only area, the embedded flash memory performs the erase operation depending on the DMES/DMEP options bits (see strike-through areas in red in [Figure 14](#)). More specifically:

- When DMEP is set in FLASH_PRAR_CUR1/2 or FLASH_PRAR_PRG1/2, the PCROP area is erased (overlapped or not with secure-only area).
- When DMES is set in FLASH_SCAR_CUR1/2 or FLASH_SCAR_PRG1/2, the secure-only area is erased (overlapped or not with PCROP area).

Note: *The sector protections (PCROP, secure-only) are removed only if the protected sector boundaries are modified by the user application.*

Figure 14. Example of protected region overlapping



About RDP protection errors

Whatever the RDP level, the corresponding error flag is raised when an illegal read or write access is detected (see [Section 4.7: FLASH error management](#)).

4.5.4 Proprietary code readout protection (PCROP)

The embedded flash memory allows the definition of an “executable-only” area in the user area of each flash memory bank. In this area, only instruction fetch transactions from the system, that is no data access (data or literal pool), are allowed. This protection is particularly efficient to protect third party software intellectual property.

Note: Executable-only area usage requires the native code to be compiled accordingly using “execute-only” option.

PCROP area programming

One PCROP area can be defined in bank 1 (respectively bank 2) by setting the PROT_AREA_END1 and PROT_AREA_START1 (respectively PROT_AREA_END2 and PROT_AREA_START2) option bytes so that the END address is strictly higher than the START address. PROT_AREA_START and PROT_AREA_END are defined with a granularity of 256 bytes. This means that the actual PCROP area size (in bytes) is defined by:

$$[(\text{PROT_AREA_END} - \text{PROT_AREA_START}) + 1] \times 256$$

As an example, to set a PCROP area on the first 4 Kbytes of user bank 1 (i.e. from address 0x0800 0000 to address 0x0800 0FFF, both included), the embedded flash memory must be configured as follows:

- PROT_AREA_START1[11:0] = 0x000
- PROT_AREA_END1[11:0] = 0x00F

The protected area size defined above is equal to:

$$[(\text{PROT_AREA_END} - \text{PROT_AREA_START}) + 1] \times 256 = 16 \times 256 \text{ bytes} = 4 \text{ Kbytes.}$$

The minimum execute-only PCROP area that can be set is 16 flash words (or 512 bytes). The maximum area is the whole user flash memory, configured by setting to the same value the PCROP area START and END addresses.

Note: It is recommended to align the PCROP area size with the flash sector granularity in order to avoid access right issues.

PCROP area properties

Each valid PCROP area has the following properties:

- Arm® Cortex®-M7 debug events are ignored while executing code in this area.
- Only the Arm® Cortex®-M7 core can access it (Master ID filtering), using only instruction fetch transactions. In all other cases, accessing the PCROP area is illegal (see below).
- Illegal transactions to a PCROP area (i.e. data read or write, not fetch) are managed as below:
 - Read operations return a zero, write operations are ignored.
 - No bus error is generated but an error flag is raised (RDPERR for read, WRPERR for write).
- A valid PCROP area is erase-protected. As a result:
 - No erase operations to a sector located in this area is possible (including the sector containing the area start address and the end address)
 - No mass erase can be performed if a single valid PCROP area is defined, except during level regression or erase with protection removal.
- Only the Arm® Cortex®-M7 core can modify the PCROP area definition and DMEP1/2 bits, as explained in [Changing user option bytes](#) in [Section 4.4.3](#).
- During an RDP level 1 to 0 regression where the PCROP area is not null
 - The PCROP area content is not erased if the corresponding DMEP1/2 bit are both cleared to 0 in FLASH_PRAR_CUR1/2 and FLASH_PRAR_PRG1/2 registers.
 - The PCROP area content is erased if either of the corresponding DMEP1/2 bit is set to 1 in FLASH_PRAR_CUR1/2 or FLASH_PRAR_PRG1/2 register.

For more information on PCROP protection errors, refer to [Section 4.7: FLASH error management](#).

4.5.5 Secure access mode

The embedded flash memory allows the definition of a secure-only area in the user area of each flash memory bank. This area can be accessed only while the CPU executes secure application code. This feature is available only if the SECURITY option bit is set to 1.

Secure-only areas help isolating secure user code from application non-secure code. As an example, they can be used to protect a customer secure firmware upgrade code, a custom secure boot library or a third party secure library.

Secure-only area programming

One secure-only area can be defined in bank 1 (respectively bank 2) by setting the SEC_AREA_END1 and SEC_AREA_START1 (respectively SEC_AREA_END2 and SEC_AREA_START2) option bytes so that the END address is strictly higher than the

START address. SEC_AREA_START and SEC_AREA_END are defined with a granularity of 256 bytes. This means that the actual secure-only area size (in bytes) is defined by:

$$[(\text{SEC_AREA_END} - \text{SEC_AREA_START}) + 1] \times 256$$

As an example, to set a secure-only area on the first 8 Kbytes of user bank 2 (i.e. from address 0x0810 0000 to address 0x0810 1FFF, both included), the embedded flash memory must be configured as follows:

- SEC_AREA_START2[11:0] = 0x000
- SEC_AREA_END2[11:0] = 0x01F

The secure-only area size defined above is equal to:

$$[(\text{SEC_AREA_END} - \text{SEC_AREA_START}) + 1] \times 256 = 32 \times 256 \text{ bytes} = 8 \text{ Kbytes.}$$

Note: These option bytes can be modified only by the Arm® Cortex®-M7 core running ST security library or application secure code, except during RDP level regression or erase with protection removal.

The minimum secure-only area that can be set is 16 flash words (or 512 bytes). The maximum area is the whole user flash memory bank, configured by setting to the same value the secure-only area START and END addresses.

Note: It is recommended to align the secure-only area size with flash sector granularity in order to avoid access right issues.

Secure access-only area properties

- Arm® Cortex®-M7 debug events are ignored while executing code in this area.
- Only the Arm® Cortex®-M7 core executing ST secure library or user secure application can access it (Master ID filtering). In all other cases, accessing the secure-only area is illegal (see below).
- Illegal transactions to a secure-only area are managed as follows:
 - Data read transactions return zero. Data write transactions are ignored. No bus error is generated but an error flag is raised (RDSERR for read, WRPERR for write).
 - Read instruction transactions generate a bus error and the RDSERR error flag is raised.
- A valid secure-only area is erase-protected. As a result:
 - No erase operations to a sector located in this area are possible (including the sector containing the area start address and the end address), unless the application software is executed from a valid secure-only area.
 - No mass erase can be performed if a single valid secure-only area is defined, except during level regression, erase with protection removal or when the application software is executed from a valid secure-only area.
- Only the Arm® Cortex®-M7 core can modify the secure-only area definition and DMES1/2 bits, as explained in [Changing user option bytes](#) in [Section 4.4.3](#).
- During an RDP level 1 to 0 regression where the secure-only area is not null:
 - the secure-only area content is not erased if the corresponding DMES1/2 bit are both cleared to 0 in FLASH_SCAR_CUR1/2 and FLASH_SCAR_PRG1/2 registers.
 - the secure-only area content is erased if either of the corresponding DMES1/2 bit is set to 1 in FLASH_SCAR_CUR1/2 or FLASH_SCAR_PRG1/2 register.

For more information on secure-only protection errors, refer to [Section 4.7: FLASH error management](#).

4.6 FLASH low-power modes

4.6.1 Introduction

The table below summarizes the behavior of the embedded flash memory in the microcontroller low-power modes. The embedded flash memory belongs to the D1 domain.

Table 25. Effect of low-power modes on the embedded flash memory

System state	Power mode		D1 domain voltage range	Allowed if FLASH busy	FLASH power mode (in D1 domain)
	D1 domain	D2 domain			
Run	DRun	DRun, DStop or DStandby	VOS1/2/3	Yes	Run
	DStop		SVOS3/4/5	No	Clock gated or Stopped
	DStandby	DRun, DStop or DStandby	Off	No	Off
Stop	DStop	DStop or DStandby	SVOS3/4/5	No	Clock gated or Stopped
	DStandby	DStop or DStandby	Off	No	Off
Standby ⁽¹⁾	DStandby	DStandby		No	

1. D3 domain must always be in DStandby mode. When all clocks are stopped and both CPUs are in CStop, the VCore domain is switched off.

When the system state changes or within a given system state, the embedded flash memory might get a different voltage supply range (VOS) according to the application. The procedure to switch the embedded flash memory into various power mode (run, clock gated, stopped, off) is described hereafter.

Note: For more information in the microcontroller power states, refer to the *Power control section (PWR)*.

4.6.2 Managing the FLASH domain switching to DStop or DStandby

As explain in [Table 25](#), if the embedded flash memory informs the reset and clock controller (RCC) that it is busy (i.e. BSY1/2, QW1/2, WBNE1/2 is set), the microcontroller cannot switch the D1 domain to DStop or DStandby mode.

Note: CRC_BUSY1/2 is not taken into account.

There are two ways to release the embedded flash memory:

- Reset the WBNE1/2 busy flag in FLASH_SR1/2 register by any of the following actions:
 - a) Complete the write buffer with missing data.
 - b) Force the write operation without filling the missing data by activating the FW1/2 bit in FLASH_CR1/2 register. This forces all missing data “high”.
 - c) Reset the PG1/2 bit in FLASH_CR1/2 register. This disables the write buffer and consequently lead to the loss of its content.
- Poll QW1/2 busy bits in FLASH_SR1/2 register until they are cleared. This will indicate that all recorded write, erase and option change operations are complete.

The microcontroller can then switch the domain to DStop or DStandby mode.

4.7 FLASH error management

4.7.1 Introduction

The embedded flash memory automatically reports when an error occurs during a read, program or erase operation. A wide range of errors are reported:

- [Write protection error \(WRPERR\)](#)
- [Programming sequence error \(PGSERR\)](#)
- [Strobe error \(STRBERR\)](#)
- [Inconsistency error \(INCERR\)](#)
- [Operation error \(OPERR\)](#)
- [Error correction code error \(SNECCERR/DBECCERR\)](#)
- [Read protection error \(RDPERR\)](#)
- [Read secure error \(RDSERR\)](#)
- [CRC read error \(CRCDERR\)](#)
- [Option byte change error \(OPTCHANGEERR\)](#)

The application software can individually enable the interrupt for each error, as detailed in [Section 4.8: FLASH interrupts](#).

Note: For some errors, the application software must clear the error flag before attempting a new operation.

Each bank has a dedicated set of error flags in order to identify which bank generated the error. They are available in flash Status register 1 or 2 (FLASH_SR1/2).

4.7.2 Write protection error (WRPERR)

When an illegal erase/program operation is attempted to the non-volatile memory bank 1 (respectively bank 2), the embedded flash memory sets the write protection error flag WRPERR1 (respectively WRPERR2) in FLASH_SR1 register (respectively FLASH_SR2).

An erase operation is rejected and flagged as illegal if it targets one of the following memory areas:

- A sector belonging to a valid PCROP area (even partially)
- A sector belonging to a valid secure-only area (even partially) except if the application software is executed from a valid secure-only area
- A sector write-locked with WRPSn

An program operation is ignored and flagged as illegal if it targets one of the following memory areas:

- The system flash memory (bank 2 only) while the device is not executing ST bootloader code
- A user flash sector belonging to a valid PCROP area while the device is not executing an ST secure library
- A user flash sector belonging to a valid secure-only area while the device is not executing user secure code or ST secure library
- A user sector write-locked with WRPSn
- The bank 1 system flash memory
- The user main flash memory when RDP level is 1 and a debugger has been detected on the device, or an Arm® Cortex® core has not booted from user flash memory.
- A reserved area

When WRPERR1/2 flag is raised, the operation is rejected and nothing is changed in the corresponding bank. If a write burst operation was ongoing, WRPERR1/2 is raised each time a flash word write operation is processed by the embedded flash memory.

Note: WRPERR1/2 flag does not block any new erase/program operation.

Not resetting the write protection error flag (WRPERR1/2) does not generate a PGSEERR error.

WRPERR1/2 flag is cleared by setting CLR_WRPERR1/2 bit to 1 in FLASH_CCR1/2 register.

If WRPERRIE1/2 bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when WRPERR1/2 flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.3 Programming sequence error (PGSEERR)

When the programming sequence to the bank 1 (respectively bank 2) is incorrect, the embedded flash memory sets the programming sequence error flag PGSEERR1 (respectively PGSEERR2) in FLASH_SR1 register (respectively FLASH_SR2).

More specifically, PGSEERR1/2 flag is set if one of below conditions is met:

- A write operation is requested but the program enable bit (PG1/2) has not been set in FLASH_CR1/2 register prior to the request.
- The inconsistency error (INCERR1/2) has not been cleared to 0 before requesting a new write operation.

When PGSEERR1/2 flag is raised, the current program operation is aborted and nothing is changed in the corresponding bank. The corresponding write data buffer is also flushed. If a write burst operation was ongoing, PGSEERR1/2 is raised at the end of the burst.

Note: When PGSEERR1/2 flag is raised, there is a risk that the last write operation performed by the application has been lost because of the above protection mechanism. Hence it is recommended to generate interrupts on PGSEERR and verify in the interrupt handler if the last write operation has been successful by reading back the value in the flash memory.

The PGSEERR1/2 flag also blocks any new program operation. This means that PGSEERR1 (respectively 2) must be cleared before starting a new program operation on bank 1 (respectively bank 2).

PGSERR1/2 flag is cleared by setting CLR_PGSERR1/2 bit to 1 in FLASH_CCR1/2 register.

If PGSERRIE1/2 bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when PGSERR1/2 flag is raised. See [Section 4.8: FLASH interrupts](#) for details.

4.7.4 Strobe error (STRBERR)

When the application software writes several times to the same byte in bank 1 (respectively bank 2) write buffer, the embedded flash memory sets the strobe error flag STRBERR1 (respectively STRBERR2) in FLASH_SR1 register (respectively FLASH_SR2).

When STRBERR1/2 flag is raised, the current program operation is not aborted and new byte data replace the old ones. The application can ignore the error, proceed with the current write operation and request new write operations. If a write burst was ongoing, STRBERR1/2 is raised at the end of the burst.

STRBERR1/2 flag is cleared by setting CLR_STRBERR1/2 bit to 1 in FLASH_CCR1/2 register.

If STRBERRIE1/2 bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when STRBERR1/2 flag is raised. See [Section 4.8: FLASH interrupts](#) for details.

4.7.5 Inconsistency error (INCERR)

When a programming inconsistency to bank 1 (respectively bank 2) is detected, the embedded flash memory sets the inconsistency error flag INCERR1 (respectively INCERR2) in register FLASH_SR1 (respectively FLASH_SR2).

More specifically, INCERR flag is set when one of the following conditions is met:

- A write operation is attempted before completion of the previous write operation, e.g.
 - The application software starts a write operation to fill the 256-bit write buffer, but sends a new write burst request to a different flash memory address before the buffer is full.
 - One master starts a write operation, but before the buffer is full, another master starts a new write operation to the same address or to a different address.
- A wrap burst request issued by a master overlaps two or more 256-bit flash-word addresses, i.e. wrap bursts must be done within 256-bit flash-word address boundaries.

Note: INCERR flag must be cleared before starting a new write operation, otherwise a sequence error (PGSERR) is raised.

It is recommended to follow the sequence below to avoid losing data when an inconsistency error occurs:

1. Execute a handler routine when INCERR1 or INCERR2 flag is raised.
2. Stop all write requests to embedded flash memory.
3. Verify that the write operations that have been requested just before the INCERR event have been successful by reading back the programmed values from the memory.
4. Clear the corresponding INCERR1/2 bit.
5. Restart the write operations where they have been interrupted.

INCERR1/2 flag is cleared by setting CLR_INCERR1/2 bit to 1 in FLASH_CCR1/2 register.

If INCERRIE1/2 bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when INCERR1/2 flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.6 Operation error (OPERR)

When an error occurred during a write or an erase operation to bank 1 (respectively bank 2), the embedded flash memory sets the operation error flag OPERR1 (respectively OPERR2) in FLASH_SR1 register (respectively FLASH_SR2). This error may be caused by an incorrect non-volatile memory behavior due to cycling issues or to a previous modify operation stopped by a system reset.

When OPERR1/2 flag is raised, the current program/erase operation is aborted.

OPERR1/2 flag is cleared by setting CLR_OPERR1/2 bit to 1 in FLASH_CCR1/2 register.

If OPERRIE1/2 bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when OPERR1/2 flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.7 Error correction code error (SNECCERR/DBECCERR)

When a single error correction is detected during a read from bank 1 (respectively bank 2) the embedded flash memory sets the single error correction flag SNECCERR1 (respectively SNECCERR2) in FLASH_SR1 register (respectively FLASH_SR2).

When two ECC errors are detected during a read to bank 1 (respectively bank 2), the embedded flash memory sets the double error detection flag DBECCERR1 (respectively DBECCERR2) in FLASH_SR1 register (respectively FLASH_SR2). When SNECCERR1/2 flag is raised, the corrected read data are returned. Hence the application can ignore the error and request new read operations.

If a read burst operation was ongoing, SNECCERR1/2 or DBECCERR1/2 flag is raised each time a new data is sent back to the requester through the AXI interface.

When SNECCERR1/2 or DBECCERR1/2 flag is raised, the address of the flash word that generated the error is saved in the FLASH_ECC_FA1/2R register. This register is automatically cleared when the associated flag that generated the error is reset.

Note: In case of successive single correction or double detection errors, only the address corresponding to the first error is stored in FLASH_ECC_FA1/2R register.

When DBECCERR1/2 flag is raised, a bus error is generated. In case of successive double error detections, a bus error is generated each time a new data is sent back to the requester through the AXI interface.

Note: It is not mandatory to clear SNECCERR1/2 or DBECCERR1/2 flags before starting a new read operation.

SNECCERR1/2 (respectively DBECCERR1/2) flag is cleared by setting to 1 CLR_SNECCERR1/2 bit (respectively CLR_DBECCERR1/2 bit) in FLASH_CCR1/2 register.

If SNECCERR1/2 (respectively DBECCERR1/2) bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when SNECCERR1/2 (respectively DBECCERR1/2) flag is raised. See [Section 4.8: FLASH interrupts](#) for details.

4.7.8 Read protection error (RDPERR)

When a read operation to a PCROP, a secure-only or a RDP protected area is attempted in non-volatile memory bank 1 (respectively bank 2), the embedded flash memory sets the read protection error flag RDPERR1 (respectively RDPERR2) in FLASH_SR1 register (respectively FLASH_SR2).

When RDPERR1/2 flag is raised, the current read operation is aborted but the application can request new read operations. If a read burst was ongoing, RDPERR1/2 is raised each time a data is sent back to the requester through the AXI interface.

Note: A bus error is raised if a standard application attempts to execute on a secure-only or a RDP protected area.

RDPERR1/2 flag is cleared by setting CLR_RDPERR1/2 bit to 1 in FLASH_CCR1/2 register.

If RDPERRIE1/2 bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when RDPERR1/2 flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.9 Read secure error (RDSERR)

When a read operation is attempted to a secure address in bank 1 (respectively bank 2), the embedded flash memory sets the read secure error flag RDSERR1 (respectively RDSERR2) in FLASH_SR1 register (respectively FLASH_SR2). For more information, refer to [Section 4.5.5: Secure access mode](#).

When RDSERR1/2 flag is raised, the current read operation is aborted and the application can request new read operations. If a read burst was ongoing, RDSERR1/2 is raised each time a data is sent back to the requester through the AXI interface.

Note: The bus error is raised only if the illegal access is due to an instruction fetch.

RDSERR1/2 flag is cleared by setting CLR_RDSERR1/2 bit to 1 in FLASH_CCR1/2 register.

If RDSERRIE1/2 bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when RDSERR1/2 flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.10 CRC read error (CRCDERR)

After a CRC computation in bank 1 (respectively bank 2), the embedded flash memory sets the CRC read error flag CRCDERR1 (respectively CRCDERR2) in FLASH_SR1 register (respectively FLASH_SR2) when one or more address belonging to a protected area was read by the CRC module. A protected area corresponds to a PCROP area (see [Section 4.5.4](#)) or to a secure-only area (see [Section 4.5.5](#)).

CRCDERR1/2 flag is raised when CRCEND1/2 bit is set to 1 (end of CRC calculation). In this case, it is likely that the CRC result is wrong since illegal read operations to protected areas return null values.

CRCDERR1/2 flag is cleared by setting CLR_CRCDERR1/2 bit to 1 in FLASH_CCR1/2 register.

If CRCDERRIE1/2 bit in FLASH_CR1/2 register is set to 1, an interrupt is generated when CRCDERR1/2 flag is raised together with CRCEND1/2 bit (see [Section 4.8: FLASH interrupts](#) for details).

4.7.11 Option byte change error (OPTCHANGEERR)

When the embedded flash memory finds an error during an option change operation, it aborts the operation and sets the option byte change error flag OPTCHANGEERR in FLASH_OPTSR_CUR register.

OPTCHANGEERR flag is cleared by setting CLR_OPTCHANGEERR bit to 1 in FLASH_OPTCCR register.

If OPTCHANGEERRIE bit in FLASH_OPTCR register is set to 1, an interrupt is generated when OPTCHANGEERR flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.12 Miscellaneous HardFault errors

The following events generate a bus error on the corresponding bus interface:

- On AXI system bus:
 - accesses to user flash memory while RDP is set to 1 and a illegal condition is detected (boot from system flash memory, boot from RAM, or debugger connected)
 - fetching to secure-only user flash memory without the correct access rights
- On AHB configuration bus:
 - wrong key input to FLASH_KEYR1/2 or FLASH_OPTKEYR

4.8 FLASH interrupts

The embedded flash memory can generate a maskable interrupt to signal the following events on a given bank:

- Read and write errors (see [Section 4.7: FLASH error management](#))
 - Single ECC error correction during read operation
 - Double ECC error detection during read operation
 - Write inconsistency error
 - Bad programming sequence
 - Strobe error during write operations
 - Non-volatile memory write/erase error (due to cycling issues)
 - option change operation error
- Security errors (see [Section 4.7: FLASH error management](#))
 - Write protection error
 - Read protection error
 - Read secure error
 - CRC computation on PCROP or secure-only area error
- Miscellaneous events (described below)
 - End of programming
 - CRC computation complete

These multiple sources are combined into a single interrupt signal, **flash_it**, which is the only interrupt signal from the embedded flash memory that drives the NVIC (nested vectored interrupt controller).

You can individually enable or disable embedded flash memory interrupt sources by changing the mask bits in the FLASH_CR1/2 register. Setting the appropriate mask bit to 1 enables the interrupt.

Note: Prior to writing, FLASH_CR1/2 register must be unlocked as explained in [Section 4.5.1: FLASH configuration protection](#)

[Table 26](#) gives a summary of the available embedded flash memory interrupt features. As mentioned in the table below, some flags need to be cleared before a new operation is triggered.

Table 26. Flash interrupt request

Interrupt event		Event flag	Enable control bit	Clear flag to resume operation	Bus error
End-of-program event	on bank 1	EOP1	EOPIE1	N/A	N/A
	on bank 2	EOP2	EOPIE2		
CRC complete event	on bank 1	CRCEND1	CRCENDIE1	N/A	N/A
	on bank 2	CRCEND2	CRCENDIE2		
Write protection error	on bank 1	WRPERR1	WRPERRIE1	No	No
	on bank 2	WRPERR2	WRPERRIE2		
Programming sequence error	on bank 1	PGSERR1	PGSERRIE1	Yes ⁽¹⁾	No
	on bank 2	PGSERR2	PGSERRIE2		
Strobe error	on bank 1	STRBERR1	STRBERRIE1	No	No
	on bank 2	STRBERR2	STRBERRIE2		
Inconsistency error	on bank 1	INCERR1	INCERRIE1	Yes ⁽¹⁾	No
	on bank 2	INCERR2	INCERRIE2		
Operation error	on bank 1	OPERR1	OPERRIE1	No	No
	on bank 2	OPERR2	OPERRIE2		
ECC single error correction event	on bank 1	SNECCERR1	SNECCERRIE1	No	No
	on bank 2	SNECCERR2	SNECCERRIE2		
ECC double error detection event	on bank 1	DBECCERR1	DBECCERRIE1	No	Yes
	on bank 2	DBECCERR2	DBECCERRIE2		
Read protection error	on bank 1	RDPERR1	RDPERRIE1	No	Yes ⁽²⁾
	on bank 2	RDPERR2	RDPERRIE2		
Read secure error	on bank 1	RDSERR1	RDSERRIE1	No	No (data) Yes (fetch)
	on bank 2	RDSERR2	RDSERRIE2		
CRC read error	on bank 1	CRCRDERR1	CRCRDERRIE1	No	No
	on bank 2	CRCRDERR2	CRCRDERRIE2		
Option Bytes operation error	all banks	OPTCHANGEERR	OPTCHANGEERRIE	No	No

1. Programming still possible on the AXI interface that did not trigger the inconsistency error, on the flash bank that does not have the flag bit raised. See [Section 4.7.5: Inconsistency error \(INCERR\)](#) for details.
2. Bus error occurs only when accessing RDP protected area, as defined in [Section 4.5.3: Readout protection \(RDP\)](#)

The status of the individual maskable interrupt sources described in [Table 26](#) (except for option byte error) can be read from the FLASH_SR1/2 register. They can be cleared by setting to 1 the adequate bit in FLASH_CCR1/2 register.

Note: No unlocking mechanism is required to clear an interrupt.

End-of-program event

Setting the end-of-operation interrupt enable bit (EOPIE1/2) in the FLASH_CR1/2 register enables the generation of an interrupt at the end of an erase operation, a program operation or an option byte change on bank 1/2. The EOP1/2 bit in the FLASH_SR1/2 register is also set when one of these events occurs.

Setting CLR_EOP1/2 bit to 1 in FLASH_CCR1/2 register clears EOP1/2 flag.

CRC end of calculation event

Setting the CRC end-of-calculation interrupt enable bit (CRCENDIE1/2) in the FLASH_CR1/2 register enables the generation of an interrupt at the end of a CRC operation on bank 1/2. The CRCEND1/2 bit in the FLASH_SR1/2 register is also set when this event occurs.

Setting CLR_CRCEND1/2 bit to 1 in FLASH_CCR1/2 register clears CRCEND1/2 flag.

4.9 FLASH registers

4.9.1 FLASH access control register (FLASH_ACR)

Address offset: 0x000 or 0x100

Reset value: 0x0000 0037

For more details, refer to [Section 4.3.8: FLASH read operations](#) and [Section 4.3.9: FLASH program operations](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRHIGHFREQ		LATENCY			
										rw	rw	rw	rw	rw	rw

Bits 31:6 Reserved, must be kept at reset value.

Bits 5:4 **WRHIGHFREQ**: Flash signal delay

These bits are used to control the delay between non-volatile memory signals during programming operations. The application software has to program them to the correct value depending on the embedded flash memory interface frequency. Please refer to [Table 16](#) for details.

Note: No check is performed by hardware to verify that the configuration is correct.

Bits 3:0 **LATENCY**: Read latency

These bits are used to control the number of wait states used during read operations on both non-volatile memory banks. The application software has to program them to the correct value depending on the embedded flash memory interface frequency and voltage conditions.

0000: zero wait state used to read a word from non-volatile memory

0001: one wait state used to read a word from non-volatile memory

0010: two wait states used to read a word from non-volatile memory

...

0111: seven wait states used to read a word from non-volatile memory

Note: No check is performed by hardware to verify that the configuration is correct.

4.9.2 FLASH key register for bank 1 (FLASH_KEYR1)

Address offset: 0x004

Reset value: 0x0000 0000

FLASH_KEYR1 is a write-only register. The following values must be programmed consecutively to unlock FLASH_CR1 register:

- 1st key = 0x4567 0123
- 2nd key = 0xCDEF 89AB

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY1R															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY1R															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **KEY1R**: Non-volatile memory bank 1 configuration access unlock key

4.9.3 FLASH option key register (FLASH_OPTKEYR)

Address offset: 0x008 or 0x108

Reset value: 0x0000 0000

FLASH_OPTKEYR is a write-only register. The following values must be programmed consecutively to unlock FLASH_OPTCR register:

- 1st key = 0x0819 2A3B
- 2nd key = 0x4C5D 6E7F

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OPTKEYR															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPTKEYR															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **OPTKEYR**: FLASH option bytes control access unlock key

4.9.4 FLASH control register for bank 1 (FLASH_CR1)

Address offset: 0x00C

Reset value: 0x0000 0031

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CRCRDERRIE1	CRCENDIE1	DBECCERRIE1	SNECCERRIE1	RDSERRIE1	RDPERRIE1	OPERRIE1	INCERRIE1	Res.	STRBERRIE1	PGSERRIE1	WRPERRIE1	EOPIE1
			r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w		r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_EN	Res.	Res.	Res.	Res.	SNB1			START1	FW1	PSIZE1		BER1	SER1	PG1	LOCK1
r/w					r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	rs

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **CRCDERRIE1**: Bank 1 CRC read error interrupt enable bit

When CRCDERRIE1 bit is set to 1, an interrupt is generated when a protected area (PCROP or secure-only) has been detected during the last CRC computation on bank 1. CRCDERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when a CRC read error occurs on bank 1
1: interrupt generated when a CRC read error occurs on bank 1

Bit 27 **CRCENDIE1**: Bank 1 CRC end of calculation interrupt enable bit

When CRCENDIE1 bit is set to 1, an interrupt is generated when the CRC computation has completed on bank 1. CRCENDIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when CRC computation complete on bank 1
1: interrupt generated when CRC computation complete on bank 1

Bit 26 **DBECCERRIE1**: Bank 1 ECC double detection error interrupt enable bit

When DBECCERRIE1 bit is set to 1, an interrupt is generated when an ECC double detection error occurs during a read operation from bank 1. DBECCERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when an ECC double detection error occurs on bank 1
1: interrupt generated if an ECC double detection error occurs on bank 1

Bit 25 **SNECCERRIE1**: Bank 1 ECC single correction error interrupt enable bit

When SNECCERRIE1 bit is set to 1, an interrupt is generated when an ECC single correction error occurs during a read operation from bank 1. SNECCERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when an ECC single correction error occurs on bank 1
1: interrupt generated when an ECC single correction error occurs on bank 1

Bit 24 **RDSERRIE1**: Bank 1 secure error interrupt enable bit

When RDSERRIE1 bit is set to 1, an interrupt is generated when a secure error (access to a secure-only protected address) occurs during a read operation from bank 1. RDSERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when a secure error occurs on bank 1
1: an interrupt is generated when a secure error occurs on bank 1

Bit 23 **RDPERRIE1**: Bank 1 read protection error interrupt enable bit

When RDPERRIE1 bit is set to 1, an interrupt is generated when a read protection error occurs (access to an address protected by PCROP or by RDP level 1) during a read operation from bank 1. RDPERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when a read protection error occurs on bank 1
1: an interrupt is generated when a read protection error occurs on bank 1

Bit 22 **OPERRIE1**: Bank 1 write/erase error interrupt enable bit

When OPERRIE1 bit is set to 1, an interrupt is generated when an error is detected during a write/erase operation to bank 1. OPERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when a write/erase error occurs on bank 1
1: interrupt generated when a write/erase error occurs on bank 1

Bit 21 **INCERRIE1**: Bank 1 inconsistency error interrupt enable bit

When INCERRIE1 bit is set to 1, an interrupt is generated when an inconsistency error occurs during a write operation to bank 1. INCERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when a inconsistency error occurs on bank 1
1: interrupt generated when a inconsistency error occurs on bank 1.

Bit 20 Reserved, must be kept at reset value.

Bit 19 **STRBERRIE1**: Bank 1 strobe error interrupt enable bit

When STRBERRIE1 bit is set to 1, an interrupt is generated when a strobe error occurs (the master programs several times the same byte in the write buffer) during a write operation to bank 1. STRBERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when a strobe error occurs on bank 1

1: interrupt generated when strobe error occurs on bank 1.

Bit 18 **PGSERRIE1**: Bank 1 programming sequence error interrupt enable bit

When PGSERRIE1 bit is set to 1, an interrupt is generated when a sequence error occurs during a program operation to bank 1. PGSERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when a sequence error occurs on bank 1

1: interrupt generated when sequence error occurs on bank 1.

Bit 17 **WRPERRIE1**: Bank 1 write protection error interrupt enable bit

When WRPERRIE1 bit is set to 1, an interrupt is generated when a protection error occurs during a program operation to bank 1. WRPERRIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated when a protection error occurs on bank 1

1: interrupt generated when a protection error occurs on bank 1.

Bit 16 **EOPIE1**: Bank 1 end-of-program interrupt control bit

Setting EOPIE1 bit to 1 enables the generation of an interrupt at the end of a program operation to bank 1. EOPIE1 can be programmed only when LOCK1 is cleared to 0.

0: no interrupt generated at the end of a program operation to bank 1.

1: interrupt enabled when at the end of a program operation to bank 1.

Bit 15 **CRC_EN**: Bank 1 CRC control bit

Setting CRC_EN bit to 1 enables the CRC calculation on bank 1. CRC_EN does not start CRC calculation but enables CRC configuration through FLASH_CRCCR1 register.

When CRC calculation is performed on bank 1, it can only be disabled by setting CRC_EN bit to 0. Resetting CRC_EN clears CRC configuration and resets the content of FLASH_CRCDATAR register.

Clearing CRC_EN to 0 sets CRCDATA to 0x0.

CRC_EN can be programmed only when LOCK1 is cleared to 0.

Bit 14 Reserved, must be kept at reset value.

Bits 13:11 Reserved, must be kept at reset value.

Bits 10:8 **SNB1**: Bank 1 sector erase selection number

These bits are used to select the target sector for a sector erase operation (they are unused otherwise). SNB1 can be programmed only when LOCK1 is cleared to 0.

000: sector 0 of bank 1

001: sector 1 of bank 1

...

111: sector 7 of bank 1

Bit 7 **START1**: Bank 1 erase start control bit

START1 bit is used to start a sector erase or a bank erase operation. START1 can be programmed only when LOCK1 is cleared to 0.

The embedded flash memory resets START1 when the corresponding operation has been acknowledged. The user application cannot access any embedded flash memory register until the operation is acknowledged.

Bit 6 **FW1**: Bank 1 write forcing control bit

FW1 forces a write operation even if the write buffer is not full. In this case all bits not written are set to 1 by hardware. FW1 can be programmed only when LOCK1 is cleared to 0.

The embedded flash memory resets FW1 when the corresponding operation has been acknowledged. The user application cannot access any embedded flash memory register until the operation is acknowledged.

Note: Using a force-write operation prevents the application from updating later the missing bits with something else than 1, because it is likely that it will lead to permanent ECC error.

Write forcing is effective only if the write buffer is not empty (in particular, FW1 does not start several write operations when the force-write operations are performed consecutively).

Bits 5:4 **PSIZE1**: Bank 1 program size

PSIZE1 selects the parallelism used by the non-volatile memory during write and erase operations to bank 1. PSIZE1 can be programmed only when LOCK1 is cleared to 0.

00: programming executed with byte parallelism

01: programming executed with half-word parallelism

10: programming executed with word parallelism

11: programming executed with double word parallelism

Bit 3 **BER1**: Bank 1 erase request

Setting BER1 bit to 1 requests a bank erase operation on bank 1 (user flash memory only).

BER1 can be programmed only when LOCK1 is cleared to 0.

BER1 has a higher priority than SER1: if both are set, the embedded flash memory executes a bank erase.

0: bank erase not requested on bank 1

1: bank erase requested on bank 1

Note: Write protection error is triggered when a bank erase is required and some sectors are protected.

Bit 2 SER1: Bank 1 sector erase request

Setting SER1 bit to 1 requests a sector erase on bank 1. SER1 can be programmed only when LOCK1 is cleared to 0.

BER1 has a higher priority than SER1: if both bits are set, the embedded flash memory executes a bank erase.

0: sector erase not requested on bank 1

1: sector erase requested on bank 1

Note: Write protection error is triggered when a sector erase is required on a protected sector.

Bit 1 PG1: Bank 1 internal buffer control bit

Setting PG1 bit to 1 enables internal buffer for write operations to bank 1. This allows preparing program operations even if a sector or bank erase is ongoing.

PG1 can be programmed only when LOCK1 is cleared to 0. When PG1 is reset, the internal buffer is disabled for write operations to bank 1, and all the data stored in the buffer but not sent to the operation queue are lost.

0: Internal buffer disabled for write operations to bank 1

1: Internal buffer enabled for write operations to bank 1

Bit 0 LOCK1: Bank 1 configuration lock bit

This bit locks the FLASH_CR1 register. The correct write sequence to FLASH_KEYR1 register unlocks this bit. If a wrong sequence is executed, or if the unlock sequence to FLASH_KEYR1 is performed twice, this bit remains locked until the next system reset.

LOCK1 can be set by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When LOCK1 changes from 0 to 1, the other bits of FLASH_CR1 register do not change.

0: FLASH_CR1 register unlocked

1: FLASH_CR1 register locked

4.9.5 FLASH status register for bank 1 (FLASH_SR1)

Address offset: 0x010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CRCRDERR1	CRCEND1	DBECCERR1	SNECCERR1	RDSERR1	RDPEERR1	OPERR1	INCERR1	Res.	STRBERR1	PGSERR1	WRPERR1	EOP1
			r	r	r	r	r	r	r	r		r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_BUSY1	QW1	WBNE1	BSY1
												r	r	r	r

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 CRCRDERR1: Bank 1 CRC read error flag

CRCRDERR1 flag is raised when a word is found read protected during a CRC operation on bank 1. An interrupt is generated if CRCRDIE1 and CRCEND1 are set to 1. Writing 1 to CLR_CRCRDERR1 bit in FLASH_CCR1 register clears CRCRDERR1.

0: no protected area detected inside address read by CRC on bank 1

1: a protected area has been detected inside address read by CRC on bank 1. CRC result is very likely incorrect.

Note: This flag is valid only when CRCEND1 bit is set to 1

Bit 27 CRCEND1: Bank 1 CRC end of calculation flag

CRCEND1 bit is raised when the CRC computation has completed on bank 1. An interrupt is generated if CRCENDIE1 is set to 1. It is not necessary to reset CRCEND1 before restarting CRC computation. Writing 1 to CLR_CRCEND1 bit in FLASH_CCR1 register clears CRCEND1.

0: CRC computation not complete on bank 1

1: CRC computation complete on bank 1

Bit 26 DBECCERR1: Bank 1 ECC double detection error flag

DBECCERR1 flag is raised when an ECC double detection error occurs during a read operation from bank 1. An interrupt is generated if DBECCERRIE1 is set to 1. Writing 1 to CLR_DBECCERR1 bit in FLASH_CCR1 register clears DBECCERR1.

0: no ECC double detection error occurred on bank 1

1: ECC double detection error occurred on bank 1

Bit 25 SNECCERR1: Bank 1 single correction error flag

SNECCERR1 flag is raised when an ECC single correction error occurs during a read operation from bank 1. An interrupt is generated if SNECCERRIE1 is set to 1. Writing 1 to CLR_SNECCERR1 bit in FLASH_CCR1 register clears SNECCERR1.

0: no ECC single correction error occurs on bank 1

1: ECC single correction error occurs on bank 1

- Bit 24 **RDSERR1**: Bank 1 secure error flag
RDSERR1 flag is raised when a read secure error (read access to a secure-only protected word) occurs on bank 1. An interrupt is generated if RDSERRIE1 is set to 1. Writing 1 to CLR_RDSERR1 bit in FLASH_CCR1 register clears RDSERR1.
0: no secure error occurs on bank 1
1: a secure error occurs on bank 1
- Bit 23 **RDPER1**: Bank 1 read protection error flag
RDPER1 flag is raised when an read protection error (read access to a PCROP-protected or a RDP-protected area) occurs on bank 1. An interrupt is generated if RDPERRIE1 is set to 1. Writing 1 to CLR_RDPER1 bit in FLASH_CCR1 register clears RDPER1.
0: no read protection error occurs on bank 1
1: a read protection error occurs on bank 1
- Bit 22 **OPERR1**: Bank 1 write/erase error flag
OPERR1 flag is raised when an error occurs during a write/erase to bank 1. An interrupt is generated if OPERRIE1 is set to 1. Writing 1 to CLR_OPERR1 bit in FLASH_CCR1 register clears OPERR1.
0: no write/erase error occurs on bank 1
1: a write/erase error occurs on bank 1
- Bit 21 **INCERR1**: Bank 1 inconsistency error flag
INCERR1 flag is raised when a inconsistency error occurs on bank 1. An interrupt is generated if INCERRIE1 is set to 1. Writing 1 to CLR_INCERR1 bit in the FLASH_CCR1 register clears INCERR1.
0: no inconsistency error occurs on bank 1
1: a inconsistency error occurs on bank 1
- Bit 20 Reserved, must be kept at reset value.
- Bit 19 **STRBERR1**: Bank 1 strobe error flag
STRBERR1 flag is raised when a strobe error occurs on bank 1 (when the master attempts to write several times the same byte in the write buffer). An interrupt is generated if the STRBERRIE1 bit is set to 1. Writing 1 to CLR_STRBERR1 bit in FLASH_CCR1 register clears STRBERR1.
0: no strobe error occurs on bank 1
1: a strobe error occurs on bank 1
- Bit 18 **PGSERR1**: Bank 1 programming sequence error flag
PGSERR1 flag is raised when a sequence error occurs on bank 1. An interrupt is generated if the PGSERRIE1 bit is set to 1. Writing 1 to CLR_PGSERR1 bit in FLASH_CCR1 register clears PGSERR1.
0: no sequence error occurs on bank 1
1: a sequence error occurs on bank 1
- Bit 17 **WRPERR1**: Bank 1 write protection error flag
WRPERR1 flag is raised when a protection error occurs during a program operation to bank 1. An interrupt is also generated if the WRPERRIE1 is set to 1. Writing 1 to CLR_WRPERR1 bit in FLASH_CCR1 register clears WRPERR1.
0: no write protection error occurs on bank 1
1: a write protection error occurs on bank 1

Bit 16 EOP1: Bank 1 end-of-program flag

EOP1 flag is set when a programming operation to bank 1 completes. An interrupt is generated if the EOPIE1 is set to 1. It is not necessary to reset EOP1 before starting a new operation. EOP1 bit is cleared by writing 1 to CLR_EOP1 bit in FLASH_CCR1 register.

0: no programming operation completed on bank 1

1: a programming operation completed on bank 1

Bits 15:4 Reserved, must be kept at reset value.

Bit 3 CRC_BUSY1: Bank 1 CRC busy flag

CRC_BUSY1 flag is set when a CRC calculation is ongoing on bank 1. This bit cannot be forced to 0. The user must wait until the CRC calculation has completed or disable CRC computation on bank 1.

0: no CRC calculation ongoing on bank 1

1: CRC calculation ongoing on bank 1

Bit 2 QW1: Bank 1 wait queue flag

QW1 flag is set when a write, erase or option byte change operation is pending in the command queue buffer of bank 1. It is not possible to know what type of programming operation is present in the queue.

This flag is reset by hardware when all write, erase or option byte change operations have been executed and thus removed from the waiting queue(s). This bit cannot be forced to 0. It is reset after a deterministic time if no other operations are requested.

0: no write, erase or option byte change operations waiting in the operation queues of bank 1

1: at least one write, erase or option byte change operation is waiting in the operation queue of bank 1

Bit 1 WBNE1: Bank 1 write buffer not empty flag

WBNE1 flag is set when the embedded flash memory is waiting for new data to complete the write buffer. In this state, the write buffer is not empty. WBNE1 is reset by hardware each time the write buffer is complete or the write buffer is emptied following one of the event below:

- the application software forces the write operation using FW1 bit in FLASH_CR1
- the embedded flash memory detects an error that involves data loss
- the application software has disabled write operations in this bank

This bit cannot be forced to 0. To reset it, clear the write buffer by performing any of the above listed actions, or send the missing data.

0: write buffer of bank 1 empty or full

1: write buffer of bank 1 waiting data to complete

Bit 0 BSY1: Bank 1 busy flag

BSY1 flag is set when an effective write, erase or option byte change operation is ongoing on bank 1. It is not possible to know what type of operation is being executed.

BSY1 cannot be forced to 0. It is automatically reset by hardware every time a step in a write, erase or option byte change operation completes.

0: no programming, erase or option byte change operation being executed on bank 1

1: programming, erase or option byte change operation being executed on bank 1

4.9.6 FLASH clear control register for bank 1 (FLASH_CCR1)

Address offset: 0x014

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CLR_CRCDERR1	CLR_CRCEND1	CLR_DBECCERR1	CLR_SNECCERR1	CLR_RDSERR1	CLR_RDPERR1	CLR_OPERR1	CLR_INCERR1	Res.	CLR_STRBERR1	CLR_PGSERR1	CLR_WRPERR1	CLR_EOP1
			w	w	w	w	w	w	w	w		w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **CLR_CRCDERR1**: Bank 1 CRCDERR1 flag clear bit

Setting this bit to 1 resets to 0 CRCDERR1 flag in FLASH_SR1 register.

Bit 27 **CLR_CRCEND1**: Bank 1 CRCEND1 flag clear bit

Setting this bit to 1 resets to 0 CRCEND1 flag in FLASH_SR1 register.

Bit 26 **CLR_DBECCERR1**: Bank 1 DBECCERR1 flag clear bit

Setting this bit to 1 resets to 0 DBECCERR1 flag in FLASH_SR1 register. If the SNECCERR1 flag of FLASH_SR1 register is cleared to 0, FLASH_ECC_FA1R register is reset to 0 as well.

Bit 25 **CLR_SNECCERR1**: Bank 1 SNECCERR1 flag clear bit

Setting this bit to 1 resets to 0 SNECCERR1 flag in FLASH_SR1 register. If the DBECCERR1 flag of FLASH_SR1 register is cleared to 0, FLASH_ECC_FA1R register is reset to 0 as well.

Bit 24 **CLR_RDSERR1**: Bank 1 RDSERR1 flag clear bit

Setting this bit to 1 resets to 0 RDSERR1 flag in FLASH_SR1 register.

Bit 23 **CLR_RDPERR1**: Bank 1 RDPERR1 flag clear bit

Setting this bit to 1 resets to 0 RDPERR1 flag in FLASH_SR1 register.

Bit 22 **CLR_OPERR1**: Bank 1 OPERR1 flag clear bit

Setting this bit to 1 resets to 0 OPERR1 flag in FLASH_SR1 register.

Bit 21 **CLR_INCERR1**: Bank 1 INCERR1 flag clear bit

Setting this bit to 1 resets to 0 INCERR1 flag in FLASH_SR1 register.

Bit 20 Reserved, must be kept at reset value.

Bit 19 **CLR_STRBERR1**: Bank 1 STRBERR1 flag clear bit

Setting this bit to 1 resets to 0 STRBERR1 flag in FLASH_SR1 register.

Bit 18 **CLR_PGSERR1**: Bank 1 PGSERR1 flag clear bit

Setting this bit to 1 resets to 0 PGSERR1 flag in FLASH_SR1 register.

Bit 17 **CLR_WRPERR1**: Bank 1 WRPERR1 flag clear bit

Setting this bit to 1 resets to 0 WRPERR1 flag in FLASH_SR1 register.

Bit 16 **CLR_EOP1**: Bank 1 EOP1 flag clear bit

Setting this bit to 1 resets to 0 EOP1 flag in FLASH_SR1 register.

Bits 15:0 Reserved, must be kept at reset value.

4.9.7 FLASH option control register (FLASH_OPTCR)

Address offset: 0x018 or 0x118

Reset value: 0xX000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWAP_BANK	OPTCHANGEERRIE	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
r	rw														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MER	Res.	Res.	OPTSTART	OPTLOCK
											w			w	rs

Bit 31 **SWAP_BANK**: Bank swapping option configuration bit

SWAP_BANK controls whether bank 1 and bank 2 are swapped or not. This bit is loaded with the SWAP_BANK_OPT bit of FLASH_OPTSR_CUR register only after reset or POR.

0: bank 1 and bank 2 not swapped

1: bank 1 and bank 2 swapped

Bit 30 **OPTCHANGEERRIE**: Option byte change error interrupt enable bit

OPTCHANGEERRIE bit controls if an interrupt has to be generated when an error occurs during an option byte change.

0: no interrupt is generated when an error occurs during an option byte change

1: an interrupt is generated when an error occurs during an option byte change.

Bits 29:5 Reserved, must be kept at reset value.

Bit 4 **MER**: mass erase request

Setting this bit launches a non-volatile memory bank erase on both banks (i.e. mass erase).

Programming MER bit to 1 automatically sets BER1, BER2, START1 and START2 to 1. FLASH_OPTCR, FLASH_CR1 and FLASH_CR2 must be unlocked prior to setting MER high.

Bits 3:2 Reserved, must be kept at reset value.

Bit 1 **OPTSTART**: Option byte start change option configuration bit

OPTSTART triggers an option byte change operation. The user can set OPTSTART only when the OPTLOCK bit is cleared to 0. The embedded flash memory resets OPTSTART when the option byte change operation has been acknowledged.

The user application cannot modify any embedded flash memory register until the option change operation has been completed.

Before setting this bit, the user has to write the required values in the FLASH_XXX_PRG registers. The FLASH_XXX_PRG registers will be locked until the option byte change operation has been executed in non-volatile memory.

It is not possible to start an option byte change operation if a CRC calculation is ongoing on bank 1 or bank 2. Trying to set OPTSTART when CRC_BUSY1/2 of FLASH_SR1/2 register is set has no effect; the option byte change does not start and no error is generated.

Bit 0 **OPTLOCK**: FLASH_OPTCR lock option configuration bit

The OPTLOCK bit locks the FLASH_OPTCR register as well as all _PRG registers. The correct write sequence to FLASH_OPTKEYR register unlocks this bit. If a wrong sequence is executed, or the unlock sequence to FLASH_OPTKEYR is performed twice, this bit remains locked until next system reset.

It is possible to set OPTLOCK by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When OPTLOCK changes from 0 to 1, the others bits of FLASH_OPTCR register do not change.

0: FLASH_OPTCR register unlocked

1: FLASH_OPTCR register locked.

4.9.8 FLASH option status register (FLASH_OPTSR_CUR)

Address offset: 0x01C or 0x11C

Reset value: 0xFFFF XXXX (see [Table 20: Option byte organization](#))

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWAP_BANK_OPT	OPTCHANGEERR	IO_HSLV	Res.	Res.	Res.	NRST_STBY_D2	NRST_STOP_D2	BOOT_CM7	BOOT_CM4	SECURITY	ST_RAM_SIZE[1:0]		IWDG_FZ_SDBY	IWDG_FZ_STOP	Res.
r	r	r				r	r	r	r	r	r	r	r	r	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDP								NRST_STDY_D1	NRST_STOP_D1	IWDG2_SW	IWDG1_SW	BOR_LEV		Res.	OPT_BUSY
r	r	r	r	r	r	r	r	r	r	r	r	r	r		r

- Bit 31 **SWAP_BANK_OPT**: Bank swapping option status bit
 SWAP_BANK_OPT reflects whether bank 1 and bank 2 are swapped or not.
 SWAP_BANK_OPT is loaded to SWAP_BANK of FLASH_OPTCR after a reset.
 0: bank 1 and bank 2 not swapped
 1: bank 1 and bank 2 swapped
- Bit 30 **OPTCHANGEERR**: Option byte change error flag
 OPTCHANGEERR flag indicates that an error occurred during an option byte change operation. When OPTCHANGEERR is set to 1, the option byte change operation did not successfully complete. An interrupt is generated when this flag is raised if the OPTCHANGEERRIE bit of FLASH_OPTCR register is set to 1.
 Writing 1 to CLR_OPTCHANGEERR of register FLASH_OPTCCR clears OPTCHANGEERR.
 0: no option byte change errors occurred
 1: one or more errors occurred during an option byte change operation.
- Bit 29 **IO_HSLV**: I/O high-speed at low-voltage status bit
 This bit indicates that the product operates below 2.7 V.
 0: Product working in the full voltage range, I/O speed optimization at low-voltage disabled
 1: Product operating below 2.7 V, I/O speed optimization at low-voltage feature allowed
- Bits 28:26 Reserved, must be kept at reset value.
- Bit 25 **NRST_STBY_D2**: D2 domain DStandby entry reset option status bit
 0: a reset is generated when entering DStandby mode on D2 domain
 1: no reset generated.
- Bit 24 **NRST_STOP_D2**: D2 domain DStop entry reset option status bit
 0: a reset is generated when entering DStop mode on D2 domain
 1: no reset generated
- Bit 23 **BOOT_CM7**: Arm® Cortex®-M7 boot option status bit
 0: Arm® Cortex®-M7 boot disabled (it is clock gated)
 1: Arm® Cortex®-M7 boot enabled.
- Bit 22 **BOOT_CM4**: Arm® Cortex®-M4 boot option status bit
 0: Arm® Cortex®-M4 boot disabled (it is clock gated)
 1: Arm® Cortex®-M4 boot enabled.
- Bit 21 **SECURITY**: Security enable option status bit
 0: Security feature disabled
 1: Security feature enabled.
- Bits 20:19 **ST_RAM_SIZE[1:0]**: ST RAM size option status
 00: 2 Kbytes reserved to ST code
 01: 4 Kbytes reserved to ST code
 10: 8 Kbytes reserved to ST code
 11: 16 Kbytes reserved to ST code
Note: This bitfield is effective only when the security is enabled (SECURITY = 1).

- Bit 18 **IWDG_FZ_SDBY**: IWDG Standby mode freeze option status bit
 When set the independent watchdogs IWDG and IWDG2 are frozen in system Standby mode.
 0: Independent watchdogs frozen in Standby mode
 1: Independent watchdogs keep running in Standby mode.
- Bit 17 **IWDG_FZ_STOP**: IWDG Stop mode freeze option status bit
 When set the independent watchdogs IWDG and IWDG2 are in system Stop mode.
 0: Independent watchdogs frozen in system Stop mode
 1: Independent watchdogs keep running in system Stop mode.
- Bit 16 Reserved, must be kept at reset value.
- Bits 15:8 **RDP**: Readout protection level option status byte
 0xAA: global readout protection level 0
 0xCC: global readout protection level 2
 others values: global readout protection level 1.
- Bit 7 **NRST_STDY_D1**: D1 domain DStandby entry reset option status bit
 0: a reset is generated when entering DStandby mode on D1 domain
 1: no reset generated when entering DStandby mode on D1 domain
- Bit 6 **NRST_STOP_D1**: D1 domain DStop entry reset option status bit
 0: a reset is generated when entering DStop mode on D1 domain
 1: no reset generated when entering DStop mode on D1 domain.
- Bit 5 **IWDG2_SW**: IWDG2 control mode option status bit
 0: IWDG2 watchdog is controlled by hardware
 1: IWDG2 watchdog is controlled by software
- Bit 4 **IWDG_SW**: IWDG control mode option status bit
 0: IWDG watchdog is controlled by hardware
 1: IWDG watchdog is controlled by software
- Bits 3:2 **BOR_LEV**: Brownout level option status bit
 These bits reflects the power level that generates a system reset. uuuuuuuuuuuu
 00: V_{BOR0} , brownout reset threshold 0
 01: V_{BOR1} , brownout reset threshold 1
 10: V_{BOR2} , brownout reset threshold 2
 11: V_{BOR3} , brownout reset threshold 3
Note: Refer to device datasheet for the values of V_{BORx} V_{DD} reset thresholds.
- Bit 1 Reserved, must be kept at reset value.
- Bit 0 **OPT_BUSY**: Option byte change ongoing flag
 OPT_BUSY indicates if an option byte change is ongoing. When this bit is set to 1, the embedded flash memory is performing an option change and it is not possible to modify any embedded flash memory register.
 0: no option byte change ongoing
 1: an option byte change ongoing and all write accesses to flash registers are blocked until the option byte change completes.

4.9.9 FLASH option status register (FLASH_OPTSR_PRG)

Address offset: 0x020 or 0x120

Reset value: 0xFFFF XXXX (see [Table 20: Option byte organization](#))

This register is used to program values in corresponding option bits. Values after reset reflects the current values of the corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWAP_BANK_OPT	Res.	IO_HSLV	Res.	Res.	Res.	NRST_STBY_D2	NRST_STOP_D2	BOOT_CM7	BOOT_CM4	SECURITY	ST_RAM_SIZE[1:0]		IWDG_FZ_SDBY	IWDG_FZ_STOP	Res.
rw		rw				rw	rw	rw	rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDP								NRST_STDY_D1	NRST_STOP_D1	IWDG2_SW	IWDG1_SW	BOR_LEV		Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 **SWAP_BANK_OPT**: Bank swapping option configuration bit

SWAP_BANK_OPT option bit is used to configure whether the bank 1 and bank 2 are swapped or not. This bit is loaded with the SWAP_BANK_OPT bit of FLASH_OPTSR_CUR register after a reset.

0: bank 1 and bank 2 not swapped

1: bank 1 and bank 2 swapped

Note:

Bit 30 Reserved, must be kept at reset value.

Bit 29 **IO_HSLV**: I/O high-speed at low-voltage configuration bit

This bit indicates that the product operates below 2.7 V.

0: Product working in the full voltage range, I/O speed optimization at low-voltage disabled

1: Product operating below 2.7 V, I/O speed optimization at low-voltage feature allowed

Bits 28:26 Reserved, must be kept at reset value.

Bit 25 **NRST_STBY_D2**: D2 domain DStandby entry reset option configuration bit

0: a reset is generated when entering DStandby mode on D2 domain

1: no reset generated.

Bit 24 **NRST_STOP_D2**: D2 domain DStop entry reset option configuration bit

0: a reset is generated when entering DStop mode on D2 domain

1: no reset generated

Bit 23 **BOOT_CM7**: Arm® Cortex®-M7 boot option configuration bit

0: Arm® Cortex®-M7 boot is disabled (it is clock gated)

1: Arm® Cortex®-M7 boot is enabled.

Bit 22 **BOOT_CM4**: Arm® Cortex®-M4 boot option configuration bit

0: Arm® Cortex®-M4 boot is disabled (it is clock gated)

1: Arm® Cortex®-M4 boot is enabled.

Bit 21 **SECURITY**: Security enable option configuration bit

The SECURITY option bit enables the secure access mode of the device during an option byte change. The change will be taken into account at next reset (next boot sequence). Once it is enabled, the security feature can be disabled if no areas are protected by PCROP or Secure access mode. If there are secure-only or PCROP protected areas, perform a level regression (from level 1 to 0) and set all the bits to unprotect secure-only areas and PCROP areas.

0: Security feature disabled

1: Security feature enabled.

Bits 20:19 **ST_RAM_SIZE[1:0]**: ST RAM size option configuration bits

00: 2 Kbytes reserved to ST code

01: 4 Kbytes reserved to ST code

10: 8 Kbytes reserved to ST code

11: 16 Kbytes reserved to ST code

Note: This bitfield is effective only when the security is enabled (SECURITY = 1).

Bit 18 **IWDG_FZ_SDBY**: IWDG Standby mode freeze option configuration bit

This option bit is used to freeze or not the independent watchdogs IWDG and IWDG2 in system Standby mode.

0: Independent watchdogs frozen in Standby mode

1: Independent watchdogs keep running in Standby mode.

Bit 17 **IWDG_FZ_STOP**: IWDG Stop mode freeze option configuration bit

This option bit is used to freeze or not the independent watchdogs IWDG and IWDG2 in system Stop mode.

0: Independent watchdogs frozen in system Stop mode

1: Independent watchdogs keep running in system Stop mode.

Bit 16 Reserved, must be kept at reset value.

Bits 15:8 **RDP**: Readout protection level option configuration bits

RDP bits are used to change the readout protection level. This change is possible only when the current protection level is different from level 2. The possible configurations are:

0xAA: global readout protection level 0

0xCC: global readout protection level 2

others values: global readout protection level 1.

Bit 7 **NRST_STDY_D1**: D1 domain DStandby entry reset option configuration bit

0: a reset is generated when entering DStandby mode on D1 domain.

1: no reset generated when entering DStandby mode on D1 domain

Bit 6 **NRST_STOP_D1**: D1 domain DStop entry reset option configuration bit

0: a reset is generated when entering DStop mode on D1 domain.

1: no reset generated when entering DStop mode on D1 domain.

Bit 5 **IWDG2_SW**: IWDG2 control mode option configuration bit

IWDG2_SW option bit is used to select if IWDG2 independent watchdog is controlled by hardware or by software.

0: IWDG2 watchdog is controlled by hardware

1: IWDG2 watchdog is controlled by software

Bit 4 **IWDG_SW**: IWDG control mode option configuration bit

IWDG_SW option bit is used to select if IWDG independent watchdog is controlled by hardware or by software.

0: IWDG watchdog is controlled by hardware

1: IWDG watchdog is controlled by software

Bits 3:2 **BOR_LEV**: Brownout level option configuration bit

These option bits are used to define the power level that generates a system reset.

00: V_{BOR0} , brownout reset threshold 0

01: V_{BOR1} , brownout reset threshold 1

02: V_{BOR2} , brownout reset threshold 2

03: V_{BOR3} , brownout reset threshold 3

Note: Refer to device datasheet for the values of V_{BORx} V_{DD} reset thresholds.

Bits 1:0 Reserved, must be kept at reset value.

4.9.10 FLASH option clear control register (FLASH_OPTCCR)

Address offset: 0x024 or 0x124

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	CLR_OPTCHANGEERR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	w														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bit 31 Reserved, must be kept at reset value.

Bit 30 **CLR_OPTCHANGEERR**: OPTCHANGEERR reset bit

This bit is used to reset the OPTCHANGEERR flag in FLASH_OPTSR_CUR register.

FLASH_OPTCCR is write-only.

It is reset by programming it to 1.

Bits 29:0 Reserved, must be kept at reset value.

4.9.11 FLASH protection address for bank 1 (FLASH_PRAR_CUR1)

Address offset: 0x028

Reset value: 0xFFFF 0XXX (see [Table 20: Option byte organization](#))

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMEP1	Res.	Res.	Res.	PROT_AREA_END1											
r				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PROT_AREA_START1											
				r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **DMEP1**: Bank 1 PCROP protected erase enable option status bit

If DMEP1 is set to 1, the PCROP protected area in bank 1 is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **PROT_AREA_END1**: Bank 1 PCROP area end status bits

These bits contain the last 256-byte block of the PCROP area in bank 1.

If this address is equal to PROT_AREA_START1, the whole bank 1 is PCROP protected.

If this address is lower than PROT_AREA_START1, no protection is set on bank 1.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **PROT_AREA_START1**: Bank 1 PCROP area start status bits

These bits contain the first 256-byte block of the PCROP area in bank 1.

If this address is equal to PROT_AREA_END1, the whole bank 1 is PCROP protected.

If this address is higher than PROT_AREA_END1, no protection is set on bank 1.

4.9.12 FLASH protection address for bank 1 (FLASH_PRAR_PRG1)

Address offset: 0x02C

Reset value: 0xFFFF 0XXX (see [Table 20: Option byte organization](#))

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMEP1	Res.	Res.	Res.	PROT_AREA_END1											
rW				rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PROT_AREA_START1											
				rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bit 31 **DMEP1**: Bank 1 PCROP protected erase enable option configuration bit

If DMEP1 is set to 1, the PCROP protected area in bank 1 is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.
Only the Arm® Cortex®-M7 master can program this bit.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **PROT_AREA_END1**: Bank 1 PCROP area end configuration bits

These bits contain the last 256-byte block of the PCROP area in bank 1.
If this address is equal to PROT_AREA_START1, the whole bank 1 is PCROP protected.
If this address is lower than PROT_AREA_START1, no protection is set on bank 1.
Only the Arm® Cortex®-M7 master can program these bits.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **PROT_AREA_START1**: Bank 1 PCROP area start configuration bits

These bits contain the first 256-byte block of the PCROP area in bank 1.
If this address is equal to PROT_AREA_END1, the whole bank 1 is PCROP protected.
If this address is higher than PROT_AREA_END1, no protection is set on bank 1.
Only the Arm® Cortex®-M7 master can program these bits.

4.9.13 FLASH secure address for bank 1 (FLASH_SCAR_CUR1)

Address offset: 0x030

Reset value: 0xFFFF 0XXX (see [Table 20: Option byte organization](#))

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMES1	Res.	Res.	Res.	SEC_AREA_END1											
r				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	SEC_AREA_START1											
				r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **DMES1**: Bank 1 secure access protected erase enable option status bit

If DMES1 is set to 1, the secure access only area in bank 1 is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **SEC_AREA_END1**: Bank 1 secure-only area end status bits

These bits contain the last 256-byte block of the secure-only area in bank 1.
If this address is equal to SEC_AREA_START1, the whole bank 1 is secure access only.
If this address is lower than SEC_AREA_START1, no protection is set on bank 1.

Note: The non-secure flash area starts at address 0x(SEC_AREA_END1 + 1)00.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **SEC_AREA_START1**: Bank 1 secure-only area start status bits

These bits contain the first 256 bytes of block of the secure-only area in bank 1.
If this address is equal to SEC_AREA_END1, the whole bank 1 is secure access only.
If this address is higher than SEC_AREA_END1, no protection is set on bank 1.

4.9.14 FLASH secure address for bank 1 (FLASH_SCAR_PRG1)

Address offset: 0x034

Reset value: 0xFFFF 0XXX (see [Table 20: Option byte organization](#))

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMES1	Res.	Res.	Res.	SEC_AREA_END1											
rw				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	SEC_AREA_START1											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **DMES1**: Bank 1 secure access protected erase enable option configuration bit

If DMES1 is set to 1, the secure access only area in bank 1 is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Only the Arm® Cortex®-M7 master can program this bit.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **SEC_AREA_END1**: Bank 1 secure-only area end configuration bits

These bits contain the last block of 256 bytes of the secure-only area in bank 1.

If this address is equal to SEC_AREA_START1, the whole bank 1 is secure access only.

If this address is lower than SEC_AREA_START1, no protection is set on bank 1.

Only the Arm® Cortex®-M7 master can program these bits.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **SEC_AREA_START1**: Bank 1 secure-only area start configuration bits

These bits contain the first block of 256 bytes of the secure-only area in bank 1.

If this address is equal to SEC_AREA_END1, the whole bank 1 is secure access only.

If this address is higher than SEC_AREA_END1, no protection is set on bank 1.

Only the Arm® Cortex®-M7 master can program these bits.

4.9.15 FLASH write sector protection for bank 1 (FLASH_WPSN_CUR1R)

Address offset: 0x038

Reset value: 0x0000 00XX

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn1							
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **WRPSn1**: Bank 1 sector write protection option status byte

Each FLASH_WPSGN_CUR1R bit reflects the write protection status of the corresponding bank 1 sector (0: sector is write protected; 1: sector is not write protected)

4.9.16 FLASH write sector protection for bank 1 (FLASH_WPSN_PRG1R)

Address offset: 0x03C

Reset value: 0x0000 00XX

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn1							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **WRPSn1**: Bank 1 sector write protection option status byte

Setting each FLASH_WPSGN_PRG1R bit to 0 write-protects the corresponding bank 1 sector (0: sector is write protected; 1: sector is not write protected)

4.9.17 FLASH register boot address (for Arm® Cortex®-M7 core (FLASH_BOOT7_CURR))

Address offset: 0x040 or 0x140

Reset value: 0xFFFF XXXX (see [Table 20: Option byte organization](#))

This register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BOOT_CM7_ADD1															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BOOT_CM7_ADD0															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 **BOOT_CM7_ADD1**: Arm® Cortex®-M7 boot address 1

These bits reflect the MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is high.

Bits 15:0 **BOOT_CM7_ADD0**: Arm® Cortex®-M7 boot address 0

These bits reflect the MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is low.

4.9.18 FLASH register boot address for Arm® Cortex®-M7 core (FLASH_BOOT7_PRGR)

Address offset: 0x044 or 0x144

Reset value: 0xFFFF XXXX (see [Table 20: Option byte organization](#))

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BOOT_CM7_ADD1															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BOOT_CM7_ADD0															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **BOOT_CM7_ADD1**: Arm® Cortex®-M7 boot address 1 configuration

These bits allow configuring the MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is high.

Bits 15:0 **BOOT_CM7_ADD0**: Arm® Cortex®-M7 boot address 0 configuration

These bits allow configuring the MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is low.

4.9.19 FLASH register boot address for Arm® Cortex®-M4 core (FLASH_BOOT4_CURR)

Address offset: 0x048 or 0x148

Reset value: 0xFFFF XXXX (see [Table 20: Option byte organization](#))

This register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BOOT_CM4_ADD1															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BOOT_CM4_ADD0															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 **BOOT_CM4_ADD1**: Arm® Cortex®-M4 boot address 1

These bits reflect the MSB of the Arm® Cortex®-M4 boot address when the BOOT pin is high.

Bits 15:0 **BOOT_CM4_ADD0**: Arm® Cortex®-M4 boot address 0

These bits reflect the MSB of the Arm® Cortex®-M4 boot address when the BOOT pin is low.

4.9.20 FLASH register boot address for Arm® Cortex®-M4 core (FLASH_BOOT4_PRGR)

Address offset: 0x04C or 0x14C

Reset value: 0xFFFF XXXX (see [Table 20: Option byte organization](#))

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BOOT_CM4_ADD1															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BOOT_CM4_ADD0															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **BOOT_CM4_ADD1**: Arm® Cortex®-M4 boot address 1 configuration

These bits allow configuring the MSB of the Arm® Cortex®-M4 boot address when the BOOT pin is high.

Bits 15:0 **BOOT_CM4_ADD0**: Arm® Cortex®-M4 boot address 0 configuration

These bits allow configuring the MSB of Arm® Cortex®-M4 boot address when the BOOT pin is low.

4.9.21 FLASH CRC control register for bank 1 (FLASH_CRCCR1)

Address offset: 0x050

Reset value: 0x001C 0000

This register can be modified only if CRC_EN bit is set to 1 in FLASH_CR1 register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALL_BANK	CRC_BURST		Res.	Res.	CLEAN_CRC	START_CRC
									w	rw	rw			w	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	CLEAN_SECT	ADD_SECT	CRC_BY_SECT	Res.	Res.	Res.	Res.	Res.	CRC_SECT		
					w	w	rw						rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **ALL_BANK**: Bank 1 CRC select bit

When ALL_BANK is set to 1, all bank 1 user sectors are added to list of sectors on which the CRC is calculated.

Bits 21:20 **CRC_BURST**: Bank 1 CRC burst size

CRC_BURST bits set the size of the bursts that are generated by the CRC calculation unit.

00: every burst has a size of 4 flash words (256-bit)

01: every burst has a size of 16 flash words (256-bit)

10: every burst has a size of 64 flash words (256-bit)

11: every burst has a size of 256 flash words (256-bit)

Bits 19:18 Reserved, must be kept at reset value.

Bit 17 **CLEAN_CRC**: Bank 1 CRC clear bit

Setting CLEAN_CRC to 1 clears the current CRC result stored in the FLASH_CRCDATAR register.

Bit 16 **START_CRC**: Bank 1 CRC start bit

START_CRC bit triggers a CRC calculation on bank 1 using the current configuration. No CRC calculation can be launched when an option byte change operation is ongoing because all write accesses to embedded flash memory registers are put on hold until the option byte change operation has completed.

Bits 15:11 Reserved, must be kept at reset value.

Bit 10 **CLEAN_SECT**: Bank 1 CRC sector list clear bit

Setting CLEAN_SECT to 1 clears the list of sectors on which the CRC is calculated.

Bit 9 **ADD_SECT**: Bank 1 CRC sector select bit

Setting ADD_SECT to 1 adds the sector whose number is CRC_SECT to the list of sectors on which the CRC is calculated.

Bit 8 **CRC_BY_SECT**: Bank 1 CRC sector mode select bit

When CRC_BY_SECT is set to 1, the CRC calculation is performed at sector level, on the sectors present in the list of sectors. To add a sector to this list, use ADD_SECT and CRC_SECT bits. To clean the list, use CLEAN_SECT bit.

When CRC_BY_SECT is reset to 0, the CRC calculation is performed on all addresses between CRC_START_ADDR and CRC_END_ADDR.

Bits 7:3 Reserved, must be kept at reset value.

Bits 2:0 **CRC_SECT**: Bank 1 CRC sector number

CRC_SECT is used to select one or more user flash sectors to be added to the list of sectors on which the CRC is calculated. The CRC can be computed either between two addresses (using registers FLASH_CRCSADD1R and FLASH_CRCEADD1R) or on a list of sectors. If this latter option is selected, it is possible to add a sector to the list of sectors by programming the sector number in CRC_SECT and then setting ADD_SECT to 1.

The list of sectors can be erased either by setting CLEAN_SECT bit or by disabling the CRC computation. CRC_SECT can be set only when CRC_EN of FLASH_CR register is set to 1.

000: sector 0 of bank 1

001: sector 1 of bank 1

...

111: sector 7 of bank 1

4.9.22 FLASH CRC start address register for bank 1 (FLASH_CRCSADD1R)

Address offset: 0x054

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_START_ADDR[19:16]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_START_ADDR[15:2]													Res.	Res.	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:2 **CRC_START_ADDR[19:2]**: CRC start address on bank 1

CRC_START_ADDR is used when CRC_BY_SECT is cleared to 0. It must be programmed to the start address of the bank 1 memory area on which the CRC calculation is performed.

Bits 1:0 Reserved, must be kept at reset value.

4.9.23 FLASH CRC end address register for bank 1 (FLASH_CRCEADD1R)

Address offset: 0x058

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_END_ADDR[19:16]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_END_ADDR[15:2]													Res.	Res.	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:2 **CRC_END_ADDR[19:2]**: CRC end address on bank 1

CRC_END_ADDR is used when CRC_BY_SECT is cleared to 0. It must be programmed to the end address of the bank 1 memory area on which the CRC calculation is performed

Bits 1:0 Reserved, must be kept at reset value.

4.9.24 FLASH CRC data register (FLASH_CRCDATAR)

Address offset: 0x05C or 0x15C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CRC_DATA															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_DATA															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **CRC_DATA**: CRC result

CRC_DATA bits contain the result of the last CRC calculation.

4.9.25 FLASH ECC fail address for bank 1 (FLASH_ECC_FA1R)

Address offset: 0x060

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	FAIL_ECC_ADDR1														
	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:15 Reserved, must be kept at reset value.

Bits 14:0 **FAIL_ECC_ADDR1**: Bank 1 ECC error address

When an ECC error occurs (both for single correction or double detection) during a read operation from bank 1, the FAIL_ECC_ADDR1 bitfield indicates the address that generated the error:

Fail address = FAIL_ECC_ADDR1 * 32 + flash memory Bank 1 address offset

FAIL_ECC_ADDR1 is reset when the flag error in the FLASH_SR1 register (CLR_SNECCERR1 or CLR_DBECCERR1) is reset.

The embedded flash memory programs the address in this register only when no ECC error flags are set. This means that only the first address that generated an ECC error is saved.

The address in FAIL_ECC_ADDR1 is relative to the flash memory area where the error occurred (user flash memory, system flash memory).

4.9.26 FLASH key register for bank 2 (FLASH_KEYR2)

Address offset: 0x104

Reset value: 0x0000 0000

FLASH_KEYR2 is a write-only register. The following values must be programmed consecutively to unlock FLASH_CR2 register and allow programming/erasing it:

- 1st key = 0x4567 0123
- 2nd key = 0xCDEF 89AB

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY2R															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY2R															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **KEY2R**: Bank 2 access configuration unlock key

4.9.27 FLASH control register for bank 2 (FLASH_CR2)

Address offset: 0x10C

Reset value: 0x0000 0031

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CRCRDERRIE2	CRCENDIE2	DBECCERRIE2	SNECCERRIE2	RDSERRIE2	RDPERIE2	OPERRIE2	INCERRIE2	Res.	STRBERIE2	PGSERRIE2	WRPERIE2	EOPIE2
			r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w		r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_EN	SPSS2	Res.	Res.	Res.	SNB2			START2	FW2	PSIZE2		BER2	SER2	PG2	LOCK2
r/w	r/w				r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	rs

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **CRCRDERRIE2**: Bank 2 CRC read error interrupt enable bit

When CRCRDERRIE2 bit is set to 1, an interrupt is generated when a protected area (PCROP or secure-only) has been detected during the last CRC computation on bank 2. CRCRDERRIE2 can be programmed only when LOCK2 is cleared to 0.

0: no interrupt generated when a CRC read error occurs on bank 2

1: interrupt generated when a CRC read error occurs on bank 2

Bit 27 **CRCENDIE2**: Bank 2 CRC end of calculation interrupt enable bit

When CRCENDIE2 bit is set to 1, an interrupt is generated when the CRC computation has completed on bank 2. CRCENDIE2 can be programmed only when LOCK2 is cleared to 0.

0: no interrupt generated when CRC computation complete on bank 2

1: interrupt generated when CRC computation complete on bank 2

Bit 26 **DBECCERRIE2**: Bank 2 ECC double detection error interrupt enable bit

When DBECCERRIE2 bit is set to 1, an interrupt is generated when an ECC double detection error occurs during a read operation from bank 2. DBECCERRIE2 can be programmed only when LOCK2 is cleared to 0.

0: no interrupt generated when an ECC double detection error occurs on bank 2

1: interrupt generated if an ECC double detection error occurs on bank 2

- Bit 25 **SNECCERRIE2**: Bank 2 ECC single correction error interrupt enable bit
When SNECCERRIE2 bit is set to 1, an interrupt is generated when an ECC single correction error occurs during a read operation from bank 2. SNECCERRIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated when an ECC single correction error occurs on bank 2
1: interrupt generated when an ECC single correction error occurs on bank 2
- Bit 24 **RDSERRIE2**: Bank 2 secure error interrupt enable bit
When RDSERRIE2 bit is set to 1, an interrupt is generated when a secure error (access to a secure-only protected address) occurs during a read operation from bank 2. RDSERRIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated when a secure error occurs on bank 2
1: an interrupt is generated when a secure error occurs on bank 2
- Bit 23 **RDPERRIE2**: Bank 2 read protection error interrupt enable bit
When RDPERRIE2 bit is set to 1, an interrupt is generated when a read protection error occurs (access to an address protected by PCROP or by RDP level 1) during a read operation from bank 2. RDPERRIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated when a read protection error occurs on bank 2
1: an interrupt is generated when a read protection error occurs on bank 2
- Bit 22 **OPERRIE2**: Bank 2 write/erase error interrupt enable bit
When OPERRIE2 bit is set to 1, an interrupt is generated when an error is detected during a write/erase operation to bank 2. OPERRIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated when a write/erase error occurs on bank 2
1: interrupt generated when a write/erase error occurs on bank 2
- Bit 21 **INCERRIE2**: Bank 2 inconsistency error interrupt enable bit
When INCERRIE2 bit is set to 1, an interrupt is generated when an inconsistency error occurs during a write operation to bank 2. INCERRIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated when a inconsistency error occurs on bank 2
1: interrupt generated when a inconsistency error occurs on bank 2.
- Bit 20 Reserved, must be kept at reset value.
- Bit 19 **STRBERRIE2**: Bank 2 strobe error interrupt enable bit
When STRBERRIE2 bit is set to 1, an interrupt is generated when a strobe error occurs (the master programs several times the same byte in the write buffer) during a write operation to bank 2. STRBERRIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated when a strobe error occurs on bank 2
1: interrupt generated when strobe error occurs on bank 2.
- Bit 18 **PGSERRIE2**: Bank 2 programming sequence error interrupt enable bit
When PGSERRIE2 bit is set to 1, an interrupt is generated when a sequence error occurs during a program operation to bank 2. PGSERRIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated when a sequence error occurs on bank 2
1: interrupt generated when sequence error occurs on bank 2.
- Bit 17 **WRPERRIE2**: Bank 2 write protection error interrupt enable bit
When WRPERRIE2 bit is set to 1, an interrupt is generated when a protection error occurs during a program operation to bank 2. WRPERRIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated when a protection error occurs on bank 2
1: interrupt generated when a protection error occurs on bank 2.

- Bit 16 **EOPIE2**: Bank 2 end-of-program interrupt control bit
Setting EOPIE2 bit to 1 enables the generation of an interrupt at the end of a program operation to bank 2. EOPIE2 can be programmed only when LOCK2 is cleared to 0.
0: no interrupt generated at the end of a program operation to bank 2.
1: interrupt enabled when at the end of a program operation to bank 2.
- Bit 15 **CRC_EN**: Bank 2 CRC control bit
Setting CRC_EN bit to 1 enables the CRC calculation on bank 2. CRC_EN does not start CRC calculation but enables CRC configuration through FLASH_CRCCR2 register.
When CRC calculation is performed on bank 2, it can only be disabled by setting CRC_EN bit to 0. Resetting CRC_EN clears CRC configuration and resets the content of FLASH_CRCDATAR register.
CRC_EN can be programmed only when LOCK2 is cleared to 0.
- Bit 14 **SPSS2**: Bank 2 special sector selection bit
Set this bit when accessing non-user sectors of the flash memory. This bit can be programmed only when LOCK2 is cleared to 0.
0: User sectors selection in bank 2
1: Non-user, special sectors selection in bank 2
- Bits 13:11 Reserved, must be kept at reset value.
- Bits 10:8 **SNB2**: Bank 2 sector erase selection number
These bits are used to select the target sector for a sector erase operation (they are unused otherwise). SNB2 can be programmed only when LOCK2 is cleared to 0.
000: sector 0 of bank 2
001: sector 1 of bank 2
...
111: sector 7 of bank 2
- Bit 7 **START2**: Bank 2 erase start control bit
START2 bit is used to start a sector erase or a bank erase operation. START2 can be programmed only when LOCK2 is cleared to 0.
The embedded flash memory resets START2 when the corresponding operation has been acknowledged. The user application cannot access any embedded flash memory register until the operation is acknowledged.
- Bit 6 **FW2**: Bank 2 write forcing control bit
FW2 forces a write operation even if the write buffer is not full. FW2 can be programmed only when LOCK2 is cleared to 0.
The embedded flash memory resets FW2 when the corresponding operation has been acknowledged. The user application cannot access any flash register until the operation is acknowledged.
Write forcing is effective only if the write buffer is not empty. In particular, FW2 does not start several write operations when the write operations are performed consecutively.
- Bits 5:4 **PSIZE2**: Bank 2 program size
PSIZE2 selects the maximum number of bits that can be written to 0 in one shot during a write operation (programming parallelism). PSIZE2 can be programmed only when LOCK2 is cleared to 0.
00: programming parallelism is byte (8 bits)
01: programming parallelism is half-word (16 bits)
10: programming parallelism is word (32 bits)
11: programming parallelism is double-word (64 bits)

Bit 3 BER2: Bank 2 erase request

Setting BER2 bit to 1 requests a bank erase operation on bank 2 (user flash memory only). BER2 can be programmed only when LOCK2 is cleared to 0.

BER2 has a higher priority than SER2: if both are set, the embedded flash memory executes a bank erase.

0: bank erase not requested on bank 2

1: bank erase requested on bank 2

Note: Write protection error is triggered when a bank erase is required and some sectors are protected.

Bit 2 SER2: Bank 2 sector erase request

Setting SER2 bit to 1 requests a sector erase on bank 2. SER2 can be programmed only when LOCK2 is cleared to 0.

BER2 has a higher priority than SER2: if both are set, the embedded flash memory executes a bank erase.

0: sector erase not requested on bank 2

1: sector erase requested on bank 2

Note: Write protection error is triggered when a sector erase is required on protected sector(s).

Bit 1 PG2: Bank 2 internal buffer control bit

Setting PG2 bit to 1 enables internal buffer for write operations to bank 2. This allows the preparation of program operations even if a sector or bank erase is ongoing.

PG2 can be programmed only when LOCK2 is cleared to 0. When PG2 is reset, the internal buffer is disabled for write operations to bank 2 and all the data stored in the buffer but not sent to the operation queue are lost.

Bit 0 LOCK2: Bank 2 configuration lock bit

This bit locks the FLASH_CR2 register. The correct write sequence to FLASH_KEYR2 register unlocks this bit. If a wrong sequence is executed, or the unlock sequence to FLASH_KEYR2 is performed twice, this bit remains locked until next system reset.

LOCK2 can be set by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When LOCK2 changes from 0 to 1, the other bits of FLASH_CR2 register do not change.

0: FLASH_CR2 register unlocked

1: FLASH_CR2 register locked

4.9.28 FLASH status register for bank 2 (FLASH_SR2)

Address offset: 0x110

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CRCRDERR2	CRCEND2	DBECCERR2	SNECCERR2	RDSERR2	RDPER2	OPERR2	INCERR2	Res.	STRBERR2	PGSERR2	WRPERR2	EOP2
			r	r	r	r	r	r	r	r		r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_BUSY2	QW2	WBNE2	BSY2
												r	r	r	r

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 CRCRDERR2: Bank 2 CRC read error flag

CRCRDERR2 flag is raised when a word is found read protected during a CRC operation on bank 2. An interrupt is generated if CRCRDIE2 and CRCEND2 are set to 1. Writing 1 to CLR_CRCRDERR2 bit in FLASH_CCR2 register clears CRCRDERR2.

0: no protected area inside the address read by CRC on bank 2

1: a protected area inside the address read by CRC on bank 2. CRC result is very likely incorrect.

Note: This flag is valid only when CRCEND2 bit is set to 1.

Bit 27 CRCEND2: Bank 2 CRC end of calculation flag

CRCEND2 bit is raised when the CRC computation has completed on bank 2. An interrupt is generated if CRCENDIE2 is set to 1. It is not necessary to reset CRCEND2 before restarting CRC computation. Writing 1 to CLR_CRCEND2 bit in FLASH_CCR2 register clears CRCEND2.

0: CRC computation not complete on bank 2

1: CRC computation complete on bank 2

Bit 26 DBECCERR2: Bank 2 ECC double detection error flag

DBECCERR2 flag is raised when an ECC double detection error occurs during a read operation from bank 2. An interrupt is generated if DBECCERRIE2 is set to 1. Writing 1 to CLR_DBECCERR2 bit in FLASH_CCR2 register clears DBECCERR2.

0: no ECC double detection error occurs on bank 2

1: ECC double detection error occurs on bank 2

Bit 25 SNECCERR2: Bank 2 single correction error flag

SNECCERR2 flag is raised when an ECC single correction error occurs during a read operation from bank 2. An interrupt is generated if SNECCERRIE2 is set to 1. Writing 1 to CLR_SNECCERR2 bit in FLASH_CCR2 register clears SNECCERR2.

0: no ECC single correction error occurs on bank 2

1: ECC single correction error occurs on bank 2

- Bit 24 **RDSERR2**: Bank 2 secure error flag
RDSERR2 flag is raised when a read secure error (read access to a secure-only protected word) occurs on bank 2. An interrupt is generated if RDSERRIE2 is set to 1. Writing 1 to CLR_RDSERR2 bit in FLASH_CCR2 register clears RDSERR2.
0: no secure error occurs on bank 2
1: a secure error occurs on bank 2
- Bit 23 **RDPER2**: Bank 2 read protection error flag
RDPER2 flag is raised when a read protection error (read access to a PCROP-protected word or a RDP-protected area) occurs on bank 2. An interrupt is generated if RDPERRIE2 is set to 1. Writing 1 to CLR_RDPER2 bit in FLASH_CCR2 register clears RDPER2.
0: no read protection error occurs on bank 2
1: a read protection error occurs on bank 2
- Bit 22 **OPERR2**: Bank 2 write/erase error flag
OPERR2 flag is raised when an error occurs during a write/erase to bank 2. An interrupt is generated if OPERRIE2 is set to 1. Writing 1 to CLR_OPERR2 bit in FLASH_CCR2 register clears OPERR2.
0: no write/erase error occurred on bank 2
1: a write/erase error occurred on bank 2
- Bit 21 **INCERR2**: Bank 2 inconsistency error flag
INCERR2 flag is raised when an inconsistency error occurs on bank 2. An interrupt is generated if INCERRIE2 is set to 1. Writing 1 to CLR_INCERR2 bit in the FLASH_CCR2 register clears INCERR2.
0: no inconsistency error occurred on bank 2
1: an inconsistency error occurred on bank 2.
- Bit 20 Reserved, must be kept at reset value.
- Bit 19 **STRBERR2**: Bank 2 strobe error flag
STRBERR2 flag is raised when a strobe error occurs on bank 2 (when the master attempts to write several times the same byte in the write buffer). An interrupt is generated if the STRBERRIE2 bit is set to 1. Writing 1 to CLR_STRBERR2 bit in FLASH_CCR2 register clears STRBERR2.
0: no strobe error occurred on bank 2
1: a strobe error occurred on bank 2.
- Bit 18 **PGSERR2**: Bank 2 programming sequence error flag
PGSERR2 flag is raised when a sequence error occurs on bank 2. An interrupt is generated if the PGSERRIE2 bit is set to 1. Writing 1 to CLR_PGSERR2 bit in FLASH_CCR2 register clears PGSERR2.
0: no sequence error occurred on bank 2
1: a sequence error occurred on bank 2.
- Bit 17 **WRPERR2**: Bank 2 write protection error flag
WRPERR2 flag is raised when a protection error occurs during a program operation to bank 2. An interrupt is also generated if the WRPERRIE2 is set to 1. Writing 1 to CLR_WRPERR2 bit in FLASH_CCR2 register clears WRPERR2.
0: no write protection error occurred on bank 2
1: a write protection error occurred on bank 2

Bit 16 EOP2: Bank 2 end-of-program flag

EOP2 flag is set when a programming operation to bank 2 completes. An interrupt is generated if the EOPIE2 is set to 1. It is not necessary to reset EOP2 before starting a new operation. EOP2 bit is cleared by writing 1 to CLR_EOP2 bit in FLASH_CCR2 register.

0: no programming operation completed on bank 2

1: a programming operation completed on bank 2

Bits 15:4 Reserved, must be kept at reset value.

Bit 3 CRC_BUSY2: Bank 2 CRC busy flag

CRC_BUSY2 flag is set when a CRC calculation is ongoing on bank 2. This bit cannot be forced to 0. The user must wait until the CRC calculation has completed or disable CRC computation on bank 2.

0: no CRC calculation ongoing on bank 2

1: CRC calculation ongoing on bank 2.

Bit 2 QW2: Bank 2 wait queue flag

QW2 flag is set when a write or erase operation is pending in the command queue buffer of bank 2. It is not possible to know what type of operation is present in the queue. This flag is reset by hardware when all write/erase operations have been executed and thus removed from the waiting queue(s). This bit cannot be forced to 0. It is reset after a deterministic time if no other operations are requested.

0: no write or erase operation is waiting in the operation queues of bank 2

1: at least one write or erase operation is pending in the operation queues of bank 2

Bit 1 WBNE2: Bank 2 write buffer not empty flag

WBNE2 flag is set when embedded flash memory is waiting for new data to complete the write buffer. In this state the write buffer is not empty. WBNE2 is reset by hardware each time the write buffer is complete or the write buffer is emptied following one of the event below:

- the application software forces the write operation using FW2 bit in FLASH_CR2
- the embedded flash memory detects an error that involves data loss
- the application software has disabled write operations in this bank

This bit cannot be forced to 0. To reset it, clear the write buffer by performing any of the above listed actions or send the missing data.

0: write buffer of bank 2 empty or full

1: write buffer of bank 2 waiting data to complete

Bit 0 BSY2: Bank 2 busy flag

BSY2 flag is set when an effective write or erase operation is ongoing to bank 2. It is not possible to know what type of operation is being executed.

BSY2 cannot be forced to 0. It is automatically reset by hardware every time a step in a write, or erase operation completes.

0: no write or erase operation is executed on bank 2

1: a write or an erase operation is being executed on bank 2.

4.9.29 FLASH clear control register for bank 2 (FLASH_CCR2)

Address offset: 0x114

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CLR_CRCDERR2	CLR_CRCEND2	CLR_DBECCERR2	CLR_SNECCERR2	CLR_RDSERR2	CLR_RDPERR2	CLR_OPERR2	CLR_INCERR2	Res.	CLR_STRBERR2	CLR_PGSERR2	CLR_WRPERR2	CLR_EOP2
			w	w	w	w	w	w	w	w		w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **CLR_CRCDERR2**: Bank 2 CRCDERR2 flag clear bit

Setting this bit to 1 resets to 0 CRCDERR2 flag in FLASH_SR2 register.

Bit 27 **CLR_CRCEND2**: Bank 2 CRCEND2 flag clear bit

Setting this bit to 1 resets to 0 CRCEND2 flag in FLASH_SR2 register.

Bit 26 **CLR_DBECCERR2**: Bank 2 DBECCERR2 flag clear bit

Setting this bit to 1 resets to 0 DBECCERR2 flag in FLASH_SR2 register. If the SNECCERR2 flag of FLASH_SR2 register is cleared to 0, FLASH_ECC_FA2R register is reset to 0 as well.

Bit 25 **CLR_SNECCERR2**: Bank 2 SNECCERR2 flag clear bit

Setting this bit to 1 resets to 0 SNECCERR2 flag in FLASH_SR2 register. If the DBECCERR2 flag of FLASH_SR2 register is cleared to 0, FLASH_ECC_FA2R register is reset to 0 as well.

Bit 24 **CLR_RDSERR2**: Bank 2 RDSERR2 flag clear bit

Setting this bit to 1 resets to 0 RDSERR2 flag in FLASH_SR2 register.

Bit 23 **CLR_RDPERR2**: Bank 2 RDPERR2 flag clear bit

Setting this bit to 1 resets to 0 RDPERR2 flag in FLASH_SR2 register.

Bit 22 **CLR_OPERR2**: Bank 2 OPERR2 flag clear bit

Setting this bit to 1 resets to 0 OPERR2 flag in FLASH_SR2 register.

Bit 21 **CLR_INCERR2**: Bank 2 INCERR2 flag clear bit

Setting this bit to 1 resets to 0 INCERR2 flag in FLASH_SR2 register.

Bit 20 Reserved, must be kept at reset value.

Bit 19 **CLR_STRBERR2**: Bank 2 STRBERR2 flag clear bit

Setting this bit to 1 resets to 0 STRBERR2 flag in FLASH_SR2 register.

Bit 18 **CLR_PGSERR2**: Bank 2 PGSERR2 flag clear bit

Setting this bit to 1 resets to 0 PGSERR2 flag in FLASH_SR2 register.

Bit 17 **CLR_WRPERR2**: Bank 2 WRPERR2 flag clear bit

Setting this bit to 1 resets to 0 WRPERR2 flag in FLASH_SR2 register.

Bit 16 **CLR_EOP2**: Bank 2 EOP2 flag clear bit

Setting this bit to 1 resets to 0 EOP2 flag in FLASH_SR2 register.

Bits 15:0 Reserved, must be kept at reset value.

4.9.30 FLASH protection address for bank 2 (FLASH_PRAR_CUR2)

Address offset: 0x128

Reset value: 0xFFFF 0XXX (see [Table 20: Option byte organization](#))

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMEP2	Res.	Res.	Res.	PROT_AREA_END2											
r				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PROT_AREA_START2											
				r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **DMEP2**: Bank 2 PCROP protected erase enable option status bit

If DMEP2 is set to 1, the PCROP protected area in bank 2 is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **PROT_AREA_END2**: Bank 2 PCROP area end status bits

These bits contain the last 256-byte block of the PCROP area in bank 2.

If this address is equal to PROT_AREA_START2, the whole bank 2 is PCROP protected.

If this address is lower than PROT_AREA_START2, no protection is set on bank 2.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **PROT_AREA_START2**: Bank 2 PCROP area start status bits

These bits contain the first 256-byte block of the PCROP area in bank 2.

If this address is equal to PROT_AREA_END2, the whole bank 2 is PCROP protected.

If this address is higher than PROT_AREA_END2, no protection is set on bank 2.

4.9.31 FLASH protection address for bank 2 (FLASH_PRAR_PRG2)

Address offset: 0x12C

Reset value: 0xFFFF 0XXX (see [Table 20: Option byte organization](#))

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMEP2	Res.	Res.	Res.	PROT_AREA_END2											
rw				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PROT_AREA_START2											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **DMEP2**: Bank 2 PCROP protected erase enable option configuration bit

If DMEP2 is set to 1, the PCROP protected area in bank 2 is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Only the Arm® Cortex®-M7 master can program this bit.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **PROT_AREA_END2**: Bank 2 PCROP area end configuration bits

These bits contain the last 256-byte block of the PCROP area in bank 2.

If this address is equal to PROT_AREA_START2, the whole bank 2 is PCROP protected.

If this address is lower than PROT_AREA_START2, no protection is set on bank 2.

Only the Arm® Cortex®-M7 master can program these bits.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **PROT_AREA_START2**: Bank 2 PCROP area start configuration bits

These bits contain the first 256-byte block of the PCROP area in bank 2.

If this address is equal to PROT_AREA_END2, the whole bank 2 is PCROP protected.

If this address is higher than PROT_AREA_END2, no protection is set on bank 2.

Only the Arm® Cortex®-M7 master can program these bits.

4.9.32 FLASH secure address for bank 2 (FLASH_SCAR_CUR2)

Address offset: 0x130

Reset value: 0xFFFF 0XXX (see [Table 20: Option byte organization](#))

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMEP2	Res.	Res.	Res.	SEC_AREA_END2											
r				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	SEC_AREA_START2											
				r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **DMES2**: Bank 2 secure protected erase enable option status bit

If DMES2 is set to 1, the secure protected area in bank 2 is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **SEC_AREA_END2**: Bank 2 secure-only area end status bits

These bits contain the last 256-byte block of the secure-only area in bank 2.

If this address is equal to SEC_AREA_START2, the whole bank 2 is secure protected.

If this address is lower than SEC_AREA_START2, no protection is set on bank 2.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **SEC_AREA_START2**: Bank 2 secure-only area start status bits

These bits contain the first 256-byte block of the secure-only area in bank 2.

If this address is equal to SEC_AREA_END2, the whole bank 2 is secure protected.

If this address is higher than SEC_AREA_END2, no protection is set on bank 2.

4.9.33 FLASH secure address for bank 2 (FLASH_SCAR_PRG2)

Address offset: 0x134

Reset value: 0xFFFF 0XXX (see [Table 20: Option byte organization](#))

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMES2	Res.	Res.	Res.	SEC_AREA_END2											
rw				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	SEC_AREA_START2											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **DMES2**: Bank 2 secure access protected erase enable option configuration bit

If DMES2 is set to 1, the secure access only area in bank 2 is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Only the Arm®Cortex®-M7 master can program this bit.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **SEC_AREA_END2**: Bank 2 secure-only area end configuration bits

These bits contain the last of 256 bytes block of the secure-only area in bank 2.

If this address is equal to SEC_AREA_START2, the whole bank 2 is secure access only.

If this address is lower than SEC_AREA_START2, no protection is set on bank 2.

Only the Arm®Cortex®-M7 master can program this bit.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **SEC_AREA_START2**: Bank 2 secure-only area start configuration bits

These bits contain the first of 256 bytes block of the secure-only area in bank 2.

If this address is equal to SEC_AREA_END2, the whole bank 2 is secure access only.

If this address is higher than SEC_AREA_END2, no protection is set on bank 2.

Only the Arm®Cortex®-M7 master can program this bit.

4.9.34 FLASH write sector protection for bank 2 (FLASH_WPSN_CUR2R)

Address offset: 0x138

Reset value: 0x0000 00XX

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn2							
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **WRPSn2**: Bank 2 sector write protection option status byte

Each FLASH_WPSGN_CUR2R bit reflects the write protection status of the corresponding bank 2 sector (0: sector is write protected; 1: sector is not write protected)

4.9.35 FLASH write sector protection for bank 2 (FLASH_WPSN_PRG2R)

Address offset: 0x13C

Reset value: 0x0000 00XX

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn2							
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **WRPSn2**: Bank 2 sector write protection option status byte

Setting WRPSn2 bits to 0 write protects the corresponding bank 2 sector (0: sector is write protected; 1: sector is not write protected)

4.9.36 FLASH CRC control register for bank 2 (FLASH_CRCCR2)

Address offset: 0x150

Reset value: 0x001C 0000

The values in this register can be changed only if CRC_EN bit is set to 1 in FLASH_CR2 register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALL_BANK	CRC_BURST		Res.	Res.	CLEAN_CRC	START_CRC
									w	rw	rw			w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	CLEAN_SECT	ADD_SECT	CRC_BY_SECT	Res.	Res.	Res.	Res.	Res.	CRC_SECT		
					w	w	rw						rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **ALL_BANK**: Bank 2 CRC select bit

When ALL_BANK is set to 1, all bank 2 user sectors are added to the list of sectors on which the CRC is calculated.

Bits 21:20 **CRC_BURST**: Bank 2 CRC burst size

CRC_BURST bits set the size of the bursts that are generated by the CRC calculation unit.

00: every burst has a size of 4 flash words (256-bit)

01: every burst has a size of 16 flash words (256-bit)

10: every burst has a size of 64 flash words (256-bit)

11: every burst has a size of 256 flash words (256-bit)

Bits 19:18 Reserved, must be kept at reset value.

Bit 17 **CLEAN_CRC**: Bank 2 CRC clear bit

Setting CLEAN_CRC to 1 clears the current CRC result stored in the FLASH_CRCDATAR register.

Bit 16 **START_CRC**: Bank 2 CRC start bit

START_CRC bit triggers a CRC calculation on bank 2 using the current configuration. It is not possible to start a CRC calculation when an option byte change operation is ongoing because all write accesses to embedded flash memory registers are put on hold until the option byte change operation has completed.

Bits 15:11 Reserved, must be kept at reset value.

Bit 10 **CLEAN_SECT**: Bank 2 CRC sector list clear bit

Setting CLEAN_SECT to 1 clears the list of sectors on which the CRC is calculated.

Bit 9 **ADD_SECT**: Bank 2 CRC sector select bit

Setting ADD_SECT to 1 adds the sector whose number is CRC_SECT to the list of sectors on which the CRC is calculated.

Bit 8 **CRC_BY_SECT**: Bank 2 CRC sector mode select bit

When CRC_BY_SECT is set to 1, the CRC calculation is performed at sector level, on the sectors selected by CRC_SECT.

When CRC_BY_SECT is reset to 0, the CRC calculation is performed on all addresses between CRC_START_ADDR and CRC_END_ADDR.

Bits 7:3 Reserved, must be kept at reset value.

Bits 2:0 **CRC_SECT**: Bank 2 CRC sector number

CRC_SECT is used to select one or more user flash sectors to be added to the CRC calculation. The CRC can be computed either between two addresses (using registers FLASH_CRCSADD2R and FLASH_CRCEADD2R) or on a list of sectors. If this latter option is selected, it is possible to add a sector to the list of sectors by programming the sector number in CRC_SECT and then setting ADD_SECT. to 1

The list of sectors can be erased either by setting CLEAN_SECT bit or by disabling the CRC computation. CRC_SECT can be set only when CRC_EN of FLASH_CR register is set to 1.

000: sector 0 of bank 2

001: sector 1 of bank 2

...

111: sector 7 of bank 2

4.9.37 FLASH CRC start address register for bank 2 (FLASH_CRCSADD2R)

Address offset: 0x154

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_START_ADDR[19:16]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_START_ADDR[15:2]													Res.	Res.	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:2 **CRC_START_ADDR[19:2]**: CRC start address on bank 2

CRC_START_ADDR is used when CRC_BY_SECT is cleared to 0. It must be programmed to the start address of the bank 2 memory area on which the CRC calculation is performed.

Bits 1:0 Reserved, must be kept at reset value.

4.9.38 FLASH CRC end address register for bank 2 (FLASH_CRCEADDR2R)

Address offset: 0x158

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_END_ADDR[19:16]			
												rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_END_ADDR[15:2]													Res.	Res.	
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW		

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:2 **CRC_END_ADDR[19:2]**: CRC end address on bank 2

CRC_END_ADDR is used when CRC_BY_SECT is cleared to 0. It must be programmed to the end address of the bank 2 memory area on which the CRC calculation is performed.

Bits 1:0 Reserved, must be kept at reset value.

4.9.39 FLASH ECC fail address for bank 2 (FLASH_ECC_FA2R)

Address offset: 0x160

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	FAIL_ECC_ADDR2														
	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:15 Reserved, must be kept at reset value.

Bits 14:0 **FAIL_ECC_ADDR2**: Bank 2 ECC error address

When an ECC error occurs (both for single error correction or double detection) during a read operation from bank 2, the FAIL_ECC_ADDR2 bitfield indicates the address that generated the error:

Fail address = FAIL_ECC_ADDR2 * 32 + flash memory Bank 2 address offset

FAIL_ECC_ADDR2 is reset when the flag error in the FLASH_SR2 register (CLR_SNECCERR2 or CLR_DBECERR2) is reset.

The embedded flash memory programs the address in this register only when no ECC error flags are set. This means that only the first address that generated an ECC error is saved.

4.9.40 FLASH register map and reset values

Table 27. Register map and reset value table

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
0x000	FLASH_ACR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WRHIGHFREQ	LATENCY								
	0x00000037																										1								1	0	1
0x004	FLASH_KEYR1	KEY1R																																			
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x008	FLASH_OPTKEYR	OPTKEYR																																			
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x00C	FLASH_CR1	Res	Res	Res	CRCRDERR1	CRCENDIE1	DBECCERRIE1	SNECCERRIE1	RDSERRIE1	RDPERRIE1	OPERRIE1	INCERRIE1	Res	STRBERRIE1	PGSERRIE1	WRPERRIE1	EOPIE1	CRC_EN	Res	Res	Res	Res	SNB1				START1	FW1	PSIZE1	BER1	SER1	PG1	LOCK1				
	0x00000031				0	0	0	0	0	0	0	0	0	0	0	0	0	0		Res	Res	Res	Res	0	0	0	0	0	0	1	1	0	0	0	1		
0x010	FLASH_SR1	Res	Res	Res	CRCRDERR1	CRCEND1	DBECCERR1	SNECCERR1	RDSERR1	RDPERRI1	OPERRIE1	INCERR1	Res	STRBERRI1	PGSERRI1	WRPERRI1	EOP1	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CRC_BUSY1	QW1	WBNE1	BSY1			
	0x00000000				0	0	0	0	0	0	0	0	0	0	0	0	0													0	0	0	0	0			
0x014	FLASH_CCR1	Res	Res	Res	CLR_CRCRDERR1	CLR_CRCEND1	CLR_DBECCERR1	CLR_SNECCERR1	CLR_RDSERR1	CLR_RDPERR1	CLR_OPERRIE1	CLR_INCERR1	Res	CLR_STRBERRI1	CLR_PGSERR1	CLR_WRPERR1	CLR_EOP1	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res			
	0x00000000						0	0	0	0	0	0	0	0	0	0	0														0	0	0	0			
0x018	FLASH_OPTCR	SWAP_BANK	OPTCHANGEERR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	MER	Res	Res	OPTSTART	OPTLOCK			
	0x00000001	0	0																										0			0	1				
0x01C	FLASH_OPTSR_CUR	SWAP_BANK_OPT	IO_HSLV	Res	Res	Res	NRST_STBY_D2	NRST_STOP_D2	BOOT_CM7	BOOT_CM4	SECURITY	ST_RAM_SIZE	Res	IWDG_FZ_SDBY	IWDG_FZ_STOP	Res	RDP										NRST_STBY_D1	NRST_STOP_D1	IWDG2_SW	IWDG1_SW	BOR_LEV	Res	OPT_BUSY				
	0xFFFF XXXX	X	X	X			X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			

Table 27. Register map and reset value table (continued)

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x020	FLASH_OPTSR_PRG	SWAP_BANK_OPT	Res	IO_HSLV	Res	Res	Res	NRST_STBY_D2	NRST_STOP_D2	BOOT_CM7	BOOT_CM4	SECURITY	ST_RAM_SIZE	Res	IWDG_FZ_SDBY	IWDG_FZ_STOP	Res	RDP										NRST_STBY_D1	NRST_STOP_D1	IWDG2_SW	IWDG1_SW	BOR_LEV	Res	Res	
	0xFFFF XXXX	X	X					X	X	X	X	X	X	X	X	X		X	X	X	X	X	X	X	X	X	X	X	X	X	X				
0x024	FLASH_OPTCCR	Res	CLR_OPTCHANGEERR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res		
	0x00000000		0																																
0x028	FLASH_PRAR_CUR1	DMEP1	Res	Res	Res	PROT_AREA_END1[11:0]										Res	Res	Res	Res	PROT_AREA_START1[11:0]															
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X			
0x02C	FLASH_PRAR_PRG1	DMEP1	Res	Res	Res	PROT_AREA_END1[11:0]										Res	Res	Res	Res	PROT_AREA_START1[11:0]															
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X			
0x030	FLASH_SCAR_CUR1	DMES1	Res	Res	Res	SEC_AREA_END1[11:0]										Res	Res	Res	Res	SEC_AREA_START1[11:0]															
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X			
0x034	FLASH_SCAR_PRG1	DMES1	Res	Res	Res	SEC_AREA_END1[11:0]										Res	Res	Res	Res	SEC_AREA_START1[11:0]															
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X			
0x038	FLASH_WPSN_CUR1R	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WRPSn1									
	0x0000 00XX																									X	X	X	X	X	X	X	X		
0x03C	FLASH_WPSN_PRG1R	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WRPSn1									
	0x0000 00XX																									X	X	X	X	X	X	X	X		
0x040	FLASH_BOOT7_CURR	BOOT_CM7_ADD1[15:0]															BOOT_CM7_ADD0[15:0]																		
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			
0x044	FLASH_BOOT7_PRGR	BOOT_CM7_ADD1[15:0]															BOOT_CM7_ADD0[15:0]																		
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			
0x048	FLASH_BOOT4_CURR	BOOT_CM4_ADD1[15:0]															BOOT_CM4_ADD0[15:0]																		
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			

Table 27. Register map and reset value table (continued)

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x04C	FLASH_BOOT4_PRGR	BOOT_CM4_ADD1[15:0]												BOOT_CM4_ADD0[15:0]																			
	0xXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x050	FLASH_CRCR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALL_BANK	CRC_BURST		Res.	Res.	CLEAN_CRC	START_CRC	Res.	Res.	Res.	Res.	Res.	CLEAN_SECT	ADD_SECT	CRC_BY_SECT	Res.	Res.	Res.	Res.	Res.		CRC_SECT	
	0x001C0000										0	0	1			0	0						0	0	0						0	0	0
0x054	FLASH_CRCSECT1R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_START_ADDR[31:0]																Res.	Res.		
	0x00000000													0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x058	FLASH_CRCECT1R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_END_ADDR[31:0]																Res.	Res.		
	0x00000000													0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x05C	FLASH_CRCDATAR	CRC_DATA[31:0]																															
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x060	FLASH_ECC_FA1R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FAIL_ECC_ADDR1[14:0]														
	0x00000000																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x100	FLASH_ACR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRHIGHFREQ		LATENCY		
	0x00000037																											1	1	0			1
0x104	FLASH_KEYR2	KEY2R																															
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x108	FLASH_OPTKEYR	OPTKEYR																															
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10C	FLASH_CR2	Res.	Res.	Res.	CRCRDERR2	CRCEND2	DBECCERR2	SNECCERR2	RDSERR2	RDPERR2	OPERR2	INCERR2	Res.	STRBERR2	PGSERR2	WRPERR2	EOPIE2	CRC_EN	SPSS2	Res.	Res.	Res.	SNB2[2:0]		START2	FW2	PSIZE2	BER2	SER2	PG2	LOCK2		
	0x00000031				0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				0	0	0	0	0	1	1	0	0	0	1
0x110	FLASH_SR2	Res.	Res.	Res.	CRCRDERR2	CRCEND2	DBECCERR2	SNECCERR2	RDSERR2	RDPERR2	OPERR2	INCERR2	Res.	STRBERR2	PGSERR2	WRPERR2	EOP2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_BUSY2	QW2	WBNE2	BSY2	
	0x00000000				0	0	0	0	0	0	0	0		0	0	0	0												0	0	0	0	0

Table 27. Register map and reset value table (continued)

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x114	FLASH_CCR2	Res.	Res.	Res.	CLR_CRCRDERR2	CLR_CRCEND2	CLR_DBECERR2	CLR_SNECERR2	CLR_RDSERR2	CLR_RDPERR2	CLR_OPERR2	CLR_INCERR2	Res.	CLR_STRBERR2	CLR_PGSERR2	CLR_WRPERR2	CLR_EOP2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	0x00000000				0	0	0	0	0	0	0	0	0	0	0	0	0																
0x118	FLASH_OPTCR	SWAP_BANK	OPTCHANGEERR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MER	Res.	Res.	OPTSTART	OPTLOCK
	0x00000001	0	0																									0			0	1	
0x11C	FLASH_OPTSR_CUR	SWAP_BANK_OPT	OPTCHANGEERR	IO_HSLV	Res.	Res.	Res.	NRST_STBY_D2	NRST_STOP_D2	BOOT_CM7	BOOT_CM4	SECURITY	ST_RAM_SIZE	IWDG_FZ_SDBY	IWDG_FZ_STOP	Res.	RDP										NRST_STBY_D1	NRST_STOP_D1	IWDG2_SW	IWDG1_SW	BOR_LEV	Res.	OPT_BUSY
	0xFFFF XXXX	X	0	X				X	X	X	X	X	X	X	X		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x120	FLASH_OPTSR_PRG	SWAP_BANK_OPT	Res.	IO_HSLV	Res.	Res.	Res.	nRST_STBY_D2	nRST_STOP_D2	BOOT_CM7	BOOT_CM4	SECURITY	ST_RAM_SIZE	FZ_IWDG_SDBY	FZ_IWDG_STOP	Res.	RDP										nRST_STDY	nRST_STOP	Res.	IWDG1_SW	BOR_LEV	Res.	Res.
	0xFFFF XXXX	X		X				X	X	X	X	X	X	X	X		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X		
0x124	FLASH_OPTCCR	Res.	CLR_OPTCHANGEERR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	0x00000000		0																														
0x128	FLASH_PRAR_CUR2	DMEP2	Res.	Res.	Res.	PROT_AREA_END2[11:0]											Res.	Res.	Res.	Res.	PROT_AREA_START2[11:0]												
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X	X	X
0x12C	FLASH_PRAR_PRG2	DMEP2	Res.	Res.	Res.	PROT_AREA_END2[11:0]											Res.	Res.	Res.	Res.	PROT_AREA_START2[11:0]												
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X	X	X
0x130	FLASH_SCAR_CUR2	DMES2	Res.	Res.	Res.	SEC_AREA_END2[11:0]											Res.	Res.	Res.	Res.	SEC_AREA_START2[11:0]												
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X	X	X

Table 27. Register map and reset value table (continued)

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
0x134	FLASH_SCAR_PRG2	DMES2	Res.	Res.	Res.	SEC_AREA_END2[11:0]												Res.	Res.	Res.	Res.	SEC_AREA_START2[11:0]															
	0xFFFF 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X						X	X	X	X	X	X	X	X	X	X	X				
0x138	FLASH_WPSN_CUR2R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn2											
	0x0000 00XX																									X	X	X	X	X	X	X	X				
0x13C	FLASH_WPSN_PRG2R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn2											
	0x0000 00XX																									X	X	X	X	X	X	X	X				
0x140	FLASH_BOOT7_CURR	BOOT_CM7_ADD1[15:0]												BOOT_CM7_ADD0[15:0]																							
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X				
0x144	FLASH_BOOT7_PRGR	BOOT_CM7_ADD1[15:0]												BOOT_CM7_ADD0[15:0]																							
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X				
0x148	FLASH_BOOT4_CURR	BOOT_CM4_ADD1[15:0]												BOOT_CM4_ADD0[15:0]																							
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X				
0x14C	FLASH_BOOT4_PRGR	BOOT_CM4_ADD1[15:0]												BOOT_CM4_ADD0[15:0]																							
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X				
0x150	FLASH_CRCCR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALL_BANK	CRC_BURST	Res.	Res.	CLEAN_CRC	START_CRC	Res.	Res.	Res.	Res.	Res.	Res.	CLEAN_SECT	ADD_SECT	CRC_BY_SECT	Res.	Res.	Res.	Res.	Res.	CRC_SECT						
	0x001C0000										0	0	1		0	0							0	0	0						0	0	0				
0x154	FLASH_CRCS_ADD2R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_START_ADDR[19:2]															Res.	Res.								
	0x00000000												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
0x158	FLASH_CRCEADD2R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_END_ADDR[19:2]															Res.	Res.								
	0x00000000												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
0x15C	FLASH_CRCDATAR	CRC_DATA[31:0]																																			
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x160	FLASH_ECC_FA2R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FAIL_ECC_ADDR2[15:0]																				
	0x00000000																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

7 Power control (PWR)

7.1 Introduction

The Power control section (PWR) provides an overview of the supply architecture for the different power domains and of the supply configuration controller.

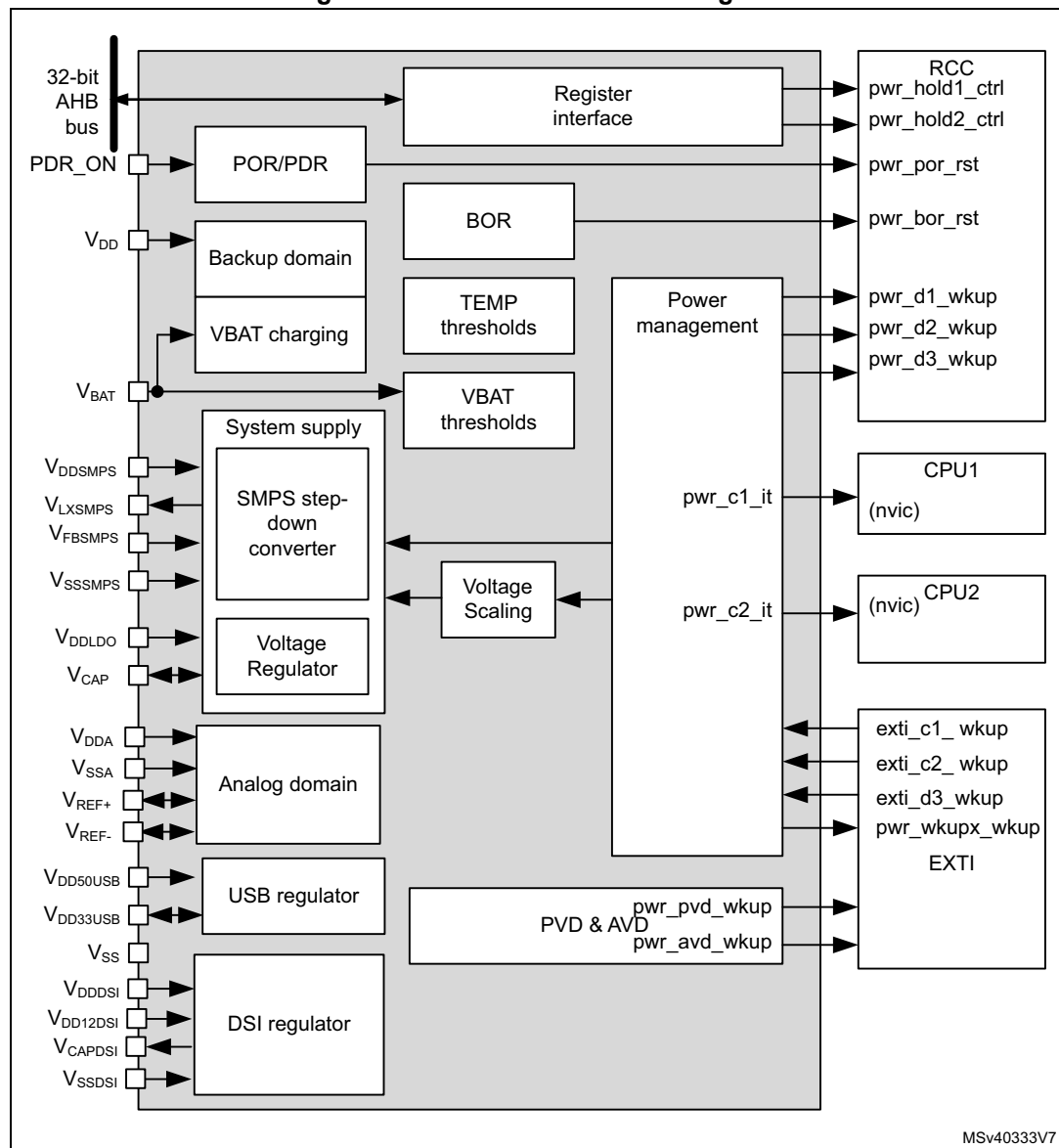
It also describes the features of the power supply supervisors and explains how the V_{CORE} supply domain is configured depending on the operating modes, the selected performance (clock frequency) and the voltage scaling.

7.2 PWR main features

- Power supplies and supply domains
 - Core domains (V_{CORE})
 - V_{DD} domain
 - Backup domain (V_{SW} , V_{BKP})
 - Analog domain (V_{DDA})
- System supply voltage regulation
 - SMPS step-down converter
 - Voltage regulator (LDO)
- Peripheral supply regulation
 - USB regulator
 - DSI regulator
- Power supply supervision
 - POR/PDR monitor
 - BOR monitor
 - PVD monitor
 - AVD monitor
 - V_{BAT} thresholds
 - Temperature thresholds
- Power management
 - V_{BAT} battery charging
 - Operating modes
 - Voltage scaling control
 - Low-power modes

7.3 PWR block diagram

Figure 20. Power control block diagram



7.3.1 PWR pins and internal signals

[Table 31](#) lists the PWR inputs and output signals connected to package pins or balls, while [Table 32](#) shows the internal PWR signals.

Table 31. PWR input/output signals connected to package pins or balls

Pin name	Signal type	Description
VDD	Supply input	Main I/O and V _{DD} domain supply input
VDDA	Supply input	External analog power supply for analog peripherals
VREF+,VREF-	Supply Input/Outputs	External reference voltage for ADCs and DAC
VBAT	Supply input	Backup battery supply input
VDDSMPS	Supply input	Step-down converter supply input
VLXSMPS	Supply output	Step-down converter supply output
VFBSMPS	Supply input	Step-down converter feedback voltage sense
VSSSMPS	Supply input	Step-down converter ground
VDDLDO	Supply input	Voltage regulator supply input
VCAP	Supply Input/Outputs	Digital core domain supply
VDD50USB	Supply input	USB regulator supply input
VDD33USB	Supply Input/Outputs	USB regulator supply output
VDDDSI	Supply input	DSI regulator supply input
VDD12DSI	Supply input	DSI PHY supply input
VCAPDSI	Supply output	DSI regulator supply output
VSSDSI	Supply input	DSI regulator ground
VSS	Supply input	Main ground

Table 31. PWR input/output signals connected to package pins or balls (continued)

Pin name	Signal type	Description
AHB	I/O	AHB register interface
PDR_ON	Digital input	Power Down Reset enable

Table 32. PWR internal input/output signals

Signal name	Signal type	Description
pwr_hold1_ctrl	Digital output	CPU1 clock hold
pwr_hold2_ctrl	Digital output	CPU2 clock hold
pwr_c1_it	Digital output	CPU2 on-hold wakeup interrupt to CPU1.
pwr_c2_it	Digital output	CPU1 on-hold wakeup interrupt to CPU2.
pwr_pvd_wkup	Digital output	Programmable voltage detector output
pwr_avd_wkup	Digital output	Analog voltage detector output
pwr_wkupx_wkup	Digital output	CPU wakeup signals (x=1 to 6)
pwr_por_rst	Digital output	Power-on reset
pwr_bor_rst	Digital output	Brownout reset
exti_c1_wkup	Digital input	CPU1 wakeup request
exti_c2_wkup	Digital input	CPU2 wakeup request
exti_d3_wkup	Digital input	D3 domain wakeup request
pwr_d1_wkup	Digital output	D1 domain bus matrix clock wakeup request
pwr_d2_wkup	Digital output	D2 domain bus matrix clock wakeup request
pwr_d3_wkup	Digital output	D3 domain bus matrix clock wakeup request

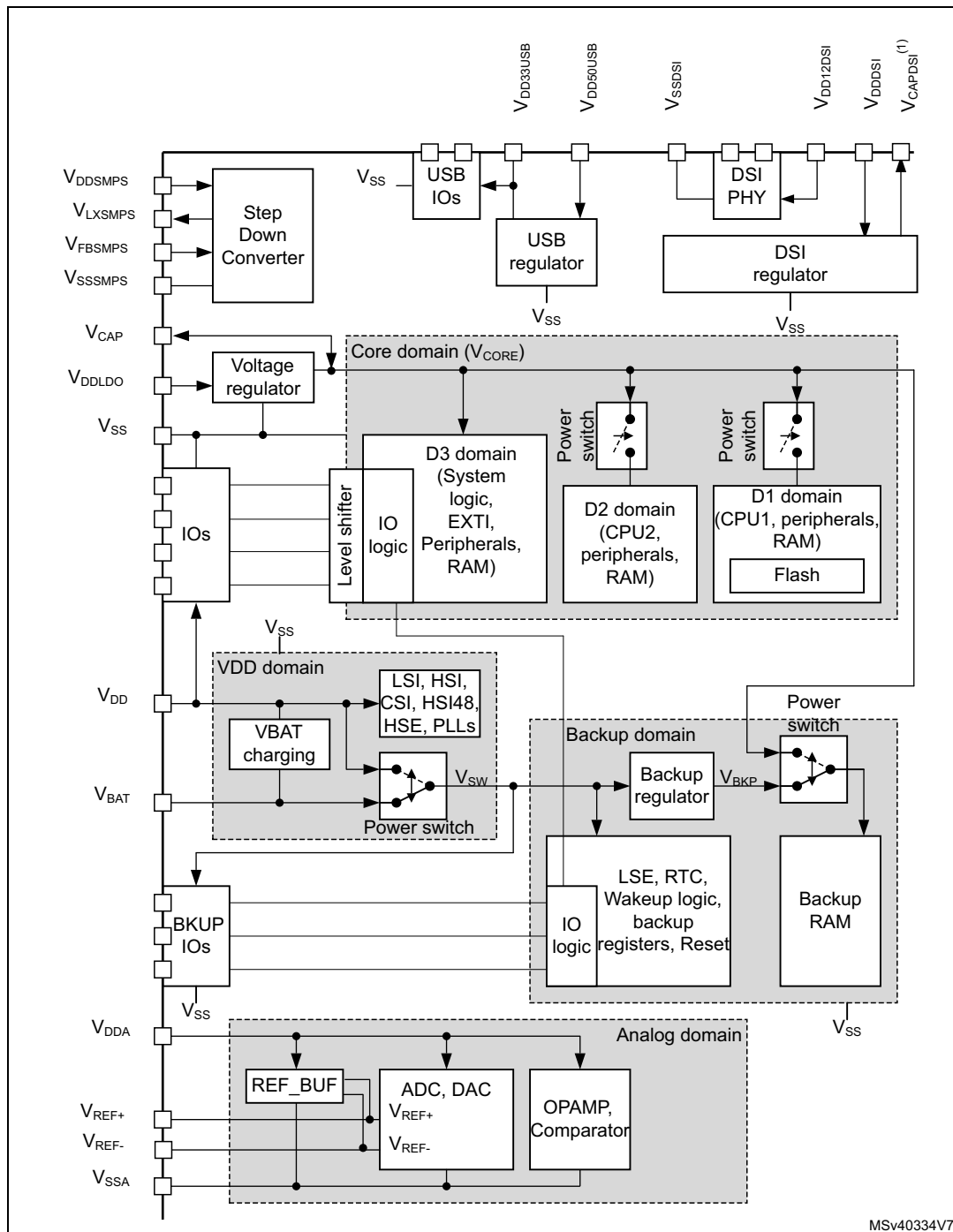
7.4 Power supplies

The device requires V_{DD} and V_{DDSMPS} power supplies as well as independent supplies for V_{DDLDO} , V_{DDA} , V_{DDUSB} , V_{DDDSI} , and V_{CAP} . It also provides regulated supplies for specific functions (step-down converter, voltage regulator, USB regulator, DSI regulator).

- V_{DD} external power supply for I/Os and system analog blocks such as reset, power management and oscillators
- V_{BAT} optional external power supply for backup domain when V_{DD} is not present (V_{BAT} mode)
This power supply shall be connected to V_{DD} when no battery is used.
- V_{DDSMPS} external power supply for step-down converter
This power supply shall be connected to V_{DD} when SMPS is used. Otherwise, it must be connected to GND.
- V_{LXSMPS} step-down converter supply output
- V_{FBSMPS} is the step-down converter sense feedback
- V_{SSSMPS} separate step-down converter ground
- V_{DDLDO} external power supply for voltage regulator
- V_{CAP} digital core domain supply
This power supply is independent from all the other power supplies:
 - When the voltage regulator is enabled, V_{CORE} is delivered by the internal voltage regulator.
 - When the voltage regulator is disabled, V_{CORE} is delivered by an external power supply through V_{CAP} pin, or by the SMPS step-down converter.
- V_{DDA} external analog power supply for ADCs, DACs, OPAMPs, comparators and voltage reference buffers
This power supply is independent from all the other power supplies.
- V_{REF+} external reference voltage for ADC and DAC.
 - When the voltage reference buffer is enabled, V_{REF+} and V_{REF-} are delivered by the internal voltage reference buffer.
 - When the voltage reference buffer is disabled, V_{REF+} is delivered by an independent external reference supply.
- V_{SSA} separate analog and reference voltage ground.
- $V_{DD50USB}$ external power supply for USB regulator.
- $V_{DD33USB}$ USB regulator supply output for USB interface.
 - When the USB regulator is enabled, $V_{DD33USB}$ is delivered by the internal USB regulator.
 - When the USB regulator is disabled, $V_{DD33USB}$ is delivered by an independent external supply input.
- V_{DDDSI} external power supply for DSI regulator.
- $V_{DD12DSI}$ DSI PHY supply input.
- V_{CAPDSI} : DSI regulator output (1.2 V) which must be externally connected to $V_{DD12DSI}$.
- V_{SSDSI} separate DSI ground.
- V_{SS} common ground for all supplies except for step-down converter, analog and DSI regulator.

Note: Depending on the operating power supply range, some peripherals might be used with limited features and performance. For more details, refer to section “General operating conditions” of the device datasheets.

Figure 21. Power supply overview



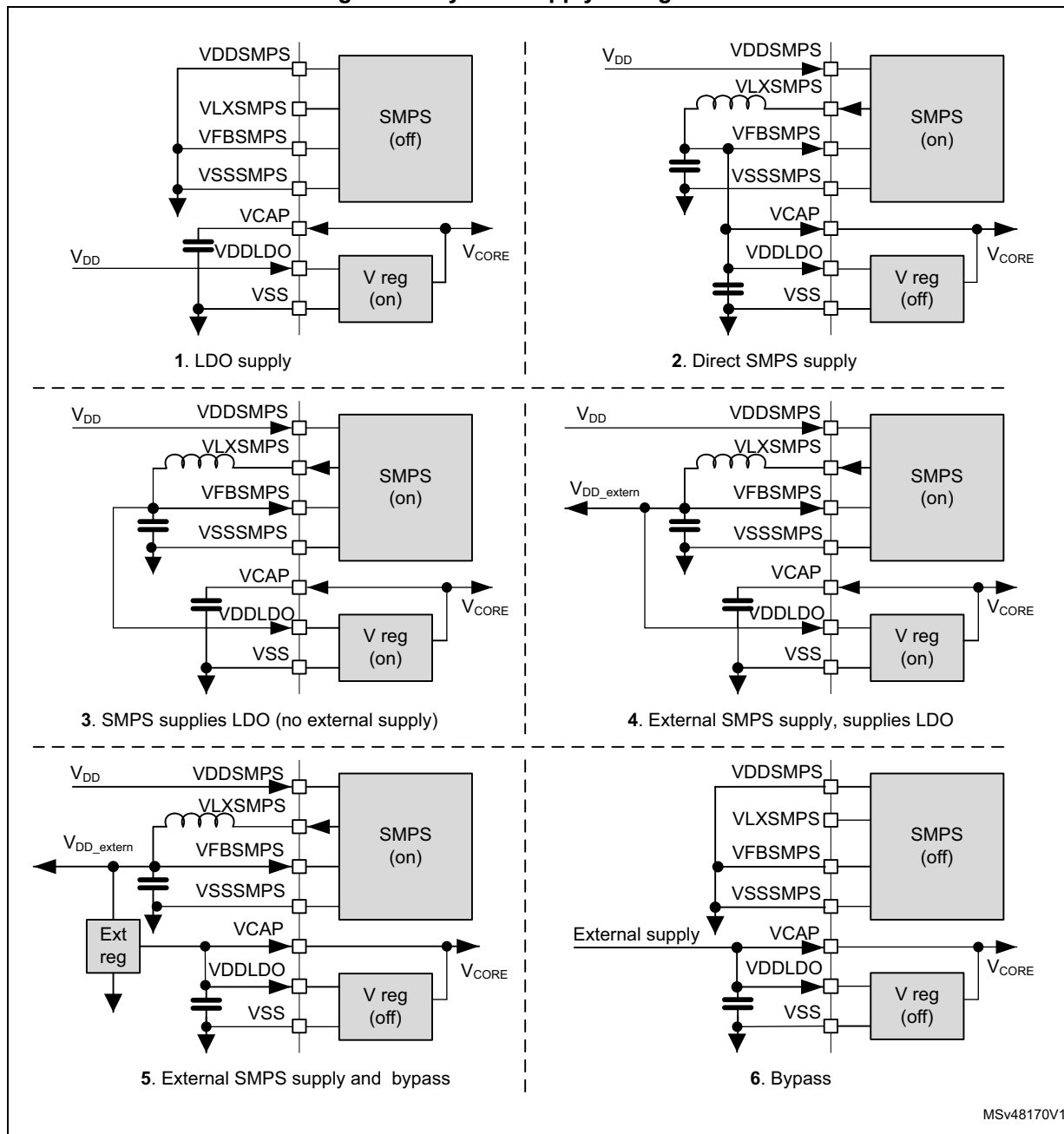
MSv40334V7

1. V_{CAPDSI} pin must be externally connected to $V_{DD12DSI}$ pin.

By configuring the SMPS step-down converter and voltage regulator, the supply configurations shown in [Figure 22](#) are supported for the V_{CORE} core domain and an external supply.

Note: The SMPS step-down converter is not available on all packages.

Figure 22. System supply configurations



The different supply configurations are controlled through the LDOEN, SDEN, SDEXTHP, SDLEVEL and BYPASS bits in *PWR control register 3 (PWR_CR3)* register according to [Table 33](#).

Table 33. Supply configuration control

ID	Supply configuration	SDLEVEL	SDEXTHP	SDEN	LDOEN	BYPASS	Description
0	Default configuration	00	0	1	1	0	– V_{CORE} Power Domains are supplied from the LDO according to VOS. – SMPS step-down converter enabled at 1.2V, may be used to supply the LDO.
1	LDO supply	x	x	0	1	0	– V_{CORE} Power Domains are supplied from the LDO according to VOS. – LDO power mode (Main, LP, Off) will follow system low-power modes. – SMPS step-down converter disabled.
2	Direct SMPS step-down converter supply	x	0	1	0	0	– V_{CORE} Power Domains are supplied from SMPS step-down converter according to VOS. – LDO bypassed. – SMPS step-down converter power mode (MR, LP, Off) will follow system low-power modes.
3	SMPS step-down converter supplies LDO,	01 or 10	0	1	1	0	– V_{CORE} Power Domains are supplied from the LDO according to VOS – LDO power mode (Main, LP, Off) will follow system low-power modes. – SMPS step-down converter enabled according to SDLEVEL, and supplies the LDO. – SMPS step-down converter power mode (MR, LP, Off) will follow system low-power modes.
4	SMPS step-down converter supplies External and LDO	01 or 10	1	1	1	0	– V_{CORE} Power Domains are supplied from voltage regulator according to VOS – LDO power mode (Main, LP, Off) will follow system low-power modes. – SMPS step-down converter enabled according to SDLEVEL used to supply external circuits and may supply the LDO. – SMPS step-down converter forced ON in MR mode.
5	SMPS step-down converter supplies external and LDO Bypass	01 or 10	1	1	0	1	– V_{CORE} supplied from external source – SMPS step-down converter enabled according to SDLEVEL used to supply external circuits and may supply the external source for V_{CORE}. – SMPS step-down converter forced ON in MR mode.

Table 33. Supply configuration control (continued)

ID	Supply configuration	SDLEVEL	SDEXTHP	SDEN	LDOEN	BYPASS	Description
6	SMPS step-down converter disabled and LDO Bypass	x	x	0	0	1	– V_{CORE} supplied from external source – SMPS step-down converter disabled and LDO bypassed, voltage monitoring still active.
NA	Illegal	x	x	0	0	0	– Illegal combination, the default configuration is kept. (write data will be ignored).
		x	x	x	1	1	
		x	0	1	0	1	
		00	x	1	1	0	
		x	1	1	0	0	
		00	1	1	0	1	

7.4.1 System supply startup

The system startup sequence from power-on in different supply configurations is the following (see [Figure 23](#) and [Figure 24](#) for LDO supply and Direct SMPS supply, respectively):

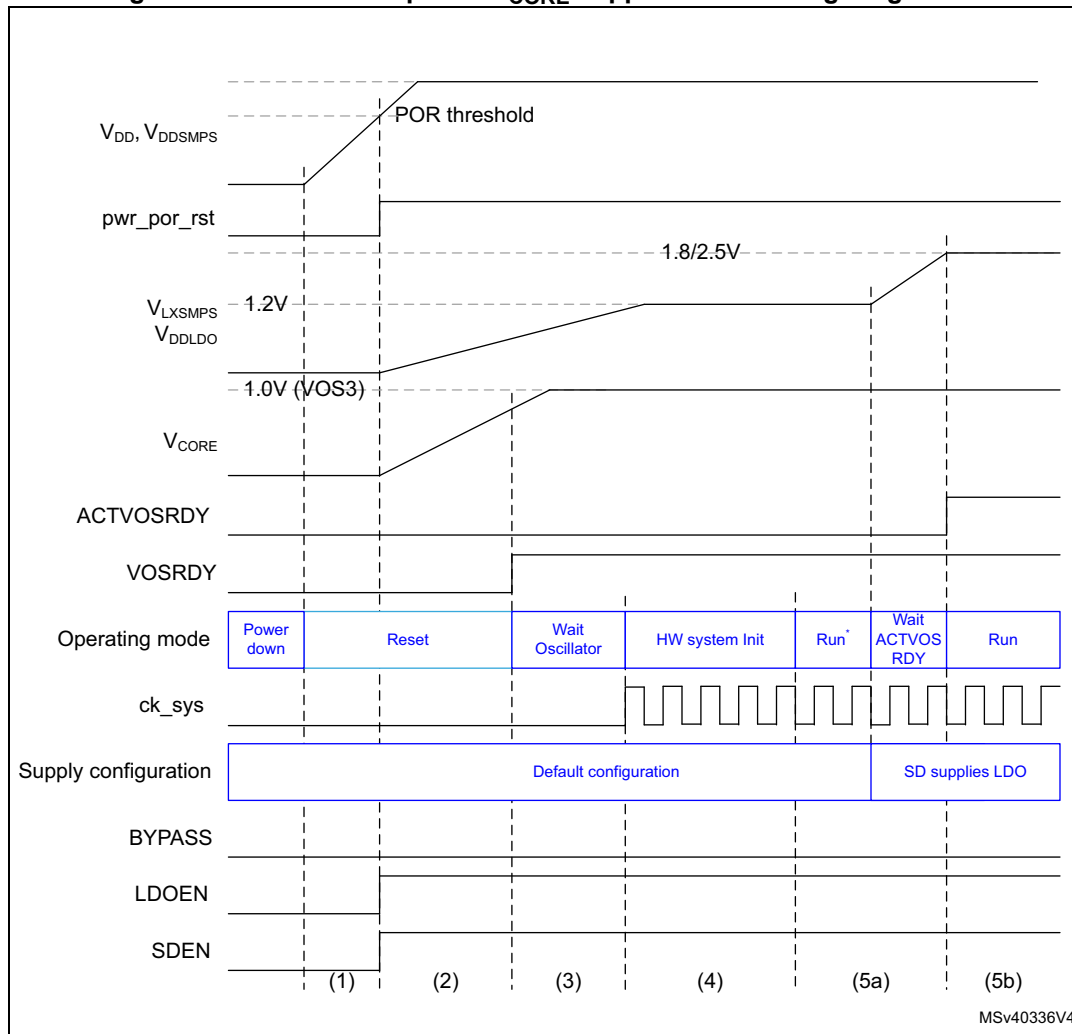
- When the system is powered on, the POR monitors V_{DD} supply. Once V_{DD} is above the POR threshold level, the SMPS step-down converter and voltage regulator are enabled in the default supply configuration:
 - The SMPS step-down converter output level is set at 1.2 V.
 - The voltage regulator output level is set at 1.0 V in accordance with the VOS3 level configured in [PWR D3 domain control register \(PWR_D3CR\)](#).
- The system is kept in reset mode as long as V_{CORE} is not ok.
- Once V_{CORE} is ok, the system is taken out of reset and the HSI oscillator is enabled.
- Once the oscillator is stable, the system is initialized: Flash memory and option bytes are loaded and the CPU starts in limited run mode (Run*).
- The software shall then initialize the system including supply configuration programming in [PWR control register 3 \(PWR_CR3\)](#). Once the supply configuration has been configured, the ACTVOSRDY bit in [PWR control status register 1 \(PWR_CSR1\)](#) shall be checked to guarantee valid voltage levels:
 - As long as ACTVOSRDY indicates that voltage levels are invalid, the system is in Run* mode, write accesses to the RAMs are not permitted and VOS shall not be changed.
 - Once ACTVOSRDY indicates that voltage levels are valid, the system is in normal Run mode, write accesses to RAMs are allowed and VOS can be changed.

V_{CORE} supplied from the voltage regulator (LDO)

When V_{CORE} is supplied from the voltage regulator (LDO), the V_{CORE} voltage settles directly at VOS3 level. However the SMPS step-down converter V_{LXSMPS} output voltage is set at 1.2 V. ACTVOSRDY bit in *PWR control status register 1 (PWR_CSR1)* indicates that the voltage levels are invalid.

The software has to program the supply configuration in *PWR control register 3 (PWR_CR3)*. In addition, the V_{LXSMPS} voltage level shall reach the programmed SDLEVEL so that ACTVOSRDY indicates valid voltage levels (see *Figure 23*).

Figure 23. Device startup with V_{CORE} supplied from voltage regulator

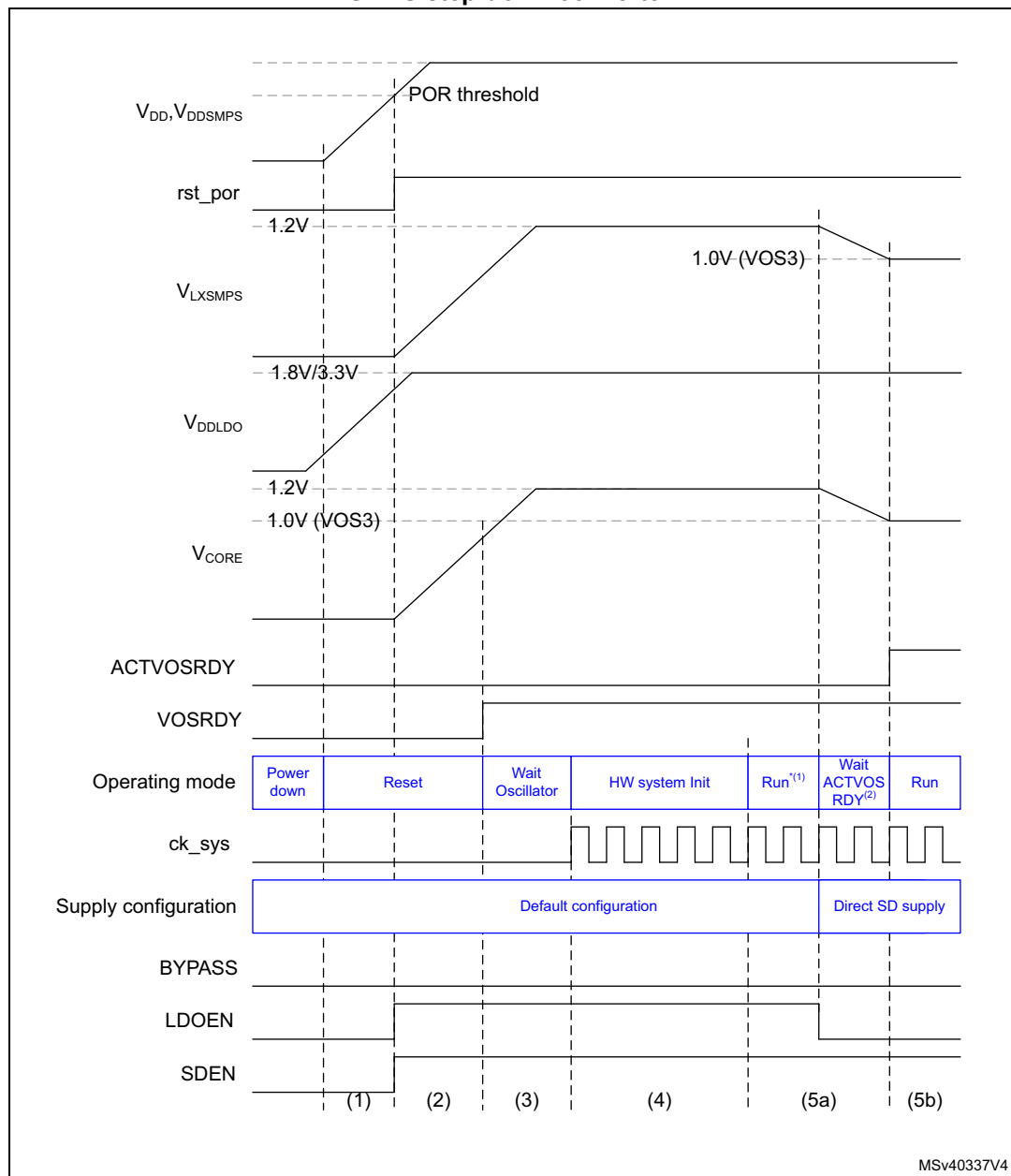


1. In Run* mode, write operations to RAM are not allowed.
2. Write operations to RAM are allowed and VOS can be changed only when ACTVOSRDY is valid.

V_{CORE} directly supplied from the SMPS step-down converter

When V_{CORE} is directly supplied from the SMPS step-down converter, the V_{CORE} voltage first settles at the SMPS step-down converter default level (1.2 V). Due to a too high supply compared to the VOS3 level, the ACTVOSRDY bit in [PWR control status register 1 \(PWR_CSR1\)](#) indicates invalid voltage levels. V_{CORE} settles at 1.0 V (VOS3 level) and ACTVOSRDY indicates valid voltage levels only when the supply configuration has been programmed in [PWR control register 3 \(PWR_CR3\)](#) (see [Figure 24](#)).

Figure 24. Device startup with V_{CORE} supplied directly from SMPS step-down converter



1. In Run* mode, write operations to RAM are not allowed.

2. Write operations to RAM are allowed and VOS can be changed only when ACTVOSRDY is valid.

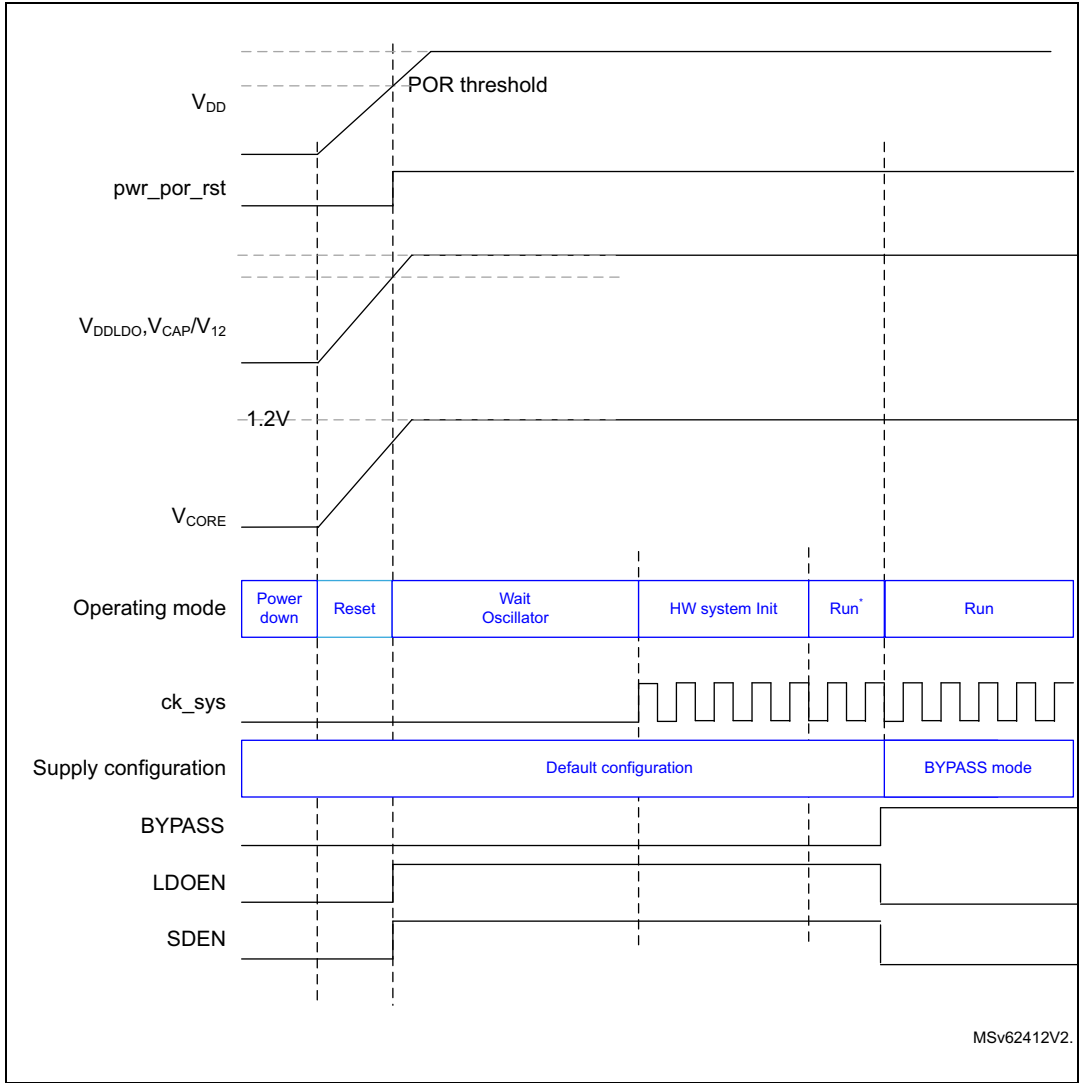
When exiting from Standby mode, the supply configuration is known by the system since the [PWR control register 3 \(PWR_CR3\)](#) register content is retained. However the software shall still wait for the ACTVOSRDY bit to be set in [PWR control status register 1 \(PWR_CSR1\)](#) to indicate V_{CORE} voltage levels are valid, before performing write accesses to RAM or changing VOS.

V_{CORE} supplied in Bypass mode (LDO and SMPS OFF)

For packages where VDDLDO is internally connected to VDD, when V_{CORE} is supplied in Bypass mode (LDO OFF), the V_{CORE} voltage must first settle at a default level higher than 1.1 V. Due to the LDO default state after power-up (enabled by default), the external V_{CORE} voltage must remain higher than 1.1 V until the LDO is disabled by software.

When the LDO is disabled, the external V_{CORE} voltage can be adjusted according to the user application needs (refer to section *General operating conditions* of the datasheet for details on V_{CORE} level versus the maximum operating frequency).

Figure 25. Device startup with V_{CORE} supplied in Bypass mode from external regulator



MSv62412V2.

7.4.2 Core domain

The V_{CORE} core domain supply can be provided by the SMPS step-down converter, voltage regulator or by an external supply (V_{CAP}). V_{CORE} supplies all the digital circuitries except for the backup domain and the Standby circuitry. The V_{CORE} domain is split into 3 sections:

- D1 domain containing the CPU (Cortex[®]-M7), Flash memory and peripherals.
- D2 domain containing peripherals and a Cortex[®]-M4 CPU.
- D3 domain containing the system control, I/O logic and low-power peripherals.

When a system reset occurs, the voltage regulator is enabled and supplies V_{CORE} . The SMPS step-down converter is also enabled to deliver 1.2 V. This allows the system to start up in any supply configurations (see [Figure 22](#)).

After a system reset, the software shall configure the used supply configuration in [PWR control register 3 \(PWR_CR3\)](#) register before changing VOS in [PWR D3 domain control register \(PWR_D3CR\)](#) or the RCC ck_sys frequency. The different system supply configurations are controlled as shown in [Table 33](#).

Note: The SMPS step-down converter is not available on all packages.

Voltage regulator

The embedded voltage regulator (LDO) requires external capacitors to be connected to V_{CAP} pins.

The voltage regulator provides three different operating modes: Main (MR), Low-power (LP) or Off. These modes will be used depending on the system operating modes (Run, Stop and Standby).

- Run mode

The LDO regulator is in Main mode and provides full power to the V_{CORE} domain (core, memories and digital peripherals). The regulator output voltage can be scaled by software to different voltage levels (VOS0^(a), VOS1, VOS2, and VOS3) that are configured through VOS bits in [PWR D3 domain control register \(PWR_D3CR\)](#). The VOS voltage scaling allows optimizing the power consumption when the system is clocked below the maximum frequency. By default VOS3 is selected after system reset. VOS can be changed on-the-fly to adapt to the required system performance.

- Stop mode

The voltage regulator supplies the V_{CORE} domain to retain the content of registers and internal memories.

The regulator can be kept in Main mode to allow fast exit from Stop mode, or can be set in LP mode to obtain a lower V_{CORE} supply level and extend the exit-from-Stop latency. The regulator mode is selected through the SVOS and LPDS bits in [PWR control register 1 \(PWR_CR1\)](#). Main mode and LP mode are allowed if SVOS3 voltage scaling is selected, while only LP mode is possible for SVOS4 and SVOS5 scaling. Due to a

a. VOS0 corresponds to V_{CORE} boost allowing to reach the system maximum frequency (refer to [Section : VOS0 activation/deactivation sequence](#))

lower voltage level for SVOS4 and SVOS5 scaling, the Stop mode consumption can be further reduced.

- Standby mode

The voltage regulator is OFF and the V_{CORE} domains are powered down. The content of the registers and memories is lost except for the Standby circuitry and the backup domain.

Note: For more details, refer to the voltage regulator section in the datasheets.

SMPS step-down converter regulator

The SMPS step-down converter requires an external coil to be connected between the dedicated V_{LXSMP} pin and, via a capacitor, to V_{SS} .

The SMPS step-down converter can be used in internal supply mode or external supply mode. The internal supply mode is used to directly supply the V_{CORE} domain, while the external supply mode is used to generate an intermediate supply level ($V_{\text{DD_extern}}$ at 1.8 or 2.5 V) which can supply the voltage regulator and optionally an external circuitry.

The SMPS step-down converter works in three different power modes: Main (MR), Low-power (LP) or Off.

When the SMPS step-down converter is used in internal supply mode, the converter operating modes depend on the system modes (Run, Stop, Standby) and are configured through the associated VOS and SVOS levels:

- Run mode

The SMPS step-down converter operates in MR mode and provides full power to the V_{CORE} domain (core, memories and digital peripherals). The regulator output voltage can be scaled by software to different voltage levels (VOS0, VOS1, VOS2, and VOS3) that are configured through VOS bits in [PWR D3 domain control register \(PWR_D3CR\)](#). The VOS voltage scaling allows optimizing the power consumption when the system is clocked below the maximum frequency. By default VOS3 is selected after system reset. VOS can be changed on-the-fly to adapt to the required system performance.

- Stop mode

The SMPS step-down converter supplies the V_{CORE} domain to retain the content of registers and internal memories. The regulator can be kept in MR mode to allow fast exit from Stop mode, or can be set in LP mode to achieve a lower V_{CORE} supply level and extend the exit-from-Stop latency. The regulator mode is selected through the SVOS and LPDS bits in [PWR control register 1 \(PWR_CR1\)](#). MR mode or LP mode are allowed if SVOS3 voltage scaling is selected, while only LP mode is possible for SVOS4 and SVOS5 scaling. Due to a lower voltage level for SVOS4 and SVOS5 scaling, the Stop mode consumption can be further reduced.

- Standby mode

The SMPS step-down converter is OFF and the V_{CORE} domains are powered down. The content of the registers and memories are lost except for the Standby circuitry and the backup domain.

When the SMPS step-down converter supplies an external circuitry by generating an intermediate voltage level, the converter is forced ON and operates in MR mode. The intermediate voltage level is selected through SDLEVEL bits in [PWR control register 3 \(PWR_CR3\)](#). $V_{\text{DD_extern}}$ is supplied at all times with full power whatever the system modes (Run, Stop, Standby).

Note: The SMPS step-down converter is not available on all packages.

7.4.3 PWR external supply

When V_{CORE} is supplied from an external source, different operating modes can be used depending on the system operating modes (Run, Stop or Standby):

- In Run mode
The external source supplies full power to the V_{CORE} domain (core, memories and digital peripherals). The external source output voltage is scalable through different voltage levels (VOS0, VOS1, VOS2 and VOS3). The externally applied voltage level shall be reflected in the VOS bits of PWR_D3CR register. The RAMs shall only be accessed for write operations when the external applied voltage level matches VOS settings.
- In Stop mode
The external source supplies V_{CORE} domain to retain the content of registers and internal memories. The regulator can select a lower V_{CORE} supply level to reduce the consumption in Stop mode.
- In Standby mode
The external source shall be switched OFF and the V_{CORE} domains powered down. The content of registers and memories is lost except for the Standby circuitry and the backup domain. The external source shall be switched ON when exiting Standby mode.

7.4.4 Backup domain

To retain the content of the backup domain (RTC, backup registers and backup RAM) when V_{DD} is turned off, V_{BAT} pin can be connected to an optional standby voltage which is supplied from a battery or from an another source.

The switching to V_{BAT} is controlled by the power-down reset embedded in the Reset block that monitors the V_{DD} supply.

Warning: During $t_{RSTTEMPO}$ (temporization at V_{DD} startup) or after a PDR is detected, the power switch between V_{BAT} and V_{DD} remains connected to V_{BAT} .
During the a startup phase, if V_{DD} is established in less than $t_{RSTTEMPO}$ (see the datasheet for the value of $t_{RSTTEMPO}$) and $V_{DD} > V_{BAT} + 0.6\text{ V}$, a current may be injected into V_{BAT} through an internal diode connected between V_{DD} and the power switch (V_{BAT}).
If the power supply/battery connected to the V_{BAT} pin cannot support this current injection, it is strongly recommended to connect an external low-drop diode between this power supply and the V_{BAT} pin.

When the V_{DD} supply is present, the backup domain is supplied from V_{DD} . This allows saving V_{BAT} power supply battery life time.

If no external battery is used in the application, it is recommended to connect V_{BAT} to V_{DD} supply and to add a 100 nF ceramic decoupling capacitor on the VBAT pin.

When the V_{DD} supply is present and higher than the PDR threshold, the backup domain is supplied by V_{DD} and the following functions are available:

- PC14 and PC15 can be used either as GPIO or as LSE pins.
- PC13 can be used either as GPIO or as RTC_AF1 or RTC_TAMP1 pin assuming they have been configured by the RTC.
- PI8/RTC_TAMP2 and PC1/RTC_TAMP3 when they are configured by the RTC as tamper pins.

Note: *Since the switch only sinks a limited amount of current, the use of PC13 to PC15 and PI8 GPIOs is restricted: only one I/O can be used as an output at a time, at a speed limited to 2 MHz with a maximum load of 30 pF. These I/Os must not be used as current sources (e.g. to drive an LED).*

In V_{BAT} mode, when the V_{DD} supply is absent and a supply is present on V_{BAT} , the backup domain is supplied by V_{BAT} and the following functions are available:

- PC14 and PC15 can be used as LSE pins only.
- PC13 can be used as RTC_AF1 or RTC_TAMP1 pin assuming they have been configured by the RTC.
- PI8/RTC_TAMP2 and PC1/RTC_TAMP3 when they are configured by the RTC as tamper pins.

Accessing the backup domain

After reset, the backup domain (RTC registers and RTC backup registers) is protected against possible unwanted write accesses. To enable access to the backup domain, set the DBP bit in the [PWR control register 1 \(PWR_CR1\)](#).

For more detail on RTC and backup RAM access, refer to [Section 9: Reset and Clock Control \(RCC\)](#).

Backup RAM

The backup domain includes 4 Kbytes of backup RAM accessible in 32-bit, 16-bit or 8-bit data mode. The backup RAM is supplied from the Backup regulator in the backup domain. When the Backup regulator is enabled through BREN bit in [PWR control register 2 \(PWR_CR2\)](#), the backup RAM content is retained even in Standby and/or V_{BAT} mode (it can be considered as an internal EEPROM if V_{BAT} is always present.)

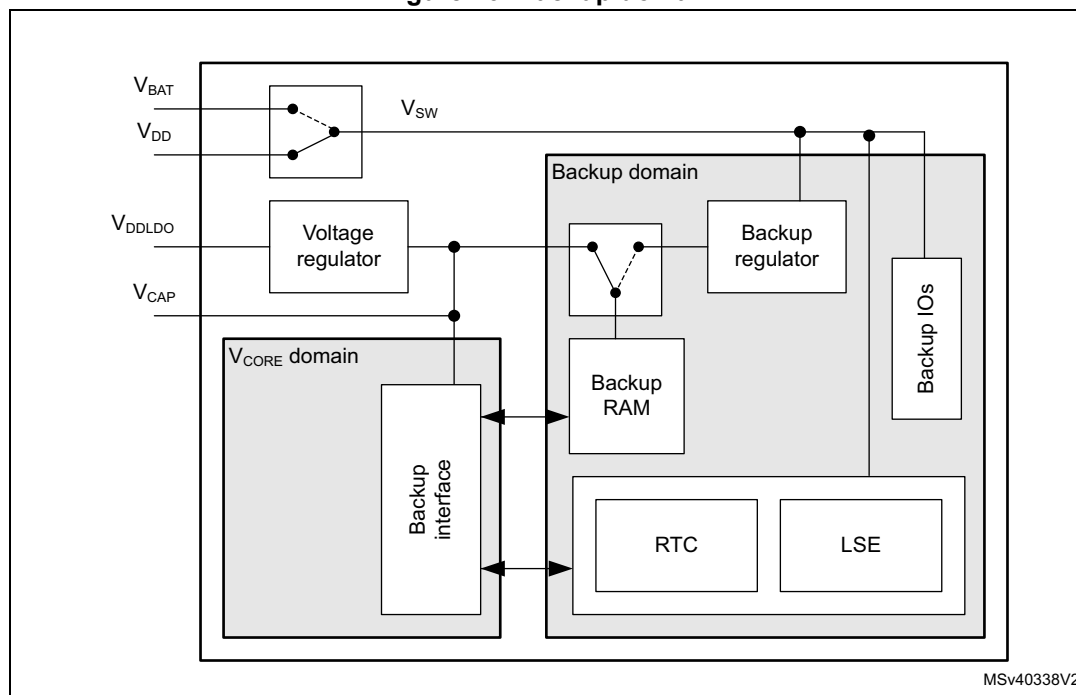
The Backup regulator can be ON or OFF depending whether the application needs the backup RAM function in Standby or V_{BAT} modes.

The backup RAM is not mass erased by a tamper event, instead it is read protected to prevent confidential data, such as cryptographic private key, from being accessed. To re-

gain access to the backup RAM after a tamper event, the memory area needs to be first erased. The backup RAM can be erased:

- through the Flash interface when a protection level change from level 1 to level 0 is requested (refer to the description of Read protection (RDP) in the Flash programming manual).
- After a tamper event, by performing a dummy write with zero as data to the backup RAM.

Figure 26. Backup domain



7.4.5 V_{BAT} battery charging

When V_{DD} is present, the external battery connected to V_{BAT} can be charged through an internal resistance.

V_{BAT} charging can be performed either through a 5 k Ω resistor or through a 1.5 k Ω resistor, depending on the VBRS bit value in [PWR control register 3 \(PWR_CR3\)](#).

The battery charging is enabled by setting the VBE bit in [PWR control register 3 \(PWR_CR3\)](#). It is automatically disabled in V_{BAT} mode.

7.4.6 Analog supply

Separate V_{DDA} analog supply

The analog supply domain is powered by dedicated V_{DDA} and V_{SSA} pads that allow the supply to be filtered and shielded from noise on the PCB, thus improving ADC and DAC conversion accuracy:

- The analog supply voltage input is available on a separate V_{DDA} pin.
- An isolated supply ground connection is provided on V_{SSA} pin.

Analog reference voltage V_{REF+}/V_{REF-}

To achieve better accuracy low-voltage signals, the ADC and DAC also have a separate reference voltage, available on V_{REF+} pin. The user can connect a separate external reference voltage on V_{REF+} .

The V_{REF+} controls the highest voltage, represented by the full scale value, the lower voltage reference (V_{REF-}) being connected to V_{SSA} .

When enabled by ENVR bit in the VREFBUF control and status register (see [Section 28: Voltage reference buffer \(VREFBUF\)](#)), V_{REF+} is provided from the internal voltage reference buffer. The internal voltage reference buffer can also deliver a reference voltage to external components through V_{REF+}/V_{REF-} pins.

When the internal voltage reference buffer is disabled by ENVR, V_{REF+} is delivered by an independent external reference supply voltage.

7.4.7 USB regulator

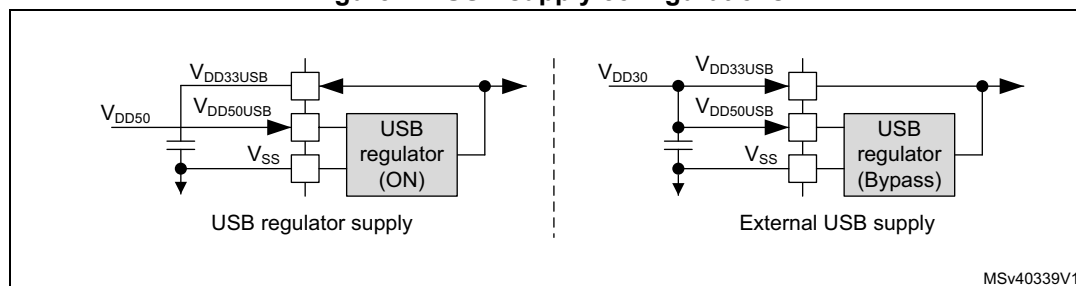
The USB transceivers are supplied from a dedicated $V_{DD33USB}$ supply that can be provided either by the integrated USB regulator, or by an external USB supply.

When enabled by USBREGEN bit in [PWR control register 3 \(PWR_CR3\)](#), the $V_{DD33USB}$ is provided from the USB regulator. Before using $V_{DD33USB}$, check that it is available by monitoring USB33RDY bit in [PWR control register 3 \(PWR_CR3\)](#). The $V_{DD33USB}$ supply level detector shall be enabled through USB33DEN bit in PWR_CR3 register.

When the USB regulator is disabled through USBREGEN bit, $V_{DD33USB}$ can be provided from an external supply. In this case $V_{DD33USB}$ and $V_{DD50USB}$ shall be connected together. The $V_{DD33USB}$ supply level detector must be enabled through USB33DEN bit in PWR_CR3 register before using the USB transceivers.

For more information on the USB regulator (see [Section 60: USB on-the-go high-speed \(OTG_HS\)](#)).

Figure 27. USB supply configurations



7.4.8 DSI regulator

The DSI interface is supplied from a dedicated $V_{DD12DSI}$ supply that can be provided either by the integrated DSI regulator or by an external DSI supply.

When enabled through REGEN bit in [Section 34.16.11: DSI Wrapper regulator and PLL control register \(DSI_WRPCR\)](#), $V_{DD12DSI}$ is delivered by the DSI regulator.

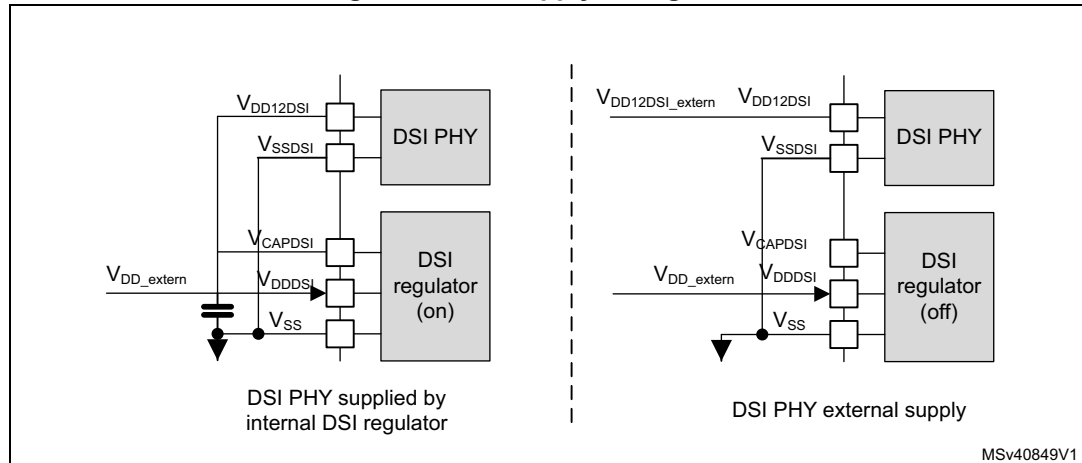
When the DSI regulator is disabled (REGEN = '0'), $V_{DD12DSI}$ can be delivered by an external supply.

For more information on the DSI regulator (see [Section 34: DSI Host \(DSI\)](#)).

If the DSI is not used at all:

- V_{DDDSI} pin must be connected to global VDD
- V_{CAPDSI} pin must be connected externally to $V_{DD12DSI}$ when both V_{CAPDSI} and $V_{DD12DSI}$ pins are available. The external capacitor is no more needed. When $V_{DD12DSI}$ pin is not available, the V_{CAPDSI} pin can be left floating.
- V_{SSDSI} pin must be grounded.

Figure 28. DSI supply configuration



7.5 Power supply supervision

Power supply level monitoring is available on the following supplies:

- V_{DD} (V_{DDSMPS}) via POR/PDR (see [Section 7.5.1](#)), BOR (see [Section 7.5.2](#)) and PVD monitor (see [Section 7.5.3](#))
- V_{DDA} via AVD monitor (see [Section 7.5.4](#))
- V_{BAT} via VBAT threshold (see [Section 7.5.5](#))
- V_{SW} via `rst_vsw`, which keeps V_{SW} domain in Reset mode as long as the level is not OK.
- V_{BKP} via a `BRRDY` bit in *PWR control register 2 (PWR_CR2)*.
- V_{FBSMPS} via a `SDEXTRDY` bit in *PWR control register 3 (PWR_CR3)*.
- $V_{DD33USB}$ via `USBRDY` bit in *PWR control register 3 (PWR_CR3)*.
- V_{CAPDSI} via a `RRS` in the DSI block in [Section 34.16.11: DSI Wrapper regulator and PLL control register \(DSI_WRPCR\)](#).

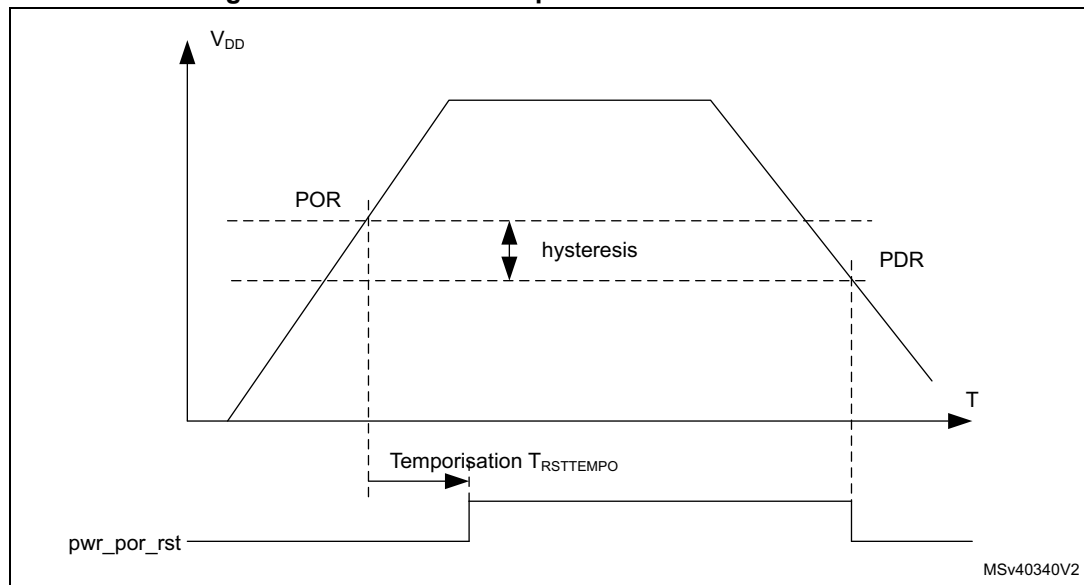
7.5.1 Power-on reset (POR)/power-down reset (PDR)

The system has an integrated POR/PDR circuitry that ensures proper startup operation.

The system remains in Reset mode when V_{DD} is below a specified V_{POR} threshold, without the need for an external reset circuit. Once the V_{DD} supply level is above the V_{POR} threshold, the system is taken out of reset (see [Figure 29](#)). For more details concerning the power-on/power-down reset threshold, refer to the electrical characteristics section of the datasheets.

The PDR can be enabled/disabled by the device PDR_ON input pin.

Figure 29. Power-on reset/power-down reset waveform



1. For thresholds and hysteresis values, please refer to the datasheets.

7.5.2 Brownout reset (BOR)

During power-on, the Brownout reset (BOR) keeps the system under reset until the V_{DD} supply voltage reaches the specified V_{BOR} threshold.

The V_{BOR} threshold is configured through system option bytes. By default, BOR is OFF. The following programmable V_{BOR} thresholds can be selected:

- BOR OFF (V_{BOR0})
- BOR Level 1 (V_{BOR1})
- BOR Level 2 (V_{BOR2})
- BOR Level 3 (V_{BOR3})

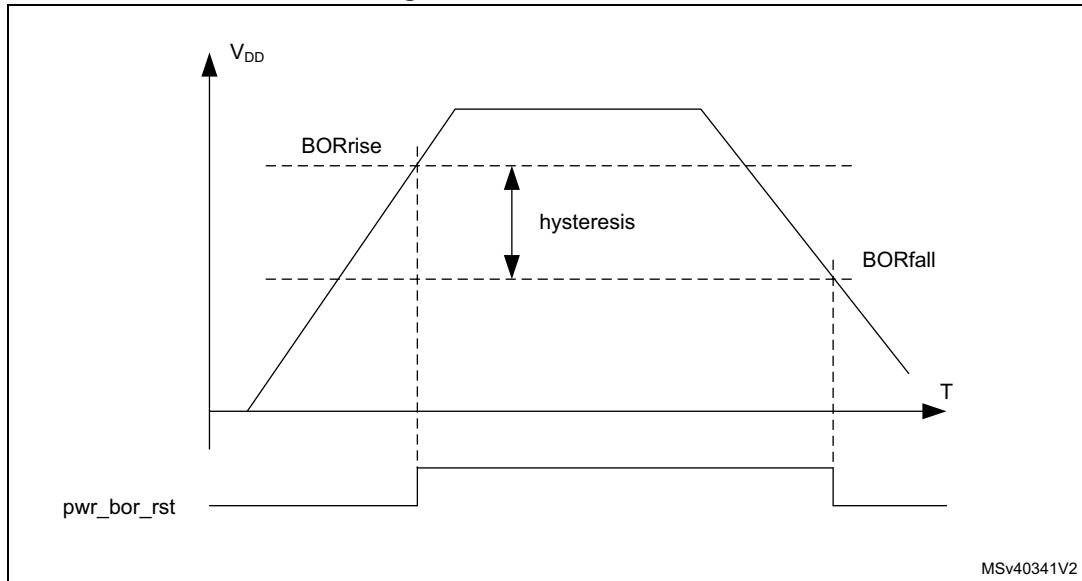
For more details on the brown-out reset thresholds, refer to the section “Electrical characteristics” of the product datasheets.

A system reset is generated when the BOR is enabled and V_{DD} supply voltage drops below the selected V_{BOR} threshold.

BOR can be disabled by programming the system option bytes. To disable the BOR function, V_{DD} must have been higher than V_{BOR0} to start the system option byte

programming sequence. The power-down is then monitored by the PDR (see [Section 7.5.1](#)).

Figure 30. BOR thresholds



1. For thresholds and hysteresis values, please refer to the datasheets.

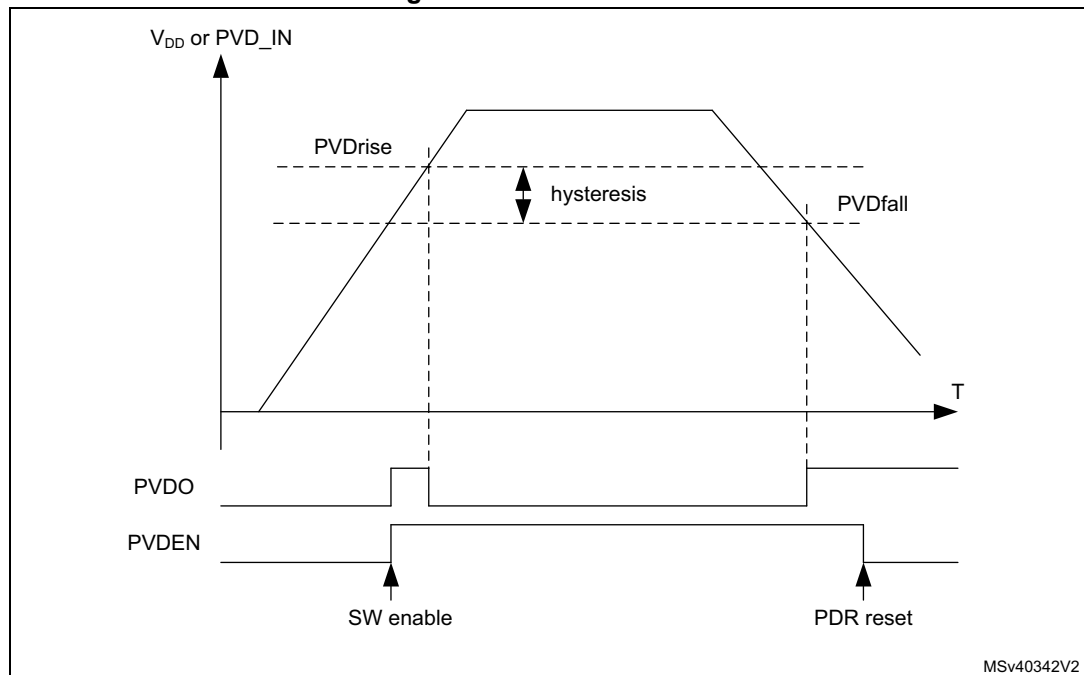
7.5.3 Programmable voltage detector (PVD)

The PVD can be used to monitor the V_{DD} power supply by comparing it to a threshold selected by the PLS[2:0] bits in the [PWR control register 1 \(PWR_CR1\)](#). The PVD can also be used to monitor a voltage level on the PVD_IN pin. In this case PVD_IN voltage is compared to the internal V_{REFINT} level.

The PVD is enabled by setting the PVDE bit in [PWR control register 1 \(PWR_CR1\)](#).

A PVDO flag is available in the [PWR control status register 1 \(PWR_CSR1\)](#) to indicate if V_{DD} or PVD_IN voltage is higher or lower than the PVD threshold. This event is internally connected to the EXTI and can generate an interrupt, assuming it has been enabled through the EXTI registers. The PVDO output interrupt can be generated when V_{DD} or PVD_IN voltage drops below the PVD threshold and/or when V_{DD} or PVD_IN voltage rises above the PVD threshold depending on EXTI rising/falling edge configuration. As an example the service routine could perform emergency shutdown tasks.

Figure 31. PVD thresholds



1. For thresholds and hysteresis values, please refer to the datasheets.

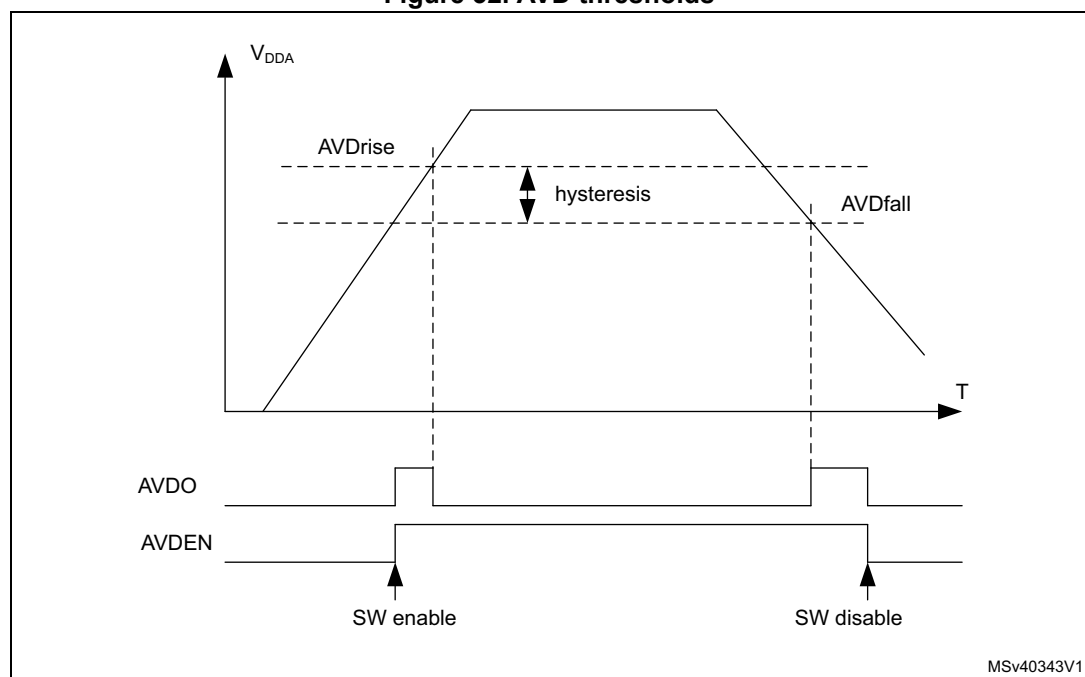
7.5.4 Analog voltage detector (AVD)

The AVD can be used to monitor the V_{DDA} supply by comparing it to a threshold selected by the ALS[1:0] bits in the *PWR control register 1 (PWR_CR1)*.

The AVD is enabled by setting the AVDEN bit in *PWR control register 1 (PWR_CR1)*.

An AVDO flag is available in the *PWR control status register 1 (PWR_CSR1)* to indicate whether V_{DDA} is higher or lower than the AVD threshold. This event is internally connected to the EXTI and can generate an interrupt if enabled through the EXTI registers. The AVDO interrupt can be generated when V_{DDA} drops below the AVD threshold and/or when V_{DDA} rises above the AVD threshold depending on EXTI rising/falling edge configuration. As an example the service routine could indicate when the V_{DDA} supply drops below a minimum level.

Figure 32. AVD thresholds



1. For thresholds and hysteresis values, please refer to the datasheets.

7.5.5 Battery voltage thresholds

The battery voltage supply monitors the backup domain V_{SW} level. V_{SW} is monitored by comparing it with two threshold levels: $V_{BAThigh}$ and V_{BATlow} . $VBATH$ and $VBATL$ flags in the [PWR control register 2 \(PWR_CR2\)](#), indicate if V_{SW} is higher or lower than the threshold.

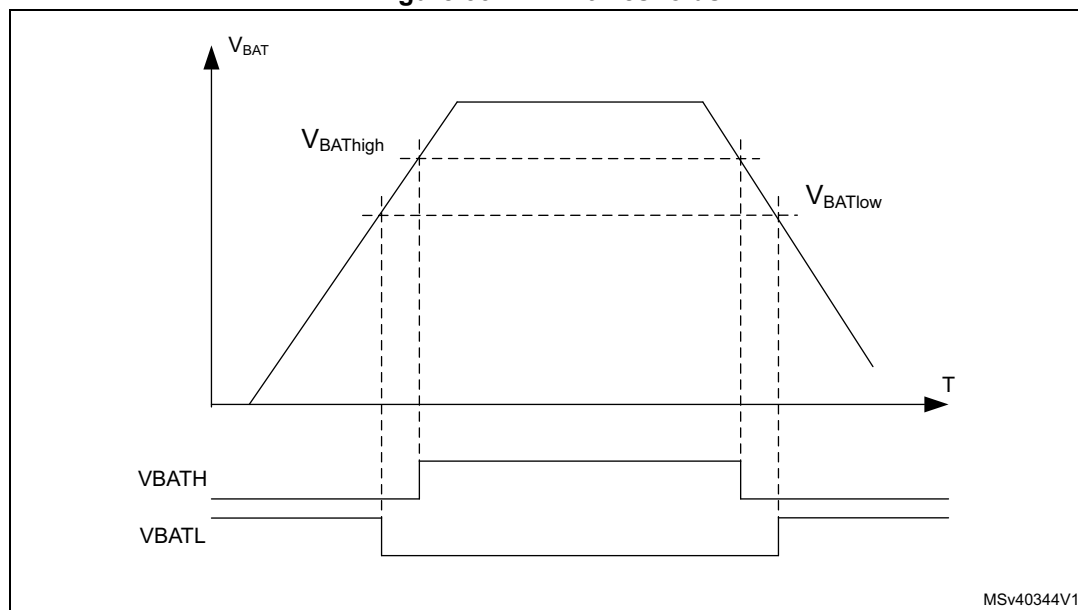
The V_{BAT} supply monitoring can be enabled/disabled via $MENEN$ bit in [PWR control register 2 \(PWR_CR2\)](#). When it is enabled, the battery voltage thresholds increase power consumption. As an example the V_{SW} levels monitoring could be used to trigger a tamper event for an over or under voltage of the RTC power supply domain (available in VBAT mode).

$VBATH$ and $VBATL$ are connected to RTC tamper signals (see [Section 49: Real-time clock \(RTC\)](#)).

Note: *Battery voltage monitoring is only available when the backup regulator is enabled ($BREN$ bit set in [PWR control register 2 \(PWR_CR2\)](#)).*

When the device does not operate in VBAT mode, the battery voltage monitoring checks V_{DD} level. When V_{DD} is available, V_{SW} is connected to V_{DD} through the internal power switch (see [Section 7.4.4: Backup domain](#)).

Figure 33. VBAT thresholds



1. For thresholds and hysteresis values, please refer to the datasheets.

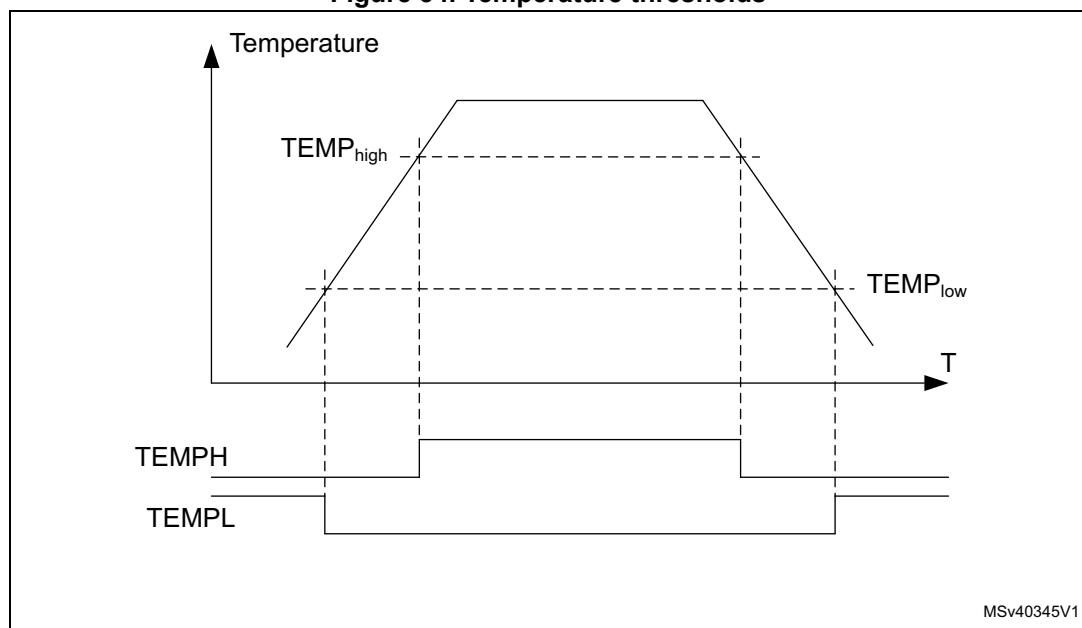
7.5.6 Temperature thresholds

The junction temperature can be monitored by comparing it with two threshold levels, $TEMP_{high}$ and $TEMP_{low}$. $TEMPH$ and $TEMPL$ flags, in the [PWR control register 2 \(PWR_CR2\)](#), indicate whether the device temperature is higher or lower than the threshold. The temperature monitoring can be enabled/disabled via $MONEN$ bit in [PWR control register 2 \(PWR_CR2\)](#). When enabled, the temperature thresholds increase power consumption. As an example the levels could be used to trigger a routine to perform temperature control tasks.

The temperature thresholds are available only when the backup regulator is enabled ($BREN$ bit set in the PWR_CR2 register).

$TEMPH$ and $TEMPL$ wakeup interrupts are available on the RTC tamper signals (see [Section 49: Real-time clock \(RTC\)](#)).

Figure 34. Temperature thresholds



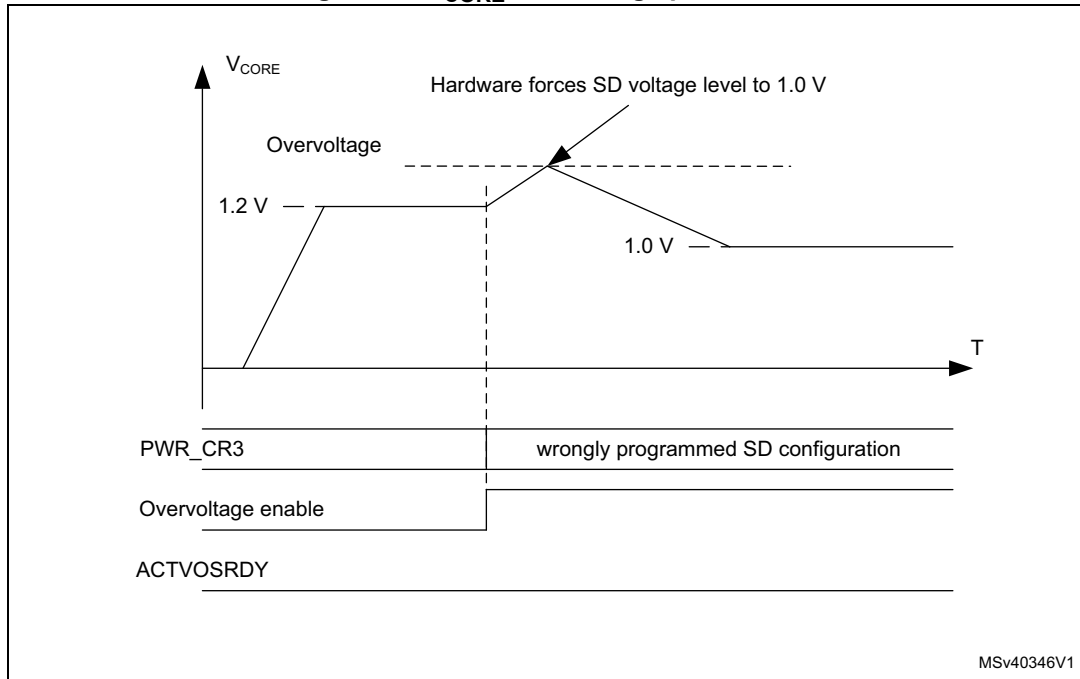
1. For thresholds and hysteresis values, please refer to the datasheets.

7.5.7 V_{CORE} maximum voltage level detector

V_{CORE} is protected against too high voltages in the direct SMPS step-down converter supply configuration. V_{CORE} overvoltage protection is enabled at startup by hardware once the SMPS step-down converter configuration has been programmed into [PWR control register 3 \(PWR_CR3\)](#):

- V_{CORE} voltage level stays within range:
 - $ACTVOSRDY$ bit in [PWR control status register 1 \(PWR_CSR1\)](#) indicate valid voltage levels.
 - The system operates normally and V_{CORE} overvoltage protection is disabled.
- V_{CORE} overvoltage (due to a wrongly programmed SMPS step-down converter configuration):

- The hardware forces the SMPS step-down converter voltage level to 1.0 V.
- The ACTVOSRDY goes on indicating invalid voltage levels. In this case the software shall be corrected and re-loaded to program a correct SMPS step-down converter configuration that matches the application supply connections. The system shall then be power cycled.

Figure 35. V_{CORE} overvoltage protection

7.6 Power management

The power management block controls the V_{CORE} supply in accordance with the system operation modes (see [Section 7.6.1](#)).

The V_{CORE} domain is split into the following power domains.

- D1 domain containing some peripherals and the Cortex[®]-M7 Core (CPU1).
- D2 domain containing a large part of the peripherals and a Cortex[®]-M4 core (CPU2).
- D3 domain containing some peripherals and the system control.

The D1, D2 and system D3 power domains can operate in one of the following operating modes:

- DRun/Run/Run* (power ON, clock ON)
- DStop/Stop (power ON, clock OFF)
- DStandby/Standby (Power OFF, clock OFF).

The operating modes for D1 domain and D2 domain are independent. However system D3 domain power modes depend on D1 and D2 domain modes:

- For system D3 domain to operate in Stop mode, both D1 and D2 domains must be in DStop or DStandby mode.
- For system D3 domain to operate in Standby mode, both D1 and D2 domains must be in DStandby too.

D1, D2 and system D3 domains are supplied from a single regulator at a common V_{CORE} level. The V_{CORE} supply level follows the system operating mode (Run, Stop, Standby). The D1 domain and/or D2 domain supply can be powered down individually when the domains are in DStandby mode.

The following voltage scaling features allow controlling the power with respect to the required system performance (see [Section 7.6.2: Voltage scaling](#)):

- To obtain a given system performance, the corresponding voltage scaling shall be set in accordance with the system clock frequency. To do this, configure the VOS bits to the Run mode voltage scaling.
- To obtain the best trade-off between power consumption and exit-from-Stop mode latency, configure the SVOS bits to Stop mode voltage scaling.

7.6.1 Operating modes

Several system operating modes are available to tune the system according to the performance required, i.e. when the CPU(s) do not need to execute code and are waiting for an external event. It is up to the user to select the operating mode that gives the best compromise between low power consumption, short startup time and available wakeup sources.

The operating modes allow controlling the clock distribution to the different system blocks and powering them. The system operating mode is driven by CPU1 subsystem, CPU2 subsystem and system D3 autonomous wakeup. A CPU subsystem can include multiple domains depending on its peripheral allocation (see [Section 9.5.11: Peripheral clock gating control](#)).

The following operating modes are available for the different system blocks (see [Table 34](#)):

- CPU subsystem modes:
 - **CRun**
CPU and CPU subsystem peripheral allocated via RCC PERxEN bits are clocked.
 - **CSleep**:
The CPU clocks is stalled and the CPU subsystem allocated peripheral(s) clock operate according to RCC PERxLPEN.
 - **CStop**:
CPU and CPU subsystem peripheral clocks are stalled.
- D1 domain and D2 domain modes:
 - **DRun**
The domain bus matrix is clocked:
 - The domain CPU subsystem^(a) is in CRun or CSleep mode,
 - or

- the other domain CPU subsystem^(a) having an allocated peripheral in the domain is in CRun or CSleep mode.
- **DStop**
The domain bus matrix clock is stalled:
 - The domain CPU subsystem is in CStop mode
 - and
 - The other domain CPU subsystem has no peripheral allocated in the domain. or the other domain CPU subsystem having an allocated peripheral in the domain is also in CStop mode
 - and
 - At least one PDDS_Dn^(b) bit for the domain select DStop.
- **DStandby**
The domain is powered down:
 - The domain CPU subsystem is in CStop mode
 - and
 - The other domain CPU subsystem has no peripheral allocated in the domain or the other domain CPU subsystem having an allocated peripheral in the domain is also in CStop mode
 - and
 - All PDDS_Dn^(b) bits for the domain select DStandby mode.
- System /D3 domain modes
 - **Run/Run***
The system clock and D3 domain bus matrix clock are running:
 - A CPU subsystem is in CRun or CSleep mode
 - or
 - A wakeup signal is active. (i.e. System D3 autonomous mode)
 The Run* mode is entered after a POR reset and a wakeup from Standby. In Run* mode, the performance is limited and the system supply configuration shall be programmed in [PWR control register 3 \(PWR_CR3\)](#). The system enters Run mode only when the ACTVOSRDY bit in [PWR control status register 1 \(PWR_CSR1\)](#) is set to 1.
 - **Stop**
The system clock and D3 domain bus matrix clock is stalled:
 - both CPU subsystems are in CStop mode.
 - and
 - all wakeup signals are inactive.
 - and
 - At least one PDDS_Dn^(b) bit for any domain select Stop mode.
 - **Standby**
The system is powered down:
 - both CPU subsystems are in CStop mode
 - and
 - all wakeup signals are inactive.

a. The domain CPU subsystem, for example CPU1 subsystem for D1 domain.

a. The other domain CPU subsystem, for example CPU1 subsystem for D2 domain.

b. The PDDS_Dn bits belong to [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#).

and

- All PDDS_Dn^(b) bits for all domains select Standby mode.

In Run mode, power consumption can be reduced by one of the following means:

- Lowering the system performance by slowing down the system clocks and reducing the V_{CORE} supply level through VOS voltage scaling bits.
- Gating the clocks to the APBx and AHBx peripherals when they are not used, through PERxEN bits.

Table 34. Low-power mode summary

System	Domain	CPU	Entry	Wakeup	Sys-oscillator	System clk	Domain bus matrix clk	Peripheral clk	CPU clk	Voltage regulator	Domain supply
Run	DRun ⁽¹⁾	CRun	-	-	ON	ON	ON	ON	ON	ON	ON
		CSleep	WFI or return from ISR or WFE	Any interrupt or event				ON/OFF ⁽²⁾			
		CStop	SLEEPDEEP bit + WFI or return from ISR or WFE	Any EXTI interrupt or event				ON/OFF ⁽³⁾			
	DStop ⁽⁴⁾		SLEEPDEEP bit + WFI or return from ISR or WFE								
	DStandby ⁽⁴⁾		SLEEPDEEP bit + WFI or return from ISR or WFE								
Stop ⁽⁵⁾	DStop ⁽⁴⁾	CStop	SLEEPDEEP bit + WFI or return from ISR or WFE or Wakeup source cleared ⁽⁶⁾		ON/OFF ⁽⁷⁾	OFF	OFF	OFF	ON	OFF	
	DStandby ⁽⁴⁾										
Standby ⁽⁸⁾	DStandby ⁽⁴⁾		All PDDS_Dn bit + SLEEPDEEP bit + WFI or return from ISR or WFE or Wakeup source cleared ⁽⁶⁾	WKUP pins rising or falling edge, RTC alarm (Alarm A or Alarm B), RTC Wakeup event, RTC tamper events, RTC time stamp event, external reset in NRST pin, IWDG reset	OFF	OFF	OFF	OFF	OFF	OFF	

1. At least one CPU subsystem that has an allocated peripheral in the domain is in CRun or CSleep.

2. The CPU subsystem peripherals that have a PERxLPEN bit will operate accordingly.

3. The CPU subsystem peripherals that have a PERxAMEN bit will operate accordingly.

4. All CPU subsystems that have an allocated peripheral in the domain need to be in CStop.

5. All domains need to be in DStop Or DStandby.

6. When both CPUs are in CStop and D3 domain in autonomous mode, the last EXTI Wakeup source is cleared.

7. When the system oscillator HSI or CSI is used, the state is controlled by HSIKERN and CSIKERN, otherwise the system oscillator is OFF.
8. All domains are in DStandby mode.

7.6.2 Voltage scaling

The D1, D2, and D3 domains are supplied from a single voltage regulator supporting voltage scaling with the following features:

- Run mode voltage scaling
 - VOS0: Scale 0 (V_{CORE} boost)
 - VOS1: Scale 1
 - VOS2: Scale 2
 - VOS3: Scale 3
- Stop mode voltage scaling
 - SVOS3: Scale 3
 - LP-SVOS4: Scale 4
 - LP-SVOS5: Scale 5

For more details on voltage scaling values, refer to the product datasheets.

After reset, the system starts on the lowest Run mode voltage scaling (VOS3). The voltage scaling can then be changed on-the-fly by software by programming VOS bits in [PWR D3 domain control register \(PWR_D3CR\)](#) according to the required system performance. When exiting from Stop mode or Standby mode, the Run mode voltage scaling is reset to the default VOS3 value.

Before entering Stop mode, the software can preselect the SVOS level in [PWR control register 1 \(PWR_CR1\)](#). The Stop mode voltage scaling for SVOS4 and SVOS5 also sets the voltage regulator in Low-power (LP) mode to further reduce power consumption. When preselecting SVOS3, the use of the voltage regulator low-power mode (LP) can be selected by LPDS register bit.

VOS0 activation/deactivation sequence

The system maximum frequency can be reached by boosting the voltage scaling level to VOS0. This is done through the ODEN bit in the SYSCFG_PWRCR register.

The sequence to activate the VOS0 is the following:

1. Ensure that the system voltage scaling is set to VOS1 by checking the VOS bits in PWR D3 domain control register ([PWR D3 domain control register \(PWR_D3CR\)](#))
2. Enable the SYSCFG clock in the RCC by setting the SYSCFGEN bit in the RCC_APB4ENR register.
3. Enable the ODEN bit in the SYSCFG_PWRCR register.
4. Wait for VOSRDY to be set.

Once the V_{CORE} supply has reached the required level, the system frequency can be increased. [Figure 36](#) shows the recommended sequence for switching V_{CORE} from VOS1 to VOS0 sequence.

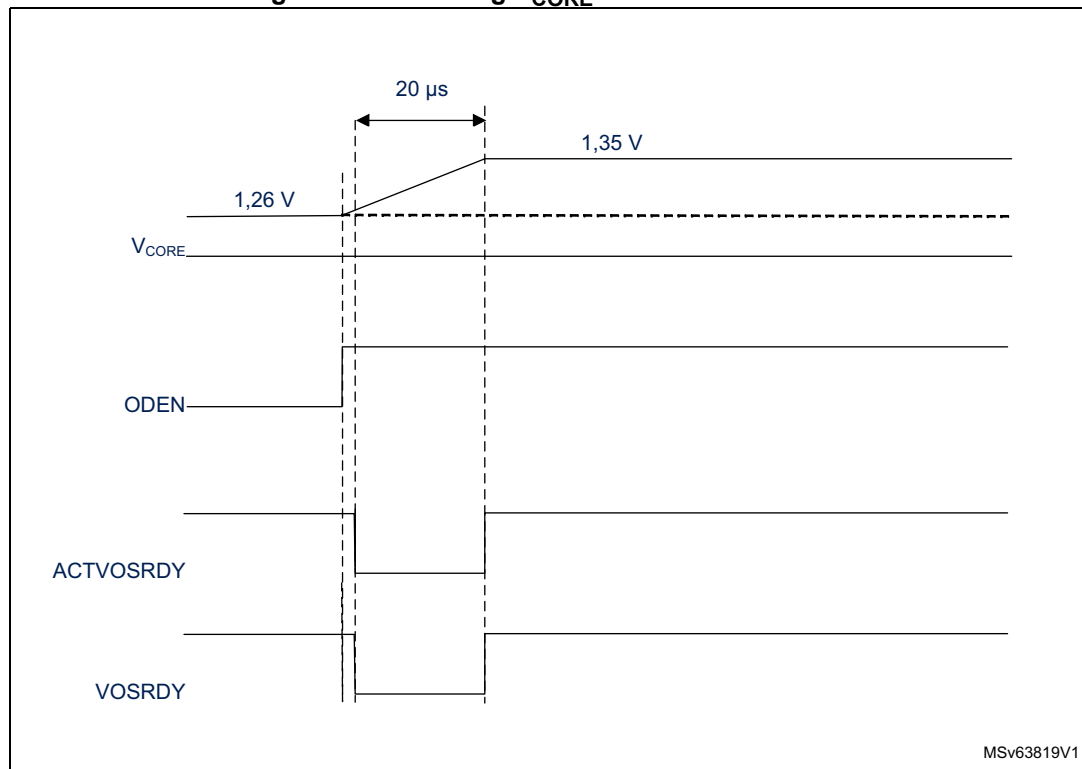
The sequence to deactivate the VOS0 is the following:

1. Ensure that the system frequency was decreased.
2. Ensure that the SYSCFG clock is enabled in the RCC by setting the SYSCFGEN bit set in the RCC_APB4ENR register.
3. Reset the ODEN bit in the SYSCFG_PWRCR register to disable VOS0.

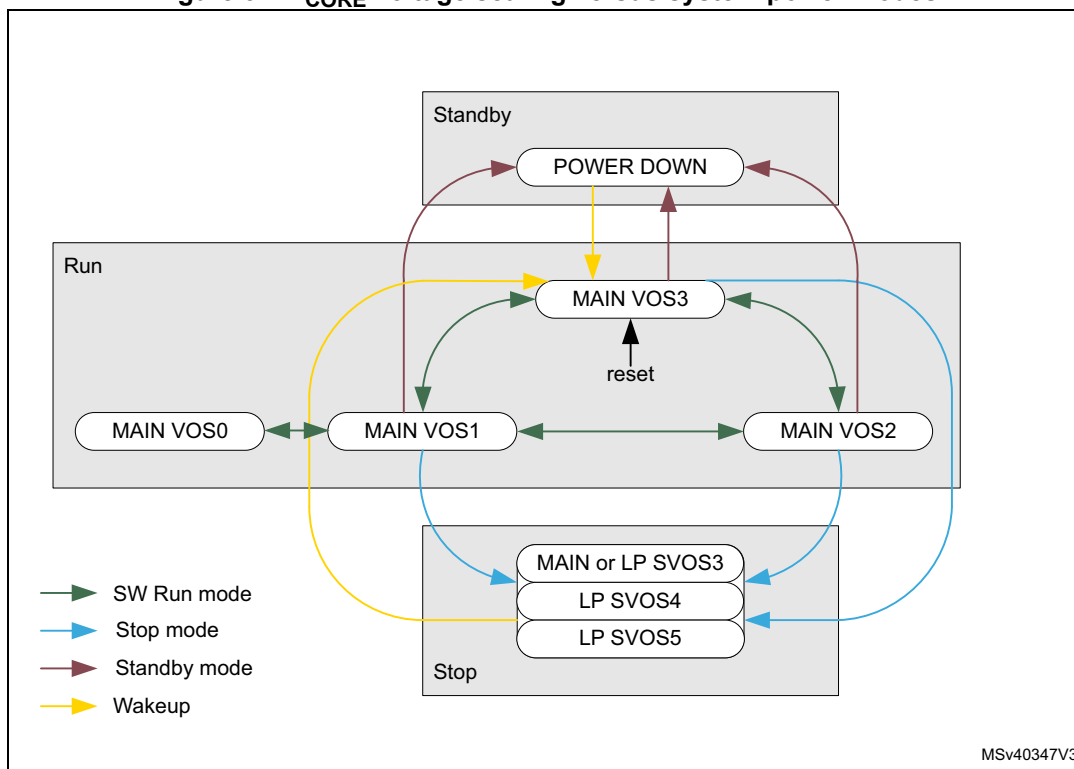
Once VOS0 is disabled, the voltage scaling can be reduced further by configuring VOS bits in PWR D3 domain control register (PWR_D3CR) according to the required system performance.

Note: VOS0 can be enabled only when VOS1 is programmed in PWR D3 domain control register (PWR_D3CR) VOS bits. VOS0 deactivation must be managed by software before the system enters low-power mode.

Figure 36. Switching V_{CORE} from VOS1 to VOS0



MSv63819V1

Figure 37. V_{CORE} voltage scaling versus system power modes

7.6.3 Power control modes

The power control block handles the V_{CORE} supply for system Run, Stop and Standby modes.

The system operating mode depends on the CPU subsystem modes (CRun, CSleep, CStop), on the domain modes (DRun, DStop, DStandby), and on the system D3 autonomous wakeup:

- In Run mode, V_{CORE} is defined by the VOS voltage scaling.
At least one CPU subsystem is in CRun or CSleep or an EXTI wakeup is active.
- In Stop mode, V_{CORE} is defined by the SVOS voltage scaling.
The CPU subsystems are in CStop mode and all EXTI wakeups are inactive. The D1 domain and D2 domain are either in DStop or DStandby mode.
- In Standby mode, V_{CORE} supply is switched off.
The CPU subsystems are in CStop mode and all EXTI wakeups are inactive. The D1 domain and D2 domain are both in DStandby mode.

The domain operating mode can depend on both CPU subsystems when peripherals are allocated in the corresponding domain. The domain mode selection between DStop and DStandby is configured via domain dedicated PDDS_Dn bits in [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#). Each CPU can choose to keep a domain in DStop, or allow a domain to enter DStandby. A domain enters DStandby only when both CPUs have allowed it.

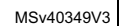
If a domain is in DStandby mode, the corresponding power is switched off.

All the domains can configure the system mode (Stop or Standby) through PDDS_Dn bits in *PWR CPU1 control register (PWR_CPU1CR)* and *PWR CPU2 control register (PWR_CPU2CR)*. The system enters Standby only when all PDDS_Dn bits for all domains have allowed it.

Table 35. PDDS_Dn low-power mode control

PWR_CPU1CR			PWR_CPU2CR			D1 mode	D2 mode	D3 mode
PDDS_D1	PDDS_D2	PDDS_D3	PDDS_D1	PDDS_D2	PDDS_D3			
0	x	x	x	x	x	DStop	any	Run or Stop
x			0			DStop	any	Run or Stop
1			1			DStandby	any	any
x	0		x	x		any	DStop	Run or Stop
	x			0		any	DStop	Run or Stop
	1			1		any	DStandby	any
at least one = 0						DStop or DStandby	DStop or DStandby	Stop
1	1	1	1	1	1	DStandby	DStandby	Standby

Figure 38. Power control modes detailed state diagram



After a system reset, both CPUs are in CRun mode.

Power control state transitions are initiated by the following events:

- CPU going to CStop mode (state transitions in Run mode are marked in green and red)
 - Green transitions: CPU wakes up as from CSleep.
 - Red transitions: CPU wakes up with domain reset. The SBF_Dn is set.
- Allocating or de-locating a peripheral in a domain (state transitions in Run mode are marked in orange and red)
 - Orange transitions: the domain wakes up from DStop
 - Red transitions: the domain wakes up from DStandby. The SBF_Dn is set.
- The system enters or exits from Stop mode (state transitions marked in blue)
 - Blue transitions the system wakes up from Stop mode. The STOPF is set.
- The system enters or exits from Standby mode (state transitions in Stop and Standby mode are marked in red).
 - When exiting from Standby mode, the SBF is set.

When a domain exits from DStandby, the domain CPU and peripherals are reset, while the domain SBF_Dn bit is set (state transitions causing a CPU reset are marked in red).

[Table 36](#) shows the flags that indicate from which mode the domain/system exits. Each CPU has its own set of flags which can be read from [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#).

Table 36. Low-power exit mode flags

System mode	D1 domain mode	D2 domain mode	SBF_D1	SBF_D2	SBF	STOPF	Comment
Run	DRun or DStop	DRun or DStop	0	0	0	0	D1, D2 and system contents retained
Run	DStandby	DRun or DStop	1	0	0	0	D1 contents lost, D2 and system contents retained
Run	DRun or DStop	DStandby	0	1	0	0	D2 contents lost, D1 and system contents retained
Run	DStandby	DStandby	1	1	0	0	D1 and D2 contents lost, system contents retained
Stop	DStop	DStop	0	0	0	1	D1, D2 and system contents retained, clock system reset.
Stop	DStandby	DStop	1	0	0	1	D1 contents lost, D2 and system contents retained, clock system reset
Stop	DStop	DStandby	0	1	0	1	D2 contents lost, D1 and system contents retained, clock system reset
Stop	DStandby	DStandby	1	1	0	1	D1 and D2 contents lost, system contents retained, clock system reset
Standby	DStandby	DStandby	0 ⁽¹⁾	0 ⁽¹⁾	1	0	D1, D2 and system contents lost

1. When returning from Standby, the SBF_D1 and SBF_D2 reflect the reset value.

7.6.4 Power management examples

- [Figure 39](#) shows V_{CORE} voltage scaling behavior in Run mode.
- [Figure 40](#) shows V_{CORE} voltage scaling behavior in Stop mode.
- [Figure 41](#) shows V_{CORE} voltage regulator and voltage scaling behavior in Standby mode.
- [Figure 42](#) shows V_{CORE} voltage scaling behavior in Run mode with D1 and D2 domains are in DStandby mode

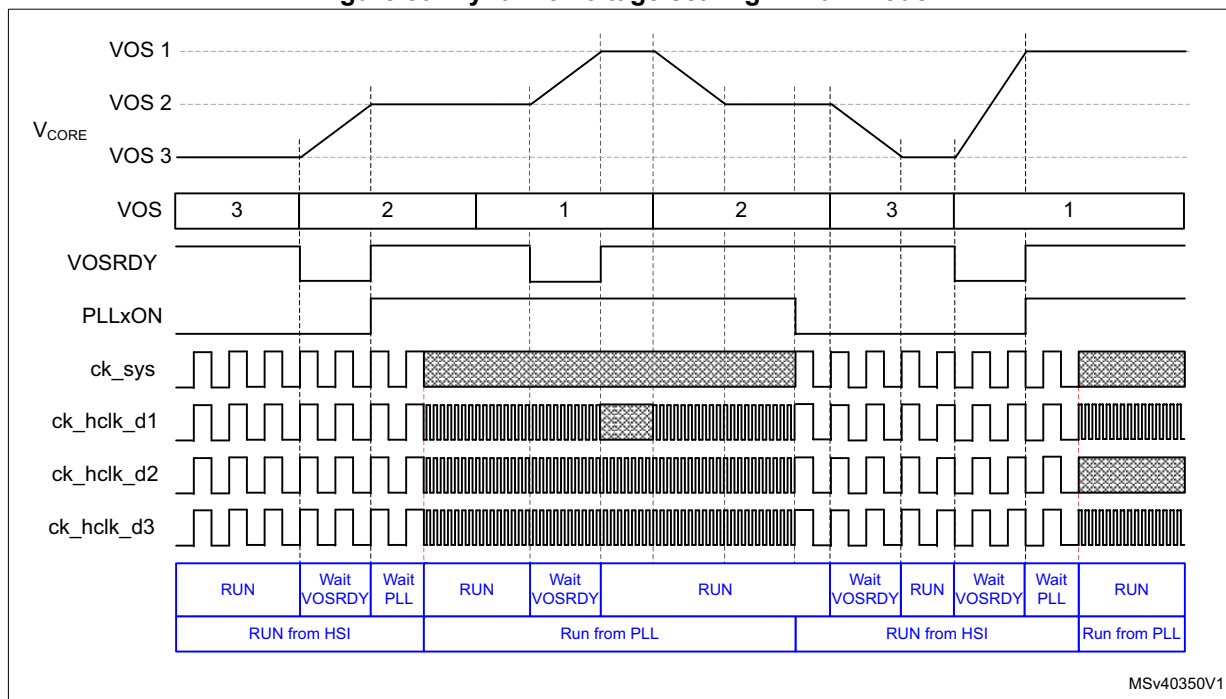
Example of V_{CORE} voltage scaling behavior in Run mode

[Figure 39](#) illustrates the following system operation sequence example:

1. After reset, the system starts from HSI with VOS3.
2. The system performance is first increased to a medium-speed clock from the PLL with voltage scaling VOS2. To do this:
 - a) Program the voltage scaling to VOS2.
 - b) Once the V_{CORE} supply has reached the required level indicated by VOSRDY, increase the clock frequency by enabling the PLL.
 - c) Once the PLL is locked, switch the system clock.
3. The system performance is then increased to high-speed clock from the PLL with voltage scaling VOS1. To do this:
 - a) Program the voltage scaling to VOS1.
 - b) Once the V_{CORE} supply has reached the required level indicated by VOSRDY, increase the clock frequency.
4. The system performance is then reduced to a medium-speed clock with voltage scaling VOS2. To do this:
 - a) First decrease the system frequency.
 - b) Then decrease the voltage scaling to VOS2.
5. The next step is to reduce the system performance to HSI clock with voltage scaling VOS3. To do this:
 - a) Switch the clock to HSI.
 - b) Disable the PLL.
 - c) Decrease the voltage scaling to VOS3.
6. The system performance can then be increased to high-speed clock from the PLL. To do this:
 - a) Program the voltage scaling to VOS1.
 - b) Once the V_{CORE} supply has reached the required level indicated by VOSRDY, increase the clock frequency by enabling the PLL.
 - c) Once the PLL is locked, switch the system clock.

When the system performance (clock frequency) is changed, VOS shall be set accordingly, otherwise the system might be unreliable.

Figure 39. Dynamic voltage scaling in Run mode



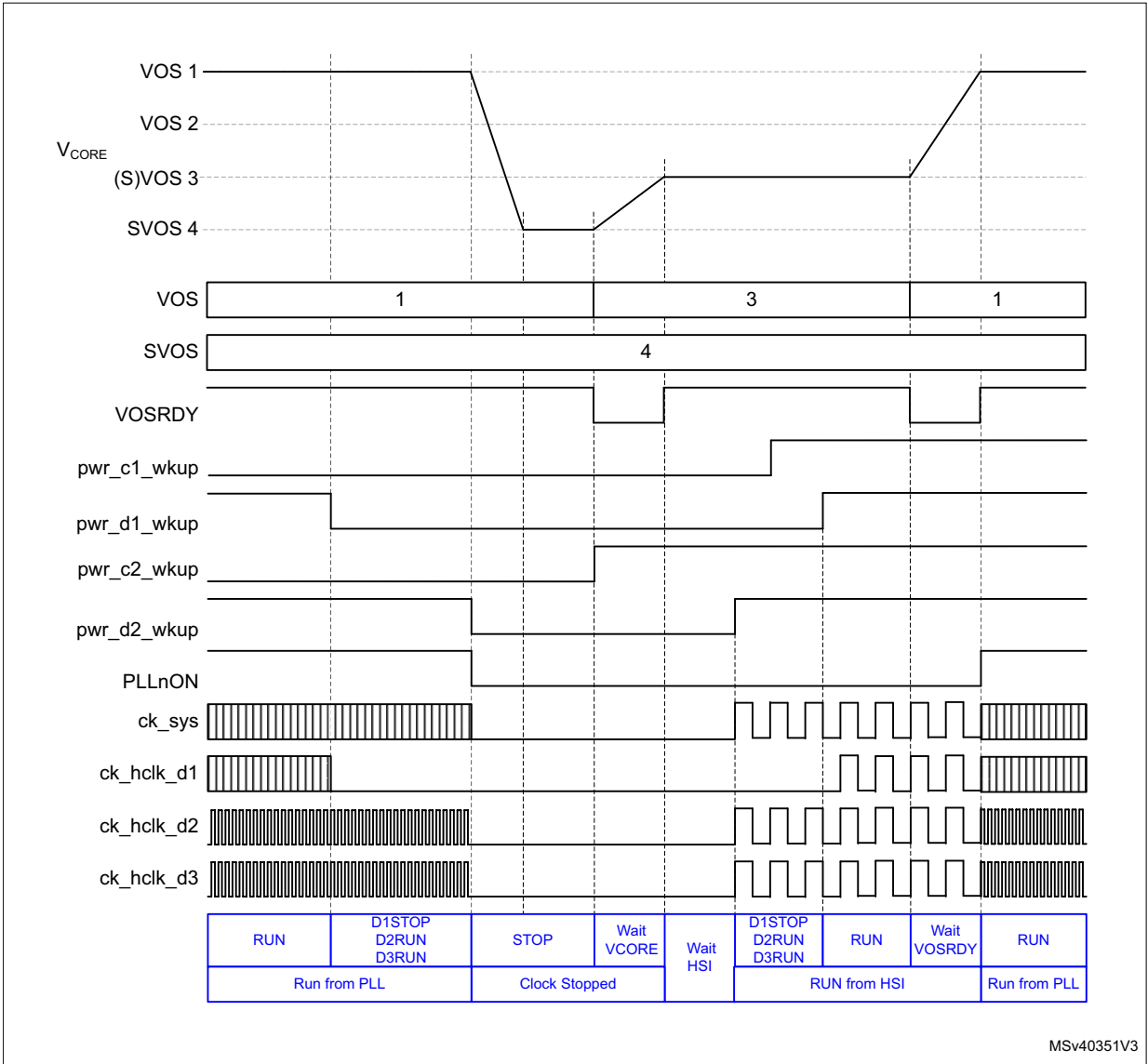
1. The status of the register bits at each step is shown in blue.

Example of V_{CORE} voltage scaling behavior in Stop mode

Figure 40 illustrates the following system operation sequence example:

- The system is running from the PLL in high-performance mode (VOS1 voltage scaling).
- CPU1 subsystem first enters CStop and D1 domain DStop mode. D1 system clock is stopped. The system still provides the high-performance system clock, hence the voltage scaling shall stay at VOS1 level.
- In a second step, CPU2 subsystem enters CStop mode, D2 domain enters DStop mode and the system enters Stop mode. The system clock is stopped and the hardware lowers the voltage scaling to the software preselected SVOS4 level.
- CPU2 subsystem is then woken up. The system exits from Stop mode, the D2 domain exits from DStop mode and the CPU2 subsystem exits from CStop mode. The hardware then sets the voltage scaling to VOS3 level and waits for the requested supply level to be reached before enabling the HSI clock. Once the HSI clock is stable, the system clock and the D2 system clock are enabled.
- The CPU1 subsystem is then woken up and exits from CStop mode. The D1 system clock is enabled.
- The system performance is then increased. To do this:
 - The software first sets the voltage scaling to VOS1.
 - Once the V_{CORE} supply has reached the required level indicated by VOSRDY, the clock frequency can be increased by enabling the PLL.
 - Once the PLL is locked, the system clock can be switched.

Figure 40. Dynamic voltage scaling behavior with D1, D2 and system in Stop mode



MSv40351V3

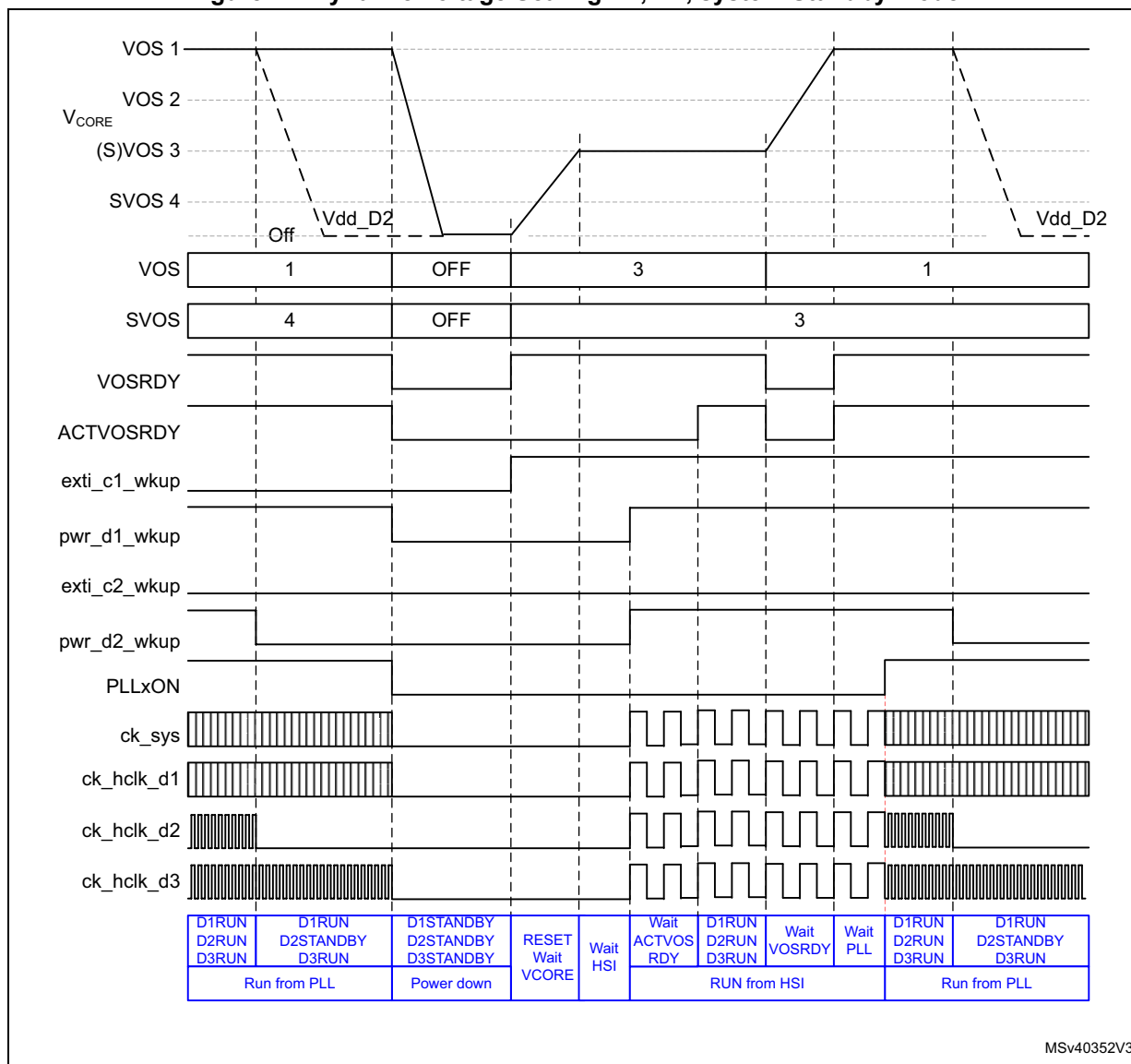
1. The status of the register bits at each step is shown in blue.

Example of V_{CORE} voltage regulator and voltage scaling behavior in Standby mode

Figure 41 illustrates the following system operation sequence example:

1. The system is running from the PLL in high-performance mode (VOS1 voltage scaling).
2. CPU2 subsystem first enters CStop mode and D2 domain enters DStandby mode. The D2 domain bus matrix clock is stopped and the power is switched off. The system performance is unchanged hence the voltage scaling does not change.
3. CPU1 subsystem then enters to CStop mode, D1 enters DStandby mode and the system enters Standby mode. The system clock is stopped and the voltage regulator switched off.
4. The system is then woken up by a wakeup source. The system exits from Standby mode. The hardware sets the voltage scaling to the default VOS3 level and waits for the requested supply level to be reached before enabling the default HSI oscillator. Once the HSI clock is stable, the system clock, D1 subsystem clock, and D2 subsystem clock are enabled. The software shall then check the ACTVOSRDY is valid before changing the system performance.
5. In a next step, increase the system performance. To do this:
 - a) The software first increases the voltage scaling to VOS1 level/
 - b) Before enabling the PLL, it waits for the requested supply level to be reached by monitoring VOSRDY bit.
 - c) Once the PLL is locked, the system clock can be switched.
6. The CPU2 completes processing and sets CPU2 subsystem in CStop mode and D2 domain in DStandby mode. The D2 domain bus matrix clock is stopped and its supply switched off.

Figure 41. Dynamic Voltage Scaling D1, D2, system Standby mode



MSv40352V3

1. The status of the register bits at each step is shown in blue.

Example of V_{CORE} voltage scaling behavior in Run mode with D1 and D2 domains in DStandby mode

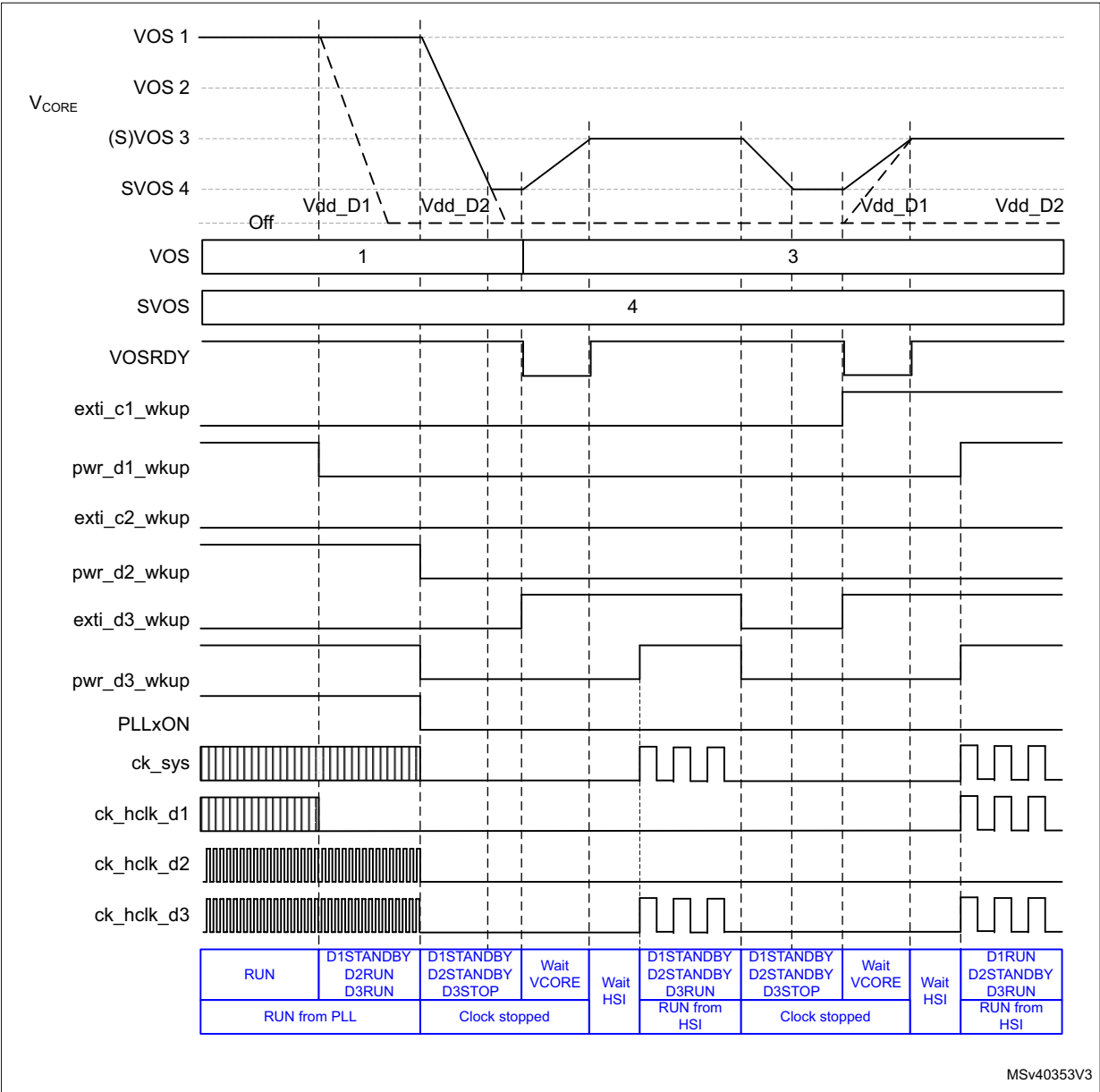
Figure 42 illustrates the following system operation sequence example:

1. The system is running from the PLL with system in high performance mode (VOS1 voltage scaling).
2. CPU1 subsystem first enters CStop mode and the D1 domain enters to DStandby mode. The D1 domain bus matrix clock is stopped and its power switched off. The system performance is unchanged hence the voltage scaling does not change.
3. CPU2 subsystem then enters CStop mode and the D2 domain enters DStandby mode. The D2 domain bus matrix clock is stopped and its power switched off. At the same

time the system enters Stop mode. The system clock is stopped and the hardware lowers the voltage scaling to the software preselected SVOS4 level.

4. The system is then woken up by a D3 autonomous mode wakeup event. The system exits from Stop mode. The hardware sets the voltage scaling to the default VOS3 level and waits for the requested supply level to be reached before enabling the HSI clock. Once the HSI clock is stable, the system clock is enabled. The system is running in D3 autonomous mode.
5. The D3 autonomous mode wakeup source is then cleared, causing the system to enter Stop mode. The system clock is stopped and the voltage scaling is lowered to the software preselected SVOS4 level.
6. CPU1 subsystem is then woken up. The system exits from Stop mode, the D1 domain exits from DStandby mode and CPU1 subsystem exits from CStop mode. The hardware sets the voltage scaling to the default VOS3 level and waits for the requested supply level to be reached before enabling the default HSI oscillator. Once the HSI clock is stable, the system clock and the D1 subsystem clock are enabled.

Figure 42. Dynamic voltage scaling behavior with D1 and D2 in DStandby mode and D3 in autonomous mode



1. The status of the register bits at each step is shown in blue.

7.7 Low-power modes

Several low-power modes are available to save power when the CPU(s) do not need to execute code (i.e. when waiting for an external event). It is up to the user application to select the mode that gives the best compromise between low power consumption, short startup time and available wakeup sources:

- Slowing down system clocks (see [Section 9.5.6: System clock \(sys_ck\)](#))
- Controlling individual peripheral clocks (see [Section 9.5.11: Peripheral clock gating control](#))
- Low-power modes
 - CSleep (CPU clock stopped)
 - CStop (CPU subsystem clock stopped)
 - DStop (Domain bus matrix clock stopped)
 - Stop (System clock stopped)
 - DStandby (Domain powered down)
 - Standby (System powered down)

7.7.1 Slowing down system clocks

In Run mode, the speed of the system clock `ck_sys` can be reduced. For more details refer to [Section 9.5.6: System clock \(sys_ck\)](#).

7.7.2 Controlling peripheral clocks

In Run mode, the `HCLKx` and `PCLKx` for individual peripherals can be stopped by configuring at any time `PERxEN` bit in `RCC_C1_XXXXENR`, `RCC_C2_XXXXENR` or `RCC_DnXXXXENR` to reduce power consumption.

To reduce power consumption in CSleep mode, the individual peripheral clocks can be disabled by configuring `PERxLPEN` bit in `RCC_C1_XXXXLPENR`, `RCC_C2_XXXXLPENR` or `RCC_DnXXXXLPENR`. For the peripherals still receiving a clock in CSleep mode, their clock can be slowed down before entering CSleep mode.

7.7.3 Entering low-power modes

CPU subsystem CSleep and CStop low-power modes are entered by the MCU when executing the WFI (Wait For Interrupt) or WFE (Wait for Event) instructions, or when the `SLEEPONEXIT` bit in the Cortex[®]-M System Control register is set on Return from ISR.

A domain can enter DStop or DStandby low-power mode when the CPU subsystem(s) that have an allocated peripheral in the domain enters CStop mode, or when the other domain CPU deallocates its last peripheral and the domain CPU subsystem is in CStop mode.

The system can enter Stop or Standby low-power mode when all EXTI wakeup sources are cleared and the other domains are in DStop or DStandby mode.

7.7.4 Exiting from low-power modes

The CPU subsystem exits from CSleep mode through any interrupt or event depending on how the low-power mode was entered:

- If the WFI instruction or Return from ISR was used to enter to low-power mode, any peripheral interrupt acknowledged by the NVIC can wake up the system.
- If the WFE instruction is used to enter to low-power mode, the CPU exits from low-power mode as soon as an event occurs. The wakeup event can be generated either by:

- An NVIC IRQ interrupt.

When SEVONPEND = 0 in the Cortex®-M4 System Control register, the interrupt must be enabled in the peripheral control register and in the NVIC.

When the MCU resumes from WFE, the peripheral interrupt pending bit and the NVIC peripheral IRQ channel pending bit in the NVIC interrupt clear pending register have to be cleared. Only NVIC interrupts with sufficient priority will wakeup and interrupt the MCU.

When SEVONPEND = 1 in the Cortex®-M4 System Control register, the interrupt must be enabled in the peripheral control register and optionally in the NVIC.

When the MCU resumes from WFE, the peripheral interrupt pending bit and, when enabled, the NVIC peripheral IRQ channel pending bit (in the NVIC interrupt clear pending register) have to be cleared.

All NVIC interrupts will wakeup the MCU, even the disabled ones.

Only enabled NVIC interrupts with sufficient priority will wakeup and interrupt the MCU.

- An event

An EXTI line must be configured in event mode. When the CPU resumes from WFE, it is not necessary to clear the EXTI peripheral interrupt pending bit or the NVIC IRQ channel pending bit as the pending bits corresponding to the event line is not set. It might be necessary to clear the interrupt flag in the peripheral.

The CPU subsystem exits from CStop, DStop and Stop modes by enabling an EXTI interrupt or event depending on how the low-power mode was entered (see above).

The system can wake up from Stop mode by enabling an EXTI wakeup, without waking up a CPU subsystem. In this case the system will operate in D3 autonomous mode.

The CPU subsystem exits from DStandby mode by enabling an EXTI interrupt or event, regardless on how DStandby mode was entered. Program execution restarts from CPU local reset (such as a reset vector fetched from System configuration block (SYSCFG)).

A domain can exit from DStop or DStandby mode when the other domain CPU allocates a first peripheral in the domain. In this case the CPU in the domain is not woken up.

The CPU subsystem exits from Standby mode by enabling an external reset (NRST pin), an IWDG reset, a rising edge on one of the enabled WKUPx pins or a RTC event. Program execution restarts in the same way as after a system reset (such as boot pin sampling, option bytes loading or reset vector fetched).

7.7.5 CSleep mode

The CSleep mode applies only to the CPU subsystem. In CSleep mode, the CPU clock is stopped. The CPU subsystem peripheral clocks operate according to the values of PERxLPEN bits in RCC_C1_xxxxENR, RCC_C2_xxxxENR or RCC_DnxxxxENR.

Entering CSleep mode

The CSleep mode is entered according to [Section 7.7.3: Entering low-power modes](#), when the SLEEPDEEP bit in the Cortex[®]-M System Control register is cleared.

Refer to [Table 37](#) for details on how to enter to CSleep mode.

Exiting from CSleep mode

The CSleep mode is exited according to [Section 7.7.4: Exiting from low-power modes](#).

Refer to [Table 37](#) for more details on how to exit from CSleep mode.

Table 37. CSleep mode

CSleep mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – SLEEPDEEP = 0 (Refer to the Cortex[®]-M System Control register.) – CPU NVIC interrupts and events cleared.
	On return from ISR while: – SLEEPDEEP = 0 and – SLEEPONEXIT = 1 (refer to the Cortex[®]-M System Control register.) – CPU NVIC interrupts and events cleared.
Mode exit	If WFI or return from ISR was used for entry: – Any Interrupt enabled in NVIC: Refer to Table 149: NVIC1 (CPU1) and NVIC2 (CPU2) If WFE was used for entry and SEVONPEND = 0: – Any event: Refer to Section 21.5.3: EXTI CPU wakeup procedure If WFE was used for entry and SEVONPEND = 1: – Any Interrupt even when disabled in NVIC: refer to Table 149: NVIC1 (CPU1) and NVIC2 (CPU2) or any event: refer to Section 21.5.3: EXTI CPU wakeup procedure
Wakeup latency	None

7.7.6 CStop mode

The CStop mode applies only to the CPU subsystem. In CStop mode, the CPU clock is stopped. Most CPU subsystem peripheral clocks are stopped too and only the CPU subsystem peripherals having a PERxAMEN bit operate accordingly.

In CStop mode, CPU subsystem peripherals having a kernel clock request can still request their kernel clock. For the peripheral having a PERxAMEN bit, this bit shall be set to be able to request the kernel clock.

Entering CStop mode

The CStop mode is entered according to [Section 7.7.3: Entering low-power modes](#), when the SLEEPDEEP bit in the Cortex[®]-M System Control register is set.

Refer to [Table 38](#) for details on how to enter to CStop mode.

Exiting from CStop mode

The CStop mode is exited according to [Section 7.7.4: Exiting from low-power modes](#).

Refer to [Table 38](#) for more details on how to exit from CStop mode.

Table 38. CStop mode

CStop mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – SLEEPDEEP = 1 (Refer to the Cortex[®]-M System Control register.) – CPU NVIC interrupts and events cleared. – All CPU EXTI Wakeup sources are cleared.
	On return from ISR while: – SLEEPDEEP = 1 and – SLEEPONEXIT = 1 (Refer to the Cortex[®]-M System Control register.) – CPU NVIC interrupts and events cleared. – All CPU EXTI Wakeup sources are cleared.
Mode exit	If WFI or return from ISR was used for entry: – EXTI Interrupt enabled in NVIC: refer to Table 149: NVIC1 (CPU1) and NVIC2 (CPU2), for peripheral which are not stopped or powered down. If WFE was used for entry and SEVONPEND = 0: – EXTI event: refer to Section 21.5.3: EXTI CPU wakeup procedure, for peripheral which are not stopped or powered down. If WFE was used for entry and SEVONPEND = 1: – EXTI Interrupt even when disabled in NVIC: refer to Table 149: NVIC1 (CPU1) and NVIC2 (CPU2) or EXTI event: refer to Section 21.5.3: EXTI CPU wakeup procedure, for peripheral which are not stopped or powered down. <i>Note: When CPU1 sends a SEV event to wakeup CPU2, the event duration is equal to 512 clock cycles.</i>
Wakeup latency	EXTI and RCC wakeup synchronization (see Section 9.4.7: Power-on and wakeup sequences)

7.7.7 DStop mode

D1 domain and/or D2 domain enters DStop mode only when all CPU subsystems having peripherals allocated in the domain are in CStop mode (see [Table 39](#)). In DStop mode the domain bus matrix clock is stopped.

The Flash memory can enter low-power Stop mode when it is enabled through FLPS in PWR_CR1 register. This allows a trade-off between domain DStop restart time and low power consumption.

Table 39. DStop mode overview

Peripheral allocation	CPU1	CPU2	D1 domain	D2 domain	Comment
CPU1 subsystem no peripheral allocated in D2 domain. and CPU2 subsystem no peripheral allocated in D1 domain.	CStop	CRun or CSleep	DStop	DRun	
	CRun or CSleep	CStop	DRun	DStop	
	CStop	CStop	DStop	DStop	
CPU1 subsystem peripheral allocated in D2 domain. and CPU2 subsystem no peripheral allocated in D1 domain.	CStop	CRun or CSleep	DStop	DRun	
	CRun or CSleep	CStop	DRun	DRun	CPU1 subsystem, keep D2 domain active.
	CStop	CStop	DStop	DStop	
CPU1 subsystem no peripheral allocated in D2 domain. and CPU2 subsystem peripheral allocated in D1 domain.	CStop	CRun or CSleep	DRun	DRun	CPU2 subsystem, keep D1 domain active.
	CRun or CSleep	CStop	DRun	DStop	
	CStop	CStop	DStop	DStop	
CPU1 subsystem peripheral allocated in D2 domain. and CPU2 subsystem peripheral allocated in D1 domain.	CStop	CRun or CSleep	DRun	DRun	CPU2 subsystem, keep D1 domain active.
	CRun or CSleep	CStop	DRun	DRun	CPU1 subsystem, keep D2 domain active.
	CStop	CStop	DStop	DStop	

In DStop mode domain peripherals using the LSI or LSE clock and peripherals having a kernel clock request are still able to operate.

Entering DStop mode

The DStop mode is entered according to [Section 7.7.3: Entering low-power modes](#), when at least one PDDS_Dn bit in [PWR CPU1 control register \(PWR_CPU1CR\)](#) or [PWR CPU2 control register \(PWR_CPU2CR\)](#) for the domain select Stop.

Refer to [Table 40](#) for details on how to enter DStop mode.

If Flash memory programming is ongoing, the DStop mode entry is delayed until the memory access is finished.

If an access to the domain bus matrix is ongoing, the DStop mode entry is delayed until the domain bus matrix access is complete.

Exiting from DStop mode

The DStop mode is exited according to [Section 7.7.4: Exiting from low-power modes](#).

Refer to [Table 40](#) for more details on how to exit from DStop mode.

When exiting from DStop mode, the CPU subsystem clocks, domain(s) bus matrix clocks and voltage scaling depend on the system mode.

- When the system did not enter Stop mode, the CPU subsystem clocks, domain(s) bus matrix clocks and voltage scaling values are the same as when entering DStop mode, except if they have been changed by the other CPU.
- When the system has entered Stop mode, the CPU subsystem clocks, domain(s) bus matrix clocks and voltage scaling are reset.

Table 40. DStop mode

DStop mode	Description
Mode entry	– The domain CPU subsystem enters CStop and the other domain CPU subsystem has no peripheral allocated in the domain or is also in CStop. – The other domain CPU subsystem has an allocated peripheral and enters to CStop and the Domain CPU subsystem is in CStop. – The other domain CPU subsystem deallocated its last peripheral in the domain and the Domain CPU subsystem is in CStop. – At least one PDDS_Dn bit for the domain selects Stop mode.
Mode exit	– The domain CPU subsystem exits from CStop mode (see Table 38) – The other domain CPU subsystem has an allocated peripheral in the domain and exits from CStop mode (see Table 38) – The other domain CPU subsystem allocates a first peripheral in the domain.
Wakeup latency	EXTI and RCC wakeup synchronization (see Section 9.4.7: Power-on and wakeup sequences).

7.7.8 Stop mode

The system D3 domain enters Stop mode only when all CPU subsystems are in CStop mode, the EXTI wakeup sources are inactive and at least one PDDS_Dn bit in [PWR CPU1 control register \(PWR_CPU1CR\)](#) or [PWR CPU2 control register \(PWR_CPU2CR\)](#) for any domain request Stop. In Stop mode, the system clock including a PLL and the D3 domain bus matrix clocks are stopped. When HSI or CSI is selected, the system oscillator operates according to the HSIKERON and CSIKERON bits in RCC_CR register. Other system oscillator sources are stopped.

In system D3 domain Stop mode, D1 domain and D2 domain are either in DStop and/or DStandby mode.

In Stop mode, the domain peripherals that use the LSI or LSE clock, and the peripherals that have a kernel clock request to select HSI or CSI as source, are still able to operate.

In system Stop mode, the following features can be selected to remain active by programming individual control bits:

- Independent watchdog (IWDG)

The IWDG is started by writing to its Key register or by hardware option. Once started it cannot be stopped except by a Reset (see [Section 48.3.3: Window option](#) in [Section 48: Independent watchdog \(IWDG\)](#)).

- Real-time clock (RTC)
This is configured via the RTCEN bit in the [RCC Backup domain control register \(RCC_BDCR\)](#).
- Internal RC oscillator (LSI RC)
This is configured via the LSION bit in the [RCC clock control and status register \(RCC_CSR\)](#).
- External 32.768 kHz oscillator (LSE OSC)
This is configured via the LSEON bit in the [RCC Backup domain control register \(RCC_BDCR\)](#).
- Peripherals capable of running on the LSI or LSE clock.
- Peripherals having a kernel clock request.
- Internal RC oscillators (HSI and CSI)
This is configured via the HSIKERON and CSIKERON bits in the [RCC clock control and status register \(RCC_CSR\)](#).
- The ADC or DAC can also consume power during Stop mode, unless they are disabled before entering this mode. To disable them, the ADON bit in the ADC_CR2 register and the ENx bit in the DAC_CR register must both be written to 0.

The selected SVOS4 and SVOS5 levels add an additional startup delay when exiting from system Stop mode (see [Table 41](#)).

Table 41. Stop mode operation

SVOS	LPDS	Stop mode Voltage regulator operation	Wake-up Latency
SVOS3	0	Main	No additional wakeup time.
	1	LP	Voltage Regulator wakeup time from LP mode.
SVOS4 or SVOS5	x	LP	Voltage Regulator wakeup time from LP mode + voltage level wakeup time for SVOS4 or SVOS5 level to VOS3 level

Entering Stop mode

The Stop mode is entered according to [Section 7.7.3: Entering low-power modes](#), when at least one PDDS_Dn bit n [PWR CPU1 control register \(PWR_CPU1CR\)](#) or [PWR CPU2 control register \(PWR_CPU2CR\)](#) for any domain request Stop.

Refer to [Table 44](#) for details on how to enter Stop mode.

If Flash memory programming is ongoing, the Stop mode entry is delayed until the memory access is finished.

If an access to a bus matrix (AXI, AHB or APB) is ongoing, the Stop mode entry is delayed until the bus matrix access is finished.

To allow peripherals having a kernel clock request to operate in Stop mode, the system must use SVOS3 level.

Note: Use a DSB instruction to ensure that outstanding memory transactions complete before entering stop mode.

Before entering Stop mode, the software must ensure that VOS0 is not active.

Exiting from Stop mode

The Stop mode is exited according to [Section 7.7.4: Exiting from low-power modes](#).

Refer to [Table 44](#) for more details on how to exit from Stop mode.

When exiting from Stop mode, the system clock, D3 domain bus matrix clocks and voltage scaling are reset.

A CPU hold mechanism is used to allow the system to be re-initialized by a “master” CPU. The “master” CPU can be woken up by its own wakeup sources and by the “slave” CPU wakeup sources. The “slave” CPU is kept on hold until it is released by the “master” CPU. The hold mechanism is controlled by HOLDn register bits. When the “slave” CPU is on hold, the “slave” CPU subsystem clocks are stalled until they are released by the “master” CPU.

A “slave” CPU will only be put on hold when exiting from Stop mode and the “slave” CPU associated HOLDn bit is set. The “slave” CPU remains on hold until the “master” CPU clears the HOLDn bit (i.e. after system re-initialization). When a wakeup event is issued for the “slave” CPU that is on hold, the “master” CPU is woken up and receives a HOLDnF interrupt.

Note: For correct operation, it is mandatory that the “master” CPU clears the “slave” CPU HOLDn bit each time it exits from CStop mode.

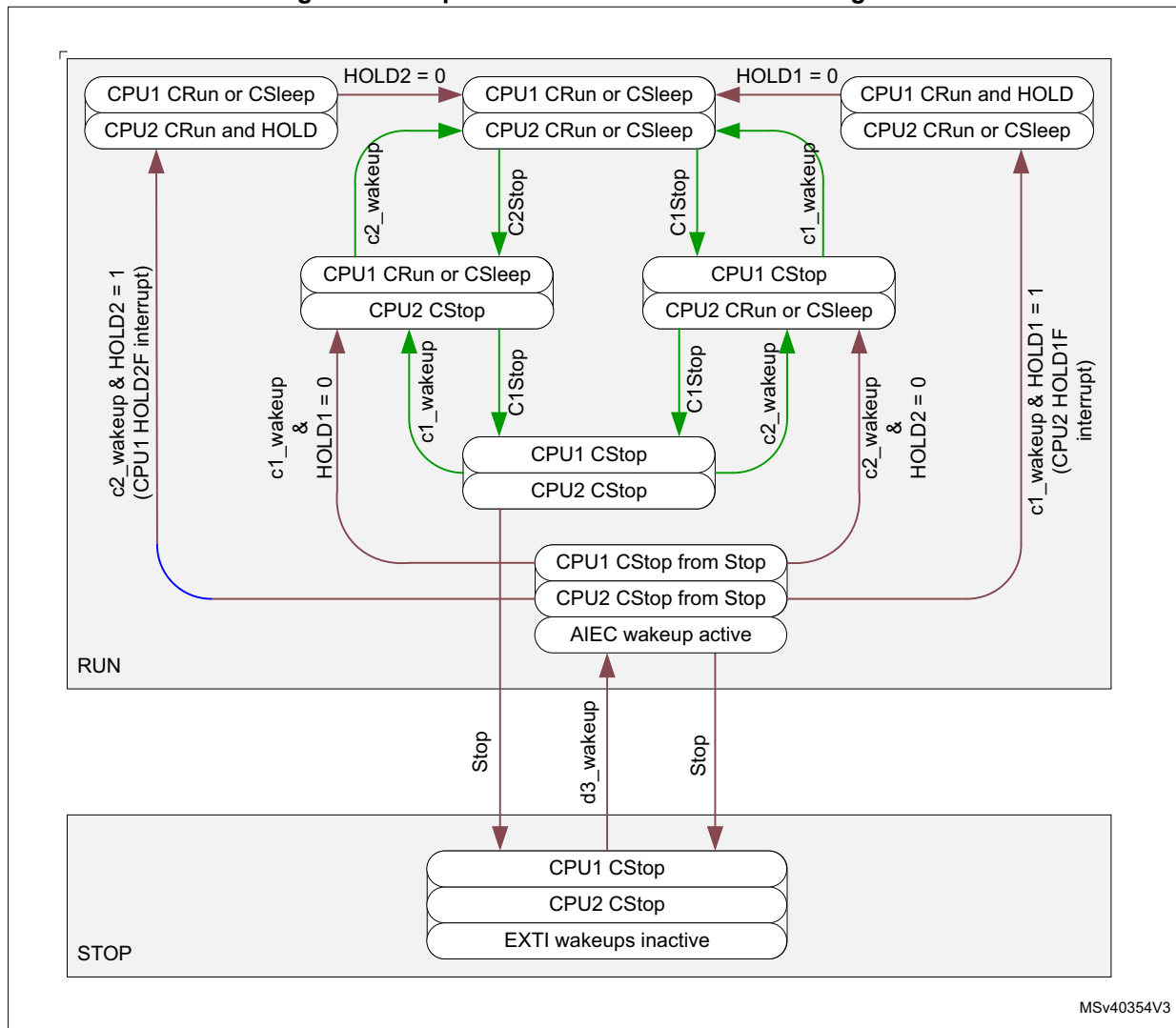
System D3 autonomous mode wakeup from Stop does not support the hold mechanism, hence there is no re-initialization and the system runs from the HSI or CSI clock.

Table 42. Stop mode hold control

HOLD1	HOLD2	Comment
0	0	Hold mechanism disabled, each CPU is only woken up from Stop mode by its own wakeup source.
0	1	CPU1 is “master” and is woken up from Stop mode by its own wakeup sources and on a CPU2 wakeup sources through PWR_CPU1CR.HOLD2F interrupt. When a CPU2 wakeup source event occurs, the CPU2 needs to be released from hold by CPU1 (i.e. after system re-initialization.)
1	0	CPU2 is “master” and is woken up from Stop mode by its own wakeup sources and on a CPU1 wakeup sources through PWR_CPU2CR.HOLD1F interrupt. When a CPU1 wakeup sources event occurs, the CPU1 need to be released from hold by CPU2 (i.e. after system re-initialization.)
1	1	Hold mechanism disabled, each CPU is only woken up from Stop mode by its own wakeup source.

[Figure 43](#) shows the Stop mode hold mechanism state diagram.

Figure 43. Stop mode hold mechanism state diagram



STOPF and HOLD1F and HOLD2F status flags in *PWR CPU1 control register (PWR_CPU1CR)* and *PWR CPU2 control register (PWR_CPU2CR)* indicate that the system has exited from Stop mode (see [Table 43](#)). Each CPU has its own set of flag.

The Hold procedure for the “master” CPU is the following:

- Before entering CStop mode, the “master” CPU sets the HOLDn bit of the “slave” CPU.
- When exiting from CStop mode, the “master” CPU re-initializes the system and clears the HOLDn bit of the “slave” CPU. The “master” CPU can be woken up either by:

- a “master” CPU wakeup event with associate interrupt or event.
- or a “slave” CPU wakeup event through a “master” CPU HOLDnF interrupt.

The HOLDn bits only take effect when it is set and the system exits from Stop mode. The associated CPU is kept on hold when the MCU exits from Stop mode and until the HOLDn bit is cleared (see state transitions in blue in [Figure 43](#)).

When the system does not enter Stop mode, the system configuration is kept and the CPU is not on hold. In this case, the HOLDn bit has no effect (see state transitions in green in [Figure 43](#)). Refer also to [Table 43](#) for a detailed description of the hold mechanism.

Table 43. Wakeup hold behavior and associated flags⁽¹⁾

System	HOLD1	HOLD2	HOLD1F	HOLD2F	STOPF	Behavior
Exit from Stop mode	1	0	x	0	1	CPU2 woken up from CStop mode with CPU2 wakeup event.
			1	0	1	CPU2 and CPU1 woken up from CStop mode with CPU1 wakeup event. CPU1 in hold until released by CPU2. CPU2 receives HOLD1F interrupt.
			x	0	1	System D3 domain woken up from Stop mode with EXTI wakeup source. CPU1 and CPU2 kept in CStop.
Run mode	x	0	0	0	0	CPU2 is only woken up from CStop mode with CPU2 wakeup event. The system has not been in Stop.
			0	0	0	CPU1 woken up from CStop mode with CPU1 wakeup event. The system has not been in Stop.

1. CPU2 is “master” and CPU1 is “Slave” (the reversed table applies when CPU1 is “master” and CPU2 is “slave”).

Table 44. Stop mode

Stop mode	Description
Mode entry	– When CPU1 and CPU2 are in CStop mode and there is no active EXTI Wakeup source and Run_D3 = 0. – At least one PDDS_Dn bit for any domain select Stop.
Mode exit	– On a EXTI Wakeup.
Wakeup latency	System oscillator startup (when disabled). + EXTI and RCC wakeup synchronization. + Voltage Scaling refer to Table 41 (see Section 7.6.2: Voltage scaling)

I/O states in Stop mode

I/O pin configuration remain unchanged in Stop mode.

7.7.9 DStandby mode

Like DStop mode, DStandby mode is based on CPU subsystems CStop mode. However the domain V_{CORE} supply is powered off. A domain enters DStandby mode only when all CPU subsystems that have peripherals allocated in the domain are in CStop mode and all PDDS_Dn bits in [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#) for the domain are configured accordingly. In DStandby mode, the domain is powered down and the domain RAM and register contents are lost.

Entering DStandby mode

The DStandby mode is entered according to [Section 7.7.3: Entering low-power modes](#), when all PDDS_Dn bits in [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#) for the Dn domain select Standby mode.

Refer to [Table 45](#) for details on how to enter DStandby mode.

If Flash memory programming is ongoing, the DStandby mode entry is delayed until the memory access is finished.

If an access to the domain bus matrix is ongoing, the DStandby mode entry is delayed until the domain bus matrix access is finished.

Note: When the other domain CPU PDDS_Dn bit selects Stop mode, the Dn domain remains in DStop. When the other domain CPU sets the PDDS_Dn bit to select Standby mode, the Dn domain will enter DStandby mode (the other domain CPU has no allocated peripherals in the Dn domain).

Exiting from DStandby mode

The DStandby mode is exited according to [Section 7.7.4: Exiting from low-power modes](#).

Refer to [Table 45](#) for more details on how to exit from DStandby mode.

Note: When a domain is in DStandby mode and the other domain CPU sets the domain PDDS_Dn bit to select Stop mode, the domain remains in DStandby mode. The domain will only exit DStandby when the other domain CPU allocates a peripheral in the domain.

When exiting from DStandby mode, the domain CPU and peripherals are reset. However the state of the CPU subsystem clocks, domain(s) bus matrix clocks and voltage scaling depends on the system mode:

- When the system did not enter Stop mode, the CPU subsystem clocks, domain(s) bus matrix clocks and voltage scaling are the same as when entering DStandby mode, except if they have been modified by the other CPU.
- When the system has entered Stop or Standby mode, the CPU subsystem clocks, domain(s) bus matrix clocks and voltage scaling are reset.

When a domain exits from DStandby mode due to the other domain CPU subsystem (i.e. when allocating a first peripheral or when the other domain CPU subsystem having peripherals allocated in the domain exits from CStop mode), the other domain CPU shall verify that the domain has exited from DStandby mode. To ensure correct operation, it is recommended to follow the sequence below:

1. First check that the domain bus matrix clock is available. The domain bus matrix clock state can be checked in RCC_CR register:
 - When RCC DnCKRDY = 0, the domain bus matrix clock is stalled.
 - If RCC DnCKRDY = 1, the domain bus matrix clock is enabled.
2. Then wait till that the domain has exited from DStandby mode. To do this, check the SBF_Dn flag in *PWR CPU1 control register (PWR_CPU1CR)* and *PWR CPU2 control register (PWR_CPU2CR)*. The domain is powered and can be accessed only when SBF_Dn is cleared. Below an example of code:

```

Loop
write PWR SBF_Dn = 0 ; try to clear bit.
read PWR SBF_Dn
While 1 ==> loop

```

Table 45. DStandby mode

DStandby mode	Description
Mode entry	– The domain CPU subsystem enters CStop and the other domain CPU subsystem has no peripheral allocated in the domain or is also in CStop. – The other domain CPU subsystem has an allocated peripheral and enters CStop and the Domain CPU subsystem is in CStop. – The other domain CPU subsystem deallocated its last peripheral in the domain and the Domain CPU subsystem is in CStop. – All PDDS_Dn bits for the domain select Standby mode. – All WKUPF bits in Power Control/Status register (PWR_WKUPFR) are cleared.
Mode exit	– The domain CPU subsystem exits from CStop mode (see Table 38) – The other domain CPU subsystem has an allocated peripheral in the domain and exits from CStop mode (see Table 38) – The other domain CPU subsystem allocates a first peripheral in the domain.
Wakeup latency	EXTI and RCC wakeup synchronization. + Domain power up and reset. (see Section 9.4.7: Power-on and wakeup sequences)

7.7.10 Standby mode

The Standby mode allows achieving the lowest power consumption. Like Stop mode, it is based on CPU subsystems CStop mode. However the V_{CORE} supply regulator is powered off.

The system D3 domain enters Standby mode only when the D1 and D2 domain are in DStandby. When the system D3 domain enters Standby mode, the voltage regulator is disabled. The complete V_{CORE} domain is consequently powered off. The PLLs, HSI oscillator, CSI oscillator, HSI48 and the HSE oscillator are also switched off. SRAM and register contents are lost except for backup domain registers (RTC registers, RTC backup register and backup RAM), and Standby circuitry (see [Section 7.4.4: Backup domain](#)).

In system Standby mode, the following features can be selected by programming individual control bits:

- Independent watchdog (IWDG)
The IWDG is started by programming its Key register or by hardware option. Once started, it cannot be stopped except by a reset (see [Section 48.3.3: Window option in Section 48: Independent watchdog \(IWDG\)](#)).
- Real-time clock (RTC)
This is configured via the RTCEN bit in the backup domain control register (RCC_BDCR).
- Internal RC oscillator (LSI RC)
This is configured by the LSION bit in the Control/status register (RCC_CSR).
- External 32.768 kHz oscillator (LSE OSC)
This is configured by the LSEON bit in the backup domain control register (RCC_BDCR).

Entering Standby mode

The Standby mode is entered according to [Section 7.7.3: Entering low-power modes](#), when all PDDS_Dn bits in [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#) for all domains request Standby.

Refer to [Table 47](#) for more details on how to enter to Standby mode.

Note: *Before entering Standby mode, the software must ensure that VOS0 is not active.*

Exiting from Standby mode

The Standby mode is exited according to [Section 7.7.4: Exiting from low-power modes](#).

Refer to [Table 47](#) for more details on how to exit from Standby mode.

The system exits from Standby mode when an external Reset (NRST pin), an IWDG Reset, a WKUP pin event, a RTC alarm, a tamper event, or a time stamp event is detected. All registers are reset after waking up from Standby except for power control and status registers ([PWR control register 2 \(PWR_CR2\)](#), [PWR control register 3 \(PWR_CR3\)](#)), SBF bit in [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#), [PWR wakeup flag register \(PWR_WKUPFR\)](#), and [PWR wakeup enable and polarity register \(PWR_WKUPEPR\)](#).

After waking up from Standby mode, the program execution restarts in the same way as after a system reset (boot option sampling, boot vector reset fetched, etc.). The SBF status flags in [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register](#)

([PWR_CPU2CR](#)) registers indicate from which mode the system has exited (see [Table 46](#)). Each CPU has its own SBF flags.

The system will boot according to the option bytes CM7B and CM4B (see [Section 2.6: Boot configuration](#)).

Table 46. Standby and Stop flags

SBF_D2	SBF_D1	SBF	STOPF	Description
0	1	0	0	D1 domain exits from DStandby while system stayed in Run
0	1	0	1	D1 domain exits from DStandby, while system has been in or exits from Stop
1	0	0	0	D2 domain exits from DStandby while system stayed in Run
1	0	0	1	D2 domain exits from DStandby while system has been in or exits from Stop
0	0	0	1	System has been in or exits from Stop
0 ⁽¹⁾	0 ⁽¹⁾	1	0	System exits from Standby

1. When exiting from Standby the SBF_D1 and SBF_D2 reflect the reset value

Table 47. Standby mode

Standby mode	Description
Mode entry	– The CPU1 subsystem and CPU2 subsystem are in CStop mode, and there is no active EXTI Wakeup source and RUN_D3 = 0. – All PDDS_Dn bits for all domains select Standby. – All WKUPF bits in Power Control/Status register (PWR_WKUPFR) are cleared.
Mode exit	– WKUP pins rising or falling edge, RTC alarm (Alarm A and Alarm B), RTC wakeup, tamper event, time stamp event, external reset in NRST pin, IWDG reset.
Wakeup latency	System reset phase (see Section 9.4.2: System reset)

I/O states in Standby mode

In Standby mode, all I/O pins are high impedance without pull, except for:

- Reset pad (still available)
- RTC_AF1 pin if configured for tamper, time stamp, RTC Alarm out, or RTC clock calibration out
- WKUP pins (if enabled). The WKUP pin pull configuration can be defined through WKUPPUPD register bits in [PWR wakeup enable and polarity register \(PWR_WKUPPEPR\)](#).

7.7.11 Monitoring low-power modes

The devices feature state monitoring pins to monitor the CPU and Domain state transition to low-power mode (refer to [Table 48](#) for the list of pins and their description). The GPIO pin corresponding to each monitoring signal has to be programmed in alternate function mode.

This feature is not available in Standby mode since these I/O pins are switched to high impedance.

Table 48. Low-power modes monitoring pin overview

Power state monitoring pins	Description
CxSLEEP	Sleeping CPU (Cx, x= 1 or 2) state
CxDSLEEP	Deepsleep CPU (Cx, x= 1 or 2) state
DxPWREN	Domain (Dx, x= 1 or 2) power enabled

The values of the monitoring pins reflect the state of the CPUs and domains. Refer to [Table 49](#) for the GPIO state depending on CPU and domain state.

Table 49. GPIO state according to CPU and domain state

Domain DxPWREN	CPUx		CPUx power state	Domainx power state
	CxSLEEP	CxDSLEEP		
1	0	0	CPUx in Run mode	DRun mode
1	1	0	CPUx in Sleep mode	
1	0	1	CPUx in Run mode	
1	1	1	CPUx in Deepsleep mode	DRun ⁽¹⁾ , DStop mode
0	-	-	_(2)	DStandby mode

1. The domain might be in Run mode if a peripheral is allocated to the other CPU.

2. The full domain is in power off state and the CPU is powered off.

7.8 PWR registers

The PWR registers can be accessed in word, half-word and byte format, unless otherwise specified.

7.8.1 PWR control register 1 (PWR_CR1)

Address offset: 0x000

Reset value: 0xF000 C000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALS		AVDEN
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SVOS		Res.	Res.	Res.	Res.	FLPS	DBP	PLS			PVDE	Res.	Res.	Res.	LPDS
rw	rw					rw	rw	rw	rw	rw	rw				rw

Bits 31:19 Reserved, must be kept at reset value

Bits 18:17 **ALS**: Analog voltage detector level selection

These bits select the voltage threshold detected by the AVD.

00: 1.7 V

01: 2.1 V

10: 2.5 V

11: 2.8 V

Bit 16 **AVDEN**: Peripheral voltage monitor on V_{DDA} enable

0: Peripheral voltage monitor on V_{DDA} disabled.

1: Peripheral voltage monitor on V_{DDA} enabled

Bits 15:14 **SVOS**: System Stop mode voltage scaling selection

These bits control the V_{CORE} voltage level in system Stop mode, to obtain the best trade-off between power consumption and performance.

00: Reserved

01: SVOS5 Scale 5

10: SVOS4 Scale 4

11: SVOS3 Scale 3 (default)

Bits 13:10 Reserved, must be kept at reset value

Bit 9 **FLPS**: Flash low-power mode in DStop mode

This bit allows to obtain the best trade-off between low-power consumption and restart time when exiting from DStop mode.

When it is set, the Flash memory enters low-power mode when D1 domain is in DStop mode.

0: Flash memory remains in normal mode when D1 domain enters DStop (quick restart time).

1: Flash memory enters low-power mode when D1 domain enters DStop mode (low-power consumption).

Bit 8 **DBP**: Disable backup domain write protection

In reset state, the RCC_BDCR register, the RTC registers (including the backup registers), BREN and MONEN bits in PWR_CR2 register, are protected against parasitic write access. This bit must be set to enable write access to these registers.

0: Access to RTC, RTC Backup registers and backup SRAM disabled

1: Access to RTC, RTC Backup registers and backup SRAM enabled

Bits 7:5 **PLS**: Programmable voltage detector level selection

These bits select the voltage threshold detected by the PVD.

000: 1.95 V

001: 2.1 V

010: 2.25 V

011: 2.4 V

100: 2.55 V

101: 2.7 V

110: 2.85 V

111: External voltage level on PVD_IN pin, compared to internal V_{REFINT} level.

Note: Refer to Section "Electrical characteristics" of the product datasheet for more details.

Bit 4 **PVDE**: Programmable voltage detector enable

0: Programmable voltage detector disabled.

1: Programmable voltage detector enabled

Bits 3:1 Reserved, must be kept at reset value

Bit 0 **LPDS**: Low-power Deepsleep with SVOS3 (SVOS4 and SVOS5 always use low-power, regardless of the setting of this bit)

0: Voltage regulator or SMPS step-down converter in Main mode (MR) when SVOS3 is selected for Stop mode

1: Voltage regulator or SMPS step-down converter in Low-power mode (LPR) when SVOS3 is selected for Stop mode

7.8.2 PWR control status register 1 (PWR_CSR1)

Address offset: 0x004

Reset value: 0x0000 4000.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	AVDO
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ACTVOS	ACTVOS RDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PVDO	Res.	Res.	Res.	Res.
r	r										r				

Bits 31:17 Reserved, must be kept at reset value

Bit 16 **AVDO**: Analog voltage detector output on V_{DDA}

This bit is set and cleared by hardware. It is valid only if AVD on V_{DDA} is enabled by the AVDEN bit.

0: V_{DDA} is equal or higher than the AVD threshold selected with the ALS[2:0] bits.

1: V_{DDA} is lower than the AVD threshold selected with the ALS[2:0] bits

Note: Since the AVD is disabled in Standby mode, this bit is equal to 0 after Standby or reset until the AVDEN bit is set.

Bits 15:14 **ACTVOS**: VOS currently applied for V_{CORE} voltage scaling selection.

These bits reflect the last VOS value applied to the PMU.

Bit 13 **ACTVOSRDY**: Voltage levels ready bit for currently used VOS and SDLEVEL

This bit is set to 1 by hardware when the voltage regulator and the SMPS step-down converter are both disabled and Bypass mode is selected in PWR control register 3 (PWR_CR3).

0: Voltage level invalid, above or below current VOS and SDLEVEL selected levels.

1: Voltage level valid, at current VOS and SDLEVEL selected levels.

Bits 12:5 Reserved, must be kept at reset value

Bit 4 **PVDO**: Programmable voltage detect output

This bit is set and cleared by hardware. It is valid only if the PVD has been enabled by the PVDE bit.

0: V_{DD} or PVD_IN voltage is equal or higher than the PVD threshold selected through the PLS[2:0] bits.

1: V_{DD} or PVD_IN voltage is lower than the PVD threshold selected through the PLS[2:0] bits.

Note: since the PVD is disabled in Standby mode, this bit is equal to 0 after Standby or reset until the PVDE bit is set.

Bits 3:0 Reserved, must be kept at reset value

7.8.3 PWR control register 2 (PWR_CR2)

Address offset: 0x008

Reset value: 0x0000 0000

This register is not reset by wakeup from Standby mode, RESET signal and V_{DD} POR. It is only reset by V_{SW} POR and VSWRST reset.

This register shall not be accessed when VSWRST bit in RCC_BDCR register resets the V_{SW} domain.

After reset, PWR_CR2 register is write-protected. Prior to modifying its content, the DBP bit in PWR_CR1 register must be set to disable the write protection.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TEMPH	TEMPL	VBATH	VBATL	Res.	Res.	Res.	BRRDY
								r	r	r	r				r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MONEN	Res.	Res.	Res.	BREN
											rw				rw

Bits 31:24 Reserved, must be kept at reset value

Bit 23 **TEMPH**: Temperature level monitoring versus high threshold

0: Temperature below high threshold level.

1: Temperature equal or above high threshold level.

Bit 22 **TEMPL**: Temperature level monitoring versus low threshold

0: Temperature above low threshold level.

1: Temperature equal or below low threshold level.

Bit 21 **VBATH**: V_{BAT} level monitoring versus high threshold

0: V_{BAT} level below high threshold level.

1: V_{BAT} level equal or above high threshold level.

Bit 20 **VBATL**: V_{BAT} level monitoring versus low threshold

0: V_{BAT} level above low threshold level.

1: V_{BAT} level equal or below low threshold level.

Bits 19:17 Reserved, must be kept at reset value

Bit 16 **BRRDY**: Backup regulator ready

This bit is set by hardware to indicate that the Backup regulator is ready.

0: Backup regulator not ready.

1: Backup regulator ready.

Bits 15:5 Reserved, must be kept at reset value

Bit 4 **MONEN**: V_{BAT} and temperature monitoring enable

When set, the V_{BAT} supply and temperature monitoring is enabled.

0: V_{BAT} and temperature monitoring disabled.

1: V_{BAT} and temperature monitoring enabled.

Note: V_{BAT} and temperature monitoring are only available when the backup regulator is enabled (BREN bit set to 1).

Bits 3:1 Reserved, must be kept at reset value

Bit 0 **BREN**: Backup regulator enable

When set, the Backup regulator (used to maintain the backup RAM content in Standby and V_{BAT} modes) is enabled.

If BREN is reset, the backup regulator is switched off. The backup RAM can still be used in Run and Stop modes. However, its content will be lost in Standby and V_{BAT} modes.

If BREN is set, the application must wait till the Backup Regulator Ready flag (BRRDY) is set to indicate that the data written into the SRAM will be maintained in Standby and V_{BAT} modes.

0: Backup regulator disabled.

1: Backup regulator enabled.

7.8.4 PWR control register 3 (PWR_CR3)

Address offset: 0x00C

Reset value: 0x0000 0006 (reset only by POR only, not reset by wakeup from Standby mode and RESET pad).

The lower byte of this register is written once after POR and shall be written before changing VOS level or ck_sys clock frequency. No limitation applies to the upper bytes.

Programming data corresponding to an invalid combination of SDLEVEL, SDEXTHP, SDEN, LDOEN and BYPASS bits (see [Table 33](#)) will be ignored: data will not be written, the written-once mechanism will lock the register and any further write access will be ignored.

The default supply configuration will be kept and the ACTVOSRDY bit in [PWR control status register 1 \(PWR_CSR1\)](#) will go on indicating invalid voltage levels. The system shall be power cycled before writing a new value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	USB33RDY	USBREGEN	USB33DEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDEXTRDY
					r	rw	rw								r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	VBR	VBE	Res.	Res.	SDLEVEL	SDEXTHP	SDEN	LDOEN	BYPASS	
						rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:27 Reserved, must be kept at reset value

Bit 26 **USB33RDY**: USB supply ready.

0: USB33 supply not ready.

1: USB33 supply ready.

- Bit 25 **USBREGEN**: USB regulator enable.
 0: USB regulator disabled.
 1: USB regulator enabled.
- Bit 24 **USB33DEN**: $V_{DD33USB}$ voltage level detector enable.
 0: $V_{DD33USB}$ voltage level detector disabled.
 1: $V_{DD33USB}$ voltage level detector enabled.
- Bits 23:17 Reserved, must be kept at reset value
- Bit 16 **SDEXTRDY**: SMPS step-down converter external supply ready
 This bit is set by hardware to indicate that the external supply from the SMPS step-down converter is ready.
 0: External supply not ready.
 1: External supply ready.
- Bits 15:10 Reserved, must be kept at reset value
- Bit 9 **VBRS**: V_{BAT} charging resistor selection
 0: Charge V_{BAT} through a 5 k Ω resistor.
 1: Charge V_{BAT} through a 1.5 k Ω resistor.
- Bit 8 **VBE**: V_{BAT} charging enable
 0: V_{BAT} battery charging disabled.
 1: V_{BAT} battery charging enabled.
- Bits 7:6 Reserved, must be kept at reset value
- Bits 5:4 **SDLEVEL**⁽¹⁾: SMPS step-down converter voltage output level selection
 This bit is used when both the LDO and SMPS step-down converter are enabled with SDEN and LDOEN enabled or when SDEXTHP is enabled. In this case SDLEVEL has to be written with a value different than 00 at system startup.
 00: Reset value
 01: 1.8 V
 10 and 11: 2.5 V
- Bit 3 **SDEXTHP**⁽¹⁾: SMPS step-down converter forced ON and in High Power MR mode.
 0: SMPS step-down converter in normal operating mode.
 1: SMPS step-down converter forced ON and in MR mode.
- Bit 2 **SDEN**⁽¹⁾⁽²⁾: SMPS step-down converter enable
 0: SMPS step-down converter disabled
 1: SMPS step-down converter enabled. (Default)
- Bit 1 **LDOEN**⁽¹⁾: Low drop-out regulator enable
 0: Low drop-out regulator disabled.
 1: Low drop-out regulator enabled (default)
- Bit 0 **BYPASS**⁽¹⁾: Power management unit bypass
 0: Power management unit normal operation.
 1: Power management unit bypassed, voltage monitoring still active.

1. Illegal combinations of SDLEVEL, SDEXTHP, SDEN, LDOEN and BYPASS are described in [Table 33](#).

2. The SMPS step-down converter is not available on all packages. In this case, the SMPS step-down converter is disabled.

7.8.5 PWR CPU1 control register (PWR_CPU1CR)

This register allows controlling CPU1 power.

Address offset: 0x010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	RUN_D3	HOLD2	CSSF	SBF_D2	SBF_D1	SBF	STOPF	HOLD2F	Res.	PDDS_D3	PDDS_D2	PDDS_D1
				rw	rw	rw	r	r	r	r	r		rw	rw	rw

Bits 31:12 Reserved, must be kept at reset value

Bit 11 **RUN_D3**: Keep system D3 domain in Run mode regardless of the CPU subsystems modes

0: D3 domain follows CPU subsystems modes.

1: D3 domain remains in Run mode regardless of CPU subsystems modes.

Bit 10 **HOLD2**: Hold the CPU2 and allocated peripherals when exiting from Stop mode.

0: CPU2 and allocated peripherals are not affected, and start running when woken up from Stop mode.

1: CPU2 and allocated peripherals are on hold when woken up from Stop mode. Writing this bit to 0 will release the CPU2 and allocated peripherals.

Bit 9 **CSSF**: Clear D1 domain CPU1 Standby, Stop and HOLD flags (always read as 0)

This bit is cleared to 0 by hardware.

0: No effect.

1: D1 domain CPU1 flags (HOLD2F, STOPF, SBF, SBF_D1, and SBF_D2) are cleared.

Bit 8 **SBF_D2**: D2 domain DStandby flag

This bit is set by hardware and cleared by any system reset or by setting the CPU1 CSSF bit. Once set, this bit can be cleared only when the D2 domain is no longer in DStandby mode.

0: D2 domain has not been in DStandby mode

1: D2 domain has been in DStandby mode.

Bit 7 **SBF_D1**: D1 domain DStandby flag

This bit is set by hardware and cleared by any system reset or by setting the CPU1 CSSF bit. Once set, this bit can be cleared only when the D1 domain is no longer in DStandby mode.

0: D1 domain has not been in DStandby mode

1: D1 domain has been in DStandby mode.

Bit 6 **SBF**: System Standby flag

This bit is set by hardware and cleared only by a POR (Power-on Reset) or by setting the CPU1 CSSF bit

0: System has not been in Standby mode

1: System has been in Standby mode

Bit 5 **STOPF**: STOP flag

This bit is set by hardware and cleared only by any reset or by setting the CPU1 CSSF bit.

0: System has not been in Stop mode

1: System has been in Stop mode

Bit 4 **HOLD2F**: CPU2 on hold wakeup flag.

This flag also generates a CPU1 interrupt. CPU1 has been woken up from a CPU2 wakeup source with CPU2 on hold. This flag is set by hardware and cleared only by a system reset or by setting the CPU1 CSSF bit.

0: No CPU2 system wake up with hold.

1: CPU2 system wake up with hold.

Bit 3 Reserved, must be kept at reset value

Bit 2 **PDDS_D3**: System D3 domain Power Down Deepsleep.

This bit allows CPU1 to define the Deepsleep mode for System D3 domain.

0: Keep Stop mode when D3 domain enters Deepsleep.

1: Allow Standby mode when D3 domain enters Deepsleep.

Bit 1 **PDDS_D2**: D2 domain Power Down Deepsleep.

This bit allows CPU1 to define the Deepsleep mode for D2 domain.

0: Keep DStop mode when D2 domain enters Deepsleep.

1: Allow DStandby mode when D2 domain enters Deepsleep.

Bit 0 **PDDS_D1**: D1 domain Power Down Deepsleep selection.

This bit allows CPU1 to define the Deepsleep mode for D1 domain.

0: Keep DStop mode when D1 domain enters Deepsleep.

1: Allow DStandby mode when D1 domain enters Deepsleep.

7.8.6 PWR CPU2 control register (PWR_CPU2CR)

This register allows controlling CPU2 power.

Address offset: 0x014

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	RUN_D3	HOLD1	CSSF	SBF_D2	SBF_D1	SBF	STOPF	HOLD1F	Res.	PDDS_D3	PDDS_D2	PDDS_D1
				rw	rw	rw	r	r	r	r	r		rw	rw	rw

Bits 31:12 Reserved, must be kept at reset value

Bit 11 **RUN_D3**: Keep D3 domain in Run mode regardless of the other CPU subsystems modes.

0: D3 domain follows the CPU subsystems modes.

1: D3 domain remains in Run mode regardless of the CPU subsystems modes.

Bit 10 **HOLD1**: Hold the CPU1 and allocated peripherals when exiting from Stop mode.

0: CPU1 and allocated peripherals are not affected, and start running when woken up from Stop mode.

1: CPU1 and allocated peripherals are on hold when woken up from Stop mode. Writing this bit to 0 will release the CPU1 and allocated peripherals.

Bit 9 **CSSF**: Clear D2 domain CPU2 Standby, Stop and HOLD flags (always read as 0)

This bit is cleared to 0 by hardware.

0: No effect.

1: D2 domain CPU2 flags (HOLD1F, STOPF, SBF, SBF_D1, and SBF_D2) cleared.

Bit 8 **SBF_D2**: D2 domain DStandby flag

This bit is set by hardware and cleared by any system Reset or by setting the CPU2 CSSF bit. Once set, it can be cleared only when the D2 domain is no longer in DStandby mode.

0: D2 domain has not been in DStandby mode.

1: D2 domain has been in DStandby mode.

Bit 7 **SBF_D1**: D1 domain DStandby flag

This bit is set by hardware and cleared by any Reset or by setting the CPU2 CSSF bit. Once set, this bit can be cleared only when the D1 domain is no longer in DStandby mode.

0: D2 domain has not been in DStandby mode

1: D2 domain has been in DStandby mode.

Bit 6 **SBF**: System Standby flag

This bit is set by hardware and cleared only by a POR (Power-on Reset) or by setting the CPU2 CSSF bit.

0: System has not been in Standby mode.

1: System has been in Standby mode.

Bit 5 **STOPF**: Stop Flag

This bit is set by hardware and cleared only by any reset or by setting the CPU2 CSSF bit.

0: System has not been in Stop mode.

1: System has been in Stop mode.

Bit 4 **HOLD1F**: CPU1 in hold wakeup flag.

This flag also generates a CPU2 interrupt.

CPU2 has been woken up from a CPU1 wakeup source with CPU1 on hold. This flag is set by hardware and cleared only by a system reset or by setting the CPU2 CSSF bit.

0: No CPU1 system wake up with hold.

1: CPU1 system wake up with hold.

Bit 3 Reserved, must be kept at reset value

Bit 2 **PDDS_D3**: System D3 domain Power Down Deepsleep.

This bit allows CPU2 to define the Deepsleep mode for System D3 domain.

0: Keep Stop mode when D3 domain enters Deepsleep.

1: Allow Standby mode when D3 domain enters Deepsleep.

Bit 1 **PDDS_D2**: D2 domain Power Down Deepsleep.

This bit allows CPU2 to define the Deepsleep mode for D2 domain.

0: Keep DStop mode when D2 domain enters Deepsleep.

1: Allow DStandby mode when D2 domain enters Deepsleep.

Bit 0 **PDDS_D1**: D1 domain Power Down Deepsleep selection.

This bit allows CPU2 to define the Deepsleep mode for D1 domain.

0: Keep DStop mode when D1 domain enters to Deepsleep.

1: Allow DStandby mode when D1 domain enters to Deepsleep.

7.8.7 PWR D3 domain control register (PWR_D3CR)

This register allows controlling D3 domain power.

Address offset: 0x018

Reset value: 0x0000 4000 (Following reset VOSRDY will be read 1 by software).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VOS	VOSRDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw	r														

Bits 31:16 Reserved, must be kept at reset value

Bits 15:14 **VOS**: Voltage scaling selection according to performance

These bits control the V_{CORE} voltage level and allow to obtains the best trade-off between power consumption and performance:

- When increasing the performance, the voltage scaling shall be changed before increasing the system frequency.
 - When decreasing performance, the system frequency shall first be decreased before changing the voltage scaling.
- 00: Reserved (Scale 3 selected).
 01: Scale 3 (default)
 10: Scale 2
 11: Scale 1

Bit 13 **VOSRDY**: VOS Ready bit for V_{CORE} voltage scaling output selection.

This bit is set to 1 by hardware when Bypass mode is selected in PWR control register 3 (PWR_CR3).

- 0: Not ready, voltage level below VOS selected level.
 1: Ready, voltage level at or above VOS selected level.

Bits 12:0 Reserved, must be kept at reset value

7.8.8 PWR wakeup clear register (PWR_WKUPCR)

Address offset: 0x020

Reset value: 0x0000 0000 (reset only by system reset, not reset by wakeup from Standby mode)

5 wait states are required when writing this register (when clearing a WKUPF bit in PWR_WKUPFR, the AHB write access will complete after the WKUPF has been cleared).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WKUPC6	WKUPC5	WKUPC4	WKUPC3	WKUPC2	WKUPC1
										rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:6 Reserved, always read as 0.

Bits 5:0 **WKUPCn**: Clear Wakeup pin flag for WKUPn.

These bits are always read as 0.

0: No effect

1: Writing 1 clears the WKUPFn Wakeup pin flag (bit is cleared to 0 by hardware)

7.8.9 PWR wakeup flag register (PWR_WKUPFR)

Address offset: 0x024

Reset value: 0x0000 0000 (reset only by system reset, not reset by wakeup from Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WKUPF6	WKUPF5	WKUPF4	WKUPF3	WKUPF2	WKUPF1
										r	r	r	r	r	r

Bits 31:6 Reserved, must be kept at reset value

Bits 5:0 **WKUPn**: Wakeup pin WKUPn flag.

This bit is set by hardware and cleared only by a Reset pin or by setting the WKUPCn bit in the [PWR wakeup clear register \(PWR_WKUPCR\)](#).

0: No wakeup event occurred

1: A wakeup event was received from WKUPn pin

7.8.10 PWR wakeup enable and polarity register (PWR_WKUPEPR)

Address offset: 0x028

Reset value: 0x0000 0000 (reset only by system reset, not reset by wakeup from Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	WKUPPUPD6		WKUPPUPD5		WKUPPUPD4		WKUPPUPD3		WKUPPUPD2		WKUPPUPD1	
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	WKUPP6	WKUPP5	WKUPP4	WKUPP3	WKUPP2	WKUPP1	Res.	Res.	WKUPEN6	WKUPEN5	WKUPEN4	WKUPEN3	WKUPEN2	WKUPEN1
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:28 Reserved, must be kept at reset value

Bits 27:16 **WKUPPUPD: Wakeup pin pull configuration for WKUP(truncate(n/2)-7)**

These bits define the I/O pad pull configuration used when WKUPEN(truncate(n/2)-7) = 1. The associated GPIO port pull configuration shall be set to the same value or to '00'.

The Wakeup pin pull configuration is kept in Standby mode.

00: No pull-up

01: Pull-up

10: Pull-down

11: Reserved

Bits 15:14 Reserved, must be kept at reset value

Bits 13:8 **WKUPPn-7**: Wakeup pin polarity bit for WKUPn-7

These bits define the polarity used for event detection on WKUPn-7 external wakeup pin.

0: Detection on high level (rising edge)

1: Detection on low level (falling edge)

Bits 7:6 Reserved, must be kept at reset value

Bits 5:0 **WKUPENn**: Enable Wakeup Pin WKUPn

Each bit is set and cleared by software.

0: An event on WKUPn pin does not wakeup the system from Standby mode.

1: A rising or falling edge on WKUPn pin wakes-up the system from Standby mode.

Note: An additional wakeup event is detected if WKUPn pin is enabled (by setting the WKUPENn bit) when WKUPn pin level is already high when WKUPPn selects rising edge, or low when WKUPPn selects falling edge.

7.8.11 PWR register map

Table 50. Power control register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x000	PWR_CR1	Reserved												ALS		AVDEN	SVOS	Reserved				FLPS	DBP	PLS			PVDE	Reserved			LPDS			
	Reset value														0	0	0	1	1					0	0	0	0	0	0				0	
0x004	PWR_CSR1	Reserved														AVDO	ACTVOS	ACTVOSRDY	Reserved						PVDO	Reserved								
	Reset value																0	0	1	0									0					
0x008	PWR_CR2	Reserved								TEMPH	TEMPL	VBATH	VBATL	Reserved			BRRDY	Reserved										MONEN	Reserved		BREN			
	Reset value									0	0	0	0				0												0				0	
0x00C	PWR_CR3	Reserved				USB33RDY	USBREGEN	USB33DEN		Reserved	Reserved	Reserved	Reserved	Reserved	Reserved	Reserved	SDEXTRDY	Reserved				VBR	VBE	Reserved			SDLEVEL	SDEXTHP	SDEN	LDOEN	BYPASS			
	Reset value					0	0	0								0												0	0	1	1	0		
0x010	PWR_CPU1CR	Reserved																			RUN_D3	HOLD2	CSSF	SBF_D2	SBF_D1	SBF	STOPF	HOLD2F	PDDS_D3	PDDS_D2	PDDS_D1			
	Reset value																				0	0	0	0	0	0	0	0	0	0	0	0	0	
0x014	PWR_CPU2CR	Reserved																			RUN_D3	HOLD1	CSSF	SBF_D2	SBF_D1	SBF	STOPF	HOLD1F	PDDS_D3	PDDS_D2	PDDS_D1			
	Reset value																				0	0	0	0	0	0	0	0	0	0	0	0	0	
0x018	PWR_D3CR	Reserved															VOS	VOSRDY	Reserved															
	Reset value																0	1	0															
0x020	PWR_WKUPCR	Reserved																											WKUPC6	WKUPC5	WKUPC4	WKUPC3	WKUPC2	WKUPC1
	Reset value																											0	0	0	0	0	0	
0x024	PWR_WKUPFR	Reserved																											WKUPF6	WKUPF5	WKUPF4	WKUPF3	WKUPF2	WKUPF1
	Reset value																											0	0	0	0	0	0	
0x028	PWR_WKUPEPR	Reserved				WKUPUPD6	WKUPUPD5			WKUPUPD4		WKUPUPD3		WKUPUPD2		WKUPUPD1		res.	WKUPP6	WKUPP5	WKUPP4	WKUPP3	WKUPP2	WKUPP1			res.	WKUPEN6	WKUPEN5	WKUPEN4	WKUPEN3	WKUPEN2	WKUPEN1	
	Reset value					0	0	0	0	0	0	0	0	0	0	0			0	0	0	0	0	0	0			0	0	0	0	0	0	

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

9 Reset and Clock Control (RCC)

The RCC block manages the clock and reset generation for the whole microcontroller, which embeds two CPUs: an Arm® Cortex®-M7 and an Arm® Cortex®-M4, called CPU1 and CPU2, respectively.

The RCC block is located in the D3 domain (refer to [Section 7: Power control \(PWR\)](#) for a detailed description).

The operating modes this section refers to are defined in [Section 7.6.1: Operating modes](#) of the PWR block.

9.1 RCC main features

Reset block

- Generation of local and system reset
- Bidirectional pin reset allowing to reset the microcontroller or external devices
- Hold Boot function
- WWDG reset supported

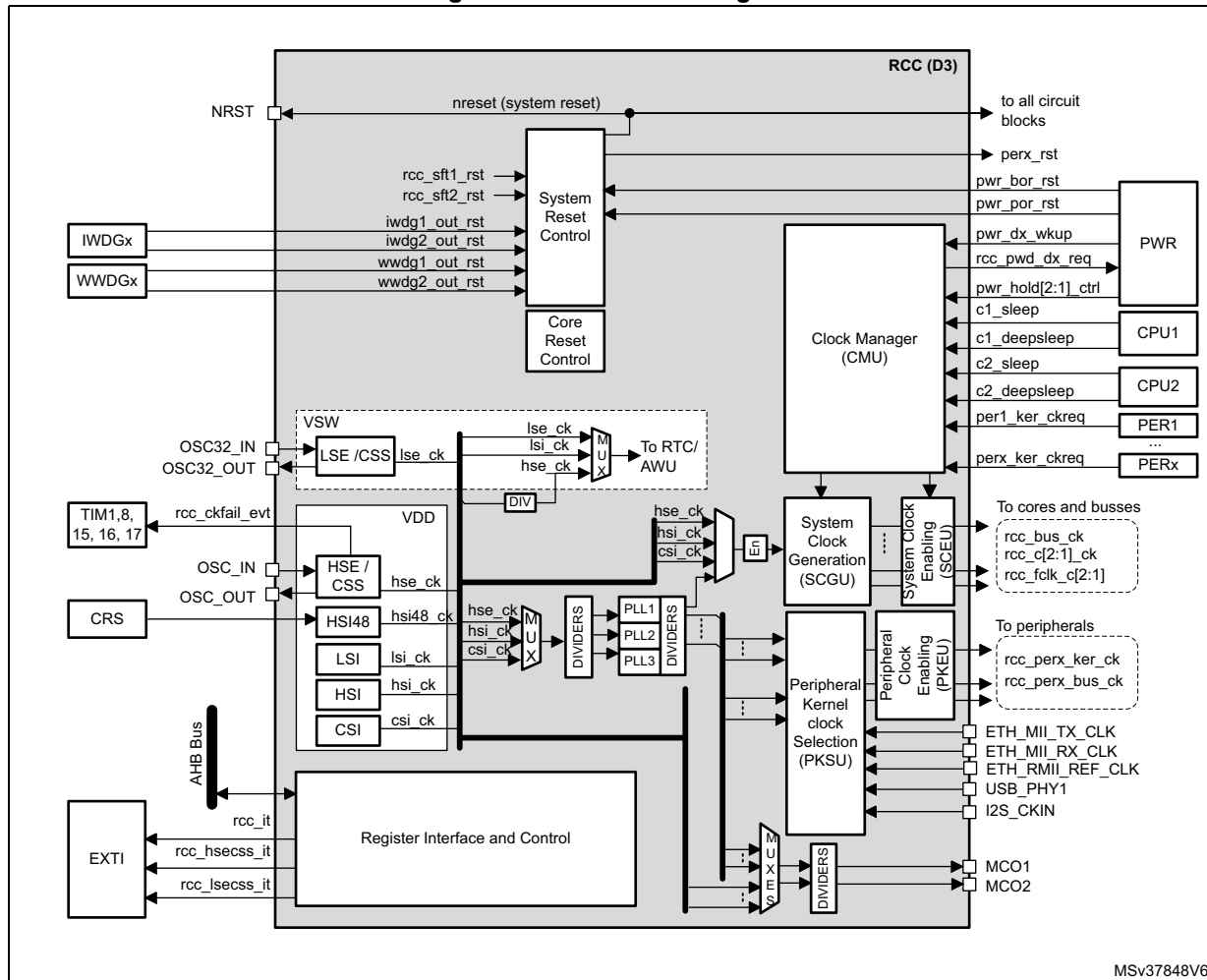
Clock generation block

- Generation and dispatching of clocks for the complete device
- 3 separate PLLs using integer or fractional ratios
- Possibility to change the PLL fractional ratios on-the-fly
- Smart clock gating to reduce power dissipation
- 2 external oscillators:
 - High-speed external oscillator (HSE) supporting a wide range of crystals from 4 to 48 MHz frequency
 - Low-speed external oscillator (LSE) for the 32 kHz crystals
- 4 internal oscillators
 - High-speed internal oscillator (HSI)
 - 48 MHz RC oscillator (HSI48)
 - Low-power Internal oscillator (CSI)
 - Low-speed internal oscillator (LSI)
- Buffered clock outputs for external devices
- Generation of two types of interrupts lines:
 - Dedicated interrupt lines for clock security management
 - One general interrupt line for other events
- Clock generation handling in Stop and Standby mode
- D3 domain Autonomous mode

9.2 RCC block diagram

Figure 48 shows the RCC block diagram.

Figure 48. RCC Block diagram



MSv37848V6

9.3 RCC pins and internal signals

Table 54 lists the RCC inputs and output signals connected to package pins or balls.

Table 54. RCC input/output signals connected to package pins or balls

Signal name	Signal type	Description
NRST	Digital input/output	System reset, can be used to provide reset to external devices
OSC32_IN	Digital input	32 kHz oscillator input
OSC32_OUT	Digital output	32 kHz oscillator output
OSC_IN	Digital input	System oscillator input

Table 54. RCC input/output signals connected to package pins or balls (continued)

Signal name	Signal type	Description
OSC_OUT	Digital output	System oscillator output
MCO1	Digital output	Output clock 1 for external devices
MCO2	Digital output	Output clock 2 for external devices
I2S_CKIN	Digital input	External kernel clock input for digital audio interfaces: SPI/I2S, SAI, and DFSDM
ETH_MII_TX_CLK	Digital input	External TX clock provided by the Ethernet MII interface
ETH_MII_RX_CLK	Digital input	External RX clock provided by the Ethernet MII interface
ETH_RMII_REF_CLK	Digital input	External reference clock provided by the Ethernet RMII interface
USB_PHY1	Digital input	USB clock input provided by the external USB PHY (OTG_HS_ULPI_CK)

The RCC exchanges a lot of internal signals with all components of the product, for that reason, the [Table 54](#) only shows the most significant internal signals.

Table 55. RCC internal input/output signals

Signal name	Signal type	Description
rcc_it	Digital output	General interrupt request line
rcc_hsecss_it	Digital output	HSE clock security failure interrupt
rcc_lsecss_it	Digital output	LSE clock security failure interrupt
rcc_ckfail_evt	Digital output	Event indicating that a HSE clock security failure is detected. This signal is connected to TIMERS
nreset	Digital input/output	System reset
iwdg[2:1]_out_rst	Digital input	Reset line driven by the IWDG1 and IWDG2, indicating that a timeout occurred.
wwdg[2:1]_out_rst	Digital input	Reset line driven by the WWDG1 and WWDG2, indicating that a timeout occurred.
pwr_bor_rst	Digital input	Brownout reset generated by the PWR block
pwr_por_rst	Digital input	Power-on reset generated by the PWR block
pwr_vsw_rst	Digital input	Power-on reset of the VSW domain generated by the PWR block
rcc_perx_rst	Digital output	Reset generated by the RCC for the peripherals.
pwr_d[3:1]_wkup	Digital input	Wake-up domain request generated by the PWR. Generally used to restore the clocks a domain when this domain exits from DStop
rcc_pwd_d[3:1]_req	Digital output	Low-Power request generated by the RCC. Generally used to ask to the PWR to set a domain into low-power mode, when a domain is in DStop.
pwr_hold[2:1]_ctrl	Digital input	Signals generated by the PWR, in order to set one processor into CStop when exiting from system Stop mode.

Table 55. RCC internal input/output signals (continued) (continued)

Signal name	Signal type	Description
c[2:1]_sleep	Digital input	Signal generated by CPU1 or CPU2, indicating if the corresponding CPU is in CRun, CSleep or CStop.
c[2:1]_deepsleep	Digital input	
perx_ker_ckreq	Digital input	Signal generated by some peripherals in order to request the activation of their kernel clock.
rcc_perx_ker_ck	Digital output	Kernel clock signals generated by the RCC, for some peripherals.
rcc_perx_bus_ck	Digital output	Bus interface clock signals generated by the RCC for peripherals.
rcc_bus_ck	Digital output	Clocks for APB (rcc_apb_ck), AHB (rcc_ahb_ck) and AXI (rcc_axi_ck) bridges generated by the RCC.
rcc_c[2:1]_ck	Digital output	CPU1 and CPU2 clocks generated by the RCC.
rcc_fclk_c[2:1]	Digital output	

9.4 RCC reset block functional description

Several sources can generate a reset:

- An external device via NRST pin
- A failure on the supply voltage applied to V_{DD}
- A watchdog timeout
- A software command

The reset scope depends on the source that generates the reset. Three reset categories exist:

- Power-on/off reset
- System reset
- Local resets

9.4.1 Power-on/off reset

The power-on/off reset (**pwr_por_rst**) is generated by the power controller block (PWR). It is activated when the input voltage (V_{DD}) is below a threshold level. This is the most complete reset since it resets the whole circuit, except for the backup domain. The power-on/off reset function can be disabled through PDR_ON pin (see [Section 7.5: Power supply supervision](#)).

Refer to [Table 56: Reset distribution summary](#) for details.

9.4.2 System reset

A system reset (**nreset**) resets all registers to their reset values unless otherwise specified in the register description.

A system reset can be generated from one of the following sources:

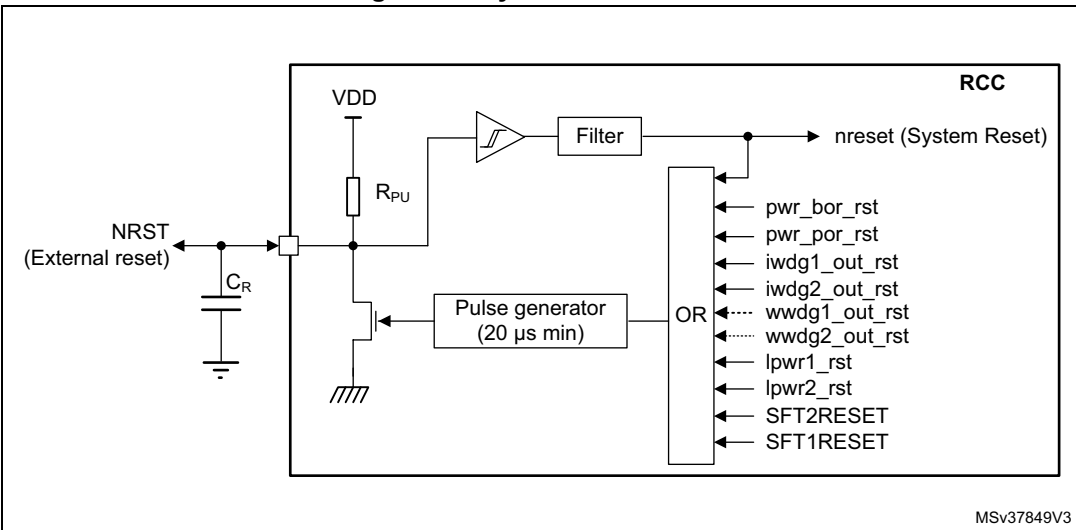
- A reset from NRST pin (external reset)
- A reset from the power-on/off reset block (**pwr_por_rst**)
- A reset from the brownout reset block (**pwr_bor_rst**)
Refer to [Section 7.5.2: Brownout reset \(BOR\)](#) for a detailed description of the BOR function.
- A reset from the independent watchdogs (**iwdg[2:1]_out_rst**)
- A software reset from the Cortex[®]-M4 core
It is generated via the SYSRESETREQ signal issued by the Cortex[®]-M4 core. This signal is also named SFT2RESET in this document.
- A software reset from the Cortex[®]-M7 core
It is generated via the SYSRESETREQ signal issued by the Cortex[®]-M7 core. This signal is also named SFT1RESET in this document.
- A reset from the window watchdogs (**wwdg[2:1]_out_rst**)
- A reset from the low-power mode security reset, depending on option byte configuration (**lpwr[2:1]_rst**)

Note: The SYSRESETREQ bit in Cortex[®]-M7 Application Interrupt and Reset Control Register must be set to force a software reset on the device. Refer to the Cortex[®]-M7 with FPU technical reference manual for more details (see <http://infocenter.arm.com>).

As shown in [Figure 49](#), some internal sources (such as **pwr_por_rst**, **pwr_bor_rst**, **iwdg[2:1]_out_rst**) perform a system reset of the circuit, which is also propagated to the NRST pin to reset the connected external devices. The pulse generator guarantees a minimum reset pulse duration of 20 µs for each internal reset source. In case of an external reset, the reset pulse is generated while the NRST pin is asserted Low.

Note: It is not recommended to let the NRST pin unconnected. When it is not used, connect this pin to ground via a 10 to 100 nF capacitor (C_R in [Figure 49](#)).

Figure 49. System reset circuit



MSv37849V3

9.4.3 Local resets

Domain reset

Some resets also depend on the domain status. For example, when D1 domain exits from DStandby, it is reset (**d1_rst**). The same mechanism applies to D2.

When the system exits from Standby mode, an **stby_rst** reset is applied. The **stby_rst** signal generates a reset of the complete V_{CORE} domain as long as the V_{CORE} voltage provided by the internal regulator is not valid.

[Table 56](#) gives a detailed overview of reset sources and scopes.

Table 56. Reset distribution summary

Reset source	Reset name	D1 CPU	D1 Interconnect	D1 Peripherals	D1 Debug	D1 WWDG1	D2 CPU	D2 Interconnect	D2 Peripherals	D2 Debug	D2 WWDG2	D3 Peripherals	IWDG1	IWDG2	FLASH	RTC domain	Backup RAM	System Supply	NRST pin	Comments
Pin	NRST	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	– Resets D1 and D2 domains, and all their peripherals – Resets D3 domain peripherals – Resets V_{DD} domain: IWDG[2:1], LDO... – Debug features, Flash memory, RTC and backup RAM are not reset

Table 56. Reset distribution summary (continued)

Reset source	Reset name	D1 CPU	D1 Interconnect	D1 Peripherals	D1 Debug	D1 WWDG1	D2 CPU	D2 Interconnect	D2 Peripherals	D2 Debug	D2 WWDG2	D3 Peripherals	IWDG1	IWDG2	FLASH	RTC domain	Backup RAM	System Supply	NRST pin	Comments
PWR	pwr_bor_rst	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	- Same as pin reset. The pin is asserted as well.
	pwr_por_rst	x	x	x	x	x	x	x	x	x	x	x	x	x	x	-	-	x	x	- Same as pwr_bor_rst reset, plus: Reset of the Flash memory digital block (including the option byte loading). Reset of the debug block
	lpwr2_rst lpwr1_rst	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	- The low-power mode security reset has the same scope than pwr_por_rst . Refer to Section 9.4.5: Low-power mode security reset (lpwr[2:1]_rst) for additional information.
RCC	BDRST	-	-	-	-	-	-	-	-	-	-	-	-	-	-	x	-	-	-	- The backup domain reset can be triggered by software. Refer to Section 9.4.6: Backup domain reset for additional information
	d1_rst	x	x	x	x	x	-	-	-	-	-	-	-	-	-	-	-	-	-	- Resets D1 domain, and all its peripherals, when the domain exits DStandby mode.
	d2_rst	-	-	-	-	-	x	x	x	x	x	-	-	-	-	-	-	-	-	- Resets D2 domain, and all its peripherals, when the domain exits DStandby mode.
	stby_rst	x	x	x	x	x	x	x	x	x	x	x	-	-	-	-	-	-	-	- When the device exits Standby mode, a reset of the complete V _{CORE} domain is performed as long the V _{CORE} voltage is not valid. The V _{CORE} is supplied by the internal regulator. NRST signal is not asserted.
CPU1	SFT1RESET	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	- This reset is generated by software when writing SYSRESETREQ bit located into AIRCR register of the Cortex [®] -M7 core. - Same scope as pwr_bor_rst reset.
CPU2	SFT2RESET	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	- This reset is generated by software when writing SYSRESETREQ bit located into AIRCR register of the Cortex [®] -M4. - Same scope as pwr_bor_rst reset.

Table 56. Reset distribution summary (continued)

Reset source	Reset name	D1 CPU	D1 Interconnect	D1 Peripherals	D1 Debug	D1 WWDG1	D2 CPU	D2 Interconnect	D2 Peripherals	D2 Debug	D2 WWDG2	D3 Peripherals	IWDG1	IWDG2	FLASH	RTC domain	Backup RAM	System Supply	NRST pin	Comments
Backup domain	pwr_vsw_rst	-	-	-	-	-	-	-	-	-	-	-	-	-	-	x	-	-	-	- This reset is generated by the backup domain when the V _{SW} supply voltage is outside the operating range.
IWDG1	iwdg1_out_rst	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	- Same as pwr_bor_rst reset.
IWDG2	iwdg2_out_rst	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	- Same as pwr_bor_rst reset.
WWDG1	wwdg1_out_rst	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	- Same as pwr_bor_rst reset.
WWDG2	wwdg2_out_rst	x	x	x	-	x	x	x	x	-	x	x	x	x	-	-	-	-	x	- Same as pwr_bor_rst reset.

9.4.4 Reset source identification

Each CPU can identify the reset source by checking the reset flags in the RCC_C1_RSR and RCC_C2_RSR registers:

- CPU1 can check the reset source by reading either RCC_RSR or RCC_C1_RSR registers.
- CPU2 can check the reset the source by reading either RCC_RSR or RCC_C2_RSR registers.

Each CPU can reset the flags of its own register by setting RMVF bit without interfering with the other CPU (refer to [RCC reset status register \(RCC_RSR\)](#) for a detailed description).

[Table 57](#) shows how the status bits of RCC_RSR register behaves, according to the situation that generated the reset. For example when an IWDG1 timeout occurs (line #10), if the CPU is reading the RCC_RSR register during the boot phase, both PINRSTF and IWDG1RSTF bits are set, indicating that the IWDG1 also generated a pin reset.

Table 57. Reset source identification (RCC_RSR)⁽¹⁾

#	Situations Generating a Reset	LPWR2RSTF	LPWR1RSTF	WWDG2RSTF	WWDG1RSTF	IWDG2RSTF	IWDG1RSTF	SFT2RSTF	SFT1RSTF	PORRSTF	PINRSTF	BORRSTF	D2RSTF	D1RSTF
1	Power-on reset (pwr_por_rst)	0	0	0	0	0	0	0	0	1	1	1	1	1
2	Pin reset (NRST)	0	0	0	0	0	0	0	0	0	1	0	0	0
3	Brownout reset (pwr_bor_rst)	0	0	0	0	0	0	0	0	0	1	1	0	0
4	System reset generated by CPU1 (STF1RESET)	0	0	0	0	0	0	0	1	0	1	0	0	0
5	System reset generated by CPU2 (STF2RESET)	0	0	0	0	0	0	1	0	0	1	0	0	0
6	WWDG1 reset (wwdg1_out_rst)	0	0	0	1	0	0	0	0	0	1	0	0	0
7	WWDG2 reset (wwdg2_out_rst)	0	0	1	0	0	0	0	0	0	1	0	0	0
8	IWDG1 reset (iwdg1_out_rst)	0	0	0	0	0	1	0	0	0	1	0	0	0
9	IWDG2 reset (iwdg2_out_rst)	0	0	0	0	1	0	0	0	0	1	0	0	0
10	D1 exits DStandby mode	0	0	0	0	0	0	0	0	0	0	0	0	1
11	D2 exits DStandby mode	0	0	0	0	0	0	0	0	0	0	0	1	0
12	D1 erroneously enters DStandby mode or CPU1 erroneously enters CStop mode	0	1	0	0	0	0	0	0	0	1	0	0	0
13	D2 erroneously enters DStandby mode or CPU2 erroneously enters CStop mode	1	0	0	0	0	0	0	0	0	1	0	0	0

1. Grayed cells highlight the register bits that are set.

9.4.5 Low-power mode security reset (lpwr[2:1]_rst)

To prevent critical applications from mistakenly enter a low-power mode, two low-power mode security resets are available. When enabled through nRST_STOP_C[2:1] option bytes, a system reset is generated if the following conditions are met:

- CPU1 and/or CPU2 accidentally enters CStop mode
This type of reset is enabled by resetting nRST_STOP_C[2:1] user option bytes. In this case, whenever a CPU1/2 CStop mode entry sequence is successfully executed, a system reset is generated.
- D1 domain and/or D2 accidentally enters DStandby mode
This type of reset is enabled by resetting nRST_STDBY_D[2:1] user option bytes. In this case, whenever a domain D1 or D2 DStandby mode entry sequence is successfully executed, a system reset is generated.

LPWR[2:1]RSTF bits in [RCC reset status register \(RCC_RSR\)](#) indicate that a low-power mode security reset occurred (see line #14 and #15 in [Table 57](#)).

lpwr1_rst is activated when a low-power mode security reset due to D1 or CPU1 occurred, while **lpwr2_rst** is activated when a low-power mode security reset due to D2 or CPU2 occurred.

Refer to [Section 4.4: FLASH option bytes](#) for additional information.

9.4.6 Backup domain reset

A backup domain reset is generated when one of the following events occurs:

- A software reset, triggered by setting BDRST bit in the [RCC Backup domain control register \(RCC_BDCR\)](#). All RTC registers and the RCC_BDCR register are reset to their default values. The backup RAM is not affected.
- V_{SW} voltage is outside the operating range. All RTC registers and the RCC_BDCR register are reset to their default values. In this case the content of the backup RAM is no longer valid.

There are two ways to reset the backup RAM:

- through the Flash memory interface by requesting a protection level change from 1 to 0
- when a tamper event occurs.

Refer to [Section 7.4.4: Backup domain](#) section of PWR block for additional information.

9.4.7 Power-on and wakeup sequences

For detailed diagrams refer to [Section 7.4.1: System supply startup](#) in the PWR section.

The time interval between the event which exits the product from a low-power and the moment where the CPUs are able to execute code, depends on the system state and on its configuration. [Figure 50](#) shows the most usual examples.

Power-on wakeup sequence

The power-on wakeup sequence shown in [Figure 50](#) gives the most significant phases of the power-on sequence. It is the longest sequence since the circuit was not powered. Note that this sequence remains unchanged, whatever V_{BAT} was present or not.

Boot from pin reset

When a pin reset occurs, V_{DD} is still present. As a result:

- The regulator settling time is faster since the reference voltage is already stable.
- The HSI restart delay may be needed if the HSI was not enabled when the NRST occurred, otherwise this restart delay phase is skipped.
- The Flash memory power recovery delay can also be skipped if the Flash memory was enabled when the NRST occurred.

Note: The boot sequence is similar for *pwr_bor_rst*, *lpwr_rst*, *STFxRESET*, *wdgx_out_rst* and *wwdgx_out_rst*.

Boot from system Standby

When waking up from system Standby, the reference voltage is stable since V_{DD} has not been removed. As a result, the regulator settling time is fast. Since V_{CORE} was not present, the restart delay for the HSI, the Flash memory power recovery and the option byte reloading cannot be skipped.

Restart from system Stop

When restarting from system Stop, V_{DD} is still present. As a result, the sequence is mainly composed of two steps:

1. Regulator settling time to reach VOS3 (default voltage)
2. HSI/CSI restart delay. This step can be skipped if HSIKERON or CSIKERON bit, in [RCC source control register \(RCC_CR\)](#) is set to '1'.

Boot from domain DStandby

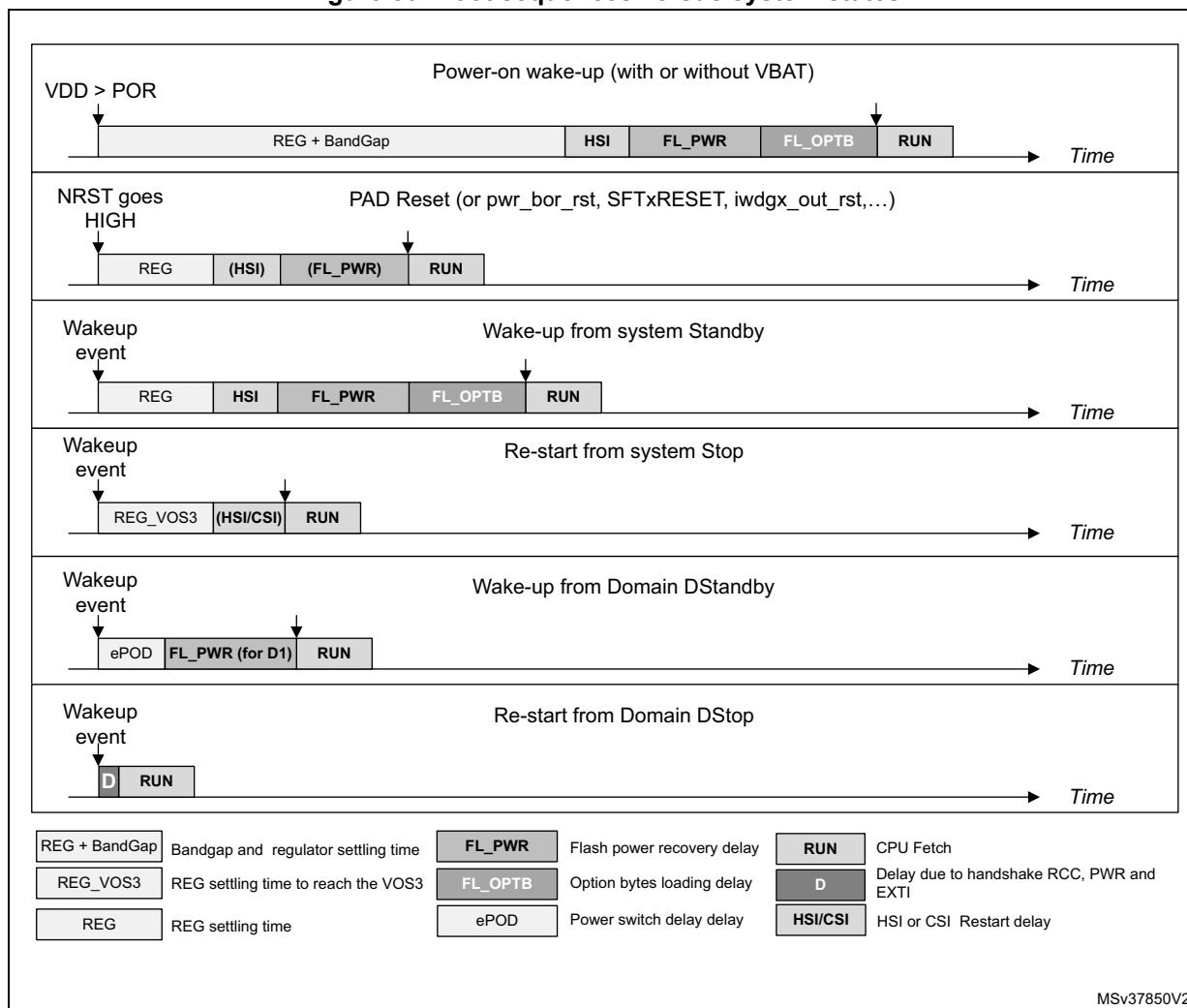
The boot sequence of a domain from domain DStandby is mainly composed of two steps:

1. The power switch settling time (the regulator is already activated).
2. The Flash memory power recovery, if the domain which exits from DStandby is D1, otherwise this step is skipped.

Restart from domain DStop

The restart sequence of a domain from domain DStop is mainly composed of the handshake between the RCC, EXTI and PWR blocks.

Figure 50. Boot sequences versus system states



Hold boot function

It is possible to put one of the two CPUs on hold after a system reset or when the **device** exits from Standby. This function is controlled by two option bytes, BCM4 and BCM7, and two bits, BOOT_C1 and BOOT_C2, located in the [RCC global control register \(RCC_GCR\)](#):

- When BCM4 or BCM7 option byte is set to '1', the corresponding CPU is allowed to boot after a system reset or when the device exits from Standby.
- When BCM4 or BCM7 option byte is set to '0', the corresponding CPU is not allowed to boot, and remains on hold as long as the corresponding BOOT_Cx bit is set to '0'.
- CPUx will be allowed to boot when BOOT_Cx is set to '1'. A protection mechanism insures that at least one of the 2 CPUs will boot (see [Table 58](#)).

The hold boot function is provided in order to support configurations where one CPU is master of the system and shall complete the initialization of the whole **device** (system clock, voltage scaling...) before activating the second CPU.

Note: After a system reset or system Standby, the BOOT_C1 and BOOT_C2 bits are set to '0'.

Table 58. Boot enable Function ⁽¹⁾

BCM7	BCM4	BOOT_C1	BOOT_C2		CPU1 State	CPU2 State	Comments
0	1	0	X	→	HOLD	ENABLED	After a system reset or System Standby, CPU2 will boot and CPU1 will remain on hold until CPU2 sets BOOT_C1 to '1'
		1	X	→	ENABLED	ENABLED	When CPU2 writes BOOT_C1 to '1', CPU1 will boot as well
X	0	X	0	→	ENABLED	HOLD	After a system reset or System Standby, CPU1 will boot, and CPU2 will remain on hold until CPU1 sets BOOT_C2 to '1'
		X	1	→	ENABLED	ENABLED	When CPU1 writes BOOT_C2 to '1', CPU2 will boot as well
1	1	X	X	→	ENABLED	ENABLED	After a system reset or System Standby, CPU1 and CPU2 will boot.

1. Setting both BCM4 and BCM7 bits to '0' forces the boot from CPU1.

9.5 RCC clock block functional description

The RCC provides a wide choice of clock generators:

- HSI (High-speed internal oscillator) clock: ~ 8, 16, 32 or 64 MHz
- HSE (High-speed external oscillator) clock: 4 to 48 MHz
- LSE (Low-speed external oscillator) clock: 32 kHz
- LSI (Low-speed internal oscillator) clock: ~ 32 kHz
- CSI (Low-power internal oscillator) clock: ~4 MHz
- HSI48 (High-speed 48 MHz internal oscillator) clock: ~48 MHz

It offers a high flexibility for the application to select the appropriate clock for CPUs and peripherals, in particular for peripherals that require a specific clock such as Ethernet, USB OTG-FS and HS, SPI/I2S, SAI and SDMMC.

To optimize the power consumption, each clock source can be switched ON or OFF independently.

The RCC provides up to 3 PLLs; each of them can be configured with integer or fractional ratios.

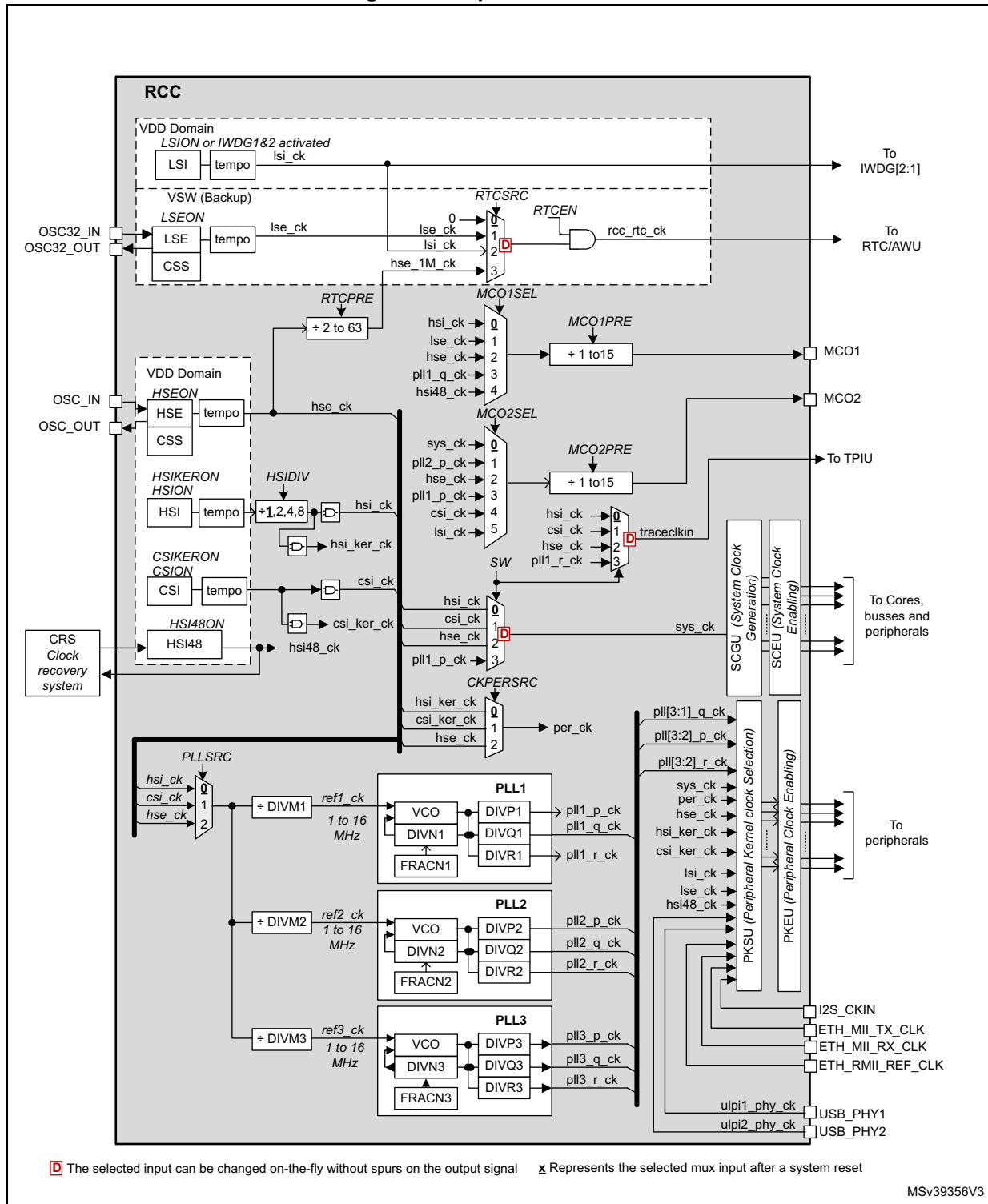
As shown in the [Figure 51](#), the RCC offers 2 clock outputs (MCO1 and MCO2), with a great flexibility on the clock selection and frequency adjustment.

The SCGU block (System Clock Generation Unit) contains several prescalers used to configure the CPU and bus matrix clock frequencies.

The PKSU block (Peripheral Kernel clock Selection Unit) provides several dynamic switches allowing a large choice of kernel clock distribution to peripherals.

The PKEU (Peripheral Kernel clock Enable Unit) and SCEU (System Clock Enable Unit) blocks perform the peripheral kernel clock gating, and the bus interface/cores/bus matrix clock gating, respectively.

Figure 51. Top-level clock tree



9.5.1 Clock naming convention

The RCC provides clocks to the complete circuit. To avoid misunderstanding, the following terms are used in this document:

- Peripheral clocks

The peripheral clocks are the clocks provided by the RCC to the peripherals. Two kinds of clock are available:

- The bus interface clocks
- The kernel clocks

A peripheral receives from the RCC a bus interface clock in order to access its registers, and thus control the peripheral operation. This clock is generally the AHB, APB or AXI clock depending on which bus the peripheral is connected to. Some peripherals only need a bus interface clock (e.g. RNG, TIMx).

Some peripherals also require a dedicated clock to handle the interface function. This clock is named “kernel clock”. As an example, peripherals such as SAI have to generate specific and accurate master clock frequencies, which require dedicated kernel clock frequencies. Another advantage of decoupling the bus interface clock from the specific interface needs, is that the bus clock can be changed without reprogramming the peripheral.

- CPU clocks

The CPU clock is the clock provided to the CPUs. It is derived from the system clock (**sys_ck**).

- Bus matrix clocks

The bus matrix clocks are the clocks provided to the different bridges (APB, AHB or AXI). These clocks are derived from the system clock (**sys_ck**).

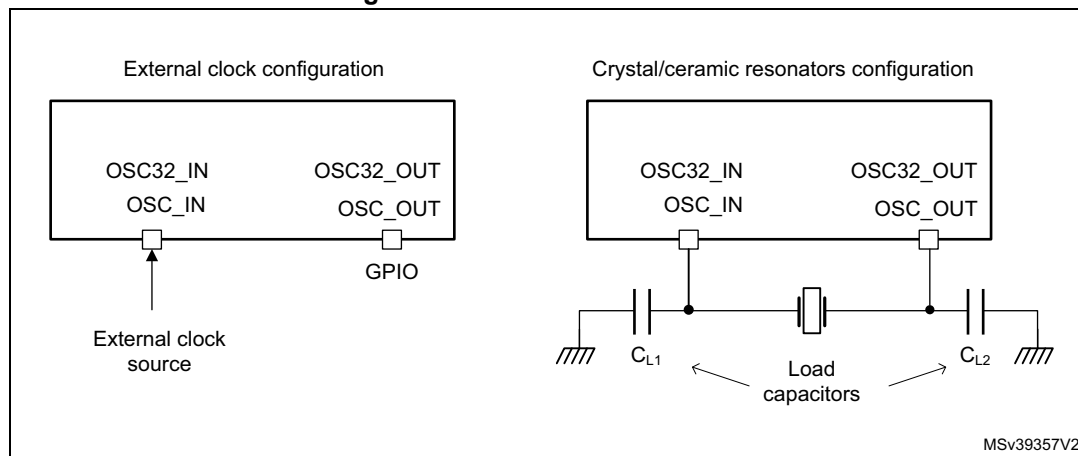
9.5.2 Oscillators description

HSE oscillator

The HSE block can generate a clock from two possible sources:

- External crystal/ceramic resonator
- External clock source

Figure 52. HSE/LSE clock source



External clock source (HSE bypass)

In this mode, an external clock source must be provided to OSC_IN pin. This mode is selected by setting the HSEBYP and HSEON bits of the [RCC source control register \(RCC_CR\)](#) to '1'. The external clock source (square, sinus or triangle) with ~50% duty cycle has to drive the OSC_IN pin.

External crystal/ceramic resonator

The oscillator is enabled by setting the HSEBYP bit to '0' and HSEON bit to '1'.

The HSE can be used when the product requires a very accurate high-speed clock.

The associated hardware configuration is shown in [Figure 52](#): the resonator and the load capacitors have to be placed as close as possible to the oscillator pins in order to minimize output distortion and startup stabilization time. The loading capacitance values must be adjusted according to the selected crystal or ceramic resonator. Refer to the electrical characteristics section of the datasheet for more details.

The HSERDY flag of the [RCC source control register \(RCC_CR\)](#), indicates whether the HSE oscillator is stable or not. At startup, the **hse_ck** clock is not released until this bit is set by hardware. An interrupt can be generated if enabled in the [RCC clock source interrupt enable register \(RCC_CIER\)](#).

The HSE can be switched ON and OFF through the HSEON bit. Note that the HSE cannot be switched OFF if one of the two conditions is met:

- The HSE is used directly (via software mux) as system clock
- The HSE is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the HSEON bit to '0'.

The HSE is automatically disabled by hardware, when the system enters Stop or Standby mode (refer to [Section 9.5.7: Handling clock generators in Stop and Standby mode](#) for additional information).

In addition, the HSE clock can be driven to the MCO1 and MCO2 outputs and used as clock source for other application components.

LSE oscillator

The LSE block can generate a clock from two possible sources:

- External crystal/ceramic resonator
- External user clock

External clock source (LSE bypass)

In this mode, an external clock source must be provided to OSC32_IN pin. The input clock can have a frequency up to 1 MHz. This mode is selected by setting the LSEBYP and LSEON bits of [RCC Backup domain control register \(RCC_BDCR\)](#) to '1'. The external clock signal (square, sinus or triangle) with ~50% duty cycle has to drive the OSC32_IN pin.

External crystal/ceramic resonator (LSE crystal)

The LSE clock is generated from a 32.768 kHz crystal or ceramic resonator. It has the advantage to provide a low-power highly accurate clock source to the real-time clock (RTC) for clock/calendar or other timing functions.

The LSERDY flag of the [RCC Backup domain control register \(RCC_BDCR\)](#) indicates whether the LSE crystal is stable or not. At startup, the LSE crystal output clock signal is not released until this bit is set by hardware. An interrupt can be generated if enabled in the [RCC clock source interrupt enable register \(RCC_CIER\)](#).

The LSE oscillator is switched ON and OFF using the LSEON bit. The LSE remains enabled when the system enters Stop or Standby mode.

In addition, the LSE clock can be driven to the MCO1 output and used as clock source for other application components.

The LSE also offers a programmable driving capability (LSEDRV[1:0]) that can be used to modulate the amplifier driving capability. The driving capability cannot be changed when the LSE oscillator is ON.

HSI oscillator

The HSI block provides the default clock to the product.

The HSI is a high-speed internal RC oscillator which can be used directly as system clock, peripheral clock, or as PLL input. A predivider allows the application to select an HSI output frequency of 8, 16, 32 or 64 MHz. This predivider is controlled by the HSIDIV.

The HSI advantages are the following:

- Low-cost clock source since no external crystal is required
- Faster startup time than HSE (a few microseconds)

The HSI frequency, even with frequency calibration, is less accurate than an external crystal oscillator or ceramic resonator.

The HSI can be switched ON and OFF using the HSION bit. Note that the HSI cannot be switched OFF if one of the two conditions is met:

- The HSI is used directly (via software mux) as system clock
- The HSI is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the HSION bit to '0'.

Note that the HSIDIV cannot be changed if the HSI is selected as reference clock for at least one enabled PLL (PLLxON bit set to '1'). In that case the hardware does not update the HSIDIV with the new value. However it is possible to change the HSIDIV if the HSI is used directly as system clock.

The HSIRDY flag indicates if the HSI is stable or not. At startup, the HSI output clock is not released until this bit is set by hardware.

The HSI clock can also be used as a backup source (auxiliary clock) if the HSE fails (refer to [Section : CSS on HSE](#)). The HSI can be disabled or not when the system enters Stop mode, please refer to [Section 9.5.7: Handling clock generators in Stop and Standby mode](#) for additional information.

In addition, the HSI clock can be driven to the MCO1 output and used as clock source for other application components.

Care must be taken when the HSI is used as kernel clock for communication peripherals, the application must take into account the following parameters:

- the time interval between the moment where the peripheral generates a kernel clock request and the moment where the clock is really available,
- the frequency accuracy.

Note: The HSI can remain enabled when the system is in Stop mode (see [Section 9.5.7](#) for additional information).

HSION, HSIRDY and HSIDIV bits are located in the [RCC source control register \(RCC_CR\)](#).

HSI calibration

RC oscillator frequencies can vary from one chip to another due to manufacturing process variations. That is why each device is factory calibrated by STMicroelectronics to improve accuracy (refer to the product datasheet for more information).

After a power-on reset, the factory calibration value is loaded in the HSICAL[11:0] bits.

If the application is subject to voltage or temperature variations, this may affect the RC oscillator frequency. The user application can trim the HSI frequency using the HSITRIM[6:0] bits.

Note: HSICAL[11:0] and HSITRIM[6:0] bits are located in the [RCC HSI configuration register \(RCC_HSIConfigR\)](#).

CSI oscillator

The CSI is a low-power RC oscillator which can be used directly as system clock, peripheral clock, or PLL input.

The CSI advantages are the following:

- Low-cost clock source since no external crystal is required
- Faster startup time than HSE (a few microseconds)
- Very low-power consumption,

The CSI provides a clock frequency of about 4 MHz, while the HSI is able to provide a clock up to 64 MHz.

CSI frequency, even with frequency calibration, is less accurate than an external crystal oscillator or ceramic resonator.

The CSI can be switched ON and OFF through the CSION bit. The CSIRDY flag indicates whether the CSI is stable or not. At startup, the CSI output clock is not released until this bit is set by hardware.

The CSI cannot be switched OFF if one of the two conditions is met:

- The CSI is used directly (via software mux) as system clock
- The CSI is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the CSION bit to '0'.

The CSI can be disabled or not when the system enters Stop mode (refer to [Section 9.5.7: Handling clock generators in Stop and Standby mode](#) for additional information).

In addition, the CSI clock can be driven to the MCO2 output and used as clock source for other application components.

Even if the CSI settling time is faster than the HSI, care must be taken when the CSI is used as kernel clock for communication peripherals: the application has to take into account the following parameters:

- the time interval between the moment where the peripheral generates a kernel clock request and the moment where the clock is really available,
- the frequency precision.

Note: *CSION and CSIRDY bits are located in the [RCC source control register \(RCC_CR\)](#).*

CSI calibration

RC oscillator frequencies can vary from one chip to another due to manufacturing process variations, this is why each device is factory calibrated by STMicroelectronics to improve accuracy (refer to the product datasheet for more information).

After reset, the factory calibration value is loaded in the CSICAL[9:0] bits.

If the application is subject to voltage or temperature variations, this may affect the RC oscillator frequency. The user application can trim the CSI frequency using the CSITRIM[5:0] bits.

Note: *CSICAL[9:0] and CSITRIM[5:0] bits are located into the [RCC CSI configuration register \(RCC_CSICFGR\)](#)*

HSI48 oscillator

The HSI48 is an RC oscillator that delivers a 48 MHz clock that can be used directly as kernel clock for some peripherals.

The internal HSI48 oscillator mainly aims at providing a high precision clock to the USB peripheral by means of a special Clock Recovery System (CRS) circuitry, which could use the USB SOF signal, the LSE, or an external signal, to automatically adjust the oscillator frequency on-the-fly, in very small granularity.

The HSI48 oscillator is disabled as soon as the system enters Stop or Standby mode. When the CRS is not used, this oscillator is free running and thus subject to manufacturing process variations. That is why each device is factory calibrated by STMicroelectronics to achieve an accuracy of ACC_{HSI48} (refer to the product datasheet of the for more information).

For more details on how to configure and use the CRS, please refer to [Section 10: Clock recovery system \(CRS\)](#).

The HSI48RDY flag indicates whether the HSI48 oscillator is stable or not. At startup, the HSI48 output clock is not released until this bit is set by hardware.

The HSI48 can be switched ON and OFF using the HSI48ON bit.

The HSI48 clock can also be driven to the MCO1 multiplexer and used as clock source for other application components.

Note: *HSI48ON and HSI48RDY bits are located in the [RCC source control register \(RCC_CR\)](#).*

LSI oscillator

The LSI acts as a low-power clock source that can be kept running when the system is in Stop or Standby mode for the independent watchdog (IWDG) and Auto-Wakeup Unit (AWU). The clock frequency is around 32 kHz. For more details, refer to the electrical characteristics section of the datasheet.

The LSI can be switched ON and OFF using the LSION bit. The LSIRDY flag indicates whether the LSI oscillator is stable or not. If an independent watchdog is started either by hardware or software, the LSI is forced ON and cannot be disabled.

The LSI remains enabled when the system enters Stop or Standby mode (refer to [Section 9.5.7: Handling clock generators in Stop and Standby mode](#) for additional information).

At LSI startup, the clock is not provided until the hardware sets the LSIRDY bit. An interrupt can be generated if enabled in the [RCC clock source interrupt enable register \(RCC_CIER\)](#).

In addition, the LSI clock can be driven to the MCO2 output and used as a clock source for other application components.

Note: Bits LSION and LSIRDY are located into the [RCC clock control and status register \(RCC_CSR\)](#).

9.5.3 Clock Security System (CSS)

CSS on HSE

The clock security system can be enabled by software via the HSECSSON bit. The HSECSSON bit can be enabled even when the HSEON is set to '0'.

The CSS on HSE is enabled by the hardware when the HSE is enabled and ready, and HSECSSON set to '1'.

The CSS on HSE is disabled when the HSE is disabled. As a result, this function does not work when the system is in Stop mode.

It is not possible to clear directly the HSECSSON bit by software.

The HSECSSON bit is cleared by hardware when a system reset occurs or when the system enters Standby mode (see [Section 9.4.2: System reset](#)).

If a failure is detected on the HSE clock, the system automatically switches to the HSI in order to provide a safe clock. The HSE is then automatically disabled, a clock failure event is sent to the break inputs of advanced-control timers (TIM1, TIM8, TIM15, TIM16, and TIM17), and an interrupt is generated to inform the software about the failure (CSS interrupt: **rcc_hsecss_it**), thus allowing the MCU to perform rescue operations. If the HSE output was used as clock source for PLLs when the failure occurred, the PLLs are also disabled.

If an HSE clock failure occurs when the CSS is enabled, the CSS generates an interrupt which causes the automatic generation of an NMI. The HSECSSF flag in [RCC clock source interrupt flag register \(RCC_CIFR\)](#) is set to '1' to allow the application to identify the failure source. The NMI routine is executed indefinitely until the HSECSSF bit is cleared. As a consequence, the application has to clear the HSECSSF flag in the NMI ISR by setting the HSECSSC bit in the [RCC clock source interrupt clear register \(RCC_CICR\)](#).

CSS on LSE

A clock security system on the LSE oscillator can be enabled by software by programming the LSECSSON bit in the [RCC Backup domain control register \(RCC_BDCR\)](#).

This bit can be disabled only by hardware when the following conditions are met:

- after a pwr_vsw_rst (V_{SW} software reset)
- or after a failure detection on LSE.

LSECSSON bit must be written after the LSE is enabled (LSEON bit set by software) and ready (LSERDY set by hardware), and after the RTC clock has been selected through the RTCSEL bit.

The CSS on LSE works in all modes (Run, Stop and Standby) except VBAT.

If an LSE failure is detected, the LSE clock is no more delivered to the RTC but the value of RTCSEL, LSECSSON and LSEON bits are not changed by the hardware.

A wakeup is generated in Standby mode. In other modes an interrupt (**rcc_lsecss_it**) can be sent to wake up the software. The software must then disable the LSECSSON bit, stop the defective LSE (clear LSEON bit), and can change the RTC clock source (no clock or LSI or HSE) through RTCSEL bits, or take any required action to secure the application.

9.5.4 Clock output generation (MCO1/MCO2)

Two micro-controller clock output (MCO) pins, MCO1 and MCO2, are available. A clock source can be selected for each output. The selected clock can be divided thanks to configurable prescaler (refer to [Figure 51](#) for additional information on signal selection).

MCO1 and MCO2 outputs are controlled via MCO1PRE[3:0], MCO1SEL[2:0], MCO2PRE[3:0] and MCO2SEL[2:0] located in the [RCC clock configuration register \(RCC_CFGR\)](#).

The GPIO port corresponding to each MCO pin, has to be programmed in alternate function mode.

The clock provided to the MCOs outputs must not exceed the maximum pin speed (refer to the product datasheet for information on the supported pin speed).

9.5.5 PLL description

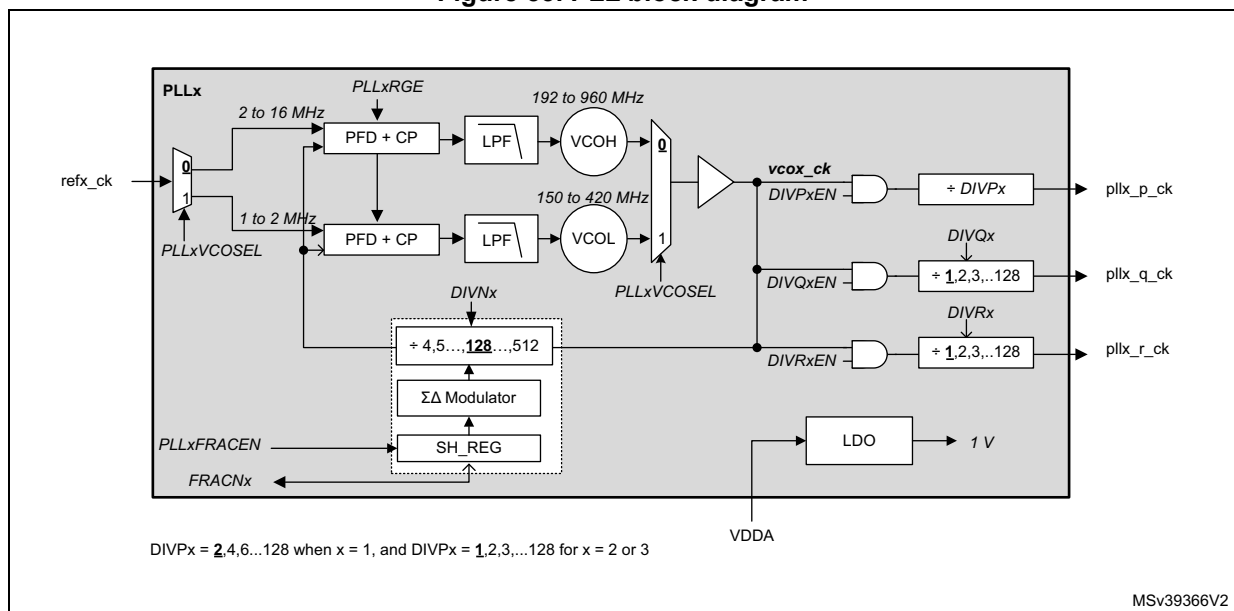
The RCC features three PLLs:

- A main PLL, PLL1, which is generally used to provide clocks to the CPUs and to some peripherals.
- Two dedicated PLLs, PLL2 and PLL3, which are used to generate the kernel clock for peripherals.

The PLLs integrated into the RCC are completely independent. They offer the following features:

- Two embedded VCOs:
 - A wide-range VCO (VCOH)
 - A low-frequency VCO (VCOL)
- Input frequency range:
 - 1 to 2 MHz when VCOL is used
 - 2 to 16 MHz when VCOH is used
- Capability to work either in integer or Fractional mode
- 13-bit Sigma-Delta modulator, allowing to fine-tune the VCO frequency by steps of 11 to 0.3 ppm.
- The Sigma-Delta modulator can be updated on-the-fly, without generating frequency overshoots on PLLs outputs.
- Each PLL offer 3 outputs with post-dividers

Figure 53. PLL block diagram



The PLLs are controlled via `RCC_PLLxDIVR`, `RCC_PLLxFRACR`, `RCC_PLLCFGR` and `RCC_CR` registers.

The frequency of the reference clock provided to the PLLs (**refx_ck**) must range from 1 to 16 MHz. The user application has to program properly the DIVMx dividers of the *RCC PLLs clock source selection register (RCC_PLLCKSELR)* in order to match this condition. In addition, the PLLxRGE of the *RCC PLL configuration register (RCC_PLLCFGR)* field must be set according to the reference input frequency to guarantee an optimal performance of the PLL.

The user application can then configure the proper VCO: if the frequency of the reference clock is lower or equal to 2 MHz, then VCOL must be selected, otherwise VCOH must be chosen. To reduce the power consumption, it is recommended to configure the VCO output to the lowest frequency.

DIVNx loop divider has to be programmed to achieve the expected frequency at VCO output. In addition, the VCO output range must be respected.

The PLLs operate in integer mode when the value of SH_REG (FRACNx shadow register) is set to '0'. The SH_REG is updated with the FRACNx value when PLLxFRACEN bit goes from '0' to '1'. The Sigma-Delta modulator is designed in order to minimize the jitter impact while allowing very small frequency steps.

The PLLs can be enabled by setting PLLxON to '1'. The bits PLLxRDY indicate that the PLL is ready (i.e. locked).

Note: *Before enabling the PLLs, make sure that the reference frequency (**refx_ck**) provided to the PLL is stable, so the hardware does not allow changing DIVMx when the PLLx is ON and it is also not possible to change PLLSRC when one of the PLL is ON.*

The hardware prevents writing PLL1ON to '0' if the PLL1 is currently used to deliver the system clock. There are other hardware protections on the clock generators (refer to sections [HSE oscillator](#), [HSI oscillator](#) and [CSI oscillator](#)).

The following PLL parameters cannot be changed once the PLL is enabled: DIVNx, PLLxRGE, PLLxVCOSEL, DIVPx, DIVQx, DIVRx, DIVPxEN, DIVQxEN and DIVRxEN.

To insure an optimal behavior of the PLL when one of the post-divider (DIVP, DIVQ or DIVR) is not used, the application shall set the enable bit (DIVyEN) as well as the corresponding post-divider bits (DIVP, DIVQ or DIVR) to '0'.

If the above rules are not respected, the PLL output frequency is not guaranteed.

Output frequency computation

When the PLL is configured in integer mode (SH_REG = '0'), the VCO frequency (F_{VCO}) is given by the following expression:

$$F_{VCO} = F_{REF_CK} \times DIVN$$

$$F_{PLL_y_CK} = (F_{VCO} / (DIVy + 1)) \text{ with } y = P, Q \text{ or } R$$

When the PLL is configured in fractional mode (SH_REG different from '0'), the DIVN divider must be initialized before enabling the PLLs. However, it is possible to change the value of FRACNx on-the-fly without disturbing the PLL output.

This feature can be used either to generate a specific frequency from any crystal value with a good accuracy, or to fine-tune the frequency on-the-fly.

For each PLL, the VCO frequency is given by the following formula:

$$F_{VCO} = F_{ref_ck} \times \left(DIVN + \frac{FRACN}{2^{(13)}} \right)$$

Note: *For PLL1, DIVP can only take odd values.*

The PLLs are disabled by hardware when:

- The system enters Stop or Standby mode.
- An HSE failure occurs when HSE or PLL (clocked by HSE) are used as system clock.

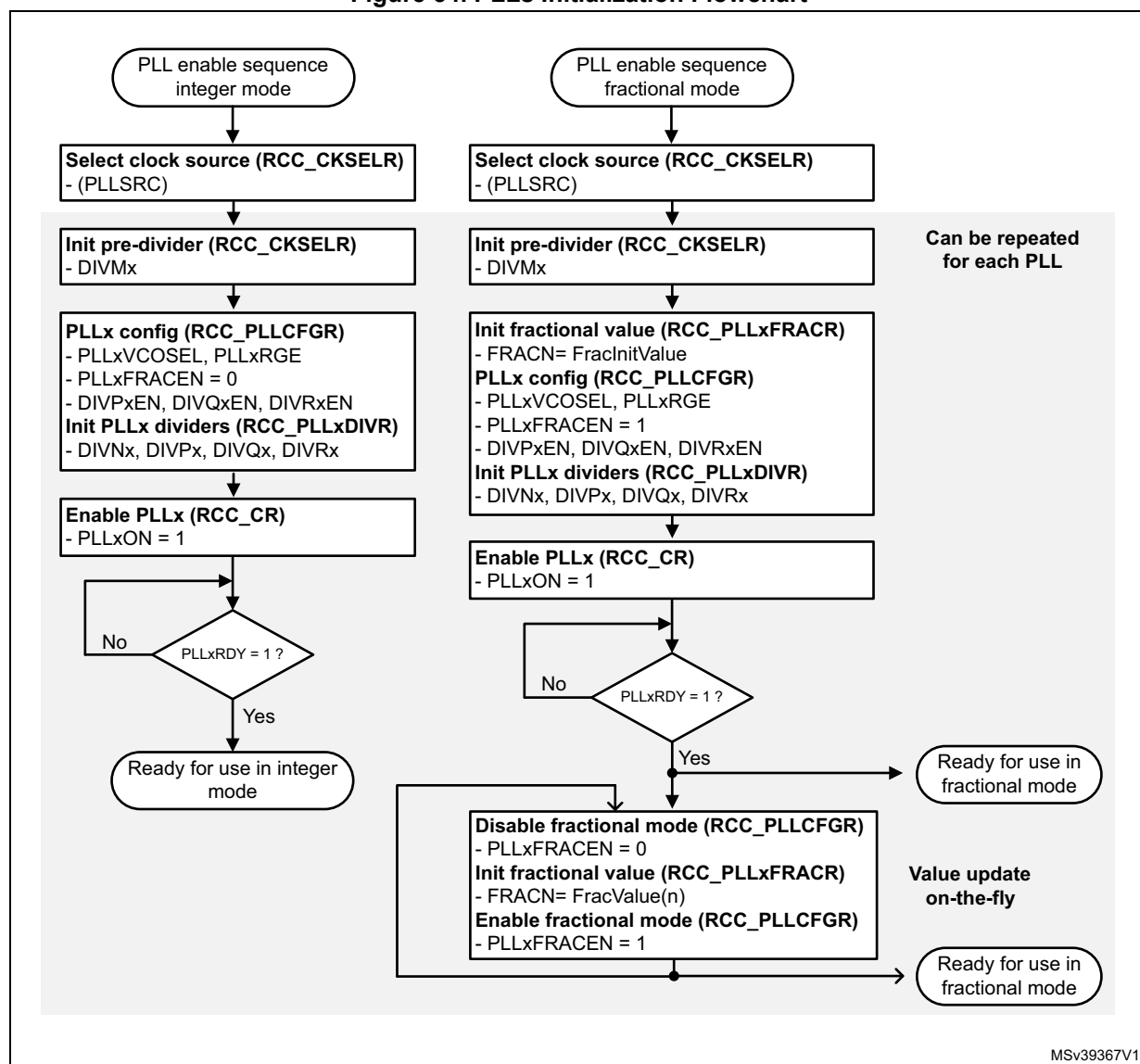
PLL initialization phase

Figure 54 shows the recommended PLL initialization sequence in integer and fractional mode. The PLLx are supposed to be disabled at the start of the initialization sequence:

1. Initialize the PLLs registers according to the required frequency.
 - Set PLLxFRACEN of *RCC PLL configuration register (RCC_PLLCFGR)* to '0' for integer mode.
 - For fractional mode, set FRACN to the required initial value (FracInitValue) and then set PLLxFRACEN to '1'.
2. Once the PLLxON bit is set to '1', the user application has to wait until PLLxRDY bit is set to '1'. If the PLLx is in fractional mode, the PLLxFRACEN bit must not be set back to '0' as long as PLLxRDY = '0'.
3. Once the PLLxRDY bit is set to '1', the PLLx is ready to be used.
4. If the application intends to tune the PLLx frequency on-the-fly (possible only in fractional mode), then:
 - a) PLLxFRACEN must be set to '0',
When PLLxFRACEN = '0', the Sigma-Delta modulator is still operating with the value latched into SH_REG.
Application must wait for 3 refx_ck clock periods (PLLxFRACEN bit propagation delay)
 - b) A new value must be uploaded into PLLxFRACR (FracValue(n)).
 - c) PLLxFRACEN must be set to '1', in order to latch the content of PLLxFRACR into its shadow register.
The new value is considered after 3 clock periods of refx_ck (PLLxFRACEN bit propagation delay)

Note: When the PLLxRDY goes to '1', it means that the difference between the PLLx output frequency and the target value is lower than $\pm 2\%$.

Figure 54. PLLs Initialization Flowchart



9.5.6 System clock (sys_ck)

System clock selection

After a system reset, the HSI is selected as system clock and all PLLs are switched OFF. When a clock source is used for the system clock, it is not possible for the software to disable the selected source via the xxxON bits.

Of course, the system clock can be stopped by the hardware when the System enters Stop or Standby mode.

When the system is running, the user application can select the system clock (**sys_ck**) among the 4 following sources:

- HSE
- HSI
- CSI
- or pll1_p_ck

This function is controlled by programming the [RCC clock configuration register \(RCC_CFGR\)](#). A switch from one clock source to another occurs only if the target clock source is ready (clock stable after startup delay or PLL locked). If a clock source that is not yet ready is selected, the switch occurs when the clock source is ready.

The SWS status bits in the [RCC clock configuration register \(RCC_CFGR\)](#) indicate which clock is currently used as system clock. The other status bits in the RCC_CR register indicate which clock(s) is (are) ready.

System clock generation

[Figure 55](#) shows a simplified view of the clock distribution for the CPUs and busses. All the dividers shown in the block diagram can be changed on-the-fly without generating timing violations. This feature is a very simply solution to adapt the busses frequencies to the application needs, thus optimizing the power consumption.

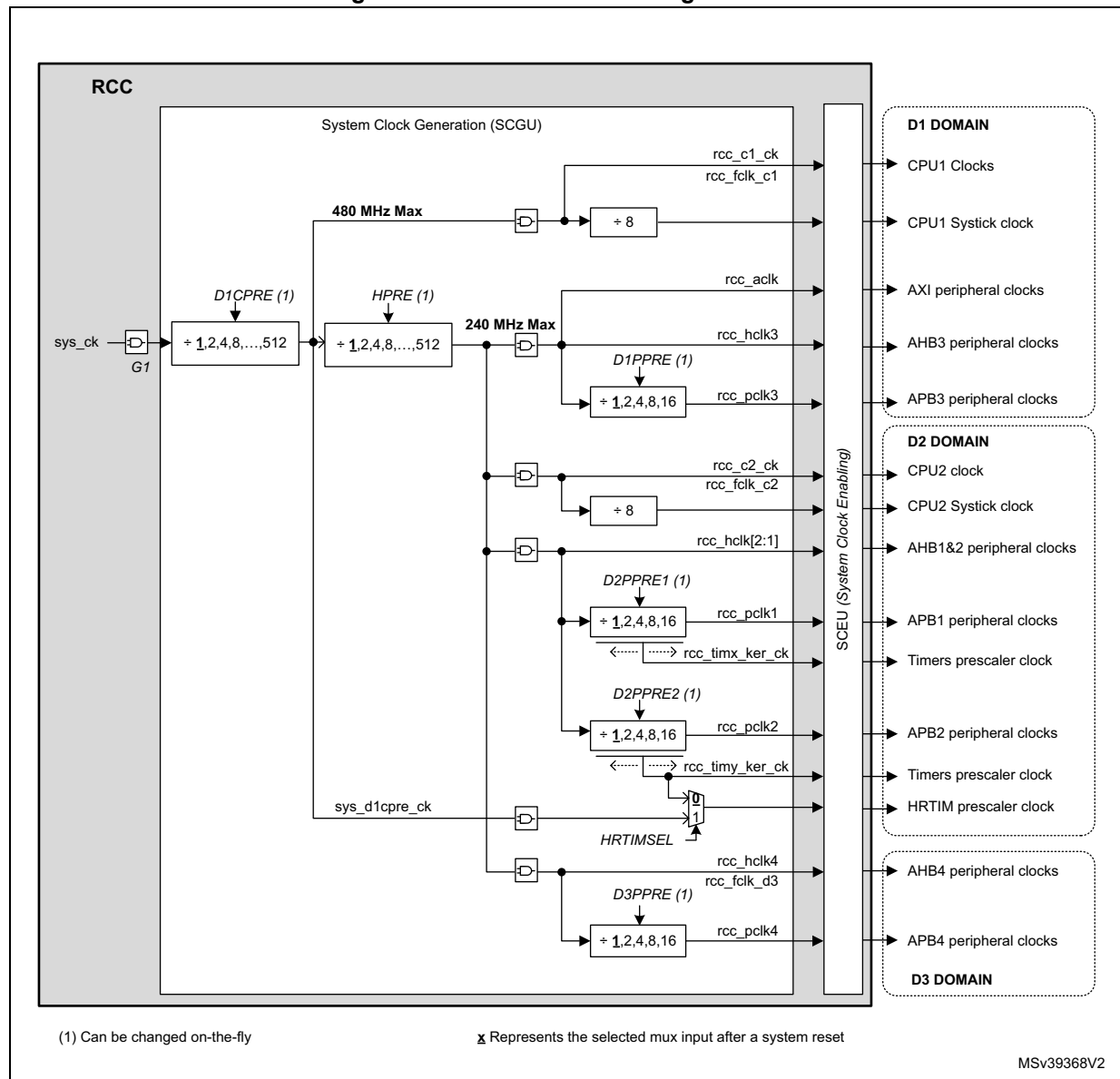
The D1CPRE divider can be used to adjust CPU1 clock. However this also impacts the clock frequency of all bus matrix, CPU2, and HRTIM.

In the same way, HPRE divider can be used to adjust the clock for D1 domain bus matrix, but this also impacts the clock frequency of bus matrix of D2 and D3 domains, and CPU2.

In addition the CPU clock frequency can be equal to the bus matrix clock frequency, but the maximum frequency for the bus matrix is limited to 200 MHz.

Most of the prescalers are controlled via RCC_D1CFGR, RCC_D2CFGR and RCC_D3CFGR registers.

Figure 55. Core and bus clock generation



This block also provides the clock for the timers (**rcc_timx_ker_ck** and **rcc_tiny_ker_ck**). The frequency of the timers clock depends on the APB prescaler corresponding to the bus to which the timer is connected, and on TIMPRE bit. [Table 59](#) shows how to select the timer clock frequency.

Table 59. Ratio between clock timer and pclk

D2PPRE1 ⁽¹⁾ D2PPRE2	TIMPRE ⁽²⁾		$F_{rcc_timx_ker_ck}$ $F_{rcc_tmy_ker_ck}$	F_{rcc_pclk1} F_{rcc_pclk2}	Comments
0xx	0	→	F_{rcc_hclk1}	F_{rcc_hclk1}	The timer clock is equal to the bus clock.
100	0	→	F_{rcc_hclk1}	$F_{rcc_hclk1} / 2$	The timer clock is twice as fast as the bus clock.
101	0	→	$F_{rcc_hclk1} / 2$	$F_{rcc_hclk1} / 4$	
110	0	→	$F_{rcc_hclk1} / 4$	$F_{rcc_hclk1} / 8$	
111	0	→	$F_{rcc_hclk1} / 8$	$F_{rcc_hclk1} / 16$	
0xx	1	→	F_{rcc_hclk1}	F_{rcc_hclk1}	The timer clock is equal to the bus clock.
100	1	→	F_{rcc_hclk1}	$F_{rcc_hclk1} / 2$	The timer clock is twice as fast as the bus clock.
101	1	→	F_{rcc_hclk1}	$F_{rcc_hclk1} / 4$	The timer clock is 4 times faster than the bus clock.
110	1	→	$F_{rcc_hclk1} / 2$	$F_{rcc_hclk1} / 8$	
111	1	→	$F_{rcc_hclk1} / 4$	$F_{rcc_hclk1} / 16$	

1. D2PPRE1 and D2PPRE2 belong to [RCC domain 2 clock configuration register \(RCC_D2CFGR\)](#).
2. TIMPRE belongs to [RCC clock configuration register \(RCC_CFGR\)](#).

9.5.7 Handling clock generators in Stop and Standby mode

When the whole system enters Stop mode, all the clocks (system and kernel clocks) are stopped as well as the following clock sources:

- CSI, HSI (depending on HSIKERON, and CSIKERON bits)
- HSE
- PLL1, PLL2 and PLL3
- HSI48

The content of the RCC registers is not altered except for PLL1ON, PLL2ON, PLL3ON, HSEON and HSI48ON which are set to '0'.

Exiting Stop mode

When the microcontroller exits system Stop mode via a wake-up event, the application can select which oscillator (HSI and/or CSI) will be used to restart. The STOPWUCK bit selects the oscillator used as system clock. The STOPKERWUCK bit selects the oscillator used as kernel clock for peripherals. The STOPKERWUCK bit is useful if after a system Stop a peripheral needs a kernel clock generated by an oscillator different from the one used for the system clock.

All these bits belong to the [RCC clock configuration register \(RCC_CFGR\)](#). [Table 60](#) gives a detailed description of their behavior.

Table 60. STOPWUCK and STOPKERWUCK description

STOPWUCK	STOPKERWUCK		Activated oscillator when the system exits Stop mode	Distributed clocks when System exits Stop mode	
				System Clock	Kernel Clock
0	0	→	HSI	HSI	HSI
	1	→	HSI and CSI		HSI and/or CSI
1	0	→		CSI	
	1	→			

During Stop mode

There are two specific cases where the HSI or CSI can be enabled during system Stop mode:

- When a dedicated peripheral requests the kernel clock:
In this case the peripheral will receive the HSI or CSI according to the kernel clock source selected for this peripheral (via PERXSRC).
- When the bits HSIKERON or CSIKERON are set:
In this case the HSI and CSI are kept running during Stop mode but the outputs are gated. In that way, the clock will be available immediately when the system exits Stop mode or when a peripheral requests the kernel clock (see [Table 61](#) for details).

HSIKERON and CSIKERON bits belong to [RCC source control register \(RCC_CR\)](#).

[Table 61](#) gives a detailed description of their behavior.

Table 61. HSIKERON and CSIKERON behavior

HSIKERON (CSIKERON)		HSI (CSI) state during Stop mode	HSI (CSI) Setting time
0	→	OFF	$t_{su(HSI)}$ ($t_{su(CSI)}$) ⁽¹⁾
1	→	Running and gated	Immediate

1. $t_{su(HSI)}$ and $t_{su(CSI)}$ are the startup times of the HSI and CSI oscillators (see refer to the product datasheet for the values of these parameters).

When the microcontroller exists system Standby mode, the HSI is selected as system and kernel clock, the RCC registers are reset to their initial values except for the RCC_RSR and RCC_BDCR registers.

Note as well that the HSI and CSI outputs provide two clock paths (see [Figure 51](#)):

- one path for the system clock (**hsi_ck** or **csi_ck**)
- one path for the peripheral kernel clock (**hsi_ker_ck** or **csi_ker_ck**).

When a peripheral requests the kernel clock in system Stop mode, only the path providing the **hsi_ker_ck** or **csi_ker_ck** is activated.

Caution: It is not guaranteed that a CPU will get automatically the same clock frequencies when leaving CStop mode: this mainly depends on the System state. For example CPU2 can switch from time to time from CRun to CStop mode. If the system does not go to Stop or Standby mode, then the system clock is not stopped and CPU2 will operate from it when

leaving CStop mode. However, if the system enters Stop while CPU2 is in CStop mode, then CPU2 will operate from HSI or CSI when it left the CStop mode.

9.5.8 Kernel clock selection

Some peripherals are designed to work with two different clock domains that operate asynchronously:

- a clock domain synchronous with the register and bus interface (**ckg_bus_perx** clock)
- and a clock domain generally synchronous with the peripheral (kernel clock).

The benefit of having peripherals supporting these two clock domains is that the user application has more freedom to choose optimized clock frequency for the CPUs, bus matrix and for the kernel part of the peripheral.

As a consequence, the user application can change the bus frequency without reprogramming the peripherals. As an example an on-going transfer with UART will not be disturbed if its APB clock is changed on-the-fly.

[Table 62](#) shows the kernel clock that the RCC can deliver to the peripherals. Each row of [Table 62](#) represents a MUX and the peripherals connected to its output. The columns starting from number 4 represent the clock sources. Column 3 gives the maximum allowed frequency at each MUX output. It is up to the user to respect these requirements.



Table 62. Kernel clock distribution overview

Peripherals	Clock mux control bits	Maximum allowed frequency [MHz]				Domain	Clock Sources																		
		VOS0	VOS1	VOS2	VOS3		pll1_q_ck	pll2_p_ck	pll2_q_ck	pll2_r_ck	pll3_p_ck	pll3_q_ck	pll3_r_ck	sys_ck	bus clocks (1)	hse_ck	hsi_ker_ck	csi_ker_ck	hsi48_ck	lse_ck	lsj_ck	per_ck (2)	I2S_CKIN	dsi_phy_ck	USB_PHY1
DSI	DSISEL	133 ⁽³⁾	133 ⁽³⁾	100 ⁽³⁾	66 ⁽³⁾	D1			1														0		
LTDC	-	150	150	75	50								x												
FMC	FMCSEL	300	250	200	133		1			2					Q							3			
QUADSPI	QSPISEL	250	200	150	100		1			2					Q							3			
SDMMC1	SDMMCSEL	250 ⁽⁵⁾	200 ⁽⁵⁾	150 ⁽⁵⁾	100 ⁽⁵⁾		Q			1															
SDMMC2			250	200	150	100																			
DFSDM1 AclK	SAI1SEL	250	200	150	100		Q	1			2										4	3			
DFSDM1 clk	DFSDM1SEL	250	200	150	100								1	Q											
FDCAN	FDCANSEL	125	100	75	50	1			2						0										
HDMI-CEC	CECSEL	66	66	66	33												2 ⁽⁴⁾	0	1						
I2C1,2,3	I2C123SEL	125	100	75	50							1		Q		2	3								
LPTIM1	LPTIM1SEL	125	100	75	50			1					2	Q					3	4	5				
TIM[8:1][17:12]	-	240	200	150	100	D2								x											
HRTIM	-	480	400	300	200										x										
RNG	RNGSEL	250	200	150	100		1											Q	2	3					
SAI1	SAI1SEL	150	150	113	75		Q	1			2											4	3		
SAI2	SAI23SEL	150	150	113	75		Q	1			2											4	3		
SAI3			150	150	113		75																		
SPDIFRX	SPDIFSEL	250	200	150	100		Q			1			2				3								
SPI(I2S)1,2,3	SPI123SEL	200	200	150	100		Q	1			2											4	3		

Table 62. Kernel clock distribution overview (continued)

Peripherals	Clock mux control bits	Maximum allowed frequency [MHz]				Domain	Clock Sources																			
		VOS0	VOS1	VOS2	VOS3		pll1_q_ck	pll2_p_ck	pll2_q_ck	pll2_r_ck	pll3_p_ck	pll3_q_ck	pll3_r_ck	sys_ck	bus clocks ⁽¹⁾	hse_ck	hsi_ker_ck	csi_ker_ck	hsi48_ck	lse_ck	lsi_ck	per_ck ⁽²⁾	I2S_CKIN	dsi_phy_ck	USB_PHY1	Disabled
SPI4,5	SPI45SEL	125	100	75	50	D2			1			2			0	5	3	4								
SWPMI	SWPSEL	125	100	75	50										0		1									
USART1,6	USART16SEL	125	100	75	50				1			2			0		3	4		5						
USART2,3 UART4,5,7,8	USART234578SEL	125	100	75	50				1			2			0		3	4		5						
USB1OTG USB2OTG	USBSEL	66	66	66	63		1					2							3							0
USB1ULPI	-	60	60	60	60																			x		
ADC1,2, 3	ADCSEL	160 ⁽⁵⁾	160 ⁽⁵⁾	60 ⁽⁵⁾	40 ⁽⁵⁾			0					1									2				
I2C4	I2C4SEL	125	100	75	50	D3						1		0		2	3									
LPTIM2	LPTIM2SEL	125	100	75	50			1					2		0					3	4	5				
LPTIM3,4,5	LPTIM345SEL	125	100	75	50			1					2		0					3	4	5				
LPUART1	LPUART1SEL	125	100	75	50				1			2			0		3	4		5						
SAI4_A	SAI4ASEL	150	150	75	75		0	1			2											4	3			
SAI4_B	SAI4BSEL	150	150	75	75		0	1			2											4	3			
SPI6	SPI6SEL	125	100	75	50				1			2			0	5	3	4								
RTC/AWU ⁽⁶⁾	RTCSEL	-	-	1	-	VSW									3				1	2					0	

1. The bus clocks are the bus interface clocks to which the peripherals are connected, it can be APB, AHB or AXI clocks.

2. The per_ck clock could be hse_ck, hsi_ker_ck or csi_ker_ck according to CKPERSEL selection.

3. For byte lane clock frequency ($F_{\text{blck_ck}}$).

4. Clock CSI divided by 122.

5. With a duty cycle close to 50%, meaning that DIV[P/Q/R]x values shall be even. For SDMMCx, the duty cycle shall be 50% when supporting DDR.

6. The RTC is not functional in V_{BAT} mode when the clock source is lsi_ck or hse_ck.

[Figure 56](#) to [Figure 65](#) provide a more detailed description of kernel clock distribution. To simplify the drawings, the bus interface clocks (pclk, hclk) are not represented, even if they are gated with enable signals. Refer to [Section 9.5.11: Peripheral clock gating control](#) for more details.

To reduce the amount of switches, some peripherals share the same kernel clock source. Nevertheless, all peripherals have their dedicated enable signal.

Peripherals dedicated to audio applications

The audio peripherals generally need specific accurate frequencies, except for SPDIFRX. As shown in [Figure 56](#), the kernel clock of the SAI or SPI(I2S)s can be generated by:

- The PLL1 when the amount of active PLLs has to be reduced
- The PLL2 or 3 for optimal flexibility in frequency generation
- HSE, HSI or CSI for use-cases where the current consumption is critical
- I2S_CKIN when an external clock reference need to be used.

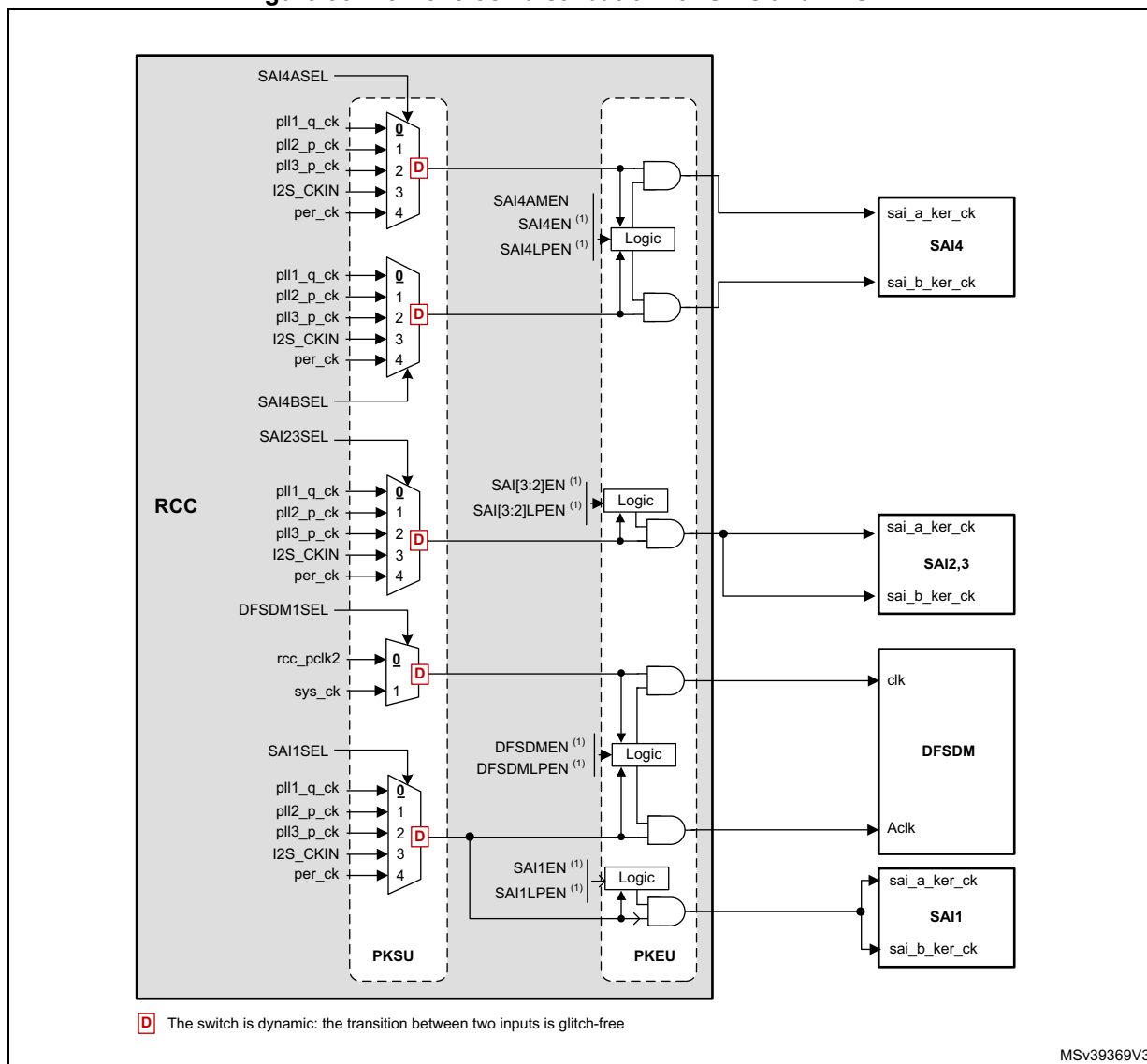
Note: The SPDIFRX does not require a specific frequency, but only a kernel clock frequency high enough to make the peripheral work properly. Refer to the SPDIFRX description for more details.

DFSDM1 can use the same clock as SAI1A. This is useful when DFSDM1 is used for audio applications.

To improve the flexibility, SAI4 can use different clock for each sub-block.

The SPI/I2S1, 2, and 3 share the same kernel clock source (see [Figure 57](#)).

Figure 56. Kernel clock distribution for SAI and DFSDM1



1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripherals. For details on each enable cell, please refer to [Section 9.5.11: Peripheral clock gating control](#).

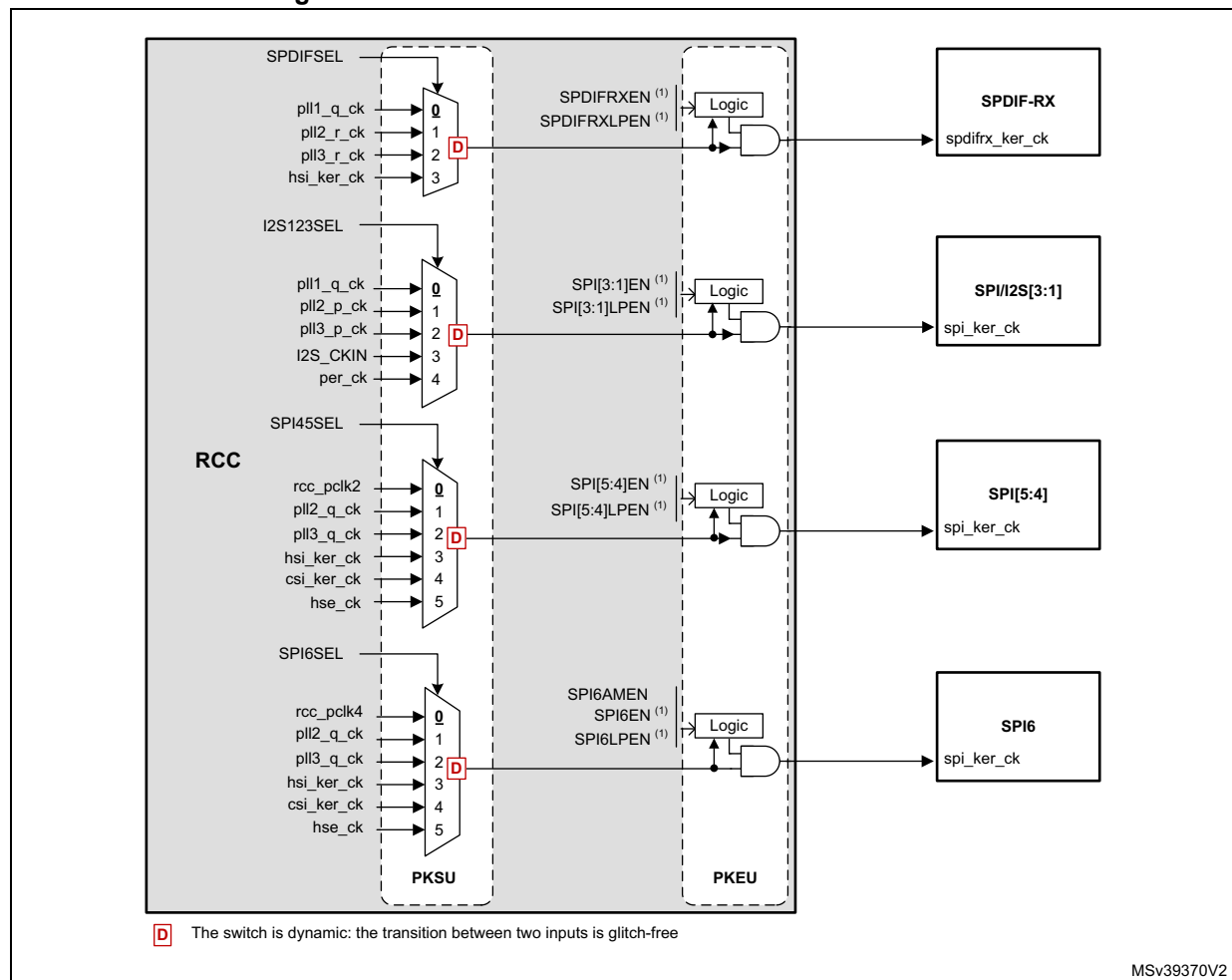
Peripherals dedicated to control and data transfer

Peripherals such as SPIs, I2Cs, UARTs do not need a specific kernel clock frequency but a clock fast enough to generate the correct baud rate, or the required bit clock on the serial interface. For that purpose the source can be selected among:

- PLL1 when the amount of active PLLs has to be reduced
- PLL2 or PLL3 if better flexibility is required. As an example, this solution allows changing the frequency bus via PLL1 without affecting the speed of some serial interfaces.
- HSI or CSI for low-power use-cases or when the peripheral has to quickly wake up from Stop mode (i.e. UART, I2C...).

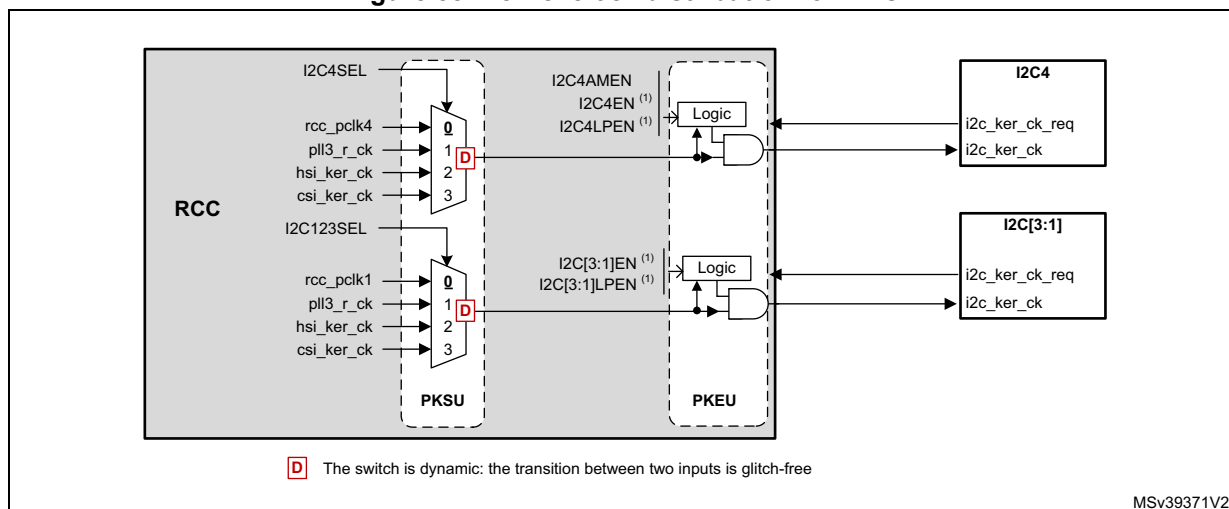
Note: UARTs also need the LSE clock when high baud rates are not required.

Figure 57. Kernel clock distribution for SPIs and SPI/I2S



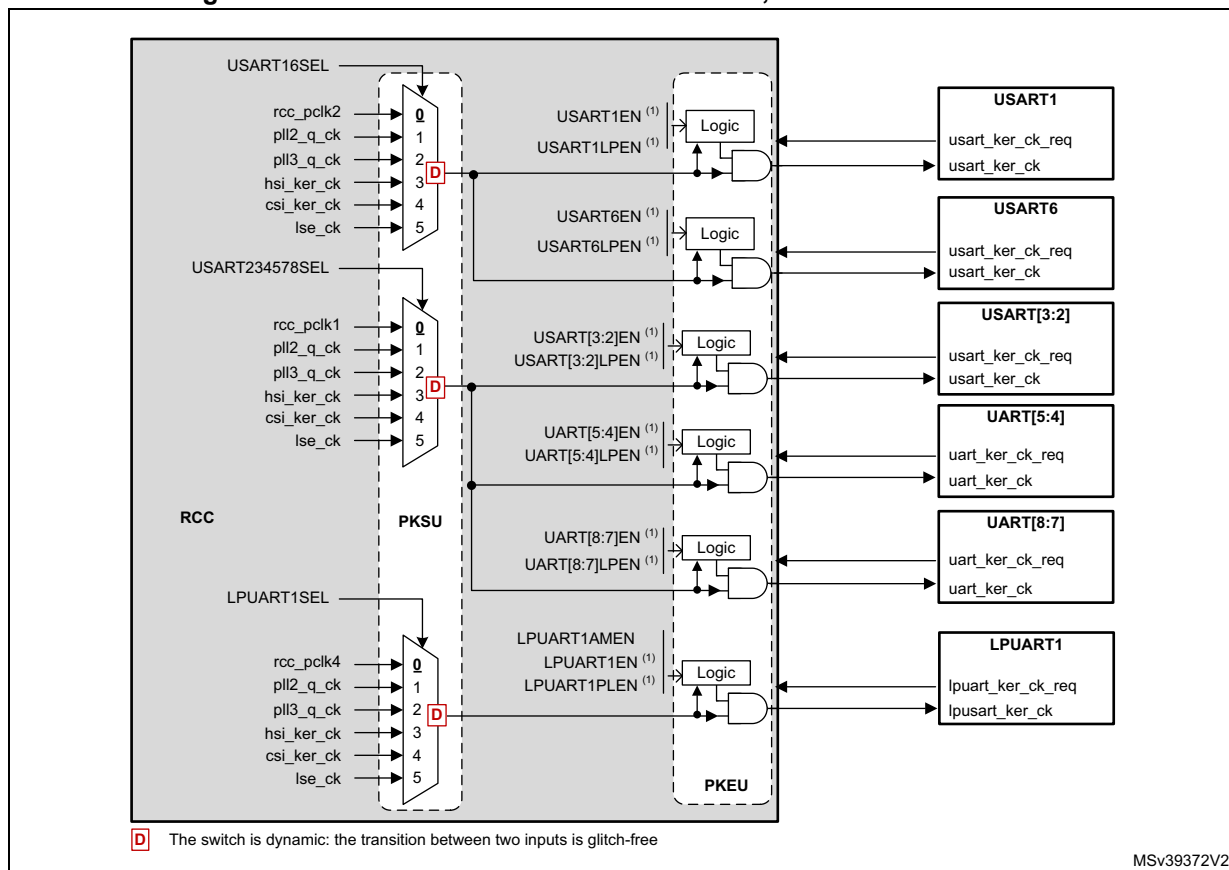
1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral. For details on each enable cell, please refer to [Section 9.5.11: Peripheral clock gating control](#).

Figure 58. Kernel clock distribution for I2Cs



1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral, for details on each enable cell, refer to [Section 9.5.11: Peripheral clock gating control](#).

Figure 59. Kernel clock distribution for UARTs, USARTs and LPUART1

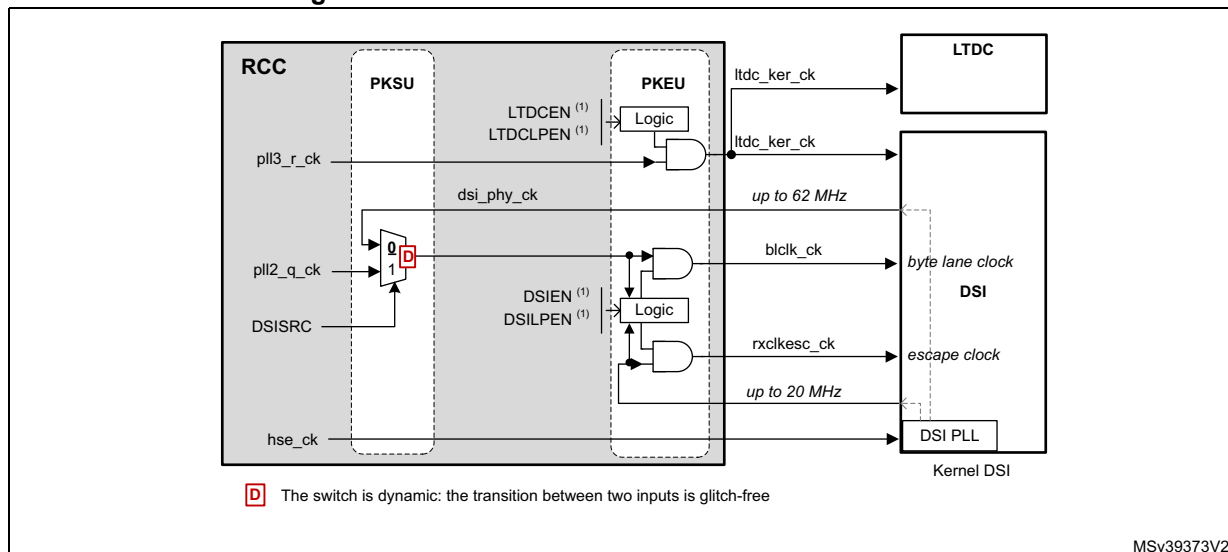


1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral, for details on each enable cell, refer to

Section 9.5.11: Peripheral clock gating control.

The DSI block has its own PLL which operates directly from the `hse_ck`.

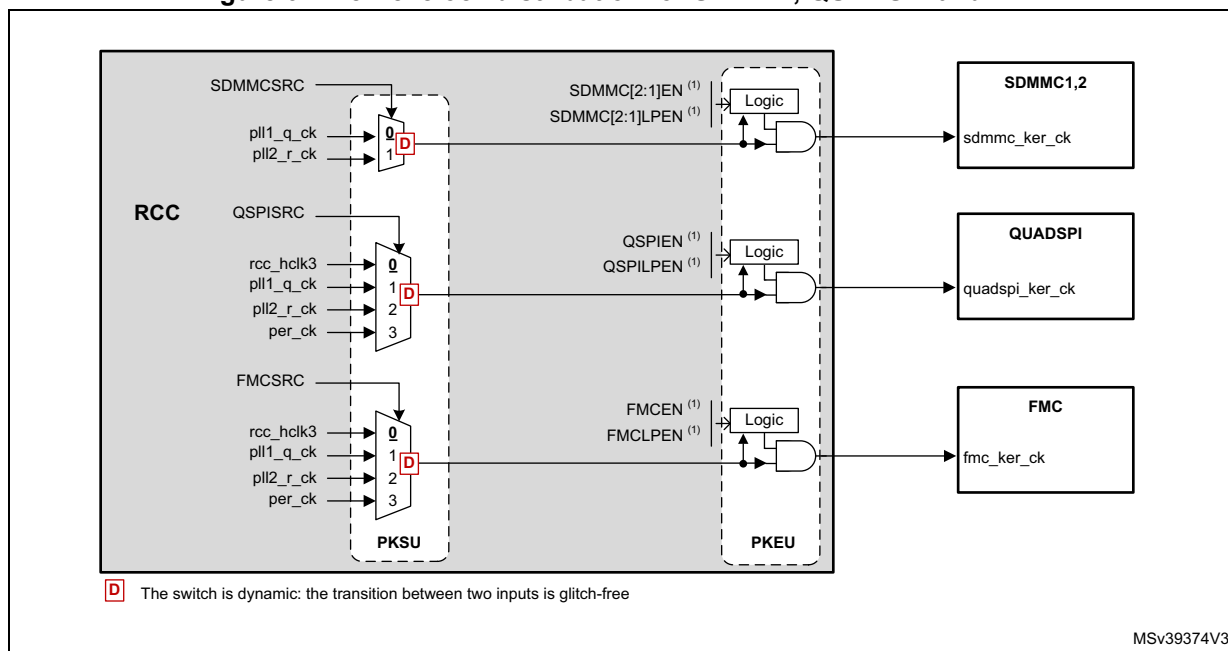
Figure 60. Kernel clock distribution for DSI and LTDC



1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral. For details on each enable cell, please refer to [Section 9.5.11: Peripheral clock gating control](#).

The FMC, QUADSPI and SDMMC1/2 can also use a clock different from the bus interface clock for more flexibility.

Figure 61. Kernel clock distribution for SDMMC, QUADSPI and FMC



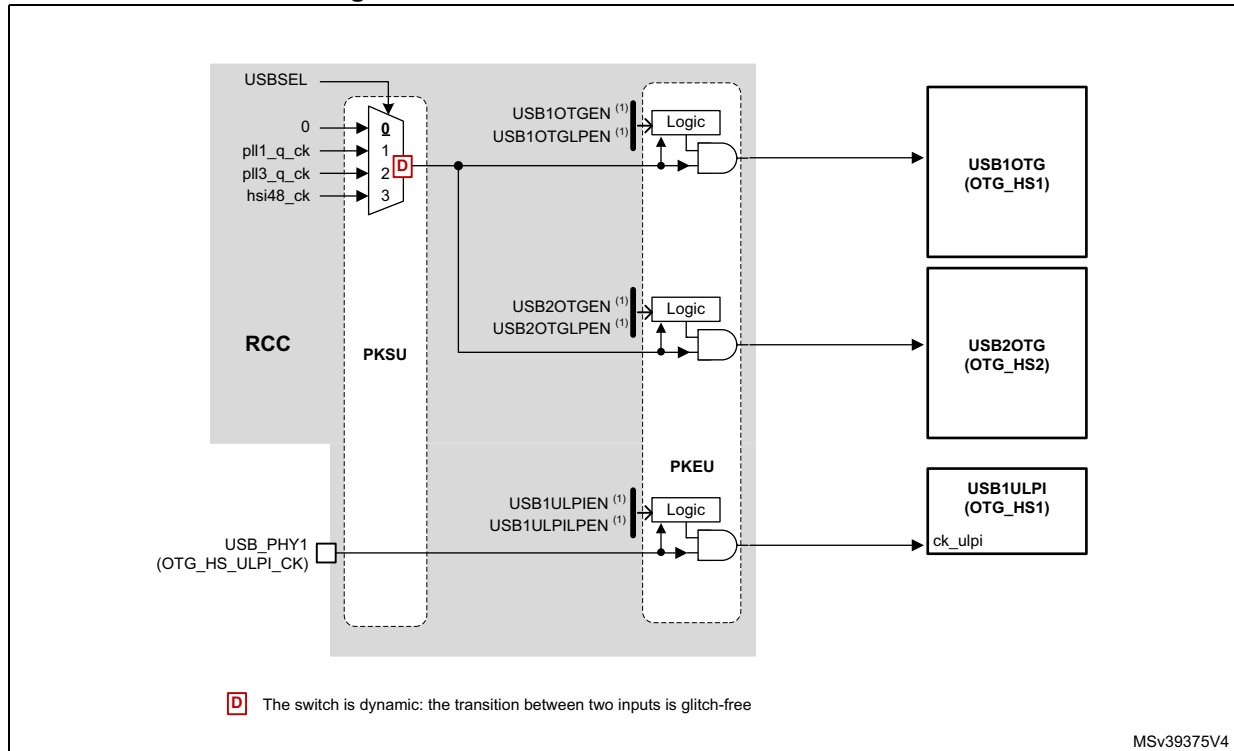
1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral. For details on each enable cell, please refer to [Section 9.5.11: Peripheral clock gating control](#).

refer to [Section 9.5.11: Peripheral clock gating control](#).

[Figure 62](#) shows the clock distribution for the USB blocks. The USB1ULPI blocks receive its clock from the external PHY.

The USBxOTG blocks receive the clock for USB communications which can be selected among different sources thanks to the MUX controlled by USBSEL.

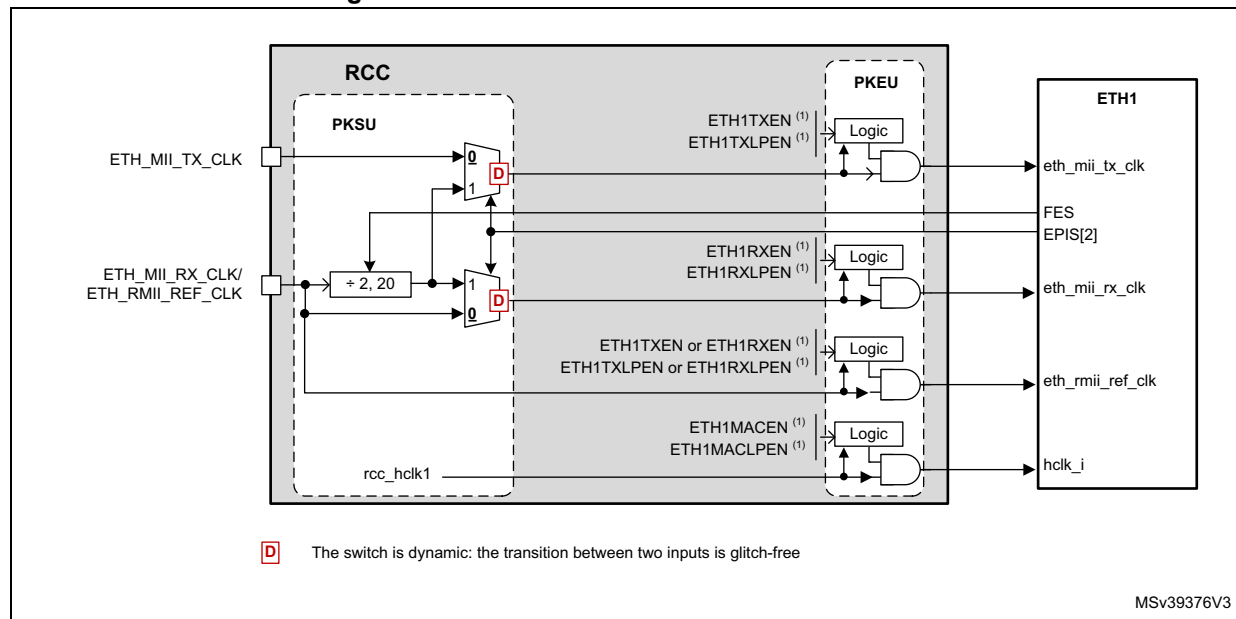
Figure 62. Kernel clock distribution For USB ⁽²⁾



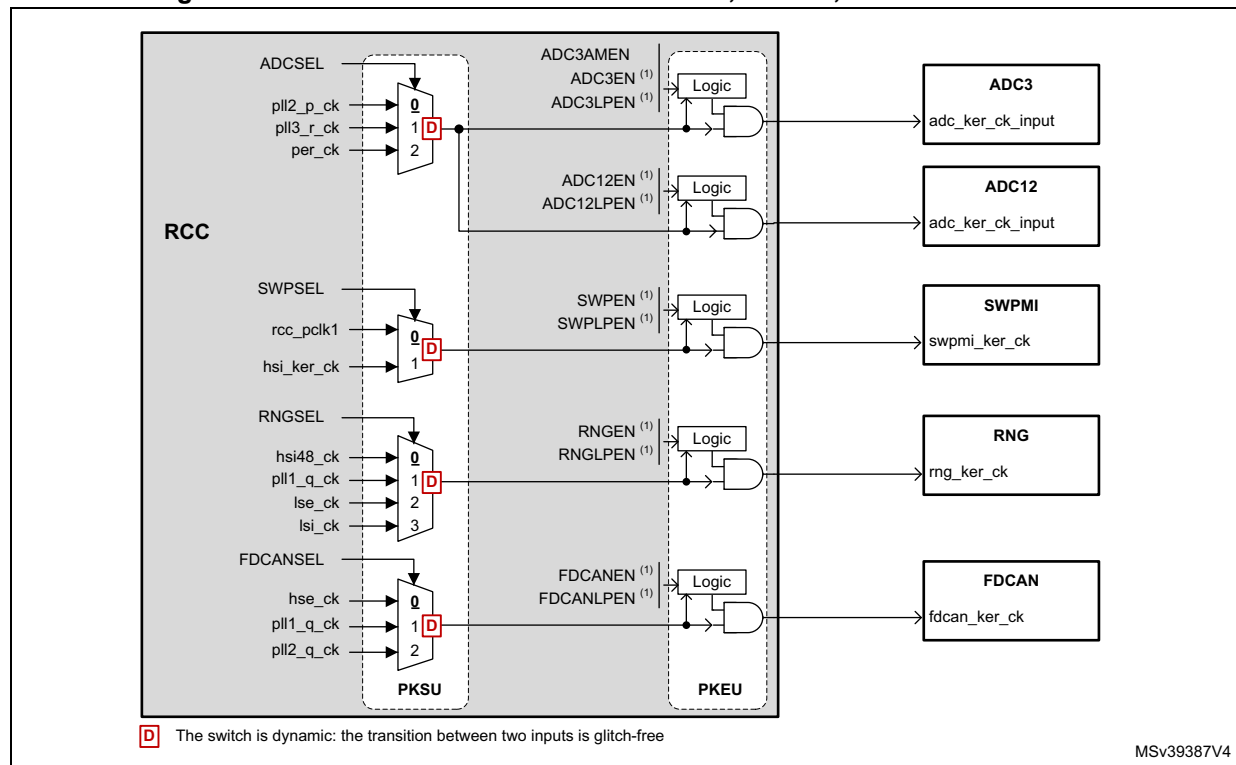
1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral. For details on each enable cell, please refer to [Section 9.5.11: Peripheral clock gating control](#).

The Ethernet transmit and receive clocks shall be provided from an external Ethernet PHY. The clock selection for the RX and TX path is controlled via the SYSCFG block.

Figure 63. Kernel clock distribution for Ethernet

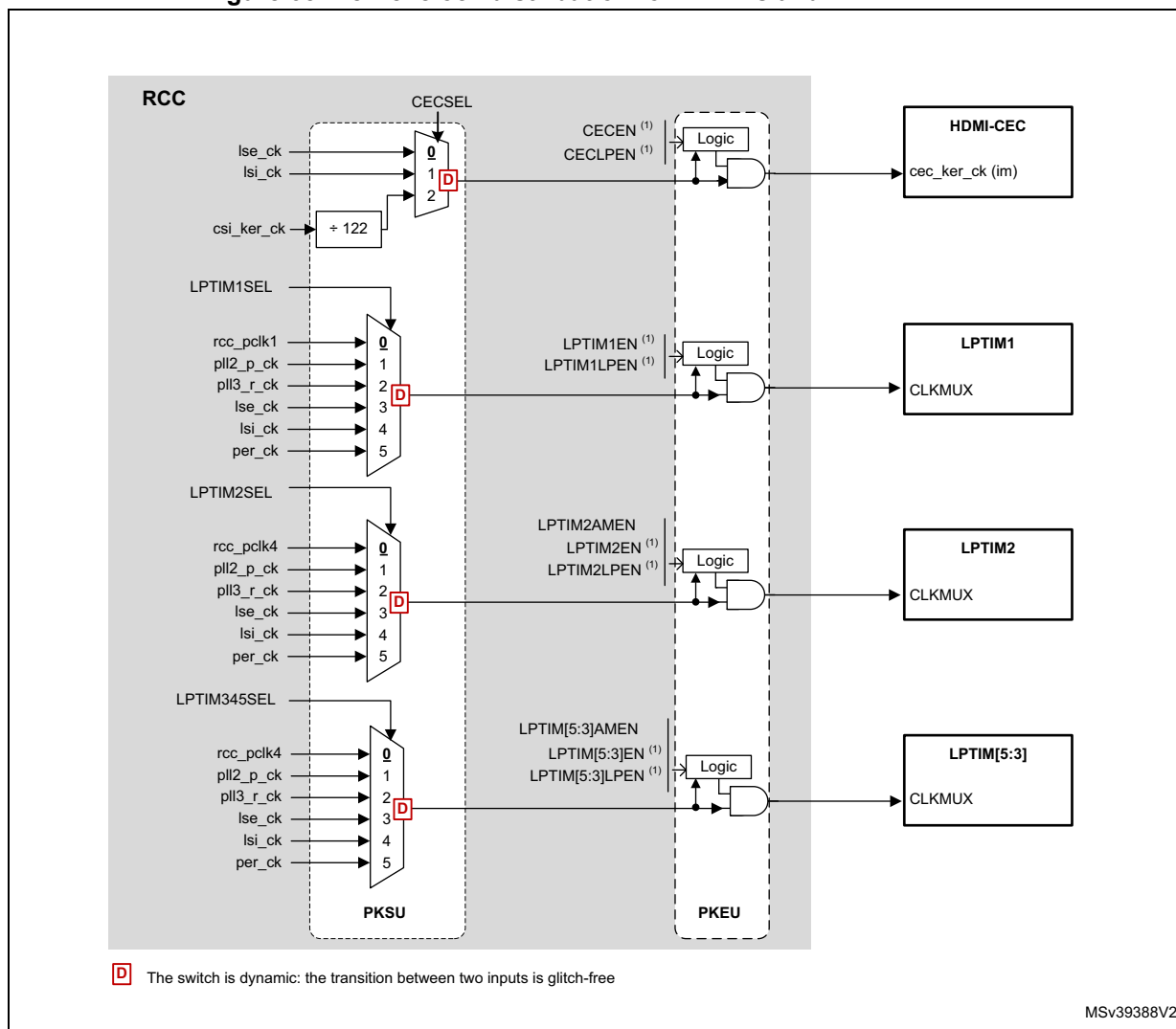


1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral. For details on each enable cell, please refer to [Section 9.5.11: Peripheral clock gating control](#).

Figure 64. Kernel clock distribution For ADCs, SWPMI, RNG and FDCAN ⁽²⁾

1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset.
2. This figure does not show the connection of the bus interface clock to the peripheral. For details on each enable cell, please refer to [Section 9.5.11: Peripheral clock gating control](#).

Figure 65. Kernel clock distribution for LPTIMs and HDMI-CEC (2)



1. There is one enable bit and one low-power enable bit from each CPU. **X** represents the selected MUX input after a system reset
2. This figure does not show the connection of the bus interface clock to the peripheral. For details on each enable cell, please refer to [Section 9.5.11: Peripheral clock gating control](#).

RTC/AWU clock

The **rtc_ck** clock source can be:

- the **hse_1M_ck** (**hse_ck** divided by a programmable prescaler)
- the **lse_ck**
- or the **lsi_ck** clock

The source clock is selected by programming the RTCSEL[1:0] bits in the *RCC Backup domain control register (RCC_BDCR)* and the RTCPRE[5:0] bits in the *RCC clock configuration register (RCC_CFGR)*.

This selection cannot be modified without resetting the Backup domain.

If the LSE is selected as RTC clock, the RTC will work normally even if the backup or the V_{DD} supply disappears.

The LSE clock is in the Backup domain, whereas the other oscillators are not. As a consequence:

- If LSE is selected as RTC clock, the RTC continues working even if the V_{DD} supply is switched OFF, provided the V_{BAT} supply is maintained.
- If LSI is selected as the RTC clock, the AWU state is not guaranteed if the V_{DD} supply is powered off.
- If the HSE clock is used as RTC clock, the RTC state is not guaranteed if the V_{DD} supply is powered off or if the V_{CORE} supply is powered off.

The **rtc_ck** clock is enabled through RTCEN bit located in the [RCC Backup domain control register \(RCC_BDCR\)](#).

The RTC bus interface clock (APB clock) is enabled through RTCAPBEN and RTCAPBLPEN bits located in RCC_APB4ENR/LPENR registers.

Note: *To read the RTC calendar register when the APB clock frequency is less than seven times the RTC clock frequency ($F_{APB} < 7 \times F_{RTCLK}$), the software must read the calendar time and date registers twice. The data are correct if the second read access to RTC_TR gives the same result than the first one. Otherwise a third read access must be performed.*

Watchdog clocks

The RCC provides the clock for the four watchdog blocks available on the circuit. The two independent watchdogs (IWDG1 and IWDG2) are connected to the LSI. The two window watchdogs (WWDG1 and WWDG2) are connected to the APB clock.

If an independent watchdog is started by either hardware option or software access, the LSI is forced ON and cannot be disabled. After the LSI oscillator setup delay, the clock is provided to the IWDGs.

Caution: Before enabling WWDG1/WWDG2, the application must set the WW1RSC/WW2RSC bit to '1'. If the WW1RSC/WW2RSC remains at '0' when the WWDG1/2 is enabled, the WWDG1/2 behavior is not guaranteed. The WW1RSC and WW2RSC bits are located in [RCC global control register \(RCC_GCR\)](#).

Clock frequency measurement using TIMx

Most of the clock source generator frequencies can be measured by means of the input capture of TIMx.

- Calibrating the HSI or CSI with the LSE
The primary purpose of having the LSE connected to a TIMx input capture is to be able to accurately measure the HSI or CSI. This requires to use the HSI or CSI as system clock source either directly or via PLL1. The number of system clock counts between consecutive edges of the LSE signal gives a measurement of the internal clock period. Taking advantage of the high precision of LSE crystals (typically a few tens of ppm) we can determine the internal clock frequency with the same resolution, and trim the

source to compensate for manufacturing-process and/or temperature- and voltage-related frequency deviations.

The basic concept consists in providing a relative measurement (e.g. HSI/LSE ratio): the precision is therefore tightly linked to the ratio between the two clock sources. The greater the ratio is, the more accurate the measurement is.

The HSI and CSI oscillators have dedicated user-accessible calibration bits for this purpose (see [RCC HSI configuration register \(RCC_HSI CFGR\)](#) and [RCC CSI configuration register \(RCC_CSI CFGR\)](#)).

When the HSI or CSI are used via the PLLx, the system clock can also be fine-tuned by using the fractional divider of the PLLs.

- Calibrating the LSI with the HSI

The LSI frequency can also be measured: this is useful for applications that do not have a crystal. The ultralow power LSI oscillator has a large manufacturing process deviation. By measuring it versus the HSI clock source, it is possible to determine its frequency with the precision of the HSI. The measured value can be used to have more accurate RTC time base timeouts (when LSI is used as the RTC clock source) and/or an IWDG timeout with an acceptable accuracy.

9.5.9 General clock concept overview

The RCC handles the distribution of the CPUs, bus interface and peripheral clocks for the system (D1, D2 and D3 domains), according to the operating mode of each function (refer to [Section 9.5.1: Clock naming convention](#) for details on clock definitions).

For each peripheral, the application can control the activation/deactivation of its kernel and bus interface clock. Prior to use a peripheral, the CPU has to enable it (by setting PERxEN to '1'), and define if this peripheral remains active in CSleep mode (by setting PERxLPEN to '1'). This is called 'allocation' of a peripheral to a CPU (refer to [Section 9.5.10: Peripheral allocation](#) for more details).

The peripheral allocation is used by the RCC to automatically control the clock gating according to the CPU and domain modes, and by the PWR to control the supply voltages of D1, D2 and D3 domains. For example if CPU1 enables a peripheral and decides that the kernel clock of this peripheral shall be gated when it goes to CSleep, then CPU2 state has no effect on the clock gating of this peripheral, but the clock gating will obey to CPU1 state. This is true if CPU2 did not allocate this peripheral as well.

Each CPU has dedicated registers where it can allocate individually the peripherals it intends to use in the application.

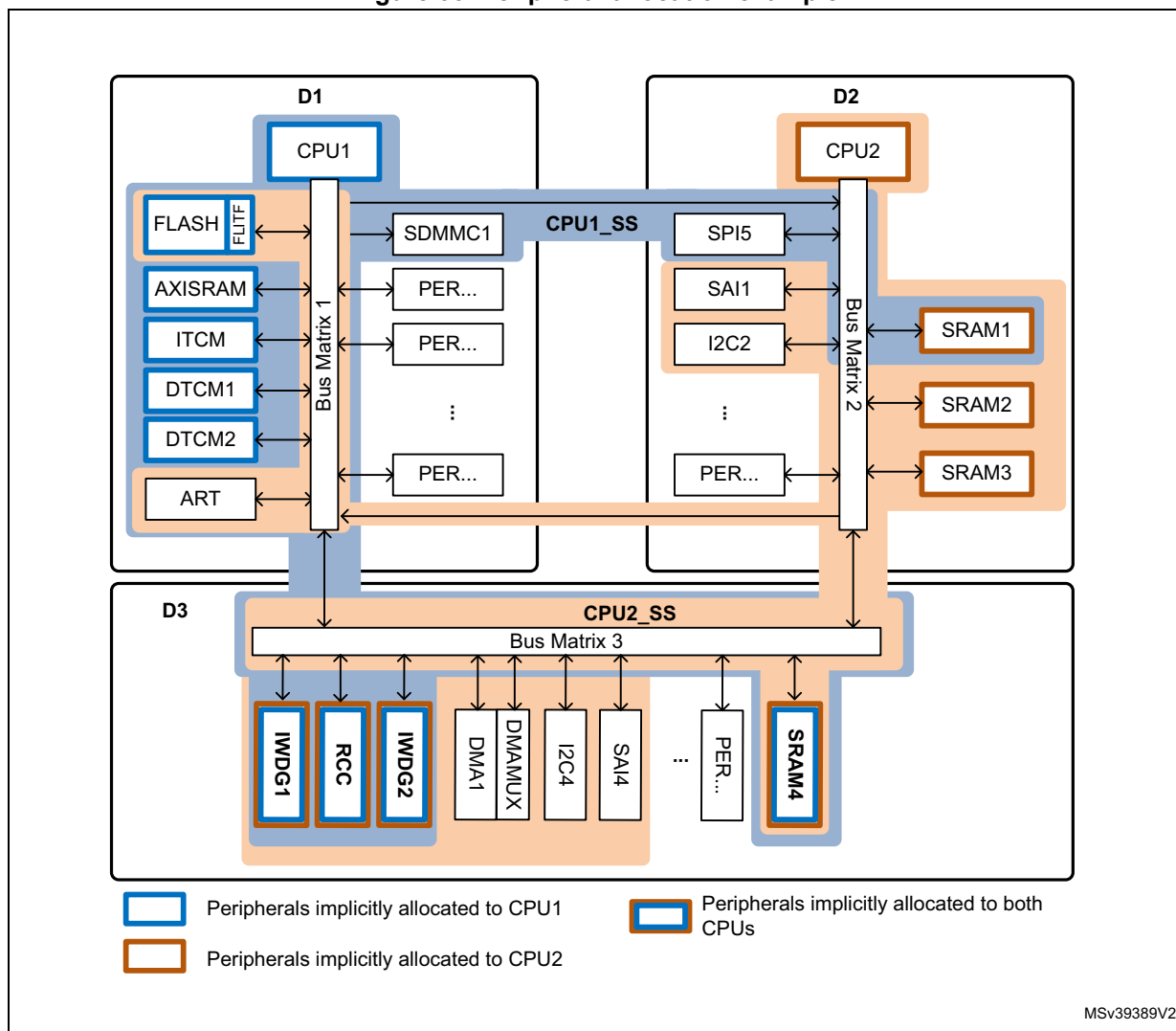
[Figure 66](#) gives an example of peripheral allocation:

- CPU1 enabled SDMMC1, SPI5 and SRAM1. AXISRAM, ITCM, DTCM1, DTCM2 and SRAM4 are implicitly allocated to CPU1. The group composed of CPU1, bus matrix 1/2/3 and allocated peripherals makes up sub-system 1 (CPU1_SS).
- The CPU2 enabled DMA1, I2C2, SAI1, FLASH, ART, I2C4, SAI4. Note that SRAM1, SRAM2, SRAM3 and SRAM4 are implicitly allocated to CPU2. The group composed by the CPU2, bus matrix 1,2 and 3 and peripherals allocated makes up sub-system 2 (CPU2_SS).

Note: SRAM4, IWGD1, IWGD2 and RCC are common resources and are implicitly allocated to both CPU1 and CPU2.

Some memories are implicitly allocated to a given CPU (see [Figure 66](#)). Refer to [Section : Memory handling](#) for more details.

Figure 66. Peripheral allocation example



Two additional functions are available to reduce power consumption:

- D3 domain can be kept in DRun mode while CPU1 and CPU2 are in CStop mode and D1 and D2 domains are in DStop or DStandby mode. This is done by setting RUN_D3 bit in PWR_D1CR or PWR_D2CR registers:
 - If RUN_D3 of PWR_CPU1CR or PWR_CPU2CR is set to '1', then D3 is maintained in DRun mode, independently from CPU1 and CPU2 modes (see [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#)).
 - If RUN_D3 of PWR_CPU1CR and RUN_D3 of PWR_CPU2CR are set to '0', then D3 domain enters DStop or DStandby mode when both CPU1 and CPU2 enter CStop mode (see [Table 63](#)).
- More autonomy can be given to some peripherals located into D3 domain (refer to [Section : D3 domain Autonomous mode](#) for details).

D3 domain Autonomous mode

The Autonomous mode allows to deliver the peripheral clocks to peripherals located in D3, even if the CPU to which they are allocated is in CStop mode. When a peripheral has its autonomous bit enabled, it receives its peripheral clocks according to D3 domain state, if the CPU to which it is allocated is in CStop mode:

- If the D3 domain is in DRun mode, peripherals with Autonomous mode activated receive their peripheral clocks,
- If the D3 domain is in DStop mode, no peripheral clock is provided.

The Autonomous mode does not prevent the D3 domain to enter DStop or DStandby mode.

The autonomous bits are located in [RCC D3 Autonomous mode register \(RCC_D3AMR\)](#).

For example, CPU1 and CPU2 can enter CStop mode, while the SAI4 is filling the SRAM4 with data received from an external device via DMA1. When the amount of received data is reached, the CPU2 can be activated by a wakeup event. This can be done by setting the SAI4, the DMA1, and SRAM4 in Autonomous mode, while keeping D3 in DRun mode (RUN_D3 set to '1'). In this example, the RCC does not switch off the PLLs as the D3 domain is always in DRun mode.

It is possible to go a step further with power consumption reduction by combining the Autonomous mode with the capability of some peripherals (UARTs, I2Cs) to request the kernel clock on their own, without waking-up the CPUs. For example, if the system is expecting messages via I2C4, the whole system can be put in Stop mode. When the I2C4 peripheral detects a START bit, it will generate a "kernel clock request". This request enables the HSI or CSI, and a kernel clock is provided only to the requester (in our example the I2C4). The I2C4 then decodes the incoming message. Several cases are then possible:

- If the device address of the message does not match, then I2C4 releases its "kernel clock request" until a new START condition is detected.
- If the device address of the incoming message matches, it has to be stored into D3 local memory. I2C4 is able to generate a wakeup event on address match to switch the D3 domain to DRun mode. The message is then transferred into memory via DMA1, and the D3 domain go back to DStop mode without any CPU activation.
- If the device address of the incoming message matches and the peripheral is setup to wake up the CPU, then I2C4 generates a wakeup event to activate CPU1_SS and CPU2_SS.

Please refer to the description of EXTI block in order see which peripheral is able to perform a wake-up event to which domain.

Memory handling

Both CPUs can access all the memory areas available in the product:

- AXISRAM, ITCM, DTCM1, DTCM2 and FLASH,
- SRAM1, SRAM2 and SRAM3,
- SRAM4 and BKPRAM.

As shown in [Figure 66](#), FLASH, AXISRAM, ITCM, DTCM1 and DTCM2 are implicitly allocated to CPU1. As a result, there is no enable bit allowing CPU1 to allocate these memories.

In the same way, SRAM1, SRAM2 and SRAM3 are implicitly allocated to CPU2, and CPU2 does not need enable bits to allocate them.

The SRAM4 does not need enable bit at all since D3 domain cannot be in DStop or DStandby mode when one of the CPU is running.

The BKPRAM has a dedicated enable bit for each CPU in order to gate the bus interface clock. The CPUs needs to enable the BKPRAM prior to use it.

When CPU1 wants to access a memory area located into D2 domain, the memory must be enabled via [RCC AHB2 clock register \(RCC_AHB2ENR\)](#). Enabling the memory insures that this memory will still be operating even if CPU2 enters CStop mode.

The same mechanism exists when CPU2 wants to use a memory located into D1 domain.

Note:

When a domain is in DRun mode, the memories located into this domain operate for both CPUs, even if memory enable bits are not set (except for the BKPRAM). The memory enable bits prevent the domain where the memory is located to enter DStop mode.

The memory interface clocks (Flash and RAM interfaces) can be stopped by software during CSleep mode (via DxSRAMyLPEN bits).

Refer to [Peripheral clock gating control](#) and [CPU and bus matrix clock gating control](#) sections for details on clock enabling.

System states overview

[Table 63](#) gives an overview of the system states with respect to CPU1, CPU2 and D3 states.

- The system remains in Run mode as long as D3 is in DRun mode. Several sub-states of system Run exist that are not detailed here (refer [Power control \(PWR\)](#) for more information).
- D3 can run while D1 and D2 are in DStop/DStandby mode thanks to RUN_D3 bits of PWR_CPU[2:1]CR registers or when D3 is in Autonomous mode.
- The system remains in Stop mode as long as D3 is in DStop mode. This means implicitly that D1 and D2 are in DStop or DStandby. As soon as D1 or D2 exits DStop or DStandby, D3 switches to DRun mode.
- The system remains in Standby mode as long as D1, D2 and D3 are in DStandby.
- Domain states versus CPU states:
 - When a domain is in DRun mode, it means that its bus matrix is clocked. The CPU of the domain (if any) can be in CRun, CSleep or CStop mode.
 - When a domain is in DStop mode, it means that its bus matrix is no longer clocked. The CPU of the domain (if any) is in CStop mode.
 - When a domain is in DStandby mode, it means that the domain including its CPU are powered down.

Table 63. System states overview

System State	D1 State	D2 State	D3 State
Run	DRun/DStop/DStandby	DRun/DStop/DStandby	DRun
Stop	DStop/DStandby	DStop/ DStandby	DStop
Standby	DStandby	DStandby	DStandby

9.5.10 Peripheral allocation

Each CPU can allocate a peripheral and hence control its kernel and bus interface clock. Each CPU has dedicated registers in order to perform this peripheral allocation. A peripheral is allocated when its PERxEN bit is set to '1' by the CPU1 and/or CPU2.

CPU1 can allocate a peripheral for itself by setting the dedicated PERxEN bit in:

- RCC_DnxxxxENR registers or
- RCC_C1_DnxxxxENR registers.

CPU1 can also allocate a peripheral for CPU2 by setting the dedicated PERxEN bit in registers RCC_C2_DnxxxxENR.

In the same way, CPU2 can allocate a peripheral for itself by setting the dedicated PERxEN bit in:

- RCC_DnxxxxENR registers or
- RCC_C2_DnxxxxENR registers.

CPU2 can also allocate a peripheral for CPU1 by setting the dedicated PERxEN bit in registers RCC_C1_DnxxxxENR.

A similar mechanism is implemented to control the peripheral clocks when the CPUs are in CSleep mode (PERxLPEN bits).

Refer to [Section 9.7.1: Register mapping overview](#) for additional information.

Note: When CPU1 or CPU2 access RCC_DnxxxxxENR registers, the hardware detects which CPU did the access (core_id) in order to allocate the peripheral to the right CPU.

The peripheral allocation bits (PERxEN bits) are used by the hardware to provide the kernel and bus interface clocks to the peripherals. However they are also used to link peripherals to a CPU (CPU sub-system). In this way, the hardware is able to safely gate the peripheral clocks and bus matrix clocks according to CPU states. The PWR block also uses this information to control properly the domain states.

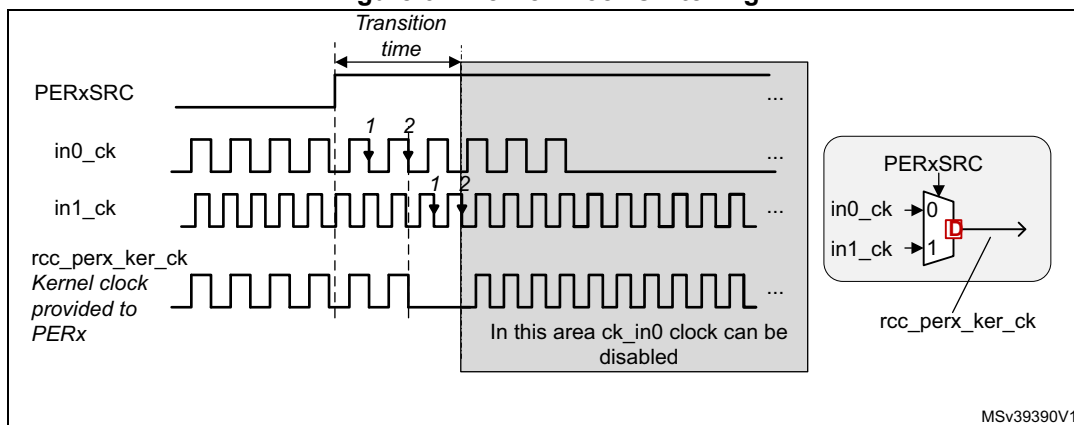
Clock switches and gating

- Clock switching delays

The input selected by the kernel clock switches can be changed dynamically without generating spurs or timing violation. As a consequence, switching from the original to the new input can only be performed if a clock is present on both inputs. If it not the case, no clock will be provided to the peripheral. To recover from this situation, the user has to provide a valid clock to both inputs.

During the transition from one input to another, the kernel clock provided to the peripheral will be gated, in the worst case, during 2 clock cycles of the previously selected clock and 2 clock cycles of the new selected clock. As shown in [Figure 67](#), both input clocks shall be present during transition time.

Figure 67. Kernel Clock switching



MSv39390V1

- Clock enabling delays

In the same way, the clock gating logic synchronizes the enable command (coming generally from a kernel clock request or PERxEN bits) with the selected clock, in order to avoid generation of spurs.

- A maximum delay of two periods of the enabled clock may occur between the enable command and the first rising edge of the clock. The enable command can be the rising edge of the PERxEN bits of RCC_xxxxENR registers, or a kernel clock request asserted by a peripheral.
- A maximum delay of 1.5 periods of the disabled clock may occur between the disable command and the last falling edge of the clock. The disable command can be the falling edge of the PERxEN bits of RCC_xxxxENR registers, or a kernel clock request released by a peripheral.

Note: Both the kernel clock and the bus interface clock are affected by this re-synchronization delay.

In addition, the clock enabling delay may strongly increase if the application is enabling for the first time a peripheral which is not located in the same domain. This is due to the fact that the domain where the peripheral is located could be in DStop or DStandby mode. The domain must be switched to DRun mode before the application can use this peripheral.

As an example, if CPU1 enables a peripheral located in the D2 domain while the D2 domain is in DStop/DStandby mode, then the power controller (PWR) has first to provide a supply voltage to D2, then the RCC has to wait for an acknowledge from the

PWR before enabling the clocks of the D2 domain. To handle properly this situation the RCC and the PWR blocks feature four flags:

- D1CKRDY/D2CKRDY located in [RCC source control register \(RCC_CR\)](#)
- SBF_D1 and SBF_D2 located in [PWR CPU1 control register \(PWR_CPU1CR\)](#) and [PWR CPU2 control register \(PWR_CPU2CR\)](#), respectively.

The following sequence can be followed to avoid this issue:

- a) Enable the peripheral clocks (i.e. allocate the peripheral) by writing the corresponding PERxEN bit to '1' in the RCC_xxxxENR register,
- b) Read back the RCC_xxxxENR register to make sure that the previous write operation is not pending into a write buffer.
- c) If the peripheral is located in a different domain, perform the two next steps:
Read DxCKRDY until it is set to '1'.
Write SBF_Dx to zero and read-back the value, in order to check if the domain where the peripheral is located is still in DStandby. If the corresponding bit is read at '1', it means that the domain is still in DStandby. Repeat this operation until SBF_Dx is equal to '0', then continue the other steps.
- d) Perform a dummy read operation into a register of the enabled peripheral. This operation will take at least 2 clock cycles, which is equal to the max delay of the enable command.
- e) The peripheral can then be used.

Note: *When the bus interface clock is not active, read or write accesses to the peripheral registers are not supported. A read access will return invalid data. A write access will be ignored and will not create any bus errors.*

9.5.11 Peripheral clock gating control

As mentioned previously, each peripheral requires a bus interface clock, named **rcc_perx_bus_ck** (for peripheral 'x'). This clock can be an APB, AHB or AXI clock, according to which bus the peripheral is connected.

The clocks used as bus interface for peripherals located in D1 domain, could be **rcc_aclk**, **rcc_hclk3** or **rcc_pclk3**, depending on the bus connected to each peripheral. For simplicity sake, these clocks are named **rcc_bus_d1_ck**.

In the same way, the signal named **rcc_bus_d2_ck** represents **rcc_hclk1**, **rcc_hclk2**, **rcc_pclk1** or **rcc_pclk2**, depending on the bus connected to each peripheral of D2 domain.

Similarly, the signal **rcc_bus_d3_ck** represents **rcc_hclk4** or **rcc_pclk4** for peripherals located in D3.

Some peripherals (SAI, UART...) also require a dedicated clock for their communication interface. This clock is generally asynchronous with respect to the bus interface clock. It is named kernel clock (**perx_ker_ckreq**). Both clocks can be gated according to several conditions detailed hereafter.

As shown in [Figure 68](#), enabling the kernel and bus interface clocks of each peripheral depends on several input signals:

- C1_PERxEN and C1_PERxLPEN bits
C1_PERxEN represents the peripheral enable (allocation) bit for CPU1. CPU1 can write these bits to '1' via RCC_C1_xxxxENR or RCC_DnxxxxENR registers.
- C2_PERxEN and C2_PERxLPEN bits
C2_PERxEN represents the peripheral enable (allocation) bit for CPU2. The CPU2 can write these bits to '1' via RCC_C2_xxxxENR or RCC_DnxxxxENR registers.
- PERxAMEN bits
The PERxAMEN bits are belong to [RCC D3 Autonomous mode register \(RCC_D3AMR\)](#).
- CPU1 state (**c1_sleep** and **c1_deepsleep** signals)
- CPU2 state (**c2_sleep** and **c2_deepsleep** signals)
- D3 domain state (**d3_deepsleep** signal)
- The kernel clock request (**perx_ker_ckreq**) of the peripheral itself, when the feature is available.

Refer to [Section 9.5.10: Peripheral allocation](#) for more details.

Figure 68. Peripheral kernel clock enable logic details

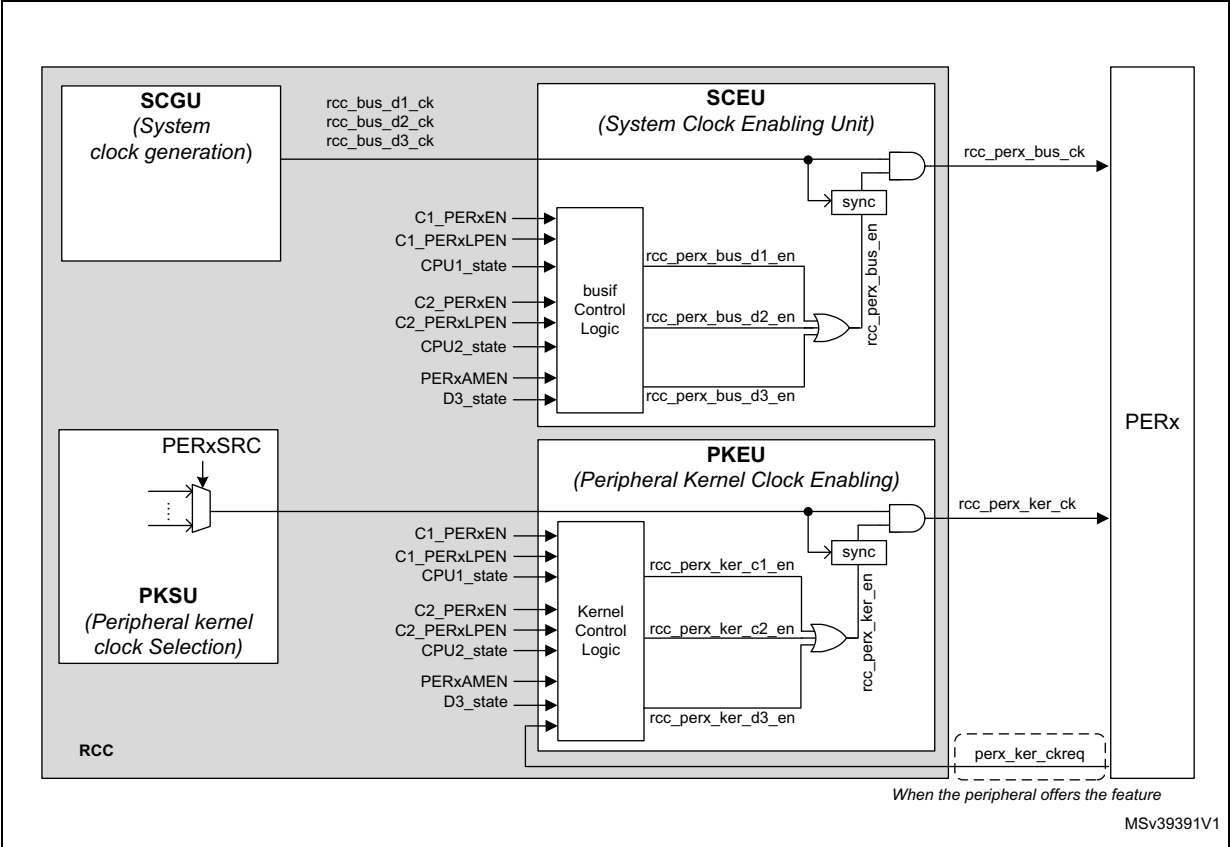


Table 64 gives a detailed description of the enabling logic of the peripheral clocks for peripherals located in D1 or D2 domain and allocated by CPU1. A similar logic applies to CPU2.

Table 64. Peripheral clock enabling for D1 and D2 peripherals

C1_PERxEN	C1_PERxLPEN	PERxSRC	perx_ker_ckreq	CPU1 State		rcc_perx_ker_c1_en	rcc_perx_bus_d1_en	Comments
0	X	X	X	X	→	0	0	No clock provided to the peripheral, because PERxEN='0'
1	X	X	X	CRun	→	1	1	Kernel and bus interface clocks are provided to the peripheral, because the CPU1 is in CRun, and PERxEN='1'
1	0	X	X	CSleep	→	0	0	No clock provided to the peripheral, because the CPU1 is in CSleep, and PERxLPEN='0'
1	1	X	X		→	1	1	Kernel and bus interface clocks are provided to the peripheral, because CPU1 is in CSleep, and PERxLPEN='1'
1	0	X	X	CStop	→	0	0	No clock provided to the peripheral because the PERxLPEN bit is set to '0'.
1	1	no lsi_ck and no lse_ck and no hsi_ker_ck and no csi_ker_ck	X		→	0	0	No clock provided to the peripheral because CPU1 is in CStop and lse_ck or lsi_ck or hsi_ker_ck or csi_ker_ck are not selected.
1	1	lsi_ck or lse_ck	X		→	1 ⁽¹⁾	0	Kernel clock is provided to the peripheral because PERxEN = PERxLPEN='1' and lsi_ck or lse_ck are selected. The bus interface clock is no provided as the CPU is in CStop
1	1	hsi_ker_ck or csi_ker_ck	1		→	1	0	Kernel clock is provided to the peripheral because req_ker_perx = '1', and PERxEN = PERxLPEN='1' and hsi_ker_ck or csi_ker_ck are selected. The bus interface clock is no provided as the CPU is in CStop
1	1	hsi_ker_ck or csi_ker_ck	0		→	0	0	No clock provided to the peripheral because CPU1 is in CStop, and no kernel clock request pending

1. For RNG block, the kernel clock is not delivered if the CPU to which it is allocated is in CStop mode, even if the clock selected is lsi_ck or lse_ck.

As a summary, we can state that the kernel clock is provided to the peripherals located on domains D1 and D2 when the following conditions are met:

1. The CPU to which the peripheral is allocated is in CRun mode.
2. The CPU to which the peripheral is allocated is in CSleep mode and PERxLPEN = '1'.
3. The CPU to which the peripheral is allocated is in CStop mode, with PERxLPEN = '1', the peripheral generates a kernel clock request, and the selected clock is **hsi_ker_ck** or **csi_ker_ck**.
4. The CPU to which the peripheral is allocated is in CStop mode, with PERxLPEN = '1', and the kernel source clock of the peripheral is **lse_ck** or **lsi_ck**.

The bus interface clock will be provided to the peripherals only when conditions 1 or 2 are met.

[Table 65](#) gives a detailed description of the enabling logic of the kernel clock for all peripherals located in D3 and allocated by CPU1. A similar logic applies to CPU2.

Table 65. Peripheral clock enabling for D3 peripherals ⁽¹⁾

C1_PERxEN	C1_PERxLPEN	PERxAMEN	PERxSRC	perx_ker_ckreq	CPU1 State	D3 State		rcc_perx_ker_d3_en	rcc_perx_bus_d3_en	Comments
0	X	X	X	X	Any	Any	= >	0	0	No clock provided to the peripheral, as C1_PERxEN='0'
1	X	X	X	X	CRun	DRun	= >	1	1	Kernel and bus interface clocks are provided to the peripheral, because the CPU1 is in CRun, and C1_PERxEN='1'
1	0	X	X	X	CSleep		= >	0	0	No clock provided to the peripheral, because the CPU1 is in CSleep, and C1_PERxLPEN='0'
1	1	X	X	X			= >	1	1	Kernel and bus interface clocks are provided to the peripheral, because the CPU1 is in CSleep, and C1_PERxLPEN='1'
1	X	0	X	X	CStop		= >	0	0	As the CPU1 is in CStop, and C1_PERxEN='1', then the kernel clock gating depends on D3 state and PERxAMEN bits. No clock provided to the peripheral because PERxAMEN = '0'.
1	X	1	X	X			= >	1	1	The kernel and bus interface clocks are provided because even if the CPU to which the peripheral is allocated is in CStop mode, D3 is in DRun mode, with C1_PERxEN and PERxAMEN bits set to '1'.
1	X	1	not lse_ck and not lsi_ck	0			DStop	= >	0	0

Table 65. Peripheral clock enabling for D3 peripherals ⁽¹⁾ (continued)

C1_PERxEN	C1_PERxLPEN	PERxAMEN	PERxSRC	perx_ker_ckreq	CPU1 State	D3 State		rcc_perx_ker_d3_en	rcc_perx_bus_d3_en	Comments
1	X	1	not hsi_ker_ck and not csi_ker_ck and not lse_ck and not lsi_ck	1	CStop	DStop	→	0	0	No clock provided to the peripheral, because even if req_ker_perx = '0', lse_ck or lsi_ck or hsi_ker_ck or csi_ker_ck are not selected.
1	X	1	hsi_ker_ck or csi_ker_ck	1			→	1	0	Kernel clock is provided to the peripheral because req_ker_perx = '1', and C1_PERxEN = PERxAMEN='1', and the selected clock is hsi_ker_ck or csi_ker_ck . The bus interface clock is not provided as D3 is in DStop.
1	X	1	lse_ck or lsi_ck	X			→	1	0	Kernel clock is provided to the peripheral because C1_PERxEN = PERxAMEN='1' and lse_ck or lsi_ck are selected, while D3 is in STOP. The bus interface clock is not provided as D3 is in DSTOP.

1. The above table is given for the peripherals allocated by CPU1. A similar truth table is applicable in the case of peripherals allocated by CPU2.

As a summary, we can state that the kernel clock is provided to the peripherals of D3 if the following conditions are met:

1. The CPU to which the peripheral is allocated is in CRun mode.
2. The CPU to which the peripheral is allocated is in CSleep mode and PERxLPEN = '1'.
3. The CPU to which the peripheral is allocated is in CStop mode and D3 domain is in DRun mode with PERxAMEN = '1'.
4. The CPU to which the peripheral is allocated is in CStop mode, D3 domain is in DStop mode with PERxAMEN = '1', the peripheral is generating a kernel clock request and the kernel clock source is **hsi_ker_ck** or **csi_ker_ck**.
5. The CPU to which the peripheral is allocated is in CStop mode, D3 domain is in DStop mode with PERxAMEN = '1', and the kernel clock source of the peripheral is **lse_ck** or **lsi_ck**.

The bus interface clock will be provided to the peripherals only when condition 1, 2 or 3 is met.

Note: When they are set to '1', the autonomous bits indicate that the associated peripheral will receive a kernel clock according to D3 state, and not according to the mode of the CPU to which it is allocated.

Only I2C, U(S)ART and LPUART peripherals are able to request the kernel clock. This feature gives to the peripheral the capability to transfer data with an optimal power consumption.

The autonomous bits dedicated to some peripherals located in D3 domain allow the data transfer with external devices without activating the CPUs.

*In order for the LPTIMER to operate with **Lse_ck** or **Isi_ck** when the circuit is in Stop mode, the user application has to select the **Isi_ck** or **Lse_ck** input via LPTIMxSEL fields, and set bit LPTIMxAMEN and LPTIMxLPEN to '1'.*

9.5.12 CPU and bus matrix clock gating control

For each domain it is possible to control the activation/deactivation of the CPU clock and bus matrix clock.

- The clock of each CPU is controlled by the **c[2:1]_sleep** signal.
- The domain bus clock is controlled by the CPU Deepsleep signal and by the other CPU Deepsleep signal when it has allocated peripheral(s).

For information about convention naming, refer to [Section 9.5.11: Peripheral clock gating control](#).

Table 66. Domain bus clock enabling for D1 peripherals

c2_per_alloc_d1 ⁽¹⁾	CPU1 State	CPU2 State		rcc_c1_ck	rcc_bus_d1_ck	Comments
X	CRun	X	=>	enabled	enabled	CPU and bus matrix clock are provided to D1, because CPU1 is in CRun.
X	CSleep	X	=>	disabled	enabled	D1 bus matrix and CPU clocks are stopped because CPU1 is in CSleep.
0	CStop	X	=>	disabled	disabled	No clock provided to D1, because there is no peripheral located on D1 enabled via RCC_C2_xxxENR registers.
1	CStop	CRun	=>	disabled	enabled	Bus matrix clock provided to D1, CPU clock stopped, because at least one peripheral located on D1 is enabled via RCC_C2_xxxENR registers, and CPU2 is not in CStop mode.
1	CStop	CSleep	=>			
X	CStop	CStop	=>	disabled	disabled	No clock provided to D1, because CPU2 is in CStop.

1. c2_per_alloc_d1 represents a logic OR of all RCC_C2_xxxENR.PERxEN bits, for peripherals located in D1 domain.

Table 67 shows the conditions which enable or disable the bus matrix clock on D3.

Table 67. Domain bus clock enabling for D3 peripherals

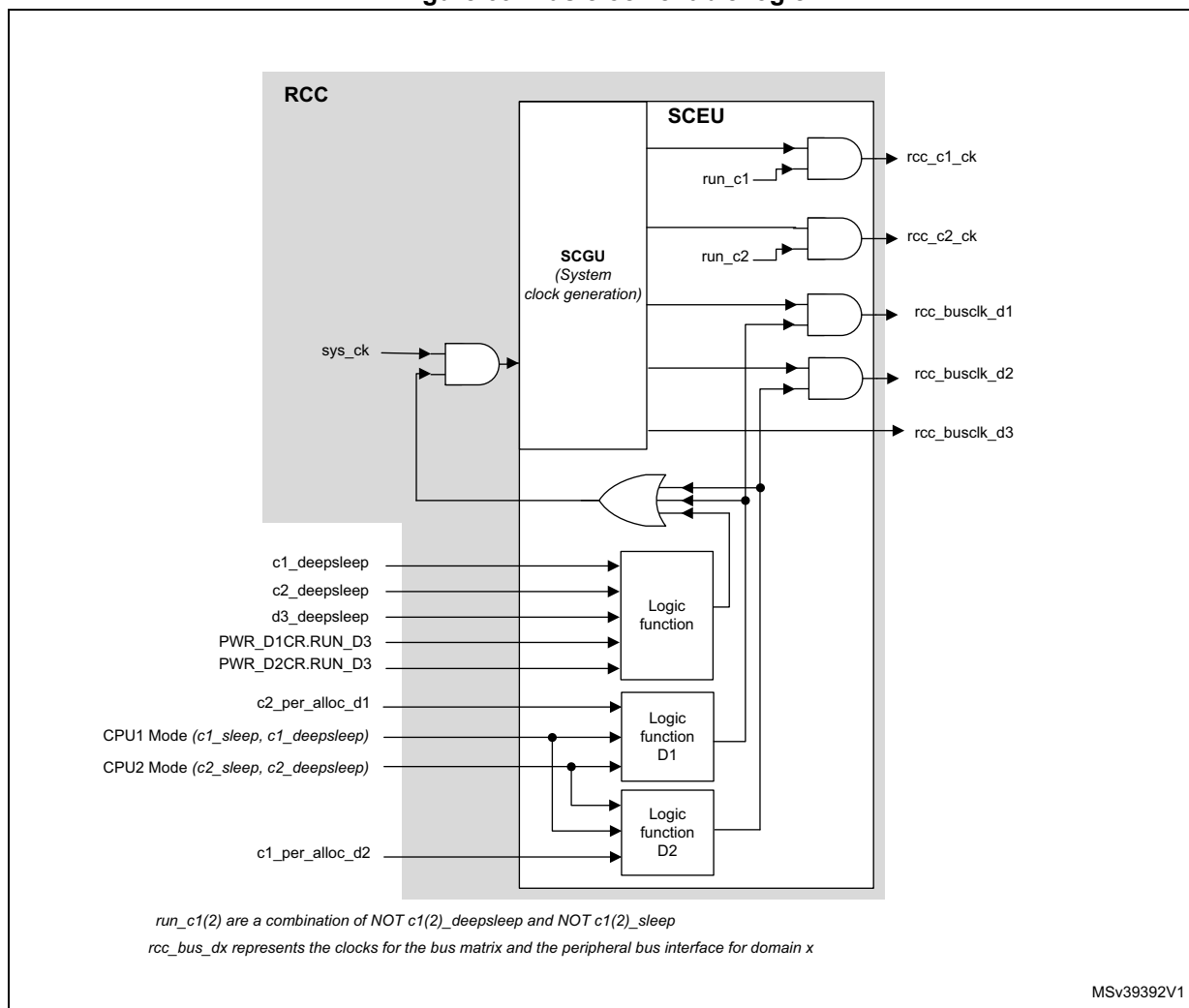
PWR_D1CR.RUN_D3	PWR_D2CR.RUN_D3	D3 State	CPU1 State	CPU2 State		rcc_bus_d3_ck	Comments
X	X	DRun	CRun	X	=>	enabled	When CPU1 or CPU2 are not in CStop mode, D3 domain is in DRun and the bus matrix clock is enabled.
X	X	DRun	X	CRun			
X	X	DRun	CSleep	X			
X	X	DRun	X	CSleep			
0	0	DStop	CStop	CStop	=>	disabled	When there is no request to run D3, and both CPU are in CStop mode then the bus matrix clock is disabled.
0	0	DRun	CStop	CStop	=>	enabled	When both CPUs are in CStop mode, if there is a request to run D3 via an enabled wake-up request, then the bus matrix clock is enabled.
1	X	DRun	X	X	=>	enabled	When bits D1CR.RUN_D3 or D2CR.RUN_D3 are set to '1', then the bus matrix clock is enabled, independently to other conditions.
X	1	DRun	X	X			

Note: When all peripherals connected to a same APB bus are disabled, the APB clock bridge is automatically gated as well to reduce the power consumption.

As shown in the Figure 69, the enabling of the core and bus clock of each domain depends on several input signals:

- **c1_sleep** and **c1_deepsleep** signals from CPU1,
- **c2_sleep** and **c2_deepsleep** signals from CPU2,
- **d3_sleepdeep** signal,
- RCC_C2_xxxxENR.PERxEN bits of peripherals located on D1 domain
- RCC_C1_xxxxENR.PERxEN bits of peripherals located on D2 domain

Figure 69. Bus clock enable logic



- When a CPU is in CRun mode, the corresponding CPU clock and bus clocks are provided to the domain.
- When a CPU is in CSleep mode, the corresponding bus clocks are provided to the domain, and the CPU clock is stopped.
- When CPU1 is in CStop mode and the other CPU has no peripherals allocated in the D1 domain, the CPU and bus matrix clocks of the D1 domain are stopped. In the same way, when CPU2 is in CStop mode and the other CPU has no peripherals allocated in the D2 domain, CPU2 and bus matrix clocks of the D2 domain are stopped.
- When CPU1 is in CStop mode and the other CPU has allocated peripherals on D1 domain, and is in CRun or CSleep, the corresponding bus matrix clock is provided to the D1 domain, and the CPU1 clock is stopped. In the same way, when CPU2 is in CStop mode and the other CPU has allocated peripherals on D2 domain, and is in

CRun or CSleep, the corresponding bus matrix clock is provided to the D2 domain, and the CPU2 clock is stopped.

- When CPU1 and CPU2 are in CStop mode, the bus matrix and the CPU clocks on both domains are stopped.
- Whenever CPU1 or CPU2 are in CRun or CSleep mode, the D3 domain bus matrix clock is running.
- When both CPU1 and CPU2 are in CStop mode, the D3 domain bus matrix clock is stopped, if RUN_D3 bits are set to '0', and d3_deepsleep = '1'.
- Whenever one of the two RUN_D3 bits is set the D3 domain bus clock is running.
- Whenever the d3_deepsleep signal is inactive (0), the D3 domain bus clock is running.

9.6 RCC Interrupts

The RCC provides 3 interrupt lines:

- **rcc_it**: a general interrupt line, providing events when the PLLs are ready, or when the oscillators are ready.
- **rcc_hsecss_it**: an interrupt line dedicated to the failure detection of the HSE Clock Security System.
- **rcc_lsecss_it**: an interrupt line dedicated to the failure detection of the LSE Clock Security System.

The interrupt enable is controlled via [RCC clock source interrupt enable register \(RCC_CIER\)](#), except for the HSE CSS failure. When the HSE CSS feature is enabled, it not possible to mask the interrupt generation.

The interrupt flags can be checked via [RCC clock source interrupt flag register \(RCC_CIFR\)](#), and those flags can be cleared via [RCC clock source interrupt clear register \(RCC_CICR\)](#).

Note: The interrupt flags are not relevant if the corresponding interrupt enable bit is not set.

[Table 68](#) gives a summary of the interrupt sources, and the way to control them.

Table 68. Interrupt sources and control

Interrupt Source	Description	Interrupt enable	Action to clear interrupt	Interrupt Line
LSIRDYF	LSI ready	LSIRDYIE	Set LSIRDYC to '1'	rcc_it
LSERDYF	LSE ready	LSERDYIE	Set LSERDYC to '1'	
HSIDRYF	HSI ready	HSIDRYIE	Set HSIRDYC to '1'	
HSERDYF	HSE ready	HSERDYIE	Set HSERDYC to '1'	
CSIRDYF	CSI ready	CSIRDYIE	Set CSIRDYC to '1'	
HSI48RDYF	HSI48 ready	HSI48RDYIE	Set HSI48RDYC to '1'	
PLL1RDYF	PLL1 ready	PLL1RDYIE	Set PLL1RDYC to '1'	
PLL2RDYF	PLL2 ready	PLL2RDYIE	Set PLL2RDYC to '1'	
PLL3RDYF	PLL3 ready	PLL3RDYIE	Set PLL3RDYC to '1'	
LSECSSF	LSE Clock security system failure	LSECSSFIE ⁽¹⁾	Set LSECSSC to '1'	rcc_lsecss_it
HSECSSF	HSE Clock security system failure	_(²)	Set HSECSSC to '1'	rcc_hsecss_it

1. The security system feature must also be enabled (LSECSSON = '1'), in order to generate interrupts.

2. It is not possible to mask this interrupt when the security system feature is enabled (HSECSSON = '1').

9.7 RCC registers

9.7.1 Register mapping overview

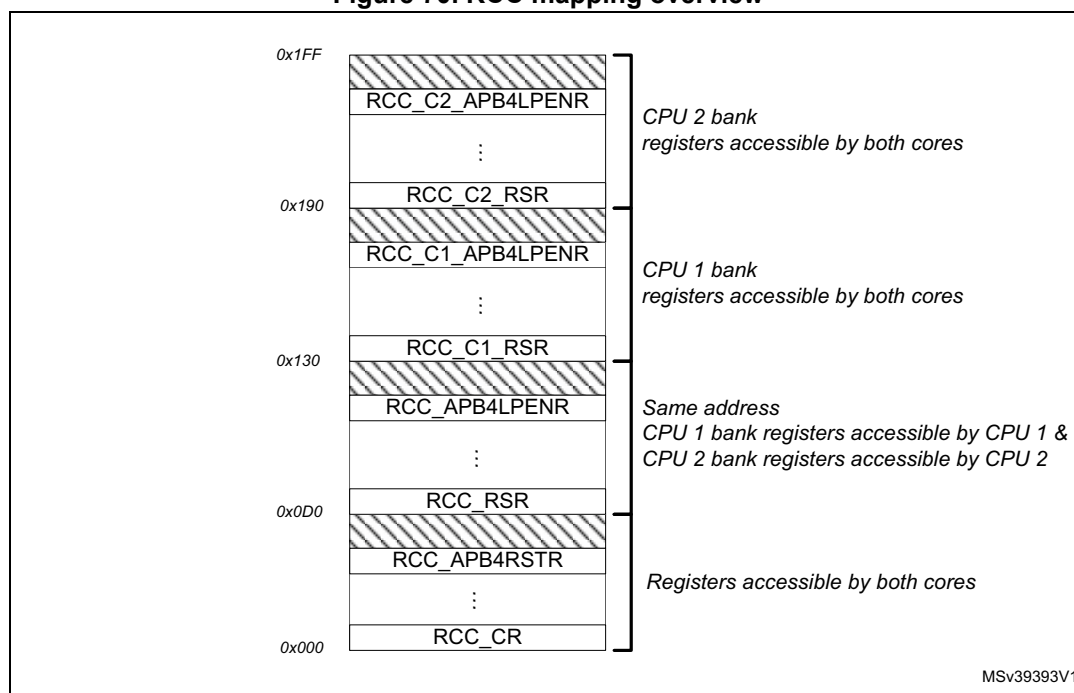
The RCC register map is divided into 4 sections.

- Common registers accessible by both cores and by any other master.
- Registers mapped at the same address:
 - When accessed by CPU1, CPU1 register bank is accessed.
 - When accessed by CPU2, CPU2 register bank is accessed.
- CPU1 register bank containing the peripheral enable bits (PERxEN), the peripheral low-power enable bits (PERxLPEN) and reset flag status bits for peripheral allocation to CPU1
- CPU2 register bank containing the peripheral enable bits (PERxEN), the peripheral low-power enable bits (PERxLPEN) and reset flag status bits for peripheral allocation to CPU2.

Note: Any master can access the Common register bank, CPU1 register bank and CPU2 register bank. However, the “same address bank” can only be accessed by CPU1 or CPU2.

Figure 70 shows the RCC mapping overview.

Figure 70. RCC mapping overview



9.7.2 RCC source control register (RCC_CR)

Address offset: 0x000

Reset value: 0x0000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	PLL3RDY	PLL3ON	PLL2RDY	PLL2ON	PLL1RDY	PLL1ON	Res.	Res.	Res.	Res.	HSECSSON	HSEBYP	HSERDY	HSEON
		r	rw	r	rw	r	rw					rs	rw	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
D2CKRDY	D1CKRDY	HSI48RDY	HSI48ON	Res.	Res.	CSIKERON	CSIRDY	CSION	Res.	HSIDIVF	HSIDIV[1:0]	HSIRDY	HSIKERON	HSION	
r	r	r	rw			rw	r	rw		r	rw	rw	r	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **PLL3RDY**: PLL3 clock ready flag

Set by hardware to indicate that the PLL3 is locked.

0: PLL3 unlocked (default after reset)

1: PLL3 locked

Bit 28 **PLL3ON**: PLL3 enable

Set and cleared by software to enable PLL3.

Cleared by hardware when entering Stop or Standby mode.

0: PLL3 OFF (default after reset)

1: PLL3 ON

Bit 27 **PLL2RDY**: PLL2 clock ready flag

Set by hardware to indicate that the PLL2 is locked.

0: PLL2 unlocked (default after reset)

1: PLL2 locked

Bit 26 **PLL2ON**: PLL2 enable

Set and cleared by software to enable PLL2.

Cleared by hardware when entering Stop or Standby mode.

0: PLL2 OFF (default after reset)

1: PLL2 ON

Bit 25 **PLL1RDY**: PLL1 clock ready flag

Set by hardware to indicate that the PLL1 is locked.

0: PLL1 unlocked (default after reset)

1: PLL1 locked

Bit 24 **PLL1ON**: PLL1 enable

Set and cleared by software to enable PLL1.

Cleared by hardware when entering Stop or Standby mode. Note that the hardware prevents writing this bit to '0', if the PLL1 output is used as the system clock.

0: PLL1 OFF (default after reset)

1: PLL1 ON

Bits 23:20 Reserved, must be kept at reset value.

Bit 19 HSECSSON: HSE Clock Security System enable

Set by software to enable Clock Security System on HSE.

This bit is “set only” (disabled by a system reset or when the system enters in Standby mode).

When HSECSSON is set, the clock detector is enabled by hardware when the HSE is ready and disabled by hardware if an oscillator failure is detected.

0: Clock Security System on HSE OFF (Clock detector OFF) (default after reset)

1: Clock Security System on HSE ON (Clock detector ON if the HSE oscillator is stable, OFF if not).

Bit 18 HSEBYP: HSE clock bypass

Set and cleared by software to bypass the oscillator with an external clock. The external clock must be enabled with the HSEON bit, to be used by the device.

The HSEBYP bit can be written only if the HSE oscillator is disabled.

0: HSE oscillator not bypassed (default after reset)

1: HSE oscillator bypassed with an external clock

Bit 17 HSERDY: HSE clock ready flag

Set by hardware to indicate that the HSE oscillator is stable.

0: HSE clock is not ready (default after reset)

1: HSE clock is ready

Bit 16 HSEON: HSE clock enable

Set and cleared by software.

Cleared by hardware to stop the HSE when entering Stop or Standby mode.

This bit cannot be cleared if the HSE is used directly (via SW mux) as system clock or if the HSE is selected as reference clock for PLL1 with PLL1 enabled (PLL1ON bit set to ‘1’).

0: HSE is OFF (default after reset)

1: HSE is ON

Bit 15 D2CKRDY: D2 domain clocks ready flag

Set by hardware to indicate that the D2 domain clocks (CPU, bus and peripheral) are available.

0: D2 domain clocks are not available (default after reset)

1: D2 domain clocks are available

Bit 14 D1CKRDY: D1 domain clocks ready flag

Set by hardware to indicate that the D1 domain clocks (CPU, bus and peripheral) are available.

0: D1 domain clocks are not available (default after reset)

1: D1 domain clocks are available

Bit 13 HSI48RDY: HSI48 clock ready flag

Set by hardware to indicate that the HSI48 oscillator is stable.

0: HSI48 clock is not ready (default after reset)

1: HSI48 clock is ready

Bit 12 HSI48ON: HSI48 clock enable

Set by software and cleared by software or by the hardware when the system enters to Stop or Standby mode.

0: HSI48 is OFF (default after reset)

1: HSI48 is ON

Bits 11:10 Reserved, must be kept at reset value.

Bit 9 **CSIKERON**: CSI clock enable in Stop mode

Set and reset by software to force the CSI to ON, even in Stop mode, in order to be quickly available as kernel clock for some peripherals. This bit has no effect on the value of CSION.

0: no effect on CSI (default after reset)

1: CSI is forced to ON even in Stop mode

Bit 8 **CSIRDY**: CSI clock ready flag

Set by hardware to indicate that the CSI oscillator is stable. This bit is activated only if the RC is enabled by CSION (it is not activated if the CSI is enabled by CSIKERON or by a peripheral request).

0: CSI clock is not ready (default after reset)

1: CSI clock is ready

Bit 7 **CSION**: CSI clock enable

Set and reset by software to enable/disable CSI clock for system and/or peripheral.

Set by hardware to force the CSI to ON when the system leaves Stop mode, if STOPWUCK = '1' or STOPKERWUCK = '1'.

This bit cannot be cleared if the CSI is used directly (via SW mux) as system clock or if the CSI is selected as reference clock for PLL1 with PLL1 enabled (PLL1ON bit set to '1').

0: CSI is OFF (default after reset)

1: CSI is ON

Bit 6 Reserved, must be kept at reset value.

Bit 5 **HSIDIVF**: HSI divider flag

Set and reset by hardware.

As a write operation to HSIDIV has not an immediate effect on the frequency, this flag indicates the current status of the HSI divider. HSIDIVF will go immediately to '0' when HSIDIV value is changed, and will be set back to '1' when the output frequency matches the value programmed into HSIDIV.

0: new division ratio not yet propagated to **hsi(_ker)_ck** (default after reset)

1: **hsi(_ker)_ck** clock frequency reflects the new HSIDIV value

Bits 4:3 **HSIDIV[1:0]**: HSI clock divider

Set and reset by software.

These bits allow selecting a division ratio in order to configure the wanted HSI clock frequency. The HSIDIV cannot be changed if the HSI is selected as reference clock for at least one enabled PLL (PLLxON bit set to '1'). In that case, the new HSIDIV value is ignored.

00: Division by 1, **hsi(_ker)_ck** = 64 MHz (default after reset)

01: Division by 2, **hsi(_ker)_ck** = 32 MHz

10: Division by 4, **hsi(_ker)_ck** = 16 MHz

11: Division by 8, **hsi(_ker)_ck** = 8 MHz

Bit 2 **HSIRDY**: HSI clock ready flag

Set by hardware to indicate that the HSI oscillator is stable.

0: HSI clock is not ready (default after reset)

1: HSI clock is ready

Bit 1 **HSIKERON**: High Speed Internal clock enable in Stop mode

Set and reset by software to force the HSI to ON, even in Stop mode, in order to be quickly available as kernel clock for peripherals. This bit has no effect on the value of HSION.

0: no effect on HSI (default after reset)

1: HSI is forced to ON even in Stop mode

Bit 0 **HSION**: High Speed Internal clock enable

Set and cleared by software.

Set by hardware to force the HSI to ON when the product leaves Stop mode, if STOPWUCK = '0' or STOPKERWUCK = '0'.

Set by hardware to force the HSI to ON when the product leaves Standby mode or in case of a failure of the HSE which is used as the system clock source.

This bit cannot be cleared if the HSI is used directly (via SW mux) as system clock or if the HSI is selected as reference clock for PLL1 with PLL1 enabled (PLL1ON bit set to '1').

0: HSI is OFF

1: HSI is ON (default after reset)

9.7.3 RCC HSI configuration register (RCC_HSI CFGR)

Address offset: 0x004

Reset value: 0x4000 0xxx

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	HSITRIM[6:0]							Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw	rw	rw	rw	rw	rw	rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	HSICAL[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 Reserved, must be kept at reset value.

Bits 30:24 **HSITRIM[6:0]**: HSI clock trimming

Set by software to adjust calibration.

HSITRIM field is added to the engineering Option Bytes loaded during reset phase (FLASH_HSI_opt) in order to form the calibration trimming value.

HSICAL = HSITRIM + FLASH_HSI_opt.

Note: The reset value of the field is 0x40.

Bits 23:12 Reserved, must be kept at reset value.

Bits 11:0 **HSICAL[11:0]**: HSI clock calibration

Set by hardware by option byte loading during system reset **nreset**.

Adjusted by software through trimming bits HSITRIM.

This field represents the sum of engineering Option Byte calibration value and HSITRIM bits value.

9.7.4 RCC clock recovery RC register (RCC_CRRCR)

Address offset: 0x008

Reset value: 0x0000 0xxx

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	HSI48CAL[9:0]									
						r	r	r	r	r	r	r	r	r	r

- Bits 31:10 Reserved, must be kept at reset value.
- Bits 9:0 **HSI48CAL[9:0]**: Internal RC 48 MHz clock calibration
- Set by hardware by option byte loading during system reset **nreset**.
- Read-only.



9.7.5 RCC CSI configuration register (RCC_CSICFGR)

Address offset: 0x00C

Reset value: 0x2000 00xx

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	CSITRIM[5:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
		rw	rw	rw	rw	rw	rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	CSICAL[9:0]									
						r	r	r	r	r	r	r	r	r	r

Bits 31:30 Reserved, must be kept at reset value.

Bits 29:24 **CSITRIM[5:0]**: CSI clock trimming

Set by software to adjust calibration.

CSITRIM bitfield is added to the engineering option bytes loaded during the reset phase (FLASH_CSI_opt) in order to build the calibration trimming value.

CSICAL = CSITRIM + FLASH_CSI_opt.

Note: The reset value of this bitfield is 0x10.

Bits 23:8 Reserved, must be kept at reset value.

Bits 9:0 **CSICAL[9:0]**: CSI clock calibration

Set by hardware by option byte loading during system reset **nreset**.

Adjusted by software through trimming bits CSITRIM.

This bitfield represents the sum of the engineering option byte calibration value and CSITRIM value.

9.7.6 RCC clock configuration register (RCC_CFGR)

Address offset: 0x010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MCO2SEL[2:0]			MCO2PRE[3:0]				MCO1SEL[2:0]			MCO11PRE[3:0]				Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TIMPRE	HRTIMSEL	RTCPRE[5:0]						STOPKERWUICK	STOPWUICK	SWS[2:0]			SW[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:29 **MCO2SEL[2:0]**: Micro-controller clock output 2

Set and cleared by software. Clock source selection may generate glitches on MCO2.

It is highly recommended to configure these bits only after reset, before enabling the external oscillators and the PLLs.

000: System clock selected (**sys_ck**) (default after reset)

001: PLL2 oscillator clock selected (**pll2_p_ck**)

010: HSE clock selected (**hse_ck**)

011: PLL1 clock selected (**pll1_p_ck**)

100: CSI clock selected (**csi_ck**)

101: LSI clock selected (**lsi_ck**)

others: reserved

Bits 28:25 **MCO2PRE[3:0]**: MCO2 prescaler

Set and cleared by software to configure the prescaler of the MCO2. Modification of this prescaler may generate glitches on MCO2. It is highly recommended to change this prescaler only after reset, before enabling the external oscillators and the PLLs.

0000: prescaler disabled (default after reset)

0001: division by 1 (bypass)

0010: division by 2

0011: division by 3

0100: division by 4

...

1111: division by 15

Bits 24:22 **MCO1SEL[2:0]**: Micro-controller clock output 1

Set and cleared by software. Clock source selection may generate glitches on MCO1.

It is highly recommended to configure these bits only after reset, before enabling the external oscillators and the PLLs.

000: HSI clock selected (**hsi_ck**) (default after reset)

001: LSE oscillator clock selected (**lse_ck**)

010: HSE clock selected (**hse_ck**)

011: PLL1 clock selected (**pll1_q_ck**)

100: HSI48 clock selected (**hsi48_ck**)

others: reserved

Bits 21:18 **MCO1PRE[3:0]**: MCO1 prescaler

Set and cleared by software to configure the prescaler of the MCO1. Modification of this prescaler may generate glitches on MCO1. It is highly recommended to change this prescaler only after reset, before enabling the external oscillators and the PLLs.

0000: prescaler disabled (default after reset)

0001: division by 1 (bypass)

0010: division by 2

0011: division by 3

0100: division by 4

...

1111: division by 15

Bits 17:16 Reserved, must be kept at reset value.

Bit 15 **TIMPRE**: Timers clocks prescaler selection

This bit is set and reset by software to control the clock frequency of all the timers connected to APB1 and APB2 domains.

0: The Timers kernel clock is equal to **rcc_hclk1** if D2PPREx is corresponding to division by 1 or 2, else it is equal to $2 \times F_{\text{rcc_pclkx_d2}}$ (default after reset)

1: The Timers kernel clock is equal to **rcc_hclk1** if D2PPREx is corresponding to division by 1, 2 or 4, else it is equal to $4 \times F_{\text{rcc_pclkx_d2}}$

Please refer to [Table 59: Ratio between clock timer and pclk](#)

Bit 14 **HRTIMSEL**: High Resolution Timer clock prescaler selection

This bit is set and reset by software to control the clock frequency of high resolution the timer (HRTIM).

0: The HRTIM prescaler clock source is the same as other timers. (default after reset)

1: The HRTIM prescaler clock source is the CPU1 clock (**rcc_c1_ck**).

Bits 13:8 **RTCPRE[5:0]**: HSE division factor for RTC clock

Set and cleared by software to divide the HSE to generate a clock for RTC.

Caution: The software has to set these bits correctly to ensure that the clock supplied to the RTC is lower than 1 MHz. These bits must be configured if needed before selecting the RTC clock source.

000000: no clock (default after reset)

000001: no clock

000010: HSE/2

000011: HSE/3

000100: HSE/4

...

111110: HSE/62

111111: HSE/63

Bit 7 **STOPKERWUCK**: Kernel clock selection after a wake up from system Stop

Set and reset by software to select the Kernel wakeup clock from system Stop.

0: The HSI is selected as wake up clock from system Stop (default after reset)

1: The CSI is selected as wake up clock from system Stop

See [Section 9.5.7: Handling clock generators in Stop and Standby mode](#) for details.

Bit 6 **STOPWUCK**: System clock selection after a wake up from system Stop

Set and reset by software to select the system wakeup clock from system Stop.

The selected clock is also used as emergency clock for the Clock Security System on HSE.

0: The HSI is selected as wake up clock from system Stop (default after reset)

1: The CSI is selected as wake up clock from system Stop

See [Section 9.5.7: Handling clock generators in Stop and Standby mode](#) for details.

Caution: STOPWUCK must not be modified when the Clock Security System is enabled (by HSECSSON bit) and the system clock is HSE (SWS="10") or a switch on HSE is requested (SW="10").

Bits 5:3 **SWS[2:0]**: System clock switch status

Set and reset by hardware to indicate which clock source is used as system clock.

000: HSI used as system clock (**hsi_ck**) (default after reset)

001: CSI used as system clock (**csi_ck**)

010: HSE used as system clock (**hse_ck**)

011: PLL1 used as system clock (**pll1_p_ck**)

others: Reserved

Bits 2:0 **SW[2:0]**: System clock switch

Set and reset by software to select system clock source (**sys_ck**).

Set by hardware in order to:

- force the selection of the HSI or CSI (depending on STOPWUCK selection) when leaving a system Stop mode
- force the selection of the HSI in case of failure of the HSE when used directly or indirectly as system clock.

000: HSI selected as system clock (**hsi_ck**) (default after reset)

001: CSI selected as system clock (**csi_ck**)

010: HSE selected as system clock (**hse_ck**)

011: PLL1 selected as system clock (**pll1_p_ck**)

others: Reserved

9.7.7 RCC domain 1 clock configuration register (RCC_D1CFGR)

Address offset: 0x018

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	D1CPRE[3:0]				Res.	D1PPRE[2:0]			HPRE[3:0]			
				rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved, must be kept at reset value.

Bits 11:8 **D1CPRE[3:0]**: D1 domain Core prescaler

Set and reset by software to control D1 domain CPU clock division factor.

Changing this division ratio has an impact on the frequency of CPU1 clock, all bus matrix clocks and CPU2 clock.

The clocks are divided by the new prescaler factor. This factor ranges from 1 to 16 periods of the slowest APB clock among **rcc_pclk[4:1]** after D1CPRE update. The application can check if the new division factor is taken into account by reading back this register.

0xxx: **sys_ck** not divided (default after reset)

1000: **sys_ck** divided by 2

1001: **sys_ck** divided by 4

1010: **sys_ck** divided by 8

1011: **sys_ck** divided by 16

1100: **sys_ck** divided by 64

1101: **sys_ck** divided by 128

1110: **sys_ck** divided by 256

1111: **sys_ck** divided by 512

Bit 7 Reserved, must be kept at reset value.

Bits 6:4 **D1PPRE[2:0]**: D1 domain APB3 prescaler

Set and reset by software to control the division factor of **rcc_pclk3**.

The clock is divided by the new prescaler factor from 1 to 16 cycles of **rcc_hclk3** after D1PPRE write.

0xx: **rcc_pclk3** = **rcc_hclk3** (default after reset)

100: **rcc_pclk3** = **rcc_hclk3** / 2

101: **rcc_pclk3** = **rcc_hclk3** / 4

110: **rcc_pclk3** = **rcc_hclk3** / 8

111: **rcc_pclk3** = **rcc_hclk3** / 16

Bits 3:0 **HPRE[3:0]**: D1 domain AHB prescaler

Set and reset by software to control the division factor of **rcc_hclk3** and **rcc_aclk**. Changing this division ratio has an impact on the frequency of all bus matrix clocks and CPU2 clock.

0xxx: **rcc_hclk3** = **sys_d1cpre_ck** (default after reset)

1000: **rcc_hclk3** = **sys_d1cpre_ck** / 2

1001: **rcc_hclk3** = **sys_d1cpre_ck** / 4

1010: **rcc_hclk3** = **sys_d1cpre_ck** / 8

1011: **rcc_hclk3** = **sys_d1cpre_ck** / 16

1100: **rcc_hclk3** = **sys_d1cpre_ck** / 64

1101: **rcc_hclk3** = **sys_d1cpre_ck** / 128

1110: **rcc_hclk3** = **sys_d1cpre_ck** / 256

1111: **rcc_hclk3** = **sys_d1cpre_ck** / 512

*Note: The clocks are divided by the new prescaler factor from 1 to 16 periods of the slowest APB clock among **rcc_pclk[4:1]** after HPRE update.*

*Note: Note also that **rcc_hclk3** = **rcc_aclk**.*

Caution: Care must be taken when using the voltage scaling. Due to the propagation delay of the new division factor, after a prescaler factor change and before lowering the V_{CORE} voltage, this register must be read in order to check that the new prescaler value has been taken into account.

Depending on the clock source frequency and the voltage range, the software application has to program a correct value in HPRE to make sure that the system frequency does not exceed the maximum frequency.

9.7.8 RCC domain 2 clock configuration register (RCC_D2CFGR)

Address offset: 0x01C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	D2PPRE2[2:0]			Res.	D2PPRE1[2:0]			Res.	Res.	Res.	Res.
					rw	rw	rw		rw	rw	rw				

Bits 31:11 Reserved, must be kept at reset value.

Bits 10:8 **D2PPRE2[2:0]**: D2 domain APB2 prescaler

Set and reset by software to control D2 domain APB2 clock division factor.

The clock is divided by the new prescaler factor from 1 to 16 cycles of **rcc_hclk1** after D2PPRE2 write.

0xx: **rcc_pclk2** = **rcc_hclk1** (default after reset)

100: **rcc_pclk2** = **rcc_hclk1** / 2

101: **rcc_pclk2** = **rcc_hclk1** / 4

110: **rcc_pclk2** = **rcc_hclk1** / 8

111: **rcc_pclk2** = **rcc_hclk1** / 16

Bit 7 Reserved, must be kept at reset value.

Bits 6:4 **D2PPRE1[2:0]**: D2 domain APB1 prescaler

Set and reset by software to control D2 domain APB1 clock division factor.

The clock is divided by the new prescaler factor from 1 to 16 cycles of **rcc_hclk1** after D2PPRE1 write.

0xx: **rcc_pclk1** = **rcc_hclk1** (default after reset)

100: **rcc_pclk1** = **rcc_hclk1** / 2

101: **rcc_pclk1** = **rcc_hclk1** / 4

110: **rcc_pclk1** = **rcc_hclk1** / 8

111: **rcc_pclk1** = **rcc_hclk1** / 16

Bits 3:0 Reserved, must be kept at reset value.

9.7.9 RCC domain 3 clock configuration register (RCC_D3CFGR)

Address offset: 0x020

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D3PPRE[2:0]			Res.			
									rw	rw	rw				

Bits 31:7 Reserved, must be kept at reset value.

Bits 6:4 **D3PPRE[2:0]**: D3 domain APB4 prescaler

Set and reset by software to control D3 domain APB4 clock division factor.

The clock is divided by the new prescaler factor from 1 to 16 cycles of **rcc_hclk4** after D3PPRE write.

0xx: **rcc_pclk4** = **rcc_hclk4** (default after reset)

100: **rcc_pclk4** = **rcc_hclk4** / 2

101: **rcc_pclk4** = **rcc_hclk4** / 4

110: **rcc_pclk4** = **rcc_hclk4** / 8

111: **rcc_pclk4** = **rcc_hclk4** / 16

Bits 3:0 Reserved, must be kept at reset value.

9.7.10 RCC PLLs clock source selection register (RCC_PLLCKSELR)

Address offset: 0x028

Reset value: 0x0202 0200

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	DIVM3[5:0]						Res.	Res.	DIVM2[5:4]	
						rw	rw	rw	rw	rw	rw			rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIVM2[3:0]				Res.	Res.	DIVM1[5:0]						Res.	Res.	PLLSRC[1:0]	
rw	rw	rw	rw			rw	rw	rw	rw	rw	rw			rw	rw

Bits 31:26 Reserved, must be kept at reset value.

Bits 25:20 **DIVM3[5:0]**: Prescaler for PLL3

Set and cleared by software to configure the prescaler of the PLL3.

The hardware does not allow any modification of this prescaler when PLL3 is enabled (PLL3ON = '1').

In order to save power when PLL3 is not used, the value of DIVM3 must be set to '0'.

000000: prescaler disabled

000001: division by 1 (bypass)

000010: division by 2

000011: division by 3

...

100000: division by 32 (default after reset)

...

111111: division by 63

Bits 19:18 Reserved, must be kept at reset value.

Bits 17:12 **DIVM2[5:0]**: Prescaler for PLL2

Set and cleared by software to configure the prescaler of the PLL2.

The hardware does not allow any modification of this prescaler when PLL2 is enabled (PLL2ON = '1').

In order to save power when PLL2 is not used, the value of DIVM2 must be set to '0'.

000000: prescaler disabled

000001: division by 1 (bypass)

000010: division by 2

000011: division by 3

...

100000: division by 32 (default after reset)

...

111111: division by 63

Bits 11:10 Reserved, must be kept at reset value.

Bits 9:4 DIVM1[5:0]: Prescaler for PLL1

Set and cleared by software to configure the prescaler of the PLL1.

The hardware does not allow any modification of this prescaler when PLL1 is enabled (PLL1ON = '1').

In order to save power when PLL1 is not used, the value of DIVM1 must be set to '0'.

000000: prescaler disabled

000001: division by 1 (bypass)

000010: division by 2

000011: division by 3

...

100000: division by 32 (default after reset)

...

111111: division by 63

Bits 3:2 Reserved, must be kept at reset value.

Bits 1:0 PLLSRC[1:0]: DIVMx and PLLs clock source selection

Set and reset by software to select the PLL clock source.

These bits can be written only when all PLLs are disabled.

In order to save power, when no PLL is used, the value of PLLSRC must be set to '11'.

00: HSI selected as PLL clock (**hsi_ck**) (default after reset)

01: CSI selected as PLL clock (**csi_ck**)

10: HSE selected as PLL clock (**hse_ck**)

11: No clock send to DIVMx divider and PLLs

9.7.11 RCC PLL configuration register (RCC_PLLCFGR)

Address offset: 0x02C

Reset value: 0x01FF 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIVR3EN	DIVQ3EN	DIVP3EN	DIVR2EN	DIVQ2EN	DIVP2EN	DIVR1EN	DIVQ1EN	DIVP1EN
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PLL3RGE[[1:0]]		PLL3VCOSEL	PLL3FRACEN	PLL2RGE[[1:0]]		PLL2VCOSEL	PLL2FRACEN	PLL1RGE[[1:0]]		PLL1VCOSEL	PLL1FRACEN
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **DIVR3EN**: PLL3 DIVR divider output enable

Set and reset by software to enable the **pll3_r_ck** output of the PLL3.

To save power, DIVR3EN and DIVR3 bits must be set to '0' when the **pll3_r_ck** is not used.

This bit can be written only when the PLL3 is disabled (PLL3ON = '0' and PLL3RDY = '0').

0: **pll3_r_ck** output is disabled

1: **pll3_r_ck** output is enabled (default after reset)

Bit 23 **DIVQ3EN**: PLL3 DIVQ divider output enable

Set and reset by software to enable the **pll3_q_ck** output of the PLL3.

To save power, DIVR3EN and DIVR3 bits must be set to '0' when the **pll3_r_ck** is not used.

This bit can be written only when the PLL3 is disabled (PLL3ON = '0' and PLL3RDY = '0').

0: **pll3_q_ck** output is disabled

1: **pll3_q_ck** output is enabled (default after reset)

Bit 22 **DIVP3EN**: PLL3 DIVP divider output enable

Set and reset by software to enable the **pll3_p_ck** output of the PLL3.

This bit can be written only when the PLL3 is disabled (PLL3ON = '0' and PLL3RDY = '0').

To save power, DIVR3EN and DIVR3 bits must be set to '0' when the **pll3_r_ck** is not used.

0: **pll3_p_ck** output is disabled

1: **pll3_p_ck** output is enabled (default after reset)

Bit 21 **DIVR2EN**: PLL2 DIVR divider output enable

Set and reset by software to enable the **pll2_r_ck** output of the PLL2.

To save power, DIVR3EN and DIVR3 bits must be set to '0' when the **pll3_r_ck** is not used.

This bit can be written only when the PLL2 is disabled (PLL2ON = '0' and PLL2RDY = '0').

0: **pll2_r_ck** output is disabled

1: **pll2_r_ck** output is enabled (default after reset)

Bit 20 **DIVQ2EN**: PLL2 DIVQ divider output enable

Set and reset by software to enable the **pll2_q_ck** output of the PLL2.

To save power, DIVR3EN and DIVR3 bits must be set to '0' when the **pll3_r_ck** is not used.

This bit can be written only when the PLL2 is disabled (PLL2ON = '0' and PLL2RDY = '0').

0: **pll2_q_ck** output is disabled

1: **pll2_q_ck** output is enabled (default after reset)

Bit 19 **DIVP2EN**: PLL2 DIVP divider output enable

Set and reset by software to enable the **pll2_p_ck** output of the PLL2.

This bit can be written only when the PLL2 is disabled (PLL2ON = '0' and PLL2RDY = '0').

To save power, DIVR3EN and DIVR3 bits must be set to '0' when the **pll3_r_ck** is not used.

0: **pll2_p_ck** output is disabled

1: **pll2_p_ck** output is enabled (default after reset)

Bit 18 **DIVR1EN**: PLL1 DIVR divider output enable

Set and reset by software to enable the **pll1_r_ck** output of the PLL1.

To save power, DIVR3EN and DIVR3 bits must be set to '0' when the **pll3_r_ck** is not used.

This bit can be written only when the PLL1 is disabled (PLL1ON = '0' and PLL1RDY = '0').

0: **pll1_r_ck** output is disabled

1: **pll1_r_ck** output is enabled (default after reset)

Bit 17 **DIVQ1EN**: PLL1 DIVQ divider output enable

Set and reset by software to enable the **pll1_q_ck** output of the PLL1.

In order to save power, when the **pll1_q_ck** output of the PLL1 is not used, the **pll1_q_ck** must be disabled.

This bit can be written only when the PLL1 is disabled (PLL1ON = '0' and PLL1RDY = '0').

0: **pll1_q_ck** output is disabled

1: **pll1_q_ck** output is enabled (default after reset)

Bit 16 **DIVP1EN**: PLL1 DIVP divider output enable

Set and reset by software to enable the **pll1_p_ck** output of the PLL1.

This bit can be written only when the PLL1 is disabled (PLL1ON = '0' and PLL1RDY = '0').

In order to save power, when the **pll1_p_ck** output of the PLL1 is not used, the **pll1_p_ck** must be disabled.

0: **pll1_p_ck** output is disabled

1: **pll1_p_ck** output is enabled (default after reset)

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:10 **PLL3RGE[1:0]**: PLL3 input frequency range

Set and reset by software to select the proper reference frequency range used for PLL3.

These bits must be written before enabling the PLL3.

00: The PLL3 input (**ref3_ck**) clock range frequency is between 1 and 2 MHz (default after reset)

01: The PLL3 input (**ref3_ck**) clock range frequency is between 2 and 4 MHz

10: The PLL3 input (**ref3_ck**) clock range frequency is between 4 and 8 MHz

11: The PLL3 input (**ref3_ck**) clock range frequency is between 8 and 16 MHz

Bit 9 **PLL3VCOSEL**: PLL3 VCO selection

Set and reset by software to select the proper VCO frequency range used for PLL3.

This bit must be written before enabling the PLL3.

0: Wide VCO range: 192 to 960 Hz (default after reset)

1: Medium VCO range: 150 to 420 MHz

Bit 8 **PLL3FRACEN**: PLL3 fractional latch enable

Set and reset by software to latch the content of FRACN3 into the Sigma-Delta modulator.

In order to latch the FRACN3 value into the Sigma-Delta modulator, PLL3FRACEN must be set to '0', then set to '1': the transition 0 to 1 transfers the content of FRACN3 into the modulator. Please refer to [Section : PLL initialization phase](#) for additional information.

Bits 7:6 PLL2RGE[1:0]: PLL2 input frequency range

Set and reset by software to select the proper reference frequency range used for PLL2.

These bits must be written before enabling the PLL2.

00: The PLL2 input (**ref2_ck**) clock range frequency is between 1 and 2 MHz (default after reset)

01: The PLL2 input (**ref2_ck**) clock range frequency is between 2 and 4 MHz

10: The PLL2 input (**ref2_ck**) clock range frequency is between 4 and 8 MHz

11: The PLL2 input (**ref2_ck**) clock range frequency is between 8 and 16 MHz

Bit 5 PLL2VCOSEL: PLL2 VCO selection

Set and reset by software to select the proper VCO frequency range used for PLL2.

This bit must be written before enabling the PLL2.

0: Wide VCO range: 192 to 960 MHz (default after reset)

1: Medium VCO range: 150 to 420 MHz

Bit 4 PLL2FRACEN: PLL2 fractional latch enable

Set and reset by software to latch the content of FRACN2 into the Sigma-Delta modulator.

In order to latch the FRACN2 value into the Sigma-Delta modulator, PLL2FRACEN must be set to '0', then set to '1': the transition 0 to 1 transfers the content of FRACN2 into the modulator. Please refer to [Section : PLL initialization phase](#) for additional information.

Bits 3:2 PLL1RGE[1:0]: PLL1 input frequency range

Set and reset by software to select the proper reference frequency range used for PLL1.

This bit must be written before enabling the PLL1.

00: The PLL1 input (**ref1_ck**) clock range frequency is between 1 and 2 MHz (default after reset)

01: The PLL1 input (**ref1_ck**) clock range frequency is between 2 and 4 MHz

10: The PLL1 input (**ref1_ck**) clock range frequency is between 4 and 8 MHz

11: The PLL1 input (**ref1_ck**) clock range frequency is between 8 and 16 MHz

Bit 1 PLL1VCOSEL: PLL1 VCO selection

Set and reset by software to select the proper VCO frequency range used for PLL1.

These bits must be written before enabling the PLL1.

0: Wide VCO range: 192 to 960 MHz (default after reset)

1: Medium VCO range: 150 to 420 MHz

Bit 0 PLL1FRACEN: PLL1 fractional latch enable

Set and reset by software to latch the content of FRACN1 into the Sigma-Delta modulator.

In order to latch the FRACN1 value into the Sigma-Delta modulator, PLL1FRACEN must be set to '0', then set to '1': the transition 0 to 1 transfers the content of FRACN1 into the modulator. Please refer to [Section : PLL initialization phase](#) for additional information.

9.7.12 RCC PLL1 dividers configuration register (RCC_PLL1DIVR)

Address offset: 0x030

Reset value: 0x0101 0280

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	DIVR1[6:0]							Res.	DIVQ1[6:0]						
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIVP1[6:0]								DIVN1[8:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bits 30:24 **DIVR1[6:0]**: PLL1 DIVR division factor

Set and reset by software to control the frequency of the **pll1_r_ck** clock.

These bits can be written only when the PLL1 is disabled (PLL1ON = '0' and PLL1RDY = '0').

0000000: not allowed

0000001: **pll1_r_ck** = **vco1_ck** / 2 (default after reset)

0000010: **pll1_r_ck** = **vco1_ck** / 3

0000011: **pll1_r_ck** = **vco1_ck** / 4

...

1111111: **pll1_r_ck** = **vco1_ck** / 128

Bit 23 Reserved, must be kept at reset value.

Bits 22:16 **DIVQ1[6:0]**: PLL1 DIVQ division factor

Set and reset by software to control the frequency of the **pll1_q_ck** clock.

These bits can be written only when the PLL1 is disabled (PLL1ON = '0' and PLL1RDY = '0').

0000000: **pll1_q_ck** = **vco1_ck**

0000001: **pll1_q_ck** = **vco1_ck** / 2 (default after reset)

0000010: **pll1_q_ck** = **vco1_ck** / 3

0000011: **pll1_q_ck** = **vco1_ck** / 4

...

1111111: **pll1_q_ck** = **vco1_ck** / 128

Bits 15:9 **DIVP1[6:0]**: PLL1 DIVP division factor

Set and reset by software to control the frequency of the **pll1_p_ck** clock.

These bits can be written only when the PLL1 is disabled (PLL1ON = '0' and PLL1RDY = '0').

Note that odd division factors are not allowed.

0000000: **pll1_p_ck** = **vco1_ck**

0000001: **pll1_p_ck** = **vco1_ck** / 2 (default after reset)

0000010: Not allowed

0000011: **pll1_p_ck** = **vco1_ck** / 4

...

1111111: **pll1_p_ck** = **vco1_ck** / 128

Bits 8:0 **DIVN1[8:0]**: Multiplication factor for PLL1 VCO

Set and reset by software to control the multiplication factor of the VCO.

These bits can be written only when the PLL is disabled (PLL1ON = '0' and PLL1RDY = '0').

0x003: DIVN1 = 4

0x004: DIVN1 = 5

0x005: DIVN1 = 6

...

0x080: DIVN1 = 129 (default after reset)

...

0x1FF: DIVN1 = 512

Others: wrong configurations

Caution: The software has to set correctly these bits to insure that the VCO output frequency is between its valid frequency range, which is:

- 192 to 960 MHz if PLL1VCOSEL = '0'
- 150 to 420 MHz if PLL1VCOSEL = '1'
-

VCO output frequency = $F_{\text{ref1_ck}} \times \text{DIVN1}$, when fractional value 0 has been loaded into FRACN1, with:

- DIVN1 between 4 and 512
- The input frequency $F_{\text{ref1_ck}}$ between 1MHz and 16 MHz

9.7.13 RCC PLL1 fractional divider register (RCC_PLL1FRACR)

Address offset: 0x034

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FRACN1[12:0]													Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw			

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:3 **FRACN1[12:0]**: Fractional part of the multiplication factor for PLL1 VCO

Set and reset by software to control the fractional part of the multiplication factor of the VCO.

These bits can be written at any time, allowing dynamic fine-tuning of the PLL1 VCO.

Caution: The software has to set correctly these bits to insure that the VCO output frequency is between its valid frequency range, which is:

- 192 to 960 MHz if PLL1VCOSEL = '0'
- 150 to 420 MHz if PLL1VCOSEL = '1'

VCO output frequency = $F_{\text{ref1_ck}} \times (\text{DIVN1} + (\text{FRACN1} / 2^{13}))$, with

- DIVN1 shall be between 4 and 512
- FRACN1 can be between 0 and $2^{13} - 1$
- The input frequency $F_{\text{ref1_ck}}$ shall be between 1 and 16 MHz.

To change the FRACN value on-the-fly even if the PLL is enabled, the application has to proceed as follow:

- set the bit PLL1FRACEN to '0',
- wait for 3 ref1_ck periods
- write the new fractional value into FRACN1,
- set the bit PLL1FRACEN to '1'.

where ref1_ck, ref2_ck, or ref3_ck are used depending on FRACNx bit index

Bits 2:0 Reserved, must be kept at reset value.

9.7.14 RCC PLL2 dividers configuration register (RCC_PLL2DIVR)

Address offset: 0x038

Reset value: 0x0101 0280

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	DIVR2[6:0]							Res.	DIVQ2[6:0]						
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIVP2[6:0]								DIVN2[8:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bits 30:24 **DIVR2[6:0]**: PLL2 DIVR division factor

Set and reset by software to control the frequency of the **pll2_r_ck** clock.

These bits can be written only when the PLL2 is disabled (PLL2ON = '0' and PLL2RDY = '0').

0000000: **pll2_r_ck** = **vco2_ck**

0000001: **pll2_r_ck** = **vco2_ck** / 2 (default after reset)

0000010: **pll2_r_ck** = **vco2_ck** / 3

0000011: **pll2_r_ck** = **vco2_ck** / 4

...

1111111: **pll2_r_ck** = **vco2_ck** / 128

Bit 23 Reserved, must be kept at reset value.

Bits 22:16 **DIVQ2[6:0]**: PLL2 DIVQ division factor

Set and reset by software to control the frequency of the **pll2_q_ck** clock.

These bits can be written only when the PLL2 is disabled (PLL2ON = '0' and PLL2RDY = '0').

0000000: **pll2_q_ck** = **vco2_ck**

0000001: **pll2_q_ck** = **vco2_ck** / 2 (default after reset)

0000010: **pll2_q_ck** = **vco2_ck** / 3

0000011: **pll2_q_ck** = **vco2_ck** / 4

...

1111111: **pll2_q_ck** = **vco2_ck** / 128

Bits 15:9 **DIVP2[6:0]**: PLL2 DIVP division factor

Set and reset by software to control the frequency of the **pll2_p_ck** clock.

These bits can be written only when the PLL2 is disabled (PLL2ON = '0' and PLL2RDY = '0').

0000000: **pll2_p_ck** = **vco2_ck**

0000001: **pll2_p_ck** = **vco2_ck** / 2 (default after reset)

0000010: **pll2_p_ck** = **vco2_ck** / 3

0000011: **pll2_p_ck** = **vco2_ck** / 4

...

1111111: **pll2_p_ck** = **vco2_ck** / 128

Bits 8:0 **DIVN2[8:0]**: Multiplication factor for PLL2 VCO

Set and reset by software to control the multiplication factor of the VCO.

These bits can be written only when the PLL is disabled (PLL2ON = '0' and PLL2RDY = '0').

Caution: The software has to set correctly these bits to insure that the VCO output frequency is between its valid frequency range, which is:

- 192 to 960 MHz if PLL2VCOSEL = '0'
- 150 to 420 MHz if PLL2VCOSEL = '1'

VCO output frequency = $F_{\text{ref2_ck}} \times \text{DIVN2}$, when fractional value 0 has been loaded into FRACN2, with

- DIVN2 between 4 and 512
- The input frequency $F_{\text{ref2_ck}}$ between 1MHz and 16MHz

0x003: DIVN2 = 4

0x004: DIVN2 = 5

0x005: DIVN2 = 6

...

0x080: DIVN2 = 129 (default after reset)

...

0x1FF: DIVN2 = 512

Others: wrong configurations

9.7.15 RCC PLL2 fractional divider register (RCC_PLL2FRACR)

Address offset: 0x03C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FRACN2[12:0]													Res.	Res.	Res.
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW			

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:3 **FRACN2[12:0]**: Fractional part of the multiplication factor for PLL2 VCO

Set and reset by software to control the fractional part of the multiplication factor of the VCO.

These bits can be written at any time, allowing dynamic fine-tuning of the PLL2 VCO.

Caution: The software has to set correctly these bits to insure that the VCO output frequency is between its valid frequency range, which is:

- 192 to 960 MHz if PLL2VCOSEL = '0'
- 150 to 420 MHz if PLL2VCOSEL = '1'

VCO output frequency = $F_{\text{ref2_ck}} \times (\text{DIVN2} + (\text{FRACN2} / 2^{13}))$, with

- DIVN2 shall be between 4 and 512
- FRACN2 can be between 0 and $2^{13} - 1$
- The input frequency $F_{\text{ref2_ck}}$ shall be between 1 and 16 MHz

In order to change the FRACN value on-the-fly even if the PLL is enabled, the application has to proceed as follow:

- set the bit PLL2FRACEN to '0',
- write the new fractional value into FRACN2,
- set the bit PLL2FRACEN to '1'.

Bits 2:0 Reserved, must be kept at reset value.

9.7.16 RCC PLL3 dividers configuration register (RCC_PLL3DIVR)

Address offset: 0x040

Reset value: 0x0101 0280

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	DIVR3[6:0]							Res.	DIVQ3[6:0]						
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIVP3[6:0]								DIVN3[8:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bits 30:24 **DIVR3[6:0]**: PLL3 DIVR division factor

Set and reset by software to control the frequency of the **pll3_r_ck** clock.

These bits can be written only when the PLL3 is disabled (PLL3ON = '0' and PLL3RDY = '0').

0000000: **pll3_r_ck** = **vco3_ck**

0000001: **pll3_r_ck** = **vco3_ck** / 2 (default after reset)

0000010: **pll3_r_ck** = **vco3_ck** / 3

0000011: **pll3_r_ck** = **vco3_ck** / 4

...

1111111: **pll3_r_ck** = **vco3_ck** / 128

Bit 23 Reserved, must be kept at reset value.

Bits 22:16 **DIVQ3[6:0]**: PLL3 DIVQ division factor

Set and reset by software to control the frequency of the **pll3_q_ck** clock.

These bits can be written only when the PLL3 is disabled (PLL3ON = '0' and PLL3RDY = '0').

0000000: **pll3_q_ck** = **vco3_ck**

0000001: **pll3_q_ck** = **vco3_ck** / 2 (default after reset)

0000010: **pll3_q_ck** = **vco3_ck** / 3

0000011: **pll3_q_ck** = **vco3_ck** / 4

...

1111111: **pll3_q_ck** = **vco3_ck** / 128

Bits 15:9 **DIVP3[6:0]**: PLL3 DIVP division factor

Set and reset by software to control the frequency of the **pll3_p_ck** clock.

These bits can be written only when the PLL3 is disabled (PLL3ON = '0' and PLL3RDY = '0').

0000000: **pll3_p_ck** = **vco3_ck**

0000001: **pll3_p_ck** = **vco3_ck** / 2 (default after reset)

0000010: **pll3_p_ck** = **vco3_ck** / 3

0000011: **pll3_p_ck** = **vco3_ck** / 4

...

1111111: **pll3_p_ck** = **vco3_ck** / 128

Bits 8:0 **DIVN3[7:0]**: Multiplication factor for PLL3 VCO

Set and reset by software to control the multiplication factor of the VCO.

These bits can be written only when the PLL is disabled (PLL3ON = '0' and PLL3RDY = '0').

Caution: The software has to set correctly these bits to insure that the VCO output frequency is between its valid frequency range, which is:

- 192 to 960 MHz if PLL3VCOSEL = '0'
- 150 to 420 MHz if PLL3VCOSEL = '1'

VCO output frequency = $F_{\text{ref3_ck}} \times \text{DIVN3}$, when fractional value 0 has been loaded into FRACN3, with

- DIVN3 between 4 and 512
- The input frequency $F_{\text{ref3_ck}}$ between 1MHz and 16MHz

0x003: DIVN3 = 4

0x004: DIVN3 = 5

0x005: DIVN3 = 6

...

0x080: DIVN3 = 129 (default after reset)

...

0x1FF: DIVN3 = 512

Others: wrong configurations

9.7.17 RCC PLL3 fractional divider register (RCC_PLL3FRACR)

Address offset: 0x044

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FRACN3[12:0]													Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw			

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:3 **FRACN3[12:0]**: Fractional part of the multiplication factor for PLL3 VCO

Set and reset by software to control the fractional part of the multiplication factor of the VCO.

These bits can be written at any time, allowing dynamic fine-tuning of the PLL3 VCO.

Caution: The software has to set correctly these bits to insure that the VCO output frequency is between its valid frequency range, which is:

- 192 to 960 MHz if PLL3VCOSEL = '0'
- 150 to 420 MHz if PLL3VCOSEL = '1'

VCO output frequency = $F_{\text{ref3_ck}} \times (\text{DIVN3} + (\text{FRACN3} / 2^{13}))$, with

- DIVN3 shall be between 4 and 512
- FRACN3 can be between 0 and $2^{13} - 1$
- The input frequency $F_{\text{ref3_ck}}$ shall be between 1 and 16 MHz

In order to change the FRACN value on-the-fly even if the PLL is enabled, the application has to proceed as follow:

- set the bit PLL1FRACEN to '0',
- write the new fractional value into FRACN1,
- set the bit PLL1FRACEN to '1'.

Bits 2:0 Reserved, must be kept at reset value.

9.7.18 RCC domain 1 kernel clock configuration register (RCC_D1CCIPR)

Address offset: 0x04C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	CKPERSEL[1:0] ⁽¹⁾		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMCSEL ⁽¹⁾
		rw													rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	DSISEL ⁽¹⁾	Res.	Res.	QSPISEL[1:0] ⁽¹⁾		Res.	Res.	FMCSEL[1:0] ⁽¹⁾	
							rw			rw				rw	rw

1. Changing the clock source on-the-fly is allowed and will not generate any timing violation. However the user has to make use that both the previous and the new clock sources are present during the switching, and during the whole transition time. Please refer to [Section : Clock switches and gating](#).

Bits 31:30 Reserved, must be kept at reset value.

Bits 29:28 **CKPERSEL[1:0]**: **per_ck** clock source selection

00: **hsi_ker_ck** clock selected as **per_ck** clock (default after reset)

01: **csi_ker_ck** clock selected as **per_ck** clock

10: **hse_ck** clock selected as **per_ck** clock

11: reserved, the **per_ck** clock is disabled

Bits 27:17 Reserved, must be kept at reset value.

Bit 16 **SDMMCSEL**: SDMMC kernel clock source selection

0: **pll1_q_ck** clock is selected as kernel peripheral clock (default after reset)

1: **pll2_r_ck** clock is selected as kernel peripheral clock

Bits 15:9 Reserved, must be kept at reset value.

Bit 8 **DSISEL**: DSI kernel clock source selection

0: DSI clock from PHY is selected as DSI byte lane clock (default after reset)

1: **pll2_q_ck** clock is selected as DSI byte lane clock

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:4 **QSPISEL[1:0]**: QUADSPI kernel clock source selection

00: **rcc_hclk3** clock selected as kernel peripheral clock (default after reset)

01: **pll1_q_ck** clock selected as kernel peripheral clock

10: **pll2_r_ck** clock selected as kernel peripheral clock

11: **per_ck** clock selected as kernel peripheral clock

Bits 3:2 Reserved, must be kept at reset value.

Bits 1:0 **FMCSEL[1:0]**: FMC kernel clock source selection

00: **rcc_hclk3** clock selected as kernel peripheral clock (default after reset)

01: **pll1_q_ck** clock selected as kernel peripheral clock

10: **pll2_r_ck** clock selected as kernel peripheral clock

11: **per_ck** clock selected as kernel peripheral clock

9.7.19 RCC domain 2 kernel clock configuration register (RCC_D2CCIP1R)

Address offset: 0x050

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWPSEL ⁽¹⁾	Res.	FDCANSEL[1:0] ⁽¹⁾		Res.	Res.	Res.	DFSDM1SEL ⁽¹⁾	Res.	Res.	SPDIFSEL[1:0] ⁽¹⁾		Res.	SPI45SEL[2:0] ⁽¹⁾		
rw		rw	rw				rw			rw	rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	SPI123SEL[2:0] ⁽¹⁾			Res.	Res.	Res.	SAI23SEL[2:0] ⁽¹⁾			Res.	Res.	Res.	SAI1SEL[2:0] ⁽¹⁾		
	rw	rw	rw				rw	rw	rw				rw	rw	rw

1. Changing the clock source on-the-fly is allowed and will not generate any timing violation. However the user has to make sure that both the previous and the new clock sources are present during the switching, and for the whole transition time. Please refer to [Section : Clock switches and gating](#).

Bit 31 **SWPSEL**: SWPMI kernel clock source selection

Set and reset by software.

0: **pclk** is selected as SWPMI kernel clock (default after reset)

1: **hsi_ker_ck** clock is selected as SWPMI kernel clock

Bit 30 Reserved, must be kept at reset value.

Bits 29:28 **FDCANSEL**: FDCAN kernel clock source selection

Set and reset by software.

00: **hse_ck** clock is selected as FDCAN kernel clock (default after reset)

01: **pll1_q_ck** clock is selected as FDCAN kernel clock

10: **pll2_q_ck** clock is selected as FDCAN kernel clock

11: reserved, the kernel clock is disabled

Bits 27:25 Reserved, must be kept at reset value.

Bit 24 **DFSDM1SEL**: DFSDM1 kernel **Clk** clock source selection

Set and reset by software.

Note: the DFSDM1 Aclk Clock Source Selection is done by SAI1SEL.

0: **rcc_pclk2** is selected as DFSDM1 Clk kernel clock (default after reset)

1: **sys_ck** clock is selected as DFSDM1 Clk kernel clock

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:20 **SPDIFSEL[1:0]**: SPDIFRX kernel clock source selection

00: **pll1_q_ck** clock selected as SPDIFRX kernel clock (default after reset)

01: **pll2_r_ck** clock selected as SPDIFRX kernel clock

10: **pll3_r_ck** clock selected as SPDIFRX kernel clock

11: **hsi_ker_ck** clock selected as SPDIFRX kernel clock

Bit 19 Reserved, must be kept at reset value.

Bits 18:16 **SPI45SEL[2:0]**: SPI4 and 5 kernel clock source selection

Set and reset by software.

000: APB clock is selected as kernel clock (default after reset)

001: **pll2_q_ck** clock is selected as kernel clock

010: **pll3_q_ck** clock is selected as kernel clock

011: **hsi_ker_ck** clock is selected as kernel clock

100: **csi_ker_ck** clock is selected as kernel clock

101: **hse_ck** clock is selected as kernel clock

others: reserved, the kernel clock is disabled

Bit 15 Reserved, must be kept at reset value.

Bits 14:12 **SPI123SEL[2:0]**: SPI/I2S1,2 and 3 kernel clock source selection

Set and reset by software.

Caution: If the selected clock is the external clock and this clock is stopped, it will not be possible to switch to another clock. Refer to [Section : Clock switches and gating](#) for additional information.

000: **pll1_q_ck** clock selected as SPI/I2S1,2 and 3 kernel clock (default after reset)

001: **pll2_p_ck** clock selected as SPI/I2S1,2 and 3 kernel clock

010: **pll3_p_ck** clock selected as SPI/I2S1,2 and 3 kernel clock

011: I2S_CKIN clock selected as SPI/I2S1,2 and 3 kernel clock

100: **per_ck** clock selected as SPI/I2S1,2 and 3 kernel clock

others: reserved, the kernel clock is disabled

Note: I2S_CKIN is an external clock taken from a pin.

Bits 11:9 Reserved, must be kept at reset value.

Bits 8:6 **SAI23SEL[2:0]**: SAI2 and SAI3 kernel clock source selection

Set and reset by software.

Caution: If the selected clock is the external clock and this clock is stopped, it will not be possible to switch to another clock. Refer to [Section : Clock switches and gating](#) for additional information.

000: **pll1_q_ck** clock selected as SAI2 and SAI3 kernel clock (default after reset)

001: **pll2_p_ck** clock selected as SAI2 and SAI3 kernel clock

010: **pll3_p_ck** clock selected as SAI2 and SAI3 kernel clock

011: **I2S_CKIN** clock selected as SAI2 and SAI3 kernel clock

100: **per_ck** clock selected as SAI2 and SAI3 kernel clock

others: reserved, the kernel clock is disabled

Note: I2S_CKIN is an external clock taken from a pin.

Bits 5:3 Reserved, must be kept at reset value.

Bits 2:0 **SAI1SEL[2:0]**: SAI1 and DFSDM1 kernel **Aclk** clock source selection

Set and reset by software.

Caution: If the selected clock is the external clock and this clock is stopped, it will not be possible to switch to another clock. Refer to [Section : Clock switches and gating](#) for additional information.

Note: DFSDM1 Clock Source Selection is done by DFSDM1SEL.

000: **pll1_q_ck** clock selected as SAI1 and DFSDM1 **Aclk** kernel clock (default after reset)

001: **pll2_p_ck** clock selected as SAI1 and DFSDM1 **Aclk** kernel clock

010: **pll3_p_ck** clock selected as SAI1 and DFSDM1 **Aclk** kernel clock

011: **I2S_CKIN** clock selected as SAI1 and DFSDM1 **Aclk** kernel clock

100: **per_ck** clock selected as SAI1 and DFSDM1 **Aclk** kernel clock

others: reserved, the kernel clock is disabled

Note: I2S_CKIN is an external clock taken from a pin.

9.7.20 RCC domain 2 kernel clock configuration register (RCC_D2CCIP2R)

Address offset: 0x054

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	LPTIM1SEL[2:0] ⁽¹⁾			Res.	Res.	Res.	Res.	CECSEL[1:0] ⁽¹⁾		USBSEL[1:0] ⁽¹⁾		Res.	Res.	Res.	Res.
	rw	rw	rw					rw	rw	rw	rw				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	I2C123SEL[1:0] ⁽¹⁾		Res.	Res.	RNGSEL[1:0] ⁽¹⁾		Res.	Res.	USART16SEL[2:0] ⁽¹⁾			USART234578SEL[2:0] ⁽¹⁾		
		rw	rw			rw	rw			rw	rw	rw	rw	rw	rw

1. Changing the clock source on-the-fly is allowed and will not generate any timing violation. However the user has to make sure that both the previous and the new clock sources are present during the switching, and for the whole transition time. Please refer to [Section : Clock switches and gating](#).

Bit 31 Reserved, must be kept at reset value.

Bits 30:28 **LPTIM1SEL[2:0]**: LPTIM1 kernel clock source selection

Set and reset by software.

000: **rcc_pclk1** clock selected as kernel peripheral clock (default after reset)

001: **pll2_p_ck** clock selected as kernel peripheral clock

010: **pll3_r_ck** clock selected as kernel peripheral clock

011: **lse_ck** clock selected as kernel peripheral clock

100: **lsi_ck** clock selected as kernel peripheral clock

101: **per_ck** clock selected as kernel peripheral clock

others: reserved, the kernel clock is disabled

Bits 27:24 Reserved, must be kept at reset value.

Bits 23:22 **CECSEL[1:0]**: HDMI-CEC kernel clock source selection

Set and reset by software.

00: **lse_ck** clock is selected as kernel clock (default after reset)

01: **lsi_ck** clock is selected as kernel clock

10: **csi_ker_ck** divided by 122 is selected as kernel clock

11: reserved, the kernel clock is disabled

Bits 21:20 **USBSEL[1:0]**: USBOTG 1 and 2 kernel clock source selection

Set and reset by software.

00: Disable the kernel clock (default after reset)

01: **pll1_q_ck** clock is selected as kernel clock

10: **pll3_q_ck** clock is selected as kernel clock

11: **hsi48_ck** clock is selected as kernel clock

Bits 19:14 Reserved, must be kept at reset value.

Bits 13:12 **I2C123SEL[1:0]**: I2C1,2,3 kernel clock source selection

Set and reset by software.

00: **rcc_pclk1** clock is selected as kernel clock (default after reset)

01: **pll3_r_ck** clock is selected as kernel clock

10: **hsi_ker_ck** clock is selected as kernel clock

11: **csi_ker_ck** clock is selected as kernel clock

Bits 11:10 Reserved, must be kept at reset value.

Bits 9:8 **RNGSEL[1:0]**: RNG kernel clock source selection

Set and reset by software.

00: **hsi48_ck** clock is selected as kernel clock (default after reset)

01: **pll1_q_ck** clock is selected as kernel clock

10: **lse_ck** clock is selected as kernel clock

11: **lsi_ck** clock is selected as kernel clock

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:3 **USART16SEL[2:0]**: USART1 and 6 kernel clock source selection

Set and reset by software.

000: **rcc_pclk2** clock is selected as kernel clock (default after reset)

001: **pll2_q_ck** clock is selected as kernel clock

010: **pll3_q_ck** clock is selected as kernel clock

011: **hsi_ker_ck** clock is selected as kernel clock

100: **csi_ker_ck** clock is selected as kernel clock

101: **lse_ck** clock is selected as kernel clock

others: reserved, the kernel clock is disabled

Bits 2:0 **USART234578SEL[2:0]**: USART2/3, UART4,5, 7/8 (APB1) kernel clock source selection

Set and reset by software.

000: **rcc_pclk1** clock is selected as kernel clock (default after reset)

001: **pll2_q_ck** clock is selected as kernel clock

010: **pll3_q_ck** clock is selected as kernel clock

011: **hsi_ker_ck** clock is selected as kernel clock

100: **csi_ker_ck** clock is selected as kernel clock

101: **lse_ck** clock is selected as kernel clock

others: reserved, the kernel clock is disabled

9.7.21 RCC domain 3 kernel clock configuration register (RCC_D3CCIPR)

Address offset: 0x058

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	SPI6SEL[2:0] ⁽¹⁾			Res.	SAI4BSEL[2:0] ⁽¹⁾			SAI4ASEL[2:0] ⁽¹⁾			Res.	Res.	Res.	ADCSEL[1:0] ⁽¹⁾	
	rw	rw	rw		rw	rw	rw	rw	rw	rw				rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPTIM345SEL[2:0] ⁽¹⁾			LPTIM2SEL[2:0] ⁽¹⁾			I2C4SEL[1:0] ⁽¹⁾		Res.	Res.	Res.	Res.	Res.	LPUART1SEL[2:0] ⁽¹⁾		
rw	rw	rw	rw	rw	rw	rw	rw						rw	rw	rw

1. Changing the clock source on-the-fly is allowed, and will not generate any timing violation. However the user has to make sure that both the previous and the new clock sources are present during the switching, and for the whole transition time. Please refer to [Section : Clock switches and gating](#).

Bit 31 Reserved, must be kept at reset value.

Bits 30:28 **SPI6SEL[2:0]**: SPI6 kernel clock source selection

Set and reset by software.

000: **rcc_pclk4** clock selected as kernel peripheral clock (default after reset)

001: **pll2_q_ck** clock selected as kernel peripheral clock

010: **pll3_q_ck** clock selected as kernel peripheral clock

011: **hsi_ker_ck** clock selected as kernel peripheral clock

100: **csi_ker_ck** clock selected as kernel peripheral clock

101: **hse_ck** clock selected as kernel peripheral clock

others: reserved, the kernel clock is disabled

Bit 27 Reserved, must be kept at reset value.

Bits 26:24 **SAI4BSEL[2:0]**: Sub-Block B of SAI4 kernel clock source selection

Set and reset by software.

Caution: If the selected clock is the external clock and this clock is stopped, it will not be possible to switch to another clock. Refer to [Section : Clock switches and gating](#) for additional information.

000: **pll1_q_ck** clock selected as kernel peripheral clock (default after reset)

001: **pll2_p_ck** clock selected as kernel peripheral clock

010: **pll3_p_ck** clock selected as kernel peripheral clock

011: **I2S_CKIN** clock selected as kernel peripheral clock

100: **per_ck** clock selected as kernel peripheral clock

others: reserved, the kernel clock is disabled

Note: I2S_CKIN is an external clock taken from a pin.

Bits 23:21 **SAI4ASEL[2:0]**: Sub-Block A of SAI4 kernel clock source selection

Set and reset by software.

Caution: If the selected clock is the external clock and this clock is stopped, it will not be possible to switch to another clock. Refer to [Section : Clock switches and gating](#) for additional information.

000: **pll1_q_ck** clock selected as kernel peripheral clock (default after reset)

001: **pll2_p_ck** clock selected as kernel peripheral clock

010: **pll3_p_ck** clock selected as kernel peripheral clock

011: **I2S_CKIN** clock selected as kernel peripheral clock

100: **per_ck** clock selected as kernel peripheral clock

others: reserved, the kernel clock is disabled

Note: I2S_CKIN is an external clock taken from a pin.

Bits 20:18 Reserved, must be kept at reset value.

Bits 17:16 **ADCSEL[1:0]**: SAR ADC kernel clock source selection

Set and reset by software.

00: **pll2_p_ck** clock selected as kernel peripheral clock (default after reset)

01: **pll3_r_ck** clock selected as kernel peripheral clock

10: **per_ck** clock selected as kernel peripheral clock

others: reserved, the kernel clock is disabled

Bits 15:13 **LPTIM345SEL[2:0]**: LPTIM3,4,5 kernel clock source selection

Set and reset by software.

000: **rcc_pclk4** clock selected as kernel peripheral clock (default after reset)

001: **pll2_p_ck** clock selected as kernel peripheral clock

010: **pll3_r_ck** clock selected as kernel peripheral clock

011: **lse_ck** clock selected as kernel peripheral clock

100: **lsi_ck** clock selected as kernel peripheral clock

101: **per_ck** clock selected as kernel peripheral clock

others: reserved, the kernel clock is disabled

Bits 12:10 **LPTIM2SEL[2:0]**: LPTIM2 kernel clock source selection

Set and reset by software.

000: **rcc_pclk4** clock selected as kernel peripheral clock (default after reset)

001: **pll2_p_ck** clock selected as kernel peripheral clock

010: **pll3_r_ck** clock selected as kernel peripheral clock

011: **lse_ck** clock selected as kernel peripheral clock

100: **lsi_ck** clock selected as kernel peripheral clock

101: **per_ck** clock selected as kernel peripheral clock

others: reserved, the kernel clock is disabled

Bits 9:8 **I2C4SEL[1:0]**: I2C4 kernel clock source selection

Set and reset by software.

00: **rcc_pclk4** clock selected as kernel peripheral clock (default after reset)

01: **pll3_r_ck** clock selected as kernel peripheral clock

10: **hsi_ker_ck** clock selected as kernel peripheral clock

11: **csi_ker_ck** clock selected as kernel peripheral clock

Bits 7:3 Reserved, must be kept at reset value.

Bits 2:0 **LPUART1SEL[2:0]**: LPUART1 kernel clock source selection

Set and reset by software.

000: **rcc_pclk_d3** clock is selected as kernel peripheral clock (default after reset)

001: **pll2_q_ck** clock is selected as kernel peripheral clock

010: **pll3_q_ck** clock is selected as kernel peripheral clock

011: **hsi_ker_ck** clock is selected as kernel peripheral clock

100: **csi_ker_ck** clock is selected as kernel peripheral clock

101: **lse_ck** clock is selected as kernel peripheral clock

others: reserved, the kernel clock is disabled

9.7.22 RCC clock source interrupt enable register (RCC_CIER)

Address offset: 0x060

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	LSECSSIE	PLL3RDYIE	PLL2RDYIE	PLL1RDYIE	HSI48RDYIE	CSIRDYIE	HSE RDYIE	HSIRDYIE	LSERDYIE	LSIRDYIE
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:10 Reserved, must be kept at reset value.

Bit 9 LSECSSIE: LSE clock security system Interrupt Enable

Set and reset by software to enable/disable interrupt caused by the Clock Security System on external 32 kHz oscillator.

0: LSE CSS interrupt disabled (default after reset)

1: LSE CSS interrupt enabled

Bit 8 PLL3RDYIE: PLL3 ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by PLL3 lock.

0: PLL3 lock interrupt disabled (default after reset)

1: PLL3 lock interrupt enabled

Bit 7 PLL2RDYIE: PLL2 ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by PLL2 lock.

0: PLL2 lock interrupt disabled (default after reset)

1: PLL2 lock interrupt enabled

Bit 6 PLL1RDYIE: PLL1 ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by PLL1 lock.

0: PLL1 lock interrupt disabled (default after reset)

1: PLL1 lock interrupt enabled

Bit 5 HSI48RDYIE: HSI48 ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by the HSI48 oscillator stabilization.

0: HSI48 ready interrupt disabled (default after reset)

1: HSI48 ready interrupt enabled

Bit 4 CSIRDYIE: CSI ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by the CSI oscillator stabilization.

0: CSI ready interrupt disabled (default after reset)

1: CSI ready interrupt enabled

Bit 3 HSE RDYIE: HSE ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by the HSE oscillator stabilization.

0: HSE ready interrupt disabled (default after reset)

1: HSE ready interrupt enabled

Bit 2 HSIRDYIE: HSI ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by the HSI oscillator stabilization.

0: HSI ready interrupt disabled (default after reset)

1: HSI ready interrupt enabled

Bit 1 LSERDYIE: LSE ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by the LSE oscillator stabilization.

0: LSE ready interrupt disabled (default after reset)

1: LSE ready interrupt enabled

Bit 0 LSIRDYIE: LSI ready Interrupt Enable

Set and reset by software to enable/disable interrupt caused by the LSI oscillator stabilization.

0: LSI ready interrupt disabled (default after reset)

1: LSI ready interrupt enabled

9.7.23 RCC clock source interrupt flag register (RCC_CIFR)

Address offset: 0x64

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	HSECSSF	LSECSSF	PLL3RDYF	PLL2RDYF	PLL1RDYF	HSI48RDYF	CSIRDYF	HSERDYF	HSIRDYF	LSERDYF	LSIRDYF
					r	r	r	r	r	r	r	r	r	r	r

Bits 31:11 Reserved, must be kept at reset value.

Bit 10 HSECSSF: HSE clock security system Interrupt Flag

Reset by software by writing HSECSSC bit.

Set by hardware in case of HSE clock failure.

0: No clock security interrupt caused by HSE clock failure (default after reset)

1: Clock security interrupt caused by HSE clock failure

Bit 9 LSECSSF: LSE clock security system Interrupt Flag

Reset by software by writing LSECSSC bit.

Set by hardware when a failure is detected on the external 32 kHz oscillator and LSECSSIE is set.

0: No failure detected on the external 32 kHz oscillator (default after reset)

1: A failure is detected on the external 32 kHz oscillator

Bit 8 PLL3RDYF: PLL3 ready Interrupt Flag

Reset by software by writing PLL3RDYC bit.

Set by hardware when the PLL3 locks and PLL3RDYIE is set.

0: No clock ready interrupt caused by PLL3 lock (default after reset)

1: Clock ready interrupt caused by PLL3 lock

Bit 7 PLL2RDYF: PLL2 ready Interrupt Flag

Reset by software by writing PLL2RDYC bit.

Set by hardware when the PLL2 locks and PLL2RDYIE is set.

0: No clock ready interrupt caused by PLL2 lock (default after reset)

1: Clock ready interrupt caused by PLL2 lock

Bit 6 PLL1RDYF: PLL1 ready Interrupt Flag

Reset by software by writing PLL1RDYC bit.

Set by hardware when the PLL1 locks and PLL1RDYIE is set.

0: No clock ready interrupt caused by PLL1 lock (default after reset)

1: Clock ready interrupt caused by PLL1 lock

Bit 5 HSI48RDYF: HSI48 ready Interrupt Flag

Reset by software by writing HSI48RDYC bit.

Set by hardware when the HSI48 clock becomes stable and HSI48RDYIE is set.

0: No clock ready interrupt caused by the HSI48 oscillator (default after reset)

1: Clock ready interrupt caused by the HSI48 oscillator

Bit 4 CSIRDYF: CSI ready Interrupt Flag

Reset by software by writing CSIRDYC bit.

Set by hardware when the CSI clock becomes stable and CSIRDYIE is set.

0: No clock ready interrupt caused by the CSI (default after reset)

1: Clock ready interrupt caused by the CSI

Bit 3 HSERDYF: HSE ready Interrupt Flag

Reset by software by writing HSERDYC bit.

Set by hardware when the HSE clock becomes stable and HSERDYIE is set.

0: No clock ready interrupt caused by the HSE (default after reset)

1: Clock ready interrupt caused by the HSE

Bit 2 HSIRDYF: HSI ready Interrupt Flag

Reset by software by writing HSIRDYC bit.

Set by hardware when the HSI clock becomes stable and HSIRDYIE is set.

0: No clock ready interrupt caused by the HSI (default after reset)

1: Clock ready interrupt caused by the HSI

Bit 1 LSERDYF: LSE ready Interrupt Flag

Reset by software by writing LSERDYC bit.

Set by hardware when the LSE clock becomes stable and LSERDYIE is set.

0: No clock ready interrupt caused by the LSE (default after reset)

1: Clock ready interrupt caused by the LSE

Bit 0 LSIRDYF: LSI ready Interrupt Flag

Reset by software by writing LSIRDYC bit.

Set by hardware when the LSI clock becomes stable and LSIRDYIE is set.

0: No clock ready interrupt caused by the LSI (default after reset)

1: Clock ready interrupt caused by the LSI

9.7.24 RCC clock source interrupt clear register (RCC_CICR)

Address offset: 0x68

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	HSECSSC	LSECSSC	PLL3RDYC	PLL2RDYC	PLL1RDYC	HSI48RDYC	CSIRDYC	HSERDYC	HSIRDYC	LSERDYC	LSIRDYC
					rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:11 Reserved, must be kept at reset value.

Bit 10 HSECSSC: HSE clock security system Interrupt Clear

Set by software to clear HSECSSF.

Reset by hardware when clear done.

0: HSECSSF no effect (default after reset)

1: HSECSSF cleared

Bit 9 LSECSSC: LSE clock security system Interrupt Clear

Set by software to clear LSECSSF.

Reset by hardware when clear done.

0: LSECSSF no effect (default after reset)

1: LSECSSF cleared

Bit 8 PLL3RDYC: PLL3 ready Interrupt Clear

Set by software to clear PLL3RDYF.

Reset by hardware when clear done.

0: PLL3RDYF no effect (default after reset)

1: PLL3RDYF cleared

Bit 7 PLL2RDYC: PLL2 ready Interrupt Clear

Set by software to clear PLL2RDYF.

Reset by hardware when clear done.

0: PLL2RDYF no effect (default after reset)

1: PLL2RDYF cleared

Bit 6 PLL1RDYC: PLL1 ready Interrupt Clear

Set by software to clear PLL1RDYF.

Reset by hardware when clear done.

0: PLL1RDYF no effect (default after reset)

1: PLL1RDYF cleared

Bit 5 HSI48RDYC: HSI48 ready Interrupt Clear

Set by software to clear HSI48RDYF.

Reset by hardware when clear done.

0: HSI48RDYF no effect (default after reset)

1: HSI48RDYF cleared

- Bit 4 **CSIRDYC**: CSI ready Interrupt Clear
Set by software to clear CSIRDYF.
Reset by hardware when clear done.
0: CSIRDYF no effect (default after reset)
1: CSIRDYF cleared
- Bit 3 **HSERDYC**: HSE ready Interrupt Clear
Set by software to clear HSERDYF.
Reset by hardware when clear done.
0: HSERDYF no effect (default after reset)
1: HSERDYF cleared
- Bit 2 **HSIRDYC**: HSI ready Interrupt Clear
Set by software to clear HSIRDYF.
Reset by hardware when clear done.
0: HSIRDYF no effect (default after reset)
1: HSIRDYF cleared
- Bit 1 **LSERDYC**: LSE ready Interrupt Clear
Set by software to clear LSERDYF.
Reset by hardware when clear done.
0: LSERDYF no effect (default after reset)
1: LSERDYF cleared
- Bit 0 **LSIRDYC**: LSI ready Interrupt Clear
Set by software to clear LSIRDYF.
Reset by hardware when clear done.
0: LSIRDYF no effect (default after reset)
1: LSIRDYF cleared

9.7.25 RCC Backup domain control register (RCC_BDCR)

Address offset: 0x070

Reset value: 0x0000 0000, reset by Backup domain reset.

Access: $0 \leq \text{wait state} \leq 7$, word, half-word and byte access. Wait states are inserted in case of successive accesses to this register.

After a system reset, the RCC_BDCR register is write-protected. To modify this register, the DBP bit in the *PWR CPU1 control register (PWR_CPU1CR)* has to be set to '1'.

RCC_BDCR bits are only reset after a backup domain reset (see [Section 9.4.6: Backup domain reset](#)). Any other internal or external reset will not have any effect on these bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BDRST
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTCEN	Res.	Res.	Res.	Res.	Res.	RTCSEL[1:0]		Res.	LSECSSD	LSECSSON	LSEDRV[1:0]		LSEBYP	LSERDY	LSEON
rw						rwo	rwo		r	rs	rw	rw	rw	r	rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **BDRST**: Backup domain software reset

Set and reset by software.

0: Reset not activated (default after backup domain reset)

1: Resets the entire VSW domain

Bit 15 **RTCEN**: RTC clock enable

Set and reset by software.

0: **rtc_ck** clock is disabled (default after backup domain reset)

1: **rtc_ck** clock enabled

Bits 14:10 Reserved, must be kept at reset value.

Bits 9:8 **RTCSEL[1:0]**: RTC clock source selection

Set by software to select the clock source for the RTC. These bits can be written only one time (except in case of failure detection on LSE). These bits must be written before LSECSSON is enabled. The BDRST bit can be used to reset them, then it can be written one time again.

If HSE is selected as RTC clock: this clock is lost when the system is in Stop mode or in case of a pin reset (NRST).

00: No clock (default after backup domain reset)

01: LSE clock used as RTC clock

10: LSI clock used as RTC clock

11: HSE clock divided by RTCPRE value is used as RTC clock

Bit 7 Reserved, must be kept at reset value.

Bit 6 LSECSSD: LSE clock security system failure detection

Set by hardware to indicate when a failure has been detected by the Clock Security System on the external 32 kHz oscillator.

0: No failure detected on 32 kHz oscillator (default after backup domain reset)

1: Failure detected on 32 kHz oscillator

Bit 5 LSECSSON: LSE clock security system enable

Set by software to enable the Clock Security System on 32 kHz oscillator.

LSECSSON must be enabled after LSE is enabled (LSEON enabled) and ready (LSERDY set by hardware), and after RTCSEL is selected.

Once enabled this bit cannot be disabled, except after a LSE failure detection (LSECSSD = '1'). In that case the software **must** disable LSECSSON.

0: Clock Security System on 32 kHz oscillator OFF (default after backup domain reset)

1: Clock Security System on 32 kHz oscillator ON

Bits 4:3 LSEDRV[1:0]: LSE oscillator driving capability

Set by software to select the driving capability of the LSE oscillator.

00: Lowest drive (default after backup domain reset)

01: Medium low drive

10: Medium high drive

11: Highest drive

Note: The driving capability cannot be changed after LSEON has been set to 1.

Bit 2 LSEBYP: LSE oscillator bypass

Set and reset by software to bypass oscillator in debug mode. This bit must not be written when the LSE is enabled (by LSEON) or ready (LSERDY = '1')

0: LSE oscillator not bypassed (default after backup domain reset)

1: LSE oscillator bypassed

Bit 1 LSERDY: LSE oscillator ready

Set and reset by hardware to indicate when the LSE is stable. This bit needs 6 cycles of **lse_ck** clock to fall down after LSEON has been set to '0'.

0: LSE oscillator not ready (default after backup domain reset)

1: LSE oscillator ready

Bit 0 LSEON: LSE oscillator enabled

Set and reset by software.

0: LSE oscillator OFF (default after backup domain reset)

1: LSE oscillator ON

9.7.26 RCC clock control and status register (RCC_CSR)

Address offset: 0x074

Reset value: 0x0000 0000

Access: $0 \leq \text{wait state} \leq 7$, word, half-word and byte access

Wait states are inserted in case of successive accesses to this register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LSIRDY	LSION
														r	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **LSIRDY**: LSI oscillator ready

Set and reset by hardware to indicate when the Low Speed Internal RC oscillator is stable.

This bit needs 3 cycles of **lsi_ck** clock to fall down after LSION has been set to '0'.

This bit can be set even when LSION is not enabled if there is a request for LSI clock by the Clock Security System on LSE or by the Low Speed Watchdog or by the RTC.

0: LSI clock is not ready (default after reset)

1: LSI clock is ready

Bit 0 **LSION**: LSI oscillator enable

Set and reset by software.

0: LSI is OFF (default after reset)

1: LSI is ON

9.7.27 RCC AHB3 reset register (RCC_AHB3RSTR)

Address offset: 0x07C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1RST
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	QSPIRST	Res.	FMCRST	Res.	Res.	Res.	Res.	Res.	Res.	JPGDECRST	DMA2DRST	Res.	Res.	Res.	MDMARST
	rw		rw							rw	rw				rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **SDMMC1RST**: SDMMC1 and SDMMC1 delay block reset

Set and reset by software.

0: does not reset SDMMC1 and SDMMC1 Delay block (default after reset)

1: resets SDMMC1 and SDMMC1 Delay block

Note: When the master is reset, all the slaves for which an access from this master is ongoing must be reset as well.

Bit 15 Reserved, must be kept at reset value.

Bit 14 **QSPIRST**: QUADSPI and QUADSPI delay block reset

Set and reset by software.

0: does not reset QUADSPI and QUADSPI Delay block (default after reset)

1: resets QUADSPI and QUADSPI Delay block

Bit 13 Reserved, must be kept at reset value.

Bit 12 **FMCRST**: FMC block reset

Set and reset by software.

0: does not reset FMC block (default after reset)

1: resets FMC block

Bits 11:6 Reserved, must be kept at reset value.

Bit 5 **JPGDECRST**: JPGDEC block reset

Set and reset by software.

0: does not reset JPGDEC block (default after reset)

1: resets JPGDEC block

Bit 4 **DMA2DRST**: DMA2D block reset

Set and reset by software.

0: does not reset DMA2D block (default after reset)

1: resets DMA2D block

Note: When the master is reset, all the slaves for which an access from this master is ongoing must be reset as well.

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 **MDMARST**: MDMA block reset

Set and reset by software.

0: does not reset MDMA block (default after reset)

1: resets MDMA block

Note: When the master is reset, all the slaves for which an access from this master is ongoing must be reset as well.

9.7.28 RCC AHB1 peripheral reset register(RCC_AHB1RSTR)

Address offset: 0x080

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	USB2OTGRST	Res.	USB1OTGRST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
				rw		rw									
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETH1MACRST	ARTRST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC12RST	Res.	Res.	Res.	DMA2RST	DMA1RST
rw	rw									rw				rw	rw

Bits 31:28 Reserved, must be kept at reset value.

Bit 27 **USB2OTGRST**: USB2OTG (OTG_HS2) block reset

Set and reset by software.

0: does not reset USB2OTG block (default after reset)

1: resets USB2OTG block

Bit 26 Reserved, must be kept at reset value.

Bit 25 **USB1OTGRST**: USB1OTG (OTG_HS1) block reset

Set and reset by software.

0: does not reset USB1OTG block (default after reset)

1: resets USB1OTG block

Bits 24:16 Reserved, must be kept at reset value.

Bit 15 **ETH1MACRST**: ETH1MAC block reset

Set and reset by software.

0: does not reset ETH1MAC block (default after reset)

1: resets ETH1MAC block

Bit 14 **ARTRST**: ART block reset

Set and reset by software.

0: does not reset ART block (default after reset)

1: resets ART block

Note: Do not set this bit while executing from a memory on which the ART is configured to fetch instructions.

Bits 13:6 Reserved, must be kept at reset value.

Bit 5 **ADC12RST**: ADC1 and 2 block reset

Set and reset by software.

0: does not reset ADC1 and 2 block (default after reset)

1: resets ADC1 and 2 block

Bits 4:2 Reserved, must be kept at reset value.

Bit 1 **DMA2RST**: DMA2 block reset

Set and reset by software.

0: does not reset DMA2 block (default after reset)

1: resets DMA2 block

Bit 0 **DMA1RST**: DMA1 block reset

Set and reset by software.

0: does not reset DMA1 block (default after reset)

1: resets DMA1 block

9.7.29 RCC AHB2 peripheral reset register (RCC_AHB2RSTR)

Address offset: 0x084

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2RST	Res.	Res.	RNGRST	HASHRST	CRYPTRST	Res.	Res.	Res.	CAMIFRST
						rw			rw	rw	rw				rw

Bits 31:10 Reserved, must be kept at reset value.

Bit 9 **SDMMC2RST**: SDMMC2 and SDMMC2 Delay block reset

Set and reset by software.

0: does not reset SDMMC2 and SDMMC2 Delay block (default after reset)

1: resets SDMMC2 and SDMMC2 Delay block

Bits 8:7 Reserved, must be kept at reset value.

Bit 6 **RNGRST**: Random Number Generator block reset

Set and reset by software.

0: does not reset RNG block (default after reset)

1: resets RNG block

Bit 5 **HASHRST**: Hash block reset

Set and reset by software.

0: does not reset hash block (default after reset)

1: resets hash block

Bit 4 **CRYPTRST**: Cryptography block reset

Set and reset by software.

0: does not reset cryptography block (default after reset)

1: resets cryptography block

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 **CAMIFRST**: CAMITF block reset

Set and reset by software.

0: does not reset the CAMITF block (default after reset)

1: resets the CAMITF block

9.7.30 RCC AHB4 peripheral reset register (RCC_AHB4RSTR)

Address offset: 0x088

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	HSEMRST	ADC3RST	Res.	Res.	BDMARST	Res.	CRCRST	Res.	Res.	Res.
						rw	rw			rw		rw			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	GPIOKRST	GPIOJRST	GPIORST	GPIOHRST	GPIOGRST	GPIOFRST	GPIOERST	GPIODRST	GPIOCRST	GPIOBRST	GPIOARST
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:26 Reserved, must be kept at reset value.

Bit 25 **HSEMRST**: HSEM block reset

Set and reset by software.

0: does not reset the HSEM block (default after reset)

1: resets the HSEM block

Bit 24 **ADC3RST**: ADC3 block reset

Set and reset by software.

0: does not reset the ADC3 block (default after reset)

1: resets the ADC3 block

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 **BDMARST**: BDMA block reset

Set and reset by software.

0: does not reset the BDMA block (default after reset)

1: resets the BDMA block

Bit 20 Reserved, must be kept at reset value.

Bit 19 **CRCRST**: CRC block reset

Set and reset by software.

0: does not reset the CRC block (default after reset)

1: resets the CRC block

Bits 18:11 Reserved, must be kept at reset value.

Bit 10 **GPIOKRST**: GPIOK block reset

Set and reset by software.

0: does not reset the GPIOK block (default after reset)

1: resets the GPIOK block

Bit 9 **GPIOJRST**: GPIOJ block reset

Set and reset by software.

0: does not reset the GPIOJ block (default after reset)

1: resets the GPIOJ block

- Bit 8 **GPIOIRST**: GPIOI block reset
Set and reset by software.
0: does not reset the GPIOI block (default after reset)
1: resets the GPIOI block
- Bit 7 **GPIOHRST**: GPIOH block reset
Set and reset by software.
0: does not reset the GPIOH block (default after reset)
1: resets the GPIOH block
- Bit 6 **GPIOGRST**: GPIOG block reset
Set and reset by software.
0: does not reset the GPIOG block (default after reset)
1: resets the GPIOG block
- Bit 5 **GPIOFRST**: GPIOF block reset
Set and reset by software.
0: does not reset the GPIOF block (default after reset)
1: resets the GPIOF block
- Bit 4 **GPIOERST**: GPIOE block reset
Set and reset by software.
0: does not reset the GPIOE block (default after reset)
1: resets the GPIOE block
- Bit 3 **GPIODRST**: GPIOD block reset
Set and reset by software.
0: does not reset the GPIOD block (default after reset)
1: resets the GPIOD block
- Bit 2 **GPIOCRST**: GPIOC block reset
Set and reset by software.
0: does not reset the GPIOC block (default after reset)
1: resets the GPIOC block
- Bit 1 **GPIOBRST**: GPIOB block reset
Set and reset by software.
0: does not reset the GPIOB block (default after reset)
1: resets the GPIOB block
- Bit 0 **GPIOARST**: GPIOA block reset
Set and reset by software.
0: does not reset the GPIOA block (default after reset)
1: resets the GPIOA block

9.7.31 RCC APB3 peripheral reset register (RCC_APB3RSTR)

Address offset: 0x08C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DSIRST	LTDCRST	Res.	Res.	Res.
											rw	rw			

Bits 31:5 Reserved, must be kept at reset value.

- Bit 4 **DSIRST**: DSI block reset
Set and reset by software.
0: does not reset the DSI block (default after reset)
1: resets the DSI block

- Bit 3 **LTDCRST**: LTDC block reset
Set and reset by software.
0: does not reset the LTDC block (default after reset)
1: resets the LTDC block

Bits 2:0 Reserved, must be kept at reset value.

9.7.32 RCC APB1 peripheral reset register (RCC_APB1LRSTR)

Address offset: 0x090

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UART8RST	UART7RST	DAC12RST	Res.	CECRST	Res.	Res.	Res.	I2C3RST	I2C2RST	I2C1RST	UART5RST	UART4RST	USART3RST	USART2RST	SPDIFXRST
rw	rw	rw		rw				rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3RST	SPI2RST	Res.	Res.	Res.	Res.	LPTIM1RST	TIM14RST	TIM13RST	TIM12RST	TIM7RST	TIM6RST	TIM5RST	TIM4RST	TIM3RST	TIM2RST
rw	rw					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **UART8RST**: UART8 block reset

Set and reset by software.

0: does not reset the UART8 block (default after reset)

1: resets the UART8 block

Bit 30 **UART7RST**: UART7 block reset

Set and reset by software.

0: does not reset the UART7 block (default after reset)

1: resets the UART7 block

Bit 29 **DAC12RST**: DAC1 and 2 Blocks Reset

Set and reset by software.

0: does not reset the DAC1 and 2 blocks (default after reset)

1: resets the DAC1 and 2 blocks

Bit 28 Reserved, must be kept at reset value.

Bit 27 **CECRST**: HDMI-CEC block reset

Set and reset by software.

0: does not reset the HDMI-CEC block (default after reset)

1: resets the HDMI-CEC block

Bits 26:24 Reserved, must be kept at reset value.

Bit 23 **I2C3RST**: I2C3 block reset

Set and reset by software.

0: does not reset the I2C3 block (default after reset)

1: resets the I2C3 block

Bit 22 **I2C2RST**: I2C2 block reset

Set and reset by software.

0: does not reset the I2C2 block (default after reset)

1: resets the I2C2 block

- Bit 21 **I2C1RST**: I2C1 block reset
Set and reset by software.
0: does not reset the I2C1 block (default after reset)
1: resets the I2C1 block
- Bit 20 **UART5RST**: UART5 block reset
Set and reset by software.
0: does not reset the UART5 block (default after reset)
1: resets the UART5 block
- Bit 19 **UART4RST**: UART4 block reset
Set and reset by software.
0: does not reset the UART4 block (default after reset)
1: resets the UART4 block
- Bit 18 **USART3RST**: USART3 block reset
Set and reset by software.
0: does not reset the USART3 block (default after reset)
1: resets the USART3 block
- Bit 17 **USART2RST**: USART2 block reset
Set and reset by software.
0: does not reset the USART2 block (default after reset)
1: resets the USART2 block
- Bit 16 **SPDIFRXRST**: SPDIFRX block reset
Set and reset by software.
0: does not reset the SPDIFRX block (default after reset)
1: resets the SPDIFRX block
- Bit 15 **SPI3RST**: SPI3 block reset
Set and reset by software.
0: does not reset the SPI3 block (default after reset)
1: resets the SPI3 block
- Bit 14 **SPI2RST**: SPI2 block reset
Set and reset by software.
0: does not reset the SPI2 block (default after reset)
1: resets the SPI2 block
- Bits 13:10 Reserved, must be kept at reset value.
- Bit 9 **LPTIM1RST**: LPTIM1 block reset
Set and reset by software.
0: does not reset the LPTIM1 block (default after reset)
1: resets the LPTIM1 block
- Bit 8 **TIM14RST**: TIM14 block reset
Set and reset by software.
0: does not reset the TIM14 block (default after reset)
1: resets the TIM14 block
- Bit 7 **TIM13RST**: TIM13 block reset
Set and reset by software.
0: does not reset the TIM13 block (default after reset)
1: resets the TIM13 block

- Bit 6 **TIM12RST**: TIM12 block reset
Set and reset by software.
0: does not reset the TIM12 block (default after reset)
1: resets the TIM12 block
- Bit 5 **TIM7RST**: TIM7 block reset
Set and reset by software.
0: does not reset the TIM7 block (default after reset)
1: resets the TIM7 block
- Bit 4 **TIM6RST**: TIM6 block reset
Set and reset by software.
0: does not reset the TIM6 block (default after reset)
1: resets the TIM6 block
- Bit 3 **TIM5RST**: TIM5 block reset
Set and reset by software.
0: does not reset the TIM5 block (default after reset)
1: resets the TIM5 block
- Bit 2 **TIM4RST**: TIM4 block reset
Set and reset by software.
0: does not reset the TIM4 block (default after reset)
1: resets the TIM4 block
- Bit 1 **TIM3RST**: TIM3 block reset
Set and reset by software.
0: does not reset the TIM3 block (default after reset)
1: resets the TIM3 block
- Bit 0 **TIM2RST**: TIM2 block reset
Set and reset by software.
0: does not reset the TIM2 block (default after reset)
1: resets the TIM2 block

9.7.33 RCC APB1 peripheral reset register (RCC_APB1HRSTR)

Address offset: 0x094

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANRST	Res.	Res.	MDIOSRST	OPAMPRST	Res.	SWPRST	CRSRST	Res.
							rw			rw	rw		rw	rw	

Bits 31:9 Reserved, must be kept at reset value.

Bit 8 **FDCANRST**: FDCAN block reset

Set and reset by software.

0: does not reset the FDCAN block (default after reset)

1: resets the FDCAN block

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **MDIOSRST**: MDIOS block reset

Set and reset by software.

0: does not reset the MDIOS block (default after reset)

1: resets the MDIOS block

Bit 4 **OPAMPRST**: OPAMP block reset

Set and reset by software.

0: does not reset the OPAMP block (default after reset)

1: resets the OPAMP block

Bit 3 Reserved, must be kept at reset value.

Bit 2 **SWPRST**: SWPMI block reset

Set and reset by software.

0: does not reset the SWPMI block (default after reset)

1: resets the SWPMI block

Bit 1 **CRSRST**: Clock Recovery System reset

Set and reset by software.

0: does not reset CRS (default after reset)

1: resets CRS

Bit 0 Reserved, must be kept at reset value.

9.7.34 RCC APB2 peripheral reset register (RCC_APB2RSTR)

Address offset: 0x098

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	HRTIMRST	DFSDM1RST	Res.	Res.	Res.	SAI3RST	SAI2RST	SAI1RST	Res.	SPI5RST	Res.	TIM17RST	TIM16RST	TIM15RST
		rw	rw				rw	rw	rw		rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	SPI4RST	SPI1RST	Res.	Res.	Res.	Res.	Res.	Res.	USART6RST	USART1RST	Res.	Res.	TIM8RST	TIM1RST
		rw	rw							rw	rw			rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **HRTIMRST**: HRTIM block reset

Set and reset by software.

0: does not reset the HRTIM block (default after reset)

1: resets the HRTIM block

Bit 28 **DFSDM1RST**: DFSDM1 block reset

Set and reset by software.

0: does not reset DFSDM1 block (default after reset)

1: resets DFSDM1 block

Bits 27:25 Reserved, must be kept at reset value.

Bit 24 **SAI3RST**: SAI3 block reset

Set and reset by software.

0: does not reset the SAI3 block (default after reset)

1: resets the SAI3 block

Bit 23 **SAI2RST**: SAI2 block reset

Set and reset by software.

0: does not reset the SAI2 block (default after reset)

1: resets the SAI2 block

Bit 22 **SAI1RST**: SAI1 block reset

Set and reset by software.

0: does not reset the SAI1 (default after reset)

1: resets the SAI1

Bit 21 Reserved, must be kept at reset value.

Bit 20 **SPI5RST**: SPI5 block reset

Set and reset by software.

0: does not reset the SPI5 block (default after reset)

1: resets the SPI5 block

Bit 19 Reserved, must be kept at reset value.

Bit 18 **TIM17RST**: TIM17 block reset
Set and reset by software.
0: does not reset the TIM17 block (default after reset)
1: resets the TIM17 block

Bit 17 **TIM16RST**: TIM16 block reset
Set and reset by software.
0: does not reset the TIM16 block (default after reset)
1: resets the TIM16 block

Bit 16 **TIM15RST**: TIM15 block reset
Set and reset by software.
0: does not reset the TIM15 block (default after reset)
1: resets the TIM15 block

Bits 15:14 Reserved, must be kept at reset value.

Bit 13 **SPI4RST**: SPI4 block reset
Set and reset by software.
0: does not reset the SPI4 block (default after reset)
1: resets the SPI4 block

Bit 12 **SPI1RST**: SPI1 block reset
Set and reset by software.
0: does not reset the SPI1 block (default after reset)
1: resets the SPI1 block

Bits 11:6 Reserved, must be kept at reset value.

Bit 5 **USART6RST**: USART6 block reset
Set and reset by software.
0: does not reset the USART6 block (default after reset)
1: resets the USART6 block

Bit 4 **USART1RST**: USART1 block reset
Set and reset by software.
0: does not reset the USART1 block (default after reset)
1: resets the USART1 block

Bits 3:2 Reserved, must be kept at reset value.

Bit 1 **TIM8RST**: TIM8 block reset
Set and reset by software.
0: does not reset the TIM8 block (default after reset)
1: resets the TIM8 block

Bit 0 **TIM1RST**: TIM1 block reset
Set and reset by software.
0: does not reset the TIM1 block (default after reset)
1: resets the TIM1 block

9.7.35 RCC APB4 peripheral reset register (RCC_APB4RSTR)

Address offset: 0x09C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4RST	Res.	Res.	Res.	Res.	Res.
										RW					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VREFRST	COMP12RST	Res.	LPTIM5RST	LPTIM4RST	LPTIM3RST	LPTIM2RST	Res.	I2C4RST	Res.	SPI6RST	Res.	LPUART1RST	Res.	SYSCFGRST	Res.
RW	RW		RW	RW	RW	RW		RW		RW		RW		RW	

Bits 31:26 Reserved, must be kept at reset value.

Bits 25:22 Reserved, must be kept at reset value.

Bit 21 **SAI4RST**: SAI4 block reset

Set and reset by software.

0: does not reset the SAI4 block (default after reset)

1: resets the SAI4 block

Bits 20:16 Reserved, must be kept at reset value.

Bit 15 **VREFRST**: VREFBUF block reset

Set and reset by software.

0: does not reset the VREFBUF block (default after reset)

1: resets the VREFBUF block

Bit 14 **COMP12RST**: COMP12 Blocks Reset

Set and reset by software.

0: does not reset the COMP1 and 2 blocks (default after reset)

1: resets the COMP1 and 2 blocks

Bit 13 Reserved, must be kept at reset value.

Bit 12 **LPTIM5RST**: LPTIM5 block reset

Set and reset by software.

0: does not reset the LPTIM5 block (default after reset)

1: resets the LPTIM5 block

Bit 11 **LPTIM4RST**: LPTIM4 block reset

Set and reset by software.

0: does not reset the LPTIM4 block (default after reset)

1: resets the LPTIM4 block

Bit 10 **LPTIM3RST**: LPTIM3 block reset

Set and reset by software.

0: does not reset the LPTIM3 block (default after reset)

1: resets the LPTIM3 block

- Bit 9 **LPTIM2RST**: LPTIM2 block reset
Set and reset by software.
0: does not reset the LPTIM2 block (default after reset)
1: resets the LPTIM2 block
- Bit 8 Reserved, must be kept at reset value.
- Bit 7 **I2C4RST**: I2C4 block reset
Set and reset by software.
0: does not reset the I2C4 block (default after reset)
1: resets the I2C4 block
- Bit 6 Reserved, must be kept at reset value.
- Bit 5 **SPI6RST**: SPI6 block reset
Set and reset by software.
0: does not reset the SPI6 block (default after reset)
1: resets the SPI6 block
- Bit 4 Reserved, must be kept at reset value.
- Bit 3 **LPUART1RST**: LPUART1 block reset
Set and reset by software.
0: does not reset the LPUART1 block (default after reset)
1: resets the LPUART1 block
- Bit 2 Reserved, must be kept at reset value.
- Bit 1 **SYSCFGRST**: SYSCFG block reset
Set and reset by software.
0: does not reset the SYSCFG block (default after reset)
1: resets the SYSCFG block
- Bit 0 Reserved, must be kept at reset value.

9.7.36 RCC global control register (RCC_GCR)

Address offset: 0x0A0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BOOT_C2	BOOT_C1	WW2RSC	WW1RSC
												rw1	rw1	rw1	rw1

Bits 31:4 Reserved, must be kept at reset value.

Bit 3 **BOOT_C2**: Allows CPU2 to boot

This bit can be set by software but is cleared by hardware after a system reset or Standby.

0: The CPU2 will not boot if the option byte BCM4 is set to '0'. (default after reset)

1: The CPU2 will boot independently of BCM4 value.

Bit 2 **BOOT_C1**: Allows CPU1 to boot

This bit can be set by software but is cleared by hardware after a system reset or Standby.

0: The CPU1 will not boot if the option byte BCM7 is set to '0' and BCM4 is set to '1'. (default after reset)

1: The CPU1 will boot independently of BCM7 value.

Bit 1 **WW2RSC**: WWDG2 reset scope control

This bit can be set by software but is cleared by hardware during a system reset

0: The WWDG2 generates a reset of CPU2, when a timeout occurs. (default after reset)

1: The WWDG2 generates a system reset, when a timeout occurs.

Bit 0 **WW1RSC**: WWDG1 reset scope control

This bit can be set by software but is cleared by hardware during a system reset

0: The WWDG1 generates a reset of CPU1, when a timeout occurs. (default after reset)

1: The WWDG1 generates a system reset, when a timeout occurs.

9.7.37 RCC D3 Autonomous mode register (RCC_D3AMR)

The Autonomous mode allows providing the peripheral clocks to peripherals located in D3, even if the CPU to which they are allocated is in CStop mode. When a peripheral has its autonomous bit enabled, it receives its peripheral clocks according to D3 domain state, if the CPU to which it is allocated is in CStop mode.

Address offset: 0x0A8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	SRAM4AMEN	BKPRAMAMEN	Res.	Res.	Res.	ADC3AMEN	Res.	Res.	SAI4AMEN	Res.	CRCAMEN	Res.	Res.	RTCAMEN
		rw	rw				rw			rw		rw			rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VREFAMEN	COMP12AMEN	Res.	LPTIM3AMEN	LPTIM4AMEN	LPTIM3AMEN	LPTIM2AMEN	Res.	I2C4AMEN	Res.	SPI6AMEN	Res.	LPUART1AMEN	Res.	Res.	BDMAAMEN
rw	rw		rw	rw	rw	rw		rw		rw		rw			rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **SRAM4AMEN**: SRAM4 Autonomous mode enable

Set and reset by software.

0: SRAM4 clock is disabled when both CPUs are in CStop (default after reset)

1: SRAM4 peripheral bus clock enabled when D3 domain is in DRun.

Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information

Bit 28 **BKPRAMAMEN**: Backup RAM Autonomous mode enable

Set and reset by software.

0: Backup RAM clock is disabled when the CPU to which it is allocated is in CStop (default after reset)

1: Backup RAM clock enabling is controlled by D3 domain state.

Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information

Bits 27:25 Reserved, must be kept at reset value.

Bit 24 **ADC3AMEN**: ADC3 Autonomous mode enable

Set and reset by software.

0: ADC3 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)

1: ADC3 peripheral clocks enabled when D3 domain is in DRun.

Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 **SAI4AMEN**: SAI4 Autonomous mode enable

Set and reset by software.

0: SAI4 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)

1: SAI4 peripheral clocks enabled when D3 domain is in DRun.

Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information

- Bit 20 Reserved, must be kept at reset value.
- Bit 19 **CRCAMEN**: CRC Autonomous mode enable
Set and reset by software.
0: CRC peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: CRC peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bits 18:17 Reserved, must be kept at reset value.
- Bit 16 **RTCAMEN**: RTC Autonomous mode enable
Set and reset by software.
0: RTC peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: RTC peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bit 15 **VREFAMEN**: VREF Autonomous mode enable
Set and reset by software.
0: VREF peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: VREF peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bit 14 **COMP12AMEN**: COMP12 Autonomous mode enable
Set and reset by software.
0: COMP12 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: COMP12 peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bit 13 Reserved, must be kept at reset value.
- Bit 12 **LPTIM5AMEN**: LPTIM5 Autonomous mode enable
Set and reset by software.
0: LPTIM5 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: LPTIM5 peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bit 11 **LPTIM4AMEN**: LPTIM4 Autonomous mode enable
Set and reset by software.
0: LPTIM4 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: LPTIM4 peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bit 10 **LPTIM3AMEN**: LPTIM3 Autonomous mode enable
Set and reset by software.
0: LPTIM3 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: LPTIM3 peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information

- Bit 9 **LPTIM2AMEN**: LPTIM2 Autonomous mode enable
Set and reset by software.
0: LPTIM2 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: LPTIM2 peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bit 8 Reserved, must be kept at reset value.
- Bit 7 **I2C4AMEN**: I2C4 Autonomous mode enable
Set and reset by software.
0: I2C4 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: I2C4 peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bit 6 Reserved, must be kept at reset value.
- Bit 5 **SPI6AMEN**: SPI6 Autonomous mode enable
Set and reset by software.
0: SPI6 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: SPI6 peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bit 4 Reserved, must be kept at reset value.
- Bit 3 **LPUART1AMEN**: LPUART1 Autonomous mode enable
Set and reset by software.
0: LPUART1 peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: LPUART1 peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information
- Bits 2:1 Reserved, must be kept at reset value.
- Bit 0 **BDMAAMEN**: BDMA and DMAMUX Autonomous mode enable
Set and reset by software.
0: BDMA and DMAMUX peripheral clocks are disabled when the CPU to which it is allocated is in CStop (default after reset)
1: BDMA and DMAMUX peripheral clocks enabled when D3 domain is in DRun.
Refer to [Section 9.5.11: Peripheral clock gating control](#) for additional information

9.7.38 RCC reset status register (RCC_RSR)

Each CPU has dedicated flags in order to check the reset status of the circuit. Please refer to [Section 9.4.4: Reset source identification](#) for additional information.

Table 69. RCC_RSR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_RSR	0x0D0	0x00FE 0000 ⁽¹⁾
RCC_C1_RSR	0x130	
RCC_C2_RSR	0x190	

1. Reset by power-on reset only

Access: $0 \leq \text{wait state} \leq 7$, word, half-word and byte access. Wait states are inserted in case of successive accesses to this register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPWR2RSTF	LPWR1RSTF	WWDG2RSTF	WWDG1RSTF	IWDG2RSTF	IWDG1RSTF	SFT2RSTF	SFT1RSTF	PORRSTF	PINRSTF	BORRSTF	D2RSTF	D1RSTF	C2RSTF	C1RSTF	RMVF
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bit 31 **LPWR2RSTF**: Reset due to illegal D2 DStandby or CPU2 CStop flag ⁽¹⁾

Reset by software by writing the RMVF bit.

Set by hardware when D2 domain goes erroneously in DStandby or when CPU2 goes erroneously in CStop.

0: No illegal reset occurred (default after power-on reset)

1: Illegal D2 DStandby or CPU2 CStop reset occurred

Bit 30 **LPWR1RSTF**: Reset due to illegal D1 DStandby or CPU1 CStop flag ⁽¹⁾

Reset by software by writing the RMVF bit.

Set by hardware when D1 domain goes erroneously in DStandby or when CPU1 goes erroneously in CStop.

0: No illegal reset occurred (default after power-on reset)

1: Illegal D1 DStandby or CPU1 CStop reset occurred

Bit 29 **WWDG2RSTF**: Window Watchdog reset flag ⁽¹⁾

Reset by software by writing the RMVF bit.

Set by hardware when a window watchdog reset occurs.

0: No window watchdog reset occurred from WWDG2 (default after power-on reset)

1: window watchdog reset occurred from WWDG2

Bit 28 **WWDG1RSTF**: Window Watchdog reset flag ⁽¹⁾

Reset by software by writing the RMVF bit.

Set by hardware when a window watchdog reset occurs.

0: No window watchdog reset occurred from WWDG1 (default after power-on reset)

1: window watchdog reset occurred from WWDG1

- Bit 27 **IWDG2RSTF**: CPU2 Independent Watchdog reset flag ⁽¹⁾
Reset by software by writing the RMVF bit.
Set by hardware when a CPU2 independent watchdog reset occurs.
0: No CPU2 independent watchdog reset occurred (default after power-on reset)
1: CPU2 independent watchdog reset occurred
- Bit 26 **IWDG1RSTF**: CPU1 Independent Watchdog reset flag ⁽¹⁾
Reset by software by writing the RMVF bit.
Set by hardware when a CPU1 independent watchdog reset occurs.
0: No CPU1 independent watchdog reset occurred (default after power-on reset)
1: CPU1 independent watchdog reset occurred
- Bit 25 **SFT2RSTF**: System reset from CPU2 reset flag ⁽¹⁾
Reset by software by writing the RMVF bit.
Set by hardware when the system reset is due to CPU2. The CPU2 can generate a system reset by writing SYSRESETREQ bit of AIRCR register of the CM4.
0: No CPU2 software reset occurred (default after power-on reset)
1: A system reset has been generated by the CPU2
- Bit 24 **SFT1RSTF**: System reset from CPU1 reset flag ⁽¹⁾
Reset by software by writing the RMVF bit.
Set by hardware when the system reset is due to CPU1. The CPU1 can generate a system reset by writing SYSRESETREQ bit of AIRCR register of the CM7.
0: No CPU1 software reset occurred (default after power-on reset)
1: A system reset has been generated by the CPU1
- Bit 23 **PORRSTF**: POR/PDR reset flag ⁽¹⁾
Reset by software by writing the RMVF bit.
Set by hardware when a POR/PDR reset occurs.
0: No POR/PDR reset occurred
1: POR/PDR reset occurred (default after power-on reset)
- Bit 22 **PINRSTF**: Pin reset flag (NRST) ⁽¹⁾
Reset by software by writing the RMVF bit.
Set by hardware when a reset from pin occurs.
0: No reset from pin occurred
1: Reset from pin occurred (default after power-on reset)
- Bit 21 **BORRSTF**: BOR reset flag ⁽¹⁾
Reset by software by writing the RMVF bit.
Set by hardware when a BOR reset occurs (**pwr_bor_rst**).
0: No BOR reset occurred
1: BOR reset occurred (default after power-on reset)
- Bit 20 **D2RSTF**: D2 domain power switch reset flag ⁽¹⁾
Reset by software by writing the RMVF bit.
Set by hardware when a D2 domain exits from DStandby or after of power-on reset. Refer to [Table 57](#) for details.
0: No D2 domain power switch reset occurred
1: A D2 domain power switch (ePOD2) reset occurred (default after power-on reset)

Bit 19 **D1RSTF**: D1 domain power switch reset flag ⁽¹⁾

Reset by software by writing the RMVF bit.

Set by hardware when a D1 domain exits from DStandby or after of power-on reset. Refer to [Table 57](#) for details.

0: No D1 domain power switch reset occurred

1: A D1 domain power switch (ePOD1) reset occurred (default after power-on reset)

Bit 18 **C2RSTF**: CPU2 reset flag ⁽¹⁾

Reset by software by writing the RMVF bit.

Set by hardware every time a CPU2 reset occurs.

0: No CPU2 reset occurred

1: A CPU2 reset occurred (default after power-on reset)

Bit 17 **C1RSTF**: CPU1 reset flag ⁽¹⁾

Reset by software by writing the RMVF bit.

Set by hardware every time a CPU1 reset occurs.

0: No CPU1 reset occurred

1: A CPU1 reset occurred (default after power-on reset)

Bit 16 **RMVF**: Remove reset flag

Set and reset by software to reset the value of the reset flags.

0: Reset of the reset flags not activated (default after power-on reset)

1: Reset the value of the reset flags

Bits 15:0 Reserved, must be kept at reset value.

1. Refer to [Table 57: Reset source identification \(RCC_RSR\)](#) for details on flag behavior.

9.7.39 RCC AHB3 clock register (RCC_AHB3ENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) for more information on peripheral allocation.

Table 70. RCC_AHB3ENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_AHB3ENR	0x0D4	0x0000 0000
RCC_C1_AHB3ENR	0x134	
RCC_C2_AHB3ENR	0x194	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AXISRAMEN ⁽¹⁾	ITCMEN ⁽¹⁾	DTCM2EN ⁽¹⁾	DTCM1EN ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1EN
rw	rw	rw	rw												rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	QSPIEN	Res.	FMCEN	Res.	Res.	Res.	FLITFEN ⁽¹⁾	Res.	Res.	JPGDECEN	DMA2DEN	Res.	Res.	Res.	MDMAEN
	rw		rw				rw			rw	rw				rw

1. This location is 'Reserved' for register RCC_C1_AHB3ENR register.

Bit 31 AXISRAMEN: AXISRAM block enable

Set and reset by software.

When set, this bit indicates that the AXISRAM is allocated by the CPU2. It causes the D1 domain to take into account also the CPU2 operation modes, i.e. keeping D1 domain in DRun when CPU2 is in CRun.

0: AXISRAM not allocated by CPU2.

AXISRAM follows the D1 domain modes. (default after reset)

1: AXISRAM allocated by CPU2.

AXISRAM and D1 domain take also the CPU2 operating modes into account.

Bit 30 ITCM1EN: D1 ITCM block enable

Set and reset by software.

When set, this bit indicates that the ITCM is allocated by the CPU2. It causes the D1 domain to take into account also the CPU2 operation modes, i.e. keeping D1 domain in DRun when CPU2 is in CRun.

0: ITCM not allocated by CPU2.

ITCM follows the D1 domain modes. (default after reset)

1: ITCM allocated by CPU2.

ITCM and D1 domain take also the CPU2 operating modes into account.

Bit 29 DTCM2EN: D1 DTCM2 block enable

Set and reset by software.

When set, this bit indicates that the DTCM2 is allocated by the CPU2. It causes the D1 domain to take into account also the CPU2 operation modes, i.e. keeping D1 domain in DRun when CPU2 is in CRun.

0: DTCM2 not allocated by CPU2.

DTCM2 follows the D1 domain modes. (default after reset)

1: DTCM2 allocated by CPU2.

DTCM2 and D1 domain take also the CPU2 operating modes into account.

Bit 28 DTCM1EN: D1 DTCM1 block enable

Set and reset by software.

When set, this bit indicates that the DTCM1 is allocated by the CPU2. It causes the D1 domain to take into account also the CPU2 operation modes, i.e. keeping D1 domain in DRun when CPU2 is in CRun.

0: DTCM1 not allocated by CPU2.

DTCM1 follows the D1 domain modes. (default after reset)

1: DTCM1 allocated by CPU2.

DTCM1 and D1 domain take also the CPU2 operating modes into account.

Bits 27:17 Reserved, must be kept at reset value.

Bit 16 SDMMC1EN: SDMMC1 and SDMMC1 Delay Clock Enable

Set and reset by software.

0: SDMMC1 and SDMMC1 Delay clock disabled (default after reset)

1: SDMMC1 and SDMMC1 Delay clock enabled

Bit 15 Reserved, must be kept at reset value.

Bit 14 QSPIEN: QUADSPI and QUADSPI Delay Clock Enable

Set and reset by software.

0: QUADSPI and QUADSPI Delay clock disabled (default after reset)

1: QUADSPI and QUADSPI Delay clock enabled

Bit 13 Reserved, must be kept at reset value.

Bit 12 FMCEN: FMC Peripheral Clocks Enable

Set and reset by software.

0: FMC peripheral clocks disabled (default after reset)

1: FMC peripheral clocks enabled

The peripheral clocks of the FMC are: the kernel clock selected by FMCSEL and provided to `fmc_ker_ck` input, and the **rcc_hclk3** bus interface clock.

Bits 11:9 Reserved, must be kept at reset value.

Bit 8 FLITFEN: D1 FLASH Interface Enable

Set and reset by software.

When set, this bit indicates that the D1 FLASH block is allocated by one or both CPUs. It causes the D1 domain to take into account also the CPU2 operation modes, i.e. keeping D1 domain in DRun when CPU2 is in CRun.

0: D1 FLASH Interface not allocated by CPU2.

D1 FLASH Interface follows the D1 domain modes.(default after reset)

1: D1 FLASH Interface allocated by CPU2.

D1 FLASH Interface and D1 domain take also the CPU2 operating modes into account.

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 JPGDECEN: JPGDEC Peripheral Clock Enable

Set and reset by software.

0: JPGDEC peripheral clock disabled (default after reset)

1: JPGDEC peripheral clock enabled

Bit 4 DMA2DEN: DMA2D Peripheral Clock Enable

Set and reset by software.

0: DMA2D peripheral clock disabled (default after reset)

1: DMA2D peripheral clock enabled

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 MDMAEN: MDMA Peripheral Clock Enable

Set and reset by software.

0: MDMA peripheral clock disabled (default after reset)

1: MDMA peripheral clock enabled

9.7.40 RCC AHB1 clock register (RCC_AHB1ENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 71. RCC_AHB1ENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_AHB1ENR	0x0D8	0x0000 0000
RCC_C1_AHB1ENR	0x138	
RCC_C2_AHB1ENR	0x198	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	USB2OTGHSULPIEN	USB2OTGHSSEN	USB1OTGHSULPIEN	USB1OTGHSSEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH1RXEN	ETH1TXEN
			r/w	r/w	r/w	r/w								r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETH1MACEN	ARTEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC12EN	Res.	Res.	Res.	DMA2EN	DMA1EN
r/w	r/w									r/w				r/w	r/w

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 USB2OTGHSULPIEN: Enable USB_PHY2 clocks

Set and reset by software.

0: USB2ULPI PHY clocks disabled (default after reset)

1: USB2ULPI PHY clocks enabled

Bit 27 USB2OTGHSSEN: Enable of USB2OTG (OTG_HS2) peripheral clocks

Set and reset by software.

0: USB2OTG peripheral clocks disabled (default after reset)

1: USB2OTG peripheral clocks enabled

The peripheral clocks of the USB2OTG are: the kernel clock selected by USBSEL and the **rcc_hclk1** bus interface clock.

Bit 26 USB1OTGHSULPIEN: Enable of USB_PHY1 clocks

Set and reset by software.

0: USB1ULPI PHY clocks disabled (default after reset)

1: USB1ULPI PHY clocks enabled

Bit 25 **USB1OTGHSEN**: Enable of USB1OTG (OTG_HS1) peripheral clocks

Set and reset by software.

0: USB1OTG peripheral clocks disabled (default after reset)

1: USB1OTG peripheral clocks enabled

The peripheral clocks of the USB1OTG are: the kernel clock selected by USBSEL and the **rcc_hclk1** bus interface clock.

Bits 24:18 Reserved, must be kept at reset value.

Bit 17 **ETH1RXEN**: Enable of Ethernet reception clock

Set and reset by software.

0: Ethernet Reception clock disabled (default after reset)

1: Ethernet Reception clock enabled

Bit 16 **ETH1TXEN**: Enable of Ethernet transmission clock

Set and reset by software.

0: Ethernet Transmission clock disabled (default after reset)

1: Ethernet Transmission clock enabled

Bit 15 **ETH1MACEN**: Enable of Ethernet MAC bus interface clock

Set and reset by software.

0: Ethernet MAC bus interface clock disabled (default after reset)

1: Ethernet MAC bus interface clock enabled

Bit 14 **ARTEN**: ART Clock enable

Set and reset by software.

0: ART clock disabled (default after reset)

1: ART clock enabled

Bits 13:6 Reserved, must be kept at reset value.

Bit 5 **ADC12EN**: Enable of ADC1/2 peripheral clocks

Set and reset by software.

0: ADC1 and 2 peripheral clocks disabled (default after reset)

1: ADC1 and 2 peripheral clocks enabled

The peripheral clocks of the ADC1 and 2 are: the kernel clock selected by ADCSEL and provided to **adc_ker_ck_input**, and the **rcc_hclk1** bus interface clock.

Bits 4:2 Reserved, must be kept at reset value.

Bit 1 **DMA2EN**: DMA2 clock enable

Set and reset by software.

0: DMA2 clock disabled (default after reset)

1: DMA2 clock enabled

Bit 0 **DMA1EN**: DMA1 clock enable

Set and reset by software.

0: DMA1 clock disabled (default after reset)

1: DMA1 clock enabled

9.7.41 RCC AHB2 clock register (RCC_AHB2ENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 72. RCC_AHB2ENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_AHB2ENR	0x0DC	0x0000 0000
RCC_C1_AHB2ENR	0x13C	
RCC_C2_AHB2ENR	0x19C	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAM3EN ⁽¹⁾	SRAM2EN ⁽¹⁾	SRAM1EN ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw	rw	rw													
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2EN	Res.	Res.	RNGEN	HASHEN	CRYPTEN	Res.	Res.	Res.	DCMIEN
						rw			rw	rw	rw				rw

1. This location is 'Reserved' for register RCC_C2_AHB2ENR register.

Bit 31 SRAM3EN: SRAM3 block enable

Set and reset by software.

When set, this bit indicates that the SRAM3 is allocated by the CPU1. It causes the D2 domain to take into account also the CPU1 operation modes, i.e. keeping D2 domain in DRun when CPU1 is in CRun.

0: SRAM3 not allocated by CPU1.

SRAM3 follows the D2 domain modes. (default after reset)

1: SRAM3 allocated by CPU1.

SRAM3 and D2 domain take also the CPU1 operating modes into account.

Bit 30 SRAM2EN: SRAM2 block enable

Set and reset by software.

When set, this bit indicates that the SRAM2 is allocated by the CPU1. It causes the D2 domain to take into account also the CPU1 operation modes, i.e. keeping D2 domain in DRun when CPU1 is in CRun.

0: SRAM2 not allocated by CPU1.

SRAM2 follows the D2 domain modes. (default after reset)

1: SRAM2 allocated by CPU1.

SRAM2 and D2 domain take also the CPU1 operating modes into account.

Bit 29 SRAM1EN: SRAM1 block enable

Set and reset by software.

When set, this bit indicates that the SRAM1 is allocated by the CPU1. It causes the D2 domain to take into account also the CPU1 operation modes, i.e. keeping D2 domain in DRun when CPU1 is in CRun.

0: SRAM1 not allocated by CPU1.

SRAM1 follows the D2 domain modes. (default after reset)

1: SRAM1 allocated by CPU1.

SRAM1 and D2 domain take also the CPU1 operating modes into account.

Bits 28:10 Reserved, must be kept at reset value.

Bit 9 SDMMC2EN: SDMMC2 and SDMMC2 delay clock enable

Set and reset by software.

0: SDMMC2 and SDMMC2 Delay clock disabled (default after reset)

1: SDMMC2 and SDMMC2 Delay clock enabled

Bits 8:7 Reserved, must be kept at reset value.

Bit 6 RNGEN: RNG peripheral clocks enable

Set and reset by software.

0: RNG peripheral clocks disabled (default after reset)

1: RNG peripheral clocks enabled:

The peripheral clocks of the RNG are: the kernel clock selected by RNGSEL and provided to **rng_ker_ck** input, and the **rcc_hclk2** bus interface clock.

Bit 5 HASHEN: HASH peripheral clock enable

Set and reset by software.

0: HASH peripheral clock disabled (default after reset)

1: HASH peripheral clock enabled

Bit 4 **CRYPTEN**: CRYPT peripheral clock enable

Set and reset by software.

0: CRYPT peripheral clock disabled (default after reset)

1: CRYPT peripheral clock enabled

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 **DCMIEN**: DCMI peripheral clock enable

Set and reset by software.

0: DCMI peripheral clock disabled (default after reset)

1: DCMI peripheral clock enabled

9.7.42 RCC AHB4 clock register (RCC_AHB4ENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 73. RCC_AHB4ENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_AHB4ENR	0x0E0	0x0000 0000
RCC_C1_AHB4ENR	0x140	
RCC_C2_AHB4ENR	0x1A0	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	BKPRAMEN	Res.	Res.	HSEMEN	ADC3EN	Res.	Res.	BDMAEN	Res.	CRCEN	Res.	Res.	Res.
			rw			rw	rw			rw		rw			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	GPIOKEN	GPIOJEN	GPIOJEN	GPIOHEN	GPIOGEN	GPIOFEN	GPIOEEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **BKPRAMEN**: Backup RAM Clock Enable

Set and reset by software.

0: Backup RAM clock disabled (default after reset)

1: Backup RAM clock enabled

Bits 27:26 Reserved, must be kept at reset value.

Bit 25 **HSEMEN**: HSEM peripheral clock enable

Set and reset by software.

0: HSEM peripheral clock disabled (default after reset)

1: HSEM peripheral clock enabled

Bit 24 **ADC3EN**: ADC3 Peripheral Clocks Enable

Set and reset by software.

0: ADC3 peripheral clocks disabled (default after reset)

1: ADC3 peripheral clocks enabled

The peripheral clocks of the ADC3 are: the kernel clock selected by ADCSEL and provided to `adc_ker_ck_input`, and the **rcc_hclk4** bus interface clock.

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 **BDMAEN**: BDMA and DMAMUX2 Clock Enable

Set and reset by software.

0: BDMA and DMAMUX2 clock disabled (default after reset)

1: BDMA and DMAMUX2 clock enabled

Bit 20 Reserved, must be kept at reset value.

Bit 19 **CRCCEN**: CRC peripheral clock enable

Set and reset by software.

0: CRC peripheral clock disabled (default after reset)

1: CRC peripheral clock enabled

Bits 18:11 Reserved, must be kept at reset value.

Bit 10 **GPIOKEN**: GPIOK peripheral clock enable

Set and reset by software.

0: GPIOK peripheral clock disabled (default after reset)

1: GPIOK peripheral clock enabled

Bit 9 **GPIOJEN**: GPIOJ peripheral clock enable

Set and reset by software.

0: GPIOJ peripheral clock disabled (default after reset)

1: GPIOJ peripheral clock enabled

Bit 8 **GPIOJEN**: GPIOI peripheral clock enable

Set and reset by software.

0: GPIOI peripheral clock disabled (default after reset)

1: GPIOI peripheral clock enabled

Bit 7 **GPIOHEN**: GPIOH peripheral clock enable

Set and reset by software.

0: GPIOH peripheral clock disabled (default after reset)

1: GPIOH peripheral clock enabled

Bit 6 **GPIOGEN**: GPIOG peripheral clock enable

Set and reset by software.

0: GPIOG peripheral clock disabled (default after reset)

1: GPIOG peripheral clock enabled

Bit 5 **GPIOFEN**: GPIOF peripheral clock enable

Set and reset by software.

0: GPIOF peripheral clock disabled (default after reset)

1: GPIOF peripheral clock enabled

Bit 4 **GPIOEEN**: GPIOE peripheral clock enable

Set and reset by software.

0: GPIOE peripheral clock disabled (default after reset)

1: GPIOE peripheral clock enabled

Bit 3 **GPIODEN**: GPIOD peripheral clock enable

Set and reset by software.

0: GPIOD peripheral clock disabled (default after reset)

1: GPIOD peripheral clock enabled

- Bit 2 **GPIOCEN**: GPIOC peripheral clock enable
Set and reset by software.
0: GPIOC peripheral clock disabled (default after reset)
1: GPIOC peripheral clock enabled
- Bit 1 **GPIOBEN**: GPIOB peripheral clock enable
Set and reset by software.
0: GPIOB peripheral clock disabled (default after reset)
1: GPIOB peripheral clock enabled
- Bit 0 **GPIOAEN**: GPIOA peripheral clock enable
Set and reset by software.
0: GPIOA peripheral clock disabled (default after reset)
1: GPIOA peripheral clock enabled

9.7.43 RCC APB3 clock register (RCC_APB3ENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 74. RCC_APB3ENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB3ENR	0x0E4	0x0000 0000
RCC_C1_APB3ENR	0x144	
RCC_C2_APB3ENR	0x1A4	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WWDG1EN	Res.	DSIEN	LTDCEN	Res.	Res.	Res.
									rw		rw	rw			

Bits 31:7 Reserved, must be kept at reset value.

Bit 6 **WWDG1EN**: WWDG1 Clock Enable

Set by software, and reset by hardware when a system reset occurs.

In order to work properly, before enabling the WWDG1, the bit WW1RSC must be set to '1'.

0: WWDG1 peripheral clock disable (default after reset)

1: WWDG1 peripheral clock enabled

There is no protection to prevent the CPU2 to set the bit WWDG1EN to '1'.

It is not recommended to enable the WWDG1 clock for CPU2 operating modes in RCC_C2_APB3ENR register.

Bit 5 Reserved, must be kept at reset value.

Bit 4 **DSIEN**: DSI Peripheral Clocks Enable

Set and reset by software.

0: DSI peripheral clocks disabled (default after reset)

1: DSI peripheral clocks enabled

Bit 3 **LTDCEN**: LTDC peripheral clock enable

Provides the pixel clock (**ltdc_ker_ck**) to both DSI and LTDC

Set and reset by software.

0: LTDC peripheral clock disabled (default after reset)

1: LTDC peripheral clock provided to both DSI and LTDC blocks

Bits 2:0 Reserved, must be kept at reset value.

9.7.44 RCC APB1 clock register (RCC_APB1LENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 75. RCC_APB1LENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB1LENR	0x0E8	0x0000 0000
RCC_C1_APB1LENR	0x148	
RCC_C2_APB1LENR	0x1A8	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UART8EN	UART7EN	DAC12EN	Res.	CECEN	Res.	Res.	Res.	I2C3EN	I2C2EN	I2C1EN	UART5EN	UART4EN	USART3EN	USART2EN	SPDIFRXEN
rw	rw	rw		rw				rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3EN	SPI2EN	Res.	Res.	WWDG2EN	Res.	LPTIM1EN	TIM14EN	TIM13EN	TIM12EN	TIM7EN	TIM6EN	TIM5EN	TIM4EN	TIM3EN	TIM2EN
rw	rw			rs		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 UART8EN: UART8 Peripheral Clocks Enable

Set and reset by software.

0: UART8 peripheral clocks disable (default after reset)

1: UART8 peripheral clocks enabled

The peripheral clocks of the UART8 are: the kernel clock selected by USART234578SEL and provided to **usart_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 30 UART7EN: UART7 Peripheral Clocks Enable

Set and reset by software.

0: UART7 peripheral clocks disable (default after reset)

1: UART7 peripheral clocks enabled

The peripheral clocks of the UART7 are: the kernel clock selected by USART234578SEL and provided to **usart_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 29 DAC12EN: DAC1 and 2 peripheral clock enable

Set and reset by software.

0: DAC1 and 2 peripheral clock disable (default after reset)

1: DAC1 and 2 peripheral clock enabled

Bit 28 Reserved, must be kept at reset value.

Bit 27 CECEN: HDMI-CEC peripheral clock enable

Set and reset by software.

0: HDMI-CEC peripheral clock disable (default after reset)

1: HDMI-CEC peripheral clock enabled

The peripheral clocks of the HDMI-CEC are: the kernel clock selected by CECSEL and provided to **cec_ker_ck** input, and the **pclk1d2** bus interface clock.

Bits 26:24 Reserved, must be kept at reset value.

Bit 23 **I2C3EN**: I2C3 Peripheral Clocks Enable

Set and reset by software.

0: I2C3 peripheral clocks disable (default after reset)

1: I2C3 peripheral clocks enabled

The peripheral clocks of the I2C3 are: the kernel clock selected by I2C123SEL and provided to **i2c_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 22 **I2C2EN**: I2C2 Peripheral Clocks Enable

Set and reset by software.

0: I2C2 peripheral clocks disable (default after reset)

1: I2C2 peripheral clocks enabled

The peripheral clocks of the I2C2 are: the kernel clock selected by I2C123SEL and provided to **i2c_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 21 **I2C1EN**: I2C1 Peripheral Clocks Enable

Set and reset by software.

0: I2C1 peripheral clocks disable (default after reset)

1: I2C1 peripheral clocks enabled

The peripheral clocks of the I2C1 are: the kernel clock selected by I2C123SEL and provided to **i2c_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 20 **UART5EN**: UART5 Peripheral Clocks Enable

Set and reset by software.

0: UART5 peripheral clocks disable (default after reset)

1: UART5 peripheral clocks enabled

The peripheral clocks of the UART5 are: the kernel clock selected by USART234578SEL and provided to **uart_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 19 **UART4EN**: UART4 Peripheral Clocks Enable

Set and reset by software.

0: UART4 peripheral clocks disable (default after reset)

1: UART4 peripheral clocks enabled

The peripheral clocks of the UART4 are: the kernel clock selected by USART234578SEL and provided to **uart_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 18 **USART3EN**: USART3 Peripheral Clocks Enable

Set and reset by software.

0: USART3 peripheral clocks disable (default after reset)

1: USART3 peripheral clocks enabled

The peripheral clocks of the USART3 are: the kernel clock selected by USART234578SEL and provided to **uart_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 17 **USART2EN**: USART2 Peripheral Clocks Enable

Set and reset by software.

0: USART2 peripheral clocks disable (default after reset)

1: USART2 peripheral clocks enabled

The peripheral clocks of the USART2 are: the kernel clock selected by USART234578SEL and provided to **uart_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 16 SPDIFRXEN: SPDIFRX Peripheral Clocks Enable

Set and reset by software.

0: SPDIFRX peripheral clocks disable (default after reset)

1: SPDIFRX peripheral clocks enabled

The peripheral clocks of the SPDIFRX are: the kernel clock selected by SPDIFSEL and provided to SPDIF_CLK input, and the **pclk1d2** bus interface clock.

Bit 15 SPI3EN: SPI3 Peripheral Clocks Enable

Set and reset by software.

0: SPI3 peripheral clocks disable (default after reset)

1: SPI3 peripheral clocks enabled

The peripheral clocks of the SPI3 are: the kernel clock selected by I2S123SRC and provided to **spi_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 14 SPI2EN: SPI2 Peripheral Clocks Enable

Set and reset by software.

0: SPI2 peripheral clocks disable (default after reset)

1: SPI2 peripheral clocks enabled

The peripheral clocks of the SPI2 are: the kernel clock selected by I2S123SRC and provided to **spi_ker_ck** input, and the **pclk1d2** bus interface clock.

Bits 13:12 Reserved, must be kept at reset value.

Bit 11 WWDG2EN: WWDG2 peripheral clock enable

Set by software, and reset by hardware when a system reset occurs

0: WWDG2 peripheral clock disabled (default after reset)

1: WWDG2 peripheral clock enabled

There is no protection to prevent the CPU1 to set the bit WWDG2EN to '1'.

It is not recommended to enable the WWDG2 clock for CPU1 operating modes in RCC_C1_APB3ENR register.

Bit 10 Reserved, must be kept at reset value.

Bit 9 LPTIM1EN: LPTIM1 Peripheral Clocks Enable

Set and reset by software.

0: LPTIM1 peripheral clocks disable (default after reset)

1: LPTIM1 peripheral clocks enabled

The peripheral clocks of the LPTIM1 are: the kernel clock selected by LPTIM1SEL and provided to **lptim_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 8 TIM14EN: TIM14 peripheral clock enable

Set and reset by software.

0: TIM14 peripheral clock disable (default after reset)

1: TIM14 peripheral clock enabled

Bit 7 TIM13EN: TIM13 peripheral clock enable

Set and reset by software.

0: TIM13 peripheral clock disable (default after reset)

1: TIM13 peripheral clock enabled

Bit 6 TIM12EN: TIM12 peripheral clock enable

Set and reset by software.

0: TIM12 peripheral clock disable (default after reset)

1: TIM12 peripheral clock enabled

- Bit 5 **TIM7EN**: TIM7 peripheral clock enable
Set and reset by software.
0: TIM7 peripheral clock disable (default after reset)
1: TIM7 peripheral clock enabled
- Bit 4 **TIM6EN**: TIM6 peripheral clock enable
Set and reset by software.
0: TIM6 peripheral clock disable (default after reset)
1: TIM6 peripheral clock enabled
- Bit 3 **TIM5EN**: TIM5 peripheral clock enable
Set and reset by software.
0: TIM5 peripheral clock disable (default after reset)
1: TIM5 peripheral clock enabled
- Bit 2 **TIM4EN**: TIM4 peripheral clock enable
Set and reset by software.
0: TIM4 peripheral clock disable (default after reset)
1: TIM4 peripheral clock enabled
- Bit 1 **TIM3EN**: TIM3 peripheral clock enable
Set and reset by software.
0: TIM3 peripheral clock disable (default after reset)
1: TIM3 peripheral clock enabled
- Bit 0 **TIM2EN**: TIM2 peripheral clock enable
Set and reset by software.
0: TIM2 peripheral clock disable (default after reset)
1: TIM2 peripheral clock enabled

9.7.45 RCC APB1 clock register (RCC_APB1HENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 76. RCC_APB1ENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB1HENR	0x0EC	0x0000 0000
RCC_C1_APB1HENR	0x14C	
RCC_C2_APB1HENR	0x1AC	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANEN	Res.	Res.	MDIOSEN	OPAMPEN	Res.	SWPEN	CRSEN	Res.
							rw			rw	rw		rw	rw	

Bits 31:9 Reserved, must be kept at reset value.

Bit 8 **FDCANEN**: FDCAN Peripheral Clocks Enable

Set and reset by software.

0: FDCAN peripheral clocks disable (default after reset)

1: FDCAN peripheral clocks enabled:

The peripheral clocks of the FDCAN are: the kernel clock selected by FDCANSEL and provided to **rcc_fdcan_ker_ck** input, and the **pclk1d2** bus interface clock.

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **MDIOSEN**: MDIOS peripheral clock enable

Set and reset by software.

0: MDIOS peripheral clock disable (default after reset)

1: MDIOS peripheral clock enabled

Bit 4 **OPAMPEN**: OPAMP peripheral clock enable

Set and reset by software.

0: OPAMP peripheral clock disable (default after reset)

1: OPAMP peripheral clock enabled

Bit 3 Reserved, must be kept at reset value.

Bit 2 **SWPEN**: SWPMI Peripheral Clocks Enable

Set and reset by software.

0: SWPMI peripheral clocks disable (default after reset)

1: SWPMI peripheral clocks enabled:

The peripheral clocks of the SWPMI are: the kernel clock selected by SWPSEL and provided to

swpmi_ker_ck input, and the **pclk1d2** bus interface clock.

Bit 1 **CRSEN**: Clock Recovery System peripheral clock enable

Set and reset by software.

0: CRS peripheral clock disable (default after reset)

1: CRS peripheral clock enabled

Bit 0 Reserved, must be kept at reset value.

9.7.46 RCC APB2 clock register (RCC_APB2ENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 77. RCC_APB2ENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB2ENR	0x0F0	0x0000 0000
RCC_C1_APB2ENR	0x150	
RCC_C2_APB2ENR	0x1B0	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	HRTIMEN	DFSDM1EN	Res.	Res.	Res.	SAI3EN	SAI2EN	SAI1EN	Res.	SPI5EN	Res.	TIM17EN	TIM16EN	TIM15EN
		rw	rw				rw	rw	rw		rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	SPI4EN	SPI1EN	Res.	Res.	Res.	Res.	Res.	Res.	USART6EN	USART1EN	Res.	Res.	TIM8EN	TIM1EN
		rw	rw							rw	rw			rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **HRTIMEN**: HRTIM peripheral clock enable

Set and reset by software.

0: HRTIM peripheral clock disabled (default after reset)

1: HRTIM peripheral clock enabled

Bit 28 **DFSDM1EN**: DFSDM1 Peripheral Clocks Enable

Set and reset by software.

0: DFSDM1 peripheral clocks disabled (default after reset)

1: DFSDM1 peripheral clocks enabled

DFSDM1 peripheral clocks are: the kernel clocks selected by SAI1SEL and DFSDM1SEL and provided to **Aclk** and **clk** inputs respectively, and the **rcc_pclk2** bus interface clock.

Bits 27:25 Reserved, must be kept at reset value.

Bit 24 **SAI3EN**: SAI3 Peripheral Clocks Enable

Set and reset by software.

0: SAI3 peripheral clocks disabled (default after reset)

1: SAI3 peripheral clocks enabled

The peripheral clocks of the SAI3 are: the kernel clock selected by SAI23SEL and provided to **sai_a_ker_ck** and **sai_b_ker_ck** inputs, and the **rcc_pclk2** bus interface clock.

- Bit 23 **SAI2EN**: SAI2 Peripheral Clocks Enable
 Set and reset by software.
 0: SAI2 peripheral clocks disabled (default after reset)
 1: SAI2 peripheral clocks enabled
 The peripheral clocks of the SAI2 are: the kernel clock selected by SAI23SEL and provided to **sai_a_ker_ck** and **sai_b_ker_ck** inputs, and the **rcc_pclk2** bus interface clock.
- Bit 22 **SAI1EN**: SAI1 Peripheral Clocks Enable
 Set and reset by software.
 0: SAI1 peripheral clocks disabled (default after reset)
 1: SAI1 peripheral clocks enabled:
 The peripheral clocks of the SAI1 are: the kernel clock selected by SAI1SEL and provided to **sai_a_ker_ck** and **sai_b_ker_ck** inputs, and the **rcc_pclk2** bus interface clock.
- Bit 21 Reserved, must be kept at reset value.
- Bit 20 **SPI5EN**: SPI5 Peripheral Clocks Enable
 Set and reset by software.
 0: SPI5 peripheral clocks disabled (default after reset)
 1: SPI5 peripheral clocks enabled:
 The peripheral clocks of the SPI5 are: the kernel clock selected by SPI45SEL and provided to **spi_ker_ck** input, and the **rcc_pclk2** bus interface clock.
- Bit 19 Reserved, must be kept at reset value.
- Bit 18 **TIM17EN**: TIM17 peripheral clock enable
 Set and reset by software.
 0: TIM17 peripheral clock disabled (default after reset)
 1: TIM17 peripheral clock enabled
- Bit 17 **TIM16EN**: TIM16 peripheral clock enable
 Set and reset by software.
 0: TIM16 peripheral clock disabled (default after reset)
 1: TIM16 peripheral clock enabled
- Bit 16 **TIM15EN**: TIM15 peripheral clock enable
 Set and reset by software.
 0: TIM15 peripheral clock disabled (default after reset)
 1: TIM15 peripheral clock enabled
- Bits 15:14 Reserved, must be kept at reset value.
- Bit 13 **SPI4EN**: SPI4 Peripheral Clocks Enable
 Set and reset by software.
 0: SPI4 peripheral clocks disabled (default after reset)
 1: SPI4 peripheral clocks enabled:
 The peripheral clocks of the SPI4 are: the kernel clock selected by SPI45SEL and provided to **spi_ker_ck** input, and the **rcc_pclk2** bus interface clock.
- Bit 12 **SPI1EN**: SPI1 Peripheral Clocks Enable
 Set and reset by software.
 0: SPI1 peripheral clocks disabled (default after reset)
 1: SPI1 peripheral clocks enabled:
 The peripheral clocks of the SPI1 are: the kernel clock selected by I2S123SRC and provided to **spi_ker_ck** input, and the **rcc_pclk2** bus interface clock.
- Bits 11:6 Reserved, must be kept at reset value.

Bit 5 USART6EN: USART6 Peripheral Clocks Enable

Set and reset by software.

0: USART6 peripheral clocks disabled (default after reset)

1: USART6 peripheral clocks enabled:

The peripheral clocks of the USART6 are: the kernel clock selected by USART16SEL and provided to **usart_ker_ck** input, and the **rcc_pclk2** bus interface clock.

Bit 4 USART1EN: USART1 Peripheral Clocks Enable

Set and reset by software.

0: USART1 peripheral clocks disabled (default after reset)

1: USART1 peripheral clocks enabled:

The peripheral clocks of the USART1 are: the kernel clock selected by USART16SEL and provided to **usart_ker_ck** input, and the **rcc_pclk2** bus interface clock.

Bits 3:2 Reserved, must be kept at reset value.

Bit 1 TIM8EN: TIM8 peripheral clock enable

Set and reset by software.

0: TIM8 peripheral clock disabled (default after reset)

1: TIM8 peripheral clock enabled

Bit 0 TIM1EN: TIM1 peripheral clock enable

Set and reset by software.

0: TIM1 peripheral clock disabled (default after reset)

1: TIM1 peripheral clock enabled

9.7.47 RCC APB4 clock register (RCC_APB4ENR)

A peripheral can be allocated (enabled) by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 78. RCC_APB4ENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB4ENR	0x0F4	0x0001 0000
RCC_C1_APB4ENR	0x154	
RCC_C2_APB4ENR	0x1B4	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4EN	Res.	Res.	Res.	Res.	RTCAPBEN
										rw					rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VREFEN	COMP12EN	Res.	LPTIM5EN	LPTIM4EN	LPTIM3EN	LPTIM2EN	Res.	I2C4EN	Res.	SPI6EN	Res.	LPUART1EN	Res.	SYSCFGEN	Res.
rw	rw		rw	rw	rw	rw		rw		rw		rw		rw	

Bits 31:26 Reserved, must be kept at reset value.

Bits 25:22 Reserved, must be kept at reset value.

Bit 21 **SAI4EN**: SAI4 Peripheral Clocks Enable

Set and reset by software.

0: SAI4 peripheral clocks disabled (default after reset)

1: SAI4 peripheral clocks enabled

The peripheral clocks of the SAI4 are: the kernel clocks selected by SAI4ASEL and SAI4BSEL, and provided to **sai_a_ker_ck** and **sai_b_ker_ck** inputs respectively, and the **pclk1d3** bus interface clock.

Bits 20:17 Reserved, must be kept at reset value.

Bit 16 **RTCAPBEN**: RTC APB Clock Enable

Set and reset by software.

0: The register clock interface of the RTC (APB) is disabled

1: The register clock interface of the RTC (APB) is enabled (default after reset)

Bit 15 **VREFEN**: VREFBUF peripheral clock enable

Set and reset by software.

0: VREFBUF peripheral clock disabled (default after reset)

1: VREFBUF peripheral clock enabled

Bit 14 **COMP12EN**: COMP1/2 peripheral clock enable

Set and reset by software.

0: COMP1/2 peripheral clock disabled (default after reset)

1: COMP1/2 peripheral clock enabled

Bit 13 Reserved, must be kept at reset value.

Bit 12 **LPTIM5EN**: LPTIM5 Peripheral Clocks Enable

Set and reset by software.

0: LPTIM5 peripheral clocks disabled (default after reset)

1: LPTIM5 peripheral clocks enabled

The peripheral clocks of the LPTIM5 are: the kernel clock selected by LPTIM345SEL and provided to **lptim_ker_ck** input, and the **pclk1d3** bus interface clock.

Bit 11 **LPTIM4EN**: LPTIM4 Peripheral Clocks Enable

Set and reset by software.

0: LPTIM4 peripheral clocks disabled (default after reset)

1: LPTIM4 peripheral clocks enabled

The peripheral clocks of the LPTIM4 are: the kernel clock selected by LPTIM345SEL and provided to **lptim_ker_ck** input, and the **pclk1d3** bus interface clock.

Bit 10 **LPTIM3EN**: LPTIM3 Peripheral Clocks Enable

Set and reset by software.

0: LPTIM3 peripheral clocks disabled (default after reset)

1: LPTIM3 peripheral clocks enabled

The peripheral clocks of the LPTIM3 are: the kernel clock selected by LPTIM345SEL and provided to **lptim_ker_ck** input, and the **pclk1d3** bus interface clock.

Bit 9 **LPTIM2EN**: LPTIM2 Peripheral Clocks Enable

Set and reset by software.

0: LPTIM2 peripheral clocks disabled (default after reset)

1: LPTIM2 peripheral clocks enabled

The peripheral clocks of the LPTIM2 are: the kernel clock selected by LPTIM2SEL and provided to **lptim_ker_ck** input, and the **pclk1d3** bus interface clock.

Bit 8 Reserved, must be kept at reset value.

Bit 7 **I2C4EN**: I2C4 Peripheral Clocks Enable

Set and reset by software.

0: I2C4 peripheral clocks disabled (default after reset)

1: I2C4 peripheral clocks enabled

The peripheral clocks of the I2C4 are: the kernel clock selected by I2C4SEL and provided to **i2c_ker_ck** input, and the **pclk1d3** bus interface clock.

Bit 6 Reserved, must be kept at reset value.

Bit 5 **SPI6EN**: SPI6 Peripheral Clocks Enable

Set and reset by software.

0: SPI6 peripheral clocks disabled (default after reset)

1: SPI6 peripheral clocks enabled

The peripheral clocks of the SPI6 are: the kernel clock selected by SPI6SEL and provided to **spi_ker_ck** input, and the **pclk1d3** bus interface clock.

Bit 4 Reserved, must be kept at reset value.

Bit 3 **LPUART1EN**: LPUART1 Peripheral Clocks Enable

Set and reset by software.

0: LPUART1 peripheral clocks disabled (default after reset)

1: LPUART1 peripheral clocks enabled

The peripheral clocks of the LPUART1 are: the kernel clock selected by LPUART1SEL and provided to **lpuart_ker_ck** input, and the **pclk1d3** bus interface clock.

Bit 2 Reserved, must be kept at reset value.

Bit 1 **SYSCFGEN**: SYSCFG peripheral clock enable

Set and reset by software.

0: SYSCFG peripheral clock disabled (default after reset)

1: SYSCFG peripheral clock enabled

Bit 0 Reserved, must be kept at reset value.

9.7.48 RCC AHB3 Sleep clock register (RCC_AHB3LPENR)

Peripheral clocks can be enabled during CPU sleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 79. RCC_AHB3LPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_AHB3LPENR	0x0FC	0xF001 5131
RCC_C1_AHB3LPENR	0x15C	
RCC_C2_AHB3LPENR	0x1BC	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AXISRAMLLEN	ITCMLLEN	DTCM2LPEN	DTCM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1LPEN
rw	rw	rw	rw												rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	QSPILLEN	Res.	FMCLLEN	Res.	Res.	Res.	FLITFPEN	Res.	Res.	JPGDECLLEN	DMA2DLPEN	Res.	Res.	Res.	MDMALLEN
	rw		rw				rw			rw	rw				rw

Bit 31 AXISRAMLLEN: AXISRAM Block Clock Enable During CSleep mode

Set and reset by software.

0: AXISRAM interface clock disabled during CSleep mode

1: AXISRAM interface clock enabled during CSleep mode (default after reset)

Bit 30 ITCMLLEN: D1ITCM Block Clock Enable During CSleep mode

Set and reset by software.

0: D1 ITCM interface clock disabled during CSleep mode

1: D1 ITCM interface clock enabled during CSleep mode (default after reset)

Bit 29 DTCM2LPEN: D1 DTCM2 Block Clock Enable During CSleep mode

Set and reset by software.

0: D1 DTCM2 interface clock disabled during CSleep mode

1: D1 DTCM2 interface clock enabled during CSleep mode (default after reset)

Bit 28 D1DTCM1LPEN: D1DTCM1 Block Clock Enable During CSleep mode

Set and reset by software.

0: D1DTCM1 interface clock disabled during CSleep mode

1: D1DTCM1 interface clock enabled during CSleep mode (default after reset)

Bits 27:17 Reserved, must be kept at reset value.

Bit 16 SDMMC1LPEN: SDMMC1 and SDMMC1 Delay Clock Enable During CSleep Mode

Set and reset by software.

0: SDMMC1 and SDMMC1 Delay clock disabled during CSleep mode

1: SDMMC1 and SDMMC1 Delay clock enabled during CSleep mode (default after reset)

Bit 15 Reserved, must be kept at reset value.

Bit 14 **QSPILPEN**: QUADSPI and QUADSPI Delay Clock Enable During CSleep Mode

Set and reset by software.

0: QUADSPI and QUADSPI Delay clock disabled during CSleep mode

1: QUADSPI and QUADSPI Delay clock enabled during CSleep mode (default after reset)

Bit 13 Reserved, must be kept at reset value.

Bit 12 **FMCLPEN**: FMC Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: FMC peripheral clocks disabled during CSleep mode

1: FMC peripheral clocks enabled during CSleep mode (default after reset):

The peripheral clocks of the FMC are: the kernel clock selected by FMCSEL and provided to `fmc_ker_ck` input, and the **rcc_hclk3** bus interface clock.

Bits 11:9 Reserved, must be kept at reset value.

Bit 8 **FLITFLPEN**: FLITF Clock Enable During CSleep Mode

Set and reset by software.

0: FLITF clock disabled during CSleep mode

1: FLITF clock enabled during CSleep mode (default after reset)

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **JPGDECLPEN**: JPGDEC Clock Enable During CSleep Mode

Set and reset by software.

0: JPGDEC peripheral clock disabled during CSleep mode

1: JPGDEC peripheral clock enabled during CSleep mode (default after reset)

Bit 4 **DMA2DLPEN**: DMA2D Clock Enable During CSleep Mode

Set and reset by software.

0: DMA2D peripheral clock disabled during CSleep mode

1: DMA2D peripheral clock enabled during CSleep mode (default after reset)

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 **MDMALPEN**: MDMA Clock Enable During CSleep Mode

Set and reset by software.

0: MDMA peripheral clock disabled during CSleep mode

1: MDMA peripheral clock enabled during CSleep mode (default after reset)

9.7.49 RCC AHB1 Sleep clock register (RCC_AHB1LPENR)

Peripheral clocks can be enabled during CPU CSleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 80. RCC_AHB1LPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_AHB1LPENR	0x100	0x1E03 C023
RCC_C1_AHB1LPENR	0x160	
RCC_C2_AHB1LPENR	0x1C0	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	USB2OTGHSULPILPEN	USB2OTGHSLPEN	USB1OTGHSULPILPEN	USB1OTGSLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH1RXLPEN	ETH1TXLPEN
			rw	rw	rw	rw								rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETH1MACLPEN	ARTLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC12LPEN	Res.	Res.	Res.	DMA2LPEN	DMA1LPEN
rw	rw									rw				rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **USB2OTGHSULPILPEN**: USB_PHY2 clock enable during CSleep mode

Set and reset by software.

0: USB_PHY2 peripheral clock disabled during CSleep mode

1: USB_PHY2 peripheral clock enabled during CSleep mode (default after reset)

Note: If the application enters Sleep mode, this bit must be cleared to avoid USB communication failure.

Bit 27 **USB2OTGHSLPEN**: USB2OTG (OTG_HS2) peripheral clock enable during CSleep mode

Set and reset by software.

0: USB2OTG peripheral clocks disabled during CSleep mode

1: USB2OTG peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the USB2OTG are: the kernel clock selected by USBSEL and the **rcc_hclk1** bus interface clock.

Bit 26 **USB1OTGHSULPILPEN**: USB_PHY1 clock enable during CSleep mode

Set and reset by software.

0: USB_PHY1 peripheral clock disabled during CSleep mode

1: USB_PHY1 peripheral clock enabled during CSleep mode (default after reset)

Bit 25 **USB1OTGHSLPEN**: USB1OTG (OTG_HS1) peripheral clock enable during CSleep mode

Set and reset by software.

0: USB1OTG peripheral clock disabled during CSleep mode

1: USB1OTG peripheral clock enabled during CSleep mode (default after reset)

The peripheral clocks of the USB1OTG are: the kernel clock selected by USBSEL and the **rcc_hclk1** bus interface clock.

Bits 24:18 Reserved, must be kept at reset value.

Bit 17 **ETH1RXLPEN**: Ethernet Reception Clock Enable During CSleep Mode

Set and reset by software.

0: Ethernet Reception clock disabled during CSleep mode

1: Ethernet Reception clock enabled during CSleep mode (default after reset)

Bit 16 **ETH1TXLPEN**: Ethernet Transmission Clock Enable During CSleep Mode

Set and reset by software.

0: Ethernet Transmission clock disabled during CSleep mode

1: Ethernet Transmission clock enabled during CSleep mode (default after reset)

Bit 15 **ETH1MACLPEN**: Ethernet MAC bus interface Clock Enable During CSleep Mode

Set and reset by software.

0: Ethernet MAC bus interface clock disabled during CSleep mode

1: Ethernet MAC bus interface clock enabled during CSleep mode (default after reset)

Bit 14 **ARTLPEN**: ART Clock Enable During CSleep Mode

Set and reset by software.

0: ART clock disabled during CSleep mode

1: ART clock enabled during CSleep mode (default after reset)

Bits 13:6 Reserved, must be kept at reset value.

Bit 5 **ADC12LPEN**: ADC1/2 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: ADC1/2 peripheral clocks disabled during CSleep mode

1: ADC1/2 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the ADC1 and 2 are: the kernel clock selected by ADCSEL and provided to **adc_ker_ck_input**, and the **rcc_hclk1** bus interface clock.

Bits 4:2 Reserved, must be kept at reset value.

Bit 1 **DMA2LPEN**: DMA2 Clock Enable During CSleep Mode

Set and reset by software.

0: DMA2 clock disabled during CSleep mode

1: DMA2 clock enabled during CSleep mode (default after reset)

Bit 0 **DMA1LPEN**: DMA1 Clock Enable During CSleep Mode

Set and reset by software.

0: DMA1 clock disabled during CSleep mode

1: DMA1 clock enabled during CSleep mode (default after reset)

9.7.50 RCC AHB2 Sleep clock register (RCC_AHB2LPENR)

Peripheral clocks can be enabled during CPU CSleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 81. RCC_AHB2LPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_AHB2LPENR	0x104	0xE000 0271
RCC_C1_AHB2LPENR	0x164	
RCC_C2_AHB2LPENR	0x1C4	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAM3LPEN	SRAM2LPEN	SRAM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw	rw	rw													
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2LPEN	Res.	Res.	RNGLPEN	HASHLPEN	CRYPTLPEN	Res.	Res.	Res.	CAMITFLPEN
						rw			rw	rw	rw				rw

Bit 31 **SRAM3LPEN**: SRAM3 Clock Enable During CSleep Mode

Set and reset by software.

0: SRAM3 clock disabled during CSleep mode

1: SRAM3 clock enabled during CSleep mode (default after reset)

Bit 30 **SRAM2LPEN**: SRAM2 Clock Enable During CSleep Mode

Set and reset by software.

0: SRAM2 clock disabled during CSleep mode

1: SRAM2 clock enabled during CSleep mode (default after reset)

Bit 29 **SRAM1LPEN**: SRAM1 Clock Enable During CSleep Mode

Set and reset by software.

0: SRAM1 clock disabled during CSleep mode

1: SRAM1 clock enabled during CSleep mode (default after reset)

Bits 28:10 Reserved, must be kept at reset value.

Bit 9 **SDMMC2LPEN**: SDMMC2 and SDMMC2 Delay Clock Enable During CSleep Mode

Set and reset by software.

0: SDMMC2 and SDMMC2 Delay clock disabled during CSleep mode

1: SDMMC2 and SDMMC2 Delay clock enabled during CSleep mode (default after reset)

Bits 8:7 Reserved, must be kept at reset value.

Bit 6 **RNGLPEN**: RNG peripheral clock enable during CSleep mode

Set and reset by software.

0: RNG peripheral clocks disabled during CSleep mode

1: RNG peripheral clock enabled during CSleep mode (default after reset)

The peripheral clocks of the RNG are: the kernel clock selected by RNGSEL and provided to **rng_ker_ck** input, and the **rcc_hclk2** bus interface clock.

Bit 5 **HASHLPEN**: HASH peripheral clock enable during CSleep mode

Set and reset by software.

0: HASH peripheral clock disabled during CSleep mode

1: HASH peripheral clock enabled during CSleep mode (default after reset)

Bit 4 **CRYPTLPEN**: CRYPT peripheral clock enable during CSleep mode

Set and reset by software.

0: CRYPT peripheral clock disabled during CSleep mode

1: CRYPT peripheral clock enabled during CSleep mode (default after reset)

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 **DCMILPEN**: DCMI peripheral clock enable during CSleep mode

Set and reset by software.

0: DCMI peripheral clock disabled during CSleep mode

1: DCMI peripheral clock enabled during CSleep mode (default after reset)

9.7.51 RCC AHB4 Sleep clock register (RCC_AHB4LPENR)

Peripheral clocks can be enabled during CPU CSleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 82. RCC_AHB4LPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_AHB4LPENR	0x108	0x3128 07FF
RCC_C1_AHB4LPENR	0x168	
RCC_C2_AHB4LPENR	0x1C8	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	SRAM4LPEN	BKPRAMLLEN	Res.	Res.	Res.	ADC3LPEN	Res.	Res.	BDMALPEN	Res.	CRCLPEN	Res.	Res.	Res.
		rw	rw				rw			rw		rw			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	GPIOKLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN	GPIODLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 SRAM4LPEN: SRAM4 Clock Enable During CSleep Mode

Set and reset by software.

0: SRAM4 clock disabled during CSleep mode

1: SRAM4 clock enabled during CSleep mode (default after reset)

Bit 28 BKPRAMLLEN: Backup RAM Clock Enable During CSleep Mode

Set and reset by software.

0: Backup RAM clock disabled during CSleep mode

1: Backup RAM clock enabled during CSleep mode (default after reset)

Bits 27:25 Reserved, must be kept at reset value.

Bit 24 ADC3LPEN: ADC3 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: ADC3 peripheral clocks disabled during CSleep mode

1: ADC3 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the ADC3 are: the kernel clock selected by ADCSEL and provided to `adc_ker_ck_input`, and the `rcc_hclk4` bus interface clock.

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 BDMALPEN: BDMA Clock Enable During CSleep Mode

Set and reset by software.

0: BDMA clock disabled during CSleep mode

1: BDMA clock enabled during CSleep mode (default after reset)

Bit 20 Reserved, must be kept at reset value.

Bit 19 **CRCLPEN**: CRC peripheral clock enable during CSleep mode

Set and reset by software.

0: CRC peripheral clock disabled during CSleep mode

1: CRC peripheral clock enabled during CSleep mode (default after reset)

Bits 18:11 Reserved, must be kept at reset value.

Bit 10 **GPIOKLPEN**: GPIOK peripheral clock enable during CSleep mode

Set and reset by software.

0: GPIOK peripheral clock disabled during CSleep mode

1: GPIOK peripheral clock enabled during CSleep mode (default after reset)

Bit 9 **GPIOJLPEN**: GPIOJ peripheral clock enable during CSleep mode

Set and reset by software.

0: GPIOJ peripheral clock disabled during CSleep mode

1: GPIOJ peripheral clock enabled during CSleep mode (default after reset)

Bit 8 **GPIOILPEN**: GPIOI peripheral clock enable during CSleep mode

Set and reset by software.

0: GPIOI peripheral clock disabled during CSleep mode

1: GPIOI peripheral clock enabled during CSleep mode (default after reset)

Bit 7 **GPIOHLPEN**: GPIOH peripheral clock enable during CSleep mode

Set and reset by software.

0: GPIOH peripheral clock disabled during CSleep mode

1: GPIOH peripheral clock enabled during CSleep mode (default after reset)

Bit 6 **GPIOGLPEN**: GPIOG peripheral clock enable during CSleep mode

Set and reset by software.

0: GPIOG peripheral clock disabled during CSleep mode

1: GPIOG peripheral clock enabled during CSleep mode (default after reset)

Bit 5 **GPIOFLPEN**: GPIOF peripheral clock enable during CSleep mode

Set and reset by software.

0: GPIOF peripheral clock disabled during CSleep mode

1: GPIOF peripheral clock enabled during CSleep mode (default after reset)

Bit 4 **GPIOELPEN**: GPIOE peripheral clock enable during CSleep mode

Set and reset by software.

0: GPIOE peripheral clock disabled during CSleep mode

1: GPIOE peripheral clock enabled during CSleep mode (default after reset)

Bit 3 **GPIODLPEN**: GPIOD peripheral clock enable during CSleep mode

Set and reset by software.

0: GPIOD peripheral clock disabled during CSleep mode

1: GPIOD peripheral clock enabled during CSleep mode (default after reset)

- Bit 2 **GPIOCLPEN**: GPIOC peripheral clock enable during CSleep mode
Set and reset by software.
0: GPIOC peripheral clock disabled during CSleep mode
1: GPIOC peripheral clock enabled during CSleep mode (default after reset)
- Bit 1 **GPIOBLPEN**: GPIOB peripheral clock enable during CSleep mode
Set and reset by software.
0: GPIOB peripheral clock disabled during CSleep mode
1: GPIOB peripheral clock enabled during CSleep mode (default after reset)
- Bit 0 **GPIOALPEN**: GPIOA peripheral clock enable during CSleep mode
Set and reset by software.
0: GPIOA peripheral clock disabled during CSleep mode
1: GPIOA peripheral clock enabled during CSleep mode (default after reset)

9.7.52 RCC APB3 Sleep clock register (RCC_APB3LPENR)

Peripheral clocks can be enabled during CPU CSleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 83. RCC_APB3LPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB3LPENR	0x10C	0x0000 0058
RCC_C1_APB3LPENR	0x16C	
RCC_C2_APB3LPENR	0x1CC	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WWDG1LPEN	Res.	DSILPEN	LTDCLPEN	Res.	Res.	Res.
									rw		rw	rw			

Bits 31:5 Reserved, must be kept at reset value.

Bit 6 WWDG1LPEN: WWDG1 Clock Enable During CSleep Mode

Set and reset by software.

0: WWDG1 clock disable during CSleep mode

1: WWDG1 clock enabled during CSleep mode (default after reset)

When accessing this bit via RCC_APB3LPENR, only the CPU1 is allowed to control the low-power clock for WWDG1.

There is no protection to prevent the CPU2 to set the bit WWDG1LPEN to '1'.

It is not recommended to enable the WWDG1 clock for CPU2 operating modes in RCC_C2_APB3LPENR register.

Bit 4 DSILPEN: DSI peripheral clock enable during CSleep mode

Set and reset by software.

0: DSI clock disabled during CSleep mode

1: DSI clock enabled during CSleep mode (default after reset)

Bit 3 LTDCLPEN: LTDC peripheral clock enable during CSleep mode

Provides the pixel clock (**ltdc_ker_ck**) to both DSI and LTDC

Set and reset by software.

0: LTDC clock disabled during CSleep mode

1: LTDC clock provided to DSI and LTDC during CSleep mode (default after reset)

Bits 2:0 Reserved, must be kept at reset value.

9.7.53 RCC APB1 Low Sleep clock register (RCC_APB1LLPENR)

Peripheral clocks can be enabled during CPU CSleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 84. RCC_APB1LLPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB1LLPENR	0x110	0xE8FF CBFF
RCC_C1_APB1LLPENR	0x170	
RCC_C2_APB1LLPENR	0x1D0	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UART8LPEN	UART7LPEN	DAC12LPEN	Res.	CECLPEN	Res.	Res.	Res.	I2C3LPEN	I2C2LPEN	I2C1LPEN	UART5LPEN	UART4LPEN	USART3LPEN	USART2LPEN	SPDIFRXLLEN
r/w	r/w	r/w		r/w				r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3LPEN	SPI2LPEN	Res.	Res.	WWDG2LPEN	Res.	LPTIM1LPEN	TIM14LPEN	TIM13LPEN	TIM12LPEN	TIM7LPEN	TIM6LPEN	TIM5LPEN	TIM4LPEN	TIM3LPEN	TIM2LPEN
r/w	r/w			r/w		r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bit 31 UART8LPEN: UART8 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: UART8 peripheral clocks disabled during CSleep mode

1: UART8 peripheral clocks enabled during CSleep mode (default after reset):

The peripheral clocks of the UART8 are: the kernel clock selected by USART234578SEL and provided to usart_ker_ck input, and the **pclk1d2** bus interface clock.

Bit 30 UART7LPEN: UART7 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: UART7 peripheral clocks disabled during CSleep mode

1: UART7 peripheral clocks enabled during CSleep mode (default after reset):

The peripheral clocks of the UART7 are: the kernel clock selected by USART234578SEL and provided to usart_ker_ck input, and the **pclk1d2** bus interface clock.

Bit 29 DAC12LPEN: DAC1/2 peripheral clock enable during CSleep mode

Set and reset by software.

0: DAC1/2 peripheral clock disabled during CSleep mode

1: DAC1/2 peripheral clock enabled during CSleep mode (default after reset)

Bit 28 Reserved, must be kept at reset value.

Bit 27 **CECLPEN**: HDMI-CEC Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: HDMI-CEC peripheral clocks disabled during CSleep mode

1: HDMI-CEC peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the HDMI-CEC are: the kernel clock selected by CECSEL and provided to **cec_ker_ck** input, and the **pclk1d2** bus interface clock.

Bits 26:24 Reserved, must be kept at reset value.

Bit 23 **I2C3LPEN**: I2C3 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: I2C3 peripheral clocks disabled during CSleep mode

1: I2C3 peripheral clocks enabled during CSleep mode (default after reset):

The peripheral clocks of the I2C3 are: the kernel clock selected by I2C123SEL and provided to **i2c_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 22 **I2C2LPEN**: I2C2 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: I2C2 peripheral clocks disabled during CSleep mode

1: I2C2 peripheral clocks enabled during CSleep mode (default after reset):

The peripheral clocks of the I2C2 are: the kernel clock selected by I2C123SEL and provided to **i2c_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 21 **I2C1LPEN**: I2C1 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: I2C1 peripheral clocks disabled during CSleep mode

1: I2C1 peripheral clocks enabled during CSleep mode (default after reset):

The peripheral clocks of the I2C1 are: the kernel clock selected by I2C123SEL and provided to **i2c_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 20 **UART5LPEN**: UART5 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: UART5 peripheral clocks disabled during CSleep mode

1: UART5 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the UART5 are: the kernel clock selected by USART234578SEL and provided to **uart_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 19 **UART4LPEN**: UART4 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: UART4 peripheral clocks disabled during CSleep mode

1: UART4 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the UART4 are: the kernel clock selected by USART234578SEL and provided to **uart_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 18 **USART3LPEN**: USART3 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: USART3 peripheral clocks disabled during CSleep mode

1: USART3 peripheral clocks enabled during CSleep mode (default after reset):

The peripheral clocks of the USART3 are: the kernel clock selected by USART234578SEL and provided to **usart_ker_ck** input, and the **pclk1d2** bus interface clock.

- Bit 17 **USART2LPEN**: USART2 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: USART2 peripheral clocks disabled during CSleep mode
1: USART2 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the USART2 are: the kernel clock selected by USART234578SEL and provided to **usart_ker_ck** input, and the **pclk1d2** bus interface clock.
- Bit 16 **SPDIFRXLPEN**: SPDIFRX Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: SPDIFRX peripheral clocks disabled during CSleep mode
1: SPDIFRX peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the SPDIFRX are: the kernel clock selected by SPDIFSEL and provided to **spdifrx_ker_ck** input, and the **pclk1d2** bus interface clock.
- Bit 15 **SPI3LPEN**: SPI3 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: SPI3 peripheral clocks disabled during CSleep mode
1: SPI3 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the SPI3 are: the kernel clock selected by I2S123SRC and provided to **spi_ker_ck** input, and the **pclk1d2** bus interface clock.
- Bit 14 **SPI2LPEN**: SPI2 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: SPI2 peripheral clocks disabled during CSleep mode
1: SPI2 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the SPI2 are: the kernel clock selected by I2S123SRC and provided to **spi_ker_ck** input, and the **pclk1d2** bus interface clock.
- Bits 13:10 Reserved, must be kept at reset value.
- Bit 11 **WWDG2LPEN**: WWDG2 Clock Enable During CSleep Mode
Set and reset by software.
0: WWDG2 clock disabled during CSleep mode
1: WWDG2 clock enabled during CSleep mode (default after reset)
When accessing this bit via RCC_APB1LLPENR register address, only the CPU2 is allowed to control the low-power clock for WWDG2.
There is no protection to prevent the CPU1 to set the WWDG2LPEN bit to '1'.
It is not recommended to enable the WWDG2 clock for CPU1 operating modes in RCC_C1_APB3LPENR register.
- Bit 10 Reserved, must be kept at reset value.
- Bit 9 **LPTIM1LPEN**: LPTIM1 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: LPTIM1 peripheral clocks disabled during CSleep mode
1: LPTIM1 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the LPTIM1 are: the kernel clock selected by LPTIM1SEL and provided to **lptim_ker_ck** input, and the **pclk1d2** bus interface clock.
- Bit 8 **TIM14LPEN**: TIM14 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM14 peripheral clock disabled during CSleep mode
1: TIM14 peripheral clock enabled during CSleep mode (default after reset)

- Bit 7 **TIM13LPEN**: TIM13 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM13 peripheral clock disabled during CSleep mode
1: TIM13 peripheral clock enabled during CSleep mode (default after reset)
- Bit 6 **TIM12LPEN**: TIM12 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM12 peripheral clock disabled during CSleep mode
1: TIM12 peripheral clock enabled during CSleep mode (default after reset)
- Bit 5 **TIM7LPEN**: TIM7 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM7 peripheral clock disabled during CSleep mode
1: TIM7 peripheral clock enabled during CSleep mode (default after reset)
- Bit 4 **TIM6LPEN**: TIM6 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM6 peripheral clock disabled during CSleep mode
1: TIM6 peripheral clock enabled during CSleep mode (default after reset)
- Bit 3 **TIM5LPEN**: TIM5 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM5 peripheral clock disabled during CSleep mode
1: TIM5 peripheral clock enabled during CSleep mode (default after reset)
- Bit 2 **TIM4LPEN**: TIM4 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM4 peripheral clock disabled during CSleep mode
1: TIM4 peripheral clock enabled during CSleep mode (default after reset)
- Bit 1 **TIM3LPEN**: TIM3 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM3 peripheral clock disabled during CSleep mode
1: TIM3 peripheral clock enabled during CSleep mode (default after reset)
- Bit 0 **TIM2LPEN**: TIM2 peripheral clock enable during CSleep mode
Set and reset by software.
0: TIM2 peripheral clock disabled during CSleep mode
1: TIM2 peripheral clock enabled during CSleep mode (default after reset)

9.7.54 RCC APB1 High Sleep clock register (RCC_APB1HLPENR)

Peripheral clocks can be enabled during CPU CSleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 85. RCC_APB1HLPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB1HLPENR	0x114	0x0000 0136
RCC_C1_APB1HLPENR	0x174	
RCC_C2_APB1HLPENR	0x1D4	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANLPEN	Res.	Res.	MDIOSLPEN	OPAMPLPEN	Res.	SWPLPEN	CRSLPEN	Res.
							rw			rw	rw		rw	rw	

Bits 31:9 Reserved, must be kept at reset value.

Bit 8 **FDCANLPEN**: FDCAN Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: FDCAN peripheral clocks disabled during CSleep mode

1: FDCAN peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the FDCAN are: the kernel clock selected by FDCANSEL and provided to **fdcan_ker_ck** input, and the **pclk1d2** bus interface clock.

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **MDIOSLPEN**: MDIOS peripheral clock enable during CSleep mode

Set and reset by software.

0: MDIOS peripheral clock disabled during CSleep mode

1: MDIOS peripheral clock enabled during CSleep mode (default after reset)

Bit 4 **OPAMPLPEN**: OPAMP peripheral clock enable during CSleep mode

Set and reset by software.

0: OPAMP peripheral clock disabled during CSleep mode

1: OPAMP peripheral clock enabled during CSleep mode (default after reset)

Bit 3 Reserved, must be kept at reset value.

Bit 2 **SWPLPEN**: SWPMI Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: SWPMI peripheral clocks disabled during CSleep mode

1: SWPMI peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the SWPMI are: the kernel clock selected by SWPSEL and provided to **swpmi_ker_ck** input, and the **pclk1d2** bus interface clock.

Bit 1 **CRSLPEN**: Clock Recovery System peripheral clock enable during CSleep mode

Set and reset by software.

0: CRS peripheral clock disabled during CSleep mode

1: CRS peripheral clock enabled during CSleep mode (default after reset)

Bit 0 Reserved, must be kept at reset value.

9.7.55 RCC APB2 Sleep clock register (RCC_APB2LPENR)

Peripheral clocks can be enabled during CPU CSleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 86. RCC_APB2LPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB2LPENR	0x118	0x31D7 3033
RCC_C1_APB2LPENR	0x178	
RCC_C2_APB2LPENR	0x1D8	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	HRTIMLPEN	DFSDM1LPEN	Res.	Res.	Res.	SAI3LPEN	SAI2LPEN	SAI1LPEN	Res.	SPI5LPEN	Res.	TIM17LPEN	TIM16LPEN	TIM15LPEN
		rw	rw				rw	rw	rw		rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	SPI4LPEN	SPI1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	USART6LPEN	USART1LPEN	Res.	Res.	TIM8LPEN	TIM1LPEN
		rw	rw							rw	rw			rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **HRTIMLPEN**: HRTIM peripheral clock enable during CSleep mode

Set and reset by software.

0: HRTIM peripheral clock disabled during CSleep mode

1: HRTIM peripheral clock enabled during CSleep mode (default after reset)

Bit 28 **DFSDM1LPEN**: DFSDM1 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: DFSDM1 peripheral clocks disabled during CSleep mode

1: DFSDM1 peripheral clocks enabled during CSleep mode (default after reset)

DFSDM1 peripheral clocks are: the kernel clocks selected by SAI1SEL and DFSDM1SEL and provided to **Aclk** and **clk** inputs respectively, and the **rcc_pclk2** bus interface clock.

Bits 27:25 Reserved, must be kept at reset value.

Bit 24 **SAI3LPEN**: SAI3 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: SAI3 peripheral clocks disabled during CSleep mode

1: SAI3 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the SAI3 are: the kernel clock selected by SAI23SEL and provided to **sai_a_ker_ck** and **sai_b_ker_ck** inputs, and the **rcc_pclk2** bus interface clock.

- Bit 23 **SAI2LPEN**: SAI2 Peripheral Clocks Enable During CSleep Mode
 Set and reset by software.
 0: SAI2 peripheral clocks disabled during CSleep mode
 1: SAI2 peripheral clocks enabled during CSleep mode (default after reset)
 The peripheral clocks of the SAI2 are: the kernel clock selected by SAI23SEL and provided to **sai_a_ker_ck** and **sai_b_ker_ck** inputs, and the **rcc_pclk2** bus interface clock.
- Bit 22 **SAI1LPEN**: SAI1 Peripheral Clocks Enable During CSleep Mode
 Set and reset by software.
 0: SAI1 peripheral clocks disabled during CSleep mode
 1: SAI1 peripheral clocks enabled during CSleep mode (default after reset)
 The peripheral clocks of the SAI1 are: the kernel clock selected by SAI1SEL and provided to **sai_a_ker_ck** and **sai_b_ker_ck** inputs, and the **rcc_pclk2** bus interface clock.
- Bit 21 Reserved, must be kept at reset value.
- Bit 20 **SPI5LPEN**: SPI5 Peripheral Clocks Enable During CSleep Mode
 Set and reset by software.
 0: SPI5 peripheral clocks disabled during CSleep mode
 1: SPI5 peripheral clocks enabled during CSleep mode (default after reset)
 The peripheral clocks of the SPI5 are: the kernel clock selected by SPI45SEL and provided to **spi_ker_ck** input, and the **rcc_pclk2** bus interface clock.
- Bit 19 Reserved, must be kept at reset value.
- Bit 18 **TIM17LPEN**: TIM17 peripheral clock enable during CSleep mode
 Set and reset by software.
 0: TIM17 peripheral clock disabled during CSleep mode
 1: TIM17 peripheral clock enabled during CSleep mode (default after reset)
- Bit 17 **TIM16LPEN**: TIM16 peripheral clock enable during CSleep mode
 Set and reset by software.
 0: TIM16 peripheral clock disabled during CSleep mode
 1: TIM16 peripheral clock enabled during CSleep mode (default after reset)
- Bit 16 **TIM15LPEN**: TIM15 peripheral clock enable during CSleep mode
 Set and reset by software.
 0: TIM15 peripheral clock disabled during CSleep mode
 1: TIM15 peripheral clock enabled during CSleep mode (default after reset)
- Bits 15:14 Reserved, must be kept at reset value.
- Bit 13 **SPI4LPEN**: SPI4 Peripheral Clocks Enable During CSleep Mode
 Set and reset by software.
 0: SPI4 peripheral clocks disabled during CSleep mode
 1: SPI4 peripheral clocks enabled during CSleep mode (default after reset)
 The peripheral clocks of the SPI4 are: the kernel clock selected by SPI45SEL and provided to **spi_ker_ck** input, and the **rcc_pclk2** bus interface clock.
- Bit 12 **SPI1LPEN**: SPI1 Peripheral Clocks Enable During CSleep Mode
 Set and reset by software.
 0: SPI1 peripheral clocks disabled during CSleep mode
 1: SPI1 peripheral clocks enabled during CSleep mode (default after reset)
 The peripheral clocks of the SPI1 are: the kernel clock selected by I2S123SRC and provided to **spi_ker_ck** input, and the **rcc_pclk2** bus interface clock.
- Bits 11:6 Reserved, must be kept at reset value.

Bit 5 USART6LPEN: USART6 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: USART6 peripheral clocks disabled during CSleep mode

1: USART6 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the USART6 are: the kernel clock selected by USART16SEL and provided to **usart_ker_ck** input, and the **rcc_pclk2** bus interface clock.

Bit 4 USART1LPEN: USART1 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: USART1 peripheral clocks disabled during CSleep mode

1: USART1 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the USART1 are: the kernel clock selected by USART16SEL and provided to **usart_ker_ck** inputs, and the **rcc_pclk2** bus interface clock.

Bits 3:2 Reserved, must be kept at reset value.

Bit 1 TIM8LPEN: TIM8 peripheral clock enable during CSleep mode

Set and reset by software.

0: TIM8 peripheral clock disabled during CSleep mode

1: TIM8 peripheral clock enabled during CSleep mode (default after reset)

Bit 0 TIM1LPEN: TIM1 peripheral clock enable during CSleep mode

Set and reset by software.

0: TIM1 peripheral clock disabled during CSleep mode

1: TIM1 peripheral clock enabled during CSleep mode (default after reset)

9.7.56 RCC APB4 Sleep clock register (RCC_APB4LPENR)

Peripheral clocks can be enabled during CPU CSleep mode by one or both CPUs. Please refer to [Section 9.5.10: Peripheral allocation](#) in order to get more information on peripheral allocation.

Table 87. RCC_APB4LPENR address offset and reset value

Register Name	Address Offset	Reset Value
RCC_APB4LPENR	0x11C	0x0421 DEAA
RCC_C1_APB4LPENR	0x17C	
RCC_C2_APB4LPENR	0x1DC	

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4LPEN	Res.	Res.	Res.	Res.	RTCAPBLPEN
										rw					rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VREFLPEN	COMP12LPEN	Res.	LPTIM5LPEN	LPTIM4LPEN	LPTIM3LPEN	LPTIM2LPEN	Res.	I2C4LPEN	Res.	SPI6LPEN	Res.	LPART1LPEN	Res.	SYSCFGLPEN	Res.
rw	rw		rw	rw	rw	rw		rw		rw		rw		rw	

Bits 31:26 Reserved, must be kept at reset value.

Bits 25:22 Reserved, must be kept at reset value.

Bit 21 SAI4LPEN: SAI4 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: SAI4 peripheral clocks disabled during CSleep mode

1: SAI4 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the SAI4 are: the kernel clocks selected by SAI4ASEL and SAI4BSEL, and provided to **sai_a_ker_ck** and **sai_b_ker_ck** inputs respectively, and the **pclk1d3** bus interface clock.

Bits 20:17 Reserved, must be kept at reset value.

Bit 16 RTCAPBLPEN: RTC APB Clock Enable During CSleep Mode

Set and reset by software.

0: The register clock interface of the RTC (APB) is disabled during CSleep mode

1: The register clock interface of the RTC (APB) is enabled during CSleep mode (default after reset)

Bit 15 VREFLPEN: VREF peripheral clock enable during CSleep mode

Set and reset by software.

0: VREF peripheral clock disabled during CSleep mode

1: VREF peripheral clock enabled during CSleep mode (default after reset)

- Bit 14 **COMP12LPEN**: COMP1/2 peripheral clock enable during CSleep mode
Set and reset by software.
0: COMP1/2 peripheral clock disabled during CSleep mode
1: COMP1/2 peripheral clock enabled during CSleep mode (default after reset)
- Bit 13 Reserved, must be kept at reset value.
- Bit 12 **LPTIM5LPEN**: LPTIM5 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: LPTIM5 peripheral clocks disabled during CSleep mode
1: LPTIM5 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the LPTIM5 are: the kernel clock selected by LPTIM345SEL and provided to **lptim_ker_ck** input, and the **pclk1d3** bus interface clock.
- Bit 11 **LPTIM4LPEN**: LPTIM4 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: LPTIM4 peripheral clocks disabled during CSleep mode
1: LPTIM4 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the LPTIM4 are: the kernel clock selected by LPTIM345SEL and provided to **lptim_ker_ck** input, and the **pclk1d3** bus interface clock.
- Bit 10 **LPTIM3LPEN**: LPTIM3 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: LPTIM3 peripheral clocks disabled during CSleep mode
1: LPTIM3 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the LPTIM3 are: the kernel clock selected by LPTIM345SEL and provided to **lptim_ker_ck** input, and the **pclk1d3** bus interface clock.
- Bit 9 **LPTIM2LPEN**: LPTIM2 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: LPTIM2 peripheral clocks disabled during CSleep mode
1: LPTIM2 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the LPTIM5 are: the kernel clock selected by LPTIM2SEL and provided to **lptim_ker_ck** input, and the **pclk1d3** bus interface clock.
- Bit 8 Reserved, must be kept at reset value.
- Bit 7 **I2C4LPEN**: I2C4 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: I2C4 peripheral clocks disabled during CSleep mode
1: I2C4 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the I2C4 are: the kernel clock selected by I2C4SEL and provided to **i2c_ker_ck** input, and the **pclk1d3** bus interface clock.
- Bit 6 Reserved, must be kept at reset value.
- Bit 5 **SPI6LPEN**: SPI6 Peripheral Clocks Enable During CSleep Mode
Set and reset by software.
0: SPI6 peripheral clocks disabled during CSleep mode
1: SPI6 peripheral clocks enabled during CSleep mode (default after reset)
The peripheral clocks of the SPI6 are: the kernel clock selected by SPI6SEL and provided to **spi_ker_ck** input, and the **pclk1d3** bus interface clock.
- Bit 4 Reserved, must be kept at reset value.

Bit 3 **LPUART1LPEN**: LPUART1 Peripheral Clocks Enable During CSleep Mode

Set and reset by software.

0: LPUART1 peripheral clocks disabled during CSleep mode

1: LPUART1 peripheral clocks enabled during CSleep mode (default after reset)

The peripheral clocks of the LPUART1 are: the kernel clock selected by LPUART1SEL and provided to **lpuart_ker_ck** input, and the **pclk1d3** bus interface clock.

Bit 2 Reserved, must be kept at reset value.

Bit 1 **SYSCFGLPEN**: SYSCFG peripheral clock enable during CSleep mode

Set and reset by software.

0: SYSCFG peripheral clock disabled during CSleep mode

1: SYSCFG peripheral clock enabled during CSleep mode (default after reset)

Bit 0 Reserved, must be kept at reset value.

9.8 RCC register map

Table 88. RCC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	RCC_CR	Res.	Res.	PLL3RDY	PLL3ON	PLL2RDY	PLL2ON	PLL1RDY	PLL1ON	Res.	Res.	Res.	Res.	HSECSSON	HSEBYP	HSERDY	HSEON	D2CKRDY	D1CKRDY	HSI48RDY	HSI48ON	Res.	Res.	CSIKERON	CSIRDY	CSION	Res.	HSIDIVF	HSIDIV[1:0]	HSIRDY	HSIKERON	HSION	
	Reset value			0	0	0	0	0	0					0	0	0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	1
0x004	RCC_HSI CFGR	Res.	HSITRIM[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HSICAL[11:0]												
	Reset value		1	0	0	0	0	0														X	X	X	X	X	X	X	X	X	X	X	X
0x008	RCC_CRR CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HSI48CAL[9:0]											
	Reset value																					X	X	X	X	X	X	X	X	X	X	X	X
0x00C	RCC_CSICFGR	Res.	Res.	CSITRIM[5:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSICAL[9:0]									
	Reset value			1	0	0	0	0	0																	X	X	X	X	X	X	X	X
0x010	RCC_CFGR	MCO2SEL[2:0]		MCO2PRE[3:0]				MCO1SEL[2:0]		MCO1PRE[3:0]				Res.		Res.	TIMPRE		HRTIMSEL		RTCPRE[5:0]				STOPKERWUICK		STOPWUICK	SWS[2:0]		SW[2:0]			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x014	reserved	Reserved																															
0x018	RCC_D1CFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D1CPRE[3:0]		Reserved		D1PPRE[2:0]		HPRE[3:0]					
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	
0x01C	RCC_D2CFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D2PPRE2[2:0]		Res.		D2PPRE1[2:0]		Res.		Res.			
	Reset value																					0	0	0	0	0	0	0					
0x020	RCC_D3CFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D3PPRE[2:0]		Res.		Res.		Res.		Res.		
	Reset value																						0	0	0	0							
0x024	reserved	Reserved																															
0x028	RCC_PL LCKSEL R	Res.	Res.	Res.	Res.	Res.	DIVM3[5:0]				Res.	Res.	DIVM2[5:0]				Res.	Res.	DIVM1[5:0]				Res.	Res.	Res.				Res.		PLLSRC[1:0]		
	Reset value						1	0	0	0	0	0	0				1	0	0	0	0	0			1	0	0	0	0	0			0

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x02C	RCC_PLLCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIVR3EN	DIVQ3EN	DIVP3EN	DIVR2EN	DIVQ2EN	DIVP2EN	DIVR1EN	DIVQ1EN	DIVP1EN	Res.	Res.	Res.	Res.	PLL3RGE[1:0]	PLL3VCOSEL	PLL3FRACEN	PLL2RGE[1:0]	PLL2VCOSEL	PLL2FRACEN	PLL1RGE[1:0]	PLL1VCOSEL	PLL1FRACEN			
	Reset value								1	1	1	1	1	1	1	1	1					0	0	0	0	0	0	0	0	0	0	0	0
0x030	RCC_PLL1DIVR	Res.	DIVR1[6:0]						Res.	DIVQ1[6:0]						DIVP1[6:0]						DIVN1[8:0]											
	Reset value		0	0	0	0	0	0	1		0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0
0x034	RCC_PLL1FRACR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FRACN1[12:0]										Res.	Res.	Res.			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x038	RCC_PLL2DIVR	Res.	DIVR2[6:0]						Res.	DIVQ2[6:0]						DIVP2[6:0]						DIVN2[8:0]											
	Reset value		0	0	0	0	0	0	1		0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0
0x03C	RCC_PLL2FRACR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FRACN2[12:0]										Res.	Res.	Res.			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x040	RCC_PLL3DIVR	Res.	DIVR3[6:0]						Res.	DIVQ3[6:0]						DIVP3[6:0]						DIVN3[8:0]											
	Reset value		0	0	0	0	0	0	1		0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0
0x044	RCC_PLL3FRACR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FRACN3[12:0]										Res.	Res.	Res.			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x048	reserved	Reserved																															
0x04C	RCC_D1CCIPR	Res.	Res.	CKPERSEL[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMCSEL	Res.	Res.	Res.	Res.	Res.	Res.	DSISEL	Res.	Res.	QSPISEL[1:0]		Res.	Res.	FMCSEL[1:0]		
	Reset value			0	0												0							0		0	0			0	0		
0x050	RCC_D2CCIP1R	SWPSEL	Res.	FDCANSEL[1:0]		Res.	Res.	Res.	DFSDM1SEL	Res.	Res.	SPDIFSEL[1:0]		Res.	Res.	SPI45SEL[2:0]		Res.	Res.	SPI123SEL[2:0]		Res.	Res.	Res.	SAI23SEL[2:0]		Res.	Res.	Res.	Res.	SAI1SEL[2:0]		
	Reset value	0		0	0				0			0	0		0	0	0		0	0	0			0	0	0					0	0	0
0x054	RCC_D2CCIP2R	Res.	LPTIM1SEL[2:0]		Res.	Res.	Res.	Res.	CECSEL[1:0]	Res.	Res.	USBSEL[1:0]		Res.	Res.	Res.	Res.	I2C123SEL[1:0]		Res.	Res.	Res.	RNGSEL[1:0]		Res.	Res.	USART16SEL[2:0]		Res.	Res.	USART234578SEL[2:0]		
	Reset value		0	0	0				0	0		0	0					0	0					0	0		0	0	0	0	0	0	0

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x058	RCC_D3CCIPR	Res.	SPI6SEL[2:0]			Res.	SAI4BSEL[2:0]			SAI4ASEL[2:0]			Res.	Res.	Res.	ADCSEL[1:0]		LPTIM345SEL[2:0]			LPTIM2SEL[2:0]		I2C4SEL[1:0]		Res.	Res.	Res.	Res.	Res.	LPUART1SEL[2:0]			
	Reset value	0	0	0		0	0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0						0	0	0
0x05C	reserved	Reserved																															
0x060	RCC_CIER	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x064	RCC_CIFR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x068	RCC_CICR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x06C	reserved	Reserved																															
0x070	RCC_BDCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BDRST	RTCN							RTCSEL[1:0]		Res.	LSECSSD	LSECSSON	LSEDRV[1:0]	LSEBYP	LSERDY	LSEON
	Reset value																0	0						0	0	0	0	0	0	0	0	0	0
0x074	RCC_CSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x078	reserved	Reserved																															
0x07C	RCC_AHB3RSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1RST		QSPIRST		FMCIRST		Res.	Res.	Res.	Res.	Res.	Res.	JPGDECRST	DMA2DRST		Res.	MDMARST
	Reset value																0		0		0						0	0					0
0x080	RCC_AHB1RSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH1MACRST	ARTRST				Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DMA2RST	DMA1RST
	Reset value																0	0	0													0	0

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x084	RCC_AHB2RSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2RST	Res.	Res.	RNGRST	HASHRST	CRYPTRST	Res.	Res.	Res.	CAMIFRST
	Reset value																							0	0	0	0	0	0	0	0	0	0
0x088	RCC_AHB4RSTR	Res.	Res.	Res.	Res.	Res.	Res.	HSEMRST	ADC3RST	Res.	Res.	BDMARST	Res.	CRCRST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOKRST	GPIORST	GPIORST	GPIORST	GPIORST	GPIORST	GPIORST	GPIORST	GPIORST	GPIORST	GPIORST
	Reset value							0	0			0		0									0	0	0	0	0	0	0	0	0	0	0
0x08C	RCC_APB3RSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DSIRST	Res.	Res.	Res.	Res.	Res.
	Reset value																											0	0	0	0	0	0
0x090	RCC_APB1LRSTR	UART8RST	UART7RST	DAC12RST	Res.	CECRST	Res.	Res.	Res.	I2C3RST	I2C2RST	I2C1RST	UART5RST	UART4RST	USART3RST	USART2RST	SPDIFRXRST	SPI3RST	SPI2RST	Res.	Res.	Res.	LPTIM1RST	TIM14RST	TIM13RST	TIM12RST	TIM7RST	TIM6RST	TIM5RST	TIM4RST	TIM3RST	TIM2RST	
	Reset value	0	0	0		0				0	0	0	0	0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0
0x094	RCC_APB1HRSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANRST	Res.	Res.	MDIOSRST	OPAMP1RST	Res.	RST	CRSRST	Res.
	Reset value																							0	0	0	0	0	0	0	0	0	0
0x098	RCC_APB2RSTR	Res.	Res.	HRTIMRST	DFSDM1RST	Res.	Res.	Res.	SAI3RST	SAI2RST	SAI1RST	Res.	SPI5RST	Res.	TIM17RST	TIM16RST	TIM15RST	Res.	Res.	SPI4RST	SPI1RST	Res.	Res.	Res.	Res.	Res.	Res.	USART6RST	USART1RST	Res.	Res.	TIM8RST	TIM1RST
	Reset value			0	0				0	0	0		0		0	0	0			0	0						0	0			0	0	0
0x09C	RCC_APB4RSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4RST	Res.	Res.	Res.	Res.	Res.	Res.	VREFRST	COMP12RST	Res.	Res.	LPTIM5RST	LPTIM4RST	LPTIM3RST	LPTIM2RST	Res.	I2C4RST	SPI6RST	Res.	Res.	LPUART1RST	Res.	
	Reset value										0							0	0			0	0	0	0	0		0		0	0	0	0
0x0A0	RCC_GCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BOOT_C2	BOOT_C1	WW2RSC	WW1RSC	
	Reset value																											0	0	0	0	0	0
0x0A4	reserved	Reserved																															

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x0A8	RCC_D3AMR	Res.		SRAM4AMEN	BKPRAMAMEN	Res.	Res.	Res.	ADC3AMEN	Res.	Res.	SAI4AMEN	Res.	CRCAMEN	Res.	Res.	RTCAMEN	VREFAMEN	COMP12AMEN	Res.	LPTIM5AMEN	LPTIM4AMEN	LPTIM3AMEN	LPTIM2AMEN	Res.	I2C4AMEN	Res.	SPI6AMEN	Res.	LPUART1AMEN	Res.	Res.	Res.	BDMAAMEN	
	Reset value			0	0				0			0		0			0	0	0		0	0	0	0		0		0		0			0		
0x0AC to 0x0CC	reserved	Reserved																																	
0x0D0	RCC_RSR	LPWR2RSTF	LPWR1RSTF	WWDG2RSTF	WWDG1RSTF	IWDG2RSTF	IWDG1RSTF	SFT2RSTF	SFT1RSTF	PORRSTF	PINRSTF	BORRSTF	D2RSTF	D1RSTF	C2RSTF	C1RSTF	RMVF	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		
	Reset value	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	0																		
0x0D4	RCC_AHB3ENR	AXISRAMEN	ITCMEN	DTCM2EN	DTCM1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1EN	Res.	QSPIEN	Res.	FMCEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MDMAEN	
	Reset value	0	0	0	0												0		0		0				0				0	0				0	
0x0D8	RCC_AHB1ENR	Res.	Res.	Res.	Res.	USB2OTGHSEN	USB1OTGHSULPIEN	USB1OTGHSEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USB2OTGHSULPIEN	ETH1RXEN	ETH1TXEN	ETH1MACEN	ARTEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DMA2EN	DMA1EN
	Reset value					0	0	0									0	0	0	0	0							0					0	0	
0x0DC	RCC_AHB2ENR	SRAM3EN	SRAM2EN	SRAM1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DCMIEN
	Reset value	0	0	0																		0							0					0	
0x0E0	RCC_AHB4ENR	Res.	Res.	Res.	BKPRAMEN	Res.	Res.	HSEMEN	ADC3EN	Res.	Res.	BDMAEN	Res.	CRCEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOAEN	
	Reset value				0			0	0			0		0															0	0	0	0	0	0	
0x0E4	RCC_APB3ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																																		

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x0E8	RCC_ APB1LENR	UART8EN	UART7EN	DAC12EN	Res.	CECEN	Res.	Res.	Res.	12C3EN	12C2EN	12C1EN	UART5EN	UART4EN	USART3EN	USART2EN	SPDIFRXEN	SPI3EN	SPI2EN	Res.	Res.	WWDG2EN	Res.	LPTIM1EN	TIM14EN	TIM13EN	TIM12EN	TIM7EN	TIM6EN	TIM5EN	TIM4EN	TIM3EN	TIM2EN	
	Reset value	0	0	0		0				0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0EC	RCC_ APB1HENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANEN	Res.	Res.	MDIOSEN	OPAMPEN	Res.	SWPEN	CRSEN	Res.	
	Reset value																								0			0	0		0	0	0	
0x0F0	RCC_ APB2ENR	Res.	Res.	HRTIMEN	DFSDM1EN	Res.	Res.	Res.	SAI3EN	SAI2EN	SAI1EN	Res.	SPI5EN	Res.	TIM17EN	TIM16EN	TIM15EN	Res.	Res.	SPI4EN	SPI1EN	Res.	Res.	Res.	Res.	Res.	Res.	USART6EN	USART1EN	Res.	Res.	TIM8EN	TIM1EN	
	Reset value			0	0				0	0	0		0		0	0	0			0	0						0	0	0			0	0	
0x0F4	RCC_ APB4ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4EN	Res.	Res.	Res.	Res.	Res.	RTCAPBEN	VREFEN	COMP12EN	Res.	LPTIM5EN	LPTIM4EN	LPTIM3EN	LPTIM2EN	Res.	12C4EN	Reserved	SPI6EN	Res.	LPUART1EN	Res.	SYSCFGEN	Res.	
	Reset value											0					1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0F8	reserved	Reserved																																
0x0FC	RCC_ AHB3LPENR	AXISRAMLPEN	ITCMLPEN	DTCM2LPEN	DTCM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1LPEN	Res.	QSPI1PEN	Res.	FMCLPEN	Res.	Res.	Res.	Res.	FLITFLPEN	Res.	Res.	JPGDECLPEN	DMA2DLPEN	Res.	Res.	Res.	MDMALPEN
	Reset value	1	1	1	1												1		1		1				1			1	1				1	
0x100	RCC_ AHB1LPENR	Res.	Res.	Res.	USB2OTGHSULPILPEN	USB2OTGHSULPEN	USB1OTGHSULPILPEN	USB1OTGHSULPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH1RXLPEN	ETH1TXLPEN	ETH1MACLPEN	ARTLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC12LPEN	Res.	Res.	Res.	DMA2LPEN	DMA1LPEN
	Reset value				1	1	1	1									1	1	1	1								1					1	1
0x104	RCC_ AHB2LPENR	SRAM3LPEN	SRAM2LPEN	SRAM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2LPEN	Res.	Res.	Res.	RNGLPEN	HASHLPEN	CRYPTLPEN	Res.	Res.	Res.	CAMITFLPEN
	Reset value	1	1	1																				1			1	1	1				1	1

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x108	RCC_AHB4LPENR	Res.	Res.	SRAM4LPEN	BKPRAML PEN	Res.	Res.	Res.	ADC3LPEN	Res.	Res.	BDMALPEN	Res.	CRCLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOKLPEN	GPIOLPEN	GPIOLPEN	GPIOHLPEN	GPIOGLPEN	GPIOFLPEN	GPIOLPEN	GPIODLPEN	GPIODLPEN	GPIODLPEN	GPIODLPEN
	Reset value			1	1				1			1		1									1	1	1	1	1	1	1	1	1	1	1
0x10C	RCC_APB3LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WWDG1LPEN	Res.	DSILPEN	LTDCLPEN	Res.	Res.	Res.	Res.
	Reset value																									1		1	1				
0x110	RCC_APB1LLPENR	UART8LPEN	UART7LPEN	DAC12LPEN	Res.	CECLPEN	Res.	Res.	Res.	I2C3LPEN	I2C2LPEN	I2C1LPEN	UART5LPEN	UART4LPEN	USART3LPEN	USART2LPEN	SPDIFRXLPEN	SPI3LPEN	SPI2LPEN	Res.	Res.	WWDG2LPEN	Res.	LPTIM1LPEN	TIM14LPEN	TIM13LPEN	TIM12LPEN	TIM7LPEN	TIM6LPEN	TIM5LPEN	TIM4LPEN	TIM3LPEN	TIM2LPEN
	Reset value	1	1	1		1				1	1	1	1	1	1	1	1	1	1			1		1	1	1	1	1	1	1	1	1	1
0x114	RCC_APB1HLPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANLPEN	Res.	MDIOSLPEN	OPAMPLPEN	Res.	SWPLPEN	CRSLPEN	Res.	Res.
	Reset value																								1		1	1		1	1	1	
0x118	RCC_APB2LPENR	Res.	Res.	HRTIMLPEN	DFSDM1LPEN	Res.	Res.	Res.	SAI3LPEN	SAI2LPEN	SAI1LPEN	Res.	SPI5LPEN	Res.	TIM17LPEN	TIM16LPEN	TIM15LPEN	Res.	Res.	SPI4LPEN	SPI1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	USART6LPEN	USART1LPEN	Res.	Res.	TIM8LPEN	TIM1LPEN
	Reset value			1	1				1	1	1		1		1	1	1			1	1						1	1				1	1
0x11C	RCC_APB4LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LPTIM5LPEN	LPTIM4LPEN	LPTIM3LPEN	LPTIM2LPEN	Res.	I2C4LPEN	SPI6LPEN	Res.	LPUART1LPEN	Res.	Res.	Res.
	Reset value										1											1	1	1	1		1			1		1	1
0x120 to 0x12C	reserved	Reserved																															
0x130	RCC_C1_RSR	LPWR2RSTF	LPWR1RSTF	WWDG2RSTF	WWDG1RSTF	IWDG2RSTF	IWDG1RSTF	SFT2RSTF	SFT1RSTF	PORRSTF	PINRSTF	BORRSTF	D2RSTF	D1RSTF	C2RSTF	C1RSTF	RMVF	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	0																

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x134	RCC_C1_AHB3ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1EN	Res.	QSPIEN	Res.	FMCEN	Res.	Res.	Res.	Res.	Res.	Res.	JPGDECEN	DMA2DEN	Res.	Res.	Res.	MDMAEN	
	Reset value																0	0	0	0	0	0					0	0				0		
0x138	RCC_C1_AHB1ENR	Res.	Res.	Res.	Res.	USB2OTGHSN	USB1OTGHSULPIEN	USB1OTGHSN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH1RXEN	ETH1TXEN	ETH1MACEN	ARTEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC12EN	Res.	Res.	Res.	DMA2EN	DMA1EN	
	Reset value					0	0	0								0	0	0	0								0				0	0	0	
0x13C	RCC_C1_AHB2ENR	SRAM3EN	SRAM2EN	SRAM1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RNGEN	HASHEN	CRYPTEN	Res.	Res.	Res.	DCMIEN
	Reset value	0	0	0														0									0	0	0				0	0
0x140	RCC_C1_AHB4ENR	Res.	Res.	Res.	BKPRAMEN	Res.	Res.	HSEMEN	ADC3EN	Res.	Res.	BDMAEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value				0			0	0			0		0														0	0	0	0	0	0	0
0x144	RCC_C1_APB3ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																										0	0	0	0	0	0	0	0
0x148	RCC_C1_APB1LENR	UART8EN	UART7EN	DAC12EN	Res.	HDMICECEN	Res.	Res.	Res.	I2C3EN	I2C2EN	I2C1EN	UART5EN	UART4EN	USART3EN	USART2EN	SPDIFRXEN	SPI3EN	SPI2EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value	0	0	0		0				0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0
0x14C	RCC_C1_APB1HENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																									0		0	0	0	0	0	0	0
0x150	RCC_C1_APB2ENR	Res.	Res.	HRTIMEN	DFSDM1EN	Res.	Res.	Res.	SAI3EN	SAI2EN	SAI1EN	Res.	SPI5EN	Res.	TIM17EN	TIM16EN	TIM15EN	Res.	Res.	SPI4EN	SPI1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value			0	0				0	0	0		0		0	0	0			0	0							0	0	0	0	0	0	0

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x154	RCC_C1_APB4ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4EN	Res.	Res.	Res.	Res.	RTCAPBEN	VREFEN	COMP12EN	Res.	LPTIM5EN	LPTIM4EN	LPTIM3EN	LPTIM2EN	Res.	I2C4EN	Res.	SPI6EN	Res.	LPUART1EN	Res.	SYSCFGEN	Res.
	Reset value											0					1	0	0		0	0	0	0		0		0		0		0	
0x158	reserved	Reserved																															
0x15C	RCC_C1_AHB3LPENR	AXISRAMLPEN	ITCMLPEN	DTCM2LPEN	DTCM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1LPEN	Res.	QSPILPEN	Res.	FMCLPEN	Res.	Res.	Res.	FLITFLPEN	Res.	Res.	JPGDECLPEN	DMA2DLPEN	Res.	Res.	Res.	MDMALPEN
	Reset value	1	1	1	1												1		1		1				1			1	1				1
0x160	RCC_C1_AHB1LPENR	Res.	Res.	Res.	Res.	USB2OTGHSLPEN	USB1OTGHSULPILPEN	USB1OTGHSLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH1RXLPEN	ETH1TXLPEN	ETH1MACLPEN	ARTLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC12LPEN	Res.	Res.	DMA2LPEN	DMA1LPEN	
	Reset value					1	1	1									1	1	1	1								1				1	1
0x164	RCC_C1_AHB2LPENR	SRAM3LPEN	SRAM2LPEN	SRAM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2LPEN	Res.	Res.	Res.	RNGLPEN	HASHLPEN	CRYPTLPEN	Res.	Res.	Res.	CAMITFLPEN
	Reset value	1	1	1																			1			1	1	1					1
0x168	RCC_C1_AHB4LPENR	Res.	Res.	SRAM4LPEN	BKPRAMLLEN	Res.	Res.	Res.	ADC3LPEN	Res.	Res.	DMA1LPEN	CRCLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOKLPEN	GPIOLPEN	GPIOLPEN	GPIOHLPEN	GPIOGLPEN	GPIOFLPEN	GPIOELPEN	GPIODLPEN	GPIOCLPEN	GPIOBLPEN	GPIOALPEN
	Reset value			1	1				1			1	1											1	1	1	1	1	1	1	1	1	1
0x16C	RCC_C1_APB3LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x170	RCC_C1_APB1LLPENR	UART8LPEN	UART7LPEN	DAC12LPEN	Res.	CECLPEN	Res.	Res.	Res.	Res.	Res.	Res.	UART4LPEN	UART3LPEN	UART2LPEN	SPDIFRXLPEN	SPI3LPEN	SPI2LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	1	1	1		1							1	1	1	1	1	1															

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x174	RCC_C1_ APB1HLPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANLPEN	Res.	Res.	Res.	MDIOSLPEN	OPAMPLPEN	Res.	SWPLPEN	CRSLPEN	Res.
	Reset value																								1				1	1		1	1	
0x178	RCC_C1_ APB2LPENR	Res.	Res.	HRTIMLPEN	DFSDM1LPEN	Res.	Res.	Res.	SA3LPEN	SAI2LPEN	SAI1LPEN	Res.	SPI5LPEN	Res.	TIM17LPEN	TIM16LPEN	TIM15LPEN	Res.	Res.	SPI4LPEN	SPI1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USART6LPEN	USART1LPEN	Res.	Res.	Res.	Res.
	Reset value			1	1				1	1	1		1		1	1	1			1	1								1	1			1	1
0x17C	RCC_C1_ APB4LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4LPEN	Res.	Res.	Res.	Res.	RTCAPBLPEN	VREFLPEN	COMP12LPEN	Res.	LPTIM5LPEN	LPTIM4LPEN	LPTIM3LPEN	LPTIM2LPEN	Res.	Res.	Res.	SPI6LPEN	Res.	LPUART1LPEN	Res.	Res.	Res.	
	Reset value											1					1	1	1		1	1	1	1		1		1		1		1		
0x180 - 0x18C	reserved	Reserved																																
0x190	RCC_C2_RSR	LPWR2RSTF	LPWR1RSTF	WWDG2RSTF	WWDG1RSTF	IWDG2RSTF	IWDG1RSTF	SFT2RSTF	SFT1RSTF	PORRSTF	PINRSTF	BORRSTF	D2RSTF	D1RSTF	C2RSTF	C1RSTF	RMVF	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	0																	
0x194	RCC_C2_AHB3EN R	AXISRSTF	ITCMEN	DTCM2EN	DTCM1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1EN	Res.	QSPIEN	Res.	FMCEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JPGDECEN	DMA2DEN	Res.	Res.	Res.	Res.
	Reset value	0	0	0	0												0		0		0				0			0	0					0
0x198	RCC_C2_AHB1EN R	Res.	Res.	Res.	Res.	USB2OTGHSN	USB1OTGHSULPIEN	USB1OTGHSN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH1RXEN	ETH1TXEN	ETH1MACEN	ARTEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC12EN	Res.	Res.	Res.	Res.	Res.
	Reset value					0	0	0									0	0	0	0								0					0	0
0x19C	RCC_C2_AHB2EN R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2EN	Res.	Res.	Res.	NGEN	HASHEN	CRYPTEN	Res.	Res.	Res.
	Reset value																							0			0	0	0					0

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x1A0	RCC_C2_AHB4ENR	Res.	Res.	Res.	BKPRAMEN	Res.	Res.	HSEMEN	ADC3EN	Res.	Res.	DMA1EN	Res.	CRCCEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOKEN	GPIOJEN	GPIOIEN	GPIOHEN	GPIOGEN	GPIOFEN	GPIOEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
	Reset value				0			0	0			0		0									0	0	0	0	0	0	0	0	0	0	0
0x1A4	RCC_C2_APB3ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WWDG1TEN	Res.	DSIEN	LTDCEN	Res.	Res.	Res.
	Reset value																									0		0	0				
0x1A8	RCC_C2_APB1LENR	UART8EN	UART7EN	DAC12EN	Res.	HDMICECEN	Res.	Res.	Res.	I2C3EN	I2C2EN	I2C1EN	UART5EN	UART4EN	USART3EN	USART2EN	SPDIFRXEN	SPI3EN	SPI2EN	Res.	Res.	WWDG2EN	Res.	LPTIM1EN	TIM14EN	TIM13EN	TIM12EN	TIM7EN	TIM6EN	TIM5EN	TIM4EN	TIM3EN	TIM2EN
	Reset value	0	0	0		0				0	0	0	0	0	0	0	0	0	0	0		0		0	0	0	0	0	0	0	0	0	0
0x1AC	RCC_C2_APB1HENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANEN	Res.	Res.	MDIOSEN	OPAMPEN	Res.	SWPEN	CRSEN	Res.
	Reset value																								0		0	0	0		0	0	0
0x1B0	RCC_C2_APB2ENR	Res.	Res.	HRTIMEN	DFSDM1EN	Res.	Res.	Res.	SAI3EN	SAI2EN	SAI1EN	SPI5EN	SPI4EN	TIM17EN	TIM16EN	TIM15EN	Res.	Res.	SPI4EN	SPI1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USART6EN	USART1EN	Res.	Res.	TIM8EN	TIM1EN
	Reset value			0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1B4	RCC_C2_APB4ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LPUART1EN	Res.	Res.	SYSCFGEN	Res.
	Reset value											0																0		0		0	
0x1B8	reserved	Reserved																															
0x1BC	RCC_C2_AHB3LPENR	AXISRAMLPEN	ITCMLPEN	DTCM2LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC1LPEN	Res.	QSPI1LPEN	Res.	FMCLPEN	Res.	Res.	Res.	Res.	Res.	Res.	JPGDECLPEN	DMA2DLPEN	Res.	Res.	Res.	MDMALPEN
	Reset value	1	1	1													1		1		1						1	1					1
0x1C0	RCC_C2_AHB1LPENR	Res.	Res.	Res.	Res.	USB2OTGHSLPEN	USB1OTGHSLPEN	USB1OTGHSLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETH1RXLPEN	ETH1TXLPEN	ETH1MACLPEN	ARTLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC12LPEN	Res.	Res.	DMA2LPEN	DMA1LPEN
	Reset value					1	1	1									1	1	1	1								1					1

Table 88. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x1C4	RCC_C2_AHB2LPENR	SRAM3LPEN	SRAM2LPEN	SRAM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMC2LPEN	1	Res.	Res.	RNGLPEN	HASHLPEN	CRYPTLPEN	Res.	Res.	Res.	CAMITFIPEN
	Reset value	1	1	1								1	Res.	CRCLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	1	1	1	1	1	1				1	
0x1C8	RCC_C2_AHB4LPENR	Res.	Res.	SRAM4LPEN	BKPRAMLLEN	Res.	Res.	Res.	ADC3LPEN	Res.	Res.	DMA1LPEN	Res.	CRCLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOKLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN	GPIOLPEN	GPIODLPEN	GPIOCLPEN	GPIOBLPEN	GPIOALPEN
	Reset value			1	1				1			1	Res.	1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	1	1	1	1	1	1	1	1	1	1	1	
0x1CC	RCC_C2_APB3LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																									1			1	1				
0x1D0	RCC_C2_APB1LLPENR	UART8LPEN	UART7LPEN	DAC12LPEN	Res.	CECLPEN	Res.	Res.	Res.	I2C3LPEN	I2C2LPEN	I2C1LPEN	UART5LPEN	UART4LPEN	USART3LPEN	USART2LPEN	SPDIFRXLPEN	SPI3LPEN	SPI2LPEN	Res.	Res.	Res.	Res.	Res.	LPTIM1LPEN	TIM14LPEN	TIM13LPEN	TIM12LPEN	TIM7LPEN	TIM8LPEN	TIM5LPEN	TIM4LPEN	TIM3LPEN	TIM2LPEN
	Reset value	1	1	1		1				1	1	1	1	1	1	1	1	1	1					1	1	1	1	1	1	1	1	1	1	
0x1D4	RCC_C2_APB1HLPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANLPEN	Res.	Res.	Res.	MDIOSLPEN	OPAMPLPEN	Res.	SWPLPEN	CRSLPEN	Res.
	Reset value																								1			1	1		1	1		
0x1D8	RCC_C2_APB2LPENR	Res.	Res.	HRTIMLPEN	DFSDM1LPEN	Res.	Res.	Res.	SAI3LPEN	SAI2LPEN	SAI1LPEN	Res.	SPI5LPEN	TIM17LPEN	TIM16LPEN	TIM15LPEN	Res.	Res.	SPI4LPEN	SPI1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USART6LPEN	USART1LPEN	Res.	Res.	TIM8LPEN	TIM1LPEN	
	Reset value			1	1				1	1	1		1	1	1	1			1	1							1	1				1	1	
0x1DC	RCC_C2_APB4LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SAI4LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SPI6LPEN	Res.	LPUART1LPEN	Res.	Res.	SYSCFGLPEN	Res.	
	Reset value											1																				1		
0x1E0 to 0x1FC	reserved	Reserved																																

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

11 Hardware semaphore (HSEM)

11.1 Introduction

The hardware semaphore block provides 32 (32-bit) register based semaphores.

The semaphores can be used to ensure synchronization between different processes running between different cores. The HSEM provides a non-blocking mechanism to lock semaphores in an atomic way. The following functions are provided:

- Semaphore lock, in two ways:
 - 2-step lock: by writing COREID and PROCID to the semaphore, followed by a read check
 - 1-step lock: by reading the COREID from the semaphore
- Interrupt generation when a semaphore is unlocked
 - Each semaphore may generate an interrupt on one of the interrupt lines
- Semaphore clear protection
 - A semaphore is only unlocked when COREID and PROCID match
- Global semaphore clear per COREID

11.2 Main features

The HSEM includes the following features:

- 32 (32-bit) semaphores
- 8-bit PROCID
- 4-bit COREID
- One interrupt line per processor
- Lock indication

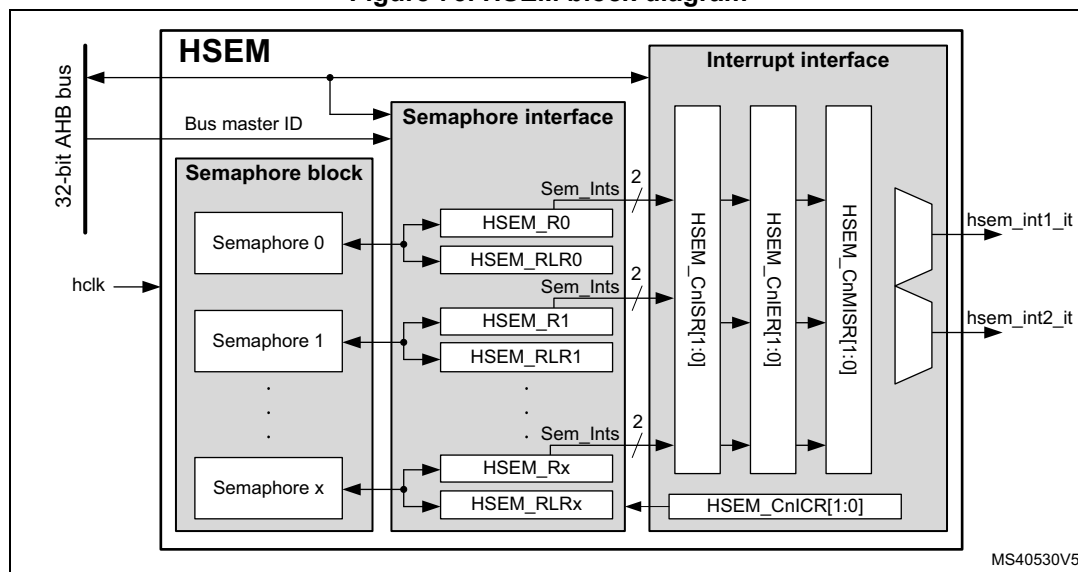
11.3 Functional description

11.3.1 HSEM block diagram

As shown in [Figure 73](#), the HSEM is based on three sub-blocks:

- the semaphore block containing the semaphore status and IDs
- the semaphore interface block providing AHB access to the semaphore via the HSEM_Rx and HSEM_RLRx registers
- the interrupt interface block providing control for the interrupts via HSEM_CnISR, HSEM_CnIER, HSEM_CnMISR, and HSEM_CnICR registers.

Figure 73. HSEM block diagram



11.3.2 HSEM internal signals

Table 94. HSEM internal input/output signals

Signal name	Signal type	Description
AHB bus	Digital input/output	AHB register access bus
BusMasterID	Digital input	AHB bus master ID
hsem_intn_it	Digital output	Interrupt n line (n = 1 to 2)

11.3.3 HSEM lock procedures

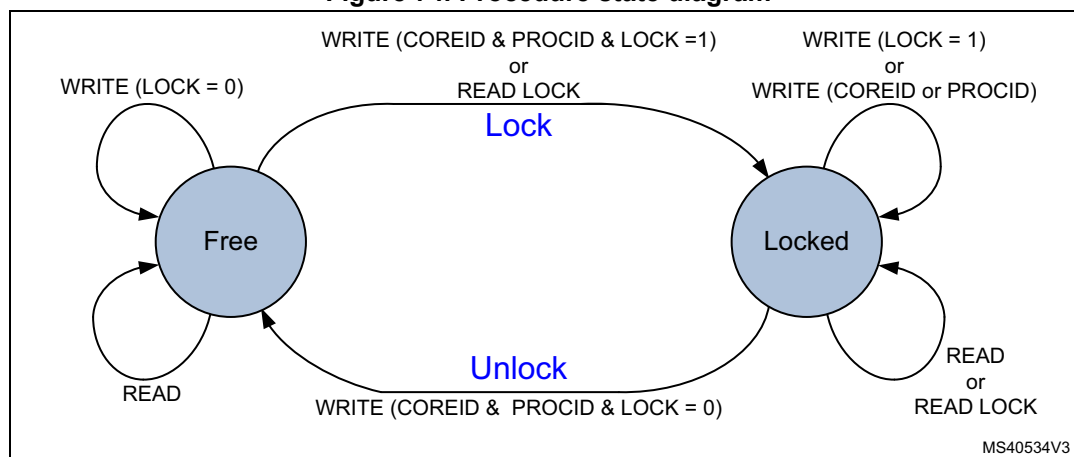
There are two lock procedures, namely 2-step (write) lock and 1-step (read) lock. The two procedures can be used concurrently.

The semaphore is free when its LOCK bit is 0. In this case, the COREID and PROCID are also 0. When the LOCK bit is 1, the semaphore is locked and the COREID indicates which AHB bus master ID has locked it. The PROCID indicates which process of that AHB bus master ID has locked the semaphore.

When write locking a semaphore, the written COREID must match the AHB bus master ID, and the PROCID is written by the AHB bus master software process taking the lock.

When read locking the semaphore, the COREID is taken from the AHB bus master ID, and the PROCID is forced to 0 by hardware. There is no PROCID available with read lock.

Figure 74. Procedure state diagram



2-step (write) lock procedure

The 2-step lock procedure consists in a write to lock the semaphore, followed by a read to check if the lock has been successful, carried out from the HSEM_Rx register.

- Write semaphore with PROCID and COREID, and LOCK = 1. The COREID data written by software must match the AHB bus master information, that is, a AHB bus master ID = 1 writes data COREID = 1.
Lock is put in place when the semaphore is free at write time.
- Read-back the semaphore
The software checks the lock status, if PROCID and COREID match the written data, then the lock is confirmed.
- Else retry (the semaphore has been locked by another process, AHB bus master ID).

A semaphore can only be locked when it is free.

A semaphore can be locked when the PROCID = 0.

Consecutive write attempts with LOCK = 1 to a locked semaphore are ignored.

1-step (read) lock procedure

The 1-step procedure consists in a read to lock and check the semaphore in a single step, carried out from the HSEM_RLRx register.

- Read lock semaphore with the AHB bus master COREID.
- If read COREID matches and PROCID = 0, then lock is put in place. If COREID matches and PROCID is not 0, this means that another process from the same COREID has locked the semaphore with a 2-step (write) procedure.
- Else retry (the semaphore has been locked by another process, AHB bus master ID).

A semaphore can only be locked when it is free. When read locking a free semaphore, PROCID is 0. Read locking a locked semaphore returns the COREID and PROCID that locked it. All read locks, including the first one that locks the semaphore, return the COREID that locks or locked the semaphore.

Note: The 1-step procedure must not be used when running multiple processes of the same AHB bus master ID. All processes using the same semaphore read the same status. When only one process locks the semaphore, each process of that AHB bus master ID reads the semaphore as locked by itself with the COREID.

11.3.4 HSEM write/read/read lock register address

For each semaphore, two AHB register addresses are provided, separated in two banks of 32-bit semaphore registers, spaced by a 0x80 address offset.

In the first register address bank the semaphore can be written (locked/unlocked) and read through the HSEM_Rx registers.

In the second register address bank the semaphore can be read (locked) through the HSEM_RLRx registers.

11.3.5 HSEM unlock procedures

Unlocking a semaphore is a protected process, to prevent accidental clearing by a AHB bus master ID or by a process not having the semaphore lock right. The procedure consists in writing to the semaphore HSEM_Rx register with the corresponding COREID and PROCID and LOCK = 0. When unlocked the semaphore, the COREID, and the PROCID are all 0.

When unlocked, an interrupt may be generated to signal the event. To this end, the semaphore interrupt must be enabled.

The unlock procedure consists in a write to the semaphore HSEM_Rx register with matching COREID regardless on how the semaphore has been locked (1- or 2-step).

- Write semaphore with PROCID, COREID, and LOCK = 0
- If the written data matches the semaphore PROCID and COREID and the AHB bus master ID, the semaphore is unlocked and an interrupt may be generated when enabled, else write is ignored, semaphore remains locked and no interrupt is generated (the semaphore is locked by another process, AHB bus master ID or the written data does not match the AHB bus master signaling).

Note: Different processes of the same AHB bus master ID can write any PROCID value. Preventing other processes of the same AHB bus master ID from unlocking a semaphore must be ensured by software, handling the PROCID correctly.

11.3.6 HSEM COREID semaphore clear

All semaphores locked by a COREID can be unlocked at once by using the HSEM_CR register. Write COREID and correct KEY value in HSEM_CR. All locked semaphores with a matching COREID are unlocked, and may generate an interrupt when enabled.

Note: This procedure may be used in case of an incorrect functioning AHB bus master ID, where another AHB bus master can unlock the locked semaphores by writing the COREID of the incorrect functioning processor into the HSEM_CR register with the correct KEY value. This unlocks all locked semaphores with a matching COREID.

An interrupt may be generated for the unlocked semaphore(s). To this end, the semaphore interrupt must be enabled in the HSEM_CnIER registers.

11.3.7 HSEM interrupts

An interrupt line hsem_intn_it per processor allows each semaphore to generate an interrupt.

An interrupt line provides the following features per semaphore:

- interrupt enable
- interrupt clear
- interrupt status
- masked interrupt status

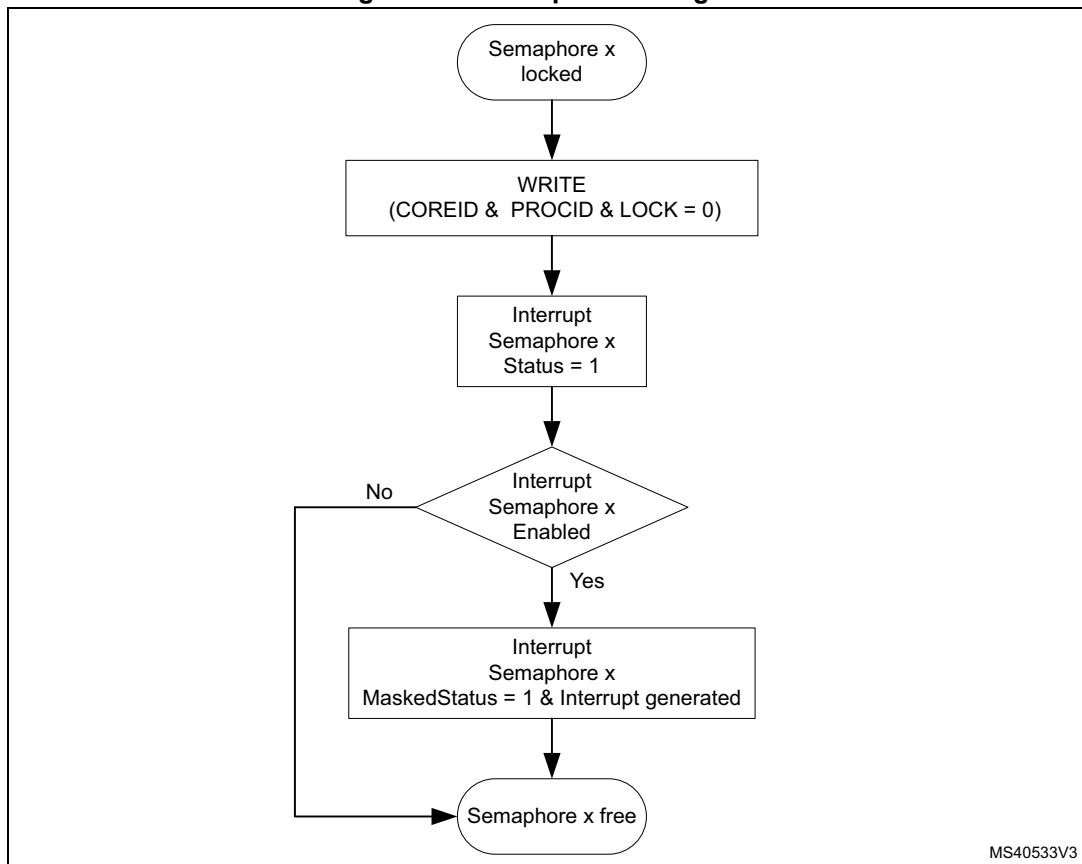
With the interrupt enable (HSEM_CnIER) the semaphores affecting the interrupt line can be enabled. Disabled (masked) semaphore interrupts do not set the masked interrupt status MISF for that semaphore, and do not generate an interrupt on the interrupt line.

The interrupt clear (HSEM_CnICR) clears the interrupt status ISF and masked interrupt status MISF of the associated semaphore for the interrupt line.

The interrupt status (HSEM_CnISR) mirrors the semaphore interrupt status ISF before the enable.

The masked interrupt status (HSEM_CnMISR) only mirrors the semaphore enabled interrupt status MISF on the interrupt line. All masked interrupt status MISF of the enabled semaphores need to be cleared to clear the interrupt line.

Figure 75. Interrupt state diagram



The procedure to get an interrupt when a semaphore becomes free is described hereafter.

Try to lock semaphore x

- If the semaphore lock is obtained, no interrupt is needed.
- If the semaphore lock fails:
- Clear pending semaphore x interrupt status for the interrupt line in HSEM_CnICR.
Re-try to lock the semaphore x again:
 - If the semaphore lock is obtained, no interrupt is needed (semaphore has been freed between first try to lock and clear semaphore interrupt status).
 - If the semaphore lock fails, enable the semaphore x interrupt in HSEM_CnIER.

On semaphore x free interrupt, try to lock semaphore x

- If the semaphore lock is obtained:
Disable the semaphore x interrupt in HSEM_CnIER.
Clear pending semaphore x interrupt status in HSEM_CnICR.
- If the semaphore x lock fails:
Clear pending semaphore x interrupt status in HSEM_CnICR.
Try again to lock the semaphore x:
 - If the semaphore lock is obtained (semaphore has been freed between first try to lock and semaphore interrupt status clear), disable the semaphore interrupt in HSEM_CnIER.

- If the semaphore lock fails, wait for semaphore free interrupt.

Note: An interrupt does not lock the semaphore. After an interrupt, either the AHB bus master or the process must still perform the lock procedure to lock the semaphore.

It is possible to have multiple AHB bus masters informed by the semaphore free interrupts. Each AHB bus master gets its interrupt, and the first one to react locks the semaphore.

11.3.8 AHB bus master ID verification

The HSEM allows only authorized AHB bus master IDs to lock and unlock semaphores.

- The AHB bus master 2-step lock write access to the semaphore HSEM_Rx register is checked against the valid bus master IDs.
 - Accesses from unauthorized AHB bus master IDs are discarded and do not lock the semaphore.
- The AHB bus master 1-step lock read access from the semaphore HSEM_RLRx register is checked against the valid bus master IDs.
 - An unauthorized AHB bus master ID read from HSEM_RLRx returns all 0.
- The semaphore unlock write access to the HSEM_CR register is checked against the valid bus master IDs. Only the valid bus master IDs can write to the HSEM_CR register and unlock any of the COREID semaphores.
 - Accesses from unauthorized AHB bus master IDs are discarded and do not clear the COREID semaphores.

[Table 95](#) details the relation between bus master/processor and COREID.

Table 95. Authorized AHB bus master IDs

Bus master 0 (processor1)	Bus master 1 (processor2)
COREID = 3	COREID = 1

Note: Accesses from unauthorized AHB bus master IDs to other registers are granted.

11.4 HSEM registers

Registers must be accessed using word format. Byte and half-word accesses are ignored and have no effect on the semaphores, they generate a bus error.

11.4.1 HSEM register semaphore x (HSEM_Rx)

Address offset: $0x000 + 0x4 * x$ ($x = 0$ to 31)

Reset value: $0x0000\ 0000$

The HSEM_Rx must be used to perform a 2-step write lock, read back, and for unlocking a semaphore. Only write accesses with authorized AHB bus master IDs are granted. Write accesses with unauthorized AHB bus master IDs are discarded.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCK	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	COREID[3:0]				PROCID[7:0]							
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **LOCK**: Lock indication

This bit can be written and read by software.

0: On write free semaphore (only when COREID and PROCID match), on read semaphore is free.

1: On write try to lock semaphore, on read semaphore is locked.

Bits 30:13 Reserved, must be kept at reset value.

Bit 12 Reserved, must be kept at reset value.

Bits 11:8 **COREID[3:0]**: Semaphore COREID

Written by software

- When the semaphore is free and the LOCK bit is at the same time written to 1 and the COREID matches the AHB bus master ID.
- When the semaphore is unlocked (LOCK written to 0 and AHB bus master ID matched COREID, the COREID is cleared to 0.
- When the semaphore is unlocked (LOCK bit written to 0 or AHB bus master ID does not match COREID, the COREID is not affected.
- Write when LOCK bit is already 1 (semaphore locked), the COREID is not affected.
- An authorized read returns the stored COREID value.

Bits 7:0 **PROCID[7:0]**: Semaphore PROCID

Written by software

- When the semaphore is free and the LOCK is written to 1, and the COREID matches the AHB bus master ID, PROCID is set to the written data.
- When the semaphore is unlocked, LOCK written to 0 and AHB bus master ID matched COREID, the PROCID is cleared to 0.
- When the semaphore is unlocked, LOCK bit written to 0 and AHB bus master ID does not match COREID, the PROCID is not affected.
- Write when LOCK bit is already 1 (semaphore locked), the PROCID is not affected.
- An authorized read returns the stored PROCID value.

11.4.2 HSEM read lock register semaphore x (HSEM_RLRx)

Address offset: $0x080 + 0x4 * x$ ($x = 0$ to 31)

Reset value: $0x0000\ 0000$

Accesses the same physical bits as HSEM_Rx. The HSEM_RLRx must be used to perform a 1-step read lock. Only read accesses with authorized AHB bus master IDs are granted. Read accesses with unauthorized AHB bus master IDs are discarded and return 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCK	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	COREID[3:0]				PROCID[7:0]							
				r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **LOCK**: Lock indication

This bit is read only by software at this address.

- When the semaphore is free:

A read with a valid AHB bus master ID locks the semaphore and returns 1.

- When the semaphore is locked:

A read with a valid AHB bus master ID returns 1 (the COREID and PROCID reflect the already locked semaphore information).

Bits 30:13 Reserved, must be kept at reset value.

Bit 12 Reserved, must be kept at reset value.

Bits 11:8 **COREID[3:0]**: Semaphore COREID

This field is read only by software at this address.

On a read, when the semaphore is free, the hardware sets the COREID to the AHB bus master ID reading the semaphore. The COREID of the AHB bus master locking the semaphore is read.

On a read when the semaphore is locked, this field returns the COREID of the AHB bus master that has locked the semaphore.

Bits 7:0 **PROCID[7:0]**: Semaphore processor ID

This field is read only by software at this address.

- On a read when the semaphore is free:

A read with a valid AHB bus master ID locks the semaphore and hardware sets the PROCID to 0.

- When the semaphore is locked:

A read with a valid AHB bus master ID returns the PROCID of the AHB bus master that has locked the semaphore.

11.4.3 HSEM interrupt enable register (HSEM_CnIER)

Address offset: $0x100 + 0x010 * (n - 1)$, ($n = 1$ to 2)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ISE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ISE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ISE[31:0]**: Interrupt(n) semaphore x enable bit ($x = 0$ to 31)

This bit is read and written by software.

0: Interrupt(n) generation for semaphore x disabled (masked)

1: Interrupt(n) generation for semaphore x enabled (not masked)

11.4.4 HSEM interrupt clear register (HSEM_CnICR)

Address offset: $0x104 + 0x010 * (n - 1)$, ($n = 1$ to 2)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ISC[31:16]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ISC[15:0]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:0 **ISC[31:0]**: Interrupt(n) semaphore x clear bit ($x = 0$ to 31)

This bit is written by software, and is always read 0.

0: Interrupt(n) semaphore x status ISFx and masked status MISFx not affected.

1: Interrupt(n) semaphore x status ISFx and masked status MISFx cleared.

11.4.5 HSEM interrupt status register (HSEM_CnISR)

Address offset: $0x108 + 0x010 * (n - 1)$, ($n = 1$ to 2)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ISF[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ISF[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **ISF[31:0]**: Interrupt semaphore x status bit before enable (mask) (x = 0 to 31)

This bit is set by hardware, and reset only by software. This bit is cleared by software writing the corresponding HSEM_CnICR bit.

0: Interrupt semaphore x status, no interrupt pending

1: Interrupt semaphore x status, interrupt pending

11.4.6 HSEM interrupt status register (HSEM_CnMISR)

Address offset: 0x10C + 0x010 * (n - 1), (n = 1 to 2)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MISF[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MISF[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **MISF[31:0]**: Masked interrupt(n) semaphore x status bit after enable (mask) (x = 0 to 31)

This bit is set by hardware and read only by software. This bit is cleared by software writing the corresponding HSEM_CnICR bit. This bit is read as 0 when semaphore x status is masked in HSEM_CnIER bit x.

0: interrupt(n) semaphore x status after masking not pending

1: interrupt(n) semaphore x status after masking pending

11.4.7 HSEM clear register (HSEM_CR)

Address offset: 0x140

Reset value: 0x0000 0000

Only write accesses with authorized AHB bus master IDs are granted. Write accesses with unauthorized AHB bus master IDs are discarded.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	COREID[3:0]				Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
				w	w	w	w								

Bits 31:16 **KEY[15:0]**: Semaphore clear key

This field can be written by software and is always read 0.

If this key value does not match HSEM_KEYR.KEY, semaphores are not affected.

If this key value matches HSEM_KEYR.KEY, all semaphores matching the COREID are cleared to the free state.

Bits 15:13 Reserved, must be kept at reset value.

Bit 12 Reserved, must be kept at reset value.

Bits 11:8 **COREID[3:0]**: COREID of semaphores to be cleared
This field can be written by software and is always read 0.
This field indicates the COREID for which the semaphores are cleared when writing the HSEM_CR.

Bits 7:0 Reserved, must be kept at reset value.

11.4.8 HSEM clear semaphore key register (HSEM_KEYR)

Address offset: 0x144
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:16 **KEY[15:0]**: Semaphore clear key
This field can be written and read by software.
Key value to match when clearing semaphores.

Bits 15:0 Reserved, must be kept at reset value.

11.4.9 HSEM register map

Table 96. HSEM register map and reset values

[illegible]

Table 96. HSEM register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x11C	HSEM_C2MISR	MISF[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x140	HSEM_CR	KEY[15:0]																Res.	Res.	Res.	Res.	COREID[3:0]				Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x144	HSEM_KEYR	KEY[15:0]																Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

12 General-purpose I/Os (GPIO)

12.1 Introduction

Each general-purpose I/O port has four 32-bit configuration registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR and GPIOx_PUPDR), two 32-bit data registers (GPIOx_IDR and GPIOx_ODR) and a 32-bit set/reset register (GPIOx_BSRR). In addition all GPIOs have a 32-bit locking register (GPIOx_LCKR) and two 32-bit alternate function selection registers (GPIOx_AFRH and GPIOx_AFRL).

12.2 GPIO main features

- Output states: push-pull or open drain + pull-up/down
- Output data from output data register (GPIOx_ODR) or peripheral (alternate function output)
- Speed selection for each I/O
- Input states: floating, pull-up/down, analog
- Input data to input data register (GPIOx_IDR) or peripheral (alternate function input)
- Bit set and reset register (GPIOx_BSRR) for bitwise write access to GPIOx_ODR
- Locking mechanism (GPIOx_LCKR) provided to freeze the I/O port configurations
- Analog function
- Alternate function selection registers
- Fast toggle capable of changing every two clock cycles
- Highly flexible pin multiplexing allows the use of I/O pins as GPIOs or as one of several peripheral functions

12.3 GPIO functional description

Subject to the specific hardware characteristics of each I/O port listed in the datasheet, each port bit of the general-purpose I/O (GPIO) ports can be individually configured by software in several modes:

- Input floating
- Input pull-up
- Input-pull-down
- Analog
- Output open-drain with pull-up or pull-down capability
- Output push-pull with pull-up or pull-down capability
- Alternate function push-pull with pull-up or pull-down capability
- Alternate function open-drain with pull-up or pull-down capability

Each I/O port bit is freely programmable, however the I/O port registers have to be accessed as 32-bit words, half-words or bytes. The purpose of the GPIOx_BSRR register is to allow atomic read/modify accesses to any of the GPIOx_ODR registers. In this way, there is no risk of an IRQ occurring between the read and the modify access.

Figure 76 and Figure 77 show the basic structures of a standard and a 5-Volt tolerant I/O port bit, respectively. Table 97 gives the possible port bit configurations.

Figure 76. Basic structure of an I/O port bit

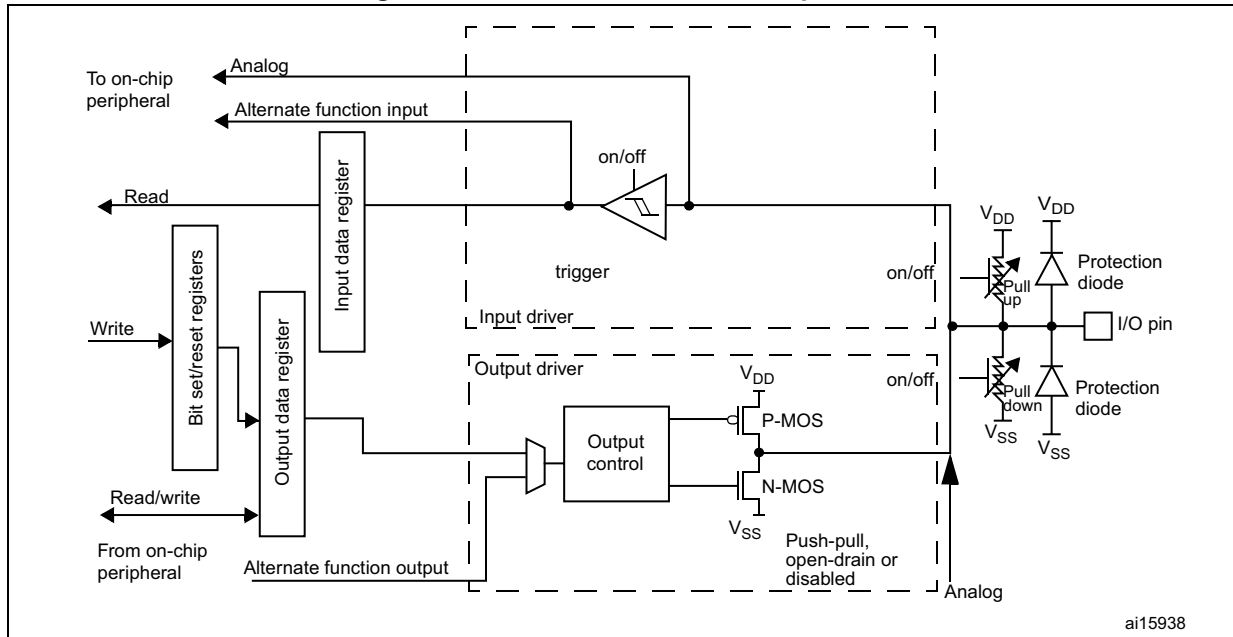
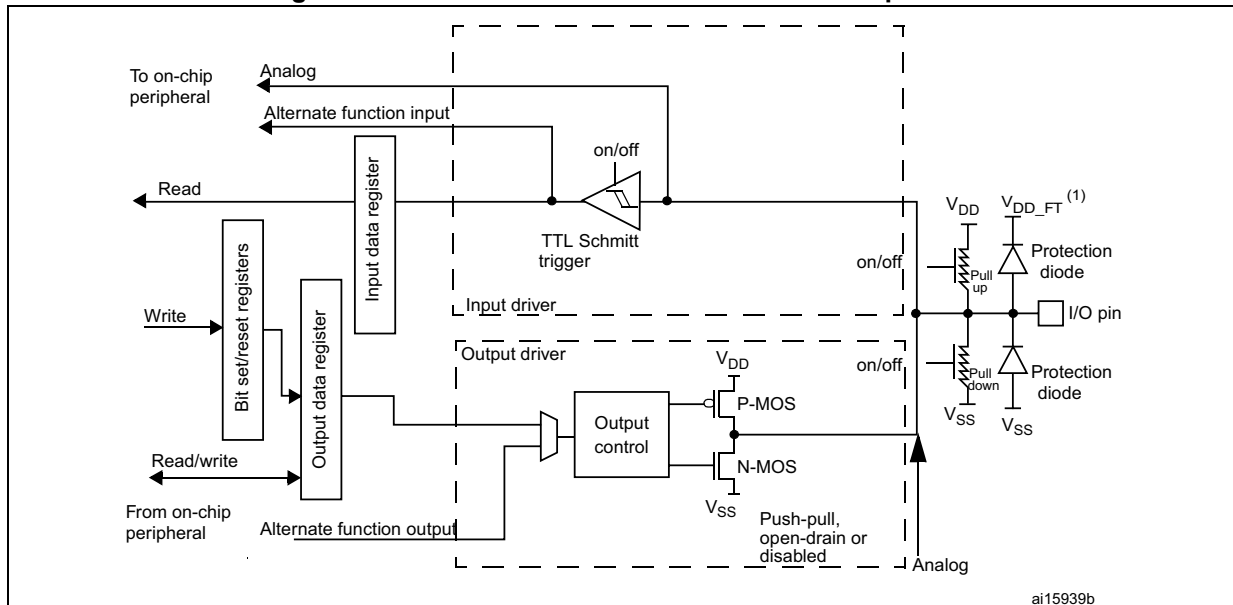


Figure 77. Basic structure of a 5-Volt tolerant I/O port bit



1. V_{DD_FT} is a potential specific to 5-Volt tolerant I/Os and different from V_{DD} .

Table 97. Port bit configuration table⁽¹⁾

MODE(i) [1:0]	OTYPER(i)	OSPEED(i) [1:0]		PUPD(i) [1:0]		I/O configuration	
01	0	SPEED [1:0]		0	0	GP output	PP
	0			0	1	GP output	PP + PU
	0			1	0	GP output	PP + PD
	0			1	1	Reserved	
	1			0	0	GP output	OD
	1			0	1	GP output	OD + PU
	1			1	0	GP output	OD + PD
	1			1	1	Reserved (GP output OD)	
10	0	SPEED [1:0]		0	0	AF	PP
	0			0	1	AF	PP + PU
	0			1	0	AF	PP + PD
	0			1	1	Reserved	
	1			0	0	AF	OD
	1			0	1	AF	OD + PU
	1			1	0	AF	OD + PD
	1			1	1	Reserved	
00	x	x	x	0	0	Input	Floating
	x	x	x	0	1	Input	PU
	x	x	x	1	0	Input	PD
	x	x	x	1	1	Reserved (input floating)	
11	x	x	x	0	0	Input/output	Analog
	x	x	x	0	1	Reserved	
	x	x	x	1	0		
	x	x	x	1	1		

1. GP = general-purpose, PP = push-pull, PU = pull-up, PD = pull-down, OD = open-drain, AF = alternate function.

12.3.1 General-purpose I/O (GPIO)

During and just after reset, the alternate functions are not active and most of the I/O ports are configured in analog mode.

The debug pins are in AF pull-up/pull-down after reset:

- PA15: JTDI in pull-up
- PA14: JTCK/SWCLK in pull-down
- PA13: JTMS/SWDAT in pull-up
- PB4: NJTRST in pull-up
- PB3: JTDO in floating state

When the pin is configured as output, the value written to the output data register (GPIOx_ODR) is output on the I/O pin. It is possible to use the output driver in push-pull mode or open-drain mode (only the low level is driven, high level is HI-Z).

The input data register (GPIOx_IDR) captures the data present on the I/O pin at every AHB clock cycle.

All GPIO pins have weak internal pull-up and pull-down resistors, which can be activated or not depending on the value in the GPIOx_PUPDR register.

12.3.2 I/O pin alternate function multiplexer and mapping

The device I/O pins are connected to on-board peripherals/modules through a multiplexer that allows only one peripheral alternate function (AF) connected to an I/O pin at a time. In this way, there can be no conflict between peripherals available on the same I/O pin.

Each I/O pin has a multiplexer with up to sixteen alternate function inputs (AF0 to AF15) that can be configured through the GPIOx_AFRL (for pin 0 to 7) and GPIOx_AFRH (for pin 8 to 15) registers:

- After reset the multiplexer selection is alternate function 0 (AF0). The I/Os are configured in alternate function mode through GPIOx_MODER register.
- The specific alternate function assignments for each pin are detailed in the device datasheet.
- Cortex-M7 with FPU EVENTOUT is mapped on AF15

In addition to this flexible I/O multiplexing architecture, each peripheral has alternate functions mapped onto different I/O pins to optimize the number of peripherals available in smaller packages.

To use an I/O in a given configuration, the user has to proceed as follows:

- **Debug function:** after each device reset these pins are assigned as alternate function pins immediately usable by the debugger host
- **System function:** MCOx pins have to be configured in alternate function mode.
- **GPIO:** configure the desired I/O as output, input or analog in the GPIOx_MODER register.
- **Peripheral alternate function:**
 - Connect the I/O to the desired AFx in one of the GPIOx_AFRL or GPIOx_AFRH register.
 - Select the type, pull-up/pull-down and output speed via the GPIOx_OTYPER, GPIOx_PUPDR and GPIOx_OSPEEDER registers, respectively.

- Configure the desired I/O as an alternate function in the GPIOx_MODER register.
- **Additional functions:**
 - For the ADC and DAC, configure the desired I/O in analog mode in the GPIOx_MODER register and configure the required function in the ADC and DAC registers.
 - For the additional functions like RTC_OUT, RTC_TS, RTC_TAMPx, WKUPx and oscillators, configure the required function in the related RTC, PWR and RCC registers. These functions have priority over the configuration in the standard GPIO registers. For details about I/O control by the RTC, refer to [Section 49.3: RTC functional description on page 2057](#).
- EVENTOUT
 - Configure the I/O pin used to output the core EVENTOUT signal by connecting it to AF15.

Refer to the “Alternate function mapping” table in the device datasheet for the detailed mapping of the alternate function I/O pins.

12.3.3 I/O port control registers

Each of the GPIO ports has four 32-bit memory-mapped control registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR) to configure up to 16 I/Os. The GPIOx_MODER register is used to select the I/O mode (input, output, AF, analog). The GPIOx_OTYPER and GPIOx_OSPEEDR registers are used to select the output type (push-pull or open-drain) and speed. The GPIOx_PUPDR register is used to select the pull-up/pull-down whatever the I/O direction.

12.3.4 I/O port data registers

Each GPIO has two 16-bit memory-mapped data registers: input and output data registers (GPIOx_IDR and GPIOx_ODR). GPIOx_ODR stores the data to be output, it is read/write accessible. The data input through the I/O are stored into the input data register (GPIOx_IDR), a read-only register.

See [Section 12.4.5: GPIO port input data register \(GPIOx_IDR\) \(x = A to K\)](#) and [Section 12.4.6: GPIO port output data register \(GPIOx_ODR\) \(x = A to K\)](#) for the register descriptions.

12.3.5 I/O data bitwise handling

The bit set reset register (GPIOx_BSRR) is a 32-bit register which allows the application to set and reset each individual bit in the output data register (GPIOx_ODR). The bit set reset register has twice the size of GPIOx_ODR.

To each bit in GPIOx_ODR, correspond two control bits in GPIOx_BSRR: BS(i) and BR(i). When written to 1, bit BS(i) **sets** the corresponding ODR(i) bit. When written to 1, bit BR(i) **resets** the ODR(i) corresponding bit.

Writing any bit to 0 in GPIOx_BSRR does not have any effect on the corresponding bit in GPIOx_ODR. If there is an attempt to both set and reset a bit in GPIOx_BSRR, the set action takes priority.

Using the GPIOx_BSRR register to change the values of individual bits in GPIOx_ODR is a “one-shot” effect that does not lock the GPIOx_ODR bits. The GPIOx_ODR bits can always

be accessed directly. The GPIOx_BSRR register provides a way of performing atomic bitwise handling.

There is no need for the software to disable interrupts when programming the GPIOx_ODR at bit level: it is possible to modify one or more bits in a single atomic AHB write access.

12.3.6 GPIO locking mechanism

It is possible to freeze the GPIO control registers by applying a specific write sequence to the GPIOx_LCKR register. The frozen registers are GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL and GPIOx_AFRH.

To write the GPIOx_LCKR register, a specific write / read sequence has to be applied. When the right LOCK sequence is applied to bit 16 in this register, the value of LCKR[15:0] is used to lock the configuration of the I/Os (during the write sequence the LCKR[15:0] value must be the same). When the LOCK sequence has been applied to a port bit, the value of the port bit can no longer be modified until the next MCU reset or peripheral reset. Each GPIOx_LCKR bit freezes the corresponding bit in the control registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL and GPIOx_AFRH).

The LOCK sequence (refer to [Section 12.4.8: GPIO port configuration lock register \(GPIOx_LCKR\) \(x = A to K\)](#)) can only be performed using a word (32-bit long) access to the GPIOx_LCKR register due to the fact that GPIOx_LCKR bit 16 has to be set at the same time as the [15:0] bits.

For more details refer to LCKR register description in [Section 12.4.8: GPIO port configuration lock register \(GPIOx_LCKR\) \(x = A to K\)](#).

12.3.7 I/O alternate function input/output

Two registers are provided to select one of the alternate function inputs/outputs available for each I/O. With these registers, the user can connect an alternate function to some other pin as required by the application.

This means that a number of possible peripheral functions are multiplexed on each GPIO using the GPIOx_AFRL and GPIOx_AFRH alternate function registers. The application can thus select any one of the possible functions for each I/O. The AF selection signal being common to the alternate function input and alternate function output, a single channel is selected for the alternate function input/output of a given I/O.

To know which functions are multiplexed on each GPIO pin refer to the device datasheet.

12.3.8 External interrupt/wake-up lines

All ports have external interrupt capability. To use external interrupt lines, the port must not be configured in analog mode.

Refer to [Section 21: Extended interrupt and event controller \(EXTI\)](#) and to [Section 21.3: EXTI functional description](#).

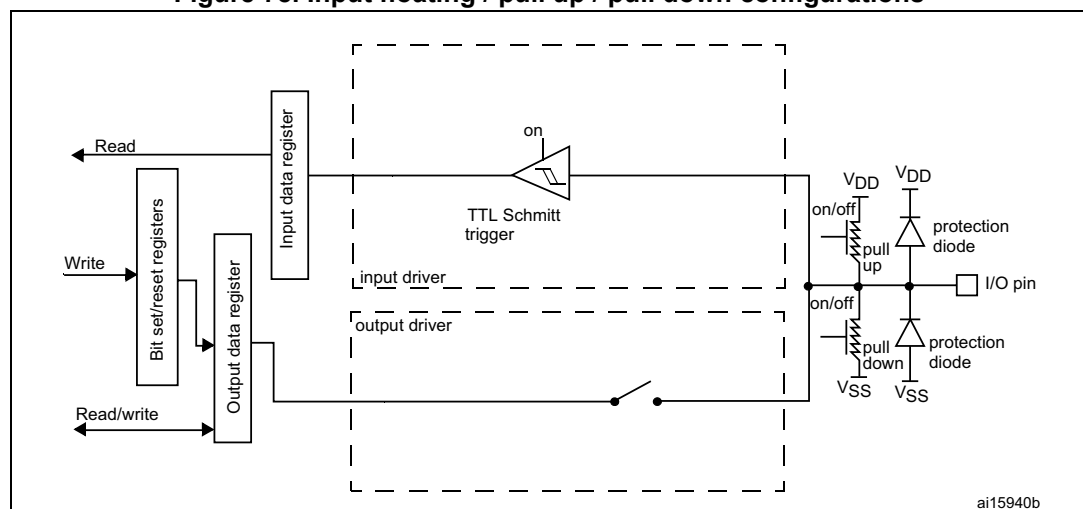
12.3.9 Input configuration

When the I/O port is programmed as input:

- The output buffer is disabled
- The Schmitt trigger input is activated
- The pull-up and pull-down resistors are activated depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register provides the I/O state

Figure 78 shows the input configuration of the I/O port bit.

Figure 78. Input floating / pull up / pull down configurations



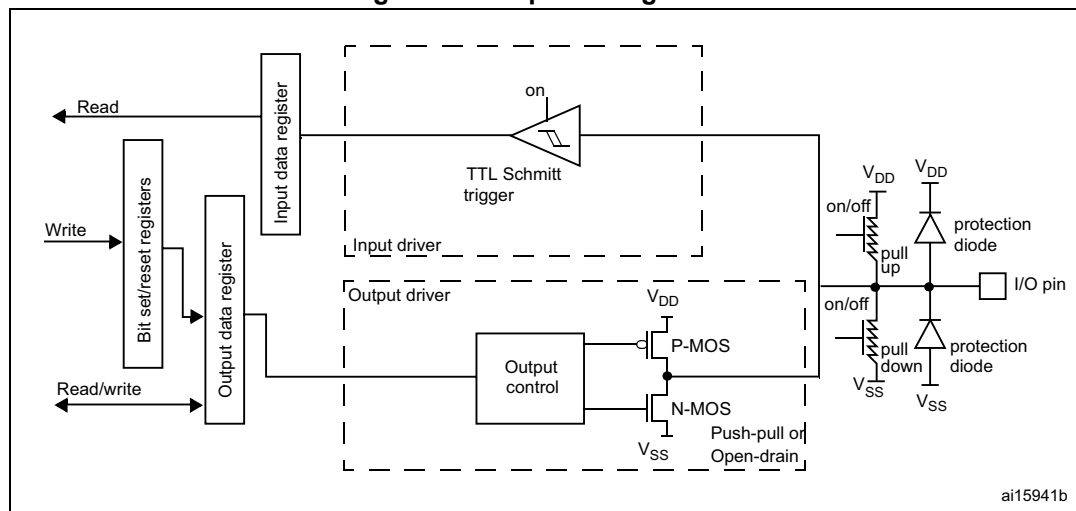
12.3.10 Output configuration

When the I/O port is programmed as output:

- The output buffer is enabled:
 - Open drain mode: a “0” in the output register activates the N-MOS whereas a “1” in the output register leaves the port in Hi-Z (the P-MOS is never activated)
 - Push-pull mode: a “0” in the output register activates the N-MOS whereas a “1” in the output register activates the P-MOS
- The Schmitt trigger input is activated
- The pull-up and pull-down resistors are activated depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register gets the I/O state
- A read access to the output data register gets the last written value

Figure 79 shows the output configuration of the I/O port bit.

Figure 79. Output configuration



12.3.11 I/O compensation cell

This cell is used to control the I/O commutation slew rate (t_{fall} / t_{rise}) to reduce the I/O noise on power supply.

The cell is split into two blocks:

- The first block provides an optimal code for the current PVT. The code stored in this block can be read when the READY flag of the SYSCFG_CCSR is set.
- The second block controls the I/O slew rate. The user selects the code to be applied and programs it by software.

The I/O compensation cell features 2 voltage ranges: 1.62 to 2.0 V and 2.7 to 3.6 V.

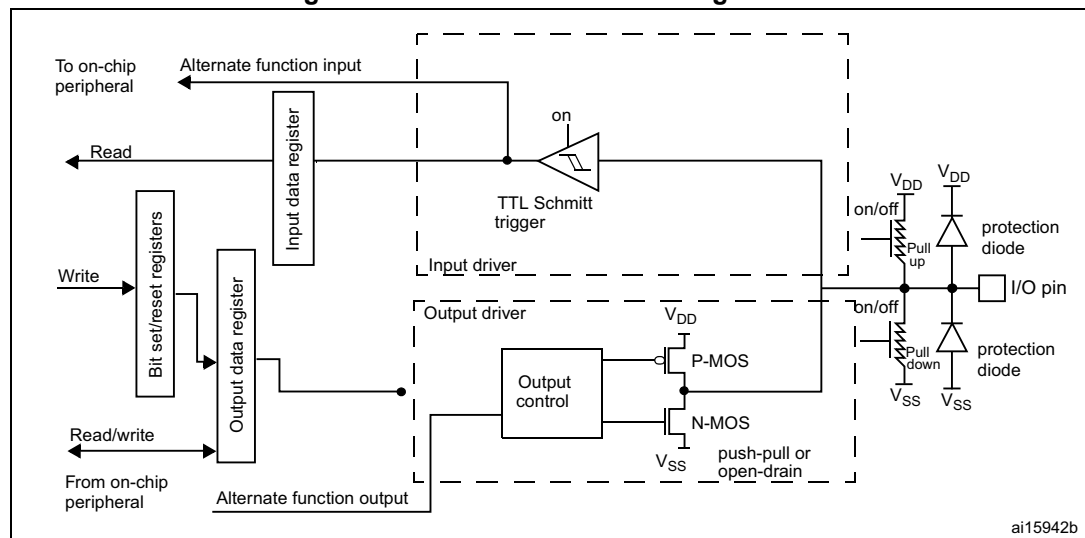
12.3.12 Alternate function configuration

When the I/O port is programmed as alternate function:

- The output buffer can be configured in open-drain or push-pull mode
- The output buffer is driven by the signals coming from the peripheral (transmitter enable and data)
- The Schmitt trigger input is activated
- The weak pull-up and pull-down resistors are activated or not depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register gets the I/O state

Figure 80 shows the alternate function configuration of the I/O port bit.

Figure 80. Alternate function configuration



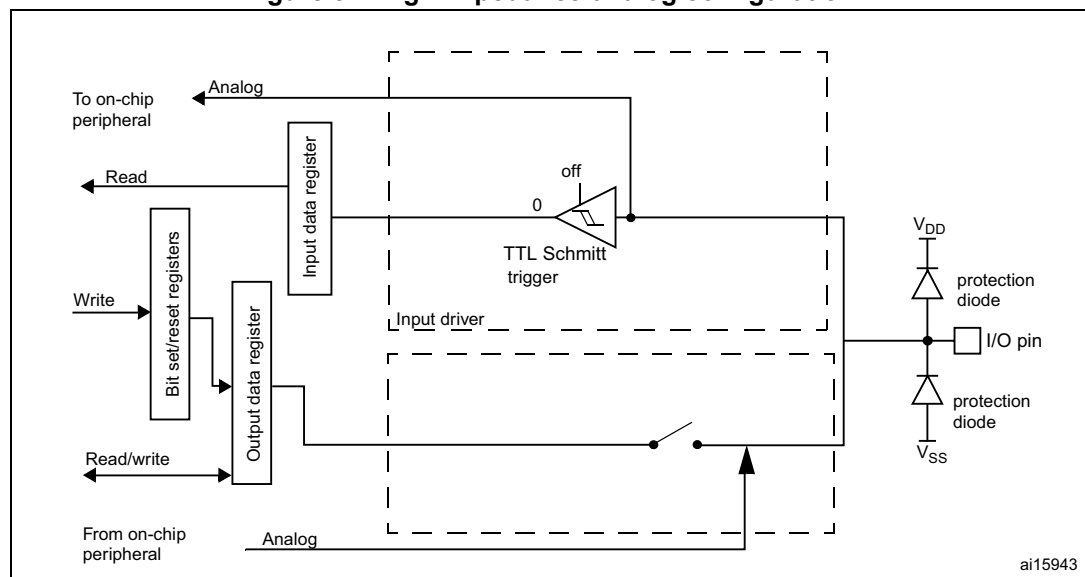
12.3.13 Analog configuration

When the I/O port is programmed as analog configuration:

- The output buffer is disabled
- The Schmitt trigger input is deactivated, providing zero consumption for every analog value of the I/O pin. The output of the Schmitt trigger is forced to a constant value (0).
- The weak pull-up and pull-down resistors are disabled by hardware
- Read access to the input data register gets the value "0"

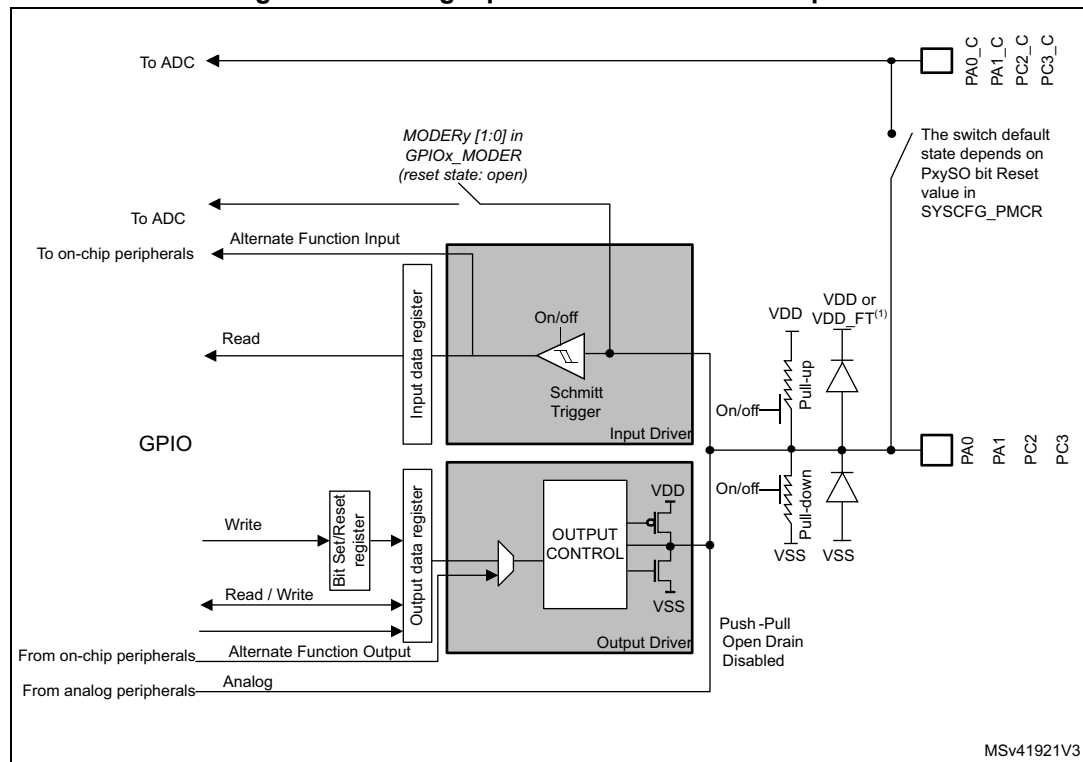
Figure 81 shows the high-impedance, analog-input configuration of the I/O port bits.

Figure 81. High impedance-analog configuration



Some pins/balls are directly connected to PA0_C, PA1_C, PC2_C and PC3_C ADC analog inputs (see [Figure 82](#)): there is a direct path between Pxy_C and Pxy pins/balls, through an analog switch (refer to [Section 13.3.1: SYSCFG peripheral mode configuration register \(SYSCFG_PMCr\)](#) for details on how to configure analog switches).

Figure 82. Analog inputs connected to ADC inputs



1. VDD_FT is a potential specific to 5V tolerant I/Os. It is distinct from VDD.

12.3.14 Using the HSE or LSE oscillator pins as GPIOs

When the HSE or LSE oscillator is switched OFF (default state after reset), the related oscillator pins can be used as normal GPIOs.

When the HSE or LSE oscillator is switched ON (by setting the HSEON or LSEON bit in the RCC_CSR register) the oscillator takes control of its associated pins and the GPIO configuration of these pins has no effect.

When the oscillator is configured in a user external clock mode, only the OSC_IN or OSC32_IN pin is reserved for clock input and the OSC_OUT or OSC32_OUT pin can still be used as normal GPIO.

12.3.15 Using the GPIO pins in the backup supply domain

The PC13/PC14/PC15/PI8 GPIO functionality is lost when the core supply domain is powered off (when the device enters Standby mode). In this case, if their GPIO configuration is not bypassed by the RTC configuration, these pins are set in an analog input mode.

12.4 GPIO registers

For a summary of register bits, register address offsets and reset values, refer to [Table 98](#).

The peripheral registers can be written in word, half word or byte mode.

12.4.1 GPIO port mode register (GPIOx_MODER) (x = A to K)

Address offset: 0x00

Reset value: 0xABFF FFFF for port A

Reset value: 0xFFFF FEBF for port B

Reset value: 0xFFFF FFFF for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **MODER[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

00: Input mode

01: General purpose output mode

10: Alternate function mode

11: Analog mode (reset state)

12.4.2 GPIO port output type register (GPIOx_OTYPER) (x = A to K)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **OT[15:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O output type.

0: Output push-pull (reset state)

1: Output open-drain

12.4.3 GPIO port output speed register (GPIOx_OSPEEDR) (x = A to K)

Address offset: 0x08

Reset value: 0x0C00 0000 (for port A)

Reset value: 0x0000 00C0 (for port B)

Reset value: 0x0000 0000 (for other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15 [1:0]		OSPEEDR14 [1:0]		OSPEEDR13 [1:0]		OSPEEDR12 [1:0]		OSPEEDR11 [1:0]		OSPEEDR10 [1:0]		OSPEEDR9 [1:0]		OSPEEDR8 [1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7 [1:0]		OSPEEDR6 [1:0]		OSPEEDR5 [1:0]		OSPEEDR4 [1:0]		OSPEEDR3 [1:0]		OSPEEDR2 [1:0]		OSPEEDR1 [1:0]		OSPEEDR0 [1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **OSPEEDR[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O output speed.

00: Low speed

01: Medium speed

10: High speed

11: Very high speed

Note: Refer to the product datasheets for the values of OSPEEDRy bits versus V_{DD} range and external load.

12.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A to K)

Address offset: 0x0C

Reset value: 0x6400 0000 (for port A)

Reset value: 0x0000 0100 (for port B)

Reset value: 0x0000 0000 (for other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **PUPDR[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

01: Pull-up

10: Pull-down

11: Reserved

12.4.5 GPIO port input data register (GPIOx_IDR) (x = A to K)

Address offset: 0x10

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDR[15:0]**: Port x input data I/O pin y (y = 15 to 0)

These bits are read-only. They contain the input value of the corresponding I/O port.

12.4.6 GPIO port output data register (GPIOx_ODR) (x = A to K)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODR[15:0]**: Port output data I/O pin y (y = 15 to 0)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and/or reset by writing to the GPIOx_BSRR register (x = A..F).

12.4.7 GPIO port bit set/reset register (GPIOx_BSRR) (x = A to K)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BR[15:0]**: Port x reset I/O pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Resets the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BS[15:0]**: Port x set I/O pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Sets the corresponding ODRx bit

12.4.8 GPIO port configuration lock register (GPIOx_LCKR) (x = A to K)

This register is used to lock the configuration of the port bits when a correct write sequence is applied to bit 16 (LCKK). The value of bits [15:0] is used to lock the configuration of the GPIO. During the write sequence, the value of LCKR[15:0] must not change. When the LOCK sequence has been applied on a port bit, the value of this port bit can no longer be modified until the next MCU reset or peripheral reset.

Note: A specific write sequence is used to write to the GPIOx_LCKR register. Only word access (32-bit long) is allowed during this locking sequence.

Each lock bit freezes a specific configuration register (control and alternate function registers).

Address offset: 0x1C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LCKK
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **LCKK**: Lock key

This bit can be read any time. It can only be modified using the lock key write sequence.

0: Port configuration lock key not active

1: Port configuration lock key active. The GPIOx_LCKR register is locked until the next MCU reset or peripheral reset.

LOCK key write sequence:

WR LCKR[16] = 1 + LCKR[15:0]

WR LCKR[16] = 0 + LCKR[15:0]

WR LCKR[16] = 1 + LCKR[15:0]

RD LCKR

RD LCKR[16] = 1 (this read operation is optional but it confirms that the lock is active)

Note: During the LOCK key write sequence, the value of LCK[15:0] must not change.

Any error in the lock sequence aborts the lock.

After the first lock sequence on any bit of the port, any read access on the LCKK bit returns 1 until the next MCU reset or peripheral reset.

Bits 15:0 **LCK[15:0]**: Port x lock I/O pin y (y = 15 to 0)

These bits are read/write but can only be written when the LCKK bit is 0.

0: Port configuration not locked

1: Port configuration locked

12.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A to K)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR7[3:0]				AFR6[3:0]				AFR5[3:0]				AFR4[3:0]			
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR3[3:0]				AFR2[3:0]				AFR1[3:0]				AFR0[3:0]			
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **AFR[7:0][3:0]**: Alternate function selection for port x I/O pin y (y = 7 to 0)

These bits are written by software to configure alternate function I/Os.

0000: AF0
 0001: AF1
 0010: AF2
 0011: AF3
 0100: AF4
 0101: AF5
 0110: AF6
 0111: AF7
 1000: AF8
 1001: AF9
 1010: AF10
 1011: AF11
 1100: AF12
 1101: AF13
 1110: AF14
 1111: AF15

12.4.10 GPIO alternate function high register (GPIOx_AFRH) (x = A to J)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR15[3:0]				AFR14[3:0]				AFR13[3:0]				AFR12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR11[3:0]				AFR10[3:0]				AFR9[3:0]				AFR8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFR[15:8][3:0]**: Alternate function selection for port x I/O pin y (y = 15 to 8)

These bits are written by software to configure alternate function I/Os.

0000: AF0
 0001: AF1
 0010: AF2
 0011: AF3
 0100: AF4
 0101: AF5
 0110: AF6
 0111: AF7
 1000: AF8
 1001: AF9
 1010: AF10
 1011: AF11
 1100: AF12
 1101: AF13
 1110: AF14
 1111: AF15

12.4.11 GPIO register map

The following table gives the GPIO register map and reset values.

Table 98. GPIO register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOA_MODER	MODER15[1:0]																															
	Reset value	1	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x00	GPIOB_MODER	MODER15[1:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1
0x00	GPIOx_MODER (where x = C..K)	MODER15[1:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1
0x04	GPIOx_OTYPER (where x = A to K)	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	GPIOA_OSPEEDR	OSPEEDR15[1:0]																															
	Reset value	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	GPIOB_OSPEEDR	OSPEEDR15[1:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
0x08	GPIOx_OSPEEDR (where x = C..K)	OSPEEDR15[1:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	GPIOA_PUPDR	PUPDR15[1:0]																															
	Reset value	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 98. GPIO register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0C	GPIOB_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0x0C	GPIOx_PUPDR (where x = C..K)	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x10	GPIOx_IDR (where x = A..I/J/K)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
	Reset value																	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
0x14	GPIOx_ODR (where x = A to K)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x18	GPIOx_BSRR (where x = A..I/J/K)	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1C	GPIOx_LCKR (where x = A to K)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LCKK	LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x20	GPIOx_AFRL (where x = A to K)	AFR7[3:0]			AFR6[3:0]			AFR5[3:0]			AFR4[3:0]			AFR3[3:0]			AFR2[3:0]			AFR1[3:0]			AFR0[3:0]										
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x24	GPIOx_AFRH (where x = A to K)	AFR15[3:0]			AFR14[3:0]			AFR13[3:0]			AFR12[3:0]			AFR11[3:0]			AFR10[3:0]			AFR9[3:0]			AFR8[3:0]										
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

13 System configuration controller (SYSCFG)

13.1 Introduction

The devices feature a set of configuration registers. The objectives of this section is to describe in details the system configuration controller.

13.2 SYSCFG main features

The system configuration controller main functions are the following:

- Analog switch configuration management
- I2C Fm+ configuration
- Selection of the Ethernet PHY interface.
- Management of the external interrupt line connection to the GPIOs
- Management of the I/O compensation cell
- Getting readout protection and Flash memory bank swap informations
- Management of boot sequences and boot addresses
- Management BOR reset level
- Management of Flash memory secured and protected sector status
- Management Flash memory write protections status
- Management of DTCM secured section status
- Management of independent watchdog behavior (hardware or software / freeze)
- Reset generation in Stop and Standby mode
- Secure mode enabling/disabling status.

13.3 SYSCFG registers

13.3.1 SYSCFG peripheral mode configuration register (SYSCFG_PMCR)

Address offset: 0x04

Reset value: 0x0X00 0000

Note: SYSCFG_PMCR reset value depends on the package.

'X' corresponds to PC3, PC2, PA1 and PA0 Switch Open bit reset value (PXnSO). PXnSO reset value is 0 when the corresponding PXn_C pin is available on the package but PXn is not. Otherwise, it is 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	PC3SO	PC2SO	PA1SO	PA0SO	EPIS[2:0]			Res.	Res.	Res.	Res.	Res.
				rw	rw	rw	rw	rw	rw	rw					

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	BOOSTV DSEL	BOOSTE	PB9 FMP	PB8 FMP	PB7 FMP	PB6 FMP	I2C4 FMP	I2C3 FMP	I2C2 FMP	I2C1 FMP
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:28 Reserved, must be kept at reset value.

Bit 27 **PC3SO**: PC3 Switch Open

This bit controls the analog switch between PC3 and PC3_C (dual pad)

0: Analog switch closed (pads are connected through the analog switch)

1: Analog switch open (2 separated pads)

Bit 26 **PC2SO**: PC2 Switch Open

This bit controls the analog switch between PC2 and PC2_C (dual pad)

0: Analog switch closed (pads are connected through the analog switch)

1: Analog switch open (2 separated pads)

Bit 25 **PA1SO**: PA1 Switch Open

This bit controls the analog switch between PA1 and PA1_C (dual pad)

0: Analog switch closed (pads are connected through the analog switch)

1: Analog switch open (2 separated pads)

Bit 24 **PA0SO**: PA0 Switch Open

This bit controls the analog switch between PA0 and PA0_C (dual pad)

0: Analog switch closed (pads are connected through the analog switch)

1: Analog switch open (2 separated pads)

Bits 23:21 **EPIS[2:0]**: Ethernet PHY Interface Selection

These bits select the Ethernet PHY interface.

000: MII

001: Reserved

010: Reserved

011: Reserved

100: RMII

101: Reserved

110: Reserved

111: Reserved

Bits 20:10 Reserved, must be kept at reset value.

Bit 9 **BOOSTVDDSEL**: Analog switch supply voltage selection (V_{DD} / V_{DDA} /booster)

To avoid current consumption due to booster activation when $V_{DDA} < 2.7$ V and $V_{DD} > 2.7$ V, V_{DD} can be selected as supply voltage for analog switches. In this case, the BOOSTE bit should be cleared to avoid unwanted power consumption.

When both $V_{DD} < 2.7$ V and $V_{DDA} < 2.7$ V, the booster is still needed to obtain full AC performances from I/O analog switches.

0: V_{DDA} selected as analog switch supply voltage (when BOOSTE bit is cleared)

1: V_{DD} selected as analog switch supply voltage

Bit 8 **BOOSTE**: Booster Enable

This bit enables the booster to reduce the total harmonic distortion of the analog switch when the supply voltage is lower than 2.7 V.

Activating the booster allows to guaranty the analog switch AC performance when the supply voltage is below 2.7 V: in this case, the analog switch performance is the same on the full voltage range.

0: Booster disabled

1: Booster enabled

Bit 7 **PB9FMP**: PB(9) Fm+

This bit enables I2C Fm+ on PB(9).

0: Fm+ disabled

1: Fm+ enabled

Bit 6 **PB8FMP**: PB(8) Fm+

This bit enables I2C Fm+ on PB(8).

0: Fm+ disabled

1: Fm+ enabled

Bit 5 **PB7FMP**: PB(7) Fm+

this bit enables I2C Fm+ on PB(7).

0: Fm+ disabled

1: Fm+ enabled

Bit 4 **PB6FMP**: PB(6) Fm+

This bit enables I2C Fm+ on PB(6).

0: Fm+ disabled

1: Fm+ enabled

Bit 3 **I2C4FMP**: I2C4 Fm+

This bit enables Fm+ on I2C4.

0: Fm+ disabled

1: Fm+ enabled

Bit 2 **I2C3FMP**: I2C3 Fm+

This bit enables Fm+ on I2C3.

0: Fm+ disabled

1: Fm+ enabled

Bit 1 **I2C2FMP**: I2C2 Fm+

This bit enables Fm+ on I2C2.

0: Fm+ disabled

1: Fm+ enabled

Bit 0 **I2C1FMP**: I2C1 Fm+

This bit enables Fm+ on I2C1.

0: Fm+ disabled

1: Fm+ enabled

13.3.2 SYSCFG external interrupt configuration register 1 (SYSCFG_EXTICR1)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 0 to 3)

These bits are written by software to select the source input for the EXTI input for external interrupt / event detection.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: PF[x] pin

0110: PG[x] pin

0111: PH[x] pin

1000: PI[x] pin

1001: PJ[x] pin

1010: PK[x] pin

Other configurations: reserved

13.3.3 SYSCFG external interrupt configuration register 2 (SYSCFG_EXTICR2)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI7[3:0]				EXTI6[3:0]				EXTI5[3:0]				EXTI4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 4 to 7)

These bits are written by software to select the source input for the EXTI input for external interrupt / event detection.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: PF[x] pin

0110: PG[x] pin

0111: PH[x] pin

1000: PI[x] pin

1001: PJ[x] pin

1010: PK[x] pin

Other configurations: reserved

13.3.4 SYSCFG external interrupt configuration register 3 (SYSCFG_EXTICR3)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI11[3:0]				EXTI10[3:0]				EXTI9[3:0]				EXTI8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 8 to 11)

These bits are written by software to select the source input for the EXTI input for external interrupt / event detection.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: PF[x] pin

0110: PG[x] pin

0111: PH[x] pin

1000: PI[x] pin

1001: PJ[x] pin

1010: PK[x] pin

Other configurations: reserved

Note: PK[11:8] are not used

13.3.5 SYSCFG external interrupt configuration register 4 (SYSCFG_EXTICR4)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 12 to 15)

These bits are written by software to select the source input for the EXTI input for external interrupt / event detection.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: PF[x] pin

0110: PG[x] pin

0111: PH[x] pin

1000: PI[x] pin

1001: PJ[x] pin

1010: PK[x] pin

Other configurations: reserved

Note: PK[15:12] are not used.

13.3.6 SYSCFG configuration register (SYSCFG_CFGR)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AXI SRAML	ITCML	DTCML	SRAM1L	SRAM2L	SRAM3L	SRAM4L	Res.	BK RAML	CM7L	Res.	Res.	FLASHL	PVDL	Res.	CM4L
rs	rs	rs	rs	rs	rs	rs		rs	rs			rs	rs		rs

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **AXISRAML**: D1 AXI-SRAM double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the AXI-SRAM double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: AXI-SRAM double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: AXI-SRAM double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs.

Bit 14 **ITCML**: D1 ITCM double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the ITCM double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: ITCM double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: ITCM double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 13 **DTCML**: D1 DTCM double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the DTCM double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: DTCM double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: DTCM double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 12 **SRAM1L**: D2 SRAM1 double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the D2 SRAM1 double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: D2 SRAM1 double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: D2 SRAM1 double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 11 SRAM2L: D2 SRAM2 double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the D2 SRAM2 double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: D2 SRAM2 double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: D2 SRAM2 double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 10 SRAM3L: D2 SRAM3 double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the D2 SRAM3 double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: D2 SRAM3 double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: D2 SRAM3 double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 9 SRAM4L: D3 SRAM4 double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the D3 SRAM4 double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: D3 SRAM4 double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: D3 SRAM4 double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 8 Reserved, must be kept at reset value.

Bit 7 BKRAML: Backup SRAM double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the Backup SRAM double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: Backup SRAM double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: Backup SRAM double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 6 CM7L: Arm[®] Cortex[®]-M7 LOCKUP (HardFault) output enable bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the connection of the Arm[®] Cortex[®]-M7 LOCKUP (HardFault) output to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: Arm[®] Cortex[®]-M7 LOCKUP output disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: Arm[®] Cortex[®]-M7 LOCKUP output connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bits 5:4 Reserved, must be kept at reset value.

Bit 3 FLASHL: FLASH double ECC error lock bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the FLASH double ECC error flag connection to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: FLASH double ECC error flag disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: FLASH double ECC error flag connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 2 PVDL: PVD lock enable bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the PVD connection to TIM1/8/15/16/17 and HRTIMER Break inputs, as well as the PVDE and PLS[2:0] in the PWR_CR1 register.

0: PVD signal disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: PVD signal connected to TIM1/8/15/16/17/HRTIMER Break inputs

Bit 1 Reserved, must be kept at reset value.**Bit 0 CM4L:** Arm® Cortex®-M4 LOCKUP (Hardfault) output enable bit

This bit is set by software and cleared only by a system reset. It can be used to enable and lock the connection of Arm® Cortex®-M4 LOCKUP (Hardfault) output to TIM1/8/15/16/17 and HRTIMER Break inputs.

0: Arm® Cortex®-M4 LOCKUP output disconnected from TIM1/8/15/16/17/HRTIMER Break inputs

1: Arm® Cortex®-M4 LOCKUP output connected to TIM1/8/15/16/17/HRTIMER Break inputs

13.3.7 SYSCFG compensation cell control/status register (SYSCFG_CCCSR)

Address offset: 0x20

Reset value: 0x0000 0000

Refer to [Section 12.3.11: I/O compensation cell](#) for a detailed description of I/O compensation mechanism.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HSLV
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	READY	Res.	Res.	Res.	Res.	Res.	Res.	CS	EN
							r							rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **HSLV**: High-speed at low-voltage

This bit is written by software to optimize the I/O speed when the product voltage is low.

This bit is active only if IO_HSLV user option bit is set. It must be used only if the product supply voltage is below 2.7 V. Setting this bit when V_{DD} is higher than 2.7 V might be destructive.

0: No I/O speed optimization

1: I/O speed optimization

Bits 15:9 Reserved, must be kept at reset value.

Bit 8 **READY**: Compensation cell ready flag

This bit provides the status of the compensation cell.

0: I/O compensation cell not ready

1: I/O compensation cell ready

Note: The CSI clock is required for the compensation cell to work properly. The compensation cell ready bit (READY) is not set if the CSI clock is not enabled.

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **CS**: Code selection

This bit selects the code to be applied for the I/O compensation cell.

0: Code from the cell (available in the SYSCFG_CCVR)

1: Code from the SYSCFG compensation cell code register (SYSCFG_CCCR)

Bit 0 **EN**: Enable

This bit enables the I/O compensation cell.

0: I/O compensation cell disabled

1: I/O compensation cell enabled

13.3.8 SYSCFG compensation cell value register (SYSCFG_CCVR)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCV[3:0]				NCV[3:0]			
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:4 **PCV[3:0]**: PMOS compensation value

This value is provided by the cell and can be used by the CPU to compute an I/O compensation cell code for PMOS transistors. This code is applied to the I/O compensation cell when the CS bit of the SYSCFG_CCCSR is reset.

Bits 3:0 **NCV[3:0]**: NMOS compensation value

This value is provided by the cell and can be used by the CPU to compute an I/O compensation cell code for NMOS transistors. This code is applied to the I/O compensation cell when the CS bit of the SYSCFG_CCCSR is reset.

13.3.9 SYSCFG compensation cell code register (SYSCFG_CCCR)

Address offset: 0x28

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCC[3:0]				NCC[3:0]			
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:4 **PCC[3:0]**: PMOS compensation code

These bits are written by software to define an I/O compensation cell code for PMOS transistors. This code is applied to the I/O compensation cell when the CS bit of the SYSCFG_CCCSR is set.

Bits 3:0 **NCC[3:0]**: NMOS compensation code

These bits are written by software to define an I/O compensation cell code for NMOS transistors. This code is applied to the I/O compensation cell when the CS bit of the SYSCFG_CCCSR is set.

13.3.10 SYSCFG power control register (SYSCFG_PWRCR)

Address Offset: 0x2C

Reset Value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ODEN
															rw

Bits 31:1 Reserved, must be kept at reset value.

- Bit 0 **ODEN**: Overdrive enable, this bit allows to activate the LDO regulator overdrive mode.
 This bit must be written only in VOS1 voltage scaling mode.
 0: Overdrive mode disabled
 1: Overdrive mode enabled (the LDO generates VOS0 for V_{CORE})

Note: VOS0 must be activated only in VOS1 mode. It must be disabled by software before entering low-power mode (execution of WFE/WFI instruction).

13.3.11 SYSCFG system register (SYSCFG_SR0)

Address offset: 0x100

Reset value: 0x0000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CM4 PRESENT
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:17 Reserved, must be kept at reset value.

- Bit 16 **CM4PRESENT**: Arm® Cortex® -M4 core
 This bit indicates that the Arm® Cortex® -M4 core is present.
 0: Arm® Cortex® -M4 core not present
 1: Arm® Cortex® -M4 core present

Bits 15:0 Reserved, must be kept at reset value.

13.3.12 SYSCFG package register (SYSCFG_PKGR)

Address offset: 0x124

Reset value: 0x0000 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PKG[3:0]			
												r	r	r	r

Bits 31:4 Reserved, must be kept at reset value.

Bits 3:0 **PKG[3:0]**: Package

These bits indicate the device package.

0010: UFBGA169/LQFP176 (STM32H7x7)

0011: LQFP144 (STM32H7x5)

0110: LQFP176 (STM32H7x5)

0111: UFBGA176 (STM32H7x5)

1001: LQFP208 (STM32H7x7)

1010: LQFP208 (STM32H7x5)

Other configurations: all pads enabled

13.3.13 SYSCFG user register 0 (SYSCFG_UR0)

Address offset: 0x300

Reset value: 0x00XX 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RDP[7:0]							
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BKS
															r

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **RDP[7:0]**: Readout protection

These bits indicate the readout protection level.

0xAA: Level 0 (no protection)

0xCC: Level 2 (Flash memory readout protected, full debug features, boot from SRAM and boundary scan disabled)

Other configurations: Level 1 (Flash memory readout protected, limited debug features and boundary scan enabled)

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **BKS**: Bank Swap

This bit indicates Flash memory bank mapping.

0: Flash memory bank addresses are inverted

1: Flash memory banks are mapped to their original addresses

13.3.14 SYSCFG user register 1 (SYSCFG_UR1)

Address offset: 0x304

Reset value: 0x000X 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BCM7
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BCM4
															rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **BCM7**: Boot Cortex-M7

This bit enables Cortex-M7 core boot.

0: Cortex-M7 core is clock gated

1: Cortex-M7 core boot is enabled

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **BCM4**: Boot Cortex-M4

This bit enables Cortex-M4 core boot.

0: Cortex-M4 core is clock gated

1: Cortex-M4 core boot is enabled

13.3.15 SYSCFG user register 2 (SYSCFG_UR2)

Address offset: 0x308

Reset value: 0xFFFF 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BCM7_ADD0[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BORH[1:0]	
														r	r

Bits 31:16 **BCM7_ADD0[15:0]**: Cortex-M7 Boot Address 0

These bits define the MSB of the Cortex-M7 core boot address when BOOT pin is low.

Bits 15:2 Reserved, must be kept at reset value.

Bits 1:0 **BORH[1:0]**: BOR_LVL Brownout Reset Threshold Level

These bits indicate the Brownout reset high level.

0x11: BOR Level 3

0x10: BOR Level 2

0x01: BOR Level 1

0x00: BOR OFF (Level 0)

13.3.16 SYSCFG user register 3 (SYSCFG_UR3)

Address offset: 0x30C

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BCM7_ADD1[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BCM4_ADD0[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **BCM7_ADD1[15:0]**: Cortex-M7 Boot Address 1

These bits define the MSB of the Cortex-M7 core boot address when BOOT pin is high.

Bits 15:0 **BCM4_ADD1[15:0]**: Boot Cortex-M4 Address 0

These bits define the MSB of the Cortex-M4 core boot address when BOOT pin is high.

13.3.17 SYSCFG user register 4 (SYSCFG_UR4)

Address offset: 0x310

Reset value: 0x000X XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MEPAD_1
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BCM4_ADD1[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **MEPAD_1**: Mass Erase Protected Area Disabled for bank 1

This bit indicates if the flash protected area (Bank 1) is affected by a mass erase.

0: When a mass erase occurs the protected area is erased

1: When a mass erase occurs the protected area is not erased

Bits 15:0 **BCM4_ADD0[15:0]**: Boot Cortex-M4 Address 1

These bits define the MSB of the Cortex-M4 core boot address when BOOT pin is low.

13.3.18 SYSCFG user register 5 (SYSCFG_UR5)

Address offset: 0x314

Reset value: 0x00XX 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPS_1[7:0]							
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MESAD_1
															r

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **WRPN_1[7:0]**: Write protection for flash bank 1

WRPN[i] bit indicates if the sector i of the Flash memory bank 1 is protected.

0: Write protection is active on sector i

1: Write protection is not active on sector i

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **MESAD_1**: Mass erase secured area disabled for bank 1

This bit indicates if the flash secured area (bank 1) is affected by a mass erase.

0: When a mass erase occurs the secured area is erased

1: When a mass erase occurs the secured area is not erased

13.3.19 SYSCFG user register 6 (SYSCFG_UR6)

Address offset: 0x318

Reset value: 0x0XXX 0XXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	PA_END_1[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PA_BEG_1[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:28 Reserved, must be kept at reset value.

Bits 23:16 **PA_END_1[11:0]**: Protected area end address for bank 1

End address for bank 1 protected area.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **PA_BEG_1[11:0]**: Protected area start address for bank 1

Start address for bank 1 protected area.

13.3.20 SYSCFG user register 7 (SYSCFG_UR7)

Address offset: 0x31C

Reset value: 0x0XXX 0XXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	SA_END_1[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	SA_BEG_1[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:28 Reserved, must be kept at reset value.

Bits 23:16 **SA_END_1[11:0]**: Secured area end address for bank 1
End address for bank 1 secured area.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **SA_BEG_1[11:0]**: Secured area start address for bank 1
Start address for bank 1 secured area.

13.3.21 SYSCFG user register 8 (SYSCFG_UR8)

Address offset: 0x320

Reset value: 0x000X 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MESAD_2
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MEPAD_2
															r

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **MESAD_2**: Mass erase secured area disabled for bank 2

This bit indicates if the Flash memory secured area (Bank 2) is affected by a mass erase.

0: When a mass erase occurs the secured area is erased

1: When a mass erase occurs the secured area is not erased

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **MEPAD_2**: Mass erase protected area disabled for bank 2

This bit indicates if the Flash memory protected area (Bank 2) is affected by a mass erase.

0: When a mass erase occurs the protected area is erased

1: When a mass erase occurs the protected area is not erased

13.3.22 SYSCFG user register 9 (SYSCFG_UR9)

Address offset: 0x324

Reset value: 0x0XXX 00XX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	PA_BEG_2[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPS_2[7:0]							
								r	r	r	r	r	r	r	r

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:16 **PA_BEG_2[11:0]**: Protected area start address for bank 2
 Start address for bank 2 protected area.

Bits 15:8 Reserved, must be kept at reset value.

Bits 7:0 **WRPN_2[7:0]**: Write protection for flash bank 2
 WRPN[i] bit indicates if the sector i of the Flash memory bank 2 is protected.
 0: Write protection is active on sector i
 1: Write protection is not active on sector i

13.3.23 SYSCFG user register 10 (SYSCFG_UR10)

Address offset: 0x328

Reset value: 0x0XXX 0XXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	SA_BEG_2[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PA_END_2[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:16 **SA_BEG_2[11:0]**: Secured area start address for bank 2
 Start address for bank 2 secured area.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **PA_END_2[11:0]**: Protected area end address for bank 2
 End address for bank 2 protected area.

13.3.24 SYSCFG user register 11 (SYSCFG_UR11)

Address offset: 0x32C

Reset value: 0x000X 0XXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IWDG1M
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	SA_END_2[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **IWDG1M**: Independent Watchdog 1 mode

This bit indicates the control mode of the Independent Watchdog 1 (IWDG1).

0: IWDG1 controlled by hardware

1: IWDG1 controlled by software

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **SA_END_2[11:0]**: Secured area end address for bank 2

End address for bank 2 secured area.

13.3.25 SYSCFG user register 12 (SYSCFG_UR12)

Address offset: 0x330

Reset value: 0x000X 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECURE
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IWDG2M
															r

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **SECURE**: Secure mode

This bit indicates the Secure mode status.

0: Secure mode disabled

1: Secure mode enabled

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **IWDG2M**: Independent Watchdog 2 mode

This bit indicates the control mode of the Independent Watchdog 2 (IWDG2).

0: IWDG2 controlled by hardware

1: IWDG2 controlled by software

13.3.26 SYSCFG user register 13 (SYSCFG_UR13)

Address offset: 0x334

Reset value: 0x000X 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D1SBRST
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDRS[1:0]	
														r	r

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **D1SBRST**: D1 Standby reset

This bit indicates if a reset is generated when D1 domain enters DStandby mode.

0: A reset is generated by entering D1 Standby mode

1: D1 Standby mode is entered without reset generation

Bits 15:2 Reserved, must be kept at reset value.

Bits 1:0 **SDRS[1:0]**: Secured DTCM RAM Size

These bits indicates the size of the secured DTCM RAM.

00: 2 Kbytes

01: 4 Kbytes

10: 8 Kbytes

11: 16 Kbytes

13.3.27 SYSCFG user register 14 (SYSCFG_UR14)

Address offset: 0x338

Reset value: 0x000X 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D2SBRST
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D1STPRST
															rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **D2SBRST**: D2 Standby Reset

This bit indicates if a reset is generated when D2 domain enters DStandby mode.

0: A reset is generated by entering D2 Standby mode

1: D2 Standby mode is entered without reset generation

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **D1STPRST**: D1 Stop Reset

This bit indicates if a reset is generated when D1 domain enters in DStop mode.

0: A reset is generated entering D1 Stop mode

1: D1 Stop mode is entered without reset generation

13.3.28 SYSCFG user register 15 (SYSCFG_UR15)

Address offset: 0x33C

Reset value: 0x000X 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FZIWGDS TB
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D2STP RST
															rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **FZIWGDS TB**: Freeze independent watchdogs in Standby mode

This bit indicates if the independent watchdogs are frozen in Standby mode.

0: Independent Watchdogs frozen in Standby mode

1: Independent Watchdogs running in Standby mode

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **D2STPRST**: D2 Stop Reset

This bit indicates if a reset is generated when D2 domain enters in DStop mode.

0: A reset is generated entering D2 Stop mode

1: D2 Stop mode is entered without reset generation

13.3.29 SYSCFG user register 16 (SYSCFG_UR16)

Address offset: 0x340

Reset value: 0x000X 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PKP
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FZIWDOG STP
															r

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **PKP**: Private key programmed

This bit indicates if the device private key is programmed.

0: Private key not programmed

1: Private key programmed

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **FZIWDOGSTP**: Freeze independent watchdogs in Stop mode

This bit indicates if the independent watchdogs are frozen in Stop mode.

0: Independent Watchdogs frozen in Stop mode

1: Independent Watchdogs running in Stop mode

13.3.30 SYSCFG user register 17 (SYSCFG_UR17)

Address offset: 0x344

Reset value: 0x0000 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IO_HSLV
															r

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **IO_HSLV**: I/O high speed / low voltage

This bit indicates that the IOHSLV option bit is set.

0: Product is working on the full voltage range

1: Product is working below 2.7 V

13.3.31 SYSCFG register maps

The following table gives the SYSCFG register map and the reset values.

Table 99. SYSCFG register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	Reserved	Reserved																															
0x04	SYSCFG_PMCRR	Res.																Res															
	Reset value					X	X	1	1	0	0	0												0	0	0	0	0	0	0	0	0	0
0x08	SYSCFG_EXTICR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		EXTI3[3:0]				EXTI2[3:0]			EXTI1[3:0]					EXTI0[3:0]		
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C	SYSCFG_EXTICR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		EXTI7[3:0]				EXTI6[3:0]			EXTI5[3:0]					EXTI4[3:0]		
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x10	SYSCFG_EXTICR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		EXTI11[3:0]				EXTI10[3:0]			EXTI9[3:0]					EXTI8[3:0]		
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x14	SYSCFG_EXTICR4	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		EXTI15[3:0]				EXTI14[3:0]			EXTI13[3:0]					EXTI12[3:0]		
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x18	SYSCFG_CFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0	AXISRAML	ITCML	DTCML	SRAM1L	SRAM2L	SRAM3L	SRAM4L	BKRAML	CM7L		Res.	FLASHL	PVDL	Res.	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x20	SYSCFG_CCSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HSLV	Res.	Res.	Res.	Res.	Res.	Res.	Res.	READY	Res.	Res.	Res.	Res.			
	Reset value																0								0						CS	EN	
0x24	SYSCFG_CCVR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.					PCV[3:0]		NCV[3:0]	
	Reset value																									0	0	0	0	0	0	0	0
0x28	SYSCFG_CCCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.					PCC[3:0]		NCC[3:0]	
	Reset value																									0	0	0	0	0	0	0	0
0x2C	SYSCFG_CCCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																															ODEN	
0x30 - 0x0FC	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x100	SYSCFG_SR0	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CM4PRESENT	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																0																

Table 99. SYSCFG register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x104 - 0x120	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.			
	Reset value																																			
0x124	SYSCFG_PKGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.			
	Reset value																																PKG[3:0]			
0x128 - 0x2FC	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.			
	Reset value																																			
0x300	SYSCFG_UR0	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RDP[7:0]								Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.			
	Reset value										x	x	x	x	x	x	x	x																BKS		
0x304	SYSCFG_UR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BCM7	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.			
	Reset value																X																BCM4			
0x308	SYSCFG_UR2	BCM7_ADD0[15:0]																Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X																X BORH[1:0]			
0x30C	SYSCFG_UR3	BCM7_ADD1[15:0]																BCM4_ADD0[15:0]																		
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x			
0x310	SYSCFG_UR4	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MEPAD_1	BCM4_ADD1[15:0]																		
	Reset value																X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			
0x314	SYSCFG_UR5	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPN_1[7:0]							Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.			
	Reset value										X	X	X	X	X	X	X	X															X MESAD_1			
0x318	SYSCFG_UR6	Res.	Res.	Res.	Res.	PA_END_1[11:0]											Res.	Res.	Res.	Res.	PA_BEG_1[11:0]															
	Reset value					X	X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X	X			
0x31C	SYSCFG_UR7	Res.	Res.	Res.	Res.	SA_END_1[11:0]											Res.	Res.	Res.	Res.	SA_BEG_1[11:0]															
	Reset value					X	X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X	X			
0x320	SYSCFG_UR8	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MESAD_2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.			
	Reset value																x																x MEPAD_2			
0x324	SYSCFG_UR9	Res.	Res.	Res.	Res.	PA_END_2[11:0]											Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value					X	X	X	X	X	X	X	X	X	X	X	X									X	X	X	X	X	X	X	X			
0x328	SYSCFG_UR10	Res.	Res.	Res.	Res.	SA_BEG_2[11:0]											Res.	Res.	Res.	Res.	PA_END_2[11:0]															
	Reset value					X	X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X	X			

Table 99. SYSCFG register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x32C	SYSCFG_UR11	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IWDG1M	Res.	Res.	Res.	Res.	SA_END_2[11:0]													
	Reset value																X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			
0x330	SYSCFG_UR12	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECURE	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	IWDG2M
	Reset value																X																X		
0x334	SYSCFG_UR13	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D1SBRST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDRS[1:0]		
	Reset value																X															X			
0x338	SYSCFG_UR14	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	D2SBRST	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	D1STPRST
	Reset value																X																X		
0x33C	SYSCFG_UR15	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FZIWGDGSTB	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	D2STPRST	
	Reset value																X																X		
0x340	SYSCFG_UR16	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PKP	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	FZIWGDGST
	Reset value																X																X		
0x344	SYSCFG_UR17	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	RES.	IO_HSLV	
	Reset value																																	Q	

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

14 Block interconnect

14.1 Peripheral interconnect

14.1.1 Introduction

Several peripherals have direct connections between them.

This enables autonomous communication and synchronization between peripherals, thus saving CPU resources and power consumption.

These hardware connections remove software latency, allow the design of a predictable system and result in a reduction of the number of pins and GPIOs.

14.1.2 Connection overview

There are several types of connections.

- Asynchronous connections (A)
The source output signal is sampled by the destination clock, leading to introduction of a possible jitter in the latency between the source output event and the destination event detection
- Synchronous connections (S)
Both source and destination are synchronous (they run on the same clock), and the latency from the source to the destination is deterministic. No jitter is introduced.
- Immediate connections (I)
Either the source or the destination is an analog signal.
- Break/fault connection for TIM/HRTIM outputs (B)
The source output signal disables the timer outputs through a pure combinational logic path, without any latency.

Table 100. Peripherals interconnect matrix (D2 domain) ^{(1) (2)}

Source			Destination																									
			D2 domain																			D3 domain						
			APB1							APB2							AHB1			AHB4	APB4							
			TIM2	TIM3	TIM4	TIM5	TIM12	LPTIM1	DAC	CRS	CAN	TIM1	TIM8	TIM15	TIM16	TIM17	DFSDM1	HRTIM	ADC1	ADC2	ETHERNET	ADC3	LPTIM2	LPTIM3	LPTIM4	LPTIM5	COMP1	COMP2
D2 domain	APB1	TIM2	-	S	S	-	-	-	S	-	A	S	S	S	-	-	-	S	S	S	A	S	-	-	-	-	I	I
		TIM3	S	-	S	S	-	-	-	-	A	S	-	S	-	-	S	S	S	S	A	S	-	-	-	-	I	I
		TIM4	S	S	-	S	S	-	S	-	-	S	S	S	-	-	S	-	S	S	-	S	-	-	-	-	-	-
		TIM5	-	-	-	-	S	-	S	-	-	-	S	-	-	-	-	-	-	-	-	S	-	-	-	-	-	-
		TIM6	-	-	-	-	-	-	S	-	-	-	-	-	-	-	S	S	S	S	-	S	-	-	-	-	-	-
		TIM7	-	-	-	-	-	-	S	-	-	-	-	-	-	-	S	S	-	-	-	-	-	-	-	-	-	-
		TIM13	-	-	-	-	S	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		TIM14	-	-	-	-	S	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		LPTIM1	-	-	-	-	-	-	A	-	-	-	-	-	-	-	A	A	A	A	-	A	-	-	-	-	-	-
		SPDIFRX	-	-	-	-	-	-	-	-	-	-	-	-	-	S	-	-	-	-	-	-	-	-	-	-	-	-
		OPAMP	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	A	-	-	-	-	-	-	-	-	-	-
		CAN	-	-	-	A	-	-	-	-	-	-	-	-	-	-	-	A	-	-	A	-	-	-	-	-	-	-
	APB2	TIM1	S	S	S	S	-	-	S	-	-	-	S	S	-	-	S	S	S	S	-	S	-	-	-	-	I	I
		TIM8	S	-	S	S	-	-	S	-	-	-	-	-	-	-	S	-	S	S	-	S	-	-	-	-	I	I
		TIM15	-	S	-	-	-	-	S	-	-	S	-	-	-	-	S	S	S	-	S	-	-	-	-	-	I	I
		TIM16	-	-	-	-	-	-	-	-	-	-	-	S	-	-	S	S	-	-	-	-	-	-	-	-	-	-
		TIM17	-	-	-	-	-	-	-	-	-	-	-	S	-	-	S	-	-	-	-	-	-	-	-	-	-	-
		SAI1	A	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	A	-	-	-	-
		SAI2	-	-	-	A	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	A	-	-	-
		DFSDM1	-	-	-	-	-	-	-	-	-	B	B	B	B	B	-	-	-	-	-	-	-	-	-	-	-	-
		HRTIM	-	-	-	-	-	-	A	-	A	-	-	-	-	-	S	-	A	A	A	A	-	-	-	-	-	-
	AHB1	ADC1	-	-	-	-	-	-	-	-	A	-	-	-	-	-	A	-	-	-	-	-	-	-	-	-	-	-
		ADC2	-	-	-	-	-	-	-	-	-	A	-	-	-	-	A	-	-	-	-	-	-	-	-	-	-	-
		ETH	A	A	-	-	-	-	-	-	A	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		USB1 (OTG_HS1)	A	-	-	-	A	-	-	A	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		USB2 (OTG_HS2)	A	-	-	-	A	-	-	A	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

- Letters in the table correspond to the type of connection described in [Section 14.1.2: Connection overview](#)
- The “-” symbol in a gray cell means no interconnect.

Table 101. Peripherals interconnect matrix (D3 domain) ^{(1) (2)}

Source			Destination																							
			D2 domain																			D3 domain AHB4 APB4				
			APB1									APB2							AHB1							
TIM2	TIM3	TIM4	TIM5	TIM12	LPTIM1	DAC	CRS	CAN	TIM1	TIM8	TIM15	TIM16	TIM17	DFSDM1	HRTIM	ADC1	ADC2	ETHERNET	ADC3	LPTIM2	LPTIM3	LPTIM4	LPTIM5			
D3 Domain	APB4	EXTI	-	-	-	-	-	-	A	-	-	-	-	-	-	A	-	A	A	-	-	-	-	-		
		LPTIM2	-	-	-	-	-	-	A	-	-	-	-	-	-	A	A	A	A	-	A	-	A	A		
		LPTIM3	-	-	-	-	-	-	-	-	-	-	-	-	-	A	-	A	A	-	A	-	-	A		
		LPTIM4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	A	-		
		LPTIM5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	A	-		
		COMP1	A	A	-	-	-	A	-	-	-	A/B	A/B	B	B	B	-	A/B	-	-	-	-	A	-		
		COMP2	A	A	-	-	-	A	-	-	-	A/B	A/B	B	B	B	-	A/B	-	-	-	-	A	-		
		SAI4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	A	-		
		RTC	-	-	-	-	-	A	-	-	-	-	-	-	A	-	-	-	-	-	-	A	-	-		
	AHB4	ADC3	-	-	-	-	-	-	-	-	A	A	-	-	-	-	-	-	-	-	-	-	-	-		
RCC		A	-	-	-	-	-	A	-	-	-	A	A	A	-	-	-	-	-	-	-	-	-			

- Letters in the table correspond to the type of connection described in [Section 14.1.2: Connection overview](#).
- The “-” symbol in a gray cell means no interconnect.

Table 102. Peripherals interconnect matrix details⁽¹⁾

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM1	TRGO	ITR0	TIM2	APB1	D2	S	-
		TIM8	TRGO	ITR1				S	-
	APB1	TIM3	TRGO	ITR2				S	-
		TIM4	TRGO	ITR3				S	-
	AHB1	ETH	PPS	ITR4				S	-
		USB1	SOF	ITR5				S	-
		USB2	SOF	ITR6				S	-
D3	APB4	COMP1	comp1_out	ETR1				I	-
		COMP2	comp2_out	ETR2				I	-
		RCC	lse_ck	ETR3				A	-
D2	APB2	SAI1	SAI1_FS_A	ETR4				A	-
		SAI1	SAI1_FS_B	ETR5				A	-
D3	APB4	COMP1	comp1_out	TI4_1				I	-
		COMP2	comp2_out	TI4_2				I	-
		COMP1 or COMP2 ⁽²⁾	comp1_out or comp2_out	TI4_3				I	-
D2	APB2	TIM1	TRGO	ITR0	TIM3	APB1	D2	S	-
	APB1	TIM2	TRGO	ITR1				S	-
	APB2	TIM15	TRGO	ITR2				S	-
	APB1	TIM4	TRGO	ITR3				S	-
	AHB1	ETH	PPS	ITR4				S	-
D3	APB4	COMP1	comp1_out	ETR1				I	-
		COMP1	comp1_out	TI1_1				I	-
		COMP2	comp2_out	TI1_2				I	-
		COMP1 or COMP2 ⁽²⁾	comp1_out or comp2_out	TI1_3				I	-
D2	APB2	TIM1	TRGO	ITR0	TIM4	APB1	D2	S	-
	APB1	TIM2	TRGO	ITR1				S	-
		TIM3	TRGO	ITR2				S	-
	APB2	TIM8	TRGO	ITR3				S	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM1	TRGO	ITR0	TIM5	APB1	D2	S	-
		TIM8	TRGO	ITR1				S	-
	APB1	TIM3	TRGO	ITR2				S	-
		TIM4	TRGO	ITR3				S	-
		CAN	SOC	ITR6				S	-
	AHB1	USB1	SOF	ITR7				S	-
		USB2	SOF	ITR8				S	-
	APB2	SAI2	SAI2_FS_A	ETR1				A	-
		SAI2	SAI2_FS_B	ETR2				A	-
	APB1	CAN	TMP	TI1_1				A	-
		CAN	RTP	TI1_2				A	-
D2	APB1	TIM4	TRGO	ITR0	TIM12	APB1	D2	S	-
		TIM5	TRGO	ITR1				S	-
		TIM13	OC1	ITR2				S	-
		TIM14	OC1	ITR3				S	-
	AHB1	USB1	SOF	crs_sync2	CRS	APB1	D2	A	-
		USB2	SOF	crs_sync3	CRS	APB1	D2	A	-
D3	AHB4	RCC	lse_ck	crs_sync1	CRS	APB1	D2	A	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM15	TRGO	ITR0	TIM1	APB2	D2	S	-
	APB1	TIM2	TRGO	ITR1				S	-
		TIM3	TRGO	ITR2				S	-
		TIM4	TRGO	ITR3				S	-
D3	APB4	COMP1	comp1_out	ETR1				I	-
		COMP2	comp2_out	ETR2				I	-
D2	AHB1	ADC1	adc1_awd1	ETR3				A	-
		ADC1	adc1_awd2	ETR4				A	-
		ADC1	adc1_awd3	ETR5				A	-
D3	AHB4	ADC3	adc3_awd1	ETR6				A	-
		ADC3	adc3_awd2	ETR7				A	-
		ADC3	adc3_awd3	ETR8				A	-
	APB4	COMP1	comp1_out	TI1_1				I	-
		COMP1	comp1_out	BRK_1				B	-
		COMP2	comp2_out	BRK_2				B	-
D2	APB2	DFSDM1	dfsdm1_break0	BRK_3				B	-
D3	APB4	COMP1	comp1_out	BRK2_1				B	-
		COMP2	comp2_out	BRK2_2				B	-
D2	APB2	DFSDM1	dfsdm1_break1	BRK2_3				B	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM1	TRGO	ITR0	TIM8	APB2	D2	S	-
	APB1	TIM2	TRGO	ITR1				S	-
		TIM4	TRGO	ITR2				S	-
		TIM5	TRGO	ITR3				S	-
D3	APB4	COMP1	comp1_out	ETR1				I	-
		COMP2	comp2_out	ETR2				I	-
D2	AHB1	ADC2	adc2_awd1	ETR3				A	-
		ADC2	adc2_awd2	ETR4				A	-
		ADC2	adc2_awd3	ETR5				A	-
D3	AHB4	ADC3	adc3_awd1	ETR6				A	-
		ADC3	adc3_awd2	ETR7				A	-
		ADC3	adc3_awd3	ETR8				A	-
	APB4	COMP2	comp2_out	TI1_1				I	-
		COMP1	comp1_out	BRK_1				B	-
		COMP2	comp2_out	BRK_2				B	-
D2	APB2	DFSDM1	dfsdm1_break2	BRK_3				B	-
D3	APB4	COMP1	comp1_out	BRK2_1				B	-
		COMP2	comp2_out	BRK2_2				B	-
D2	APB2	DFSDM1	dfsdm1_break3	BRK2_3				B	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM1	TRGO	ITR0	TIM15	APB2	D2	S	-
	APB1	TIM3	TRGO	ITR1				S	-
	APB2	TIM16	OC1	ITR2				S	-
		TIM17	OC1	ITR3				S	-
	APB1	TIM2	CH1	TI1_1				A	-
		TIM3	CH1	TI1_2				A	-
		TIM4	CH1	TI1_3				A	-
D3	AHB4	RCC	lse_ck	TI1_4				A	-
		RCC	csi_ck	TI1_5				A	-
		RCC	MCO2	TI1_6				A	-
D2	APB1	TIM2	CH2	TI2_1				A	-
		TIM3	CH2	TI2_2				A	-
		TIM4	CH2	TI2_3				A	-
D3	APB4	COMP1	comp1_out	BRK_1				B	-
		COMP2	comp2_out	BRK_2				B	-
D2	APB2	DFSDM1	dfsdm_break0	BRK_3				B	-
D3	AHB4	RCC	lsi_ck	TI1_1	TIM16	APB2	D2	A	-
		RCC	lse_ck	TI1_2				A	-
	APB4	RTC	WKUP_IT	TI1_3				A	-
		COMP1	comp1_out	BRK_1				B	-
		COMP2	comp2_out	BRK_2				B	-
D2	APB2	DFSDM1	dfsdm_break1	BRK_3				B	-
D2	APB1	SPDIFRX	spdifrx_frame_sync	TI1_1	TIM17	APB2	D2	A	-
D3	AHB4	RCC	HSE_1MHZ	TI1_2				A	-
		RCC	MCO1	TI1_3				A	-
	APB4	COMP1	comp1_out	BRK_1				B	-
		COMP2	comp2_out	BRK_2				B	-
	APB2	DFSDM1	dfsdm_break2	BRK_3				B	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D3	APB4	COMP1	comp1_out	hrtim_evt11	HRTIM	APB2	D2	B	-
D2	APB2	TIM1	TRGO	hrtim_evt12				B	-
	AHB1	ADC1	adc1_awd1	hrtim_evt13				B	-
D3	APB4	COMP2	OUT	hrtim_evt21				B	-
D2	APB2	TIM2	TRGO	hrtim_evt22				B	-
	AHB1	ADC1	adc1_awd2	hrtim_evt23				B	-
	NC	NC	NC	hrtim_evt31				B	-
	APB2	TIM3	TRGO	hrtim_evt32				B	-
	AHB1	ADC1	adc1_awd3	hrtim_evt33				B	-
	APB1	OPAMP1	opamp1_out	hrtim_evt41				B	-
		TIM7	TRGO	hrtim_evt42				B	-
	AHB1	ADC2	adc2_awd1	hrtim_evt43				B	-
	NC	NC	NC	hrtim_evt51				B	-
	APB1	LPTIM1	lptim1_out	hrtim_evt52				B	-
	AHB1	ADC2	adc2_awd2	hrtim_evt53				B	-
D3	APB4	COMP1	comp1_out	hrtim_evt61				I	-
D2	APB1	TIM6	TRGO	hrtim_evt62				S	-
	AHB1	ADC2	adc2_awd3	hrtim_evt63				A	-
D3	APB4	COMP2	comp2_out	hrtim_evt71				I	-
D2	APB1	TIM7	TRGO	hrtim_evt72				S	-
	NC	NC	NC	hrtim_evt73				-	-
				hrtim_evt81				-	-
	APB1	TIM6	TRGO	hrtim_evt82				S	-
	APB1	CAN	TTCAN_TMP	hrtim_evt83				A	-
		OPAMP1	opamp1_out	hrtim_evt91				I	-
	APB2	TIM15	TRGO	hrtim_evt92				S	-
	APB1	CAN	TTCAN_RTP	hrtim_evt93				A	-
	NC	NC	NC	hrtim_evt101					-
D3	APB4	LPTIM2	lptim2_out	hrtim_evt102				A	-
D2	APB1	CAN	TTCAN_SOC	hrtim_evt103				A	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D3	APB4	COMP1	comp1_out	hrtim_in_ft1	HRTIM	APB2	D2	B	-
		COMP2	comp2_out	hrtim_in_ft2				B	-
D2	APB2	TIM16	OC	hrtim_upd_en1				S	-
		TIM17	OC	hrtim_upd_en2				S	-
	APB1	TIM6	TRGO	hrtim_upd_en3				S	-
		TIM7	TRGO	hrtim_bm_trg				S	-
	APB2	TIM16	OC	hrtim_bm_ck1				S	-
		TIM17	OC	hrtim_bm_ck2				S	-
	APB1	TIM7	TRGO	hrtim_bm_ck3				S	-
D3	APB4	RTC	rtc_alarm_a_evt	lptim1_ext_trg0	LPTIM1	APB1	D2	A	-
		RTC	rtc_alarm_b_evt	lptim1_ext_trg1				A	-
		RTC	rtc_tamp1_evt	lptim1_ext_trg2				A	-
		RTC	rtc_tamp2_evt	lptim1_ext_trg3				A	-
		RTC	rtc_tamp3_evt	lptim1_ext_trg4				A	-
		COMP1	comp1_out	lptim1_ext_trg5				I	-
		COMP2	comp2_out	lptim1_ext_trg6				I	-
		COMP1	comp1_out	lptim1_in1_mu_x1				I	-
		COMP2	comp2_out	lptim1_in2_mu_x2				I	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D3	APB4	RTC	rtc_alarm_a_evt	lptim2_ext_trg0	LPTIM2	APB4	D3	A	-
		RTC	rtc_alarm_b_evt	lptim2_ext_trg1				A	-
		RTC	rtc_tamp1_evt	lptim2_ext_trg2				A	-
		RTC	rtc_tamp2_evt	lptim2_ext_trg3				A	-
		RTC	rtc_tamp3_evt	lptim2_ext_trg4				A	-
		COMP1	comp1_out	lptim2_ext_trg5				I	-
		COMP2	comp2_out	lptim2_ext_trg6				I	-
		COMP1	comp1_out	lptim2_in1_mux1				I	-
		COMP2	comp2_out	lptim2_in1_mux2				I	-
		COMP1 or COMP2 ⁽²⁾	comp1_out or comp2_out	lptim2_in1_mux3				I	-
		COMP2	comp2_out	lptim2_in2_mux1				I	-
D3	APB4	LPTIM2	lptim2_out	lptim3_ext_trg0	LPTIM3	APB4	D3	S	If same kernel clock source
		NC	NC	lptim3_ext_trg1				-	-
		LPTIM4	lptim4_out	lptim3_ext_trg2				S	If same kernel clock source
		LPTIM5	lptim5_out	lptim3_ext_trg3				S	If same kernel clock source
D2	APB2	SAI1	SAI1_FS_A	lptim3_ext_trg4				A	-
		SAI1	SAI1_FS_B	lptim3_ext_trg5				A	-
		SAI4	SAI4_FS_A	lptim3_in1_mux1				A	-
		SAI4	SAI4_FS_B	lptim3_in1_mux2				A	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D3	APB4	LPTIM2	lptim2_out	lptim4_ext_trg_0	LPTIM4	APB4	D3	S	If same kernel clock source
		LPTIM3	lptim3_out	lptim4_ext_trg_1				S	If same kernel clock source
		NC	NC	lptim4_ext_trg_2					-
		LPTIM5	lptim5_out	lptim4_ext_trg_3				S	If same kernel clock source
D2	APB2	SAI2	SAI2_FS_A	lptim4_ext_trg_4				A	-
		SAI2	SAI2_FS_B	lptim4_ext_trg_5				A	-
D3	APB4	LPTIM2	lptim2_out	lptim5_ext_trg_0	LPTIM5	APB4	D3	S	If same kernel clock source
		LPTIM3	lptim3_out	lptim5_ext_trg_1				S	If same kernel clock source
		LPTIM4	lptim4_out	lptim5_ext_trg_2				S	If same kernel clock source
		SAI4	SAI4_FS_A	lptim5_ext_trg_3				A	-
		SAI4	SAI4_FS_B	lptim5_ext_trg_4				A	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM1	TRGO	dac_ch1/2_trg 0	DAC channel 1/channel 2	APB1	D2	S	-
	APB1	TIM2	TRGO	dac_ch1/2_trg 1				S	-
		TIM4	TRGO	dac_ch1/2_trg 02				S	-
		TIM5	TRGO	dac_ch1/2_trg 3				S	-
		TIM6	TRGO	dac_ch1/2_trg 4				S	-
		TIM7	TRGO	dac_ch1/2_trg 5				S	-
	APB2	TIM8	TRGO	dac_ch1/2_trg 6				S	-
		TIM15	TRGO	dac_ch1/2_trg 7				S	-
		HRTIM1	hrtim_dac_trg1	dac_ch1/2_trg 8				S	-
			hrtim_dac_trg2	dac_ch1/2_trg 9				S	-
	APB1	LPTIM1	lptim1_out	dac_ch1/2_trg 10				S	-
		LPTIM2	lptim2_out	dac_ch1/2_trg 11				S	-
D3	APB4	SYSCFG	EXTI9	dac_ch1/2_trg 12				S	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM1	TRGO	TRG0	DFSDM1	APB2	D2	S	-
		TIM1	TRGO2	TRG1				S	-
		TIM8	TRGO	TRG2				S	-
		TIM8	TRGO2	TRG3				S	-
	APB1	TIM3	TRGO	TRG4				S	-
		TIM4	TRGO	TRG5				S	-
	APB2	TIM16	OC1	TRG6				S	-
	APB1	TIM6	TRGO	TRG7				S	-
		TIM7	TRGO	TRG8				S	-
	APB2	HRTIM1	hrtim_adc_trg1	TRG9				S	-
		HRTIM1	hrtim_adc_trg3	TRG10				S	-
D3	APB4	SYSCFG	EXTI11	TRG24	ADC1 / ADC2	AHB1	D2	A	-
		SYSCFG	EXTI15	TRG25				A	-
D2	APB1	LPTIM1	lptim1_out	TRG26				A	-
D3	APB4	LPTIM2	lptim2_out	TRG27				A	-
		LPTIM3	lptim3_out	TRG28				A	-
D2	APB2	TIM1	CC1	adc_ext_trg0				S	-
		TIM1	CC2	adc_ext_trg1				S	-
		TIM1	CC3	adc_ext_trg2				S	-
	APB1	TIM2	CC2	adc_ext_trg3				S	-
		TIM3	TRGO	adc_ext_trg4				S	-
		TIM4	CC4	adc_ext_trg5				S	-
D3	APB4	SYSCFG	EXTI11	adc_ext_trg6				A	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM8	TRGO	adc_ext_trg7	ADC1 / ADC2	AHB1	D2	S	-
		TIM8	TRGO2	adc_ext_trg8				S	-
		TIM1	TRGO	adc_ext_trg9				S	-
		TIM1	TRGO2	adc_ext_trg10				S	-
	APB1	TIM2	TRGO	adc_ext_trg11				S	-
		TIM4	TRGO	adc_ext_trg12				S	-
		TIM6	TRGO	adc_ext_trg13				S	-
	APB2	TIM15	TRGO	adc_ext_trg14				S	-
	APB1	TIM3	CC4	adc_ext_trg15				S	-
	APB2	HRTIM1	hrtim_adc_trg1	adc_ext_trg16				A	-
		HRTIM1	hrtim_adc_trg3	adc_ext_trg17				A	-
		LPTIM1	lptim1_out	adc_ext_trg18				A	-
D3	APB4	LPTIM2	lptim2_out	adc_ext_trg19				A	-
		LPTIM3	lptim3_out	adc_ext_trg20				A	-
D2	APB2	TIM1	TRGO	adc_jext_trg0	ADC1 / ADC2	AHB1	D2	S	-
		TIM1	CC4	adc_jext_trg1				S	-
	APB1	TIM2	TRGO	adc_jext_trg2				S	-
		TIM2	CC1	adc_jext_trg3				S	-
		TIM3	CC4	adc_jext_trg4				S	-
		TIM4	TRGO	adc_jext_trg5				S	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D3	APB4	SYSCFG	EXTI15	adc_jext_trg6	ADC1 / ADC2	AHB1	D2	A	-
D2	APB2	TIM8	CC4	adc_jext_trg7				S	-
		TIM1	TRGO2	adc_jext_trg8				S	-
		TIM8	TRGO	adc_jext_trg9				S	-
		TIM8	TRGO2	adc_jext_trg10				S	-
		TIM3	CC3	adc_jext_trg11				S	-
	APB1	TIM3	TRGO	adc_jext_trg12				S	-
		TIM3	CC1	adc_jext_trg13				S	-
		TIM6	TRGO	adc_jext_trg14				S	-
		TIM15	TRGO	adc_jext_trg15				S	-
	APB2	HRTIM1	hrtim_adc_trg2	adc_jext_trg16				A	-
		HRTIM1	hrtim_adc_trg4	adc_jext_trg17				A	-
	APB1	LPTIM1	lptim1_out	adc_jext_trg18				A	-
D3	APB4	LPTIM2	lptim2_out	adc_jext_trg19				A	-
		LPTIM3	lptim2_out	adc_jext_trg20				A	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM1	CC1	adc_ext_trg0	ADC3	AHB4	D3	S	-
		TIM1	CC2	adc_ext_trg1				S	-
		TIM1	CC3	adc_ext_trg2				S	-
	APB1	TIM2	CC2	adc_ext_trg3				S	-
		TIM3	TRGO	adc_ext_trg4				S	-
		TIM4	CC4	adc_ext_trg5				S	-
D3	APB4	SYSCFG	EXTI11	adc_ext_trg6				A	-
D2	APB2	TIM8	TRGO	adc_ext_trg7				S	-
		TIM8	TRGO2	adc_ext_trg8				S	-
		TIM1	TRGO	adc_ext_trg9				S	-
		TIM1	TRGO2	adc_ext_trg10				S	-
	APB1	TIM2	TRGO	adc_ext_trg11				S	-
		TIM4	TRGO	adc_ext_trg12				S	-
		TIM6	TRGO	adc_ext_trg13				S	-
	APB2	TIM15	TRGO	adc_ext_trg14				S	-
	APB1	TIM3	CC4	adc_ext_trg15				S	-
	APB2	HRTIM1	hrtim_adc_trg1	adc_ext_trg16				A	-
		HRTIM1	hrtim_adc_trg3	adc_ext_trg17				A	-
		LPTIM1	lptim1_out	adc_ext_trg18				A	-
D3	APB4	LPTIM2	lptim2_out	adc_ext_trg19				A	-
		LPTIM3	lptim3_out	adc_ext_trg20				A	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB2	TIM1	TRGO	adc_jext_trg0	ADC3	AHB4	D3	SI	-
		TIM1	CC4	adc_jext_trg1				S	-
	APB1	TIM2	TRGO	adc_jext_trg2				S	-
		TIM2	CC1	adc_jext_trg3				S	-
		TIM3	CC4	adc_jext_trg4				S	-
		TIM4	TRGO	adc_jext_trg5				S	-
D3	APB4	SYSCFG	EXTI15	adc_jext_trg6				A	-
D2	APB2	TIM8	CC4	adc_jext_trg7				S	-
		TIM1	TRGO2	adc_jext_trg8				S	-
		TIM8	TRGO	adc_jext_trg9				S	-
		TIM8	TRGO2	adc_jext_trg10				S	-
	APB1	TIM3	CC3	adc_jext_trg11				S	-
		TIM3	TRGO	adc_jext_trg12				S	-
		TIM3	CC1	adc_jext_trg13				S	-
		TIM6	TRGO	adc_jext_trg14				S	-
	APB2	TIM15	TRGO	adc_jext_trg15				S	-
		HRTIM1	hrtim_adc_trg2	adc_jext_trg16				A	-
		HRTIM1	hrtim_adc_trg4	adc_jext_trg17				A	-
	APB1	LPTIM1	OUT	adc_jext_trg18				A	-
D3	APB4	LPTIM2	OUT	adc_jext_trg19				A	-
		LPTIM3	OUT	adc_jext_trg20				A	-
D2	APB2	TIM1	OC5	comp_blk1	COMP1 / COMP2	APB4	D3	I	-
		TIM1	OC3	comp_blk2				I	-
	APB1	TIM3	OC3	comp_blk3				I	-
		TIM3	OC4	comp_blk4				I	-
	APB2	TIM8	OC5	comp_blk5				I	-
		TIM15	OC1	comp_blk6				I	-

Table 102. Peripherals interconnect matrix details⁽¹⁾ (continued)

Source				Destination				Type	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain		
D2	APB1	TIM2	TRGO	SWT0	FDCAN	APB1	D2	A	-
		TIM3	TRGO	SWT1				A	-
	AHB1	ETH	PPS	SWT2				A	-
	APB2	HRTIM1	hrtim_dac_trg1	SWT3				A	-
	APB1	TIM2	TRGO	EVT0				A	-
		TIM3	TRGO	EVT1				A	-
	AHB1	ETH	PPS	EVT2				A	-
	APB2	HRTIM1	hrtim_dac_trg1	EVT3				A	-
	APB1	TIM2	TRGO	PTP0	ETH	AHB1	D2	A	-
		TIM3	TRGO	PTP1				A	-
	APB2	HRTIM1	hrtim_dac_trg2	PTP2				A	-
	APB1	CAN	TMP	PTP3				A	-

- Letters in the table correspond to the type of connection described in [Section 14.1.2: Connection overview](#).
- comp1_out and comp2_out are connected to the inputs of an OR gate. The output of this OR gate is connected to the The lptim2_in1_mux3 input.

14.2 Wakeup from low power modes

The Extended interrupt and event controller module (EXTI) allows to wake up the system from Stop mode and/or a CPU from CStop mode. Wakeup events are coming from peripherals.

These events are handled by the EXTI either as Configurable events (**C**), or as Direct events (**D**). See *Type* column in [Table 103](#). Refer to [Section 21: Extended interrupt and event controller \(EXTI\)](#) for further details.

Three types of peripheral output signals are connected to the EXTI input events:

- The wake up signals. These signals can be generated by the peripheral without any bus interface clock, they are referred to as xxx_wkup in [Table 103](#). Some peripherals do not have this capability.
- The interrupt signals. These signals can be generated only if the peripheral bus interface clock is running. These interrupt signals are generally directly connected to the NVIC of CPU. They are referred to as xxx_it.
- The signals, i.e. the pulses generated by the peripheral. Once a peripheral has generated a signal, no action (flag clearing) is required at peripheral level.

Each EXTI input event has a different wakeup capability or possible target (see *Target* column in [Table 103](#)):

- CPU1 wakeup (**CPU1**): the input event can be enabled to wake up the CPU1
- CPU2 wakeup (**CPU2**): the input event can be enabled to wake up the CPU2
- CPU1 and CPU2 wakeups (**CPU**): the input event can be enabled to wake up the CPU1 only, the CPU2 only, or both CPU1 and CPU2
- CPU1, CPU2 and D3 domain wakeup for autonomous Run mode (**ANY**): the input event can be enabled to wake up the CPU1 only, the CPU2 only, both CPU1 and CPU2, or the D3 domain only for an autonomous Run mode phase.

Table 103. EXTI wakeup inputs⁽¹⁾

Source				Destination		Type	Target	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral			
D3	APB4	SYSCFG	exti0_wkup	WKUP0	EXTI	C	ANY	-
			exti1_wkup	WKUP1				-
			exti2_wkup	WKUP2				-
			exti3_wkup	WKUP3				-
			exti4_wkup	WKUP4				-
			exti5_wkup	WKUP5				-
			exti6_wkup	WKUP6				-
			exti7_wkup	WKUP7				-
			exti8_wkup	WKUP8				-
			exti9_wkup	WKUP9				-
			exti10_wkup	WKUP10				-
			exti11_wkup	WKUP11				-
			exti12_wkup	WKUP12				-
			exti13_wkup	WKUP13				-
			exti14_wkup	WKUP14				-
			exti15_wkup	WKUP15				-
D3	AHB4	PWR	pvd_avd_wkup	WKUP16	C	CPU	-	
D3	APB4	RTC	ALARMS	WKUP17	D	CPU	-	
D3	APB4	RTC	TAMPER TIMESTAMP	WKUP18	C	CPU	-	
D3	AHB4	RCC	CSS_LSE				-	
D3	APB4	RTC	WKUP	WKUP19	C	ANY	-	
D3	APB4	COMP1	comp1_out	WKUP20	C	ANY	-	
D3	APB4	COMP2	comp2_out	WKUP21	C	ANY	-	
D2	APB1	I2C1	i2c1_wkup	WKUP22	C	CPU	-	
D2	APB1	I2C2	i2c2_wkup	WKUP23	D	CPU	-	
D2	APB1	I2C3	i2c3_wkup	WKUP24	D	CPU	-	
D2	APB1	I2C4	i2c4_wkup	WKUP25	D	ANY	-	
D2	APB2	USART1	usart1_wkup	WKUP26	D	CPU	-	
D2	APB1	USART2	usart2_wkup	WKUP27	D	CPU	-	

Table 103. EXTI wakeup inputs⁽¹⁾ (continued)

Source				Destination		Type	Target	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral			
D2	APB1	USART3	usart3_wkup	WKUP28	EXTI	D	CPU	-
D2	APB2	USART6	usart6_wkup	WKUP29		D	CPU	-
D2	APB1	UART4	uart4_wkup	WKUP30		D	CPU	-
D2	APB1	UART5	uart5_wkup	WKUP31		D	CPU	-
D2	APB1	UART7	uart7_wkup	WKUP32		D	CPU	-
D2	APB1	UART8	uart8_wkup	WKUP33		D	CPU	-
D3	APB4	LPUART	lpuart_rx_wkup	WKUP34		D	ANY	-
D3	APB4	LPUART	lpuart_tx_wkup	WKUP35		D	ANY	-
D2	APB2	SPI1	spi1_wkup	WKUP36		D	CPU	-
D2	APB1	SPI2	spi2_wkup	WKUP37		D	CPU	-
D2	APB1	SPI3	spi3_wkup	WKUP38		D	CPU	-
D2	APB2	SPI4	spi4_wkup	WKUP39		D	CPU	-
D2	APB2	SPI5	spi5_wkup	WKUP40		D	CPU	-
D3	APB4	SPI6	spi6_wkup	WKUP41		D	ANY	-
D2	APB1	MDIOS	mdios_wkup	WKUP42		D	CPU	-
D2	AHB1	USB1	usb1_wkup	WKUP43		D	CPU	-
D2	AHB1	USB2	usb2_wkup	WKUP44		D	CPU	-
-	-	NC	NC	WKUP45		-	-	-
D1	APB3	DSI	dsi_it	WKUP46		D	CPU	(1)
D2	APB1	LPTIM1	lptim1_wkup	WKUP47		D	CPU	-
D3	APB4	LPTIM2	lptim2_wkup	WKUP48		D	ANY	-
D3	APB4	LPTIM2	lptim2_out	WKUP49		C	ANY	(2)
D3	APB4	LPTIM3	lptim3_wkup	WKUP50		D	ANY	-
D3	APB4	LPTIM3	lptim3_out	WKUP51		C	ANY	(2)
D3	APB4	LPTIM4	lptim4_wkup	WKUP52		D	ANY	-
D3	APB4	LPTIM5	lptim5_wkup	WKUP53		D	ANY	-
D2	APB1	SWPMI	swpmi_wkup	WKUP54		D	CPU	-

Table 103. EXTI wakeup inputs⁽¹⁾ (continued)

Source				Destination		Type	Target	Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral			
D3	AHB4	PWR	pwr_wkup1_wkup	WKUP55	EXTI	D	CPU	-
			pwr_wkup2_wkup	WKUP56				-
			pwr_wkup3_wkup	WKUP57				-
			pwr_wkup4_wkup	WKUP58				-
			pwr_wkup5_wkup	WKUP59				-
			pwr_wkup6_wkup	WKUP60				-
D3	AHB4	RCC	rcc_it	WKUP61		D	CPU	-
D3	APB4	I2C4	i2c4_ev_it	WKUP62		D	CPU	(1)
		I2C4	i2c4_err_it	WKUP63		D	CPU	(1)
D3	APB4	LPUART1	lpuart1_it	WKUP64		D	CPU	(1)
D3	APB4	SPI6	spi6_it	WKUP64		D	CPU	(1)
D3	AHB4	BDMA	bdma_ch0_it	WKUP66		D	CPU	(1)
			bdma_ch1_it	WKUP67		D	CPU	(1)
			bdma_ch2_it	WKUP68		D	CPU	(1)
			bdma_ch3_it	WKUP69		D	CPU	(1)
			bdma_ch4_it	WKUP70		D	CPU	(1)
			bdma_ch5_it	WKUP71		D	CPU	(1)
			bdma_ch6_it	WKUP72		D	CPU	(1)
			bdma_ch7_it	WKUP73		D	CPU	(1)
D3	AHB4	DMAMUX2	dmamux2_it	WKUP74			CPU	(1)
D3	AHB4	ADC3	adc3_it	WKUP75		D	CPU	(1)
D3	APB4	SAI4	sai4_gbl_it	WKUP76		D	CPU	(1)
D3	AHB4	HSEM	hsem_int0_it	WKUP77		D	CPU1	(1)
D3	AHB4	HSEM	hsem_int1_it	WKUP78		D	CPU2	(1)
-	-	CPU2	cpu2_sev_it	WKUP79		D	CPU2	(2)
-	-	CPU1	cpu1_sev_it	WKUP80		D	CPU1	(2)
-	-	NC	NC	WKUP81		-	-	-
D1	APB3	WWDG1	wwdg1_out_rst	WKUP82		C	CPU1	(1)
-	-	NC	NC	WKUP83		-	-	-
D1	APB1	WWDG2	wwdg2_out_rst	WKUP84		C	CPU2	(1)
D1	APB1	CEC	cec_wkup	WKUP85		C	CPU	-
D2	AHB1	ETH	eth	WKUP86		C	CPU	-
D3	AHB4	RCC	hse_css_rcc_wkup	WKUP87		D	CPU	-

1. The source peripheral needs its bus clock in order to generate the event. This is either PCLK4 or HCLK4 in D3 domain, PCLK3 in D1 domain.
2. The source peripheral signal is not connected to the NVIC.

The Extended Interrupt and Event Controller (EXTI) module event inputs able to wake up the D3 domain for autonomous Run mode have a pending request logic that can be cleared by 4 different input sources ([Table 104](#)). Refer to [Section 21: Extended interrupt and event controller \(EXTI\)](#) for further details.

Table 104. EXTI pending requests clear inputs

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D3	AHB4	DMAMUX2	dmamux2_evt6	d3_pendclear_in[0]	EXTI	APB4	D3	-
			dmamux2_evt7	d3_pendclear_in[1]				-
	APB4	LPTIM4	lptim4_out	d3_pendclear_in[2]				-
		LPTIM5	lptim5_out	d3_pendclear_in[3]				-

14.3 DMA

In D1 domain, the MDMA allows the memory to transfer data. It can be triggered by software or by hardware, according to the connections described in [Section 14.3.1](#).

DMA Multiplexer in D2 domain (DMAMUX1) allows to map any peripheral DMA request to any stream of the DMA1 or the DMA2. In addition to this, The DMAMUX provides two other functionalities:

- It's possible to synchronize a peripheral DMA request with a timer, with an external pin or with a DMA transfer complete of another stream.
- DMA requests can be generated on a stream by the DMAMUX1 itself. This event can be triggered by a timer, by an external pin event, or by a DMA transfer complete of another stream. The number of DMA requests generated is configurable.

The connections on DMAMUX1 and DMA1/DMA2 are described in [Section 18: DMA request multiplexer \(DMAMUX\)](#), [Section 16: Direct memory access controller \(DMA\)](#) and [Section 17: Basic direct memory access controller \(BDMA\)](#).

DMA Multiplexer in D3 domain (DMAMUX2) has the same functionality of DMAMUX1, it is connected to the basic DMA (BDMA).

The connections on DMAMUX2 and BDMA are described in [Section 14.3.3: DMAMUX2, BDMA \(D3 domain\)](#). Refer to [Section 14.3.3: DMAMUX2, BDMA \(D3 domain\)](#) and [Section 17: Basic direct memory access controller \(BDMA\)](#) for more details.

14.3.1 MDMA (D1 domain)

Table 105. MDMA

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D2	AHB1	DMA1	dma1_tcif0	mdma_str0	MDMA	AXI	D1	DMA1 stream 0 transfer complete
			dma1_tcif1	mdma_str1				DMA1 stream 1 transfer complete
			dma1_tcif2	mdma_str2				DMA1 stream 2 transfer complete
			dma1_tcif3	mdma_str3				DMA1 stream 3 transfer complete
			dma1_tcif4	mdma_str4				DMA1 stream 4 transfer complete
			dma1_tcif5	mdma_str5				DMA1 stream 5 transfer complete flag
			dma1_tcif6	mdma_str6				DMA1 stream 6 transfer complete
			dma1_tcif7	mdma_str7				DMA1 stream 7 transfer complete
D2	AHB1	DMA2	dma2_tcif0	mdma_str8				DMA2 stream 0 transfer complete
			dma2_tcif1	mdma_str9				DMA2 stream 1 transfer complete
			dma2_tcif2	mdma_str10				DMA2 stream 2 transfer complete
			dma2_tcif3	mdma_str11				DMA2 stream 3 transfer complete
			dma2_tcif4	mdma_str12				DMA2 stream 4 transfer complete
			dma2_tcif5	mdma_str13				DMA2 stream 5 transfer complete
			dma2_tcif6	mdma_str14				DMA2 stream 6 transfer complete
			dma2_tcif7	mdma_str15				DMA2 stream 7 transfer complete
D1	APB3	LTDC	ltdc_li_it	mdma_str16				LTDC line interrupt

Table 105. MDMA (continued)

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D1	AHB3	JPEG	jpeg_ift_trg	mdma_str17	MDMA	AXI	D1	JPEG input FIFO threshold
			jpeg_ifnt_trg	mdma_str18				JPEG input FIFO not full
			jpeg_ofn_trg	mdma_str19				JPEG output FIFO threshold
			jpeg_ofne_trg	mdma_str20				JPEG output FIFO not empty
			jpeg_oec_trg	mdma_str21				JPEG end of conversion
D1	AHB3	QUADSPI	quadspi_ft_trg	mdma_str22				QUADSPI FIFO threshold
			quadspi_tc_trg	mdma_str23				QUADSPI transfer complete
D1	AHB3	DMA2D	dma2d_clut_trg	mdma_str24				DMA2D CLUT transfer complete
			dma2d_tc_trg	mdma_str25				DMA2D transfer complete
			dma2d_tw_trg	mdma_str26				DMA2D transfer watermark
D1	APB3	DSI	dsi_te_trg	mdma_str27				Tearing effect
			dsi_eor_trg	mdma_str28				End of refresh
D1	AHB3	SDMMC1	sdmmc1_buffend_trg	mdma_str30				SDMMC1 internal DMA buffer end
			sdmmc1_cmdend_trg	mdma_str31				SDMMC1 command end
			sdmmc1_dataend_trg	mdma_str29				End of data

14.3.2 DMAMUX1, DMA1 and DMA2 (D2 domain)

Table 106. DMAMUX1, DMA1 and DMA2 connections⁽¹⁾

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D3	AHB4	dmamux1 internal (Request generator)		dmamux1_req_in1	DMAMUX1	AHB1	D2	Requests
				dmamux1_req_in2				
				dmamux1_req_in3				
				dmamux1_req_in4				
				NC				
				NC				
				NC				
				NC				
D2	AHB1	ADC1	adc1_dma	dmamux1_req_in9				
D2	AHB1	ADC2	adc2_dma	dmamux1_req_in10				
D2	APB2	TIM1	tim1_ch1_dma	dmamux1_req_in11				
			tim1_ch2_dma	dmamux1_req_in12				
			tim1_ch3_dma	dmamux1_req_in13				
			tim1_ch4_dma	dmamux1_req_in14				
			tim1_up_dma	dmamux1_req_in15				
			tim1_trig_dma	dmamux1_req_in16				
			tim1_com_dma	dmamux1_req_in17				
D2	APB1	TIM2	tim2_ch1_dma	dmamux1_req_in18				
			tim2_ch2_dma	dmamux1_req_in19				
			tim2_ch3_dma	dmamux1_req_in20				
			tim2_ch4_dma	dmamux1_req_in21				
			tim2_up_dma	dmamux1_req_in22				
D2	APB1	TIM3	tim3_ch1_dma	dmamux1_req_in23				
			tim3_ch2_dma	dmamux1_req_in24				
			tim3_ch3_dma	dmamux1_req_in25				
			tim3_ch4_dma	dmamux1_req_in26				
			tim3_up_dma	dmamux1_req_in27				
			tim3_trig_dma	dmamux1_req_in28				
D2	APB1	TIM4	tim4_ch1_dma	dmamux1_req_in29				
			tim4_ch2_dma	dmamux1_req_in30				
			tim4_ch3_dma	dmamux1_req_in31				
			tim4_up_dma	dmamux1_req_in32				

Table 106. DMAMUX1, DMA1 and DMA2 connections⁽¹⁾ (continued)

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D2	APB1	I2C1	i2c1_rx_dma	dmamux1_req_in33	DMAMUX1	AHB1	D2	Requests
			i2c1_tx_dma	dmamux1_req_in34				
D2	APB1	I2C2	i2c2_rx_dma	dmamux1_req_in35				
			i2c2_tx_dma	dmamux1_req_in36				
D2	APB2	SPI1	spi1_rx_dma	dmamux1_req_in37				
			spi1_tx_dma	dmamux1_req_in38				
D2	APB1	SPI2	spi2_rx_dma	dmamux1_req_in39				
			spi2_tx_dma	dmamux1_req_in40				
D2	APB2	USART1	usart1_rx_dma	dmamux1_req_in41				
			usart1_tx_dma	dmamux1_req_in42				
D2	APB1	USART2	usart2_rx_dma	dmamux1_req_in43				
			usart2_tx_dma	dmamux1_req_in44				
D2	APB1	USART3	usart3_rx_dma	dmamux1_req_in45				
			usart3_tx_dma	dmamux1_req_in46				
D2	APB2	TIM8	tim8_ch1_dma	dmamux1_req_in47				
			tim8_ch2_dma	dmamux1_req_in48				
			tim8_ch3_dma	dmamux1_req_in49				
			tim8_ch4_dma	dmamux1_req_in50				
			tim8_up_dma	dmamux1_req_in51				
			tim8_trig_dma	dmamux1_req_in52				
			tim8_com_dma	dmamux1_req_in53				
-	-	NC	NC	NC				
D1	APB1	TIM3	tim5_ch1_dma	dmamux1_req_in55				
			tim5_ch2_dma	dmamux1_req_in56				
			tim5_ch3_dma	dmamux1_req_in57				
			tim5_ch4_dma	dmamux1_req_in58				
			tim5_up_dma	dmamux1_req_in59				
			tim5_trig_dma	dmamux1_req_in60				
D2	APB1	SPI3	spi3_rx_dma	dmamux1_req_in61				
			spi3_tx_dma	dmamux1_req_in62				
D1	APB1	UART4	uart4_rx_dma	dmamux1_req_in63				
			uart4_tx_dma	dmamux1_req_in64				

Table 106. DMAMUX1, DMA1 and DMA2 connections⁽¹⁾ (continued)

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D1	APB1	UART5	uart5_rx_dma	dmamux1_req_in65	DMAMUX1	AHB1	D2	Requests
			uart5_tx_dma	dmamux1_req_in66				
D2	APB1	DAC1	dac_ch1_dma	dmamux1_req_in67				
D2	APB1	DAC2	dac_ch2_dma	dmamux1_req_in68				
D2	APB1	TIM6	tim6_up_dma	dmamux1_req_in69				
D2	APB1	TIM7	tim7_up_dma	dmamux1_req_in70				
D2	APB2	USART6	usart6_rx_dma	dmamux1_req_in71				
			usart6_tx_dma	dmamux1_req_in72				
D2	APB1	I2C3	i2c3_rx_dma	dmamux1_req_in73				
			i2c3_tx_dma	dmamux1_req_in74				
D2	AHB2	DCMI	dcmi_dma	dmamux1_req_in75				
D2	AHB2	CRYP	cryp_in_dma	dmamux1_req_in76				
			cryp_out_dma	dmamux1_req_in77				
D2	AHB2	HASH	hash_in_dma	dmamux1_req_in78				
D2	APB1	UART7	uart7_rx_dma	dmamux1_req_in79				
			uart7_tx_dma	dmamux1_req_in80				
D2	APB1	UART8	uart8_rx_dma	dmamux1_req_in81				
			uart8_tx_dma	dmamux1_req_in82				
D2	APB2	SPI4	spi4_rx_dma	dmamux1_req_in83				
			spi4_tx_dma	dmamux1_req_in84				
D2	APB2	SPI5	spi5_rx_dma	dmamux1_req_in85				
			spi5_tx_dma	dmamux1_req_in86				
D2	APB2	SAI1	sai1_a_dma	dmamux1_req_in87				
			sai1_b_dma	dmamux1_req_in88				
D2	APB2	SAI2	sai2_a_dma	dmamux1_req_in89				
			sai2_b_dma	dmamux1_req_in90				
D2	APB1	SWPMI	swpmi_rx_dma	dmamux1_req_in91				
			swpmi_tx_dma	dmamux1_req_in92				
D2	APB1	SPDIFRX	spdifrx_dt_dma	dmamux1_req_in93				
			spdifrx_cs_dma	dmamux1_req_in94				

Table 106. DMAMUX1, DMA1 and DMA2 connections⁽¹⁾ (continued)

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D2	APB2	HRTIM1	hrtim_dma1	dmamux1_req_in95	DMAMUX1	AHB1	D2	Requests
			hrtim_dma2	dmamux1_req_in96				
			hrtim_dma3	dmamux1_req_in97				
			hrtim_dma4	dmamux1_req_in98				
			hrtim_dma5	dmamux1_req_in99				
			hrtim_dma6	dmamux1_req_in100				
D2	APB2	DFSDM1	dfsdm1_dma0	dmamux1_req_in101				
			dfsdm1_dma1	dmamux1_req_in102				
			dfsdm1_dma2	dmamux1_req_in103				
			dfsdm1_dma3	dmamux1_req_in104				
D2	APB2	TIM15	tim15_ch1_dma	dmamux1_req_in105				
			tim15_up_dma	dmamux1_req_in106				
			tim15_trig_dma	dmamux1_req_in107				
			tim15_com_dma	dmamux1_req_in108				
D2	APB2	TIM16	tim16_ch1_dma	dmamux1_req_in109				
			tim16_up_dma	dmamux1_req_in110				
D2	APB2	TIM17	tim17_ch1_mda	dmamux1_req_in111				
			tim17_up_dma	dmamux1_req_in112				
D2	APB2	SAI3	sai3_a_dma	dmamux1_req_in113				
			sai3_b_dma	dmamux1_req_in114				
D3	AHB4	ADC3	adc3_dma	dmamux1_req_in115				
D2	AHB1	DMAMUX1	dmamux1_evt0	dmamux1_gen0	DMAMUX1	AHB1	D2	Request generation
			dmamux1_evt1	dmamux1_gen1				
			dmamux1_evt2	dmamux1_gen2				
D2	APB1	LPTIM1	lptim1_out	dmamux1_gen3				
D2	APB1	LPTIM2	lptim2_out	dmamux1_gen4				
D2	APB1	LPTIM3	lptim3_out	dmamux1_gen5				
D3	APB4	EXTI	exti_exti0_it	dmamux1_gen6				
D2	APB1	TIM12	tim12_trgo	dmamux1_gen7				

Table 106. DMAMUX1, DMA1 and DMA2 connections⁽¹⁾ (continued)

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D2	AHB1	DMAMUX1	dmamux1_evt0	dmamux1_trg0	DMAMUX1	AHB1	D2	Triggers
			dmamux1_evt1	dmamux1_trg1				
			dmamux1_evt2	dmamux1_trg2				
D2	APB1	LPTIM1	lptim1_out	dmamux1_trg3				
D2	APB1	LPTIM2	lptim2_out	dmamux1_trg4				
D2	APB1	LPTIM3	lptim3_out	dmamux1_trg5				
D3	APB4	EXTI	exti_exti0_it	dmamux1_trg6				
D2	APB1	TIM12	tim12_trgo	dmamux1_trg7				
D2	AHB1	DMAMUX1	dmamux1_req_out0	dma1_str0	DMA1	AHB1	D2	Requests out
			dmamux1_req_out1	dma1_str1				
			dmamux1_req_out2	dma1_str2				
			dmamux1_req_out3	dma1_str3				
			dmamux1_req_out4	dma1_str4				
			dmamux1_req_out5	dma1_str5				
			dmamux1_req_out6	dma1_str6				
			dmamux1_req_out7	dma1_str7				
			dmamux1_req_out8	dma2_str0	DMA2	AHB1	D2	
			dmamux1_req_out9	dma2_str1				
			dmamux1_req_out10	dma2_str2				
			dmamux1_req_out11	dma2_str3				
			dmamux1_req_out12	dma2_str4				
			dmamux1_req_out13	dma2_str5				
			dmamux1_req_out14	dma2_str6				
			dmamux1_req_out15	dma2_str7				

1. The “-” symbol in grayed cells means no interconnect.

14.3.3 DMAMUX2, BDMA (D3 domain)

Table 107. DMAMUX2 and BDMA connections

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D3	AHB4	dmamux2 internal (Request generator)		dmamux2_req_in1	DMAMUX2	AHB4	D3	Requests
				dmamux2_req_in2				
				dmamux2_req_in3				
				dmamux2_req_in4				
				NC				
				NC				
				NC				
				NC				
D3	APB4	LPUART	dma_rx_lpuart	dmamux2_req_in9	DMAMUX2	AHB4	D3	Requests
			dma_tx_lpuart	dmamux2_req_in10				
D3	APB4	SPI6	dma_rx_spi6	dmamux2_req_in11				
			dma_tx_spi6	dmamux2_req_in12				
D2	APB1	I2C4	dma_rx_i2c4	dmamux2_req_in13				
			dma_tx_i2c4	dmamux2_req_in14				
D3	APB4	SAI4	dma_a_sai4	dmamux2_req_in15				
			dma_b_sai4	dmamux2_req_in16				
D3	APB4	ADC3	dma_adc3	dmamux2_req_in17				

Table 107. DMAMUX2 and BDMA connections (continued)

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D3	AHB4	DMAMUX2	dmamux2_evt0	dmamux2_gen0	DMAMUX2	AHB4	D3	Request generation
			dmamux2_evt1	dmamux2_gen1				
			dmamux2_evt2	dmamux2_gen2				
			dmamux2_evt3	dmamux2_gen3				
			dmamux2_evt4	dmamux2_gen4				
			dmamux2_evt5	dmamux2_gen5				
			dmamux2_evt6	dmamux2_gen6				
D3	APB4	EXTI	exti_lpuart_rx_it	dmamux2_gen7				
			exti_lpuart_tx_it	dmamux2_gen8				
			exti_lptim2_wkup	dmamux2_gen9				
			exti_lptim2_out	dmamux2_gen10				
			exti_lptim3_wkup	dmamux2_gen11				
			exti_lptim3_out	dmamux2_gen12				
			exti_lptim4_wkup	dmamux2_gen13				
			exti_lptim5_wkup	dmamux2_gen14				
			exti_i2c4_wkup	dmamux2_gen15				
			exti_spi6_wkup	dmamux2_gen16				
			exti_comp1_out	dmamux2_gen17				
			exti_comp2_out	dmamux2_gen18				
			exti_rtc_wkup	dmamux2_gen19				
			exti_syscfg_exti0	dmamux2_gen20				
			exti_syscfg_exti2	dmamux2_gen21				
D3	APB4	I2C4	it_evt_i2c4	dmamux2_gen22				
D3	APB4	SPI6	it_spi6	dmamux2_gen23				
D3	APB4	LPUART	it_tx_lpuart1	dmamux2_gen24				
			it_rx_lpuart1	dmamux2_gen25				
D3	AHB4	ADC3	it_adc3	dmamux2_gen26				
			out_awd1_adc3	dmamux2_gen27				
D3	AHB4	BDMA	it_ch0_bdma	dmamux2_gen28				
			it_ch1_bdma	dmamux2_gen29				

Table 107. DMAMUX2 and BDMA connections (continued)

Source				Destination				Comment
Domain	Bus	Peripheral	Signal	Signal	Peripheral	Bus	Domain	
D3	AHB4	DMAMUX2	dmamux2_evt0	dmamux2_trg0	DMAMUX2	AHB4	D3	Triggers
			dmamux2_evt1	dmamux2_trg1				
			dmamux2_evt2	dmamux2_trg2				
			dmamux2_evt3	dmamux2_trg3				
			dmamux2_evt4	dmamux2_trg4				
			dmamux2_evt5	dmamux2_trg5				
D3	APB4	EXTI	it_exti_tx_lpuart1	dmamux2_trg6				
			it_exti_rx_lpuart1	dmamux2_trg7				
			it_exti_out_lptim2	dmamux2_trg8				
			it_exti_out_lptim3	dmamux2_trg9				
			it_exti_wkup_i2c4	dmamux2_trg10				
			it_exti_wkup_spi6	dmamux2_trg11				
			it_exti_out_comp1	dmamux2_trg12				
			it_exti_wkup_rtc	dmamux2_trg13				
			it_exti_exti0_syscfg	dmamux2_trg14				
			it_exti_exti2_syscfg	dmamux2_trg15				
D3	AHB4	DMAMUX2	dmamux1_req_out0	bdma_ch0	BDMA	AHB4	D3	Requests out
			dmamux1_req_out1	bdma_ch1				
			dmamux1_req_out2	bdma_ch2				
			dmamux1_req_out3	bdma_ch3				
			dmamux1_req_out4	bdma_ch4				
			dmamux1_req_out5	bdma_ch5				
			dmamux1_req_out6	bdma_ch6				
			dmamux1_req_out7	bdma_ch7				

16 Direct memory access controller (DMA)

16.1 DMA introduction

Direct memory access (DMA) is used in order to provide high-speed data transfer between peripherals and memory and between memory and memory. Data can be quickly moved by DMA without any CPU action. This keeps CPU resources free for other operations.

The DMA controller combines a powerful dual AHB master bus architecture with independent FIFO to optimize the bandwidth of the system, based on a complex bus matrix architecture.

The two DMA controllers (DMA1, DMA2) have 8 streams each, dedicated to managing memory access requests from one or more peripherals.

Each DMA stream is driven by one DMAMUX1 output channel (request). Any DMAMUX1 output request can be individually programmed in order to select the DMA request source signal, from any of the 115 available request input signals.

Refer to the [Section 18.3: DMAMUX implementation](#) for more information about the DMA requests and streams mapping.

Each DMA controller has an arbiter for handling the priority between DMA requests.

16.2 DMA main features

The main DMA features are:

- Dual AHB master bus architecture, one dedicated to memory accesses and one dedicated to peripheral accesses
- AHB slave programming interface supporting only 32-bit accesses
- 8 streams for each DMA controller, up to 115 channels (requests) per stream
- Four-word depth 32 first-in, first-out memory buffers (FIFOs) per stream, that can be used in FIFO mode or direct mode:
 - FIFO mode: with threshold level software selectable between 1/4, 1/2 or 3/4 of the FIFO size
 - Direct mode: each DMA request immediately initiates a transfer from/to the memory. When it is configured in direct mode (FIFO disabled), to transfer data in memory-to-peripheral mode, the DMA preloads only one data from the memory to the internal FIFO to ensure an immediate data transfer as soon as a DMA request is triggered by a peripheral.
- Each stream can be configured to be:
 - a regular channel that supports peripheral-to-memory, memory-to-peripheral and memory-to-memory transfers
 - a double buffer channel that also supports double buffering on the memory side
- Priorities between DMA stream requests are software-programmable (four levels consisting of very high, high, medium, low) or hardware in case of equality (for example, request 0 has priority over request 1)
- Each stream also supports software trigger for memory-to-memory transfers

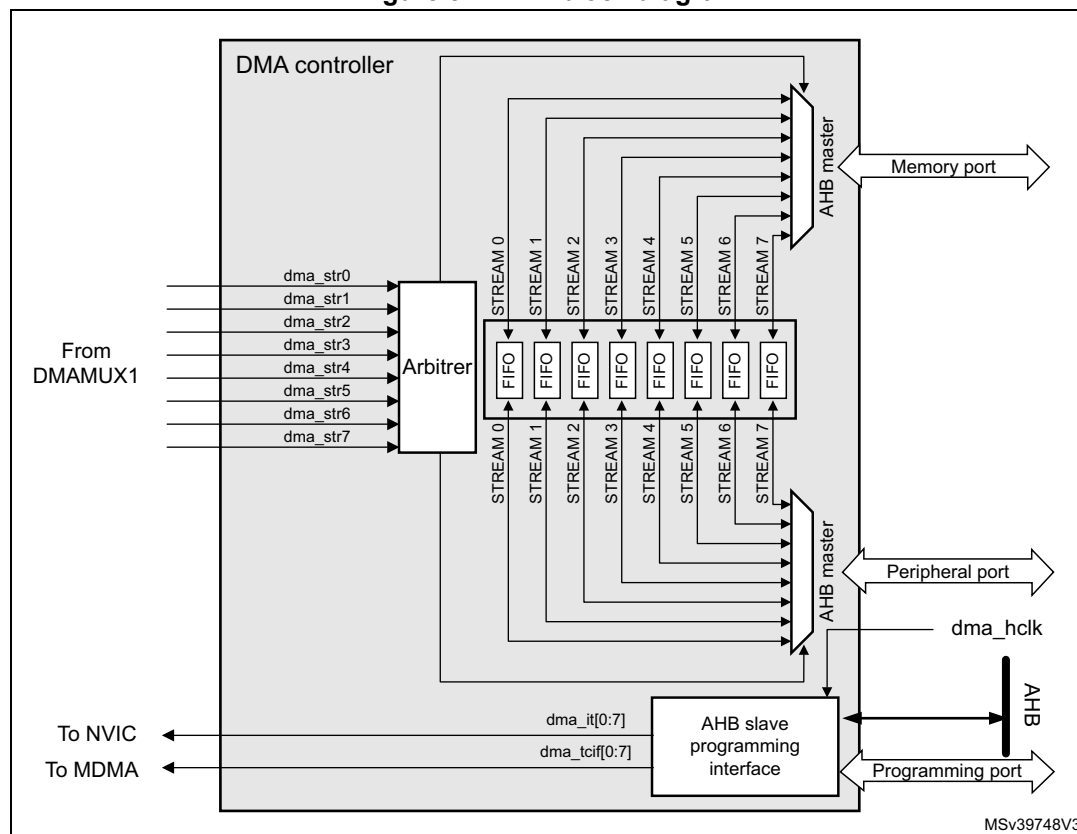
- Each stream request can be selected among up to 115 possible channel requests. This selection is software-configurable by the DMAMUX1 and allows 107 peripherals to initiate DMA requests
- The number of data items to be transferred can be managed either by the DMA controller or by the peripheral:
 - DMA flow controller: the number of data items to be transferred is software-programmable from 1 to 65535
 - Peripheral flow controller: the number of data items to be transferred is unknown and controlled by the source or the destination peripheral that signals the end of the transfer by hardware
- Independent source and destination transfer width (byte, half-word, word): when the data widths of the source and destination are not equal, the DMA automatically packs/unpacks the necessary transfers to optimize the bandwidth. This feature is only available in FIFO mode
- Incrementing or non-incrementing addressing for source and destination
- Supports incremental burst transfers of 4, 8 or 16 beats. The size of the burst is software-configurable, usually equal to half the FIFO size of the peripheral
- Each stream supports circular buffer management
- 5 event flags (DMA half transfer, DMA transfer complete, DMA transfer error, DMA FIFO error, direct mode error) logically ORed together in a single interrupt request for each stream

16.3 DMA functional description

16.3.1 DMA block diagram

The figure below shows the block diagram of a DMA.

Figure 84. DMA block diagram



16.3.2 DMA internal signals

The table below shows the internal DMA signals.

Table 111. DMA internal input/output signals

Signal name	Signal type	Description
dma_hclk	Digital input	DMA AHB clock
dma_it[0:7]	Digital outputs	DMA stream [0:7] global interrupts
dma_tcif[0:7]	Digital outputs	MDMA triggers
dma_str[0:7]	Digital input	DMA stream [0:7] requests

16.3.3 DMA overview

The DMA controller performs direct memory transfer: as an AHB master, the DMA controller can take the control of the AHB bus matrix to initiate AHB transactions.

The DMA controller carries out the following transactions:

- peripheral-to-memory
- memory-to-peripheral
- memory-to-memory

The DMA controller provides two AHB master ports: the AHB memory port, intended to be connected to memories and the AHB peripheral port, intended to be connected to peripherals. However, to allow memory-to-memory transfers, the AHB peripheral port must also have access to the memories.

The AHB slave port is used to program the DMA controller (it supports only 32-bit accesses).

16.3.4 DMA transactions

A DMA transaction consists of a sequence of a given number of data transfers. The number of data items to be transferred and their width (8-bit, 16-bit or 32-bit) are software-programmable.

Each DMA transfer consists of three operations:

- a loading from the peripheral data register or a location in memory, addressed through the DMA_SxPAR or DMA_SxM0AR register
- a storage of the data loaded to the peripheral data register or a location in memory addressed through the DMA_SxPAR or DMA_SxM0AR register
- a post-decrement of the DMA_SxNDTR register, containing the number of transactions that still have to be performed

After an event, the peripheral sends a request signal to the DMA controller. The DMA controller serves the request depending on the channel priorities. As soon as the DMA controller accesses the peripheral, an Acknowledge signal is sent to the peripheral by the DMA controller. The peripheral releases its request as soon as it gets the Acknowledge signal from the DMA controller. Once the request has been deasserted by the peripheral, the DMA controller releases the Acknowledge signal. If there are more requests, the peripheral can initiate the next transaction.

16.3.5 DMA request mapping

The DMA request mapping from peripherals to DMA streams is described in [Section 18.3: DMAMUX implementation](#).

16.3.6 Arbiter

An arbiter manages the 8 DMA stream requests based on their priority for each of the two AHB master ports (memory and peripheral ports) and launches the peripheral/memory access sequences.

Priorities are managed in two stages:

- Software: each stream priority can be configured in the DMA_SxCR register. There are four levels:
 - Very high priority
 - High priority
 - Medium priority
 - Low priority
- Hardware: If two requests have the same software priority level, the stream with the lower number takes priority over the stream with the higher number. For example, stream 2 takes priority over stream 4.

16.3.7 DMA streams

Each of the eight DMA controller streams provides a unidirectional transfer link between a source and a destination.

Each stream can be configured to perform:

- Regular type transactions: memory-to-peripherals, peripherals-to-memory or memory-to-memory transfers
- Double-buffer type transactions: double buffer transfers using two memory pointers for the memory (while the DMA is reading/writing from/to a buffer, the application can write/read to/from the other buffer).

The amount of data to be transferred (up to 65535) is programmable and related to the source width of the peripheral that requests the DMA transfer connected to the peripheral AHB port. The register that contains the amount of data items to be transferred is decremented after each transaction.

16.3.8 Source, destination and transfer modes

Both source and destination transfers can address peripherals and memories in the entire 4-Gbyte area, at addresses comprised between 0x0000 0000 and 0xFFFF FFFF.

The direction is configured using the DIR[1:0] bits in the DMA_SxCR register and offers three possibilities: memory-to-peripheral, peripheral-to-memory or memory-to-memory transfers.

The table below describes the corresponding source and destination addresses.

Table 112. Source and destination address

Bits DIR[1:0] of the DMA_SxCR register	Direction	Source address	Destination address
00	Peripheral-to-memory	DMA_SxPAR	DMA_SxM0AR
01	Memory-to-peripheral	DMA_SxM0AR	DMA_SxPAR

Table 112. Source and destination address (continued)

Bits DIR[1:0] of the DMA_SxCR register	Direction	Source address	Destination address
10	Memory-to-memory	DMA_SxPAR	DMA_SxM0AR
11	Reserved	-	-

When the data width (programmed in the PSIZE or MSIZE bits in the DMA_SxCR register) is a half-word or a word, respectively, the peripheral or memory address written into the DMA_SxPAR or DMA_SxM0AR/M1AR registers has to be aligned on a word or half-word address boundary, respectively.

Peripheral-to-memory mode

[Figure 85](#) describes this mode.

When this mode is enabled (by setting the bit EN in the DMA_SxCR register), each time a peripheral request occurs, the stream initiates a transfer from the source to fill the FIFO.

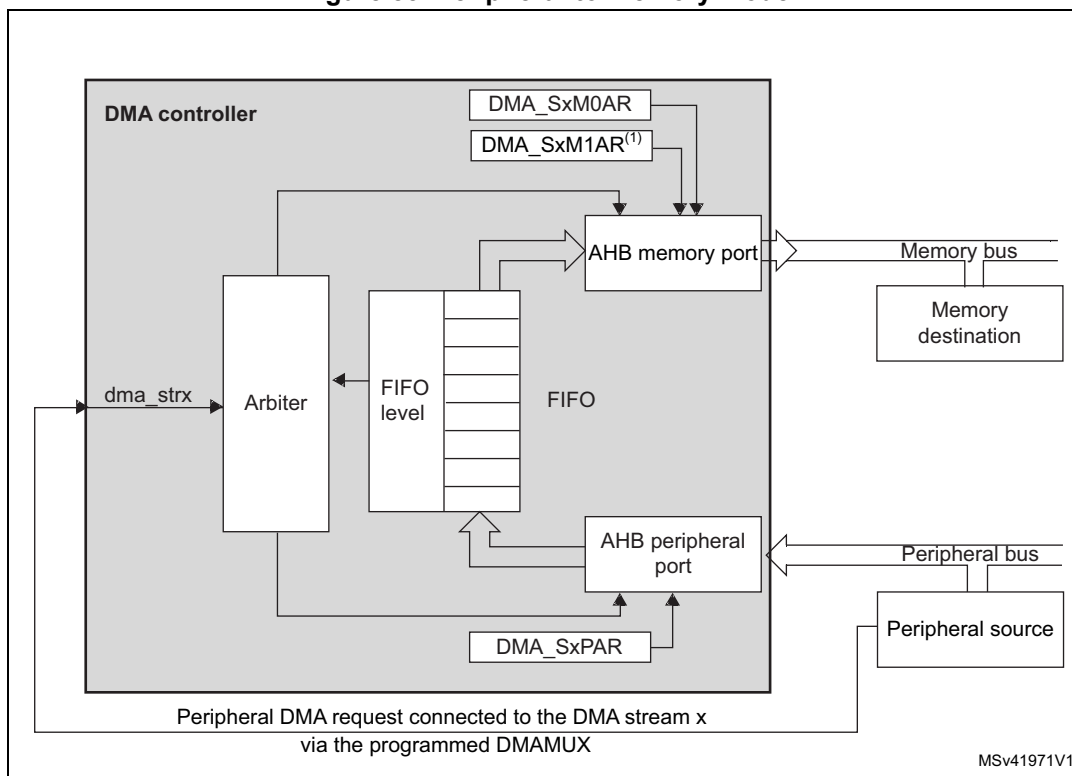
When the threshold level of the FIFO is reached, the contents of the FIFO are drained and stored into the destination.

The transfer stops once the DMA_SxNDTR register reaches zero, when the peripheral requests the end of transfers (in case of a peripheral flow controller) or when the EN bit in the DMA_SxCR register is cleared by software.

In direct mode (when the DMDIS value in the DMA_SxFCR register is 0), the threshold level of the FIFO is not used: after each single data transfer from the peripheral to the FIFO, the corresponding data are immediately drained and stored into the destination.

The stream has access to the AHB source or destination port only if the arbitration of the corresponding stream is won. This arbitration is performed using the priority defined for each stream using the PL[1:0] bits in the DMA_SxCR register.

Figure 85. Peripheral-to-memory mode



1. For double-buffer mode.

Memory-to-peripheral mode

[Figure 86](#) describes this mode.

When this mode is enabled (by setting the EN bit in the DMA_SxCR register), the stream immediately initiates transfers from the source to entirely fill the FIFO.

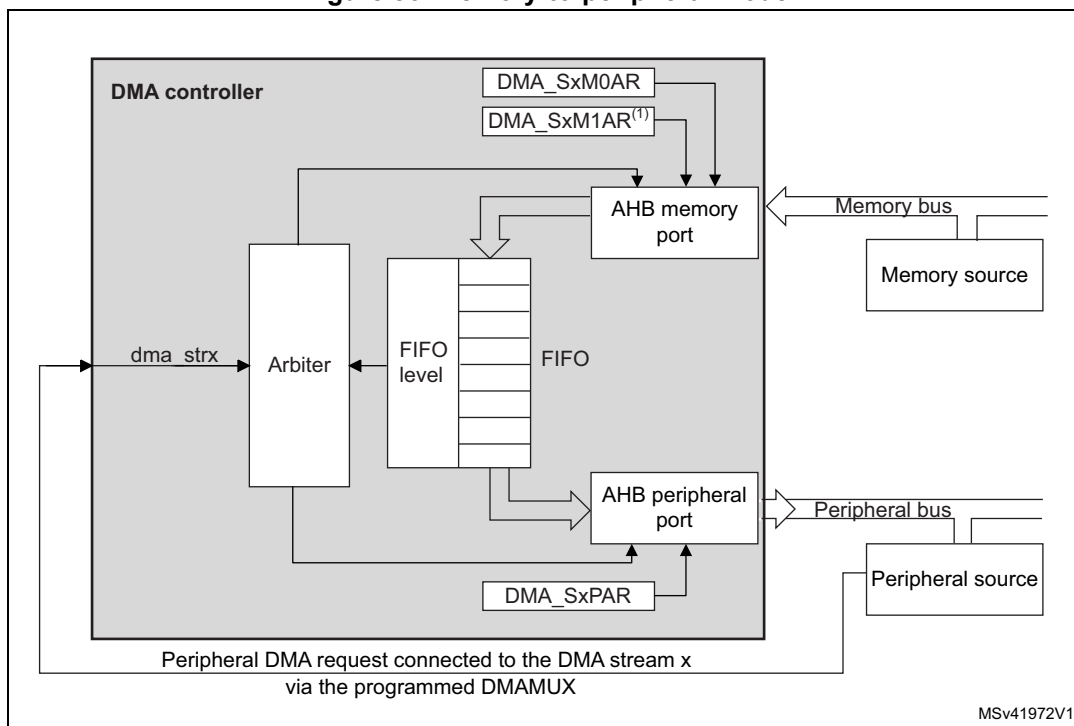
Each time a peripheral request occurs, the contents of the FIFO are drained and stored into the destination. When the level of the FIFO is lower than or equal to the predefined threshold level, the FIFO is fully reloaded with data from the memory.

The transfer stops once the DMA_SxNDTR register reaches zero, when the peripheral requests the end of transfers (in case of a peripheral flow controller) or when the EN bit in the DMA_SxCR register is cleared by software.

In direct mode (when the DMDIS value in the DMA_SxFCR register is 0), the threshold level of the FIFO is not used. Once the stream is enabled, the DMA preloads the first data to transfer into an internal FIFO. As soon as the peripheral requests a data transfer, the DMA transfers the preloaded value into the configured destination. It then reloads again the empty internal FIFO with the next data to be transfer. The preloaded data size corresponds to the value of the PSIZE bitfield in the DMA_SxCR register.

The stream has access to the AHB source or destination port only if the arbitration of the corresponding stream is won. This arbitration is performed using the priority defined for each stream using the PL[1:0] bits in the DMA_SxCR register.

Figure 86. Memory-to-peripheral mode



1. For double-buffer mode.

Memory-to-memory mode

The DMA channels can also work without being triggered by a request from a peripheral. This is the memory-to-memory mode, described in [Figure 87](#).

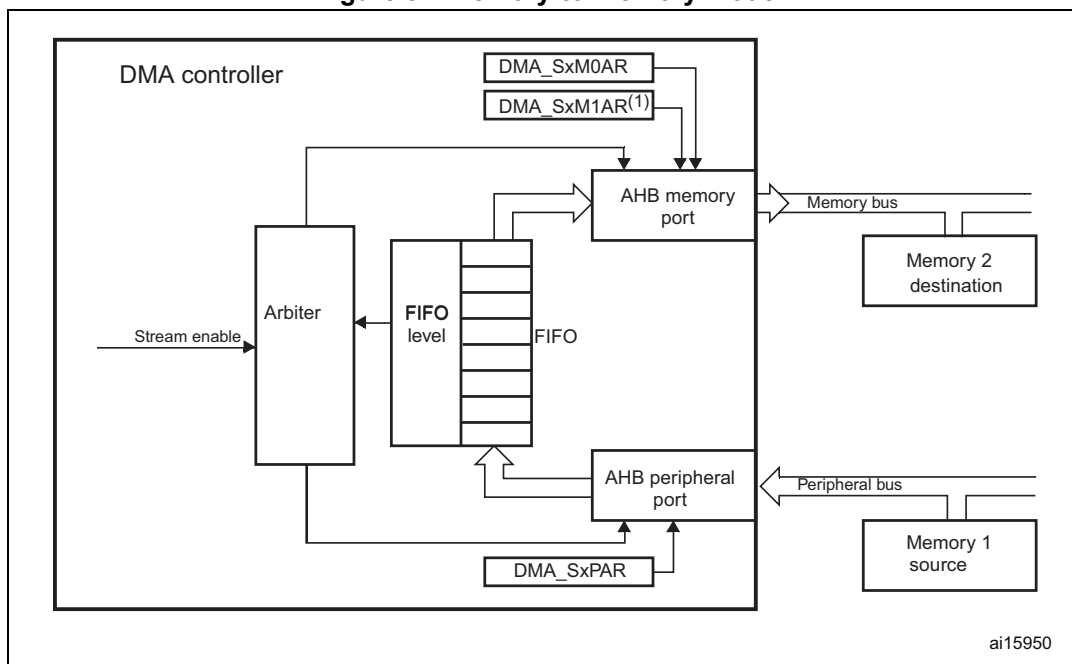
When the stream is enabled by setting the Enable bit (EN) in the DMA_SxCR register, the stream immediately starts to fill the FIFO up to the threshold level. When the threshold level is reached, the FIFO contents are drained and stored into the destination.

The transfer stops once the DMA_SxNDTR register reaches zero or when the EN bit in the DMA_SxCR register is cleared by software.

The stream has access to the AHB source or destination port only if the arbitration of the corresponding stream is won. This arbitration is performed using the priority defined for each stream using the PL[1:0] bits in the DMA_SxCR register.

Note: When memory-to-memory mode is used, the circular and direct modes are not allowed.

Figure 87. Memory-to-memory mode



1. For double-buffer mode.

16.3.9 Pointer incrementation

Peripheral and memory pointers can optionally be automatically post-incremented or kept constant after each transfer depending on the PINC and MINC bits in the DMA_SxCR register.

Disabling the increment mode is useful when the peripheral source or destination data is accessed through a single register.

If the increment mode is enabled, the address of the next transfer is the address of the previous one incremented by 1 (for bytes), 2 (for half-words) or 4 (for words) depending on the data width programmed in the PSIZE or MSIZE bits in the DMA_SxCR register.

In order to optimize the packing operation, it is possible to fix the increment offset size for the peripheral address whatever the size of the data transferred on the AHB peripheral port. The PINCOS bit in the DMA_SxCR register is used to align the increment offset size with the data size on the peripheral AHB port, or on a 32-bit address (the address is then incremented by 4). The PINCOS bit has an impact on the AHB peripheral port only.

If the PINCOS bit is set, the address of the following transfer is the address of the previous one incremented by 4 (automatically aligned on a 32-bit address), whatever the PSIZE value. The AHB memory port, however, is not impacted by this operation.

16.3.10 Circular mode

The circular mode is available to handle circular buffers and continuous data flows (e.g. ADC scan mode). This feature can be enabled using the CIRC bit in the DMA_SxCR register.

When the circular mode is activated, the number of data items to be transferred is automatically reloaded with the initial value programmed during the stream configuration phase, and the DMA requests continue to be served.

Note: *In the circular mode, it is mandatory to respect the following rule in case of a burst mode configured for memory:*

$DMA_SxNDTR = \text{Multiple of } ((Mburst\ beat) \times (Msize)/(Psize)), \text{ where:}$

- $(Mburst\ beat) = 4, 8 \text{ or } 16$ (depending on the MBURST bits in the DMA_SxCR register)
- $((Msize)/(Psize)) = 1, 2, 4, 1/2 \text{ or } 1/4$ (Msize and Psize represent the MSIZE and PSIZE bits in the DMA_SxCR register. They are byte dependent)
- $DMA_SxNDTR = \text{Number of data items to transfer on the AHB peripheral port}$

For example: Mburst beat = 8 (INCR8), MSIZE = 00 (byte) and PSIZE = 01 (half-word), in this case: DMA_SxNDTR must be a multiple of $(8 \times 1/2 = 4)$.

If this formula is not respected, the DMA behavior and data integrity are not guaranteed.

NDTR must also be a multiple of the Peripheral burst size multiplied by the peripheral data size, otherwise this could result in a bad DMA behavior.

16.3.11 Double-buffer mode

This mode is available for all the DMA1 and DMA2 streams.

The double-buffer mode is enabled by setting the DBM bit in the DMA_SxCR register.

A double-buffer stream works as a regular (single buffer) stream with the difference that it has two memory pointers. When the double-buffer mode is enabled, the circular mode is automatically enabled (CIRC bit in DMA_SxCR is not relevant) and at each end of transaction, the memory pointers are swapped.

In this mode, the DMA controller swaps from one memory target to another at each end of transaction. This allows the software to process one memory area while the second memory area is being filled/used by the DMA transfer. The double-buffer stream can work in both directions (the memory can be either the source or the destination) as described in [Table 113: Source and destination address registers in double-buffer mode \(DBM = 1\)](#).

Note: *In double-buffer mode, it is possible to update the base address for the AHB memory port on-the-fly (DMA_SxM0AR or DMA_SxM1AR) when the stream is enabled, by respecting the following conditions:*

- *When the CT bit is 0 in the DMA_SxCR register, the DMA_SxM1AR register can be written. Attempting to write to this register while CT = 1 sets an error flag (TEIF) and the stream is automatically disabled.*
- *When the CT bit is 1 in the DMA_SxCR register, the DMA_SxM0AR register can be written. Attempting to write to this register while CT = 0, sets an error flag (TEIF) and the stream is automatically disabled.*

To avoid any error condition, it is advised to change the base address as soon as the TCIF flag is asserted because, at this point, the targeted memory must have changed from

memory 0 to 1 (or from 1 to 0) depending on the value of CT in the DMA_SxCR register in accordance with one of the two above conditions.

For all the other modes (except the double-buffer mode), the memory address registers are write-protected as soon as the stream is enabled.

Table 113. Source and destination address registers in double-buffer mode (DBM = 1)

Bits DIR[1:0] of the DMA_SxCR register	Direction	Source address	Destination address
00	Peripheral-to-memory	DMA_SxPAR	DMA_SxM0AR / DMA_SxM1AR
01	Memory-to-peripheral	DMA_SxM0AR / DMA_SxM1AR	DMA_SxPAR
10	Not allowed ⁽¹⁾		
11	Reserved	-	-

1. When the double-buffer mode is enabled, the circular mode is automatically enabled. Since the memory-to-memory mode is not compatible with the circular mode, when the double-buffer mode is enabled, it is not allowed to configure the memory-to-memory mode.

16.3.12 Programmable data width, packing/unpacking, endianness

The number of data items to be transferred has to be programmed into DMA_SxNDTR (number of data items to transfer bit, NDT) before enabling the stream (except when the flow controller is the peripheral, PFCTRL bit in DMA_SxCR is set).

When using the internal FIFO, the data widths of the source and destination data are programmable through the PSIZE and MSIZE bits in the DMA_SxCR register (can be 8-, 16- or 32-bit).

When PSIZE and MSIZE are not equal:

- The data width of the number of data items to transfer, configured in the DMA_SxNDTR register is equal to the width of the peripheral bus (configured by the PSIZE bits in the DMA_SxCR register). For instance, in case of peripheral-to-memory, memory-to-peripheral or memory-to-memory transfers and if the PSIZE[1:0] bits are configured for half-word, the number of bytes to be transferred is equal to $2 \times \text{NDT}$.
- The DMA controller only copes with little-endian addressing for both source and destination. This is described in [Table 114: Packing/unpacking and endian behavior \(bit PINC = MINC = 1\)](#).

This packing/unpacking procedure may present a risk of data corruption when the operation is interrupted before the data are completely packed/unpacked. So, to ensure data coherence, the stream may be configured to generate burst transfers: in this case, each group of transfers belonging to a burst are indivisible (refer to [Section 16.3.13: Single and burst transfers](#)).

In direct mode (DMDIS = 0 in the DMA_SxFCR register), the packing/unpacking of data is not possible. In this case, it is not allowed to have different source and destination transfer data widths: both are equal and defined by the PSIZE bits in the DMA_SxCR register. MSIZE bits are not relevant.

Table 114. Packing/unpacking and endian behavior (bit PINC = MINC = 1)

AHB memory port width	AHB peripheral port width	Number of data items to transfer (NDT)	Memory transfer number	Memory port address / byte lane	Peripheral transfer number	Peripheral port address / byte lane	
						PINCOS = 1	PINCOS = 0
8	8	4	1	0x0 / B0[7:0]	1	0x0 / B0[7:0]	0x0 / B0[7:0]
			2	0x1 / B1[7:0]	2	0x4 / B1[7:0]	0x1 / B1[7:0]
			3	0x2 / B2[7:0]	3	0x8 / B2[7:0]	0x2 / B2[7:0]
			4	0x3 / B3[7:0]	4	0xC / B3[7:0]	0x3 / B3[7:0]
8	16	2	1	0x0 / B0[7:0]	1	0x0 / B1[B0[15:0]	0x0 / B1[B0[15:0]
			2	0x1 / B1[7:0]	2	0x4 / B3[B2[15:0]	0x1 / B1[B0[15:0]
			3	0x2 / B2[7:0]			0x2 / B3[B2[15:0]
			4	0x3 / B3[7:0]			
8	32	1	1	0x0 / B0[7:0]	1	0x0 / B3[B2[B1[B0[31:0]	0x0 / B3[B2[B1[B0[31:0]
			2	0x1 / B1[7:0]			
			3	0x2 / B2[7:0]			
			4	0x3 / B3[7:0]			
16	8	4	1	0x0 / B1[B0[15:0]	1	0x0 / B0[7:0]	0x0 / B0[7:0]
			2	0x2 / B3[B2[15:0]	2	0x4 / B1[7:0]	0x1 / B1[7:0]
					3	0x8 / B2[7:0]	0x2 / B2[7:0]
					4	0xC / B3[7:0]	0x3 / B3[7:0]
16	16	2	1	0x0 / B1[B0[15:0]	1	0x0 / B1[B0[15:0]	0x0 / B1[B0[15:0]
			2	0x2 / B1[B0[15:0]	2	0x4 / B3[B2[15:0]	0x2 / B3[B2[15:0]
16	32	1	1	0x0 / B1[B0[15:0]	1	0x0 / B3[B2[B1[B0[31:0]	0x0 / B3[B2[B1[B0[31:0]
			2	0x2 / B3[B2[15:0]			
32	8	4	1	0x0 / B3[B2[B1[B0[31:0]	1	0x0 / B0[7:0]	0x0 / B0[7:0]
					2	0x4 / B1[7:0]	0x1 / B1[7:0]
					3	0x8 / B2[7:0]	0x2 / B2[7:0]
					4	0xC / B3[7:0]	0x3 / B3[7:0]
32	16	2	1	0x0 / B3[B2[B1[B0[31:0]	1	0x0 / B1[B0[15:0]	0x0 / B1[B0[15:0]
					2	0x4 / B3[B2[15:0]	0x2 / B3[B2[15:0]
32	32	1	1	0x0 / B3[B2[B1[B0 [31:0]	1	0x0 / B3[B2[B1[B0 [31:0]	0x0 / B3[B2[B1[B0[31:0]

Note: Peripheral port may be the source or the destination (it can also be the memory source in the case of memory-to-memory transfer).

PSIZE, MSIZE and NDT[15:0] must be configured so as to ensure that the last transfer is not incomplete. This can occur when the data width of the peripheral port (PSIZE bits) is lower than the data width of the memory port (MSIZE bits). This constraint is summarized in the table below.

Table 115. Restriction on NDT versus PSIZE and MSIZE

PSIZE[1:0] of DMA_SxCR	MSIZE[1:0] of DMA_SxCR	NDT[15:0] of DMA_SxNDTR
00 (8-bit)	01 (16-bit)	Must be a multiple of 2.
00 (8-bit)	10 (32-bit)	Must be a multiple of 4.
01 (16-bit)	10 (32-bit)	Must be a multiple of 2.

16.3.13 Single and burst transfers

The DMA controller can generate single transfers or incremental burst transfers of 4, 8 or 16 beats.

The size of the burst is configured by software independently for the two AHB ports by using the MBURST[1:0] and PBURST[1:0] bits in the DMA_SxCR register.

The burst size indicates the number of beats in the burst, not the number of bytes transferred.

To ensure data coherence, each group of transfers that form a burst are indivisible: AHB transfers are locked and the arbiter of the AHB bus matrix does not degrant the DMA master during the sequence of the burst transfer.

Depending on the single or burst configuration, each DMA request initiates a different number of transfers on the AHB peripheral port:

- When the AHB peripheral port is configured for single transfers, each DMA request generates a data transfer of a byte, half-word or word depending on the PSIZE[1:0] bits in the DMA_SxCR register
- When the AHB peripheral port is configured for burst transfers, each DMA request generates 4, 8 or 16 beats of byte, half word or word transfers depending on the PBURST[1:0] and PSIZE[1:0] bits in the DMA_SxCR register.

The same as above has to be considered for the AHB memory port considering the MBURST and MSIZE bits.

In direct mode, the stream can only generate single transfers and the MBURST[1:0] and PBURST[1:0] bits are forced by hardware.

The address pointers (DMA_SxPAR or DMA_SxM0AR registers) must be chosen so as to ensure that all transfers within a burst block are aligned on the address boundary equal to the size of the transfer.

The burst configuration has to be selected in order to respect the AHB protocol, where bursts **must not** cross the 1 Kbyte address boundary because the minimum address space that can be allocated to a single slave is 1 Kbyte. This means that the 1-Kbyte address boundary **must not** be crossed by a burst block transfer, otherwise an AHB error is generated, that is not reported by the DMA registers.

16.3.14 FIFO

FIFO structure

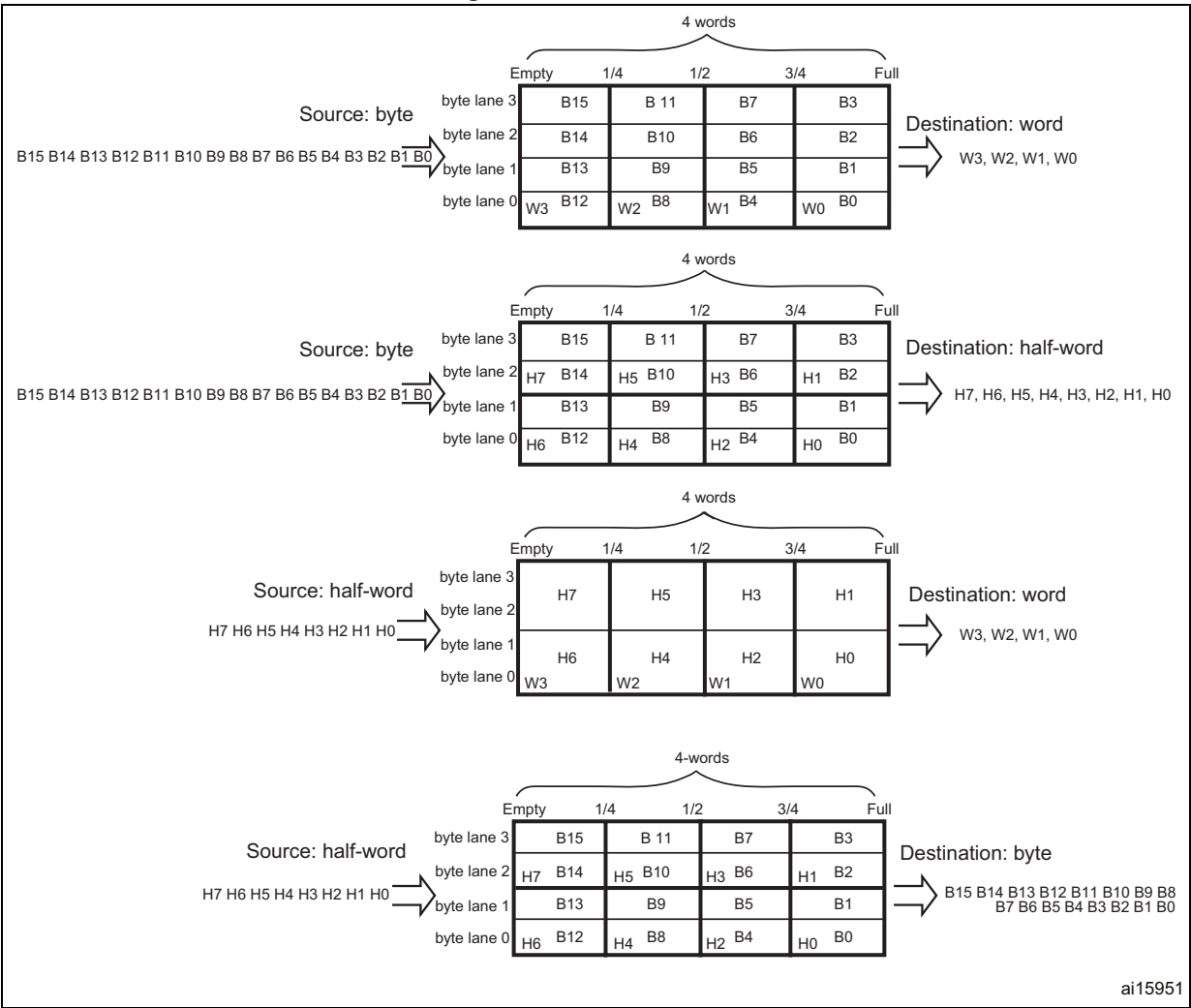
The FIFO is used to temporarily store data coming from the source before transmitting them to the destination.

Each stream has an independent 4-word FIFO and the threshold level is software-configurable between 1/4, 1/2, 3/4 or full.

To enable the use of the FIFO threshold level, the direct mode must be disabled by setting the DMDIS bit in the DMA_SxFCR register.

The structure of the FIFO differs depending on the source and destination data widths, and is described in the figure below.

Figure 88. FIFO structure



ai15951

FIFO threshold and burst configuration

Caution is required when choosing the FIFO threshold (bits FTH[1:0] of the DMA_SxFCR register) and the size of the memory burst (MBURST[1:0] of the DMA_SxCR register): The content pointed by the FIFO threshold must exactly match an integer number of memory burst transfers. If this is not in the case, a FIFO error (flag FEIFx of the DMA_HISR or DMA_LISR register) is generated when the stream is enabled, then the stream is automatically disabled. The allowed and forbidden configurations are described in the table below. The forbidden configurations are highlighted in gray in the table.

Table 116. FIFO threshold configurations

MSIZE	FIFO level	MBURST = INCR4	MBURST = INCR8	MBURST = INCR16	
Byte	1/4	1 burst of 4 beats	Forbidden	Forbidden	
	1/2	2 bursts of 4 beats	1 burst of 8 beats		
	3/4	3 bursts of 4 beats	Forbidden		
	Full	4 bursts of 4 beats	2 bursts of 8 beats	1 burst of 16 beats	
Half-word	1/4	Forbidden	Forbidden	Forbidden	
	1/2	1 burst of 4 beats			
	3/4	Forbidden			
	Full	2 bursts of 4 beats	1 burst of 8 beats		
Word	1/4	Forbidden	Forbidden		
	1/2				
	3/4				
	Full	1 burst of 4 beats			

In all cases, the burst size multiplied by the data size must not exceed the FIFO size (data size can be: 1 (byte), 2 (half-word) or 4 (word)).

Incomplete burst transfer at the end of a DMA transfer may happen if one of the following conditions occurs:

- For the AHB peripheral port configuration: the total number of data items (set in the DMA_SxNDTR register) is not a multiple of the burst size multiplied by the data size.
- For the AHB memory port configuration: the number of remaining data items in the FIFO to be transferred to the memory is not a multiple of the burst size multiplied by the data size.

In such cases, the remaining data to be transferred is managed in single mode by the DMA, even if a burst transaction is requested during the DMA stream configuration.

Note: *When burst transfers are requested on the peripheral AHB port and the FIFO is used (DMDIS = 1 in the DMA_SxCR register), it is mandatory to respect the following rule to avoid permanent underrun or overrun conditions, depending on the DMA stream direction:*

If $(PBURST \times PSIZE) = FIFO_SIZE$ (4 words), $FIFO_Threshold = 3/4$ is forbidden with $PSIZE = 1, 2$ or 4 and $PBURST = 4, 8$ or 16 .

This rule ensures that enough FIFO space at a time is free to serve the request from the peripheral.

FIFO flush

The FIFO can be flushed when the stream is disabled by resetting the EN bit in the DMA_SxCR register and when the stream is configured to manage peripheral-to-memory or memory-to-memory transfers. If some data are still present in the FIFO when the stream is disabled, the DMA controller continues transferring the remaining data to the destination (even though stream is effectively disabled). When this flush is completed, the transfer complete status bit (TCIFx) in the DMA_LISR or DMA_HISR register is set.

The remaining data counter DMA_SxNDTR keeps the value in this case to indicate how many data items are currently available in the destination memory.

Note that during the FIFO flush operation, if the number of remaining data items in the FIFO to be transferred to memory (in bytes) is less than the memory data width (for example 2 bytes in FIFO while MSIZE is configured to word), data is sent with the data width set in the MSIZE bit in the DMA_SxCR register. This means that memory is written with an undesired value. The software may read the DMA_SxNDTR register to determine the memory area that contains the good data (start address and last address).

If the number of remaining data items in the FIFO is lower than a burst size (if the MBURST bits in DMA_SxCR register are set to configure the stream to manage burst on the AHB memory port), single transactions are generated to complete the FIFO flush.

Direct mode

By default, the FIFO operates in direct mode (DMDIS bit in the DMA_SxFCR is reset) and the FIFO threshold level is not used. This mode is useful when the system requires an immediate and single transfer to or from the memory after each DMA request.

When the DMA is configured in direct mode (FIFO disabled), to transfer data in memory-to-peripheral mode, the DMA preloads one data from the memory to the internal FIFO to ensure an immediate data transfer as soon as a DMA request is triggered by a peripheral.

To avoid saturating the FIFO, it is recommended to configure the corresponding stream with a high priority.

This mode is restricted to transfers where:

- the source and destination transfer widths are equal and both defined by the PSIZE[1:0] bits in DMA_SxCR (MSIZE[1:0] bits are not relevant)
- burst transfers are not possible (PBURST[1:0] and MBURST[1:0] bits in DMA_SxCR are don't care)

Direct mode must not be used when implementing memory-to-memory transfers.

16.3.15 DMA transfer completion

Different events can generate an end of transfer by setting the TCIFx bit in the DMA_LISR or DMA_HISR status register:

- In DMA flow controller mode:
 - The DMA_SxNDTR counter has reached zero in the memory-to-peripheral mode.
 - The stream is disabled before the end of transfer (by clearing the EN bit in the DMA_SxCR register) and (when transfers are peripheral-to-memory or memory-

to-memory) all the remaining data have been flushed from the FIFO into the memory.

- In Peripheral flow controller mode:
 - The last external burst or single request has been generated from the peripheral and (when the DMA is operating in peripheral-to-memory mode) the remaining data have been transferred from the FIFO into the memory
 - The stream is disabled by software, and (when the DMA is operating in peripheral-to-memory mode) the remaining data have been transferred from the FIFO into the memory

Note: The transfer completion is dependent on the remaining data in FIFO to be transferred into memory only in the case of peripheral-to-memory mode. This condition is not applicable in memory-to-peripheral mode.

If the stream is configured in non-circular mode, after the end of the transfer (that is when the number of data to be transferred reaches zero), the DMA is stopped (EN bit in DMA_SxCR register is cleared by Hardware) and no DMA request is served unless the software reprograms the stream and re-enables it (by setting the EN bit in the DMA_SxCR register).

16.3.16 DMA transfer suspension

At any time, a DMA transfer can be suspended to be restarted later on or to be definitively disabled before the end of the DMA transfer.

There are two cases:

- The stream disables the transfer with no later-on restart from the point where it was stopped. There is no particular action to do, except to clear the EN bit in the DMA_SxCR register to disable the stream. The stream may take time to be disabled (ongoing transfer is completed first). The transfer complete interrupt flag (TCIF in the DMA_LISR or DMA_HISR register) is set in order to indicate the end of transfer. The value of the EN bit in DMA_SxCR is now 0 to confirm the stream interruption. The DMA_SxNDTR register contains the number of remaining data items at the moment when the stream was stopped so that the software can determine how many data items have been transferred before the stream was interrupted.
- The stream suspends the transfer before the number of remaining data items to be transferred in the DMA_SxNDTR register reaches 0. The aim is to restart the transfer later by re-enabling the stream. In order to restart from the point where the transfer was stopped, the software has to read the DMA_SxNDTR register after disabling the stream by writing the EN bit in DMA_SxCR register (and then checking that it is at 0) to know the number of data items already collected. Then:
 - The peripheral and/or memory addresses have to be updated in order to adjust the address pointers
 - The SxNDTR register has to be updated with the remaining number of data items to be transferred (the value read when the stream was disabled)
 - The stream may then be re-enabled to restart the transfer from the point it was stopped

Note: A transfer complete interrupt flag (TCIF in DMA_LISR or DMA_HISR) is set to indicate the end of transfer due to the stream interruption.

16.3.17 Flow controller

The entity that controls the number of data to be transferred is known as the flow controller. This flow controller is configured independently for each stream using the PFCTRL bit in the DMA_SxCR register.

The flow controller can be:

- The DMA controller: in this case, the number of data items to be transferred is programmed by software into the DMA_SxNDTR register before the DMA stream is enabled.
- The peripheral source or destination: this is the case when the number of data items to be transferred is unknown. The peripheral indicates by hardware to the DMA controller when the last data are being transferred. This feature is only supported for peripherals that are able to signal the end of the transfer.

When the peripheral flow controller is used for a given stream, the value written into the DMA_SxNDTR has no effect on the DMA transfer. Actually, whatever the value written, it is forced by hardware to 0xFFFF as soon as the stream is enabled, to respect the following schemes:

- Anticipated stream interruption: EN bit in DMA_SxCR register is reset to 0 by the software to stop the stream before the last data hardware signal (single or burst) is sent by the peripheral. In such a case, the stream is switched off and the FIFO flush is triggered in the case of a peripheral-to-memory DMA transfer. The TCIFx flag of the corresponding stream is set in the status register to indicate the DMA completion. To know the number of data items transferred during the DMA transfer, read the DMA_SxNDTR register and apply the following formula:
 – $\text{Number_of_data_transferred} = 0xFFFF - \text{DMA_SxNDTR}$
- Normal stream interruption due to the reception of a last data hardware signal: the stream is automatically interrupted when the peripheral requests the last transfer (single or burst) and when this transfer is complete. the TCIFx flag of the corresponding stream is set in the status register to indicate the DMA transfer completion. To know the number of data items transferred, read the DMA_SxNDTR register and apply the same formula as above.
- The DMA_SxNDTR register reaches 0: the TCIFx flag of the corresponding stream is set in the status register to indicate the forced DMA transfer completion. The stream is automatically switched off even though the last data hardware signal (single or burst) has not been yet asserted. The already transferred data is not lost. This means that a maximum of 65535 data items can be managed by the DMA in a single transaction, even in peripheral flow control mode.

Note: When configured in memory-to-memory mode, the DMA is always the flow controller and the PFCTRL bit is forced to 0 by hardware.

The circular mode is forbidden in the peripheral flow controller mode.

16.3.18 Summary of the possible DMA configurations

The table below summarizes the different possible DMA configurations. The forbidden configurations are highlighted in gray in the table.

Table 117. Possible DMA configurations

DMA transfer mode	Source	Destination	Flow controller	Circular mode	Transfer type	Direct mode	Double-buffer mode
Peripheral-to-memory	AHB peripheral port	AHB memory port	DMA	Possible	single	Possible	Possible
					burst	Forbidden	
			Peripheral	Forbidden	single	Possible	Forbidden
					burst	Forbidden	
Memory-to-peripheral	AHB memory port	AHB peripheral port	DMA	Possible	single	Possible	Possible
					burst	Forbidden	
			Peripheral	Forbidden	single	Possible	Forbidden
					burst	Forbidden	
Memory-to-memory	AHB peripheral port	AHB memory port	DMA only	Forbidden	single	Forbidden	Forbidden
					burst		

16.3.19 Stream configuration procedure

The following sequence must be followed to configure a DMA stream x (where x is the stream number):

1. If the stream is enabled, disable it by resetting the EN bit in the DMA_SxCR register, then read this bit in order to confirm that there is no ongoing stream operation. Writing this bit to 0 is not immediately effective since it is actually written to 0 once all the current transfers are finished. When the EN bit is read as 0, this means that the stream is ready to be configured. It is therefore necessary to wait for the EN bit to be cleared before starting any stream configuration. All the stream dedicated bits set in the status register (DMA_LISR and DMA_HISR) from the previous data block DMA transfer must be cleared before the stream can be re-enabled.
2. Set the peripheral port register address in the DMA_SxPAR register. The data is moved from/ to this address to/ from the peripheral port after the peripheral event.
3. Set the memory address in the DMA_SxMA0R register (and in the DMA_SxMA1R register in the case of a double-buffer mode). The data is written to or read from this memory after the peripheral event.
4. Configure the total number of data items to be transferred in the DMA_SxNDTR register. After each peripheral event or each beat of the burst, this value is decremented.
5. Use DMAMUX1 to route a DMA request line to the DMA channel.
6. If the peripheral is intended to be the flow controller and if it supports this feature, set the PFCTRL bit in the DMA_SxCR register.
7. Configure the stream priority using the PL[1:0] bits in the DMA_SxCR register.
8. Configure the FIFO usage (enable or disable, threshold in transmission and reception)

9. Configure the data transfer direction, peripheral and memory incremented/fixed mode, single or burst transactions, peripheral and memory data widths, circular mode, double-buffer mode and interrupts after half and/or full transfer, and/or errors in the DMA_SxCR register.
10. Activate the stream by setting the EN bit in the DMA_SxCR register.

As soon as the stream is enabled, it can serve any DMA request from the peripheral connected to the stream.

Once half the data have been transferred on the AHB destination port, the half-transfer flag (HTIF) is set and an interrupt is generated if the half-transfer interrupt enable bit (HTIE) is set. At the end of the transfer, the transfer complete flag (TCIF) is set and an interrupt is generated if the transfer complete interrupt enable bit (TCIE) is set.

Warning: To switch off a peripheral connected to a DMA stream request, it is mandatory to, first, switch off the DMA stream to which the peripheral is connected, then to wait for EN bit = 0. Only then can the peripheral be safely disabled.

16.3.20 Error management

The DMA controller can detect the following errors:

- **Transfer error:** the transfer error interrupt flag (TEIFx) is set when:
 - a bus error occurs during a DMA read or a write access
 - a write access is requested by software on a memory address register in double-buffer mode whereas the stream is enabled and the current target memory is the one impacted by the write into the memory address register (refer to [Section 16.3.11: Double-buffer mode](#))
- **FIFO error:** the FIFO error interrupt flag (FEIFx) is set if:
 - a FIFO underrun condition is detected
 - a FIFO overrun condition is detected (no detection in memory-to-memory mode because requests and transfers are internally managed by the DMA)
 - the stream is enabled while the FIFO threshold level is not compatible with the size of the memory burst (refer to [Table 116: FIFO threshold configurations](#))
- **Direct mode error:** the direct mode error interrupt flag (DMEIFx) can only be set in the peripheral-to-memory mode while operating in direct mode and when the MINC bit in the DMA_SxCR register is cleared. This flag is set when a DMA request occurs while the previous data have not yet been fully transferred into the memory (because the memory bus was not granted). In this case, the flag indicates that two data items were be transferred successively to the same destination address, which could be an issue if the destination is not able to manage this situation

In direct mode, the FIFO error flag can also be set under the following conditions:

- In the peripheral-to-memory mode, the FIFO can be saturated (overrun) if the memory bus is not granted for several peripheral requests.
- In the memory-to-peripheral mode, an underrun condition may occur if the memory bus has not been granted before a peripheral request occurs.

If the TEIFx or the FEIFx flag is set due to incompatibility between burst size and FIFO threshold level, the faulty stream is automatically disabled through a hardware clear of its EN bit in the corresponding stream configuration register (DMA_SxCR).

If the DMEIFx or the FEIFx flag is set due to an overrun or underrun condition, the faulty stream is not automatically disabled and it is up to the software to disable or not the stream by resetting the EN bit in the DMA_SxCR register. This is because there is no data loss when this kind of errors occur.

When the stream's error interrupt flag (TEIF, FEIF, DMEIF) in the DMA_LISR or DMA_HISR register is set, an interrupt is generated if the corresponding interrupt enable bit (TEIE, FEIE, DMIE) in the DMA_SxCR or DMA_SxFCR register is set.

Note: *When a FIFO overrun or underrun condition occurs, the data is not lost because the peripheral request is not acknowledged by the stream until the overrun or underrun condition is cleared. If this acknowledge takes too much time, the peripheral itself may detect an overrun or underrun condition of its internal buffer and data might be lost.*

16.4 DMA interrupts

For each DMA stream, an interrupt can be produced on the following events:

- Half-transfer reached
- Transfer complete
- Transfer error
- FIFO error (overrun, underrun or FIFO level error)
- Direct mode error

Separate interrupt enable control bits are available for flexibility as shown in the table below.

Table 118. DMA interrupt requests

Interrupt event	Event flag	Enable control bit
Half-transfer	HTIF	HTIE
Transfer complete	TCIF	TCIE
Transfer error	TEIF	TEIE
FIFO overrun/underrun	FEIF	FEIE
Direct mode error	DMEIF	DMEIE

Note: *Before setting an enable control bit EN = 1, the corresponding event flag must be cleared, otherwise an interrupt is immediately generated.*

16.5 DMA registers

The DMA registers have to be accessed by words (32 bits).

16.5.1 DMA low interrupt status register (DMA_LISR)

Address offset: 0x000

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TCIF3	HTIF3	TEIF3	DMEIF3	Res.	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Res.	FEIF2
				r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TCIF1	HTIF1	TEIF1	DMEIF1	Res.	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Res.	FEIF0
				r	r	r	r		r	r	r	r	r		r

Bits 31:28, 15:12 Reserved, must be kept at reset value.

Bits 27, 21, 11, 5 **TCIF[3:0]**: Stream x transfer complete interrupt flag (x = 3 to 0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_LIFCR register.

0: No transfer complete event on stream x

1: A transfer complete event occurred on stream x.

Bits 26, 20, 10, 4 **HTIF[3:0]**: Stream x half transfer interrupt flag (x = 3 to 0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_LIFCR register.

0: No half transfer event on stream x

1: An half transfer event occurred on stream x.

Bits 25, 19, 9, 3 **TEIF[3:0]**: Stream x transfer error interrupt flag (x = 3 to 0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_LIFCR register.

0: No transfer error on stream x

1: A transfer error occurred on stream x.

Bits 24, 18, 8, 2 **DMEIF[3:0]**: Stream x direct mode error interrupt flag (x = 3 to 0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_LIFCR register.

0: No direct mode error on stream x

1: A direct mode error occurred on stream x.

Bits 23, 17, 7, 1 Reserved, must be kept at reset value.

Bits 22, 16, 6, 0 **FEIF[3:0]**: Stream x FIFO error interrupt flag (x = 3 to 0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_LIFCR register.

0: No FIFO error event on stream x

1: A FIFO error event occurred on stream x.

16.5.2 DMA high interrupt status register (DMA_HISR)

Address offset: 0x004

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TCIF7	HTIF7	TEIF7	DMEIF7	Res.	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Res.	FEIF6
				r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TCIF5	HTIF5	TEIF5	DMEIF5	Res.	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Res.	FEIF4
				r	r	r	r		r	r	r	r	r		r

Bits 31:28, 15:12 Reserved, must be kept at reset value.

Bits 27, 21, 11, 5 **TCIF[7:4]**: Stream x transfer complete interrupt flag (x = 7 to 4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_HIFCR register.

0: No transfer complete event on stream x

1: A transfer complete event occurred on stream x.

Bits 26, 20, 10, 4 **HTIF[7:4]**: Stream x half transfer interrupt flag (x = 7 to 4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_HIFCR register.

0: No half transfer event on stream x

1: An half transfer event occurred on stream x.

Bits 25, 19, 9, 3 **TEIF[7:4]**: Stream x transfer error interrupt flag (x = 7 to 4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_HIFCR register.

0: No transfer error on stream x

1: A transfer error occurred on stream x.

Bits 24, 18, 8, 2 **DMEIF[7:4]**: Stream x direct mode error interrupt flag (x = 7 to 4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_HIFCR register.

0: No direct mode error on stream x

1: A direct mode error occurred on stream x.

Bits 23, 17, 7, 1 Reserved, must be kept at reset value.

Bits 22, 16, 6, 0 **FEIF[7:4]**: Stream x FIFO error interrupt flag (x = 7 to 4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in DMA_HIFCR register.

0: No FIFO error event on stream x

1: A FIFO error event occurred on stream x.

16.5.3 DMA low interrupt flag clear register (DMA_LIFCR)

Address offset: 0x008

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	CTCIF3	CHTIF3	CTEIF3	CDMEIF3	Res.	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Res.	CFEIF2
				w	w	w	w		w	w	w	w	w		w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Res.	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Res.	CFEIF0
				w	w	w	w		w	w	w	w	w		w

Bits 31:28, 15:12 Reserved, must be kept at reset value.

Bits 27, 21, 11, 5 **CTCIF[3:0]**: Stream x clear transfer complete interrupt flag (x = 3 to 0)

Writing 1 to this bit clears the corresponding TCIFx flag in the DMA_LISR register.

Bits 26, 20, 10, 4 **CHTIF[3:0]**: Stream x clear half transfer interrupt flag (x = 3 to 0)

Writing 1 to this bit clears the corresponding HTIFx flag in the DMA_LISR register

Bits 25, 19, 9, 3 **CTEIF[3:0]**: Stream x clear transfer error interrupt flag (x = 3 to 0)

Writing 1 to this bit clears the corresponding TEIFx flag in the DMA_LISR register.

Bits 24, 18, 8, 2 **CDMEIF[3:0]**: Stream x clear direct mode error interrupt flag (x = 3 to 0)

Writing 1 to this bit clears the corresponding DMEIFx flag in the DMA_LISR register.

Bits 23, 17, 7, 1 Reserved, must be kept at reset value.

Bits 22, 16, 6, 0 **CFEIF[3:0]**: Stream x clear FIFO error interrupt flag (x = 3 to 0)

Writing 1 to this bit clears the corresponding CFEIFx flag in the DMA_LISR register.

16.5.4 DMA high interrupt flag clear register (DMA_HIFCR)

Address offset: 0x00C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	CTCIF7	CHTIF7	CTEIF7	CDMEIF7	Res.	CFEIF7	CTCIF6	CHTIF6	CTEIF6	CDMEIF6	Res.	CFEIF6
				w	w	w	w		w	w	w	w	w		w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	CTCIF5	CHTIF5	CTEIF5	CDMEIF5	Res.	CFEIF5	CTCIF4	CHTIF4	CTEIF4	CDMEIF4	Res.	CFEIF4
				w	w	w	w		w	w	w	w	w		w

Bits 31:28, 15:12 Reserved, must be kept at reset value.

Bits 27, 21, 11, 5 **CTCIF[7:4]**: Stream x clear transfer complete interrupt flag (x = 7 to 4)

Writing 1 to this bit clears the corresponding TCIFx flag in the DMA_HISR register.

Bits 26, 20, 10, 4 **CHTIF[7:4]**: Stream x clear half transfer interrupt flag (x = 7 to 4)

Writing 1 to this bit clears the corresponding HTIFx flag in the DMA_HISR register.

Bits 25, 19, 9, 3 **CTEIF[7:4]**: Stream x clear transfer error interrupt flag (x = 7 to 4)

Writing 1 to this bit clears the corresponding TEIFx flag in the DMA_HISR register.

Bits 24, 18, 8, 2 **CDMEIF[7:4]**: Stream x clear direct mode error interrupt flag (x = 7 to 4)

Writing 1 to this bit clears the corresponding DMEIFx flag in the DMA_HISR register.

Bits 23, 17, 7, 1 Reserved, must be kept at reset value.

Bits 22, 16, 6, 0 **CFEIF[7:4]**: Stream x clear FIFO error interrupt flag (x = 7 to 4)

Writing 1 to this bit clears the corresponding CFEIFx flag in the DMA_HISR register.

16.5.5 DMA stream x configuration register (DMA_SxCR)

This register is used to configure the concerned stream.

Address offset: $0x010 + 0x18 * x$, (x = 0 to 7)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	MBURST[1:0]		PBURST[1:0]		TRBUF F	CT	DBM	PL[1:0]	
							r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINC OS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PF CTRL	TCIE	HTIE	TEIE	DMEIE	EN
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:25 Reserved, must be kept at reset value.

Bits 24:23 **MBURST[1:0]**: Memory burst transfer configuration

These bits are set and cleared by software.

00: Single transfer

01: INCR4 (incremental burst of 4 beats)

10: INCR8 (incremental burst of 8 beats)

11: INCR16 (incremental burst of 16 beats)

These bits are protected and can be written only if EN = 0.

In direct mode, these bits are forced to 0x0 by hardware as soon as bit EN = 1.

Bits 22:21 **PBURST[1:0]**: Peripheral burst transfer configuration

These bits are set and cleared by software.

00: Single transfer

01: INCR4 (incremental burst of 4 beats)

10: INCR8 (incremental burst of 8 beats)

11: INCR16 (incremental burst of 16 beats)

These bits are protected and can be written only if EN = 0.

In direct mode, these bits are forced to 0x0 by hardware.

Bit 20 **TRBUFF**: Enable the DMA to handle bufferable transfers

0: Bufferable transfers not enabled

1: Bufferable transfers enabled

Note: This bit must be set to 1 if the DMA stream manages UART/USART/LPUART transfers.

- Bit 19 **CT**: Current target (only in double-buffer mode)
 This bit is set and cleared by hardware. It can also be written by software.
 0: Current target memory is memory 0 (addressed by the DMA_SxM0AR pointer).
 1: Current target memory is memory 1 (addressed by the DMA_SxM1AR pointer).
 This bit can be written only if EN = 0 to indicate the target memory area of the first transfer.
 Once the stream is enabled, this bit operates as a status flag indicating which memory area is the current target.
- Bit 18 **DBM**: Double-buffer mode
 This bit is set and cleared by software.
 0: No buffer switching at the end of transfer
 1: Memory target switched at the end of the DMA transfer
 This bit is protected and can be written only if EN = 0.
- Bits 17:16 **PL[1:0]**: priority level
 These bits are set and cleared by software.
 00: Low
 01: Medium
 10: High
 11: Very high
 These bits are protected and can be written only if EN = 0.
- Bit 15 **PINCOS**: Peripheral increment offset size
 This bit is set and cleared by software
 0: The offset size for the peripheral address calculation is linked to the PSIZE.
 1: The offset size for the peripheral address calculation is fixed to 4 (32-bit alignment).
 This bit has no meaning if bit PINC = 0.
 This bit is protected and can be written only if EN = 0.
 This bit is forced low by hardware when the stream is enabled (EN = 1) if the direct mode is selected or if PBURST are different from 00.
- Bits 14:13 **MSIZE[1:0]**: Memory data size
 These bits are set and cleared by software.
 00: Byte (8-bit)
 01: Half-word (16-bit)
 10: Word (32-bit)
 11: Reserved
 These bits are protected and can be written only if EN = 0.
 In direct mode, MSIZE is forced by hardware to the same value as PSIZE as soon as EN = 1.
- Bits 12:11 **PSIZE[1:0]**: Peripheral data size
 These bits are set and cleared by software.
 00: Byte (8-bit)
 01: Half-word (16-bit)
 10: Word (32-bit)
 11: Reserved
 These bits are protected and can be written only if EN = 0.
- Bit 10 **MINC**: Memory increment mode
 This bit is set and cleared by software.
 0: Memory address pointer fixed
 1: Memory address pointer incremented after each data transfer (increment is done according to MSIZE)
 This bit is protected and can be written only if EN = 0.

Bit 9 PINC: Peripheral increment mode

This bit is set and cleared by software.

0: Peripheral address pointer fixed

1: Peripheral address pointer incremented after each data transfer (increment done according to PSIZE)

This bit is protected and can be written only if EN = 0.

Bit 8 CIRC: Circular mode

This bit is set and cleared by software and can be cleared by hardware.

0: Circular mode disabled

1: Circular mode enabled

When the peripheral is the flow controller (bit PFCTRL = 1), and the stream is enabled (EN = 1), then this bit is automatically forced by hardware to 0.

It is automatically forced by hardware to 1 if the DBM bit is set, as soon as the stream is enabled (EN = 1).

Bits 7:6 DIR[1:0]: Data transfer direction

These bits are set and cleared by software.

00: Peripheral-to-memory

01: Memory-to-peripheral

10: Memory-to-memory

11: Reserved

These bits are protected and can be written only if EN = 0.

Bit 5 PFCTRL: Peripheral flow controller

This bit is set and cleared by software.

0: DMA is the flow controller.

1: The peripheral is the flow controller.

This bit is protected and can be written only if EN = 0.

When the memory-to-memory mode is selected (bits DIR[1:0] = 10), then this bit is automatically forced to 0 by hardware.

Bit 4 TCIE: Transfer complete interrupt enable

This bit is set and cleared by software.

0: TC interrupt disabled

1: TC interrupt enabled

Bit 3 HTIE: Half transfer interrupt enable

This bit is set and cleared by software.

0: HT interrupt disabled

1: HT interrupt enabled

Bit 2 TEIE: Transfer error interrupt enable

This bit is set and cleared by software.

0: TE interrupt disabled

1: TE interrupt enabled

Bit 1 DMEIE: Direct mode error interrupt enable

This bit is set and cleared by software.

0: DME interrupt disabled

1: DME interrupt enabled

Bit 0 **EN**: Stream enable/flag stream ready when read low

This bit is set and cleared by software.

0: Stream disabled

1: Stream enabled

This bit is cleared by hardware:

- on a DMA end of transfer (stream ready to be configured)
- if a transfer error occurs on the AHB master buses
- when the FIFO threshold on memory AHB port is not compatible with the burst size

When this bit is read as 0, the software is allowed to program the configuration and FIFO bits registers. It is forbidden to write these registers when the EN bit is read as 1.

Note: Before setting EN bit to 1 to start a new transfer, the event flags corresponding to the stream in DMA_LISR or DMA_HISR register must be cleared.

16.5.6 DMA stream x number of data register (DMA_SxNDTR)

Address offset: $0x014 + 0x18 * x$, ($x = 0$ to 7)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **NDT[15:0]**: Number of data items to transfer (0 up to 65535)

This bitfield can be written only when the stream is disabled. When the stream is enabled, this bitfield is read-only, indicating the remaining data items to be transmitted. This bitfield decrements after each DMA transfer.

Once the transfer is completed, this bitfield can either stay at zero (when the stream is in normal mode), or be reloaded automatically with the previously programmed value in the following cases:

- when the stream is configured in circular mode
- when the stream is enabled again by setting EN bit to 1

If the value of this bitfield is zero, no transaction can be served even if the stream is enabled.

16.5.7 DMA stream x peripheral address register (DMA_SxPAR)

Address offset: $0x018 + 0x18 * x$, ($x = 0$ to 7)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PAR[31:16]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **PAR[31:0]**: Peripheral address

Base address of the peripheral data register from/to which the data is read/written.

These bits are write-protected and can be written only when bit EN = 0 in DMA_SxCR.

16.5.8 DMA stream x memory 0 address register (DMA_SxM0AR)

Address offset: $0x01C + 0x18 * x$, ($x = 0$ to 7)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M0A[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M0A[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **M0A[31:0]**: Memory 0 address

Base address of memory area 0 from/to which the data is read/written.

These bits are write-protected. They can be written only if:

- the stream is disabled (EN = 0 in DMA_SxCR) or
- the stream is enabled (EN = 1 in DMA_SxCR) and CT = 1 in DMA_SxCR (in double-buffer mode).

16.5.9 DMA stream x memory 1 address register (DMA_SxM1AR)

Address offset: $0x020 + 0x18 * x$, ($x = 0$ to 7)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M1A[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M1A[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **M1A[31:0]**: Memory 1 address (used in case of double-buffer mode)

Base address of memory area 1 from/to which the data is read/written.

This bitfield is used only for the double-buffer mode.

These bits are write-protected. They can be written only if:

- the stream is disabled (EN = 0 in DMA_SxCR) or
- the stream is enabled (EN = 1 in DMA_SxCR) and bit CT = 0 in DMA_SxCR .

16.5.10 DMA stream x FIFO control register (DMA_SxFCR)

Address offset: $0x024 + 0x18 * x$, ($x = 0$ to 7)

Reset value: 0x0000 0021

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FEIE	Res.	FS[2:0]			DMDIS	FTH[1:0]	
								rw		r	r	r	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bit 7 **FEIE**: FIFO error interrupt enable

This bit is set and cleared by software.

0: FE interrupt disabled

1: FE interrupt enabled

Bit 6 Reserved, must be kept at reset value.

Bits 5:3 **FS[2:0]**: FIFO status

These bits are read-only.

000: $0 < \text{fifo_level} < 1/4$

001: $1/4 \leq \text{fifo_level} < 1/2$

010: $1/2 \leq \text{fifo_level} < 3/4$

011: $3/4 \leq \text{fifo_level} < \text{full}$

100: FIFO is empty.

101: FIFO is full.

Others: Reserved (no meaning)

These bits are not relevant in the direct mode (DMDIS = 0).

Bit 2 **DMDIS**: Direct mode disable

This bit is set and cleared by software. It can be set by hardware.

0: Direct mode enabled

1: Direct mode disabled

This bit is protected and can be written only if EN = 0.

This bit is set by hardware if the memory-to-memory mode is selected (DIR = 10 in DMA_SxCR), and EN = 1 in DMA_SxCR because the direct mode is not allowed in the memory-to-memory configuration.

Bits 1:0 **FTH[1:0]**: FIFO threshold selection

These bits are set and cleared by software.

00: 1/4 full FIFO

01: 1/2 full FIFO

10: 3/4 full FIFO

11: Full FIFO

These bits are not used in the direct mode when the DMIS = 0.

These bits are protected and can be written only if EN = 0.

16.5.11 DMA register map

Table 119. DMA register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	DMA_LISR	Res	Res	Res	Res	TCIF3	HTIF3	TEIF3	DMEIF3	Res	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Res	FEIF2	Res	Res	Res	Res	TCIF1	HTIF1	TEIF1	DMEIF1	Res	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Res	FEIF0
	Reset value					0	0	0	0		0	0	0	0	0		0	Res	Res	Res	Res	0	0	0	0	0	0	0	0	0	0	0	0
0x004	DMA_HISR	Res	Res	Res	Res	TCIF7	HTIF7	TEIF7	DMEIF7	Res	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Res	FEIF6	Res	Res	Res	Res	TCIF5	HTIF5	TEIF5	DMEIF5	Res	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Res	FEIF4
	Reset value					0	0	0	0		0	0	0	0	0		0	Res	Res	Res	Res	0	0	0	0	0	0	0	0	0	0	0	0
0x008	DMA_LIFCR	Res	Res	Res	Res	CTCIF3	CHTIF3	TEIF3	CDMEIF3	Res	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Res	CFEIF2	Res	Res	Res	Res	CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Res	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Res	CFEIF0
	Reset value					0	0	0	0		0	0	0	0	0		0	Res	Res	Res	Res	0	0	0	0	0	0	0	0	0	0	0	0
0x00C	DMA_HIFCR	Res	Res	Res	Res	CTCIF7	CHTIF7	CTEIF7	CDMEIF7	Res	CFEIF7	CTCIF6	CHTIF6	CTEIF6	CDMEIF6	Res	CFEIF6	Res	Res	Res	Res	CTCIF5	CHTIF5	CTEIF5	CDMEIF5	Res	CFEIF5	CTCIF4	CHTIF4	CTEIF4	CDMEIF4	Res	CFEIF4
	Reset value					0	0	0	0		0	0	0	0	0		0	Res	Res	Res	Res	0	0	0	0	0	0	0	0	0	0	0	0
0x010	DMA_S0CR	Res	Res	Res	Res	Res	Res	Res	MBURST [1:0]	Res	PBURST [1:0]	Res	TRBUFF	CT	DBM	PL[1:0]	Res	PINCOS	MSIZE[1:0]	PSIZE[1:0]	Res	CTCIF5	CHTIF5	PINC	CIRC	DIR[1:0]	PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN	
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x014	DMA_S0NDTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NDT[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x018	DMA_S0PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x01C	DMA_S0M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x020	DMA_S0M1AR	M1A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x024	DMA_S0FCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FEIE	Res	FS[2:0]		DMDIS	FTH[1:0]		
	Reset value																									0		1	0	0	0	0	1
0x028	DMA_S1CR	Res	Res	Res	Res	Res	Res	Res	MBURST[1:0]	Res	PBURST[1:0]	Res	TRBUFF	CT	DBM	PL[1:0]	Res	PINCOS	MSIZE[1:0]	PSIZE[1:0]	Res	CTCIF5	CHTIF5	PINC	CIRC	DIR[1:0]	PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN	
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x02C	DMA_S1NDTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NDT[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x030	DMA_S1PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 119. DMA register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x034	DMA_S1M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x038	DMA_S1M1AR	M1A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x03C	DMA_S1FCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FEIE	Res	FS[2:0]		DMDIS	FTH[1:0]		
	Reset value																									0		1	0	0	0	0	1
0x040	DMA_S2CR	Res	Res	Res	Res	Res	Res		MBURST[1:0]		PBURST[1:0]		TRBUFF	CT	DBM	PL[1:0]		PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCCTRL	TCIE	HTIE	TEIE	DMEIE	EN
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x044	DMA_S2NDTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NDT[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x048	DMA_S2PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04C	DMA_S2M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x050	DMA_S2M1AR	M1A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x054	DMA_S2FCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FEIE	Res	FS[2:0]		DMDIS	FTH[1:0]		
	Reset value																									0		1	0	0	0	0	1
0x058	DMA_S3CR	Res	Res	Res	Res	Res	Res		MBURST[1:0]		PBURST[1:0]		TRBUFF	CT	DBM	PL[1:0]		PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCCTRL	TCIE	HTIE	TEIE	DMEIE	EN
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x05C	DMA_S3NDTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NDT[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x060	DMA_S3PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x064	DMA_S3M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x068	DMA_S3M1AR	M1A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 119. DMA register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x06C	DMA_S3FCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FEIE	Res	FS[2:0]			DMDIS	FTH[1:0]		
	Reset value																								0		1	0	0	0	0	0	1
0x070	DMA_S4CR	Res	Res	Res	Res	Res	Res	Res	MBURST [1:0]		PBURST [1:0]		TRBUFF	CT	DBM	PL[1:0]	PINCOS		MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]	PFCCTRL	TCIE	HTIE	TEIE	DMEIE	EN	
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x074	DMA_S4NDTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NDT[15:0]																
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x078	DMA_S4PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x07C	DMA_S4M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x080	DMA_S4M1AR	M1A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x084	DMA_S4FCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FEIE	Res	FS[2:0]			DMDIS	FTH[1:0]		
	Reset value																								0		1	0	0	0	0	0	1
0x088	DMA_S5CR	Res	Res	Res	Res	Res	Res	Res	MBURST [1:0]		PBURST [1:0]		TRBUFF	CT	DBM	PL[1:0]	PINCOS		MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]	PFCCTRL	TCIE	HTIE	TEIE	DMEIE	EN	
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08C	DMA_S5NDTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NDT[15:0]																
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x090	DMA_S5PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x094	DMA_S5M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x098	DMA_S5M1AR	M1A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x09C	DMA_S5FCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FEIE	Res	FS[2:0]			DMDIS	FTH[1:0]		
	Reset value																								0		1	0	0	0	0	0	1
0x0A0	DMA_S6CR	Res	Res	Res	Res	Res	Res	Res	MBURST [1:0]		PBURST [1:0]		TRBUFF	CT	DBM	PL[1:0]	PINCOS		MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]	PFCCTRL	TCIE	HTIE	TEIE	DMEIE	EN	
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 119. DMA register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x0A4	DMA_S6NDTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NDT[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x0A8	DMA_S6PAR	PA[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x0AC	DMA_S6M0AR	M0A[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x0B0	DMA_S6M1AR	M1A[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x0B4	DMA_S6FCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FEIE	Res	FS[2:0]		DMDIS		FTH[1:0]				
	Reset value																								0		1	0	0	0	0	0	1		
0x0B8	DMA_S7CR	Res	Res	Res	Res	Res	Res	Res	MBURST [1:0]		PBURST [1:0]		TRBUFF		CT	DBM	PL[1:0]		PINCOS		MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
	Reset value								0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0BC	DMA_S7NDTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NDT[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C0	DMA_S7PAR	PA[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C4	DMA_S7M0AR	M0A[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C8	DMA_S7M1AR	M1A[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0CC	DMA_S7FCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FEIE	Res	FS[2:0]		DMDIS		FTH[1:0]				
	Reset value																								0		1	0	0	0	0	0	0	1	

Refer to [Section 2.3](#) for the register boundary addresses.

18 DMA request multiplexer (DMAMUX)

18.1 Introduction

A peripheral indicates a request for DMA transfer by setting its DMA request signal. The DMA request is pending until served by the DMA controller that generates a DMA acknowledge signal, and the corresponding DMA request signal is deasserted.

In this document, the set of control signals required for the DMA request/acknowledge protocol is not explicitly shown or described, and it is referred to as DMA request line.

The DMAMUX request multiplexer enables routing a DMA request line between the peripherals and the DMA controllers of the product. The routing function is ensured by a programmable multi-channel DMA request line multiplexer. Each channel selects a unique DMA request line, unconditionally or synchronously with events from its DMAMUX synchronization inputs. The DMAMUX may also be used as a DMA request generator from programmable events on its input trigger signals.

The number of DMAMUX instances and their main characteristics are specified in [Section 18.3.1](#).

The assignment of DMAMUX request multiplexer inputs to the DMA request lines from peripherals and to the DMAMUX request generator outputs, the assignment of DMAMUX request multiplexer outputs to DMA controller channels, and the assignment of DMAMUX synchronizations and trigger inputs to internal and external signals depend upon product implementation. They are detailed in [Section 18.3.2](#).

18.2 DMAMUX main features

- Up to 16-channel programmable DMA request line multiplexer output
- Up to 8-channel DMA request generator
- Up to 32 trigger inputs to DMA request generator
- Up to 16 synchronization inputs
- Per DMA request generator channel:
 - DMA request trigger input selector
 - DMA request counter
 - Event overrun flag for selected DMA request trigger input
- Per DMA request line multiplexer channel output:
 - Up to 107 input DMA request lines from peripherals
 - One DMA request line output
 - Synchronization input selector
 - DMA request counter
 - Event overrun flag for selected synchronization input
 - One event output, for DMA request chaining

18.3 DMAMUX implementation

18.3.1 DMAMUX1 and DMAMUX2 instantiation

The product integrates two instances of DMA request multiplexer:

- DMAMUX1 for DMA1 and DMA2 (D2 domain)
- DMAMUX2 for BDMA (D3 domain)

DMAMUX1 and DMAMUX2 are instantiated with the hardware configuration parameters listed in the following table.

Table 125. DMAMUX1 and DMAMUX2 instantiation

Feature	DMAMUX1	DMAMUX2
Number of DMAMUX output request channels	16	8
Number of DMAMUX request generator channels	8	8
Number of DMAMUX request trigger inputs	8	32
Number of DMAMUX synchronization inputs	8	16
Number of DMAMUX peripheral request inputs	107	12

18.3.2 DMAMUX1 mapping

The mapping of resources to DMAMUX1 is hardwired.

DMAMUX1 is used with DMA1 and DMA2 in D2 domain

- DMAMUX1 channels 0 to 7 are connected to DMA1 channels 0 to 7
- DMAMUX1 channels 8 to 15 are connected to DMA2 channels 0 to 7

Table 126. DMAMUX1: assignment of multiplexer inputs to resources

DMA request MUX input	Resource	DMA request MUX input	Resource	DMA request MUX input	Resource
1	dmamux1_req_gen0	44	usart2_tx_dma	87	sai1a_dma
2	dmamux1_req_gen1	45	usart3_rx_dma	88	sai1b_dma
3	dmamux1_req_gen2	46	usart3_tx_dma	89	sai2a_dma
4	dmamux1_req_gen3	47	TIM8_CH1	90	sai2b_dma
5	dmamux1_req_gen4	48	TIM8_CH2	91	swpmi_rx_dma
6	dmamux1_req_gen5	49	TIM8_CH3	92	swpmi_tx_dma
7	dmamux1_req_gen6	50	TIM8_CH4	93	spdifrx_dat_dma
8	dmamux1_req_gen7	51	TIM8_UP	94	spdifrx_ctrl_dma
9	adc1_dma	52	TIM8_TRIG	95	HR_REQ(1)
10	adc2_dma	53	TIM8_COM	96	HR_REQ(2)
11	TIM1_CH1	54	Reserved	97	HR_REQ(3)
12	TIM1_CH2	55	TIM5_CH1	98	HR_REQ(4)
13	TIM1_CH3	56	TIM5_CH2	99	HR_REQ(5)
14	TIM1_CH4	57	TIM5_CH3	100	HR_REQ(6)
15	TIM1_UP	58	TIM5_CH4	101	dfsdm1_dma0
16	TIM1_TRIG	59	TIM5_UP	102	dfsdm1_dma1
17	TIM1_COM	60	TIM5_TRIG	103	dfsdm1_dma2
18	TIM2_CH1	61	spi3_rx_dma	104	dfsdm1_dma3
19	TIM2_CH2	62	spi3_tx_dma	105	TIM15_CH1
20	TIM2_CH3	63	uart4_rx_dma	106	TIM15_UP
21	TIM2_CH4	64	uart4_tx_dma	107	TIM15_TRIG
22	TIM2_UP	65	uart5_rx_dma	108	TIM15_COM
23	TIM3_CH1	66	uart5_tx_dma	109	TIM16_CH1
24	TIM3_CH2	67	dac_ch1_dma	110	TIM16_UP
25	TIM3_CH3	68	dac_ch2_dma	111	TIM17_CH1
26	TIM3_CH4	69	TIM6_UP	112	TIM17_UP
27	TIM3_UP	70	TIM7_UP	113	sai3_a_dma
28	TIM3_TRIG	71	usart6_rx_dma	114	sai3_b_dma
29	TIM4_CH1	72	usart6_tx_dma	115	adc3_dma
30	TIM4_CH2	73	i2c3_rx_dma	116	Reserved
31	TIM4_CH3	74	i2c3_tx_dma	117	Reserved
32	TIM4_UP	75	dcmi_dma	118	Reserved
33	i2c1_rx_dma	76	crypt_in_dma	119	Reserved
34	i2c1_tx_dma	77	crypt_out_dma	120	Reserved
35	i2c2_rx_dma	78	hash_in_dma	121	Reserved
36	i2c2_tx_dma	79	uart7_rx_dma	122	Reserved

Table 126. DMAMUX1: assignment of multiplexer inputs to resources (continued)

DMA request MUX input	Resource	DMA request MUX input	Resource	DMA request MUX input	Resource
37	spi1_rx_dma	80	uart7_tx_dma	123	Reserved
38	spi1_tx_dma	81	uart8_rx_dma	124	Reserved
39	spi2_rx_dma	82	uart8_tx_dma	125	Reserved
40	spi2_tx_dma	83	spi4_rx_dma	126	Reserved
41	usart1_rx_dma	84	spi4_tx_dma	127	Reserved
42	usart1_tx_dma	85	spi5_rx_dma	-	-
43	usart2_rx_dma	86	spi5_tx_dma	-	-

Table 127. DMAMUX1: assignment of multiplexer inputs to resources

DMA request MUX input	Resource	DMA request MUX input	Resource	DMA request MUX input	Resource
1	dmamux1_req_gen0	44	usart2_tx_dma	87	sai1a_dma
2	dmamux1_req_gen1	45	usart3_rx_dma	88	sai1b_dma
3	dmamux1_req_gen2	46	usart3_tx_dma	89	sai2a_dma
4	dmamux1_req_gen3	47	TIM8_CH1	90	sai2b_dma
5	dmamux1_req_gen4	48	TIM8_CH2	91	swpmi_rx_dma
6	dmamux1_req_gen5	49	TIM8_CH3	92	swpmi_tx_dma
7	dmamux1_req_gen6	50	TIM8_CH4	93	spdifrx_dat_dma
8	dmamux1_req_gen7	51	TIM8_UP	94	spdifrx_ctrl_dma
9	adc1_dma	52	TIM8_TRIG	95	HR_REQ(1)
10	adc2_dma	53	TIM8_COM	96	HR_REQ(2)
11	TIM1_CH1	54	Reserved	97	HR_REQ(3)
12	TIM1_CH2	55	TIM5_CH1	98	HR_REQ(4)
13	TIM1_CH3	56	TIM5_CH2	99	HR_REQ(5)
14	TIM1_CH4	57	TIM5_CH3	100	HR_REQ(6)
15	TIM1_UP	58	TIM5_CH4	101	dfsdm1_dma0
16	TIM1_TRIG	59	TIM5_UP	102	dfsdm1_dma1
17	TIM1_COM	60	TIM5_TRIG	103	dfsdm1_dma2
18	TIM2_CH1	61	spi3_rx_dma	104	dfsdm1_dma3
19	TIM2_CH2	62	spi3_tx_dma	105	TIM15_CH1
20	TIM2_CH3	63	uart4_rx_dma	106	TIM15_UP
21	TIM2_CH4	64	uart4_tx_dma	107	TIM15_TRIG
22	TIM2_UP	65	uart5_rx_dma	108	TIM15_COM
23	TIM3_CH1	66	uart5_tx_dma	109	TIM16_CH1
24	TIM3_CH2	67	dac_ch1_dma	110	TIM16_UP
25	TIM3_CH3	68	dac_ch2_dma	111	TIM17_CH1

Table 127. DMAMUX1: assignment of multiplexer inputs to resources (continued)

DMA request MUX input	Resource	DMA request MUX input	Resource	DMA request MUX input	Resource
26	TIM3_CH4	69	TIM6_UP	112	TIM17_UP
27	TIM3_UP	70	TIM7_UP	113	sai3_a_dma
28	TIM3_TRIG	71	usart6_rx_dma	114	sai3_b_dma
29	TIM4_CH1	72	usart6_tx_dma	115	adc3_dma
30	TIM4_CH2	73	i2c3_rx_dma	116	Reserved
31	TIM4_CH3	74	i2c3_tx_dma	117	Reserved
32	TIM4_UP	75	dcmi_dma	118	Reserved
33	i2c1_rx_dma	76	crypt_in_dma	119	Reserved
34	i2c1_tx_dma	77	crypt_out_dma	120	Reserved
35	i2c2_rx_dma	78	hash_in_dma	121	Reserved
36	i2c2_tx_dma	79	uart7_rx_dma	122	Reserved
37	spi1_rx_dma	80	uart7_tx_dma	123	Reserved
38	spi1_tx_dma	81	uart8_rx_dma	124	Reserved
39	spi2_rx_dma	82	uart8_tx_dma	125	Reserved
40	spi2_tx_dma	83	spi4_rx_dma	126	Reserved
41	usart1_rx_dma	84	spi4_tx_dma	127	Reserved
42	usart1_tx_dma	85	spi5_rx_dma	-	-
43	usart2_rx_dma	86	spi5_tx_dma	-	-

Table 128. DMAMUX1: assignment of trigger inputs to resources

Trigger input	Resource	Trigger input	Resource
0	dmamux1_evt0	4	lptim2_out
1	dmamux1_evt1	5	lptim3_out
2	dmamux1_evt2	6	extit0
3	lptim1_out	7	TIM12_TRGO

Table 129. DMAMUX1: assignment of synchronization inputs to resources

Sync. input	Resource	Sync. input	Resource
0	dmamux1_evt0	4	lptim2_out
1	dmamux1_evt1	5	lptim3_out
2	dmamux1_evt2	6	extit0
3	lptim1_out	7	TIM12_TRGO

18.3.3 DMAMUX2 mapping

DMAMUX2 channels 0 to 7 are connected to BDMA channels 0 to 7.

Table 130. DMAMUX2: assignment of multiplexer inputs to resources

DMA request MUX input	Resource	DMA request MUX input	Resource
1	dmamux2_req_gen0	17	adc3_dma
2	dmamux2_req_gen1	18	Reserved
3	dmamux2_req_gen2	19	Reserved
4	dmamux2_req_gen3	20	Reserved
5	dmamux2_req_gen4	21	Reserved
6	dmamux2_req_gen5	22	Reserved
7	dmamux2_req_gen6	23	Reserved
8	dmamux2_req_gen7	24	Reserved
9	lpuart1_rx_dma	25	Reserved
10	lpuart1_tx_dma	26	Reserved
11	spi6_rx_dma	27	Reserved
12	spi6_tx_dma	28	Reserved
13	i2c4_rx_dma	29	Reserved
14	i2c4_tx_dma	30	Reserved
15	sai4_a_dma	31	Reserved
16	sai4_b_dma	32	Reserved

Table 131. DMAMUX2: assignment of trigger inputs to resources

Trigger input	Resource	Trigger input	Resource
0	dmamux2_evt0	16	spi6_wkup
1	dmamux2_evt1	17	Comp1_out
2	dmamux2_evt2	18	Comp2_out
3	dmamux2_evt3	19	RTC_wkup
4	dmamux2_evt4	20	Syscfg_exti0_mux
5	dmamux2_evt5	21	Syscfg_exti2_mux
6	dmamux2_evt6	22	I2c4_event_it
7	lpuart_rx_wkup	23	spi6_it
8	lpuart_tx_wkup	24	Lpuart1_it_T
9	lptim2_wkup	25	Lpuart1_it_R
10	lptim2_out	26	adc3_it
11	lptim3_wkup	27	adc3_awd1
12	lptim3_out	28	bdma_ch0_it
13	Lptim4_ait	29	bdma_ch1_it
14	Lptim5_ait	30	Reserved
15	I2c4_wkup	31	Reserved

Table 132. DMAMUX2: assignment of synchronization inputs to resources

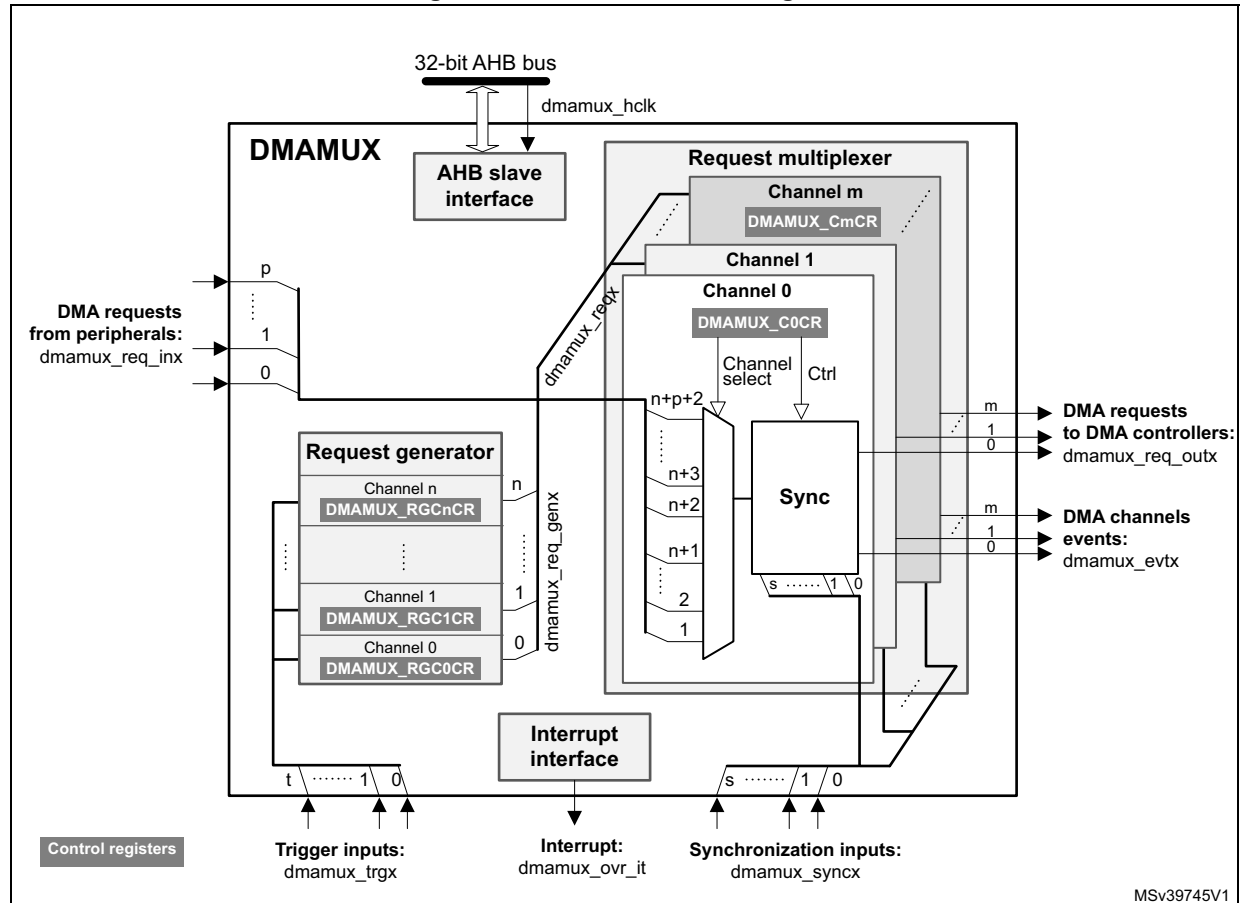
Sync input	Resource	Sync input	Resource
0	dmamux2_evt0	16	Reserved
1	dmamux2_evt1	17	Reserved
2	dmamux2_evt2	18	Reserved
3	dmamux2_evt3	19	Reserved
4	dmamux2_evt4	20	Reserved
5	dmamux2_evt5	21	Reserved
6	lpuart1_rx_wkup	22	Reserved
7	lpuart1_tx_wkup	23	Reserved
8	Lptim2_out	24	Reserved
9	Lptim3_out	25	Reserved
10	I2c4_wkup	26	Reserved
11	spi6_wkup	27	Reserved
12	Comp1_out	28	Reserved
13	RTC_wkup	29	Reserved
14	Syscfg_exti0_mux	30	Reserved
15	Syscfg_exti2_mux	31	Reserved

18.4 DMAMUX functional description

18.4.1 DMAMUX block diagram

Figure 90 shows the DMAMUX block diagram.

Figure 90. DMAMUX block diagram



DMAMUX features two main sub-blocks: the request line multiplexer and the request line generator.

The implementation assigns:

- DMAMUX request multiplexer sub-block inputs (**dmamux_reqx**) from peripherals (**dmamux_req_inx**) and from channels of the DMAMUX request generator sub-block (**dmamux_req_genx**)
- DMAMUX request outputs to channels of DMA controllers (**dmamux_req_outx**)
- Internal or external signals to DMA request trigger inputs (**dmamux_trgx**)
- Internal or external signals to synchronization inputs (**dmamux_syncx**)

18.4.2 DMAMUX signals

Table 133 lists the DMAMUX signals.

Table 133. DMAMUX signals

Signal name	Description
dmamux_hclk	DMAMUX AHB clock
dmamux_req_inx	DMAMUX DMA request line inputs from peripherals
dmamux_trgx	DMAMUX DMA request triggers inputs (to request generator sub-block)
dmamux_req_genx	DMAMUX request generator sub-block channels outputs
dmamux_reqx	DMAMUX request multiplexer sub-block inputs (from peripheral requests and request generator channels)
dmamux_syncx	DMAMUX synchronization inputs (to request multiplexer sub-block)
dmamux_req_outx	DMAMUX requests outputs (to DMA controllers)
dmamux_evtx	DMAMUX events outputs
dmamux_ovr_it	DMAMUX overrun interrupts

18.4.3 DMAMUX channels

A DMAMUX channel is a request multiplexer channel that can include, depending upon the selected input of the request multiplexer, an additional DMAMUX request generator channel.

A DMAMUX request multiplexer channel is connected and dedicated to a single channel of DMA controller(s).

Channel configuration procedure

Follow the sequence below to configure a DMAMUX x channel and the related DMA channel y:

1. Set and configure completely the DMA channel y, except enabling the channel y.
2. Set and configure completely the related DMAMUX y channel.
3. Last, activate the DMA channel y by setting the EN bit in the DMA y channel register.

18.4.4 DMAMUX request line multiplexer

The DMAMUX request multiplexer with its multiple channels ensures the actual routing of DMA request/acknowledge control signals, named DMA request lines.

Each DMA request line is connected in parallel to all the channels of the DMAMUX request line multiplexer.

A DMA request is sourced either from the peripherals, or from the DMAMUX request generator.

The DMAMUX request line multiplexer channel x selects the DMA request line number as configured by the DMAREQ_ID field in the DMAMUX_CxCR register.

Note: The null value in the field DMAREQ_ID corresponds to no DMA request line selected.

Caution: A same non-null DMAREQ_ID cannot be programmed to different x and y DMAMUX request multiplexer channels (via DMAMUX_CxCR and DMAMUX_CyCR), except when the application guarantees that the two connected DMA channels are not simultaneously active.

On top of the DMA request selection, the synchronization mode and/or the event generation may be configured and enabled, if required.

Synchronization mode and channel event generation

Each DMAMUX request line multiplexer channel x can be individually synchronized by setting the synchronization enable (SE) bit in the DMAMUX_CxCR register.

DMAMUX has multiple synchronization inputs. The synchronization inputs are connected in parallel to all the channels of the request multiplexer.

The synchronization input is selected via the SYNC_ID field in the DMAMUX_CxCR register of a given channel x.

When a channel is in this synchronization mode, the selected input DMA request line is propagated to the multiplexer channel output, once a programmable rising/falling edge is detected on the selected input synchronization signal, via the SPOL[1:0] field of the DMAMUX_CxCR register.

Additionally, internally to the DMAMUX request multiplexer, there is a programmable DMA request counter, which can be used for the channel request output generation, and for an event generation. An event generation on the channel x output is enabled through the EGE bit (event generation enable) of the DMAMUX_CxCR register.

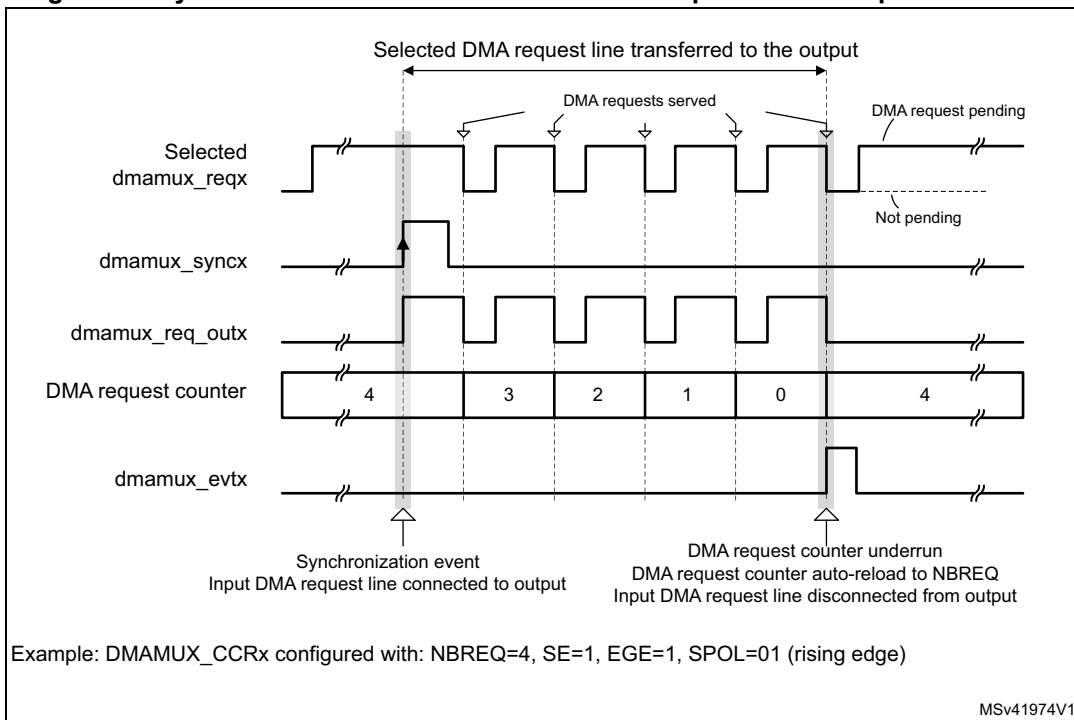
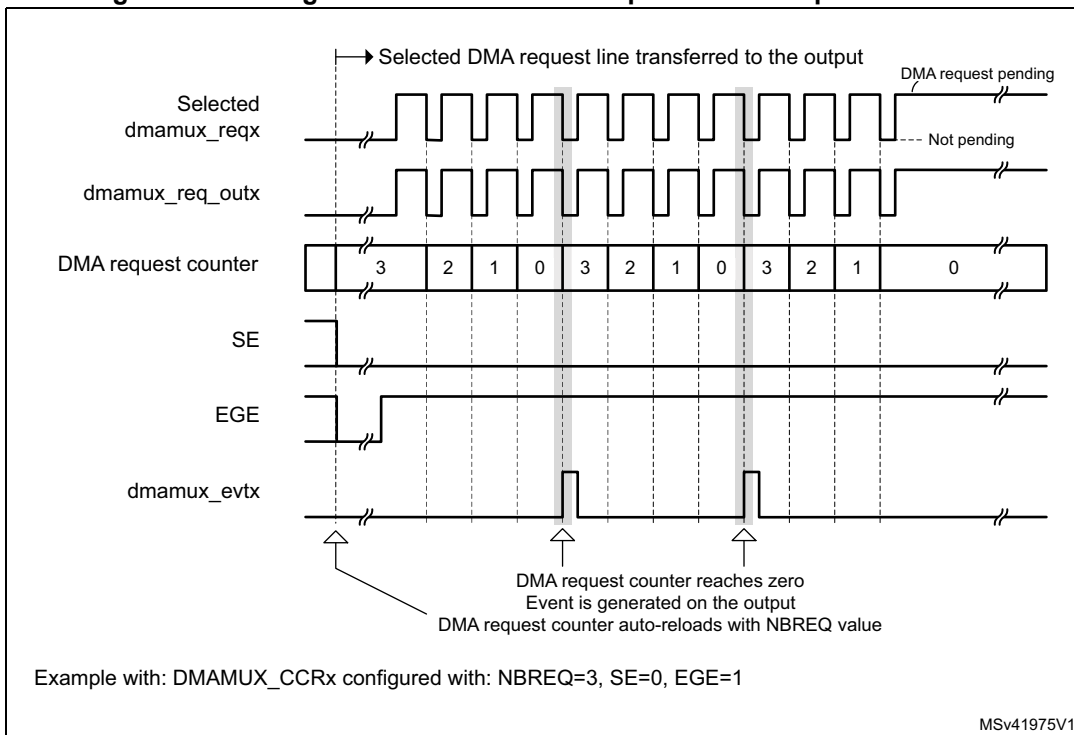
As shown in [Figure 92](#), upon the detected edge of the synchronization input, the pending selected input DMA request line is connected to the DMAMUX multiplexer channel x output.

Note: *If a synchronization event occurs while there is no pending selected input DMA request line, it is discarded. The following asserted input request lines is not connected to the DMAMUX multiplexer channel output until a synchronization event occurs again.*

From this point on, each time the connected DMAMUX request is served by the DMA controller (a served request is deasserted), the DMAMUX request counter is decremented. At its underrun, the DMA request counter is automatically loaded with the value in the NBREQ field of the DMAMUX_CxCR register and the input DMA request line is disconnected from the multiplexer channel x output.

Thus, the number of DMA requests transferred to the multiplexer channel x output following a detected synchronization event, is equal to the value in the NBREQ field, plus one.

Note: *The NBREQ field value can be written by software only when both synchronization enable bit (SE) and event generation enable bit (EGE) of the corresponding multiplexer channel x are disabled.*

Figure 91. Synchronization mode of the DMAMUX request line multiplexer channel**Figure 92. Event generation of the DMA request line multiplexer channel**

If EGE is enabled, the multiplexer channel generates a channel event, as a pulse of one AHB clock cycle, when its DMA request counter is automatically reloaded with the value of the programmed NBREQ field, as shown in [Figure 91](#) and [Figure 92](#).

Note: If EGE is enabled and NBREQ = 0, an event is generated after each served DMA request.

Note: A synchronization event (edge) is detected if the state following the edge remains stable for more than two AHB clock cycles.

Upon writing into DMAMUX_CxCR register, the synchronization events are masked during three AHB clock cycles.

Synchronization overrun and interrupt

If a new synchronization event occurs before the request counter underrun (the internal request counter programmed via the NBREQ field of the DMAMUX_CxCR register), the synchronization overrun flag bit SOFx is set in the DMAMUX_CSR register.

Note: The request multiplexer channel x synchronization must be disabled (DMAMUX_CxCR.SE = 0) when the use of the related channel of the DMA controller is completed. Else, upon a new detected synchronization event, there is a synchronization overrun due to the absence of a DMA acknowledge (that is, no served request) received from the DMA controller.

The overrun flag SOFx is reset by setting the associated clear synchronization overrun flag bit CSOFx in the DMAMUX_CFR register.

Setting the synchronization overrun flag generates an interrupt if the synchronization overrun interrupt enable bit SOIE is set in the DMAMUX_CxCR register.

18.4.5 DMAMUX request generator

The DMAMUX request generator produces DMA requests following trigger events on its DMA request trigger inputs.

The DMAMUX request generator has multiple channels. DMA request trigger inputs are connected in parallel to all channels.

The outputs of DMAMUX request generator channels are inputs to the DMAMUX request line multiplexer.

Each DMAMUX request generator channel x has an enable bit GE (generator enable) in the corresponding DMAMUX_RGxCR register.

The DMA request trigger input for the DMAMUX request generator channel x is selected through the SIG_ID (trigger signal ID) field in the corresponding DMAMUX_RGxCR register.

Trigger events on a DMA request trigger input can be rising edge, falling edge or either edge. The active edge is selected through the GPOL (generator polarity) field in the corresponding DMAMUX_RGxCR register.

Upon the trigger event, the corresponding generator channel starts generating DMA requests on its output. Each time the DMAMUX generated request is served by the connected DMA controller (a served request is deasserted), a built-in (inside the DMAMUX request generator) DMA request counter is decremented. At its underrun, the request generator channel stops generating DMA requests and the DMA request counter is automatically reloaded to its programmed value upon the next trigger event.

Thus, the number of DMA requests generated after the trigger event is GNBREQ + 1.

Note: The GNBREQ field value can be written by software only when the enable GE bit of the corresponding generator channel x is disabled.

There is no hardware write protection.

A trigger event (edge) is detected if the state following the edge remains stable for more than two AHB clock cycles.

Upon writing into DMAMUX_RGxCR register, the trigger events are masked during three AHB clock cycles.

Trigger overrun and interrupt

If a new DMA request trigger event occurs before the DMAMUX request generator counter underrun (the internal counter programmed via the GNBREQ field of the DMAMUX_RGxCR register), and if the request generator channel x was enabled via GE, then the request trigger event overrun flag bit OFx is asserted by the hardware in the DMAMUX_RGSR register.

Note: The request generator channel x must be disabled (DMAMUX_RGxCR.GE = 0) when the usage of the related channel of the DMA controller is completed. Else, upon a new detected trigger event, there is a trigger overrun due to the absence of an acknowledge (that is, no served request) received from the DMA.

The overrun flag OFx is reset by setting the associated clear overrun flag bit COFx in the DMAMUX_RGCFR register.

Setting the DMAMUX request trigger overrun flag generates an interrupt if the DMA request trigger event overrun interrupt enable bit OIE is set in the DMAMUX_RGxCR register.

18.5 DMAMUX interrupts

An interrupt can be generated upon:

- a synchronization event overrun in each DMA request line multiplexer channel
- a trigger event overrun in each DMA request generator channel

For each case, per-channel individual interrupt enable, status, and clear flag register bits are available.

Table 134. DMAMUX interrupts

Interrupt signal	Interrupt event	Event flag	Clear bit	Enable bit
dmamuxovr_it	Synchronization event overrun on channel x of the DMAMUX request line multiplexer	SOFx	CSOFx	SOIE
	Trigger event overrun on channel x of the DMAMUX request generator	OFx	COFx	OIE

18.6 DMAMUX registers

Refer to the table containing register boundary addresses for the DMAMUX1 and DMAMUX2 base address.

DMAMUX registers may be accessed per byte (8-bit), half-word (16-bit), or word (32-bit). The address must be aligned with the data size.

18.6.1 DMAMUX1 request line multiplexer channel x configuration register (DMAMUX1_CxCR)

Address offset: $0x000 + 0x04 * x$ ($x = 0$ to 15)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	SYNC_ID[2:0]			NBREQ[4:0]					SPOL[1:0]		SE
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	EGE	SOIE	Res.	DMAREQ_ID[6:0]						
						rw	rw		rw	rw	rw	rw	rw	rw	rw

Bits 31:27 Reserved, must be kept at reset value.

Bits 26:24 **SYNC_ID[2:0]**: Synchronization identification

Selects the synchronization input (see [Table 129: DMAMUX1: assignment of synchronization inputs to resources](#) and [Table 132: DMAMUX2: assignment of synchronization inputs to resources](#)).

Bits 23:19 **NBREQ[4:0]**: Number of DMA requests minus 1 to forward

Defines the number of DMA requests to forward to the DMA controller after a synchronization event, and/or the number of DMA requests before an output event is generated.

This field must only be written when both SE and EGE bits are low.

Bits 18:17 **SPOL[1:0]**: Synchronization polarity

Defines the edge polarity of the selected synchronization input:

00: No event (no synchronization, no detection).

01: Rising edge

10: Falling edge

11: Rising and falling edges

Bit 16 **SE**: Synchronization enable

0: Synchronization disabled

1: Synchronization enabled

Bits 15:10 Reserved, must be kept at reset value.

Bit 9 **EGE**: Event generation enable

0: Event generation disabled

1: Event generation enabled

Bit 8 **SOIE**: Synchronization overrun interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bit 7 Reserved, must be kept at reset value.

Bits 6:0 **DMAREQ_ID[6:0]**: DMA request identification

Selects the input DMA request. See the DMAMUX table about assignments of multiplexer inputs to resources.

18.6.2 DMAMUX2 request line multiplexer channel x configuration register (DMAMUX2_CxCR)

Address offset: $0x000 + 0x04 * x$, where $x = 0$ to 7

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	SYNC_ID[4:0]					NBREQ[4:0]					SPOL[1:0]		SE
			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	EGE	SOIE	Res.	Res.	Res.	DMAREQ_ID[4:0]				
						rw	rw				rw	rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bits 28:24 **SYNC_ID[4:0]**: Synchronization identification

Selects the synchronization input (see [Table 131: DMAMUX2: assignment of trigger inputs to resources](#))

Bits 23:19 **NBREQ[4:0]**: Number of DMA requests minus 1 to forward

Defines the number of DMA requests to forward to the DMA controller after a synchronization event, and/or the number of DMA requests before an output event is generated.

This field shall only be written when both SE and EGE bits are low.

Bits 18:17 **SPOL[1:0]**: Synchronization polarity

Defines the edge polarity of the selected synchronization input:

00: no event (no synchronization, no detection).

01: rising edge

10: falling edge

11: rising and falling edges

Bit 16 **SE**: Synchronization enable

0: synchronization disabled

1: synchronization enabled

Bits 15:10 Reserved, must be kept at reset value.

Bit 9 **EGE**: Event generation enable

0: event generation disabled

1: event generation enabled

Bit 8 **SOIE**: Synchronization overrun interrupt enable

0: interrupt disabled

1: interrupt enabled

Bits 7:5 Reserved, must be kept at reset value.

Bits 4:0 **DMAREQ_ID[4:0]**: DMA request identification

Selects the input DMA request. (see the DMAMUX table about assignments of multiplexer inputs to resources).

18.6.3 DMAMUX1 request line multiplexer interrupt channel status register (DMAMUX1_CSR)

Address offset: 0x080

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SOF15	SOF14	SOF13	SOF12	SOF11	SOF10	SOF9	SOF8	SOF7	SOF6	SOF5	SOF4	SOF3	SOF2	SOF1	SOF0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **SOF[15:0]**: Synchronization overrun event flag

The flag is set when a synchronization event occurs on a DMA request line multiplexer channel x, while the DMA request counter value is lower than NBREQ.

The flag is cleared by writing 1 to the corresponding CSOFx bit in DMAMUX_CFR register.
For DMAMUX2 bits 15:8 are reserved, keep them at reset value.

18.6.4 DMAMUX2 request line multiplexer interrupt channel status register (DMAMUX2_CSR)

Address offset: 0x080

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SOF7	SOF6	SOF5	SOF4	SOF3	SOF2	SOF1	SOF0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **SOF[7:0]**: Synchronization overrun event flag

The flag is set when a new synchronization event occurs on a DMA request line multiplexer channel x before the request counter underrun (the internal request counter programmed via the NBREQ field of the DMAMUX_CxCR register).

The flag is cleared by writing 1 to the corresponding CSOFx bit in DMAMUX2_CFR register.

18.6.5 DMAMUX1 request line multiplexer interrupt clear flag register (DMAMUX1_CFR)

Address offset: 0x084

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CSOF 15	CSOF 14	CSOF 13	CSOF 12	CSOF 11	CSOF 10	CSOF 9	CSOF 8	CSOF 7	CSOF 6	CSOF 5	CSOF 4	CSOF 3	CSOF 2	CSOF 1	CSOF 0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **CSOF[15:0]**: Clear synchronization overrun event flag

Writing 1 in each bit clears the corresponding overrun flag SOF_x in the DMAMUX_CSR register.

18.6.6 DMAMUX2 request line multiplexer interrupt clear flag register (DMAMUX2_CFR)

Address offset: 0x084

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSOF6	CSOF6	CSOF5	CSOF4	CSOF3	CSOF2	CSOF1	CSOF0
								w	w	w	w	w	w	w	w

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **CSOF[7:0]**: Clear synchronization overrun event flag

Writing 1 in each bit clears the corresponding overrun flag SOF_x in the DMAMUX2_CSR register.

18.6.7 DMAMUX1 request generator channel x configuration register (DMAMUX1_RGxCR)

Address offset: $0x100 + 0x04 * x$ ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GNBREQ[4:0]					GPOL[1:0]		GE
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OIE	Res.	Res.	Res.	Res.	Res.	SIG_ID[2:0]		
							rw						rw	rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:19 **GNBREQ[4:0]**: Number of DMA requests to be generated (minus 1)

Defines the number of DMA requests to be generated after a trigger event. The actual number of generated DMA requests is $GNBREQ + 1$.

Note: This field must be written only when GE bit is disabled.

Bits 18:17 **GPOL[1:0]**: DMA request generator trigger polarity

Defines the edge polarity of the selected trigger input

00: No event, i.e. no trigger detection nor generation.

01: Rising edge

10: Falling edge

11: Rising and falling edges

Bit 16 **GE**: DMA request generator channel x enable

0: DMA request generator channel x disabled

1: DMA request generator channel x enabled

Bits 15:9 Reserved, must be kept at reset value.

Bit 8 **OIE**: Trigger overrun interrupt enable

0: Interrupt on a trigger overrun event occurrence is disabled

1: Interrupt on a trigger overrun event occurrence is enabled

Bits 7:3 Reserved, must be kept at reset value.

Bits 2:0 **SIG_ID[2:0]**: Signal identification

Selects the DMA request trigger input used for the channel x of the DMA request generator

18.6.8 DMAMUX2 request generator channel x configuration register (DMAMUX2_RGxCR)

Address offset: $0x100 + 0x04 * x$ ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GNBREQ[4:0]				GPOL[1:0]		GE	
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OIE	Res.	Res.	Res.	SIG_ID[4:0]				
							rw				rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:19 **GNBREQ[4:0]**: Number of DMA requests to be generated (minus 1)

Defines the number of DMA requests to be generated after a trigger event. The actual number of generated DMA requests is GNBREQ+1.

Note: This field shall only be written when GE bit is disabled.

Bits 18:17 **GPOL[1:0]**: DMA request generator trigger polarity

Defines the edge polarity of the selected trigger input

00: no event. I.e. none trigger detection nor generation.

01: rising edge

10: falling edge

11: rising and falling edge

Bit 16 **GE**: DMA request generator channel x enable

0: DMA request generator channel x disabled

1: DMA request generator channel x enabled

Bits 15:9 Reserved, must be kept at reset value.

Bit 8 **OIE**: Trigger overrun interrupt enable

0: interrupt on a trigger overrun event occurrence is disabled

1: interrupt on a trigger overrun event occurrence is enabled

Bits 7:5 Reserved, must be kept at reset value.

Bits 4:0 **SIG_ID[4:0]**: Signal identification

Selects the DMA request trigger input used for the channel x of the DMA request generator

18.6.9 DMAMUX1 request generator interrupt status register (DMAMUX1_RGSR)

Address offset: 0x140

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OF7	OF6	OF5	OF4	OF3	OF2	OF1	OF0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **OF[7:0]**: Trigger overrun event flag

The flag is set when a new trigger event occurs on DMA request generator channel x, before the request counter underrun (the internal request counter programmed via the GNBREQ field of the DMAMUX_RGxCR register).

The flag is cleared by writing 1 to the corresponding COFx bit in the DMAMUX_RGCFR register.

18.6.10 DMAMUX2 request generator interrupt status register (DMAMUX2_RGSR)

Address offset: 0x140

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OF7	OF6	OF5	OF4	OF3	OF2	OF1	OF0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **OF[7:0]**: Trigger overrun event flag

The flag is set when a new trigger event occurs on DMA request generator channel x.

The flag is cleared by writing 1 to the corresponding COFx bit in the DMAMUX2_RGCFR register.

18.6.11 DMAMUX1 request generator interrupt clear flag register (DMAMUX1_RGCFR)

Address offset: 0x144

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	COF7	COF6	COF5	COF4	COF3	COF2	COF1	COF0
								w	w	w	w	w	w	w	w

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **COF[7:0]**: Clear trigger overrun event flag

Writing 1 in each bit clears the corresponding overrun flag OFx in the DMAMUX_RGSR register.

18.6.12 DMAMUX2 request generator interrupt clear flag register (DMAMUX2_RGCFR)

Address offset: 0x144

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	COF7	COF6	COF5	COF4	COF3	COF2	COF1	COF0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **COF[7:0]**: Clear trigger overrun event flag
Writing 1 in each bit clears the corresponding overrun flag OFx in the DMAMUX2_RGSR register.

18.6.13 DMAMUX register map

The following table summarizes the DMAMUX registers and reset values. Refer to the register boundary address table for the DMAMUX register base address.

Table 135. DMAMUX register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	DMAMUX_C0CR ⁽¹⁾⁽²⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x004	DMAMUX_C1CR ⁽¹⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x008	DMAMUX_C2CR ⁽¹⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x00C	DMAMUX_C3CR ⁽¹⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x010	DMAMUX_C4CR ⁽¹⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x014	DMAMUX_C5CR ⁽¹⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x018	DMAMUX_C6CR ⁽¹⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x01C	DMAMUX_C7CR ⁽¹⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x020	DMAMUX_C8CR ⁽³⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x024	DMAMUX_C9CR ⁽⁴⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x028	DMAMUX_C10CR ⁽³⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x02C	DMAMUX_C11CR ⁽³⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x030	DMAMUX_C12CR ⁽³⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x034	DMAMUX_C13CR ⁽³⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x038	DMAMUX_C14CR ⁽³⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x03C	DMAMUX_C15CR ⁽³⁾	Res	Res	Res	Res	Res	SYNC_ID [2:0]			NBREQ[4:0]					SPOL [1:0]	SE		Res	Res	Res	Res	Res	Res	Res	EGE	SOIE	Res	DMAREQ_ID[6:0]					
	Reset value						0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0
0x040 - 0x07C	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res

Table 135. DMAMUX register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x080	DMAMUX_CSR ⁽⁴⁾	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x084	DMAMUX_CFR ⁽⁴⁾	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x088 - 0x0FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
0x100	DMAMUX_RG0CR ⁽⁵⁾	Res	Res	Res	Res	Res	Res	Res	Res	GNBREQ[4:0]					GPOL	[1:0]	GE	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SIG_ID	[2:0]
	Reset value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x104	DMAMUX_RG1CR	Res	Res	Res	Res	Res	Res	Res	Res	GNBREQ[4:0]					GPOL	[1:0]	GE	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SIG_ID	[2:0]
	Reset value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x108	DMAMUX_RG2CR	Res	Res	Res	Res	Res	Res	Res	Res	GNBREQ[4:0]					GPOL	[1:0]	GE	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SIG_ID	[2:0]
	Reset value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10C	DMAMUX_RG3CR	Res	Res	Res	Res	Res	Res	Res	Res	GNBREQ[4:0]					GPOL	[1:0]	GE	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SIG_ID	[2:0]
	Reset value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x140	DMAMUX_RGSR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x144	DMAMUX_RGCFR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x140	DMAMUX_RGSR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x144	DMAMUX_RGCFR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x148 - 0x3FC	Reserved	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res

1. For DMAMUX2 bits 6:5 are reserved.
2. For DMAMUX2 bits 28:24 correspond to SYNC_ID[4:0].
3. Only applies to DMAMUX1. For DMAMUX2 the word is reserved.
4. For DMAMUX2 bits 15:8 are reserved.
5. Valid for DMAMUX1. For DMAMUX2 bits 4:0 are for SIG_ID[4:0].

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

20 Nested vectored interrupt controllers (NVIC1 and NVIC2)

20.1 NVIC features

Both CPU1 (Cortex[®]-M7) and CPU2 (Cortex[®]-M4) cores have their own nested vector interrupt controller (respectively NVIC1 and NVIC2). The NVIC includes the following features:

- up to 150 maskable interrupt channels for STM32H7xxx (not including the 16 interrupt lines of Cortex[®]-M7 / Cortex[®]-M4 with FPU)
- 16 programmable priority levels (4 bits of interrupt priority are used)
- low-latency exception and interrupt handling
- power management control
- implementation of system control registers

The NVIC and the processor core interface are closely coupled, which enables low latency interrupt processing and efficient processing of late arriving interrupts.

All interrupts, including the core exceptions, are managed by the NVIC.

For more information on exceptions and NVIC programming, refer to PM0253 and PM0214 programming manuals for Cortex[®]-M7 and for Cortex[®]-M4, respectively.

20.1.1 SysTick calibration value register

The SysTick calibration value (SYST_CALIB) is fixed to 0x3E8. It provides a reference timebase of 1 ms based when the SysTick clock frequency is 1 MHz. To match the 1 ms timebase whatever the application frequency, the SysTick reload value must be programmed as follows in the SYST_RVR register:

- The SysTick clock source is the 100 MHz CPU clock (HCLK):

$$\text{reload value} = (F_{\text{HCLK}} \times \text{SYST_CALIB}) - 1$$

- or the SysTick clock source is an external clock:

$$\text{reload value} = ((F_{\text{HCLK}} / 8) \times \text{SYST_CALIB}) - 1$$

where F_{HCLK} refers to the AHB frequency expressed in MHz.

For example, to achieve a timebase of 1 ms when the SysTick clock source is the 100 MHz HCLK:

$$\text{reload value} = (100 \times \text{SYST_CALIB}) - 1 = 0x1869F$$

20.1.2 Interrupt and exception vectors

Each CPU has its own exceptions connected to its Nested vectored interrupt controller (NVIC): reset, NMI, HardFault, MemManage, Bus Fault, UsageFault, SVCall, DebugMonitor, PendSV, SysTick.

Each CPU also has its own Floating Point Unit (FPU) interrupt connected to its Nested Vectored Interrupt Controller (NVIC), in position 81.

The Window Watchdog 1 (WWDG1) interrupt is connected to the CPU1 NVIC (NVIC1) position 0, while Window Watchdog 2 (WWDG2) interrupt is connected to the CPU2 NVIC (NVIC2) position 0.

The Window Watchdog 1 (WWDG1) reset output signal is connected to the CPU2 NVIC (NVIC2) position 143 via the Extended Interrupt and Event Controller (EXTI). Similarly, the Window Watchdog 2 (WWDG2) reset output signal is connected to the CPU1 NVIC (NVIC1) position 143 via the Extended Interrupt and Event Controller (EXTI).

Refer to [Section 47: System window watchdog \(WWDG\)](#) and [Section 48: Independent watchdog \(IWDG\)](#) for the description of watchdog interrupts and reset.

Hardware Semaphore (HSEM) interrupt line 0 is connected to the CPU1 NVIC (NVIC1) position 125, while Hardware Semaphore (HSEM) interrupt line 1 is connected to the CPU2 NVIC (NVIC2) position 126.

The CPU2 hold interrupt is connected to the CPU1 NVIC (NVIC1) position 148. Similarly, the CPU1 hold interrupt is connected to the CPU2 NVIC (NVIC2) position 148. Refer to [Power control \(PWR\)](#) for the description of the CPU hold mechanism.

All other interrupt lines coming from peripherals are connected in a symmetrical way on both NVIC1 and NVIC2.

Table 149. NVIC1 (CPU1) and NVIC2 (CPU2)⁽¹⁾

Signal	Priority	NVIC1 position	Acronym	NVIC2 position	Description	Address offset
-	-	-	-	-	Reserved	0x0000 0000
-	-3	-	Reset	-	Reset	0x0000 0004
-	-2	-	NMI	-	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	-	HardFault	-	All classes of fault	0x0000 000C
-	0	-	MemManage	-	Memory management	0x0000 0010
-	1	-	BusFault	-	Prefetch fault, memory access fault	0x0000 0014
-	2	-	UsageFault	-	Undefined instruction or illegal state	0x0000 0018
-	-	-	-	-	Reserved	0x0000 001C-0x0000 002B
-	3	-	SVCall	-	System service call via SWI instruction	0x0000 002C
-	4	-	DebugMonitor	-	Debug monitor	0x0000 0030
-	-	-	-	-	Reserved	0x0000 0034
-	5	-	PendSV	-	Pendable request for system service	0x0000 0038
-	6	-	Systick	-	System tick timer	0x0000 003C

Table 149. NVIC1 (CPU1) and NVIC2 (CPU2)⁽¹⁾ (continued)

Signal	Priority	NVIC1 position	Acronym	NVIC2 position	Description	Address offset
wwdg1_it	7	0	WWDG1	-	Window Watchdog interrupt	0x0000 0040
wwdg2_it		-	WWDG2	0		
exti_pwr_pvd_wkup	8	1	PVD_PVM	1	PVD through EXTI line detection interrupt	0x0000 0044
exti_tamp_rtc_wkup	9	2	RTC_TAMP_STAMP_CSS_LSE	2	RTC tamper, timestamp	0x0000 0048
lsecss_rcc_it					CSS LSE	
exti_wkup_rtc_wkup	10	3	RTC_WKUP	3	RTC Wakeup interrupt through the EXTI line	0x0000 004C
flash_it	11	4	FLASH	4	Flash memory global interrupt	0x0000 0050
rcc_it	12	5	RCC	5	RCC global interrupt	0x0000 0054
exti_exti0_wkup	13	6	EXTI0	6	EXTI Line 0 interrupt	0x0000 0058
exti_exti1_wkup	14	7	EXTI1	7	EXTI Line 1 interrupt	0x0000 005C
exti_exti2_wkup	15	8	EXTI2	8	EXTI Line 2 interrupt	0x0000 0060
exti_exti3_wkup	16	9	EXTI3	9	EXTI Line 3 interrupt	0x0000 0064
exti_exti4_wkup	17	10	EXTI4	10	EXTI Line 4 interrupt	0x0000 0068
dma1_it0	18	11	DMA_STR0	11	DMA1 Stream0 global interrupt	0x0000 006C
dma1_it1	19	12	DMA_STR1	12	DMA1 Stream1 global interrupt	0x0000 0070
dma1_it2	20	13	DMA_STR2	13	DMA1 Stream2 global interrupt	0x0000 0074
dma1_it3	21	14	DMA_STR3	14	DMA1 Stream3 global interrupt	0x0000 0078
dma1_it4	22	15	DMA_STR4	15	DMA1 Stream4 global interrupt	0x0000 007C
dma1_it5	23	16	DMA_STR5	16	DMA1 Stream5 global interrupt	0x0000 0080
dma1_it6	24	17	DMA_STR6	17	DMA1 Stream6 global interrupt	0x0000 0084
adc1_it	25	18	ADC1_2	18	ADC1 and ADC2 global interrupt	0x0000 0088
adc2_it						
ttfdcan_intr0_it	26	19	FDCAN1_IT0	19	FDCAN1 Interrupt 0	0x0000 008C
fdcan_intr0_it	27	20	FDCAN2_IT0	20	FDCAN2 Interrupt 0	0x0000 0090
ttfdcan_intr1_it	28	21	FDCAN1_IT1	21	FDCAN1 Interrupt 1	0x0000 0094
fdcan_intr1_it	29	22	FDCAN2_IT1	22	FDCAN2 Interrupt 1	0x0000 0098

Table 149. NVIC1 (CPU1) and NVIC2 (CPU2)⁽¹⁾ (continued)

Signal	Priority	NVIC1 position	Acronym	NVIC2 position	Description	Address offset
exti_exti5_wkup	30	23	EXTI9_5	23	EXTI Line[9:5] interrupts	0x 0000 009C
exti_exti6_wkup						
exti_exti7_wkup						
exti_exti8_wkup						
exti_exti9_wkup						
tim1_brk_it	31	24	TIM1_BRK	24	TIM1 break interrupt	0x0000 00A0
tim1_upd_it	32	25	TIM1_UP	25	TIM1 update interrupt	0x0000 00A4
tim1_trg_it	33	26	TIM1_TRG_COM	26	TIM1 trigger and commutation interrupts	0x0000 00A8
tim1_cc_it	34	27	TIM_CC	27	TIM1 capture / compare interrupt	0x0000 00AC
tim2_it	35	28	TIM2	28	TIM2 global interrupt	0x0000 00B0
tim3_it	36	29	TIM3	29	TIM3 global interrupt	0x0000 00B4
tim4_it	37	30	TIM4	30	TIM4 global interrupt	0x0000 00B8
i2c1_ev_it	38	31	I2C1_EV	31	I2C1 event interrupt	0x0000 00BC
exti_i2c1_ev_wkup						
i2c1_err_it	39	32	I2C1_ER	32	I2C1 error interrupt	0x0000 00C0
i2c2_ev_it	40	33	I2C2_EV	33	I2C2 event interrupt	0x0000 00C4
exti_i2c2_ev_wkup						
i2c2_err_it	41	34	I2C2_ER	34	I2C2 error interrupt	0x0000 00C8
spi1_it	42	35	SPI1	35	SPI1 global interrupt	0x0000 00CC
exti_spi1_it						
spi2_it	43	36	SPI2	36	SPI2 global interrupt	0x0000 00D0
exti_spi2_it						
usart1_gbl_it	44	37	USART1	37	USART1 global interrupt	0x0000 00D4
exti_usart1_wkup						
usart2_gbl_it	45	38	USART2	38	USART2 global interrupt	0x0000 00D8
exti_usart2_wkup						
usart3_gbl_it	46	39	USART3	39	USART3 global interrupt	0x0000 00DC
exti_usart3_wkup						

Table 149. NVIC1 (CPU1) and NVIC2 (CPU2)⁽¹⁾ (continued)

Signal	Priority	NVIC1 position	Acronym	NVIC2 position	Description	Address offset
exti_exti10_it	47	40	EXTI15_10	40	EXTI Line[15:10] interrupts	0x0000 00E0
exti_exti11_wkup						
exti_exti12_wkup						
exti_exti13_wkup						
exti_exti14_wkup						
exti_exti15_wkup						
exti_rtc_al	48	41	RTC_ALARM	41	RTC alarms (A and B) through EXTI Line interrupts	0x0000 00E4
-	49	42	-	42	-	0x0000 00E8
tim8_brk_it	50	43	TIM8_BRK_TIM12	43	TIM8 break and TIM12 global interrupts	0x0000 00EC
tim12_gbl_it						
tim8_upd_it	51	44	TIM8_UP_TIM13	44	TIM8 update and TIM13 global interrupts	0x0000 00F0
tim13_gbl_it						
tim8_trg_it	52	45	TIM8_TRG_COM_TIM14	45	TIM8 trigger /commutation and TIM14 global interrupts	0x0000 00F4
tim14_gbl_it						
tim8_cc_it	53	46	TIM8_CC	46	TIM8 capture / compare interrupts	0x0000 00F8
dma1_it7	54	47	DMA1_STR7	47	DMA1 Stream7 global interrupt	0x0000 00FC
fmc_gbl_it	55	48	FMC	48	FMC global interrupt	0x0000 0100
sdmmc1_it	56	49	SDMMC1	49	SDMMC global interrupt	0x0000 0104
tim5_gbl_it	57	50	TIM5	50	TIM5 global interrupt	0x0000 0108
spi3_it	58	51	SPI3	51	SPI3 global interrupt	0x0000 010C
exti_spi3_wkup						
uart4_gbl_it	59	52	UART4	52	UART4 global interrupt	0x0000 0110
exti_uart4_wkup						
uart5_gbl_it	60	53	UART5	53	UART5 global interrupt	0x0000 0114
exti_uart5_wkup						
tim6_gbl_it	61	54	TIM6_DAC	54	TIM6 global interrupt	0x0000 0118
dac_unr_it					DAC underrun error interrupt	
tim7_gbl_it	62	55	TIM7	55	TIM7 global interrupt	0x0000 011C
dma2_it0	63	56	DMA2_STR0	56	DMA2 Stream0 interrupt	0x0000 0120
dma2_it1	64	57	DMA2_STR1	57	DMA2 Stream1 interrupt	0x0000 0124
dma2_it2	65	58	DMA2_STR2	58	DMA2 Stream2 interrupt	0x0000 0128

Table 149. NVIC1 (CPU1) and NVIC2 (CPU2)⁽¹⁾ (continued)

Signal	Priority	NVIC1 position	Acronym	NVIC2 position	Description	Address offset
dma2_it3	66	59	DMA2_STR3	59	DMA2 Stream3 interrupt	0x0000 012C
dma2_it4	67	60	DMA2_STR4	60	DMA2 Stream4 interrupt	0x0000 0130
eth_sbd_intr_it	68	61	ETH	61	Ethernet global interrupt	0x0000 0134
exti_eth_wkup	69	62	ETH_WKUP	62	Ethernet wakeup through EXTI line interrupt	0x0000 0138
fdcan_cal_it	70	63	FDCAN_CAL	63	FDCAN calibration interrupts	0x0000 013C
cm7_sev_it	71	-	-	64	Arm® Cortex®-M7 Send even interrupt	0x0000 0140
cm4_sev_it	72	65	-	-	Arm® Cortex®-M4 Send even interrupt	0x0000 0144
NC	73	66	-	66	-	0x0000 0148
NC	74	67	-	67	-	0x0000 014C
dma2_it5	75	68	DMA2_STR5	68	DMA2 Stream5 interrupt	0x0000 0150
dma2_it6	76	69	DMA2_STR6	69	DMA2 Stream6 interrupt	0x0000 0154
dma2_it7	77	70	DMA2_STR7	70	DMA2 Stream7 interrupt	0x0000 0158
usart6_gbl_it	78	71	USART6	71	USART6 global interrupt	0x0000 015C
exti_usart6_wkup					USART6 wakeup interrupt	
i2c3_ev_it	79	72	I2C3_EV	72	I2C3 event interrupt	0x0000 0160
exti_i2c3_ev_wkup						
i2c3_err_it	80	73	I2C3_ER	73	I2C3 error interrupt	0x0000 0164
usb1_out_it	81	74	OTG_HS_EP1_OUT	74	OTG_HS out global interrupt	0x0000 0168
usb1_in_it	82	75	OTG_HS_EP1_IN	75	OTG_HS in global interrupt	0x0000 016C
exti_usb1_wkup	83	76	OTG_HS_WKUP	76	OTG_HS wakeup interrupt	0x0000 0170
usb1_gbl_it	84	77	OTG_HS	77	OTG_HS global interrupt	0x0000 0174
dcmi_it	85	78	DCMI	78	DCMI global interrupt	0x0000 0178
cryp_it	86	79	CRYP	79	CRYP global interrupt	0x0000 017C
hash_rng_it	87	80	HASH_RNG	80	HASH and RNG global interrupt	0x0000 0180
cpu1_fpu_it	88	81	FPU	-	CPU1 FPU	0x0000 0184
cpu2_fpu_it		-	FPU	81	CPU2 FPU	
uart7_gbl_it	89	82	UART7	82	UART7 global interrupt	0x0000 0188
exti_uart7_wkup						
uart8_gbl_it	90	83	UART8	83	UART8 global interrupt	0x0000 018C
exti_uart8_wkup						

Table 149. NVIC1 (CPU1) and NVIC2 (CPU2)⁽¹⁾ (continued)

Signal	Priority	NVIC1 position	Acronym	NVIC2 position	Description	Address offset
spi4_it	91	84	SPI4	84	SPI4 global interrupt	0x0000 0190
exti_spi4_wkup						
spi5_it	92	85	SPI5	85	SPI5 global interrupt	0x0000 0194
exti_spi5_wkup						
spi6_it	93	86	SPI6	86	SPI6 global interrupt	0x0000 0198
exti_spi6_wkup						
sai1_glb_it	94	87	SAI1	87	SAI1 global interrupt	0x0000 019C
ltdc_it	95	88	LTDC	88	LCD-TFT global interrupt	0x0000 01A0
ltdc_err_it	96	89	LTDC_ER	89	LCD-TFT error interrupt	0x0000 01A4
dma2d_gbl_it	97	90	DMA2D	90	DMA2D global interrupt	0x0000 01A8
sai2_gbl_it	98	91	SAI2	91	SAI2 global interrupt	0x0000 01AC
quadspi_it	99	92	QUADSPI	92	QuadSPI global interrupt	0x0000 01B0
lptim1_it	100	93	LPTIM1	93	LPTIM1 global interrupt	0x0000 01B4
exti_lptim_wkup						
cec_it	101	94	CEC	94	HDMI-CEC global interrupt	0x0000 01B8
exti_cec_it						
i2c4_ev_it	102	95	I2C4_EV	95	I2C4 event interrupt	0x0000 01BC
exti_i2c4_ev_it						
i2c4_err_it	103	96	I2C4_ER	96	I2C4 error interrupt	0x0000 01C0
spdifrx_it	104	97	SPDIF	97	SPDIFRX global interrupt	0x0000 01C4
usb2_out_it	105	98	OTG_FS_EP1_OUT	98	OTG_FS out global interrupt	0x0000 01C8
usb2_in_it	106	99	OTG_FS_EP1_IN	99	OTG_FS in global interrupt	0x0000 01CC
exti_usb2_wkup	107	100	OTG_FS_WKUP	100	OTG_FS wakeup	0x0000 01D0
usb2_gbl_it	108	101	OTG_FS	101	OTG_FS global interrupt	0x0000 01D4
dmamux1_ovr_it	109	102	DMAMUX1_OV	102	DMAMUX1 overrun interrupt	0x0000 01D8
hrtim1_mst_it	110	103	HRTIM1_MST	103	HRTIM1 master timer interrupt	0x0000 01DC
hrtim1_tima_it	111	104	HRTIM1_TIMA	104	HRTIM1 timer A interrupt	0x0000 01E0
hrtim1_timb_it	112	105	HRTIM1_TIMB	105	HRTIM1 timer B interrupt	0x0000 01E4
hrtim1_timc_it	113	106	HRTIM1_TIMC	106	HRTIM1 timer C interrupt	0x0000 01E8
hrtim1_timd_it	114	107	HRTIM1_TIMD	107	HRTIM1 timer D interrupt	0x0000 01EC
hrtim1_time_it	115	108	HRTIM1_TIME	108	HRTIM1 timer E interrupt	0x0000 01F0
hrtim1_fault_it	116	109	HRTIM1_FLT	109	HRTIM1 fault interrupt	0x0000 01F4
dfsdm1_it0	117	110	DFSDM1_FLT0	110	DFSDM1 filter 0 interrupt	0x0000 01F8

Table 149. NVIC1 (CPU1) and NVIC2 (CPU2)⁽¹⁾ (continued)

Signal	Priority	NVIC1 position	Acronym	NVIC2 position	Description	Address offset
dfsdm1_it1	118	111	DFSDM1_FLT1	111	DFSDM1 filter 1 interrupt	0x0000 01FC
dfsdm1_it2	119	112	DFSDM1_FLT2	112	DFSDM1 filter 2 interrupt	0x0000 0200
dfsdm1_it3	120	113	DFSDM1_FLT3	113	DFSDM1 filter 3 interrupt	0x0000 0204
sai3_gbl_it	121	114	SAI3	114	SAI3 global interrupt	0x0000 0208
swpmi_gbl_it	122	115	SWPMI1	115	SWPMI global interrupt	0x0000 020C
exti_swpmi_wkup					SWPMI wakeup	
tim15_gbl_it	123	116	TIM15	116	TIM15 global interrupt	0x0000 0210
tim16_gbl_it	124	117	TIM16	117	TIM16 global interrupt	0x0000 0214
tim17_gbl_it	125	118	TIM17	118	TIM17 global interrupt	0x0000 0218
exti_mdios_wkup	126	119	MDIOS_WKUP	119	MDIOS wakeup	0x0000 021C
mdios_it	127	120	MDIOS	120	MDIOS global interrupt	0x0000 0220
jpeg_it	128	121	JPEG	121	JPEG global interrupt	0x0000 0224
mdma_it	129	122	MDMA	122	MDMA	0x0000 0228
dsi_it	130	123	DSI	123	DSI Host global interrupt	0x0000 022C
exti_dsi_wkup			DSI_WKUP		DSI Host wakeup interrupt	
sdmmc2_it	131	124	SDMMC2	124	SDMMC global interrupt	0x0000 0230
hsem1_it	132	125	HSEM0	-	HSEM global interrupt 1	0x0000 0234
hsem2_it	133	-	HSEM1	126	HSEM global interrupt 2	0x0000 0238
adc3_it	134	127	ADC3	127	ADC3 global interrupt	0x0000 023C
dmamux2_ovr_it	135	128	DMAMUX2_OVR	128	DMAMUX2 overrun interrupt	0x0000 0240
bdma_ch0_it	136	129	BDMA_CH0	129	BDMA channel 0 interrupt	0x0000 0244
bdma_ch1_it	137	130	BDMA_CH1	130	BDMA channel 1 interrupt	0x0000 0248
bdma_ch2_it	138	131	BDMA_CH2	131	BDMA channel 2 interrupt	0x0000 024C
bdma_ch3_it	139	132	BDMA_CH3	132	BDMA channel 3 interrupt	0x0000 0250
bdma_ch4_it	140	133	BDMA_CH4	133	BDMA channel 4 interrupt	0x0000 0254
bdma_ch5_it	141	134	BDMA_CH5	134	BDMA channel 5 interrupt	0x0000 0258
bdma_ch6_it	142	135	BDMA_CH6	135	BDMA channel 6 interrupt	0x0000 025C
bdma_ch7_it	143	136	BDMA_CH7	136	BDMA channel 7 interrupt	0x0000 0260
comp_gbl_it	144	137	COMP	137	COMP1 and COMP2 global interrupt	0x0000 0264
exti_comp1_wkup						
exti_comp2_wkup						
lptim2_it	145	138	LPTIM2	138	LPTIM2 timer interrupt	0x0000 0268
exti_lptim2_wkup						

Table 149. NVIC1 (CPU1) and NVIC2 (CPU2)⁽¹⁾ (continued)

Signal	Priority	NVIC1 position	Acronym	NVIC2 position	Description	Address offset
lptim3_it	146	139	LPTIM3	139	LPTIM2 timer interrupt	0x0000 026C
exti_lptim3_wkup						
lptim4_it	147	140	LPTIM4	140	LPTIM2 timer interrupt	0x0000 0270
exti_lptim4_wkup						
lptim5_it	148	141	LPTIM5	141	LPTIM2 timer interrupt	0x0000 0274
exti_lptim5_wkup						
lpuart_gbl_it	149	142	LPUART	142	LPUART global interrupt	0x0000 0278
exti_lpuart_rx_it						
exti_lpuart_tx_it						
exti_d2_wwdg_wkup	150	143	WWDG2_RST	-	Window Watchdog interrupt	0x0000 027C
exti_d1_wwdg_wkup		-	WWDG1_RST	143	Window Watchdog interrupt	
crs_it	151	144	CRS	144	Clock Recovery System global interrupt	0x0000 0280
ramecc1_it	152	145	ECC	145	ECC diagnostic global interrupt for RAM D1	0x0000 0284
ramecc2_it					ECC diagnostic global interrupt for RAM D2	
ramecc3_it					ECC diagnostic global interrupt for RAM D3	
sai4_glb_it	153	146	SAI4	146	SAI4 global interrupt	0x0000 0288
cpu1hold_it	155	148	HOLD_CORE (CPU1_HOLD)	-	CPU1 hold	0x0000 0290
cpu2hold_it		-	HOLD_CORE (CPU2_HOLD)	148	CPU2 hold	
exti_wkup1_wkup	156	149	WKUP	149	WKUP0 to WKUP5 pins	0x0000 0294
exti_wkup2_wkup						
exti_wkup3_wkup						
exti_wkup4_wkup						
exti_wkup5_wkup						
exti_wkup6_wkup						

1. When different signals are connected to the same NVIC interrupt line, they are OR-ed.

21 Extended interrupt and event controller (EXTI)

The Extended Interrupt and event controller (EXTI) manages wakeup through configurable and direct event inputs. It provides wakeup requests to the Power Control, and generates interrupt requests to the CPU NVIC and to the D3 domain DMAMUX2, and events to the CPU event input.

The EXTI wakeup requests allow the system to be woken up from Stop mode, and the CPU to be woken up from CStop mode.

Both the interrupt request and event request generation can also be used in Run modes.

21.1 EXTI main features

The EXTI main features are the following:

- All Event inputs allow a CPU to wakeup and to generate a CPU interrupt and/or CPU event
- Some Event inputs allow the user to wakeup the D3 domain for autonomous Run mode and generate an interrupt to the D3 domain, i.e. the DMAMUX2

The asynchronous event inputs are classified in 2 groups:

- Configurable events (signals from I/Os or peripherals able to generate a pulse), they have the following features:
 - Selectable active trigger edge
 - Interrupt pending status register bit
 - Individual Interrupt and Event generation mask
 - SW trigger possibility
 - Configurable System D3 domain wakeup events have a D3 Pending mask and status register and may have a D3 interrupt signal.
- Direct events (interrupt and wakeup sources from other peripherals, requiring to be cleared in the peripheral), they feature
 - Fixed rising edge active trigger
 - No interrupt pending status register bit in the EXTI (the interrupt pending status is provided by the peripheral generating the event)
 - Individual Interrupt and Event generation mask
 - No SW trigger possibility
 - Direct system D3 domain wakeup events have a D3 Pending mask and status register and may have a D3 interrupt signal

21.2 EXTI block diagram

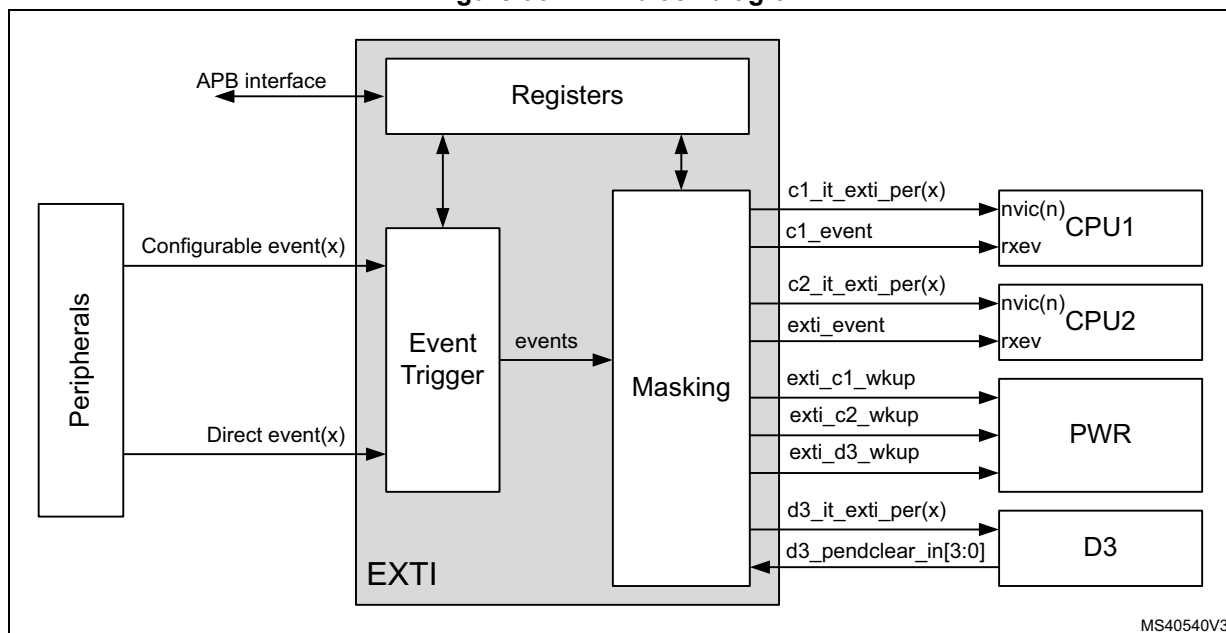
As shown in [Figure 96](#), the EXTI consists of a Register block accessed via an APB interface, an Event input Trigger block, and a Masking block.

The Register block contains all EXTI registers.

The Event input trigger block provides Event input edge triggering logic.

The Masking block provides the Event input distribution to the different wakeup, interrupt and event outputs, and their masking.

Figure 96. EXTI block diagram



21.2.1 EXTI connections between peripherals, CPU, and D3 domain

The peripherals able to generate wakeup events when the system is in Stop mode or a CPU is in CStop mode are connected to an EXTI Configurable event input or Direct Event input:

- Peripheral signals that generate a pulse are connected to an EXTI Configurable Event input. For these events the EXTI provides a CPU status pending bit that has to be cleared.
- Peripheral Interrupt and Wakeup sources that have to be cleared in the peripheral are connected to an EXTI Direct Event input. There is no CPU status pending bit within the EXTI. The Interrupt or Wakeup is cleared by the CPU in the peripheral.

The Event inputs able to wakeup D3 for autonomous Run mode are provided with a D3 domain pending request function, that has to be cleared. This clearing request is taken care of by the signal selected by the Pending clear selection.

The CPU(n) interrupts are connected to their respective CPU(n) NVIC, and, similarly, the CPU(n) event is connected to the CPU(n) rxev input.

The EXTI Wakeup signals are connected to the PWR block, and are used to wakeup the D3 domain and/or the CPU(n).

The D3 domain interrupts allow the system to trigger events for D3 domain autonomous Run mode operation.

21.3 EXTI functional description

Depending on the EXTI Event input type and wakeup target(s), different logic implementations are used. The applicable features are controlled from register bits:

- Active trigger edge enable, by *EXTI rising trigger selection register (EXTI_RTSR1)*, *EXTI rising trigger selection register (EXTI_RTSR2)*, *EXTI rising trigger selection register (EXTI_RTSR3)*, and *EXTI falling trigger selection register (EXTI_FTSR1)*, *EXTI falling trigger selection register (EXTI_FTSR2)*, *EXTI falling trigger selection register (EXTI_FTSR3)*
- Software trigger, by *EXTI software interrupt event register (EXTI_SWIER1)*, *EXTI software interrupt event register (EXTI_SWIER2)*, *EXTI software interrupt event register (EXTI_SWIER3)*
- CPU Interrupt enable, by *EXTI interrupt mask register (EXTI_CnIMR1)*, *EXTI interrupt mask register (EXTI_CnIMR2)*, *EXTI interrupt mask register (EXTI_CnIMR3)*
- CPU Event enable, by *EXTI event mask register (EXTI_CnEMR1)*, *EXTI event mask register (EXTI_CnEMR2)*, *EXTI event mask register (EXTI_CnEMR3)*
- D3 domain wakeup pending, by *EXTI D3 pending mask register (EXTI_D3PMR1)*, *EXTI D3 pending mask register (EXTI_D3PMR2)*, *EXTI D3 pending mask register (EXTI_D3PMR3)*

Table 150. EXTI Event input configurations and register control⁽¹⁾

Event input type	Wakeup target(s)	Logic implementation	EXTI_RTSR	EXTI_FTSR	EXTI_SWIER	EXTI_CnIMR	EXTI_CnEMR	EXTI_D3PMR
Configurable	CPU(n) ⁽²⁾	Configurable event input, CPU wakeup logic	X	X	X	X	X	-
	Any ⁽³⁾	Configurable event input, Any wakeup logic						X
Direct	CPU(n) ⁽²⁾	Direct event input, CPU wakeup logic	-	-	-	X	X	-
	Any ⁽³⁾	Direct event input, Any wakeup logic						X

1. X indicates that functionality is available.

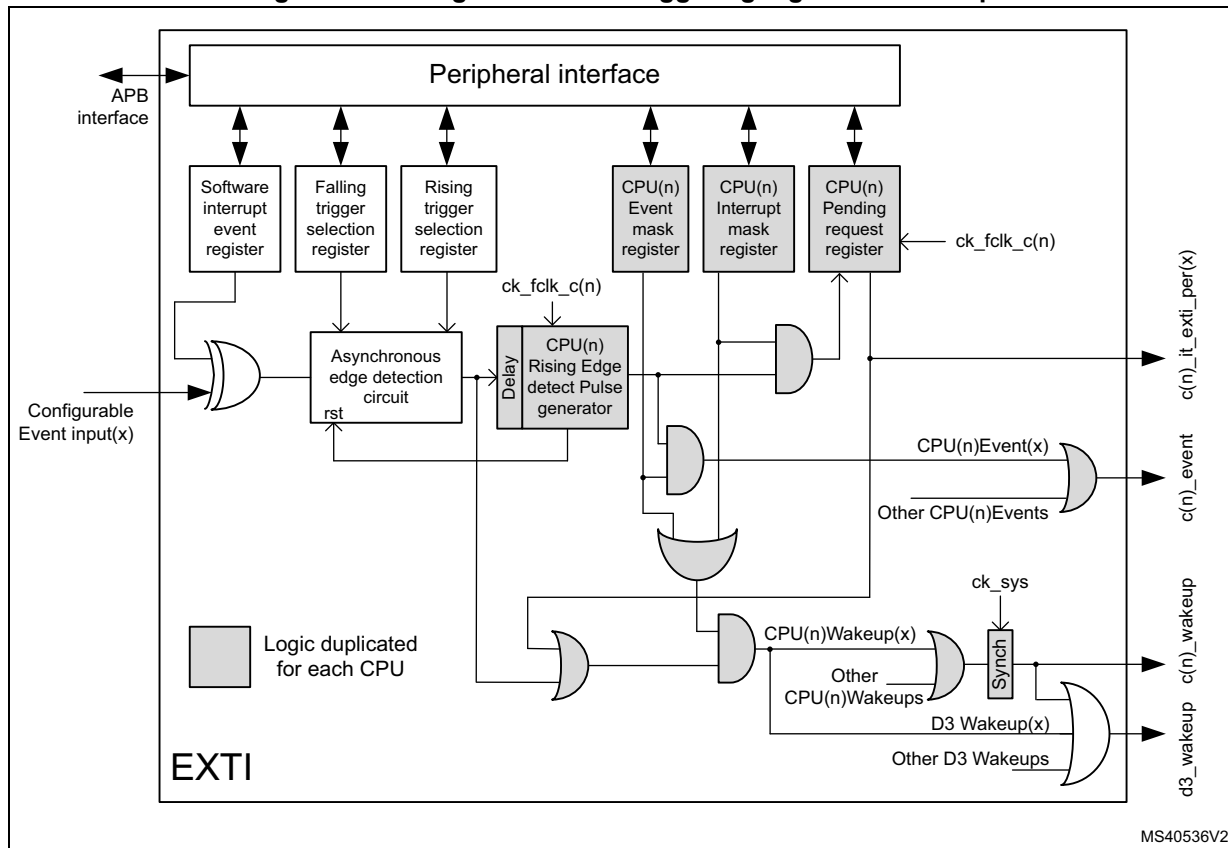
2. Waking-up CPU1 and/or CPU2.

3. Waking-up D3 domain for autonomous Run mode, and/or CPU1, and/or CPU2.

21.3.1 EXTI Configurable event input CPU wakeup

Figure 98 is a detailed representation of the logic associated with Configurable Event inputs which will always wake up a CPU(n).

Figure 97. Configurable event triggering logic CPU wakeup



The Software interrupt event register allows the system to trigger Configurable events by software, writing the *EXTI software interrupt event register (EXTI_SWIER1)*, the *EXTI software interrupt event register (EXTI_SWIER2)*, or the *EXTI software interrupt event register (EXTI_SWIER3)* register bit.

The rising edge *EXTI rising trigger selection register (EXTI_RTSR1)*, *EXTI rising trigger selection register (EXTI_RTSR2)*, *EXTI rising trigger selection register (EXTI_RTSR3)*, and falling edge *EXTI falling trigger selection register (EXTI_FTSR1)*, *EXTI falling trigger selection register (EXTI_FTSR2)*, *EXTI falling trigger selection register (EXTI_FTSR3)* selection registers allow the system to enable and select the Configurable event active trigger edge or both edges.

Each CPU has its dedicated interrupt mask register, namely *EXTI interrupt mask register (EXTI_CnIMR1)* and *EXTI interrupt mask register (EXTI_CnIMR2)*, *EXTI interrupt mask register (EXTI_CnIMR3)*, and *EXTI pending register (EXTI_CnPR1)*, *EXTI pending register (EXTI_CnPR2)*, *EXTI pending register (EXTI_CnPR3)* for Configurable events pending request registers. The CPU pending register will only be set for an unmasked CPU(n) interrupt. Each event provides a individual CPU(n) interrupt to the CPU(n) NVIC. The Configurable events interrupts need to be acknowledged by software in the EXTI_CnPR register.

Each CPU has dedicated event mask registers, i.e. *EXTI event mask register (EXTI_CnEMR1)*, *EXTI event mask register (EXTI_CnEMR2)*, and *EXTI event mask register (EXTI_CnEMR3)*. The enabled event then generates an event on a CPU. All events for a CPU are OR-ed together into a single CPU CPU(n) event signal. The CPU Pending register (EXTI_CnPR) will not be set for an unmasked CPU event.

When a CPU(n) interrupt or CPU(n) event is enabled, the Asynchronous edge detection circuit is reset by the clocked Delay and Rising edge detect pulse generator. This guarantees that the CPU(n) clock is woken up before the Asynchronous edge detection circuit is reset.

Note: *A detected Configurable event, enabled by CPU(n), is only cleared when CPU(n) wakes up. When the CPU(n) is kept in hold (see [Section 7: Power control \(PWR\)](#)), the detected Configurable event will not be cleared and the system will be kept in Run mode. To clear the detected Configurable event the other CPU shall release the CPU(n) from hold.*

21.3.2 EXTI configurable event input Any wakeup

[Figure 98](#) is a detailed representation of the logic associated with Configurable Event inputs that can wakeup D3 domain for autonomous Run mode and/or CPU(n) (“Any” target). It provides the same functionality as the Configurable event input CPU wakeup, with additional functionality to wake up the D3 domain independently.

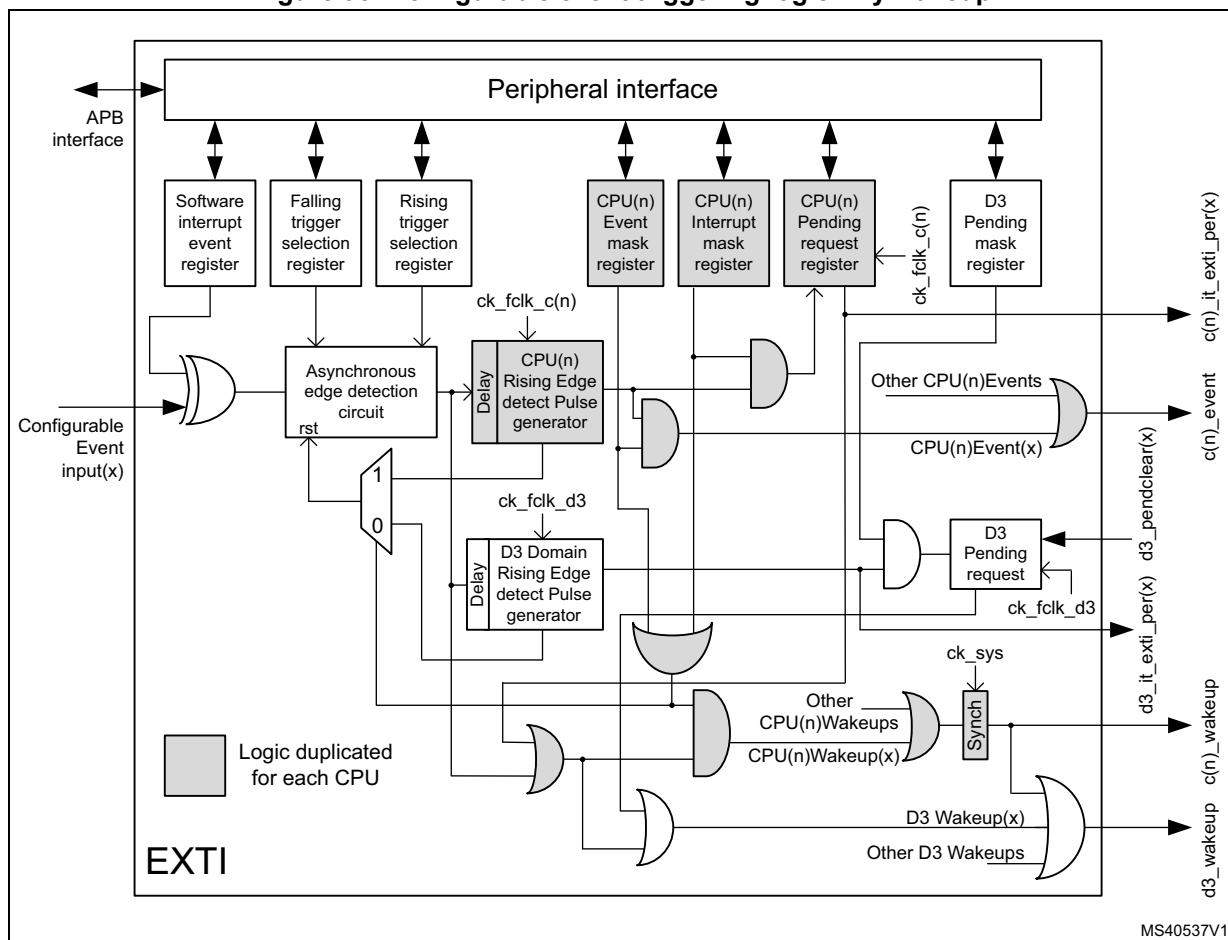
When all CPU(n) interrupts and CPU(n) events are disabled, the Asynchronous edge detection circuit is reset by the D3 domain clocked Delay and Rising edge detect pulse generator. This guarantees that the D3 domain clock is woken up before the Asynchronous edge detection circuit is reset.

Table 151. Configurable Event input Asynchronous Edge detector reset

EXTI_C1IMR	EXTI_C1EMR	EXTI_C2IMR	EXTI_C2EMR	Asynchronous Edge detector reset by
Both = 0		Both = 0		D3 domain clock rising edge detect pulse generator
At least one = 1		Both = 0		CPU1 clock rising edge detect pulse generator
Both = 0		At least one = 1		CPU2 clock rising edge detect pulse generator
At least one = 1		At least one = 1		CPU1 clock rising edge detect pulse generator OR-ed ⁽¹⁾ with CPU2 clock rising edge detect pulse generator

1. The first rising edge detect pulse generator will reset the Asynchronous Edge detection circuit. A new Configurable Event input edge for both CPUs will only be detected after the last rising edge detect pulse generator has completed. A Configurable Event input edge arriving between the detection of the first and the last rising edge detect pulse generator detection will only signal a new event to the first CPU.

Figure 98. Configurable event triggering logic Any wakeup



The event triggering logic for “Any” target has additional D3 Pending mask register [EXTI D3 pending mask register \(EXTI_D3PMR1\)](#), [EXTI D3 pending mask register \(EXTI_D3PMR2\)](#), [EXTI D3 pending mask register \(EXTI_D3PMR3\)](#) and D3 Pending request logic. The D3 Pending request logic will only be set for unmasked D3 Pending events. The D3 Pending request logic keeps the D3 domain in Run mode until the D3 Pending request logic is cleared by the selected D3 domain pendclear source.

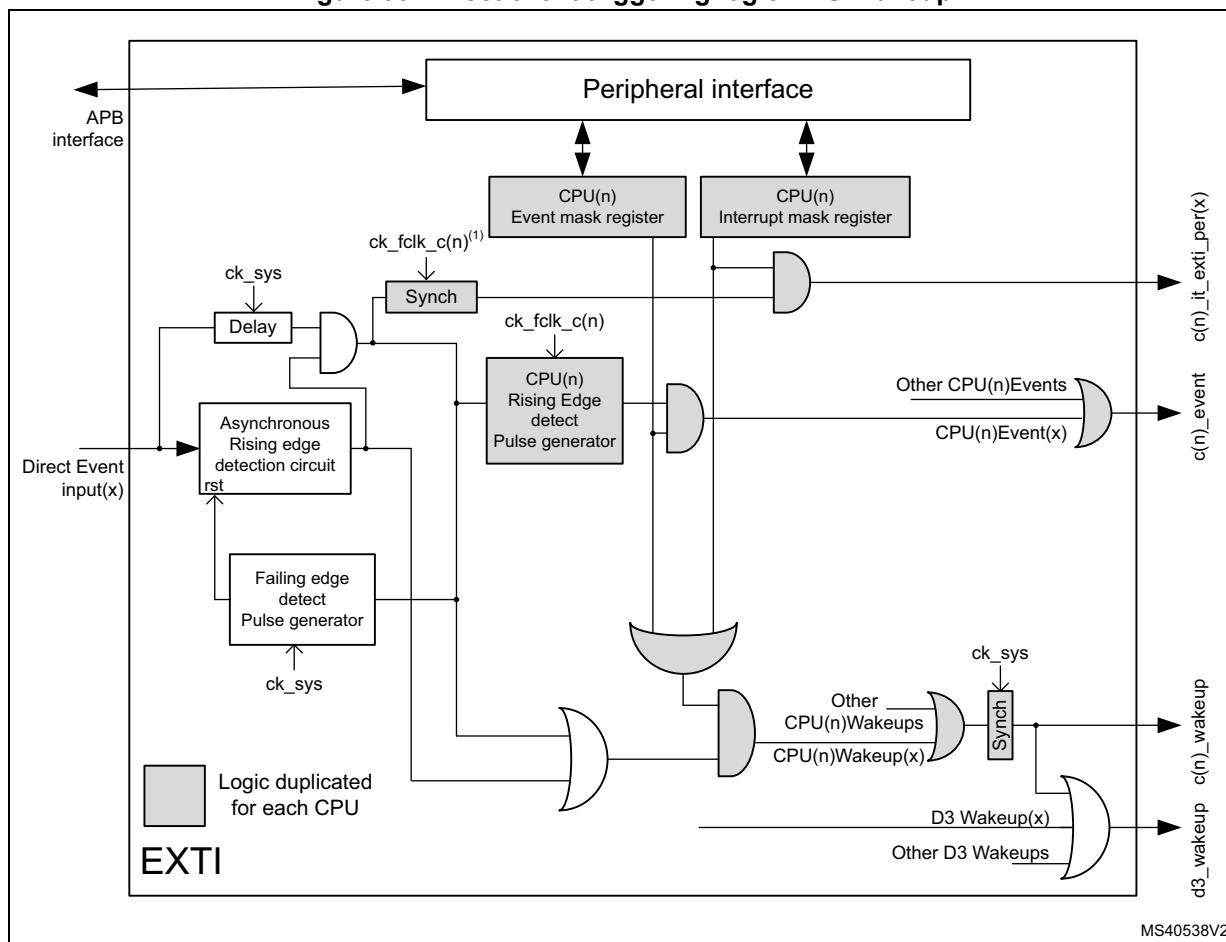
21.3.3 EXTI direct event input CPU wakeup

[Figure 99](#) is a detailed representation of the logic associated with Direct Event inputs waking up a CPU(n).

Direct events only provide CPU(n) interrupt enable and CPU(n) event enable functionality.

Note: *Direct events are cleared in the peripheral generating the event. When a Direct event input enabled by CPU(n) is cleared by the other CPU before the CPU(n) clock is running, (i.e. when CPU(n) is kept in hold, see [Section 7: Power control \(PWR\)](#)), the CPU(n) will no longer receive neither a CPU(n) interrupt nor a CPU(n) event, and will not wakeup. However the system will stay in Run mode, generating the CPU(n) clock. For this reason CPU(n) Direct events shall not be cleared by the other CPU.*

Figure 99. Direct event triggering logic CPU Wakeup

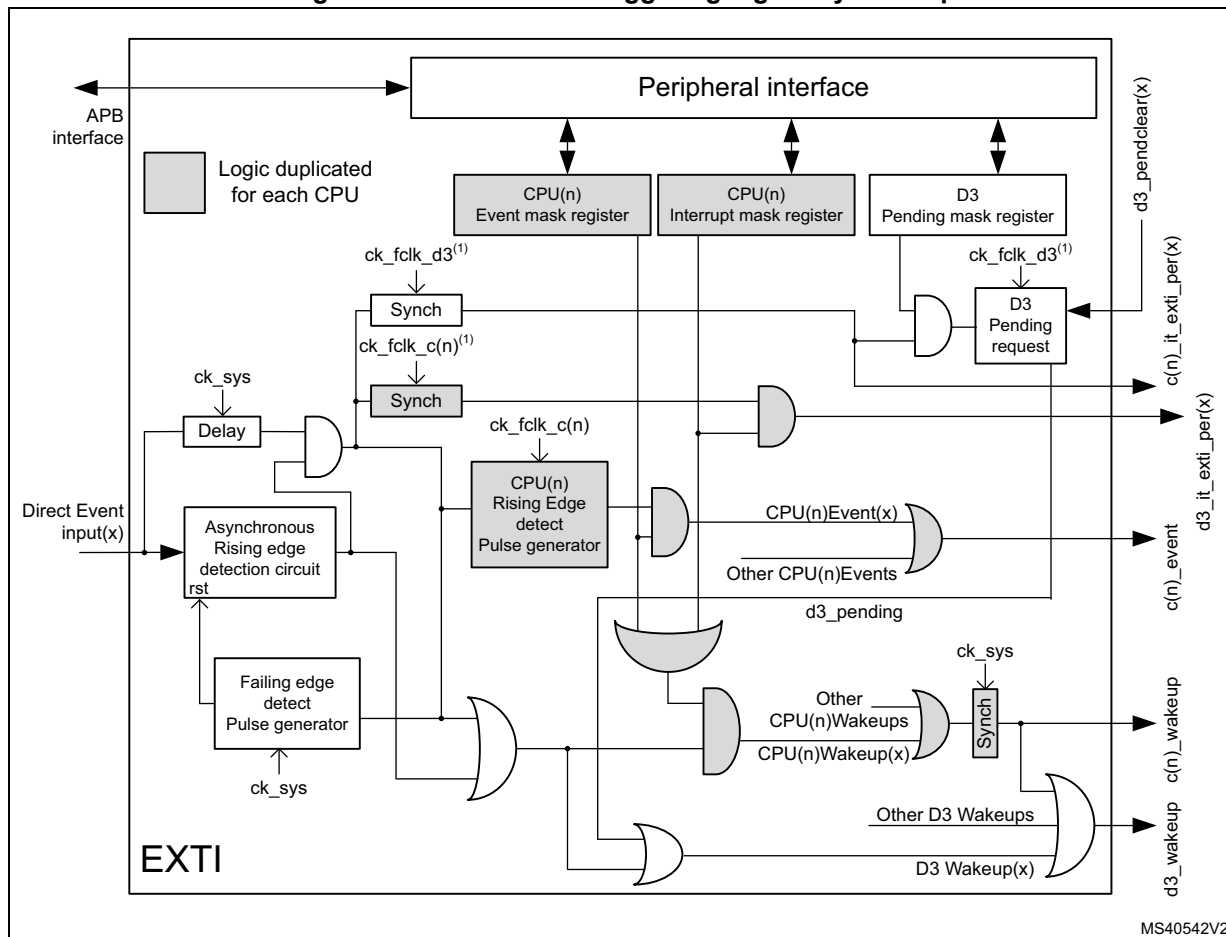


1. The CPU(n) interrupt for asynchronous Direct Event inputs (peripheral Wakeup signals) is synchronized with the CPU(n) clock. The synchronous Direct Event inputs (peripheral interrupt signals), after the asynchronous edge detection, are directly sent to the CPU(n) interrupt without resynchronization.

21.3.4 EXTI direct event input Any wakeup

Figure 100 is a detailed representation of the logic associated with Direct Event inputs waking up D3 domain for autonomous Run mode and/or CPU(n), (“Any” target). It provides the same functionality as the Direct event input CPU wakeup, plus additional functionality to wakeup the D3 domain independently.

Figure 100. Direct event triggering logic Any Wakeup



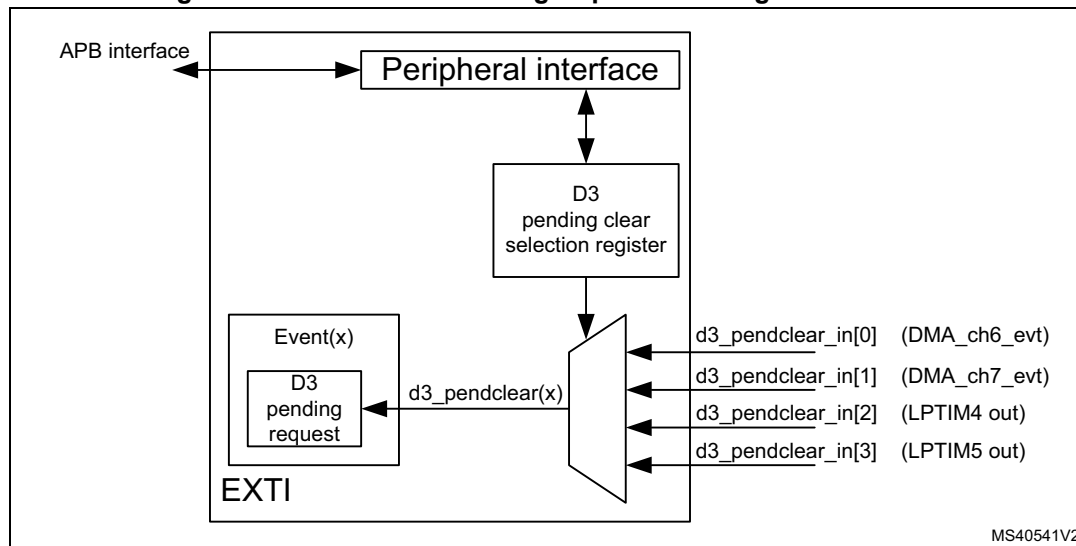
1. The CPU(n) interrupt and D3 domain interrupt for asynchronous Direct Event inputs (peripheral Wakeup signals) are synchronized, respectively, with the CPU(n) clock and the D3 domain clock. The synchronous Direct Event inputs (peripheral interrupt signals), after the asynchronous edge detection, are directly sent to the CPU(n) interrupt and the D3 domain interrupt without resynchronization in the EXTI.

21.3.5 EXTI D3 pending request clear selection

Event inputs able to wake up D3 domain for autonomous Run mode have D3 Pending request logic that can be cleared by the selected D3 pendclear source. For each D3 Pending request a D3 domain pendclear source can be selected from four different inputs.

Figure 101 is a detailed representation of the logic selecting the D3 pendclear source.

Figure 101. D3 domain Pending request clear logic



The D3 Pending request clear selection registers [EXTI D3 pending clear selection register low \(EXTI_D3PCR1L\)](#), [EXTI D3 pending clear selection register high \(EXTI_D3PCR1H\)](#), [EXTI D3 pending clear selection register low \(EXTI_D3PCR2L\)](#), [EXTI D3 pending clear selection register high \(EXTI_D3PCR2H\)](#), [EXTI D3 pending clear selection register low \(EXTI_D3PCR3L\)](#) and [EXTI D3 pending clear selection register high \(EXTI_D3PCR3H\)](#) allow the system to select the source to reset the D3 Pending request.

21.4 EXTI event input mapping

For the sixteen GPIO Event inputs the associated IOPORT pin has to be selected in the SYSCFG register SYSCFG_EXTICRn. The same pin from each IOPORT maps to the corresponding EXTI Event input.

The wakeup capabilities of each Event input are detailed in [Table 152](#). An Event input can either wake up CPU1, CPU2 or both, and in the case of “Any” can also wake up D3 domain for autonomous Run mode.

The EXTI Event inputs with a connection to the CPU NVIC are indicated in the *Connection to NVIC* column. For the EXTI events not having a connection to the NVIC, the peripheral interrupt is directly connected to the NVIC in parallel with the connection to the EXTI.

All EXTI Event inputs are OR-ed together and connected to the CPU event input (rxev).

Table 152. EXTI Event input mapping

Event input	Source	Event input type	Wakeup target(s)	Connection to NVIC
0 - 15	EXTI[15:0]	Configurable	Any	Yes
16	PVD and AVD ⁽¹⁾	Configurable	CPU1 or CPU2	Yes
17	RTC alarms	Configurable	CPU1 or CPU2	Yes
18	RTC tamper, RTC timestamp, RCC LSECSS ⁽²⁾	Configurable	CPU1 or CPU2	Yes
19	RTC wakeup timer	Configurable	Any	Yes
20	COMP1	Configurable	Any	Yes
21	COMP2	Configurable	Any	Yes
22	I2C1 wakeup	Direct	CPU1 or CPU2	Yes
23	I2C2 wakeup	Direct	CPU1 or CPU2	Yes
24	I2C3 wakeup	Direct	CPU1 or CPU2	Yes
25	I2C4 wakeup	Direct	Any	Yes
26	USART1 wakeup	Direct	CPU1 or CPU2	Yes
27	USART2 wakeup	Direct	CPU1 or CPU2	Yes
28	USART3 wakeup	Direct	CPU1 or CPU2	Yes
29	USART6 wakeup	Direct	CPU1 or CPU2	Yes
30	UART4 wakeup	Direct	CPU1 or CPU2	Yes
31	UART5 wakeup	Direct	CPU1 or CPU2	Yes
32	UART7 wakeup	Direct	CPU1 or CPU2	Yes
33	UART8 wakeup	Direct	CPU1 or CPU2	Yes
34	LPUART1 RX wakeup	Direct	Any	Yes
35	LPUART1 TX wakeup	Direct	Any	Yes
36	SPI1 wakeup	Direct	CPU1 or CPU2	Yes
37	SPI2 wakeup	Direct	CPU1 or CPU2	Yes
38	SPI3 wakeup	Direct	CPU1 or CPU2	Yes
39	SPI4 wakeup	Direct	CPU1 or CPU2	Yes
40	SPI5 wakeup	Direct	CPU1 or CPU2	Yes
41	SPI6 wakeup	Direct	Any	Yes
42	MDIO wakeup	Direct	CPU1 or CPU2	Yes
43	USB1 wakeup	Direct	CPU1 or CPU2	Yes
44	USB2 wakeup	Direct	CPU1 or CPU2	Yes
45	Reserved	-	-	-
46	DSI wakeup	Direct	CPU1 or CPU2	Yes
47	LPTIM1 wakeup	Direct	CPU1 or CPU2	Yes
48	LPTIM2 wakeup	Direct	Any	Yes

Table 152. EXTI Event input mapping (continued)

Event input	Source	Event input type	Wakeup target(s)	Connection to NVIC
49	LPTIM2 output	Configurable	Any	No ⁽³⁾
50	LPTIM3 wakeup	Direct	Any	Yes
51	LPTIM3 output	Configurable	Any	No ⁽³⁾
52	LPTIM4 wakeup	Direct	Any	Yes
53	LPTIM5 wakeup	Direct	Any	Yes
54	SWPMI wakeup	Direct	CPU1 or CPU2	Yes
55 ⁽⁴⁾	WKUP1	Direct	CPU1 or CPU2	Yes
56 ⁽⁴⁾	WKUP2	Direct	CPU1 or CPU2	Yes
57 ⁽⁴⁾	WKUP3	Direct	CPU1 or CPU2	Yes
58 ⁽⁴⁾	WKUP4	Direct	CPU1 or CPU2	Yes
59 ⁽⁴⁾	WKUP5	Direct	CPU1 or CPU2	Yes
60 ⁽⁴⁾	WKUP6	Direct	CPU1 or CPU2	Yes
61	RCC interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
62	I2C4 Event interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
63	I2C4 Error interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
64	LPUART1 global Interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
65	SPI6 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
66	BDMA CH0 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
67	BDMA CH1 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
68	BDMA CH2 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
69	BDMA CH3 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
70	BDMA CH4 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
71	BDMA CH5 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
72	BDMA CH6 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
73	BDMA CH7 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
74	DMAMUX2 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
75	ADC3 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
76	SAI4 interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
77	HSEM0 interrupt	Direct	CPU1 only	No ⁽⁵⁾
78	HSEM1 interrupt	Direct	CPU2 only	No ⁽⁵⁾
79	CortexM4 SEV interrupt	Direct	CPU1 only	No ⁽³⁾
80	CortexM7 SEV interrupt ⁽⁶⁾	Direct	CPU2 only	No ⁽³⁾
81	Reserved	-	-	-
82	WWDG1 reset	Configurable	CPU2 only	Yes
83	Reserved	-	-	-

Table 152. EXTI Event input mapping (continued)

Event input	Source	Event input type	Wakeup target(s)	Connection to NVIC
84	WWDG2 reset	Configurable	CPU1 only	Yes
85	HDMI-CEC wakeup	Configurable	CPU1 or CPU2	Yes
86	ETHERNET wakeup	Configurable	CPU1 or CPU2	Yes
87	HSECSS interrupt	Direct	CPU1 or CPU2	No ⁽⁵⁾
88	Reserved	-	-	-

1. PVD and AVD signals are OR-ed together on the same EXTI event input.
2. RTC Tamper, RTC timestamp and RCC LSECSS signals are OR-ed together on the same EXTI event input.
3. Not available on CPU NVIC, to be used for system wakeup only or CPU event input (rxev).
4. Signals of WKUP1 to WKUP6 correspond to WKUPn pin+1.
5. Available on CPU NVIC directly from the peripheral
6. Event duration is equal to 512 clock cycles.

21.5 EXTI functional behavior

The Direct event inputs are enabled in the respective peripheral generating the event. The Configurable events are enabled by enabling at least one of the trigger edges.

When in Stop mode an event will always wake up the D3 domain. In system Run and Stop modes an event will always generate an associated D3 domain interrupt. An event will only wake up a CPU when the event associated CPU interrupt is unmasked and/or the CPU event is unmasked.

Table 153. Masking functionality

CPU(n)		Configurable event inputs PRx bits of EXTI_CnPR	CPU(n)			D3 domain wakeup
Interrupt enable MRx bits of EXTI_CnIMR	Event enable MRx bits of EXTI_CnEMR		Interrupt	Event	Wakeup	
0	0	No	Masked	Masked	Masked	Yes ⁽¹⁾ / Masked ⁽²⁾
0	1	No	Masked	Yes	Yes	Yes
1	0	Status latched	Yes	Masked	Yes	Yes
1	1	Status latched	Yes	Yes	Yes	Yes

1. Only for Event inputs that allow the system to wakeup D3 domain for autonomous Run mode (Any target).
2. For Event inputs that will always wake up CPU(n).

For Configurable event inputs, when the enabled edge(s) occur on the event input, an event request is generated. When the associated CPU(n) interrupt is unmasked, the corresponding pending PRx bit in EXTI_CnPR is set and the CPU(n) interrupt signal is activated. EXTI_CnPR PRx pending bit shall be cleared by software writing it to '1'. This will clear the CPU(n) interrupt.

For Direct event inputs, when enabled in the associated peripheral, an event request is generated on the rising edge only. There is no corresponding CPU(n) pending bit. When the

associated CPU(n) interrupt is unmasked the corresponding CPU(n) interrupt signal is activated.

The CPU(n) event has to be unmasked to generate an event. When the enabled edge(s) occur on the Event input a CPU(n) event pulse is generated. There is no CPU(n) Event pending bit.

Both a CPU(n) interrupt and a CPU(n) event may be enabled on the same Event input. They will both trigger the same Event input condition(s).

For the Configurable Event inputs an event input request can be generated by software when writing a '1' in the software interrupt/event register EXTI_SWIER.

Whenever an Event input is enabled and a CPU(n) interrupt and/or CPU(n) event is unmasked, the Event input will also generate a D3 domain wakeup next to the CPU(n) wakeup.

Some Event inputs are able to wakeup the D3 domain autonomous Run mode, in this case the CPU(n) interrupt and CPU(n) event are masked, preventing the CPU(n) to be woken up. Two D3 domain autonomous Run mode wakeup mechanisms are supported:

- D3 domain wakeup without pending (EXTI_D3PMR = 0)
 - On a Configurable Event input this mechanism will wake up D3 domain and clear the D3 domain wakeup signal automatically after the Delay + Rising Edge detect Pulse generator.
 - On a Direct Event input this mechanism will wake up D3 domain and clear the D3 domain wakeup signal after the Direct Event input signal is cleared.
- D3 domain wakeup with pending (EXTI_D3PMR = 1)
 - On a Configurable Event input this mechanism will wake up D3 domain and clear the D3 domain wakeup signal after the Delay + Rising Edge detect Pulse generator and when the D3 Pending request is cleared.
 - On a Direct Event input this mechanism will wake up D3 domain and clear the D3 domain wakeup signal after the Direct Event input signal is cleared and when the D3 Pending request is cleared.

21.5.1 EXTI CPU interrupt procedure

- Unmask the Event input interrupt by setting the corresponding mask bits in the EXTI_CnIMR register.
- For Configurable Event inputs, enable the event input by setting either one or both the corresponding trigger edge enable bits in EXTI_RTSR and EXTI_FTSR registers.
- Enable the associated interrupt source in the CPU(n) NVIC or use the SEVONPEND, so that an interrupt coming from the CPU(n) interrupt signal is detectable by the CPU after a WFI/WFE instruction.
 - For Configurable event inputs the associated EXTI pending bit needs to be cleared.

21.5.2 EXTI CPU event procedure

- Unmask the Event input by setting the corresponding mask bits of the EXTI_CnEMR register.
- For Configurable Event inputs, enable the event input by setting either one or both the corresponding trigger edge enable bits in EXTI_RTISR and EXTI_FTISR registers.
- The CPU(n) event signal is detected by the CPU after a WFE instruction.
 - For Configurable event inputs there is no EXTI pending bit to clear.

21.5.3 EXTI CPU wakeup procedure

- Unmask the Event input by setting at least one of the corresponding mask bits in the EXTI_CnIMR and/or EXTI_CnEMR registers. The CPU(n) wakeup is generated at the same time as the unmasked CPU(n) interrupt and/or CPU(n) event.
- For Configurable Event inputs, enable the event input by setting either one or both the corresponding trigger edge enable bits in EXTI_RTISR and EXTI_FTISR registers.
- Direct Events will automatically generate a CPU(n) wakeup.

21.5.4 EXTI D3 domain wakeup for autonomous Run mode procedure

- Mask the Event input for waking up the CPU(n), by clearing both the corresponding mask bits in the EXTI_CnIMR and/or EXTI_CnEMR registers.
- For Configurable Event inputs, enable the event input by setting either one or both the corresponding trigger edge enable bits in EXTI_RTISR and EXTI_FTISR registers.
- Direct Events will automatically generate a D3 domain wakeup.
- Select the D3 domain wakeup mechanism in EXTI_D3PMR.
 - When D3 domain wakeup without pending (EXTI_PMR = 0) is selected, the Wakeup will be cleared automatically following the clearing of the Event input.
 - When D3 domain wakeup with pending (EXTI_PMR = 1) is selected the Wakeup needs to be cleared by a selected D3 domain pendclear source.
A pending D3 domain wakeup signal can also be cleared by FW clearing the associated EXTI_D3PMR register bit.
- After the D3 domain wakeup a D3 domain interrupt is generated.
 - Configurable Event inputs will generate a pulse on D3 domain interrupt.
 - Direct Event inputs will activate the D3 domain interrupt until the event input is cleared in the peripheral.

21.5.5 EXTI software interrupt/event trigger procedure

Any of the Configurable Event inputs can be triggered from the software interrupt/event register (the associated CPU(n) interrupt and/or CPU(n) event shall be enabled by their respective procedure).

- Enable the Event input by setting at least one of the corresponding edge trigger bits in the EXTI_RTISR and/or EXTI_FTISR registers.
- Unmask the software interrupt/event trigger by setting at least one of the corresponding mask bits in the EXTI_CnIMR and/or EXTI_CnEMR registers.
- Trigger the software interrupt/event by writing “1” to the corresponding bit in the EXTI_SWIER register.
- The Event input may be disabled by clearing the EXTI_RTISR and EXTI_FTISR register bits.

Note: *An edge on the Configurable event input will also trigger an interrupt/event.*

A software trigger can be used to set the D3 Pending request logic, keeping the D3 domain in Run until the D3 Pending request logic is cleared.

21.6 EXTI registers

Every register can only be accessed with 32-bit (word). A byte or half-word cannot be read or written.

21.6.1 EXTI rising trigger selection register (EXTI_RTSR1)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR21	TR20	TR19	TR18	TR17	TR16
										rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:0 **TRx**: Rising trigger event configuration bit of Configurable Event input x.⁽¹⁾

0: Rising trigger disabled (for Event and Interrupt) for input line

1: Rising trigger enabled (for Event and Interrupt) for input line

- The Configurable event inputs are edge triggered, no glitch must be generated on these inputs. If a rising edge on the Configurable event input occurs during writing of the register, the associated pending bit will not be set. Rising and falling edge triggers can be set for the same Configurable Event input. In this case, both edges generate a trigger.

21.6.2 EXTI falling trigger selection register (EXTI_FTSR1)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR21	TR20	TR19	TR18	TR17	TR16
										rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:0 **TRx**: Falling trigger event configuration bit of Configurable Event input x.⁽¹⁾

0: Falling trigger disabled (for Event and Interrupt) for input line

1: Falling trigger enabled (for Event and Interrupt) for input line.

- The Configurable event inputs are edge triggered, no glitch must be generated on these inputs. If a falling edge on the Configurable event input occurs during writing of the register, the associated pending bit will not be set. Rising and falling edge triggers can be set for the same Configurable Event input. In this case, both edges generate a trigger.

21.6.3 EXTI software interrupt event register (EXTI_SWIER1)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIER 21	SWIER 20	SWIER 19	SWIER 18	SWIER 17	SWIER 16
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIER 15	SWIER 14	SWIER 13	SWIER 12	SWIER 11	SWIER 10	SWIER 9	SWIER 8	SWIER 7	SWIER 6	SWIER 5	SWIER 4	SWIER 3	SWIER 2	SWIER 1	SWIER 0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:0 **SWIERx**: Software interrupt on line x

Will always return 0 when read.

0: Writing 0 has no effect.

1: Writing a 1 to this bit will trigger an event on line x. This bit is auto cleared by HW.

21.6.4 EXTI D3 pending mask register (EXTI_D3PMR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	MR25	Res.	Res.	Res.	MR21	MR20	MR19	Res.	Res.	Res.
						rw				rw	rw	rw			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:26 Reserved, must be kept at reset value.

Bit 25 **MRx**: D3 Pending Mask on Event input x

0: D3 Pending request from Line x is masked. Writing this bit to 0 will also clear the D3 Pending request.

1: D3 Pending request from Line x is unmasked. The D3 domain pending signal when triggered will keep D3 domain wakeup active until cleared.

Bits 24:22 Reserved, must be kept at reset value.

Bits 21:19 **MRx**: D3 Pending Mask on Event input x

0: D3 Pending request from Line x is masked. Writing this bit to 0 will also clear the D3 Pending request.

1: D3 Pending request from Line x is unmasked. The D3 domain pending signal when triggered will keep D3 domain wakeup active until cleared.

Bits 18:16 Reserved, must be kept at reset value.

Bits 15:0 **MRx**: D3 Pending Mask on Event input x

0: D3 Pending request from Line x is masked. Writing this bit to 0 will also clear the D3 Pending request.

1: D3 Pending request from Line x is unmasked. The D3 domain pending signal when triggered will keep D3 domain wakeup active until cleared.

21.6.5 EXTI D3 pending clear selection register low (EXTI_D3PCR1L)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PCS15		PCS14		PCS13		PCS12		PCS11		PCS10		PCS9		PCS8	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PCS7		PCS6		PCS5		PCS4		PCS3		PCS2		PCS1		PCS0	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **PCSx**: D3 Pending request clear input signal selection on Event input x = truncate (n/2)

00: DMA ch6 event selected as D3 domain pendclear source

01: DMA ch7 event selected as D3 domain pendclear source

10: LPTIM4 out selected as D3 domain pendclear source

11: LPTIM5 out selected as D3 domain pendclear source

21.6.6 EXTI D3 pending clear selection register high (EXTI_D3PCR1H)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCS25		Res.	Res.
												rw	rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PCS21		PCS20		PCS19		Res.	Res.	Res.	Res.	Res.	Res.
				rw	rw	rw	rw	rw	rw						

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:18 **PCSx**: D3 Pending request clear input signal selection on Event input x = truncate ((n+32)/2)

00: DMA ch6 event selected as D3 domain pendclear source

01: DMA ch7 event selected as D3 domain pendclear source

10: LPTIM4 out selected as D3 domain pendclear source

11: LPTIM5 out selected as D3 domain pendclear source

Bits 17:12 Reserved, must be kept at reset value.

Bits 11:6 **PCSx**: D3 Pending request clear input signal selection on Event input x = truncate ((n+32)/2)

00: DMA ch6 event selected as D3 domain pendclear source

01: DMA ch7 event selected as D3 domain pendclear source

10: LPTIM4 out selected as D3 domain pendclear source

11: LPTIM5 out selected as D3 domain pendclear source

Bits 5:0 Reserved, must be kept at reset value.

21.6.7 EXTI rising trigger selection register (EXTI_RTSTR2)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR51	Res.	TR49	Res.
												rw		rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:20 Reserved, must be kept at reset value.

Bit 19 **TRx**: Rising trigger event configuration bit of Configurable Event input x+32.⁽¹⁾

0: Rising trigger disabled (for Event and Interrupt) for input line

1: Rising trigger enabled (for Event and Interrupt) for input line

Bit 18 Reserved, must be kept at reset value.

Bit 17 **TRx**: Rising trigger event configuration bit of Configurable Event input x+32.⁽¹⁾

0: Rising trigger disabled (for Event and Interrupt) for input line

1: Rising trigger enabled (for Event and Interrupt) for input line

Bits 16:0 Reserved, must be kept at reset value.

- The Configurable event inputs are edge triggered, no glitch must be generated on these inputs.
If a rising edge on the configurable event input occurs during writing of the register, the associated pending bit will not be set.
Rising and falling edge triggers can be set for the same Configurable Event input. In this case, both edges generate a trigger.

21.6.8 EXTI falling trigger selection register (EXTI_FTSR2)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR51	Res.	TR49	Res.
												rw		rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:20 Reserved, must be kept at reset value.

Bit 19 **TRx**: Falling trigger event configuration bit of Configurable Event input x+32.⁽¹⁾

0: Falling trigger disabled (for Event and Interrupt) for input line

1: Falling trigger enabled (for Event and Interrupt) for input line

Bit 18 Reserved, must be kept at reset value.

Bit 17 **TRx**: Falling trigger event configuration bit of Configurable Event input x+32.⁽¹⁾

0: Falling trigger disabled (for Event and Interrupt) for input line

1: Falling trigger enabled (for Event and Interrupt) for input line

Bits 16:0 Reserved, must be kept at reset value.

1. The Configurable event inputs are edge triggered, no glitch must be generated on these inputs. If a falling edge on the Configurable event input occurs during writing of the register, the associated pending bit will not be set. Rising and falling edge triggers can be set for the same Configurable Event input. In this case, both edges generate a trigger.

21.6.9 EXTI software interrupt event register (EXTI_SWIER2)

Address offset: 0x28

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIER 51	Res.	SWIER 49	Res.
												rw		rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:20 Reserved, must be kept at reset value.

Bit 19 **SWIERx**: Software interrupt on line x+32

Will always return 0 when read.

0: Writing 0 has no effect.

1: Writing a 1 to this bit will trigger an event on line x. This bit is auto cleared by HW.

Bit 18 Reserved, must be kept at reset value.

Bit 17 **SWIERx**: Software interrupt on line x+32

Will always return 0 when read.

0: Writing 0 has no effect.

1: Writing a 1 to this bit will trigger an event on line x. This bit is auto cleared by HW.

Bits 16:0 Reserved, must be kept at reset value.

21.6.10 EXTI D3 pending mask register (EXTI_D3PMR2)

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MR53	MR52	MR51	MR50	MR49	MR48
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	MR41	Res.	Res.	Res.	Res.	Res.	MR35	MR34	Res.	Res.
						rw						rw	rw		

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:16 **MRx**: D3 Pending Mask on Event input x+32

0: D3 Pending request from Line x+32 is masked. Writing this bit to 0 will also clear the D3 Pending request.

1: D3 Pending request from Line x+32 is unmasked. The D3 domain pending signal when triggered will keep D3 domain wakeup active until cleared.

Bits 15:10 Reserved, must be kept at reset value.

Bit 9 **MRx**: D3 Pending Mask on Event input x+32

0: D3 Pending request from Line x+32 is masked. Writing this bit to 0 will also clear the D3 Pending request.

1: D3 Pending request from Line x+32 is unmasked. The D3 domain pending signal when triggered will keep D3 domain wakeup active until cleared.

Bits 8:4 Reserved, must be kept at reset value.

Bits 3:2 **MRx**: D3 Pending Mask on Event input x+32

0: D3 Pending request from Line x+32 is masked. Writing this bit to 0 will also clear the D3 Pending request.

1: D3 Pending request from Line x+32 is unmasked. The D3 domain pending signal when triggered will keep D3 domain wakeup active until cleared.

Bits 1:0 Reserved, must be kept at reset value.

21.6.11 EXTI D3 pending clear selection register low (EXTI_D3PCR2L)

Address offset: 0x30

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCS41		Res.	Res.
												rw	rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCS35		PCS34		Res.	Res.	Res.	Res.
								rw	rw	rw	rw				

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:18 **PCSx**: D3 Pending request clear input signal selection on Event input x = truncate ((n+64)/2)

00: DMA ch6 event selected as D3 domain pendclear source

01: DMA ch7 event selected as D3 domain pendclear source

10: LPTIM4 out selected as D3 domain pendclear source

11: LPTIM5 out selected as D3 domain pendclear source

Bits 17:8 Reserved, must be kept at reset value.

Bits 7:4 **PCSx**: D3 Pending request clear input signal selection on Event input x= truncate ((n+64)/2)

00: DMA ch6 event selected as D3 domain pendclear source

01: DMA ch7 event selected as D3 domain pendclear source

10: LPTIM4 out selected as D3 domain pendclear source

11: LPTIM5 out selected as D3 domain pendclear source

Bits 3:0 Reserved, must be kept at reset value.

21.6.12 EXTI D3 pending clear selection register high (EXTI_D3PCR2H)

Address offset: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PCS53		PCS52		PCS51		PCS50		PCS49		PCS48	
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved, must be kept at reset value.

Bits 11:0 **PCSx**: D3 Pending request clear input signal selection on Event input x= truncate ((n+96)/2)

00: DMA ch6 event selected as D3 domain pendclear source

01: DMA ch7 event selected as D3 domain pendclear source

10: LPTIM4 out selected as D3 domain pendclear source

11: LPTIM5 out selected as D3 domain pendclear source

21.6.13 EXTI rising trigger selection register (EXTI_RTSTR3)

Address offset: 0x40

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR86	TR85	TR84	Res.	TR82	Res.	Res.
									rw	rw	rw		rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:20 **TRx**: Rising trigger event configuration bit of Configurable Event input x+64.⁽¹⁾

0: Rising trigger disabled (for Event and Interrupt) for input line

1: Rising trigger enabled (for Event and Interrupt) for input line

Bit 19 Reserved, must be kept at reset value.

Bit 18 **TRx**: Rising trigger event configuration bit of Configurable Event input x+64.⁽¹⁾

0: Rising trigger disabled (for Event and Interrupt) for input line

1: Rising trigger enabled (for Event and Interrupt) for input line

Bits 17:0 Reserved, must be kept at reset value.

1. The Configurable event inputs are edge triggered, no glitch must be generated on these inputs.
If a rising edge on the Configurable event input occurs during writing of the register, the associated pending bit will not be set.
Rising and falling edge triggers can be set for the same Configurable Event input. In this case, both edges generate a trigger.

21.6.14 EXTI falling trigger selection register (EXTI_FTSR3)

Address offset: 0x44

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR86	TR85	TR84	Res.	TR82	Res.	Res.
									rw	rw	rw		rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:20 **TRx**: Falling trigger event configuration bit of Configurable Event input x+64.⁽¹⁾

0: Falling trigger disabled (for Event and Interrupt) for input line

1: Falling trigger enabled (for Event and Interrupt) for input line

Bit 19 Reserved, must be kept at reset value.

Bit 18 **TRx**: Falling trigger event configuration bit of Configurable Event input x+64.⁽¹⁾

0: Falling trigger disabled (for Event and Interrupt) for input line

1: Falling trigger enabled (for Event and Interrupt) for input line

Bits 17:0 Reserved, must be kept at reset value.

1. The Configurable event inputs are edge triggered, no glitch must be generated on these inputs. If a falling edge on the Configurable event input occurs during writing of the register, the associated pending bit will not be set. Rising and falling edge triggers can be set for the same Configurable Event input. In this case, both edges generate a trigger.

21.6.15 EXTI software interrupt event register (EXTI_SWIER3)

Address offset: 0x48

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIER 86	SWIER 85	SWIER 84	Res.	SWIER 82	Res.	Res.
									rw	rw	rw		rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:20 **SWIERx**: Software interrupt on line x+64

Will always return 0 when read.

0: Writing 0 has no effect.

1: Writing a 1 to this bit will trigger an event on line x. This bit is auto cleared by HW.

Bit 19 Reserved, must be kept at reset value.

Bit 18 **SWIERx**: Software interrupt on line x+64

Will always return 0 when read.

0: Writing 0 has no effect.

1: Writing a 1 to this bit will trigger an event on line x. This bit is auto cleared by HW.

Bits 17:0 Reserved, must be kept at reset value.

21.6.16 EXTI D3 pending mask register (EXTI_D3PMR3)

Address offset: 0x4C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	MR88	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
							rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **MRx**: D3 Pending Mask on Event input x+64

0: D3 Pending request from Line x+64 is masked. Writing this bit to 0 will also clear the D3 Pending request.

1: D3 Pending request from Line x+64 is unmasked. The D3 domain pending signal when triggered will keep D3 domain wakeup active until cleared.

Bits 23:0 Reserved, must be kept at reset value.

21.6.17 EXTI D3 pending clear selection register low (EXTI_D3PCR3L)

Address offset: 0x50

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:0 Reserved, must be kept at reset value.

21.6.18 EXTI D3 pending clear selection register high (EXTI_D3PCR3H)

Address offset: 0x54

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCS88	
														rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:18 Reserved, must be kept at reset value.

Bits 17:16 **PCSx**: D3 Pending request clear input signal selection on Event input x= truncate ((n+160)/2)

00: DMA ch6 event selected as D3 domain pendclear source

01: DMA ch7 event selected as D3 domain pendclear source

10: LPTIM4 out selected as D3 domain pendclear source

11: LPTIM5 out selected as D3 domain pendclear source

Bits 15:0 Reserved, must be kept at reset value.

21.6.19 EXTI interrupt mask register (EXTI_CnIMR1)

Address offset: 0x80 (EXTI_C1IMR1), 0xC0 (EXTI_C2IMR1)

Reset value: 0xFFC0 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MR31	MR30	MR29	MR28	MR27	MR26	MR25	MR24	MR23	MR22	MR21	MR20	MR19	MR18	MR17	MR16
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:22 **MRx**: CPUn interrupt Mask on Direct Event input x⁽¹⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

Bits 21:0 **MRx**: CPUn interrupt Mask on Configurable Event input x⁽²⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

1. The reset value for Direct Event inputs is set to '1' in order to enable the interrupt by default.
2. The reset value for Configurable Event inputs is set to '0' in order to disable the interrupt by default.

21.6.20 EXTI event mask register (EXTI_CnEMR1)

Address offset: 0x84 (EXTI_C1EMR1), 0xC4 (EXTI_C2EMR1)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MR31	MR30	MR29	MR28	MR27	MR26	MR25	MR24	MR23	MR22	MR21	MR20	MR19	MR18	MR17	MR16
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **MRx**: CPUn Event mask on Event input x

0: Event request from Line x is masked

1: Event request from Line x is unmasked

21.6.21 EXTI pending register (EXTI_CnPR1)

Address offset: 0x88 (EXTI_C1PR1), 0xC8 (EXTI_C2PR1)

Reset value: undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR21	PR21	PR19	PR18	PR17	PR16
										rc1	rc1	rc1	rc1	rc1	rc1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:0 **PRx**: Configurable event inputs x Pending bit

0: No trigger request occurred

1: selected trigger request occurred

This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing a 1 into the bit or by changing the sensitivity of the edge detector.

21.6.22 EXTI interrupt mask register (EXTI_CnIMR2)

Address offset: 0x90 (EXTI_C1IMR2), 0xD0 (EXTI_C2IMR2)

Reset value: 0xFFFF 5 FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MR63	MR62	MR61	MR60	MR59	MR58	MR57	MR56	MR55	MR54	MR53	MR52	MR51	MR50	MR49	MR48
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR47	MR46	Res.	MR44	MR43	MR42	MR41	MR40	MR39	MR38	MR37	MR36	MR35	MR34	MR33	MR32
rw	rw	1	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:20 **MRx**: CPUn Interrupt Mask on Direct Event input x+32⁽¹⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

Bit 19 **MRx**: CPUn interrupt Mask on Configurable Event input x+32⁽²⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

Bit 18 **MRx**: CPUn Interrupt Mask on Direct Event input x+32⁽¹⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

Bit 17 **MRx**: CPUn interrupt Mask on Configurable Event input x+32⁽²⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

Bits 16:14 **MRx**: CPUn Interrupt Mask on Direct Event input x+32 ⁽¹⁾

- 0: Interrupt request from Line x is masked
- 1: Interrupt request from Line x is unmasked

Bit 13 Reserved, must be kept at reset value (1).

Bits 12:0 **MRx**: CPUn Interrupt Mask on Direct Event input x+32 ⁽¹⁾

- 0: Interrupt request from Line x is masked
- 1: Interrupt request from Line x is unmasked

1. The reset value for Direct Event inputs is set to '1' in order to enable the interrupt by default.
2. The reset value for Configurable Event inputs is set to '0' in order to disable the interrupt by default.

21.6.23 EXTI event mask register (EXTI_CnEMR2)

Address offset: 0x94 (EXTI_C1EMR2), 0xD4 (EXTI_C2EMR2)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MR63	MR62	MR61	MR60	MR59	MR58	MR57	MR56	MR55	MR54	MR53	MR52	MR51	MR50	MR49	MR48
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR47	MR46	Res.	MR44	MR43	MR42	MR41	MR40	MR39	MR38	MR37	MR36	MR35	MR34	MR33	MR32
r/w	r/w	0	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:14 **MRx**: CPUn Event mask on Event input x+32

- 0: Event request from Line x is masked
- 1: Event request from Line x is unmasked

Bit 13 Reserved, must be kept at reset value.

Bits 12:0 **MRx**: CPUn Event mask on Event input x+32

- 0: Event request from Line x is masked
- 1: Event request from Line x is unmasked

21.6.24 EXTI pending register (EXTI_CnPR2)

Address offset: 0x98 (EXTI_C1PR2), 0xD8 (EXTI_C2PR2)

Reset value: undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR51	Res.	PR49	Res.
												rc1		rc1	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:20 Reserved, must be kept at reset value.

Bit 19 **PRx**: Configurable event inputs x+32 Pending bit

0: No trigger request occurred

1: selected trigger request occurred

This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing a 1 into the bit or by changing the sensitivity of the edge detector.

Bit 18 Reserved, must be kept at reset value.

Bit 17 **PRx**: Configurable event inputs x+32 Pending bit

0: No trigger request occurred

1: selected trigger request occurred

This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing a 1 into the bit or by changing the sensitivity of the edge detector.

Bits 16:0 Reserved, must be kept at reset value.

21.6.25 EXTI interrupt mask register (EXTI_CnIMR3)

Address offset: 0xA0 (EXTI_C1IMR3), 0xE0 (EXTI_C2IMR3)

Reset value: 0x018B FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res	Res	Res	Res	Res	Res	Res	MR88	MR87	MR86	MR85	MR84	Res	MR82	Res	MR80
							r/w	r/w	r/w	r/w	r/w		r/w		r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR79	MR78	MR77	MR76	MR75	MR74	MR73	MR72	MR71	MR70	MR69	MR68	MR67	MR66	MR65	MR64
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:25 Reserved, must be kept at reset value.

Bits 24:23 **MRx**: CPUn Interrupt Mask on Direct Event input x+64 ⁽¹⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

Bits 22:20 **MRx**: CPUn interrupt Mask on Configurable Event input x+64 ⁽²⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

Bit 19 Reserved, must be kept at reset value (1).

Bit 18 **MRx**: CPUn interrupt Mask on Configurable Event input x+64 ⁽²⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

Bit 17 Reserved, must be kept at reset value (1).

Bits 16:0 **MRx**: CPUn Interrupt Mask on Direct Event input x+64 ⁽¹⁾

0: Interrupt request from Line x is masked

1: Interrupt request from Line x is unmasked

1. The reset value for Direct Event inputs is set to '1' in order to enable the interrupt by default.

2. The reset value for Configurable Event inputs is set to '0' in order to disable the interrupt by default.

21.6.26 EXTI event mask register (EXTI_CnEMR3)

Address offset: 0xA4 (EXTI_C1EMR3), 0xE4 (EXTI_C2EMR3)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	MR88	MR87	MR86	MR85	MR84	Res.	MR82	Res.	MR80
							rw	rw	rw	rw	rw		rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR79	MR78	MR77	MR76	MR75	MR74	MR73	MR72	MR71	MR70	MR69	MR68	MR67	MR66	MR65	MR64
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bits 24:0 **MRx**: CPUx Event mask on Event input x+64

0: Event request from Line x is masked

1: Event request from Line x is unmasked

21.6.27 EXTI pending register (EXTI_CnPR3)

Address offset: 0xA8 (EXTI_C1PR3), 0xE8 (EXTI_C2PR3)

Reset value: undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR86	PR85	PR84	Res.	PR82	Res.	Res.
									rc1	rc1	rc1		rc1		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:20 **PRx**: Configurable event inputs x+64 Pending bit

0: No trigger request occurred

1: selected trigger request occurred

This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing a 1 into the bit or by changing the sensitivity of the edge detector.

Bit 19 Reserved, must be kept at reset value.

Bit 18 **PRx**: Configurable event inputs x+64 Pending bit

0: No trigger request occurred

1: selected trigger request occurred

This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing a 1 into the bit or by changing the sensitivity of the edge detector.

Bits 17:0 Reserved, must be kept at reset value.

21.6.28 EXTI register map

The following table gives the EXTI register map and the reset values.

Table 154. Asynchronous interrupt/event controller register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	EXTI_RTISR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR[21:0]																						
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x04	EXTI_FTSR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR[21:0]																						
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x08	EXTI_SWIER1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIER[21:0]																						
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C	EXTI_D3PMR1	Res.	Res.	Res.	Res.	Res.	Res.	MR[25]	Res.	Res.	Res.	MR[21:19]				Res.	Res.	MR[15:0]																
	Reset value							0				0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	EXTI_D3PCR1L	PCS[15]			PCS[14]		PCS[13]		PCS[12]		PCS[11]		PCS[10]		PCS[9]		PCS[8]		PCS[7]		PCS[6]		PCS[5]		PCS[4]		PCS[3]		PCS[2]		PCS[1]		PCS[0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x14	EXTI_D3PCR1H	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		PCS[25]	Res.	Res.	Res.	Res.	Res.	Res.		PCS[21]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value														0	0						0	0	0	0	0	0	0	0	0	0	0	0	
0x20	EXTI_RTISR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR[51]	Res.	TR[49]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value														0		0																	
0x24	EXTI_FTSR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR[51]	Res.	TR[49]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value														0		0																	
0x28	EXTI_SWIER2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIER[51]	Res.	SWIER[49]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value														0		0																	
0x2C	EXTI_D3PMR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MR[53:48]						Res.	Res.	Res.	Res.	Res.	Res.	MR[41]	Res.	Res.	Res.	Res.	Res.	MR[35]	Res.	MR[34]	Res.	
	Reset value											0	0	0	0	0	0						0						0	0	0	0	0	0
0x30	EXTI_D3PCR2L	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCS[41]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCS[35]	Res.	PCS[34]	Res.	Res.	Res.	Res.	Res.	
	Reset value														0	0									0	0	0	0						
0x34	EXTI_D3PCR2H	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																					0	0	0	0	0	0	0	0	0	0	0	0	0
0x40	EXTI_RTISR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR[86]	TR[85]	TR[84]	Res.	TR[82]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value										0	0	0	0	0																			

Table 154. Asynchronous interrupt/event controller register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x44	EXTI_FTSR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR[86]	TR[85]	TR[84]	Res.	TR[82]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value										0	0	0	0	0																			
0x48	EXTI_SWIER3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWIER[86]	SWIER[85]	SWIER[84]	Res.	SWIER[82]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value										0	0	0	0	0																			
0x4C	EXTI_D3PMR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MR[88]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value							0																										
0x50	EXTI_D3PCR3L	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																																	
0x54	EXTI_D3PCR3H	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCS[88]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value															0	0																	
0x58-0x7C	Reserved																																	
0x80	EXTI_C1IMR1	MR[31:22]										MR[21:0]																						
	Reset value	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x84	EXTI_C1EMR1	MR[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x88	EXTI_C1PR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR[21:0]																						
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x90	EXTI_C1IMR2	MR[63:52]										MR[51]	MR[50]	MR[49]	MR[48:46]				Res.	MR[44:32]														
	Reset value	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	1	1		1	1	1	1	1	1	1	1	1	1	1	1	1	1	
0x94	EXTI_C1EMR2	MR[63:46]																		Res.	MR[44:32]													
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	
0x98	EXTI_C1PR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR[51]	Res.	PR[49]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value													0		0																		
0xA0	EXTI_C1IMR3	Res.	Res.	Res.	Res.	Res.	Res.	MR[88]	MR[87]	MR[86]	MR[85]	MR[84]	Res.	Res.	MR[82]	Res.	MR[80:64]																	
	Reset value							1	1	0	0	0			0		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
0xA4	EXTI_C1EMR3	Res.	Res.	Res.	Res.	Res.	Res.	MR[88:84]				Res.	Res.	MR[82]	Res.	MR[80:64]																		
	Reset value							0	0	0	0	0			0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0xA8	EXTI_C1PR3	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR[86]	PR[85]	PR[84]	Res.	Res.	PR[82]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value									0	0	0			0																			

Table 154. Asynchronous interrupt/event controller register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0xAC-0xBC	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		
0xC0	EXTI_C2IMR1	MR[31:22]										MR[21:0]																							
	Reset value	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0xC4	EXTI_C2EMR1	MR[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0xC8	EXTI_C2PR1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR[21:0]																							
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0xD0	EXTI_C2IMR2	MR[63:52]										MR[51]	MR[50]	MR[49]	MR[48:46]			Res.	MR[44:32]																
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	1			1	1	1	1	1	1	1	1	1	1	1	1	1	
0xD4	EXTI_C2EMR2	MR[63:46]																			Res.	MR[44:32]													
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0		
0xD8	EXTI_C2PR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR[51]	Res.	PR[49]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		
	Reset value													0		0																			
0xE0	EXTI_C2IMR3	Res.								MR[88]	MR[87]	MR[86]	MR[85]	MR[84]	Res.	MR[82]	Res.	MR[80:64]																	
	Reset value									1	1	0	0	0		0		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1		
0xE4	EXTI_C2EMR3	Res.								MR[88:84]				Res.	MR[82]	Res.	MR[80:64]																		
	Reset value									0	0	0	0	0		0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0xE8	EXTI_C2PR3	Res.									PR[86]	PR[85]	PR[84]	Res.	PR[82]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		
	Reset value										0	0	0		0																				

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

22 Cyclic redundancy check calculation unit (CRC)

22.1 Introduction

The CRC (cyclic redundancy check) calculation unit is used to get a CRC code from 8-, 16- or 32-bit data word and a generator polynomial.

Among other applications, CRC-based techniques are used to verify data transmission or storage integrity. In the scope of the functional safety standards, they offer a means of verifying the flash memory integrity. The CRC calculation unit helps compute a signature of the software during runtime, to be compared with a reference signature generated at link time and stored at a given memory location.

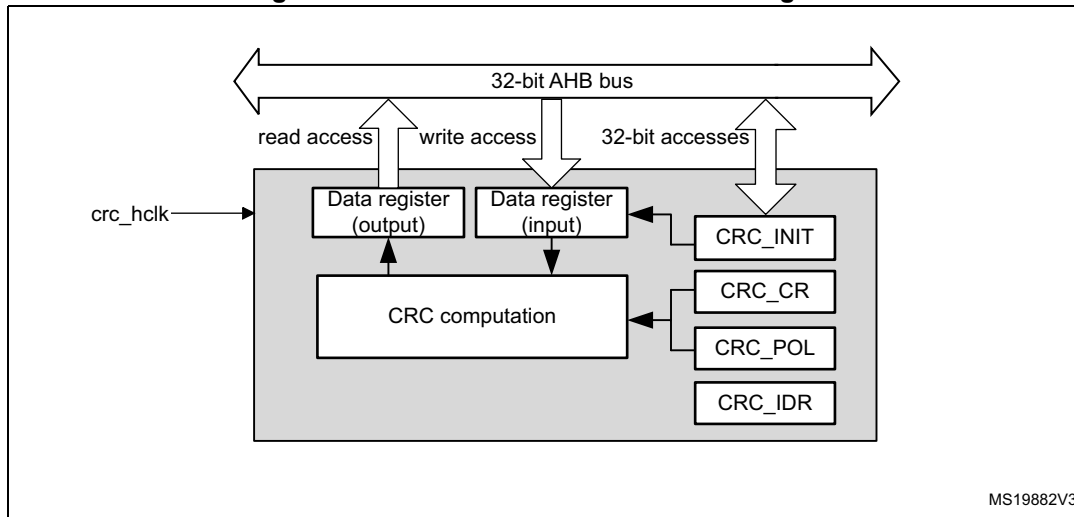
22.2 CRC main features

- Uses CRC-32 (Ethernet) polynomial: 0x4C11DB7
$$X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$$
- Alternatively, uses fully programmable polynomial with programmable size (7, 8, 16, 32 bits)
- Handles 8-, 16-, 32-bit data size
- Programmable CRC initial value
- Single input/output 32-bit data register
- Input buffer to avoid bus stall during calculation
- CRC computation done in 4 AHB clock cycles (HCLK) for the 32-bit data size
- General-purpose 8-bit register (can be used for temporary storage)
- Reversibility option on I/O data
- Accessed through AHB slave peripheral by 32-bit words only, with the exception of CRC_DR register that can be accessed by words, right-aligned half-words and right-aligned bytes

22.3 CRC functional description

22.3.1 CRC block diagram

Figure 102. CRC calculation unit block diagram



22.3.2 CRC internal signals

Table 155. CRC internal input/output signals

Signal name	Signal type	Description
crc_hclk	Digital input	AHB clock

22.3.3 CRC operation

The CRC calculation unit has a single 32-bit read/write data register (CRC_DR). It is used to input new data (write access), and holds the result of the previous CRC calculation (read access).

Each write operation to the data register creates a combination of the previous CRC value (stored in CRC_DR) and the new one. CRC computation is done on the whole 32-bit data word or byte by byte depending on the format of the data being written.

The CRC_DR register can be accessed by word, right-aligned half-word and right-aligned byte. For the other registers only 32-bit accesses are allowed.

The duration of the computation depends on data width:

- 4 AHB clock cycles for 32 bits
- 2 AHB clock cycles for 16 bits
- 1 AHB clock cycles for 8 bits

An input buffer allows a second data to be immediately written without waiting for any wait states due to the previous CRC calculation.

The data size can be dynamically adjusted to minimize the number of write accesses for a given number of bytes. For instance, a CRC for 5 bytes can be computed with a word write followed by a byte write.

The input data can be reversed to manage the various endianness schemes. The reversing operation can be performed on 8 bits, 16 bits and 32 bits depending on the REV_IN[1:0] bits in the CRC_CR register.

For example, 0x1A2B3C4D input data are used for CRC calculation as:

- 0x58D43CB2 with bit-reversal done by byte
- 0xD458B23C with bit-reversal done by half-word
- 0xB23CD458 with bit-reversal done on the full word

The output data can also be reversed by setting the REV_OUT bit in the CRC_CR register.

The operation is done at bit level. For example, 0x11223344 output data are converted to 0x22CC4488.

The CRC calculator can be initialized to a programmable value using the RESET control bit in the CRC_CR register (the default value is 0xFFFFFFFF).

The initial CRC value can be programmed with the CRC_INIT register. The CRC_DR register is automatically initialized upon CRC_INIT register write access.

The CRC_IDR register can be used to hold a temporary value related to CRC calculation. It is not affected by the RESET bit in the CRC_CR register.

Polynomial programmability

The polynomial coefficients are fully programmable through the CRC_POL register, and the polynomial size can be configured to be 7, 8, 16 or 32 bits by programming the POLYSIZE[1:0] bits in the CRC_CR register. Even polynomials are not supported.

Note: The type of an even polynomial is $X+X^2+..+X^n$, while the type of an odd polynomial is $1+X+X^2+..+X^n$.

If the CRC data is less than 32-bit, its value can be read from the least significant bits of the CRC_DR register.

To obtain a reliable CRC calculation, the change on-fly of the polynomial value or size can not be performed during a CRC calculation. As a result, if a CRC calculation is ongoing, the application must either reset it or perform a CRC_DR read before changing the polynomial.

The default polynomial value is the CRC-32 (Ethernet) polynomial: 0x4C11DB7.

22.4 CRC registers

The CRC_DR register can be accessed by words, right-aligned half-words and right-aligned bytes. For the other registers only 32-bit accesses are allowed.

22.4.1 CRC data register (CRC_DR)

Address offset: 0x00

Reset value: 0xFFFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DR[31:16]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **DR[31:0]**: Data register bits

This register is used to write new data to the CRC calculator.

It holds the previous CRC calculation result when it is read.

If the data size is less than 32 bits, the least significant bits are used to write/read the correct value.

22.4.2 CRC independent data register (CRC_IDR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IDR[31:16]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **IDR[31:0]**: General-purpose 32-bit data register bits

These bits can be used as a temporary storage location for four bytes.

This register is not affected by CRC resets generated by the RESET bit in the CRC_CR register

22.4.3 CRC control register (CRC_CR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	REV_OUT	REV_IN[1:0]		POLYSIZE[1:0]		Res.	Res.	RESET
								rw	rw	rw	rw	rw			rs

Bits 31:8 Reserved, must be kept at reset value.

Bit 7 **REV_OUT**: Reverse output data

This bit controls the reversal of the bit order of the output data.

0: Bit order not affected

1: Bit-reversed output format

Bits 6:5 **REV_IN[1:0]**: Reverse input data

This bitfield controls the reversal of the bit order of the input data

00: Bit order not affected

01: Bit reversal done by byte

10: Bit reversal done by half-word

11: Bit reversal done by word

Bits 4:3 **POLYSIZE[1:0]**: Polynomial size

These bits control the size of the polynomial.

00: 32 bit polynomial

01: 16 bit polynomial

10: 8 bit polynomial

11: 7 bit polynomial

Bits 2:1 Reserved, must be kept at reset value.

Bit 0 **RESET**: RESET bit

This bit is set by software to reset the CRC calculation unit and set the data register to the value stored in the CRC_INIT register. This bit can only be set, it is automatically cleared by hardware

22.4.4 CRC initial value (CRC_INIT)

Address offset: 0x10

Reset value: 0xFFFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CRC_INIT[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_INIT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **CRC_INIT[31:0]**: Programmable initial CRC value
This register is used to write the CRC initial value.

22.4.5 CRC polynomial (CRC_POL)

Address offset: 0x14

Reset value: 0x04C1 1DB7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
POL[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
POL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **POL[31:0]**: Programmable polynomial
This register is used to write the coefficients of the polynomial to be used for CRC calculation.
If the polynomial size is less than 32 bits, the least significant bits have to be used to program the correct value.

22.4.6 CRC register map

Table 156. CRC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	CRC_DR	DR[31:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x04	CRC_IDR	IDR[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	CRC_CR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	REV_OUT	REV_IN[1:0]	POLYSIZE[1:0]		Res	Res	RESET
	Reset value																									0	0	0	0	0			0
0x10	CRC_INIT	CRC_INIT[31:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x14	CRC_POL	POL[31:0]																															
	Reset value	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	1	0	0	0	1	1	1	0	1	1	0	1	1	0	1	1	1

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

46 Watchdog overview

The devices feature four embedded watchdog blocks which offer a combination of high safety, timing accuracy and flexibility. Option bytes allow adapting the behavior of the watchdogs to the user application.

46.1 Watchdog main features

- Two independent watchdogs (IWDG1/2), each dedicated to a CPU
- Two window watchdogs (WWDG1/2), each dedicated to a CPU
- Each CPU can receive an interrupt if the other CPU has been reset due to a window watchdog timeout.
- Each CPU can receive the early interrupt of its dedicated window watchdog.
- Watchdog functions can be configured through option bytes.

46.2 Watchdog brief functional description

IWDG1 and WWDG1 are dedicated to CPU1, while IWDG2 and WWDG2 are dedicated to CPU2.

Both watchdog peripherals (Independent and Window) allow detecting and resolving malfunctions due to software or hardware failures.

The window watchdogs (WWDG1/2) clocks are derived from the APB clocks and have a configurable time window that can be programmed to detect abnormally late or early application behavior. The WWDGs are best suited for monitoring software execution.

Each WWDG provides a reset and an early interrupt signal.

As shown in [Figure 575](#), the early wakeup interrupt output (wwdg1_ewit) of WWDG1 is connected to CPU1 interrupt controller.

The WWDG1 reset signal output (wwdg1_out_rst) is input to the reset and clock controller (RCC), and generates either a CPU1 reset or a system reset according to the value of WW1RSC bit in RCC_GRC register. In both cases, the wwdg1_out_rst also resets the WWDG1 block.

In addition, the wwdg1_out_rst signal is connected to CPU2 interrupt controller via the EXTI block. This function allows each core to be interrupted when the other core has a failure due to a window watchdog reset.

WWDG2 connection is completely symmetrical compared to WWDG1 connection.

Note: *The WWDG1 and WWDG2 are hard-wired to be used by CPU1 and CPU2, respectively. There is no register protection. As a result it is not recommended that CPU1 uses WWDG2, and CPU2, WWDG1.*

Refer to [Section 47: System window watchdog \(WWDG\)](#) for a detailed description of WWDG blocks.

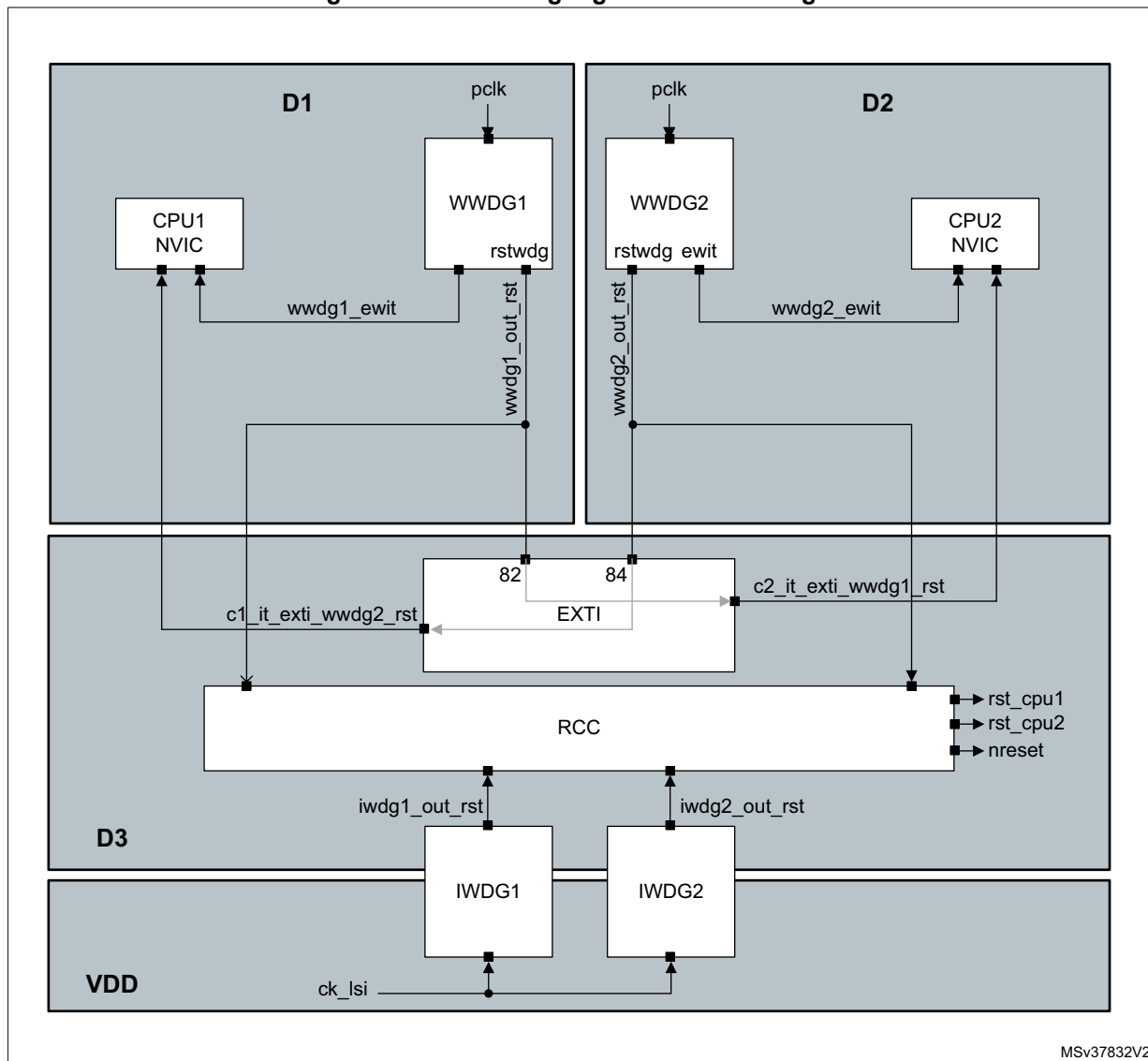
The independent watchdogs (IWDG1/2) are clocked by the low-speed clock (LSI) and thus stay active even if the main clock fails. They are consequently best suited for applications

which require the watchdog to run totally independently of the main application. The IWDGs are ideal solutions to recover from unexpected software or hardware failures.

Refer to [Section 48: Independent watchdog \(IWDG\)](#) for a detailed description of IWDG blocks.

Note: Like WWDG blocks, IWDG1 block is designed to work with CPU1, and IWDG2 with CPU2.

Figure 575. Watchdog high-level block diagram



46.2.1 Enabling the watchdog clock

Enabling WWDG1/2 clock

Each core can enable the window watchdog clocks via the RCC block. Setting the WWDG1EN bit in RCC_C1_D1APB1ENR register enables WWDG1 block clock, while setting WWDG2EN bit in RCC_C2_D2APB1LENR register enables WWDG2 block clock.

The software cannot stop WWDG1 and WWDG2 down-counting by setting WWDG1EN and WWDG2EN bit to '0', respectively.

CPU1 can also enable the WWDG1 block via the RCC_D1APB1ENR register, while CPU2 cannot program the WWDG1EN bit through this register.

Similarly, CPU2 can also enable the WWDG2 block via the RCC_D2APB1LENR register, while CPU1 cannot program the WWDG2EN bit through this register.

Enabling IWDG1/2 clock

The independent watchdogs do not need their clock to be enabled by the RCC block. IWDG1 is implicitly allocated to CPU1, and IWDG2 to CPU2. An option byte allows IWDG1 and IWDG2 to be automatically enabled after a system reset. Refer to [Section 48.3: IWDG functional description](#) for additional information.

46.2.2 Window watchdog reset scope

The reset scope of the window watchdogs can be controlled via WW1RSC and WW2RSC bits in the RCC_GRC register (see [Section 9.7.36: RCC global control register \(RCC_GCR\)](#)).

After a system reset, WW[2:1]RSC bits are set to '0', meaning that the reset scope of the WWDG1/2 is by default limited to CPU1 and CPU2.

When the WW1RSC bit is set to '1', the WWDG1 will generate a system reset if a timeout occurs. The software cannot set the WW1RSC bit back to '0' once it has been set to '1'. A similar description applies to WW2RSC bit with respect to WWDG2 block.

46.2.3 Watchdog behavior versus CPU state

A WWDG block is frozen when the corresponding CPU enters CSTOP mode. When the domain goes in DSTANDBY, the WWDG block located into this domain is reset.

IWDG blocks remain always enabled once enabled. Two option bytes allow freezing IWDG down-counting:

- IWDG_FZ_STOP option byte allows freezing WWDG1/2 down-counting when the CPU1/CPU2 is entering CSTOP mode or deeper low-power mode (DxSTOP, DxSTANDBY or product Standby mode).
- IWDG_FZ_STANDBY option byte allows freezing WWDG1/2 down-counting only when the product enters Standby mode.

Refer to [Section 4.4: FLASH option bytes](#) for a detailed description of the option bytes.

47 System window watchdog (WWDG)

47.1 Introduction

The system window watchdog (WWDG) is used to detect the occurrence of a software fault, usually generated by external interference or by unforeseen logical conditions, which causes the application program to abandon its normal sequence.

The watchdog circuit generates a reset on expiry of a programmed time period, unless the program refreshes the contents of the down-counter before the T6 bit is cleared. A reset is also generated if the 7-bit down-counter value (in the control register) is refreshed before the down-counter reaches the window register value. This implies that the counter must be refreshed in a limited window.

The WWDG clock is prescaled from the APB clock and has a configurable time-window that can be programmed to detect abnormally late or early application behavior.

The WWDG is best suited for applications requiring the watchdog to react within an accurate timing window.

47.2 WWDG main features

- Programmable free-running down-counter
- Conditional reset
 - Reset (if watchdog activated) when the down-counter value becomes lower than 0x40
 - Reset (if watchdog activated) if the down-counter is reloaded outside the window (see [Figure 577](#))
- Early wake-up interrupt (EWI): triggered (if enabled and the watchdog activated) when the down-counter is equal to 0x40

47.3 WWDG functional description

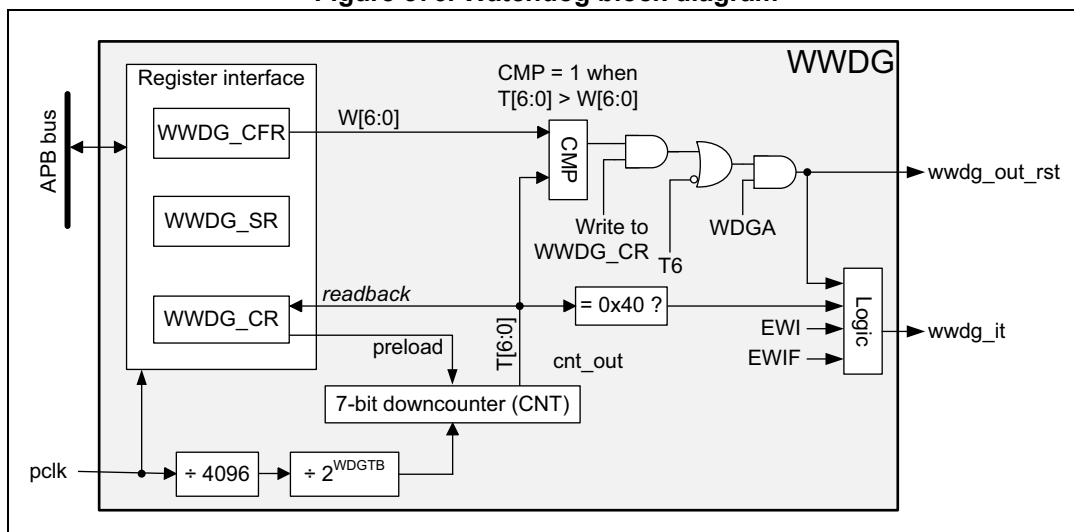
If the watchdog is activated (the WDGA bit is set in the WWDG_CR register), and when the 7-bit down-counter (T[6:0] bits) is decremented from 0x40 to 0x3F (T6 becomes cleared), it initiates a reset. If the software reloads the counter while the counter is greater than the value stored in the window register, then a reset is generated.

The application program must write in the WWDG_CR register at regular intervals during normal operation to prevent a reset. This operation can take place only when the counter value is lower than or equal to the window register value, and higher than 0x3F. The value to be stored in the WWDG_CR register must be between 0xFF and 0xC0.

Refer to [Figure 576](#) and to [Section 47.3.2: WWDG internal signals](#) for the WWDG block diagram.

47.3.1 WWDG block diagram

Figure 576. Watchdog block diagram



47.3.2 WWDG internal signals

Table 390 gives the list of WWDG internal signals.

Table 390. WWDG internal input/output signals

Signal name	Signal type	Description
pclk	Digital input	APB bus clock
wwdg_out_rst	Digital output	WWDG reset signal output
wwdg_it	Digital output	WWDG early interrupt output

47.3.3 Enabling the watchdog

The watchdog is always disabled after a reset. It is enabled by setting the WDGA bit in the WWDG_CR register, then it cannot be disabled again except by a reset.

47.3.4 Controlling the down-counter

This down-counter is free-running, counting down even if the watchdog is disabled. When the watchdog is enabled, the T6 bit must be set to prevent generating an immediate reset.

The T[5:0] bits contain the number of increments that represent the time delay before the watchdog produces a reset. The timing varies between a minimum and a maximum value, due to the unknown status of the prescaler when writing to the WWDG_CR register (see [Figure 577](#)). The [WWDG configuration register \(WWDG_CFR\)](#) contains the high limit of the window: to prevent a reset, the down-counter must be reloaded when its value is lower than or equal to the window register value, and greater than 0x3F. [Figure 577](#) describes the window watchdog process.

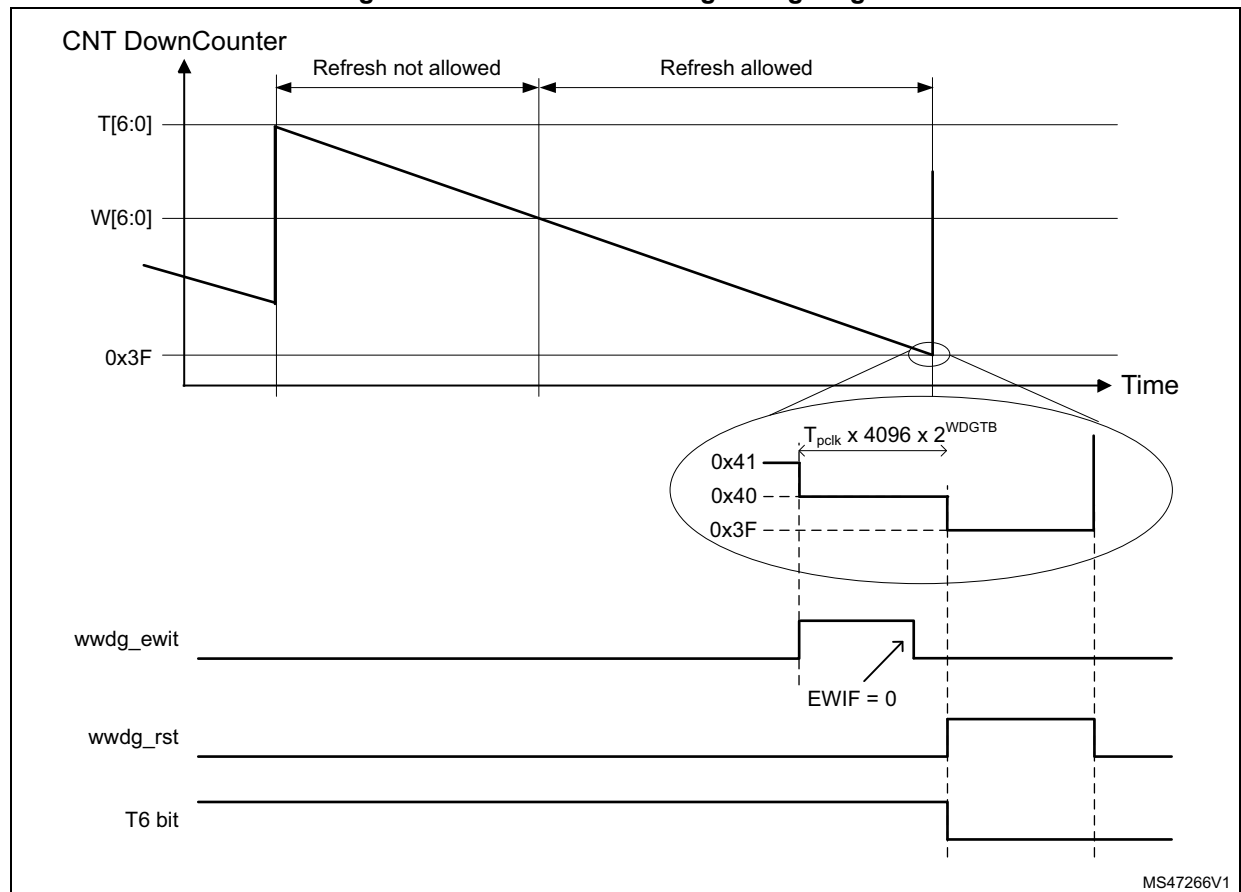
Note: The T6 bit can be used to generate a software reset (the WDGA bit is set and the T6 bit is cleared).

47.3.5 How to program the watchdog timeout

Use the formula in [Figure 577](#) to calculate the WWDG timeout.

Warning: When writing to the WWDG_CR register, always write 1 in the T6 bit to avoid generating an immediate reset.

Figure 577. Window watchdog timing diagram



The formula to calculate the timeout value is given by:

$$t_{WWDG} = t_{PCLK} \times 4096 \times 2^{WDGTB[2:0]} \times (T[5:0] + 1) \quad (\text{ms})$$

where:

t_{WWDG} : WWDG timeout

t_{PCLK} : APB clock period measured in ms

4096: value corresponding to internal divider

As an example, if APB frequency is 48 MHz, WDGTB[2:0] is set to 3, and T[5:0] is set to 63:

$$t_{\text{WWDG}} = (1 / 48000) \times 4096 \times 2^3 \times (63 + 1) = 43.69\text{ms}$$

Refer to the datasheet for the minimum and maximum values of t_{WWDG} .

47.3.6 Debug mode

When the CPU1/2 enter debug mode, WWDG1 and WWDG2 counters either continue to work normally or stop, depending on DBGMCU_APB3LFZ1/2 and DBGMCU_APB1LFZ1/2, respectively. For more details, refer to [Section 63: Debug infrastructure](#).

47.4 WWDG interrupts

The early wake-up interrupt (EWI) can be used if specific safety operations or data logging must be performed before the reset is generated. To enable the early wake-up interrupt, the application must:

- Write EWIF bit of WWDG_SR register to 0, to clear unwanted pending interrupt
- Write EWI bit of WWDG_CFR register to 1, to enable interrupt

When the down-counter reaches the value 0x40, a watchdog interrupt is generated, and the corresponding interrupt service routine (ISR) can be used to trigger specific actions (such as communications or data logging), before resetting the device.

In some applications, the EWI interrupt can be used to manage a software system check and/or system recovery/graceful degradation, without generating a WWDG reset. In this case the corresponding ISR must reload the WWDG counter to avoid the WWDG reset, then trigger the required actions.

The watchdog interrupt is cleared by writing '0' to the EWIF bit in the WWDG_SR register.

Note: When the watchdog interrupt cannot be served (for example due to a system lock in a higher priority task), the WWDG reset is eventually generated.

47.5 WWDG registers

Refer to [Section 1.2 on page 106](#) for a list of abbreviations used in register descriptions.

The peripheral registers can be accessed by halfwords (16-bit) or words (32-bit).

47.5.1 WWDG control register (WWDG_CR)

Address offset: 0x000

Reset value: 0x0000 007F

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WDGA	T[6:0]						
								rs	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bit 7 **WDGA**: Activation bit

This bit is set by software and only cleared by hardware after a reset. When WDGA = 1, the watchdog can generate a reset.

0: Watchdog disabled

1: Watchdog enabled

Bits 6:0 **T[6:0]**: 7-bit counter (MSB to LSB)

These bits contain the value of the watchdog counter, decremented every $(4096 \times 2^{WDGTB[2:0]})$ PCLK cycles. A reset is produced when it is decremented from 0x40 to 0x3F (T6 becomes cleared).

47.5.2 WWDG configuration register (WWDG_CFR)

Address offset: 0x004

Reset value: 0x0000 007F

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	WDGTB[2:0]			Res.	EWI	Res.	Res.	W[6:0]						
		rw	rw	rw		rs			rw	rw	rw	rw	rw	rw	rw

Bits 31:14 Reserved, must be kept at reset value.

Bits 13:11 **WDGTB[2:0]**: Timer base

The timebase of the prescaler can be modified as follows:

000: CK counter clock (PCLK div 4096) div 1

001: CK counter clock (PCLK div 4096) div 2

010: CK counter clock (PCLK div 4096) div 4

011: CK counter clock (PCLK div 4096) div 8

100: CK counter clock (PCLK div 4096) div 16

101: CK counter clock (PCLK div 4096) div 32

110: CK counter clock (PCLK div 4096) div 64

111: CK counter clock (PCLK div 4096) div 128

Bit 10 Reserved, must be kept at reset value.

Bit 9 **EWI**: Early wake-up interrupt enable

Set by software and cleared by hardware after a reset. When set, an interrupt occurs whenever the counter reaches the value 0x40.

Bits 8:7 Reserved, must be kept at reset value.

Bits 6:0 **W[6:0]**: 7-bit window value

These bits contain the window value to be compared with the down-counter.

47.5.3 WWDG status register (WWDG_SR)

Address offset: 0x008

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EWIF
															rc_w0

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **EWIF**: Early wake-up interrupt flag

This bit is set by hardware when the counter has reached the value 0x40. It must be cleared by software by writing 0. Writing 1 has no effect. This bit is also set if the interrupt is not enabled.

47.5.4 WWDG register map

The following table gives the WWDG register map and reset values.

Table 391. WWDG register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x000	WWDG_CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WDGA	T[6:0]									
	Reset value																									0	1	1	1	1	1	1	1			
0x004	WWDG_CFR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WDGTB [2:0]		Res.	Res.	EWI	Res.	Res.	W[6:0]									
	Reset value																			0	0	0		0			1	1	1	1	1	1	1			
0x008	WWDG_SR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EWIF			
	Reset value																																			

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

48 Independent watchdog (IWDG)

48.1 Introduction

The devices feature an embedded watchdog peripheral that offers a combination of high safety level, timing accuracy and flexibility of use. The Independent watchdog peripheral detects and solves malfunctions due to software failure, and triggers system reset when the counter reaches a given timeout value.

The independent watchdog (IWDG) is clocked by its own dedicated low-speed clock (LSI) and thus stays active even if the main clock fails.

The IWDG is best suited for applications that require the watchdog to run as a totally independent process outside the main application, but have lower timing accuracy constraints. For further information on the window watchdog, refer to [Section 47: System window watchdog \(WWDG\)](#).

48.2 IWDG main features

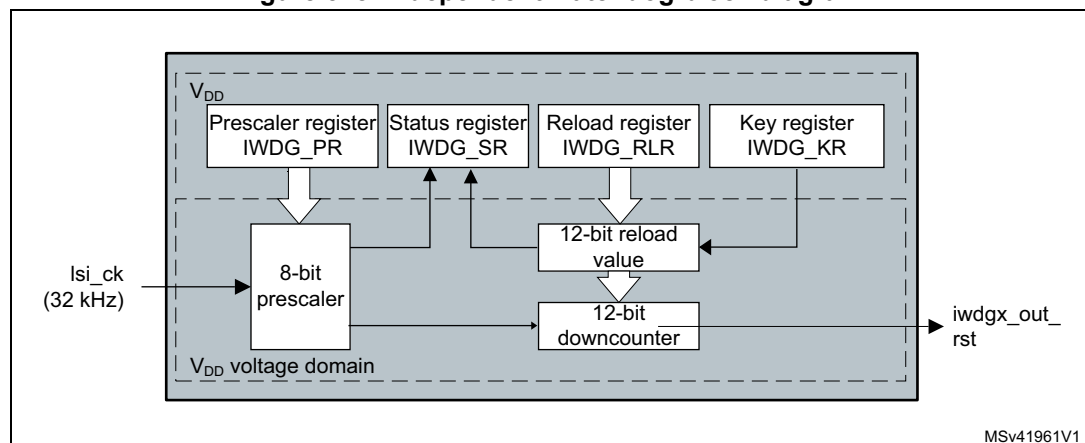
- Free-running downcounter
- Clocked from an independent RC oscillator (can operate in Standby and Stop modes)
- Conditional reset
 - Reset (if watchdog activated) when the downcounter value becomes lower than 0x000
 - Reset (if watchdog activated) if the downcounter is reloaded outside the window

48.3 IWDG functional description

48.3.1 IWDG block diagram

[Figure 578](#) shows the functional blocks of the independent watchdog module.

Figure 578. Independent watchdog block diagram



1. The register interface is located in the V_{DD} voltage domain. The watchdog function is located in the V_{DD} voltage domain, still functional in Stop and Standby modes.

When the independent watchdog is started by writing the value 0x0000 CCCC in the *IWDG key register (IWDG_KR)*, the counter starts counting down from the reset value of 0xFFFF. When it reaches the end of count value (0x000) a reset signal is generated (IWDG reset).

Whenever the key value 0x0000 AAAA is written in the *IWDG key register (IWDG_KR)*, the IWDG_RLR value is reloaded in the counter and the watchdog reset is prevented.

Once running, the IWDG cannot be stopped.

48.3.2 IWDG internal signals

Table 392 gives the list of IWDG internal signals.

Table 392. IWDG internal input/output signals

Signal name	Signal type	Description
lsi_ck	Digital input	LSI clock
iwdg1_out_rst, iwdg2_out_rst	Digital output	IWDG1 and IWDG2 reset signal outputs

48.3.3 Window option

The IWDG can also work as a window watchdog by setting the appropriate window in the *IWDG window register (IWDG_WINR)*.

If the reload operation is performed while the counter is greater than the value stored in the *IWDG window register (IWDG_WINR)*, then a reset is provided.

The default value of the *IWDG window register (IWDG_WINR)* is 0x0000 0FFF, so if it is not updated, the window option is disabled.

As soon as the window value is changed, a reload operation is performed in order to reset the downcounter to the *IWDG reload register (IWDG_RLR)* value and ease the cycle number calculation to generate the next reload.

Configuring the IWDG when the window option is enabled

1. Enable the IWDG by writing 0x0000 CCCC in the *IWDG key register (IWDG_KR)*.
2. Enable register access by writing 0x0000 5555 in the *IWDG key register (IWDG_KR)*.
3. Write the IWDG prescaler by programming *IWDG prescaler register (IWDG_PR)* from 0 to 7.
4. Write the *IWDG reload register (IWDG_RLR)*.
5. Wait for the registers to be updated (IWDG_SR = 0x0000 0000).
6. Write to the *IWDG window register (IWDG_WINR)*. This automatically refreshes the counter value in the *IWDG reload register (IWDG_RLR)*.

Note: Writing the window value allows the counter value to be refreshed by the RLR when *IWDG status register (IWDG_SR)* is set to 0x0000 0000.

Configuring the IWDG when the window option is disabled

When the window option it is not used, the IWDG can be configured as follows:

1. Enable the IWDG by writing 0x0000 CCCC in the *IWDG key register (IWDG_KR)*.
2. Enable register access by writing 0x0000 5555 in the *IWDG key register (IWDG_KR)*.
3. Write the prescaler by programming the *IWDG prescaler register (IWDG_PR)* from 0 to 7.
4. Write the *IWDG reload register (IWDG_RLR)*.
5. Wait for the registers to be updated (IWDG_SR = 0x0000 0000).
6. Refresh the counter value with IWDG_RLR (IWDG_KR = 0x0000 AAAA).

48.3.4 Hardware watchdog

If the “Hardware watchdog” feature is enabled through the device option bits, the watchdog is automatically enabled at power-on, and generates a reset unless the *IWDG key register (IWDG_KR)* is written by the software before the counter reaches end of count or if the downcounter is reloaded inside the window.

48.3.5 Low-power freeze

Depending on the IWDG_FZ_STOP and IWDG_FZ_STBY options configuration, the IWDG can continue counting or not during the Stop mode and the Standby mode, respectively. If the IWDG is kept running during Stop or Standby modes, it can wake up the device from this mode. Refer to [Section 4.4.5: Description of user and system option bytes](#) for more details.

48.3.6 Register access protection

Write access to *IWDG prescaler register (IWDG_PR)*, *IWDG reload register (IWDG_RLR)* and *IWDG window register (IWDG_WINR)* is protected. To modify them, the user must first write the code 0x0000 5555 in the *IWDG key register (IWDG_KR)*. A write access to this register with a different value breaks the sequence and register access is protected again. This is the case of the reload operation (writing 0x0000 AAAA).

A status register is available to indicate that an update of the prescaler or of the downcounter reload value or of the window value is ongoing.

48.3.7 Debug mode

When the device enters Debug mode (core halted), the IWDG counter either continues to work normally or stops, depending on the configuration of the corresponding bit in DBGMCU freeze register.

48.4 IWDG registers

Refer to [Section 1.2 on page 106](#) for a list of abbreviations used in register descriptions.

The peripheral registers can be accessed by half-words (16-bit) or words (32-bit).

48.4.1 IWDG key register (IWDG_KR)

Address offset: 0x00

Reset value: 0x0000 0000 (reset by Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **KEY[15:0]**: Key value (write only, read 0x0000)

These bits must be written by software at regular intervals with the key value 0xAAAA, otherwise the watchdog generates a reset when the counter reaches 0.

Writing the key value 0x5555 to enable access to the IWDG_PR, IWDG_RLR and IWDG_WINR registers (see [Section 48.3.6: Register access protection](#))

Writing the key value 0xCCCC starts the watchdog (except if the hardware watchdog option is selected)

48.4.2 IWDG prescaler register (IWDG_PR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR[2:0]		
													rw	rw	rw

Bits 31:3 Reserved, must be kept at reset value.

Bits 2:0 **PR[2:0]**: Prescaler divider

These bits are write access protected see [Section 48.3.6: Register access protection](#). They are written by software to select the prescaler divider feeding the counter clock. PVU bit of the [IWDG status register \(IWDG_SR\)](#) must be reset in order to be able to change the prescaler divider.

000: divider /4

001: divider /8

010: divider /16

011: divider /32

100: divider /64

101: divider /128

110: divider /256

111: divider /256

Note: Reading this register returns the prescaler value from the V_{DD} voltage domain. This value may not be up to date/valid if a write operation to this register is ongoing. For this reason the value read from this register is valid only when the PVU bit in the [IWDG status register \(IWDG_SR\)](#) is reset.

48.4.3 IWDG reload register (IWDG_RLR)

Address offset: 0x08

Reset value: 0x0000 0FFF (reset by Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	RL[11:0]											
				r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:12 Reserved, must be kept at reset value.

Bits 11:0 **RL[11:0]**: Watchdog counter reload value

These bits are write access protected see [Register access protection](#). They are written by software to define the value to be loaded in the watchdog counter each time the value 0xAAAA is written in the [IWDG key register \(IWDG_KR\)](#). The watchdog counter counts down from this value. The timeout period is a function of this value and the clock prescaler. Refer to the datasheet for the timeout information.

The RVU bit in the [IWDG status register \(IWDG_SR\)](#) must be reset to be able to change the reload value.

Note: Reading this register returns the reload value from the V_{DD} voltage domain. This value may not be up to date/valid if a write operation to this register is ongoing on it. For this reason the value read from this register is valid only when the RVU bit in the [IWDG status register \(IWDG_SR\)](#) is reset.

48.4.4 IWDG status register (IWDG_SR)

Address offset: 0x0C

Reset value: 0x0000 0000 (not reset by Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WVU	RVU	PVU
													r	r	r

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 WVU: Watchdog counter window value update

This bit is set by hardware to indicate that an update of the window value is ongoing. It is reset by hardware when the reload value update operation is completed in the V_{DD} voltage domain (takes up to five RC 40 kHz cycles).

Window value can be updated only when WVU bit is reset.

Bit 1 RVU: Watchdog counter reload value update

This bit is set by hardware to indicate that an update of the reload value is ongoing. It is reset by hardware when the reload value update operation is completed in the V_{DD} voltage domain (takes up to five RC 40 kHz cycles).

Reload value can be updated only when RVU bit is reset.

Bit 0 PVU: Watchdog prescaler value update

This bit is set by hardware to indicate that an update of the prescaler value is ongoing. It is reset by hardware when the prescaler update operation is completed in the V_{DD} voltage domain (takes up to five RC 40 kHz cycles).

Prescaler value can be updated only when PVU bit is reset.

Note: *If several reload, prescaler, or window values are used by the application, it is mandatory to wait until RVU bit is reset before changing the reload value, to wait until PVU bit is reset before changing the prescaler value, and to wait until WVU bit is reset before changing the window value. However, after updating the prescaler and/or the reload/window value it is not necessary to wait until RVU or PVU or WVU is reset before continuing code execution except in case of low-power mode entry.*

48.4.5 IWDG window register (IWDG_WINR)

Address offset: 0x10

Reset value: 0x0000 0FFF (reset by Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	WIN[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:12 Reserved, must be kept at reset value.

Bits 11:0 **WIN[11:0]**: Watchdog counter window value

These bits are write access protected, see [Section 48.3.6](#), they contain the high limit of the window value to be compared with the downcounter.

To prevent a reset, the downcounter must be reloaded when its value is lower than the window register value and greater than 0x0

The WVU bit in the [IWDG status register \(IWDG_SR\)](#) must be reset in order to be able to change the reload value.

Note: Reading this register returns the reload value from the V_{DD} voltage domain. This value may not be valid if a write operation to this register is ongoing. For this reason the value read from this register is valid only when the WVU bit in the [IWDG status register \(IWDG_SR\)](#) is reset.

48.4.6 IWDG register map

The following table gives the IWDG register map and reset values.

Table 393. IWDG register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	IWDG_KR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	KEY[15:0]																
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x04	IWDG_PR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PR[2:0]		
	Reset value																														0	0	0	
0x08	IWDG_RLR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	RL[11:0]												
	Reset value																					1	1	1	1	1	1	1	1	1	1	1	1	
0x0C	IWDG_SR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WVU	RVU	PVU
	Reset value																														0	0	0	
0x10	IWDG_WINR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WIN[11:0]												
	Reset value																					1	1	1	1	1	1	1	1	1	1	1	1	

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

51 Universal synchronous/asynchronous receiver transmitter (USART/UART)

This section describes the universal synchronous asynchronous receiver transmitter (USART).

51.1 USART introduction

The USART offers a flexible means to perform Full-duplex data exchange with external equipments requiring an industry standard NRZ asynchronous serial data format. A very wide range of baud rates can be achieved through a fractional baud rate generator.

The USART supports both synchronous one-way and Half-duplex Single-wire communications, as well as LIN (local interconnection network), Smartcard protocol, IrDA (infrared data association) SIR ENDEC specifications, and Modem operations (CTS/RTS). Multiprocessor communications are also supported.

High-speed data communications are possible by using the DMA (direct memory access) for multibuffer configuration.

51.2 USART main features

- Full-duplex asynchronous communication
- NRZ standard format (mark/space)
- Configurable oversampling method by 16 or 8 to achieve the best compromise between speed and clock tolerance
- Baud rate generator systems
- Two internal FIFOs for transmit and receive data
Each FIFO can be enabled/disabled by software and come with a status flag.
- A common programmable transmit and receive baud rate
- Dual clock domain with dedicated kernel clock for peripherals independent from PCLK
- Auto baud rate detection
- Programmable data word length (7, 8 or 9 bits)
- Programmable data order with MSB-first or LSB-first shifting
- Configurable stop bits (1 or 2 stop bits)
- Synchronous master/slave mode and clock output/input for synchronous communications
- SPI slave transmission underrun error flag
- Single-wire Half-duplex communications
- Continuous communications using DMA
- Received/transmitted bytes are buffered in reserved SRAM using centralized DMA.
- Separate enable bits for transmitter and receiver
- Separate signal polarity control for transmission and reception
- Swappable Tx/Rx pin configuration
- Hardware flow control for modem and RS-485 transceiver
- Communication control/error detection flags
- Parity control:
 - Transmits parity bit
 - Checks parity of received data byte
- Interrupt sources with flags
- Multiprocessor communications: wake-up from Mute mode by idle line detection or address mark detection
- Wake-up from Stop mode

51.3 USART extended features

- LIN master synchronous break send capability and LIN slave break detection capability
 - 13-bit break generation and 10/11 bit break detection when USART is hardware configured for LIN
- IrDA SIR encoder decoder supporting 3/16 bit duration for normal mode
- Smartcard mode
 - Supports the T = 0 and T = 1 asynchronous protocols for smartcards as defined in the ISO/IEC 7816-3 standard
 - 0.5 and 1.5 stop bits for Smartcard operation
- Support for Modbus communication
 - Timeout feature
 - CR/LF character recognition

51.4 USART implementation

The table(s) below describe(s) USART implementation. It(they) also include(s) LPUART for comparison.

Table 419. USART / LPUART features

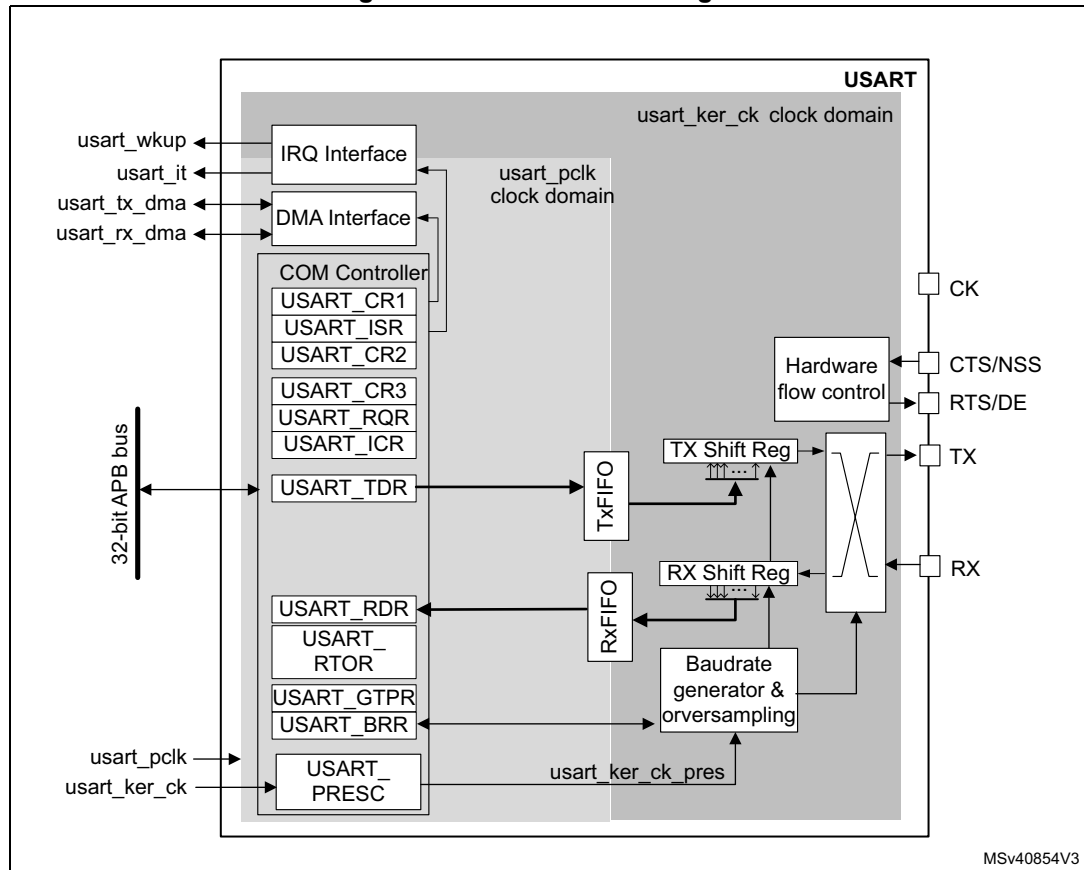
USART / LPUART modes/features ⁽¹⁾	USART1/2/3/6	UART4/5/7/8	LPUART1
Hardware flow control for modem	X	X	X
Continuous communication using DMA	X	X	X
Multiprocessor communication	X	X	X
Synchronous mode (Master/Slave)	X	-	-
Smartcard mode	X	-	-
Single-wire Half-duplex communication	X	X	X
IrDA SIR ENDEC block	X	X	-
LIN mode	X	X	-
Dual clock domain and wake-up from low-power mode	X	X	X
Receiver timeout interrupt	X	X	-
Modbus communication	X	X	-
Auto baud rate detection	X	X	-
Driver Enable	X	X	X
USART data length	7, 8 and 9 bits		
Tx/Rx FIFO	X	X	X
Tx/Rx FIFO size	16		

1. X = supported.

51.5 USART functional description

51.5.1 USART block diagram

Figure 613. USART block diagram



The simplified block diagram given in [Figure 613](#) shows two fully-independent clock domains:

- The **usart_pclk** clock domain
The **usart_pclk** clock signal feeds the peripheral bus interface. It must be active when accesses to the USART registers are required.
- The **usart_ker_ck** kernel clock domain.
The **usart_ker_ck** is the USART clock source. It is independent from **usart_pclk** and delivered by the RCC. The USART registers can consequently be written/read even when the **usart_ker_ck** clock is stopped.
When the dual clock domain feature is disabled, the **usart_ker_ck** clock is the same as the **usart_pclk** clock.

There is no constraint between **usart_pclk** and **usart_ker_ck**: **usart_ker_ck** can be faster or slower than **usart_pclk**. The only limitation is the software ability to manage the communication fast enough.

When the USART operates in SPI slave mode, it handles data flow using the serial interface clock derived from the external CK signal provided by the external master SPI device. The **usart_ker_ck** clock must be at least 3 times faster than the clock on the CK input.

51.5.2 USART signals

USART bidirectional communications

USART bidirectional communications require a minimum of two pins: Receive Data In (RX) and Transmit Data Out (TX):

- **RX** (Receive Data Input)
RX is the serial data input. Oversampling techniques are used for data recovery. They discriminate between valid incoming data and noise.
- **TX** (Transmit Data Output)
When the transmitter is disabled, the output pin returns to its I/O port configuration. When the transmitter is enabled and no data needs to be transmitted, the TX pin is High. In Single-wire and Smartcard modes, this I/O is used to transmit and receive data.

RS232 Hardware flow control mode

The following pins are required in RS232 Hardware flow control mode:

- **CTS** (Clear To Send)
When driven high, this signal blocks the data transmission at the end of the current transfer.
- **RTS** (Request To Send)
When it is low, this signal indicates that the USART is ready to receive data.

RS485 Hardware control mode

The following pin is required in RS485 Hardware control mode:

- **DE** (Driver Enable)
This signal activates the transmission mode of the external transceiver.

Note: DE and RTS share the same pin.

Synchronous master/slave mode and Smartcard mode

The following pin is required in synchronous master/slave mode and Smartcard mode:

- **CK**
This pin acts as Clock output in Synchronous master and Smartcard modes.
It acts as Clock input in Synchronous slave mode.
In Synchronous Master mode, this pin outputs the transmitter data clock for synchronous transmission corresponding to SPI master mode (no clock pulses on start bit and stop bit, and a software option to send a clock pulse on the last data bit). In parallel, data can be received synchronously on RX pin. This mechanism can be used to control peripherals featuring shift registers (e.g. LCD drivers). The clock phase and polarity are software programmable.
In Smartcard mode, CK output provides the clock to the smartcard.
- **NSS**
This pin acts as Slave Select input in Synchronous slave mode.

Note: NSS and CTS share the same pin.

51.5.3 USART character description

The word length can be set to 7, 8 or 9 bits, by programming the M bits (M0: bit 12 and M1: bit 28) in the USART_CR1 register (see [Figure 614](#)):

- 7-bit character length: M[1:0] = '10'
- 8-bit character length: M[1:0] = '00'
- 9-bit character length: M[1:0] = '01'

Note: In 7-bit data length mode, the Smartcard mode, LIN master mode and auto baud rate (0x7F and 0x55 frames detection) are not supported.

By default, the signal (TX or RX) is in low state during the start bit. It is in high state during the stop bit.

These values can be inverted, separately for each signal, through polarity configuration control.

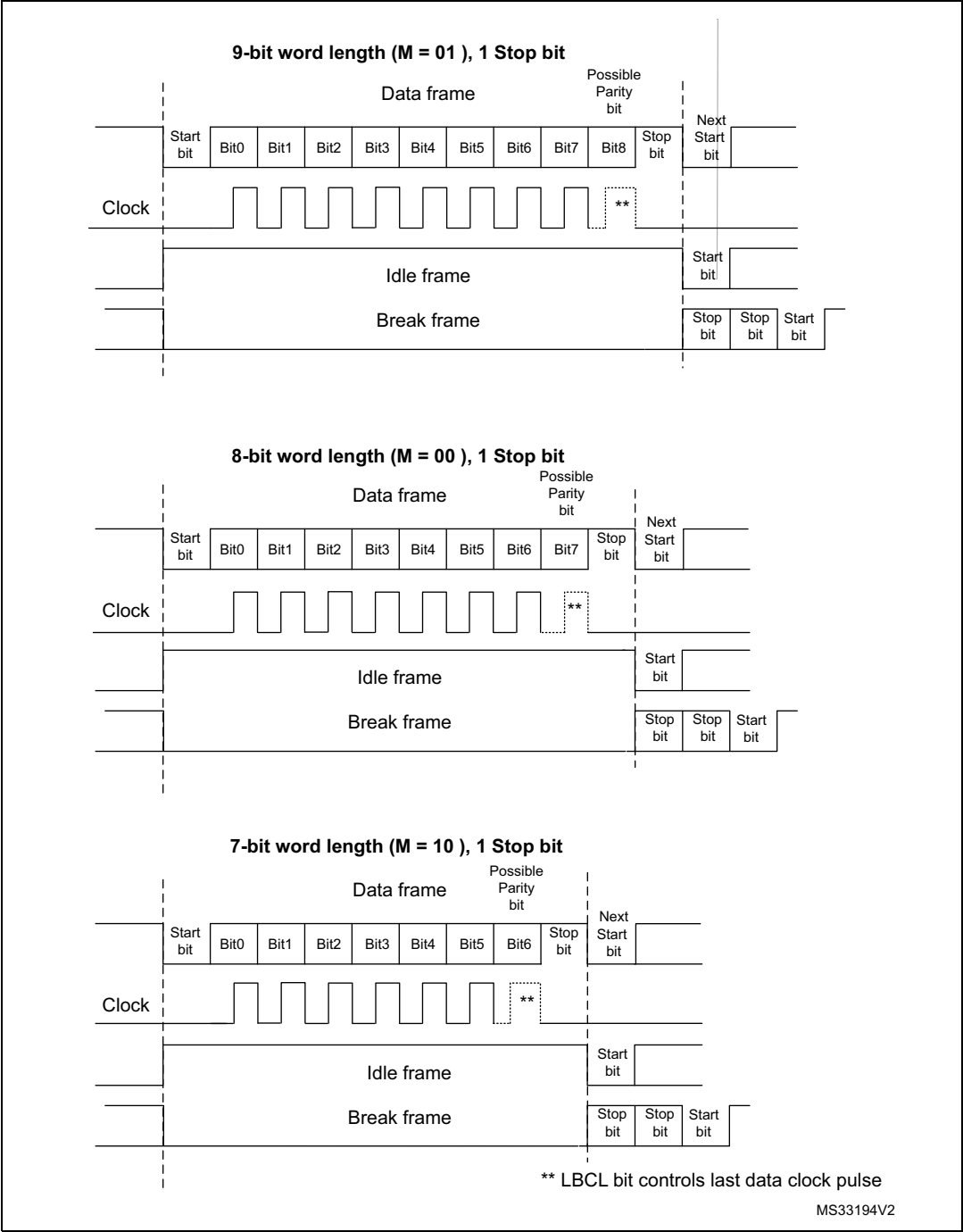
An **Idle character** is interpreted as an entire frame of "1"s (the number of "1"s includes the number of stop bits).

A **Break character** is interpreted on receiving "0"s for a frame period. At the end of the break frame, the transmitter inserts 2 stop bits.

Transmission and reception are driven by a common baud rate generator. The transmission and reception clock are generated when the enable bit is set for the transmitter and receiver, respectively.

A detailed description of each block is given below.

Figure 614. Word length programming



51.5.4 USART FIFOs and thresholds

The USART can operate in FIFO mode.

The USART comes with a Transmit FIFO (TXFIFO) and a Receive FIFO (RXFIFO). The FIFO mode is enabled by setting FIFOEN in USART_CR1 register (bit 29). This mode is supported only in UART, SPI and Smartcard modes.

Since the maximum data word length is 9 bits, the TXFIFO is 9-bit wide. However the RXFIFO default width is 12 bits. This is due to the fact that the receiver does not only store the data in the FIFO, but also the error flags associated to each character (Parity error, Noise error and Framing error flags).

Note: The received data is stored in the RXFIFO together with the corresponding flags. However, only the data are read when reading the RDR.

The status flags are available in the USART_ISR register.

It is possible to configure the TXFIFO and RXFIFO levels at which the Tx and RX interrupts are triggered. These thresholds are programmed through RXFTCFG and TXFTCFG bitfields in USART_CR3 control register.

In this case:

- The RXFT flag is set in the USART_ISR register and the corresponding interrupt (if enabled) is generated, when the number of received data in the RXFIFO reaches the threshold programmed in the RXFTCFG bits fields.
This means that the RXFIFO is filled until the number of data in the RXFIFO is equal to the programmed threshold.
RXFTCFG data have been received: one data in USART_RDR and (RXFTCFG - 1) data in the RXFIFO. As an example, when the RXFTCFG is programmed to '101', the RXFT flag is set when a number of data corresponding to the FIFO size has been received (FIFO size - 1 data in the RXFIFO and 1 data in the USART_RDR). As a result, the next received data is not set the overrun flag.
- The TXFT flag is set in the USART_ISR register and the corresponding interrupt (if enabled) is generated when the number of empty locations in the TXFIFO reaches the threshold programmed in the TXFTCFG bits fields.
This means that the TXFIFO is emptied until the number of empty locations in the TXFIFO is equal to the programmed threshold.

51.5.5 USART transmitter

The transmitter can send data words of either 7 or 8 or 9 bits, depending on the M bit status. The Transmit Enable bit (TE) must be set in order to activate the transmitter function. The data in the transmit shift register is output on the TX pin while the corresponding clock pulses are output on the CK pin.

Character transmission

During an USART transmission, data shifts out the least significant bit first (default configuration) on the TX pin. In this mode, the USART_TDR register consists of a buffer (TDR) between the internal bus and the transmit shift register.

When FIFO mode is enabled, the data written to the transmit data register (USART_TDR) are queued in the TXFIFO.

Every character is preceded by a start bit which corresponds to a low logic level for one bit period. The character is terminated by a configurable number of stop bits.

The number of stop bits can be configured to 0.5, 1, 1.5 or 2.

Note: *The TE bit must be set before writing the data to be transmitted to the USART_TDR.*
The TE bit should not be reset during data transmission. Resetting the TE bit during the transmission corrupts the data on the TX pin as the baud rate counters get frozen. The current data being transmitted are then lost.
An idle frame is sent when the TE bit is enabled.

Configurable stop bits

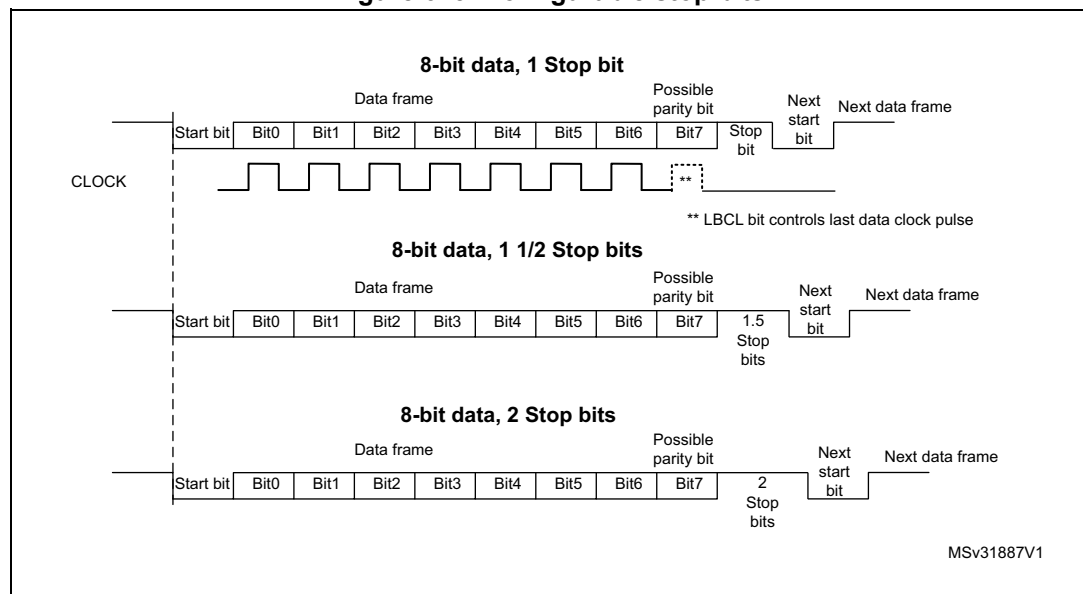
The number of stop bits to be transmitted with every character can be programmed in USART_CR2, bits 13,12.

- **1 stop bit:** This is the default value of number of stop bits.
- **2 stop bits:** This is supported by normal USART, Single-wire and Modem modes.
- **1.5 stop bits:** To be used in Smartcard mode.

An idle frame transmission includes the stop bits.

A break transmission features 10 low bits (when M[1:0] = '00') or 11 low bits (when M[1:0] = '01') or 9 low bits (when M[1:0] = '10') followed by 2 stop bits (see [Figure 615](#)). It is not possible to transmit long breaks (break of length greater than 9/10/11 low bits).

Figure 615. Configurable stop bits



Character transmission procedure

To transmit a character, follow the sequence below:

1. Program the M bits in USART_CR1 to define the word length.
2. Select the desired baud rate using the USART_BRR register.
3. Program the number of stop bits in USART_CR2.
4. Enable the USART by writing the UE bit in USART_CR1 register to 1.
5. Select DMA enable (DMAT) in USART_CR3 if multibuffer communication must take place. Configure the DMA register as explained in [Section 51.5.19: Continuous communication using USART and DMA](#).
6. Set the TE bit in USART_CR1 to send an idle frame as first transmission.
7. Write the data to send in the USART_TDR register. Repeat this for each data to be transmitted in case of single buffer.
 - When FIFO mode is disabled, writing a data to the USART_TDR clears the TXE flag.
 - When FIFO mode is enabled, writing a data to the USART_TDR adds one data to the TXFIFO. Write operations to the USART_TDR are performed when TXFNF flag is set. This flag remains set until the TXFIFO is full.
8. When the last data is written to the USART_TDR register, wait until TC = 1.
 - When FIFO mode is disabled, this indicates that the transmission of the last frame is complete.
 - When FIFO mode is enabled, this indicates that both TXFIFO and shift register are empty.

This check is required to avoid corrupting the last transmission when the USART is disabled or enters Halt mode.

Single byte communication

- When FIFO mode is disabled

Writing to the transmit data register always clears the TXE bit. The TXE flag is set by hardware. It indicates that:

- the data have been moved from the USART_TDR register to the shift register and the data transmission has started;
- the USART_TDR register is empty;
- the next data can be written to the USART_TDR register without overwriting the previous data.

This flag generates an interrupt if the TXEIE bit is set.

When a transmission is ongoing, a write instruction to the USART_TDR register stores the data in the TDR buffer. It is then copied in the shift register at the end of the current transmission.

When no transmission is ongoing, a write instruction to the USART_TDR register places the data in the shift register, the data transmission starts, and the TXE bit is set.

- When FIFO mode is enabled, the TXFNF (TXFIFO not full) flag is set by hardware to indicate that:

- the TXFIFO is not full;
- the USART_TDR register is empty;
- the next data can be written to the USART_TDR register without overwriting the previous data. When a transmission is ongoing, a write operation to the USART_TDR register stores the data in the TXFIFO. Data are copied from the TXFIFO to the shift register at the end of the current transmission.

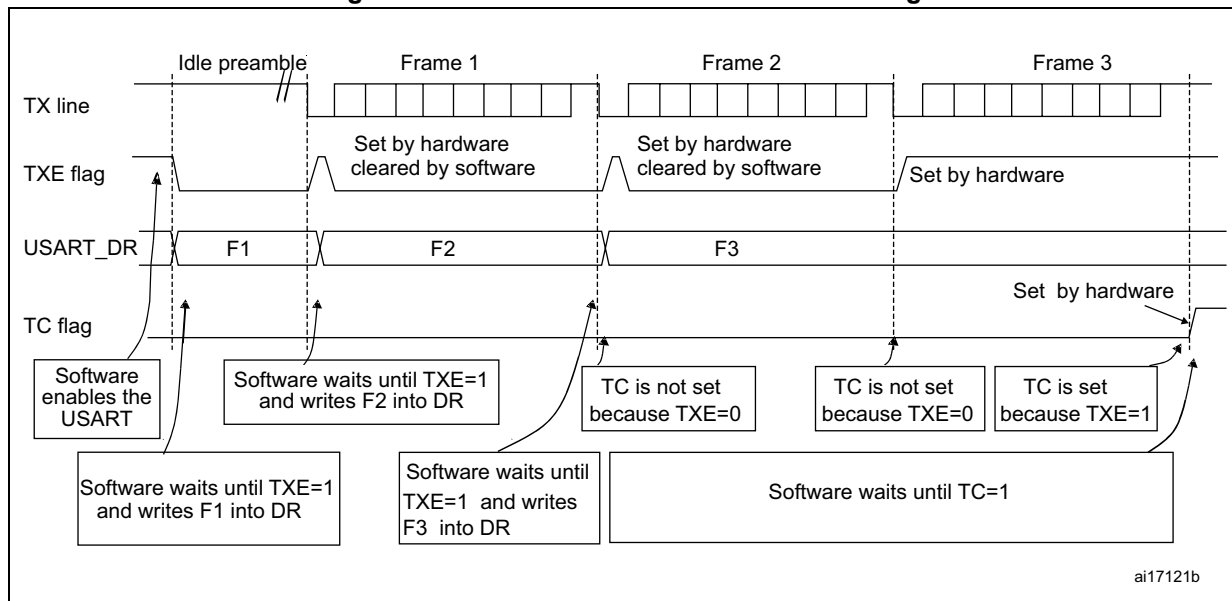
When the TXFIFO is not full, the TXFNF flag stays at '1' even after a write operation to USART_TDR register. It is cleared when the TXFIFO is full. This flag generates an interrupt if the TXFNFIE bit is set.

Alternatively, interrupts can be generated and data can be written to the FIFO when the TXFIFO threshold is reached. In this case, the CPU can write a block of data defined by the programmed trigger level.

If a frame is transmitted (after the stop bit) and the TXE flag (TXFE in case of FIFO mode) is set, the TC flag goes high. An interrupt is generated if the TCIE bit is set in the USART_CR1 register.

After writing the last data to the USART_TDR register, it is mandatory to wait until TC is set before disabling the USART or causing the device to enter the low-power mode (see [Figure 616: TC/TXE behavior when transmitting](#)).

Figure 616. TC/TXE behavior when transmitting



Note: When FIFO management is enabled, the TXFNF flag is used for data transmission.

Break characters

Setting the SBKRQ bit transmits a break character. The break frame length depends on the M bit (see [Figure 614](#)).

If a '1' is written to the SBKRQ bit, a break character is sent on the TX line after completing the current character transmission. The SBKF bit is set by the write operation and it is reset by hardware when the break character is completed (during the stop bits after the break character). The USART inserts a logic 1 signal (stop) for the duration of 2 bits at the end of the break frame to guarantee the recognition of the start bit of the next frame.

When the SBKRQ bit is set, the break character is sent at the end of the current transmission.

When FIFO mode is enabled, sending the break character has priority on sending data even if the TXFIFO is full.

Idle characters

Setting the TE bit drives the USART to send an idle frame before the first data frame.

51.5.6 USART receiver

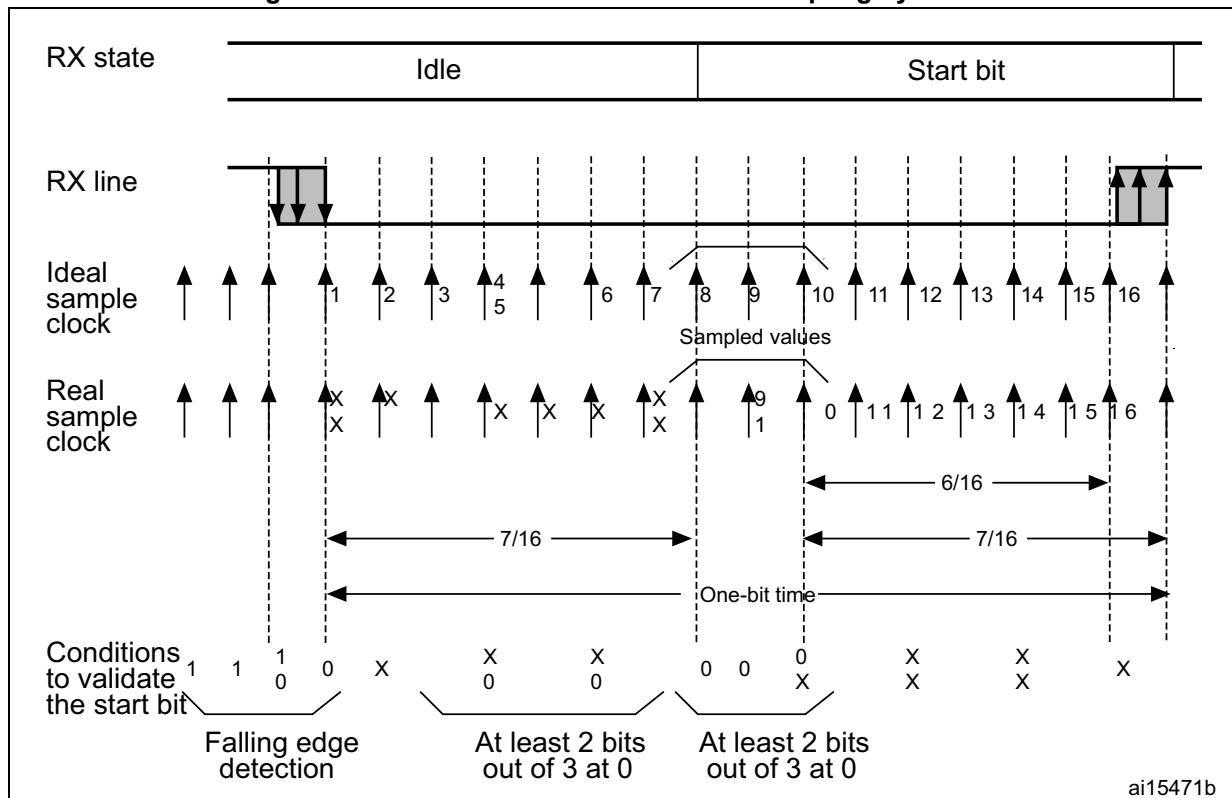
The USART can receive data words of either 7 or 8 or 9 bits depending on the M bits in the USART_CR1 register.

Start bit detection

The start bit detection sequence is the same when oversampling by 16 or by 8.

In the USART, the start bit is detected when a specific sequence of samples is recognized. This sequence is: 1 1 1 0 X 0 X 0X 0X 0 X 0X 0.

Figure 617. Start bit detection when oversampling by 16 or 8



Note: If the sequence is not complete, the start bit detection aborts and the receiver returns to the idle state (no flag is set), where it waits for a falling edge.

The start bit is confirmed (RXNE flag set and interrupt generated if RXNEIE = 1, or RXFNE flag set and interrupt generated if RXFNEIE = 1 if FIFO mode enabled) if the 3 sampled bits are at '0' (first sampling on the 3rd, 5th and 7th bits finds the 3 bits at '0' and second sampling on the 8th, 9th and 10th bits also finds the 3 bits at '0').

The start bit is validated but the NE noise flag is set if,

- a) for both samplings, 2 out of the 3 sampled bits are at '0' (sampling on the 3rd, 5th and 7th bits and sampling on the 8th, 9th and 10th bits)
- or
- b) for one of the samplings (sampling on the 3rd, 5th and 7th bits or sampling on the 8th, 9th and 10th bits), 2 out of the 3 bits are found at '0'.

If neither of the above conditions are met, the start detection aborts and the receiver returns to the idle state (no flag is set).

Character reception

During an USART reception, data are shifted out least significant bit first (default configuration) through the RX pin.

Character reception procedure

To receive a character, follow the sequence below:

1. Program the M bits in USART_CR1 to define the word length.
2. Select the desired baud rate using the baud rate register USART_BRR
3. Program the number of stop bits in USART_CR2.
4. Enable the USART by writing the UE bit in USART_CR1 register to '1'.
5. Select DMA enable (DMAR) in USART_CR3 if multibuffer communication is to take place. Configure the DMA register as explained in [Section 51.5.19: Continuous communication using USART and DMA](#).
6. Set the RE bit USART_CR1. This enables the receiver which begins searching for a start bit.

When a character is received:

- When FIFO mode is disabled, the RXNE bit is set to indicate that the content of the shift register is transferred to the RDR. In other words, data have been received and can be read (as well as their associated error flags).
- When FIFO mode is enabled, the RXFNE bit is set to indicate that the RXFIFO is not empty. Reading the USART_RDR returns the oldest data entered in the RXFIFO. When a data is received, it is stored in the RXFIFO together with the corresponding error bits.
- An interrupt is generated if the RXNEIE (RXFNEIE when FIFO mode is enabled) bit is set.
- The error flags can be set if a frame error, noise, parity or an overrun error was detected during reception.
- In multibuffer communication mode:
 - When FIFO mode is disabled, the RXNE flag is set after every byte reception. It is cleared when the DMA reads the Receive data Register.
 - When FIFO mode is enabled, the RXFNE flag is set when the RXFIFO is not empty. After every DMA request, a data is retrieved from the RXFIFO. A DMA request is triggered when the RXFIFO is not empty i.e. when there are data to be read from the RXFIFO.
- In single buffer mode:
 - When FIFO mode is disabled, clearing the RXNE flag is done by performing a software read from the USART_RDR register. The RXNE flag can also be cleared by programming RXFRQ bit to '1' in the USART_RQR register. The RXNE flag must be cleared before the end of the reception of the next character to avoid an overrun error.
 - When FIFO mode is enabled, the RXFNE is set when the RXFIFO is not empty. After every read operation from USART_RDR, a data is retrieved from the RXFIFO. When the RXFIFO is empty, the RXFNE flag is cleared. The RXFNE flag can also be cleared by programming RXFRQ bit to '1' in USART_RQR. When the RXFIFO is full, the first entry in the RXFIFO must be read before the end of the reception of the next character, to avoid an overrun error. The RXFNE flag generates an interrupt if the RXFNEIE bit is set. Alternatively, interrupts can be

generated and data can be read from RXFIFO when the RXFIFO threshold is reached. In this case, the CPU can read a block of data defined by the programmed threshold.

Break character

When a break character is received, the USART handles it as a framing error.

Idle character

When an idle frame is detected, it is handled in the same way as a data character reception except that an interrupt is generated if the IDLEIE bit is set.

Overrun error

- FIFO mode disabled

An overrun error occurs if a character is received and RXNE has not been reset. Data can not be transferred from the shift register to the RDR register until the RXNE bit is cleared. The RXNE flag is set after every byte reception.

An overrun error occurs if RXNE flag is set when the next data is received or the previous DMA request has not been serviced. When an overrun error occurs:

 - the ORE bit is set;
 - the RDR content is not lost. The previous data is available by reading the USART_RDR register.
 - the shift register is overwritten. After that, any data received during overrun is lost.
 - an interrupt is generated if either the RXNEIE or the EIE bit is set.
- FIFO mode enabled

An overrun error occurs when the shift register is ready to be transferred and the receive FIFO is full.

Data can not be transferred from the shift register to the USART_RDR register until there is one free location in the RXFIFO. The RXFNE flag is set when the RXFIFO is not empty.

An overrun error occurs if the RXFIFO is full and the shift register is ready to be transferred. When an overrun error occurs:

 - The ORE bit is set.
 - The first entry in the RXFIFO is not lost. It is available by reading the USART_RDR register.
 - The shift register is overwritten. After that point, any data received during overrun is lost.
 - An interrupt is generated if either the RXFNEIE or EIE bit is set.

The ORE bit is reset by setting the ORECF bit in the USART_ICR register.

Note: The ORE bit, when set, indicates that at least 1 data has been lost.

When the FIFO mode is disabled, there are two possibilities

- *if RXNE = 1, then the last valid data is stored in the receive register (RDR) and can be read,*
- *if RXNE = 0, the last valid data has already been read and there is nothing left to be read in the RDR register. This case can occur when the last valid data is read in the RDR register at the same time as the new (and lost) data is received.*

Selecting the clock source and the appropriate oversampling method

The choice of the clock source is done through the Clock Control system (see Section *Reset and clock control (RCC)*). The clock source must be selected through the UE bit before enabling the USART.

The clock source must be selected according to two criteria:

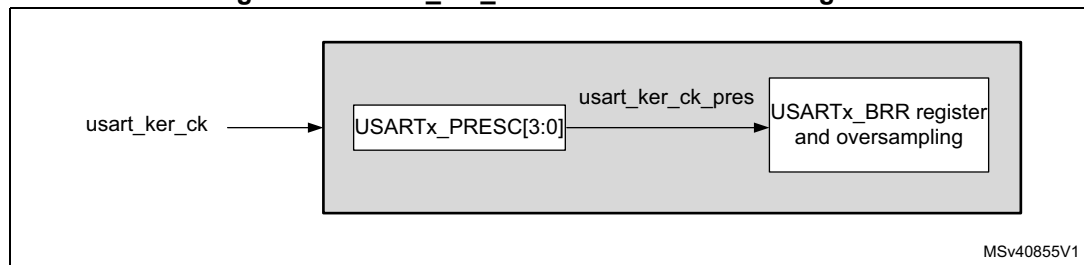
- Possible use of the USART in low-power mode
- Communication speed.

The clock source frequency is `usart_ker_ck`.

When the dual clock domain and the wake-up from low-power mode features are supported, the `usart_ker_ck` clock source can be configurable in the RCC (see Section *Reset and clock control (RCC)*). Otherwise the `usart_ker_ck` clock is the same as `usart_pclk`.

The `usart_ker_ck` clock can be divided by a programmable factor, defined in the USART_PRESC register.

Figure 618. usart_ker_ck clock divider block diagram



Some `usart_ker_ck` sources enable the USART to receive data while the MCU is in low-power mode. Depending on the received data and wake-up mode selected, the USART wakes up the MCU, when needed, in order to transfer the received data, by performing a software read to the USART_RDR register or by DMA.

For the other clock sources, the system must be active to enable USART communications.

The communication speed range (specially the maximum communication speed) is also determined by the clock source.

The receiver implements different user-configurable oversampling techniques (except in synchronous mode) for data recovery by discriminating between valid incoming data and noise. This enables obtaining the best a trade-off between the maximum communication speed and noise/clock inaccuracy immunity.

The oversampling method can be selected by programming the OVER8 bit in the USART_CR1 register either to 16 or 8 times the baud rate clock (see [Figure 619](#) and [Figure 620](#)).

Depending on your application:

- select oversampling by 8 (OVER8 = 1) to achieve higher speed (up to `usart_ker_ck_pres/8`). In this case the maximum receiver tolerance to clock deviation is reduced (refer to [Section 51.5.8: Tolerance of the USART receiver to clock deviation on page 2189](#))
- select oversampling by 16 (OVER8 = 0) to increase the tolerance of the receiver to clock deviations. In this case, the maximum speed is limited to maximum

$\text{usart_ker_ck_pres}/16$ (where usart_ker_ck_pres is the USART input clock divided by a prescaler).

Programming the ONEBIT bit in the USART_CR3 register selects the method used to evaluate the logic level. Two options are available:

- The majority vote of the three samples in the center of the received bit. In this case, when the 3 samples used for the majority vote are not equal, the NE bit is set.
- A single sample in the center of the received bit

Depending on your application:

- select the three sample majority vote method (ONEBIT = 0) when operating in a noisy environment and reject the data when a noise is detected (refer to [Figure 420](#)) because this indicates that a glitch occurred during the sampling.
- select the single sample method (ONEBIT = 1) when the line is noise-free to increase the receiver tolerance to clock deviations (see [Section 51.5.8: Tolerance of the USART receiver to clock deviation on page 2189](#)). In this case the NE bit is never set.

When noise is detected in a frame:

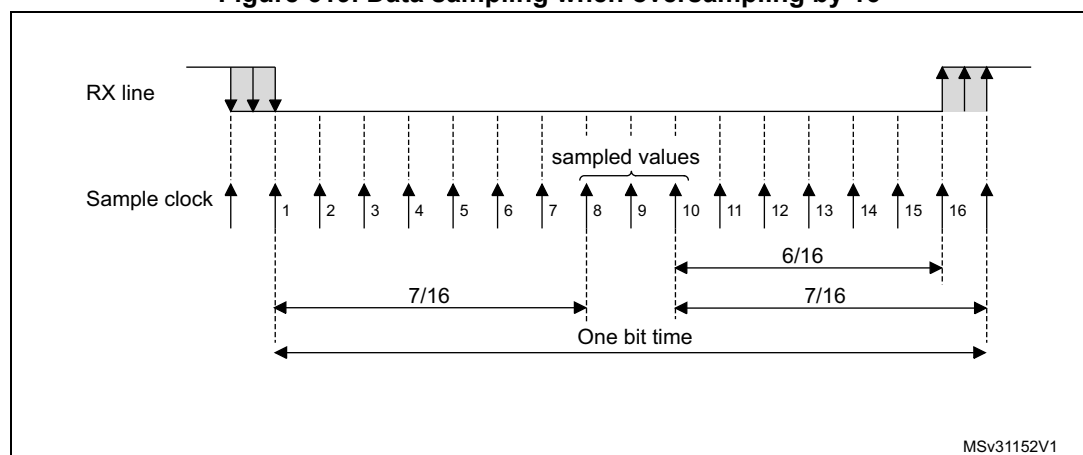
- The NE bit is set at the rising edge of the RXNE bit (RXFNE in case of FIFO mode enabled).
- The invalid data is transferred from the Shift register to the USART_RDR register.
- No interrupt is generated in case of single byte communication. However this bit rises at the same time as the RXNE bit (RXFNE in case of FIFO mode enabled) which itself generates an interrupt. In case of multibuffer communication an interrupt is issued if the EIE bit is set in the USART_CR3 register.

The NE bit is reset by setting NECF bit in USART_ICR register.

Note: Noise error is not supported in SPI mode.

Oversampling by 8 is not available in the Smartcard, IrDA and LIN modes. In those modes, the OVER8 bit is forced to '0' by hardware.

Figure 619. Data sampling when oversampling by 16



The diagram illustrates the timing relationship between the RX line and the Sample clock (x8). The RX line is shown as a horizontal line with three shaded regions: the first two samples (1 and 2) are high, the third sample (3) is low, and the last three samples (7, 8, and the final sample) are high. The Sample clock (x8) is shown as a series of vertical arrows labeled 1 through 8. The time intervals are defined as follows: the first three samples (1, 2, 3) take a total time of $3/8$; the next three samples (4, 5, 6) take a total time of $3/8$; and the last two samples (7, 8) take a total time of $2/8$. The total time for all eight samples is labeled as "One bit time".

Sampled value	NE status	Received bit value
000	0	0
001	1	0
010	1	0
011	1	1
100	1	0
101	1	1
110	1	1
111	0	1

Configurable stop bits during reception

The number of stop bits to be received can be configured through the control bits of USART_CR: it can be either 1 or 2 in normal mode and 0.5 or 1.5 in Smartcard mode.

- **0.5 stop bit (reception in Smartcard mode):** no sampling is done for 0.5 stop bit. As a consequence, no framing error and no break frame can be detected when 0.5 stop bit is selected.
- **1 stop bit:** sampling for 1 stop bit is done on the 8th, 9th and 10th samples.
- **1.5 stop bits (Smartcard mode)**

When transmitting in Smartcard mode, the device must check that the data are correctly sent. The receiver block must consequently be enabled (RE = 1 in USART_CR1) and the stop bit is checked to test if the Smartcard has detected a parity error.

In the event of a parity error, the Smartcard forces the data signal low during the sampling (NACK signal), which is flagged as a framing error. The FE flag is then set through RXNE flag (RXFNE if the FIFO mode is enabled) at the end of the 1.5 stop bit. Sampling for 1.5 stop bits is done on the 16th, 17th and 18th samples (1 baud clock period after the beginning of the stop bit). The 1.5 stop bit can be broken into 2 parts: one 0.5 baud clock period during which nothing happens, followed by 1 normal stop bit period during which sampling occurs halfway through (refer to [Section 51.5.16: USART receiver timeout on page 2203](#) for more details).

- **2 stop bits**
Sampling for 2 stop bits is done on the 8th, 9th and 10th samples of the first stop bit. The framing error flag is set if a framing error is detected during the first stop bit. The second stop bit is not checked for framing error. The RXNE flag (RXFNE if the FIFO mode is enabled) is set at the end of the first stop bit.

51.5.7 USART baud rate generation

The baud rate for the receiver and transmitter (Rx and Tx) are both set to the value programmed in the USART_BRR register.

Equation 1: baud rate for standard USART (SPI mode included) (OVER8 = '0' or '1')

In case of oversampling by 16, the baud rate is given by the following formula:

$$\text{Tx/Rx baud} = \frac{\text{usart_ker_ckpres}}{\text{USARTDIV}}$$

In case of oversampling by 8, the baud rate is given by the following formula:

$$\text{Tx/Rx baud} = \frac{2 \times \text{usart_ker_ckpres}}{\text{USARTDIV}}$$

Equation 2: baud rate in Smartcard, LIN and IrDA modes (OVER8 = 0)

The baud rate is given by the following formula:

$$\text{Tx/Rx baud} = \frac{\text{usart_ker_ckpres}}{\text{USARTDIV}}$$

USARTDIV is an unsigned fixed point number that is coded on the USART_BRR register.

- When OVER8 = 0, BRR = USARTDIV.
- When OVER8 = 1
 - BRR[2:0] = USARTDIV[3:0] shifted 1 bit to the right.
 - BRR[3] must be kept cleared.
 - BRR[15:4] = USARTDIV[15:4]

Note: The baud counters are updated to the new value in the baud registers after a write operation to USART_BRR. Hence the baud rate register value should not be changed during communication.

In case of oversampling by 16 and 8, USARTDIV must be greater than or equal to 16.

How to derive USARTDIV from USART_BRR register values

Example 1

To obtain 9600 baud with usart_ker_ck_pres = 8 MHz:

- In case of oversampling by 16:

$$\text{USARTDIV} = 8\,000\,000/9600$$

$$\text{BRR} = \text{USARTDIV} = 0\text{d}833 = 0\text{x}0341$$
- In case of oversampling by 8:

$$\text{USARTDIV} = 2 * 8\,000\,000/9600$$

$$\text{USARTDIV} = 1666,66 \text{ (}0\text{d}1667 = 0\text{x}683\text{)}$$

$$\text{BRR}[3:0] = 0\text{x}3 \gg 1 = 0\text{x}1$$

$$\text{BRR} = 0\text{x}681$$

Example 2

To obtain 921.6 Kbaud with usart_ker_ck_pres = 48 MHz:

- In case of oversampling by 16:

$$\text{USARTDIV} = 48\,000\,000/921\,600$$

$$\text{BRR} = \text{USARTDIV} = 0\text{d}52 = 0\text{x}34$$
- In case of oversampling by 8:

$$\text{USARTDIV} = 2 * 48\,000\,000/921\,600$$

$$\text{USARTDIV} = 104 \text{ (}0\text{d}104 = 0\text{x}68\text{)}$$

$$\text{BRR}[3:0] = \text{USARTDIV}[3:0] \gg 1 = 0\text{x}8 \gg 1 = 0\text{x}4$$

$$\text{BRR} = 0\text{x}64$$

51.5.8 Tolerance of the USART receiver to clock deviation

The USART asynchronous receiver operates correctly only if the total clock system deviation is less than the tolerance of the USART receiver.

The causes which contribute to the total deviation are:

- DTRA: deviation due to the transmitter error (which also includes the deviation of the transmitter's local oscillator)
- DQUANT: error due to the baud rate quantization of the receiver
- DREC: deviation of the receiver local oscillator
- DTCL: deviation due to the transmission line (generally due to the transceivers which can introduce an asymmetry between the low-to-high transition timing and the high-to-low transition timing)

$$DTRA + DQUANT + DREC + DTCL + DWU < \text{USART receiver tolerance}$$

where

DWU is the error due to sampling point deviation when the wake-up from low-power mode is used.

when M[1:0] = 01:

$$DWU = \frac{t_{WUUSART}}{11 \times T_{bit}}$$

when M[1:0] = 00:

$$DWU = \frac{t_{WUUSART}}{10 \times T_{bit}}$$

when M[1:0] = 10:

$$DWU = \frac{t_{WUUSART}}{9 \times T_{bit}}$$

$t_{WUUSART}$ is the time between the detection of the start bit falling edge and the instant when the clock (requested by the peripheral) is ready and reaching the peripheral, and the regulator is ready.

The USART receiver can receive data correctly at up to the maximum tolerated deviation specified in [Table 421](#), [Table 422](#), depending on the following settings:

- 9-, 10- or 11-bit character length defined by the M bits in the USART_CR1 register
- Oversampling by 8 or 16 defined by the OVER8 bit in the USART_CR1 register
- Bits BRR[3:0] of USART_BRR register are equal to or different from 0000.
- Use of 1 bit or 3 bits to sample the data, depending on the value of the ONEBIT bit in the USART_CR3 register.

Table 421. Tolerance of the USART receiver when BRR [3:0] = 0000

M bits	OVER8 bit = 0		OVER8 bit = 1	
	ONEBIT = 0	ONEBIT = 1	ONEBIT = 0	ONEBIT = 1
00	3.75%	4.375%	2.50%	3.75%
01	3.41%	3.97%	2.27%	3.41%
10	4.16%	4.86%	2.77%	4.16%

Table 422. Tolerance of the USART receiver when BRR[3:0] is different from 0000

M bits	OVER8 bit = 0		OVER8 bit = 1	
	ONEBIT = 0	ONEBIT = 1	ONEBIT = 0	ONEBIT = 1
00	3.33%	3.88%	2%	3%
01	3.03%	3.53%	1.82%	2.73%
10	3.7%	4.31%	2.22%	3.33%

Note: The data specified in [Table 421](#) and [Table 422](#) may slightly differ in the special case when the received frames contain some Idle frames of exactly 10-bit times when M bits = 00 (11-bit times when M = 01 or 9-bit times when M = 10).

51.5.9 USART auto baud rate detection

The USART can detect and automatically set the USART_BRR register value based on the reception of one character. Automatic baud rate detection is useful under two circumstances:

- The communication speed of the system is not known in advance.
- The system is using a relatively low accuracy clock source and this mechanism enables the correct baud rate to be obtained without measuring the clock deviation.

The clock source frequency must be compatible with the expected communication speed.

- When oversampling by 16, the baud rate ranges from $\text{usart_ker_ck_pres}/65535$ and $\text{usart_ker_ck_pres}/16$.
- When oversampling by 8, the baud rate ranges from $\text{usart_ker_ck_pres}/65535$ and $\text{usart_ker_ck_pres}/8$.

Before activating the auto baud rate detection, the auto baud rate detection mode must be selected through the ABRMOD[1:0] field in the USART_CR2 register. There are four modes based on different character patterns. In these auto baud rate modes, the baud rate is measured several times during the synchronization data reception and each measurement is compared to the previous one.

These modes are the following:

- **Mode 0:** Any character starting with a bit at '1'.
In this case the USART measures the duration of the start bit (falling edge to rising edge).
- **Mode 1:** Any character starting with a 10xx bit pattern.
In this case, the USART measures the duration of the Start and of the 1st data bit. The measurement is done falling edge to falling edge, to ensure a better accuracy in the case of slow signal slopes.
- **Mode 2:** A 0x7F character frame (it may be a 0x7F character in LSB first mode or a 0xFE in MSB first mode).
In this case, the baud rate is updated first at the end of the start bit (BRs), then at the end of bit 6 (based on the measurement done from falling edge to falling edge: BR6). Bit0 to bit6 are sampled at BRs while further bits of the character are sampled at BR6.
- **Mode 3:** A 0x55 character frame.
In this case, the baud rate is updated first at the end of the start bit (BRs), then at the end of bit0 (based on the measurement done from falling edge to falling edge: BR0), and finally at the end of bit6 (BR6). Bit 0 is sampled at BRs, bit 1 to bit 6 are sampled at BR0, and further bits of the character are sampled at BR6. In parallel, another check is performed for each intermediate RX line transition. An error is generated if the transitions on RX are not sufficiently synchronized with the receiver (the receiver being based on the baud rate calculated on bit 0).

Prior to activating the auto baud rate detection, the USART_BRR register must be initialized by writing a non-zero baud rate value.

The automatic baud rate detection is activated by setting the ABREN bit in the USART_CR2 register. The USART then waits for the first character on the RX line. The auto baud rate operation completion is indicated by the setting of the ABRF flag in the USART_ISR register. If the line is noisy, the correct baud rate detection cannot be guaranteed. In this case the BRR value may be corrupted and the ABRE error flag is set. This also happens if the communication speed is not compatible with the automatic baud rate detection range (bit duration not between 16 and 65536 clock periods (oversampling by 16) and not between 8 and 65536 clock periods (oversampling by 8)).

The auto baud rate detection can be re-launched later by resetting the ABRF flag (by writing a '0').

When FIFO management is disabled and an auto baud rate error occurs, the ABRE flag is set through RXNE and FE bits.

When FIFO management is enabled and an auto baud rate error occurs, the ABRE flag is set through RXFNE and FE bits.

If the FIFO mode is enabled, the auto baud rate detection should be made using the data on the first RXFIFO location. So, prior to launching the auto baud rate detection, make sure that the RXFIFO is empty by checking the RXFNE flag in USART_ISR register.

Note: The BRR value might be corrupted if the USART is disabled (UE = 0) during an auto baud rate operation.

51.5.10 USART multiprocessor communication

It is possible to perform USART multiprocessor communications (with several USARTs connected in a network). For instance one of the USARTs can be the master with its TX output connected to the RX inputs of the other USARTs, while the others are slaves with their respective TX outputs logically ANDed together and connected to the RX input of the master.

In multiprocessor configurations, it is often desirable that only the intended message recipient actively receives the full message contents, thus reducing redundant USART service overhead for all non addressed receivers.

The non-addressed devices can be placed in Mute mode by means of the muting function. To use the Mute mode feature, the MME bit must be set in the USART_CR1 register.

Note: When FIFO management is enabled and MME is already set, MME bit must not be cleared and then set again quickly (within two usart_ker_ck cycles), otherwise Mute mode might remain active.

When the Mute mode is enabled:

- none of the reception status bits can be set;
- all the receive interrupts are inhibited;
- the RWU bit in USART_ISR register is set to '1'. RWU can be controlled automatically by hardware or by software, through the MMRQ bit in the USART_RQR register, under certain conditions.

The USART can enter or exit from Mute mode using one of two methods, depending on the WAKE bit in the USART_CR1 register:

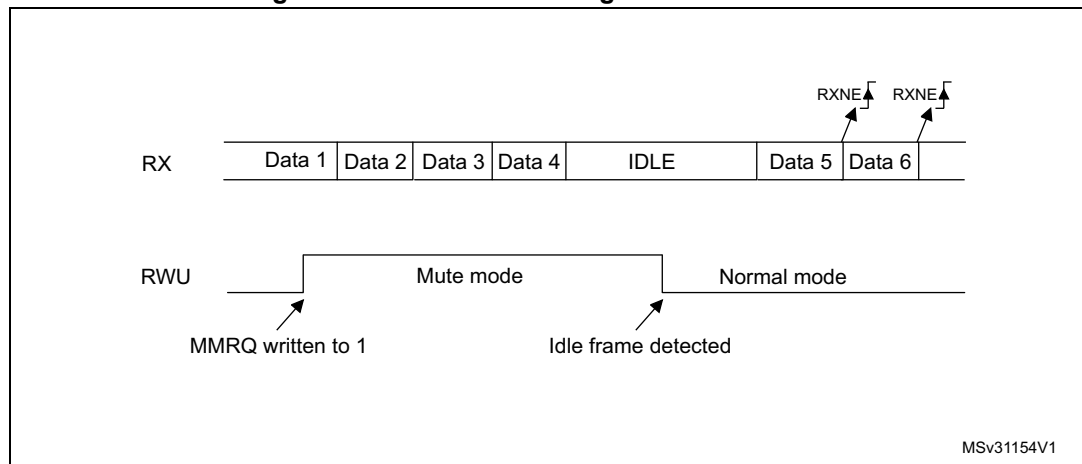
- Idle Line detection if the WAKE bit is reset,
- Address Mark detection if the WAKE bit is set.

Idle line detection (WAKE = 0)

The USART enters Mute mode when the MMRQ bit is written to '1' and the RWU is automatically set.

The USART wakes up when an Idle frame is detected. The RWU bit is then cleared by hardware but the IDLE bit is not set in the USART_ISR register. An example of Mute mode behavior using Idle line detection is given in [Figure 621](#).

Figure 621. Mute mode using Idle line detection



Note: If the MMRQ is set while the IDLE character has already elapsed, Mute mode is not entered (RWU is not set).

If the USART is activated while the line is IDLE, the idle state is detected after the duration of one IDLE frame (not only after the reception of one character frame).

4-bit/7-bit address mark detection (WAKE = 1)

In this mode, bytes are recognized as addresses if their MSB is a '1', otherwise they are considered as data. In an address byte, the address of the targeted receiver is put in the 4 or 7 LSBs. The choice of 7 or 4 bit address detection is done using the ADDM7 bit. This 4-bit/7-bit word is compared by the receiver with its own address which is programmed in the ADD bits in the USART_CR2 register.

Note: In 7-bit and 9-bit data modes, address detection is done on 6-bit and 8-bit addresses (ADD[5:0] and ADD[7:0]) respectively.

The USART enters Mute mode when an address character is received which does not match its programmed address. In this case, the RWU bit is set by hardware. The RXNE flag is not set for this address byte and no interrupt or DMA request is issued when the USART enters Mute mode. When FIFO management is enabled, the software should ensure that there is at least one empty location in the RXFIFO before entering Mute mode.

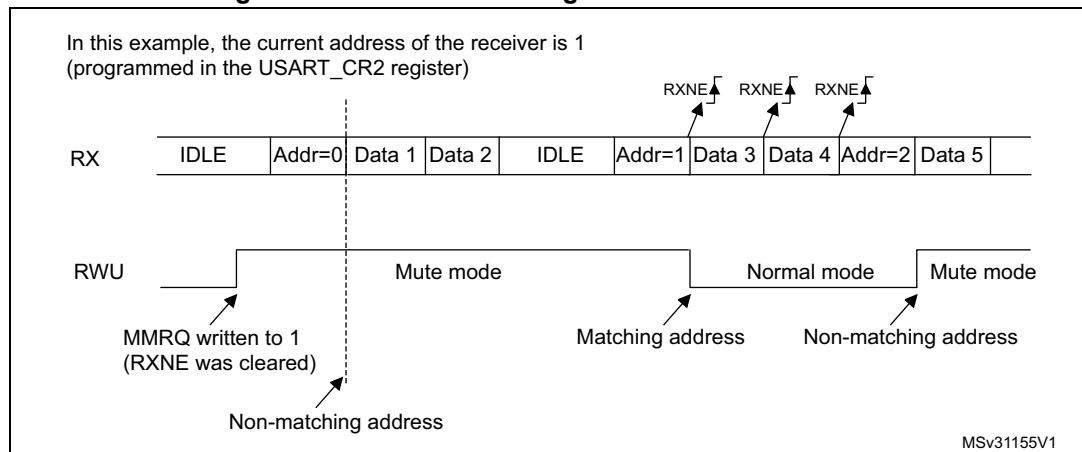
The USART also enters Mute mode when the MMRQ bit is written to 1. The RWU bit is also automatically set in this case.

The USART exits from Mute mode when an address character is received which matches the programmed address. Then the RWU bit is cleared and subsequent bytes are received normally. The RXNE/RXFNE bit is set for the address character since the RWU bit has been cleared.

Note: When FIFO management is enabled, when MMRQ is set while the receiver is sampling last bit of a data, this data may be received before effectively entering in Mute mode

An example of Mute mode behavior using address mark detection is given in [Figure 622](#).

Figure 622. Mute mode using address mark detection



51.5.11 USART Modbus communication

The USART offers basic support for the implementation of Modbus/RTU and Modbus/ASCII protocols. Modbus/RTU is a Half-duplex, block-transfer protocol. The control part of the protocol (address recognition, block integrity control and command interpretation) must be implemented in software.

The USART offers basic support for the end of the block detection, without software overhead or other resources.

Modbus/RTU

In this mode, the end of one block is recognized by a “silence” (idle line) for more than 2 character times. This function is implemented through the programmable timeout function.

The timeout function and interrupt must be activated, through the RTOEN bit in the USART_CR2 register and the RTOIE in the USART_CR1 register. The value corresponding to a timeout of 2 character times (for example 22 x bit time) must be programmed in the RTO register. When the receive line is idle for this duration, after the last stop bit is received, an interrupt is generated, informing the software that the current block reception is completed.

Modbus/ASCII

In this mode, the end of a block is recognized by a specific (CR/LF) character sequence. The USART manages this mechanism using the character match function.

By programming the LF ASCII code in the ADD[7:0] field and by activating the character match interrupt (CMIE = 1), the software is informed when a LF has been received and can check the CR/LF in the DMA buffer.

51.5.12 USART parity control

Parity control (generation of parity bit in transmission and parity checking in reception) can be enabled by setting the PCE bit in the USART_CR1 register. Depending on the frame length defined by the M bits, the possible USART frame formats are as listed in [Table 423](#).

Table 423. USART frame formats

M bits	PCE bit	USART frame ⁽¹⁾
00	0	SB 8 bit data STB
00	1	SB 7-bit data PB STB
01	0	SB 9-bit data STB
01	1	SB 8-bit data PB STB
10	0	SB 7bit data STB
10	1	SB 6-bit data PB STB

1. Legends: SB: start bit, STB: stop bit, PB: parity bit. In the data register, the PB is always taking the MSB position (8th or 7th, depending on the M bit value).

Even parity

The parity bit is calculated to obtain an even number of “1s” inside the frame of the 6, 7 or 8 LSB bits (depending on M bit values) and the parity bit.

As an example, if data = 00110101 and 4 bits are set, the parity bit is equal to 0 if even parity is selected (PS bit in USART_CR1 = 0).

Odd parity

The parity bit is calculated to obtain an odd number of “1s” inside the frame made of the 6, 7 or 8 LSB bits (depending on M bit values) and the parity bit.

As an example, if data = 00110101 and 4 bits set, then the parity bit is equal to 1 if odd parity is selected (PS bit in USART_CR1 = 1).

Parity checking in reception

If the parity check fails, the PE flag is set in the USART_ISR register and an interrupt is generated if PEIE is set in the USART_CR1 register. The PE flag is cleared by software writing 1 to the PECF in the USART_ICR register.

Parity generation in transmission

If the PCE bit is set in USART_CR1, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit (even number of “1s” if even parity is selected (PS = 0) or an odd number of “1s” if odd parity is selected (PS=1).

51.5.13 USART LIN (local interconnection network) mode

This section is relevant only when LIN mode is supported. Refer to [Section 51.4: USART implementation on page 2172](#).

The LIN mode is selected by setting the LINEN bit in the USART_CR2 register. In LIN mode, the following bits must be kept cleared:

- CLKEN in the USART_CR2 register,
- STOP[1:0], SCEN, HDSEL and IREN in the USART_CR3 register.

LIN transmission

The procedure described in [Section 51.5.4](#) has to be applied for LIN Master transmission. It must be the same as for normal USART transmission with the following differences:

- Clear the M bit to configure 8-bit word length.
- Set the LINEN bit to enter LIN mode. In this case, setting the SBKRQ bit sends 13 '0 bits as a break character. Then two bits of value '1 are sent to enable the next start detection.

LIN reception

When LIN mode is enabled, the break detection circuit is activated. The detection is totally independent from the normal USART receiver. A break can be detected whenever it occurs, during Idle state or during a frame.

When the receiver is enabled (RE = 1 in USART_CR1), the circuit looks at the RX input for a start signal. The method for detecting start bits is the same when searching break characters or data. After a start bit has been detected, the circuit samples the next bits exactly like for the data (on the 8th, 9th and 10th samples). If 10 (when the LBDL = 0 in USART_CR2) or 11 (when LBDL = 1 in USART_CR2) consecutive bits are detected as '0, and are followed by a delimiter character, the LBDF flag is set in USART_ISR. If the LBDIE bit = 1, an interrupt is generated. Before validating the break, the delimiter is checked for as it signifies that the RX line has returned to a high level.

If a '1 is sampled before the 10 or 11 have occurred, the break detection circuit cancels the current detection and searches for a start bit again.

If the LIN mode is disabled (LINEN = 0), the receiver continues working as normal USART, without taking into account the break detection.

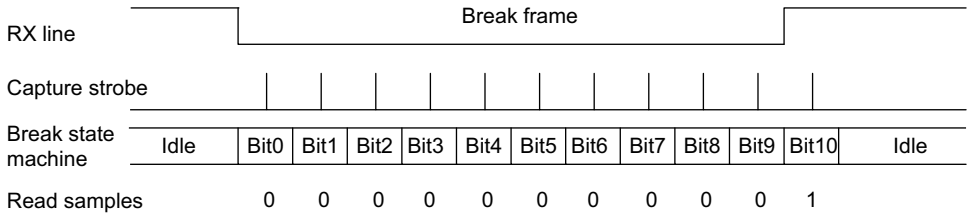
If the LIN mode is enabled (LINEN = 1), as soon as a framing error occurs (i.e. stop bit detected at '0, which is the case for any break frame), the receiver stops until the break detection circuit receives either a '1, if the break word was not complete, or a delimiter character if a break has been detected.

The behavior of the break detector state machine and the break flag is shown on the [Figure 623: Break detection in LIN mode \(11-bit break length - LBDL bit is set\) on page 2198](#).

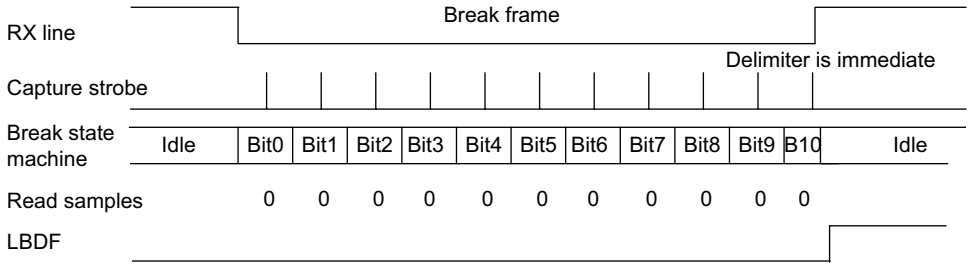
Examples of break frames are given on [Figure 624: Break detection in LIN mode vs. Framing error detection on page 2199](#).

Figure 623. Break detection in LIN mode (11-bit break length - LBDL bit is set)

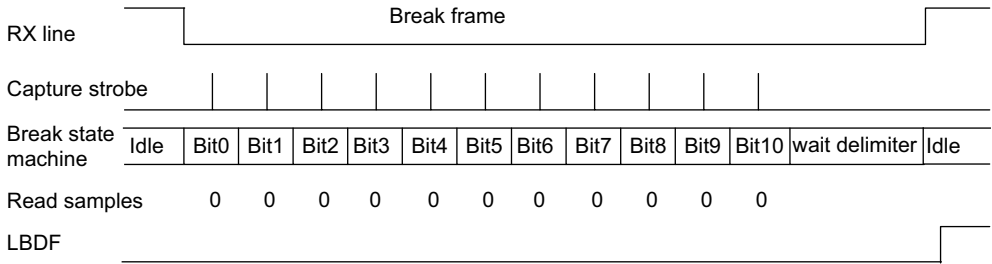
Case 1: break signal not long enough => break discarded, LBDF is not set



Case 2: break signal just long enough => break detected, LBDF is set

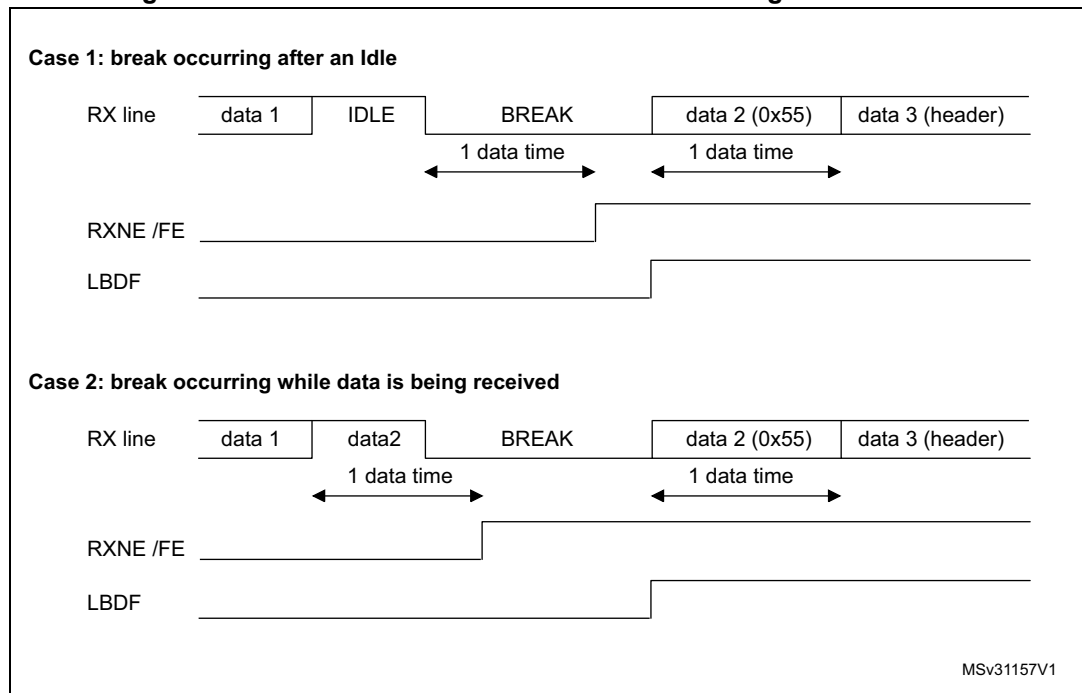


Case 3: break signal long enough => break detected, LBDF is set



MSv31156V1

Figure 624. Break detection in LIN mode vs. Framing error detection



51.5.14 USART synchronous mode

Master mode

The synchronous master mode is selected by programming the CLKEN bit in the USART_CR2 register to '1'. In synchronous mode, the following bits must be kept cleared:

- LINEN bit in the USART_CR2 register,
- SCEN, HDSEL and IREN bits in the USART_CR3 register.

In this mode, the USART can be used to control bidirectional synchronous serial communications in master mode. The CK pin is the output of the USART transmitter clock. No clock pulses are sent to the CK pin during start bit and stop bit. Depending on the state of the LBCL bit in the USART_CR2 register, clock pulses are, or are not, generated during the last valid data bit (address mark). The CPOL bit in the USART_CR2 register is used to select the clock polarity, and the CPHA bit in the USART_CR2 register is used to select the phase of the external clock (see [Figure 625](#), [Figure 626](#) and [Figure 627](#)).

During the Idle state, preamble and send break, the external CK clock is not activated.

In synchronous master mode, the USART transmitter operates exactly like in asynchronous mode. However, since CK is synchronized with TX (according to CPOL and CPHA), the data on TX is synchronous.

In synchronous master mode, the USART receiver operates in a different way compared to asynchronous mode. If RE is set to 1, the data are sampled on CK (rising or falling edge, depending on CPOL and CPHA), without any oversampling. A given setup and a hold time must be respected (which depends on the baud rate: 1/16 bit time).

Note: In master mode, the CK pin operates in conjunction with the TX pin. Thus, the clock is provided only if the transmitter is enabled (TE = 1) and data are being transmitted (USART_TDR data register written). This means that it is not possible to receive synchronous data without transmitting data.

Figure 625. USART example of synchronous master transmission

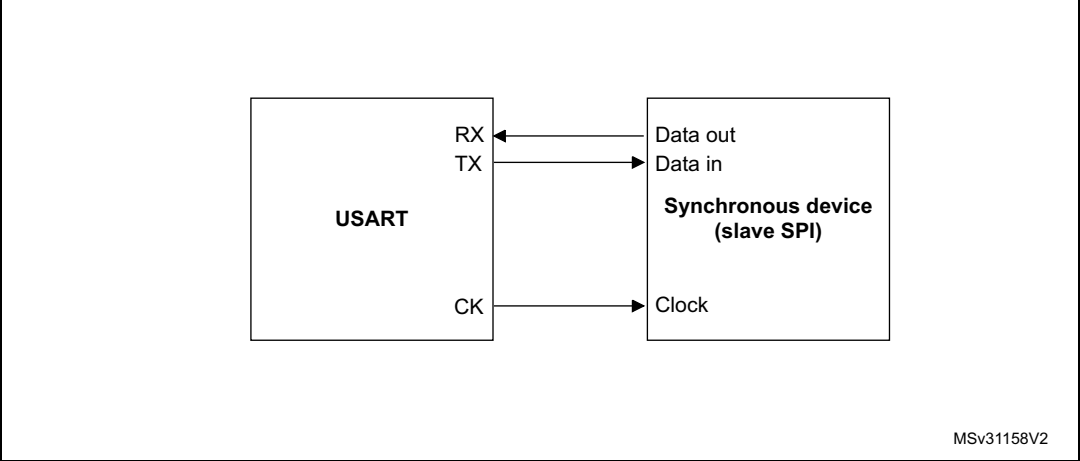


Figure 626. USART data clock timing diagram in synchronous master mode (M bits = 00)

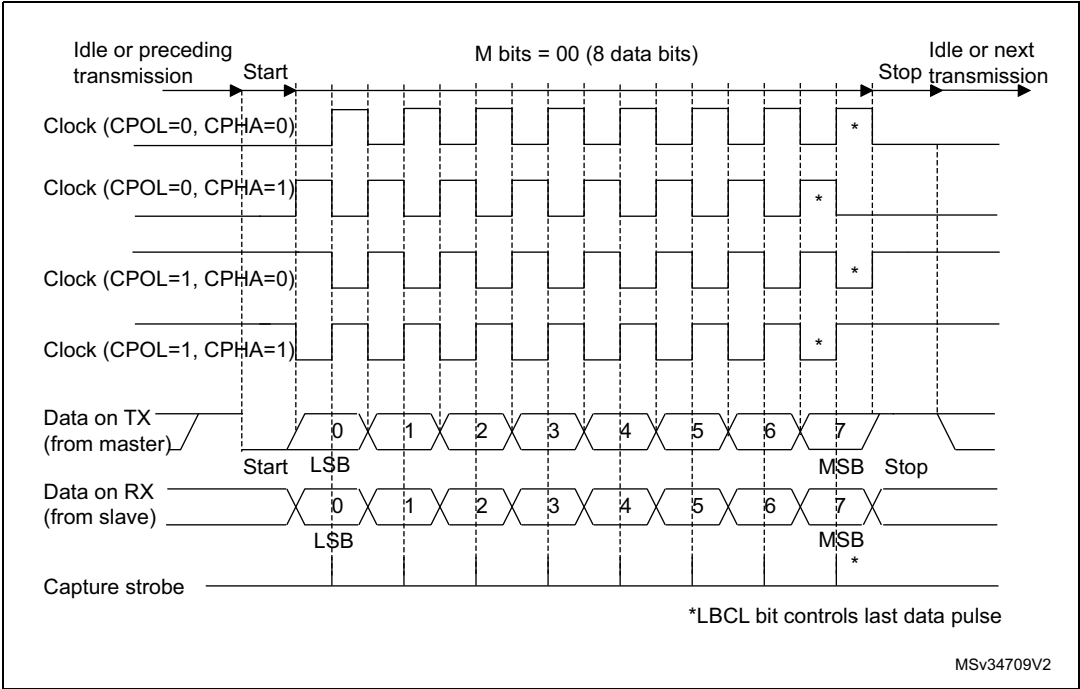
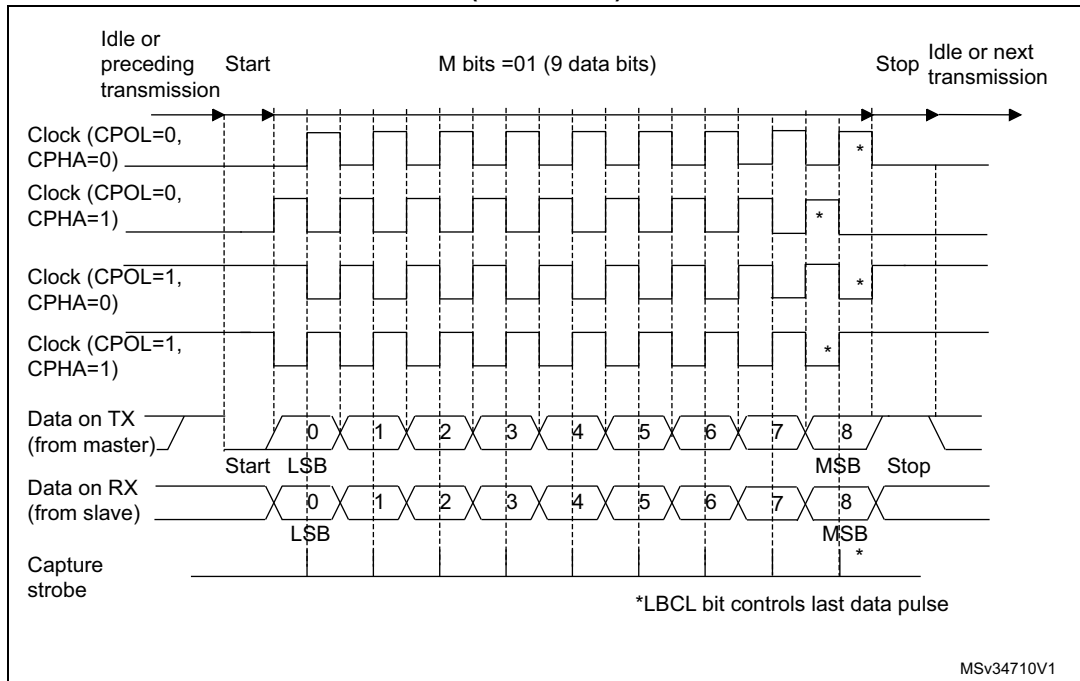


Figure 627. USART data clock timing diagram in synchronous master mode (M bits = 01)



Slave mode

The synchronous slave mode is selected by programming the SLVEN bit in the USART_CR2 register to '1'. In synchronous slave mode, the following bits must be kept cleared:

- LINEN and CLKEN bits in the USART_CR2 register,
- SCEN, HDSEL and IREN bits in the USART_CR3 register.

In this mode, the USART can be used to control bidirectional synchronous serial communications in slave mode. The CK pin is the input of the USART in slave mode.

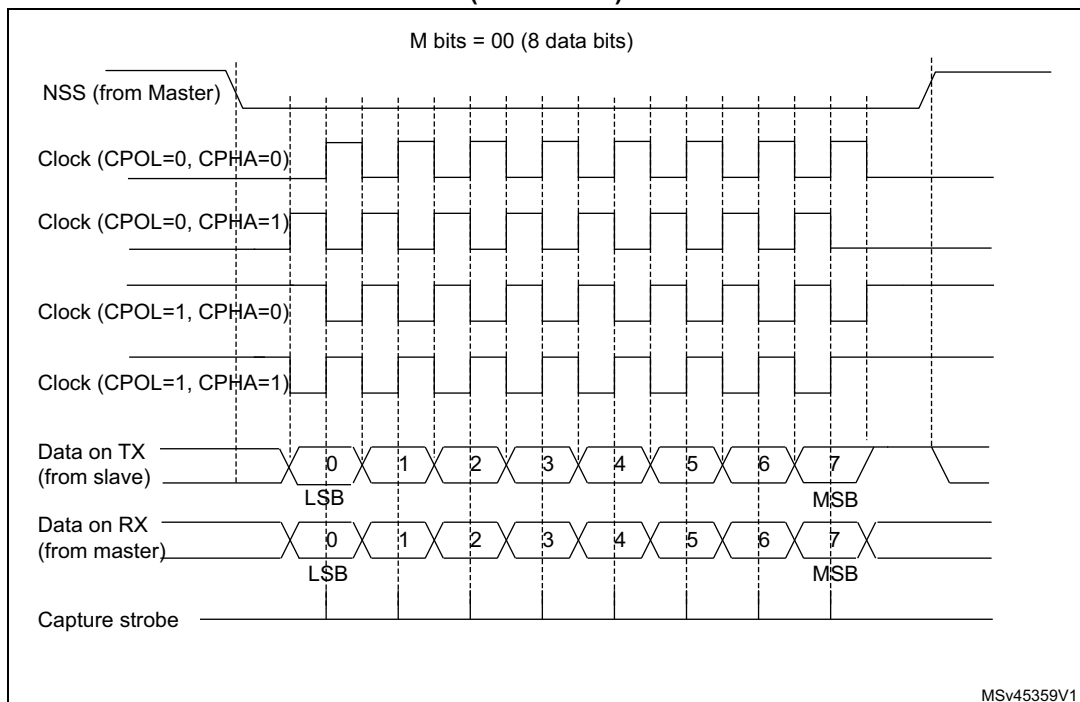
Note: *When the peripheral is used in SPI slave mode, the frequency of peripheral clock source (usart_ker_ck_pres) must be greater than 3 times the CK input frequency.*

The CPOL bit and the CPHA bit in the USART_CR2 register are used to select the clock polarity and the phase of the external clock, respectively (see [Figure 628](#)).

An underrun error flag is available in slave transmission mode. This flag is set when the first clock pulse for data transmission appears while the software has not yet loaded any value to USART_TDR.

The slave supports the hardware and software NSS management.

**Figure 628. USART data clock timing diagram in synchronous slave mode
(M bits = 00)**



Slave Select (NSS) pin management

The hardware or software slave select management can be set through the DIS_NSS bit in the USART_CR2 register:

- Software NSS management (DIS_NSS = 1)
The SPI slave is always selected and NSS input pin is ignored.
The external NSS pin remains free for other application uses.
- Hardware NSS management (DIS_NSS = 0)
The SPI slave selection depends on NSS input pin. The slave is selected when NSS is low and deselected when NSS is high.

Note: The LBCL (used only on SPI master mode), CPOL and CPHA bits have to be selected when the USART is disabled (UE = 0) to ensure that the clock pulses function correctly.

In SPI slave mode, the USART must be enabled before starting the master communications (or between frames while the clock is stable). Otherwise, if the USART slave is enabled while the master is in the middle of a frame, it becomes desynchronized with the master. The data register of the slave needs to be ready before the first edge of the communication clock or before the end of the ongoing communication, otherwise the SPI slave transmits zeros.

SPI Slave underrun error

When an underrun error occurs, the UDR flag is set in the USART_ISR register, and the SPI slave goes on sending the last data until the underrun error flag is cleared by software.

The underrun flag is set at the beginning of the frame. An underrun error interrupt is triggered if EIE bit is set in the USART_CR3 register.

The underrun error flag is cleared by setting bit UDRCF in the USART_ICR register.

In case of underrun error, it is still possible to write to the TDR register. Clearing the underrun error enables sending new data.

If an underrun error occurred and there is no new data written in TDR, then the TC flag is set at the end of the frame.

Note: An underrun error may occur if the moment the data is written to the USART_TDR is too close to the first CK transmission edge. To avoid this underrun error, the USART_TDR should be written 3 usart_ker_ck cycles before the first CK edge.

51.5.15 USART single-wire Half-duplex communication

Single-wire Half-duplex mode is selected by setting the HDSEL bit in the USART_CR3 register. In this mode, the following bits must be kept cleared:

- LINEN and CLKEN bits in the USART_CR2 register,
- SCEN and IREN bits in the USART_CR3 register.

The USART can be configured to follow a Single-wire Half-duplex protocol where the TX and RX lines are internally connected. The selection between half- and Full-duplex communication is made with a control bit HDSEL in USART_CR3.

As soon as HDSEL is written to '1':

- The TX and RX lines are internally connected.
- The RX pin is no longer used.
- The TX pin is always released when no data is transmitted. Thus, it acts as a standard I/O in idle or in reception. It means that the I/O must be configured so that TX is configured as alternate function open-drain with an external pull-up.

Apart from this, the communication protocol is similar to normal USART mode. Any conflict on the line must be managed by software (for instance by using a centralized arbiter). In particular, the transmission is never blocked by hardware and continues as soon as data are written in the data register while the TE bit is set.

51.5.16 USART receiver timeout

The receiver timeout feature is enabled by setting the RTOEN bit in the USART_CR2 control register.

The timeout duration is programmed using the RTO bitfields in the USART_RTOR register.

The receiver timeout counter starts counting:

- from the end of the stop bit if STOP = '00' or STOP = '11'
- from the end of the second stop bit if STOP = '10'.
- from the beginning of the stop bit if STOP = '01'.

When the timeout duration has elapsed, the RTOF flag in the USART_ISR register is set. A timeout is generated if RTOIE bit in USART_CR1 register is set.

51.5.17 USART Smartcard mode

This section is relevant only when Smartcard mode is supported. Refer to [Section 51.4: USART implementation on page 2172](#).

Smartcard mode is selected by setting the SCEN bit in the USART_CR3 register. In Smartcard mode, the following bits must be kept cleared:

- LINEN bit in the USART_CR2 register,
- HDSEL and IREN bits in the USART_CR3 register.

The CLKEN bit can also be set to provide a clock to the Smartcard.

The Smartcard interface is designed to support asynchronous Smartcard protocol as defined in the ISO 7816-3 standard. Both T = 0 (character mode) and T = 1 (block mode) are supported.

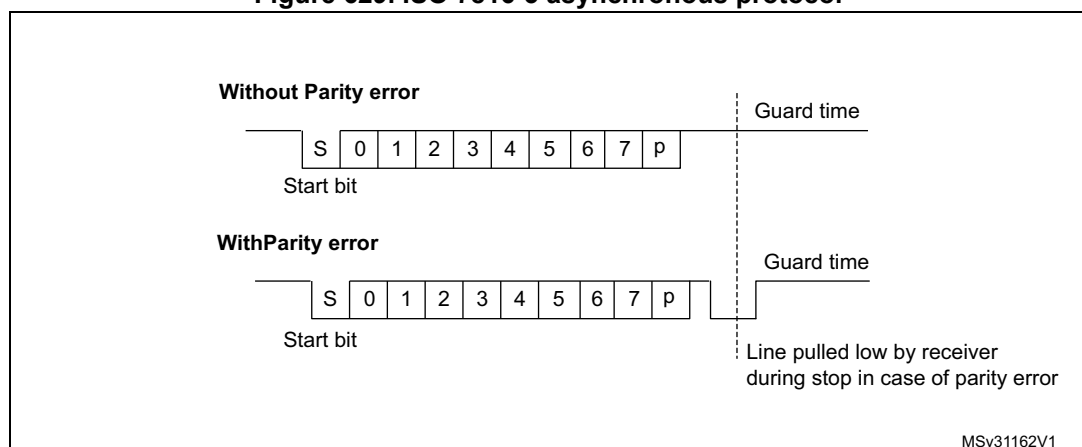
The USART should be configured as:

- 8 bits plus parity: M = 1 and PCE = 1 in the USART_CR1 register
- 1.5 stop bits when transmitting and receiving data: STOP = '11' in the USART_CR2 register. It is also possible to choose 0.5 stop bit for reception.

In T = 0 (character) mode, the parity error is indicated at the end of each character during the guard time period.

[Figure 629](#) shows examples of what can be seen on the data line with and without parity error.

Figure 629. ISO 7816-3 asynchronous protocol



When connected to a Smartcard, the TX output of the USART drives a bidirectional line that is also driven by the Smartcard. The TX pin must be configured as open drain.

Smartcard mode implements a single wire half duplex communication protocol.

- Transmission of data from the transmit shift register is guaranteed to be delayed by a minimum of 1/2 baud clock. In normal operation a full transmit shift register starts shifting on the next baud clock edge. In Smartcard mode this transmission is further delayed by a guaranteed 1/2 baud clock.
- In transmission, if the Smartcard detects a parity error, it signals this condition to the USART by driving the line low (NACK). This NACK signal (pulling transmit line low for 1 baud clock) causes a framing error on the transmitter side (configured with 1.5 stop bits). The USART can handle automatic re-sending of data according to the protocol.

The number of retries is programmed in the SCARCNT bitfield. If the USART continues receiving the NACK after the programmed number of retries, it stops transmitting and signals the error as a framing error. The TXE bit (TXFNF bit in case FIFO mode is enabled) may be set using the TXFRQ bit in the USART_RQR register.

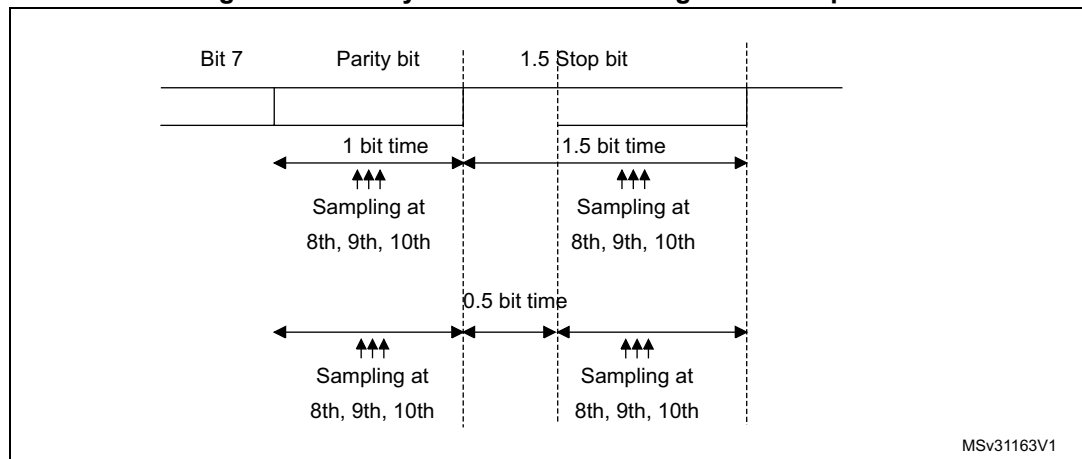
- Smartcard auto-retry in transmission: A delay of 2.5 baud periods is inserted between the NACK detection by the USART and the start bit of the repeated character. The TC bit is set immediately at the end of reception of the last repeated character (no guardtime). If the software wants to repeat it again, it must insure the minimum 2 baud periods required by the standard.
- If a parity error is detected during reception of a frame programmed with a 1.5 stop bit period, the transmit line is pulled low for a baud clock period after the completion of the receive frame. This is to indicate to the Smartcard that the data transmitted to the USART has not been correctly received. A parity error is NACKed by the receiver if the NACK control bit is set, otherwise a NACK is not transmitted (to be used in T = 1 mode). If the received character is erroneous, the RXNE (RXFNE in case FIFO mode is enabled)/receive DMA request is not activated. According to the protocol specification, the Smartcard must resend the same character. If the received character is still erroneous after the maximum number of retries specified in the SCARCNT bitfield, the USART stops transmitting the NACK and signals the error as a parity error.
- Smartcard auto-retry in reception: the BUSY flag remains set if the USART NACKs the card but the card doesn't repeat the character.
- In transmission, the USART inserts the Guard Time (as programmed in the Guard Time register) between two successive characters. As the Guard Time is measured after the stop bit of the previous character, the GT[7:0] register must be programmed to the desired CGT (Character Guard Time, as defined by the 7816-3 specification) minus 12 (the duration of one character).
- The assertion of the TC flag can be delayed by programming the Guard Time register. In normal operation, TC is asserted when the transmit shift register is empty and no further transmit requests are outstanding. In Smartcard mode an empty transmit shift register triggers the Guard Time counter to count up to the programmed value in the Guard Time register. TC is forced low during this time. When the Guard Time counter reaches the programmed value TC is asserted high. The TCBGT flag can be used to detect the end of data transfer without waiting for guard time completion. This flag is set just after the end of frame transmission and if no NACK has been received from the card.
- The deassertion of TC flag is unaffected by Smartcard mode.
- If a framing error is detected on the transmitter end (due to a NACK from the receiver), the NACK is not detected as a start bit by the receive block of the transmitter. According to the ISO protocol, the duration of the received NACK can be 1 or 2 baud clock periods.
- On the receiver side, if a parity error is detected and a NACK is transmitted the receiver does not detect the NACK as a start bit.

Note: *Break characters are not significant in Smartcard mode. A 0x00 data with a framing error is treated as data and not as a break.*

No Idle frame is transmitted when toggling the TE bit. The Idle frame (as defined for the other configurations) is not defined by the ISO protocol.

Figure 630 shows how the NACK signal is sampled by the USART. In this example the USART is transmitting data and is configured with 1.5 stop bits. The receiver part of the USART is enabled in order to check the integrity of the data and the NACK signal.

Figure 630. Parity error detection using the 1.5 stop bits



The USART can provide a clock to the Smartcard through the CK output. In Smartcard mode, CK is not associated to the communication but is simply derived from the internal peripheral input clock through a 5-bit prescaler. The division ratio is configured in the USART_GTPR register. CK frequency can be programmed from $\text{usart_ker_ck_pres}/2$ to $\text{usart_ker_ck_pres}/62$, where usart_ker_ck_pres is the peripheral input clock divided by a programmed prescaler.

Block mode (T = 1)

In T = 1 (block) mode, the parity error transmission can be deactivated by clearing the NACK bit in the USART_CR3 register.

When requesting a read from the Smartcard, in block mode, the software must program the RTOR register to the BWT (block wait time) - 11 value. If no answer is received from the card before the expiration of this period, a timeout interrupt is generated. If the first character is received before the expiration of the period, it is signaled by the RXNE/RXFNE interrupt.

Note: *The RXNE/RXFNE interrupt must be enabled even when using the USART in DMA mode to read from the Smartcard in block mode. In parallel, the DMA must be enabled only after the first received byte.*

After the reception of the first character (RXNE/RXFNE interrupt), the RTO register must be programmed to the CWT (character wait time - 11 value), in order to enable the automatic check of the maximum wait time between two consecutive characters. This time is expressed in baud time units. If the Smartcard does not send a new character in less than the CWT period after the end of the previous character, the USART signals it to the software through the RTOF flag and interrupt (when RTOIE bit is set).

Note: *As in the Smartcard protocol definition, the BWT/CWT values should be defined from the beginning (start bit) of the last character. The RTO register must be programmed to BWT - 11 or CWT - 11, respectively, taking into account the length of the last character itself.*

A block length counter is used to count all the characters received by the USART. This counter is reset when the USART is transmitting. The length of the block is communicated by the Smartcard in the third byte of the block (prologue field). This value must be programmed to the BLEN field in the USART_RTOR register. When using DMA mode, before the start of the block, this register field must be programmed to the minimum value

(0x0). With this value, an interrupt is generated after the 4th received character. The software must read the LEN field (third byte), its value must be read from the receive buffer.

In interrupt driven receive mode, the length of the block may be checked by software or by programming the BLEN value. However, before the start of the block, the maximum value of BLEN (0xFF) may be programmed. The real value is programmed after the reception of the third character.

If the block is using the LRC longitudinal redundancy check (1 epilogue byte), the $BLEN = LEN$. If the block is using the CRC mechanism (2 epilog bytes), $BLEN = LEN + 1$ must be programmed. The total block length (including prologue, epilogue and information fields) equals $BLEN + 4$. The end of the block is signaled to the software through the EOBFF flag and interrupt (when EOBIE bit is set).

In case of an error in the block length, the end of the block is signaled by the RTO interrupt (Character Wait Time overflow).

Note: The error checking code (LRC/CRC) must be computed/verified by software.

Direct and inverse convention

The Smartcard protocol defines two conventions: direct and inverse.

The direct convention is defined as: LSB first, logical bit value of 1 corresponds to a H state of the line and parity is even. In order to use this convention, the following control bits must be programmed: MSBFIRST = 0, DATAINV = 0 (default values).

The inverse convention is defined as: MSB first, logical bit value 1 corresponds to an L state on the signal line and parity is even. In order to use this convention, the following control bits must be programmed: MSBFIRST = 1, DATAINV = 1.

Note: When logical data values are inverted (0 = H, 1 = L), the parity bit is also inverted in the same way.

In order to recognize the card convention, the card sends the initial character, TS, as the first character of the ATR (Answer To Reset) frame. The two possible patterns for the TS are: LHHL LLL LLH and LHHL HHH LLH.

- (H) LHHL LLL LLH sets up the inverse convention: state L encodes value 1 and moment 2 conveys the most significant bit (MSB first). When decoded by inverse convention, the conveyed byte is equal to '3F'.
- (H) LHHL HHH LLH sets up the direct convention: state H encodes value 1 and moment 2 conveys the least significant bit (LSB first). When decoded by direct convention, the conveyed byte is equal to '3B'.

Character parity is correct when there is an even number of bits set to 1 in the nine moments 2 to 10.

As the USART does not know which convention is used by the card, it needs to be able to recognize either pattern and act accordingly. The pattern recognition is not done in hardware, but through a software sequence. Moreover, assuming that the USART is configured in direct convention (default) and the card answers with the inverse convention, TS = LHHL LLL LLH results in a USART received character of 03 and an odd parity.

Therefore, two methods are available for TS pattern recognition:

Method 1

The USART is programmed in standard Smartcard mode/direct convention. In this case, the TS pattern reception generates a parity error interrupt and error signal to the card.

- The parity error interrupt informs the software that the card did not answer correctly in direct convention. Software then reprograms the USART for inverse convention
- In response to the error signal, the card retries the same TS character, and it is correctly received this time, by the reprogrammed USART.

Alternatively, in answer to the parity error interrupt, the software may decide to reprogram the USART and to also generate a new reset command to the card, then wait again for the TS.

Method 2

The USART is programmed in 9-bit/no-parity mode, no bit inversion. In this mode it receives any of the two TS patterns as:

(H) LHHH LLL LLH = 0x103: inverse convention to be chosen

(H) LHHH HHH LLH = 0x13B: direct convention to be chosen

The software checks the received character against these two patterns and, if any of them match, then programs the USART accordingly for the next character reception.

If none of the two is recognized, a card reset may be generated in order to restart the negotiation.

51.5.18 USART IrDA SIR ENDEC block

This section is relevant only when IrDA mode is supported. Refer to [Section 51.4: USART implementation on page 2172](#).

IrDA mode is selected by setting the IREN bit in the USART_CR3 register. In IrDA mode, the following bits must be kept cleared:

- LINEN, STOP and CLKEN bits in the USART_CR2 register,
- SCEN and HDSEL bits in the USART_CR3 register.

The IrDA SIR physical layer specifies use of a Return to Zero, Inverted (RZI) modulation scheme that represents logic 0 as an infrared light pulse (see [Figure 631](#)).

The SIR Transmit encoder modulates the Non Return to Zero (NRZ) transmit bit stream output from USART. The output pulse stream is transmitted to an external output driver and infrared LED. USART supports only bit rates up to 115.2 kbaud for the SIR ENDEC. In normal mode the transmitted pulse width is specified as 3/16 of a bit period.

The SIR receive decoder demodulates the return-to-zero bit stream from the infrared detector and outputs the received NRZ serial bit stream to the USART. The decoder input is normally high (marking state) in the Idle state. The transmit encoder output has the opposite polarity to the decoder input. A start bit is detected when the decoder input is low.

- IrDA is a half duplex communication protocol. If the Transmitter is busy (when the USART is sending data to the IrDA encoder), any data on the IrDA receive line is ignored by the IrDA decoder and if the Receiver is busy (when the USART is receiving decoded data from the USART), data on the TX from the USART to IrDA is not

encoded. While receiving data, transmission should be avoided as the data to be transmitted could be corrupted.

- A '0' is transmitted as a high pulse and a '1' is transmitted as a '0'. The width of the pulse is specified as 3/16th of the selected bit period in normal mode (see [Figure 632](#)).
- The SIR decoder converts the IrDA compliant receive signal into a bit stream for USART.
- The SIR receive logic interprets a high state as a logic one and low pulses as logic zeros.
- The transmit encoder output has the opposite polarity to the decoder input. The SIR output is in low state when Idle.
- The IrDA specification requires the acceptance of pulses greater than 1.41 μ s. The acceptable pulse width is programmable. Glitch detection logic on the receiver end filters out pulses of width less than 2 PSC periods (PSC is the prescaler value programmed in the USART_GTPR). Pulses of width less than 1 PSC period are always rejected, but those of width greater than one and less than two periods may be accepted or rejected, those greater than two periods are accepted as a pulse. The IrDA encoder/decoder doesn't work when PSC = 0.
- The receiver can communicate with a low-power transmitter.
- In IrDA mode, the stop bits in the USART_CR2 register must be configured to '1 stop bit'.

IrDA low-power mode

- Transmitter
In low-power mode, the pulse width is not maintained at 3/16 of the bit period. Instead, the width of the pulse is 3 times the low-power baud rate which can be a minimum of 1.42 MHz. Generally, this value is 1.8432 MHz ($1.42 \text{ MHz} < \text{PSC} < 2.12 \text{ MHz}$). A low-power mode programmable divisor divides the system clock to achieve this value.
- Receiver
Receiving in low-power mode is similar to receiving in normal mode. For glitch detection the USART should discard pulses of duration shorter than 1/PSC. A valid low is accepted only if its duration is greater than 2 periods of the IrDA low-power Baud clock (PSC value in the USART_GTPR).

Note: A pulse of width less than two and greater than one PSC period(s) may or may not be rejected.

The receiver set up time should be managed by software. The IrDA physical layer specification specifies a minimum of 10 ms delay between transmission and reception (IrDA is a half duplex protocol).

Figure 631. IrDA SIR ENDEC block diagram

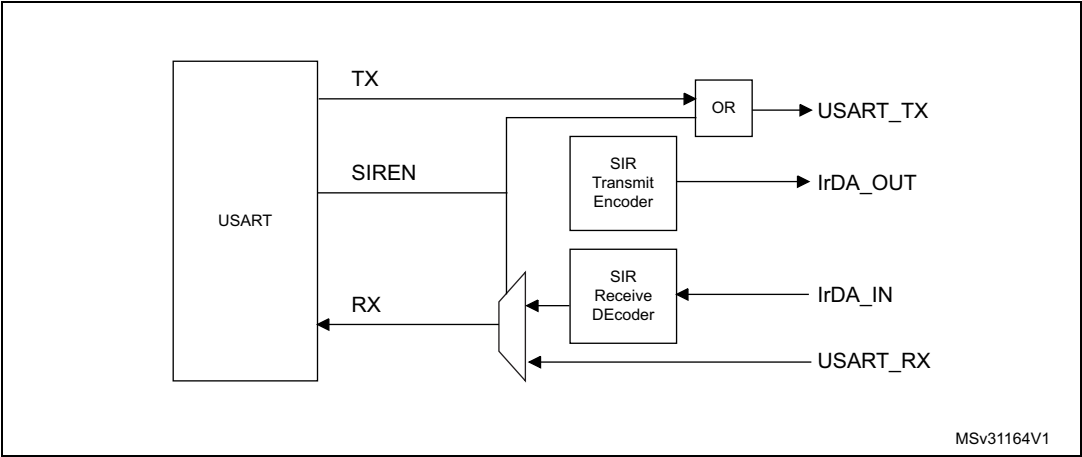
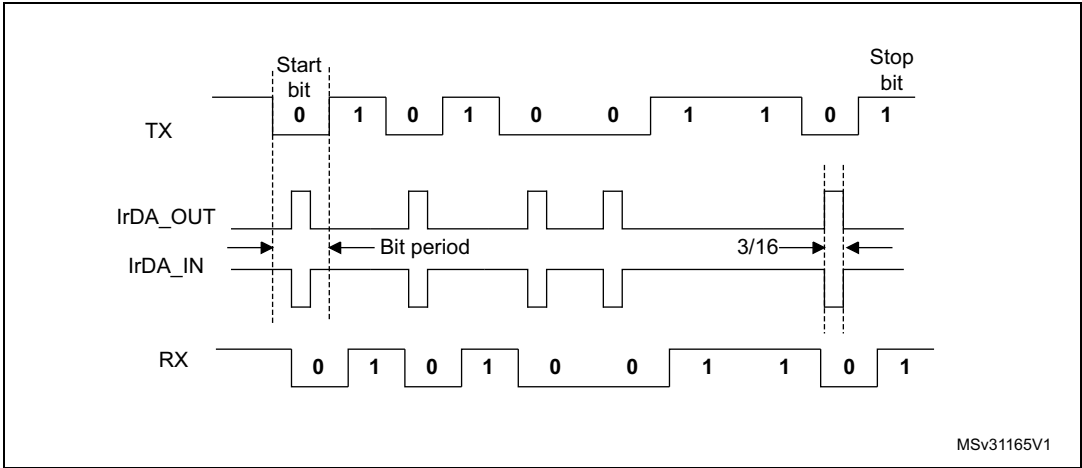


Figure 632. IrDA data modulation (3/16) - Normal mode



51.5.19 Continuous communication using USART and DMA

The USART is capable of performing continuous communications using the DMA. The DMA requests for Rx buffer and Tx buffer are generated independently.

Note: Refer to [Section 51.4: USART implementation on page 2172](#) to determine if the DMA mode is supported. If DMA is not supported, use the USART as explained in [Section 51.5.6](#). To perform continuous communications when the FIFO is disabled, clear the TXE/ RXNE flags in the USART_ISR register.

Transmission using DMA

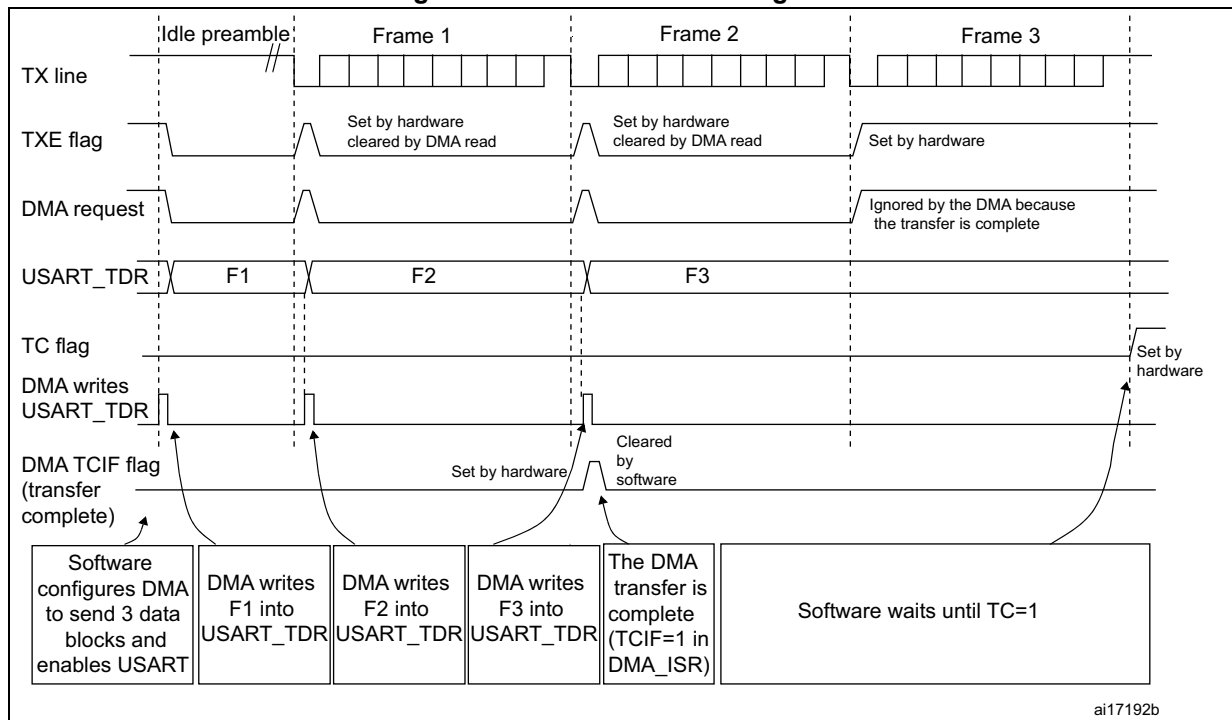
DMA mode can be enabled for transmission by setting DMAT bit in the USART_CR3 register. Data are loaded from an SRAM area configured using the DMA peripheral (refer to the corresponding *Direct memory access controller* section) to the USART_TDR register whenever the TXE flag (TXFNF flag if FIFO mode is enabled) is set. To map a DMA channel for USART transmission, use the following procedure (x denotes the channel number):

1. Write the USART_TDR register address in the DMA control register to configure it as the destination of the transfer. The data is moved to this address from memory after each TXE (or TXFNF if FIFO mode is enabled) event.
2. Write the memory address in the DMA control register to configure it as the source of the transfer. The data is loaded into the USART_TDR register from this memory area after each TXE (or TXFNF if FIFO mode is enabled) event.
3. Configure the total number of bytes to be transferred to the DMA control register.
4. Configure the channel priority in the DMA register
5. Configure DMA interrupt generation after half/ full transfer as required by the application.
6. Clear the TC flag in the USART_ISR register by setting the TCCF bit in the USART_ICR register.
7. Activate the channel in the DMA register.

When the number of data transfers programmed in the DMA Controller is reached, the DMA controller generates an interrupt on the DMA channel interrupt vector.

In transmission mode, once the DMA has written all the data to be transmitted (the TCIF flag is set in the DMA_ISR register), the TC flag can be monitored to make sure that the USART communication is complete. This is required to avoid corrupting the last transmission before disabling the USART or before the system enters a low-power mode when the peripheral clock is disabled. Software must wait until TC = 1. The TC flag remains cleared during all data transfers and it is set by hardware at the end of transmission of the last frame.

Figure 633. Transmission using DMA



Note: When FIFO management is enabled, the DMA request is triggered by Transmit FIFO not full (i.e. $TXFNF = 1$).

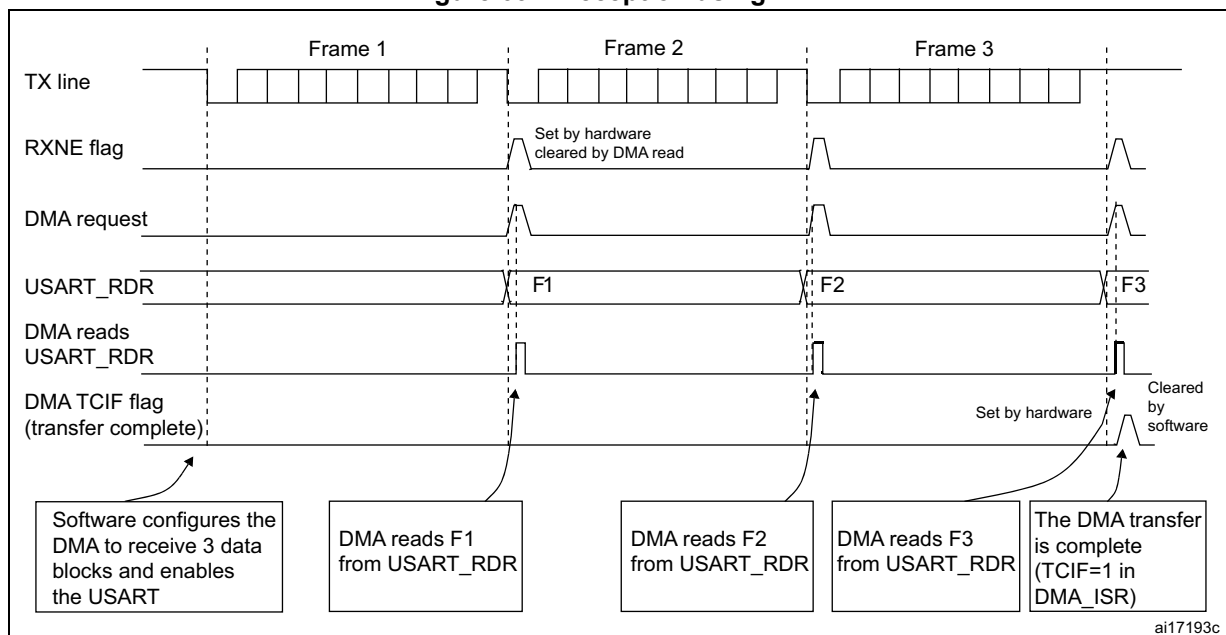
Reception using DMA

DMA mode can be enabled for reception by setting the DMAR bit in USART_CR3 register. Data are loaded from the USART_RDR register to an SRAM area configured using the DMA peripheral (refer to the corresponding *Direct memory access controller* section) whenever a data byte is received. To map a DMA channel for USART reception, use the following procedure:

1. Write the USART_RDR register address in the DMA control register to configure it as the source of the transfer. The data is moved from this address to the memory after each RXNE (RXFNE in case FIFO mode is enabled) event.
2. Write the memory address in the DMA control register to configure it as the destination of the transfer. The data is loaded from USART_RDR to this memory area after each RXNE (RXFNE in case FIFO mode is enabled) event.
3. Configure the total number of bytes to be transferred to the DMA control register.
4. Configure the channel priority in the DMA control register
5. Configure interrupt generation after half/ full transfer as required by the application.
6. Activate the channel in the DMA control register.

When the number of data transfers programmed in the DMA Controller is reached, the DMA controller generates an interrupt on the DMA channel interrupt vector.

Figure 634. Reception using DMA



Note: When FIFO management is enabled, the DMA request is triggered by Receive FIFO not empty (i.e. $RXFNE = 1$).

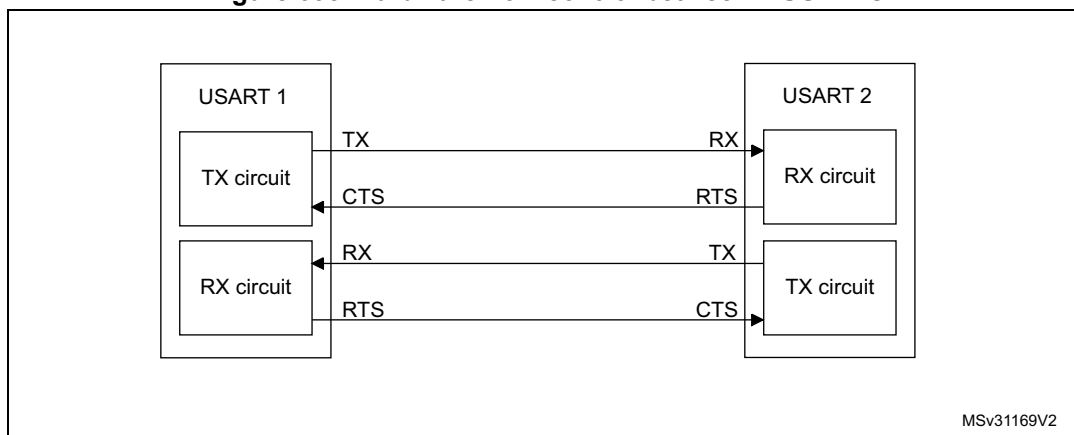
Error flagging and interrupt generation in multibuffer communication

If any error occurs during a transaction in multibuffer communication mode, the error flag is asserted after the current byte. An interrupt is generated if the interrupt enable flag is set. For framing error, overrun error and noise flag which are asserted with RXNE (RXFNE in case FIFO mode is enabled) in single byte reception, there is a separate error flag interrupt enable bit (EIE bit in the USART_CR3 register), which, if set, enables an interrupt after the current byte if any of these errors occur.

51.5.20 RS232 Hardware flow control and RS485 Driver Enable

It is possible to control the serial data flow between 2 devices by using the CTS input and the RTS output. The [Figure 635](#) shows how to connect 2 devices in this mode:

Figure 635. Hardware flow control between 2 USARTs

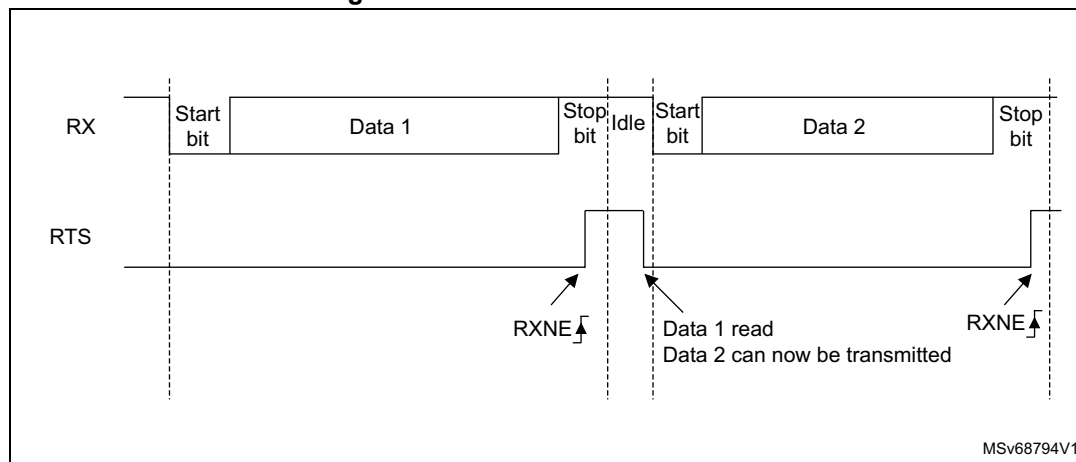


RS232 RTS and CTS flow control can be enabled independently by writing the RTSE and CTSE bits to '1' in the USART_CR3 register.

RS232 RTS flow control

If the RTS flow control is enabled (RTSE = 1), then RTS is deasserted (tied low) as long as the USART receiver is ready to receive a new data. When the receive register is full, RTS is asserted, indicating that the transmission is expected to stop at the end of the current frame. [Figure 636](#) shows an example of communication with RTS flow control enabled.

Figure 636. RS232 RTS flow control



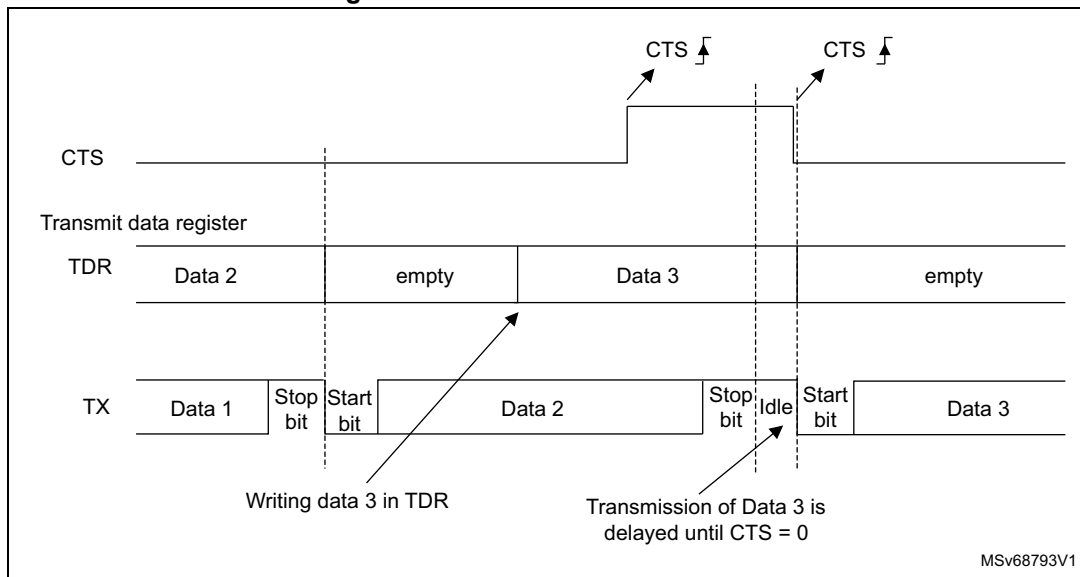
Note: When FIFO mode is enabled, RTS is asserted only when RXFIFO is full.

RS232 CTS flow control

If the CTS flow control is enabled (CTSE = 1), then the transmitter checks the CTS input before transmitting the next frame. If CTS is deasserted (tied low), then the next data is transmitted (assuming that data is to be transmitted, in other words, if TXE/TXFE = 0), else the transmission does not occur. When CTS is asserted during a transmission, the current transmission is completed before the transmitter stops.

When CTSE = 1, the CTSIF status bit is automatically set by hardware as soon as the CTS input toggles. It indicates when the receiver becomes ready or not ready for communication. An interrupt is generated if the CTSIE bit in the USART_CR3 register is set. [Figure 637](#) shows an example of communication with CTS flow control enabled.

Figure 637. RS232 CTS flow control



Note: For correct behavior, CTS must be deasserted at least 3 USART clock source periods before the end of the current character. In addition it should be noted that the CTSCF flag may not be set for pulses shorter than $2 \times PCLK$ periods.

RS485 driver enable

The driver enable feature is enabled by setting bit DEM in the USART_CR3 control register. This enables the user to activate the external transceiver control, through the DE (Driver Enable) signal. The assertion time is the time between the activation of the DE signal and the beginning of the start bit. It is programmed using the DEAT [4:0] bitfields in the USART_CR1 control register. The deassertion time is the time between the end of the last stop bit, in a transmitted message, and the de-activation of the DE signal. It is programmed using the DEDT [4:0] bitfields in the USART_CR1 control register. The polarity of the DE signal can be configured using the DEP bit in the USART_CR3 control register.

In USART, the DEAT and DEDT are expressed in sample time units ($1/8$ or $1/16$ bit time, depending on the oversampling rate).

51.5.21 USART low-power management

The USART has advanced low-power mode functions, that enables transferring properly data even when the `usart_pclk` clock is disabled.

The USART is able to wake up the MCU from low-power mode when the `UESM` bit is set.

When the `usart_pclk` is gated, the USART provides a wake-up interrupt (**`usart_wkup`**) if a specific action requiring the activation of the **`usart_pclk`** clock is needed:

- If FIFO mode is disabled
 `usart_pclk` clock has to be activated to empty the USART data register.
 In this case, the `usart_wkup` interrupt source is `RXNE` set to '1'. The `RXNEIE` bit must be set before entering low-power mode.
- If FIFO mode is enabled
 `usart_pclk` clock has to be activated to:
 - to fill the `TXFIFO`
 - or to empty the `RXFIFO` In this case, the `usart_wkup` interrupt source can be:
 - `RXFIFO` not empty. In this case, the `RXFNEIE` bit must be set before entering low-power mode.
 - `RXFIFO` full. In this case, the `RXFFIE` bit must be set before entering low-power mode, the number of received data corresponds to the `RXFIFO` size, and the `RXFF` flag is not set.
 - `TXFIFO` empty. In this case, the `TXFEIE` bit must be set before entering low-power mode.

This enables sending/receiving the data in the `TXFIFO`/`RXFIFO` during low-power mode.

To avoid overrun/underrun errors and transmit/receive data in low-power mode, the `usart_wkup` interrupt source can be one of the following events:

- `TXFIFO` threshold reached. In this case, the `TXFTIE` bit must be set before entering low-power mode.
- `RXFIFO` threshold reached. In this case, the `RXFTIE` bit must be set before entering low-power mode.

For example, the application can set the threshold to the maximum `RXFIFO` size if the wake-up time is less than the time required to receive a single byte across the line.

Using the `RXFIFO` full, `TXFIFO` empty, `RXFIFO` not empty and `RXFIFO`/`TXFIFO` threshold interrupts to wake up the MCU from low-power mode enables doing as many USART transfers as possible during low-power mode with the benefit of optimizing consumption.

Alternatively, a specific **`usart_wkup`** interrupt can be selected through the `WUS` bitfields.

When the wake-up event is detected, the `WUF` flag is set by hardware and a **`usart_wkup`** interrupt is generated if the `WUFIE` bit is set.

Note: *Before entering low-power mode, make sure that no USART transfers are ongoing. Checking the BUSY flag cannot ensure that low-power mode is never entered when data reception is ongoing.*

The WUF flag is set when a wake-up event is detected, independently of whether the MCU is in low-power or active mode.

When entering low-power mode just after having initialized and enabled the receiver, the REACK bit must be checked to make sure the USART is enabled.

When DMA is used for reception, it must be disabled before entering low-power mode and re-enabled when exiting from low-power mode.

When the FIFO is enabled, waking up from low-power mode on address match is only possible when Mute mode is enabled.

Using Mute mode with low-power mode

If the USART is put into Mute mode before entering low-power mode:

- Wake-up from Mute mode on idle detection must not be used, because idle detection cannot work in low-power mode.
- If the wake-up from Mute mode on address match is used, then the low-power mode wake-up source must also be the address match. If the RXNE flag was set when entering the low-power mode, the interface remains in Mute mode upon address match and wake up from low-power mode.

Note: *When FIFO management is enabled, Mute mode can be used with wake-up from low-power mode without any constraints (i.e. the two points mentioned above about Mute and low-power mode are valid only when FIFO management is disabled).*

Wake-up from low-power mode when USART kernel clock (usart_ker_ck) is OFF in low-power mode

If during low-power mode, the usart_ker_ck clock is switched OFF when a falling edge on the USART receive line is detected, the USART interface requests the usart_ker_ck clock to be switched ON thanks to the usart_ker_ck_req signal. usart_ker_ck is then used for the frame reception.

If the wake-up event is verified, the MCU wakes up from low-power mode and data reception goes on normally.

If the wake-up event is not verified, usart_ker_ck is switched OFF again, the MCU is not woken up and remains in low-power mode, and the kernel clock request is released.

The example below shows the case of a wake-up event programmed to “address match detection” and FIFO management disabled.

Figure 638 shows the USART behavior when the wake-up event is verified.

Figure 638. Wake-up event verified (wake-up event = address match, FIFO disabled)

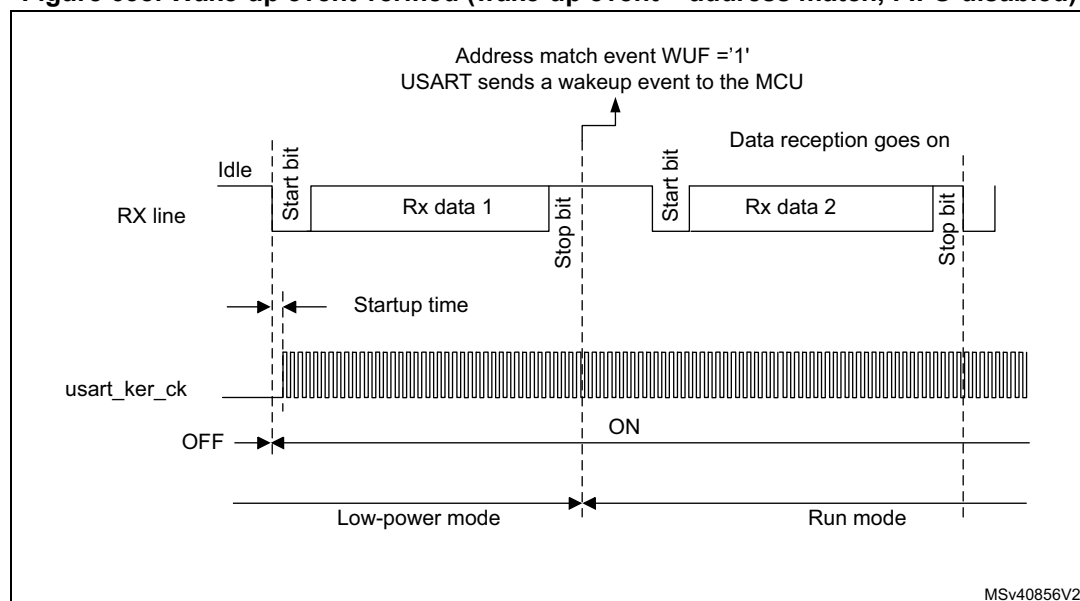
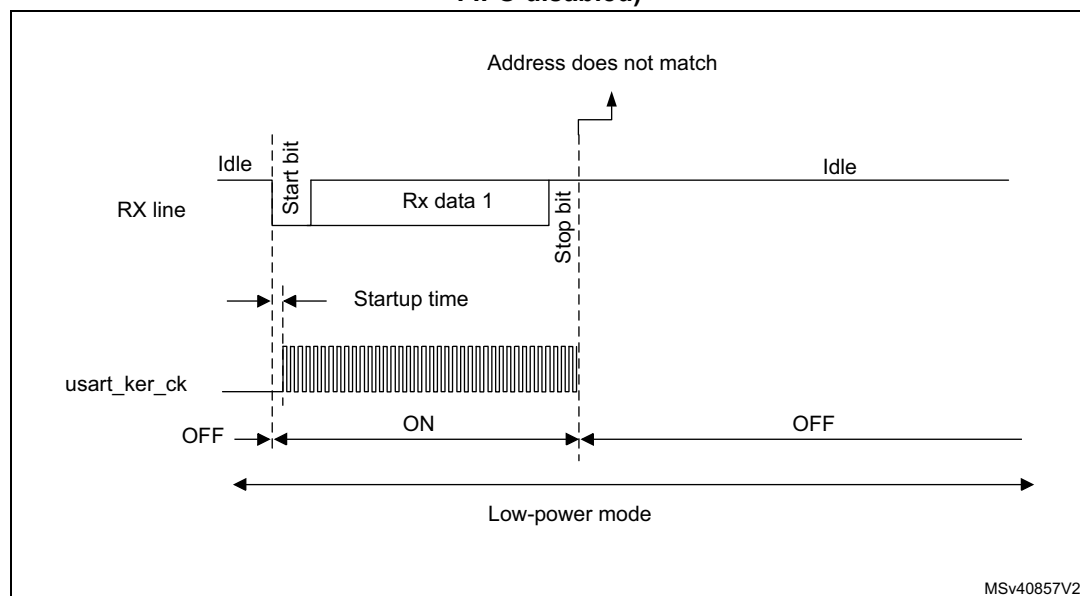


Figure 639 shows the USART behavior when the wake-up event is not verified.

Figure 639. Wake-up event not verified (wake-up event = address match, FIFO disabled)



Note:

The figures above are valid when address match or any received frame is used as wake-up event. If the wake-up event is the start bit detection, the USART sends the wake-up event to the MCU at the end of the start bit.

Determining the maximum USART baud rate that enables to correctly wake up the device from low-power mode

The maximum baud rate that enables to correctly wake up the device from low-power mode depends on the wake-up time parameter (refer to the device datasheet) and on the USART receiver tolerance (see [Section 51.5.8: Tolerance of the USART receiver to clock deviation](#)).

Let us take the example of OVER8 = 0, M bits = '01', ONEBIT = 0 and BRR [3:0] = 0000.

In these conditions, according to [Table 421: Tolerance of the USART receiver when BRR \[3:0\] = 0000](#), the USART receiver tolerance equals 3.41%.

$$DTRA + DQUANT + DREC + DTCL + DWU < \text{USART receiver tolerance}$$

$$D_{WUmax} = t_{WUUSART} / (11 \times T_{bit \text{ Min}})$$

$$T_{bit \text{ Min}} = t_{WUUSART} / (11 \times D_{WUmax})$$

where $t_{WUUSART}$ is the wake-up time from low-power mode.

If we consider the ideal case where DTRA, DQUANT, DREC and DTCL parameters are at 0%, the maximum value of DWU is 3.41%. In reality, we need to consider at least the usart_ker_ck inaccuracy.

For example, if HSI is used as usart_ker_ck, and the HSI inaccuracy is of 1%, then we obtain:

$t_{WUUSART} = 3 \mu\text{s}$ (values provided only as examples; for correct values, refer to the device datasheet).

$$D_{WUmax} = 3.41\% - 1\% = 2.41\%$$

$$T_{bit \text{ min}} = 3 \mu\text{s} / (11 \times 2.41\%) = 11.32 \mu\text{s}.$$

As a result, the maximum baud rate that enables to wake up correctly from low-power mode is: $1/11.32 \mu\text{s} = 88.36 \text{ Kbaud}$.

51.6 USART in low-power modes

Table 424. Effect of low-power modes on the USART

Mode	Description
Sleep	No effect. USART interrupts cause the device to exit Sleep mode.
Stop ⁽¹⁾	The content of the USART registers is kept. The USART is able to wake up the microcontroller from Stop mode when the USART is clocked by an oscillator available in Stop mode.
Standby	The USART peripheral is powered down and must be reinitialized after exiting Standby mode.

1. Refer to [Section 51.4: USART implementation](#) to know if the wake-up from Stop mode is supported for a given peripheral instance. If an instance is not functional in a given Stop mode, it must be disabled before entering this Stop mode.

51.7 USART interrupts

Refer to [Table 425](#) for a detailed description of all USART interrupt requests.

Table 425. USART interrupt requests

Interrupt vector	Interrupt event	Event flag	Enable Control bit	Interrupt clear method	Exit from Sleep mode	Exit from Stop ⁽¹⁾ modes	Exit from Standby mode
USART or UART	Transmit data register empty	TXE	TXEIE	Write TDR	Yes	No	No
	Transmit FIFO not Full	TXFNF	TXFNFI	TXFIFO full		No	
	Transmit FIFO Empty	TXFE	TXFEIE	Write TDR or write 1 in TXFRQ		Yes	
	Transmit FIFO threshold reached	TXFT	TXFTIE	Write TDR		Yes	
	CTS interrupt	CTSIF	CTSIE	Write 1 in CTSCF		No	
	Transmission Complete	TC	TCIE	Write TDR or write 1 in TCCF		No	
	Transmission Complete Before Guard Time	TCBGT	TCBGTIE	Write TDR or write 1 in TCBGT		No	
USART or UART	Receive data register not empty (data ready to be read)	RXNE	RXNEIE	Read RDR or write 1 in RXFRQ	Yes	Yes	No
	Receive FIFO Not Empty	RXFNE	RXFNEIE	Read RDR until RXFIFO empty or write 1 in RXFRQ		Yes	
	Receive FIFO Full	RXFF ⁽²⁾	RXFFIE	Read RDR		Yes	
	Receive FIFO threshold reached	RXFT	RXFTIE	Read RDR		Yes	
	Overrun error detected	ORE	RXNEIE/RXFNEIE	Write 1 in ORECF		No	
	Idle line detected	IDLE	IDLEIE	Write 1 in IDLECF		No	
	Parity error	PE	PEIE	Write 1 in PECF		No	
	LIN break	LBDF	LBDIE	Write 1 in LBDCF		No	
	Noise error in multibuffer communication	NE	EIE	Write 1 in NFCF		No	
	Overrun error in multibuffer communication	ORE ⁽³⁾		Write 1 in ORECF		No	
	Framing Error in multibuffer communication	FE		Write 1 in FECF		No	
	Character match	CMF	CMIE	Write 1 in CMCF		No	
	Receiver timeout	RTOF	RTOFIE	Write 1 in RTOCCF		No	
	End of Block	EOBF	EOBIE	Write 1 in EOBCF		No	
	Wake-up from low-power mode	WUF	WUFIE	Write 1 in WUC		Yes	
	SPI slave underrun error	UDR	EIE	Write 1 in UDRCF		No	

1. The USART can wake up the device from Stop mode only if the peripheral instance supports the wake-up from Stop mode feature. Refer to [Section 51.4: USART implementation](#) for the list of supported Stop modes.

2. RXFF flag is asserted if the USART receives n+1 data (n being the RXFIFO size): n data in the RXFIFO and 1 data in USART_RDR. In Stop mode, USART_RDR is not clocked. As a result, this register is not written and once n data are received and written in the RXFIFO, the RXFF interrupt is asserted (RXFF flag is not set).
3. When OVRDIS = 0.

51.8 USART registers

Refer to [Section 1.2 on page 106](#) for a list of abbreviations used in register descriptions.

The peripheral registers have to be accessed by words (32 bits).

51.8.1 USART control register 1 (USART_CR1)

Address offset: 0x00

Reset value: 0x0000 0000

The same register can be used in FIFO mode enabled (this section) and FIFO mode disabled (next section).

FIFO mode enabled

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RXF FIE	TXFE IE	FIFO EN	M1	EOBIE	RTOIE	DEAT[4:0]					DEDT[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	CMIE	MME	M0	WAKE	PCE	PS	PEIE	TXFNFI E	TCIE	RXFNE IE	IDLEIE	TE	RE	UESM	UE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **RXFFIE**: RXFIFO full interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated when RXFF = 1 in the USART_ISR register

Bit 30 **TXFEIE**: TXFIFO empty interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated when TXFE = 1 in the USART_ISR register

Bit 29 **FIFOEN**: FIFO mode enable

This bit is set and cleared by software.

0: FIFO mode is disabled.

1: FIFO mode is enabled.

This bitfield can only be written when the USART is disabled (UE = 0).

Note: FIFO mode can be used on standard UART communication, in SPI master/slave mode and in Smartcard modes only. It must not be enabled in IrDA and LIN modes.

Bit 28 M1: Word length

This bit must be used in conjunction with bit 12 (M0) to determine the word length. It is set or cleared by software.

M[1:0] = '00': 1 start bit, 8 Data bits, n Stop bit

M[1:0] = '01': 1 start bit, 9 Data bits, n Stop bit

M[1:0] = '10': 1 start bit, 7 Data bits, n Stop bit

This bit can only be written when the USART is disabled (UE = 0).

Note: In 7-bits data length mode, the Smartcard mode, LIN master mode and auto baud rate (0x7F and 0x55 frames detection) are not supported.

Bit 27 EOBF: End-of-block interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated when the EOBF flag is set in the USART_ISR register

Note: If the USART does not support Smartcard mode, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bit 26 RTOIE: Receiver timeout interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated when the RTOF bit is set in the USART_ISR register.

Note: If the USART does not support the Receiver timeout feature, this bit is reserved and must be kept at reset value. [Section 51.4: USART implementation on page 2172](#).

Bits 25:21 DEAT[4:0]: Driver enable assertion time

This 5-bit value defines the time between the activation of the DE (Driver Enable) signal and the beginning of the start bit. It is expressed in sample time units (1/8 or 1/16 bit time, depending on the oversampling rate).

This bitfield can only be written when the USART is disabled (UE = 0).

Note: If the Driver Enable feature is not supported, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bits 20:16 DEDT[4:0]: Driver enable deassertion time

This 5-bit value defines the time between the end of the last stop bit, in a transmitted message, and the de-activation of the DE (Driver Enable) signal. It is expressed in sample time units (1/8 or 1/16 bit time, depending on the oversampling rate).

If the USART_TDR register is written during the DEDT time, the new data is transmitted only when the DEDT and DEAT times have both elapsed.

This bitfield can only be written when the USART is disabled (UE = 0).

Note: If the Driver Enable feature is not supported, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bit 15 OVER8: Oversampling mode

0: Oversampling by 16

1: Oversampling by 8

This bit can only be written when the USART is disabled (UE = 0).

Note: In LIN, IrDA and Smartcard modes, this bit must be kept cleared.

Bit 14 CMIE: Character match interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated when the CMF bit is set in the USART_ISR register.

Bit 13 MME: Mute mode enable

This bit enables the USART Mute mode function. When set, the USART can switch between active and Mute mode, as defined by the WAKE bit. It is set and cleared by software.

0: Receiver in active mode permanently

1: Receiver can switch between Mute mode and active mode.

Bit 12 M0: Word length

This bit is used in conjunction with bit 28 (M1) to determine the word length. It is set or cleared by software (refer to bit 28 (M1) description).

This bit can only be written when the USART is disabled (UE = 0).

Bit 11 WAKE: Receiver wake-up method

This bit determines the USART wake-up method from Mute mode. It is set or cleared by software.

0: Idle line

1: Address mark

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 10 PCE: Parity control enable

This bit selects the hardware parity control (generation and detection). When the parity control is enabled, the computed parity is inserted at the MSB position (9th bit if M = 1; 8th bit if M = 0) and the parity is checked on the received data. This bit is set and cleared by software. Once it is set, PCE is active after the current byte (in reception and in transmission).

0: Parity control disabled

1: Parity control enabled

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 9 PS: Parity selection

This bit selects the odd or even parity when the parity generation/detection is enabled (PCE bit set). It is set and cleared by software. The parity is selected after the current byte.

0: Even parity

1: Odd parity

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 8 PEIE: PE interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever PE = 1 in the USART_ISR register

Bit 7 TXFNFIE: TXFIFO not-full interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever TXFNF = 1 in the USART_ISR register

Bit 6 TCIE: Transmission complete interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever TC = 1 in the USART_ISR register

Bit 5 RXFNEIE: RXFIFO not empty interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever ORE = 1 or RXFNE = 1 in the USART_ISR register

Bit 4 **IDLEIE**: IDLE interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever IDLE = 1 in the USART_ISR register

Bit 3 **TE**: Transmitter enable

This bit enables the transmitter. It is set and cleared by software.

0: Transmitter is disabled

1: Transmitter is enabled

Note: During transmission, a low pulse on the TE bit ('0' followed by '1') sends a preamble (idle line) after the current word, except in Smartcard mode. In order to generate an idle character, the TE must not be immediately written to '1'. To ensure the required duration, the software can poll the TEACK bit in the USART_ISR register.

In Smartcard mode, when TE is set, there is a 1 bit-time delay before the transmission starts.

Bit 2 **RE**: Receiver enable

This bit enables the receiver. It is set and cleared by software.

0: Receiver is disabled

1: Receiver is enabled and begins searching for a start bit

Bit 1 **UESM**: USART enable in low-power mode

When this bit is cleared, the USART cannot wake up the MCU from low-power mode.

When this bit is set, the USART can wake up the MCU from low-power mode.

This bit is set and cleared by software.

0: USART not able to wake up the MCU from low-power mode.

1: USART able to wake up the MCU from low-power mode.

Note: It is recommended to set the UESM bit just before entering low-power mode and clear it when exit from low-power mode.

If the USART does not support the wake-up from Stop feature, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bit 0 **UE**: USART enable

When this bit is cleared, the USART prescalers and outputs are stopped immediately, and all current operations are discarded. The USART configuration is kept, but all the USART_ISR status flags are reset. This bit is set and cleared by software.

0: USART prescaler and outputs disabled, low-power mode

1: USART enabled

Note: To enter low-power mode without generating errors on the line, the TE bit must be previously reset and the software must wait for the TC bit in the USART_ISR to be set before resetting the UE bit.

The DMA requests are also reset when UE = 0 so the DMA channel must be disabled before resetting the UE bit.

In Smartcard mode, (SCEN = 1), the CK is always available when CLKEN = 1, regardless of the UE bit value.

51.8.2 USART control register 1 [alternate] (USART_CR1)

Address offset: 0x00

Reset value: 0x0000 0000

The same register can be used in FIFO mode enabled (previous section) and FIFO mode disabled (this section).

FIFO mode disabled

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	FIFO EN	M1	EOBIE	RTOIE	DEAT[4:0]					DEDT[4:0]				
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	CMIE	MME	M0	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	UESM	UE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **FIFOEN**: FIFO mode enable

This bit is set and cleared by software.

0: FIFO mode is disabled.

1: FIFO mode is enabled.

This bitfield can only be written when the USART is disabled (UE = 0).

Note: FIFO mode can be used on standard UART communication, in SPI master/slave mode and in Smartcard modes only. It must not be enabled in IrDA and LIN modes.

Bit 28 **M1**: Word length

This bit must be used in conjunction with bit 12 (M0) to determine the word length. It is set or cleared by software.

M[1:0] = '00': 1 start bit, 8 Data bits, n Stop bit

M[1:0] = '01': 1 start bit, 9 Data bits, n Stop bit

M[1:0] = '10': 1 start bit, 7 Data bits, n Stop bit

This bit can only be written when the USART is disabled (UE = 0).

Note: In 7-bits data length mode, the Smartcard mode, LIN master mode and auto baud rate (0x7F and 0x55 frames detection) are not supported.

Bit 27 **EOBIE**: End of Bblock interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated when the EOBIF flag is set in the USART_ISR register

Note: If the USART does not support Smartcard mode, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bit 26 **RTOIE**: Receiver timeout interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated when the RTOF bit is set in the USART_ISR register.

Note: If the USART does not support the Receiver timeout feature, this bit is reserved and must be kept at reset value. [Section 51.4: USART implementation on page 2172](#).

Bits 25:21 DEAT[4:0]: Driver enable assertion time

This 5-bit value defines the time between the activation of the DE (Driver Enable) signal and the beginning of the start bit. It is expressed in sample time units (1/8 or 1/16 bit time, depending on the oversampling rate).

This bitfield can only be written when the USART is disabled (UE = 0).

Note: If the Driver Enable feature is not supported, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bits 20:16 DEDT[4:0]: Driver enable deassertion time

This 5-bit value defines the time between the end of the last stop bit, in a transmitted message, and the de-activation of the DE (Driver Enable) signal. It is expressed in sample time units (1/8 or 1/16 bit time, depending on the oversampling rate).

If the USART_TDR register is written during the DEDT time, the new data is transmitted only when the DEDT and DEAT times have both elapsed.

This bitfield can only be written when the USART is disabled (UE = 0).

Note: If the Driver Enable feature is not supported, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bit 15 OVER8: Oversampling mode

0: Oversampling by 16

1: Oversampling by 8

This bit can only be written when the USART is disabled (UE = 0).

Note: In LIN, IrDA and Smartcard modes, this bit must be kept cleared.

Bit 14 CMIE: Character match interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated when the CMF bit is set in the USART_ISR register.

Bit 13 MME: Mute mode enable

This bit enables the USART Mute mode function. When set, the USART can switch between active and Mute mode, as defined by the WAKE bit. It is set and cleared by software.

0: Receiver in active mode permanently

1: Receiver can switch between Mute mode and active mode.

Bit 12 M0: Word length

This bit is used in conjunction with bit 28 (M1) to determine the word length. It is set or cleared by software (refer to bit 28 (M1) description).

This bit can only be written when the USART is disabled (UE = 0).

Bit 11 WAKE: Receiver wake-up method

This bit determines the USART wake-up method from Mute mode. It is set or cleared by software.

0: Idle line

1: Address mark

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 10 PCE: Parity control enable

This bit selects the hardware parity control (generation and detection). When the parity control is enabled, the computed parity is inserted at the MSB position (9th bit if M = 1; 8th bit if M = 0) and the parity is checked on the received data. This bit is set and cleared by software. Once it is set, PCE is active after the current byte (in reception and in transmission).

0: Parity control disabled

1: Parity control enabled

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 9 PS: Parity selection

This bit selects the odd or even parity when the parity generation/detection is enabled (PCE bit set). It is set and cleared by software. The parity is selected after the current byte.

0: Even parity

1: Odd parity

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 8 PEIE: PE interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever PE = 1 in the USART_ISR register

Bit 7 TXEIE: Transmit data register empty

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever TXE = 1 in the USART_ISR register

Bit 6 TCIE: Transmission complete interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever TC = 1 in the USART_ISR register

Bit 5 RXNEIE: Receive data register not empty

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever ORE = 1 or RXNE = 1 in the USART_ISR register

Bit 4 IDLEIE: IDLE interrupt enable

This bit is set and cleared by software.

0: Interrupt inhibited

1: USART interrupt generated whenever IDLE = 1 in the USART_ISR register

Bit 3 TE: Transmitter enable

This bit enables the transmitter. It is set and cleared by software.

0: Transmitter is disabled

1: Transmitter is enabled

Note: During transmission, a low pulse on the TE bit ('0' followed by '1') sends a preamble (idle line) after the current word, except in Smartcard mode. In order to generate an idle character, the TE must not be immediately written to '1'. To ensure the required duration, the software can poll the TEACK bit in the USART_ISR register.

In Smartcard mode, when TE is set, there is a 1 bit-time delay before the transmission starts.

Bit 2 **RE**: Receiver enable

This bit enables the receiver. It is set and cleared by software.

0: Receiver is disabled

1: Receiver is enabled and begins searching for a start bit

Bit 1 **UESM**: USART enable in low-power mode

When this bit is cleared, the USART cannot wake up the MCU from low-power mode.

When this bit is set, the USART can wake up the MCU from low-power mode.

This bit is set and cleared by software.

0: USART not able to wake up the MCU from low-power mode.

1: USART able to wake up the MCU from low-power mode.

Note: It is recommended to set the UESM bit just before entering low-power mode and clear it when exit from low-power mode.

If the USART does not support the wake-up from Stop feature, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bit 0 **UE**: USART enable

When this bit is cleared, the USART prescalers and outputs are stopped immediately, and all current operations are discarded. The USART configuration is kept, but all the USART_ISR status flags are reset. This bit is set and cleared by software.

0: USART prescaler and outputs disabled, low-power mode

1: USART enabled

Note: To enter low-power mode without generating errors on the line, the TE bit must be previously reset and the software must wait for the TC bit in the USART_ISR to be set before resetting the UE bit.

The DMA requests are also reset when UE = 0 so the DMA channel must be disabled before resetting the UE bit.

In Smartcard mode, (SCEN = 1), the CK pin is always available when CLKEN = 1, regardless of the UE bit value.

51.8.3 USART control register 2 (USART_CR2)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADD[7:0]								RTOEN	ABRMOD[1:0]		ABREN	MSBFIRST	DATAINV	TXINV	RXINV
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWAP	LINEN	STOP[1:0]		CLKEN	CPOL	CPHA	LBCL	Res.	LBDIE	LBDL	ADDM7	DISNSS	Res.	Res.	SLVEN
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw			rw

Bits 31:24 **ADD[7:0]**: Address of the USART node

These bits give the address of the USART node in Mute mode or a character code to be recognized in low-power or Run mode:

- In Mute mode: they are used in multiprocessor communication to wake up from Mute mode with 4-bit/7-bit address mark detection. The MSB of the character sent by the transmitter should be equal to 1. In 4-bit address mark detection, only ADD[3:0] bits are used.
- In low-power mode: they are used for wake up from low-power mode on character match. When WUS[1:0] is programmed to 0b00 (WUF active on address match), the wake-up from low-power mode is performed when the received character corresponds to the character programmed through ADD[6:0] or ADD[3:0] bitfield (depending on ADDM7 bit), and WUF interrupt is enabled by setting WUFIE bit. The MSB of the character sent by transmitter should be equal to 1.
- In Run mode with Mute mode inactive (for example, end-of-block detection in ModBus protocol): the whole received character (8 bits) is compared to ADD[7:0] value and CMF flag is set on match. An interrupt is generated if the CMIE bit is set.

These bits can only be written when the reception is disabled (RE = 0) or when the USART is disabled (UE = 0).

Bit 23 **RTOEN**: Receiver timeout enable

This bit is set and cleared by software.

0: Receiver timeout feature disabled.

1: Receiver timeout feature enabled.

When this feature is enabled, the RTOF flag in the USART_ISR register is set if the RX line is idle (no reception) for the duration programmed in the RTOR (receiver timeout register).

Note: If the USART does not support the Receiver timeout feature, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bits 22:21 **ABRMOD[1:0]**: Auto baud rate mode

These bits are set and cleared by software.

00: Measurement of the start bit is used to detect the baud rate.

01: Falling edge to falling edge measurement (the received frame must start with a single bit = 1 and Frame = Start10xxxxxx)

10: 0x7F frame detection.

11: 0x55 frame detection

This bitfield can only be written when ABREN = 0 or the USART is disabled (UE = 0).

Note: If DATAINV = 1 and/or MSBFIRST = 1 the patterns must be the same on the line, for example 0xAA for MSBFIRST)

If the USART does not support the auto baud rate feature, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bit 20 **ABREN**: Auto baud rate enable

This bit is set and cleared by software.

0: Auto baud rate detection is disabled.

1: Auto baud rate detection is enabled.

Note: If the USART does not support the auto baud rate feature, this bit is reserved and must be kept at reset value. Refer to [Section 51.4: USART implementation on page 2172](#).

Bit 19 **MSBFIRST**: Most significant bit first

This bit is set and cleared by software.

0: data is transmitted/received with data bit 0 first, following the start bit.

1: data is transmitted/received with the MSB (bit 7/8) first, following the start bit.

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 18 DATAINV: Binary data inversion

This bit is set and cleared by software.

0: Logical data from the data register are send/received in positive/direct logic. (1 = H, 0 = L)

1: Logical data from the data register are send/received in negative/inverse logic. (1 = L, 0 = H).

The parity bit is also inverted.

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 17 TXINV: TX pin active level inversion

This bit is set and cleared by software.

0: TX pin signal works using the standard logic levels (V_{DD} = 1/idle, Gnd = 0/mark)

1: TX pin signal values are inverted (V_{DD} = 0/mark, Gnd = 1/idle).

This enables the use of an external inverter on the TX line.

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 16 RXINV: RX pin active level inversion

This bit is set and cleared by software.

0: RX pin signal works using the standard logic levels (V_{DD} = 1/idle, Gnd = 0/mark)

1: RX pin signal values are inverted (V_{DD} = 0/mark, Gnd = 1/idle).

This enables the use of an external inverter on the RX line.

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 15 SWAP: Swap TX/RX pins

This bit is set and cleared by software.

0: TX/RX pins are used as defined in standard pinout

1: The TX and RX pins functions are swapped. This enables to work in the case of a cross-wired connection to another UART.

This bitfield can only be written when the USART is disabled (UE = 0).

Bit 14 LINEN: LIN mode enable

This bit is set and cleared by software.

0: LIN mode disabled

1: LIN mode enabled

The LIN mode enables the capability to send LIN synchronous breaks (13 low bits) using the SBKRQ bit in the USART_CR1 register, and to detect LIN Sync breaks.

This bitfield can only be written when the USART is disabled (UE = 0).

Note: If the USART does not support LIN mode, this bit is reserved and must be kept at reset value.

Refer to [Section 51.4: USART implementation on page 2172](#).

Bits 13:12 STOP[1:0]: Stop bits

These bits are used for programming the stop bits.

00: 1 stop bit

01: 0.5 stop bit.

10: 2 stop bits

11: 1.5 stop bits

This bitfield can only be written when the USART is disabled (UE = 0).

59 Controller area network with flexible data rate (FDCAN)

59.1 Introduction

The controller area network (CAN) subsystem (see [Figure 775](#)) consists of two CAN modules, a shared message RAM, and a clock calibration unit. Refer to the product memory organization for the base address of each of them.

FDCAN modules are compliant with ISO 11898-1: 2015 (CAN protocol specification version 2.0 part A, B) and CAN FD protocol specification version 1.0.

In addition, the first CAN module FDCAN1 supports time triggered CAN (TTCAN), specified in ISO 11898-4, including event synchronized time-triggered communication, global system time, and clock drift compensation. The FDCAN1 contains additional registers, specific to the time triggered feature. The CAN FD option can be used together with event-triggered and time triggered CAN communication.

A 10-Kbyte message RAM implements filters, receive FIFOs, receive buffers, transmit event FIFOs, transmit buffers (and triggers for TTCAN). This memory is shared between the FDCAN modules.

The common clock calibration unit is optional. It can be used to generate a calibrated clock for each FDCAN from the HSI internal RC oscillator and the PLL, by evaluating CAN messages received by the FDCAN1.

The CAN subsystem I/O signals and pins are detailed, respectively, in [Table 499](#) and [Table 500](#).

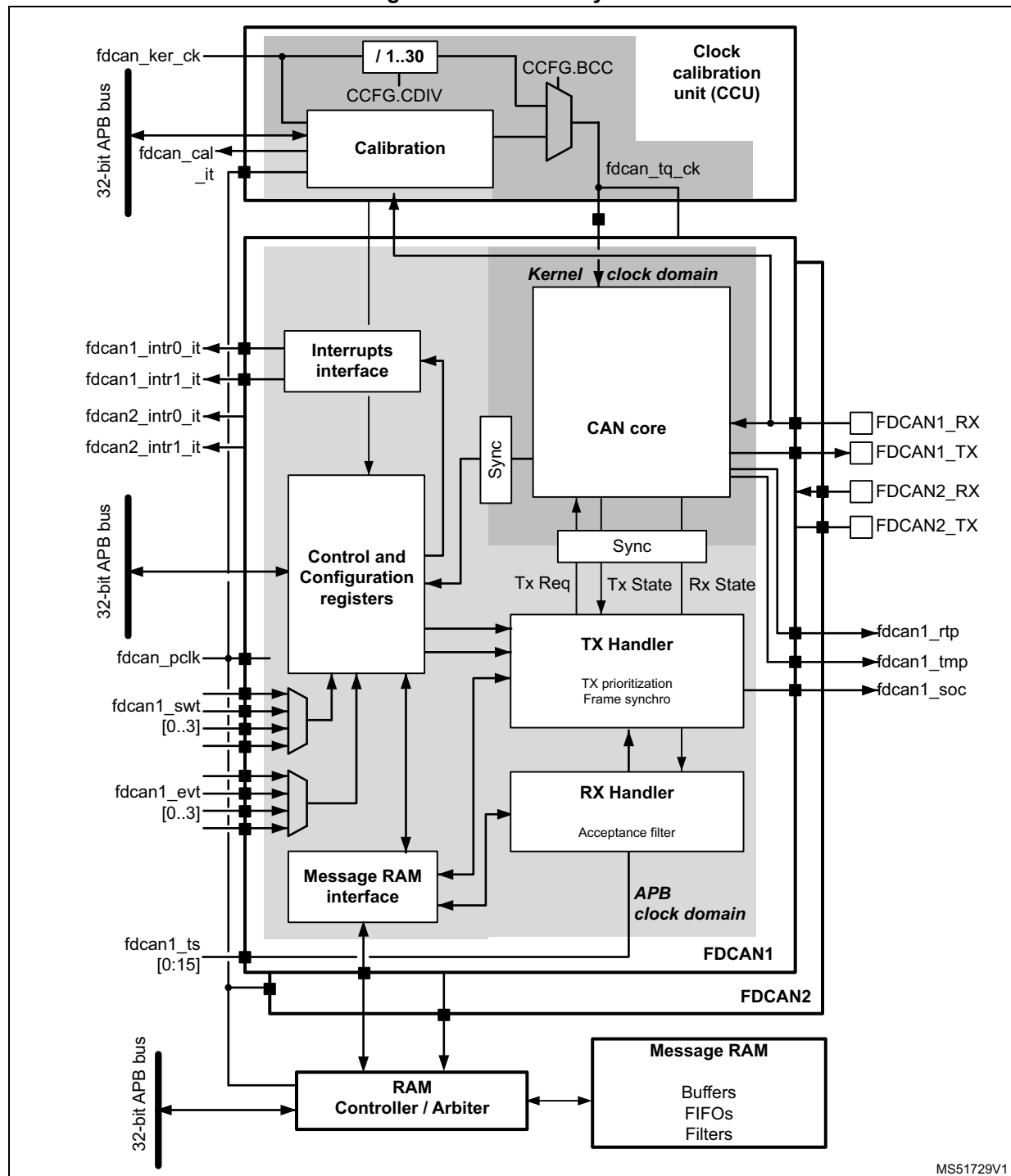
Table 499. CAN subsystem I/O signals

Name	Type	Description
fdcan_ker_ck	Digital input	CAN subsystem kernel clock input
fdcan_pclk		CAN subsystem APB interface clock input
fdcan_cal_it	Digital output	FDCAN calibration interrupt
fdcan1_intr0_it	Digital output	FDCAN1 interrupt0
fdcan1_intr1_it		FDCAN1 interrupt1
fdcan2_intr0_it	Digital output	FDCAN2 interrupt0
fdcan2_intr1_it		FDCAN2 interrupt1
fdcan1_swt[0:3]	Digital input	Stop watch trigger input
fdcan1_evt[0:3]		Event trigger input
fdcan1_ts[0:15]		External timestamp vector
fdcan1_soc	Digital output	Start of cycle pulse
fdcan1_rtp		Register time mark pulse
fdcan1_tmp		Trigger time mark pulse

Table 500. CAN subsystem I/O pins

Name	Type	Description
FDCAN1_RX	Digital input	FDCAN1 receive pin
FDCAN1_TX	Digital output	FDCAN1 transmit pin
FDCAN2_RX	Digital input	FDCAN2 receive pin
FDCAN2_TX	Digital output	FDCAN2 transmit pin

Figure 775. CAN subsystem



MS51729V1

59.2 FDCAN main features

- Conform with CAN protocol version 2.0 part A, B, and ISO 11898-1: 2015, -4
- CAN FD with max. 64 data bytes supported
- TTCAN protocol level 1 and level 2 completely in hardware (FDCAN1 only)
- Event synchronized time-triggered communication supported (FDCAN1 only)
- CAN error logging
- AUTOSAR and J1939 support
- Improved acceptance filtering
- Two configurable receive FIFOs
- Separate signaling on reception of high priority messages
- Up to 64 dedicated receive buffers
- Up to 32 dedicated transmit buffers
- Configurable transmit FIFO / queue
- Configurable transmit event FIFO
- FDCAN modules share the same message RAM
- Programmable loop-back test mode
- Maskable module interrupts
- Two clock domains: APB bus interface and CAN core kernel clock
- Power-down support

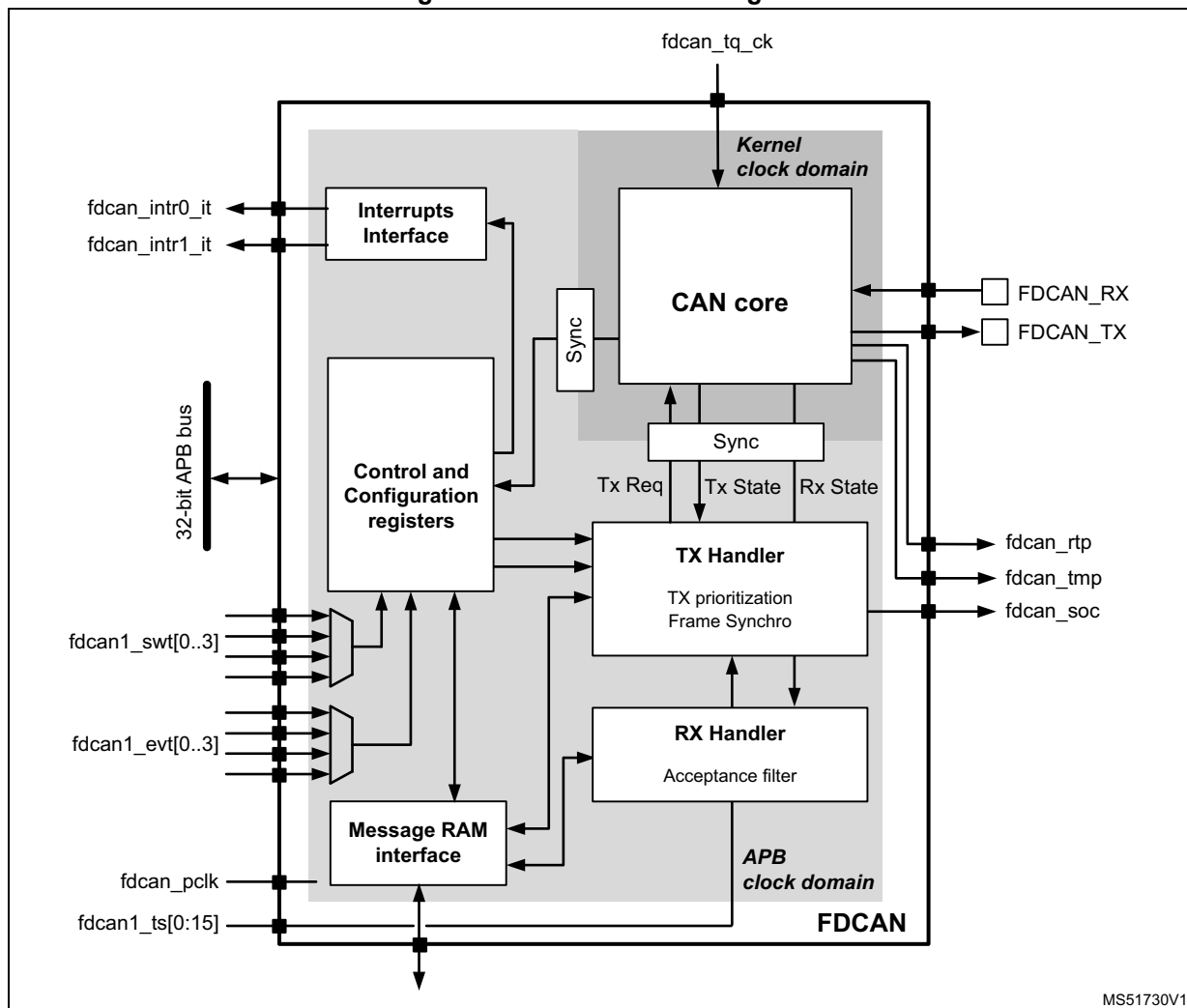
59.3 FDCAN implementation

Table 501. Main features

Module	FDCAN1	FDCAN2
TTCAN	X	-

59.4 FDCAN functional description

Figure 776. FDCAN block diagram



Dual interrupt lines

The FDCAN peripheral provides two interrupt lines, `fdcan_intr0_it` and `fdcan_intr1_it`. By programming `EINT0` and `EINT1` bits in `FDCAN_ILE` register, the interrupt lines can be enabled or disabled separately.

CAN core

The CAN core contains the protocol controller and the receive/transmit shift registers. It handles all ISO 11898-1: 2015 protocol functions, and supports both 11-bit and 29-bit identifiers.

Sync

This block synchronizes signals from the APB clock domain to the CAN kernel clock domain, and vice versa.

Tx handler

Controls the message transfer from the message RAM to the CAN core. A maximum of 32 Tx buffers can be configured for transmission. Tx buffers can be used as dedicated Tx buffers, as Tx FIFO, part of a Tx queue, or as a combination of them. A Tx event FIFO stores Tx timestamps together with the corresponding message ID. Transmit cancellation is also supported.

On FDCAN1, the Tx handler also implements the frame synchronization entity (FSE) which controls time-triggered communication according to ISO11898-4. It synchronizes itself with the reference messages on the CAN bus, controls cycle time and global time, and handles transmissions according to the predefined message schedule, the system matrix. It also handles the time marks of the system matrix that are linked to the messages in the message RAM. Stop watch trigger, event trigger, and time mark interrupt are synchronization interfaces.

Rx handler

Controls the transfer of received messages from the CAN core to the external message RAM. The Rx handler supports two receive FIFOs, each of configurable size, and up to 64 dedicated Rx buffers for storage of all messages that have passed acceptance filtering. A dedicated Rx buffer, in contrast to a receive FIFO, is used to store only messages with a specific identifier. An Rx timestamp is stored together with each message. Up to 128 filters can be defined for 11-bit IDs and up to 64 filters for 29-bit IDs.

APB interface

Connects the FDCAN to the APB bus.

Message RAM interface

Connects the FDCAN access to an external 10 Kbytes message RAM through a RAM controller/arbitrator.

59.4.1 Operating modes

Software initialization

Software initialization is started by setting INIT bit in FDCAN_CCCR register, either by software or by a hardware reset, or by going Bus_Off. While INIT bit in FDCAN_CCCR register is set, message transfer from and to the CAN bus is stopped, the status of the CAN bus output FDCAN_TX is recessive (high). The counters of the error management logic (EML) are unchanged. Setting INIT bit in FDCAN_CCCR does not change any configuration register. Clearing INIT bit in FDCAN_CCCR finishes the software initialization. Afterwards, the bit stream processor (BSP) synchronizes itself to the data transfer on the CAN bus by waiting for the occurrence of a sequence of 11 consecutive recessive bits (Bus_Idle) before it can take part in bus activities and start the message transfer.

Access to the FDCAN configuration registers is only enabled when both INIT bit in FDCAN_CCCR register and CCE bit in FDCAN_CCCR register are set.

CCE bit in FDCAN_CCCR register can be set/cleared only while INIT bit in FDCAN_CCCR is set. CCE bit in FDCAN_CCCR register is automatically cleared when INIT bit in FDCAN_CCCR is cleared.

The following registers are reset when CCE bit in FDCAN_CCCR register is set:

- FDCAN_HPMS - high priority message status
- FDCAN_RXF0S - Rx FIFO 0 status
- FDCAN_RXF1S - Rx FIFO 1 status
- FDCAN_TXFQS - Tx FIFO/queue status
- FDCAN_TXBRP - Tx buffer request pending
- FDCAN_TXBTO - Tx buffer transmission occurred
- FDCAN_TXBCF - Tx buffer cancellation finished
- FDCAN_TXEFS - Tx event FIFO status
- FDCAN_TTOST - TT (time trigger) operation status (FDCAN1 only)
- FDCAN_TTLGT - TT local and global time, only global time FDCAN_TTLGT.GT is reset (FDCAN1 only)
- FDCAN_TTCTC - TT cycle time and count (FDCAN1 only)
- FDCAN_TTCSM - TT cycle sync mark (FDCAN1 only)

The timeout counter value TOC bit in FDCAN_TOCV register is preset to the value configured by TOP bit in FDCAN_TOCC register when CCE bit in FDCAN_CCCR is set.

In addition, the state machines of the Tx handler and Rx handler are held in idle state while CCE bit in FDCAN_CCCR is set.

The following registers can be written only when CCE bit in FDCAN_CCCR register is cleared:

- FDCAN_TXBAR - Tx buffer add request
- FDCAN_TXBCR - Tx buffer cancellation request

TEST bit in FDCAN_CCCR and MON bit in FDCAN_CCCR can be set only by software while both INIT bit and CCE bit in FDCAN_CCCR register are set. Both bits may be reset at any time. DAR bit in FDCAN_CCCR can be set/cleared only while both INIT bit in FDCAN_CCCR and CCE bit in FDCAN_CCCR are set.

Normal operation

The FDCAN1 default operating mode after hardware reset is event-driven CAN communication without time triggers (FDCAN_TTOCF.OM = 00). It is required that both INIT bit and CCE bit in FDCAN_CCCR register are set before the TT operation mode can be changed.

Once the FDCAN is initialized and INIT bit in FDCAN_CCCR register is cleared, the FDCAN synchronizes itself to the CAN bus and is ready for communication.

After passing the acceptance filtering, received messages including message ID and DLC are stored into a dedicated Rx buffer or into the Rx FIFO 0 or Rx FIFO 1.

For messages to be transmitted dedicated Tx buffers and/or a Tx FIFO or a Tx queue can be initialized or updated. Automated transmission on reception of remote frames is not supported.

CAN FD operation

There are two variants in the CAN FD protocol. The first is the long frame mode (LFM), where the data field of a CAN frame can be longer than eight bytes. The second variant is the fast frame mode (FFM), where the control, data, and CRC fields of a CAN frame are

transmitted with a higher bitrate than the beginning and the end of the frame. Fast frame mode can be used in combination with long frame mode.

The previously reserved bit in CAN frames with 11-bit identifiers and the first previously reserved bit in CAN frames with 29-bit identifiers are now decoded as FDF bit. FDF recessive signifies a CAN FD frame, while FDF dominant signifies a classic CAN frame. In a CAN FD frame, the two bits following FDF, res and BRS, decide whether the bitrate inside this CAN FD frame is switched. A CAN FD bitrate switch is signified by res dominant and BRS recessive. The coding of res recessive is reserved for protocol expansions. If the FDCAN receives a frame with FDF recessive and res recessive, it signals a protocol exception event by setting bit FDCAN_PSR.PXE. When protocol exception handling is enabled (FDCAN_CCCR.PXHD = 0), this causes the operation state to change from receiver (FDCAN_PSR.ACT = 10) to integrating (FDCAN_PSR.ACT = 00) at the next sample point. In case protocol exception handling is disabled (FDCAN_CCCR.PXHD = 1), the FDCAN treats a recessive res bit as a form error and responds with an error frame.

CAN FD operation is enabled by programming FDCAN_CCCR.FDOE. If FDCAN_CCCR.FDOE = 1, transmission and reception of CAN FD frames is enabled. Transmission and reception of classic CAN frames is always possible. Whether a CAN FD or a classic CAN frame is transmitted, it can be configured via bit FDF in the respective Tx buffer element. With FDCAN_CCCR.FDOE = 0, received frames are interpreted as classic CAN frames, which leads to the transmission of an error frame when receiving a CAN FD frame. When CAN FD operation is disabled, no CAN FD frames are transmitted, even if bit FDF of a Tx buffer element is set. FDCAN_CCCR.FDOE and FDCAN_CCCR.BRSE can only be changed while FDCAN_CCCR.INIT and FDCAN_CCCR.CCE are both set.

With FDCAN_CCCR.FDOE = 0, the setting of bits FDF and BRS is ignored and frames are transmitted in classic CAN format. With FDCAN_CCCR.FDOE = 1 and FDCAN_CCCR.BRSE = 0, only bit FDF of a Tx buffer element is evaluated. With FDCAN_CCCR.FDOE = 1 and FDCAN_CCCR.BRSE = 1, transmission of CAN FD frames with bitrate switching is enabled. All Tx buffer elements with bits FDF and BRS set are transmitted in CAN FD format with bitrate switching.

A mode change during CAN operation is only recommended under the following conditions:

- The failure rate in the CAN FD data phase is significant higher than in the CAN FD arbitration phase. In this case disable the CAN FD bitrate switching option for transmissions.
- During system startup all nodes are transmitting classic CAN messages until it is verified that they are able to communicate in CAN FD format. If this is true, all nodes switch to CAN FD operation.
- Wake-up messages in CAN partial networking have to be transmitted in classic CAN format.
- End-of-line programming in case not all nodes are CAN FD capable. Non CAN FD nodes are held in Silent mode until programming has completed. Then all nodes switch back to classic CAN communication.

In the CAN FD format, the coding of the DLC differs from the standard CAN format. DLC codes 0 to 8 have the same coding as in standard CAN, codes 9 to 15 (that in standard CAN all code a data field of 8 bytes) are coded according to [Table 502](#).

Table 502. DLC coding in FDCAN

DLC	9	10	11	12	13	14	15
Number of data bytes	12	16	20	24	32	48	64

In CAN FD fast frames, the bit timing is switched inside the frame, after the BRS (bitrate switch) bit, if this bit is recessive. Before the BRS bit, in the CAN FD arbitration phase, the nominal CAN bit timing is used as defined by the bit timing and prescaler register FDCAN_NBTP. In the following CAN FD data phase, the fast CAN bit timing is used as defined by the fast bit timing and prescaler register FDCAN_DBTP. The bit timing is switched back from the fast timing at the CRC delimiter or when an error is detected, whichever occurs first.

The maximum configurable bitrate in the CAN FD data phase depends on the FDCAN kernel clock frequency. For example, with an FDCAN kernel clock frequency of 20 MHz and the shortest configurable bit time of four time quanta (tq), the bitrate in the data phase is 5 Mbit/s.

In both data frame formats, CAN FD long frames and CAN FD fast frames, the value of the bit ESI (error status indicator) is determined by the transmitter error state at the start of the transmission. If the transmitter is error passive, ESI is transmitted recessive, else it is transmitted dominant. In CAN FD remote frames the ESI bit is always transmitted dominant, independent of the transmitter error state. The data length code of CAN FD remote frames is transmitted as 0.

In case an FDCAN Tx buffer is configured for CAN FD transmission with DLC > 8, the first eight bytes are transmitted as configured in the Tx buffer while the remaining part of the data field is padded with 0xCC. When the FDCAN receives an FDCAN frame with DLC > 8, the first eight bytes of that frame are stored into the matching Rx buffer or Rx FIFO. The remaining bytes are discarded.

Transceiver delay compensation

During the data phase of a CAN FD transmission only one node is transmitting, all others are receivers. The length of the bus line has no impact. When transmitting via pin FDCAN_TX the protocol controller receives the transmitted data from its local CAN transceiver via pin FDCAN_RX. The received data is delayed by the CAN transceiver loop delay. If this delay is greater than TSEG1 (time segment before sample point), a bit error is detected. Without transceiver delay compensation, the bitrate in the data phase of a CAN FD frame is limited by the transceivers loop delay.

The FDCAN implements a delay compensation mechanism to compensate the CAN transceiver loop delay, thereby enabling transmission with higher bitrates during the CAN FD data phase, independent from the delay of a specific CAN transceiver.

To check for bit errors during the data phase of transmitting nodes, the delayed transmit data is compared against the received data at the Secondary Sample Point SSP. If a bit error is detected, the transmitter reacts on this bit error at the next following regular sample point. During the arbitration phase the delay compensation is always disabled.

The transmitter delay compensation enables configurations where the data bit time is shorter than the transmitter delay, it is described in detail in the new ISO11898-1. It is enabled by setting bit FDCAN_DBTP.TDC.

The received bit is compared against the transmitted bit at the SSP. The SSP position is defined as the sum of the measured delay from the FDCAN transmit output pin FDCAN_TX through the transceiver to the receive input pin FDCAN_RX plus the transmitter delay compensation offset as configured by FDCAN_TDCR.TDCO. The transmitter delay compensation offset is used to adjust the position of the SSP inside the received bit (for example half of the bit time in the data phase). The position of the secondary sample point is rounded down to the next integer number of minimum time quanta (mtq, that is, one period of fdcan_tq_ck clock).

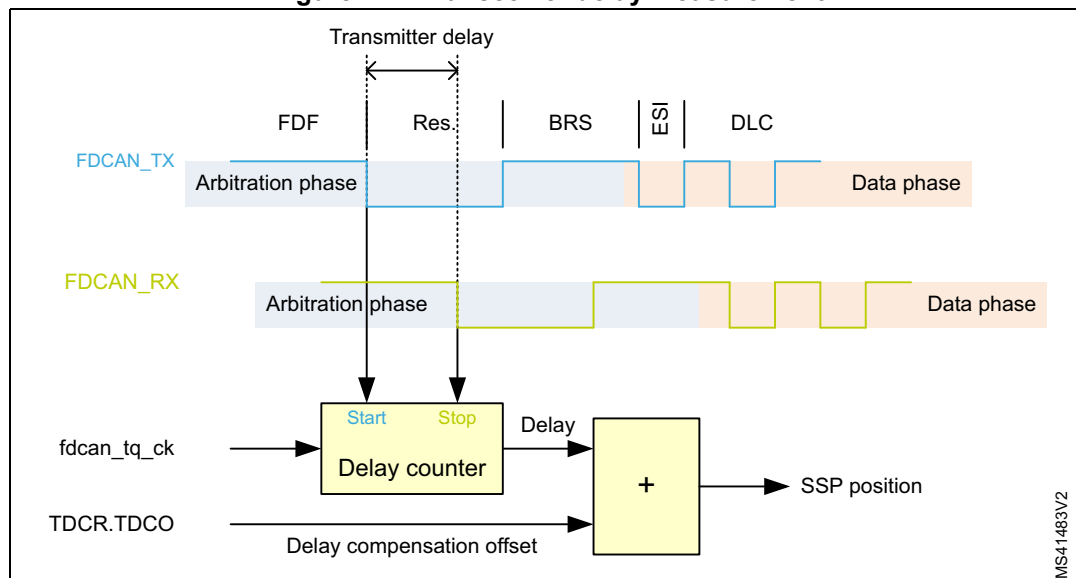
FDCAN_PSR.TDCV shows the actual transmitter delay compensation value, cleared when FDCAN_CCCR.INIT is set, and updated at each transmission of an FD frame while FDCAN_DBTP.TDC is set.

The following boundary conditions have to be considered for the transmitter delay compensation implemented in the FDCAN:

- The sum of the measured delay from FDCANx_TX to FDCANx_RX and the configured transmitter delay compensation offset FDCAN_TDCR.TDCO must be less than six bit times in the data phase.
- The sum of the measured delay from FDCANx_TX to FDCANx_RX and the configured transmitter delay compensation offset FDCAN_TDCR.TDCO must be less than or equal to 127 mtq. If this sum exceeds 127 mtq, the maximum value (127 mtq) is used for transmitter delay compensation.
- The data phase ends at the sample point of the CRC delimiter, which stops checking received bits at the SSPs

If transmitter delay compensation is enabled by programming FDCAN_DBTP.TDC = 1, the measurement is started within each transmitted CAN FD frame at the falling edge of bit FDF to bit res. The measurement is stopped when this edge is seen at the receive input pin FDCAN_TX of the transmitter. The resolution of this measurement is 1 mtq.

Figure 777. Transceiver delay measurement



To avoid that a dominant glitch inside the received FDF bit ends the delay compensation measurement before the falling edge of the received res bit (resulting in a too early SSP position), the use of a transmitter delay compensation filter window can be enabled by programming FDCAN_TDCR.TDCF. This defines a minimum value for the SSP position.

Dominant edges on FDCANx_RX that would result in an earlier SSP position are ignored for transmitter delay measurement. The measurement is stopped when the SSP position is at least FDCAN_TDCR.TDCF and FDCAN_RX is low.

Restricted operation mode

In restricted operation mode the node is able to receive data and remote frames and to give acknowledge to valid frames, but it does not send data frames, remote frames, active error frames, or overload frames. In case of an error condition or overload condition, it does not send dominant bits, but waits for the occurrence of bus idle condition to resynchronize itself to the CAN communication. The error counters (FDCAN_ECR.REC, FDCAN_ECR.TEC) are frozen while error logging (FDCAN_ECR.CEL) is active. The software can set the FDCAN into restricted operation mode by setting bit FDCAN_CCCR.ASM. The bit can be set only by software when both FDCAN_CCCR.CCE and FDCAN_CCCR.INIT are set to 1. The bit can be cleared by software at any time.

Restricted operation mode is automatically entered when the Tx handler was not able to read data from the message RAM in time. To leave restricted operation mode, the software has to reset FDCAN_CCCR.ASM.

The restricted operation mode can be used in applications that adapt themselves to different CAN bitrates. In this case the application tests different bitrates and leaves the restricted operation mode after it has received a valid frame.

FDCAN_CCCR.ASM is also controlled by the clock calibration unit. When the calibration process is enabled, the restricted operation mode is entered and the FDCAN_CCR.ASM bit is set. Once the calibration is completed, FDCAN_CCR.ASM bit is cleared.

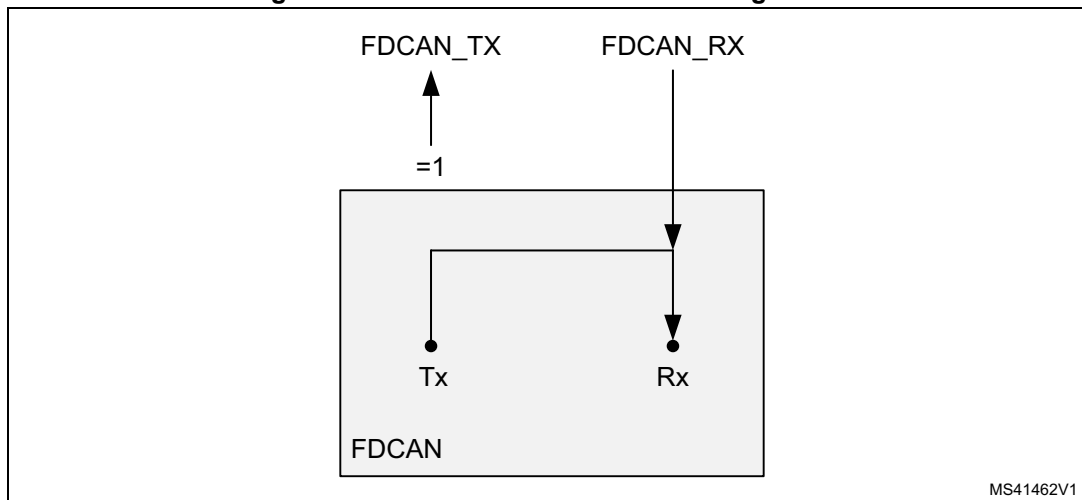
Note: The restricted operation mode must not be combined with the loop back mode (internal or external).

Bus monitoring mode

The FDCAN is set in bus monitoring mode by setting FDCAN_CCCR.MON bit or when error level S3 (FDCAN_TTOST.EL = 11) is entered. In bus monitoring mode (for more details refer to ISO11898-1, 10.12 bus monitoring), the FDCAN is able to receive valid data frames and valid remote frames, but cannot start a transmission. In this mode, it sends only recessive bits on the CAN bus, if the FDCAN is required to send a dominant bit (ACK bit, overload flag, active error flag), the bit is rerouted internally so that the FDCAN monitors this dominant bit, although the CAN bus may remain in recessive state. In bus monitoring mode register FDCAN_TXBRP is held in reset state.

The bus monitoring mode can be used to analyze the traffic on a CAN bus without affecting it by the transmission of dominant bits. [Figure 778](#) shows the connection of FDCAN_TX and FDCAN_RX signals to the FDCAN in bus monitoring mode.

Figure 778. Pin control in bus monitoring mode



MS41462V1

Disabled automatic retransmission (DAR) mode

According to the CAN Specification (see ISO11898-1, 6.3.3 recovery management), the FDCAN provides means for automatic retransmission of frames that have lost arbitration or have been disturbed by errors during transmission. By default, automatic retransmission is enabled.

To support time-triggered communication as described in ISO 11898-1: 2015, chapter 9.2, the automatic retransmission may be disabled via FDCAN_CCCR.DAR.

Frame transmission in disabled automatic retransmission (DAR) mode

In DAR mode all transmissions are automatically canceled after they started on the CAN bus. A Tx buffer Tx request pending bit FDCAN_TXBRP.TRPx is reset after successful transmission, when a transmission has not yet been started at the point of cancellation, has been aborted due to lost arbitration, or when an error occurred during frame transmission.

- Successful transmission:
 - Corresponding Tx buffer transmission occurred bit FDCAN_TXBTO.TOx set
 - Corresponding Tx buffer cancellation finished bit FDCAN_TXBCF.CFx not set
- Successful transmission in spite of cancellation:
 - Corresponding Tx buffer transmission occurred bit FDCAN_TXBTO.TOx set
 - Corresponding Tx buffer cancellation finished bit FDCAN_TXBCF.CFx set
- Arbitration loss or frame transmission disturbed:
 - Corresponding Tx buffer transmission occurred bit FDCAN_TXBTO.TOx not set
 - Corresponding Tx buffer cancellation finished bit FDCAN_TXBCF.CFx set

In case of a successful frame transmission, and if storage of Tx events is enabled, a Tx event FIFO element is written with event type ET = 10 (transmission in spite of cancellation).

Power down (Sleep mode)

The FDCAN can be set into power down mode controlled by clock stop request input via register FDCAN_CCCR.CSR. As long as the clock stop request is active, bit FDCAN_CCCR.CSR is read as 1.

When all pending transmission requests have completed, the FDCAN waits until bus idle state is detected. Then the FDCAN sets FDCAN_CCCR.INIT to 1 to prevent any further CAN transfers. Now, the FDCAN acknowledges that it is ready for power down by setting FDCAN_CCCR.CSA to 1. In this state, before the clocks are switched off, further register accesses can be made. A write access to FDCAN_CCCR.INIT has no effect. Now the module clock inputs may be switched off.

To leave power down mode, the application has to turn on the module clocks before resetting CC control register flag FDCAN_CCCR.CSR. The FDCAN acknowledges this by resetting FDCAN_CCCR.CSA. Afterwards, the application can restart CAN communication by resetting bit FDCAN_CCCR.INIT.

Test modes

To enable write access to FDCAN test register (see [Section 59.5.4](#)), bit FDCAN_CCCR.TEST must be set to 1, thus enabling the configuration of test modes and functions.

Four output functions are available for the CAN transmit pin FDCAN_TX by programming FDCAN_TEST.TX. Additionally to its default function (the serial data output) it can drive the CAN Sample Point signal to monitor the FDCAN bit timing and it can drive constant dominant or recessive values. The actual value at pin FDCAN_RX can be read from FDCAN_TEST.RX. Both functions can be used to check the CAN bus physical layer.

Due to the synchronization mechanism between CAN kernel clock and APB clock domain, there may be a delay of several APB clock periods between writing to FDCAN_TEST.TX until the new configuration is visible at FDCAN_TX output pin. This applies also when reading FDCAN_RX input pin via FDCAN_TEST.RX.

Note: Test modes should be used for production tests or self test only. The software control for FDCAN_TX pin interferes with all CAN protocol functions. It is not recommended to use test modes for application.

External loop back mode

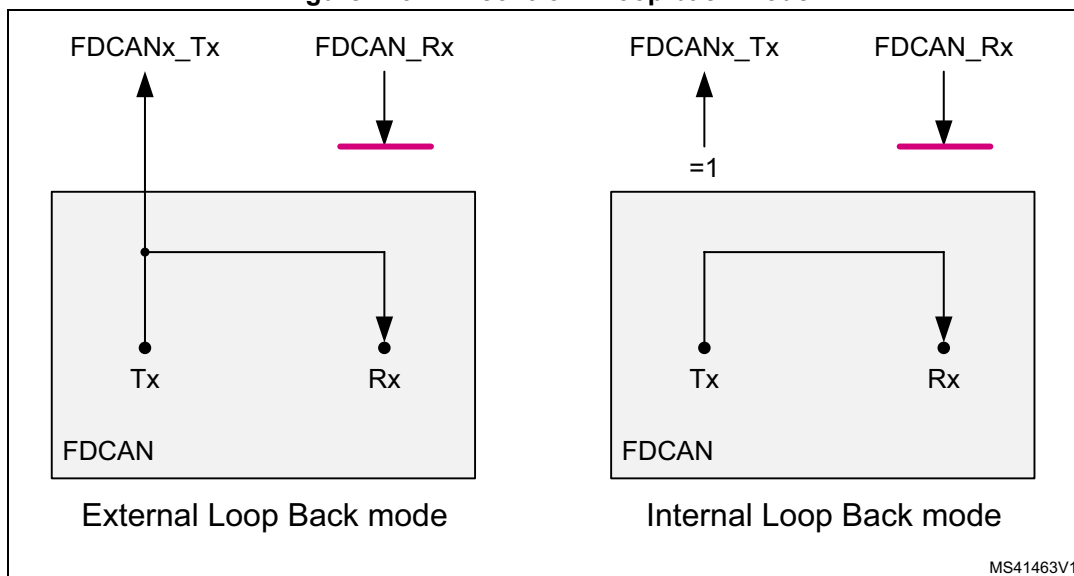
The FDCAN can be set in external loop back mode by programming FDCAN_TEST.LBCK to 1. In loop back mode, the FDCAN treats its own transmitted messages as received messages and stores them (if they pass acceptance filtering) into Rx FIFOs. [Figure 779](#) (left side) shows the connection of transmit and receive signals FDCAN_TX and FDCAN_RX to the FDCAN in external loop back mode.

This mode is provided for hardware self-test. To be independent from external stimulation, the FDCAN ignores acknowledge errors (recessive bit sampled in the acknowledge slot of a data/remote frame) in loop back mode. In this mode the FDCAN performs an internal feedback from its transmit output to its receive input. The actual value of the FDCAN_RX input pin is disregarded by the FDCAN. The transmitted messages can be monitored at the FDCAN_TX transmit pin.

Internal loop back mode

Internal loop back mode is entered by programming bits FDCAN_TEST.LBCK and FDCAN_CCCR.MON to 1. This mode can be used for a “Hot Selftest”, meaning the FDCAN can be tested without affecting a running CAN system connected to the FDCAN_TX and FDCAN_RX pins. In this mode, FDCAN_RX pin is disconnected from the FDCAN and FDCAN_TX pin is held recessive. [Figure 779](#) (right side) shows the connection of FDCAN_TX and FDCAN_RX pins to the FDCAN in case of internal loop back mode.

Figure 779. Pin control in loop back mode



Application watchdog (FDCAN1 only)

The application watchdog is served by reading register FDCAN_TTOST. When the application watchdog is not served in time, bit FDCAN_TTOST.AWE is set, all TTCAN communication is stopped, and the FDCAN1 is set into bus monitoring mode.

The TT application watchdog can be disabled by programming the application watchdog limit FDCAN_TTOCF.AWL to 0x00. The TT application watchdog must not be disabled in a TTCAN application program.

Timestamp generation

For timestamp generation the FDCAN supplies a 16-bit wraparound counter. A prescaler FDCAN_TSCC.TCP can be configured to clock the counter in multiples of CAN bit times (1 ... 16). The counter is readable via FDCAN_TSCV.TCV. A write access to register FDCAN_TSCV resets the counter to 0. When the timestamp counter wraps around interrupt flag FDCAN_IR.TSW is set.

On start of frame reception/transmission the counter value is captured and stored into the timestamp section of an Rx buffer/Rx FIFO (RXTS[15:0]) or Tx event FIFO (TXTS[15:0]) element.

By programming bit FDCAN_TSCC.TSS, a 16-bit timestamp can be used.

Timeout counter

To signal timeout conditions for Rx FIFO 0, Rx FIFO 1, and the Tx event FIFO the FDCAN supplies a 16-bit timeout counter. It operates as down-counter and uses the same prescaler controlled by FDCAN_TSCC.TCP as the timestamp counter. The timeout counter is configured via register FDCAN_TOCC. The actual counter value can be read from FDCAN_TOCV.TOC. The timeout counter can only be started while FDCAN_CCCR.INIT = 0. It is stopped when FDCAN_CCCR.INIT = 1, for example when the FDCAN enters Bus_Off state.

The operation mode is selected by FDCAN_TOCC.TOS. When operating in Continuous mode, the counter starts when FDCAN_CCCR.INIT is reset. A write to FDCAN_TOCV

presets the counter to the value configured by FDCAN_TOCC.TOP and continues down-counting.

When the timeout counter is controlled by one of the FIFOs, an empty FIFO presets the counter to the value configured by FDCAN_TOCC.TOP. Down-counting is started when the first FIFO element is stored. Writing to FDCAN_TOCV has no effect.

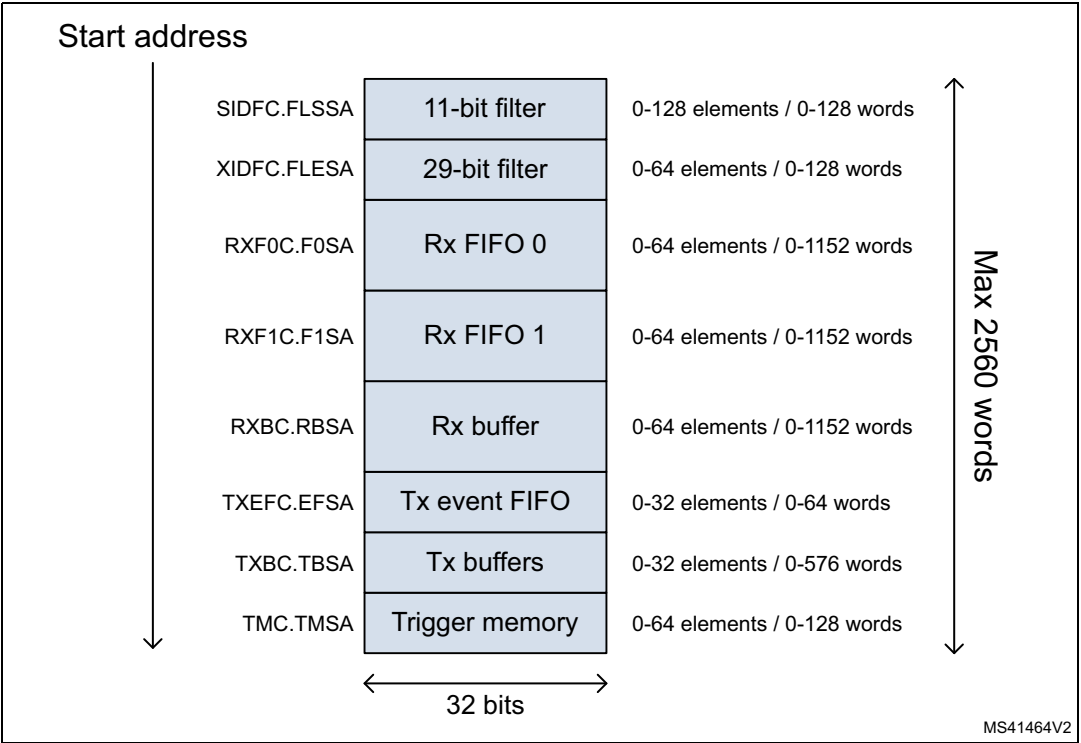
When the counter reaches 0, interrupt flag FDCAN_IR.TOO is set. In Continuous mode, the counter is immediately restarted at FDCAN_TOCC.TOP.

Note: The clock signal for the timeout counter is derived from the CAN core sample point signal. Therefore, the point in time where the timeout counter is decremented can vary due to the synchronization/re-synchronization mechanism of the CAN core. If the baudrate switch feature in FDCAN is used, the timeout counter is clocked differently in arbitration and data fields.

59.4.2 Message RAM

The message RAM has a width of 32 bits. The FDCAN module can be configured to allocate up to 2560 words in the message RAM. It is not necessary to configure each of the sections listed in [Figure 780](#), nor is there any restriction with respect to the sequence of the sections.

Figure 780. Message RAM configuration



When the FDCAN addresses the message RAM it addresses 32-bit words, not single bytes. The configured start addresses are 32-bit word addresses, that is. only bits 15 to 2 are evaluated, the two least significant bits are ignored.

Note: The FDCAN does not check for erroneous configuration of the message RAM. In particular, the configuration of the start addresses of the different sections and the number of elements of each section must be done carefully to avoid falsification or loss of data.

Rx handling

The Rx handler controls the acceptance filtering, the transfer of received messages to Rx buffers or to 1 of the two Rx FIFOs, as well as the Rx FIFO put and get Indices.

Acceptance filter

The FDCAN offers the possibility to configure two sets of acceptance filters, one for standard identifiers and one for extended identifiers. These filters can be assigned to Rx buffer, Rx FIFO 0 or Rx FIFO 1. For acceptance filtering each list of filters is executed from element #0 until the first matching element. Acceptance filtering stops at the first matching element. The following filter elements are not evaluated for this message.

The main features are:

- Each filter element can be configured as
 - range filter (from 0 to 128 elements for the 11-bit filter and from 0 to 64 for the 29-bit filter)
 - filter for one or two dedicated IDs
 - classic bit mask filter
- Each filter element is configurable for acceptance or rejection filtering
- Each filter element can be enabled/disabled individually
- Filters are checked sequentially, execution stops with the first matching filter element

Related configuration registers are:

- Global filter configuration (FDCAN_GFC)
- Standard ID filter configuration (FDCAN_SIDFC)
- Extended ID filter configuration (FDCAN_XIDFC)
- Extended ID AND Mask (FDCAN_XIDAM)

Depending on the configuration of the filter element (SFEC / EFEC) a match triggers one of the following actions:

- Store received frame in FIFO 0 or FIFO 1
- Store received frame in Rx buffer
- Store received frame in Rx buffer and generate pulse at filter event pin
- Reject received frame
- Set high priority message interrupt flag FDCAN_IR.HPM
- Set high priority message interrupt flag FDCAN_IR.HPM and store received frame in FIFO 0 or FIFO 1
- Set high priority message interrupt flag FDCAN_IR.HPM and store received frame in FIFO 0 or FIFO 1

Acceptance filtering is started after the complete identifier has been received. After acceptance filtering has completed, and if a matching Rx buffer or Rx FIFO has been found, the message handler starts writing the received message data in 32-bit portions to the

matching Rx buffer or Rx FIFO. If the CAN protocol controller has detected an error condition (for example CRC error), this message is discarded with the following impact:

- **Rx buffer**
New data flag of matching Rx buffer is not set, but Rx buffer (partly) overwritten with received data. For error type see FDCAN_PSR.LEC and FDCAN_PSR.DLEC.
- **Rx FIFO**
Put index of matching Rx FIFO is not updated, but related Rx FIFO element (partly) overwritten with received data. For error type see FDCAN_PSR.LEC and FDCAN_PSR.DLEC. In case the matching Rx FIFO is operated in overwrite mode, the boundary conditions described in [Rx FIFO overwrite mode](#) have to be considered.

Note: When an accepted message is written to one of the two Rx FIFOs, or into an Rx buffer, the unmodified received identifier is stored independently of the filter(s) used. The result of the acceptance filter process depends strongly upon the sequence of configured filter elements.

Range filter

The filter matches for all received frames with message IDs in the range defined by SF1ID / SF2ID and EF1ID / EF2ID.

There are two possibilities when range filtering is used together with extended frames:

- EFT = 00: The message ID of received frames is AND-ed with the extended ID AND Mask (FDCAN_XIDAM) before the range filter is applied
- EFT = 11: The extended ID AND Mask (FDCAN_XIDAM) is not used for range filtering

Filter for dedicated IDs

A filter element can be configured to filter for one or two specific message IDs. To filter for one specific message ID, the filter element must be configured with SF1ID = SF2ID and EF1ID = EF2ID.

Classic bit mask filter

Classic bit mask filtering is intended to filter groups of message IDs by masking single bits of a received message ID. With classic bit mask filtering SF1ID / EF1ID is used as message ID filter, while SF2ID / EF2ID is used as filter mask.

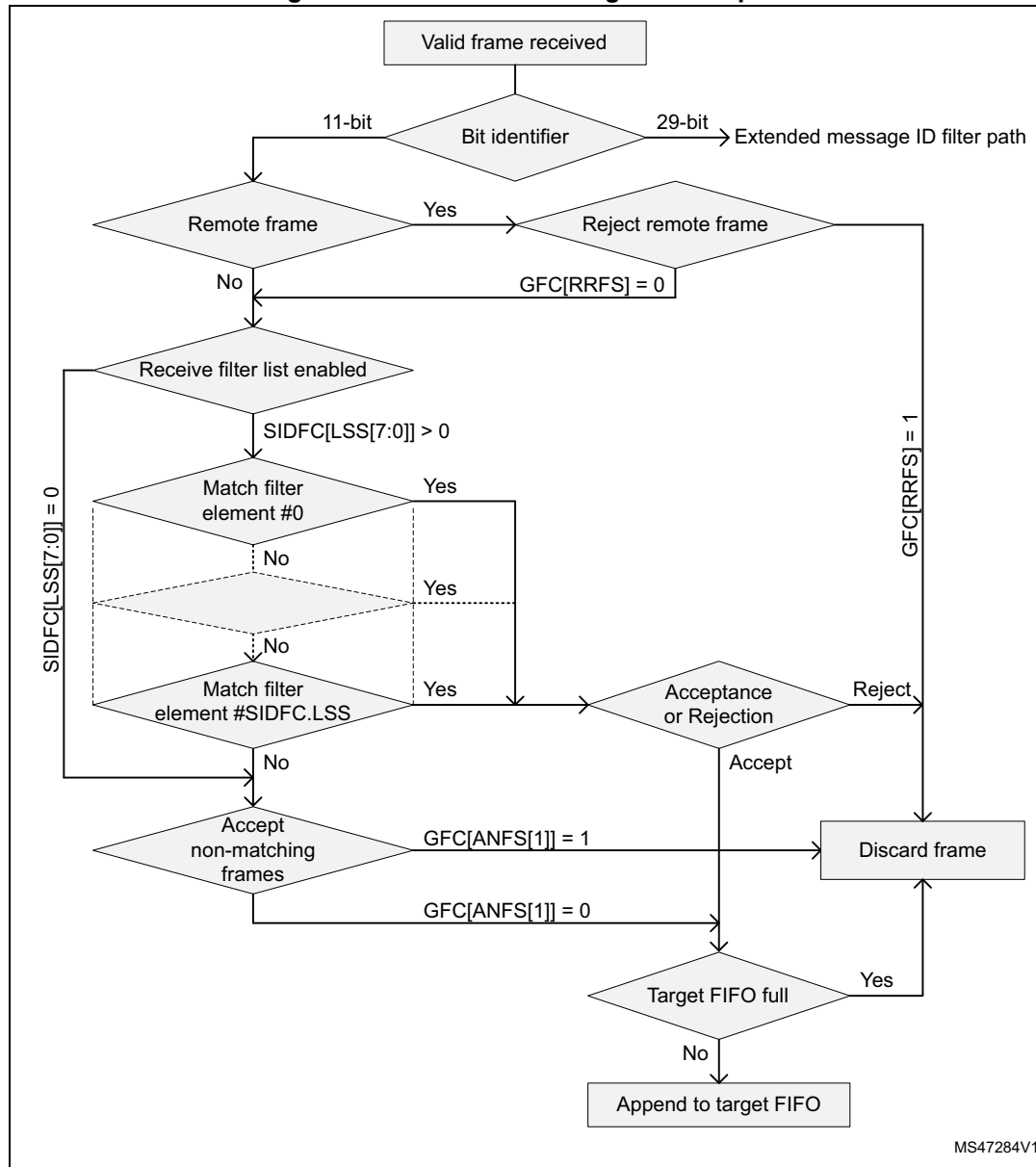
A 0 bit at the filter mask masks out the corresponding bit position of the configured ID filter, for example, the value of the received message ID at that bit position is not relevant for acceptance filtering. Only those bits of the received message ID where the corresponding mask bits are one are relevant for acceptance filtering.

In case all mask bits are one, a match occurs only when the received message ID and the message ID filter are identical. If all mask bits are 0, all message IDs match.

Standard message ID filtering

Figure 781 shows the flow for standard message ID (11-bit Identifier) filtering. The standard message ID filter element is described in Section 59.4.21.

Figure 781. Standard message ID filter path

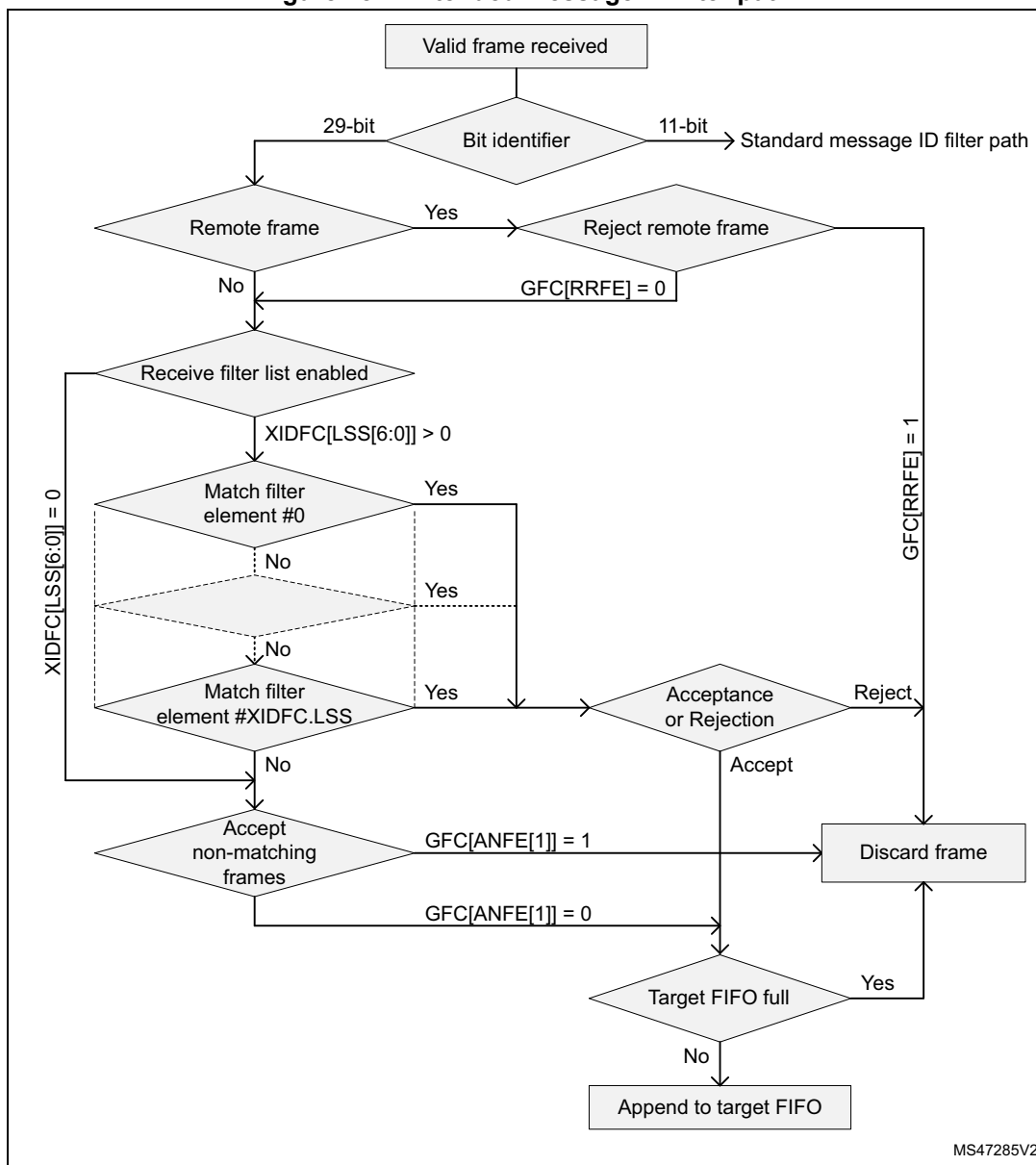


Controlled by the global filter configuration (FDCAN_GFC) and the standard ID filter configuration (FDCAN_SIDFC) message ID, remote transmission request bit (RTR), and the Identifier Extension bit (IDE) of received frames are compared against the list of configured filter elements.

Extended message ID filtering

Figure 782 shows the flow for extended message ID (29-bit Identifier) filtering. The extended message ID filter element is described in Section 59.4.22.

Figure 782. Extended message ID filter path



Controlled by the global filter configuration FDCAN_GFC and the extended ID filter configuration FDCAN_XIDFC message ID, remote transmission request bit (RTR), and the Identifier Extension bit (IDE) of received frames are compared against the list of configured filter elements.

The extended ID AND Mask (FDCAN_XIDAM) is AND-ed with the received identifier before the filter list is executed.

Rx FIFOs

Rx FIFO 0 and Rx FIFO 1 can be configured to hold up to 64 elements each. Configuration of the two Rx FIFOs is done via registers FDCAN_RXF0C and FDCAN_RXF1C.

Received messages that passed acceptance filtering are transferred to the Rx FIFO as configured by the matching filter element. For a description of the filter mechanisms available for Rx FIFO 0 and Rx FIFO 1, see [Acceptance filter](#). The Rx buffer and FIFO element are described in [Section 59.4.18](#).

When an Rx FIFO full condition is signaled by FDCAN_IR.RFnF, no further messages are written to the corresponding Rx FIFO until at least one message has been read out, and the Rx FIFO get index has been incremented. In case a message is received while the corresponding Rx FIFO is full, this message is discarded and interrupt flag FDCAN_IR.RFnL is set.

To avoid an Rx FIFO overflow, the Rx FIFO watermark can be used. When the Rx FIFO fill level reaches the Rx FIFO watermark configured by FDCAN_RXFnC.FnWM, interrupt flag FDCAN_IR.RFnW is set.

When reading from an Rx FIFO, Rx FIFO get index RXFnS[FnGI] + FIFO element size must be added to the corresponding Rx FIFO start address RXFnC[FnSA].

Rx FIFO blocking mode

The Rx FIFO blocking mode is configured by RXFnC.FnOM = 0. This is the default operation mode for the Rx FIFOs.

When an Rx FIFO full condition is reached (RXFnS.FnPI = RXFnS.FnGI), no further messages are written to the corresponding Rx FIFO until at least one message has been read out, and the Rx FIFO get index has been incremented. An Rx FIFO full condition is signaled by RXFnS.FnF = 1. In addition, interrupt flag FDCAN_IR.RFnF is set.

In case a message is received while the corresponding Rx FIFO is full, this message is discarded and the message lost condition is signaled by RXFnS.RFnL = 1. In addition, interrupt flag FDCAN_IR.RFnL is set.

Rx FIFO overwrite mode

The Rx FIFO overwrite mode is configured by RXFnC.FnOM = 1.

When an Rx FIFO full condition (RXFnS.FnPI = RXFnS.FnGI) is signaled by RXFnS.FnF = 1, the next message accepted for the FIFO overwrites the oldest FIFO message. Put and get index are both incremented by one.

When an Rx FIFO is operated in overwrite mode and an Rx FIFO full condition is signaled, reading of the Rx FIFO elements should start at least at get index + 1. The reason for that is that it can happen that a received message is written to the message RAM (put index) while the CPU is reading from the message RAM (get index). In this case inconsistent data may be read from the respective Rx FIFO element. Adding an offset to the get index when reading from the Rx FIFO avoids this problem. The offset depends on how fast the CPU accesses the Rx FIFO. [Figure 784](#) shows an offset of two with respect to the get index when reading the Rx FIFO. In this case the two messages stored in elements 1 and 2 are lost.

After reading from the Rx FIFO, the number of the last element read must be written to the Rx FIFO acknowledge index RXFnA.FnA. This increments the get index to that element number. In case the put index has not been incremented to this Rx FIFO element, the Rx FIFO full condition is reset (RXFnS.FnF = 0).

Dedicated Rx buffers

The FDCAN supports up to 64 dedicated Rx buffers. The start address of the dedicated Rx buffer section is configured via FDCAN_RXBC.RBSA.

For each Rx buffer a standard or extended message ID filter element with SFEC / EFEC=111 and SFID2 / EFID2[10:9] = 00 must be configured (see [Section 59.4.21](#) and [Section 59.4.22](#)).

After a received message has been accepted by a filter element, the message is stored into the Rx buffer in the message RAM referenced by the filter element. The format is the same as for an Rx FIFO element. In addition, the flag FDCAN_IR.DRX (message stored in dedicated Rx buffer) in the interrupt register is set.

Table 503. Example of filter configuration for Rx buffers

Filter element	SFID1[10:0] EFID1[28:0]	SFID2[10:9] EFID2[10:9]	SFID2[5:0] EFID2[5:0]
0	ID message 1	00	00 0000
1	ID message 2	00	00 0001
2	ID message 3	00	00 0010

After the last word of a matching received message has been written to the message RAM, the respective New data flag in register NDAT1,2 is set. As long as the New data flag is set, the respective Rx buffer is locked against updates from received matching frames. The New data flags have to be reset by the user by writing a 1 to the respective bit position.

While an Rx buffer New data flag is set, a message ID filter element referencing this specific Rx buffer is not matched, causing the acceptance filtering to continue. The following message ID filter elements may cause the received message to be stored into another Rx buffer, or into an Rx FIFO, or the message may be rejected, depending on filter configuration.

Rx buffer handling

- Reset interrupt flag FDCAN_IR.DRX
- Read New data registers
- Read messages from message RAM
- Reset New data flags of processed messages

Filtering for Debug messages

Filtering for debug messages is done by configuring one standard/extended message ID filter element for each of the three debug messages. To enable a filter element to filter for debug messages SFEC/EFEC must be programmed to 111. In this case fields SFID1 / SFID2 and EFID1 / EFID2 have a different meaning. While SFID2 / EFID2[10:9] controls the debug message handling state machine, SFID2 / EFID2[5:0] controls the location for storage of a received debug message.

When a debug message is stored, neither the respective New data flag nor FDCAN_IR.DRX are set. The reception of debug messages can be monitored via FDCAN_RXF1S.DMS.

Table 504. Example of filter configuration for Debug messages

Filter element	SFID1[10:0] EFID1[28:0]	SFID2[10:9] EFID2[10:9]	SFID2[5:0] EFID2[5:0]
0	ID debug message A	01	11 1101
1	ID debug message B	10	11 1110
2	ID debug message C	11	11 1111

Tx handling

The Tx handler handles transmission requests for the dedicated Tx buffers, the Tx FIFO, and the Tx queue. It controls the transfer of transmit messages to the CAN core, the put and get indices, and the Tx event FIFO. Up to 32 Tx buffers can be set up for message transmission (see [Dedicated Tx buffers](#)). Depending on the configuration of the element size (FDCAN_RXESC), between two and sixteen 32-bit words ($R_n = 3 \dots 17$) are used for storage of a CAN message data field.

Table 505. Possible configurations for frame transmission

FDCAN_CCCR		Tx buffer element		Frame transmission
BRSE	FDOE	FDF	BRS	
Ignored	0	Ignored	Ignored	Classic CAN
0	1	0	Ignored	Classic CAN
0	1	1	Ignored	FD without bitrate switching
1	1	0	Ignored	Classic CAN
1	1	1	0	FD without bitrate switching
1	1	1	1	FD with bitrate switching

Note: AUTOSAR requires at least three Tx queue buffers and support of transmit cancellation.

The Tx handler starts a Tx scan to check for the highest priority pending Tx request (Tx buffer with lowest message ID) when the Tx buffer request pending register FDCAN_TXBRP is updated, or when a transmission has been started.

Transmit pause

This feature is intended for use in CAN systems where the CAN message identifiers are (permanently) specified to specific values and cannot easily be changed. These message identifiers may have a higher CAN arbitration priority than other defined messages, while in a specific application their relative arbitration priority should be inverse. This may lead to a case where one ECU sends a burst of CAN messages that cause another ECU CAN messages to be delayed because that other messages have a lower CAN arbitration priority.

If, as an example, CAN ECU-1 has the feature enabled and is requested by its application software to transmit four messages, after the first successful message transmission, it waits for two CAN bit times of bus idle before it is allowed to start the next requested message. If there are other ECUs with pending messages, those messages are started in the idle time, they would not need to arbitrate with the next message of ECU-1. After having received a

message, ECU-1 is allowed to start its next transmission as soon as the received message releases the CAN bus.

The feature is controlled by TXP bit in FDCAN_CCCR register. If the bit is set, the FDCAN, each time it has successfully transmitted a message, pauses for two CAN bit times before starting the next transmission. This enables other CAN nodes in the network to transmit messages even if their messages have lower prior identifiers. Default is disabled (FDCAN_CCCR.TXP = 0).

This feature looses up burst transmissions coming from a single node and it protects against "babbling idiot" scenarios where the application program erroneously requests too many transmissions.

Dedicated Tx buffers

Dedicated Tx buffers are intended for message transmission under complete control of the CPU. Each dedicated Tx buffer is configured with a specific message ID. In case that multiple Tx buffers are configured with the same message ID, the Tx buffer with the lowest buffer number is transmitted first.

If the data section has been updated, a transmission is requested by an add request via FDCAN_TXBAR.ARn. The requested messages arbitrate internally with messages from an optional Tx FIFO or Tx queue and externally with messages on the CAN bus, and are sent out according to their message ID.

A dedicated Tx buffer allocates four 32-bit words in the message RAM. Therefore the start address of a dedicated Tx buffer in the message RAM is calculated by adding four times the transmit buffer index (0 ... 31) to the Tx buffer start address FDCAN_TXBC.TBSA.

Table 506. Tx buffer/FIFO - queue element size

FDCAN_TXESC.TBDS[2;0]	Data field (bytes)	Element size (RAM words)
000	8	4
001	12	5
010	16	6
011	20	7
100	24	8
101	32	10
110	48	14
111	64	18

Tx FIFO

Tx FIFO operation is configured by programming FDCAN_TXBC.TFQM to 0. Messages stored in the Tx FIFO are transmitted starting with the message referenced by the get index FDCAN_TXFQS.TFGI. After each transmission the get index is incremented cyclically until the Tx FIFO is empty. The Tx FIFO enables transmission of messages with the same message ID from different Tx buffers in the order these messages have been written to the Tx FIFO. The FDCAN calculates the Tx FIFO free level FDCAN_TXFQS.TFFL as difference between get and put index. It indicates the number of available (free) Tx FIFO elements.

New transmit messages have to be written to the Tx FIFO starting with the Tx buffer referenced by the put index FDCAN_TXFQS.TFQPI. An add request increments the put index to the next free Tx FIFO element. When the put index reaches the get index, Tx FIFO full (FDCAN_TXFQS.TFQF = 1) is signaled. In this case no further messages should be written to the Tx FIFO until the next message has been transmitted and the get index has been incremented.

When a single message is added to the Tx FIFO, the transmission is requested by writing a 1 to the FDCAN_TXBAR bit related to the Tx buffer referenced by the Tx FIFO put index.

When multiple (n) messages are added to the Tx FIFO, they are written to n consecutive Tx buffers starting with the put index. The transmissions are then requested via FDCAN_TXBAR. The put index is then cyclically incremented by n. The number of requested Tx buffers should not exceed the number of free Tx buffers as indicated by the Tx FIFO free level.

When a transmission request for the Tx buffer referenced by the get index is canceled, the get index is incremented to the next Tx buffer with pending transmission request and the Tx FIFO free level is recalculated. When transmission cancellation is applied to any other Tx buffer, the get index and the FIFO free level remain unchanged.

A Tx FIFO element allocates four 32-bit words in the message RAM. Therefore the start address of the next available (free) Tx FIFO buffer is calculated by adding four times the put index FDCAN_TXFQS.TFQPI (0 ... 31) to the Tx buffer start address FDCAN_TXBC.TBSA.

Tx queue

Tx queue operation is configured by programming FDCAN_TXBC.TFQM to 1. Messages stored in the Tx queue are transmitted starting with the message with the lowest message ID (highest priority). In case that multiple queue buffers are configured with the same message ID, the queue buffer with the lowest buffer number is transmitted first.

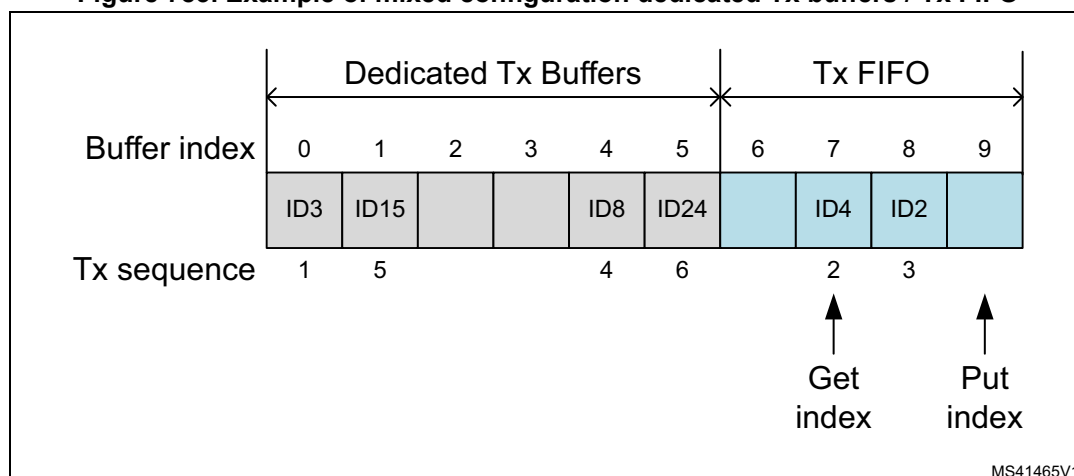
New messages have to be written to the Tx buffer referenced by the put index FDCAN_TXFQS.TFQPI. An add request cyclically increments the put index to the next free Tx buffer. In case that the Tx queue is full (FDCAN_TXFQS.TFQF = 1), the put index is not valid and no further message should be written to the Tx queue until at least one of the requested messages has been sent out or a pending transmission request has been canceled.

The application may use register FDCAN_TXBRP instead of the put index and may place messages to any Tx buffer without pending transmission request.

A Tx queue buffer allocates four 32-bit words in the message RAM. Therefore the start address of the next available (free) Tx queue buffer is calculated by adding four times the Tx queue put index FDCAN_TXFQS.TFQPI (0 ... 31) to the Tx buffer start address FDCAN_TXBC.TBSA.

Mixed dedicated Tx buffers / Tx FIFO

In this case the Tx buffers section in the message RAM is subdivided into a set of dedicated Tx buffers and a Tx FIFO. The number of dedicated Tx buffers is configured by FDCAN_TXBC.NDTB. The number of Tx buffers assigned to the Tx FIFO is configured by FDCAN_TXBC.TFQS. In case, FDCAN_TXBC.TFQS is programmed to 0, only dedicated Tx buffers are used.

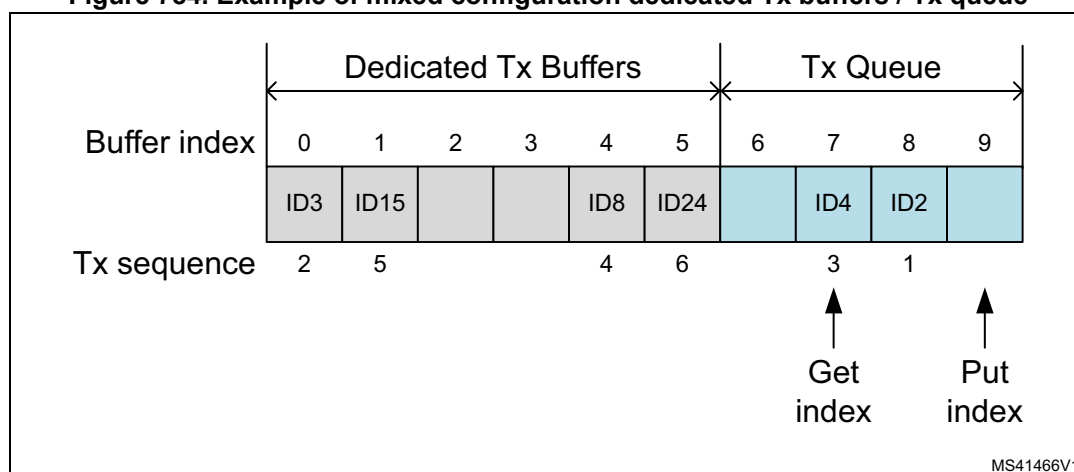
Figure 783. Example of mixed configuration dedicated Tx buffers / Tx FIFO

Tx prioritization:

- Scan dedicated Tx buffers and oldest pending Tx FIFO buffer (referenced by FDCAN_TXFS.TFGI)
- Buffer with lowest message ID gets highest priority and is transmitted next

Mixed dedicated Tx buffers / Tx queue

In this case the Tx buffers section in the message RAM is subdivided into a set of dedicated Tx buffers and a Tx queue. The number of dedicated Tx buffers is configured by FDCAN_TXBC.NDTB. The number of Tx queue buffers is configured by FDCAN_TXBC.TFQS. If FDCAN_TXBC.TFQS is programmed to 0, only dedicated Tx buffers are used.

Figure 784. Example of mixed configuration dedicated Tx buffers / Tx queue

Tx priority setting:

- Scan all Tx buffers with activated transmission request
- Tx buffer with lowest message ID gets highest priority and is transmitted next

Transmit cancellation

The FDCAN supports transmit cancellation. To cancel a requested transmission from a dedicated Tx buffer or a Tx queue buffer the user has to write a 1 to the corresponding bit position (= number of Tx buffer) of register FDCAN_TXBCR. Transmit cancellation is not intended for Tx FIFO operation.

Successful cancellation is signaled by setting the corresponding bit of register FDCAN_TXBCF to 1.

In case a transmit cancellation is requested while a transmission from a Tx buffer is already ongoing, the corresponding FDCAN_TXBRP bit remains set as long as the transmission is in progress. If the transmission was successful, the corresponding FDCAN_TXBTO and FDCAN_TXBCF bits are set. If the transmission was not successful, it is not repeated and only the corresponding FDCAN_TXBCF bit is set.

Note: If a pending transmission is canceled immediately before this transmission starts, a short time window follows where no transmission is started even if another message is pending in this node. This may enable another node to transmit a message that may have a priority lower than that of the second message in this node.

Tx event handling

To support Tx event handling the FDCAN has implemented a Tx event FIFO. After the FDCAN has transmitted a message on the CAN bus, message ID and timestamp are stored in a Tx event FIFO element. To link a Tx event to a Tx event FIFO element, the message marker from the transmitted Tx buffer is copied into the Tx event FIFO element.

The Tx event FIFO can be configured to a maximum of 32 elements. The Tx event FIFO element is described in [Tx FIFO](#). Depending on the configuration of the element size (FDCAN_TXESC), between two and sixteen 32-bit words ($T_n = 3 \dots 17$) are used for storage of a CAN message data field.

The purpose of the Tx event FIFO is to decouple handling transmit status information from transmit message handling i.e. a Tx buffer holds only the message to be transmitted, while the transmit status is stored separately in the Tx event FIFO. This has the advantage, especially when operating a dynamically managed transmit queue, that a Tx buffer can be used for a new message immediately after successful transmission. There is no need to save transmit status information from a Tx buffer before overwriting that Tx buffer.

When a Tx event FIFO full condition is signaled by FDCAN_IR.TEFF, no further elements are written to the Tx event FIFO until at least one element has been read out and the Tx event FIFO get index has been incremented. In case a Tx event occurs while the Tx event FIFO is full, this event is discarded and interrupt flag FDCAN_IR.TEFL is set.

To avoid a Tx event FIFO overflow, the Tx event FIFO watermark can be used. When the Tx event FIFO fill level reaches the Tx event FIFO watermark configured by FDCAN_TXEFC.EFWM, interrupt flag FDCAN_IR.TEFW is set.

When reading from the Tx event FIFO, two times the Tx event FIFO get index FDCAN_TXEFS.EFGI must be added to the Tx event FIFO start address FDCAN_TXEFC.EFSA.

59.4.3 FIFO acknowledge handling

The get indices of Rx FIFO 0, Rx FIFO 1, and the Tx event FIFO are controlled by writing to the corresponding FIFO acknowledge index, see [FDCAN Rx FIFO 0 acknowledge register](#)

([FDCAN_RXF0A](#)), [FDCAN Rx FIFO 1 acknowledge register \(FDCAN_RXF1A\)](#), and [FDCAN Tx event FIFO configuration register \(FDCAN_TXEFC\)](#). Writing to the FIFO acknowledge index sets the FIFO get index to the FIFO acknowledge index plus one and thereby updates the FIFO fill level. There are two use cases:

- When only a single element has been read from the FIFO (the one being pointed to by the get index), this get index value is written to the FIFO acknowledge index.
- When a sequence of elements has been read from the FIFO, it is sufficient to write the FIFO acknowledge index only once at the end of that read sequence (value: index of the last element read), to update the FIFO get index.

Due to the fact that the CPU has free access to the FDCAN message RAM, special care must be taken when reading FIFO elements in an arbitrary order (get index not considered). This might be useful when reading a high priority message from one of the two Rx FIFOs. In this case the FIFO acknowledge index should not be written because this would set the get index to a wrong position and also alters the FIFO fill level. In this case some of the older FIFO elements would be lost.

Note: The application has to ensure that a valid value is written to the FIFO acknowledge index. The FDCAN does not check for erroneous values.

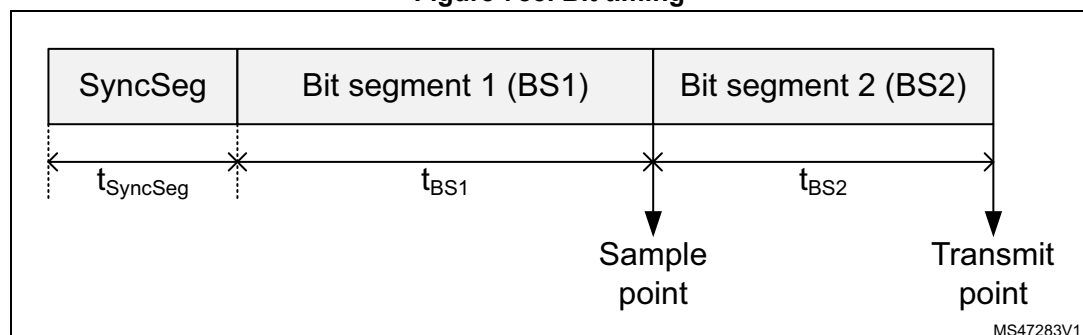
59.4.4 Bit timing

The bit timing logic monitors the serial bus-line and performs sampling and adjustment of the sample point by synchronizing on the start-bit edge and resynchronizing on the following edges.

As shown in [Figure 785](#), its operation may be explained simply by splitting the bit time in three segments, as follows:

- Synchronization segment (SYNC_SEG): a bit change is expected to occur within this time segment, that has a fixed length of one time quantum ($1 \times tq$).
- Bit segment 1 (BS1): defines the location of the sample point. It includes the PROP_SEG and PHASE_SEG1 of the CAN standard, its duration is programmable between 1 and 16 time quanta, but may be automatically lengthened to compensate for positive phase drifts due to differences in the frequency of the various nodes of the network.
- Bit segment 2 (BS2): defines the location of the transmit point. It represents the PHASE_SEG2 of the CAN standard, its duration is programmable between one and eight time quanta, but may also be automatically shortened to compensate for negative phase drifts.

Figure 785. Bit timing



The baudrate is the inverse of bit time (baudrate = 1 / bit time), which, in turn, is the sum of three components. [Figure 785](#) indicates that bit time = $t_{\text{SyncSeg}} + t_{\text{BS1}} + t_{\text{BS2}}$, where:

- for the nominal bit time
 - $t_q = (\text{FDCAN_NBTP.NBRP}[8:0] + 1) * t_{\text{fdcan_tq_ck}}$
 - $t_{\text{SyncSeg}} = 1 t_q$
 - $t_{\text{BS1}} = t_q * (\text{FDCAN_NBTP.NTSEG1}[7:0] + 1)$
 - $t_{\text{BS2}} = t_q * (\text{FDCAN_NBTP.NTSEG2}[6:0] + 1)$
- for the data bit time
 - $t_q = (\text{FDCAN_DBTP.DBRP}[4:0] + 1) * t_{\text{fdcan_tq_ck}}$
 - $t_{\text{SyncSeg}} = 1 t_q$
 - $t_{\text{BS1}} = t_q * (\text{FDCAN_DBTP.DTSEG1}[4:0] + 1)$
 - $t_{\text{BS2}} = t_q * (\text{FDCAN_DBTP.DTSEG2}[3:0] + 1)$

The (re)synchronization jump width (SJW) defines an upper bound for the amount of lengthening or shortening of the bit segments. It is programmable between one and four time quanta.

A valid edge is defined as the first transition in a bit time from dominant to recessive bus level, provided the controller itself does not send a recessive bit.

If a valid edge is detected in BS1 instead of SYNC_SEG, BS1 is extended by up to SJW so that the sample point is delayed.

Conversely, if a valid edge is detected in BS2 instead of SYNC_SEG, BS2 is shortened by up to SJW so that the transmit point is moved earlier.

As a safeguard against programming errors, the configuration of the bit timing register is only possible while the device is in Standby mode. Registers FDCAN_DBTP and FDCAN_NBTP (dedicated, respectively, to data and nominal bit timing) are only accessible when FDCAN_CCCR.CCE and FDCAN_CCCR.INIT are set.

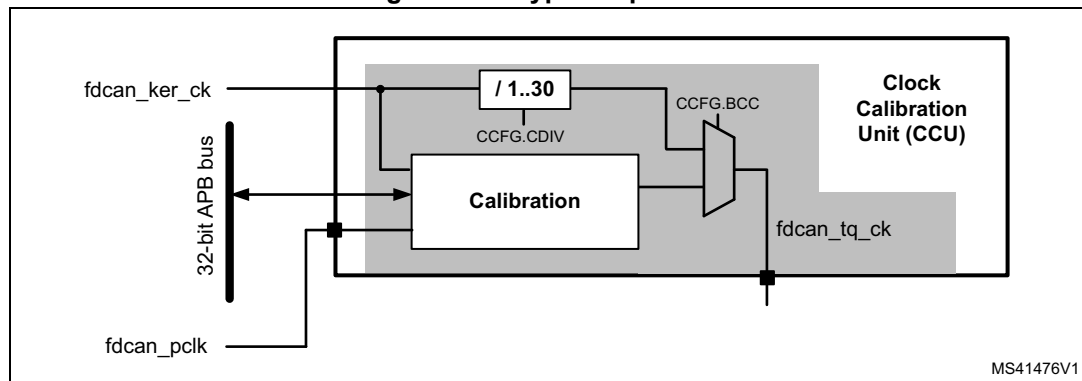
Note: For a detailed description of the CAN bit timing and resynchronization mechanism, refer to the ISO 11898-1 standard.

59.4.5 Clock calibration on CAN

After device reset the clock calibration unit (CCU) does not provide a valid clock signal to the FDCAN(s). The CCU must be initialized via FDCAN_CCFG register. The FDCAN_CCFG register can be written only when FDCAN1 has both FDCAN_CCCR.CCE and FDCAN_CCCR.INIT bits set. In consequence the CCU and the FDCAN1 initialization needs to be completed before any FDCAN module can operate.

Clock calibration is bypassed when FDCAN_CCFG.BCC = 1 (see [Figure 786](#)).

Figure 786. Bypass operation



Operating conditions

The clock calibration on CAN unit is designed to operate under the following conditions:

- a CAN kernel clock frequency `fdcan_ker_ck` up of at least 80 MHz
- FDCAN bitrates:
 - Nominal bitrate: up to 1 Mbit/s
 - Data bitrate: between nominal bitrate and 8 Mbit/s

The clock calibration on FDCAN unit generates a calibrated time quanta clock `fdcan_tq_ck` in the range from 0.5 to 25 MHz.

Note: *The FDCAN requires that the CAN time quanta clock is always below or equal to the APB clock ($fdcan_tq_ck < fdcan_pclk$). This must be considered when the clock calibration on CAN unit is bypassed (`FDCAN_CCFC.BCC = 1`).*

Calibration accuracy

The calibration accuracy in state `Precision_Calibrated` depends upon the factors listed below.

- Dynamic clock tolerance at the CAN kernel clock input `fdcan_ker_ck`
- Measurement error. For each bit sequence used for calibration measurement, there is a maximum error of one `fdcan_pclk` period. The number of bits used for measurement of the bit time is 32 or 64-bit, depending on configuration of `FDCAN_CCFC.CFL`.
- Tolerable error in calibration mechanism

The distance between two calibration messages must be chosen to fit the clock tolerance requirements of the FDCAN1 module.

Note: *Dynamic clock tolerance is the clock frequency variation between two calibration messages for example caused by change of temperature or operating voltage.*

Functional description

Calibration of the time quanta clock `fdcan_tq_ck` via CAN messages is performed by adapting a clock divider that generates the CAN protocol time quantum `tq` from the clock `fdcan_ker_ck`.

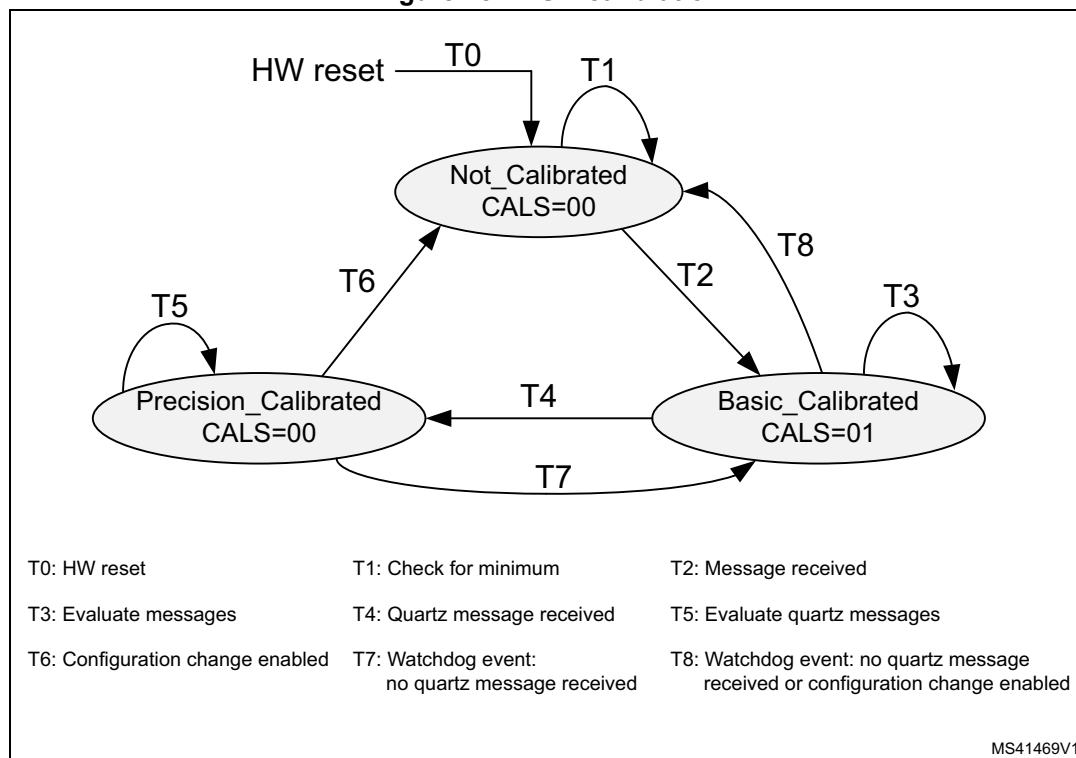
1. First step: basic calibration
The minimum distance between two consecutive falling edges from recessive to dominant is measured, this time to be assumed two CAN bit times, counted in PLL clock periods. The clock divider (`FDCAN_CCFC.CDIV`) is updated each time a new

measurement finds a smaller distance between edges. Basic calibration is achieved when the CAN protocol controller detects a valid CAN message.

2. Second step: Precision calibration

The calibration state machine measures the length of a longer bit sequence inside a CAN frame by counting the number of `fdcan_ker_ck` periods. The length of this bit sequence can be configured to 32 or 64 bits via `FDCAN_CCFG.CLF`. For a calibration field length of 32/64 bit a calibration message with at least 2/6-byte data field is required. Precision calibration is based on the new clock divider value calculated from the measurement of the longer bit sequence.

Figure 787. FSM calibration



A change in the calibration state sets interrupt flag `FDCAN_CCU_IR.CSC`, if enabled by the interrupt enable `FDCAN_CCU_IE.CSCE`. It remains set until cleared by writing 1 in `FDCAN_CCU_IR.CSC`.

Until precision calibration is achieved, FDCANs operate in a restricted mode (no frame transmission, no error or overload flag transmission, no error counting). In case calibration of the `fdcan_ker_clk` is done by software by evaluating the calibration status from register `FDCAN_CCU_CSTAT`, FDCANs have to be set to restricted operation mode (`FDCAN_CCCR.ASM = 1`) until the calibration on CAN unit is in state `Precision_Calibrated` (see [Application](#)).

Precision calibration may be performed only on valid CAN frames transmitted by a node with a stable, quartz-controlled clock. Calibration frames are detected by the FDCAN1 acceptance filtering. A filter element and a Rx buffer have to be configured in the FDCAN1 to identify and store calibration messages. After reception of a calibration message the Rx buffer new data flag must be reset to enable signaling of the next calibration message.

In case there is only one CAN transmitter with a quartz clock in the network, this node has to transmit its first message after startup with at least one 1010 binary sequence in the data field or in the identifier. This assures that the non-quartz nodes can enter state Basic_Calibrated and then acknowledge the quartz node messages.

Precision calibration must be repeated in predefined maximum intervals supervised by the calibration watchdog.

Note: *When the clock calibration on CAN unit transits from state Precision_Calibrated back to Basic_Calibrated, the calibration OK signal is deasserted, the FDCAN1 complete ongoing transmissions, and then enter restricted operation (no frame transmission, no error or overload flag transmission, no error counting).*

Configuration

The clock calibration on CAN unit is configured via register FDCAN_CCFG, i.e. when FDCAN1 has FDCAN_CCCR.CCE and FDCAN_CCCR.INIT bits set.

For basic calibration the minimum number of oscillator periods between two consecutive falling edges at pin FDCAN1_RX is measured. The number of clock periods depends on the clock frequency applied at input fdcan_ker_ck. In case the measured number of clock periods is below the minimum configured by FDCAN_CCFG.OCPM (as an example, because of a glitch on FDCAN1_RX) the value is discarded and measurement continues.

It is recommended to configure FDCAN_CCFG.OCPM slightly below two CAN bit times:

$$\text{FDCAN_CCFG.OCPM} < ((2 \times \text{CAN bit time}) / \text{fdcan_ker_ck period}) / 32$$

The length of the bit field used for precision calibration can be configured to 32 or 64 bits via FDCAN_CCFG.CFL. The number of bits used for precision calibration has an impact on calibration accuracy and the maximum distance between two calibration messages.

The number of time quanta per bit time configured by FDCAN_CCFG.TQBT is used together with the measured number of oscillator clock periods FDCAN_CCU_CSTAT.OCPC to define the number of oscillator clocks per bit time.

When the clock calibration is bypassed by configuring FDCAN_CCFG.BCC = 1, the internal clock divider must be configured via FDCAN_CCFG.CDIV to fulfill the condition $\text{fdcan_tq_ck} < \text{fdcan_pclk}$.

Note: *When clock calibration on CAN is active (FDCAN_CCFG.BCC = 0), the baudrate prescalers of FDCAN modules have to be configured to inactive.*

Status signaling

The status of the clock calibration on CAN unit can be monitored by reading register FDCAN_CCU_CSTAT. When in state Precision_Calibrated the oscillator clock period counter FDCAN_CCU_CSTAT.OCPC signals the number of oscillator clock periods in the calibration field while FDCAN_CCU_CSTAT.TQC signals the number of time quanta in the calibration field.

The calibration state is monitored by FDCAN_CCU_CSTAT.CALS. A change in the calibration state sets interrupt flag FDCAN_CCU_IR.CSC. If enabled by the interrupt enable FDCAN_CCU_IE.CSCE it remains set until cleared by writing 1 in FDCAN_CCU_IR.CSC.

A calibration watchdog event also sets interrupt flag FDCAN_CCU_IR.CWE. If enabled by FDCAN_CCU_IE.CWEE (set to high), it remains active until reset by FDCAN_CCU_IE.CWE.

59.4.6 Application

Clock calibration bypassed

The CCU internal clock divider is configured for division by one (FDCAN_CCFG.CDIV = 0x0000). In this operation mode the input clock `fdcan_ker_ck` is directly routed to the clock output `fdcan_tq_ck`. In this case `fdcan_tq_ck` is independent from the configuration and status of FDCAN1 and FDCAN2 connected to the CCU. CAN FD operation is possible with an `FDcan_ker_ck` above 80 MHz.

Software calibration

The clock calibration on CAN unit also supports software calibration of `fdcan_ker_ck` by trimming of an on-chip oscillator. For calculation of the trimming values the user has to read the CCU state from FDCAN_CCU_CSTAT. The clock from `fdcan_ker_ck` is routed to output `fdcan_tq_ck` (FDCAN_CCFG.BCC = 1).

The input clock `fdcan_ker_ck` must be at least 80 MHz. The clock divider of CCU must be configured via FDCAN_CCFG.CDIV to bring `fdcan_tq_ck` to a valid range. All other configuration parameters have to be set via FDCAN_CCFG. For correct operation of FDCAN1 and FDCAN2, the APB clock `fdcan_pclk` needs to be equal to or higher than the time quanta clock (`fdcan_tq_ck`). CAN FD operation is not possible.

For startup FDCAN modules have to be both configured for restricted operation (FDCAN_CCCR.ASM = 1) before FDCAN_CCCR.INIT is reset. The input clock `fdcan_ker_ck` must be adjusted until the clock calibration on CAN unit has reached state `Precision_Calibrated`. Now the software can reset FDCAN_CCCR.ASM and the CANFD1 and CANFD2 can start normal operation.

During operation the software has to check regularly whether the clock calibration on CAN unit is still in state `Precision_Calibrated`. In case the clock calibration on CAN unit has left state `Precision_Calibrated` due to drift of `fdcan_ker_ck`, FDCAN modules have to be set into restricted operation mode by programming FDCAN_CCCR.INIT, FDCAN_CCCR.CCE, and FDCAN_CCCR.ASM to 1. After `fdcan_ker_ck` has been adjusted successfully (clock calibration on CAN unit is in state `Precision_Calibrated`), FDCAN modules can resume normal operation.

Note: Trimming accuracy must be sufficient to meet the CAN clock tolerance requirements for the configured bitrate.

Clock calibration active

This operation mode is entered by resetting FDCAN_CCFG.BCC to 0. In this operation mode the `fdcan_ker_ck` is controlled by the CCU.

The generation of CCU output signal `fdcan_tq_ck` depends upon the state of the FDCAN1. Input clock `fdcan_ker_ck` must be above 80 MHz. Configuration of the CCU and FDCAN1 is required. CAN FD operation is not possible.

If FDCAN1 turns to `Bus_Off` or when its INIT bit is set by the user command (FDCAN_CCCR.INIT = 1), the CCU enters state `Not_Calibrated`. CANFD1 and CANFD2 enter restricted operation mode.

Note: This is the default operation mode after reset in case the reset value of FDCAN_CCFG.BCC is configured to 0.

59.4.7 TTCAN operations (FDCAN1 only)

Reference message

A reference message is a data frame characterized by a specific CAN identifier. It is received and accepted by all nodes except the time master (sender of the reference message).

For level 1 the data length must be at least one; for level 0, 2 the data length must be at least four; otherwise, the message is not accepted as reference message. The reference message may be extended by other data up to the sum of eight CAN data bytes. All bits of the identifier except the three LSBs characterize the message as a reference message. The last three bits specify the priorities of up to eight potential time masters. Reserved bits are transmitted as logical 0 and are ignored by the receivers. The reference message is configured via register FDCAN_TTRMC.

The time master transmits the reference message. If the reference message is disturbed by an error, it is retransmitted immediately. In case of a retransmission, the transmitted Master_Ref_Mark is updated. The reference message is sent periodically, but is allowed to stop the periodic transmission (Next_is_Gap bit) and to initiate transmission event-synchronized at the start of the next basic cycle by the current time master or by one of the other potential time masters.

The node transmitting the reference message is the current time master. The time master is allowed to transmit other messages. If the current time master fails, its function is replicated by the potential time master with the highest priority. Nodes that are neither time master nor potential time master are time-receiving nodes.

Level 1

Level 1 operation is configured via FDCAN_TTOCF.OM = 01 and FDCAN_TTOCF.GEN. external clock synchronization is not available in level 1. The information related to the reference message is stored in the first data byte as shown in [Table 507](#). Cycle_Count is optional.

Table 507. First byte of level 1 reference message

Bits	0	1	2	3	4	5	6	7
First byte	Next_is_Gap	Reserved	Cycle_Count[5:0]					

Level 2

Level 2 operation is configured via FDCAN_TTOCF.OM = 10 and FDCAN_TTOCF.GEN. The information related to the reference message is stored in the first four data bytes as shown in [Table 508](#). Cycle_Count and the lower four bits of FDCAN_NTU_Res are optional. The TTCAN does not evaluate NTU_Res[3:0] from received reference messages, it always transmits these bits as 0.

Table 508. First four bytes of level 2 reference message

Bits	0	1	2	3	4	5	6	7
First byte	Next_is_Gap	Reserved	Cycle_Count[5;0]					
Second byte	NTU_Res[6:4]			NTU_Res[3:0]				Disc_Bit
Third byte	Master_Ref_Mark[7:0]							
Fourth byte	Master_Ref_Mark[15:8]							

Level 0

Level 0 operation is configured via FDCAN_TTOCF.OM = 11. External event-synchronized time-triggered operation is not available in level 0. The information related to the reference message is stored in the first four data bytes as shown in the table below. In level 0 Next_is_Gap is always 0. Cycle_Count and the lower four bits of NTU_Res are optional. The TTCAN does not evaluate NTU_Res[3:0] from received reference messages, it always transmits these bits as 0.

Table 509. First four bytes of level 0 reference message

Bits	0	1	2	3	4	5	6	7
First byte	Next_is_Gap	Reserved	Cycle_Count[5:0]					
Second byte	NTU_Res[6:4]			NTU_Res[3:0]				Disc_Bit
Third byte	Master_Ref_Mark[7:0]							
Fourth byte	Master_Ref_Mark[15:8]							

59.4.8 TTCAN configuration**TTCAN timing**

The network time unit (NTU) is the unit in which all times are measured. The NTU is a constant of the whole network and is defined as a priority by the network system designer. In TTCAN level 1 the NTU is the nominal CAN bit time. In TTCAN level 0 and level 2 the NTU is a fraction of the physical second.

The NTU is the time base for the local time. The integer part of the local time (16-bit value) is incremented once each NTU. Cycle time and global time are both derived from local time. The fractional part (3-bit value) of local time, cycle time, and global time is not readable.

In TTCAN level 0 and level 2 the length of the NTU is defined by the time unit Ratio TUR. The TUR is in general a non-integer number, given by

$$TUR = FDCAN_TURNA.NAV / FDCAN_TURCF.DC.$$
The NTU length is given by

$$NTU = CAN \text{ clock period} \times TUR.$$

The TUR numerator configuration NC is an 18-bit number, FDCAN_TURCF[NCL[15:0]] can be programmed in the range 0x0000 to 0xFFFF. FDCAN_TURCF[NCH[17:16]] is hard wired to 0b01. When 0xnxxx is written to FDCAN_TURCF[NCL[15:0]], FDCAN_TURNA.NAV starts with the value $0x10000 + 0x0nnnn = 0x1nnnn$. The TUR denominator configuration FDCAN_TURCF.DC is a 14-bit number. FDCAN_TURCF.DC may be programmed in the range 0x0001 to 0x3FFF (0x0000 is an illegal value).

In level 1, NC must be equal to or higher than $4 \times \text{FDCAN_TURCF.DC}$. In level 0 and level 2 NC must be equal to or higher than $8 \times \text{FDCAN_TURCF.DC}$ to get the 3-bit resolution for the internal fractional part of the NTU.

A hardware reset presets FDCAN_TURCF.DC to 0x1000 and FDCAN_TURCF.NCL to 0x10000, resulting in an NTU consisting of sixteen CAN clock periods. Local time and application watchdog are not started before either the FDCAN_CCCR.INIT is reset, or FDCAN_TURCF.ELT is set. FDCAN_TURCF.ELT may not be set before the NTU is configured. Setting FDCAN_TURCF.ELT to 1 also locks the write access to register FDCAN_TURCF.

At startup FDCAN_TURNA.NAV is updated from NC ($= \text{FDCAN_TURCF.NCL} + 0x10000$) when FDCAN_TURCF.ELT is set. In TTCAN level 1 there is no drift compensation. FDCAN_TURNA.NAV does not change during operation, it is always equal to NC.

In TTCAN level 0 and level 2 there are two possibilities for FDCAN_TURNA.NAV to change. When operating as time slave or backup time master, and when FDCAN_TTOCF.ECC is set, FDCAN_TURNA.NAV is updated automatically to the value calculated from the monitored global time speed, as long as the TTCAN is in synchronization state In_Schedule or In_Gap. When it loses synchronization, it returns to NC. When operating as the actual time master, and when FDCAN_TTOCF.EECS is set, the user may update FDCAN_TURCF.NCL. When the user sets FDCAN_TTOCN.ECS, FDCAN_TURNA.NAV is updated from the new value of NC at the next reference message. The status flag FDCAN_TTOST.WECS as is set when FDCAN_TTOCN.ECS is set and is cleared when FDCAN_TURNA.NAV is updated. FDCAN_TURCF.NCL is write locked while FDCAN_TTOST.WECS is set.

In TTCAN level 0 and level 2 the clock calibration process adapts FDCAN_TURNA.NAV in the range of the synchronization deviation limit SDL of $NC \pm 2(\text{FDCAN_TTOCF.LDSDL} + 5)$. FDCAN_TURCF.NCL should be programmed to the largest applicable numerical value in order to achieve the best accuracy in the calculation of FDCAN_TURNA.NAV.

The synchronization deviation SD is the difference between NC and FDCAN_TURNA.NAV ($SD = |NC - \text{FDCAN_TURNA.NAV}|$). It is limited by the synchronization deviation limit SDL, which is configured by its dual logarithm FDCAN_TTOCF.LDSDL ($SDL = 2(\text{FDCAN_TTOCF.LDSDL} + 5)$) and should not exceed the clock tolerance given by the CAN bit timing configuration. SD is calculated at each new basic cycle. When the calculated TURNA[NAV deviates by more than SDL from NC, or if the Disc_Bit in the reference message is set, the drift compensation is suspended and FDCAN_TTIR.GTE is set and FDCAN_TTOSC.QCS is reset, or in case of the Disc_Bit = 1, FDCAN_TTIR.GTD is set.

Table 510. TUR configuration example

TUR	8	10	24	50	510	125000	32.5	100/12	529/17
NC	0x1FFF8	0x1FFFE	0x1FFF8	0x1FFFEA	0x1FFFE	0x1E848	0x1FFE0	0x19000	0x10880
FDCAN_TURCF.DC	0x3FFF	0x3333	0x1555	0x0A3D	0x0101	0x0001	0x0FC0	0x3000	0x0880

FDCAN_TTOCN.ECS schedules NC for activation by the next reference message. FDCAN_TTOCN.SGT schedules FDCAN_TTGTP.TP for activation by the next reference message. Setting FDCAN_TTOCN.ECS and FDCAN_TTOCN.SGT requires FDCAN_TTOCF.EECS to be set (external clock synchronization enabled) while the FDCAN is actual time master.

The TTCAN module provides an application watchdog to verify the function of the application program. The user has to serve this watchdog regularly, else all CAN bus activity is stopped. The application watchdog limit FDCAN_TTOCF.AWL specifies the number of NTUs between two times the watchdog must be served. The maximum number of NTUs is 256. The application watchdog is served by reading register FDCAN_TTOST. FDCAN_TTOST.AWE indicates whether the watchdog has been served in time. In case the application failed to serve the application watchdog, interrupt flag FDCAN_TTIR.AW is set. For software development, the application watchdog may be disabled by programming FDCAN_TTOCF.AWL to 0x00, see [Section 59.6.3](#).

Timing of interface signals

The timing events that cause a pulse at output FDCAN trigger time mark interrupt pulse `fdcan1_tmp` for more than one instance and `fdcan_tmp` if only one instance; FDCAN register time mark interrupt pulse `fdcan1_rtp` for more than one instance and `fdcan_rtp` if only one instance are generated in the CAN clock domain.

There is a clock domain crossing delay to be considered before the same event is visible in the APB clock domain (when FDCAN_TTIR.TTMI is set or FDCAN_TTIR.RTMI is set). As an example, the signals can be connected to the timing input(s) of another FDCAN node (`fdcan_swt/fdcan_evt`), in order to automatically synchronize two TTCAN networks.

Output FDCAN start of cycle `fdcan1_soc` for more than one instance and `cycle_fdcan1_soc` if only one instance gets active whenever a reference message is completed (either transmitted or received). The output is controlled in the APB clock domain.

59.4.9 Message scheduling

FDCAN_TTOCF.TM controls whether the TTCAN operates as potential time master or as a time slave. If it is a potential time master, the three LSBs of the reference message identifier FDCAN_TTRMC.RID define the master priority, 0 giving the highest and 7 giving the lowest. There cannot be two nodes in the network using the same master priority. FDCAN_TTRMC.RID is used for recognition of reference messages. FDCAN_TTRMC.RMPS is not relevant for time slaves.

The initial reference trigger offset FDCAN_TTOCF.IRTO is a 7-bit-value that defines (in NTUs) how long a backup time master waits before it starts the transmission of a reference message when a reference message is expected but the bus remains idle. The recommended value for FDCAN_TTOCF.IRTO is the master priority multiplied with a factor depending on the expected clock drift between the potential time masters in the network. The sequential order of the backup time masters, when one of them starts the reference message in case the current time master fails, should correspond to their master priority, even with maximum clock drift.

FDCAN_TTOCF.OM decides whether the node operates in TTCAN level 0, level 1, or level 2. In one network, all potential time masters have to operate on the same level. Time slaves may operate on level 1 in a level 2 network, but not vice versa. The configuration of the TTCAN operation mode via FDCAN_TTOCF.OM is the last step in the setup. With FDCAN_TTOCF.OM = 00 (event-driven CAN communication), the FDCAN operates

according to ISO 11898-1: 2015, without time triggers. With FDCAN_TTOCF.OM = 01 (level 1), the FDCAN operates according to ISO 11898-4, but without the possibility to synchronize the basic cycles to external events, the Next_is_Gap bit in the reference message is ignored. With FDCAN_TTOCF.OM = 10 (level 2), the TTCAN operates according to ISO 11898-4, including the event-synchronized start of a basic cycle. With FDCAN_TTOCF.OM = 11 (level 0), the FDCAN operates as event-driven CAN but maintains a calibrated global time base as in level 2.

FDCAN_TTOCF.EECS enables the external clock synchronization, allowing the application program of the current time master to update the TUR configuration during time-triggered operation, to adapt the clock speed and (in levels 0 and level 2 only) the global clock phase to an external reference.

FDCAN_TTMLM.ENTT in the TT matrix limits register specifies the number of expected Tx_Triggers in the system matrix. This is the sum of Tx_Triggers for exclusive, single arbitrating and merged arbitrating windows, excluding the Tx_Ref_Triggers. Note that this is usually not the number of Tx_Trigger memory elements; the number of basic cycles in the system matrix and the trigger repeat factors have to be taken into account.

An inaccurate configuration of FDCAN_TTMLM.ENTT results either in a TxCount Underflow (FDCAN_TTIR.TXU = 1 and FDCAN_TTOST.EL = 01, severity 1), or in a Tx count Overflow (FDCAN_TTIR.TXO = 1 and FDCAN_TTOST.EL = 10, severity 2).

Note: In case the first reference message seen by a node does not have Cycle_Count 0, this node may finish its first matrix cycle with its Tx count resulting in a Tx count underflow condition. As long as a node is in state Synchronizing, its Tx_Triggers do not lead to transmissions.

FDCAN_TTMLM.CCM specifies the number of the last basic cycle in the system matrix. The counting of basic cycles starts at 0. In a system matrix consisting of eight basic cycles FDCAN_TTMLM.CCM would be 7. FDCAN_TTMLM.CCM is ignored by time slaves, a receiver of a reference message considers the received cycle count as the valid cycle count for the actual basic cycle.

FDCAN_TTMLM.TXEW specifies the length of the Tx enable window in NTUs. The Tx enable window is that period of time at the beginning of a time window where a transmission may be started. If a transmission of a message cannot be started inside the Tx enable window because of for example, a slight overlap from the previous time window message, the transmission cannot be started in that time window at all. FDCAN_TTMLM.TXEW must be chosen with respect to the network synchronization quality and with respect to the relation between the length of the time windows and the length of the messages.

Trigger memory

The trigger memory is part of the external message RAM to which the TTCAN is connected to (see [Section 59.5.26](#)). It stores up to 64 trigger elements. A trigger memory element consists of time mark TM, cycle code CC, trigger type TYPE, filter type FTYPE, message number MNR, message status count MSC, time mark event internal TMIN, time mark event external TMEX (see [Section 59.4.23](#)).

The time mark defines at which cycle time a trigger becomes active. The triggers in the trigger memory have to be sorted by their time marks. The trigger element with the lowest time mark is written to the first trigger memory word. Message number and cycle code are ignored for triggers of type Tx_Ref_Trigger, Tx_Ref_Trigger_Gap, Watch_Trigger, Watch_Trigger_Gap, and End_of_List.

When the cycle time reaches the time mark of the actual trigger, the FSE switches to the next trigger and starts to read the following trigger from the trigger memory. In case of a

transmit trigger, the Tx handler starts to read the message from the message RAM as soon as the FSE switches to its trigger. The RAM access speed defines the minimum time step between a transmit trigger and its preceding trigger, the Tx handler must be able to prepare the transmission before the transmit trigger time mark is reached. The RAM access speed also limits the number of non-matching (with regard to their cycle code) triggers between two matching triggers, the next matching trigger must be read before its time mark is reached. If the reference message is n NTU long, a trigger with a time mark lower than n never becomes active and is treated as a configuration error.

Starting point of the cycle time is the sample point of the reference message start of frame bit. The next reference message is requested when cycle time reaches the Tx_Ref_Trigger time mark. The FDCAN reacts on the transmission request at the next sample point. A new Sync_Mark is captured at the start of frame bit, but the cycle time is incremented until the reference message is successfully transmitted (or received) and the Sync_Mark is taken as the new Ref_Mark. At that point in time, cycle time is restarted. As a consequence, cycle time can never (with the exception of initialization) be seen at a value lower than n , with n being the length of the reference message measured in NTU.

Length of a basic cycle: Tx_Ref_Trigger time mark + 1 NTU + 1 CAN bit time.

The trigger list is different for all nodes in the CAN FD network. Each node knows only the Tx_Triggers for its own transmit messages, the Rx_Triggers for those receive messages that are processed by this node, and the triggers concerning the reference messages.

Trigger types

Tx_Ref_Trigger (TYPE = 0000) and Tx_Ref_Trigger_Gap (TYPE = 0001) cause the transmission of a reference message by a time master. A configuration error (FDCAN_TTOST.EL = 11, severity 3) is detected when a time slave encounters a Tx_Ref_Trigger(_Gap) in its trigger memory. Tx_Ref_Trigger_Gap is only used in external event-synchronized time-triggered operation mode. In that mode, Tx_Ref_Trigger is ignored when the FDCAN synchronization state is In_Gap (FDCAN_TTOST.SYS = 10).

Tx_Trigger_Single (TYPE = 0010), Tx_Trigger_Continuous (TYPE = 0011), Tx_Trigger_Arbitration (TYPE = 0100), and Tx_Trigger_Merged (TYPE = 0101) cause the start of a transmission. They define the start of a time window.

Tx_Trigger_Single starts a single transmission in an exclusive time window when the message buffer transmission request pending bit is set. After successful transmission the transmission request pending bit is reset.

Tx_Trigger_Continuous starts a transmission in an exclusive time window when the message buffer transmission request pending bit is set. After successful transmission the transmission request pending bit remains set, and the message buffer is transmitted again in the next matching time window.

Tx_Trigger_Arbitration starts an arbitrating time window, Tx_Trigger_Merged a merged arbitrating time window. The last Tx_Trigger of a merged arbitrating time window must be of type Tx_Trigger_Arbitration. A configuration error (FDCAN_TTOST.EL = 11, severity 3) is detected when a trigger of type Tx_Trigger_Merged is followed by any other Tx_Trigger than one of type Tx_Trigger_Merged or Tx_Trigger_Arbitration. Several Tx_Triggers may be defined for the same Tx message buffer. Depending on their cycle code, they may be ignored in some basic cycles. The cycle code must be considered when the expected number of Tx_Triggers (FDCAN_TTMLM.ENTT) is calculated.

Watch_Trigger (TYPE = 0110) and Watch_Trigger_Gap (TYPE = 0111) check for missing reference messages. They are used by both time masters and time slaves. Watch_Trigger_Gap is only used in external event-synchronized time-triggered operation mode. In that mode, a Watch_Trigger is ignored when the FDCAN synchronization state is In_Gap (FDCAN_TTOST.SYS = 10).

Rx_Trigger (TYPE = 1000) is used to check for the reception of periodic messages in exclusive time windows. Rx_Triggers are not active until state In_Schedule or In_Gap is reached. The time mark of an Rx_Trigger must be placed after the end of that message transmission, independent of time window boundaries. Depending on their cycle code, Rx_Triggers may be ignored in some basic cycles. At the time mark of the Rx_Trigger, it is checked whether the last received message before this time mark and after start of cycle or previous Rx_Trigger had matched the acceptance filter element referenced by MNR. Accepted messages are stored in one of the two receive FIFOs, according to the acceptance filtering, independent of the Rx_Trigger. Acceptance filter elements referenced by Rx_Triggers should be placed at the beginning of the filter list to ensure that the filtering is finished before the Rx_Trigger time mark is reached.

Time_Base_Trigger (TYPE = 1001) are used to generate internal/external events depending on the configuration of TMIN and TMEX.

End_of_List (TYPE = 1010 ... 1111) is an illegal trigger type, a configuration error (FDCAN_TTOST.EL = 11, severity 3) is detected when an End_of_List trigger is encountered in the trigger memory before the Watch_Trigger and Watch_Trigger_Gap.

Restrictions for the node trigger list

There may not be two triggers that are active at the same cycle time and cycle count, but triggers that are active in different basic cycles (different cycle code) may share the same time mark.

Rx_Triggers and Time_Base_Triggers may not be placed inside the Tx enable windows of Tx_Trigger_Single/Continuous/Arbitration, but they may be placed after Tx_Trigger_Merged.

Triggers that are placed after the Watch_Trigger (or the Watch_Trigger_Gap when FDCAN_TTOST.SYS = 10) never become active. The watch triggers themselves do not become active when the reference messages are transmitted on time.

All unused trigger memory words (after the Watch_Trigger or after the Watch_Trigger_Gap when FDCAN_TTOST.SYS = 10) must be set to trigger type End_of_List.

A typical trigger list for a potential time master begins with a number of Tx_Triggers and Rx_Triggers followed by the Tx_Ref_Trigger and the Watch_Trigger. For networks with external event-synchronized time-triggered communication, this is followed by the Tx_Ref_Trigger_Gap and the Watch_Trigger_Gap. The trigger list for a time slave is the same but without the Tx_Ref_Trigger and the Tx_Ref_Trigger_Gap.

At the beginning of each basic cycle, that is at each reception or transmission of a reference message, the trigger list is processed starting with the first trigger memory element. The FSE looks for the first trigger with a cycle code that matches the current cycle count. The FSE waits until cycle time reaches the trigger time mark and activates the trigger. Afterwards the FSE looks for the next trigger in the list with a cycle code that matches the current cycle count.

Special consideration is needed for the time around Tx_Ref_Trigger and Tx_Ref_Trigger_Gap. In a time master competing for master ship, the effective time mark of

a Tx_Ref_Trigger may be decremented in order to be the first node to start a reference message. In backup time masters the effective time mark of a Tx_Ref_Trigger or Tx_Ref_Trigger_Gap is the sum of its configured time mark and the reference trigger offset FDCAN_TTOCF. IRT0. In case error level 2 is reached (FDCAN_TTOST.EL = 10), the effective time mark is the sum of its time mark and 0x127. No other trigger elements should be placed in this range otherwise it may happen that the time marks appear out of order and are flagged as a configuration error. Trigger elements which are coming after Tx_Ref_Trigger may never become active as long as the reference messages come in time.

There are interdependencies between the following parameters:

- APB clock frequency
- Speed and waiting time for trigger RAM accesses
- Length of the acceptance filter list
- Number of trigger elements
- Complexity of cycle code filtering in the trigger elements
- Offset between time marks of the trigger elements

Example for trigger handling

The example shows how the trigger list is derived from a node system matrix. Assumption is that node A is first time master and has knowledge of the section of the system matrix shown in [Table 511](#).

Table 511. System matrix, Node A

Cycle count	Time mark						
	1	2	3	4	5	6	7
0	Tx7	-	-	-	-	TxRef	Error
1	Rx3	-	Tx2, Tx4		-	TxRef	Error
2	-	-	-	-	-	TxRef	Error
3	Tx7	-	Rx5	-	-	TxRef	Error
4	Tx7	-	-	Rx6	-	TxRef	Error

The cycle count starts with 0 and runs until 0, 1, 3, 7, 15, 31, 63 (the corresponding number of basic cycles in the system matrix is, respectively, 1, 2, 4, 8, 16, 32, 64). The maximum cycle count is configured by FDCAN_TTMLM.CCM. The cycle code CC is composed of repeat factor (= value of most significant 1) and the number of the first basic cycle in the system matrix (= bit field after most significant 1).

As an example, with a cycle code of 0b0010011 (repeat factor = 16, first basic cycle = 3) and a maximum cycle count of FDCAN_TTMLM.CCM = 0x3F matches occur at cycle counts 3, 19, 35 and 51.

A trigger element consists of time mark TM, cycle code CC, trigger type TYPE, and message number MNR. For transmission MNR references the Tx buffer number (0 ... 31). For reception MNR references the number of the filter element (0 ... 127) that matched during acceptance filtering. Depending on the configuration of the filter type FTYPE, the 11-bit or 29-bit message ID filter list is referenced.

In addition, a trigger element can be configured for generation of time mark event internal TMIN, and time mark event external TMEX. The message status count MSC holds the counter value (0 ... 7) for scheduling errors for periodic messages in exclusive time windows at the point in time when the time mark of the trigger element became active.

Table 512. Trigger list, Node A

Trigger	Time mark TM[15:0]	Cycle code CC [6:0]	Trigger type TYPE [3:0]	Mess No. MNR [6:0]
0	Mark1	0b0000100	Tx_Trigger_Single	7
1	Mark 1	0b1000000	Rx_Trigger	3
2	Mark 1	0b1000011	Tx_Trigger_Single	7
3	Mark 3	0b1000001	Tx_Trigger_Merged	2
4	Mark 3	0b1000011	Rx_Trigger	5
5	Mark 4	0b1000001	Tx_Trigger_Arbitration	4
6	Mark 4	0b1000100	Rx_Trigger	6
7	Mark 6	N/A	Tx_Ref_Trigger	0 (Ref)
8	Mark 7	N/A	Watch_Trigger	N/A
9	N/A	N/A	End_of_List	N/A

Tx_Trigger_Single, Tx_Trigger_Continuous, Tx_Trigger_Merged, Tx_Trigger_Arbitration, Rx_Trigger, and Time_Base_Trigger are only valid for the specified cycle code. For all other trigger types the cycle code is ignored.

The FSE starts the basic cycle with scanning the trigger list starting from 0 until a trigger with time mark higher than cycle time and with its cycle code CC matching the actual cycle count is reached, or a trigger of type Tx_Ref_Trigger, Tx_Ref_Trigger_Gap, Watch_Trigger, or Watch_Trigger_Gap is encountered.

When the cycle time reached the time mark TM, the action defined by trigger type TYPE and message number MNR is started. There is an error in the configuration when End_of_List is reached.

At mark 6 the reference message (always TxRef) is transmitted. After transmission of the reference message the FSE returns to the beginning of the trigger list. When the watch trigger at mark 7 is reached, the node was not able to transmit the reference message; error treatment is started.

Detection of configuration errors

A configuration error is signaled via FDCAN_TTOST.EL = 11 (severity 3) when:

- The FSE comes to a trigger in the list with a cycle code that matches the current cycle count but with a time mark that is less than the cycle time.
- The previous active trigger was a Tx_Trigger_Merged and the FSE comes to a trigger in the list with a cycle code that matches the current cycle count but that is neither a

Tx_Trigger_Merged nor a Tx_Trigger_Arbitration nor a Time_Base_Trigger nor an Rx_Trigger.

- The FSE of a node with FDCAN_TTOCF.TM=0 (time slave) encounters a Tx_Ref_Trigger or a Tx_Ref_Trigger_Gap.
- Any time mark placed inside the Tx enable window (defined by FDCAN_TTMLM.TXEW) of a Tx_Trigger with a matching cycle code.
- A time mark is placed near the time mark of a Tx_Ref_Trigger and the reference trigger offset FDCAN_TTOST.RTO causes a reversal of their sequential order measured in cycle time.

TTCAN schedule initialization

The synchronization to the TTCAN message schedule starts when FDCAN_CCCR.INIT is reset. The TTCAN can operate strictly time-triggered (FDCAN_TTOCF.GEN = 0) or external event-synchronized time-triggered (FDCAN_TTOCF.GEN = 1). All nodes start with cycle time 0 at the beginning of their trigger list with FDCAN_TTOST.SYS = 00 (out of synchronization), no transmission is enabled with the exception of the reference message. Nodes in external event-synchronized time-triggered operation mode ignore Tx_Ref_Trigger and Watch_Trigger and use instead Tx_Ref_Trigger_Gap and Watch_Trigger_Gap until the first reference message decides whether a Gap is active.

Time slaves

After configuration, a time slave ignores its Watch_Trigger and Watch_Trigger_Gap if it does not receive any message before reaching the Watch_Triggers. When it reaches Init_Watch_Trigger, interrupt flag FDCAN_TTIR.IWT is set, the FSE is frozen, and the cycle time becomes invalid, but the node is still able to take part in CAN bus communication (to acknowledge or to send error flags). The first received reference message restarts the FSE and the cycle time.

Note: Init_Watch_Trigger is not part of the trigger list. It is implemented as an internal counter that counts up to 0xFFFF = maximum cycle time.

When a time slave has received any message but the reference message before reaching the Watch_Triggers, it assumes a fatal error (FDCAN_TTOST.EL = 11, severity 3), set interrupt flag FDCAN_TTIR.WT, switch off its CAN bus output, and enter the bus monitoring mode (FDCAN_CCCR.MON set to 1). In the bus monitoring mode it is still able to receive messages, but cannot send any dominant bits and therefore cannot give acknowledge.

Note: To leave the fatal error state, the user has to set FDCAN_CCCR.INIT = 1. After reset of FDCAN_CCCR.INIT, the node restarts TTCAN communication.

When no error is encountered during synchronization, the first reference message sets FDCAN_TTOST.SYS = 01 (Synchronizing), the second sets the FDCAN synchronization state (depending on its Next_is_Gap bit) to FDCAN_TTOST.SYS = 11 (In_Schedule) or FDCAN_TTOST.SYS = 10 (In_Gap), enabling all Tx_Triggers and Rx_Triggers.

Potential time masters

After configuration, a potential time master starts the transmission of a reference message when it reaches its Tx_Ref_Trigger (or its Tx_Ref_Trigger_Gap when in external event-synchronized time-triggered operation). It ignores its Watch_Trigger and Watch_Trigger_Gap when it does not receive any message or transmit the reference message successfully before reaching the Watch_Triggers (assumed reason: all other nodes still in reset or configuration, giving no acknowledge). When it reaches

Init_Watch_Trigger, the attempted transmission is aborted, interrupt flag FDCAN_TTIR.IWT is set, the FSE is frozen, and the cycle time becomes invalid, but the node is still able to take part in CAN bus communication (to give acknowledge or to send error flags). Resetting FDCAN_TTIR.IWT re-enables the transmission of reference messages until next time the Init_Watch_Trigger condition is met, or another CAN message is received. The FSE is not be restarted by the reception of a reference message.

When a potential time master reaches the Watch_Triggers after it has received any message but the reference message, it assumes a fatal error (FDCAN_TTOST.EL = 11, severity 3), sets interrupt flag FDCAN_TTIR.WT, switches off its CAN bus output, and enters the bus monitoring mode (FDCAN_CCCR.MON set to 1). In bus monitoring mode, it is still able to receive messages, but cannot send any dominant bits and therefore cannot give acknowledge.

When no error is detected during initialization, the first reference message sets FDCAN_TTOST.SYS = 01 (synchronizing), the second sets the FDCAN synchronization state (depending on its Next_is_Gap bit) to FDCAN_TTOST.SYS = 11 (In_Schedule) or FDCAN_TTOST.SYS = 10 (In_Gap), enabling all Tx_Triggers and Rx_Triggers.

A potential time master is current time master (FDCAN_TTOST.MS = 11) when it was the transmitter of the last reference message, else it is backup time master (FDCAN_TTOST.MS = 10).

When all potential time masters have finished configuration, the node with the highest time master priority in the network becomes the current time master.

59.4.10 TTCAN gap control

All functions related to gap control apply only when the FDCAN is operated in external event synchronized time-triggered mode (FDCAN_TTOCF.GEN = 1). In this operation mode the FDCAN message schedule may be interrupted by inserting gaps between the basic cycles of the system matrix. All nodes connected to the CAN network have to be configured for external event- synchronized time-triggered operation.

During a gap, all transmissions are stopped and the CAN bus remains idle. A gap is finished when the next reference message starts a new basic cycle. A gap starts at the end of a basic cycle that itself was started by a reference message with bit Next_is_Gap = 1 for example gaps are initiated by the current time master.

The current time master has two options to initiate a gap. A gap can be initiated under software control when the application program writes FDCAN_TTOCN.NIG = 1. The Next_is_Gap bit is transmitted as 1 with the next reference message. A gap can also be initiated under hardware control when the application program enables the event trigger input pin fdcan_evt by writing FDCAN_TTOCN.GCS = 1. When a reference message is started and FDCAN_TTOCN.GCS is set, a HIGH level at event trigger pin fdcan_evt sets Next_is_Gap = 1.

As soon as that reference message is completed, the FDCAN_TTOST.WFE bit announces the gap to the time master as well as to the time slaves. The current basic cycle continues until its last time window. The time after the last time window is the gap time.

For the actual time master and the potential time masters, FDCAN_TTOST.GSI is set when the last basic cycle has finished and the gap time starts. In nodes that are time slaves, bit FDCAN_TTOST.GSI remains at 0.

When a potential time master is in synchronization state In_Gap (FDCAN_TTOST.SYS = 10), it has four options to intentionally finish a gap:

1. Under software control by writing FDCAN_TTOCN.FGP = 1.
2. Under hardware control (FDCAN_TTOCN.GCS = 1) an edge from HIGH to LOW at the event-trigger input pin fdcan_evt sets FDCAN_TTOCN.FGP and restarts the schedule.
3. The third option is a time-triggered restart. When FDCAN_TTOCN.TMG = 1, the next register time mark interrupt (FDCAN_TTIR.RTMI = 1) sets FDCAN_TTOCN.FGP and starts the reference message.
4. Finally any potential time master finishes a gap when it reaches its Tx_Ref_Trigger_Gap, assuming that the event to synchronize on did not occur in time.

None of these options can cause a basic cycle to be interrupted with a reference message.

Setting of FDCAN_TTOCN.FGP after the gap time begins starts the transmission of a reference message immediately and thereby synchronizes the message schedule. When FDCAN_TTOCN.FGP is set before the gap time has started (while the basic cycle is still in progress), the next reference message is started at the end of the basic cycle, at the Tx_Ref_Trigger – there is no gap time in the message schedule.

In strictly time-triggered operation, bit Next_is_Gap = 1 in the reference message is ignored, as well as the event-trigger input pin fdcan_evt and the bits FDCAN_TTOCN.NIG, FDCAN_TTOCN.FGP, and FDCAN_TTOCN.TMG.

59.4.11 Stop watch

The stop watch function enables capturing of FDCAN internal time values (local time, cycle time, or global time) triggered by an external event.

To enable the stop watch function, the application program first has to define local time, cycle time, or global time as stop watch source via FDCAN_TTOCN.SWS. When FDCAN_TTOCN.SWS is different from 00 and TT interrupt register flag FDCAN_TTIR.SWE is 0, the actual value of the time selected by FDCAN_TTOCN.SWS is copied into FDCAN_TTCPT.SWV on the next rising / falling edge (as configured via FDCAN_TTOCN.SWP) on stop watch trigger pin fdcan_swt. This sets interrupt flag FDCAN_TTIR.SWE. After the application program has read FDCAN_TTCPT.SWV, it may enable the next stop watch event by resetting FDCAN_TTIR.SWE to 0.

59.4.12 Local time, cycle time, global time, and external clock synchronization

There are two possible levels in time-triggered CAN:

1. Level 1 only provides time-triggered operation using cycle time.
2. Level 2 additionally provides increased synchronization quality, global time and external clock synchronization. In both levels, all timing features are based on a local time base - the local time.

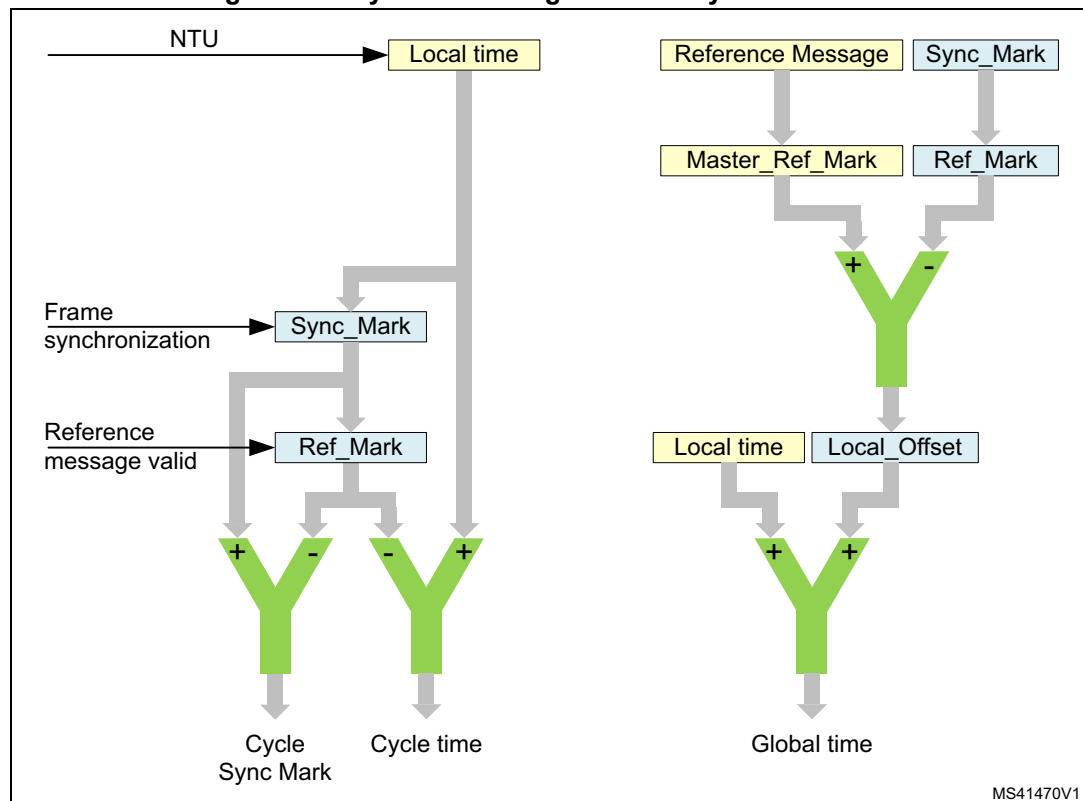
The local time is a 16-bit cyclic counter, it is incremented once each NTU. Internally the NTU is represented by a 3-bit counter which can be regarded as a fractional part (three binary digits) of the local time. Generally, the 3-bit NTU counter is incremented eight times each NTU. If the length of the NTU is shorter than eight CAN clock periods (as may be configured in level 1, or as a result of clock calibration in level 2), the length of the NTU fraction is adapted, and the NTU counter is incremented only four times each NTU.

Figure 788 describes the synchronization of the cycle time and global time, performed in the same manner by all FDCAN nodes, including the time master. Any message received or transmitted invokes a capture of the local time taken at the message is frame

synchronization event. This frame synchronization event occurs at the sample point of each start of frame (SoF) bit and causes the local time to be stored as Sync_Mark. Sync_Marks and Ref_Marks are captured including the 3-bit fractional part.

Whenever a valid reference message is transmitted or received, the internal Ref_Mark is updated from the Sync_Mark. The difference between Ref_Mark and Sync_Mark is the cycle sync mark (cycle sync mark = Sync_Mark - Ref_Mark) stored in register FDCAN_TTCSM. The most significant 16 bits of the difference between Ref_Mark and the actual value of the local time is the cycle time (cycle time = local time - Ref_Mark).

Figure 788. Cycle time and global time synchronization



MS41470V1

The cycle time that can be read from FDCAN_TTCTC.CT is the difference of the node local time and Ref_Mark, both synchronized into the APB clock domain and truncated to 16 bit.

The global time exists for TTCAN level 0 and level 2 only, in level 1 it is invalid. The node view of the global time is the local image of the global time in (local) NTUs. After configuration, a potential time master uses its own local time as global time. The time master establishes its own local time as global time by transmitting its own Ref_Marks as Master_Ref_Marks in the reference message (bytes 3 and 4). The global time that can be read from FDCAN_TTLGT.GT is the sum of the node local time and its local offset, both synchronized into the APB clock domain and truncated to 16 bit. The fractional part is used for clock synchronization only.

A node that receives a reference message calculates its local offset to the global time by comparing its local Ref_Mark with the received Master_Ref_Mark (see [Figure 789](#)). The node view of the global time is local time plus local offset. In a potential time master that has never received another time master reference message, Local_Offset is 0. When a node becomes the current time master after first having received other reference messages,

Local_Offset is frozen at its last value. In the time receiving nodes, Local_Offset may be subject to small adjustments, due to clock drift, when another node becomes time master, or when there is a global time discontinuity, signaled by Disc_Bit in the reference message. With the exception of global time discontinuity, the global time provided to the application program by register FDCAN_TTLGT is smoothed by a low-pass filtering to have a continuous monotonic value.

Figure 789. TTCAN level 0 and level 2 drift compensation

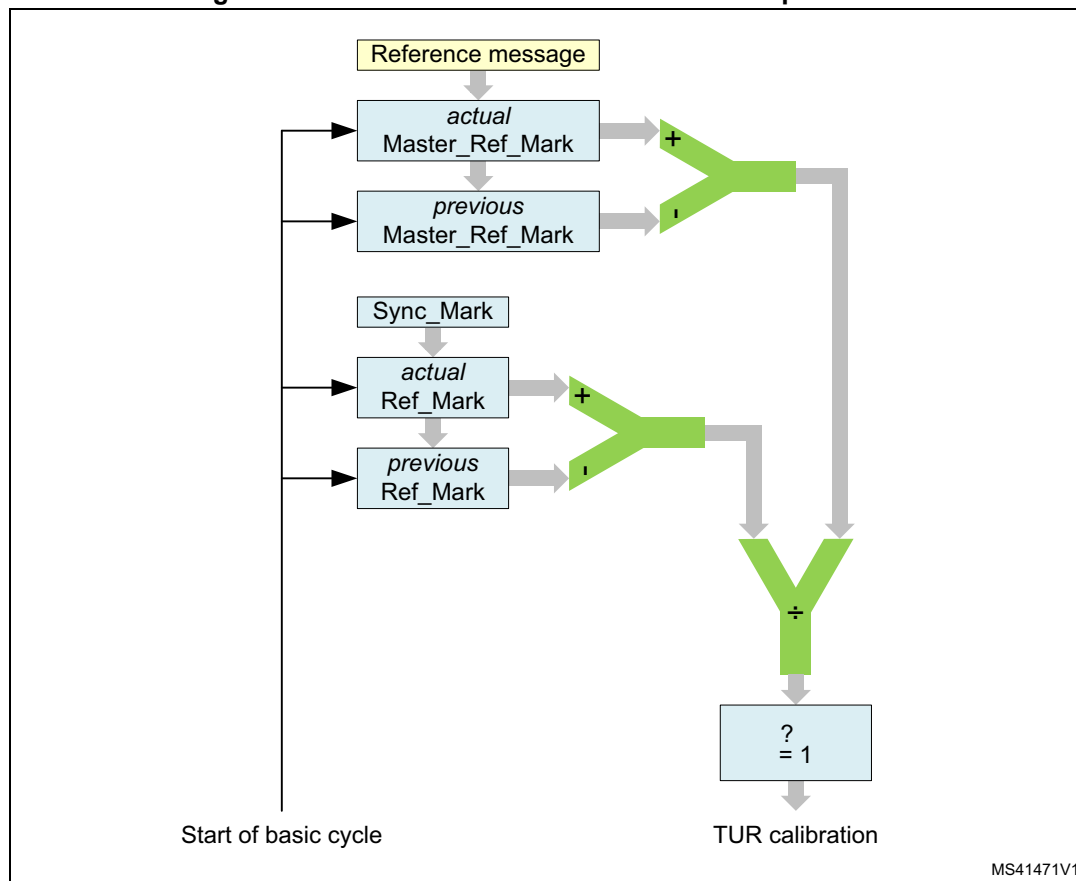


Figure 789 describes how in TTCAN levels 0 and 2 each time receiving node compensates the drift between its own local clock and the time master clock by comparing the length of a basic cycle in local time and in global time. If there is a difference between the two values and the Disc_Bit in the reference message is not set, a new value for FDCAN_TURNA.NAV is calculated. If the synchronization deviation $SD = |NC - FDCAN_TURNA.NAV| \leq SDL$ (synchronization deviation limit), the new value for FDCAN_TURNA.NAV takes effect. Else the automatic drift compensation is suspended.

In TTCAN level 0 and level 2, FDCAN_TTOST.QCS indicates whether the automatic drift compensation is active or suspended. In TTCAN level 1, FDCAN_TOST.QCS is always 1.

The current time master may synchronize its local clock speed and the global time phase to an external clock source. This is enabled by bit FDCAN_TTOCF.EECS.

The stop watch function (see [Section 59.4.11](#)) may be used to measure the difference in clock speed between the local clock and the external clock. The local clock speed is adjusted by first writing the newly calculated numerator configuration low to

FDCAN_TURCF.NCL (FDCAN_TURCF.DC cannot be updated during operation). The new value takes effect by writing FDCAN_TTOCN.ECS to 1.

The global time phase is adjusted by first writing the phase offset into the TT global time Preset register FDCAN_TTGTP. The new value takes effect by writing FDCAN_TTOCN.SGT to 1. The first reference message transmitted after the global time phase adjustment has the Disc_Bit set to 1.

FDCAN_TTOST.QGTP shows whether the node global time is in phase with the time master global time. FDCAN_TTOST.QGTP is permanently 0 in TTCAN level 1 and when the synchronization deviation limit is exceeded in TTCAN level 0, 2 (FDCAN_TTOST.QCS = 0). It is temporarily 0 while the global time is low-pass filtered to supply the application with a continuous monotonic value. There is no low-pass filtering when the last reference message contained a Disc_Bit = 1 or when FDCAN_TTOST.QCS = 0.

59.4.13 TTCAN error level

The ISO 11898-4 specifies four levels of error severity:

1. S0 - No error
2. S1 - Warning - Only notification of application, reaction application-specific.
3. S2 error - Notification of application. All transmissions in exclusive or arbitrating time windows are disabled (i.e. no data or remote frames may be started). Potential time masters still transmit reference messages with the reference trigger offset FDCAN_TTOST.RTO set to the maximum value of 127.
4. S3 - Severe error - Notification of application. All CAN bus operations are stopped, i.e. transmission of dominant bits is not allowed, and FDCAN_CCCR.MON is set. The S3 error condition remains active until the application updates the configuration (set FDCAN_CCCR.CCE).

If several errors are detected at the same time, the highest severity prevails. When an error is detected, the application is notified by FDCAN_TTIR.ELC. The error level is monitored by FDCAN_TTOST.EL.

The TTCAN signals the following error conditions as required by ISO 11898-4:

- Config_error (S3)
 - Sets error level FDCAN_TTOST.EL to 11 when a merged arbitrating time window is not properly closed or when there is a Tx_Trigger with a time mark beyond the Tx_Ref_Trigger.
- Watch_Trigger_Reached (S3)
 - Sets error level FDCAN_TTOST.EL to 11 when a watch trigger was reached because the reference message is missing.
- Application_Watchdog (S3)
 - Sets error level FDCAN_TTOST.EL to 11 when the application failed to serve the application watchdog. The application watchdog is configured via FDCAN_TTOCF.AWL. It is served by reading register FDCAN_TTOST. When the watchdog is not served in time, bit FDCAN_TTOST.AWE and interrupt flag

FDCAN_TTIR.AW are set, all FDCAN communication is stopped, and the FDCAN is set into bus monitoring mode (FDCAN_CCCR.MON set to 1).

- CAN_Bus_Off (S3)
 - Entering CAN_Bus_Off state sets error level FDCAN_TTOST.EL to 11. CAN_Bus_Off state is signaled by FDCAN_PSR.BO = 1 and FDCAN_CCCR.INIT = 1.
- Scheduling_Error_2 (S2)
 - Sets error level FDCAN_TTOST.EL to 10 if the MSC of one Tx_Trigger has reached 7. In addition, interrupt flag FDCAN_TTIR.SE2 is set. The error level FDCAN_TTOST.EL is reset to 00 at the beginning of a matrix cycle when no Tx_Trigger has an MSC of 7 in the preceding matrix cycle.
- Tx_Overflow (S2)
 - Sets error level FDCAN_TTOST.EL to 10 when the Tx count is equal to or higher than the expected number of Tx_T riggers FDCAN_TTMLM.ENTT and a Tx_Trigger event occurs. In addition, interrupt flag FDCAN_TTIR.TXO is set. The error level FDCAN_TTOST.EL is reset to 00 when the Tx count is no more than FDCAN_TTMLM.ENTT at the start of a new matrix cycle.
- Scheduling_Error_1 (S1)
 - Sets error level FDCAN_TTOST.EL to 01 if within one matrix cycle the difference between the maximum MSC and the minimum MSC for all trigger memory elements (of exclusive time windows) is larger than two, or if one of the MSCs of an exclusive Rx_Trigger has reached seven. In addition, interrupt flag FDCAN_TTIR.SE1 is set. If within one matrix cycle none of these conditions is valid, the error level FDCAN_TTOST.EL is reset to 00.
- Tx_Underflow (S1)
 - Sets error level FDCAN_TTOST.EL to 01 when the Tx count is less than the expected number of Tx_Triggers FDCAN_TTMLM.ENTT at the start of a new matrix cycle. In addition, interrupt flag FDCAN_TTIR.TXU is set. The error level FDCAN_TTOST.EL is reset to 00 when the Tx count is at least FDCAN_TTMLM.ENTT at the start of a new matrix cycle.

59.4.14 TTCAN message handling

Reference message

For potential time masters the identifier of the reference message is configured via FDCAN_TTRMC.RID. No dedicated Tx buffer is required for transmission of the reference message. When a reference message is transmitted, the first data byte for TTCAN level 1 (that is, the first four data bytes for TTCAN level 0 and the first four data bytes for TTCAN level 2) is provided by the FSE.

In case the reference message Payload select FDCAN_TTRMC.RMPS is set, the rest of the reference message payload (level 1: bytes 2-8, level 0, 2: bytes 5-6) is taken from Tx buffer 0. In this case the data length DLC code from message buffer 0 is used.

Table 513. Number of data bytes transmitted with a reference message

FDCAN_TTRMC.RMPS	FDCAN_TXBRP.TRP0	Level 0	Level 1	Level 2
0	0	4	1	4
0	1	4	1	4

Table 513. Number of data bytes transmitted with a reference message (continued)

FDCAN_TTRMC.RMPS	FDCAN_TXBRP.TRP0	Level 0	Level 1	Level 2
1	0	4	1	4
1	1	4 + MBO	1 + MBO	4 + MBO

To send additional payload with the reference message in level 1 a DLC > 1 must be configured, for level 0 and level 2 a DLC > 4 is required. In addition, the transmission request pending bit FDCAN_TXBRP.TRP0 of message buffer 0 must be set (see [Table 513](#)). In case bit FDCAN_TXBRP.TRP0 is not set when a reference message is started, the reference message is transmitted with the data bytes supplied by the FSE only.

For acceptance filtering of reference messages the reference identifier FDCAN_TTRMC.RID is used.

Message reception

Message reception is done via the two Rx FIFOs in the same way as for event-driven CAN communication (see [Rx handler](#)).

The message status count MSC is part of the corresponding trigger memory element and must be initialized to 0 during configuration. It is updated while the TTCAN is in synchronization states In_Gap or In_Schedule. The update happens at the message Rx_Trigger. At this point in time it is checked at which acceptance filter element the latest message received in this basic cycle had matched. The matching filter number is stored as the acceptance filter result. If this is the same the filter number as defined in this trigger memory element, the MSC is decremented by one. If the acceptance filter result is not the same filter number as defined for this filter element, or if the acceptance filter result is cleared, the MSC is incremented by one. At each Rx_Trigger and at each start of cycle, the last acceptance filter result is cleared.

The time mark of an Rx_Trigger should be set to a value where it is ensured that reception and acceptance filtering for the targeted message has completed. This has to take into consideration the RAM access time and the order of the filter list. It is recommended, that filters which are used for Rx_Triggers are placed at the beginning of the filter list. It is not recommended to use an Rx_Trigger for the reference message.

Message transmission

For time-triggered message transmission the TTCAN supplies 32 dedicated Tx buffers (see [Transmit pause](#)). A Tx FIFO or Tx queue is not available when the FDCAN is configured for time-triggered operation (FDCAN_TTOCF.OM = 01 or 10).

Each Tx_Trigger in the trigger memory points to a particular Tx buffer containing a specific message. There may be more than one Tx_Trigger for a given Tx buffer if that Tx buffer contains a message that is to be transmitted more than once in a basic cycle or matrix cycle.

The application program has to update the data regularly and on time, synchronized to the cycle time. The user is responsible that no partially updated messages are transmitted. To assure this the user has to proceed in the following way:

Tx_Trigger_Single / Tx_Trigger_Merged / Tx_Trigger_Arbitration

- Check whether the previous transmission has completed by reading FDCAN_TXBTO
- Update the Tx buffer configuration and/or payload
- Issue an add request to set the Tx buffer request pending bit

Tx_Trigger_Continuous

- Issue a cancellation request to reset the Tx buffer request pending bit
- Check whether the cancellation has finished by reading FDCAN_TXBCF
- Update Tx buffer configuration and/or payload
- Issue an add request to set the Tx buffer request pending bit

The message MSC stored with the corresponding Tx_Trigger provides information on the success of the transmission.

The MSC is incremented by one when the transmission cannot be started because the CAN bus was not idle within the corresponding transmit enable window or when the message was started and cannot be completed successfully. The MSC is decremented by one when the message was transmitted successfully or when the message may have been started within its transmit enable window but was not started because transmission was disabled (TTCAN in error level S2 or user has disabled this particular message).

The Tx buffers may be managed dynamically, i.e. several messages with different identifiers may share the same Tx buffer element. In this case the user must ensure that no transmission request is pending for the Tx buffer element to be reconfigured by checking FDCAN_TXBRP.

If a Tx buffer with pending transmission request should be updated, the user must first issue a cancellation request and check whether the cancellation has completed by reading FDCAN_TXBCF before it starts updating.

The Tx handler transfers a message from the message RAM to its intermediate output buffer at the trigger element, which becomes active immediately before the Tx_Trigger element (which defines the beginning of the transmit window). During and after the transfer time the transmit message may not be updated and its FDCAN_TXBRP bit may not be changed. To control this transfer time, an additional trigger element may be placed before the Tx_Trigger. This may be example of a Time_Base_Trigger which need not cause any other action. The difference in time marks between the Tx_Trigger and the preceding trigger must be large enough to guarantee that the Tx handler can read four words from the message RAM even at high RAM access load from other modules.

Transmission in exclusive time windows

A transmission is started time-triggered when the cycle time reaches the time mark of a Tx_Trigger_Single or Tx_Trigger_Continuous. There is no arbitration on the bus with messages from other nodes. The MSC is updated according the result of the transmission attempt. After successful transmission started by a Tx_Trigger_Single the respective Tx buffer request pending bit is reset. After successful transmission started by a Tx_Trigger_Continuous the respective Tx buffer request pending remains set. When the transmission was not successful due to disturbances, it is repeated next time (one of) its Tx_Trigger(s) become(s) active.

Transmission in arbitrating time windows

A transmission is started time-triggered when the cycle time reaches the time mark of a Tx_Trigger_Arbitration. Several nodes may start to transmit at the same time. In this case the message has to arbitrate with the messages from other nodes. The MSC is not updated. When the transmission was not successful (lost arbitration or disturbance), it is repeated next time (one of) its Tx_Trigger(s) become(s) active.

Transmission in merged arbitrating time windows

The purpose of a merged arbitrating time window is, to enable multiple nodes to send a limited number of frames which are transmitted in immediate sequence, the order given by CAN arbitration. It is not intended for burst transmission by a node. Since the node does not have exclusive access within this time window, it may happen that not all requested transmissions are successful.

Messages which have lost arbitration or were disturbed by an error, may be re-transmitted inside the same merged arbitrating time window. The re-transmission is not started if the corresponding transmission request pending flag is reset by a successful Tx cancellation.

In single transmit windows, the Tx handler transmits the message indicated by the message number of the trigger element. In merged arbitrating time windows, it can handle up to three message numbers from the trigger list. Their transmission is attempted in the sequence defined by the trigger list. If the time mark of a fourth message is reached before the first is transmitted (or canceled by the user), the fourth request is ignored.

The transmission inside a merged arbitrating time window is not time-triggered. The transmission of a message may start before its time mark, or after the time mark if the bus was not idle.

The messages transmitted by a specific node inside a merged arbitrating time window are started in the order of their Tx_Triggers, so a message with low CAN priority may prevent the successful transmission of a following message with higher priority, if there is compelling bus traffic. This must be considered for the configuration of the trigger list.

Time_Base_Triggers may be placed between consecutive Tx_Triggers to define the time until the data of the corresponding Tx buffer needs to be updated.

59.4.15 TTCAN interrupt and error handling

The TT interrupt register FDCAN_TTIR consists of four segments. Each interrupt can be enabled separately by the corresponding bit in the TT interrupt enable register FDCAN_TTIE. The flags remain set until the user clears them. A flag is cleared by writing a 1 to the corresponding bit position.

The first segment consists of flags CER, AW, WT, and IWT. Each flag indicates a fatal error condition where the CAN communication is stopped. With the exception of IWT, these error conditions require a re-configuration of the FDCAN module before the communication can be restarted.

The second segment consists of flags ELC, SE1, SE2, TXO, TXU, and GTE. Each flag indicates an error condition where the CAN communication is disturbed. If they are caused by a transient failure, for example by disturbances on the CAN bus, they are handled by the FDCAN protocol failure handling and do not require intervention by the application program.

The third segment consists of flags GTD, GTW, SWE, TTMI, and RTMI. The first two flags are controlled by global time events (level 0, 2 only) that require a reaction by the application program. With a stop watch event triggered by a rising edge on pin fdcan_swt internal time

values are captured. The trigger time mark interrupt notifies the application that a specific Time_Base_Trigger is reached. The register time mark interrupt signals that the time referenced by FDCAN_TTOCN.TMC (cycle, local, or global) is equal to the time mark FDCAN_TTTMK.TM. It can also be used to finish a gap.

The fourth segment consists of flags SOG, CSM, SMC, and SBC. These flags provide a means to synchronize the application program to the communication schedule.

59.4.16 Level 0

TTCAN level 0 is not part of ISO11898-4. This operation mode makes the hardware, that in TTCAN level 2 maintains the calibrated global time base, also available for event-driven CAN according to ISO11898-1.

Level 0 operation is configured via FDCAN_TTOCF.OM = 11. In this mode the FDCAN operates in event driven CAN communication, there is no fixed schedule, the configuration of FDCAN_TTOCF.GEN is ignored. External event-synchronized operation is not available in level 0. A synchronized time base is maintained by transmission of reference messages.

In level 0 the trigger memory is not active and therefore needs not to be configured. The time mark interrupt flag (FDCAN_TTIR.TTMI) is set when the cycle time has reached FDCAN_TTOCF.IRTO = 0x200, it reminds the user to set a transmission request for message buffer 0. The Watch_Trigger interrupt flag (FDCAN_TTIR.WT) is set when the cycle time has reached 0xFF00. These values were chosen to have enough margin for a stable clock calibration. There are no further TT-error-checks.

Register time mark interrupts (FDCAN_TTIR.RTMI) are also possible.

The reference message is configured as for level 2 operation. Received reference messages are recognized by the identifier configured in register FDCAN_TTRMC. For the transmission of reference messages only message buffer 0 may be used. The node transmits reference messages any time the user sets a transmission request for message buffer 0, there is no reference trigger offset.

Level 0 operation is configured via:

- FDCAN_TTRMC
- FDCAN_TTOCF except EVTP, AWL, GEN
- FDCAN_TTMLM except ENTT, TXEW
- FDCAN_TURCF

Level 0 operation is controlled via:

- FDCAN_TTOCN except NIG, TMG, FGP, GCS, TTMIE
- FDCAN_TTGTP
- FDCAN_TTTMK
- FDCAN_TTIR excluding bits CER, AW, IWT SE2, SE1, TXO, TXU, SOG (no function)
- FDCAN_TTIR the following bits have changed function
 - TTMI not defined by trigger memory - activated at cycle time FDCAN_TTOCF.IRTO 0x200
 - WT not defined by trigger memory - activated at cycle time 0xFF00

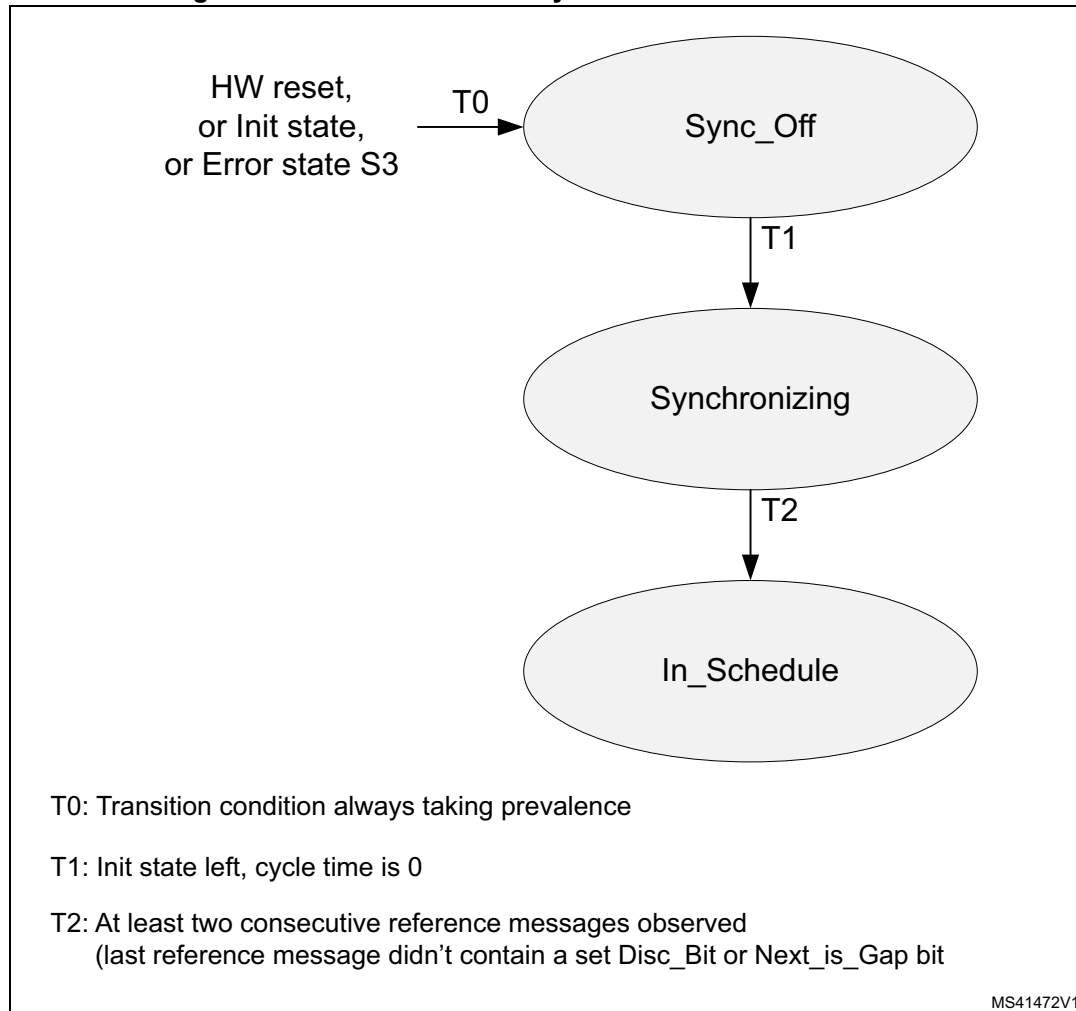
Level 0 operation is signaled via:

- TTOST excluding bits AWE, WFE, GSI, GFI, RTO (no function)

Synchronizing

Figure 790 describes the states and the state transitions in TTCAN level 0 operation. level 0 has no In_Gap state.

Figure 790. Level 0 schedule synchronization state machine



Handling of error levels

During level 0 operation only the following error conditions may occur:

- Watch_Trigger_Reached (S3), reached cycle time 0xFF00
- CAN_Bus_Off (S3)

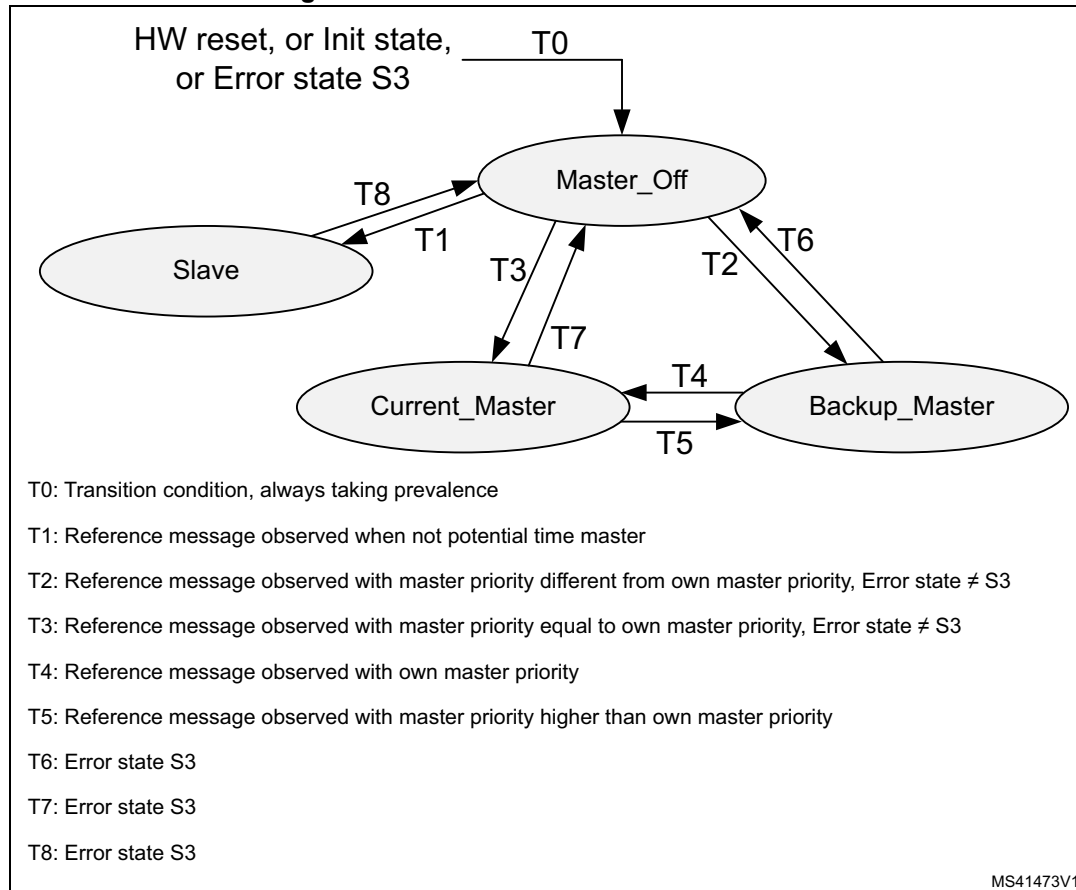
Since no S1 and S2 errors are possible, the error level can only switch between S0 (No error) and S3 (Severe error). In TTCAN level 0 an S3 error is handled differently. When error level S3 is reached, both FDCAN_TTOST.SYS and FDCAN_TTOST.MS are reset, and interrupt flags FDCAN_TTIR.GTE and FDCAN_TTIR.GTD are set.

When error level S3 (FDCAN_TTOST.EL = 11) is entered, bus monitoring mode is (contrary to TTCAN level 1 and level 2) not entered. S3 error level is left automatically after transmission (time master) or reception (time slave) of the next reference message.

Master slave relation

Figure 791 describes the master slave relation in TTCAN level 0. In case of an S3 error the FDCAN returns to state Master_Off.

Figure 791. Level 0 master to slave relation



59.4.17 Synchronization to external time schedule

This feature can be used to synchronize the phase of the FDCAN schedule to an external schedule (for example that of a second TTCAN network or FlexRay network). It is applicable only when the FDCAN is current time master (FDCAN_TTOST.MS = 11).

External synchronization is controlled by event trigger input pin `fdcan_evt`. If bit `FDCAN_TTOCN.ESCN` is set, a rising edge at event trigger pin `fdcan_evt` the FDCAN compares its actual cycle time with the target phase value configured by `FDCAN_TTGTP.CTP`.

Before setting `FDCAN_TTOCN.ESCN` the user has to adapt the phases of the two time schedules for example by using the FDCAN gap control (see [Section 59.4.10](#)). When the user sets `FDCAN_TTOCN.ESCN`, `FDCAN_TTOSTSPL` is set.

If the difference between the cycle time and the target phase value `FDCAN_TTGTP.CTP` at the rising edge at event trigger pin `fdcan_evt` is greater than 9 NTU, the phase lock bit `FDCAN_TTOST.SPL` is reset, and interrupt flag `FDCAN_TTIR.CSM` is set.

FDCAN_TTOST.SPL is also reset (and FDCAN_TTIR.CSM is set), when another node becomes time master.

If both FDCAN_TTOST.SPL and FDCAN_TTOCN.ESCN are set, and if the difference between the cycle time and the target phase value FDCAN_TTGTP.CTP at the rising edge at event trigger pin fdcan_evt is lower or equal to nine NTU, the phase lock bit FDCAN_TTOST.SPL remains set, and the measured difference is used as reference trigger offset value to adjust the phase at the next transmitted reference message.

Note: The rising edge detection at event trigger pin fdcan_evt is enabled with the start of each basic cycle. The first rising edge triggers the compare of the actual cycle time with FDCAN_TTGTP.CTP. All further edges until the beginning of the next basic cycle are ignored.

59.4.18 FDCAN Rx buffer and FIFO element

Up to 64 Rx buffers and two Rx FIFOs can be configured in the message RAM. Each Rx FIFO section can be configured to store up to 64 received messages. The structure of a Rx buffer / FIFO element is shown in [Table 514](#), the description is provided in [Table 515](#).

Table 514. Rx buffer and FIFO element

Bit	31				24	23				16	15	8	7	0
R0	ESI	XTD	RTR	ID[28:0]										
R1	ANMF	FIDX[6:0]			Res.	FDF	BRS	DLC[3:0]	RXTS[15:0]					
R2	DB3[7:0]				DB2[7:0]				DB1[7:0]		DB0[7:0]			
R3	DB7[7:0]				DB6[7:0]				DB5[7:0]		DB4[7:0]			
⋮	⋮				⋮				⋮					
Rn	DBm[7:0]				DBm-1[7:0]				DBm-2[7:0]		DBm-3[7:0]			

The element size can be configured for storage of CAN FD messages with up to 64 bytes data field via register FDCAN_RXESC.

Table 515. Rx buffer and FIFO element description

Field	Description
R0 bit 31 ESI	Error state indicator 0: Transmitting node is error active 1: Transmitting node is error passive
R0 bit 30 XTD	Extended identifier Signals to the user whether the received frame has a standard or extended identifier. 0: 11-bit standard identifier 1: 29-bit extended identifier
R0 bit 29 RTR	Remote transmission request Signals to the user whether the received frame is a data frame or a remote frame. 0: Received frame is a data frame 1: Received frame is a remote frame

Table 515. Rx buffer and FIFO element description (continued)

Field	Description
R0 bits 28:0 ID[28:0]	Identifier Standard or extended identifier depending on bit XTD. A standard identifier is stored into ID[28:18].
R1 bit 31 ANMF	Accepted non-matching frame Acceptance of non-matching frames may be enabled via FDCAN_GFC.ANFS and FDCAN_GFC.ANFE. 0: Received frame matching filter index FIDX 1: Received frame did not match any Rx filter element
R1 bits 30:24 FIDX[6:0]	Filter index 0-127 = index of matching Rx acceptance filter element (invalid if ANMF = 1). Range is 0 to FDCAN_SIDFC.LSS. - 1 or FDCAN_XIDFC.LSE. - 1.
R1 bit 21 FDF	FD Format 0: Standard frame format 1: FDCAN frame format (new DLC-coding and CRC)
R1 bit 20 BRS	Bitrate Switch 0: Frame received without bitrate switching 1: Frame received with bitrate switching
R1 bits 19:16 DLC[3:0]	Data length code 0-8: Classic CAN + CAN FD: received frame has 0-8 data bytes 9-15: Classic CAN: received frame has 8 data bytes 9-15: CAN FD: received frame has 12/16/20/24/32/48/64 data bytes
R1 bits 15:0 RXTS[15:0]	Rx timestamp Timestamp counter value captured on start of frame reception. Resolution depending on configuration of the timestamp counter prescaler FDCAN_TSICC.TCP.
R2 bits 31:24 DB3[7:0]	Data Byte 3
R2 bits 23:16 DB2[7:0]	Data Byte 2
R2 bits 15:8 DB1[7:0]	Data Byte 1
R2 bits 7:0 DB0[7:0]	Data Byte 0
R3 bits 31:24 DB7[7:0]	Data Byte 7
R3 bits 23:16 DB6[7:0]	Data Byte 6
R3 bits 15:8 DB5[7:0]	Data Byte 5
R3 bits 7:0 DB4[7:0]	Data Byte 4
⋮	⋮

Table 515. Rx buffer and FIFO element description (continued)

Field	Description
Rn bits 31:24 DBm[7:0]	Data Byte m
Rn bits 23:16 DBm-1[7:0]	Data Byte m-1
Rn bits 15:8 DBm-2[7:0]	Data Byte m-2
Rn bits 7:0 DBm-3[7:0]	Data Byte m-3

59.4.19 FDCAN Tx buffer element

The Tx buffers section can be configured to hold dedicated Tx buffers as well as a Tx FIFO / Tx queue. In case that the Tx buffers section is shared by dedicated Tx buffers and a Tx FIFO / Tx queue, the dedicated Tx buffers start at the beginning of the Tx buffers section followed by the buffers assigned to the Tx FIFO or Tx queue. The Tx handler distinguishes between dedicated Tx buffers and Tx FIFO / Tx queue by evaluating the Tx buffer configuration FDCAN_TXBC.TFQS and FDCAN_TXBC.NDTB. The element size can be configured for storage of CAN FD messages with up to 64 bytes data field via register FDCAN_TXESC.

Table 516. Tx buffer and FIFO element

Bit	31	24	23	16	15	8	7	0
T0	ESI	XTD	RTR	ID[28:0]				
T1	MM[7:0]			EFC	Res.	FDF	BPS	DLC[3:0]
T2	DB3[7:0]			DB2[7:0]			DB1[7:0]	DB0[7:0]
T3	DB7[7:0]			DB6[7:0]			DB5[7:0]	DB4[7:0]
⋮	⋮			⋮			⋮	
Tn	DBm[7:0]			DBm-1[7:0]			DBm-2[7:0]	DBm-3[7:0]

Table 517. Tx buffer element description

Field	Description
T0 bit 31 ESI ⁽¹⁾	Error state indicator 0: ESI bit in CAN FD format depends only on error passive flag 1: ESI bit in CAN FD format transmitted recessive
T0 bit 30 XTD	Extended identifier 0: 11-bit standard identifier 1: 29-bit extended identifier
T0 bit 29 RTR ⁽²⁾	Remote transmission request 0: Transmit data frame 1: Transmit remote frame

Table 517. Tx buffer element description (continued)

Field	Description
T0 bits 28:0 ID[28:0]	Identifier Standard or extended identifier depending on bit XTD. A standard identifier must be written to ID[28:18].
T1 bits 31:24 MM[7:0]	Message marker Written by CPU during Tx buffer configuration. Copied into Tx event FIFO element for identification of Tx message status.
T1 bit 23 EFC	Event FIFO control 0: Don't store Tx events 1: Store Tx events
T1 bit 21 FDF	FD Format 0: Frame transmitted in classic CAN format 1: Frame transmitted in CAN FD format
T1 bit 20 BRS ⁽³⁾	Bitrate switching 0: CAN FD frames transmitted without bitrate switching 1: CAN FD frames transmitted with bitrate switching
T1 bits 19:16 DLC[3:0]	Data length code 0 - 8: Classic CAN + CAN FD: received frame has 0-8 data bytes 9-15: Classic CAN: received frame has 8 data bytes 9 - 15: CAN FD: received frame has 12/16/20/24/32/48/64 data bytes
T2 bits 31:24 DB3[7:0]	Data Byte 3
T2 bits 23:16 DB2[7:0]	Data Byte 2
T2 bits 15:8 DB1[7:0]	Data Byte 1
T2 bits 7:0 DB0[7:0]	Data Byte 0
T3 bits 31:24 DB7[7:0]	Data Byte 7
T3 bits 23:16 DB6[7:0]	Data Byte 6
T3 bits 15:8 DB5[7:0]	Data Byte 5
T3 bits 7:0 DB4[7:0]	Data Byte 4
⋮	⋮
Tn bits 31:24 DBm[7:0]	Data Byte m
Tn bits 23:16 DBm-1[7:0]	Data Byte m-1

Table 517. Tx buffer element description (continued)

Field	Description
Tn bits 15:8 DBm-2[7:0]	Data Byte m-2
Tn bits 7:0 DBm-3[7:0]	Data Byte m-3

1. The ESI bit of the transmit buffer is OR-ed with the error passive flag to decide the value of the ESI bit in the transmitted FD frame. As required by the CAN FD protocol specification, an error active node may optionally transmit the ESI bit recessive, but an error passive node always transmit the ESI bit recessive.
2. When RTR = 1, the FDCAN transmits a remote frame according to ISO11898-1, even if FDCAN_CCCR.FDOE enables the transmission in CAN FD format.
3. Bits ESI, FDF, and BRS are only evaluated when CAN FD operation is enabled FDCAN_CCCR.FDOE = 1. Bit BRS is only evaluated when in addition FDCAN_CCCR.BRSE = 1.

59.4.20 FDCAN Tx event FIFO element

Each element stores information about transmitted messages. By reading the Tx event FIFO the user gets this information in the order the messages were transmitted. Status information about the Tx event FIFO can be obtained from register FDCAN_TXEFS.

Table 518. Tx Event FIFO element

Bit	31	24	23	16	15	8	7	0
E0	ESI	XTD	RTR	ID[28:0]				
E1	MM[7:0]			ET[1:0]	EDL	BRS	DLC[3:0]	TXTS[15:0]

Table 519. Tx Event FIFO element description

Field	Description
E0 bit 31 ESI	Error state indicator – 0: Transmitting node is error active – 1: Transmitting node is error passive
E0 bit 30 XTD	Extended Identifier – 0: 11-bit standard identifier – 1: 29-bit extended identifier
E0 bit 29 RTR	Remote transmission request – 0: Transmit data frame – 1: Transmit remote frame
E0 bits 28:0 ID[28:0]	Identifier Standard or extended identifier depending on bit XTD. A standard identifier must be written to ID[28:18].
E1 bits 31:24 MM[7:0]	Message marker Copied from Tx buffer into Tx event FIFO element for identification of Tx message status.

Table 519. Tx Event FIFO element description (continued)

Field	Description
E1 bits 23:22 EFC	Event type – 00: Reserved – 01: Tx event – 10: Transmission in spite of cancellation (always set for transmissions in DAR mode) – 11: Reserved
E1 bit 21 EDL	Extended data length – 0: Standard frame format – 1: FDCAN frame format (new DLC-coding and CRC)
E1 bit 20 BRS	Bitrate switching – 0: Frame transmitted without bitrate switching – 1: Frame transmitted with bitrate switching
T1 bits 19:16 DLC[3:0]	Data length code – 0 - 8: Frame with 0-8 data bytes transmitted – 9 - 15: Frame with eight data bytes transmitted
E1 bits 15:0 TXTS[15:0]	Tx timestamp Timestamp counter value captured on start of frame transmission. Resolution depending on configuration of the timestamp counter prescaler FDCAN_TSCC.TCP.

59.4.21 FDCAN standard message ID filter element

Up to 128 filter elements can be configured for 11-bit standard IDs. When accessing a standard message ID filter element, its address is the filter list standard start address FDCAN_SIDFC.FLSSA plus the index of the filter element (0 ... 127).

Table 520. Standard message ID filter element

Bit	31	24	23	16	15	8	7	0
S0	SFT[1:0]	SFEC[2:0]	SFID1[10:0]			Res.	SFID2[10:0]	

Table 521. Standard message ID filter element field description

Field		Description
Bit 31:30 SFT[1:0] ⁽¹⁾		Standard filter type – 00: Range filter from SFID1 to SFID2 – 01: Dual ID filter for SFID1 or SFID2 – 10: Classic filter: SFID1 = filter, SFID2 = mask – 11: Filter element disabled
Bit 29:27 SFEC[2:0]		Standard filter element configuration All enabled filter elements are used for acceptance filtering of standard frames. Acceptance filtering stops at the first matching enabled filter element or when the end of the filter list is reached. If SFEC = “100”, “101”, or “110” a match sets interrupt flag FDCAN_IR.HPM and, if enabled, an interrupt is generated. In this case register FDCAN_HPMS is updated with the status of the priority match. – 000: Disable filter element – 001: Store in Rx FIFO 0 if filter matches – 010: Store in Rx FIFO 1 if filter matches – 011: Reject ID if filter matches – 100: Set priority if filter matches – 101: Set priority and store in FIFO 0 if filter matches – 110: Set priority and store in FIFO 1 if filter matches – 111: Store into Rx buffer or as debug message, configuration of FDCAN_SFT[1:0] ignored
Bits 26:16 SFID1[10:0]		Standard filter ID 1 First ID of standard ID filter element. When filtering for Rx buffers or for debug messages this field defines the ID of a standard message to be stored. The received identifiers must match exactly, no masking mechanism is used.
Bits 15:0	SFID2[15:10]	Standard filter ID 2 This bit field has a different meaning depending on the configuration of SFEC: – SFEC = 001 ... 110 Second ID of standard ID filter element – SFEC = 111 Filter for Rx buffers or for debug messages
	SFID2[10:9]	Decides whether the received message is stored into an Rx buffer or treated as message A, B, or C of the debug message sequence. – 00: Store message into an Rx buffer – 01: Debug message A – 10: Debug message B – 11: Debug message C
	SFID2[8:6]	Is used to control the filter event pins at the Extension Interface. A 1 at the respective bit position enables generation of a pulse at the related filter event pin with the duration of one fdcan_pclk period in case the filter matches. SFID2[8] is used by the calibration unit.
	SFID2[5:0]	Defines the offset to the Rx buffer start address FDCAN_RXBC.RBSA for storage of a matching message.

1. With SFT = “11” the filter element is disabled and the acceptance filtering continues (same behavior as with SFEC = “000”).

Note: In case a reserved value is configured, the filter element is considered disabled.

59.4.22 FDCAN extended message ID filter element

Up to 64 filter elements can be configured for 29-bit extended IDs. When accessing an Extended message ID filter element, its address is the filter list extended start address FDCAN_XIDFC.FLESA plus two times the index of the filter element (0 ... 63).

Table 522. Extended message ID filter element

Bit	31	30	29	28	0
F0	EFEC[2:0]			EFID1[28:0]	
F1	EFTI[1:0]		Reserved	EFID2[28:0]	

Table 523. Extended message ID filter element field description

Field	Description
F0 bits 31:29 EFEC[2:0]	Extended filter element configuration All enabled filter elements are used for acceptance filtering of extended frames. Acceptance filtering stops at the first matching enabled filter element or when the end of the filter list is reached. If EFEC = 100, 101, or 110 a match sets interrupt flag FDCAN_IR.HPM and, if enabled, an interrupt is generated. In this case register FDCAN_HPMS is updated with the status of the priority match. – 000: Disable filter element – 001: Store in Rx FIFO 0 if filter matches – 010: Store in Rx FIFO 1 if filter matches – 011: Reject ID if filter matches – 100: Set priority if filter matches – 101: Set priority and store in FIFO 0 if filter matches – 110: Set priority and store in FIFO 1 if filter matches – 111: Store into Rx buffer, configuration of EFT[1:0] ignored
F0 bits 28:0 EFID1[28:0]	Extended filter ID 1 First ID of extended ID filter element. When filtering for Rx buffers or for debug messages this field defines the ID of an extended message to be stored. The received identifiers must match exactly, only FDCAN_XIDAM masking mechanism.
F1 bits 31:30 EFT[1:0]	Extended filter type – 00: Range filter from EF1ID to EF2ID ($EF2ID \geq EF1ID$) – 01: Dual ID filter for EF1ID or EF2ID – 10: Classic filter: EF1ID = filter, EF2ID = mask – 11: Range filter from EF1ID to EF2ID ($EF2ID \geq EF1ID$), FDCAN_XIDAM mask not applied

Table 523. Extended message ID filter element field description (continued)

Field		Description
F1 bits 28:0	EFID2[10:0]	Extended filter ID 2 This bit field has a different meaning depending on the configuration of EFEC: – SFEC = 001 ... 110 Second ID of extended ID filter element – SFEC = 111 Filter for Rx buffers or for debug messages
	EFID2[10:9]	Decides whether the received message is stored into an Rx buffer or treated as message A, B, or C of the debug message sequence. – 00: Store message into an Rx buffer – 01: Debug message A – 10: Debug message B – 11: Debug message C
	EFID2[8:6]	Is used to control the filter event pins at the Extension Interface. A 1 at the respective bit position enables generation of a pulse at the related filter event pin with the duration of one fdcan_pclk period in case the filter matches. EFID2[8] interface is used by the calibration unit.
	EFID2[5:0]	Defines the offset to the Rx buffer start address FDCAN_RXBC.RBSA for storage of a matching message.

59.4.23 FDCAN trigger memory element

Up to 64 trigger memory elements can be configured. When accessing a trigger memory element, its address is the trigger memory start address FDCAN_TTTMC.TMSA plus the index of the trigger memory element (0 ... 63).

Table 524. Trigger memory element

Bit	31	24	23	16	15	8	7				0
T0	TM[15:0]				Res.	CC[6:0]	Res.	TMIN	TMEX	TYPE[3:0]	
T1	Res.	FTYPE	MNR[6:0]		Res.					MSC[2:0]	

Table 525. Trigger memory element description

Field	Description
T0 bits 31:16 TM[15:0]	Time mark Cycle time for which the trigger becomes active.
T0 bit 14:8 CC[6:0]	Cycle code Cycle count for which the trigger is valid. Ignored for trigger types Tx_Ref_Trigger, Tx_Ref_Trigger_Gap, Watch_Trigger, Watch_Trigger_Gap, End_of_List. – 0b000000x valid for all cycles – 0b000001c valid every 2 nd cycle at cycle count mod2 = c – 0b00001cc valid every 4 th cycle at cycle count mod4 = cc – 0b0001ccc valid every 8 th cycle at cycle count mod8 = ccc – 0b001cccc valid every 16 th cycle at cycle count mod16 = cccc – 0b01ccccc valid every 32 nd cycle at cycle count mod32 = ccccc – 0b1cccccc valid every 64 th cycle at cycle count mod64 = ccccccc

Table 525. Trigger memory element description (continued)

Field	Description
T0 bit 5 TMIN	Time mark event internal – 0: No action – 1: FDCAN_TTIR.TTMI is set when trigger memory element becomes active
T0 bit 4 TMEX	Time mark event external – 0: No action – 1: Pulse at output fdcan1_tmp for more than one instance and fdcan_tmp if only one instance with the length of one period is generated when the time arc of the trigger memory element becomes active and FDCAN_TTOCN.TTMIE = 1
T0 bit 3:0 TYPE[3:0]	Trigger type – 0000 Tx_Ref_Trigger - valid when not in gap – 0001 Tx_Ref_Trigger_Gap - valid when in gap – 0010 Tx_Trigger_Single - starts a single transmission in an exclusive time window – 0011 Tx_Trigger_Continuous - starts continuous transmission in an exclusive time window – 0100 Tx_Trigger_Arbitration - starts a transmission in an arbitrating time window – 0101 Tx_Trigger_Merged - starts a merged arbitration window – 0110 Watch_Trigger - valid when not in gap – 0111 Watch_Trigger_Gap - valid when in gap – 1000 Rx_Trigger - check for reception – 1001 Time_Base_Trigger - only control TMIN, TMEX – 1010 ... 1111=End_of_List - illegal type, causes configuration error
T1 bit 23FTYPE	Filter type – 0: 11-bit standard message ID – 1: 29-bit extended message ID
T1 bit 22:16 MNR[6:0] ⁽¹⁾	Message number – Transmission: trigger is valid for configured Tx buffer number. Valid values are 0 to 31. – Reception: trigger is valid for standard/extended message ID filter element number. Valid values are, respectively 0 to 63 and 0 to 127.
T1 bits 2:0 MSC[2:0]	Message status count Counts scheduling errors for periodic messages in exclusive time windows. It has no function for arbitrating messages and in event-driven CAN communication (ISO11898-1). – 0-7= Actual status

1. The trigger memory elements have to be written when the FDCAN is in INIT state. Write access to the trigger memory elements outside INIT state is not allowed. There is an exception for TMIN and TMEX when they are defined as part of a trigger memory element of TYPE Tx_Ref_Trigger. In this case they become active at the time mark modified by the actual reference trigger offset (TTOST[RTO]).

59.5 FDCAN registers

59.5.1 FDCAN core release register (FDCAN_CREL)

Address offset: 0x0000

Reset value: 0x3214 1218

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
REL[3:0]				STEP[3:0]				SUBSTEP[3:0]				YEAR[3:0]			
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MON[7:0]								DAY[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:28 **REL[3:0]**: Core release = 3

Bits 27:24 **STEP[3:0]**: Step of core release = 2

Bits 23:20 **SUBSTEP[3:0]**: Sub-step of core release = 1

Bits 19:16 **YEAR[3:0]**: Timestamp year = 4

Bits 15:8 **MON[7:0]**: Timestamp month = 12

Bits 7:0 **DAY[7:0]**: Timestamp day = 18

59.5.2 FDCAN Endian register (FDCAN_ENDN)

Address offset: 0x0004

Reset value: 0x8765 4321

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ETV[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETV[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **ETV[31:0]**: Endianness test value

The endianness test value is 0x8765 4321.

59.5.3 FDCAN data bit timing and prescaler register (FDCAN_DBTP)

Address offset: 0x000C

Reset value: 0x0000 0A33

This register is dedicated to data bit timing phase and only writable if bits FDCAN_CCCR.CCE and FDCAN_CCCR.INIT are set. The CAN time quantum may be programmed in the range from 1 to 32 FDCAN clock periods. $t_q = (DBRP + 1)$ FDCAN clock periods.

DTSEG1 is the sum of Prop_Seg and Phase_Seg1. DTSEG2 is Phase_Seg2. Therefore the length of the bit time is $(DTSEG1 + DTSEG2 + 3) t_q$ for programmed values, or $(Sync_Seg + Prop_Seg + Phase_Seg1 + Phase_Seg2) t_q$ for functional values.

The information processing time (IPT) is zero, meaning the data for the next bit is available at the first clock edge after the sample point.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TDC	Res.	Res.	DBRP[4:0]				
								rw			rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	DTSEG1[4:0]					DTSEG2[3:0]				DSJW[3:0]			
			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **TDC**: Transceiver delay compensation

0: Transceiver delay compensation disabled

1: Transceiver delay compensation enabled

Bits 22:21 Reserved, must be kept at reset value.

Bits 20:16 **DBRP[4:0]**: Data bitrate prescaler

The value by which the oscillator frequency is divided to generate the bit time quanta. The bit time is built up from a multiple of these quanta. Valid values for the baudrate prescaler are 0 to 31. The hardware interpreters this value as the programmed value plus 1.

Bits 15:13 Reserved, must be kept at reset value.

Bits 12:8 **DTSEG1[4:0]**: Data time segment before sample point

Valid values are 0 to 31. The value used by the hardware is the one programmed, incremented by 1, i.e. $t_{BS1} = (DTSEG1 + 1) \times t_q$.

Bits 7:4 **DTSEG2[3:0]**: Data time segment after sample point

Valid values are 0 to 15. The value used by the hardware is the one programmed, incremented by 1, i.e. $t_{BS2} = (DTSEG2 + 1) \times t_q$.

Bits 3:0 **DSJW[3:0]**: Synchronization jump width

Valid values are 0 to 15. The value used by the hardware is the one programmed, incremented by 1, i.e. $t_{SJW} = (DSJW + 1) \times t_q$.

Note: With an FDCAN clock of 8 MHz, the reset value 0x00000A33 configures the FDCAN for a fast bitrate of 500 kbit/s.

Note: The data phase bit rate must be higher than or equal to the nominal bit rate.

59.5.4 FDCAN test register (FDCAN_TEST)

Write access to this register must be enabled by setting bit FDCAN_CCCR.TEST to 1. All register functions are set to their reset values when bit FDCAN_CCCR.TEST is reset.

Loop back mode and software control of Tx pin FDCANx_TX are hardware test modes. Programming TX differently from 00 may disturb the message transfer on the CAN bus.

Address offset: 0x0010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RX	TX[1:0]		LBCK	Res.	Res.	Res.	Res.
								r	rw	rw	rw				

Bits 31:8 Reserved, must be kept at reset value.

Bit 7 **RX**: Receive pin

Monitors the actual value of transmit pin FDCANx_RX

0: The CAN bus is dominant (FDCANx_RX = 0)

1: The CAN bus is recessive (FDCANx_RX = 1)

Bits 6:5 **TX[1:0]**: Control of transmit pin

00: Reset value, FDCANx_TX TX is controlled by the CAN core, updated at the end of the CAN bit time

01: Sample point can be monitored at pin FDCANx_TX

10: Dominant (0) level at pin FDCANx_TX

11: Recessive (1) at pin FDCANx_TX

Bit 4 **LBCK**: Loop back mode

0: Reset value, loop back mode is disabled

1: Loop back mode is enabled (see [Test modes](#))

Bits 3:0 Reserved, must be kept at reset value.

59.5.5 FDCAN RAM watchdog register (FDCAN_RWD)

The RAM watchdog monitors the READY output of the message RAM. A message RAM access starts the message RAM watchdog counter with the value configured by the FDCAN_RWD.WDC bits.

The counter is reloaded with FDCAN_RWD.WDC bits when the message RAM signals successful completion by activating its READY output. In case there is no response from the message RAM until the counter has counted down to 0, the counter stops and interrupt flag FDCAN_IR.WDI bit is set. The RAM watchdog counter is clocked by the fdcan_pclk clock.

Address offset: 0x0014

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WDV[7:0]								WDC[7:0]							
r	r	r	r	r	r	r	r	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:8 **WDV[7:0]**: Watchdog value

Actual message RAM watchdog counter value.

Bits 7:0 **WDC[7:0]**: Watchdog configuration

Start value of the message RAM watchdog counter. With the reset value of 00 the counter is disabled.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

59.5.6 FDCAN CC control register (FDCAN_CCCR)

Address offset: 0x0018

Reset value: 0x0000 0001

For details about setting and resetting of single bits, see [Software initialization](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NISO	TXP	EFBI	PXHD	Res.	Res.	BRSE	FDOE	TEST	DAR	MON	CSR	CSA	ASM	CCE	INIT
rW	rW	rW	rW			rW	rW	rW	rW	rW	rW	r	rW	rW	rW

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **NISO**: Non ISO operation

If this bit is set, the FDCAN uses the CAN FD frame format as specified by the Bosch CAN FD Specification V1.0.

0: CAN FD frame format according to ISO11898-1

1: CAN FD frame format according to Bosch CAN FD Specification V1.0

Bit 14 **TXP**:

If this bit is set, the FDCAN pauses for two CAN bit times before starting the next transmission after successfully transmitting a frame.

0: Disabled

1: Enabled

Bit 13 **EFBI**: Edge filtering during bus integration

0: Edge filtering disabled

1: Two consecutive dominant tq required to detect an edge for hard synchronization

Bit 12 **PXHD**: Protocol exception handling disable

- 0: Protocol exception handling enabled
- 1: Protocol exception handling disabled

Bits 11:10 Reserved, must be kept at reset value.

Bit 9 **BRSE**: FDCAN bitrate switching

- 0: Bitrate switching for transmissions disabled
- 1: Bitrate switching for transmissions enabled

Bit 8 **FDOE**: FD operation enable

- 0: FD operation disabled
- 1: FD operation enabled

Bit 7 **TEST**: Test mode enable

- 0: Normal operation, register TEST holds reset values
- 1: Test mode, write access to register TEST enabled

Bit 6 **DAR**: Disable automatic retransmission

- 0: Automatic retransmission of messages not transmitted successfully enabled
- 1: Automatic retransmission disabled

Bit 5 **MON**: Bus monitoring mode

Bit MON can be set only by software when both CCE and INIT are set to 1. The bit can be reset by the user at any time.

- 0: Bus monitoring mode is disabled
- 1: Bus monitoring mode is enabled

Bit 4 **CSR**: Clock stop request

- 0: No clock stop is requested
- 1: Clock stop requested. When clock stop is requested, first INIT and then CSA is set after all pending transfer requests have been completed and the CAN bus reached idle.

Bit 3 **CSA**: Clock stop acknowledge

- 0: No clock stop acknowledged
- 1: FDCAN may be set in power down by stopping APB clock and kernel clock

Bit 2 **ASM**: ASM restricted operation mode

The restricted operation mode is intended for applications that adapt themselves to different CAN bitrates. The application tests different bitrates and leaves the restricted operation mode after it has received a valid frame. In the optional restricted operation mode the node is able to transmit and receive data and remote frames and it gives acknowledge to valid frames, but it does not send active error frames or overload frames. In case of an error condition or overload condition, it does not send dominant bits, instead it waits for the occurrence of bus idle condition to resynchronize itself to the CAN communication. The error counters are not incremented. Bit ASM can be set only by software when both CCE and INIT are set to 1. The bit can be reset by the software at any time. This bit is set automatically set to 1 when the Tx handler was not able to read data from the message RAM in time. If the FDCAN is connected to a clock calibration on CAN unit, ASM bit is set by hardware as long as the calibration is not completed.

- 0: Normal CAN operation
- 1: Restricted operation mode active

Bit 1 **CCE**: Configuration change enable

- 0: The CPU has no write access to the protected configuration registers
- 1: The CPU has write access to the protected configuration registers (while FDCAN_CCCR.INIT = 1 CCE bit is automatically cleared when INIT bit is cleared)

Bit 0 **INIT**: Initialization

0: Normal operation

1: Initialization is started (while FDCAN_CCCR.INIT = 1 CCE bit is automatically cleared when INIT bit is cleared)

Note: Due to the synchronization mechanism between the two clock domains, there may be a delay until the value written to INIT can be read back. Therefore the programmer must ensure that the previous value written to INIT has been accepted by reading INIT before setting INIT to a new value.

59.5.7 FDCAN nominal bit timing and prescaler register (FDCAN_NBTP)

Address offset: 0x001C

Reset value: 0x0600 0A03

This register is dedicated to the nominal bit timing used during the arbitration phase, and is only writable if bits FDCAN_CCCR.CCE and FDCAN_CCCR.INIT are set. The CAN bit time may be programmed in the range of 4 to 81 tq. The CAN time quantum may be programmed in the range of [1 ... 1024] FDCAN kernel clock periods.

$tq = (BRP + 1) \text{ FDCAN clock period } fdcan_tq_ck$

NTSEG1 is the sum of Prop_Seg and Phase_Seg1. NTSEG2 is Phase_Seg2. Therefore the length of the bit time is (programmed values) [NTSEG1 + NTSEG2 + 3] tq or (functional values) [Sync_Seg + Prop_Seg + Phase_Seg1 + Phase_Seg2] tq.

The information processing time (IPT) is zero, meaning the data for the next bit is available at the first clock edge after the sample point.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NSJW[6:0]							NBRP[8:0]								
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NTSEG1[7:0]							Res.	NTSEG2[6:0]							
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

Bits 31:25 **NSJW[6:0]**: Nominal (re)synchronization jump width

Should be smaller than NTSEG2, valid values are 0 to 127. The value used by the hardware is the one programmed, incremented by 1, i.e. $t_{SJW} = (NSJW + 1) \times tq$.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 24:16 **NBRP[8:0]**: Bitrate prescaler

Value by which the oscillator frequency is divided for generating the bit time quanta. The bit time is built up from a multiple of this quanta. Valid values are 0 to 511. The value used by the hardware is the one programmed, incremented by 1.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:8 **NTSEG1[7:0]**: Nominal time segment before sample point

Valid values are 0 to 255. The value used by the hardware is the one programmed, incremented by 1 ($t_{BS1} = (NTSEG1 + 1) \times tq$).

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 7 Reserved, must be kept at reset value.

Bits 6:0 **NTSEG2[6:0]**: Nominal time segment after sample point

Valid values are 0 to 127. The value used by the hardware is the one programmed, incremented by 1 ($t_{BS2} = (NTSEG2 + 1) \times tq$).

Note: With a CAN kernel clock of 8 MHz, the reset value of 0x00000A33 configures the FDCAN for a bitrate of 125 kbit/s.

59.5.8 FDCAN timestamp counter configuration register (FDCAN_TSCC)

Address offset: 0x0020

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TCP[3:0]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TSS[1:0]	
														rw	rw

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:16 **TCP[3:0]**: Timestamp counter prescaler

Configures the timestamp and timeout counters time unit in multiples of CAN bit times [1 ... 16].

The actual interpretation by the hardware of this value is such that one more than the value programmed here is used.

In CAN FD mode the internal timestamp counter TCP does not provide a constant time base due to the different CAN bit times between arbitration phase and data phase. Thus CAN FD requires an external counter for timestamp generation (TSS = 10).

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:2 Reserved, must be kept at reset value.

Bits 1:0 **TSS[1:0]**: Timestamp select

00: Timestamp counter value always 0x0000

01: Timestamp counter value incremented according to TCP

10: External timestamp counter from TIM3 value used (tim3_cnt[0:15])

11: Same as 00.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

59.5.9 FDCAN timestamp counter value register (FDCAN_TSCV)

Address offset: 0x0024

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TSC[15:0]															
rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W	rc_W

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **TSC[15:0]**: Timestamp counter

The internal/external timestamp counter value is captured on start of frame (both Rx and Tx). When FDCAN_TSCC.TSS = 01, the timestamp counter is incremented in multiples of CAN bit times [1 ... 16] depending on the configuration of FDCAN_TSCC.TCP. A wrap around sets interrupt flag FDCAN_IR.TSW. Write access resets the counter to 0. When FDCAN_TSCC.TSS = 10, TSC reflects the external timestamp counter value. A write access has no impact.

Note: A “wrap around” is a change of the timestamp counter value from non-0 to 0 not caused by write access to FDCAN_TSCV.

59.5.10 FDCAN timeout counter configuration register (FDCAN_TOCC)

Address offset: 0x0028

Reset value: 0xFFFF 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TOP[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TOS[1:0]		ETOC
													rw	rw	rw

Bits 31:16 **TOP[15:0]**: Timeout period

Start value of the timeout counter (down-counter). Configures the timeout period.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:3 Reserved, must be kept at reset value.

Bits 2:1 **TOS[1:0]**: Timeout select

When operating in Continuous mode, a write to FDCAN_TOCV presets the counter to the value configured by FDCAN_TOCC.TOP and continues down-counting. When the timeout counter is controlled by one of the FIFOs, an empty FIFO presets the counter to the value configured by FDCAN_TOCC.TOP. Down-counting is started when the first FIFO element is stored.

00: Continuous operation

01: Timeout controlled by Tx event FIFO

10: Timeout controlled by Rx FIFO 0

11: Timeout controlled by Rx FIFO 1

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 0 **ETOC**: Enable timeout counter

0: Timeout counter disabled

1: Timeout counter enabled

This is a write-protected bit, write access is possible only when the bit 1 (CCE) and bit 0 (INIT) of FDCAN_CCCR register are set to 1.

For more details see [Timeout counter](#).

59.5.11 FDCAN timeout counter value register (FDCAN_TOCV)

Address offset: 0x002C

Reset value: 0x0000 FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TOC[15:0]															
rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w	rc_w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **TOC[15:0]**: Timeout counter

The timeout counter is decremented in multiples of CAN bit times [1 ... 16] depending on the configuration of FDCAN_TSCC.TCP. When decremented to 0, interrupt flag FDCAN_IR.TOO is set and the timeout counter is stopped. Start and reset/restart conditions are configured via FDCAN_TOCC.TOS.

59.5.12 FDCAN error counter register (FDCAN_ECR)

Address offset: 0x0040

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CEL[7:0]							
								rc_r	rc_r	rc_r	rc_r	rc_r	rc_r	rc_r	rc_r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RP	REC[6:0]							TEC[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **CEL[7:0]**: CAN error logging

The counter is incremented each time when a CAN protocol error causes the transmit error counter or the receive error counter to be incremented. It is reset by read access to CEL. The counter stops at 0xFF; the next increment of TEC or REC sets interrupt flag FDCAN_IR.ELO.

Bit 15 **RP**: Receive error passive

0: The receive error counter is below the error passive level of 128

1: The receive error counter has reached the error passive level of 128

Bits 14:8 **REC[6:0]**: Receive error counter

Actual state of the receive error counter, values between 0 and 127.

Bits 7:0 **TEC[7:0]**: Transmit error counter

Actual state of the transmit error counter, values between 0 and 255.

When FDCAN_CCCR.ASM is set, the CAN protocol controller does not increment TEC and REC when a CAN protocol error is detected, but CEL is still incremented.

59.5.13 FDCAN protocol status register (FDCAN_PSR)

Address offset: 0x0044

Reset value: 0x0000 0707

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TDCV[6:0]						
									r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	PXE	REDL	RBR	RESI	DLEC[2:0]			BO	EW	EP	ACT[1:0]		LEC[2:0]		
	rc_r	rc_r	rc_r	rc_r	rs_r	rs_r	rs_r	r	r	r	r	r	rs_r	rs_r	rs_r

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **TDCV[6:0]**: Transmitter delay compensation value

Position of the secondary sample point, defined by the sum of the measured delay from FDCAN_TX to FDCAN_RX and FDCAN_TDCR.TDCO. The SSP position is, in the data phase, the number of minimum time quanta (mtq) between the start of the transmitted bit and the secondary sample point. Valid values are 0 to 127 mtq.

- Bit 15 Reserved, must be kept at reset value.
- Bit 14 **PXE**: Protocol exception event
0: No protocol exception event occurred since last read access
1: Protocol exception event occurred
- Bit 13 **REDL**: Received FDCAN message
This bit is set independent of acceptance filtering.
0: Since this bit was reset by the CPU, no FDCAN message has been received
1: Message in FDCAN format with EDL flag set has been received
Access type is RX: reset on read.
- Bit 12 **RBS**: BRS flag of last received FDCAN message
This bit is set together with REDL, independent of acceptance filtering.
0: Last received FDCAN message did not have its BRS flag set
1: Last received FDCAN message had its BRS flag set
Access type is RX: reset on read.
- Bit 11 **RESI**: ESI flag of last received FDCAN message
This bit is set together with REDL, independent of acceptance filtering.
0: Last received FDCAN message did not have its ESI flag set
1: Last received FDCAN message had its ESI flag set
Access type is RX: reset on read.
- Bits 10:8 **DLEC[2:0]**: Data last error code
Type of last error that occurred in the data phase of an FDCAN format frame with its BRS flag set. Coding is the same as for LEC. This field is cleared to 0 when an FDCAN format frame with its BRS flag set has been transferred (reception or transmission) without error.
Access type is RS: set on read.
- Bit 7 **BO**: Bus_Off status
0: The FDCAN is not Bus_Off
1: The FDCAN is in Bus_Off state
- Bit 6 **EW**: Warning status
0: Both error counters are below the Error_Warning limit of 96
1: At least one of error counter has reached the Error_Warning limit of 96
- Bit 5 **EP**: Error passive
0: The FDCAN is in the Error_Active state. It normally takes part in bus communication and sends an active error flag when an error has been detected
1: The FDCAN is in the Error_Passive state
- Bits 4:3 **ACT[1:0]**: Activity
Monitors the module CAN communication state.
00: Synchronizing: node is synchronizing on CAN communication
01: Idle: node is neither receiver nor transmitter
10: Receiver: node is operating as receiver
11: Transmitter: node is operating as transmitter

Bits 2:0 **LEC[2:0]**: Last error code

The LEC indicates the type of the last error to occur on the CAN bus. This field is cleared to 0 when a message has been transferred (reception or transmission) without error.

000: No error: No error occurred since LEC has been reset by successful reception or transmission.

001: Stuff error: More than 5 equal bits in a sequence have occurred in a part of a received message where this is not allowed.

010: Form error: A fixed format part of a received frame has the wrong format.

011: AckError: The message transmitted by the FDCAN was not acknowledged by another node.

100: Bit1Error: During the transmission of a message (with the exception of the arbitration field), the device wanted to send a recessive level (bit of logical value 1), but the monitored bus value was dominant.

101: Bit0Error: During the transmission of a message (or acknowledge bit, or active error flag, or overload flag), the device wanted to send a dominant level (data or identifier bit logical value 0), but the monitored bus value was recessive. During Bus_Off recovery this status is set each time a sequence of 11 recessive bits has been monitored. This enables the CPU to monitor the proceeding of the Bus_Off recovery sequence (indicating the bus is not stuck at dominant or continuously disturbed).

110: CRCError: The CRC check sum of a received message was incorrect. The CRC of an incoming message does not match with the CRC calculated from the received data.

111: NoChange: Any read access to the protocol status register re-initializes the LEC to '7. When the LEC shows the value 7, no CAN bus event was detected since the last CPU read access to the protocol status register.

Access type is RS: set on read.

Note: When a frame in FDCAN format has reached the data phase with BRS flag set, the next CAN event (error or valid frame) is shown in FLEC instead of LEC. An error in a fixed stuff bit of an FDCAN CRC sequence is shown as a Form error, not Stuff error

Note: The Bus_Off recovery sequence (see CAN Specification Rev. 2.0 or ISO11898-1) cannot be shortened by setting or resetting FDCAN_CCCR.INIT. If the device goes Bus_Off, it sets FDCAN_CCCR.INIT of its own, stopping all bus activities. Once FDCAN_CCCR.INIT has been cleared by the CPU, the device waits for 129 occurrences of bus Idle (129×11 consecutive recessive bits) before resuming normal operation. At the end of the Bus_Off recovery sequence, the error management counters are reset. During the waiting time after the reset of FDCAN_CCCR.INIT, each time a sequence of 11 recessive bits has been monitored, a Bit0 error code is written to FDCAN_PSR.LEC, enabling the CPU to readily check up whether the CAN bus is stuck at dominant or continuously disturbed and to monitor the Bus_Off recovery sequence. FDCAN_ECR.REC is used to count these sequences.

59.5.14 FDCAN transmitter delay compensation register (FDCAN_TDCR)

Address offset: 0x0048

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TDCO[6:0]							Res.	TDCF[6:0]						
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

Bits 31:15 Reserved, must be kept at reset value.

Bits 14:8 **TDCO[6:0]**: Transmitter delay compensation offset

Offset value defining the distance between the measured delay from FDCAN_TX to FDCAN_RX and the secondary sample point. Valid values are 0 to 127 mtq.

These are write-protected bits, which means that write access by the bits is possible only when the bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 7 Reserved, must be kept at reset value.

Bits 6:0 **TDCF[6:0]**: Transmitter delay compensation filter window length

Defines the minimum value for the SSP position, dominant edges on FDCAN_RX that would result in an earlier SSP position are ignored for transmitter delay measurements.

These are write-protected bits, which means that write access by the bits is possible only when the bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

59.5.15 FDCAN interrupt register (FDCAN_IR)

The flags are set when one of the listed conditions is detected (edge-sensitive). The flags remain set until the user clears them. A flag is cleared by writing a 1 to the corresponding bit position.

Writing a 0 has no effect. A hard reset clears the register. The configuration of IE controls whether an interrupt is generated. The configuration of ILS controls on which interrupt line an interrupt is signaled.

Address offset: 0x0050

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	ARA	PED	PEA	WDI	BO	EW	EP	ELO	Res.	Res.	DRX	TOO	MRAF	TSW
		rw	rw	rw	rw	rw	rw	rw	rw			rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TEFL	TEFF	TEFW	TEFN	TFE	TCF	TC	HPM	RF1L	RF1F	RF1W	RF1N	RF0L	RF0F	RF0W	RF0N
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **ARA**: Access to reserved address

0: No access to reserved address occurred

1: Access to reserved address occurred

Bit 28 **PED**: Protocol error in data phase (data bit time is used)

0: No protocol error in data phase

1: Protocol error in data phase detected (PSR.DLEC different from 0,7)

Bit 27 **PEA**: Protocol error in arbitration phase (nominal bit time is used)

0: No protocol error in arbitration phase

1: Protocol error in arbitration phase detected (PSR.LEC different from 0,7)

Bit 26 **WDI**: Watchdog interrupt

0: No message RAM watchdog event occurred

1: Message RAM watchdog event due to missing READY

- Bit 25 **BO**: Bus_Off status
 0: Bus_Off status unchanged
 1: Bus_Off status changed
- Bit 24 **EW**: Warning status
 0: Error_Warning status unchanged
 1: Error_Warning status changed
- Bit 23 **EP**: Error passive
 0: Error_Passive status unchanged
 1: Error_Passive status changed
- Bit 22 **ELO**: Error logging overflow
 0: CAN error logging counter did not overflow
 1: Overflow of CAN error logging counter occurred
- Bits 21:20 Reserved, must be kept at reset value.
- Bit 19 **DRX**: Message stored to dedicated Rx buffer
 The flag is set whenever a received message has been stored into a dedicated Rx buffer.
 0: No Rx buffer updated
 1: At least one received message stored into a Rx buffer
- Bit 18 **TOO**: Timeout occurred
 0: No timeout
 1: Timeout reached
- Bit 17 **MRAF**: Message RAM access failure
 The flag is set when the Rx handler
- l Has not completed acceptance filtering or storage of an accepted message until the arbitration field of the following message has been received. In this case acceptance filtering or message storage is aborted and the Rx handler starts processing of the following message.
 - l Was unable to write a message to the message RAM. In this case message storage is aborted. In both cases the FIFO put index is not updated or the New data flag for a dedicated Rx buffer is not set. The partly stored message is overwritten when the next message is stored to this location. The flag is also set when the Tx handler was not able to read a message from the message RAM in time. In this case message transmission is aborted. In case of a Tx handler access failure the FDCAN is switched into restricted operation mode (see [Restricted operation mode](#)). To leave restricted operation mode, the user has to reset FDCAN_CCCR.ASM.
- 0: No message RAM access failure occurred
 1: Message RAM access failure occurred
- Bit 16 **TSW**: Timestamp wraparound
 0: No timestamp counter wraparound
 1: Timestamp counter wraparound
- Bit 15 **TEFL**: Tx event FIFO element lost
 0: No Tx event FIFO element lost
 1: Tx event FIFO element lost, also set after write attempt to Tx event FIFO of size 0
- Bit 14 **TEFF**: Tx event FIFO full
 0: Tx event FIFO not full
 1: Tx event FIFO full
- Bit 13 **TEFW**: Tx event FIFO watermark reached
 0: Tx event FIFO fill level below watermark
 1: Tx event FIFO fill level reached watermark

- Bit 12 **TEFN**: Tx event FIFO new entry
0: Tx event FIFO unchanged
1: Tx handler wrote Tx event FIFO element
- Bit 11 **TFE**: Tx FIFO empty
0: Tx FIFO non-empty
1: Tx FIFO empty
- Bit 10 **TCF**: Transmission cancellation finished
0: No transmission cancellation finished
1: Transmission cancellation finished
- Bit 9 **TC**: Transmission completed
0: No transmission completed
1: Transmission completed
- Bit 8 **HPM**: High priority message
0: No high priority message received
1: High priority message received
- Bit 7 **RF1L**: Rx FIFO 1 message lost
0: No Rx FIFO 1 message lost
1: Rx FIFO 1 message lost, also set after write attempt to Rx FIFO 1 of size 0
- Bit 6 **RF1F**: Rx FIFO 1 full
0: Rx FIFO 1 not full
1: Rx FIFO 1 full
- Bit 5 **RF1W**: Rx FIFO 1 watermark reached
0: Rx FIFO 1 fill level below watermark
1: Rx FIFO 1 fill level reached watermark
- Bit 4 **RF1N**: Rx FIFO 1 new message
0: No new message written to Rx FIFO 1
1: New message written to Rx FIFO 1
- Bit 3 **RF0L**: Rx FIFO 0 message lost
0: No Rx FIFO 0 message lost
1: Rx FIFO 0 message lost, also set after write attempt to Rx FIFO 0 of size 0
- Bit 2 **RF0F**: Rx FIFO 0 full
0: Rx FIFO 0 not full
1: Rx FIFO 0 full
- Bit 1 **RF0W**: Rx FIFO 0 watermark reached
0: Rx FIFO 0 fill level below watermark
1: Rx FIFO 0 fill level reached watermark
- Bit 0 **RF0N**: Rx FIFO 0 New message
0: No new message written to Rx FIFO 0
1: New message written to Rx FIFO 0

59.5.16 FDCAN interrupt enable register (FDCAN_IE)

The settings in the interrupt enable register determine which status changes in the interrupt register is signaled on an interrupt line.

Address offset: 0x0054

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	ARAE	PEDE	PEAE	WDIE	BOE	EWE	EPE	ELOE	Res.	Res.	DRXE	TOOE	MRAFE	TSWE
		rw	rw	rw	rw	rw	rw	rw	rw			rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TEFLE	TEFFE	TEFWE	TEFNE	TFEE	TCFE	TCE	HPME	RF1LE	RF1FE	RF1WE	RF1NE	RF0LE	RF0FE	RF0WE	RF0NE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **ARAE**: Access to Reserved address enable

Bit 28 **PEDE**: Protocol error in data phase enable

Bit 27 **PEAE**: Protocol error in Arbitration phase enable

Bit 26 **WDIE**: Watchdog interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bit 25 **BOE**: Bus_Off status

0: Interrupt disabled

1: Interrupt enabled

Bit 24 **EWE**: Warning status interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bit 23 **EPE**: Error passive interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bit 22 **ELOE**: Error logging overflow interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bits 21:20 Reserved, must be kept at reset value.

Bit 19 **DRXE**: Message stored to dedicated Rx buffer interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bit 18 **TOOE**: Timeout occurred interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bit 17 **MRAFE**: Message RAM access failure interrupt enable

0: Interrupt disabled

1: Interrupt enabled

- Bit 16 **TSWE**: Timestamp wraparound interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 15 **TEFLE**: Tx event FIFO element lost interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 14 **TEFFE**: Tx event FIFO full interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 13 **TEFWE**: Tx event FIFO watermark reached interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 12 **TEFNE**: Tx event FIFO new entry interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 11 **TFEE**: Tx FIFO empty interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 10 **TCFE**: Transmission cancellation finished interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 9 **TCE**: Transmission completed interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 8 **HPME**: High priority message interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 7 **RF1LE**: Rx FIFO 1 message lost interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 6 **RF1FE**: Rx FIFO 1 full interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 5 **RF1WE**: Rx FIFO 1 watermark reached interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 4 **RF1NE**: Rx FIFO 1 new message interrupt enable
0: Interrupt disabled
1: Interrupt enabled
- Bit 3 **RF0LE**: Rx FIFO 0 message lost interrupt enable
0: Interrupt disabled
1: Interrupt enabled

Bit 2 **RF0FE**: Rx FIFO 0 full interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bit 1 **RF0WE**: Rx FIFO 0 watermark reached interrupt enable

0: Interrupt disabled

1: Interrupt enabled

Bit 0 **RF0NE**: Rx FIFO 0 new message interrupt enable

0: Interrupt disabled

1: Interrupt enabled

59.5.17 FDCAN interrupt line select register (FDCAN_ILS)

This register assigns an interrupt generated by a specific interrupt flag from the interrupt register to one of the two module interrupt lines. For interrupt generation the respective interrupt line must be enabled via FDCAN_ILE.EINT0 and FDCAN_ILE.EINT1.

Address offset: 0x0058

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	ARAL	PEDL	PEAL	WDIL	BOL	EWL	EPL	ELOL	Res.	Res.	DRXL	TOOL	MRAFL	TSWL
		rW	rW	rW	rW	rW	rW	rW	rW			rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TEFLL	TEFFL	TEFWL	TEFNL	TFEL	TCFL	TCL	HPML	RF1LL	RF1FL	RF1WL	RF1NL	RF0LL	RF0FL	RF0WL	RF0NL
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **ARAL**: Access to reserved address line

Bit 28 **PEDL**: Protocol error in data phase line

Bit 27 **PEAL**: Protocol error in arbitration phase line

Bit 26 **WDIL**: Watchdog interrupt line

Bit 25 **BOL**: Bus_Off status

Bit 24 **EWL**: Warning status interrupt line

Bit 23 **EPL**: Error passive interrupt line

Bit 22 **ELOL**: Error logging overflow interrupt line

Bits 21:20 Reserved, must be kept at reset value.

Bit 19 **DRXL**: Message stored to dedicated Rx buffer interrupt line

Bit 18 **TOOL**: Timeout occurred interrupt line

Bit 17 **MRAFL**: Message RAM access failure interrupt line

Bit 16 **TSWL**: Timestamp wraparound interrupt line

Bit 15 **TEFLL**: Tx event FIFO element Lost interrupt line

Bit 14 **TEFFL**: Tx event FIFO full interrupt line

Bit 13 **TEFWL**: Tx event FIFO watermark reached interrupt line

Bit 12 **TEFNL**: Tx event FIFO new entry interrupt line

Bit 11 **TFEL**: Tx FIFO empty interrupt line

Bit 10 **TCFL**: Transmission cancellation finished interrupt line

Bit 9 **TCL**: Transmission completed interrupt line

Bit 8 **HPML**: High priority message interrupt line

Bit 7 **RF1LL**: Rx FIFO 1 message lost interrupt line

Bit 6 **RF1FL**: Rx FIFO 1 full interrupt line

Bit 5 **RF1WL**: Rx FIFO 1 watermark reached interrupt line

Bit 4 **RF1NL**: Rx FIFO 1 new message interrupt line

Bit 3 **RF0LL**: Rx FIFO 0 message lost interrupt line

Bit 2 **RF0FL**: Rx FIFO 0 full interrupt line

Bit 1 **RF0WL**: Rx FIFO 0 watermark reached interrupt line

Bit 0 **RF0NL**: Rx FIFO 0 new message interrupt line

59.5.18 FDCAN interrupt line enable register (FDCAN_ILE)

Each of the two interrupt lines to the CPU can be enabled/disabled separately by programming bits EINT0 and EINT1.

Address offset: 0x005C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EINT1	EINT0
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **EINT1**: Enable interrupt line 1

0: Interrupt line fdcan_intr1_it disabled

1: Interrupt line fdcan_intr1_it enabled

Bit 0 **EINT0**: Enable interrupt line 0

0: Interrupt line fdcan_intr0_it disabled

1: Interrupt line fdcan_intr0_it enabled

59.5.19 FDCAN global filter configuration register (FDCAN_GFC)

Global settings for message ID filtering. The global filter configuration register controls the filter path for standard and extended messages as described in [Figure 781](#) and [Figure 782](#).

Address offset: 0x0080

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ANFS[1:0]		ANFE[1:0]		RRFS	RRFE
										rw	rw	rw	rw	rw	rw

Bits 31:6 Reserved, must be kept at reset value.

Bits 5:4 **ANFS[1:0]**: Accept non-matching frames standard

Defines how received messages with 11-bit ID that do not match any element of the filter list are treated.

00: Accept in Rx FIFO 0

01: Accept in Rx FIFO 1

10: Reject

11: Reject

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 3:2 **ANFE[1:0]**: Accept non-matching frames extended

Defines how received messages with 29-bit ID that do not match any element of the filter list are treated.

00: Accept in Rx FIFO 0

01: Accept in Rx FIFO 1

10: Reject

11: Reject

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 1 **RRFS**: Reject remote frames standard

0: Filter remote frames with 11-bit standard ID

1: Reject all remote frames with 11-bit standard ID

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 0 **RRFE**: Reject remote frames extended

0: Filter remote frames with 29-bit standard ID

1: Reject all remote frames with 29-bit standard ID

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

59.5.20 FDCAN standard ID filter configuration register (FDCAN_SIDFC)

Settings for 11-bit standard message ID filtering. The standard ID filter configuration register controls the filter path for standard messages as described in [Figure 781](#).

Address offset: 0x0084

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LSS[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLSSA[13:0]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **LSS[7:0]**: List size standard

0: No standard message ID filter

1-128: Number of standard message ID filter elements

>128: Values greater than 128 are interpreted as 128.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:2 **FLSSA[13:0]**: Filter list standard start address

Start address of standard message ID filter list (32-bit word address, see [Table 520](#)). These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 1:0 Reserved, must be kept at reset value.

59.5.21 FDCAN extended ID filter configuration register (FDCAN_XIDFC)

Settings for 29-bit extended message ID filtering. The FDCAN extended ID filter configuration register controls the filter path for standard messages as described in [Figure 782](#).

Address offset: 0x0088

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LSE[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLESA[13:0]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **LSE[7:0]**: List size extended

0: No extended message ID filter

1-128: Number of extended ID filter elements

>128: Values greater than 128 are interpreted as 128.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:2 **FLESA[13:0]**: Filter list extended start address

Start address of extended message ID filter list (32-bit word address, see [Table 522: Extended message ID filter element](#)).

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 1:0 Reserved, must be kept at reset value.

59.5.22 FDCAN extended ID and mask register (FDCAN_XIDAM)

Address offset: 0x0090

Reset value: 0x1FFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	EIDM[28:16]												
			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EIDM[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bits 28:0 **EIDM[28:0]**: Extended ID Mask

For acceptance filtering of extended frames the extended ID AND Mask is AND-ed with the message ID of a received frame. Intended for masking of 29-bit IDs in SAE J1939. With the reset value of all bits set to 1 the mask is not active.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

59.5.23 FDCAN high priority message status register (FDCAN_HPMS)

This register is updated every time a message ID filter element configured to generate a priority event match. This can be used to monitor the status of incoming high priority messages and to enable fast access to these messages.

Address offset: 0x0094

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLST	FIDX[6:0]							MSI[1:0]		BIDX[5:0]					
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **FLST**: Filter list

Indicates the filter list of the matching filter element.

0: Standard filter list

1: Extended filter list

Bits 14:8 **FIDX[6:0]**: Filter index

Index of matching filter element. Range is 0 to FDCAN_SIDFC[LSS] - 1 or FDCAN_XIDFC[LSE] - 1.

Bits 7:6 **MSI[1:0]**: Message storage indicator

00: No FIFO selected

01: FIFO overrun

10: Message stored in FIFO 0

11: Message stored in FIFO 1

Bits 5:0 **BIDX[5:0]**: Buffer index

Index of Rx FIFO element to which the message was stored. Only valid when MSI[1] = 1.

59.5.24 FDCAN new data 1 register (FDCAN_NDAT1)

Address offset: 0x0098

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ND31	ND30	ND29	ND28	ND27	ND26	ND25	ND24	ND23	ND22	ND21	ND20	ND19	ND18	ND17	ND16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ND15	ND14	ND13	ND12	ND11	ND10	ND9	ND8	ND7	ND6	ND5	ND4	ND3	ND2	ND1	ND0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ND[31:0]**: New data[31:0]

The register holds the new data flags of Rx buffers 0 to 31. The flags are set when the respective Rx buffer has been updated from a received frame. The flags remain set until the user clears them. A flag is cleared by writing a 1 to the corresponding bit position. Writing a 0 has no effect. A hard reset clears the register.

0: Rx buffer not updated

1: Rx buffer updated from new message

59.5.25 FDCAN new data 2 register (FDCAN_NDAT2)

Address offset: 0x009C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ND63	ND62	ND61	ND60	ND59	ND58	ND57	ND56	ND55	ND54	ND53	ND52	ND51	ND50	ND49	ND48
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ND47	ND46	ND45	ND44	ND43	ND42	ND41	ND40	ND39	ND38	ND37	ND36	ND35	ND34	ND33	ND32
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **ND[63:32]**: New data[63:32]

The register holds the new data flags of Rx buffers 32 to 63. The flags are set when the respective Rx buffer has been updated from a received frame. The flags remain set until the user clears them. A flag is cleared by writing a 1 to the corresponding bit position. Writing a 0 has no effect. A hard reset clears the register.

0: Rx buffer not updated

1: Rx buffer updated from new message

59.5.26 FDCAN Rx FIFO 0 configuration register (FDCAN_RXF0C)

Address offset: 0x00A0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
F0OM	F0WM[6:0]							Res.	F0S[6:0]						
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
F0SA[13:0]													Res.	Res.	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 **F0OM**: FIFO 0 operation mode

FIFO 0 can be operated in blocking or in overwrite mode.

0: FIFO 0 blocking mode

1: FIFO 0 overwrite mode

Bits 30:24 **F0WM[6:0]**: FIFO 0 watermark

0: Watermark interrupt disabled

1-64: Level for Rx FIFO 0 watermark interrupt (FDCAN_IR.RF0W)

>64: Watermark interrupt disabled

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 23 Reserved, must be kept at reset value.

Bits 22:16 **F0S[6:0]**: Rx FIFO 0 size

0: No Rx FIFO 0

1-64: Number of Rx FIFO 0 elements

>64: Values greater than 64 are interpreted as 64

The Rx FIFO 0 elements are indexed from 0 to F0S-1.

Bits 15:2 **F0SA[13:0]**: Rx FIFO 0 start address

Start address of Rx FIFO 0 in message RAM (32-bit word address, see [Figure 780](#)).

Bits 1:0 Reserved, must be kept at reset value.

59.5.27 FDCAN Rx FIFO 0 status register (FDCAN_RXF0S)

Address offset: 0x00A4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	RF0L	F0F	Res.	Res.	F0PI[5:0]					
						rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	F0GI[5:0]						Res.	F0FL[6:0]						
		rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

Bits 31:26 Reserved, must be kept at reset value.

Bit 25 **RF0L**: Rx FIFO 0 message lost

This bit is a copy of interrupt flag FDCAN_IR.RF0L. When FDCAN_IR.RF0L is reset, this bit is also reset.

0: No Rx FIFO 0 message lost

1: Rx FIFO 0 message lost, also set after write attempt to Rx FIFO 0 of size 0

Bit 24 **F0F**: Rx FIFO 0 full

0: Rx FIFO 0 not full

1: Rx FIFO 0 full

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:16 **F0PI[5:0]**: Rx FIFO 0 put index

Rx FIFO 0 write index pointer, range 0 to 63.

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:8 **F0GI[5:0]**: Rx FIFO 0 get index

Rx FIFO 0 read index pointer, range 0 to 63.

Bit 7 Reserved, must be kept at reset value.

Number of elements stored in Rx FIFO 0, range 0 to 64.

Address offset: 0x00A8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FOA[5:0]					
										rw	rw	rw	rw	rw	rw

Bits 5:0 **F0AI[5:0]**: Rx FIFO 0 acknowledge index

After the user has read a message or a sequence of messages from Rx FIFO 0 it has to write the buffer index of the last element read from Rx FIFO 0 to F0AI. This sets the Rx FIFO 0 get index FDCAN_RXF0S.F0GI to F0AI + 1 and update the FIFO 0 fill level FDCAN_RXF0S.F0FL.

Address offset: 0x00AC

Reset value: 0x0000 0000

[illegible]

Bits 15:2 **RBSA[13:0]**: Rx buffer start address

Configures the start address of the Rx buffers section in the message RAM (32-bit word address). Also used to reference debug messages A, B, C.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 1:0 Reserved, must be kept at reset value.

59.5.30 FDCAN Rx FIFO 1 configuration register (FDCAN_RXF1C)

Address offset: 0x00B0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
F1OM	F1WM[6:0]							Res.	F1S[6:0]						
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
F1SA[13:0]													Res.	Res.	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 **F1OM**: FIFO 1 operation mode

FIFO 1 can be operated in blocking or in overwrite mode.

0: FIFO 1 blocking mode

1: FIFO 1 overwrite mode

Bits 30:24 **F1WM[6:0]**: Rx FIFO 1 watermark

0: Watermark interrupt disabled

1-64: Level for Rx FIFO 1 watermark interrupt (FDCAN_IR.RF1W)

>64: Watermark interrupt disabled.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 23 Reserved, must be kept at reset value.

Bits 22:16 **F1S[6:0]**: Rx FIFO 1 size

0: No Rx FIFO 1

1-64: Number of Rx FIFO 1 elements

>64: Values greater than 64 are interpreted as 64

The Rx FIFO 1 elements are indexed from 0 to F1S - 1.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:2 **F1SA[13:0]**: Rx FIFO 1 start addressstart address of Rx FIFO 1 in message RAM (32-bit word address, see [Figure 780](#)).

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 1:0 Reserved, must be kept at reset value.

59.5.31 FDCAN Rx FIFO 1 status register (FDCAN_RXF1S)

Address offset: 0x00B4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMS[1:0]		Res.	Res.	Res.	Res.	RF1L	F1F	Res.	Res.	F1PI[5:0]					
r	r					r	r			r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	F1GI[5:0]						Res.	F1FL[6:0]						
		r	r	r	r	r	r		r	r	r	r	r	r	r

Bits 31:30 **DMS[1:0]**: Debug message status

00: Idle state, wait for reception of debug messages

01: Debug message A received

10: Debug messages A, B received

11: Debug messages A, B, C received

Bits 29:26 Reserved, must be kept at reset value.

Bit 25 **RF1L**: Rx FIFO 1 message lost

This bit is a copy of interrupt flag FDCAN_IR.RF1L. When FDCAN_IR.RF1L is reset, this bit is also reset.

0: No Rx FIFO 1 message lost

1: Rx FIFO 1 message lost, also set after write attempt to Rx FIFO 1 of size 0.

Bit 24 **F1F**: Rx FIFO 1 full

0: Rx FIFO 1 not full

1: Rx FIFO 1 full

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:16 **F1PI[5:0]**: Rx FIFO 1 put index

Rx FIFO 1 write index pointer, range 0 to 63.

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:8 **F1GI[5:0]**: Rx FIFO 1 get index

Rx FIFO 1 read index pointer, range 0 to 63.

Bit 7 Reserved, must be kept at reset value.

Bits 6:0 **F1FL[6:0]**: Rx FIFO 1 fill level

Number of elements stored in Rx FIFO 1, range 0 to 64

59.5.32 FDCAN Rx FIFO 1 acknowledge register (FDCAN_RXF1A)

Address offset: 0x00B8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	F1AI[5:0]					
										rw	rw	rw	rw	rw	rw

Bits 31:6 Reserved, must be kept at reset value.

Bits 5:0 **F1AI[5:0]**: Rx FIFO 1 acknowledge index

After the user has read a message or a sequence of messages from Rx FIFO 1 it has to write the buffer index of the last element read from Rx FIFO 1 to F1AI. This sets the Rx FIFO 1 get index FDCAN_RXF1S.F1GI. to F1AI + 1 and update the FIFO 1 fill level FDCAN_RXF1S.F1FL.

59.5.33 FDCAN Rx buffer element size configuration register (FDCAN_RXESC)

Configures the number of data bytes belonging to an Rx buffer / Rx FIFO element. Data field sizes higher than 8 bytes are intended for CAN FD operation only.

Address offset: 0x00BC

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	RBDS[2:0]			Res.	F1DS[2:0]			Res.	F0DS[2:0]		
					r	r	r		r	r	r		r	r	r

Bits 31:11 Reserved, must be kept at reset value.

Bits 10:8 **RBDS[2:0]**: Rx buffer data field size

000: 8-byte data field
 001: 12-byte data field
 010: 16-byte data field
 011: 20-byte data field
 100: 24-byte data field
 101: 32-byte data field
 110: 48-byte data field
 111: 64-byte data field

Bit 7 Reserved, must be kept at reset value.

Bits 6:4 **F1DS[2:0]**: Rx FIFO 0 data field size

000: 8-byte data field
 001: 12-byte data field
 010: 16-byte data field
 011: 20-byte data field
 100: 24-byte data field
 101: 32-byte data field
 110: 48-byte data field
 111: 64-byte data field

Bit 3 Reserved, must be kept at reset value.

Bits 2:0 **F0DS[2:0]**: Rx FIFO 1 data field size

000: 8-byte data field
 001: 12-byte data field
 010: 16-byte data field
 011: 20-byte data field
 100: 24-byte data field
 101: 32-byte data field
 110: 48-byte data field
 111: 64-byte data field

59.5.34 FDCAN Tx buffer configuration register (FDCAN_TXBC)

Address offset: 0x00C0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	TFQM	TFQS[5:0]						Res.	Res.	NDTB[5:0]					
	rw	rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TBSA[13:0]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 Reserved, must be kept at reset value.

Bit 30 **TFQM**: Tx FIFO/queue mode

0: Tx FIFO operation
 1: Tx queue operation.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 29:24 **TFQS[5:0]**: Transmit FIFO/queue size

0: No Tx FIFO/queue
 1-32: Number of Tx buffers used for Tx FIFO/queue
 >32: Values greater than 32 are interpreted as 32.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:16 **NDTB[5:0]**: Number of dedicated transmit buffers

0: No dedicated Tx buffers

1-32: Number of dedicated Tx buffers

>32: Values greater than 32 are interpreted as 32.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:2 **TBSA[13:0]**: Tx buffers start address

Start address of Tx buffers section in message RAM (32-bit word address, see [Figure 780](#)).

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 1:0 Reserved, must be kept at reset value.

Note: *The sum of TFQS and NDTB cannot be larger than 32. There is no check for erroneous configurations. The Tx buffers section in the message RAM starts with the dedicated Tx buffers.*

59.5.35 FDCAN Tx FIFO/queue status register (FDCAN_TXFQS)

The Tx FIFO/queue status is related to the pending Tx requests listed in register FDCAN_TXBRP. Therefore the effect of add/cancellation requests may be delayed due to a running Tx scan (FDCAN_TXBRP not yet updated).

Address offset: 0x00C4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TFQF	TFQPI[4:0]				
										r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	TFGI[4:0]					Res.	Res.	TFFL[5:0]					
			r	r	r	r	r			r	r	r	r	r	r

Bits 31:22 Reserved, must be kept at reset value.

Bit 21 **TFQF**: Tx FIFO/queue full

0 Tx FIFO/queue not full

1 Tx FIFO/queue full

Bits 20:16 **TFQPI[4:0]**: Tx FIFO/queue put index

Tx FIFO/queue write index pointer, range 0 to 31

Bits 15:13 Reserved, must be kept at reset value.

Bits 12:8 **TFGI[4:0]**:

Tx FIFO get index.

Tx FIFO read index pointer, range 0 to 31. Read as 0 when Tx queue operation is configured (FDCAN_TXBC.TFQM = 1)

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:0 **TFFL[5:0]**: Tx FIFO free level

Number of consecutive free Tx FIFO elements starting from TFGI, range 0 to 32. Read as 0 when Tx queue operation is configured (FDCAN_TXBC.TFQM = 1).

Note: *In case of mixed configurations where dedicated Tx buffers are combined with a Tx FIFO or a Tx queue, the put and get index indicate the number of the Tx buffer starting with the first dedicated Tx buffers. For example: For a configuration of 12 dedicated Tx buffers and a Tx FIFO of 20 buffers a put index of 15 points to the fourth buffer of the Tx FIFO.*

59.5.36 FDCAN Tx buffer element size configuration register (FDCAN_TXESC)

Configures the number of data bytes belonging to a Tx buffer element. Data field sizes >8 bytes are intended for CAN FD operation only.

Address offset: 0x00C8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TBDS[2:0]		
													r	r	r

Bits 31:3 Reserved, must be kept at reset value.

Bits 2:0 **TBDS[2:0]**: Tx buffer data Field size:

000: 8-byte data field

001: 12-byte data field

010: 16-byte data field

011: 20-byte data field

100: 24-byte data field

101: 32-byte data field

110: 48-byte data field

111: 64-byte data field

59.5.37 FDCAN Tx buffer request pending register (FDCAN_TXBRP)

Address offset: 0x00CC

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TRP[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TRP[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **TRP[31:0]**:Transmission request pending

Each Tx buffer has its own transmission request pending bit. The bits are set via register FDCAN_TXBAR. The bits are reset after a requested transmission has completed or has been canceled via register FDCAN_TXBCR.

FDCAN_TXBRP bits are set only for those Tx buffers configured via FDCAN_TXBC. After an FDCAN_TXBRP bit has been set, a Tx scan (see [Filtering for Debug messages](#)) is started to check for the pending Tx request with the highest priority (Tx buffer with lowest message ID).

A cancellation request resets the corresponding transmission request pending bit of register FDCAN_TXBRP. In case a transmission has already been started when a cancellation is requested, this is done at the end of the transmission, regardless whether the transmission was successful or not. The cancellation request bits are reset directly after the corresponding FDCAN_TXBRP bit has been reset.

After a cancellation has been requested, a finished cancellation is signaled via FDCAN_TXBCF after successful transmission together with the corresponding FDCAN_TXBTO bit when the transmission has not yet been started at the point of cancellation when the transmission has been aborted due to lost arbitration when an error occurred during frame transmission

In DAR mode all transmissions are automatically canceled if they are not successful. The corresponding FDCAN_TXBCF bit is set for all unsuccessful transmissions.

0: No transmission request pending

1: Transmission request pending

Note: *FDCAN_TXBRP bits set while a Tx scan is in progress are not considered during this particular Tx scan. In case a cancellation is requested for such a Tx buffer, this add request is canceled immediately, the corresponding FDCAN_TXBRP bit is reset.*

59.5.38 FDCAN Tx buffer add request register (FDCAN_TXBAR)

Address offset: 0x00D0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AR[31:0]**:Add request

Each Tx buffer has its own add request bit. Writing a 1 sets the corresponding add request bit; writing a 0 has no impact. This enables the user to set transmission requests for multiple Tx buffers with one write to FDCAN_TXBAR. FDCAN_TXBAR bits are set only for those Tx buffers configured via FDCAN_TXBC. When no Tx scan is running, the bits are reset immediately, else the bits remain set until the Tx scan process has completed.

0: No transmission request added

1: Transmission requested added.

Note: *If an add request is applied for a Tx buffer with pending transmission request (corresponding FDCAN_TXBRP bit already set), the request is ignored.*

59.5.39 FDCAN Tx buffer cancellation request register (FDCAN_TXBCR)

Address offset: 0x00D4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **CR[31:0]**: Cancellation request

Each Tx buffer has its own cancellation request bit. Writing a 1 sets the corresponding cancellation request bit; writing a 0 has no impact.

This enables the user to set cancellation requests for multiple Tx buffers with one write to FDCAN_TXBCR. FDCAN_TXBCR bits are set only for those Tx buffers configured via FDCAN_TXBC. The bits remain set until the corresponding FDCAN_TXBRP bit is reset.

0: No cancellation pending

1: Cancellation pending

59.5.40 FDCAN Tx buffer transmission occurred register (FDCAN_TXBTO)

Address offset: 0x00D8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TO[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TO[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **TO[31:0]**: Transmission occurred

Each Tx buffer has its own transmission occurred bit. The bits are set when the corresponding FDCAN_TXBRP bit is cleared after a successful transmission. The bits are reset when a new transmission is requested by writing a 1 to the corresponding bit of register FDCAN_TXBAR.

0: No transmission occurred

1: Transmission occurred

59.5.41 FDCAN Tx buffer cancellation finished register (FDCAN_TXBCF)

Address offset: 0x00DC

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CF[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CF[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **CF[31:0]**: Cancellation finished

Each Tx buffer has its own cancellation finished bit. The bits are set when the corresponding FDCAN_TXBRP bit is cleared after a cancellation was requested via FDCAN_TXBCR. In case the corresponding FDCAN_TXBRP bit was not set at the point of cancellation, CF is set immediately. The bits are reset when a new transmission is requested by writing a 1 to the corresponding bit of register FDCAN_TXBAR.

0: No transmit buffer cancellation

1: Transmit buffer cancellation finished

59.5.42 FDCAN Tx buffer transmission interrupt enable register (FDCAN_TXBTIE)

Address offset: 0x00E0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIE[31:16]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TIE[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **TIE[31:0]**: Transmission interrupt enable

Each Tx buffer has its own transmission interrupt enable bit.

0: Transmission interrupt disabled

1: Transmission interrupt enable

59.5.43 FDCAN Tx buffer cancellation finished interrupt enable register (FDCAN_TXBCIE)

Address offset: 0x00E4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CFIE[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CFIE[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **CFIE[31:0]**: Cancellation finished interrupt enable

Each Tx buffer has its own cancellation finished interrupt enable bit.

0: Cancellation finished interrupt disabled

1: Cancellation finished interrupt enabled

59.5.44 FDCAN Tx event FIFO configuration register (FDCAN_TXEFC)

Address offset: 0x00F0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	EFWM[5:0]						Res.	Res.	EFS[5:0]					
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EFSA[13:0]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:30 Reserved, must be kept at reset value.

Bits 29:24 **EFWM[5:0]**: Event FIFO watermark

0: Watermark interrupt disabled

1-32: Level for Tx event FIFO watermark interrupt (FDCAN_IR.TEFW)

>32: Watermark interrupt disabled

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:16 **EFS[5:0]**: Event FIFO size.

0: Tx event FIFO disabled

1-32: Number of Tx event FIFO elements

>32: Values greater than 32 are interpreted as 32

The Tx event FIFO elements are indexed from 0 to EFS-1.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:2 **EFSA[13:0]**: Event FIFO start address

Start address of Tx event FIFO in message RAM (32-bit word address, see [Figure 780](#)).

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 1:0 Reserved, must be kept at reset value.

59.5.45 FDCAN Tx event FIFO status register (FDCAN_TXEFS)

Address offset: 0x00F4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TEFL	EFF	Res.	Res.	Res.	EFPI[4:0]				
						r	r				r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	EFGI[4:0]					Res.	Res.	EFFL[5:0]					
			r	r	r	r	r			r	r	r	r	r	r

Bits 31:26 Reserved, must be kept at reset value.

Bit 25 **TEFL**: Tx event FIFO element lost

This bit is a copy of interrupt flag FDCAN_IR.TEFL. When FDCAN_IR.TEFL is reset, this bit is also reset.

0 No Tx event FIFO element lost

1 Tx event FIFO element lost, also set after write attempt to Tx event FIFO of size 0.

Bit 24 **EFF**: Event FIFO full

0: Tx event FIFO not full

1: Tx event FIFO full

Bits 23:21 Reserved, must be kept at reset value.

Bits 20:16 **EFPI[4:0]**: Event FIFO put index

Tx event FIFO write index pointer, range 0 to 31.

Bits 15:13 Reserved, must be kept at reset value.

Bits 12:8 **EFGI[4:0]**: Event FIFO get index

Tx event FIFO read index pointer, range 0 to 31.

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:0 **EFFL[5:0]**: Event FIFO fill level

Number of elements stored in Tx event FIFO, range 0 to 31.

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EFA[4:0]				
											rw	rw	rw	rw	rw

After the user has read an element or a sequence of elements from the Tx event FIFO, it must write the index of the last element read from Tx event FIFO to EFAI. This sets the Tx event FIFO get index FDCAN_TXEFS.EFGI to EFAI + 1 and update the FIFO 0 fill level FDCAN_TXEFS.EFFL.

[illegible]

Table 526. FDCAN register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0024	FDCAN_TSCV																	TSC[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0028	FDCAN_TOCC	TOP[15:0]																Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TOS [1:0]	ETOC
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1														0	0	0
0x002C	FDCAN_TOCV																	TOC[15:0]															
	Reset value																	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x0030- 0x003F	Reserved	Reserved																															
0x0040	FDCAN_ECR	Res	Res	Res	Res	Res	Res	Res	Res	CEL[7:0]							RP	REC[6:0]						TEC[7:0]									
	Reset value										0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0044	FDCAN_PSR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PXE	REDL	RBR	RESI	DLEC [2:0]			BO	EW	EP	ACT [1:0]		LEC[2:0]		
	Reset value																		0	0	0	0	1	1	1	0	0	0	0	0	1	1	1
0x0048	FDCAN_TDCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TDCO[6:0]						Res	TDCF[6:0]							
	Reset value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0050	FDCAN_IR	Res	Res	ARA	PED	PEA	WDI	BO	EW	EP	ELO	Res	Res	DRX	TOO	MRAF	TSW	TEFL	TEFF	TEFW	TEFN	TFE	TCF	TC	HPM	RF1L	RF1F	RF1W	RF1N	RF0L	RF0F	RF0W	RF0N
	Reset value			0	0	0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0054	FDCAN_IE	Res	Res	ARAE	PEDE	PEAE	WDIE	BOE	EWE	EPE	ELOE	Res	Res	DRXE	TOOE	MRAFE	TSWE	TEFLE	TEFFE	TEFWE	TEFNE	TFEE	TCFE	TCE	HPME	RF1LE	RF1FE	RF1WE	RF1NE	RF0LE	RF0FE	RF0WE	RF0NE
	Reset value			0	0	0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0058	FDCAN_ILS	Res	Res	ARAL	PEDL	PEAL	WDIL	BOL	EWL	EPL	ELOL	Res	Res	DRXL	TOOL	MRAFL	TSWL	TEFLL	TEFFL	TEFWL	TEFNL	TFEL	TCFL	TCL	HPML	RF1LL	RF1FL	RF1WL	RF1NL	RF0LL	RF0FL	RF0WL	RF0NL
	Reset value			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x005C	FDCAN_ILE	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	EINT1	EINT0
	Reset value																														0	0	
0x006- 0x007F	Reserved	Reserved																															
0x0080	FDCAN_GFC	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ANFS [1:0]	ANFE [1:0]		EINT1	EINT0
	Reset value																											0	0	0	0	0	0
0x0084	FDCAN_SIDFC	Res	Res	Res	Res	Res	Res	Res	Res	LSS[7:0]							FLSSA[13:0]															Res	Res
	Reset value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x0088	FDCAN_XIDFC	Res	Res	Res	Res	Res	Res	Res	Res	LSE[7:0]							FLESA[13:0]															Res	Res
	Reset value									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Table 526. FDCAN register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0090	FDCAN_XIDAM	Res	Res	Res	EIDM[28:0]																												
	Reset value				1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x0094	FDCAN_HPMS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	FLST	FIDX[6:0]						MISI[1:0]		MIDX[5:0]						
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0098	FDCAN_NDAT1	ND31	ND30	ND29	ND28	ND27	ND26	ND25	ND24	ND23	ND22	ND21	ND20	ND19	ND18	ND17	ND16	ND15	ND14	ND13	ND12	ND11	ND10	ND9	ND8	ND7	ND6	ND5	ND4	ND3	ND2	ND1	ND0
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x009C	FDCAN_NDAT2	ND63	ND62	ND61	ND60	ND59	ND58	ND57	ND56	ND55	ND54	ND53	ND52	ND51	ND50	ND49	ND48	ND47	ND46	ND45	ND44	ND43	ND42	ND41	ND40	ND39	ND38	ND37	ND36	ND35	ND34	ND33	ND32
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00A0	FDCAN_RXF0C	F0M	F0WM[6:0]						Res	F0S[6:0]						F0SA[13:0]												Res	Res				
	Reset value	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x00A4	FDCAN_RXF0S	Res	Res	Res	Res	Res	Res	RF0	F0F	Res	Res	F0PI[5:0]					Res	Res	F0GI[5:0]					Res	F0FL[6:0]								
	Reset value							0	0			0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00A8	FDCAN_RXF0A	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	F0AI[5:0]					
	Reset value																										0	0	0	0	0	0	0
0x00AC	FDCAN_RXBC	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	RBSA[13:0]												Res	Res			
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x00B0	FDCAN_RXF1C	F1M	F1WM[6:0]						Res	F1S[6:0]						F1SA[13:0]												Res	Res				
	Reset value	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00B4	FDCAN_RXF1S	DMS[1:0]	Res	Res	Res	Res	Res	RF1	F1F	Res	Res	F1PI[5:0]					Res	Res	F1GI[5:0]					Res	F1FL[6:0]								
	Reset value	0	0					0	0			0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00B8	FDCAN_RXF1A	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	F1AI[5:0]					
	Reset value																										0	0	0	0	0	0	0
0x00BC	FDCAN_RXESC	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	RBDS[2:0]		F1DS[2:0]		Res	F0DS[2:0]		Res	Res	Res	Res		
	Reset value																				0	0	0		0	0	0				0	0	0
0x00C0	FDCAN_TXBC	Res	TFQM	TFQS[5:0]						Res	Res	NTDB[5:0]					TBSA[13:0]												Res	Res			
	Reset value		0	0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00C4	FDCAN_TXFQS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TFQF	TFQP[4:0]				Res	Res	Res	TFGI[4:0]				Res	Res	TFFL[5:0]							
	Reset value											0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0
0x00C8	FDCAN_TXESC	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TBDS[2:0]	
	Reset value																														0	0	0

Table 526. FDCAN register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00CC	FDCAN_TXBRP	TRP31	TRP30	TRP29	TRP28	TRP27	TRP26	TRP25	TRP24	TRP23	TRP22	TRP21	TRP20	TRP19	TRP18	TRP17	TRP16	TRP15	TRP14	TRP13	TRP12	TRP11	TRP10	TRP9	TRP8	TRP7	TRP6	TRP5	TRP4	TRP3	TRP2	TRP1	TRP0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00D0	FDCAN_TXBAR	AR31	AR30	AR29	AR28	AR27	AR26	AR25	AR24	AR23	AR22	AR21	AR20	AR19	AR18	AR17	AR16	AR15	AR14	AR13	AR12	AR11	AR10	AR9	AR8	AR7	AR6	AR5	AR4	AR3	AR2	AR1	AR0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00D4	FDCAN_TXBCR	CR31	CR30	CR29	CR28	CR27	CR26	CR25	CR24	CR23	CR22	CR21	CR20	CR19	CR18	CR17	CR16	CR15	CR14	CR13	CR12	CR11	CR10	CR9	CR8	CR7	CR6	CR5	CR4	CR3	CR2	CR1	CR0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00D8	FDCAN_TXBTO	TO31	TO30	TO29	TO28	TO27	TO26	TO25	TO24	TO23	TO22	TO21	TO20	TO19	TO18	TO17	TO16	TO15	TO14	TO13	TO12	TO11	TO10	TO9	TO8	TO7	TO6	TO5	TO4	TO3	TO2	TO1	TO0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00DC	FDCAN_TXBCF	CF31	CF30	CF29	CF28	CF27	CF26	CF25	CF24	CF23	CF22	CF21	CF20	CF19	CF18	CF17	CF16	CF15	CF14	CF13	CF12	CF11	CF10	CF9	CF8	CF7	CF6	CF5	CF4	CF3	CF2	CF1	CF0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00E0	FDCAN_TXBTIE	TIE31	TIE30	TIE29	TIE28	TIE27	TIE26	TIE25	TIE24	TIE23	TIE22	TIE21	TIE20	TIE19	TIE18	TIE17	TIE16	TIE15	TIE14	TIE13	TIE12	TIE11	TIE10	TIE9	TIE8	TIE7	TIE6	TIE5	TIE4	TIE3	TIE2	TIE1	TIE0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00E4	FDCAN_TXBCIE	CFIE31	CFIE30	CFIE29	CFIE28	CFIE27	CFIE26	CFIE25	CFIE24	CFIE23	CFIE22	CFIE21	CFIE20	CFIE19	CFIE18	CFIE17	CFIE16	CFIE15	CFIE14	CFIE13	CFIE12	CFIE11	CFIE10	CFIE9	CFIE8	CFIE7	CFIE6	CFIE5	CFIE4	CFIE3	CFIE2	CFIE1	CFIE0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00F0	FDCAN_TXEFC	Res	Res	EFWM[5:0]						Res	Res	EFS[5:0]						EFSA[15:2]															Res	Res
	Reset value			0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x00F4	FDCAN_TXEFS	Res	Res	Res	Res	Res	Res	TEF	EFF	Res	Res	Res	EFPI[4:0]				Res	Res	Res	EFGI[4:0]				Res	Res	EFFL[5:0]								
	Reset value							0	0				0	0	0	0	0				0	0	0	0	0				0	0	0	0	0	
0x00F8	FDCAN_TXEFA	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	EFAI[4:0]						
	Reset value																											0	0	0	0	0	0	

Refer to [Section 2.3](#) for the register boundary addresses.

59.6 TTCAN registers

These registers are available only for FDCAN1.

59.6.1 FDCAN TT trigger memory configuration register (FDCAN_TTTMC)

Address offset: 0x100

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TME[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TMSA[13:0]													Res.	Res.	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **TME[6:0]**: Trigger memory elements

0: No trigger memory

1-64: Number of trigger memory elements

>64: Values greater than 64 are interpreted as 64

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:2 **TMSA[13:0]**: Trigger memory start address.

Start address of trigger memory in message RAM (32-bit word address, see [Figure 780](#)).

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 1:0 Reserved, must be kept at reset value.

59.6.2 FDCAN TT reference message configuration register (FDCAN_TTRMC)

Address offset: 0x0104

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RMPS	XTD	Res.	RID[28:16]												
rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RID[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **RMPS**: Reference message payload select

Ignored in case of time slaves.

0: Reference message has no additional payload

1: The following elements are taken from Tx buffer 0:

Message marker MM,

Event FIFO control EFC,

Data length code DLC,

Data Bytes DB (level 1: bytes 2-8, level 0, 2: bytes 5-8)

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 30 **XTD**: Extended identifier

0: 11-bit standard identifier

1: 29-bit extended identifier

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 29 Reserved, must be kept at reset value.

Bits 28:0 **RID[28:0]**: Reference identifier.

Identifier transmitted with reference message and used for reference message filtering. Standard or extended reference identifier depending on bit XTD. A standard identifier must be written to ID[28:18].

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

59.6.3 FDCAN TT operation configuration register (FDCAN_TTOCF)

Address offset: 0x0108

Reset value: 0x0001 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	EVTP	ECC	EGTF	AWL[7:0]							
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EECS	IRTO[6:0]						LSDSL[2:0]				TM	GEN	Res.	OM[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw

Bits 31:27 Reserved, must be kept at reset value.

Bit 26 **EVTP**: Event trigger polarity.

0: Rising edge trigger

1: Falling edge trigger

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 25 **ECC**: Enable clock calibration.

0: Automatic clock calibration in FDCAN level 0, 2 is disabled

1: Automatic clock calibration in FDCAN level 0, 2 is enabled

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 24 **EGTF**: Enable global time filtering.

0: Global time filtering in FDCAN level 0, 2 is disabled

1: Global time filtering in FDCAN level 0, 2 is enabled

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 23:16 **AWL[7:0]**: Application watchdog limit.

The application watchdog can be disabled by programming AWL to 0x00.

0x00 to FF: Maximum time after which the application has to serve the application watchdog. The application watchdog is incremented once each 256 NTUs.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 15 **EECS**: Enable external clock synchronization

If enabled, TUR configuration (FDCAN_TURCF.NCL only) may be updated during FDCAN operation.

0: External clock synchronization in FDCAN level 0, 2 disabled

1: External clock synchronization in FDCAN level 0, 2 enabled

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 14:8 **IRTO[6:0]**: Initial reference trigger offset.

0x00 to 7F Positive offset, range from 0 to 127

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 7:5 **LDSDL[2:0]**: LD of synchronization deviation limit.

The synchronization deviation limit SDL is configured by its dual logarithm LDSDL with $SDL = 2 * (LDSDL + 5)$. SDL is comprised between 32 and 4096. It should not exceed the clock tolerance given by the CAN bit timing configuration.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 4 **TM**: Time master.

0: Time master function disabled

1: Potential time master

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 3 **GEN**: Gap enable.

0: Strictly time-triggered operation

1: External event-synchronized time-triggered operation

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bit 2 Reserved, must be kept at reset value.

Bits 1:0 **OM[1:0]**: Operation mode.

00: Event-driven CAN communication, default

01: TTCAN level 1

10: TTCAN level 2

11: TTCAN level 0

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

59.6.4 FDCAN TT matrix limits register (FDCAN_TTMLM)

Address offset: 0x010C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	ENTT[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TXEW[3:0]				CSS[1:0]		CCM[5:0]					
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:16 **ENTT[11:0]**: Expected number of Tx triggers

0x000 to FFF Expected number of Tx triggers in one matrix cycle.

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:8 **TXEW[3:0]**: Tx enable window

0x0 to F Length of Tx enable window, 1-16 NTU cycles

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 7:6 **CSS[1:0]**: Cycle start synchronization

Enables sync pulse output.

00: No sync pulse

01: Sync pulse at start of basic cycle

10: Sync pulse at start of matrix cycle

11: Reserved

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 5:0 **CCM[5:0]**: Cycle count Max

0x00: 1 basic cycle per matrix cycle

0x01: 2 basic cycles per matrix cycle

0x03: 4 basic cycles per matrix cycle

0x07: 8 basic cycles per matrix cycle

0x0F: 16 basic cycles per matrix cycle

0x1F: 32 basic cycles per matrix cycle

0x3F: 64 basic cycles per matrix cycle

Others: Reserved

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Note: *ISO 11898-4, Section 5.2.1 requires that only the listed cycle count values are configured. Other values are possible, but may lead to inconsistent matrix cycles.*

59.6.5 FDCAN TUR configuration register (FDCAN_TURCF)

The length of the NTU is given by: $NTU = \text{CAN clock period} \times NC/DC$.

NC is an 18-bit value. Its high part, NCH[17:16] is hard wired to 0b01. Therefore the range of NC extends from 0x10000 to 0x1FFFF. The value configured by NCL is the initial value for FDCAN_TURNA.NAV[15:0]. DC is set to 0x1000 by hardware reset and it may not be written to 0x0000.

- Level 1: $NC \times 4$ and $NTU = \text{CAN bit time}$
- Levels 0 and 2: $NC \times 8$

The actual value of FDCAN_TUR may be changed by the clock drift compensation function of TTCAN level 0 and level 2 in order to adjust the node local view of the NTU to the time master view of the NTU. DC is not changed by the automatic drift compensation, FDCAN_TURNA.NAV may be adjusted around NC in the range of the synchronization deviation limit given by FDCAN_TTOCF.LDSDL. NC and DC should be programmed to the largest suitable values in achieve the best computational accuracy for the drift compensation process.

Address offset: 0x0110

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ELT	Res.	DC[13:0]													
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NCL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **ELT**: Enable local time.

0: Local time is stopped, default

1: Local time is enabled

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Note: The local time is started by setting ELT, it remains active until ELT is reset or until the next hardware reset. FDCAN_TURCF.DC is locked when FDCAN_TURCF.ELT = 1. If ELT is written to 0, the readable value stays at 1 until the new value has been synchronized into the CAN clock domain. During this time write access to the other bits of the register is locked.

Bit 30 Reserved, must be kept at reset value.

Bits 29:16 **DC[13:0]**: Denominator configuration.

0x0000: Illegal value

0x0001 to 0x3FFF: Denominator configuration

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Bits 15:0 **NCL[15:0]**: Numerator configuration low.

Write access to the TUR numerator configuration low is only possible during configuration with FDCAN_TURCF.ELT = 0 or if FDCAN_TTOCF.EECS (external clock synchronization enabled) is set. When a new value for NCL is written outside TT configuration mode, the new value takes effect when FDCAN_TTOST.WECS is cleared to 0. NCL is locked FDCAN_TTOST.WECS is 1.

0x0000 to FFFF Numerator configuration low

These are write-protected bits, write access is possible only when bit CCE and bit INIT of FDCAN_CCCR register are set to 1.

Note: *If $NC < 7 \times DC$ in TTCAN level 1, subsequent time marks in the trigger memory must differ by at least two NTUs.*

59.6.6 FDCAN TT operation control register (FDCAN_TTOCN)

Address offset: 0x0114

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCKC	Res.	ESCN	NIG	TMG	FGP	GCS	TTIE	TMC[1:0]		RTIE	SWS[1:0]		SWP	ECS	SGT
r		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **LCKC**: TT operation control register locked.

Set by a write access to register FDCAN_TTOCN. Reset when the updated configuration has been synchronized into the CAN clock domain.

0: Write access to FDCAN_TTOCN enabled

1: Write access to FDCAN_TTOCN locked

Bit 14 Reserved, must be kept at reset value.

Bit 13 **ESCN**: External synchronization control

If enabled the FDCAN synchronizes its cycle time phase to an external event signaled by a rising edge at event trigger pin (see [Section 59.4.17: Synchronization to external time schedule](#)).

0: External synchronization disabled

1: External synchronization enabled

Bit 12 **NIG**: Next is gap.

This bit can be set only when the FDCAN is the actual time master and when it is configured for external event-synchronized time-triggered operation (FDCAN_TTOCF.GEN = 1)

0: No action, reset by reception of any reference message

1: Transmit next reference message with Next_is_Gap = 1

Bit 11 **TMG**: Time mark gap.

0: Reset by each reference message

1: Next reference message started when register time mark interrupt FDCAN_TTIR.RTMI is activated

Bit 10 **FGP**: Finish gap.

- Set by the CPU, reset by each reference message
- 0: No reference message requested
- 1: Application requested start of reference message

Bit 9 **GCS**: Gap control select

- 0: Gap control independent from event trigger
- 1: Gap control by input event trigger pin

Bit 8 **TTIE**: Trigger time mark interrupt pulse enable

External time mark events are configured by trigger memory element TMEX. A trigger time mark interrupt pulse is generated when the trigger memory element becomes active, and the FDCAN is in synchronization state In_Schedule or In_Gap.

- 0: Trigger time mark interrupt output fdcan1_tmp for more than one instance and fdcan_tmp if only one instance disabled
- 1: Trigger time mark interrupt output fdcan1_tmp for more than one instance and fdcan_tmp if only one instance enabled

Bits 7:6 **TMC[1:0]**: Register time mark compare.

- 00: No register time mark interrupt generated
- 01: Register time mark interrupt if time mark = cycle time
- 10: Register time mark interrupt if time mark = local time
- 11: Register time mark interrupt if time mark = global time

Note: When changing the time mark reference (cycle, local, global time), it is recommended to first write TMC = 00, then reconfigure FDCAN_TTTMK, and finally set FDCAN_TMC to the intended time reference.

Bit 5 **RTIE**: Register time mark interrupt pulse enable.

Register time mark interrupts are configured by register FDCAN_TTTMK. A register time mark interrupt pulse with the length of one fdcan_tq_ck period is generated when time referenced by FDCAN_TTOCN.TMC (cycle, local, or global) is equal to FDCAN_TTTMK.TM, independently from the synchronization state.

- 0: Register time mark interrupt output disabled
- 1: Register time mark interrupt output enabled

Bits 4:3 **SWS[1:0]**: Stop watch source.

- 00: Stop watch disabled
- 01: Actual value of cycle time is copied to FDCAN_TTCPT.SWV
- 10: Actual value of local time is copied to FDCAN_TTCPT.SWV
- 11: Actual value of global time is copied to FDCAN_TTCPT.SWV

Bit 2 **SWP**: Stop watch polarity.

- 0: Rising edge trigger
- 1: Falling edge trigger

Bit 1 **ECS**: External clock synchronization.

Writing a 1 to ECS sets FDCAN_TTOST.WECS if the node is the actual time master. ECS is reset after one APB clock period. The external clock synchronization takes effect at the start of the next basic cycle.

Bit 0 **SGT**: Set global time.

Writing a 1 to SGT sets FDCAN_TTOST.WGDT if the node is the actual time master. SGT is reset after one APB clock period. The global time preset takes effect when the node transmits the next reference message with the Master_Ref_Mark modified by the preset value written to FDCAN_TTGTP.

59.6.7 FDCAN TT global time preset register (FDCAN_TTGTP)

If TTOST.WGDT is set, the next reference message is transmitted with the Master_Ref_Mark modified by the preset value and with Disc_Bit = 1, presetting the global time in all nodes simultaneously.

TP is reset to 0x0000 each time a reference message with Disc_Bit = 1 becomes valid or if the node is not the current time master. TP is locked while FDCAN_TTOST.WGTD = 1 after setting FDCAN_TTOCN.SGT until the reference message with Disc_Bit = 1 becomes valid or until the node is no longer the current time master.

Address offset: 0x0118

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CTP[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TP[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **CTP[15:0]**: Cycle time target phase

CTP is write-protected while FDCAN_TTOCN.ESCN or FDCAN_TTOST.SPL are set (see [Section 59.4.17: Synchronization to external time schedule](#)).

0x0000 to FFFF defines the target value of cycle time when a rising edge of event trigger is expected

Bits 15:0 **TP[15:0]**: Time preset

TP is write-protected while FDCAN_TTOST.WGTD is set.

0x0000 to 7FFF next master reference mark = master reference mark + TP

0x8000 reserved

0x8001–FFFF Next master reference mark = master reference mark - (0x10000 - TP).

59.6.8 FDCAN TT time mark register (FDCAN_TTTMK)

A time mark interrupt (FDCAN_TTIR.TMI = 1) is generated when the time base indicated by FDCAN_TTOCN.TMC (cycle time, local time, or global time) has the same value as TM.

Address offset: 0x011C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LCKM	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TICC[6:0]						
r									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TM[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **LCKM**: TT time mark register locked

Always set by a write access to registers FDCAN_TTOCN. Set by write access to register FDCAN_TTTMK when FDCAN_TTOCN.TMC 00. Reset when the registers have been synchronized into the CAN clock domain.

0: Write access to FDCAN_TTTMK enabled

1: Write access to FDCAN_TTTMK locked

Bits 30:23 Reserved, must be kept at reset value.

Bits 22:16 **TICC[6:0]**: Time mark cycle code

Cycle count for which the time mark is valid.

0b000000x valid for all cycles

0b000001c valid every second cycle at cycle count mod2 = c

0b00001cc valid every fourth cycle at cycle count mod4 = cc

0b0001ccc valid every eighth cycle at cycle count mod8 = ccc

0b001cccc valid every sixteenth cycle at cycle count mod16 = cccc

0b01ccccc valid every thirty-second cycle at cycle count mod32 = ccccc

0b1cccccc valid every sixty-fourth cycle at cycle count mod64 = ccccccc

Bits 15:0 **TM[15:0]**: Time mark

0x0000 to FFFF time mark

Note: When using byte access to register FDCAN_TTTMK it is recommended to first disable the time mark compare function (FDCAN_TTOCN.TMC = 00) to avoid comparisons on inconsistent register values.

59.6.9 FDCAN TT interrupt register (FDCAN_TTIR)

The flags are set when one of the listed conditions is detected (edge-sensitive). The flags remain set until the user clears them. A flag is cleared by writing a 1 to the corresponding bit position. Writing a 0 has no effect. A hard reset clears the register.

Address offset: 0x0120

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CER	AW	WT
													rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IWTG	ELC	SE2	SE1	TXO	TXU	GTE	GTD	GTW	SWE	TTMI	RTMI	SOG	CSM	SMC	SBC
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:19 Reserved, must be kept at reset value.

Bit 18 **CER**: Configuration error

Trigger out of order.

0: No error found in trigger list

1: Error found in trigger list

Bit 17 **AW**: Application watchdog

0: Application watchdog served in time

1: Application watchdog not served in time

- Bit 16 **WT**: Watch trigger
0: No missing reference message
1: Missing reference message (level 0: cycle time 0xFF00)
- Bit 15 **IWTG**: Initialization watch trigger
The initialization is restarted by resetting IWT.
0 No missing reference message during system startup
1 No system startup due to missing reference message
- Bit 14 **ELC**: Error level changed
Not set when error level changed during initialization.
0: No change in error level
1: Error level changed
- Bit 13 **SE2**: Scheduling error 2
0: No scheduling error 2
1: Scheduling error 2 occurred
- Bit 12 **SE1**: Scheduling error 1
0: No scheduling error 1
1: Scheduling error 1 occurred
- Bit 11 **TXO**: Tx count overflow
0: Number of Tx trigger as expected
1: More Tx trigger than expected in one cycle
- Bit 10 **TXU**: Tx count underflow
0: Number of Tx trigger as expected
1: Less Tx trigger than expected in one cycle
- Bit 9 **GTE**: Global time error
Synchronization deviation SD exceeds limit specified by FDCAN_TTOCF.LDSDL, TTCAN level 0, 2 only.
0: Synchronization deviation within limit
1: Synchronization deviation exceeded limit
- Bit 8 **GTD**: Global time discontinuity
0: No discontinuity of global time
1: Discontinuity of global time
- Bit 7 **GTW**: Global time wrap
0: No global time wrap occurred
1: Global time wrap from 0xFFFF to 0x0000 occurred
- Bit 6 **SWE**: Stop watch event
0: No rising/falling edge at stop watch trigger pin detected
1: Rising/falling edge at stop watch trigger pin detected
- Bit 5 **TTMI**: Trigger time mark event internal
Internal time mark events are configured by trigger memory element TMIN (see [Section 59.4.23](#)).
Set when the trigger memory element becomes active, and the FDCAN is in synchronization state In_Gap or In_Schedule.
0: Time mark not reached
1: Time mark reached (level 0: cycle time FDCAN_TTOCF.RTO x 0x200)

Bit 4 **RTMI**: Register time mark interrupt

Set when time referenced by TTOCN.TMC (cycle, local, or global) is equal to FDCAN_TTTMK.TM, independently from the synchronization state.

0: Time mark not reached

1: Time mark reached

Bit 3 **SOG**: Start of gap

0 No reference message seen with Next_is_Gap bit set

1 Reference message with Next_is_Gap bit set becomes valid

Bit 2 **CSM**: Change of synchronization mode

0: No change in master to slave relation or schedule synchronization

1: Master to slave relation or schedule synchronization changed, also set when FDCAN_TTOST.SPL is reset

Bit 1 **SMC**: Start of matrix cycle

0: No matrix cycle started since bit has been reset

1: Matrix cycle started

Bit 0 **SBC**: Start of basic cycle

0: No basic cycle started since bit has been reset

1: Basic cycle started

59.6.10 FDCAN TT interrupt enable register (FDCAN_TTIE)

The settings in the TT interrupt enable register determine which status changes in the TT interrupt register result in an interrupt.

Address offset: 0x0124

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CERE	AWE	WTE
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IWTE	ELCE	SE2E	SE1E	TXOE	TXUE	GTEE	GTDE	GTWE	SWEE	TTMIE	RTMIE	SOG	CSME	SMCE	SBCE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:19 Reserved, must be kept at reset value.

Bit 18 **CERE**: Configuration error interrupt enable

0: TT interrupt disabled

1: TT interrupt enabled

Bit 17 **AWE**: Application watchdog interrupt enable

0: TT interrupt disabled

1: TT interrupt enabled

Bit 16 **WTE**: Watch trigger interrupt enable

0: TT interrupt disabled

1: TT interrupt enabled

- Bit 15 **IWTE**: Initialization watch trigger interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 14 **ELCE**: Change error level interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 13 **SE2E**: Scheduling error 2 interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 12 **SE1E**: Scheduling error 1 interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 11 **TXOE**: Tx count overflow interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 10 **TXUE**: Tx count underflow interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 9 **GTEE**: Global time error interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 8 **GTDE**: Global time discontinuity interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 7 **GTWE**: Global time wrap interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 6 **SWEE**: Stop watch event interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 5 **TTMIE**: Trigger time mark event internal interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 4 **RTMIE**: Register time mark interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 3 **SOGE**: Start of gap interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 2 **CSME**: Change of synchronization mode interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled
- Bit 1 **SMCE**: Start of matrix cycle interrupt enable
0: TT interrupt disabled
1: TT interrupt enabled

- Bit 0 **SBCE**: Start of basic cycle interrupt enable
 0: TT interrupt disabled
 1: TT interrupt enabled

59.6.11 FDCAN TT interrupt line select register (FDCAN_TTILS)

The TT interrupt line select register assigns an interrupt generated by a specific interrupt flag from the TT interrupt register to one of the two module interrupt lines. For interrupt generation the respective interrupt line must be enabled via FDCAN_ILE.EINT0 and FDCAN_ILE.EINT1.

Address offset: 0x0128

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CERL	AWL	WTL
													r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IWTL	ELCL	SE2L	SE1L	TXOL	TXUL	GTEL	GTDL	GTWL	SWEL	TTMIL	RTMIL	SOGL	CSML	SMCL	SBCL
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:19 Reserved, must be kept at reset value.

- Bit 18 **CERL**: Configuration error interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 17 **AWL**: Application watchdog interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 16 **WTL**: Watch trigger interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 15 **IWTL**: Initialization watch trigger interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 14 **ELCL**: Change error level interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 13 **SE2L**: Scheduling error 2 interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 12 **SE1L**: Scheduling error 1 interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 11 **TXOL**: Tx count overflow interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1

- Bit 10 **TXUL**: Tx count underflow interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 9 **GTEL**: Global time error interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 8 **GTDL**: Global time discontinuity interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 7 **GTWL**: Global time wrap interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 6 **SWEL**: Stop watch event interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 5 **TTMIL**: Trigger time mark event internal interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 4 **RTMIL**: Register time mark interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 3 **SOGL**: Start of gap interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 2 **CSML**: Change of synchronization mode interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 1 **SMCL**: Start of matrix cycle interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1
- Bit 0 **SBCL**: Start of basic cycle interrupt line
 0: TT interrupt assigned to interrupt line 0
 1: TT interrupt assigned to interrupt line 1

59.6.12 FDCAN TT operation status register (FDCAN_TTOST)

Address offset: 0x012C

Reset value: 0x0000 0080

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SPL	WECS	AWE	WFE	GSI	TMP[2:0]			GFI	WGTD	Res.	Res.	Res.	Res.	Res.	Res.
r	r	r	r	r	r	r	r	r	r						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTO[7:0]								QCS	QGTP	SYS[1:0]		MS[1:0]		EL[1:0]	
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

- Bit 31 **SPL**: Schedule phase lock
 The bit is valid only when external synchronization is enabled (FDCAN_TTOCN.ESCN = 1). In this case it signals that the difference between cycle time configured by FDCAN_TTGTP.CTP and the cycle time at the rising edge at event trigger pin is less than or equal to 9 NTU (see [Section 59.4.17: Synchronization to external time schedule](#)).
 0: Phase outside range
 1: Phase inside range
- Bit 30 **WECS**: Wait for external clock synchronization.
 0: No external clock synchronization pending
 1: Node waits for external clock synchronization to take effect. The bit is reset at the start of the next basic cycle.
- Bit 29 **AWE**: Application watchdog event
 The application watchdog is served by reading FDCAN_TTOST. When the watchdog is not served in time, bit AWE is set, all FDCAN communication is stopped, and the FDCAN is set into bus monitoring mode.
 0: Application watchdog served in time
 1: Failed to serve application watchdog in time
- Bit 28 **WFE**: Wait for event
 0: No gap announced, reset by a reference message with Next_is_Gap = 0
 1: Reference message with Next_is_Gap = 1 received
- Bit 27 **GSI**: Gap started indicator
 0: No gap in schedule, reset by each reference message and for all time slaves
 1: Gap time after basic cycle has started
- Bits 26:24 **TMP[2:0]**: Time master priority
 0x0-7 Priority of actual time master
- Bit 23 **GFI**: Gap finished indicator
 Set when the CPU writes FDCAN_TTOCN.FGP, or by a time mark interrupt if TMG = 1, or via input pin (event trigger) if FDCAN_TTOCN.GCS = 1. Not set by Ref_Trigger_Gap or when Gap is finished by another node sending a reference message.
 0: Reset at the end of each reference message
 1: Gap finished by FDCAN
- Bit 22 **WGTD**: Wait for global time discontinuity
 0: No global time preset pending
 1: Node waits for the global time preset to take effect. The bit is reset when the node has transmitted a reference message with Disc_Bit = 1 or after it received a reference message.
- Bits 21:16 Reserved, must be kept at reset value.
- Bits 15:8 **RTO[7:0]**: Reference trigger offset
 The reference trigger offset value is a signed integer with a range from -127 (0x81) to 127 (0x7F). There is no notification when the lower limit of -127 is reached. In case the FDCAN becomes time master (MS[1:0] = 11), the reset of RTO is delayed due to synchronization between user and CAN clock domain. For time slaves the value configured by FDCAN_TTOCF.IRTO is read.
 0x00-FF Actual reference trigger offset value
- Bit 7 **QCS**: Quality of clock speed
 Only relevant in TTCAN level 0 and level 2, otherwise fixed to 1.
 0: Local clock speed not synchronized to time master clock speed
 1: Synchronization deviation $\leq$ SDL

Bit 6 **QGTP**: Quality of global time phase

Only relevant in TTCAN level 0 and level 2, otherwise fixed to 0.

0: Global time not valid

1: Global time in phase with time master

Bits 5:4 **SYS[1:0]**: Synchronization state

00: Out of Synchronization

01: Synchronizing to FDCAN communication

10: Schedule suspended by gap (In_Gap)

11: Synchronized to schedule (In_Schedule)

Bits 3:2 **MS[1:0]**: Master state

00: Master_Off, no master properties relevant

01: Operating as time Slave

10: Operating as backup time master

11: Operating as current time master

Bits 1:0 **EL[1:0]**: Error level

00: Severity 0 - No error

01: Severity 1 - Warning

10: Severity 2 - error

11: Severity 3 - Severe error

59.6.13 FDCAN TUR numerator actual register (FDCAN_TURNA)

There is no drift compensation in TTCAN level 1 (NAV = NC). In TTCAN level 0 and level 2, the drift between the node local clock and the time master local clock is calculated. The drift is compensated when the synchronization deviation (difference between NC and the calculated NAV) is lower than $2 * (FDCAN_TTOCF.LDSDDL + 5)$. With $FDCAN_TTOCF.LDSDDL < 7$, this results in a maximum range for NAV of $(NC - 0x1000) \leq NAV \leq (NC + 0x1000)$.

Address offset: 0x0130

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NAV[17:16]	
														r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NAV[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:18 Reserved, must be kept at reset value.

Bits 17:0 **NAV[17:0]**: Numerator actual value

0x0EFFF: illegal value

0x0F000 to 20FFF: actual numerator value

0x21000: illegal value

59.6.14 FDCAN TT local and global time register (FDCAN_TTLGT)

Address offset: 0x0134

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
GT[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LT[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 **GT[15:0]**: Global time

Non-fractional part of the sum of the node local time and its local offset (see [Section 59.4.12](#)).
0x0000 to FFFF Global time value of FDCAN network

Bits 15:0 **LT[15:0]**: Local time

Non-fractional part of local time, incremented once each local NTU (see [Section 59.4.12](#)).
0x0000 to FFFF Local time value of FDCAN node

59.6.15 FDCAN TT cycle time and count register (FDCAN_TTCTC)

Address offset: 0x0138

Reset value: 0x003F 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CC[5:0]					
										r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CT[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:16 **CC[5:0]**: Cycle count

0x00 to 3F Number of actual basic cycle in the system matrix

Bits 15:0 **CT[15:0]**: Cycle time

Non-fractional part of the difference of the node local time and Ref_Mark (see [Section 59.4.12](#)).
0x0000 to FFFF: cycle time value of FDCAN basic cycle

59.6.16 FDCAN TT capture time register (FDCAN_TTCPT)

Address offset: 0x013C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWV[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CCV[5:0]					
										r	r	r	r	r	r

Bits 31:16 **SWV[15:0]**: Stop watch value

On a rising / falling edge (as configured via FDCAN_TTOCN.SWP) at the stop watch trigger pin, when FDCAN_TTOCN.SWS is different from 00 and FDCAN_TTIR.SWE is 0, the actual time value as selected by FDCAN_TTOCN.SWS (cycle, local, global) is copied to SWV and FDCAN_TTIR.SWE is set to 1. Capturing of the next stop watch value is enabled by resetting FDCAN_TTIR.SWE.

0x0000 to FFFF Captured stop watch value

Bits 15:6 Reserved, must be kept at reset value.

Bits 5:0 **CCV[5:0]**: Cycle count value

Cycle count value captured together with SWV.

0x00 to 3F: captured cycle count value

59.6.17 FDCAN TT cycle sync mark register (FDCAN_TTCSM)

Address offset: 0x0140

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CSM[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **CSM[15:0]**: Cycle sync mark

The cycle sync mark is measured in cycle time. It is updated when the reference message becomes valid and retains its value until the next reference message becomes valid.

0x0000 to FFFF Captured cycle time

59.6.18 FDCAN TT trigger select register (FDCAN_TTTS)

The settings in the FDCAN_TTTS register select the input to be used as event trigger and stop watch trigger.

Address offset: 0x0300

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EVTSEL[1:0]		Res.	Res.	SWTDEL[1:0]	
										rw	rw			rw	rw

Bits 31:6 Reserved, must be kept at reset value.

Bits 5:4 **EVTSEL[1:0]**: Event trigger input selection

These bits are used to select the input to be used as event trigger

00: fdcan1_evt0

01: fdcan1_evt1

10: fdcan1_evt2

11: fdcan1_evt3

Bits 3:2 Reserved, must be kept at reset value.

Bits 1:0 **SWTDEL[1:0]**: Stop watch trigger input selection

These bits are used to select the input to be used as stop watch trigger

00: fdcan1_swat0

01: fdcan1_swat1

10: fdcan1_swat2

11: fdcan1_swat3

59.6.19 FDCAN TT register map

Table 527. FDCAN TT register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0100	FDCAN_TTTMC	Res	Res	Res	Res	Res	Res	Res	Res	Res	TME[6:0]						TMSA[13:0]														Res	Res	
	Reset value										0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x0104	FDCAN_TTRMC	RMP	XTD	RID[28:0]																													
	Reset value	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0108	FDCAN_TTOCF	Res	Res	Res	Res	Res	EVTP	ECC	EGTF	AWL[7:0]							EECS	IRTO[6:0]						LDSDL[2:0]		TM		GEN	Res	OM[1:0]			
	Reset value						0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x010C	FDCAN_TTMLM	Res	Res	Res	Res	ENTT[11:0]										Res	Res	Res	Res	TXEW[3:0]				CSS[1:0]		CCM[5:0]							
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 527. FDCAN TT register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0110	FDCAN_TURCF	ELT	Res	DC[13:0]														NCL[15:0]															
	Reset value	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0114	FDCAN_TTOCN	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	LCKC	Res	ESCN	NIG	TMG	FGP	GCS	TTIE	TMC[1:0]	RTIE	SWS[1:0]	SWP	ECS	SGT		
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0118	FDCAN_TTGTP	CTP[15:0]														TP[15:0]																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x011C	FDCAN_TTTMK	LCKM	Res	Res	Res	Res	Res	Res	Res	Res	TICC[6:0]						TM[15:0]																
	Reset value	0									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0120	FDCAN_TTIR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CER	AW	WT	IWTG	ELC	SE2	SE1	TXO	TXU	GTE	GTD	GTW	SWE	TTMI	RTMI	SOG	CSM	SMC	SBC
	Reset value														0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0124	FDCAN_TTIE	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CERE	AWE	WTE	IWTE	ELCE	SE2E	SE1E	TXOE	TXUE	GTEE	GTDE	GTWE	SWEE	TTMIE	RTMIE	SOGE	CSME	SMCE	SBCE
	Reset value														0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0128	FDCAN_TTILS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CERL	AWL	WTL	IWTL	ELCL	SE2L	SE1L	TXOL	TXUL	GTEL	GTDL	GTWL	SWEL	TTMIL	RTMIL	SOGL	CSWL	SMCL	SBCL
	Reset value														0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x012C	FDCAN_TTOST	SPL	WECS	AVE	WFE	GSI	TMP [2:0]			GFI	WGTD	Res	Res	Res	Res	Res	Res	RTO[7:0]						QCS			QGTP	SYS [1:0]		MS [1:0]	EL [1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0x0130	FDCAN_TURNA	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	NAV[17:0]																
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0134	FDCAN_TTLGT	GT[15:0]														LT[15:0]																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0138	FDCAN_TTCTC	Res	Res	Res	Res	Res	Res	Res	Res	Res	CC[5:0]						CT[15:0]																
	Reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x013C	FDCAN_TTCPT	SWV[15:0]														Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CCV[5:0]							
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0											0	0	0	0	0	0	0
0x0140	FDCAN_TTCSM	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CSM[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0144 to 0x01FC	Reserved																																
	Reset value																																

Table 527. FDCAN TT register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x0300	FDCAN_TTTS	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	EVTSEL [1:0]					SWTSEL [1:0]	
	Reset value																											0	0			0	0	

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.

59.7 CCU registers

59.7.1 Clock calibration unit core release register (FDCAN_CCU_CREL)

Address offset: 0x0000

Reset value: 0x1114 1218

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
REL[3:0]				STEP[3:0]				SUBSTEP[3:0]				YEAR[3:0]			
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MON[7:0]								DAY[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:28 **REL[3:0]**: Core release = 1

Bits 27:24 **STEP[3:0]**: Step of core release = 1

Bits 23:20 **SUBSTEP[3:0]**: Sub-step of core release = 1

Bits 19:16 **YEAR[3:0]**: Timestamp year =

Bits 15:8 **MON[7:0]**: Timestamp month = 12

Bits 7:0 **DAY[7:0]**: Timestamp day = 18

59.7.2 Calibration configuration register (FDCAN_CCU_CCFG)

Address offset: 0x0004

Reset value: 0x0000 0004

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CDIV[3:0]			
rw												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OCPM[7:0]								CFL	BCC	Res.	TQBT[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

Bit 31 **SWR**: Software reset

Writing a 1 to this bit resets the calibration FSM to state `Not_Calibrated` (FDCAN_CCU_CSTAT.CALS = 00). The calibration watchdog value CWD.WDV is also reset. Registers FDCAN_CCFG, FDCAN_CCU_CSTAT and the calibration watchdog configuration CWD.WDC are unchanged. The bit remains set until reset is completed.

Write access by the user to registers/bits marked with "P = Protected Write" is possible only when FDCAN control bits FDCAN_CCCR.CCE = 1 AND FDCAN_CCCR.INIT = 1.

Bits 30:20 Reserved, must be kept at reset value.

Bits 19:16 **CDIV[3:0]**: Clock divider

The clock divider must be configured when the clock calibration is bypassed (BCC = 1) to ensure that the FDCAN requirement is fulfilled.

0000: Divide by 1
 0001: Divide by 2
 0010: Divide by 4
 0011: Divide by 6
 0100: Divide by 8
 0101: Divide by 10
 0110: Divide by 12
 0111: Divide by 14
 1000: Divide by 16
 1001: Divide by 18
 1010: Divide by 20
 1011: Divide by 22
 1100: Divide by 24
 1101: Divide by 26
 1110: Divide by 28
 1111: Divide by 30

Write access by the user to registers/bits marked with "P = Protected Write" is possible only when FDCAN control bits FDCAN_CCCR.CCE = 1 AND FDCAN_CCCR.INIT = 1.

Bits 15:8 **OCPM[7:0]**: Oscillator clock periods minimum

Configures the minimum number of periods in two CAN bit times. OCPM is used in basic calibration to avoid false measurements in case of glitches on the bus line. The configured number of periods is $OCPM \times 32$. The configuration depends on the frequency and the bitrate configured in FDCAN modules (from 125 kbit/s up to 1 Mbit/s). It is recommended to configure a value slightly below two CAN bit times. The reset value is 1.6 bit times at 80 MHz `fdcan_ker_ck` and 1 Mbit/s CAN bitrate.

Write access by the user to registers/bits marked with "P = Protected Write" is possible only when FDCAN control bits FDCAN_CCCR.CCE = 1 AND FDCAN_CCCR.INIT = 1.

Bit 7 **CFL**: Calibration field length

0: Calibration field length is 32 bits
 1: Calibration field length is 64 bits

Write access by the user to registers/bits marked with "P = Protected Write" is possible only when FDCAN control bits FDCAN_CCCR.CCE = 1 AND FDCAN_CCCR.INIT = 1.

Bit 6 **BCC**: Bypass clock calibration

If this bit is set, the clock input `fdcan_ker_ck` is routed to the time quanta clock through a clock divider configurable via CDIV. In this case the baudrate prescaler of the connected FDCANs must be configured to generate the FDCAN internal time quanta clock.

0: Clock calibration unit generates time quanta clock
 1: Clock calibration unit bypassed (default configuration)

Bit 5 Reserved, must be kept at reset value.

Bits 4:0 **TQBT[4:0]**: Time quanta per bit time

Configures the number of time quanta per bit time. Same value as configured in FDCAN modules.

The range of the resulting time quanta clock `fdcan_tq_ck` is from 0.5 MHz (bitrate of 125 kbit/s with 4 tq per bit time) to 25 MHz (bitrate of 1 Mbit/s with 25 tq per bit time). Valid values are 4 to 25.

Configured values below 4 are interpreted as 4, values above 25 are interpreted as 25.

Write access by the user to registers/bits marked with "P = Protected Write" is possible only when FDCAN control bits `FDCAN_CCCR.CCE = 1` AND `FDCAN_CCCR.INIT = 1`.

59.7.3 Calibration status register (FDCAN_CCU_CSTAT)

Address offset: 0x0008

Reset value: 0x0203 FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CALS[1:0]		Res.	TQC[10:0]											OCPC[17:16]	
r	r		r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OCPC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:30 **CALS[1:0]**: Calibration state

00: Not_Calibrated

01: Basic_Calibrated

10: Precision_Calibrated

11: Reserved

Bit 29 Reserved, must be kept at reset value.

Bits 28:18 **TQC[10:0]**: Time quanta counter

Captured number of time quanta in calibration field (32 or 64 bits). Only valid when the clock calibration unit is in state Precision_Calibrated.

Bits 17:0 **OCPC[17:0]**: Oscillator clock period counter

Captured number of oscillator clock periods in calibration field (32 or 64 bits). Only valid when the clock calibration unit is in state Precision_Calibrated.

59.7.4 Calibration watchdog register (FDCAN_CCU_CWD)

Address offset: 0x000C

Reset value: 0x0000 0000

The calibration watchdog is started after the first falling edge when the calibration FSM is in state Not_Calibrated (`FDCAN_CCU_CSTAT.CALS = 00`). In this state the calibration watchdog monitors the message received. In case no message was received until the calibration watchdog has counted down to 0, the calibration FSM stays in state Not_Calibrated (`FDCAN_CCU_CSTAT.CALS = 00`), the counter is reloaded with `FDCAN_RWD.WDC` and basic calibration is restarted after the next falling edge.

When in state Basic_Calibrated (`FDCAN_CCU_CSTAT.CALS = 01`), the calibration watchdog is restarted with each received message. In case no message was received until the calibration watchdog has counted down to 0, the calibration FSM returns to state

Not_Calibrated (FDCAN_CCU_CSTAT.CALS = 00), the counter is reloaded with FDCAN_RWD.WDC and basic calibration is restarted after the next falling edge.

When a quartz message is received, state Precision_Calibrated (FDCAN_CCU_CSTAT.CALS = 10) is entered and the calibration watchdog is restarted. In this state the calibration watchdog monitors the quartz message received input. In case no message from a quartz controlled node is received by the attached TTCAN until the calibration watchdog has counted down to 0, the calibration FSM transits back to state Basic_Calibrated (FDCAN_CCU_CSTAT.CALS = 01). The signal is active when the CAN protocol engine on the attached TTCAN is started i.e. when the INIT bit is reset.

A calibration watchdog event also sets interrupt flag FDCAN_CCU_IR.CWE. If enabled by FDCAN_CCU_IE.CWEE, interrupt line is activated (set to high). Interrupt line remains active until interrupt flag FDCAN_CCU_IR.CWE is reset.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WDV[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WDC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **WDV[15:0]**: Watchdog value
Actual calibration watchdog counter value.

Bits 15:0 **WDC[15:0]**: WDC
Watchdog configuration
Start value of the calibration watchdog counter. With the reset value of 00 the counter is disabled.
Write access by the user to registers/bits marked with "P = Protected Write" is possible only when FDCAN control bits FDCAN_CCCR.CCE = 1 AND FDCAN_CCCR.INIT = 1.

59.7.5 Clock calibration unit interrupt register (FDCAN_CCU_IR)

The flags are set when one of the listed conditions is detected (edge-sensitive). The flags remain set until the user clears them. A flag is cleared by writing a 1 to the corresponding bit position. Writing a 0 has no effect. A hard reset clears the register. The configuration of FDCAN_CCU_IE controls whether an interrupt is generated or not.

Address offset: 0x0010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSC	CWE
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

- Bit 1 **CSC**: Calibration state changed
 0: Calibration state unchanged
 1: Calibration state has changed
- Bit 0 **CWE**: Calibration watchdog event
 0: No calibration watchdog event
 1: Calibration watchdog event occurred

59.7.6 Clock calibration unit interrupt enable register (FDCAN_CCU_IE)

Address offset: 0x0014

Reset value: 0x0000 0000

The settings in the CU interrupt enable register determine whether a status change in the CU interrupt register is signaled on an interrupt line.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSCE	CWEE
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

- Bit 1 **CSCE**: Calibration state changed enable
 0: Interrupt disabled
 1: Interrupt enabled
- Bit 0 **CWEE**: Calibration watchdog event enable
 0: Interrupt disabled
 1: Interrupt enabled

59.7.7 CCU register map

Table 528. CCU register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0000	FDCAN_CCU_CREL	REL[3:0]			STEP[3:0]				SUBSTEP[3:0]				YEAR[3:0]				MON[7:0]							DAY[7:0]									
	Reset value	0	0	0	1	0	0	0	1	0	0	0	1	0	1	0	0	0	0	0	1	0	0	1	0	0	0	0	0	1	1	0	0
0x0004	FDCAN_CCU_CCFG	SWR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CDIV[3:0]				OCPM[7:0]							CFL	BCC	Res	TQBT[4:0]					
	Reset value	0												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0008	FDCAN_CCU_CSTAT	CALS[1:0]	Res	TQC[10:0]										OCPM[17:0]																			
	Reset value	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x000C	FDCAN_CCU_CWD	WDV[15:0]												WDC[15:0]																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 528. CCU register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0010	FDCAN_CCU_IR	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras
	Reset value																															0	0
0x0014	FDCAN_CCU_IE	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras	Ras
	Reset value																															0	0

Refer to [Section 2.3 on page 134](#) for the register boundary addresses.



65 Important security notice

The STMicroelectronics group of companies (ST) places a high value on product security, which is why the ST product(s) identified in this documentation may be certified by various security certification bodies and/or may implement our own security measures as set forth herein. However, no level of security certification and/or built-in security measures can guarantee that ST products are resistant to all forms of attacks. As such, it is the responsibility of each of ST's customers to determine if the level of security provided in an ST product meets the customer needs both in relation to the ST product alone, as well as when combined with other components and/or software for the customer end product or application. In particular, take note that:

- ST products may have been certified by one or more security certification bodies, such as Platform Security Architecture (www.psacertified.org) and/or Security Evaluation standard for IoT Platforms (www.trustcb.com). For details concerning whether the ST product(s) referenced herein have received security certification along with the level and current status of such certification, either visit the relevant certification standards website or go to the relevant product page on www.st.com for the most up to date information. As the status and/or level of security certification for an ST product can change from time to time, customers should re-check security certification status/level as needed. If an ST product is not shown to be certified under a particular security standard, customers should not assume it is certified.
- Certification bodies have the right to evaluate, grant and revoke security certification in relation to ST products. These certification bodies are therefore independently responsible for granting or revoking security certification for an ST product, and ST does not take any responsibility for mistakes, evaluations, assessments, testing, or other activity carried out by the certification body with respect to any ST product.
- Industry-based cryptographic algorithms (such as AES, DES, or MD5) and other open standard technologies which may be used in conjunction with an ST product are based on standards which were not developed by ST. ST does not take responsibility for any flaws in such cryptographic algorithms or open technologies or for any methods which have been or may be developed to bypass, decrypt or crack such algorithms or technologies.
- While robust security testing may be done, no level of certification can absolutely guarantee protections against all attacks, including, for example, against advanced attacks which have not been tested for, against new or unidentified forms of attack, or against any form of attack when using an ST product outside of its specification or intended use, or in conjunction with other components or software which are used by customer to create their end product or application. ST is not responsible for resistance against such attacks. As such, regardless of the incorporated security features and/or any information or support that may be provided by ST, each customer is solely responsible for determining if the level of attacks tested for meets their needs, both in relation to the ST product alone and when incorporated into a customer end product or application.
- All security features of ST products (inclusive of any hardware, software, documentation, and the like), including but not limited to any enhanced security features added by ST, are provided on an "AS IS" BASIS. AS SUCH, TO THE EXTENT PERMITTED BY APPLICABLE LAW, ST DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE, unless the applicable written and signed contract terms specifically provide otherwise.

66 Revision history

Table 657. Document revision history

Date	Revision	Changes
05-Jul-2018	1	Initial release.
04-Apr-2019	2	Added case of reset during half word write in Section : Error code correction (ECC). Section 4: Embedded Flash memory (FLASH) Updated Figure 8: Embedded Flash memory usage. Section : Adjusting programming timing constraints: added note related to WRHIGHFREQ modification during Flash memory programming/erasing. Section : Adjusting programming parallelism: added note related to PSIZE1/2 modification during Flash memory programming/erasing. Secure DTCM size (ST_RAM_SIZE): Removed Secure DTCM size (ST_RAM_SIZE) from Section : Changing security option bytes. Updated ST_RAM_SIZE description and added ST_RAM_SIZE to the values programmed in the data protection option bytes in Section 4.4.6: Description of data protection option bytes. Section : Definitions of RDP global protection level: updated description of RDP level 1 to 0 regression and RDP level 2. Section : RDP protection transitions: user Flash memory can be partially or mass erased when doing a level regression from RDP level 1 to 0. Section 5: Secure internal Flash memory (SIFM) (former Secure memory management section) Removed Section Flash protections. Updated Section 5.3.1: Associated features and Section 5.3.2: Boot state machine. Added and added Note 2. below Figure 15: Flash memory areas and services in Standard and Secure access modes. Updated Figure 16: Bootloader state machine in Secure access mode. Updated Section 5.3.3: Secure access mode configuration. Restructured Section 5.4: Root secure services (RSS): Added Section 5.4.1: Secure area setting service and Section 5.4.2: Secure area exiting service Renamed RSS services (removed RSS_ prefix). Removed RSS_resetAndDestroyPCROPArea Updated Section 5.5.1: Access rules and Section 5.5.2: Setting secure user memory areas. Remove sections Removing secure memory areas and Selecting secure user software. Updated Figure 17: Core access to Flash memory areas